



AC8025 软件开发指南

文档版本	1.8
发布日期	2024-3-28

航盛2024-09-12 16:50:24

© 2013 - 2024 杰发科技

本文档包含杰发科技的专有信息。未经授权，严禁复制或披露本文档包含的任何信息。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。

修订记录

版本	日期	作者	修订说明
0.1	2023-3-31	autochips	文档初版，主要为 IVI 相关部分
1.0	2023-6-8	autochips	- 各模块新增/更新使用说明和注意事项、用户定制化和典型问题部分 - 各模块新增 Cockpit 部分 - 新增章节： - 系统：Android Security、IPC、Safety、SecureBoot、Recovery - 音视频：Watermark、Audio - 网络和连接：Ethernet、GNSS - 显示：PQ、Safety Display、Atcsurface、LCM、Display Debug、Projection - 存储：SD4HC、UFS - App：AtcCamera App、Cluster App、Control Panel UI、HUD App - 工具：Debug Tool
1.1	2023-7-28	autochips	- 新增章节： - 外设：PCIe - Toolkit：NPU - 更新章节： - Camera 和 LCM - 删除 Flash tool 章节，相关内容随 tool 一起 release
1.2	2023-9-12	autochips	- 更新章节 - Filesystem：主要更新 32.4.3 - NPU
1.3	2023-12-04	autochips	- 更新 HSM OS API 部分文字描述 - 更新 ipc_chan_dev_init 描述 - 更新 safety 代码结构及文件描述 - 更新代码添加章节的描述
1.4	2023-12-19	autochips	- 更新 USB 35.4.4 节，修正 BC1.2 说明，改用 metazone 配置，移除 dts 描述 - 更新 Camera 14.4.2.3 节，示例摄像头配置。
1.5	2023-12-27	autochips	更新 NPU 章节： 增加 Lite driver 说明，增加 Java 接口说明，修改部分接口描述
1.6	2024-2-27	autochips	更新 LK 升级章节

			- 更新 OTA 升级 章节 - 更新 11.3.1 ~ 11.3.4 章节 - 更新 Audio 章节 - 更新 Camera 章节 - 更新 R5 RVC 章节 - 更新 Wi-Fi 章节 - 新增 19.2.1.1 ~ 19.2.1.3 - 更新 Multimedia 章节 - 更新 22.5.1Debug 方法 - 更新 Filesystem 章节 - 更新 32.4.4 FBE 章节 - 新增 32.5.2 统计文件写入频率与写入量的方法 章节 - 更新 Metazone 章节 - 新增 33.4.9 Metazone Reserved 类型数据的使用注意事项 章节 - 新增 33.4.10 扩展 Metazone 导致 size 增加后注意事项 章节 - 更新 Datazone 章节 - 新增 34.4.8Datazone 资源添加 章节 - 更新 SD4HC 章节 - 更新 GPIO PINMUX 章节 - 更新 39.3.2API in hsm (AGPIO) 章节 - 更新 39.3.3API in hsm (SGPIO) 章节 - 新增 39.4.4hsm 使用 AGPIO/SGPIO 外部中断 章节 - 新增 39.4.5 在 AP 端查看 SPIN 状态 章节 - 更新 I2C 章节: - 新增 40.4.4 Kernel 打开 I2C 死锁检测 章节 - 新增 40.5.3 I2C 常见异常及确认方法 章节 - 更新 SPI 章节 - 新增 42.3.2 API in Hsm 章节 - 新增 42.4.2 Kernel 使用 SPI 章节 - 新增 42.4.3 hsm 使用 SPI 章节 - 更新 UART 章节 - 新增 43.3.2 API in hsm 章节 - 新增 43.4.2 hsm 使用 uart 章节 - 新增 43.4.3 uart 软硬件对应关系 章节 - 更新 Cluster App 章节: - 增加 cluster 端使用 idc 的流程和 idc 相关 api 的介绍
--	--	--	--

1.7	2024-3-17	autochips	- 更新 Backlight 章节 添加 API 接口，更新数据结构说明 - 更新 FreeRTOS Display 章节 更新设置 color range / color encoding / 冻屏检测 / 完整性校验 / Display Rsv memory 等相关接口描述 增加 QT/QD 适配说明 - 更新 Safety Display 章节 更新 API 和数据接口 新增使用说明和注意事项、典型问题 更新用户定制化章节 - 更新 LCM 章节 更新 29.4.1 新屏幕调试步骤，添加注意事项 - 更新 Projection 章节 更新使用说明和注意事项、用户定制化
1.8	2024-3-28	autochips	更新 Safety 章节 更新简介中的描述 更新 Misc Service 中的描述，增加 System trace service 内容 更新 Watch dog 和 Fault Monitor 章节

版权声明

本文档包含杰发科技的机密信息。禁止未经授权使用或披露本文档包含的信息。对因未经杰发科技授权而全部或部分披露此文档内容而给杰发科技带来的任何损失或损害，杰发科技将追究责任。

杰发科技保留对此处任何信息进行更改的权利，此处的信息如有变更，恕不另行通知。杰发科技对使用或依赖此处包含的信息不承担任何责任。

本文档的所有信息均“按原样”提供，不提供任何形式的明示、暗示、法定或其他形式的保证。杰发科技明确拒绝对适销性，非侵权性和针对特定用途的适用性方面的所有暗示保证。杰发科技对本文档可能使用、包含或提供的任何第三方软件不提供任何担保，并且用户同意仅向该等第三方寻求与此相关的任何担保索赔。杰发科技对于根据用户规格或为符合特定标准或公开论坛而产生的任何交付物，也不承担任何责任。

文档目录

修订记录.....	2
版权声明.....	5
文档目录.....	6
插图目录.....	39
表格目录.....	52
缩略语.....	61
1 系统架构.....	67
1.1 简介.....	67
1.1.1 软硬件环境	67
1.2 软件设计	67
1.2.1 软件架构	67
1.2.2 系统启动流程.....	75
1.3 系统资源管理	77
1.3.1 CPU.....	78
1.3.2 IRQ.....	78
1.3.3 Memory.....	79
2 系统软件目录.....	81
2.1 简介.....	81
2.1.1 软硬件环境	81
2.2 代码结构及目录说明	81
2.2.1 顶层目录结构.....	81
2.2.2 Cockpit 代码整体目录结构.....	83
2.2.3 ADAS/Cluster 子系统目录结构.....	84
2.2.4 重点目录介绍.....	84
2.3 使用说明和注意事项	87
2.3.1 模块定制化	87
2.3.2 模块添加	87

3	系统编译与配置	88
3.1	编译环境设定	88
3.2	编译系统配置介绍	88
3.2.1	编译配置	88
3.2.2	完整编译	89
3.2.3	子系统编译	91
3.2.4	模块编译	92
3.2.5	Build Log	94
3.3	使用 docker 打造编译环境	94
3.3.1	安装 docker	94
3.3.2	配置私服	98
3.3.3	下载 & 安装镜像	98
3.3.4	运行镜像	99
3.3.5	下载 git 代码	100
3.3.6	关闭 & 恢复容器	102
3.4	Android 编译附加说明	103
3.4.1	概述	103
3.4.2	SELinux 配置	103
3.4.3	Android.bp 语法	104
3.4.4	增加动态 jar 模块	105
3.4.5	增加静态 jar 模块	106
3.4.6	增加 apk 模块	106
3.4.7	增加动态库模块	106
3.4.8	增加静态库模块	107
3.4.9	同时增加动态和静态库模块	108
3.4.10	增加可执行程序模块	108
3.4.11	增加导出头文件模块	109
3.4.12	增加 HIDL interface 模块	110
3.4.13	条件编译	111

3.5	编译工具链(编译器版本)	114
4	Android Security	116
4.1	简介	116
4.1.1	软硬件环境	116
4.1.2	功能	116
4.1.3	代码结构及文件描述	117
4.2	软件设计	118
4.2.1	架构图	118
4.2.2	流程图	118
4.3	接口描述	120
4.4	使用说明和注意事项	120
4.4.1	OP-TEE 更新 SDK 目录	120
4.5	用户定制化	120
4.5.1	OPTEE-OS 常规宏配置	120
4.5.2	Android 原生 GateKeeper 配置	120
4.6	典型问题	121
4.6.1	OP-TEE 错误码分析	121
4.6.2	OP-TEE 中添加宏定义和配置	121
4.6.3	TA 应用实例	122
5	IPC	126
5.1	简介	126
5.1.1	功能描述	127
5.1.2	代码结构及文件描述	127
5.2	软件设计	127
5.2.1	架构图	127
5.2.2	流程图	128
5.3	接口描述	129
5.3.1	API	131
5.4	使用说明和注意事项	142

5.4.1	算法描述	142
5.4.2	参考代码	145
5.5	典型问题	147
6	Safety	148
6.1	简介	148
6.1.1	功能	148
6.1.2	架构图	149
6.1.3	代码结构及文件描述	149
6.2	使用说明和注意事项	150
6.2.1	编译注意事项	150
6.3	Fast service	151
6.4	Safety Service	151
6.4.1	Safety Monitor	151
6.4.2	Safety Display	170
6.5	Crypto Service	170
6.6	Misc Service	170
6.6.1	Shutdown and reboot service	170
6.6.2	System Trace service	171
6.7	Memory	178
6.7.1	分布图	178
6.7.2	系统内存	179
6.8	IPC	180
6.9	接口说明	180
6.9.1	中断处理	180
6.9.2	Task	181
6.9.3	AT 接口说明	187
7	SecureBoot	189
7.1	简介	189
7.1.1	功能	189
7.1.2	代码结构及文件描述	189

7.2	软件设计	190
7.2.1	架构图	190
7.3	接口描述	193
7.3.1	verify_image_header	194
7.3.2	verify_image_body	194
7.4	使用说明和注意事项	195
7.4.1	平台 Key	195
7.4.2	Secure boot enable 配置	195
7.4.3	Preloader 签名	195
7.4.4	启动关键 image 签名	197
7.4.5	enable secure boot 步骤	197
8	Shutdown	198
8.1	简介	198
8.1.1	软硬件环境	198
8.1.2	功能	199
8.1.3	代码结构以及文件描述	199
8.2	软件设计	200
8.2.1	架构图	200
8.2.2	流程图	201
8.3	接口描述	202
8.3.1	API	202
8.3.2	数据结构	207
8.4	使用说明和注意事项	208
8.4.1	使用说明	208
8.4.2	注意事项	211
8.5	用户定制化	212
8.5.1	按键触发 Shutdown 流程	212
8.5.2	调用接口触发 Shutdown 流程	212
8.5.3	Shutdown 通信配置	213

8.5.4	Safety OSs 配置 GPIO	213
8.6	典型问题	214
8.6.1	输入关机指令系统重启	214
9	Thermal.....	215
9.1	简介	215
9.1.1	功能	215
9.1.2	代码结构及文件描述	215
9.2	软件设计	216
9.2.1	架构图	216
9.2.2	流程图	216
9.3	接口描述	217
9.3.1	API	217
9.4	使用说明和注意事项	219
9.4.1	温控策略	219
9.5	典型问题	220
10	LK 升级.....	221
10.1	简介	221
10.1.1	软硬件环境	221
10.1.2	功能	221
10.1.3	代码结构及文件描述	222
10.2	软件设计	223
10.2.1	架构图	223
10.2.2	流程图	224
10.3	使用说明和注意事项	226
10.3.1	拷贝升级	226
10.3.2	fastboot 升级	231
10.4	用户定制化.....	231
10.5	典型问题	232
10.5.1	拷贝升级	232

10.5.2	Fastboot 升级	235
11	OTA 升级	236
11.1	简介	236
11.1.1	功能	236
11.1.2	代码结构及文件描述	236
11.2	软件设计	237
11.2.1	架构图	237
11.2.2	流程图	238
11.2.3	AB 分区信息	239
11.3	使用说明和注意事项	242
11.3.1	AB OTA 分区添加	242
11.3.2	AB OTA 包编译	244
11.3.3	AB 分区命名规则	245
11.3.4	AB OTA 升级步骤	245
11.4	典型问题	250
11.4.1	如何知道当前系统是哪一个 Slot	250
11.4.2	如何手动切换 slot	250
11.4.3	哪些分区需要做成 AB 分区	251
11.4.4	Slot 的 successful 状态设置	251
11.4.5	A/B 升级过程中异常终止	251
11.4.6	A/B 无感升级反复升级同一个升级包	252
12	Recovery	253
12.1	简介	253
12.1.1	功能	253
12.1.2	代码结构及文件描述	253
12.2	软件设计	254
12.2.1	架构图	254
12.2.2	流程图	255
12.2.3	Recovery 基本流程	256

12.3	使用说明和注意事项	263
12.3.1	Recovery 菜单选项	263
12.3.2	Recovery 功能	263
12.4	用户定制化	267
12.4.1	Recovery FS Table 的修改	267
12.4.2	Recovery UI 的修改	267
12.5	典型问题	267
12.5.1	Recovery 的 log 问题	267
13	Debug Tool	269
13.1	DebugTool 简介	269
13.2	AEE	269
13.2.1	简介	269
13.2.2	软件设计	272
13.2.3	接口描述	277
13.2.4	使用说明和注意事项	280
13.2.5	用户定制化	287
13.3	ATCLogger	290
13.3.1	简介	290
13.3.2	软件设计	291
13.3.3	接口描述	296
14	Camera	300
14.1	简介	300
14.1.1	软硬件环境	300
14.1.2	功能	301
14.1.3	代码结构及文件描述	302
14.2	软件设计	304
14.2.1	架构图	304
14.3	接口描述	304
14.3.1	API	304

14.3.2	数据结构	327
14.4	使用说明和注意事项	328
14.4.1	Android Camera 框架定制	328
14.4.2	Sensor 使用的预配置	329
14.4.3	Camera 数据流的使用	331
14.4.4	Camera 配置文件	333
14.4.5	Camera 图像质量调节	337
14.4.6	CameraSource 使用	337
14.4.7	Camera Debug 方法	338
14.5	用户定制化	338
14.5.1	Sensor 移植和定制化	338
15	R5 RVC	353
15.1	简介	353
15.1.1	软硬件环境	353
15.1.2	功能	353
15.1.3	代码结构及文件描述	353
15.2	软件设计	354
15.2.1	架构图	354
15.2.2	流程图	355
15.3	接口描述	359
15.3.1	API	359
15.3.2	数据结构	365
15.4	使用说明和注意事项	367
15.4.1	倒车响应时间	367
15.4.2	倒车资源冲突	368
15.5	用户定制化	369
15.5.1	倒车触发方式修改	369
15.5.2	倒车显示配置	369
15.5.3	RVC 的输入源设置	370

15.5.4	R5 RVC UI 定制	370
15.5.5	R5 RVC 模块开关	376
15.6	典型问题	376
15.6.1	R5 RVC 显示异常	376
16	Watermark	378
16.1	简介	378
16.1.1	软硬件环境	378
16.1.2	功能	378
16.1.3	代码结构及文件描述	378
16.2	软件设计	380
16.2.1	架构图	380
16.2.2	流程图	381
16.3	接口描述	385
16.3.1	WatermarkWrapper API	385
16.3.2	Watermarkcore API	387
16.3.3	数据结构	393
16.4	使用说明和注意事项	396
16.4.1	库名及头文件	396
16.4.2	水印资源配置文件	396
16.4.3	Watermarkcore 使用示例	398
16.4.4	WatermarkWrapper 使用示例	399
16.5	用户定制化	400
16.5.1	概述	400
16.5.2	水印的起始点	400
16.5.3	水印的内容	401
16.6	典型问题	402
16.6.1	水印显示异常	402
17	Audio	404
17.1	Audio 简介	404

17.1.1	内容介绍	404
17.1.2	软硬件环境	405
17.2	通路部分	406
17.2.1	框架	407
17.2.2	文件描述	409
17.2.3	Framework 音频分离	410
17.2.4	ALSA device 节点配置	418
17.2.5	输出声道配置	430
17.3	音量部分	431
17.3.1	音量框架	431
17.3.2	音量接口介绍	433
17.3.3	ATC volumepolicy	442
17.3.4	导航白名单说明	456
17.3.5	典型问题	457
17.4	音效部分	462
17.4.1	音效框架	463
17.4.2	音效类型说明	463
17.4.3	音效参数持久化实现方式	466
17.4.4	Java 接口	472
17.4.5	Kernel 接口	488
17.4.6	Debug 方法	527
17.4.7	用户定制化	528
17.5	蓝牙/Carplay 电话部分	529
17.5.1	框架说明	529
17.5.2	接口描述	530
17.5.3	HAL ECNR	530
17.5.4	DSP ECNR	534
17.6	FM/AuxIn 部分	534
17.6.1	硬件说明	535

17.6.2	Direct mode	535
17.6.3	Indirect mode	537
17.7	语音识别 support 部分	539
17.7.1	DSP 四声道拼接	539
17.8	FastAudio	541
17.8.1	硬件环境	542
17.8.2	代码结构及文件描述	542
17.8.3	框架图	543
17.8.4	功能描述	544
17.8.5	接口描述	545
17.8.6	定制方法	550
17.9	Audio IO	553
17.9.1	硬件环境	554
17.9.2	软件支持说明	554
17.9.3	接口描述	558
17.10	通用 debug 方法	568
17.10.1	AtcEngineerMode Audio 使用说明	568
17.10.2	Dump 文件说明	571
17.10.3	Aurisys 相关 dump	573
17.10.4	设置参数	573
18	Bluetooth	575
18.1	简介	575
18.1.1	软硬件环境	575
18.1.2	功能	575
18.1.3	代码结构及文件描述	577
18.2	软件设计	579
18.2.1	架构图	579
18.2.2	流程图	579
18.3	接口描述	589

18.3.1	Android 原生接口	589
18.3.2	ATC 特有接口	589
18.4	使用说明和注意事项	593
18.4.1	构建蓝牙 Settings	594
18.4.2	构建蓝牙电话 Dialer	595
18.4.3	构建蓝牙音乐播放器	598
18.5	用户定制化	600
18.5.1	关闭 in-band ring tone	600
18.5.2	蓝牙地址	601
18.5.3	微信语音、语音识别通话界面弹框配置	603
18.5.4	关闭蓝牙音乐服务端控制 audioFocus 功能	603
18.5.5	关闭蓝牙通话音量同步功能	603
18.5.6	A2dp source 功能配置	603
18.5.7	蓝牙短信同步时间限制配置	604
18.5.8	关闭电话本联系人头像同步	604
18.5.9	关闭原生电话本自动同步功能	604
18.5.10	蓝牙网络评分配置	605
18.5.11	蓝牙 HFP 连接上限配置	605
18.5.12	蓝牙通话记录同步条数上限设置	605
18.5.13	关闭蓝牙 A2DP Sink AAC Codec 功能	606
18.5.14	配置蓝牙芯片	606
18.6	典型问题	606
18.6.1	如何获取蓝牙 HCI log	606
18.6.2	如何进入蓝牙测试模式	607
18.6.3	蓝牙通话声音质量差或者近距离下 A2DP 卡顿	608
18.6.4	如何知道蓝牙走远了信号丢失断开	608
18.6.5	如何知道蓝牙设备重新回到可连接范围了	608
18.6.6	如何控制蓝牙音乐是否可以播放出声音	609
18.6.7	如何打开 Bluedroid 协议栈 log	610

18.6.8	如何打开蓝牙通话 audio dump	611
18.6.9	当多 Dialer 存在时，如何配置系统默认电话 Dialer	612
18.6.10	使用其他第三方蓝牙模块时，如何关闭 ATC 蓝牙的相关代码	612
19	Wi-Fi	613
19.1	简介	613
19.1.1	软硬件环境	613
19.1.2	功能	613
19.1.3	代码结构及文件描述	614
19.2	软件设计	616
19.2.1	架构图	616
19.2.2	流程图	627
19.3	接口描述	634
19.3.1	原生接口	634
19.4	使用说明和注意事项	634
19.4.1	如何使用 Wi-Fi	634
19.4.2	Wi-Fi 芯片的配置规则	640
19.4.3	Wi-Fi 配置	641
19.4.4	Wi-Fi MAC 的读取和修改	645
19.5	用户定制化	645
19.5.1	热点开机自动打开功能	645
19.5.2	热点主动断开对端 station 功能	646
19.5.3	根据 MAC 地址获取指定设备的连接状态	647
19.5.4	获取对端 station 断开时的 reasonCode	647
19.5.5	获取当前热点的信道	648
19.6	典型问题	649
19.6.1	Wi-Fi 难以搜索到周边设备	649
19.6.2	Wi-Fi Mac 地址说明	650
19.6.3	开发 Wi-Fi APP 时需要添加的权限	651
19.6.4	如何开启 Wi-Fi 的 Debug log	652

19.6.5	如何获取 TCP dump 文件	653
19.6.6	Wi-Fi P2P GO 和 GC 如何决定	654
19.6.7	如何修改 P2P 配置将其投屏在 5G 信道	655
19.6.8	如何打开 AP 的随机 MAC 地址	655
19.6.9	若修改了接口或者 wifiserver 中的内容，如何编译，单独 push 什么可以生效	656
19.6.10	需要根据 Vendor 厂商定制化的系统属性，以及这些系统属性的意义是什么？	656
19.6.11	如何设置 country code，本地若要修改 country code，怎么处理？	657
19.6.12	如何通过 overlay 机制定制化 Vendor Feature	657
19.6.13	如何查看当前 AP 使用的信道和带宽	658
20	Ethernet	659
20.1	简介	659
20.1.1	软硬件环境	659
20.1.2	功能	660
20.1.3	代码结构及文件描述	660
20.2	软件设计	661
20.2.1	架构图	661
20.2.2	流程图	662
20.3	接口描述	662
20.4	使用说明和注意事项	662
20.4.1	如何确认 Ethernet 是否已通	662
20.4.2	如何用命令测试 Ethernet	662
20.5	用户定制化	663
20.5.1	已适配过的 PHY 芯片型号	663
20.5.2	PHY 芯片适配流程	663
20.6	典型问题	665
20.6.1	PHY link up 成功的参考 log	665
20.6.2	PHY 芯片工作异常	665
20.6.3	PHY link up 但 ping 失败	666
21	GNSS	667

21.1	简介.....	667
21.1.1	软硬件环境	667
21.1.2	功能	667
21.1.3	代码结构及文件描述	667
21.2	软件设计	669
21.2.1	架构图	669
21.2.2	数据流	670
21.3	使用说明和注意事项	670
21.4	用户定制化.....	672
21.4.1	GNSS Mode 配置	672
21.5	典型问题	673
21.5.1	如何进行 GPS 测试.....	673
21.5.2	如何打开 GPS debug log.....	673
21.5.3	3D 定位和 2D 定位的区别	674
21.5.4	各个卫星系统的卫星编号是多少	674
21.5.5	硬件传导条件要求.....	675
21.5.6	晶体要求	675
21.5.7	各种启动方式的 TTFF 是多少	675
21.5.8	各种启动方式 TTFF 的测试方法	676
21.5.9	各种启动方式的条件要求.....	676
21.5.10	如何快速获取 GNSS 软件版本	677
21.5.11	如何从 NMEA 数据中看是否定位	677
21.5.12	如何修改 framework 代码，强制输出某个地区经纬度的内容	677
21.5.13	如何关闭在线获取辅助星历功能	678
21.5.14	如何配置位置更新频率	678
21.5.15	第三方 GNSS 芯片如何适配.....	679
22	Multimedia	681
22.1	简介.....	681
22.1.1	软硬件环境	681

22.1.2 功能 681

22.2 软件设计 686

22.2.1 架构图 686

22.2.2 流程图 690

22.3 接口描述 693

22.4 使用说明和注意事项 694

22.4.1 示例代码 694

22.5 典型问题 702

22.5.1 Debug 方法 702

22.5.2 常见问题分析 705

23 VBA 709

23.1 简介 709

23.1.1 软硬件环境 709

23.1.2 功能 709

23.1.3 代码结构及文件描述 710

23.2 软件设计 710

23.2.1 架构图 710

23.2.2 流程图 711

23.3 接口描述 716

23.3.1 API 716

23.3.2 数据结构 720

23.4 用户定制化 723

23.4.1 开机资源客制化 723

24 Backlight 724

24.1 简介 724

24.1.1 软硬件环境 724

24.1.2 功能 724

24.2 代码结构及文件描述 724

24.3 软件设计 726

24.3.1	架构图.....	726
24.3.2	流程图.....	727
24.4	接口描述	728
24.4.1	API	728
24.4.2	数据结构	731
24.5	使用说明和注意事项	732
24.6	用户定制化.....	733
24.6.1	AP 保存背光	733
24.6.2	HSM 设置背光.....	734
24.6.3	开关背光一致性功能	734
24.7	典型问题	737
24.7.1	设置背光常见问题.....	737
25	PQ.....	738
25.1	简介.....	738
25.1.1	软硬件环境	738
25.1.2	功能	738
25.1.3	代码结构及文件描述	740
25.2	软件设计	741
25.2.1	架构图.....	741
25.2.2	流程图.....	742
25.3	接口描述	744
25.3.1	java 层接口	744
25.3.2	hal 层接口	751
25.3.3	Hsm 接口.....	757
25.4	使用说明和注意事项	759
25.4.1	PQ 调节工具介绍.....	759
25.4.2	Android 如何进入 PQ 调节界面	759
25.4.3	Android 调节界面介绍	763
25.4.4	Cluster/adas 如何调节 PQ.....	768

25.5	用户定制化.....	768
25.5.1	应用参数	768
25.5.2	Android PQ 接口.....	770
25.5.3	linux PQ 接口.....	771
25.5.4	Hsm 调用 PQ 接口	771
25.6	典型问题	771
25.6.1	拖动某一调节项，显示没有变化.....	771
25.6.2	PQ tool 中调节好一组参数后，重启平台，PQ 效果又恢复到原始状态.....	771
26	FreeRTOS Display.....	772
26.1	简介.....	772
26.1.1	软硬件环境	772
26.1.2	功能	772
26.1.3	代码结构及文件描述.....	772
26.2	软件设计	773
26.2.1	架构图.....	773
26.2.2	流程图.....	774
26.3	接口描述	775
26.3.1	API	775
26.3.2	数据结构	793
26.4	使用说明和注意事项	795
26.4.1	Normal 显示.....	795
26.4.2	冻屏检测 / 完整性校验	795
26.5	用户定制化.....	796
26.5.1	冻屏以及完整性校验定制化	796
26.5.2	QT/QD 显示适配说明	796
27	Safety Display	797
27.1	简介.....	797
27.1.1	软硬件环境	797
27.1.2	功能	797

27.1.3	代码结构及文件描述	797
27.2	软件设计	798
27.2.1	架构图	798
27.2.2	流程图	799
27.3	接口描述	800
27.3.1	API	800
27.3.2	数据结构	810
27.4	使用说明和注意事项	811
27.5	用户定制化	812
27.5.1	安全图标完整性校验客制化	812
27.5.2	冻屏检测客制化	813
27.6	典型问题	814
28	Atcsurface	817
28.1	简介	817
28.1.1	功能	817
28.1.2	代码结构及文件描述	817
28.2	软件设计	818
28.2.1	架构图	818
28.2.2	流程图	819
28.3	接口描述	820
28.3.1	API	820
28.3.2	数据结构	833
28.4	使用说明和注意事项	837
28.5	典型问题	837
28.5.1	用户使用不当，导致内存泄露	837
28.5.2	用户如何设置 atcsurface 的显示层级/优先级	837
29	LCM	838
29.1	简介	838
29.1.1	软硬件环境	838

29.1.2	功能	838
29.1.3	代码结构及文件描述	838
29.2	软件设计	840
29.2.1	架构图	840
29.2.2	流程图	841
29.3	使用说明和注意事项	841
29.4	用户定制化	842
29.4.1	新屏幕调试步骤	842
29.5	典型问题	847
29.5.1	生成 datazone 烧录后屏幕没有正常点亮	847
29.5.2	出现巨齿纹	849
29.5.3	黑屏	850
29.5.4	镜像显示	850
30	Display Debug	851
30.1	简介	851
30.1.1	功能	851
30.2	软件设计	851
30.2.1	架构图	852
30.2.2	Android System Debug 功能	852
30.2.3	HWC Dump Debug 功能	855
30.2.4	Atcsurface debug 功能	857
30.2.5	Drm screenshot 功能	858
30.3	常见场景分析	858
30.3.1	UI 问题	858
30.3.2	Video 问题	860
30.3.3	开机黑屏	862
31	Projection	863
31.1	简介	863
31.1.1	功能	863

31.1.2	代码结构及文件描述	863
31.2	软件设计	864
31.2.1	架构图	864
31.2.2	流程图	865
31.3	接口描述	867
31.3.1	API	867
31.3.2	数据结构	876
31.4	使用说明和注意事项	877
31.4.1	InstrumentClusterApp 开发注意事项	877
31.4.2	Android 异常仪表保留最后一帧验证方法	880
31.5	用户定制化	883
31.5.1	投屏显示配置	883
31.5.2	QT/QD /Kancuster app 集成 idc 功能步骤	883
31.6	典型问题	886
32	Filesystem	887
32.1	简介	887
32.1.1	软硬件环境	887
32.1.2	功能	887
32.1.3	代码结构及文件描述	888
32.2	软件设计	889
32.2.1	架构图	889
32.2.2	流程图	890
32.3	接口描述	892
32.3.1	StorageManager 接口	892
32.3.2	StorageManagerEx 接口	892
32.4	使用说明和注意事项	896
32.4.1	如何使用 Adoptable Storage	896
32.4.2	如何使用 Auto Recovery	904
32.4.3	如何使用添加/删除/修改分区	906

32.4.4	FBE	914
32.5	用户定制化	918
32.5.1	定制化低存储保护机制	918
32.5.2	统计文件写入频率与写入量的方法	921
32.6	典型问题	922
32.6.1	拔出外置设备，导致使用外置设备内文件的进程被 kill 掉	922
33	Metazone	923
33.1	简介	923
33.1.1	软硬件环境	923
33.1.2	功能	923
33.1.3	代码结构及文件描述	923
33.2	软件设计	925
33.2.1	架构图	925
33.2.2	流程图	930
33.3	接口描述	931
33.3.1	AtcMetazone Class	931
33.3.2	Native APIs	937
33.3.3	LK APIs	944
33.3.4	Safety OS APIs	950
33.3.5	Kernel Driver APIs	955
33.3.6	注意事项	962
33.4	使用说明和注意事项	962
33.4.1	Metazone 生成方式	962
33.4.2	各层 Metazone 接口使用注意	963
33.4.3	Java 层使用 Metazone	963
33.4.4	Native 层使用 MetaZone	965
33.4.5	LK 中使用 Metazone	967
33.4.6	Safety 中使用 Metazone	968
33.4.7	Kernel Driver 中使用 metazone	968

33.4.8	通过 adb 操作平台上的数据	968
33.4.9	Metazone Reserved 类型数据的使用注意事项	969
33.4.10	扩展 Metazone 导致 size 增加后注意事项	969
33.5	用户定制化	969
33.5.1	新增 index, 修改 metazone.csv 文件	969
33.6	典型问题	971
34	Datazone	972
34.1	简介	972
34.1.1	功能	972
34.1.2	代码结构及文件描述	972
34.2	软件设计	974
34.2.1	架构图	974
34.2.2	Datazone 结构	975
34.2.3	流程图	976
34.3	接口描述	977
34.3.1	Safety os API	978
34.3.2	LK API	981
34.3.3	Kernel API	984
34.3.4	Native API	987
34.3.5	Framework API	992
34.4	使用说明和注意事项	995
34.4.1	各层 Datazone 接口使用注意	995
34.4.2	java 层使用 Datazone	995
34.4.3	Native 层使用 datazone	996
34.4.4	LK 中使用 Datazone	997
34.4.5	Safety OS 中使用 Datazone	997
34.4.6	Kernel Driver 中使用 Datazone	997
34.4.7	通过 adb debug Datazone	998
34.4.8	Datazone 资源添加	998

34.4.9	Datazone CRC 校验	999
34.5	典型问题	999
35	USB	1000
35.1	简介	1000
35.1.1	软硬件环境	1000
35.1.2	功能	1001
35.1.3	代码结构及文件描述	1002
35.2	软件设计	1003
35.2.1	OTG 架构图	1003
35.2.2	Host30 架构图	1007
35.3	使用说明和注意事项	1009
35.3.1	设置 USB OTG host/device 模式切换	1009
35.3.2	设置 USB OTG protocol 为 full speed 或 high speed	1012
35.3.3	设置 USB Host 30 Protocol 为 Full Speed/High Speed/Super Speed	1015
35.3.4	设置 USB Host 30 Protocol 默认值	1017
35.3.5	切换 adb 连接域	1018
35.3.6	查看当前 adb 连接域	1018
35.4	典型问题	1018
35.4.1	如何查看 USB 工作规范和 USB OTG 当前工作模式	1018
35.4.2	如果 U 盘连接到车机的 USB port 上，没有出现挂载点，如何确定 driver 是否正确识别到 U 盘	1019
35.4.3	BC1.2 功能支持多大的充电电流	1020
35.4.4	如何开关 OTG 口 BC 功能	1020
35.4.5	如何查看 USBHOST30 工作的规范和当前工作模式	1021
36	SD4HC	1023
36.1	简介	1023
36.1.1	软硬件环境	1023
36.1.2	功能	1025
36.1.3	代码结构及文件描述	1025

36.2	软件设计	1027
36.2.1	架构图	1027
36.2.2	控制流和数据流	1030
36.2.3	dts 配置介绍	1032
36.2.4	eMMC/SD 工作时钟说明	1034
36.2.5	硬件写保护	1034
36.2.6	软件写保护	1037
36.3	接口描述	1038
36.3.1	eMMC/SD Driver 在 preloader 中提供的接口	1038
36.3.2	eMMC/SD Driver 在 Safety OS 中提供的接口	1038
36.3.3	eMMC/SD Driver 在 LK 中提供的接口	1039
36.4	使用说明	1039
36.4.1	如何在 Preloader 中配置 eMMC/SD 工作模式	1039
36.4.2	如何在 Kernel 中配置 eMMC/SD/SDIO 工作模式	1041
36.4.3	如何查看当前 SD/eMMC 工作状态	1042
36.4.4	硬件写保护功能验证	1043
36.4.5	如何验证软件写保护	1043
36.5	常见问题	1044
36.5.1	系统 remount 之后的写保护状态	1044
36.5.2	重要 error log 分析	1044
36.5.3	硬件写保护 group size	1045
37	UFS	1046
37.1	简介	1046
37.1.1	软硬件环境	1046
37.1.2	功能	1047
37.1.3	代码结构及文件描述	1047
37.2	软件设计	1047
37.2.1	UFS 在 HSM Loader/HSM OS/ LK 中的架构图	1047
37.2.2	UFS 在 Kernel 中的架构图	1048

37.3	接口描述	1049
37.3.1	UFS Driver 在 Loader 中提供的接口	1049
37.3.2	UFS Driver 在 HSM OS 中提供的接口	1050
37.3.3	UFS Driver 在 LK 中提供的接口	1050
37.4	使用说明和注意事项	1053
37.4.1	配置 UFS 的写保护区间	1053
38	FastTouch.....	1054
38.1	简介	1054
38.1.1	软硬件环境	1054
38.1.2	功能	1054
38.1.3	代码结构及文件描述	1054
38.2	软件设计	1055
38.2.1	架构图	1055
38.2.2	流程图	1056
38.3	接口描述	1057
38.3.1	API in Kernel	1057
38.3.2	API in hsm.....	1058
38.4	使用说明与注意事项	1059
39	GPIO PINMUX.....	1060
39.1	简介	1060
39.1.1	软硬件环境	1060
39.1.2	功能	1060
39.1.3	代码结构及文件描述	1060
39.2	软件设计	1061
39.2.1	架构图	1061
39.3	接口描述	1062
39.3.1	API in Kernel	1062
39.3.2	API in hsm (AGPIO)	1066
39.3.3	API in hsm (SGPIO)	1074

39.4	使用说明与注意事项	1084
39.4.1	Kernel 中使用 GPIO	1084
39.4.2	通过 GPIOLIB 设置管脚方向及输入输出	1086
39.4.3	Kernel 注册 GPIO 外部中断	1087
39.4.4	hsm 使用 AGPIO/SGPIO 外部中断	1087
39.4.5	在 AP 端查看 SPIN 状态	1090
39.5	典型问题	1091
39.5.1	如何通过节点方式进行 GPIO 相关设置/测试/查看	1091
40	I2C	1093
40.1	简介	1093
40.1.1	软硬件环境	1093
40.1.2	功能	1093
40.1.3	代码结构及文件描述	1094
40.2	软件设计	1094
40.2.1	架构图	1094
40.2.2	流程图	1094
40.3	接口描述	1095
40.3.1	API in Kernel	1095
40.3.2	API in hsm	1096
40.3.3	数据结构	1098
40.4	使用说明和注意事项	1099
40.4.1	I2C 速率设置	1099
40.4.2	在 DTS 中添加设备节点	1099
40.4.3	设备驱动的编写	1099
40.4.4	Kernel 打开 I2C 死锁检测	1102
40.5	典型问题	1103
40.5.1	目前平台调通的触摸屏 IC	1103
40.5.2	I2C 支持的最大速率	1103
40.5.3	I2C 常见异常及确认方法	1104

41	PWM	1106
41.1	简介	1106
41.1.1	软硬件环境	1106
41.1.2	功能	1106
41.1.3	代码结构及文件描述	1106
41.2	软件设计	1107
41.2.1	架构图	1107
41.2.2	流程图	1107
41.3	接口描述	1107
41.3.1	API in Kernel	1107
41.3.2	API in hsm	1109
41.3.3	数据结构	1111
41.4	使用说明和注意事项	1112
41.4.1	时钟说明	1112
41.4.2	Vsync Mode	1113
42	SPI	1114
42.1	简介	1114
42.1.1	软硬件环境	1114
42.1.2	功能	1114
42.1.3	代码结构及文件描述	1114
42.2	软件设计	1115
42.2.1	架构图	1115
42.2.2	流程图	1115
42.3	接口描述	1116
42.3.1	API in Kernel	1116
42.3.2	API in Hsm	1116
42.4	使用说明和注意事项	1118
42.4.1	通过 spidev 使用 SPI	1118
42.4.2	Kernel 使用 SPI	1119

42.4.3	hsm 使用 SPI	1121
43	UART	1126
43.1	简介	1126
43.1.1	软硬件环境	1126
43.1.2	功能	1126
43.1.3	代码结构及文件描述	1126
43.2	软件设计	1127
43.2.1	架构图	1127
43.2.2	流程图	1127
43.3	接口描述	1128
43.3.1	API in Kernel	1128
43.3.2	API in hsm.....	1130
43.4	使用说明和注意事项	1135
43.4.1	用户空间使用 uart.....	1135
43.4.2	hsm 使用 uart	1138
43.4.3	uart 软硬件对应关系	1140
44	PCIe	1142
44.1	简介	1142
44.1.1	软硬件环境	1142
44.1.2	功能	1142
44.1.3	代码结构及文件描述	1143
44.2	使用说明和注意事项	1143
44.2.1	EP DMA 支持情况	1143
44.2.2	参考时钟	1144
45	AtcCamera App	1145
45.1	简介	1145
45.1.1	软硬件环境	1145
45.1.2	功能	1145
45.1.3	代码结构及文件描述	1147

45.2	软件设计	1150
45.2.1	架构图	1150
45.2.2	流程图	1151
45.3	接口描述	1154
45.3.1	类 MIPIDVRActivity	1154
45.3.2	类 CameraServiceImpl	1154
45.3.3	类 CameraStateMachine	1155
45.3.4	类 CameraOperation	1156
45.3.5	类 StorageUtil	1157
45.3.6	类 DVRConst	1158
45.4	使用说明和注意事项	1158
45.4.1	水印开关	1158
45.4.2	AtcCamera 所使用的摄像头	1160
45.4.3	倒车事件的监听	1160
45.4.4	Metazone 在倒车中的使用	1161
45.4.5	倒车镜像	1162
45.4.6	倒车视频源的裁剪	1162
45.4.7	同步刷新紧急视频到外置存储中	1163
45.5	用户定制化	1164
45.5.1	录制时长设置	1164
45.5.2	循环录制策略设置	1164
45.5.3	打开 Camera id 设置	1164
45.6	典型问题	1165
45.6.1	写外置存储卡权限	1165
46	Cluster App	1166
46.1	简介	1166
46.1.1	功能	1166
46.1.2	代码结构及文件描述	1166
46.2	软件设计	1167

46.2.1	架构图	1167
46.2.2	流程图	1168
46.3	接口描述	1173
46.3.1	ClusterCtrlManager	1173
46.3.2	IInterdomainChannelProxy	1176
46.3.3	数据结构	1178
46.4	使用说明和注意事项	1179
46.4.1	Kanzi app 使用问题	1179
46.4.2	投屏功能	1179
46.4.3	QT/QD /kancluster app 集成 idc 功能步骤	1181
46.5	典型问题	1184
47	Control Panel UI.....	1185
47.1	简介	1185
47.1.1	软硬件环境	1185
47.1.2	功能	1185
47.1.3	代码结构及文件描述	1185
47.2	软件设计	1187
47.2.1	架构图	1187
47.2.2	流程图	1188
47.3	接口描述	1188
47.4	使用说明和注意事项	1189
48	HUD App.....	1190
48.1	简介	1190
48.1.1	功能	1190
48.1.2	代码结构及文件描述	1190
48.2	软件设计	1191
48.2.1	架构图	1191
48.2.2	流程图	1192
48.3	接口描述	1193

48.3.1	HudServiceManager	1193
48.4	使用说明和注意事项	1193
49	NPU	1194
49.1	简介	1194
49.1.1	软硬件环境	1194
49.1.2	功能	1195
49.2	软件设计	1198
49.2.1	架构图	1198
49.2.2	流程图	1200
49.3	接口及数据结构描述	1203
49.3.1	ATCNN ToolKit	1203
49.3.2	ATCNN SDK	1206
49.3.3	ATCNN SDK 数据结构	1221
49.4	使用说明和注意事项	1228
49.4.1	ATCNN TooKit 安装使用说明	1228
49.4.2	ATCNN C++接口基本调用流程	1245
49.4.3	ATCNN Service 接口基本调用流程	1248
49.4.4	ATCNN SDK 接口调用示例	1252
50	参考文档	1253

插图目录

图 1-1 软件总体架构图-IVI 形态	68
图 1-2 软件总体架构图-Cockpit 形态	69
图 1-3 软件系统架构图	70
图 1-4 Android OS 架构图	71
图 1-5 Trustzone 架构图	73
图 1-6 Safety OS 架构图	74
图 1-7 Cluster/ADAS 系统软件架构	75
图 1-8 系统启动流程	75
图 1-9 Cockpit 形态系统启动时序示意	77
图 1-10 IVI 形态的 CPU 资源分配	78
图 1-11 Cockpit 形态的 CPU 资源分配	78
图 1-12 中断管理模块架构图	78
图 1-13 内存布局	79
图 1-14 OS 间内存隔离示意	79
图 2-1 顶层目录	81
图 2-2 项目整体目录结构	83
图 2-3 ADAS/Cluste 子系统目录结构	84
图 3-1 Json 配置文件	89
图 3-2 编译整体架构	90
图 3-3 docker 安装版本查看	95
图 4-1 Android Security 架构图	118
图 4-2 OP-TEE 流程图	118
图 4-3 KeyStore 流程图	119
图 4-4 GateKeeper 流程图	119
图 4-5 TA open session	123
图 4-6 TA invoke command	124
图 4-7 TA close session	124
图 5-1 IPC 框图	126
图 5-2 IPC 系统构架图	128
图 5-3 IPC send and ack 流程	129
图 5-4 IPC message	143

图 5-5 IPC understanding message buffer	144
图 6-1 Safety 系统架构图	149
图 6-2 OS layout	149
图 6-3 Safety monitor 框架	151
图 6-4 Clock Monitor 硬件框图	152
图 6-5 Exception handler 流程	153
图 6-6 Watchdog 工作流程	154
图 6-7 AWDt 硬件框图	155
图 6-8 SWDt 硬件框图	156
图 6-9 Fault Monitor 框图	157
图 6-10 Hardware Fault 框图	158
图 6-11 Fault Signal State Machine	159
图 6-12 Fault 信号处理流程	160
图 6-13 HSM SYSTEM TRACE	172
图 6-14 Memory layout	178
图 6-15 AT 映射图	179
图 7-1 preloader 子模块	190
图 7-2 image 签名子模块	191
图 7-3 image 校验	193
图 8-1 AC8025 Shutdown 整体架构图	200
图 8-2 Shutdown 关键流程	201
图 9-1 Thermal 架构图	216
图 9-2 Thermal 流程图	216
图 10-1 LK Upgrade 系统架构图	223
图 10-2 LK 拷贝升级流程	224
图 10-3 Fastboot 升级流程	225
图 10-4 Partition Flag	228
图 10-5 LK Copy Upgrade Display UI	233
图 11-1 AB OTA 升级架构图	237
图 11-2 AB OTA 升级流程图	238
图 11-3 AB 分区示意图	239
图 11-4 Slot 属性图	240
图 11-5 默认属性	241
图 11-6 升级完成后的属性	241

图 12-1 recovery 系统架构图	254
图 12-2 recovery 数据流图	255
图 12-3 Recovery main 数据流图	256
图 12-4 Recovery UI 显示	259
图 12-5 OTA 包文件结构	261
图 12-6 recovery 下 U 盘/Sdcard 的路径示例	262
图 13-1 NE 可捕获异常信号量说明图	270
图 13-2 AEE 总体架构	272
图 13-3 JE 流程图	274
图 13-4 NE 流程图	275
图 13-5 KE 流程图	276
图 13-6 IVI 系统 AEE log 分析流程总结	285
图 13-7 Adas/Cluster 系统 AEE Log 分析流程总结	286
图 13-8 IVI 系统 aee 命令参数	288
图 13-9 ivi log 架构图	291
图 13-10 adas/cluster log 架构图	292
图 13-11 Starting recording 流程图	293
图 13-12 Stop recording 流程图	294
图 13-13 Stop recording 流程图	295
图 13-14 ATCLogger UI Starting	296
图 13-15 ATCLogger UI Stop	297
图 13-16 ATCLogger UI Settings	298
图 14-1 AC8025 MIPI 接口示意图	300
图 14-2 AC8025 MIPI 硬件原理图	301
图 14-3 AC8025 Camera 架构图	304
图 14-4 AC8025 Camera 数据流使用	332
图 15-1 R5 RVC 架构图	354
图 15-2 R5 RVC 和 RVC Setting App 交互流程	355
图 15-3 Videoin 预初始化流程	356
图 15-4 R5 RVC 初始化流程	356
图 15-5 R5 RVC 开启流程及关闭流程	357
图 15-6 R5 RVC 拔插信号通知流程	358
图 15-7 RVC Setting App Set/Get 流程	359
图 15-8 R5 RVC UI 更新流程	371

图 15-9 MRF Tool RVC 界面	375
图 16-1 Watermark 整体架构图	380
图 16-2 Watermarkcore 架构图	380
图 16-3 Watermarkcore 工作流程图	382
图 16-4 WatermarkWrapper 工作流程	383
图 16-5 Watermark Service 位置信息获取流程图	384
图 16-6 Watermarkcore 示例程序调用流程	399
图 16-7 WatermarkWrapper 水印叠加效果示例	400
图 16-8 WatermarkWrapper 示例程序调用流程	400
图 16-9 车速水印效果	402
图 17-1 CarAudio VS PhoneAudio 音频通路分离简化图	406
图 17-2 Audio 系统框图	407
图 17-3 Audio 数据流图	408
图 17-4 DSP 输出框图	409
图 17-5 多喇叭输出示例	410
图 17-6 CarAudio 音频通路分离实现流程	414
图 17-7 音频分离实现流程图	415
图 17-8 debug 音频分离	418
图 17-9 PCM 节点	429
图 17-10 声道映射示意图	430
图 17-11 IVI 音量框架图	432
图 17-12 Cockpit 音量框图	433
图 17-13 AudioVolumePolicy 架构图	442
图 17-14 startVolumePolicy 调用流程图	445
图 17-15 stopVolumePolicy 调用流程图	446
图 17-16 设置音量流程图	447
图 17-17 deviceport address 示例	455
图 17-18 volumepolicy dump 信息	456
图 17-19 CarAudio output device 显示	457
图 17-20 disable volumepolicy 方式	458
图 17-21 当前 active 通路示例	458
图 17-22 audioCustomConfig.xml 中节点映射示例	459
图 17-23 非 IVI 系统音源音量曲线	461
图 17-24 Android 音效框图	463

图 17-25 均衡器中心频率曲线	465
图 17-26 混响反馈叠加	466
图 17-27 DSP 架构图	489
图 17-28 蓝牙通话数据通路	530
图 17-29 削顶的 MIC 录音数据	534
图 17-30 Direct Mode 数据通路图	536
图 17-31 Indirect Mode 数据通路图	538
图 17-32 四声道拼接格式	540
图 17-33 DSP 四声道拼接数据流图	540
图 17-34 Fast Audio 整体框架图	543
图 17-35 Safety & IVI OS 架构图	544
图 17-36 Audio IO 模块框图	555
图 17-37 Audio IO 软件框图	557
图 18-1 蓝牙框架图	579
图 18-2 蓝牙 Settings 初始化流程	580
图 18-3 打开蓝牙流程	581
图 18-4 搜索设备流程	582
图 18-5 蓝牙连接流程	582
图 18-6 电话应用初始化准备	583
图 18-7 拨打及挂断电话流程	584
图 18-8 蓝牙通话语音切换流程	585
图 18-9 主动下载电话本的流程	586
图 18-10 音乐播放器初始化流程	586
图 18-11 音乐播放流程	587
图 18-12 列表浏览流程	587
图 18-13 接收新短信流程	588
图 18-14 短信同步流程	588
图 18-15 短信发送流程	589
图 18-16 蓝牙地址使用流程	601
图 18-17 打开 Hci log	607
图 18-18 打开 Stack log	610
图 18-19 打开 Audio Dump	611
图 19-1 Wi-Fi 架构图	617
图 19-2 Wi-Fi Station 架构图	618

图 19-3 Wi-Fi AP 架构图	622
图 19-4 Wi-Fi 架构图	625
图 19-5 Wi-Fi 整体流程图	627
图 19-6 Wi-Fi Enable Flow	628
图 19-7 Wi-Fi Scan Flow	629
图 19-8 Wi-Fi Connect Flow	630
图 19-9 Wi-Fi Disconnect Flow	630
图 19-10 AP Enable Flow	631
图 19-11 AP connected Flow	631
图 19-12 Enable P2P Flow	632
图 19-13 P2P Discovery Flow	632
图 19-14 P2P Connect Flow	633
图 19-15 Wi-Fi MAC Address handle in WLAN Driver	633
图 19-16 ATC Wi-Fi 配置框架	640
图 19-17 Wi-Fi MAC Address 策略	651
图 19-18 打开 TCP Dump 功能	654
图 20-1 Ethernet HW 连接图	659
图 20-2 Ethernet 架构图	661
图 20-3 Ethernet 流程图	662
图 21-1 GNSS 架构图	669
图 21-2 UC6226 数据流示意图	670
图 21-3 GNSS HAL 类图	679
图 22-1 MediaPlayer 架构图	686
图 22-2 MediaRecorder 框架图	687
图 22-3 MediaRecorder 数据流控制流图	688
图 22-4 MediaScanner 框架图	689
图 22-5 设置编码数据源时序图_Surface 模块	690
图 22-6 设置录制参数时序图	691
图 22-7 消息处理	692
图 22-8 获取 Video Thumbnail	693
图 23-1 IVI 系统 VBA 架构图	710
图 23-2 cockpit 系统 VBA 架构图	711
图 23-3 VBA 初始化调用流程图	711
图 23-4 APM 初始化 VBA 流程图	712

图 23-5 VBA 资源读取流程图.....	713
图 23-6 VBA 初始化流程图	714
图 23-7 VBA 退出消息流程图.....	715
图 23-8 VBA 单屏退出流程图.....	716
图 24-1 Backlight 架构图.....	726
图 24-2 Android 保存背光值流程图	727
图 24-3 HSM 读取背光值并设置	728
图 24-4 AP 背光设置操作界面图	733
图 25-1 亮度对比图	738
图 25-2 对比度对比图	739
图 25-3 饱和度对比图	739
图 25-4 色温对比图	739
图 25-5 Android PQ 架构图	741
图 25-6 Cluster PQ 架构图	742
图 25-7 PQ 初始化流程图.....	743
图 25-8 CarSetting APP.....	760
图 25-9 ATC Settings	760
图 25-10 Display.....	761
图 25-11 PQ 调节	761
图 25-12 gallery APP	762
图 25-13 进入 PQ 调节界面.....	762
图 25-14 UI 整体界面.....	763
图 25-15 主屏亮度	764
图 25-16 主屏对比度.....	764
图 25-17 主屏饱和度.....	765
图 25-18 主屏色温	766
图 25-19 副屏亮度	766
图 25-20 副屏对比度	767
图 25-21 副屏饱和度.....	767
图 25-22 主屏色温	768
图 25-23 参数应用	769
图 25-24 Metazone 配置	770
图 26-1 FreeRtos Display 架构图.....	773
图 26-2 FreeRTOS application 流程图	774

图 27-1 Safety Display 架构图	798
图 27-2 Safety Display 流程图	799
图 4-3 safety display(icon)示意图	811
图 28-1 Atcsurface 架构图	818
图 28-2 Atcsurface 流程图	819
图 29-1 LCM 架构图	840
图 29-2 LCM 流程图	841
图 29-3 屏幕分辨率、显示区域物理尺寸信息.....	842
图 29-4 屏幕 Timing、Clk、同步模式信息（Interface Timing-DE mod）	843
图 29-5 屏幕接口类型、数据格式信息（8Bit LVDS input）	844
图 29-6 VESA format（24.1）	844
图 29-7 JEIDA format（24.0，也叫做 DISM）	845
图 29-8 LOG 信息	848
图 29-9 DPIM 详细 log.....	849
图 29-10 锯齿纹.....	849
图 30-1 Android 显示架构图	852
图 30-2 dump SF	853
图 30-3 dump HWC.....	856
图 30-4 dump HWC layer config	857
图 31-1 Projection 架构图	864
图 31-2 Cluster Rendering Service onCreate 流程图	865
图 31-3 投屏 Buffer 同步流程图.....	866
图 31-4 nav map demo 投屏示例图 1	877
图 31-5 nav map demo 投屏示例图 2	877
图 31-6 nav map demo 投屏示例图 3	879
图 31-7 nav map demo 投屏示例图 4	879
图 31-8 nav map demo 投屏示例图 5	879
图 31-9 nav map demo 投屏示例图 6	880
图 31-10 nav map demo 投屏示例图 7	880
图 31-11 nav map demo 投屏示例图 8	881
图 31-12 nav map demo 投屏示例图 9	882
图 32-1 文件系统架构图	889
图 32-2 文件系统流程图	890
图 32-3 U 盘/SD 卡卸载流程	891

图 32-4 Settings.....	896
图 32-5 Setting Storage 显示界面	896
图 32-6 进入 Storage.....	897
图 32-7 Storage settings	897
图 32-8 格式化.....	897
图 32-9 确认	898
图 32-10 进度	898
图 32-11 结束	898
图 32-12 Migrate Data	899
图 32-13 Move Content	900
图 32-14 adoptable storage 完成.....	900
图 32-15 点击作为 Primary Storage 的外置存储设备	901
图 32-16 点击 forget 并确认	901
图 32-17 Storage 1.....	902
图 32-18 把 Adoptable Storage 格式化成外置存储设备	902
图 32-19 格式化.....	903
图 32-20 进度	903
图 32-21 查看外置存储设备状态	903
图 32-22 Auto Recovery 分区格式化流程	904
图 32-23 Auto Restore 分区格式化流程.....	905
图 32-24 添加 customer 分区	907
图 32-25 添加 customer 分区 flag.....	908
图 32-26 Customer 目录内容	908
图 32-27 image.mk 目录内容.....	909
图 32-28 customer.mk 内容	909
图 32-29 fstab 添加 customer 分区挂载.....	909
图 32-30 allmake_split.sh 添加 customer.img	910
图 32-31 8025 adas fstab 内容	910
图 32-32 8025 cluster fstab 内容.....	910
图 32-33 selinux 添加方法内容	912
图 32-34 config.fs 文件添加 customer 挂载点权限	913
图 32-35 project_config.json 中 FBE 的配置项	914
图 32-36 FBE 打开后的属性	914
图 32-37 密钥安装路径	916

图 32-38 debugfs 查看目录是否被配置了 FBE	917
图 32-39 使用 debugfs 根据查看目录是否加密	917
图 32-40 查看加密目录	917
图 32-41 低存储保护机制	918
图 32-42 状态转换流程图	919
图 32-43 Low Storage Get Status	920
图 32-44 MonitoredPathes.json 文件内容	920
图 32-45 打开写入频率与写入量统计工具	921
图 32-46 打开写入频率与写入量统计工具	922
图 33-1 metazone 系统架构图	925
图 33-2 metazone.bin 结构图	926
图 33-3 Dword item 数据结构	927
图 33-4 Binary item 数据结构	928
图 33-5 Reserved item 数据结构	929
图 33-6 Dword read flow	930
图 33-7 Dword write flow	931
图 33-8 metazone.csv 格式	962
图 33-9 metazone.csv 中新增 index 示意图	970
图 33-10 客户定制化 index 和 ATC 原生的 index 分区域	970
图 33-11 autochips.jar 解压后可看到 AtcMetazone.class	971
图 33-12 autochips.jar 解压后可看到 MetazoneIndex.class	971
图 34-1 Datazone 系统结构图	974
图 34-2 Datazone 结构图	975
图 34-3 Datazone 写数据流程图	976
图 34-4 Datazone 读数据流程图	977
图 34-5 Datazone adb 查看所有资源	998
图 34-6 Datazone adb 查看指定资源	998
图 34-7 datazone_config.json 中资源添加	999
图 35-1 高速设备连接电路示意图	1000
图 35-2 低速设备连接电路示意图	1000
图 35-3 USB3.0 连接电路示意图	1001
图 35-4 USB OTG host 模式架构	1003
图 35-5 USB OTG host 数据流	1004
图 35-6 USB OTG device 架构	1005

图 35-7 USB OTG device 数据流.....	1006
图 35-8 USB Host 30 模式架构	1007
图 35-9 USB Host 30 模式数据流.....	1008
图 35-10 Car Settings	1009
图 35-11 ATC Settings	1010
图 35-12 Other.....	1010
图 35-13 Usb Mode	1011
图 35-14 切换 host 模式成功	1011
图 35-15 切换 device 模式成功.....	1012
图 35-16 Car Settings	1013
图 35-17 ATC Settings	1013
图 35-18 Other.....	1014
图 35-19 OTG Protocol.....	1014
图 35-20 Car Settings	1015
图 35-21 ATC Settings	1016
图 35-22 Other.....	1016
图 35-23 USB Port1 Protocol.....	1017
图 35-24 Mode & protocol	1019
图 35-25 USB disk sd*	1020
图 35-26 Protocol.....	1022
图 36-1 SD0 和 eMMC 连接示意图.....	1023
图 36-2 SD1 和 SD 卡连接示意图	1024
图 36-3 SD1 和 SDIO Device 连接示意图	1024
图 36-4 Preloader/Safety os/LK eMMC/SD driver 软件架构图	1028
图 36-5 kernel mmc driver 软件架构图	1029
图 36-6 Preloader/Safety os/LK mmc driver 控制流和数据流	1030
图 36-7 lk mmc driver 控制流和数据流.....	1031
图 36-8 kernel mmc driver 控制流和数据流	1032
图 36-9 AC8025 Protect Region 示例.....	1036
图 36-10 eMMC 工作模式配置项	1039
图 36-11 lk 中 SD Card 工作模式配置项	1040
图 36-12 kernel 中 eMMC dts 配置示例	1041
图 37-1 UFS Host 和 UFS Device 原图示意图	1046
图 37-2 UFS HL/HSM/LK Driver 架构图.....	1048

图 37-3 UFS Kernel driver 架构图	1049
图 38-1 FastTouch 方式一架构图	1055
图 38-2 FastTouch 方式二架构图	1056
图 38-3 FastTouch 方案一工作流程图	1056
图 38-4 FastTouch 方案二工作流程图	1057
图 39-1 GPIO PINMUX 示意图	1060
图 39-2 Pinctrl 架构图	1061
图 40-1 GPIO PINMUX 示意图	1093
图 40-2 I2C 架构图	1094
图 40-3 I2C 读写流程图	1095
图 41-1 PWM 架构图	1107
图 41-2 PWM 流程图	1107
图 42-1 SPI PIN 示意图	1114
图 42-2 SPI 架构图	1115
图 42-3 SPI 读写流程图	1115
图 43-1 UART 架构图	1127
图 43-2 UART 读写流程图	1127
图 45-1 预览和录制 UI	1146
图 45-2 倒车 UI	1147
图 45-3 AtcCamera 架构图	1150
图 45-4 创建录制文件	1151
图 45-5 常规录制	1152
图 45-6 紧急录制	1153
图 46-1 Cluster App 架构图	1167
图 46-2 IVI 信息显示流程图	1169
图 46-3 Cluster 转场动画流程图	1170
图 46-4 Cluster 端接受 IVI 信息流程图	1171
图 46-5 Cluster 端接受 IVI 投屏信息流程图	1172
图 47-1 ControlPanelUI 架构图	1187
图 47-2 流程图	1188
图 48-1 HUD App 架构图	1191
图 48-2 HUD 导航信息显示流程图	1192
图 49-1 ToolKit 架构图	1198
图 49-2 ATCNN SDK 架构图	1199

图 49-3 ATCNN ToolKit 流程图.....	1200
图 49-4 ATCNN C++接口调用时序图.....	1201
图 49-5 ATCNN Service C++接口调用时序图	1202
图 49-6 ATCNN Java 接口调用时序图.....	1203
图 49-7 ATCNN-Toolkit tar 包内容示意图	1228
图 49-8 Caffe 模型配置示例	1236
图 49-9 ONNX 模型配置示例.....	1236
图 49-10 Tensorflow-Lite 模型配置示例	1237
图 49-11 Darknet 模型配置示例	1237
图 49-12 Keras 模型配置示例	1238
图 49-13 Tensorflow 模型配置示例	1238
图 49-14 Pytorch 模型配置示例.....	1239
图 49-15 yolov3 检测网络预处理参数设置	1240
图 49-16 模型剪枝示例	1241
图 49-17 模型量化数据集配置示例.....	1241
图 49-18 多输入模型数据集描述文件示例	1241
图 49-19 模型量化信息配置示例	1242
图 49-20 lenet 模型转换过程打印的逐层精度信息	1243
图 49-21 lenet 推理示例.....	1244
图 49-22 lenet 性能分析示例.....	1245

表格目录

表 2-1 Android 版本	85
表 4-1 Android Security 代码结构及描述	117
表 5-1 IPC 源码目录和文件	127
表 5-2 IPC Kernel driver API	131
表 5-3 IPC HSM OS API	139
表 5-4 LIBIPC API	139
表 5-5 Channel 配置	144
表 5-6 专有 Channel 配置	145
表 5-7 IPC 典型问题	147
表 6-1 Safety 源码目录	150
表 6-2 Fast service	151
表 6-3 fault 描述	161
表 6-4 FM 接口说明	166
表 6-5 Crypto service	170
表 6-6 中断函数列表	180
表 6-7 AT 接口说明	187
表 7-1 secure boot 代码结构及描述	189
表 7-2 secure boot 接口描述	193
表 8-1 Shutdown 相关代码路径	199
表 8-2 Shutdown 相关 API	202
表 8-3 Shutdown 相关数据结构	207
表 8-4 Shutdown 相关数据结构	208
表 9-1 Thermal 代码结构及描述	215
表 9-2 Thermal API	217
表 10-1 LK 拷贝升级代码结构和描述	222
表 10-2 LK Fastboot 升级代码结构和描述	222
表 10-3 flag 说明表	228
表 11-1 AB OTA 代码结构及描述	236
表 12-1 recovery 代码结构及描述	253
表 12-2 系统支持的 recovery 命令描述	257
表 12-3 RecoveryUI 及描述	259

表 13-1 JEC 可捕获类型表	269
表 13-2 代码路径及文件描述	271
表 13-3 AEE Kernel 层 API 参数说明	277
表 13-4 AEE Native 层 API 参数说明	278
表 13-5 AEE Native 层 API 参数说明	278
表 13-6 AEE 数据结构	279
表 13-7 AEE 文件内容说明	281
表 13-8 ATCLogger 代码结构及描述	290
表 13-9 ivi log 操作描述	296
表 14-1 CameraSource 代码结构及描述	302
表 14-2 Videoin Driver 代码结构及描述	303
表 14-3 Sensor Driver 代码结构及描述	303
表 14-4 IEventCallback 类	304
表 14-5 ICameraStream 类	305
表 14-6 ICameraSource 类	317
表 14-7 ICameraCallback 类	323
表 14-8 ICameraManager 类	324
表 14-9 CameraSource 数据结构	327
表 14-10 定制化 Vendor Tag 说明	329
表 14-11 Sensor 工作域配置 Metazone 说明	329
表 14-12 Sensor 通道使能配置 Metazone 说明	330
表 14-13 Sensor 通道制式配置 Metazone 说明	330
表 14-14 MIPI1 工作模式配置 Metazone 说明	331
表 14-15 atc_camera_config.xml 配置字段说明	333
表 14-16 接口调用顺序表	337
表 15-1 RVC 代码结构及描述	354
表 15-2 RVC Framework API	359
表 15-3 RVC Main 接口	363
表 15-4 RVC Video 接口	363
表 15-5 RVC UI 接口	364
表 15-6 RVC Audio 接口	364
表 15-7 RVC Event Monitor 接口	365
表 15-8 R5 RVC 数据结构	365
表 15-9 Display 优先级设定	368

表 15-10 R5 RVC 显示配置相关 Metazone	369
表 15-11 R5 RVC 输入源配置相关 Metazone.....	370
表 16-1 Watermark 代码结构及描述	378
表 16-2 Watermarkcore 代码结构及描述	379
表 16-3 IWatermarkWrapper 类	385
表 16-4 IWatermarkManger 类	387
表 16-5 IMergeExcutor 类.....	389
表 16-6 Watermark 数据结构.....	393
表 16-7 水印配置文件各字段说明.....	396
表 16-8 示例 watermark_log.cmd 各字段含义	403
表 16-9 示例 watermark_dump.cmd 各字段含义	403
表 17-1 配置文件描述	409
表 17-2 streamType 和 Int 值映射关系表	417
表 17-3 ALSA 节点介绍	419
表 17-4 ALSA Patg 相关 Kcontrol	420
表 17-5 sink 标识与含义关系表.....	427
表 17-6 CarAudioManager 接口	433
表 17-7 AudioManager 类音量设置接口	437
表 17-8 ATCAudioManager 接口	439
表 17-9 非 IVIstreamType 说明	440
表 17-10 代码接口和文件描述	442
表 17-11 AudioVolumePolicy 接口.....	447
表 17-12 volumepolicy.xml 字段及其含义.....	451
表 17-13 非 IVI 系统音源 name 标识与 streamType 对应关系表	451
表 17-14 PhoneAudio IVI 系统音源 name 标识及含义	455
表 17-15 alsa device 与音量 Kcontrol 映射表.....	459
表 17-16 音效代码结构和文件描述.....	472
表 17-17 Bass 类接口说明.....	472
表 17-18 Loudness 类接口说明.....	473
表 17-19 Balance 类接口说明	473
表 17-20 SpeakerLayout 类接口说明	473
表 17-21 AutoSurround 类接口说明.....	474
表 17-22 ChannelDelay 类接口说明.....	475
表 17-23 EQ 类接口说明	475

表 17-24 AudioSettings 类	476
表 17-25 Effect ID 列举	477
表 17-26 ALSA SoC driver 代码结构及描述	488
表 17-27 DSP 音效 Kcontrol API	489
表 17-28 bit 与声道对应表	500
表 17-29 Bass Gain 值与 dB 值对应表	503
表 17-30 Bass 中心频率 value 表.....	504
表 17-31 Bass Q 值 value 表.....	505
表 17-32 Bass Gain 值与 dB 映射表.....	505
表 17-33 Middle Gain 值与 dB 值对应表.....	505
表 17-34 Middle 中心频率 value 表	505
表 17-35 Middle Q 值 value 表	506
表 17-36 Middle Gain 值与 dB 映射表	506
表 17-37 Treble Gain 值与 dB 值对应表	506
表 17-38 Treble 中心频率 value 表.....	506
表 17-39 Treble Q 值 value 表.....	507
表 17-40 Treble Gain 值与 dB 映射表	507
表 17-41 Master Volume dB.....	507
表 17-42 DL1 volume dB	509
表 17-43 Balance Fader index dB 对应表.....	511
表 17-44 EQ 预设类型表	517
表 17-45 limiter pre gain 对应表.....	524
表 17-46 limiter post gain 对应表	525
表 17-47 音效参数读取方式.....	527
表 17-48 NR 力度控制.....	532
表 17-49 NLP 控制.....	532
表 17-50 ES 控制.....	533
表 17-51 AES 控制	533
表 17-52 Fast Audio 代码结构及描述	542
表 17-53 I2S 资源.....	554
表 17-54 PCM 资源	554
表 17-55 Audio ALSA SOC Driver 代码结构及描述	556
表 17-56 Kcontrol API.....	558
表 17-57 Alsa Api 代码结构及描述	559

表 17-58 Alsa Api API Overview	559
表 17-59 framework 层普通 dump	571
表 17-60 HAL 层普通 dump	571
表 17-61 Framework 层 cycledump	572
表 17-62 HAL 层 cycledump	573
表 17-63 常用 Key 值	574
表 18-1 蓝牙功能列表	575
表 18-2 BT 代码结构及描述	577
表 18-3 BluetoothAtcGap API	589
表 18-4 BluetoothAtcGap 广播参数	590
表 18-5 BluetoothAtcMapClient API	591
表 18-6 BluetoothAtcMapClient 广播参数	591
表 18-7 BluetoothAtcPbapClient API	592
表 18-8 电话本广播	592
表 19-1 Wi-Fi 代码结构及描述	614
表 20-1 Ethernet 代码结构及描述	660
表 20-2 Ethernet 已适配过的 PHY 芯片型号	663
表 20-3 Ethernet PHY datasheet 确认项	663
表 20-4 Ethernet PHY 硬件确认项	663
表 20-5 Ethernet PHY 软件确认项	664
表 21-1 GNSS 代码路径及描述	667
表 21-2 GNSS 动态库及描述	668
表 21-3 GNSS 配置文件及描述	668
表 21-4 卫星编号	674
表 21-5 TTFF 标准	676
表 21-6 TTFF 启动条件	677
表 22-1 MediaRecorder 支持列表	682
表 22-2 存储设备扫描性能	683
表 22-3 Multimedia 源文件	684
表 22-4 Multimedia 动态库	685
表 22-5 MediaCodec 编解码支持情况	700
表 23-1 VBA 交互行为表	709
表 23-2 VBA 代码结构及描述	710
表 23-3 VBA API	716

表 23-4 VBA 数据结构	720
表 24-1 BackLight 代码结构及描述	724
表 24-2 BackLight API.....	728
表 24-3 BackLight 数据结构	731
表 25-1 PQ 代码结构及描述	740
表 25-2 Java 层接口	744
表 25-3 Java 层接口	751
表 25-4 Hsm 接口.....	757
表 26-1 Display Framework 代码结构及描述	772
表 26-2 FreeRTOS Display Framework API	775
表 26-3 FreeRTOS Display 数据结构	793
表 27-1 Safety Display 代码结构及描述	797
表 27-2 Safety Display API.....	800
表 27-3 Safety Display 数据结构.....	810
表 28-1 Atcsurface 代码结构及描述.....	817
表 28-2 Atcsurface API	820
表 28-3 Atcsurface 数据结构	833
表 29-1 LCM 代码结构及描述	838
表 29-2 Kernel 部分代码结构及描述	839
表 29-3 屏参填写说明	843
表 29-4 数据格式填表说明	845
表 31-1 Projection 代码结构及描述	863
表 31-2 CarAppFocusManager API	867
表 31-3 InterdomainChannel API.....	869
表 31-4 IInterdomainChannelProxy API.....	873
表 31-5 IInterdomainChannelMonitor API	874
表 31-6 Projection 数据结构	876
表 32-1 Filesystem 代码结构及描述	888
表 32-2 Filesystem StorageManagerEx 接口	892
表 32-3 Filesystem Methods Summary	892
表 32-4 mkdir encryption 行为	915
表 33-1 metazone 代码结构和描述	923
表 33-2 Dword Flag 标志位定义	927
表 33-3 Binary Flag 标志位定义.....	928

表 33-4 Reserved Flag 标志位定义	929
表 33-5 AtcMetazone class	931
表 33-6 Native APIs	937
表 33-7 LK APIs	944
表 33-8 Safety OS APIs	950
表 33-9 kernel driver APIs	956
表 34-1 datazone 代码结构和描述	972
表 34-2 Datazone API 错误码表	977
表 34-3 Safety os API	978
表 34-4 LK API	981
表 34-5 Kernel API	984
表 34-6 Native API	987
表 34-7 Framework API	992
表 35-1 USB OTG/ USB Host30 Controller 代码结构及描述	1002
表 35-2 USB 设备树配置	1002
表 35-3 切换参数说明	1018
表 35-4 参数说明	1021
表 36-1 Preloader SD4HC driver 代码结构及描述	1025
表 36-2 Safety os SD4HC driver 代码结构及描述	1026
表 36-3 lk SD4HC driver 代码结构及描述	1026
表 36-4 kernel SD4HC driver 代码结构及描述	1027
表 36-5 kernel mmc driver dts 配置项描述	1033
表 36-6 eMMC/SD 各阶段 clock 及 mode 配置	1034
表 36-7 eMMC/SD 在 preloader 的接口描述	1038
表 36-8 eMMC/SD 在 safety os 的接口描述	1038
表 36-9 eMMC/SD 在 LK 的接口描述	1039
表 37-1 UFS 在 hsm loader 的接口描述	1049
表 37-2 UFS 在 hsm os 的接口描述	1050
表 37-3 UFS 在 lk 的接口描述	1050
表 38-1 FastTouch 代码结构及描述	1054
表 38-2 API in kernel	1057
表 38-3 API in hsm	1058
表 39-1 GPIO PINMUX 代码结构及描述	1061
表 39-2 API in kernel	1062

表 39-3 AGPIO API in hsm	1066
表 39-4 SGPIO API in hsm.....	1074
表 40-1 I2C 代码结构及描述	1094
表 40-2 I2C kernel API	1095
表 40-3 I2C hsm API.....	1096
表 40-4 I2C 数据结构	1098
表 41-1 PWM 代码结构及描述	1106
表 41-2 PWM API.....	1107
表 41-3 PWM API.....	1109
表 41-4 PWM 数据结构	1111
表 42-1 SPI 代码结构及描述	1114
表 42-2 SPI kernel API	1116
表 42-3 SPI API in hsm	1116
表 43-1 UART 代码结构及描述	1126
表 43-2 UART kernel API.....	1128
表 43-3 UART hsm API	1130
表 44-1 PCIe 代码结构及描述	1143
表 45-1 AtcCamera APP 代码结构及描述	1147
表 45-2 MIPIDVRActivity 类方法	1154
表 45-3 类 CameraServiceImpl 方法说明	1154
表 45-4 类 CameraStateMachine 方法说明	1155
表 45-5 类 CameraOperation 方法说明	1156
表 45-6 类 StorageUtil 方法说明	1158
表 45-7 倒车中用到的 metazone 地址	1161
表 46-1 Cluster 代码结构及描述	1166
表 46-2 ClusterCtrlManager API.....	1173
表 46-3 IInterdomainChannelProxy API.....	1176
表 46-4 idc 数据结构	1178
表 47-1 ControlPanelUI 代码结构及描述	1185
表 48-1 HUD 代码结构及描述	1190
表 48-2 HudServiceManager API	1193
表 49-1 ToolKit 第三方依赖库	1194
表 49-2 ToolKit 依赖的硬件环境	1195
表 49-3 ToolKit 类名/接口	1203

表 49-4 ATCNN SDK	1206
表 49-5 ATCNN Service C++ SDK	1210
表 49-6 ATCNN JAVA SDK	1215
表 49-7 ATCNN SDK 数据结构	1221
表 49-8 ATCNN SDK 数据结构	1225
表 49-9 配置文件属性域信息	1230
表 49-10 模型描述信息	1230
表 49-11 输入节点描述信息(input_node)	1231
表 49-12 预处理节点描述信息	1231
表 49-13 预处理节点具体操作描述信息	1231
表 49-14 预处理节点之缩放操作描述信息	1232
表 49-15 预处理节点之裁剪操作描述信息	1232
表 49-16 网络优化项描述信息	1233
表 49-17 量化数据集描述信息(dataset_node)	1233
表 49-18 输入数据描述信息(node_info)	1233
表 49-19 剪枝方法描述(opt_prune_node)	1233
表 49-20 量化方法描述(quantization_node)	1234
表 49-21 混合量化层描述信息	1235
表 49-22 单层量化描述信息	1235
表 49-23 Caffe 模型导入配置	1235
表 49-24 ONNX 模型导入配置	1236
表 49-25 Tensorflow-Lite 模型导入配置	1236
表 49-26 Darknet 模型导入配置	1237
表 49-27 Keras 模型导入配置	1237
表 49-28 Tensorflow 模型导入配置	1238
表 49-29 Pytorch 模型导入配置	1238
表 49-30 模型预处理设置示例	1239
表 49-31 模型剪枝设置示例	1240
表 49-32 量化数据库配置示例	1241
表 49-33 量化方式设置示例	1241

缩略语

A2DP	Advanced Audio Distribution Profile	音频传输协议
AAC	Advanced Audio Coding	一种音频编码方式
AAH	Autochips Automotive Hypervisor	ATC 自动驾驶管理程序
AAOS	Android Automotive OS	Android 的 Automotive 版本系统
ACA	Accessory Charger Adapter	附件充电器适配器
ADAS	Advanced Drive Assist System	
ADB	Android Debug Bridge	安卓调试桥
AED	Android Exception Daemon/ AEE Native Service	
AED Device	AED Kernel Driver	
AEE	Autochips Exception Engine/Enhancement	
AES	Aggressive Echo Suppression	强回声抑制
AIDL	Android Interface Definition Language	安卓接口定义语言
ANR	Application Not Responding	
AP	Application Processor	平台主要功能业务处理器
AP	Access Point	访问点，接入点
API	Application Programming Interface	应用程序接口
APM	ARM2(Vehicle) Policy Manager	Vehicle 资源管理器
AT	Address Tranlation	
ATC	AutoChips	杰发科技
ATC-DPU	Autochips Display Process Unit	ATC 显示处理器
ATF	ARM Trusted Firmware	ARM 可信任固件
AVB	Android Verified Boot	Android 启动验证功能
AVM	Around View Monitor	周围视野监测器
AVRCP	Audio/Video Remote Control Profile	音频/视频远程控制协议
AWB	Audio Write Back	DSP 输出的音频数据写回 Dram
AWDT	AP Watchdog	
BC	Battery Charging	电池充电

BT	Bluetooth	蓝牙
CA	Client Application	客户端应用
CarAudio	AutoMotive Audio	Android 中支持多音区的 Audio 框架
CDP	Charging Downstream Port	充电下接端口
CSI-2	Camera Serial Interface 2	摄像头串行接口 2
DAL	Display Assert Layer	发生异常后，在桌面覆盖一层红色，提醒用户发生异常并显示异常信息
DCP	Dedicated Charging Port	专用充电端口
DHCP	Dynamic Host Configuration Protocol	动态主机设置协议
DL	Downlink	下行
DMS	Driver Monitoring System	驾驶员监测系统
DNS	Domain Name System	域名服务器
DP	DisplayPort	VESA 协会下的一种视频传输协议
DPIM	Display Path Information Manage	使用 MRF tool 生成的文件的名字
DPU	Display Process Unit	显示处理器
DRAM	Dynamic Random Access Memory	动态随机存取存储器
DRC	Dynamic Range Control	动态范围控制
DSI	Display Serial Interface	显示串行接口，MIPI 的一种接口类型
DSMS	Device Storage Monitor Service	
DTB	Device Tree Blob	Linux 系统设备树描述二进制文件
DTMF	Dual-tone multifrequency	电话按键编码
DVR	Digital Video Recorder	行车记录仪功能
EEC	External Exception Cather	
EMI	Electromagnetic Interference	电磁干扰
eMMC	embedded Multi Media Card	嵌入式设备上常用的内嵌式存储器
EP	Endpoint	端点
ES	Echo Suppression	回声抑制
EXFAT	Extended File Allocation Table	

FM	Fault Monitor	
GAP	Generic Access Profile	通用访问协议
GC	P2P Group Client	P2P 组用户
GNSS	Global Navigation Satellite System	全球导航卫星系统
GO	P2P Group Owner	P2P 组所有者
HAL	Hardware Abstraction Layer	Android 系统的硬件抽象层
HFP	Hand Free Profile	免提协议
HIDL	Hardware Interface Definition Language	硬件接口描述语言
H-MTD	High Speed-Modular Twisted-pair Data	车载双绞线高速数据传输模块
HSM	Hardware Security Module	硬件安全模块
HUD	Head Up Display	抬头显示
HVAC	Heating Ventilation and Air Conditioning	供暖通风与空气调节
HWT	Hardware Watchdog Timeout	
I2C	Inter-Integrated Circuit	I2C 总线
I2S	Inter-IC Sound	
I2S SC	I2S single channel	每组 I2S 有一根 DIO 和一根 DO PIN
I2S MC	I2S Multi channel	此 I2S 有三根 DO PIN
IDC	Inter-Domain Channel	用于实现不同域之间的安全通信和数据共享
INTx	Pin-based Interrupt	
IPC	Inter-Process Communication	进程间通信机制
ISP	Image Signal Processor	图像信号处理器
IR	Intermediate Representation	中间表达形式
IVI	In-Vehicle Infotainment	车载信息娱乐系统
JE	Jave Exception	
JEC	Java Exception Catcher	
KE	Kernel Exception	
KEC	Kernel Exception Catcher	
L2	Data Link Layer	数据链路层

L3	Network Layer	网络层
LCM	Liquid Crystal Module	液晶显示模组
LE	Low Energy	蓝牙低功耗
LK	Little Kernel	作为 Andriod 系统的 bootloader
LVDS	Low-Voltage Differential Signaling	低压差分信号，一种信号传输模式
MAC	Medium Access Control	介质访问控制
MCU	Micro programmed Control Unit	用于自动化测试，或车载控制的外部单片
MDIO	Management Data Input / Output	管理数据输入输出，控制 PHY 芯片
MIPI	Mobile Industry Processor Interface	移动产业处理器接口
MRF	Multi Resource File	用于把配置信息打包为一个 dpim.bin 文件的工具软件的名字
MSI	Message Signaled Interrupt	
NDC	NR + DRC	降噪与动态范围控制
NDK	Native Development Kit	允许开发人员使用 C、C++ 和其他 本地语言编写 Android 应用程序的 工具集
NE	Native Exception	
NEC	Native Exception Catcher	
NLP	Non-Linear Process	非线性处理
NMEA	National Marine Electronics Association	美国国家海洋电子协会
NPU	Neural Processing Unit	神经处理单元
NR	Noise Reduction	降噪
NTFS	New Technology File System	
NTFS-3g	Third Generation Read/Write NTFS Driver	
OPP	Object Push Profile	目标文件传输协议
OP-TEE	Open Platform Trusted Execution Environment	开源可信执行环境
OS	Operating System	操作系统
OSS	Open Source Software	开放源码软件
OTA	Over the air	Android 系统更新系统软件的一种升

		级方式
OTG	On The Go	电子设备数据交换技术
P2P	Peer-to-Peer	点对点
PAN	Personal Area Networking Profile	蓝牙热点协议
PBAP	Phone Book Access Profile	电话本访问协议
PCIe	PCI Express (Peripheral Component Interconnect Express)	
PCM	Pulse-code modulation	脉宽调制
PhoneAudio		Android 非 Automotive Audio 框架
PHY	Physical layer	物理层协议或芯片
PM	Power Management	电源管理
PQ	Picture Quality	图像质量
RC	Root Complex	
RDS	Remote Domain Surface	用于实现在不同安全域之间的图形渲染共享 surface
REE	Rich Execution Environment	丰富执行环境
RGMII	Reduced Gigabit Media Independent Interface	精简的千兆介质无关接口
RMII	Reduced Media Independent Interface	精简介质无关接口，用于大数据传输
RP	Root Port	
PWM	Pulse Width Modulation	脉冲宽度调制
RVC	Rear View Camera	倒车影像系统
SCO Link	Synchronous Connection Oriented Link	面向连接的同步连接
SDP	Service Discovery Protocol	服务发现协议
SL	Software Link	Linux 软连接
SoC	System on Chip	
SPI	Serial Peripheral Interface	串行外设接口
SPP	Serial Port Profile	串口协议
SRC	Sample rate converter	采样率转换
SSID	Service Set Identifier	服务组标识符
SSP	Secure Simple Pairing	简单配对方式

SWDT	Safety Watchdog	
SWT	Software Watchdog Timeout	
TA	Trusted Application	运行于 TEE 中的安全应用
TDM	Time Division Multiplexing	时分复用方式，支持传输多个声道的数据
TEE	Trusted Execution Environment	可信任的软件安全执行环境
TEE OS	Trusted Execution Environment Operation System	可信内核系统
UART	Universal Asynchronous Receiver/Transmitter	通用非同步收发传输器
UDP	User Datagram Protocol	用户数据报协议
UFS	Universal Flash Storage	嵌入式设备上常用的内嵌式存储器
UI	User Interface	用户界面
UL	Uplink	上行
URB	USB Request Block	USB 请求块
USB	Universal Serial Bus	通用串行总线
V4L2	Video for Linux 2	用于 Linux 操作系统中的视频捕获和输出设备的框架
VBA	Video Boot Animation	快速开机动画
VFAT	Virtual File Allocation Table	
VFS	Virtual File System	虚拟文件系统
VHAL	Vehicle Hardware Abstraction Layer	Android Automotive 中 HAL 层实现
VOLD	Volume Daemon	
WDT	Watchdog Timer	
Wi-Fi	Wireless Fidelity	无线保真技术

1 系统架构

1.1 简介

1.1.1 软硬件环境

1.1.1.1 软件环境

AC8025 默认支持两种软件形态，分别为IVI形态和Cockpit形态。

IVI形态默认在AP端只有一个IVI OS，基于Google Android 12 Automotive，Kernel基于Android Kernel-5.10；R5F Lock-Step端对应的Safety OS基于FreeRTOS v10.0.1。

Cockpit系统对应座舱系统，AP端支持多个OS同时运行，比如默认支持IVI（Android 12）、Cluster（Linux）、ADAS（Linux）；R5F（默认Lock-Step模式）端对应的Safety OS则同IVI形态一样，基于FreeRTOS v10.0.1。

1.1.1.2 硬件环境

AC8025 IC 是一款高度集成平台，集成了 AP 子系统（基于 A76 * 2 + A55 * 6），Safety 子系统（基于 R5F Lock-Step），存储子系统，显示子系统，Audio 子系统，Video Input 和 Video Codec 子系统等。

1.2 软件设计

1.2.1 软件架构

IVI 形态的 AC8025 软件总体架构图如下图所示：

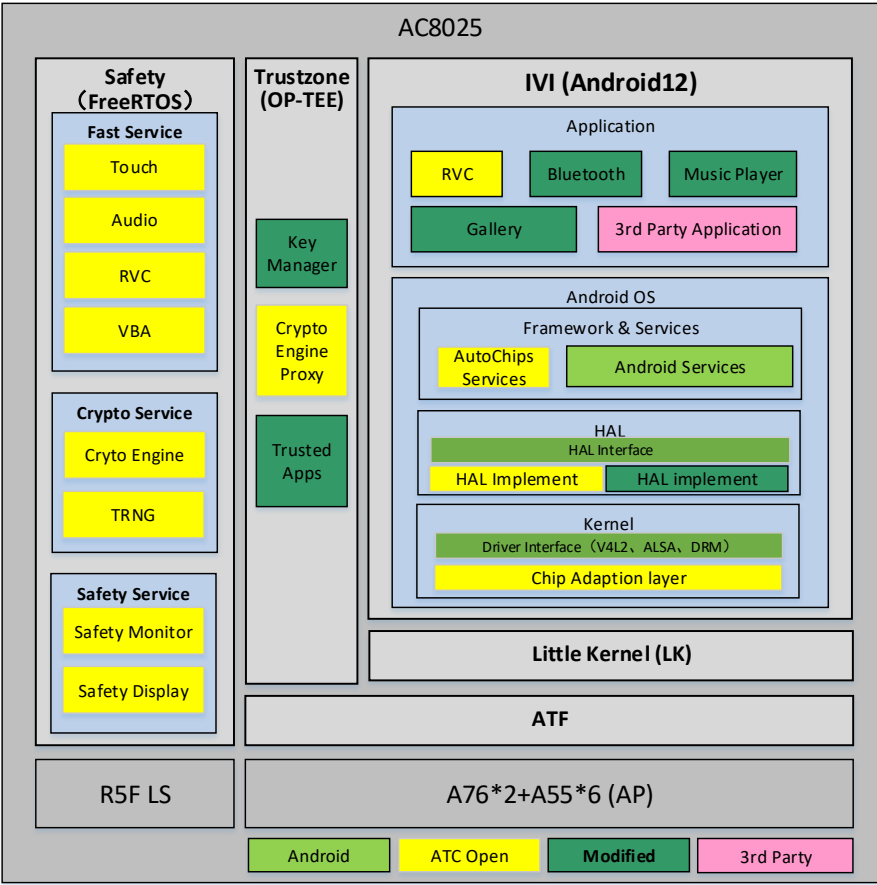


图 1-1 软件总体架构图-IVI 形态

Cockpit 形态的 AC8025 软件总体架构图如下图所示：

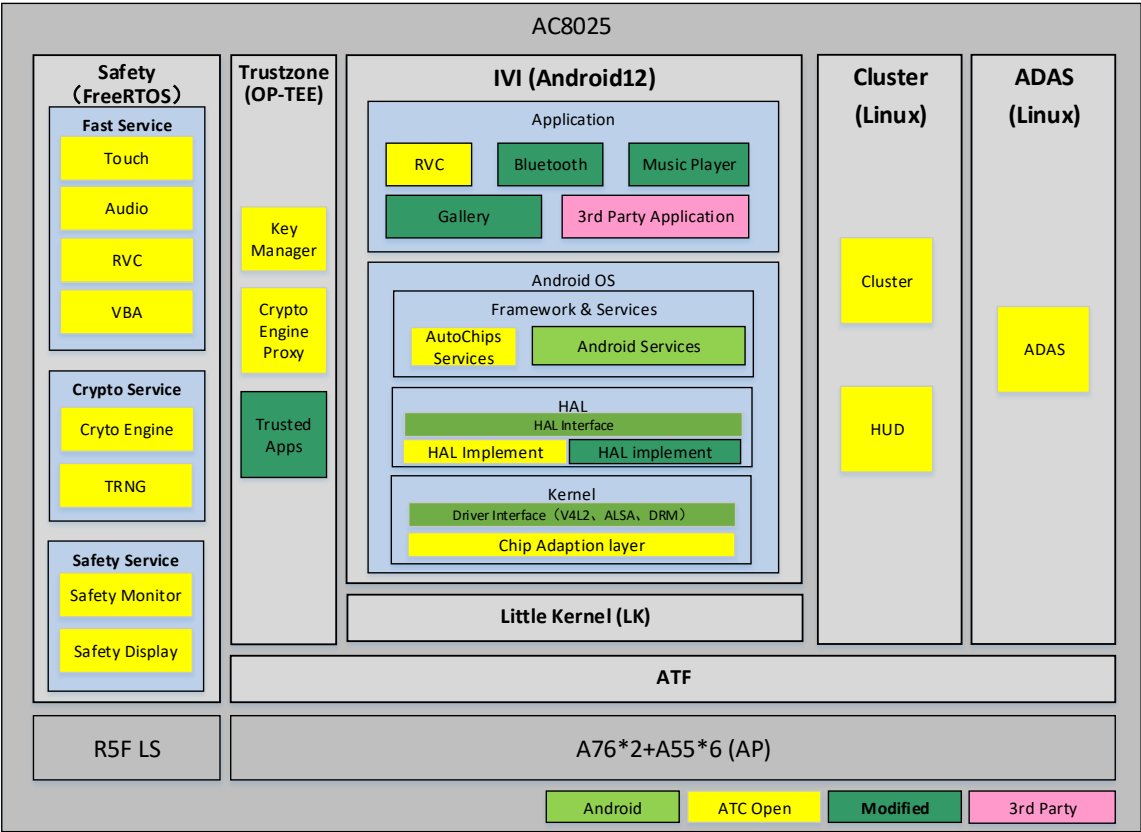


图 1-2 软件总体架构图-Cockpit 形态

其中 Safety OS 都是运行在 R5F 核（Lock-Step）上，是 AC8025 Safety 子系统的重要组成部分。

AP 端的系统区分 IVI 形态和 Cockpit 形态，Cockpit 形态相较于 IVI 形态，会在 AP 端运行更多的系统（默认多了两个 Linux OS），多系统通过物理分核的方式支持运行。IVI 形态包含 OP-TEE、Android OS、Little Kernel、ATF 等系统，Little Kernel 小系统是 Android 系统在冷启动阶段对 bootloader 功能的增强，因此在 Android 系统启动后即不再运行。

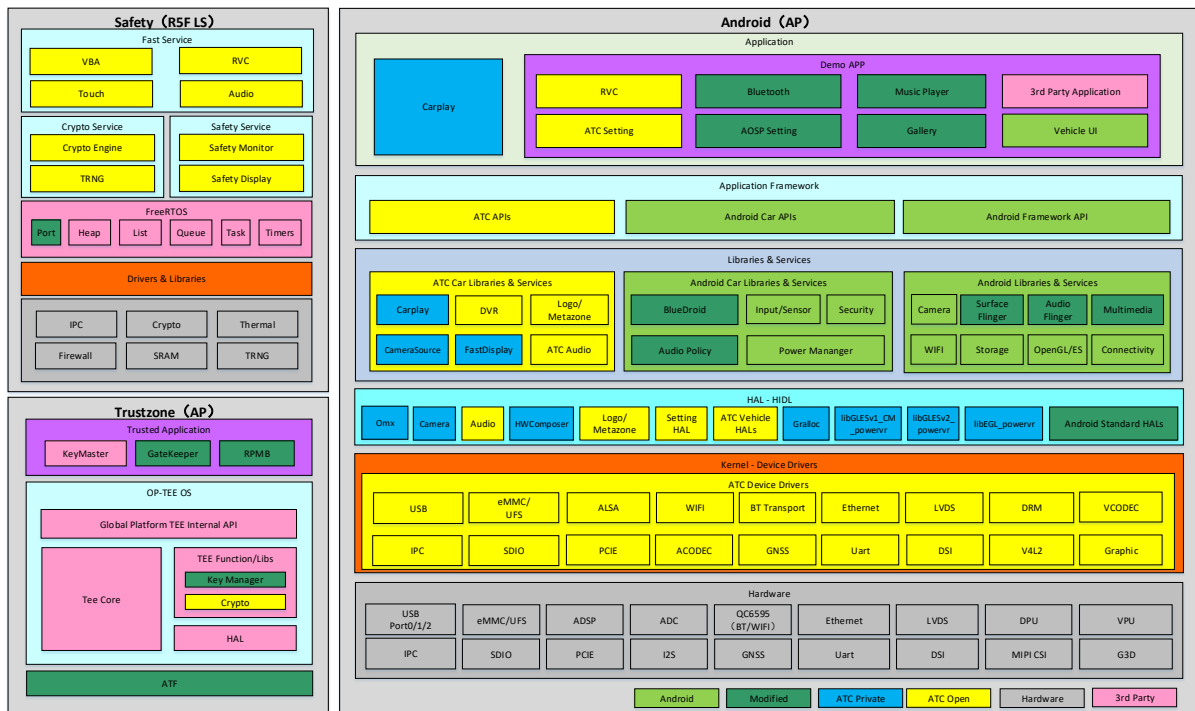


图 1-3 软件系统架构图

IVI 形态的 AC8025 在运行期，软件上主要分为 3 个操作系统模块（如图 1-3 所示），分别为：

- Android OS，IVI 主要软件功能实现的载体系统；
- Trustzone，包含 ATF 和基于 OP-TEE OS 实现的 TEE 环境；
- Safety OS，由 R5F 核（Lock-Step）运行的、基于 FreeRTOS 操作系统，业务包含快速启动相关功能（Fast RVC，Fast Touch 等）及安全功能支持等。

Cockpit 形态则在上述操作系统模块基础上，默认多出 Cluster 和 ADAS 操作系统（可按需配置操作系统数量或类型），均基于 Linux OS 并根据需要搭载 Cluster 相关应用或者 ADAS 应用。

1.2.1.1 Android OS

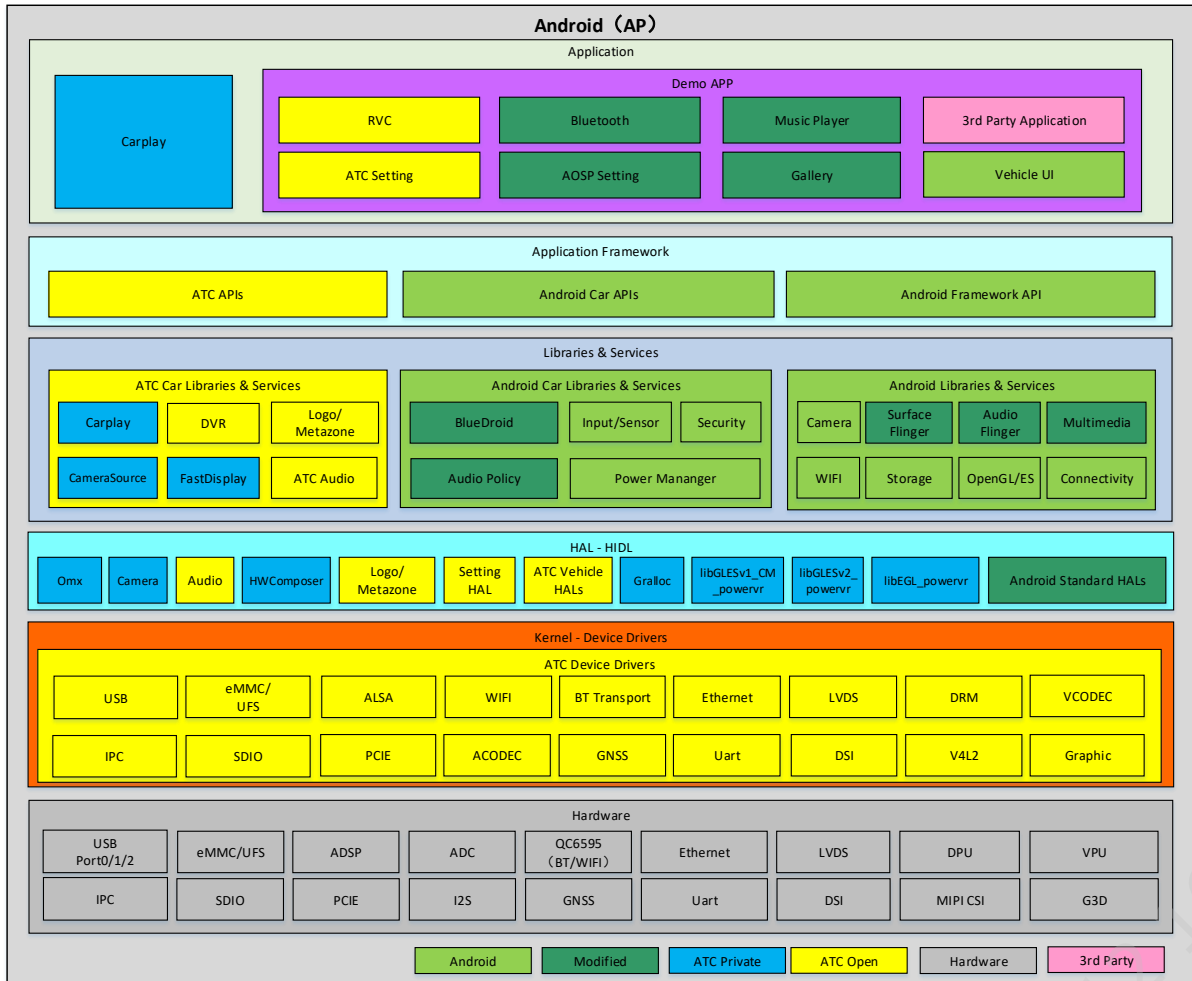


图 1-4 Android OS 架构图

Android OS 包含如下架构分层:

- Application
- Application Framework
- Libraries & Services
- HAL-HIDL
- Device Drivers

1.2.1.1.1 Application

AC8025 Android Application 层是使用 Java 程序语言开发的应用程序所在的系统架构层级。应用程序的开发基于 Android 开源开发包和 ATC 提供的 SDK。AC8025 Android Application 层包含了

AOSP 自带的必要系统应用程序、项目需要的三方应用程序和 ATC 根据项目需求开发发布的应用程序。

1.2.1.1.2 Application Framework

AC8025 Android Application Framework 层包含基于 ATC (AutoChips) Framework 和 AAOS (Android Automotive OS) Framework 所提供的 ATC Framework APIs、Android Framework APIs 以及 Android Automotive (Car) APIs, 以支撑基于 AAOS SDK 和 ATC SDK 的 APP 功能开发, 该些 API 均为 Java API。ATC SDK 提供了平台级定制的功能支持, 如 Audio、RVC 等相关接口开发包。

1.2.1.1.3 Libraries & Services

AC8025 Android libraries 层包含系统运行所需的各类库文件和 Services 实现, 该层业务都是基于 C/C++ 开发, 是承接 Application Framework 和 HAL 层的核心运行层。

Android 系统中重要的 Native System Service 包括多媒体播放服务 (MediaPlayerService)、显示相关服务 (SurfaceFlinger)、音频相关服务 (AudioFlinger)、摄像头相关服务 (Camera Service) 等。

ATC Car Services 部分包括音频相关服务 (ATC Audio)、摄像头录制服务 (DVR) 等。

1.2.1.1.4 HAL-HIDL

Hardware Abstract Layer (HAL) 层是对平台硬件功能的抽象封装, 在较新的 Android OS 版本中, 为了适配 Android Treble 化, 一个 HAL 通常实现为一个 HIDL (Hardware Interface Definition Language) 服务, 以 HIDL 接口形式向 System 分区的系统进程提供跨进程通信功能。

AC8025 全面支持 Android Treble 化, 所有 HAL 层实现均会符合 Android Treble 化要求, 并实现 HIDL 接口的版本化管理, 支持 Android 系统快速升级。

1.2.1.1.5 Device Drivers

Android 平台的基础是 Linux 内核, Device Drivers 层是基于 Linux 内核的硬件驱动程序实现层。

1.2.1.2 Trustzone

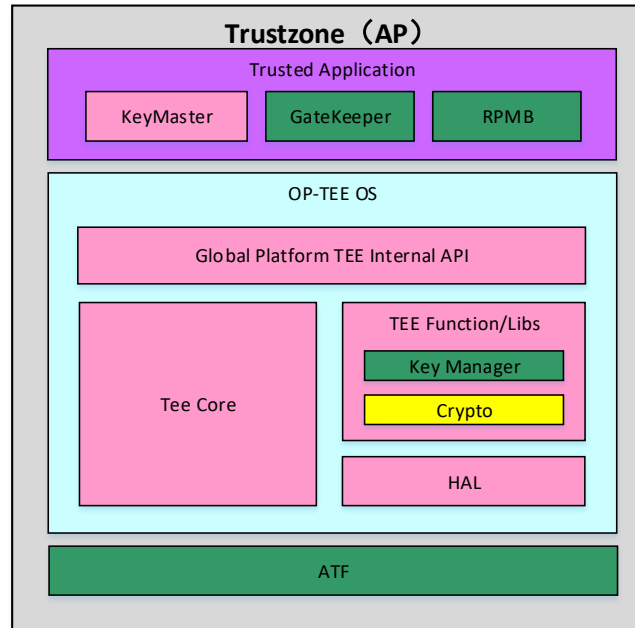


图 1-5 Trustzone 架构图

和 Android OS 一样运行于 AP 核上的 Trustzone，包含 ATF（ARM Trusted Firmware）和 OP-TEE 两个部分。其中，OP-TEE 是 AC8025 的 Trusted OS 解决方案，主要包含 TA（Trusted Application）和 OP-TEE Framework 两部分。

1.2.1.2.1 ATF

AC8025 ATF 是 v2.3 版本，主要做了一些平台相关适配，以及和 OP-TEE 的对接。

1.2.1.2.2 OP-TEE TA

AC8025 上的 OP-TEE TA，是基于 OP-TEE 实现的若干 Trustzone 应用，用以支撑相关安全功能。包含 Android Keystore 对应的 Key Master TA，以及 Android GateKeeper TA，和支持 RPMB Storage 访问的 RPMB TA 等。

1.2.1.2.3 OP-TEE OS

AC8025 上的 OP-TEE OS，在开源 OP-TEE 的 OS 基础上，主要做了平台级硬件功能的对接，比如 Key Manager 等，同时包含 Safety 端 Crypto Engine 的在 AP 端 Proxy 实现，提供基于 Crypto Engine 的各种加解密、TRNG 等功能。

1.2.1.3 Safety OS

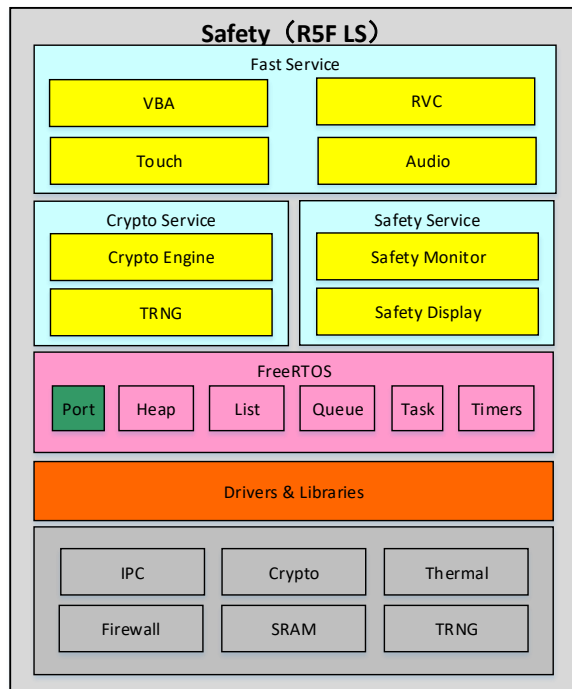


图 1-6 Safety OS 架构图

R5F 核默认以 Lock-Step 形式运行 FreeRTOS 操作系统，在功能层面主要包含三部分功能：

- Fast Service
- Crypto Service
- Safety Service

1.2.1.3.1 Fast Service

该 R5F 运行的 FreeRTOS 在系统启动过程中承接了 bootloader 的一部工作，由于同时满足部分 Fast Feature 所需要的外设资源访问能力，因此部分冷启动过程中的 Fast Feature 软件实现放在了该 FreeRTOS 中，按需在系统启动过程中进行及早启动，从而满足 Fast Feature 的性能需求。

AC8025 Fast RVC、Fast Touch 等均在这里实现。

1.2.1.3.2 Crypto Service

Crypto Service 主要由 Crypto Engine 和 TRNG 两个子功能组成，分别为 AC8025 平台提供基于硬件的加解密、真随机数等功能，该相关硬件只能由 R5F 核访问，所以 AP 端对相关功能访问，需要通过 Trustzone 利用 IPC 模块进行核间通信实现。

1.2.1.3.3 Safety Service

R5F 端的 Safety Service 主要实现了 Safety Monitor、Safety Display 和 Safety Audio 等功能。Safety Monitor 实现对系统关键模块的监控，保证系统关键功能的正常运行。Safety Display 和 Safety Audio 在相关具有功能安全设计的硬件基础上，实现了高安全性的显示、音频播放功能。Safety Service 和 AP 端的交互也是通过 IPC 模块进行核间通信实现。

1.2.1.4 Cluster/ADAS OS

Cockpit 形态默认搭载的 Cluster/ADAS 系统的软件架构如下图所示：

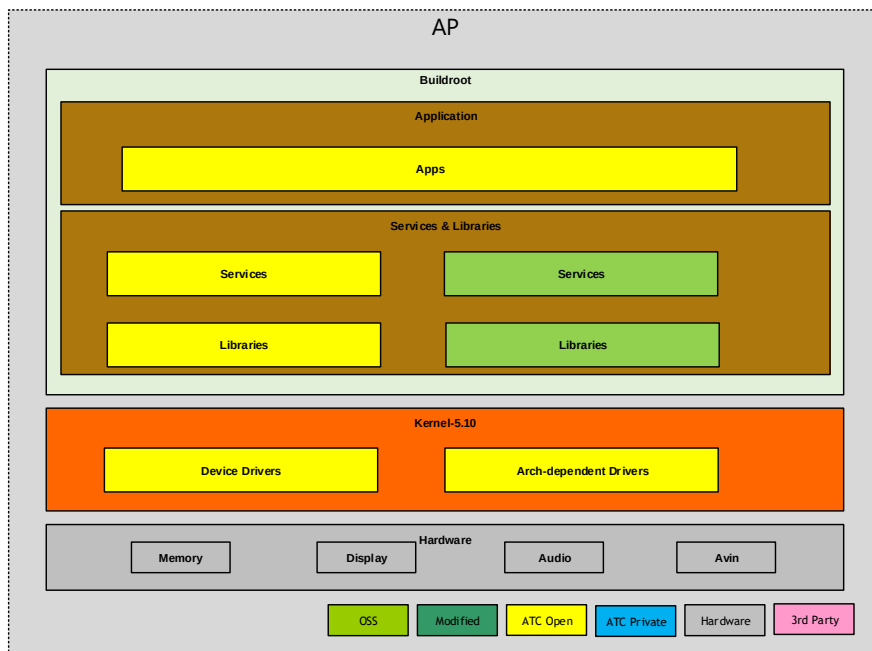


图 1-7 Cluster/ADAS 系统软件架构

在 AC8025 默认的设计中，Cluster 和 ADAS 系统都是基于 Linux，也可以根据需要配置成其他操作系统，如 FreeRTOS、QNX 等（可参考系统编译与配置章节），相应 Linux Kernel 和 IVI 系统所用的 Kernel 代码是公用的，通过配置区分编译出相应的内核。Cluster 系统主要提供了 Cluster 和 HUD 相关应用功能支持，而 ADAS 系统则主要是实现 ADAS 功能的支持。

1.2.2 系统启动流程

1.2.2.1 IVI 形态系统启动流程



图 1-8 系统启动流程

1. 平台上电以后，Boot ROM 开始执行。

Boot ROM 检测 eMMC/UFS 设备并做初始化，从而访问设备并加载 Preloader 镜像到 SRAM 中。Boot ROM 加载并校验 Preloader 镜像后，跳转到 Preloader 继续执行。

2. Preloader 做 DRAM 初始化，并加载 Trustzone、HSM 镜像到 DRAM 中，启动 HSM。
3. HSM 做必要的硬件初始化，读取 datazone 等配置数据、加载 boot 镜像（Android kernel + DTB），然后启动 AP。
4. AP 启动后首先从 Trustzone 开始运行，待 AP 端 Trustzone 环境初始化完成后，跳转到 LK。
5. LK 负责准备 Android Kernel 运行环境，并跳转到 Kernel 继续执行。
6. Kernel 在配置完基本的平台环境后即启动 Android 系统。

针对 Secure Boot，上述流程中会增加 Image 签名校验相关处理，详见 Secure Boot 章节。

1.2.2.2 Cockpit 形态系统启动流程

Cockpit 形态下，Android 系统的启动流程和上述 IVI 形态是一致的，差别在于多出了另外的 ADAS/Cluster OS 的启动。

总体的启动时序参考下图：

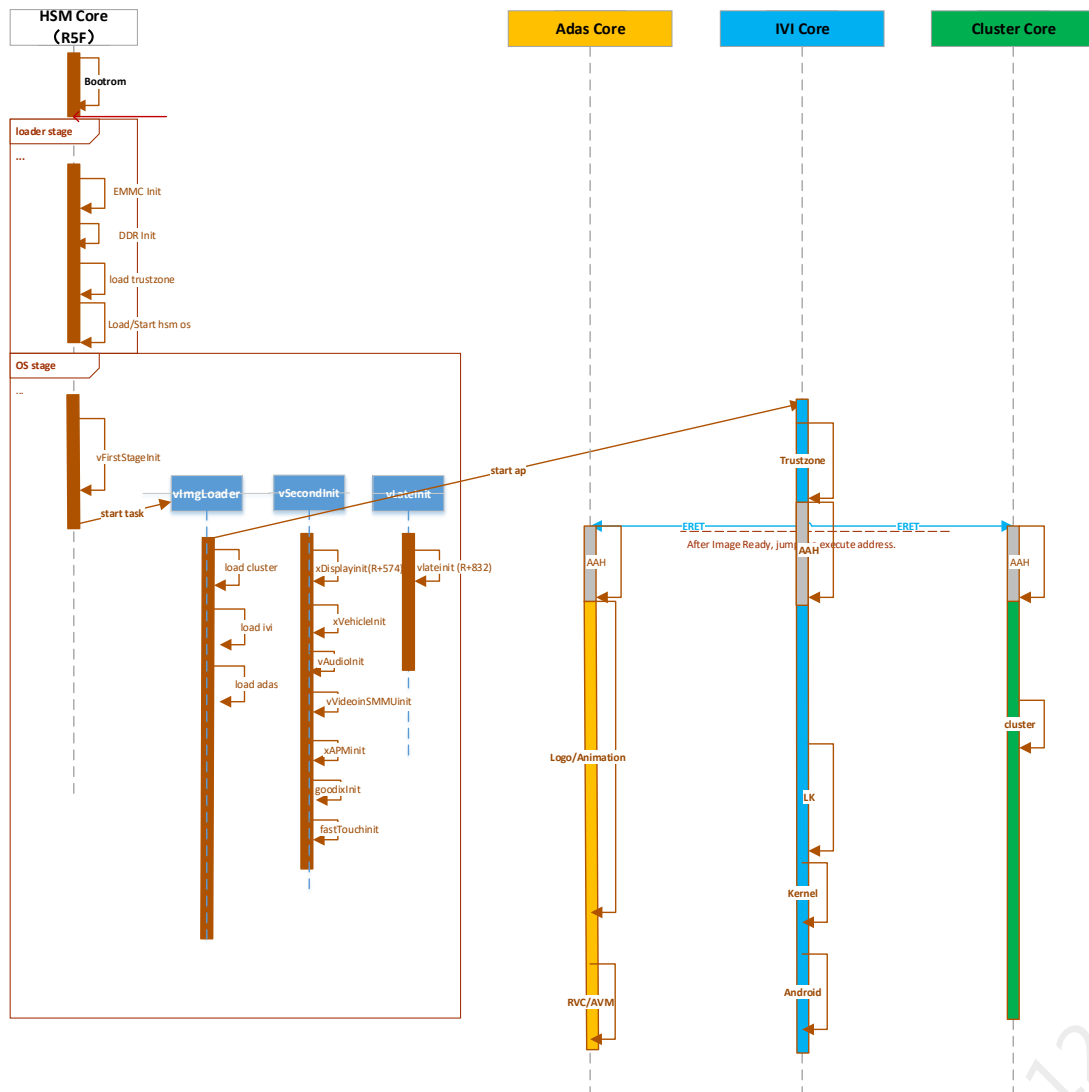


图 1-9 Cockpit 形态系统启动时序示意

HSM（对应上图 OS stage）在开始 image 加载前，会 reset AP 以开始做 trustzone 初始化，AP 初始化完成后由 AAH 模块（负责管理 AP 端的 core 分配）接管，等待各 OS 对应的 image 加载完成信号。HSM 中的对应 OS image 加载完成后，通知到 AP 端的 AAH 模块，由 AAH 将对应 OS 代码地址给到相应 core 进行运行，实现相应的系统启动。

1.3 系统资源管理

本节主要描述 AC8025 平台的 CPU/IRQ/Memory 相关资源管理情况，AC8025 的编译架构支持这些资源的静态配置，可以参考系统编译与配置相关章节了解具体配置方法。

本章节所描述的资源分配状态，均是 AC8025 平台默认配置状态，可以根据需求配置修改。

1.3.1 CPU

IVI 形态的 AC8025 AP 端的 CPU 完全由 Android 系统使用，参考下图所示：

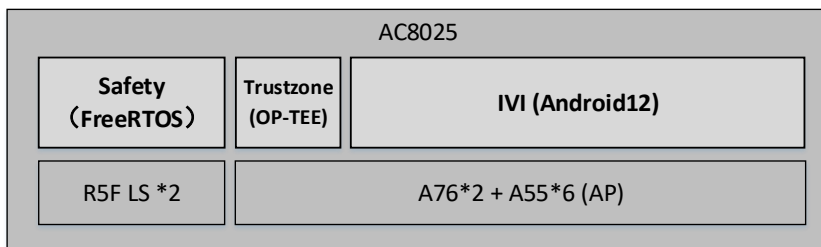


图 1-10 IVI 形态的 CPU 资源分配

Cockpit 形态的 AP 端 CPU，则根据需要通过物理分核的形式，给每个 OS 按需配置了相应的核。AC8025 Cockpit 形态默认的 CPU 分配参考下图：

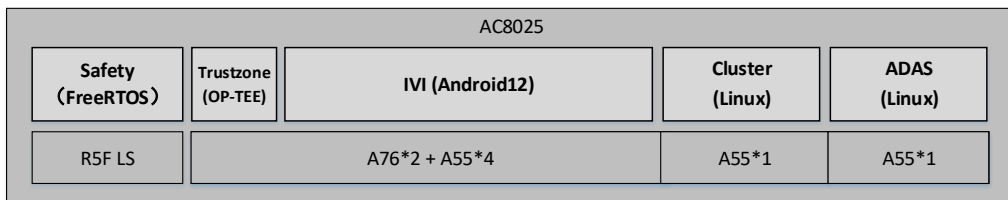


图 1-11 Cockpit 形态的 CPU 资源分配

AC8025 默认 Cockpit 形态中，IVI 占用 6 核（2 个 A76, 4 个 A55），Cluster 占用 1 个 A55，ADAS 也是占用 1 个 A55。

1.3.2 IRQ

AC8025 使用 GIC600 做为中断控制器。在 AP 端，AutoChips 的中断模块负责管理所有中断。

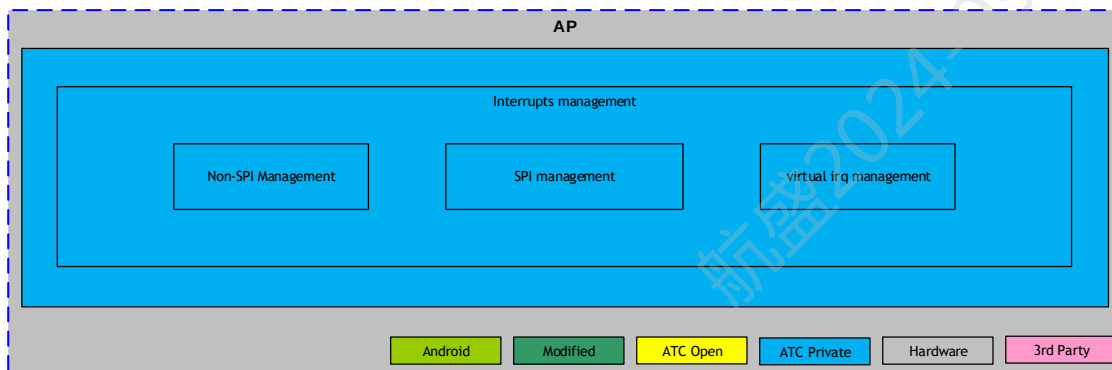


图 1-12 中断管理模块架构图

中断管理模块主要包含对 3 种类型中断的管理，分别是：Non-SPI（PPI or SGI）、SPI、Virtual IRQ。

针对 Cockpit 形态多系统的需求，除 Virtual IRQ 外，其他 IRQ 均是通过配置文件的方式指定所属 OS（具体可参考[编译配置](#)相关章节），用户在工程中通过配置文件添加指定 OS 所需要处理的中断号，中断管理模块在启动中会依据中断号与 OS ID 的连接建立一一映射的关系。默认不允许同一中断号绑定不同的 OS，当用户进行错误的配置后，系统会在启动解析阶段报错。不过对于一些特殊场景下的同一中断号支持多个 OS 接收情况，也可以针对性的实现相关处理服务。

1.3.3 Memory

AC8025 多系统的内存使用采用了静态隔离的方式，默认整体布局参考下图：

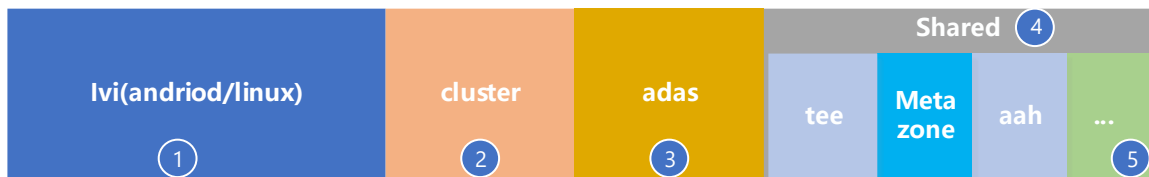


图 1-13 内存布局

①，②，③配置后，剩余统一规划为共享内存 ④，多系统内存的配置可以参考[编译配置](#)相关章节描述。

针对 Cockpit 系统，在上述整体的内存隔离配置基础上，每个域系统还会有内部预留内存的配置，这部分主要是基于 Linux 内核的 Reserved Memory 机制，在 Cockpit 系统顶层配置目录中，有相应的配置项。

进一步的，为了避免 OS 间的内存访问越界，AC8025 通过 MMU 配置保证多个 OS 间的内存隔离，各系统的内存空间视图如下图所示：

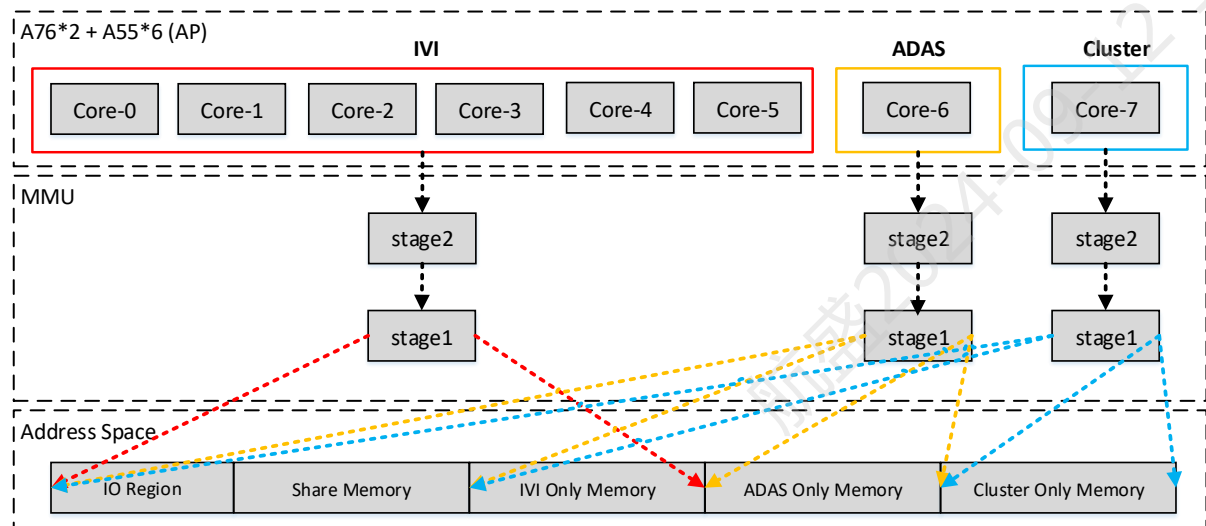


图 1-14 OS 间内存隔离示意

各子系统间对于 share memory 以及 IO 区域的映射是相同的，因此能够保证各个系统间正常通信，并且访问外设寄存器无影响。同时，各个子系统在 DRAM 中还会有其独占的内存空间，该段空间对于子系统的 stage1 映射而言是透明的，通过 stage2 映射实现内存隔离。

航盛2024-09-12 16:50:26

2 系统软件目录

2.1 简介

2.1.1 软硬件环境

2.1.1.1 软件环境

- OS: Ubuntu18.04.1 or above
- repo 版本: v1.13.11 or above
- git 版本: 2.28.0 or above

2.1.1.2 硬件环境

2.1.1.2.1 磁盘空间需求

- Cockpit 代码 Size: ~122GB
- IVI 代码 Size: ~120GB
- 编译之后, Cockpit 整体 Size: ~264GB
- 编译之后, IVI 整体 Size: ~231GB

2.2 代码结构及目录说明

2.2.1 顶层目录结构

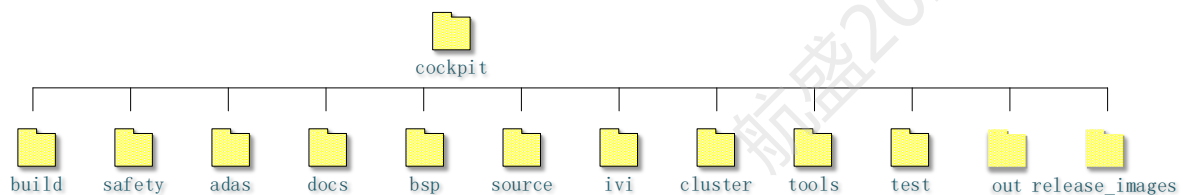


图 2-1 顶层目录

▪ /build

此目录中包括整个项目编译配置脚本，以及不同产品形态配置参数文件。

▪ /safety

此目录下有 safety 子系统的编译入口脚本，以及 safety 子系统的 prebuilt 库。

▪ /docs

此目录中编译 IVI 系统的 SDK 压缩包，以及 IVI 系统的 doxygen 文档。

▪ /bsp

此目录中包括 aah/lk/ preloader 三个子系统的编译脚本，以及这三个子系统的 prebuild 库，编译中间文件以及目标文件等。



说明：

prebuild 库：因为一些模块的源代码为 ATC 私有，不对外开放，所以需要预编译成库文件的形式 release 给客户。

▪ /source

整个 IVI 产品形态所有的源代码存放与此目录，包括 Android 以及其他各个子系统的代码。

▪ /ivi

此目录中包括 Android 编译脚本以及 Android 系统的编译 Log。

▪ /adas

此目录包括 adas 子系统 prebuild 库，adas 编译脚本、编译 Log，编译中间文件和目标文件等。

▪ /cluster

此目录包括 cluster 子系统 prebuild 库，cluster 编译脚本、编译 Log、编译中间文件和目标文件等。

▪ /tools

此目录中包括编译 dtb、reserve memory 以及 Partition Scatter Info。

▪ /test

此目录中包括测试程序编译脚本，动态编译配置，以及测试脚本。分为模块测试和集成测试，对应各个子系统的不同 level 分为 jtest/gtest/ctest。

▪ /out

此目录为编译时动态生成，其中包括各个子系统编译时所需要的整体配置文档，xml/mk/h/dts/dtsi 等。

▪ /release_images

此目录为编译时动态生成，是编译之后平台烧写所需要的所有 Image 文件以及烧写工具本身。

2.2.2 Cockpit 代码整体目录结构

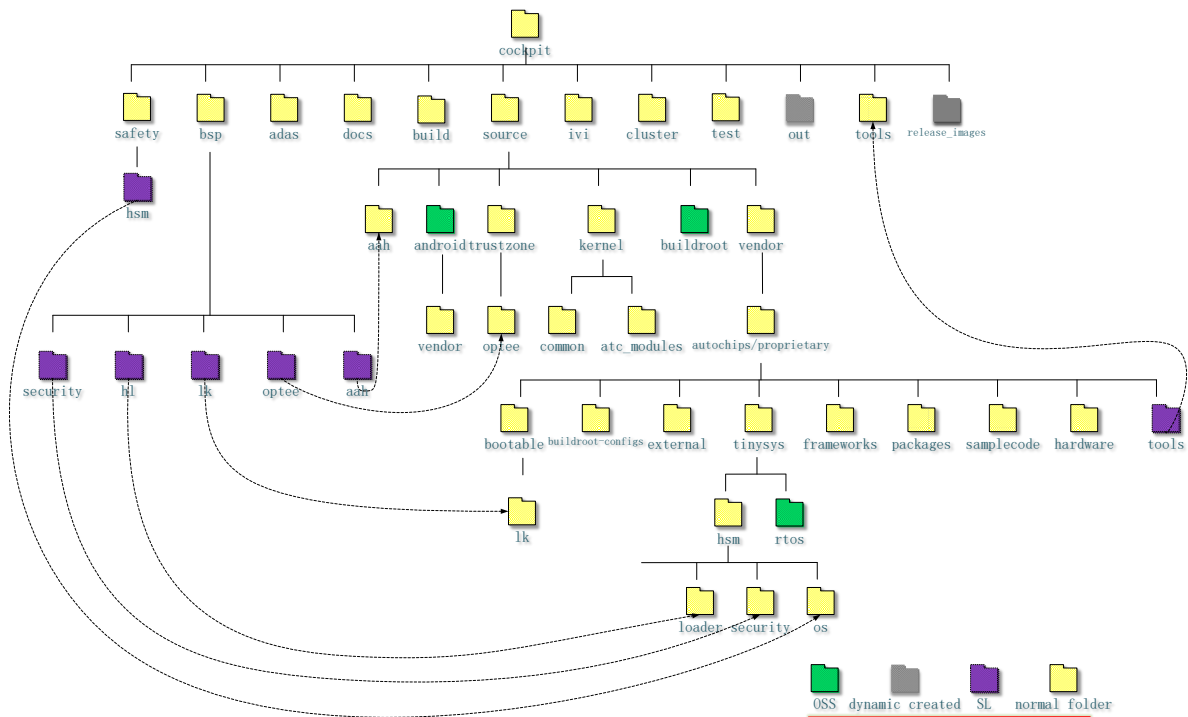


图 2-2 项目整体目录结构

2.2.3 ADAS/Cluster 子系统目录结构

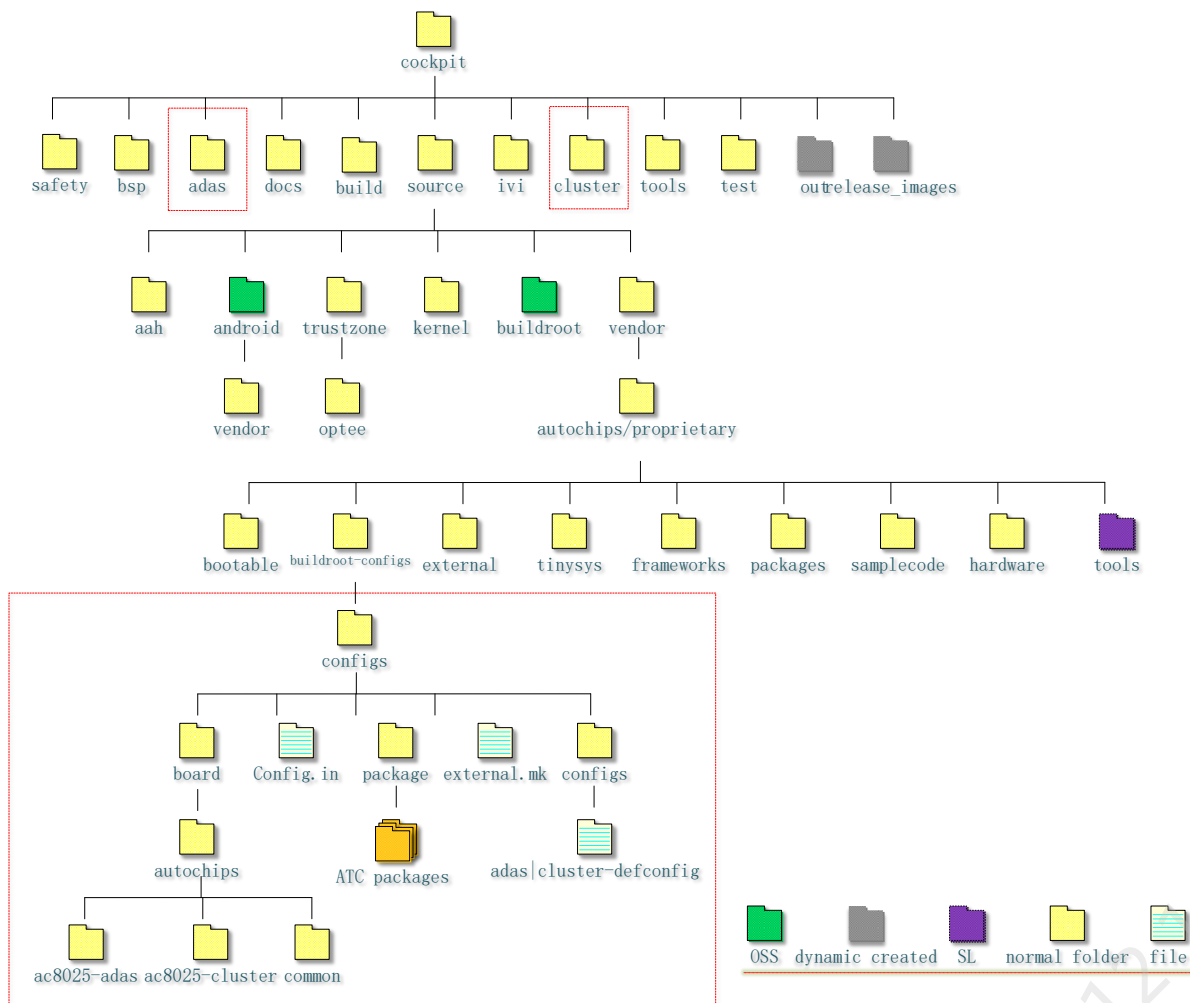


图 2-3 ADAS/Cluster 子系统目录结构

2.2.4 重点目录介绍

2.2.4.1 build/

allmake.sh – 编译命令执行入口，不带参数编译默认配置的 Image，可以加参数编译出不同配置的 Image。

编译出 cockpit 产品形态的命令举例: (以下两种方式都可以)

- ./allmake
- ./build/allmak_cockpit.sh

2.2.4.1.1 builder/

此目录下是各个 builder 编译脚本，创建每一个编译对象，支持各个子系统的整体编译和模块编译，以及 clean, release image 等动作。

2.2.4.1.2 parser/

此目录下是需要支持编译配置各个模块添加的 parser 脚本，负责解析各个模块提供的配置 json 文件，编译时，parser 对象解析 json 文件内容，生成对应的 xml/mk/h/sh 等各个子系统编译时可识别的编译配置。

2.2.4.1.3 utils/

整体编译系统实现的基础功能，比如默认参数、prebuild 文件上传 git 操作等。

2.2.4.1.4 projects/\$(product_name)

此目录中是不同产品形态的编译配置文件，这些文件格式包括 json, xml, csv 等，编译时根据编译参数，到对应不同的产品形态命令中找到这些配置文件，解析编译配置文件生成各个子系统编译时需要的配置。

2.2.4.2 source/

2.2.4.2.1 buildroot/

buildroot OSS 代码环境，用于构建 ADAS 和 Cluster 两个子系统，此目录即 ADAS/Cluster 两个子系统的编译基础环境。

2.2.4.2.2 android/

基于 Android AOSP 版本：12.1.0_r7

表 2-1 Android 版本

Code name	Version	Kernel version	Release date	API level
Android S	12.1	5.10	4-Oct-21	32

- ATC 对于 AOSP 的深度定制即源码修改都在此目录；
- ATC 自研 Android 模块也在此目录下。

2.2.4.2.2.1 vendor/autochips/libs/

此目录是 Android 系统中，存放 ATC 自研模块的 prebuild 库以及相应的模块编译脚本。Android 系统编译时，如果对应的模块 source code 路径不存在，则直接编译 prebuild 库；如果对应的模块 source code 路径存在，则不编译 prebuild 库。

2.2.4.2.2.2 vendor/autochips/proprietary/

此目录下是对于 Android 系统定制扩展以及 ATC 自研的单独属于 Android 系统的模块，此目录下的目录结构，完全 follow Android 根目录的目录结构，比如 frameworks/base，external/等。

2.2.4.2.3 kernel/common

目录中是 kernel 的源码(当前是 5.10 版本)以及 kernel 编译环境。

2.2.4.2.4 kernel/atc_modules

此目录下存放 autochips 自研的部分驱动模块的代码，这些代码都是编译成 ko 形式，运行时加载到 kernel。

2.2.4.2.5 trustzone/

此目录下有 atf 和 optee 系统的源代码。

2.2.4.2.6 vendor/

此目录中全部是 ATC 自研模块以及各个子系统的源代码路径，比如 lk、hsm、preloader 等。

2.2.4.2.6.1 autochips/proprietary/

- bootable/

此目录下存放经过 ATC 定制化之后的 LK 子系统的 source code，包括定制化编译和定制化功能。

- buildroot-configs/

此目录中，是 ATC 添加到 buildroot 系统的模块编译脚本以及系统配置，当前 ADAS/Cluster 两个子系统都是用 buildroot 编译系统，所以相关的模块编译脚本分别加入到 ADAS/Cluster 子系统编译。

- external/、frameworks/、hardware/、packages

这些目录中，全部是 ATC 自研模块的源代码，并且是 IVI 以及其他多系统共用的模块。

- tinysys/

此目录下是 rtos 系统源码，以及 ATC 自研的安全模块的源代码目录，包括 preloader、hsm os、efuse、security 等。

2.3 使用说明和注意事项

2.3.1 模块定制化

2.3.1.1 Android

- Android 自研代码统一放置在 source/android/vendor/autochips/proprietary/ 目录中。
- Android 代码保护的 prebuilt 库统一放置在 source/android/vendor/autochips/libs 目录中。

2.3.1.2 LK 定制化

- LK 编译脚本定制化修改：bsp/lk_main.mk
- LK 源码定制化修改：source/vendor/autochips/proprietary/bootable/lk

2.3.1.3 Kernel 定制化

- Kernel 编译配置脚本：source/kernel/build
- Kernel 编译 toolchain：source/kernel/prebuilts/
- Kernel 源码：source/kernel/common/

2.3.2 模块添加

2.3.2.1 Android

Android 系统添加模块，模块源码建议放置：

source/android/vendor/\$(OEM_NAME)/proprietary/ 目录下对应的子目录中。

3 系统编译与配置

3.1 编译环境设定

编译环境要求：

- OS version: Ubuntu 18.04.1 or above
- JDK Version: 1.8 or above
- python version: 3.8.4 or above
- bash version: 4.4.20 or above

3.2 编译系统配置介绍

3.2.1 编译配置

IVI 编译系统代码实现是在 codebase 的 build 目录中。

build/

builder/ -- 其中实现针对各个子系统的编译脚本

parser/ -- 解析配置文件，生成 mk/h/sh/dts 等各个子系统编译配置

projects/ivi_s_ac8025/ -- 各个模块的编译配置 json 文件

projects/cockpit_s_ac8025/ -- 针对 Cockpit 产品形态各个模块的编译配置 json 文件

utils/ -- 其中实现编译功能基础功能

IVI 编译系统，编译配置有两个来源：

- Json 文件 config
- 命令行参数 config

按照优先级，Json 文件 config < 命令行参数 config。

以上优先级的作用是：在 Json 配置文件中有某项配置，编译命令行参数也加了此项配置，且他们的值又不一致时，那么最终编译时生效的以哪个为准呢？

这时候就根据优先级决定，以优先级高的参数值为准。

3.2.1.1 Json 文件 config

当前系统中支持 json 文件如下：

```
build/projects/ivi_s_ac8025/aah_config.json
build/projects/ivi_s_ac8025/avb.json
build/projects/ivi_s_ac8025/datazone_config.json
build/projects/ivi_s_ac8025/display_config.json
build/projects/ivi_s_ac8025/domain.json
build/projects/ivi_s_ac8025/memory.json
build/projects/ivi_s_ac8025/mix_config.json
build/projects/ivi_s_ac8025/project_config.json
build/projects/ivi_s_ac8025/safety.json
build/projects/ivi_s_ac8025/storage_config.json
build/projects/ivi_s_ac8025/system_config.json
build/projects/ivi_s_ac8025/tee.json
build/projects/ivi_s_ac8025/videoin_config.json
```

图 3-1 Json 配置文件

Json 文件中对于各个模块的具体配置，这些配置被 parser 脚本解析成模块希望的宏定义，导入到各个子系统编译环境中。

3.2.1.2 命令行 config

执行 allmake 命令时，可以用 -a 参数添加配置，命令行参数是配置参数中优先级最高的，如果遇到配置参数值不一致的情况，以命令行参数值为准，如下是一个编译命令行示例：

```
./allmake -a
clean_br=True,ab_update=True,allota=False,with_symbol=False,docs=False,sdk=False,api_check=True,bt_wifi_chip=cypress_pcie,lite_asan=False,kernel_asan=False,fs_config=ext4,dynamic_partition=False,user=False,default_img_avb=False,coverage=True,hires=True,aosp_enhancement=True,totalmemsize=4G
```

特别说明：当前 WiFi 芯片赛普拉斯的两款的 PCIE 和 SDIO 两个版本以及 QCA6595，如果客户使用其他 WiFi 芯片，需要做相应的配置。

3.2.2 完整编译

3.2.2.1 Lunch

lunch 命令用于编译各个子系统之前的平台配置，即执行了 lunch 命令之后，无需做整体编译，就可以编译 android、preloader、lk、safety 等子系统。

```

$ ./lunch
You're building on Linux ahfscf07 5.4.0-84-generic #94~18.04.1-Ubuntu SMP Thu Aug 26 23:17:46 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
Lunch menu... pick one:
1. cockpit_s_linux_ac8025, ac8x_demo
2. cockpit_s_linux_ac8025, ac8025_demo
3. cockpit_s_linux_ac8025, ac8025_fpga
4. ivi_s_ac8025, ac8x_demo
5. ivi_s_ac8025, ac8025_demo
6. ivi_s_ac8025, ac8025_fpga
Which would you like? [atc_product_type]
  
```

然后输入选择配置的 index 即可。



注意：

lunch 之后，选择的配置起效于单独编译各个子系统时。如果 lunch 之后使用 allmake 完整编译整个系统，那么参数配置是依据 allmake 参数，或者系统的默认配置参数。

3.2.2.2 系统的编译框架

编译配置 Tool 配置编译时需要的配置参数。

根据配置参数，生成配置文件，并串行执行各个子系统的编译，每一个子系统均是独立编译。

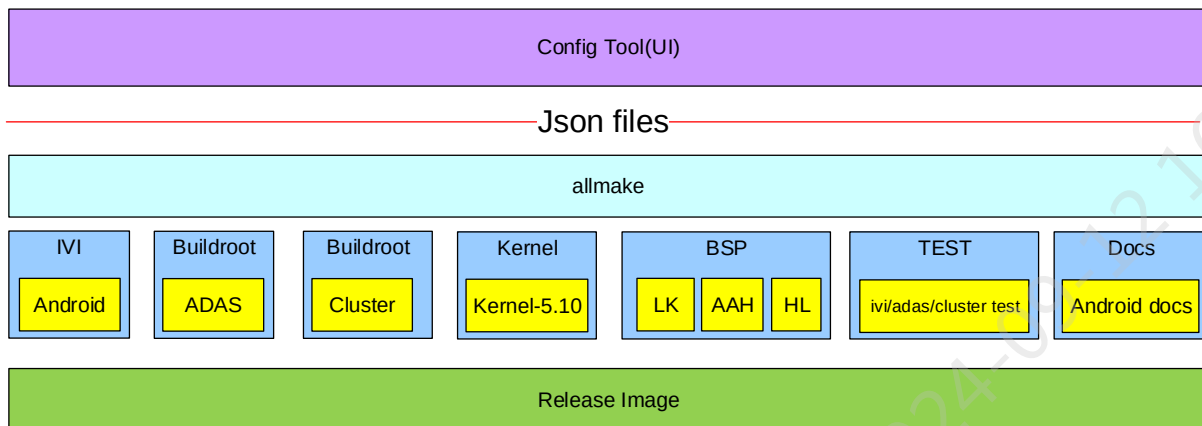


图 3-2 编译整体架构

3.2.2.3 allmake – 编译完整系统

```
$ allmake
```

此命令是软连接指向 build/allmake_cockpit.sh， build/allmake_cockpit.sh 是封装了 build/cockpit_build.py 编译脚本，可以不加任何参数编译。

```
$ python3 ./build/cockpit_build.py build -p $(PRODUCT_NAME) [-d $(DEVICE)] [-b domain|-a $ARG1=True[, $ARG2=False]]
```

直接用 python 编译脚本编译，参数如下：

- [build/clean/version]：build 表示编译；clean 表示删除编译目录包括中间文件；version 表示当前编译系统版本；
- [-p \$(PRODUCT_NAME)]：后接产品形态，目前支持 ivi_s_ac8025, cockpit_s_linux_ac8025；
- [-d \$(DEVICE)]：device 选择有 ac8x_demo、ac8025_demo；
- [-b domain]：后接 domain，可选 domain 有：bsp、android、safety、tools、test、docs；
- [-a args]：后接编译配置参数，具体参见后续章节介绍。
- 举例：

【编译 IVI 产品形态】

IVI 产品形态两个 device 可以选择：

- ./allmake.py build -p ivi_s_ac8025 -d ac8x_demo
- ./allmake.py build -p ivi_s_ac8025 -d ac8025_demo

【编译 cockpit 产品形态】

cockpit 产品形态也两个 device 可以选择：

- ./allmake.py build -p cockpit_s_linux_ac8025 -d ac8x_demo
- ./allmake.py build -p cockpit_s_linux_ac8025 -d ac8025_demo

3.2.3 子系统编译

3.2.3.1 Android

编译 Android 子系统：命令如下：

```
./allmake -b android
```

Android 编译支持 vendor freeze 编译，即 system 和 vendor 分开 lunch 以及分开编译。

如下具体命令供参考：

lunch & build vendor image:

```

[redacted]@ubuntu:~/android$ lunch
You're building on Linux
Lunch menu... pick a combo:
  1. aosp_arm-eng
  2. aosp_arm64-eng
  3. aosp_x86-eng
  4. aosp_x86_64-eng
  5. sys_assi_64-eng
  6. sys_assi_64-user
  7. sys_assi_64-userdebug
  8. vnd_ac8025_demo-eng
  9. vnd_ac8025_demo-user
 10. vnd_ac8025_demo-userdebug
 11. vnd_ac8025_fpga-eng
 12. vnd_ac8025_fpga-user
 13. vnd_ac8025_fpga-userdebug
 14. vnd_ac8x_demo-eng
 15. vnd_ac8x_demo-user
 16. vnd_ac8x_demo-userdebug

which would you like? [aosp_arm-eng] 16

```

make vnd_images

同理，如果 lunch 7，则可以使用 `make sys_images` 命令编译 system image。

另外，编译 system image 是指编译 device/autochips/system/目录下添加到 package 的目标，具体可以参考对饮的 device.mk 文件中的内容。编译 vendor image 是指编译 device/autochips/vendor/目录下对应 device 中添加到 package 的目标，具体可以参考对饮的 device.mk 文件中的内容。

3.2.3.2 Other subsystem

编译 bsp、safety 命令如下：

```
./allmake -b android|adas|cluster|bsp|safety|tools|docs
```

3.2.4 模块编译

模块编译需要在相应的子系统编译环境设置完成之后，即至少完成一次整体编译，才可以进行模块编译。

3.2.4.1 Android 模块编译

Android 模块编译有三种两种方法：

1. allmake 命令

```
allmake -b android -m $MODULE_NAME
```

具体实现是 Andorid 自带的 mmm 或者 ninja 编译，所以需要模块有编译过。

如果实际修改了编译文件 Android.mk/bp 等编译文件，\$MODULE_NAME 应该是 module 所在的 path，编译脚本解析到 Path，调用 Android mmm 命令编译。

如果实际修改了 source code，\$MODULE_NAME 只需要传入模块目标名，实现时调用的是 ninja 命令编译。

2. Android 原生命令：mm/mmm，步骤如下：

```
a) cd source/android
b) lunch vnd_ac8025_demo_userdebug/vnd_ac8025_demo_user
c) source build/envsetup.sh
d) mmm $MODULE_PATH or cd $MODULE_PATH && mm
```

3. Android 原生命令：ninja，步骤如下（vnd）：

```
a) cd source/android
b) ./prebuilts/build-tools/linux-x86/bin/ninja -f out/combined-
vnd_ac8025_demo.ninja $MODULE_NAME
```

4. Android 原生命令：ninja，步骤如下（sys）：

```
a) cd source/android
b) ./prebuilts/build-tools/linux-x86/bin/ninja -f out_sys/combined-
sys_assi_64.ninja $MODULE_NAME
```

3.2.4.2 Build Kernel

当前有三个系统的 kernel：android、adas、cluster。

```
$ ./allmake -b android -m kernel
```

如上步骤生成的 kernel image 位于：

```
source/android/out/target/product/${Device}/obj/KERNEL_OBJ/common/arch/arm64/boot/Image
source/android/out/target/product/${Device}/boot.img
```

```
$ ./allmake -b adas -m kernel
```

如上步骤生成的 kernel image 位于： adas/out/obj/KERNEL_OBJ/common/arch/arm64/boot/Image
adas/out/images/boot.img

```
$ ./allmake -b cluster -m kernel
```

如上步骤生成的 kernel image 位于：

```
cluster/out/obj//KERNEL_OBJ/common/arch/arm64/boot/Image
cluster/out/images/boot.img
```

3.2.4.3 Build LK

```
$ ./allmake -b bsp -m lk
```

如上步骤生成的 lk.img 位于 bsp/release_images/ivi_s_ac8025/。

3.2.4.4 Build preloader

```
$ ./allmake -b bsp -m hl
```

如上步骤生成的 hl.img&hl_verify.img 位于 bsp/release_images/\$(PRODUCT_NAME)/。

3.2.4.5 Build hsm/safety

```
$ ./allmake -b safety
```

如上步骤生成的 hsm.img&hsm_verify.img 位于 safety/release_images/\$(PRODUCT_NAME)/。

3.2.5 Build Log

- Android 编译 Log: ivi/build_\$(DEVICE).log
- Bsp 编译 Log: out/(PRODUCT_NAME)/log/build_bsp.log
- Safety 编译 Log: out/(PRODUCT_NAME)/log/build_safety.log

3.3 使用 docker 打造编译环境

3.3.1 安装 docker

3.3.1.1 Linux

执行如下命令，进行安装：

```
sudo apt-get install docker.io
```

安装完毕之后，可以通过如下指令查看当前安装的版本号：

```
sudo docker version
```

结果如下：

```
autochips@600-G6:~$ sudo docker version
Client:
Version:           20.10.7
API version:       1.41
Go version:        go1.13.8
Git commit:        20.10.7-0ubuntu1~20.04.1
Built:             Wed Aug  4 22:52:25 2021
OS/Arch:           linux/amd64
Context:           default
Experimental:      true

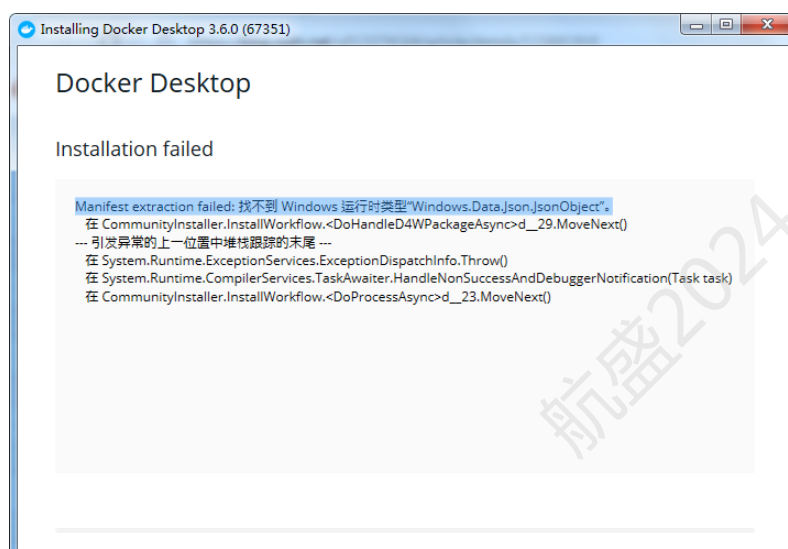
Server:
Engine:
Version:           20.10.7
API version:       1.41 (minimum version 1.12)
Go version:        go1.13.8
Git commit:        20.10.7-0ubuntu1~20.04.1
Built:             Wed Aug  4 19:07:47 2021
OS/Arch:           linux/amd64
Experimental:      false
containerd:
Version:           1.5.2-0ubuntu1~20.04.2
GitCommit:
runc:
Version:           1.0.0~rc95-0ubuntu1~20.04.2
GitCommit:
docker-init:
Version:           0.19.0
GitCommit:
```

图 3-3 docker 安装版本查看

3.3.1.2 Windows

首先，找到 Windows 版本的 docker 下载站点：<https://www.docker.com/get-started>

安装过程中，如果提示如下错误：



原因是最新版本 docker 需要安装在 Win10 系统环境下。如果是在 Win7 系统安装的话，可在如下网址下载旧版本：

<http://mirrors.aliyun.com/docker-toolbox/windows/docker-toolbox/>

安装过程较为直接，选择 full，然后一路 next 即可。

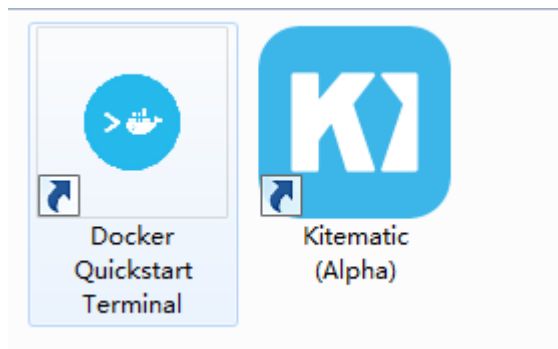
安装完毕之后，为了避免我们的 C 盘太过于臃肿，还必须要做一些事情。

1. 设置一个 MACHINE_STORAGE_PATH 环境变量。

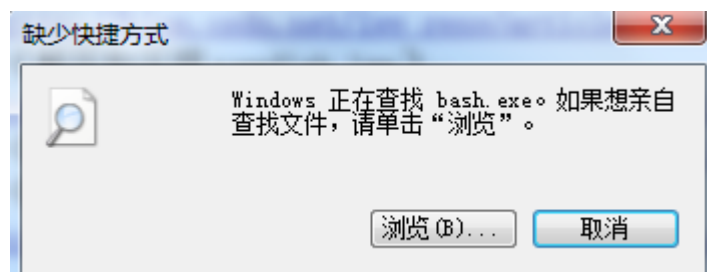
`MACHINE_STORAGE_PATH=D:\VirtualBox Image\docker`

如果不设置该环境变量，就会将虚拟机装在 C:\Users\XXX.docker\machine 目录下。当然咯，如果用户 C 盘容量够大的，完全可以省略这步。

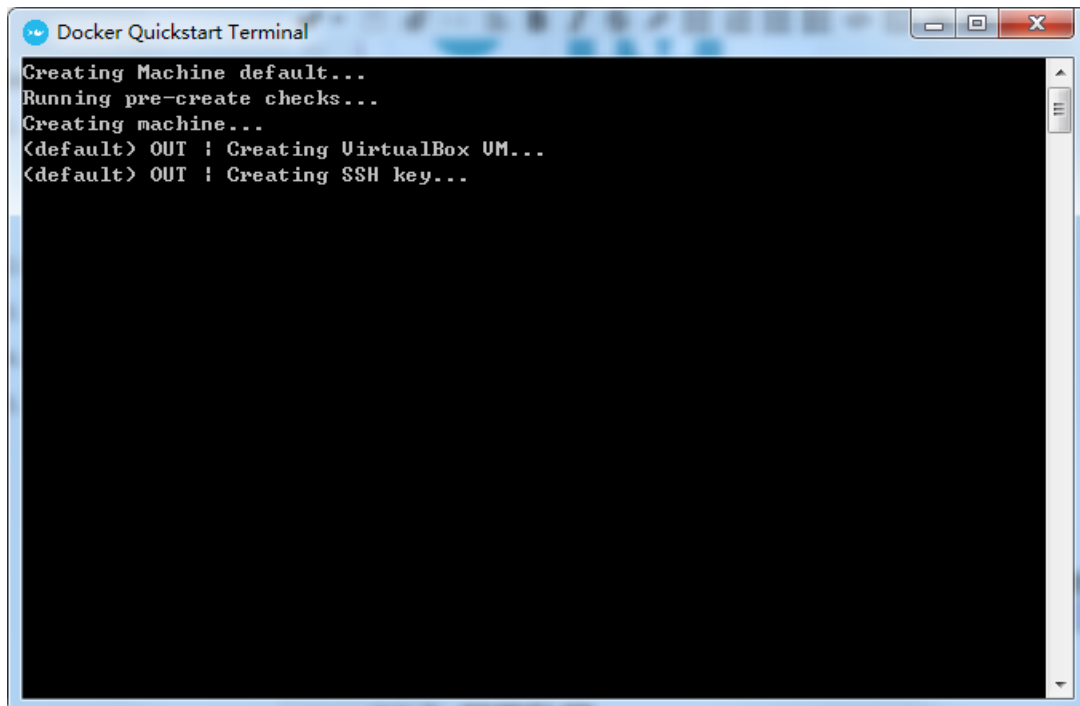
2. 将 docker 安装目录下的 boot2docker.iso 文件拷贝到 D:\VirtualBox Image\docker\cache 目录下，以免还需要从网络下载。
3. 点击 Docker Quickstart Terminal:



这时可能会提示找不到 bash.exe，如：



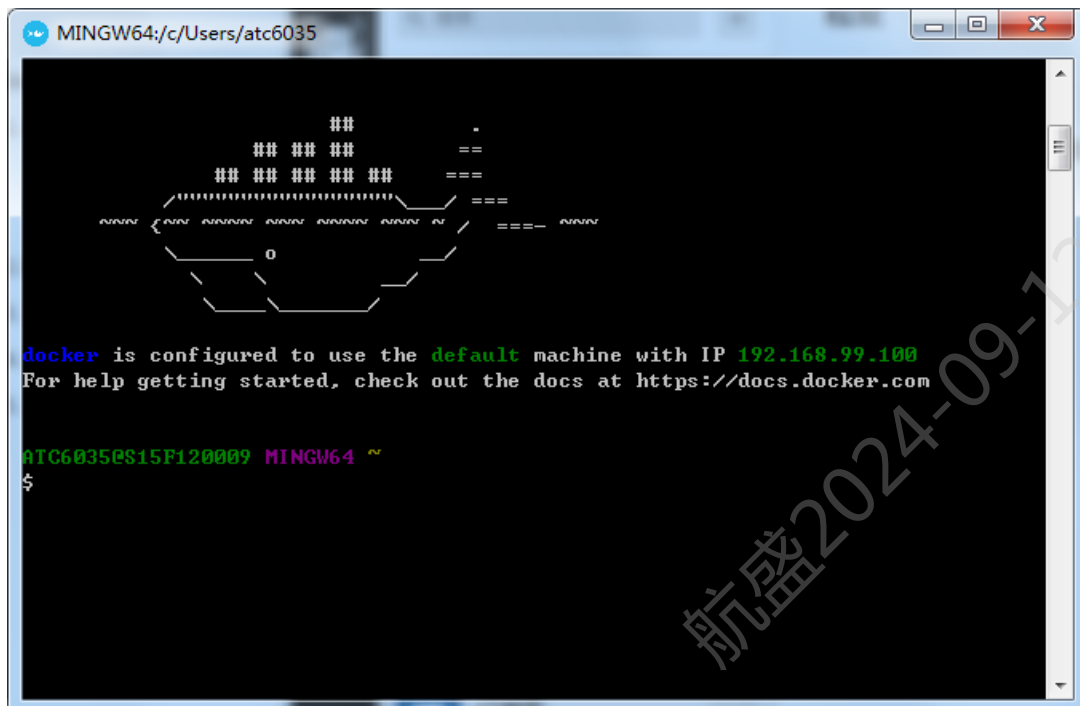
此时，可选择让 Windows 自行查找。找到之后，点击"修复"，即可进到命令行：



```

Docker Quickstart Terminal
Creating Machine default...
Running pre-create checks...
Creating machine...
(default) OUT : Creating VirtualBox VM...
(default) OUT : Creating SSH key...
  
```

初始化完毕，就进入到下图所示的虚拟机的界面：



```

MINGW64:/c/Users/atc6035
      ##
    ##  ##
  ##  ##  ##
##  ##  ##  ##
/#####\
|  o      |
|  / \    |
| /   \   |
|/     \  |
\       /
 \     /
  \   /
   \ /
    o

docker is configured to use the default machine with IP 192.168.99.100
For help getting started, check out the docs at https://docs.docker.com

ATC6035ES15F120009 MINGW64 ~
$
  
```

此时，就可以跟实际的 Linux 系统一下，进行相关的操作了。

3.3.2 配置私服

ATC 专门为编译环境打造了一个 image，位于 60.174.195.116 地址。但此地址并不是在注册的可信任服务列表上的，所以直接 pull image 的时候会提示如下错误：

```
autochips@600-G6:~/yu.mo$ docker pull 60.174.195.116:5000/ubuntu-atc-buildsystem:18.04
Error response from daemon: Get https://60.174.195.116:5000/v2/: http: server gave HTTP response to HTTPS client
```

可以新建或编辑一个 daemon.json 文件，在其中添加我们私服的地址：

```
vim /etc/docker/daemon.json
```

```
{
  "insecure-registries": [ "60.174.195.116:5000" ],
  "max-concurrent-uploads": 1
}
```

上述例子是添加 1 个的，如果是多个的话，用“,”隔开即可，如：

```
vim /etc/docker/daemon.json
```

```
{
  "insecure-registries": [ "60.174.195.116:5000", "172.28.134.8:5000" ],
  "max-concurrent-uploads": 1
}
```

保存之后，如果仍没有生效的，需要重启一下 docker 服务，依次执行如下命令：

```
sudo systemctl daemon-reload
sudo systemctl restart docker.service
sudo systemctl enable docker.service
```

3.3.3 下载 & 安装镜像

配置私服完毕之后，可以在私服上执行如下命令下载 ATC 定制的编译 image：

```
docker pull 60.174.195.116:5000/ubuntu-atc-buildsystem:18.04
```

执行结果如下图所示：

```
autochips@600-G6:~/yu.mo/docker$ docker pull 60.174.195.116:5000/ubuntu-atc-buildsystem:18.04
18.04: Pulling from ubuntu-atc-buildsystem
feac53061382: Pull complete
012b6f047ffd: Pull complete
db4378b2c560: Pull complete
Digest: sha256:2f3f02f52099fdc8fb50b5906f6756368e789d13ff21133a0669f4c53276816a
Status: Downloaded newer image for 60.174.195.116:5000/ubuntu-atc-buildsystem:18.04
```

可以执行如下命令查看当前有多少可用的镜像：

```
docker images
```

结果如下：

```
autochips@600-G6:~/yu.mo/docker$ docker images
REPOSITORY          TAG          IMAGE ID       CREATED        SIZE
60.174.195.116:5000/ubuntu-atc-buildsystem  18.04       dae26deec2c1  7 days ago    2.03GB
```

3.3.4 运行镜像

下载完毕之后，可使用如下命令运行镜像：

```
docker run -d -ti 60.174.195.116:5000/ubuntu-atc-buildsystem:18.04
```



注意：

上述启动命令无法让 docker 和 host 有共享目录，如果需要共享目录的话，需要添加-v 参数。

如下为将 host 主机的~/yu.mo/docker/ubuntu-atc-buildsystem 目录挂载为 docker 的/host_share 目录：

```
docker run -d -ti -v ~/yu.mo/docker/ubuntu-atc-buildsystem:/host_share
60.174.195.116:5000/ubuntu-atc-buildsystem:18.04
```

对于上述命令，需要注意的是：

1. 命令中-v 后面跟的 docker 的目录，必须为绝对路径；
2. 不建议再加上--rm 参数，这个参数意味着调用 docker stop 和 docker kill 之后，容器会将运行时的数据给删掉，并且无法通过 docker start 再调用起来。

至于 run 还可以跟多少参数，可以执行如下命令进行查看：

```
docker run --help
```

运行镜像之后，可以通过如下命令查看当前运行容器的 CONTAINER ID：

```
docker ps
```

结果如下所示：

```
autochips@600-G6:~/yu.mo/docker/ubuntu-atc-buildsystem$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
32d3c2991f44   60.174.195.116:5000/ubuntu-atc-buildsystem:18.04  "/bin/bash"            3 minutes ago Up 3 minutes          naughty_yalow
```

如果在 ps 命令后面添加“-a”参数，那么会列出之前已经退出的容器，如：

```
autochips@600-G6:~/yu.mo/docker/ubuntu-atc-buildsystem$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
32d3c2991f44   60.174.195.116:5000/ubuntu-atc-buildsystem:18.04  "/bin/bash"            5 minutes ago Up 5 minutes          naughty_yalow
7d0107b5f77e   60.174.195.116:5000/ubuntu-atc-buildsystem:18.04  "/bin/bash"            5 minutes ago Exited (0) 5 minutes ago wizardly_carson
b0ca42a3d0ca   60.174.195.116:5000/ubuntu-atc-buildsystem:18.04  "/bin/bash"            15 minutes ago Exited (0) 15 minutes ago competent_goldwasser
```

获取正在运行的容器的 ID 后，可以使用此 ID 进到容器：

```
docker exec -it 32d3c2991f44 /bin/bash
```

执行之后，可以看到提示已经变了：

```
autochips@600-G6:~/yu.mo/docker/ubuntu-atc-buildsystem$ docker exec -it 32d3c2991f44 /bin/bash
root@32d3c2991f44:/home/autochips/yu.mo/docker/ubuntu-atc-buildsystem#
```

乍一看似乎没有多大区别，但如果我们分别在容器的命令行以及本机的命令行执行“ls / -all”命令，就可以从此区别中看到我们已经运行于 docker 容器里了。

DOCKER

```
root@32d3c2991f44:/home/autochips/yu.mo/docker/ubuntu-atc-buildsystem# ls / -all
total 84
drwxr-xr-x 1 root root 4096 Aug 19 03:40 .
drwxr-xr-x 1 root root 4096 Aug 19 03:40 ..
drwxr-xr-x 1 root root 0 Aug 19 03:40 .dockerenv
drwxr-xr-x 1 root root 4096 Aug 11 07:41 bin
drwxr-xr-x 2 root root 4096 Apr 24 20:18 boot
drwxr-xr-x 5 root root 360 Aug 19 03:40 dev
drwxr-xr-x 1 root root 4096 Aug 19 03:40 etc
drwxr-xr-x 1 root root 4096 Aug 19 03:40 home
drwxr-xr-x 1 root root 4096 Aug 11 07:41 lib
drwxr-xr-x 2 root root 4096 Aug 11 03:39 lib32
drwxr-xr-x 2 root root 4096 Jul 23 13:59 lib64
drwxr-xr-x 2 root root 4096 Aug 11 03:35 libx32
drwxr-xr-x 2 root root 4096 Jul 23 13:49 media
drwxr-xr-x 2 root root 4096 Jul 23 13:49 mnt
drwxr-xr-x 2 root root 4096 Jul 23 13:49 opt
dr-xr-xr-x 352 root root 0 Aug 19 03:40 proc
drwx----- 1 root root 4096 Aug 11 08:01 root
drwxr-xr-x 1 root root 4096 Aug 11 03:36 run
drwxr-xr-x 1 root root 4096 Aug 11 07:41/sbin
drwxr-xr-x 2 root root 4096 Jul 23 13:49 srv
dr-xr-xr-x 13 root root 0 Aug 19 03:40 sys
drwxrwxrwt 1 root root 4096 Aug 11 07:44 tmp
drwxr-xr-x 1 root root 4096 Aug 11 03:35 usr
drwxr-xr-x 1 root root 4096 Jul 23 13:59 var
```

LOCAL SSH

```
autochips@600-G6:~/yu.mo/docker/ubuntu-atc-buildsystem$ ls / -all
total 2097240
drwxr-xr-x 20 root root 4096 8月 14 06:40 .
drwxr-xr-x 20 root root 4096 8月 14 06:40 ..
lrwxrwxrwx 1 root root 7 8月 14 06:39 bin -> usr/bin
drwxr-xr-x 4 root root 4096 8月 14 06:45 boot
drwxrwxr-x 2 root root 4096 8月 14 06:40 cdrom
drwxr-xr-x 20 root root 4260 8月 13 15:39 dev
drwxr-xr-x 132 root root 12288 8月 19 10:14 etc
drwxr-xr-x 3 root root 4096 8月 14 06:40 home
lrwxrwxrwx 1 root root 7 8月 14 06:39 lib -> usr/lib
lrwxrwxrwx 1 root root 9 8月 14 06:39 lib32 -> usr/lib32
lrwxrwxrwx 1 root root 9 8月 14 06:39 lib64 -> usr/lib64
lrwxrwxrwx 1 root root 10 8月 14 06:39 libx32 -> usr/libx32
drwx----- 2 root root 16384 8月 14 06:39 lost+found
drwxr-xr-x 2 root root 4096 2月 10 2021 media
drwxr-xr-x 2 root root 4096 2月 10 2021 mnt
drwxr-xr-x 3 root root 4096 8月 19 10:13 opt
dr-xr-xr-x 348 root root 0 8月 13 15:39 proc
drwx----- 4 root root 4096 8月 19 10:57 root
drwxr-xr-x 36 root root 1060 8月 19 11:08 run
lrwxrwxrwx 1 root root 8 8月 14 06:39/sbin -> usr/sbin
drwxr-xr-x 8 root root 4096 2月 10 2021 snap
drwxr-xr-x 2 root root 4096 2月 10 2021 srv
-rw----- 1 root root 2147483648 8月 14 06:39 swapfile
dr-xr-xr-x 13 root root 0 8月 13 15:39 sys
drwxrwxrwt 18 root root 4096 8月 19 11:50 tmp
drwxr-xr-x 14 root root 4096 2月 10 2021 usr
drwxr-xr-x 14 root root 4096 2月 10 2021 var
```

3.3.5 下载 git 代码

下载回来的 image，可能是没有 repo 的，docker 通过如下命令进行下载：

```
curl https://mirrors.tuna.tsinghua.edu.cn/git/git-repo -o /bin/repo
chmod a+x /bin/repo
```

因为 Google 网站在国内是很难访问到的，所以我们必须在.bashrc 里面添加国内的镜像地址。简单点来说，就是在~/.bashrc 里面添加一行代码：

```
vim ~/.bashrc
```

```
...
export REPO_URL='https://mirrors.tuna.tsinghua.edu.cn/git/git-repo'
```

然后执行如下命令将 repo 给安装起来：

```
source ~/.bashrc
repo init
```

接下来，我们需要生成 ssh-key。直接在命令行上执行如下命令：

```
ssh-keygen
```

然后一路回车，便可生成一个密钥，如图：


```
root@99e3c4c2334e:~# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Created directory '/root/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa.
Your public key has been saved in /root/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:IET4WxRvw4Dcs4FH7VUpzUfcFByRiygU34tN3986Cig root@99e3c4c2334e
The key's randomart image is:
+---[RSA 2048]-----+
|
|  +o=+o ..+.+o**|
| ..+ *+..+ = oo.|
| ..o.=*. o.+..|
| ..oo.o .+.o..|
| o S .. o ..|
| . . o|
| E . . o|
| . . ..|
| . . ....|
+---[SHA256]-----+
```

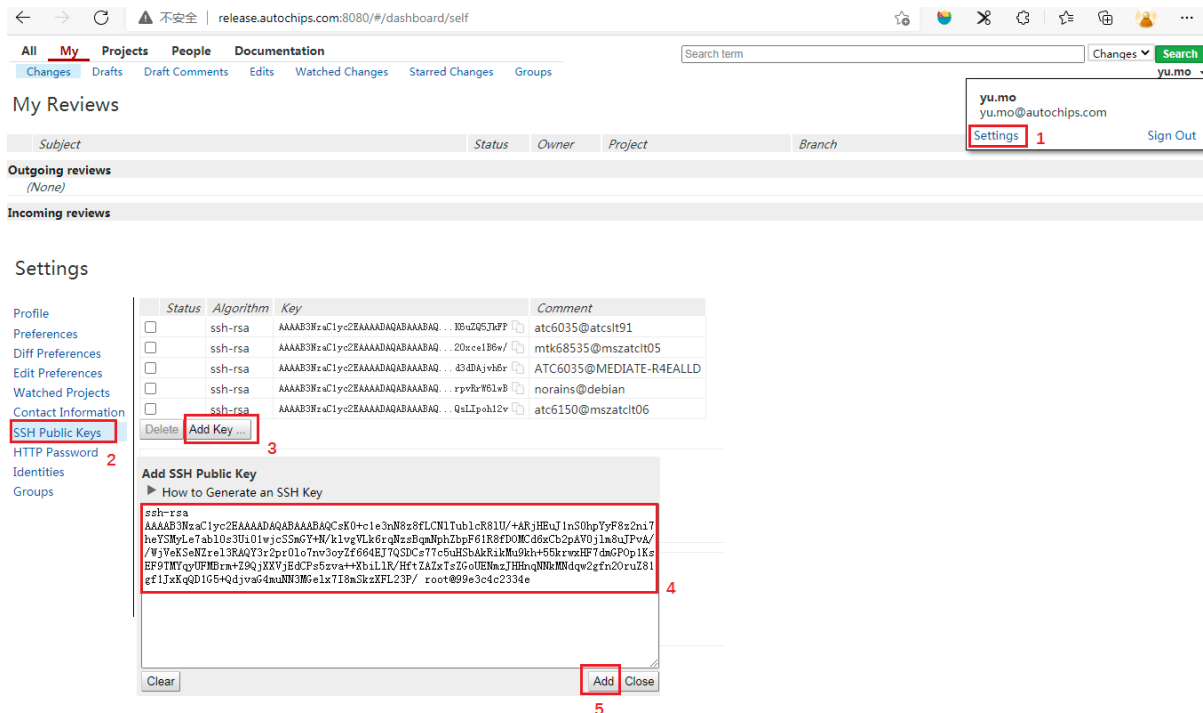
接着执行如下命令：

```
cat ~/.ssh/id_rsa.pub
```

然后将红框里面的字符复制出来：

```
t@99e3c4c2334e:~# cat ~/.ssh/id_rsa.pub
-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCSK0+c1e3nN8z8fLCNlTublcR8lU/+ARjHEuJ1nS0hpYyF8z2ni7heYSMyLe7abl0s3Ui01wjcSSmGY+N/klvgVLk6rqNzsBqmN
bpF61R8fD0MCd6xCb2pAV0jlm8uJPvA//WjVeKSeNZrel3RAQY3r2pr0lo7nv3oyZf664EJ7QSDCs77c5uHSbAkRikMu9kh+55krwxHF7dmGP0p1KsEF9TMYqyUFMBrm+Z9QjX
EdCPs5zva++XbiLLR/HftZAZxTsZGoUENmzJHHnqNNkMNdqW2qfn20ruZ81qf1JxKqQD1G5+Qd1vaG4muNN3MGeLx7I8mSkzXFL23P/ root@99e3c4c2334e
t@99e3c4c2334e:~#
```

打开 <http://release.autochips.com:8080> 网页，使用账户登录，按如下步骤添加 ssh-key：



配置完毕，使用添加了 ssh-key 的账号就可以直接下载代码了。比如，下载 AC8025 IVI 的代码，可执行如下命令：

```
repo init -u ssh://atc6035@release.autochips.com:29418/atc/software/manifest -b
ivi-release-s.ac8025 -g all,-notdefault
repo sync -c
```

如果是第一次执行的话，还需要设置邮件和用户名，直接执行类似下列指令即可：

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

配置完毕之后，执行下载命令，代码就可以正常下载了。下载完毕之后，可以按照通用的做法，直接进行编译了。

对于 AC8025 来说，由于默认不带 iso 生成器，还需要执行一下如下命令进行安装之后才进行编译：

```
apt-get -y install mkisofs
```

3.3.6 关闭 & 恢复容器

如果需要关闭容器的话，只需要调用 stop 或 kill 命令即可，如：

```
# 使用 stop 命令
docker stop 32d3c2991f44
```

```
# 使用 kill 命令
```

```
docker kill 32d3c2991f44
```

恢复容器，不能再使用 run，因为调用 run 意味着会新建一个容器，直接使用 start 命令即可：

```
docker start 32d3c2991f44
```

如果使用 start 命令失败，很可能是因为调用 run 的时候添加了 --rm 参数，导致退出的时候会将所有的资源给清掉，从而导致无法再恢复。

3.4 Android 编译附加说明

3.4.1 概述

从 Android 8.0 开始，引入了 Android.bp (bp 是 blueprint 的缩写) 文件来替代之前的 Android.mk 文件。Android.bp 只是纯粹的配置文件，不包括分支、循环等流程控制。对于逻辑控制的部分（比如当 ProjectConfig.mk 中的变量为 true 时就在代码中引入相应的宏），需要增加 go 文件来实现。推荐新增模块使用 Android.bp 来编译。

本章介绍 Android.bp 的使用方法，Android.mk 在现版本还依然支持。

3.4.2 SELinux 配置

SELinux 工作模式分为 3 种：Disabled、Permissive 和 Enforcing。

3.4.2.1 SELinux 工作模式静态配置

可以通过修改 Safety OS 中 cmdline 参数修改系统 SELinux 工作模式

修改 SELINUX_STATUS 值，default 值为 3。

- Disabled 模式
- Permissive 模式
- Enforcing 模式

source/vendor/autochips/proprietary/tinsys/hsm/os/build/config.mk

```
GLOBAL_DEFINES += SELINUX_STATUS=2
```

3.4.2.2 SELinux 工作模式动态配置

```
setenforce 0 //关闭 SELinux，设置为 Permissive 模式
```

```
setenforce 1 //打开 SELinux，设置为 Enforcing 模式
```

```
getenforce //获取当前 SELinux 状态
```

3.4.3 Android.bp 语法

1. Android.bp 中定义的模块，要以模块类型为开头，然后跟着 “key: value”样式的属性集合。

```
cc_binary {
    name: "gzip",
    srcs: ["src/test/minigzip.c"],
    shared_libs: ["libz"],
    stl: "none",
}
```

2. 每个模块必须要有一个 “name” 属性，并且是全局唯一的。
3. 可使用 C/C++ style 注释。
4. Android.bp 中可使用变量，作用域是声明之后，以及子 bp 文件。

```
gzip_srcs = ["src/test/minigzip.c"],
cc_binary {
    name: "gzip",
    srcs: gzip_srcs,
    shared_libs: ["libz"],
    stl: "none",
}
```

5. 变量可使用 “+=”追加。
6. Android.bp 中变量和属性都是强类型，变量是根据第一次赋值来确定类型，属性类型在模块类型中是固定的。
7. 可使用的类型：

```
* Bool (`true` or `false`)
* Integers (`int`)
* Strings (`"string"`)
* Lists of strings (`["string1", "string2"]`)
* Maps (`{key1: "value1", key2: ["value2"]}`)
```

8. Android.bp 中可以使用 defaults 类型的模块，达到属性复用的目的。

```
cc_defaults {
    name: "gzip_defaults",
    shared_libs: ["libz"],
    stl: "none",
}
cc_binary {
    name: "gzip",
```

```
defaults: ["gzip_defaults"],
srcs: ["src/test/minigzip.c"],
}
```

3.4.4 增加动态 jar 模块

使用 `java_library` 属性类型定义。

```
java_library {
    name: "services", // 模块名称

    dex_preopt: {
        app_image: true,
        profile: "art-profile",
    },

    srcs: [ // 包含的源文件，可使用通配符，也可指定特定的文件
        "java/**/*.java",
    ],

    static_libs: [ // 依赖的静态 jar
        "services.core",
        "services.accessibility",
        "services.appwidget",
        "services.autofill",
        "services.backup",
        "services.companion",
        "services.coverage",
        "services.devicepolicy",
        "services.midi",
        "services.net",
        "services.print",
        "services.restrictions",
        "services.usage",
        "services.usb",
        "services.voiceinteraction",
        "android.hidl.base-V1.0-java",
    ],

    libs: [ // 依赖的动态 jar
        "android.hidl.manager-V1.0-java",
    ],
}
```

```
}
```

3.4.5 增加静态 jar 模块

使用 `java_library_static` 属性类型定义。

```
java_library_static {  
    name: "services.midi", // 模块名称  
    srcs: ["java/**/*.java"], // 包含的源文件  
    libs: ["services.core"], // 依赖的动态 jar  
}
```

3.4.6 增加 apk 模块

目前使用 `Android.bp` 编译 apk 的例子还比较少，推荐仍然使用 `Android.mk`。

3.4.7 增加动态库模块

使用 `cc_library_shared` 属性类型定义。

```
cc_library_shared {  
    name: "libdrmframework", // 模块名称  
  
    srcs: [ // 包含的源文件  
        "DrmManagerClientImpl.cpp",  
        "DrmManagerClient.cpp",  
        "NoOpDrmManagerClientImpl.cpp",  
    ],  
  
    shared_libs: [ // 依赖的动态库  
        "libutils",  
        "libcutils",  
        "liblog",  
        "libbinder",  
        "libdl",  
    ],  
  
    static_libs: ["libdrmframeworkcommon"], // 依赖的静态库  
  
    export_include_dirs: ["include"], // 导出给其他模块的头文件目录  
    export_static_lib_headers: ["libdrmframeworkcommon"],  
    // 导出其他静态模块导出的头文件目录，如果是动态模块的话使用
```

```
export_shared_lib_headers

cflags: ["-Werror"], // 编译使用的 cflags
}
```

对于“导出给其他模块的头文件目录”，表示其他依赖此模块的代码，可以直接使用这里导出的头文件目录中的头文件。

对于“导出其他静态模块导出的头文件目录”，此例中定义如下：

```
cc_library_static {
    name: "libdrmframeworkcommon", // 模块名称

    srcs: [ // 包含的源文件
        "DrmConstraints.cpp",
        "DrmMetadata.cpp",
        "DrmConvertedStatus.cpp",
        "DrmEngineBase.cpp",
        "DrmInfo.cpp",
        "DrmInfoRequest.cpp",
        "DrmInfoStatus.cpp",
        "DrmRights.cpp",
        "DrmSupportInfo.cpp",
        "IDrmManagerService.cpp",
        "IDrmServiceListener.cpp",
        "DrmInfoEvent.cpp",
        "ReadWriteUtils.cpp",
    ],

    cflags: ["-Wall", "-Werror"], // 编译使用的 cflags

    static_libs: ["libbinder"], // 依赖的静态库

    export_include_dirs: ["include"],
    // 对于依赖 libdrmframework 的模块，libdrmframework 会同时导出这个目录
}
```

3.4.8 增加静态库模块

使用 `cc_library_static` 属性类型定义，如上例中的 `libdrmframeworkcommon` 模块。

3.4.9 同时增加动态和静态库模块

使用 `cc_library` 属性类型定义，会同时编译出动/静态库。

```
cc_library {
    name: "libmedia_helper", // 模块名称
    vendor_available: true, // 对 vendor 模块可用
    vndk: { // enable vndk
        enabled: true,
    },
    // 使用如上 2 个属性后，最终编译可能会生成 2 份 so，1 份在/system/lib[64]目录，
    // 1 份在/system/lib[64]/vndk-*目录
    srcs: ["AudioParameter.cpp", "TypeConverter.cpp", "TimeCheck.cpp"],
    // 包含的源文件
    cflags: [ // 编译使用的 cflags
        "-Werror",
        "-Wno-error=deprecated-declarations",
        "-Wall",
        "-DATC_AOSP_ENHANCEMENT",
    ],
    shared_libs: ["libutils", "liblog"], // 依赖的动态库
    header_libs: [ // 依赖的头文件模块
        "libmedia_headers",
        "libaudioclient_headers",
        "libaudio_system_headers",
    ],
    clang: true, // 使用 clang 编译工具
}
```

3.4.10 增加可执行程序模块

使用 `cc_binary` 属性类型定义。

```
cc_binary {
    name: "vendor.autochips.hardware.ipc@1.0-service", // 模块名称
    init_rc: ["vendor.autochips.hardware.ipc@1.0-service.rc"],
    // 对于需要 init 启动的可执行程序，这里可以使用 init_rc 属性来指定独立的 init rc 文件
    // init 启动时会读取此文件
    relative_install_path: "hw", // 生成到相对于可执行程序的 hw 目录
    vendor: true, // 生成到 vendor 分区
    srcs: [ // 包含的源文件
        "Ipc.cpp",
    ],
}
```



```
    "service.cpp",
  ],
  cflags: [ // 编译使用的 cflags
    "-Wall",
    "-Werror",
  ],
  include_dirs: [ //头文件搜索路径
    "vendor/autochips/proprietary/hardware/libipc",
  ],

  shared_libs: [ // 依赖的动态库
    "libbase",
    "libhidlbase",
    "libhidltransport",
    "libutils",
    "liblog",
    "vendor.autochips.hardware.ipc@1.0",
    "libipc",
  ],
}
```

3.4.11 增加导出头文件模块

使用 `cc_library_headers` 属性类型定义。

```
cc_library_headers {
  name: "libmedia_headers", // 模块名称
  vendor_available: true, // 对 vendor 模块可用
  export_include_dirs: ["include"], // 导出给其他模块的头文件目录
  header_libs:[ // 依赖的导出头文件模块
    "libgui_headers",
    "libstagefright_headers",
    "media_plugin_headers",
  ],
  export_header_lib_headers: [ // 导出其他导出头文件模块导出的头文件目录
    "libgui_headers",
    "libstagefright_headers",
    "media_plugin_headers",
  ],
}
```

导出头文件模块只是为了声明导出头文件路径给其他模块使用，推荐使用这种方式来导出头文件路径。

可以这样使用：

```
cc_library {
    name: "libmedia_helper",
    vendor_available: true,
    vndk: {
        enabled: true,
    },
    srcs: ["AudioParameter.cpp", "TypeConverter.cpp", "TimeCheck.cpp"],
    cflags: [
        "-Werror",
        "-Wno-error=deprecated-declarations",
        "-Wall",
        "-DATC_AOSP_ENHANCEMENT",
    ],
    shared_libs: ["libutils", "liblog"],
    header_libs: [ // 依赖的导出头文件模块，代码中可直接使用导出路径下的头文件
        "libmedia_headers",
        "libaudioclient_headers",
        "libaudio_system_headers",
    ],
    clang: true,
}
```

3.4.12 增加 HIDL interface 模块

使用 hidl_interface 类型属性定义，此文件不需要手写，由 hidl-gen 工具自动生成。

```
hidl_interface {
    name: "vendor.autochips.hardware.ipc@1.0",
    root: "vendor.autochips.hardware",
    srcs: [
        "types.hal",
        "IIPC.hal",
    ],
    interfaces: [
        "android.hidl.base@1.0",
    ],
    types: [
        "IpcCoreId",
        "IpcDirectionId",
        "IpcStatus",
    ],
    gen_java: true,
```

```
}
```

3.4.13 条件编译

前文说过 Android.bp 本身不包含逻辑判断，而需要逻辑判断的地方就需要 go 文件辅助或者使用扩展的方式。

3.4.13.1 Android 源生的支持

在 Android.bp 中使用 product_variables 属性，可使用的值和类型参考 source/android/build/core/soong_config.mk 文件。

```
cc_defaults {
    name: "neuralnetworks_defaults",
    cflags: [
        "-Wall",
        "-Wextra",
        "-Werror",
        "-O3",
    ],
    product_variables: {
        debuggable: { // eng and userdebug builds
            cflags: ["-DNN_DEBUGGABLE"],
        },
    },
    // 当 debuggable 属性为 true 时，则编译时会使用此处的 cflags
}
```

还可使用整形和字符串类型的 product_variables。

```
cc_object {
    name: "crtbrand",
    // crtbrand.c needs <stdint.h> and a #define for the platform SDK version.
    local_include_dirs: ["include"],
    product_variables: {
        platform_sdk_version: {
            asflags: ["-DPLATFORM_SDK_VERSION=%d"],
        },
    },
    // 编译时会把 PLATFORM_SDK_VERSION 对应的整型值引入到代码的 PLATFORM_SDK_VERSION 宏
    srcs: ["arch-common/bionic/crtbrand.S"],

    defaults: ["crt_so_defaults"],
}
```

```
cc_library_shared {
    name: "libRS_internal",

    product_variables: {
        override_rs_driver: {
            cflags: ["-DOVERRIDE_RS_DRIVER=%s"],
        },
        pdk: {
            enabled: false,
        },
    },
    // 编译时会把 OVERRIDE_RS_DRIVER 对应的字符串值引入到代码的 OVERRIDE_RS_DRIVER 宏
}
```

3.4.13.2 扩展的支持

类似 `product_variables` 的用法，使用 `autochips_variables` 属性。此处 `autochips_variables` 属性里面可使用 `ProjectConfig.mk` 中定义的宏。

```
atc_cc_library_static {
    name: "libfstab",
    vendor_available: true,
    defaults: ["fs_mgr_defaults"],
    autochips_variables:{
        atc_fs_auto_recovery_support:{
            cflags:[
                "-DATC_FS_AUTO_RECOVERY_SUPPORT"
            ],
        },
    },
    // 当 ATC_FS_AUTO_RECOVERY_SUPPORT 属性为 true 时，则编译时会使用此处的 cflags
    // 编译时会把属性转换成小写来和 Android.bp 中属性做比较，所以，Android.bp 中都使用小写
    srcs: [
        "fs_mgr_fstab.cpp",
        "fs_mgr_boot_config.cpp",
        "fs_mgr_slotselect.cpp",
    ],
    export_include_dirs: ["include_fstab"],
    header_libs: ["libbase_headers"],
}
```

对应 `atc_cc_library_static` 属性的类型，还有 `atc_cc_library/atc_cc_default/atc_cc_library_shared`。

3.4.13.3 使用 go 文件控制逻辑

Android.bp 中定义 bootstrap_go_package 类型的模块，以及自定义名称的模块。

```
bootstrap_go_package {
    name:"soong-services-core-atc", // Package 名称（自定义）
    pkgPath:" android/soong/services/core/atc ", // Package 路径（自定义）
    deps:[
        "soong-android",
        "soong-cc",
    ],
    srcs:[
        "services.core.go", // 使用的 go 文件
    ],
    pluginFor:["soong_build"],
}

services_core_atc_defaults { // 自定义类型模块，需要和 go 文件中定义一致
    name: "services_core_atc_defaults",
}
```

Android.bp 中继承自定义名称的模块：

```
java_library_static {
    name: "services.core.unboosted",
    .....
    defaults: [
        "services_core_atc_defaults",
    ]
}
```

Go 文件实现：

```
package services_core_atc // Package 名称（自定义）

import (
    "android/soong/android"
    "android/soong/cc"
)

func servicesCoreAtcDefaults(ctx android.LoadHookContext) { // Hook 函数名称（自定义）
    type props struct {
        Srcs []string
    }
    p := &props{}
```

```
vars := ctx.Config().VendorConfig("atcPlugin")
if vars.Bool("ATC_FS_LOW_STORAGE_PROTECTION_SUPPORT") {
    // 当 ATC_FS_LOW_STORAGE_PROTECTION_SUPPORT 为 yes 或 true 时，添加这些源文件
    p.Srcs = append(p.Srcs,
        "java/com/android/server/atc/StorageManagerService.java",
        "java/com/android/server/atc/storage/DeviceStorageMonitorService.java",
        "java/com/android/server/atc/storage/DeviceStorageMonitorInternal.java",
        "java/com/android/server/atc/am/AppErrorDialog.java")
} else {
    // 当 ATC_FS_LOW_STORAGE_PROTECTION_SUPPORT 不为 yes 或 true 时，添加这些源文件
    p.Srcs = append(p.Srcs,
        "java/com/android/server/StorageManagerService.java",
        "java/com/android/server/storage/DeviceStorageMonitorService.java",
        "java/com/android/server/storage/DeviceStorageMonitorInternal.java",
        "java/com/android/server/am/AppErrorDialog.java")
}
ctx.AppendProperties(p)
}

func init() { // 必须要 init 函数
    android.RegisterModuleType("services_core_atc_defaults",
        servicesCoreAtcDefaultsFactory)
}

func servicesCoreAtcDefaultsFactory () android.Module { // 类型解析函数（名称自定义，内容固定）
    module := cc.DefaultsFactory()
    android.AddLoadHook(module, servicesCoreAtcDefaults) // 添加上面的 hook 函数
    return module
}
```

如上 go 文件格式很固定，目前推荐使用这种方式来引入相应的宏。除此之外，还可以处理更加复杂的逻辑判断，具体可搜索“bootstrap_go_package”关键字，查看实现。

3.5 编译工具链(编译器版本)

▪ Android S

参见 source/android/build/soong/cc/config/global.go

```
ClangDefaultVersion = "clang-r416183b"
```

▪ Kernel5.10

参见 source/kernel/common/build.config.common

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

114 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

```
CLANG_PREBUILT_BIN=prebuilts-master/clang/host/linux-x86/clang-r416183b/bin/
```

- Bsp

参见 bsp/lk_makefile

```
LK_CROSS_COMPILE := $(ROOT)/bsp/prebuilts/gcc/linux-x86/aarch64/aarch64-linux-android-4.9/bin/aarch64-linux-android-
```

- Safety

参见 source/vendor/autochips/proprietary/tinysys/hsm/os/Makefile

```
export TINYSYS_CROSS_COMPILE ?= $(HL_TOP_PATH)/../../rtos/toolchain/7.2.1/bin/arm-none-eabi-
```

- Preloader

参见 source/vendor/autochips/proprietary/tinysys/hsm/loader/Makefile

```
export TINYSYS_CROSS_COMPILE ?= $(HL_TOP_PATH)/../../rtos/toolchain/7.2.1/bin/arm-non-eabi-
```

4 Android Security

4.1 简介

4.1.1 软硬件环境

- TEE 版本: OP-TEE-3.12

4.1.2 功能

Android Security 功能旨在保护用户、数据和设备的安全性和隐私。其中 OP-TEE、KeyStore、GateKeeper 是 Android Security 平台对接实现的几个关键功能，这些功能的结合使得 Android 系统能够提供多层次的安全保护。KeyStore 用于安全管理密钥和敏感数据，GateKeeper 用于设备解锁和用户身份验证，而 OP-TEE 提供了一个隔离的执行环境，用于处理安全敏感的任务。这些功能结合起来有助于保护用户数据的机密性、完整性和可用性，并确保设备和应用程序的安全性。

4.1.2.1 OP-TEE

OP-TEE (Open Portable Trusted Execution Environment) 是一个开源的可信执行环境，运行于安全处理器 (如 ARM TrustZone) 中。它提供了一个隔离的执行环境，用于安全敏感的计算和操作。OP-TEE 可以与 Android 系统集成，提供更高级别的安全保护。其主要功能包括：

- 安全应用程序的执行和管理
- 安全存储和管理敏感数据
- 安全通信和加密功能

4.1.2.2 KeyStore

KeyStore 是 Android 系统中的密钥和证书存储解决方案。其主要功能包括：

- 密钥和证书的安全存储和管理
- 支持对称密钥、非对称密钥和证书的生成、导入、导出和删除
- 提供加密和解密操作
- 与硬件安全模块 (如 Trusted Execution Environment) 结合使用，提供更高级别的安全保护

4.1.2.3 GateKeeper

GateKeeper 是 Android 系统中的身份验证框架，用于设备解锁和访问控制。其主要功能包括：

- 提供完善的设备解锁机制，如密码、图案、指纹等
- 安全存储和管理用户的身份验证凭据
- 防止未经授权的用户访问设备和敏感数据
- 与硬件安全模块（如 Trusted Execution Environment）结合使用，提供更高级别的安全保护

4.1.3 代码结构及文件描述

表 4-1 Android Security 代码结构及描述

文件名	文件路径	文件描述
optee_os	source/trustzone/optee/optee_os	Optee os 系统代码
optee_client	source/trustzone/optee/optee_client	Libtee 和 tee-suppllicant 代码
gatekeeper	source/trustzone/optee/optee_os/optee_applications/kmgk/gatekeeper	gatekeeper service 代码
keymaster	source/trustzone/optee/optee_os/optee_applications/kmgk/keymaster	Keymaster Service 和 TA

4.2 软件设计

4.2.1 架构图

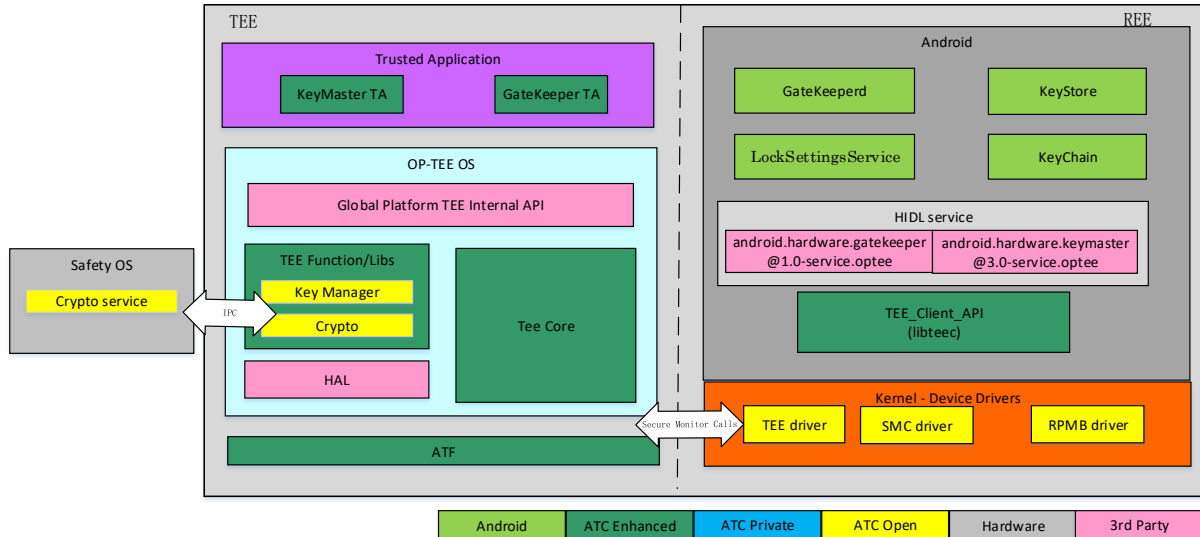


图 4-1 Android Security 架构图

4.2.2 流程图

4.2.2.1 OP-TEE 流程图

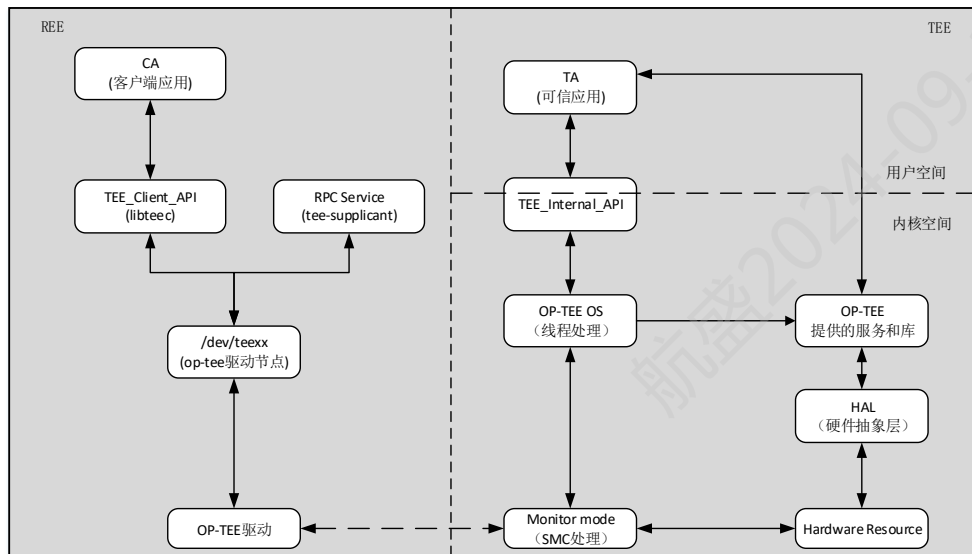


图 4-2 OP-TEE 流程图

4.2.2.2 KeyStore 流程图

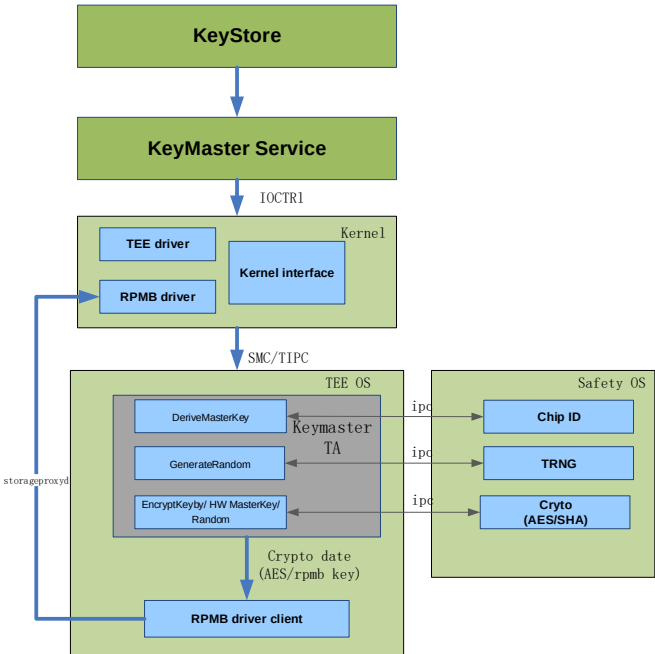


图 4-3 KeyStore 流程图

4.2.2.3 GateKeeper 流程图

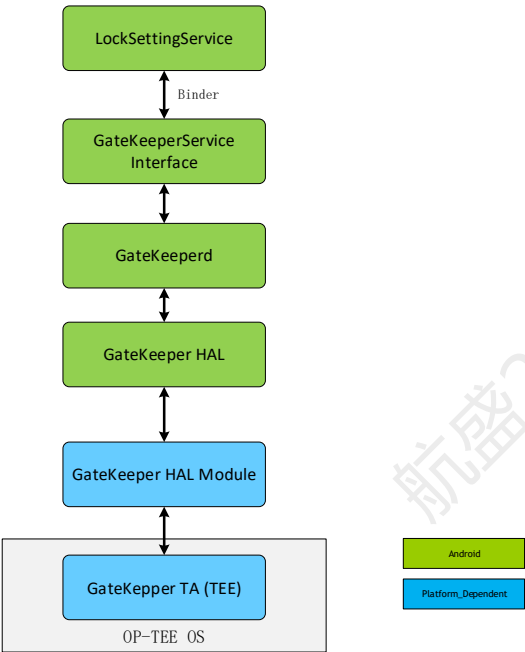


图 4-4 GateKeeper 流程图

4.3 接口描述

- OP-TEE 接口属于 GP 标准 API，具体可参考：

https://optee.readthedocs.io/en/latest/architecture/globalplatform_api.html#

- KeyStore 和 GateKeeper 采用 Android API，接口参考 Android 官方网站：

<https://source.android.com/docs/security/features/keystore?hl=zh-cn>

<https://source.android.com/docs/security/features/authentication/gatekeeper?hl=zh-cn>

4.4 使用说明和注意事项

4.4.1 OP-TEE 更新 SDK 目录

当使用 SDK 开发 OP-TEE 应用时，由于 OP-TEE 编译嵌套在 Android 编译框架中，无法直接更新 SDK。因此，在修改了 OP-TEE OS 的配置文件 conf.mk 或在 libteec 库中添加接口后，需要手动更新 SDK 目录以使修改生效，更新步骤如下：

在 OP-TEE OS 编译完成后，将路径 `source/android/out/target/product/${device}/optee/arm-plat-autochips/export-ta_arm64/` 下的内容复制或移动到 OP-TEE SDK 目录的路径 `source/trustzone/optee/sdk/export-ta_arm64/` 中。

通过执行上述操作，将修改后的 OP-TEE 应用程序更新到 OP-TEE SDK 的目录中，以便在使用 SDK 开发 OP-TEE 应用时使用最新的修改。

4.5 用户定制化

4.5.1 OPTEE-OS 常规宏配置

- CFG_TEE_CORE_LOG_LEVEL OP-TEE log 等级修改，Max=4，Mix=0
- CFG_RPMB_FS RPMB 文件系统功能使能关闭，secure storage 使用 RPMB
- CFG_RPMB_RESET_FAT 系统无法正确读取 RPMB 中 fat 文件系统时复位 RPMB 数据
- CFG_REE_FS REE 文件系统开关，存储安全文件，关闭时保存在 RPMB 中
- CFG_TEE_FS_PARNT_PATH 定义在 optee_client 中，REE 文件路径，默认路径/teedata/

4.5.2 Android 原生 GateKeeper 配置

Android 原生 Keymaster 和 GateKeeper 服务配置方法：

在 Android 代码：source/android/device/autochips/vendor/ac8x/common/device.mk 中

Remove optee services:

```
PRUDUCT_PACKGES += android.hardware.keymaster@3.0-service.optee
```

```
PRUDUCT_PACKGES += android.hardware.gatekeeper@1.0-service.optee
```

Add android services:

```
PRUDUCT_PACKGES += android.hardware.keymaster@3.0-impl
```

```
PRUDUCT_PACKGES += android.hardware.keymaster@3.0-service
```

```
PRUDUCT_PACKGES += android.hardware.gatekeeper@1.0-service.software
```

4.6 典型问题

4.6.1 OP-TEE 错误码分析

【问题描述】

调用 TA 时出错，查看 TEE log 中错误码，例如 D/TC:? 1 ldelf_syscall_open_bin:144 res=0xffff0008。

【原因分析】

OP-TEE 中在服务出错会打印对应的错误码。

【解决方案】

查看 source/trustzone/optee/optee_os/lib/liutee/include/tee_api_defines.h 中相关定义。

例如 0xFFFF0008 TEE_ERROR_ITEM_NOT_FOUND，表明 loadelf 时无法找到对应 bin 文件。

4.6.2 OP-TEE 中添加宏定义和配置

4.6.2.1 OS 中常规宏定义

正常情况下，OPTEE-OS 的宏定义位于 source/trustzone/optee_os/core/arch/arm/plat-autochips/conf.mk 中，其中包含一些内存分配和平台差异性定义。

4.6.2.2 TA 中使用宏配置

TA 代码中无法直接调用 Android.mk 或者 optee-os 中 conf.mk 的配置定义，需要在对应 Makefile 中添加对应配置，具体添加方法参考如下：

【步骤 1】source/trustzone/optee_os/build/Android.mk 把宏定义传递到 OPTEE_EXTRA_FLAGS 中作为整体编译变量：

```
OPTEE_EXTRA_FLAGS += OPTEE_PLATFORM_FLAVOR=$(OPTEE_PLATFORM_FLAVOR)
```

【步骤 2】TA 对应 make 中添加 CFLAGS：

```
ifeq ("$(OPTEE_PLATFORM_FLAVOR)","ac8025")  
CFLAGS += -DPLATFORM_FLAVOR_ac8025=1  
endif
```

4.6.3 TA 应用实例

借助 OP-TEE 来实现特定安全需求时，一次完整的功能调用一般都是起源于 CA，TA 做具体功能实现并返回数据到 CA，而整个过程需要经过 OP-TEE 的 client 端接口，OP-TEE 在 Linux Kernel 端的驱动，Monitor 模式下的 SMC 处理，OP-TEE OS 的 thread 处理，OP-TEE 中的 TA 程序运行。

完成一次完整的 CA 请求为例，在 userspace 端需要执行的操作依次如下。

4.6.3.1 初始化 context

先会调用 TEEC_InitializeContext 函数来初始化 context。open 一个 tee0 的节点，会对应的创建一个 context，这个 kernel 侧的 context 是和 userspace 侧的 fd 相对应。

4.6.3.2 创建 session

调用 TEEC_OpenSession 函数通过获取到的 TEE_Context 类型的变量创建一个特定 CA 与特定 TA 之间进行通信的通道。通过 ioctl 转入 kernel 后，通过 SMC 调用后到达 TEE 侧，最终是在 Ta_OpenSessionEntryPoint 中处理的。

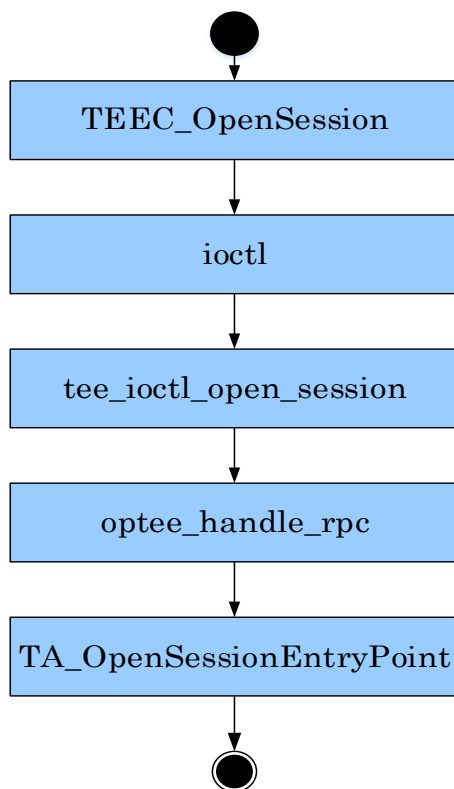


图 4-5 TA open session

4.6.3.3 发起请求

使用已经创建好的会话（session），CA 端规定的命令 ID（command ID），以及配置好的 TEEC_Operation 类型变量，作为参数调用 TEEC_InvokeCommand 函数，实现真正的请求发起。成功调用 TEEC_InvokeCommand 后，请求通过 ioctl 传递到内核，并通过 SMC 调用到达 TEE 侧。在 TEE 侧，请求由 Ta_InvokeCommandEntryPoint 函数处理，并通过 TEEC_Operation 类型变量的 params 成员将结果和相关数据返回给 CA。Android 会发送 check permission 跟 decrypt 的请求到 TA 端做相应的服务。

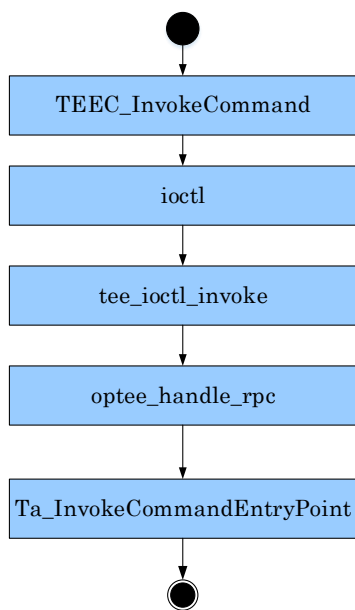


图 4-6 TA invoke command

4.6.3.4 释放资源

调用成功之后如果不需要再次调用该 TA 则需要注销 session 和释放掉 context，这两个操作通过调用 TEEC_CloseSession 函数和 TEEC_FinalizeContext 函数来实现。

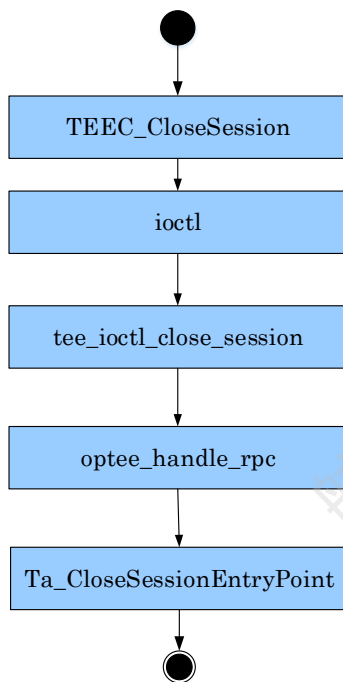


图 4-7 TA close session

4.6.3.5 实例参考

CA 会调用 tee_connect 先初始化 context 跟 open session 与 optee 建立连接。

```
TEEC_InitializeContext(NULL,&dctx);  
TEEC_OpenSession(&dctx,&dsess,&uuid.TEEC_LOGIN_PUBLIC,NULL,NULL,&err_origin);
```

然后通过 tee_call 调用 invokecommand 发送需要 TA 做的服务请求。

```
TEEC_Operation op;  
TEEC_InvokeCommand(&dsess.msg_in->cmd.&op.&err_origin);
```

5 IPC

5.1 简介

IPC 模块用于与之相连的任意两个处理器间的通信。简单来说，就是提供一个门铃（doorbell，实际中也就是中断）功能，发起者按接收者的门铃让其知道在预配置的地方有数据等待处理，接收者在检索完这块数据之后再按发起者的门铃确认数据已接收。将发起通信的处理器称作 **originator** 或 **initiator**，接收通信的处理器称作 **responder**，两者之间的通信渠道称作 **channel**。注意，这里描述的所有通信都是单向的。**originator** 仅通知有新的消息，**responder** 也仅确认消息接收。如需反向通信，则需要 **originator** 和 **responder** 的身份互转并建立新的 **channel**。

图 5-1 演示了这样的一个过程：任务 A 和 B 分别处在不同的处理器上，A 若向 B 发起通信：

- 1) 任务 A 先把共享数据放置 A 和 B 均能访问的内存上
- 2) 然后任务 A 调用 IPC 发送消息通知到 B 处理器
- 3) B 处理器收到发送中断后，告知任务 B 查看共享数据
- 4) B 处理器再调用 IPC 确认消息已收到，反馈给处理器 A
- 5) A 处理器收到确认中断后，告知任务 A（非必须，AC8025 屏蔽了确认中断）

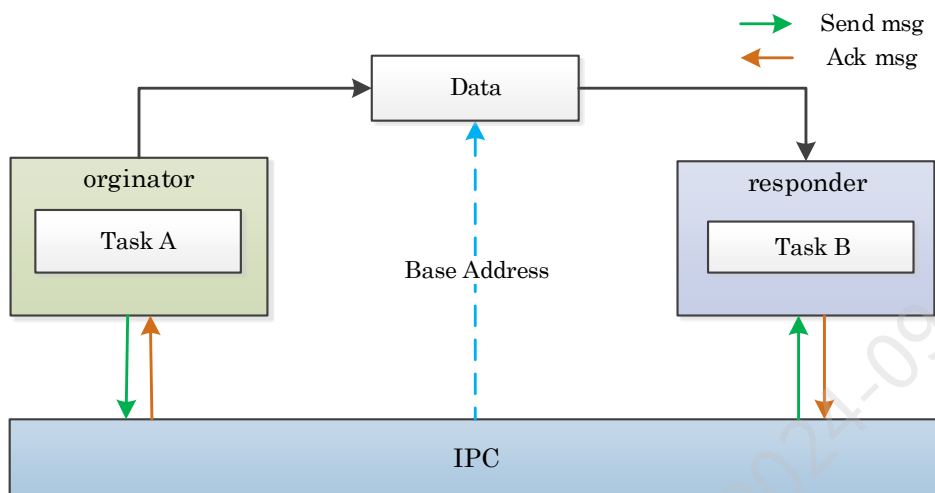


图 5-1 IPC 框图

AC8025 的 IPC 模块实际配置如下：

- 1) 共支持 32 个 **originator** 和 32 个 **responder**，理论上可建立多达 32 x 32 个 **channel**，且每条 **channel** 的使用和用途没有限制。发起者和接收者通过协商选定 **originator** 和 **responder**，或者静态地定好都可以；
- 2) 通信的消息内容、格式、大小等和 HW 无关，一切由 **originator** 和 **responder** 两边的 SW 商定。IPC 为 **originator** 提供了 2 个 32-bit 的寄存器供设置消息存放的基地址，为 **responder** 提供了 2 个 32-bit 的寄存器查询消息存放的基地址。另外提供最大可达 255 个未处理消息长度计数；

- 3) 可用中断数目仅有 32 个。originator 和 responder 可以独立绑定终端，当 originator 发送消息时 responder 会收到中断，当 responder 确认消息时 originator 也可以收到。此外还可以配置命令错误、消息溢出等异常中断。

5.1.1 功能描述

IPC 用于实现跨处理器、跨操作系统间的通信。

5.1.2 代码结构及文件描述

表 5-1 IPC 源码目录和文件

目录	描述
source/kernel/common/drivers/soc/autochips/ipc	内核的 ipc engine, proxy 驱动
source/vendor/autochips/proprietary/hardware/libipc	用户层的 ipc library
source/vendor/autochips/proprietary/tinysys/hsm/os/driver/misc	Hsm os 的 engine 驱动

5.2 软件设计

5.2.1 架构图

图 5-2 展示了多处理器间通信的系统框架图。在 AP 上运行的 ATF, TEE, AAH, 各 domain 的内核层和用户层, 已经 HSM 上运行的 Safety OS 都有 IPC 的 engine 驱动, 这些驱动的功能都是相似的:

- 1) 预设好 channel, 中断, 消息 base address、buffer length 的配置, 提供 channel 的申请、释放和消息发送、接收 API 接口;
- 2) 仅负责发送、接收消息, 不负责消息的解析;
- 3) 对 IPC HAL 做一个封装, 使用户无需关心 IPC 内部是如何工作的;
- 4) 基于 Linux kernel 的 domain 的驱动中还包含 proxy 驱动, 是对 engine 驱动的一个简单封装字符设备, 以 IOCTL 的访问方式抛给 HAL 层访问。

在各 domain 的用户层 java 和 native apps 通信需求, 提供 libipc 库。ibipc 库会对驱动层提供的 IOCTL 再做一个简单封装, 使之类似于 socket 的使用方式。

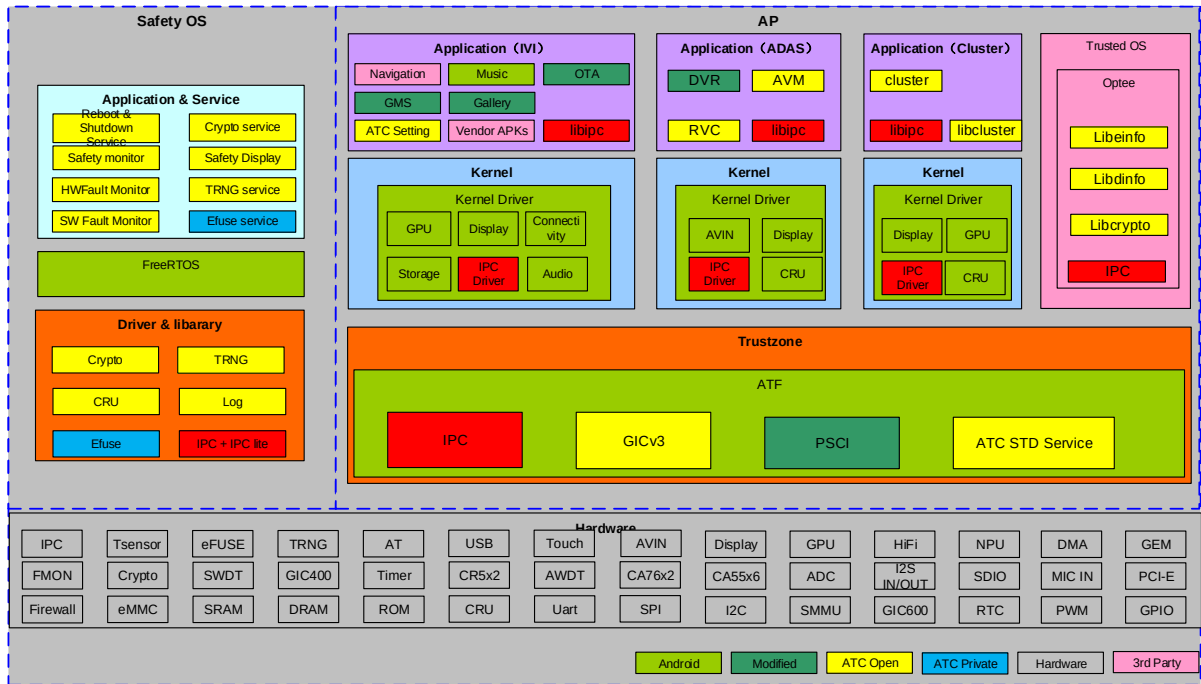


图 5-2 IPC 系统架构图

5.2.2 流程图

IPC engine 基本操作 flow 如图 5-3:

- Originator 和 responder 的 IPC driver 做初始化的配置
- Originator 调用 ipc_send 发送消息
 - IPC driver 查询消息是否已满。如果已满返回 EFULL 错误，否则根据 last_entry 往对应偏移的地址填写 message;
 - 触发 Add message, 返回 ok。
- Responder 会收到中断，进入到 receiver 的 irq handler
 - 先 mask 该中断，查询 responder 的中断 status，判断是否为 add/remove msg 的 flag。如果不是则进入对应的错误处理流程，如果是则查找对应的 receiver 并从 first_entry 处读取消息的 receiver 字段 channel 中 copy 消息;
 - 通知对应 receiver 收到新的消息，使之退出阻塞态;
 - 触发 remove message。
- Originator 会收到中断，进入到 sender 的 irq handler（非必须）

先 mask 该中断查询 originator 的中断 status，判断是否为 add/remove msg 的 flag，如果不是进入对应的错误处理流程。

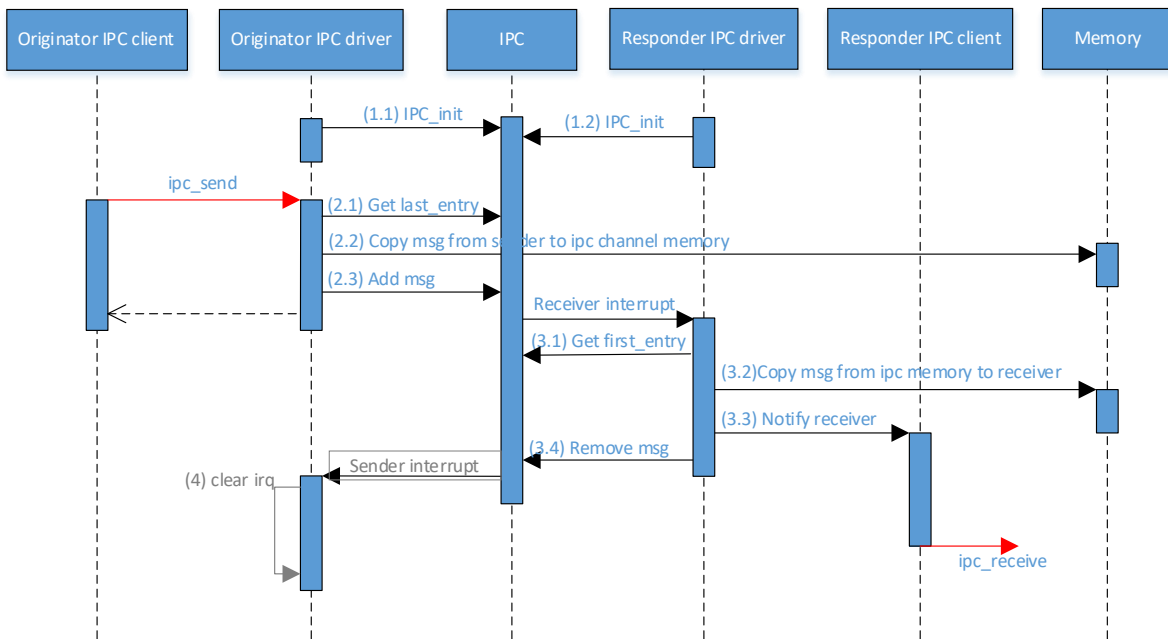


图 5-3 IPC send and ack 流程

5.3 接口描述

以 KERNEL 为例，同系统会有略微差别。

IPC 的 sender/receiver，默认设计是不同的 core，实际使用中，sender/receiver 的定义按需进行了扩展。当前 AC8025 设计的 IPC sender/receiver 定义参考如下 IPC_CORE 枚举定义：

```
enum IPC_CORE {
    CORE_SAFETY = 1,    //HSM OS   R5F
    CORE_ATF  = 4,    //trustzone
    CORE_TEE  = 0x10, // trustzone
    //下述不同产品可能只存在一个 dom 域。
    CORE_DOM0  = 0x20, //DOM0   系统 1
    CORE_DOM1  = 0x40, //DOM1   系统 2
    CORE_DOM2  = 0x80, //DOM2   系统 3
};
```

消息的方向的描述为：

```
enum IPC_DIRECTION {
    MSGBIDIRECTION = 0, //双向   目前都是单向
};
```

```
MSGSEND = 1, //发送
MSGRECV = 2, //接收
MSGNONE = 3, //
};
```

消息格式如下:

```
#define MAX_MSG_LEN 236
typedef struct IPC_msg {
    char sender[8]; //message sender
    char receiver[8]; //message receiver
    uint32_t size; //message size
    uint8_t msg[MAX_MSG_LEN]; //direct message content
} ipc_msg_t;
```

对一个物理 IPC channel 设备的抽象:

```
typedef struct IPC_chan_dev {
    uint8_t oid; //发送端 id
    uint8_t rid; //接收端 id
    uint8_t irq_hw; //物理中断号
    uint8_t buffer_nr; // 消息的 buffer 长度
    uint32_t packet_size; //包长度
    uint32_t buffer_size; //存放消息的长度
    uint64_t buffer_address; //存放消息的地址

    /* software */
    ipc_msg_t *base_addr; //存放消息的虚拟地址
    uint32_t irq; //逻辑中断号
    uint32_t idr_ref; //reference count
    struct list_head ipc_list; // receiver list
    spinlock_t lock;
    enum IPC_DIRECTION dir; //direction
} ipc_chan_dev_t;
```

对 ipc 使用者来说的软件 channel 抽象:

```
#define IPC_MSG_QUEUE_LEN 4
typedef struct IPC_msg_data {
    uint32_t size;
```

```
void *msg;
} ipc_msg_data_t;

typedef struct IPC_chan {
    struct list_head list_node;
    wait_queue_head_t wait;
    ipc_chan_dev_t *dev;//物理通道设备
    char sender[8];//发送端名称
    char receiver[8];//接收端名称
    unsigned msg_count, msg_free, msg_drop;//消息数量, free 数量, 丢失的消息数量
    ipc_msg_data_t msg_data[IPC_MSG_QUEUE_LEN];
    spinlock_t lock;
    ipc_handler_t rx_callback;//callback 函数
    ipc_handler_t terminate_callback;//发送端断开, 接收到 0xdeadbeaf 回调处理函数
} ipc_chan_t ;
```

5.3.1 API

常用的 API 如下表。

5.3.1.1 Kernel Driver API

表 5-2 IPC Kernel driver API

API	说明
ipc_request_chan	请求channel，用于存储消息。一次channel请求后可以触发多次send或者receive
ipc_request_chan_detailed	请求channel，比ipc_request_chan多了sender和receiver两个参数
ipc_release_chan	释放channel
ipc_sendmsg	发送消息，带msg和size的参数
ipc_receive	接收消息信号
ipc_receive_timeout	接收消息信号，如果等待超时会退出等待
ipc_receivemsg	接收消息
ipc_receivemsg_timeout	接收消息，如果等待超时会退出等待
ipc_chan_dev_init	ipc物理通道的初始化
ipc_register	为ipc物理通道的发送或接收中断，绑定中断处理函数
ipc_notify	ipc发送端通知接收端

ipc_ack	ipc接收端处理完消息后，向发送端回馈
ipc_clear_interrupt	清除ipc的中断状态

5.3.1.1.1 ipc_request_chan

【描述】

申请 ipc channel，用于存储消息。

【语法】

ipc_chan_t *ipc_request_chan (enum IPC_CORE proc, enum IPC_DIRECTION direction)

【返回值】

返回值	描述
非 NULL	IPC_chan 句柄
NULL	申请通道失败。

【参数】

参数名称	描述	输入/输出
proc	IPC_CORE	输入
direction	发送 or 接收	输入

5.3.1.1.2 ipc_request_chan_detailed

【描述】

申请 ipc channel，用于存储消息。

【语法】

ipc_chan_t *ipc_request_chan_detailed (enum IPC_CORE proc, enum IPC_DIRECTION direction, char *sender, char *receiver)

【返回值】

返回值	描述
非 NULL	IPC_chan 句柄
NULL	申请通道失败

【参数】

参数名称	描述	输入/输出
proc	IPC_CORE	输入
direction	发送 or 接收	输入
sender	发送者的名字（最大 8 字节）	输入
receiver	接受者的名字（最大 8 字节）	输入

5.3.1.1.3 ipc_release_chan

【描述】

释放 ipc channel。

【语法】

```
void ipc_release_chan (ipc_chan_t* chan)
```

【返回值】

无

【参数】

参数名称	描述	输入/输出
chan	IPC_chan client instant	输入

5.3.1.1.4 ipc_sendmsg

【描述】

发送 ipc 消息。

【语法】

```
int ipc_sendmsg (ipc_chan_t *chan, void *msg, uint32_t size)
```

【返回值】

返回值	描述
非负值	成功发送的消息长度
负值	发送消息失败，参见 错误码 。

【参数】

参数名称	描述	输入/输出
chan	IPC_chan 句柄	输入
msg	存放发送消息的指针	输入
size	存放发送消息的大小（长度有限制）	输入

5.3.1.1.5 ipc_receive

【描述】

无限期的等待 ipc 消息。消息的内容和长度存在 chan 结构体对应的字段中。

【语法】

```
int ipc_receive(ipc_chan_t * chan)
```

【返回值】

返回值	描述
0	正常接收到消息信号
负值	接收消息信号失败，参见 错误码 。

【参数】

参数名称	描述	输入/输出
chan	IPC_chan 句柄	输入

5.3.1.1.6 ipc_receive_timeout

【描述】

在指定限期内等待 ipc 消息信号，超时返回错误。消息的内容和长度存在 chan 结构体对应的字段中。

【语法】

```
int ipc_receive_timeout(ipc_chan_t * chan, int timeout)
```

【返回值】

返回值	描述
0	正常接收到消息信号
负值	接收消息信号失败，参见 错误码 。

【参数】

参数名称	描述	输入/输出
chan	IPC_chan 句柄	输入
timeout	超时时间（单位 ms）	输入

5.3.1.1.7 ipc_receivemsg

【描述】

无限期的等待 ipc 消息。

【语法】

```
int ipc_receivemsg(ipc_chan_t * chan, void *msg, uint32_t size)
```

【返回值】

返回值	描述
非负值	正常接收到消息长度
负值	接收消息失败，参见 错误码 。

【参数】

参数名称	描述	输入/输出
chan	IPC_chan 句柄	输入
msg	存放接收消息的指针	输出
size	最大能接收的消息长度	输出

5.3.1.1.8 ipc_receivemsg_timeout

【描述】

在指定限期内等待 ipc 消息，超时返回错误。

【语法】

```
int ipc_receivemsg_timeout(ipc_chan_t * chan, void *msg, uint32_t size, int timeout)
```

【返回值】

返回值	描述
-----	----

返回值	描述
非负值	正常接收到消息长度
负值	接收消息失败，参见 错误码 。

【参数】

参数名称	描述	输入/输出
chan	IPC_chan 句柄	输入
msg	存放接收消息的指针	输出
size	最大能接收的消息长度	输出
timeout	超时时间（单位 ms）	输入

5.3.1.1.9 ipc_chan_dev_init

【描述】

物理通道初始化。

【语法】

ipc_chan_dev_t * ipc_chan_dev_init (uint8_t oid, uint8_t rid, uint8_t num, int irq, uint64_t addr, uint32_t size)

【返回值】

返回值	描述
非 NULL	IPC_chan_dev 句柄(即物理通道创建的句柄)
NULL	申请通道失败。

【参数】

参数名称	描述	输入/输出
oid	发送者 id	输入
rid	接受者 id	输入
num	可存放消息的数量	输入
irq	中断号	输入
addr	消息地址	输入

参数名称	描述	输入/输出
size	一个消息包的大小	输入

5.3.1.1.10 ipc_register

【描述】

为物理通道的发送或接收中断绑定中断处理函数。

【语法】

```
int ipc_register(ipc_chan_dev_t *dev)
```

【返回值】

返回值	描述
0	注册成功
非 0	失败，参见 错误码 。

【参数】

参数名称	描述	输入/输出
dev	IPC_chan_dev 句柄	输入

5.3.1.1.11 ipc_notify

【描述】

用于发送端触发 ipc 通知到接收端。

【语法】

```
int ipc_notify (ipc_chan_dev_t *dev)
```

【返回值】

返回值	描述
0	触发消息成功
非 0	触发消息失败

【参数】

参数名称	描述	输入/输出
------	----	-------

参数名称	描述	输入/输出
dev	IPC_chan_dev 句柄	输入

5.3.1.1.12 ipc_ack

【描述】

用于接收端在处理完消息后，确认消息已经处理完成。

【语法】

```
int ipc_ack(ipc_chan_dev_t *dev)
```

【返回值】

返回值	描述
0	回应消息成功
非 0	回应消息失败

【参数】

参数名称	描述	输入/输出
dev	IPC_chan_dev 句柄	输入

5.3.1.1.13 ipc_clear_interrupt

【描述】

用于发送或接收中断处理函数中，清除 ipc 中断状态寄存器。

【语法】

```
int ipc_clear_interrupt(ipc_chan_dev_t *dev)
```

【返回值】

返回值	描述
0	成功
非 0	失败

【参数】

参数名称	描述	输入/输出
------	----	-------

参数名称	描述	输入/输出
dev	IPC_chan_dev 句柄	输入

5.3.1.2 HSM OS API

表 5-3 IPC HSM OS API

API	说明
pxIpcRequestChan	ipc_request_chan()的别名，请求channel，用于存储消息。类似 kernel 中ipc_request_chan_detailed
vIpcReleaseChan	ipc_release_chan()的别名，释放channel
sIpcSendMsg	ipc_sendmsg()的别名，发送消息，带msg和size的参数
sIpcReceiveMsg	ipc_receivemsg()的别名，接收消息
sIpcReceiveMsgTimeOut	ipc_receivemsg_timeout()的别名，接收消息，如果等待超时会退出等待

以上接口调用参考 HSM OS tasks/pwrkey_task.c 中使用即可。

5.3.1.3 LIBIPC API

表 5-4 LIBIPC API

API	说明
ipc_init	初始化ipc channel
ipc_deinit	反初始化ipc channel
ipc_send	发送消息
ipc_recv	阻塞接收消息
ipc_recv_timeout	接收消息，带超时属性

5.3.1.3.1 ipc_init

【描述】

申请 ipc channel, 用于存储消息。

【语法】

```
int ipc_init (enum IPC_CORE core_id, enum IPC_DIRECTION direction, char *sender, char *receiver)
```

【返回值】

返回值	描述
非负值	ipc 文件句柄
负值	失败

【参数】

参数名称	描述	输入/输出
core_id	IPC_CORE	输入
direction	发送 or 接收	输入
sender	发送者的名字（最大 8 字节）	输入
receiver	接受者的名字（最大 8 字节）	输入

5.3.1.3.2 ipc_deinit

【描述】

释放 ipc channel。

【语法】

```
void ipc_deinit (int ipcfid)
```

【返回值】

无

【参数】

参数名称	描述	输入/输出
ipcfid	ipc 文件句柄	输入

5.3.1.3.3 ipc_send

【描述】

发送 ipc 消息。

【语法】

```
int ipc_send (int ipcfid, void *msg, uint32_t size)
```

【返回值】

返回值	描述
-----	----

返回值	描述
非负值	成功发送的消息长度
负值	失败，参见 错误码 。

【参数】

参数名称	描述	输入/输出
ipcfid	ipc 文件句柄	输入
msg	存放发送消息的指针	输入
size	存放发送消息的大小	输入

5.3.1.3.4 ipc_recv

【描述】

在指定限期内等待 ipc 消息，超时返回错误。

【语法】

```
int ipc_recv(int ipcfid, void *msg, uint32_t size)
```

【返回值】

返回值	描述
非负值	正常接收到消息长度
负值	接收消息失败，参见 错误码 。

【参数】

参数名称	描述	输入/输出
ipcfid	ipc 文件句柄	输入
timeout	超时时间（单位 ms）	输入

5.3.1.3.5 ipc_recv_timeout

【描述】

在指定限期内等待 ipc 消息，超时返回错误。

【语法】

```
int ipc_recv_timeout(int ipcfid, void *msg, uint32_t size, int timeout)
```

【返回值】

返回值	描述
非负值	正常接收到消息长度
负值	接收消息失败，参见 错误码 。

【参数】

参数名称	描述	输入/输出
ipcfid	ipc 文件句柄	输入
msg	存放接收消息的指针	输出
size	最大能接收的消息长度	输出
timeout	超时时间（单位 ms）	输入

5.3.1.4 错误码

错误代码	定义	描述	建议处理
0x1	IPC_HANDLE_ERROR	Ipc 接口中调用 ioctl 出现错误	根据调用场景做容错处理
0x2	IPC_INVALID_ARGS	无效的参数	检查参数
0x3	IPC_RCV_TIMEOUT	接收消息超时	超时后容错处理
0x4	IPC_SND_BUSY	发送通道满	Buf 满后，请等待一段时间，retry，或者做容错处理
0x5	IPC_NODE_NO_FOUND	IPC 节点信息没找到	检查 dtb 中信息
0x6	IPC_MEMMAP_ERROR	Iomap 失败	检查 iomap 失败原因
0x7	IPC_INIT_DEV_ERROR	初始化创建物理通道失败	检查 ipc_init 中初始化逻辑

非上述错误码可以参看 errno.h 中的错误值。

5.4 使用说明和注意事项

5.4.1 算法描述

IPC 驱动负责消息的发送和接收，以及并发通信的处理（对用户是透明的）。而消息内容的设定和解析由发送者和接收者约定。

5.4.1.1 信息格式

消息格式如下结构体，其中 `sender` 和 `receiver` 字段代表发送者和接收者的名字，`receiver` 是判断该消息发给谁的关键，`sender` 保留暂不解析。

```
struct IPC_msg {
    char sender[8]; //message sender
    char receiver[8]; //message receiver
    uint32_t msg_size; //message size
    uint8_t msg[MAX_MSG_LEN]; // message content
};
```

对不同 `size` 的通信内容，处理方式也会有所差别：

- 1) `size` 很小（少于或等于 `MAX_MSG_LEN`），比如几个参数或不长的一段字符串，`msg[]` 数组完全可以容纳下这段通信内容。推荐方案是将想要发送的内容存储到 `msg[]` 数组中，通信内容大小赋值给 `size`。
- 2) `size` 很大（超过 `MAX_MSG_LEN`），比如下图 5-4 一块 dump 信息或真实消息内容不方便放置在 `msg[]` 数组中。推荐方案是将这个消息的所在 `memory` 物理地址和 `size` 存储在 `msg[]` 数组中，通信内容大小为 4 赋值给 `size`。

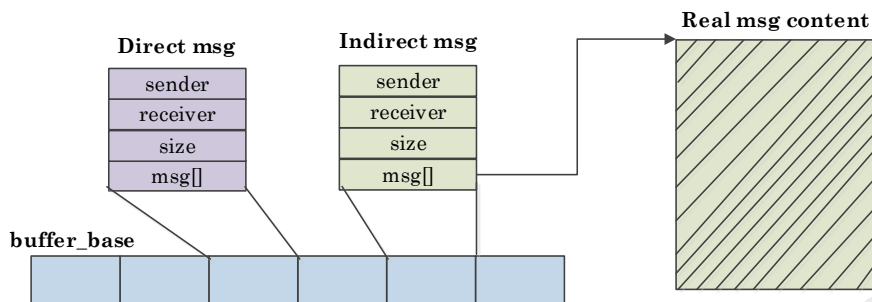


图 5-4 IPC message

5.4.1.2 单个通信 channel 上并发通信

在某个通信 `channel` 上并发通信的情况可能有如下两种：

- 1) 两个或以上的信息需要同时发送，或者确认；
- 2) 一个发送的信息还未得到 `responder` 确认处理的情况下，又一个信息等待发送。

对于情况一，需在 SW 上使用锁做好同步处理，即 `add msg` 和 `remove msg` 不能重入。

对于情况二，每次都等到上一条消息被 `responder` 处理完毕才能下一条消息（也就是消息的处理是顺序的）。但消息的处理可能会消耗一定时间，IPC HW 提供了未处理消息的应对机制，其中未处理消息的 `buffer` 长度最大可达 255。如下图 5-5，Originator 每新增一条消息 `last_entry` 以 `buffer` 长度为模递

增 1（即消息总数累积超过 buffer 长度后 last_entry 会从 0 开始），responder 每删除一条消息 first_entry 以 buffer 长度为模递增 1。具体考虑如下三种情况：

- 1) 如果 add msg 的频次一直等同于 remove msg 的时候（即每次 originator 发消息 responder 都及时处理，期间没有未处理的消息），first_entry 会在 responder remove msg 后等于 last_entry；
- 2) 如果 add msg 的频次多于 remove msg 的时候，last_entry 和 first_entry 就会有空隙，空隙等于 buffer 长度的时候会 full 溢出报异常；
- 3) 如果 remove msg 的频次多于 add msg 的时候，在没有报异常中断的情况是正常可被忽略的。

看起来 buffer 长度越大越好，尤其是 add msg 很频繁的情况下。但考虑到 buffer 长度越大，我们 reserve memory 就越多，所以 buffer 长度初步设定等于 4，如果出现溢出的情况，再做调整(修改方法为更改各驱动的 IPC_OUTSTANDING_MSG_LEN 值)。

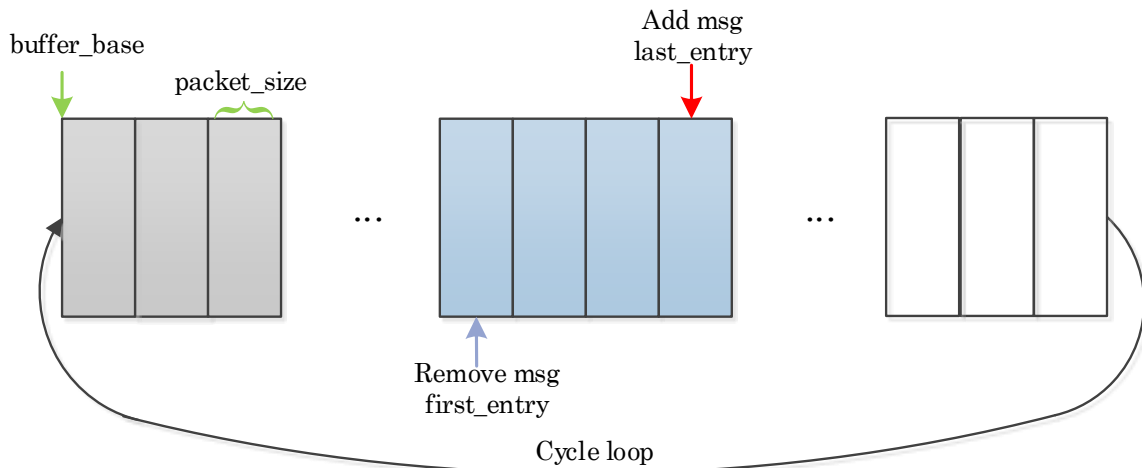


图 5-5 IPC understanding message buffer

5.4.1.3 多处理器间并发通信

为保证每个系统的通信互不干扰，为每个系统间通信建立一对独立的发送和接收 channel。每个 changnel 都有自己独立的状态控制寄存器，通信通道，buffer 和中断，因此从理论上可以做到互不干扰。

默认 IPC 为每个系统之间的通信都预设了一对通信 channel，这一类的通信 channel 被用作对实时性要求不高的模块使用。表 5-5 展示了的 originator 和 responder ID，originator 和 responder 中断，以及消息所在 reserved memory 的配置原理(此图仅供参考，实际以代码中定义的号为准,后续可能会有变动)。

表 5-5 Channel 配置

Channel	Orig ID	Resp ID	Orig Irq	Resp Irq	Size
Safety->ATF	0	6	N/A	6	0x400
Safety ->TEE	0	8	N/A	8	0x400

Channel	Orig ID	Resp ID	Orig Irq	Resp Irq	Size
Safety ->DOM0	0	9	N/A	9	0x400
Safety->DOM1	0	12	N/A	12	0x400
Safety -> DOM2	0	15	N/A	15	0x400
ATF->Safety	2	0	N/A	0	0x400
TEE->Safety	4	2	N/A	2	0x400
DOM0-> Safety	5	3	N/A	3	0x400
DOM1-> Safety	6	4	N/A	4	0x400
DOM2-> Safety	7	5	N/A	5	0x400
DOM0->DOM1	5	13	N/A	13	0x400
DOM0->DOM2	5	16	N/A	16	0x400
DOM1-> DOM0	6	10	N/A	10	0x400
DOM1-> DOM2	6	17	N/A	17	0x400
DOM2-> DOM0	7	11	N/A	11	0x400
DOM2-> DOM1	7	14	N/A	14	0x400

此外有的模块对实时性有较高的要求，需要申请专有的通道 channel，可以参考如下配置(去除 COMMON 通道的，M and N 小于 32)。

表 5-6 专有 Channel 配置

Channel	Orig ID	Resp ID	Orig Irq	Resp Irq	Memory offset	Size
XXX->YYY	M	N	N/A	N	N/A	N/A
YYY->XXX	N	M	N/A	M	N/A	N/A

5.4.2 参考代码

如下以 kernel ipc 接口 为例（简单用法，接口返回值可以根据需要细化做容错处理），分为发送端和接收端。发送端的用法如下：

1. 先调用 ipc_request_chan()或 ipc_request_chan_detail()申请一个 channel;
2. 然后调用 ipc_sendmsg()发送消息，可以多次发送;
3. 当不再发送消息的时候，调用 ipc_release_chan()释放该 channel。

```
#include "ipc_engine.h"

int ipc_send(void)
{
    int ret = 0;
    char *sender = "Aos";
    char *receiver = "Vos";
```

```
char *msg = "Hello World";
uint32_t size = strlen(msg) + 1;
ipc_chan_t *chan;

chan = ipc_request_chan_detailed(CORE_SAFETY, MSGSEND, sender, receiver);
if (!chan) {
    pr_err("%s: request send chan fail\n", __func__);
    return -1;
}

ret = ipc_sendmsg(chan, msg, size);
if (ret < 0) {
    pr_err("%s: send fail %d\n", __func__, ret);
    return -2;
}

ipc_release_chan(chan);
return ret;
}
```

接收端的用法如下：

1. 先调用 ipc_request_chan()或 ipc_request_chan_detail()申请一个 channel;
2. 然后调用 ipc_receivemsg()等接口等待消息，该接口在没有收到消息前是一直阻塞的;
3. 当不再等待消息的时候，调用 ipc_release_chan()释放该 channel。

```
#include "ipc_engine.h"

int ipc_receiver(void)
{
    int ret = 0;
    char *sender = "Vos";
    char *receiver = "Aos";
    char msg[128];

    ipc_chan_t *chan;

    chan = ipc_request_chan_detailed(CORE_SAFETY, MSGRECV, sender, receiver);
```

```
if (!chan) {
    pr_err("%s: request receive chan fail\n", __func__);
    return -1;
}

ret = ipc_receivemsg(chan, msg, 128);
if (ret < 0) {
    pr_err("%s: receive fail %d\n", __func__, ret);
    return -2;
}

pr_info("%s: receive msg %s\n", __func__, msg);

ipc_release_chan(chan);
return ret;
}
```

5.5 典型问题

表 5-7 IPC 典型问题

问题	原因	建议
提示 chan is full	1. 发送端发的太快，或者接受端处理的太慢，导致 FIFO full 2. 接收端 hang 住或者崩溃	1. 合理判断返回值，做 ERROR handle 2. 排查对端进程是否异常
接收消息出错返回-512	接收者所在的进程处在了 exit 状态，ipc 也自然不再接收消息	PIC 分析这种是不是 ipc_receivemsg 所在的进程发生了 crash
接收到大小为 4 内容为 0xDEADBEAF 的消息	表示发送端调用了 ipc_release_chan()	考虑忽略或处理这条消息

6 Safety

6.1 简介

6.1.1 功能

Autochips Safety OS:

- 基于 FreeRTOS v10.0.1 所开发的 RTOS，在操作系统的内核层对操作系统的功能安全、信息安全做了增强处理，包括以下：
 - 增加 MPU/System MPU，做到基于硬件的任务隔离，增强系统的可靠性；
 - 对关键任务进行容错管理，确保关键任务的可执行性；
 - 增强 Trace 机制，对于关键模块实时性能无损的 trace 机制，并提供 offline 分析工具，方便客户查找与分析复杂问题；
 - 增强 Critical 任务的处理，与系统监控、信息安全等模块，指定在芯片 internal 中的 ECC SRAM 上支持，做到外界隔离，确保其稳定与安全性。
- 其他优化处理：
 - LDS 文件整合优化，隔离重要数据。
 - Watch dog，Fault monitor 异常检测
 - 支持在系统提供的任务切换堆栈溢出检测基础上，增加了动态开启功能，切换任务和中断前后都可以进行堆栈溢出检测
 - 支持对于系统异常后，会 dump 相关寄存器，以及 SP 堆栈中的信息（包括中断异常后，对应当前 task 的堆栈。）
 - 提供 PC 侧 hsm 异常分析工具（hsm_panic.exe）

经过多次迭代，Autochips safety OS 已稳定量产，应用在杰发 HSM、仪表、监控等高安全与实时性要求的子系统中。

6.1.2 架构图

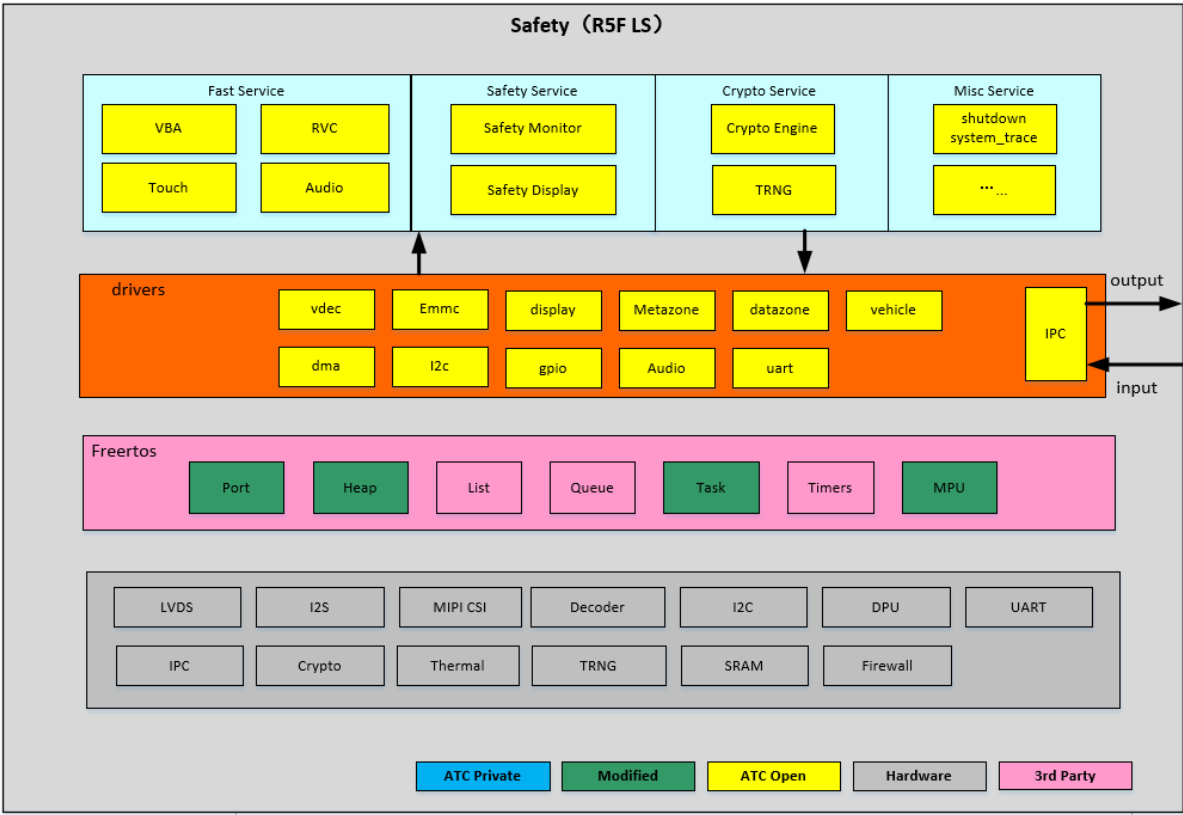


图 6-1 Safety 系统架构图

6.1.3 代码结构及文件描述

整个 satety 代码结构如下图。

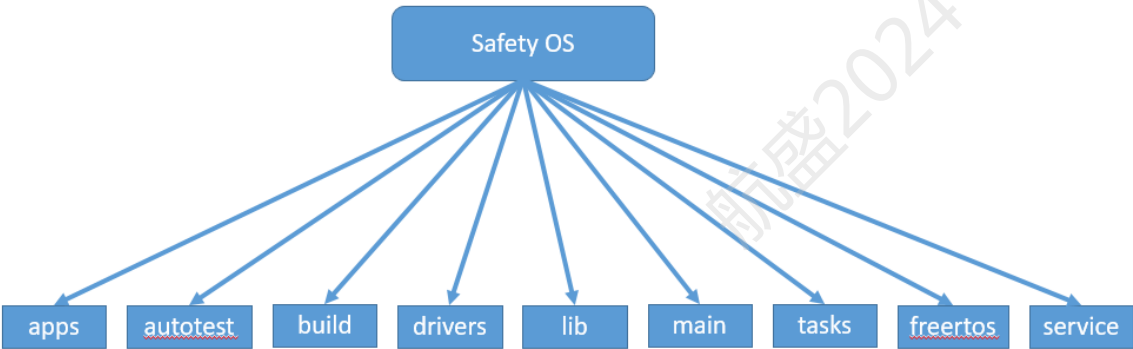


图 6-2 OS layout

表 6-1 Safety 源码目录

模块名	代码路径	功能说明
Apps	source/vendor/autochips/proprietary/tinysys/hsm/os/apps	应用相关 app
autotest	source/vendor/autochips/proprietary/tinysys/hsm/os/autotest	自动化测试
build	source/vendor/autochips/proprietary/tinysys/hsm/os/build	编译相关配置
Drivers	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers	Drviders
lib	source/vendor/autochips/proprietary/tinysys/rtos/freertos/Demo/ CORTEX_R5_Autochips_AC8015/bsp_common_hal/atc_cortexr5_0/libsrc	HAL 层 Efuse
Main	source/vendor/autochips/proprietary/tinysys/hsm/os/main	OS core
Tasks	source/vendor/autochips/proprietary/tinysys/hsm/os/tasks	运行期 任务（会慢慢被 service 取代）
freertos	source/vendor/autochips/proprietary/tinysys/rtos/freertos/Source	Freertos 代码
service	source/vendor/autochips/proprietary/tinysys/hsm/os/service	服务类任务

6.2 使用说明和注意事项

6.2.1 编译注意事项

6.2.1.1 模块代码编译

在顶层目录使用 ./allmake -b safety 编译代码。

6.2.1.2 代码添加

如果需要添加目录，请参考 source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/dummy 目录（其他目录下类似）。

1. 创建类似 dummy 的目录
2. 然后添加代码，比如 dummy.c
3. 参考 Makefile，创建自己的 Makefile，然后再 makefile 中选择 hsm_sram.lds or hsm_dram.lds
4. 然后在 drivers 目录层的 makefile 中加入自己的目录 dummy 的依赖

source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/dummy

----- dummy.c

----- Makefile

6.3 Fast service

Fast service 主要定义为快速启动服务程序，其中包含但不限于 display, audio 等，主要为 VBA, RVC 等应用提供服务。

表 6-2 Fast service

VBA	快速开机动画使用视频硬件解码器进行开机视频/图片解码，调用显示接口进行全屏显示，调用 Audio 接口进行音频输出
RVC	快速倒车服务在开机阶段快速响应倒车信号，实时显示后视摄像头视频画面以及倒车辅助信息等，同时使用音频输出播放警报音

6.4 Safety Service

6.4.1 Safety Monitor

主要用于监控 safety OS，关键进程的正常运行。

- Exception handler，用于处理 safety OS 的 prefetch abort, data abort, undefined instruction 等异常
- Clock monitor，用于监控和对策 clock 的稳定
- Watchdog，用于监控关键线程的运行，和 safety OS 是否发生 crash 或 hang，当发生 crash 或 hang 之后触发 reset 或者导出信号到外部 MCU
- Fault monitor，用于处理系统发生的各种 fault 信号，对于特殊错误可能发到外部 MCU 处理

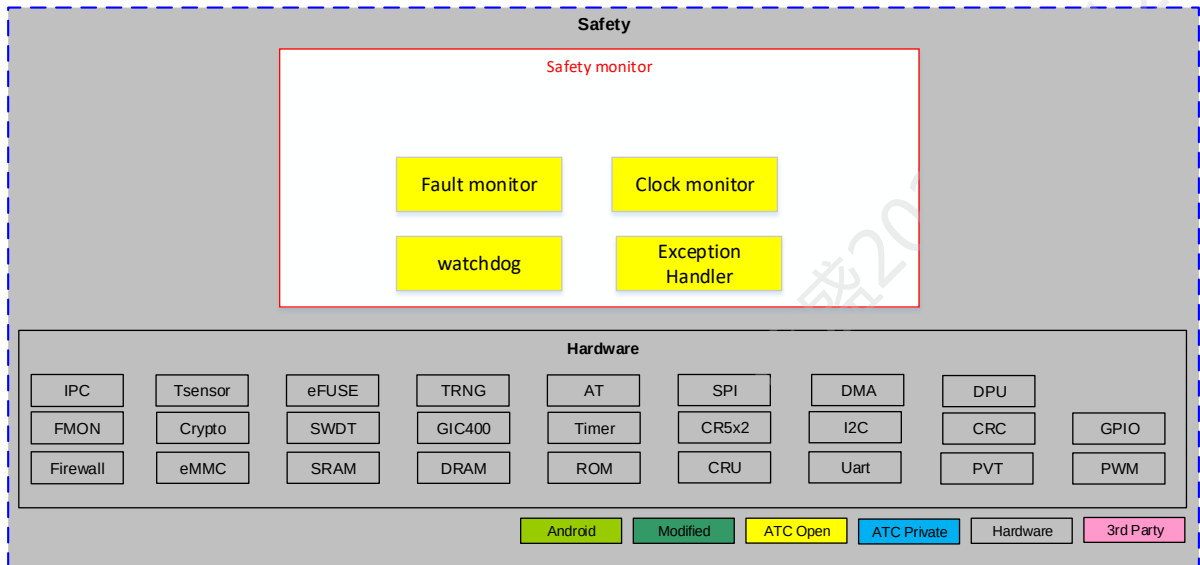


图 6-3 Safety monitor 框架

6.4.1.1 Clock Monitor

- CMU 模块可以检测到 clock 的丢失，和不正确的频率
- 每个 CMU 都是独立编程的
- 可以监控 XOSC

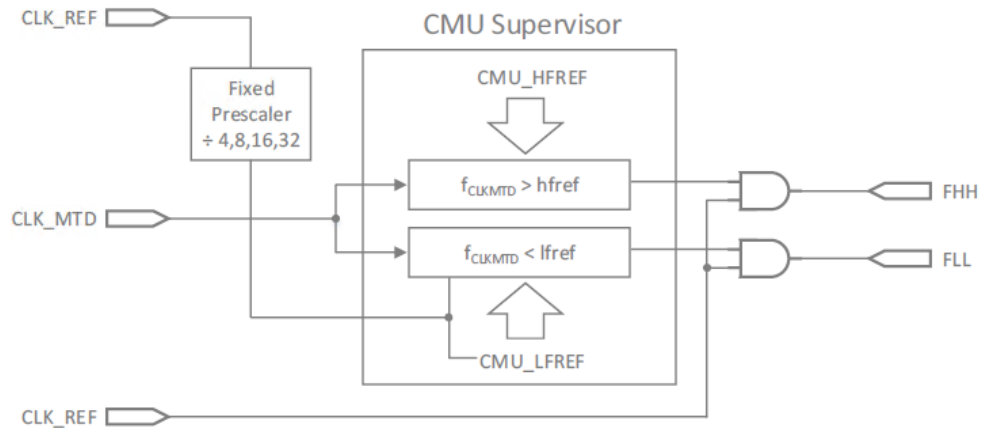


图 6-4 Clock Monitor 硬件框图

6.4.1.2 Exception Handler

- 非关键线程发生 abort 不会影响关键进程的运行
- 关键线程发生 abort，会尝试恢复
- 如果发生在中断上下文，会尝试 reset 该中断对应的 HW 或 SW

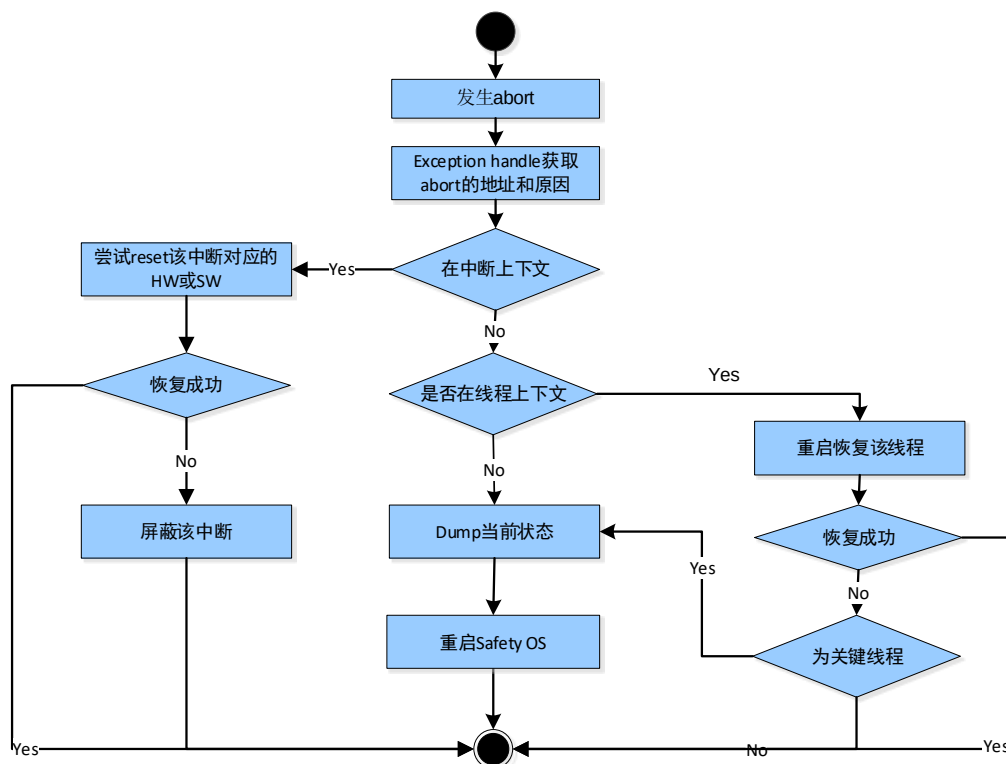


图 6-5 Exception handler 流程

6.4.1.3 Watch dog

Watchdog 用来监控系统的调度是否正常，CPU 需要在预先设定好的时间内对其进行喂狗操作，否则就会导致 Watchdog 超时，Watchdog 大致工作流程如下：

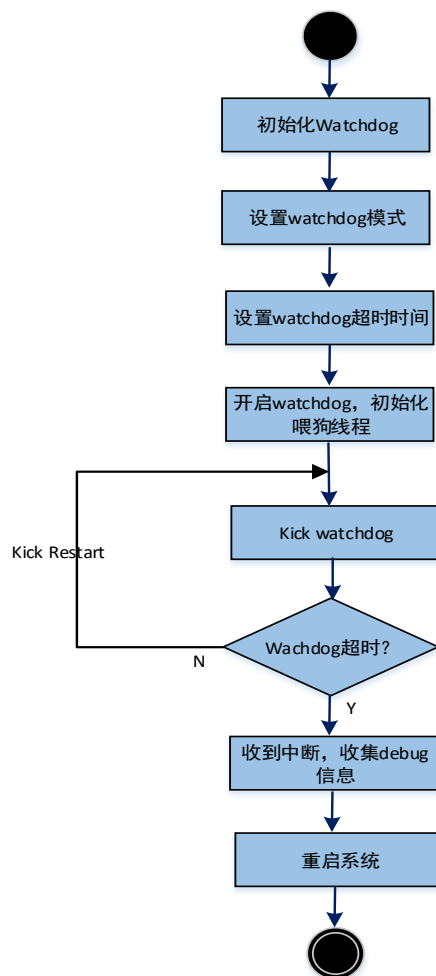


图 6-6 Watchdog 工作流程

AC8025 提供两个 watchdog，分别为 AP watchdog(AWDT)和 Safety watchdog(SWDT)。

6.4.1.3.1 AP watchdog(AWDT)

AWDT 位于 IVI Domain，用于监控 AP 是否挂死或者陷入死循环。

AWDT 模块的整体架构如下图所示：

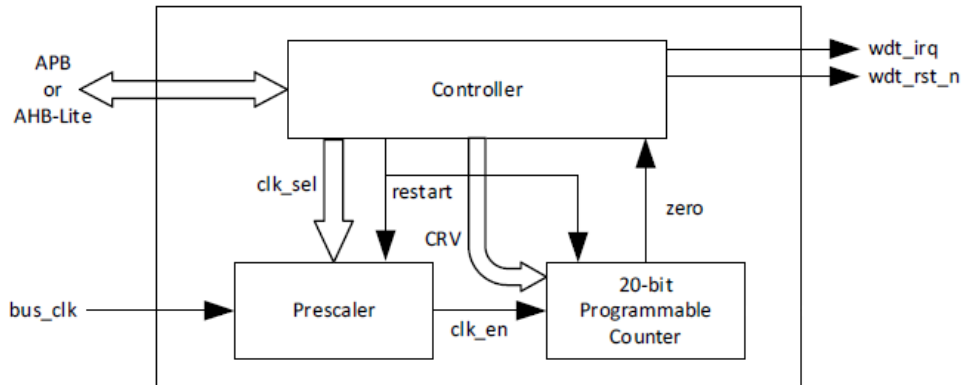


图 6-7 AWDT 硬件框图

AP 在预先设定好的时间内对其进行喂狗操作，否则就会导致 AWDT 超时。发生超时时会输出如下信号：

- wdt_irq: 中断信号，可同时发给 SCPU 与 AP
- wdt_rst_n: 系统复位请求信号（该信号作为一个故障信号传输给 FM）

AWDT 超时，会同时在 AP 和 Safety 域产生中断

- 1) 在 AP 域产生 FIQ 中断由 ATF 处理，并最终通知到 Kernel dump 系统超时相关的 debug 信息。
- 2) 在 Safety 域产生 IRQ 中断由 Safety 系统处理，Safety 系统在等待 AP 完成 debug 信息 dump 后，最终执行复位 AP 操作。。

所以 AWDT 超时会导致整个 AP 域的系统重启，safety 域不重启。

AWDT 默认超时时间为 30 秒，watchdog 线程每 10 秒 kick 一次。

AWDT 的初始化在 AP 侧，代码路径如下：

source/kernel/common/driver/watchdog/autochips/wdt8025/atc_wdt.c

6.4.1.3.2 Safety watchdog(SWDT)

SWDT 用于监控 Safety 域是否挂死或者陷入死循环。

SWDT 模块的整体架构如下图所示：

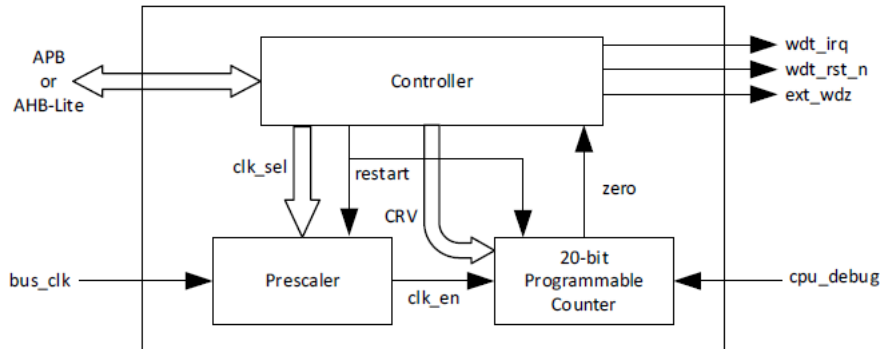


图 6-8 SWDT 硬件框图

SWDT 位于 Safety Subsystem 中，用于监控 SCPU 是否挂死或者陷入死循环。SCPU 需要在预先设定好的时间内对其进行喂狗操作，否则就会导致 SWDT 超时。发生超时可以输出如下信号：

- wdt_irq: 中断信号, 仅发给 SCPU
- wdt_rst_n: 系统复位请求信号（当前软件配置没有产生此信号，而是在中断处理中执行复位操作）
- ext_wdz: 通过 GPIO 输出电平信号(暂时未使用)

当前软件实现中，SWDT 超时仅发送中断信号到 SCPU，SCPU 在中断处理函数中收集相关 debug 信息后重启整个系统。

SWDT 默认超时时间为 5 秒，watchdog 线程每 2 秒 kick 一次。

SWDT 的初始化在 Safety 域，代码路径如下：

source/vendor/autochips/proprietary/tinysys/hsm/os/driver/wdt/atc_wdt_ac8025.c

6.4.1.3.3 Watchdog 错误说明

AWDT、SWDT 超时都会在 Safety 域产生中断（AWDT 还会在 AP 域产生中断，由 ATF 处理，最终路由到 kernel，用于收集 AP 域异常 debug 信息），由 Safety 域在中断中执行最终的 Action。

Watchdog 超时属于系统级别错误，会导致系统重启（AWDT 超时重启 AP，SWDT 超时重启整个系统），重启前会在串口和 safety 的 log 文件中产生相应 error 级别的 log 输出。

AWDT 超时相关 log:

```
[E][WDT] File:atc_wdt_ac8025.c FUNC:awdt_irq_handler Line:306: ++++++ AP
Watchdog timeout 71507017[ms]+++++
```

AWDT 超时中断处理函数: void awdt_irq_handler(unsigned int irq, void *arg);

SWDT 超时相关 log:

```
[E][WDT] File:atc_wdt_ac8025.c FUNC:wdt_irq_handler Line:294: ++++++ Safety
Watchdog timeout 71206013[ms]+++++
```


SWDT 超时中断处理函数: void wdt_irq_handler(unsigned int irq, void *arg)。

6.4.1.4 Fault Monitor

Fault Monitor(FM), 负责整机硬件异常检测及恢复。Safety OS 的 Fault Monitor 框架配合硬件 FM 模块, 用于监测内部及外部硬件模块是否有异常, 有异常时提供相应的容错处理机制。

6.4.1.4.1 FM Introduction

FM 模块的整体架构如下图所示:

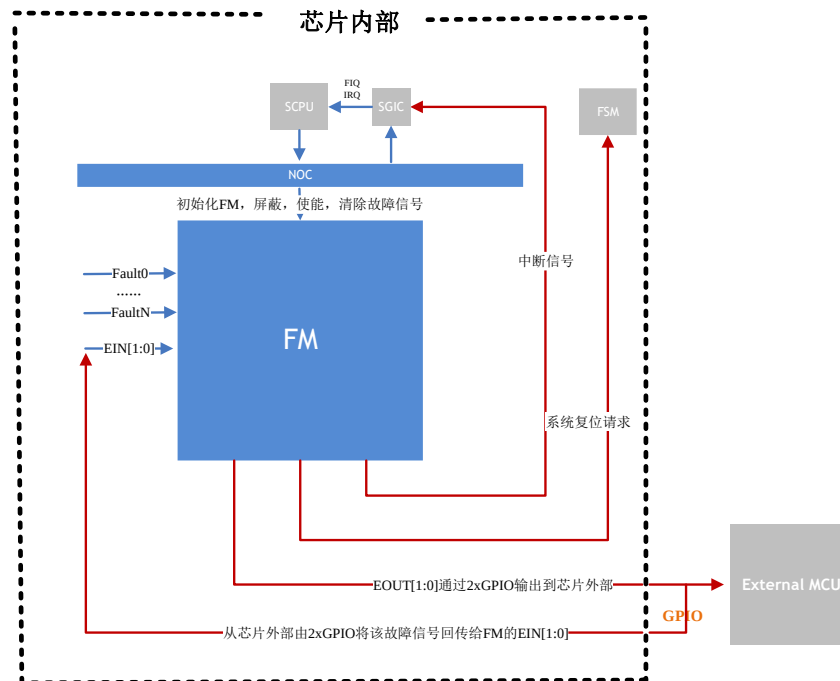


图 6-9 Fault Monitor 框图

FM 模块主要用于对整个系统中的故障信号进行汇总收集, 并可通过软件配置在收到故障信号后触发系统复位, 触发 SCPU (R5F) 中断, 或者通过 GPIO 将故障信号输出到芯片外部。上图中各缩写名称所代表的内容说明如下:

- NOC(Network on chip): 此处代表 8025 中总线(由 SNOC, MNOC, PNOC, HNOC 组成)
- FM(Fault Monitor): 即 FCCU(Fault Collection and Control Unit)模块
- FSM(Finite State Machine): 该状态机在启动阶段起作用, 用于控制各模块的复位时序以及系统复位
- SCPU: Safety CPU, 即双核 R5F

AC8025 FM 模块包含如下四个外部 PIN，可以软件配置其通过任意一个 Safety GPIO 输出到芯片外部。

- EOUT_0 (Output pin): 当 FM 模块监听到故障信号时，通过该管脚通知外部 MCU
- EOUT_1(Output pin): 当 FM 模块监听到故障信号时，通过该管脚通知外部 MCU
- EIN_0(Input pin): 用于将输出到芯片外部的 EOUT_0 信号回传到 FM 模块，以验证 FM EOUT_0 通路是否可以正常工作
- EIN_1(Input pin): 用于将输出到芯片外部的 EOUT_1 信号回传到 FM 模块，以验证 FM EOUT_1 通路是否可以正常工作

说明：当前软件没有实现 Safety GPIO 输出功能，但提供了相关功能配置接口。

FM 模块最多支持 16 个 32-bit 的 fault 源，每个 bit 代表一种 fault（注意：此 fault 并非翻译的“错误”，而是更广泛意义上的硬件“状态报告”，“状态报告”的类型，可能为 warning、可能为 error、也可能是正常的状态上报），AC8025 仅实现了其中的 11 个 fault 源。当 fault 发生时：

- 其对应 fault 寄存器会填充相关信息
- 如果对应的 fault 没有屏蔽的话，则 FM 模块会触发中断给 SCPU
- SCPU 的中断处理函数先读取 fault 寄存器以确定 fault 源，然后清除 active 的 fault
- 如果该 fault 在处理的过程中再次发生，它会被再次 active
- 注意 fault 是持续性的（sticky），即若错误的源头不清掉的话，FM 会一直报中断

FM 保留了 1 个 32bit fault 源给 safety 域，其余的 15 个 32bit fault 源（共 480 个）供外部使用。FM 模块收集 fault 信号，然后对这些 fault signal 集中处理。

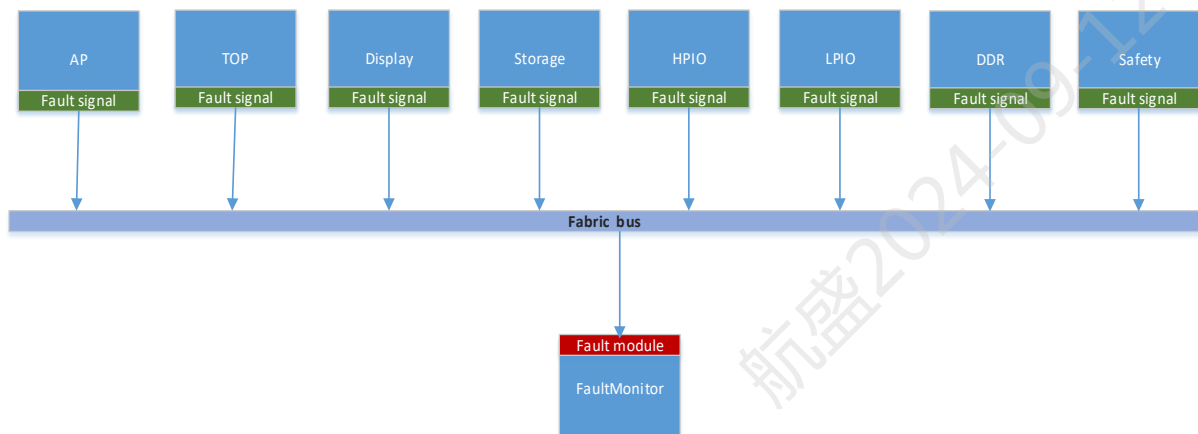


图 6-10 Hardware Fault 框图

6.4.1.4.2 Fault Signal State Machine

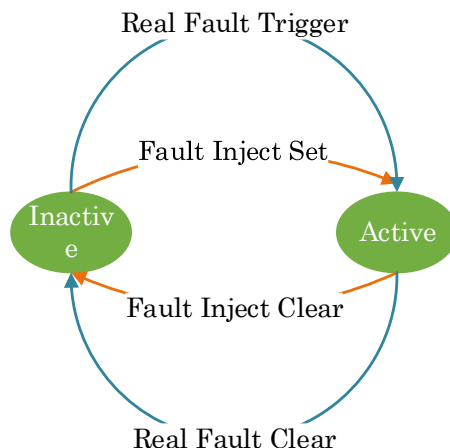


图 6-11 Fault Signal State Machine

每个故障信号的默认状态为 Inactive 状态，当该故障信号实际发生且被 FM 模块采样到，或者当其所对应的 Fault Inject 寄存器被设为 1 时，该故障信号的状态机由 Inactive 状态转化为 Active 状态；当该故障信号故障源被清除，并且其对应的 Fault Inject 寄存器被设为 0 时，该故障信号的状态机由 Active 状态转化为 Inactive 状态。

6.4.1.4.3 Fault 处理流程

软件按如下流程处理 Fault:

1. 各 fault 源根据需要向 FM 软件框架注册对应各个 fault 的处理函数。
2. FM 模块采集 Safety 内部和外部 subsystem 的 fault 信号，如果有 fault 产生，相应 fault 源对应该 fault 的 bit 会置位，同时产生 FM 中断。
3. Safety OS 收到 FM 的中断，在中断处理函数中解析 fault 源和对应的 fault id，生成一个 fault 消息插入到 fault 消息队列 xFaultQueue，同时清除该 fault 源对应 fault 的 bit 置位状态。
4. Fault 处理任务从 fault 消息队列 xFaultQueue 读取 fault 消息，根据 fault 源和 fault id 回调相应的处理函数。
5. 各个 fault 的处理函数根据该 fault 对系统的影响，做相应的处理（各 fault 默认的动作见 6.4.1.4.4）。

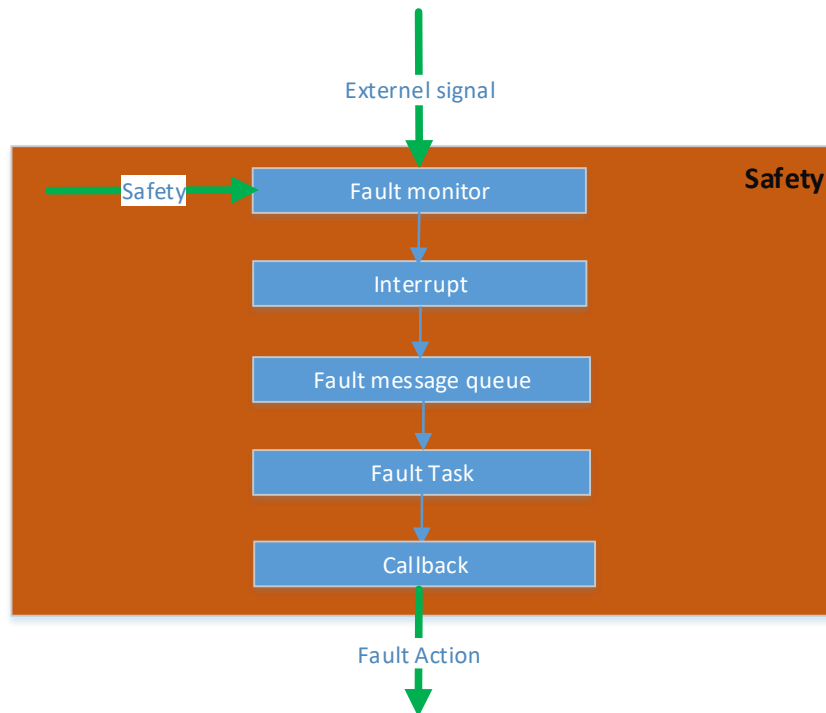


图 6-12 Fault 信号处理流程

- Fault Monitor 收到 fault signal 最终执行其注册的 callback，执行 fault action。

6.4.1.4.4 Fault Description And Action

AC8025 设计支持的 fault 如下表所示，通过对每个 fault（硬件状态通知）产生后对系统的影响评估，各个 fault 产生后的 Action 见 Action 列。

表中的部分 fault 可能与 IP 的硬件中断存在重合，如果在对应 IP 模块的驱动中已经对异常中断中进行处理，则 Fault Monitor 模块可以忽略对应的 fault。

还有一部分 fault 可能仅仅是硬件正常的状态改变通知，这类 fault 不会对系统产生任何影响，也可以忽略掉。

所以，软件上针对各个 fault 的 Action 主要分为以下几类：

- 1、严重级别硬件异常 fault：记录错误信息到 log，然后重启系统。
- 2、一般级别硬件异常 fault：记录错误信息到 log（当前暂未实现模块级别的复位）
- 3、Low 级别的 fault，不会对系统产生影响：忽略掉该 fault
- 4、部分与 IP 模块中断有重合的 fault，并且 IP 模块驱动已经处理相应的中断，在 FM 模块也做忽略处理。

表 6-3 fault 描述

Subsystem ID	Subsystem Name	Bits	Fault Name	Comment	Action
0	LPIO	[31:15]	Reserved	Reserved	
		[14:14]	wdt_fault	watchdog resetn	忽略（已在 watchdog 中断中 reset 系统处理）
		[13:13]	Reserved	Reserved	
		[12:12]	mmc_boot_done	emmc boot done flag	忽略
		[11:9]	lgc_dmac_fault	dmac_fault[0]: ecc_fault[0] dmac_fault[1]: ecc_fault[1] dmac_fault[2]: security_violation	记录错误信息到 log，清除 fault
		[8:8]	ipc_err_irq	ipc error interrupt	记录错误信息到 log，清除 fault
		[7:4]	gmac_fault	gmac_fault[0]: rxram_ecc_sts[0] gmac_fault[1]: rxram_ecc_sts[1] gmac_fault[2]: txram_ecc_sts[0] gmac_fault[3]: txram_ecc_sts[1]	忽略
		[3:1]	acru_pll_lock_loss	acru pll lock lost error	记录错误信息 log 并 reset 系统
		[0:0]	pnoc_irq	pnoc error, such as decode error	记录错误信息后，清除 fault
1	HPIO	[31:4]	Reserved	Reserved	
		[3:3]	hnoc_irq_err	hnoc error, such as decode error	记录错误信息到 log，清除 fault
		[2:2]	irq_pcie_correctable_error	PCIe correctable error	忽略
		[1:1]	irq_pcie_non_fatal_error	PCIe non-fatal error	忽略
		[0:0]	Reserved	Reserved	
2	AP Group 0	[31:29]	Reserved	Reserved	
		[28:28]	apu_axim_fault_interrupt	GIC fault interrupt	记录错误信息 log 并 reset 系统
		[27:27]	apu_axim_err_int	GIC error interrupt	记录错误信息 log 并 reset 系统
		[26:26]	apu_axim_nclust_erpμirq	AP pmu interrupt	忽略
		[25:17]	apu_axim_nfaultirq	AP fault handling interrupt	忽略
		[16:8]	apu_axim_nerrir	AP error handling interrupt	忽略

Subsystem ID	Subsystem Name	Bits	Fault Name	Comment	Action
3	AP Group 1		q		
		[7:0]	apu_axim_dbg_rst_req	AP debug reset request	忽略
		[31:8]	Reserved	Reserved	
4	Display Group 0	[7:0]	apu_axim_wake_request	AP wake up request	忽略
		[31:10]	Reserved	Reserved	
		[9:4]	lvds0_fault	LVDS fault[0]: correctable ECC error LVDS fault[1]: uncorrectable ECC error LVDS fault[2]: encode error LVDS fault[3]: PHY bist fail LVDS fault[4]: PHY disconnect fail LVDS fault[5]: PLL lock fail	忽略（在 IP 中断中处理，无需 FM 处理）
		[3:2]	adpu1_crc_int	CRC error, such as crc frozen or crc mismatch	忽略（在 IP 中断中处理，无需 FM 处理）
		[1:0]	adpu0_crc_int	CRC error, such as crc frozen or crc mismatch	忽略（在 IP 中断中处理，无需 FM 处理）
5	Display Group 1	[31:0]	crc0_fault	LVDS CRC error detected by controller	忽略（在 IP 中断中处理，无需 FM 处理）
6	DDR	[31:12]	Reserved	Reserved	
		[11:11]	dmc1_ctrl_int	DDR1 controller interrupt	忽略
		[10:10]	dmc1_phy_error_int	DDR1 PHY error detected	记录错误信息到 log，清除 fault
		[9:9]	dmc1_ctrl_glb_err_int_nonfat	DDR1 non fatal error detected by controller	记录错误信息到 log，清除 fault
		[8:8]	dmc1_ctrl_glb_err_int	DDR1 error detected by controller	记录错误信息到 log，清除 fault
		[7:7]	dmc1_ecc_uncorrected_1	DDR1 uncorrectable ECC error	忽略
		[6:6]	dmc1_ecc_uncorrected_0	DDR1 correctable ECC error	忽略
		[5:5]	dmc0_ctrl_int	DDR0 controller interrupt	忽略
		[4:4]	dmc0_phy_error_int	DDR0 PHY error detected	记录错误信息到 log，清除 fault
		[3:3]	dmc0_ctrl_glb_err_int_nonfat	DDR0 non fatal error detected by controller	记录错误信息到 log，清除 fault
		[2:2]	dmc0_ctrl_glb_err_int	DDR0 error detected by controller	记录错误信息到 log，清除 fault
		[1:1]	dmc0_ecc_uncorr	DDR0 uncorrectable ECC error	忽略

Subsystem ID	Subsystem Name	Bits	Fault Name	Comment	Action
7	HIFI		ected_1		
		[0:0]	dmc0_ecc_uncorrected_0	DDR0 correctable ECC error	忽略
		[31:2]	Reserved	Reserved	
		[1:1]	irq_hifi1	HIFI DSP1 error	记录错误信息到 log，清除 fault
8	Top Group 0	[0:0]	irq_hifi0	HIFI DSP0 error	记录错误信息到 log，清除 fault
		[31:31]	mnoc_g3d1_latenint	mnoc_g3d1_latenint	忽略
		[30:30]	mnoc_g3d0_missionintbar	mnoc_g3d0_missionintbar	忽略
		[29:29]	mnoc_g3d0_missionint	mnoc_g3d0_missionint	忽略
		[28:28]	mnoc_g3d0_latenintbar	mnoc_g3d0_latenintbar	忽略
		[27:27]	mnoc_g3d0_latenint	mnoc_g3d0_latenint	忽略
		[26:26]	mnoc_main_missionintbar	mnoc_main_missionintbar	记录错误信息 log 并 reset 系统
		[25:25]	mnoc_main_missionint	mnoc_main_missionint	记录错误信息 log 并 reset 系统
		[24:24]	mnoc_main_latenintbar	mnoc_main_latenintbar	记录错误信息 log 并 reset 系统
		[23:23]	mnoc_main_latenint	mnoc_main_latenint	记录错误信息 log 并 reset 系统
		[22:22]	ocm_write_err_status	OCM write uncorrectable ecc error	记录错误信息 log 并 reset 系统
		[21:21]	ocm_write_cor_status	OCM write correctable ecc error	忽略
		[20:20]	ocm_read_err_status	OCM read uncorrectable ecc error	记录错误信息 log 并 reset 系统
		[19:19]	ocm_read_cor_status	OCM read correctable ecc error	忽略
		[18:18]	fw_ocm_intr	OCM firewalls interrupt	记录错误信息到 log，清除 fault
		[17:17]	fw_ddr_intr	DDR firewalls interrupt	记录错误信息到 log，清除 fault
		[16:16]	noc_irq_err	mnoc error, such as decode error	记录错误信息到 log，清除 fault
		[15:10]	Reserved	Reserved	
		[9:0]	mcru_pll_lock_loss	mcru pll lock lost error	记录错误信息 log 并 reset

Subsystem ID	Subsystem Name	Bits	Fault Name	Comment	Action
			t		系统
9	Top Group 1	[31:3]	Reserved	Reserved	
		[2:2]	mnoc_g3d1_missionintbar	mnoc_g3d1_missionintbar	忽略
		[1:1]	mnoc_g3d1_missionint	mnoc_g3d1_missionint	忽略
		[0:0]	mnoc_g3d1_latenintbar	mnoc_g3d1_latenintbar	忽略
10	Safety	[31:23]	Reserved	Reserved	
		[22:22]	snoc_missionintbar	snoc mission fault interrupt negation	记录错误信息 log 并 reset 系统
		[21:21]	snoc_missionint	snoc mission fault interrupt	记录错误信息 log 并 reset 系统
		[20:20]	snoc_latentintbar	snoc latent fault interrupt negation	记录错误信息 log 并 reset 系统
		[19:19]	snoc_latentint	snoc latent fault interrupt	记录错误信息 log 并 reset 系统
		[18:18]	wd_rst_n	watchdog resetn	忽略（已在 watchdog 中断中 reset 系统处理）
		[17:17]	Reserved	Reserved	
		[16:16]	snoc_irq_err	snoc error, such as decode error	记录错误信息到 log，清除 fault
		[15:15]	scru_pll_lock_lost	scru pll lock lost error	记录错误信息 log 并 reset 系统
		[14:14]	dbgrstreq1	R5F_1 debug reset request	忽略
		[13:13]	dbgrstreq0	R5F_0 debug reset request	忽略
		[12:12]	pd_alarmb	PVT-Sensor, Alarm B interrupt	忽略
		[11:11]	pd_alarma	PVT-Sensor, Alarm A interrupt	忽略
		[10:10]	ts_alarmb	ATC T-Sensor, Alarm B interrupt	忽略
		[9:9]	ts_alarma	ATC T-Sensor, Alarm A interrupt	忽略
		[8:7]	sram_ecc_status(ospir)	sram_ecc_status[0]: correctable ecc error sram_ecc_status[1]: uncorrectable ecc error	忽略
		[6:5]	Reserved	Reserved	忽略
		[4:4]	efuse_dbit_ecc_fault	eFuse 2bit uncorrectable ecc error	记录错误信息 log 并 reset 系统
		[3:3]	efuse_sbit_ecc_fa	eFuse 1bit correctable ecc error	忽略

Subsystem ID	Subsystem Name	Bits	Fault Name	Comment	Action
			ult		
		[2:0]	dmac_fault	dmac_fault[0]: ecc_fault[0] dmac_fault[1]: ecc_fault[1] dmac_fault[2]: security_violation	记录错误信息到 log，清除 fault

6.4.1.4.5 FM 注入与 log 说明

AC8025 支持 FM 故障注入，以验证 FM 是否可正常工作。

为了方便 Fault monitor 的注入测试，在 Android 域的 linux shell 下提供 /proc/aed/generate-fault 节点用于验证 fault 注入测试。具体命令格式如下：

```
echo [subsystem-id] [fault-bit] > /proc/aed/generate-fault
```

其中：

- subsystem-id 参考 6.4.1.4.4 表格中的 Subsystem ID，取值范围为[0-10]
- fault-bit 的取值范围为 [0-31], 对应每个 subsystem 的 32 个 bit。

例如，注入 LPIO 子系统(subsystem-id 为 0)的 ipc_err_irq (bit 8)类型的 fault，

```
echo 0 8 > /proc/aed/generate-fault
```

Fault 注入后，在串口下会产生如下输出：

```
[0][W][FaultMonitor] [Fulat Monitor]fault_src:(LPIO subsystem) fault_id:(IPC_PAC_ERR) fault happen 1 times.
```

主要字段解释如下：

- Fulat Monitor: 说明是 FM 模块的 log
- fault_src:(LPIO subsystem): 产生该 fault 的 subsystem 为 LPIO
- fault_id:(IPC_PAC_ERR): 具体到 subsystem 下的 fault 类型为 IPC_PAC_ERR
- fault happen 1 times: 该 fault 产生的次数。

在 safety 系统的 log 中会记录如下 log：

```
[D][FaultMonitor] Inject Fault happened: fsr(0) status(100) fault_id(8)
```

```
[D][FaultMonitor] read fault from queue fault_src:0, fault_id:8
```

```
[W][FaultMonitor] [Fulat Monitor]fault_src:(LPIO subsystem) fault_id:(IPC_PAC_ERR) fault happen 1 times.
```

6.4.1.4.6 FM 接口说明

FM 模块涉及相关接口如下表，具体定义在：

source/vendor/autochips/proprietary/tinysys/hsm/os/include /fm_driver.h

source/vendor/autochips/proprietary/tinysys/hsm/os/libsrc/include/ac8025/fm_hal_if.h

Fault monitor 相关实现代码路径：

source/vendor/autochips/proprietary/tinysys/hsm/os/driver/fm/fm_drv_8025.c

表 6-4 FM 接口说明

API	说明
fm_intr_handler_reg	基于 Subsystem 注册整个 fault 源处理函数
fm_intr_handler_reg_by_faultid	基于 Subsystem +Sub faultid 注册 fault 处理函数
fm_fault_source_intr_mask	基于 Subsystem enable 整个 fault 源的 fault 上报
fm_fault_source_intr_unmask	基于 Subsystem disable 整个 fault 源的 fault 上报
fm_fault_intr_enable	基于 Subsystem+Sub faultid 使能 fault 上报
fm_fault_intr_disable	基于 Subsystem+Sub faultid 去使能 fault 上报
fm_fault_inject	基于 Subsystem+Sub faulted 注入 fault
fm_fault_eout_enable	基于 Subsystem+Sub faulted 使能 fault Eout 功能
fm_fault_eout_disable	基于 Subsystem+Sub faulted 去使能 fault Eout 功能

6.4.1.4.6.1 fm_intr_handler_reg

【语法】

```
int fm_intr_handler_reg(int fault_src, fm_error_handler fun)
```

【描述】

基于 Subsystem 注册整个 fault 源处理函数

【参数】

- fault_src: fault source id, 取值范围参考 fm_hal_if.h enum FMSubsystemID 定义
- fun : fault 处理函数

【返回值】

- 0：成功
- 非 0：失败

6.4.1.4.6.2 fm_intr_handler_reg_by_faultid

【语法】

```
int fm_intr_handler_reg_by_faultid (int fault_src, int fault_id, fm_error_handler fun)
```

【描述】

基于 Subsystem +Sub faultid 注册 fault 处理函数

【参数】

- fault_src: fault source id, 取值范围参考 enum FMSubsystemID 定义
- fault_id: fault srouce 内部的 fault id, 取值 范围[0 – 31], 参考各个 subsystem 的 fault 枚举。
- fun: fault 处理函数

【返回值】

- 0: 成功
- 非 0: 失败

6.4.1.4.6.3 fm_fault_source_intr_mask

【语法】

```
int fm_fault_source_intr_mask (int fault_src)
```

【描述】

基于 Subsystem enable 整个 fault 源的 fault 上报

【参数】

fault_src: fault source id, 取值范围参考 fm_hal_if.h enum FMSubsystemID 定义

【返回值】

- 0: 成功
- 非 0: 失败

6.4.1.4.6.4 fm_fault_source_intr_unmask

【语法】

```
int fm_fault_source_intr_unmask (int fault_src)
```

【描述】

基于 Subsystem disable 整个 fault 源的 fault 上报

【参数】

fault_src: fault source id, 取值范围参考 fm_hal_if.h enum FMSubsystemID 定义

【返回值】

- 0：成功
- 非 0：失败

6.4.1.4.6.5 fm_fault_intr_enable

【语法】

```
int fm_fault_intr_enable (int fault_src, int fault_id)
```

【描述】

基于 Subsystem+Sub faultid 使能 fault 上报

【参数】

- fault_src: fault source id, 取值范围参考 fm_hal_if.h enum FMSubsystemID 定义
- fault_id : fault srouce 内部的 fault id, 取值 范围[0 – 31], 参考各个 subsystem 的 fault 枚举。

【返回值】

- 0：成功
- 非 0：失败

6.4.1.4.6.6 fm_fault_intr_disable

【语法】

```
int fm_fault_intr_disable (int fault_src, int fault_id)
```

【描述】

基于 Subsystem+Sub faultid 去使能 fault 上报

【参数】

- fault_src: fault source id, 取值范围参考 fm_hal_if.h enum FMSubsystemID 定义
- fault_id : fault srouce 内部的 fault id, 取值 范围[0 – 31], 参考各个 subsystem 的 fault 枚举。

【返回值】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

168 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

- 0：成功
- 非 0：失败

6.4.1.4.6.7 fm_fault_inject

【语法】

```
int fm_fault_inject (int fault_src, int fault_id)
```

【描述】

基于 Subsystem+Sub faulted 注入 fault，用于 FM 测试

【参数】

- fault_src: fault source id, 取值范围参考 fm_hal_if.h enum FMSubsystemID 定义
- fault_id : fault srouce 内部的 fault id, 取值 范围[0 – 31], 参考各个 subsystem 的 fault 枚举。

【返回值】

- 0：成功
- 非 0：失败

6.4.1.4.6.8 fm_fault_eout_enable

【语法】

```
int fm_fault_eout_enable (int fault_src, int fault_id)
```

【描述】

基于 Subsystem+Sub faulted 使能 fault Eout 功能

【参数】

- fault_src: fault source id, 取值范围参考 fm_hal_if.h enum FMSubsystemID 定义
- fault_id : fault srouce 内部的 fault id, 取值 范围[0 – 31], 参考各个 subsystem 的 fault 枚举。

【返回值】

- 0：成功
- 非 0：失败

6.4.1.4.6.9 fm_fault_eout_disable

【语法】

int fm_fault_eout_disable (int fault_src, int fault_id)

【描述】

基于 Subsystem+Sub faulted 去使能 fault Eout 功能

【参数】

- fault_src: fault source id, 取值范围参考 fm_hal_if.h enum FMSubsystemID 定义
- fault_id : fault srouce 内部的 fault id, 取值 范围[0 – 31], 参考各个 subsystem 的 fault 枚举。

【返回值】

- 0 : 成功
- 非 0: 失败

6.4.2 Safety Display

ATC 产品形态目前存在 IVI 和 Cockpit 两种形态。

其中 IVI 形态不存在 Safety Display。Cockpit 形态请参考 [27 Safety Display](#)。

6.5 Crypto Service

在 Safety OS 中主要由 crypto_task.c 来提供该项服务。该项服务涉及到下述定义：

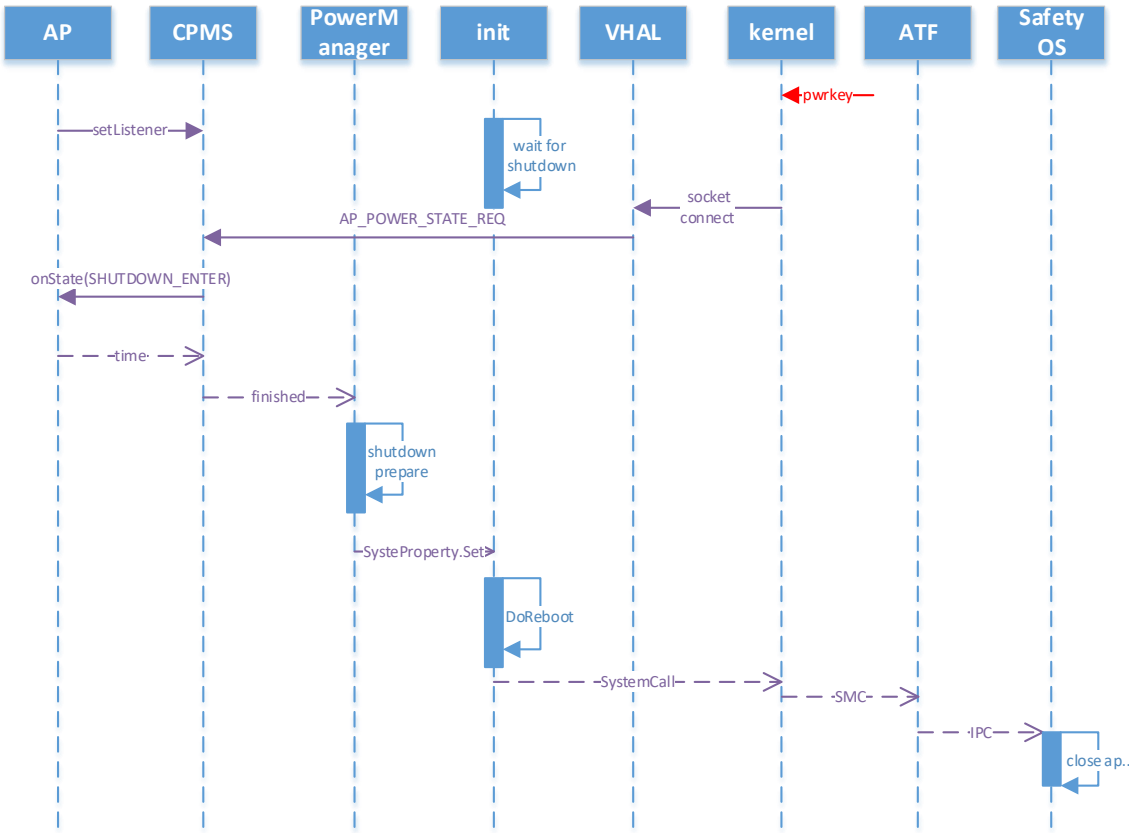
表 6-5 Crypto service

TRNG	用于生成随机数
AES	负责 AES 加解密服务
SHA	负责哈希值计算
EFUSE	读 EFUSE 中的相关属性

6.6 Misc Service

6.6.1 Shutdown and reboot service

ATC 芯片，在系统中进行 shutdown and reboot 请求，具体流程如下。



详情参考 [8 Shutdown](#)。

6.6.2 System Trace service

HSM SYSTEM TRACE 主要是为了监控系统的状态：

- CPU 信息
- Memory 信息
- Task 信息
- Video data 信息
- Interrupt 信息

6.6.2.1 架构图

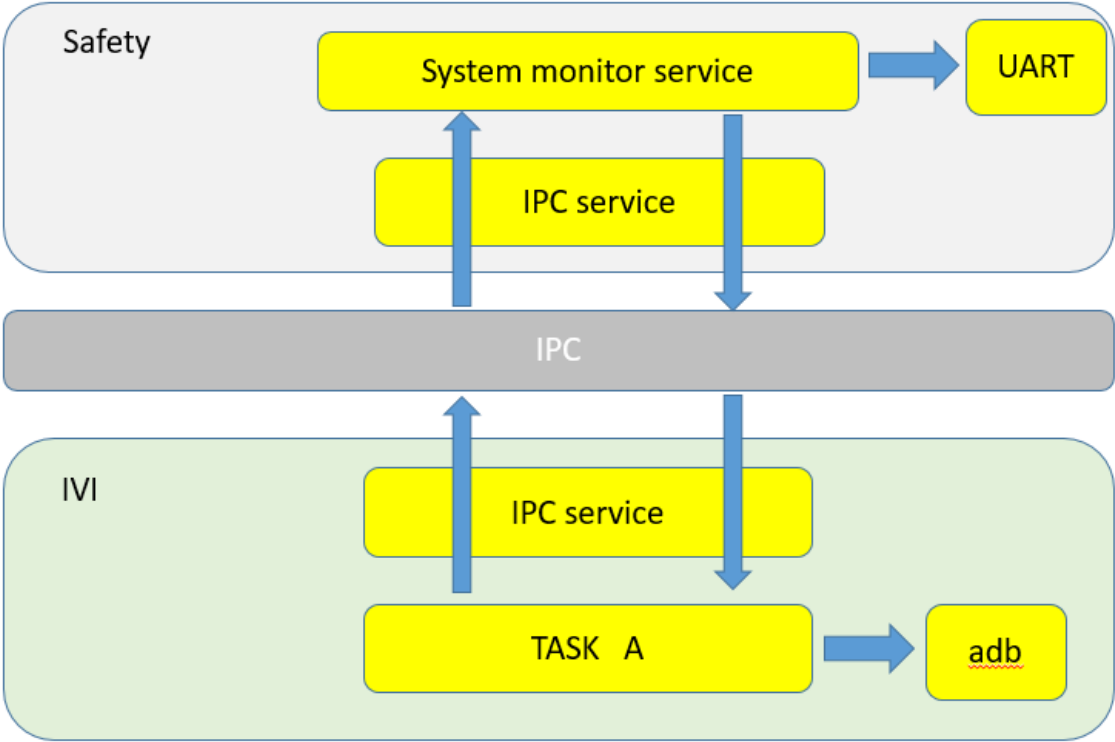


图 6-13 HSM SYSTEM TRACE

6.6.2.2 模块说明

通过 kernel 侧发送请求，然后 HSM 收到请求后，根据请求类型展示对应的任务状态信息。以下是请求类型说明：

通过 ivi 侧 echo “command” > /proc/hsm_trace, 然后 cat /proc/hsm_trace,终端会显示对应的数据信息，具体的展示内容后面章节描述。

Type	Command
CPU info	cpu
Memory info	memory
Task info	task
Video info	video
Interrupte info	int

6.6.2.3 CPU Info

ATC 提供系统启动后，在 kernel 中查看对应的 safety 中对应 cpu 状态

使用 `echo cpu > /proc/hsm_trace` 可以看到相关信息，具体展示项如下：

Type	Description	Instructions
TN	Task_name	HSM task 名称
ACV	Abs Count_Value	运行的 timer_counter 数量
Tmp	Time Percent	占 R5F cpu 百分比
AT	Abs_Times	启动后运行的总时间

说明：日志中包含各项的 UNIT 介绍。

最终展示如下：

```
ac8025_demo:/ # cat /proc/hsm_trace
TN for TASK_NAME, ACV for Abs counter value, TmP for Time percent, AT for Abs timeTime unit is (s), ulStatTimerTicks:(411)Hz
*****show_cpu*****
TN          ACV      TmP    AT
system_communication_Task  0      <1%  0
IDLE        98681     90%  240
1_VISS Audio Playback Task  2252     2%   5
VehicleCallbackTask        473     <1%  1
disp-pv-task3              2250     2%   5
VehicleTask                56     <1%  0
Multi OS Manager Task      52     <1%  0
audio communication task   424     <1%  1
vVehicleGPIONonitorTask(15) 357     <1%  0
vVehicleGPIONonitorTask(14) 357     <1%  0
xOsFreeMemTask             16     <1%  0
WDT                         7      <1%  0
wakeup task                4      <1%  0
Tmr Svc                    378     <1%  0
refuse ready task          1      <1%  0
disp-pv-task0              1669     1%   4
trace_rec                  0      <1%  0
trace_snd                  0      <1%  0
trace_rec                  0      <1%  0
trace_snd                  0      <1%  0
trace_rec                  0      <1%  0
trace_snd                  0      <1%  0
Recv ADAS Task             0      <1%  0
Recv Cluster Task          0      <1%  0
vVehicleSlaveMonitorTask(1) 0      <1%  0
APM_ADAS_IPC               0      <1%  0
APM_CLUSTER_IPC            0      <1%  0
pwrkey task                0      <1%  0
mcu task                   1      <1%  0
shutdown task              1      <1%  0
vVehicleSlaveMonitorTask(2) 1      <1%  0
noc svc task               0      <1%  0
irq_sync task              1      <1%  0
vRVCIPCtrlTask             0      <1%  0
RVCMainTask                23     <1%  0
vVehicleSlaveMonitorTask(0) 3      <1%  0
Recv IVI Task              1      <1%  0
APM_IPC                    4      <1%  0
vdecpowermanager           6      <1%  0
Apm                        2      <1%  0
unittest                   0      <1%  0
hsm_crypto task            20     <1%  0
cpy_refresh1               614     <1%  1
cpy_refresh0               537     <1%  1
monitor pvt task           0      <1%  0
*****
ac8025_demo:/ # cat /proc/hsm_trace
```

如果想要查看 cpu 一段时间内的情况，可以使用提供的脚本 `hsm_cpu.py`（在 hsm 的 tools 目录下面）查看。

可以在 windows,powershell 中运行下面程序

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

173 /1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

Example: python hsm_cpu.py time

Time: 是你希望监控的大致时间

脚本运行后会输出这段时间所有任务的 tick 信息，由于程序的时间会存在部分误差，所以以最终计算出来的总时间为准。

python.exe hsm_cpu.py 20

```

WDT                                0      -      0
wakeup task                        0      -      0
efuse ready task                   0      -      0
vVehicleGPIOMonitorTask(14)       23      -      55
vRVCIPCCtrlTask                   0      -      0
RVCMaintTask                       0      -      0
vVehicleSlaveMonitorTask(0)       0      -      0
Recv_IVI Task                     0      -      0
APM_IPC                           0      -      0
vdecpowermanager                  0      -      0
Apm                               0      -      0
unittest                          0      -      0
monitor pvt task                  0      -      0
trace_rec                         0      -      0
trace_rec                         0      -      0
trace_rec                         0      -      0
trace_snd                         0      -      0
trace_snd                         0      -      0
trace_snd                         0      -      0
trace_rec                         0      -      0
trace_rec                         0      -      0
trace_rec                         0      -      0
trace_snd                         0      -      0
trace_snd                         0      -      0
trace_snd                         0      -      0
trace_rec                         0      -      0
trace_rec                         0      -      0
trace_rec                         0      -      0
trace_snd                         0      -      0
trace_snd                         0      -      0
trace_snd                         0      -      0
Recv ADAS Task                    0      -      0
Recv Cluster Task                 0      -      0
APM_ADAS_IPC                      0      -      0
APM_CLUSTER_IPC                   0      -      0
pwrkey task                       0      -      0
noc svc task                      0      -      0
vVehicleSlaveMonitorTask(2)       0      -      0
irq sync task                     0      -      0
shutdown task                     0      -      0
hsm crypto task                   1      -      2
dpy_refresh1                      50      -     121
dpy_refresh0                      34      -      82
mcu task                          0      -      0
vVehicleSlaveMonitorTask(1)       0      -      0
Total Diff Value 2: 20686 ms
C:\Users\atc0265\Desktop>

```

6.6.2.4 Memory Info

ATC 提供系统启动后，在 kernel 中查看对应的 safety 中对应 memory 使用状态,包含 DRAM and SRAM。

使用 echo memory > /proc/hsm_trace 可以看到相关信息，具体展示项如下：

Type	Description	Instructions
DMemTotal	Memory total size	dram 内存总大小
DMemFree	Memory available	dram 可使用内存
DMemRate	Memory rate	dram 内存占用率情况
SMemTotal	Memory total size	sram 内存总大小
SMemFree	Memory available	sram 可使用内存
SMemRate	Memory rate	sram 内存占用率情况

说明: 日志中包含各项的 UNIT 介绍。

最终展示如下:

```
hsm_prof hsm_trace
ac8025_demo:/ # cat /proc/hsm_trace

*****show_mem*****

DMemTotal:      4194304 B
DMemFree:       3813712 B
DMemRate:       9 %
SMemTotal:      262144 B
SMemFree:       35712 B
SMemRate:       86 %

*****

ac8025_demo:/ #
```

6.6.2.5 TASK Info

ATC 提供系统启动后, 在 kernel 中查看对应的 safety 中各个任务 STACK 使用状态

使用 echo task:xx > /proc/hsm_trace 可以看到相关信息, 具体展示项如下:

xx 可以是 all 代表全部, 也可以是任务名称。

Type	Description	Instructions
TN	Task name	任务名称
PR	Task priority	任务优先级
STSZ	Task stack size	任务栈大小
UNSTSZ	Task used stack size	已用任务栈
RT	Task unused rate	未使用的栈大小百分比

说明： 日志中包含各项的 UNIT 介绍。

最终展示如下：

```
ac8025_demo:/ # echo task:all >/proc/hsm_trace
ac8025_demo:/ # cat /proc/hsm_trace
TN for TASK_NAME, PR for priority, STSZ for task stack size(unit B), UNSTSZ for task unused stack_size, RT for stack use rate
*****show_task*****
TN          PR          STSZ  UNSTSZ  RT
system_communication_Task  1          4000    384    77
IDLE        0          4096    994    75
disp-pw-task0  3        16384   4014    75
audio communication task  1          4000    920    77
VVehicleCPIDMonitorTask(14) 1          8192   1998    75
VVehicleCPIDMonitorTask(15) 1          8192   1998    75
VVISS Audio Playback Task  2          4000    956    76
VehicleCallTask  1          8192   1947    76
VehicleTask  1          8192   1982    75
Multi OS Manager Task  1          4000    940    76
VOSFreeMemTask  2          2048    468    77
Tmr Svc      4          8192   2010    75
WDT          1          4096    980    76
wakeuptask  1          2048    463    77
refuse ready task  1        16000   3958    75
disp-pw-task3  3        16384   4014    75
trace_snd    1          800     146    81
trace_rec    1          800     140    82
trace_snd    1          800     146    81
trace_rec    1          800     140    82
trace_snd    1          800     146    81
Recv ADAS Task  1          4000    944    76
Recv Cluster Task  1          4000    944    76
VVehicleSlaveMonitorTask(1) 1          8192   1948    76
AFM_ADAS_IPC  1          800     124    84
AFM_CLUSTER_IPC  1          800     122    84
pwkey task   1          4096    961    76
mcu task     1          2048    448    78
shutdown task 1          2048    448    78
VVehicleSlaveMonitorTask(2) 1          8192   1948    76
rpc svc task 1          1600    340    78
irq sync task 1          800      88    89
VVCIPCCtrlTask 1          2048    430    79
VVCMainTask  1          4000    934    76
VVehicleSlaveMonitorTask(0) 1          8192   1948    76
Recv IVI Task  1          4000    924    76
AFM_IPC      1          800     130    83
Vvdecpowermanager 2          4096    960    76
Apm          1          4096    966    76
unittest     1          4096    922    77
hsm crypto task 1        16000   3748    76
key_refresh1 3          8192   1990    75
key_refresh0 3          8192   1990    75
monitor pvt task 1        8000    1936    75
trace_rec    1          800     140    82
*****
ac8025_demo:/ #
ac8025_demo:/ #
```

6.6.2.6 Video Data Info

ATC 提供系统启动后，在 kernel 中查看对应的 safety 中对应 RAM 使用状态

使用 echo video > /proc/hsm_trace 可以看到相关信息，具体展示项如下：

Type	Description	Instructions
MOD	Module name	模块名称
ADDR	Video buffer addr	视频缓冲地址
SIZE	Video buffer size	视频缓冲大小
TYPE	Video buffer type	视频缓冲类型（eg:queue）
NUM	Video buffer num	申请的数量

说明： 日志中包含各项的 UNIT 介绍。

最终展示如下：

```
(231205_09:50:01.613)*****show_video*****
(231205_09:50:01.613)DMemTotal:    100664320 B
(231205_09:50:01.613)DMemFree:     76698540 B
(231205_09:50:01.613)MOD          TYPE      ADDR          SIZE  NUM
(231205_09:50:01.613)rvs          rsv       0x89116600        921600  1
(231205_09:50:01.613)rvs          rsv       0x89296600        921600  1
(231205_09:50:01.613)rvs          rsv       0x89416600        921600  1
(231205_09:50:01.613)rvs          rsv       0x89596600        921600  1
(231205_09:50:01.613)rvs          rsv       0x89716600        921600  1
(231205_09:50:01.613)rvs          rsv       0x89896600        921600  1
(231205_09:50:01.613)rvs          rsv       0x89a16600       2457600  1
(231205_09:50:01.613)rvs          rsv       0x89d16600       2457600  1
(231205_09:50:01.613)rvs          rsv       0x8a016600       2457600  1
(231205_09:50:01.613)rvs          rsv       0x8a316600       2457600  1
(231205_09:50:01.613)rvs          rsv       0x8a616600       2457600  1
(231205_09:50:01.613)rvs          rsv       0x8a916600       2457600  1
(231205_09:50:01.613)vba          rsv       0x88516600        921600  1
(231205_09:50:01.613)vba          rsv       0x88816600        921600  1
(231205_09:50:01.613)vba          rsv       0x88996600        921600  1
(231205_09:50:01.613)vba          rsv       0x88b16600        921600  1
(231205_09:50:01.613)misc          rsv       0x88696600         4180  1
(231205_09:50:01.613)*****
```

6.6.2.7 Interrupt Info

ATC 提供系统启动后，在 kernel 中查看对应的 safety 中对应 RAM 使用状态。

使用 `echo int:xx > /proc/hsm_trace` 可以看到相关信息，具体展示项如下：

xx 可以是 all 代表全部，也可以是中断号看某一个中断信息。

Type	Description	Instructions
mod_name	Int module	中断所属模块
ID	Int number	中断号
FREQ	Int count	中断发生次数
PROR	Int priority	中断优先级
NA	Int time	中断处理时间
MT	Int max time	中断最大处理时间
NA	Int last time	中断上次触发时间(以 os 启动时间算)
IT	Int min interval	中断最小间隔

其中 mod_name 在代码中需要各个模块包含 system_trace.h，然后按照下图调用注册。

```
system_int_register_t reg_int = {0};
strncpy(reg_int.mod_name, "IPC", 3);
system_trace_register_int(irq, &reg_int);
```

说明：日志中包含各项的 UNIT 介绍。

最终展示如下：

```
ac8025_demo:/ # echo int:all >/proc/hsm_trace
ac8025_demo:/ # cat /proc/hsm_trace
ID for interrupt ID, PROR for priority, FREQ(hex) for occur frequency, MT(hex) for once MAX time expend(us), IT() for min interval of secondary occurrence(us)

*****show_int*****

ID   PROR  FREQ    MT   IT      mod_name
33   0x00  0        0    0
44   0x00  0        0    0
46   0x00  0        0    0
48   0x00  4b881    15   16
107  0x00  1        a    5cf96f
109  0x00  51       12   3aa      IPC
111  0x00  0        0    0        IPC
112  0x00  9        f    12      IPC
113  0x00  0        0    0        IPC
114  0x00  0        0    0        IPC
142  0x00  0        0    0
269  0x00  4990     170   38a3
275  0x00  4990     1a2   38a2
281  0x00  0        0    0
219  0x00  0        0    0
320  0x00  0        0    0
334  0x00  0        0    0
335  0x00  0        0    0

*****
ac8025_demo:/ # echo int:1 >/proc/hsm_trace
```

6.7 Memory

AC8025 上 Safety 模块使用 Cortex R5F，属于 32 位架构，无法直接访问 64 位地址，但是 HSM 访问 DRAM 的物理基地址是 0x8_0000_0000，因此必须添加地址转换工具，AC8025 使用 Address Translator（后续简称 AT）作为地址转换器。

6.7.1 分布图

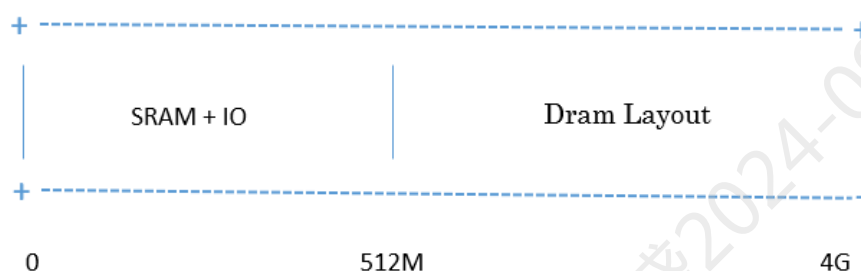


图 6-14 Memory layout

由于 AT IP 支持的源地址的配置范围为 0x0000_0000~0xFFFF_FFFF 即 4GB，在 AC8025 中由于 0x0000_0000~0x1FFF_FFFF 地址空间内分布这 ROM, TCM SRAM, OCM, QSPI Data Space 以及外设寄存器等，因此 0x0000_0000~0x1FFF_FFFF 无法用作源地址。请使用 0x2000_0000~0xFFFF_FFFF 地址范围内的值作为源地址的配置参数。

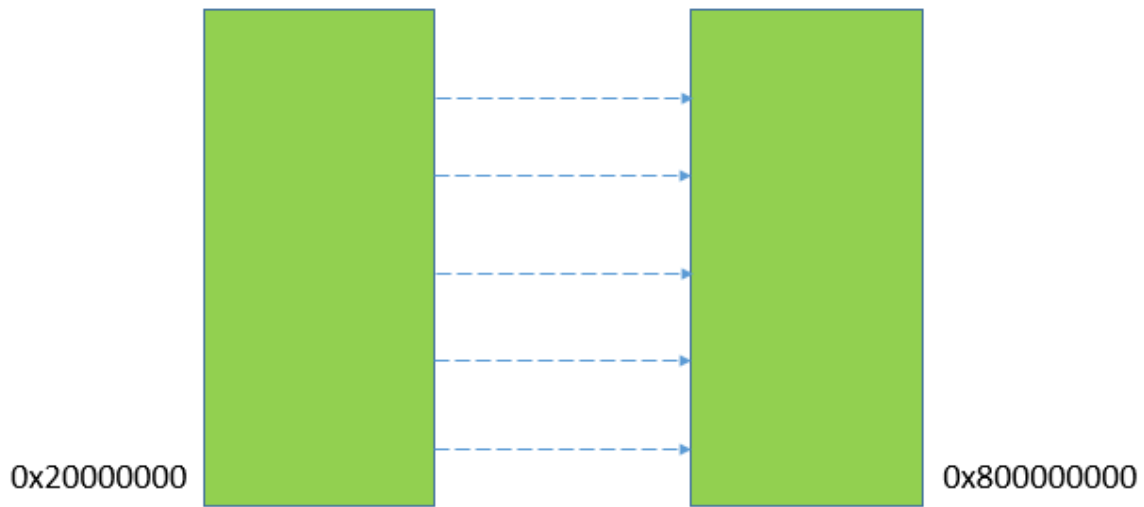


图 6-15 AT 映射图

6.7.2 系统内存

6.7.2.1 运行内存

HSM 使用的内存大小是在 RSV memory 中配置的。文件路径为

`$(PROJECT_ROOT)/build/projects/$(PROJECT_NAME)/ac8x_reserved_memory.csv`

其中配置 HSM RSV 截图如下：

```
1 Index,Label,Node Name,Size(B),Size(KB or MB),Size_HEX(B),Compatible,Mapped,Static,alignment (B),alloc-ranges start,al
2 1,aah_reserved,aah,16777216,16 MB,0x1000000,""autochips,aah""",no-map,yes,0,0x0,0x0,
3 2,trustzone_reserved,trustzone,13631488,13 MB,0xD000000,""autochips,tzram""",no-map,yes,0,0x0,0x0,
4 3,entropy_reserved,entropy,3145728,3 MB,0x3000000,""autochips,entropy""",no-map,yes,0,0x0,0x0,
5 4,hsm_reserved,hsm,10485760,10 MB,0xA000000,""autochips,hsm""",no-map,yes,0,0x0,0x0,
6 5,tk_reserved,tk,16777216,16 MB,0x1000000,""autochips,tk""",no-map,yes,0,0x0,0x0,
7 6,linux_cma_reserved,"linux,cma",104857600,100 MB,0x6400000,""shared-dma-pool""",no,0,0x0,0x0,
8 7,smmu_table_reserved,smmu_table,65536,64 KB,0x10000,""autochips,smmu_table""",no-map,yes,0,0x0,0x0,
9 8,ipc_reserved_memory,ipc,65536,64 KB,0x10000,""autochips,ipc-reserved-memory""",map,yes,0,0x0,0x0,
```

不建议修改该文件的内存大小，如果非常需要可以根据这里修改。

6.7.2.2 HEAP

HSM 定义了 FreeRTOSConfig.h 中定义了 HEAP 的大小，可以适当修改。

```
4 #define configMINIMAL_STACK_SIZE ( ( unsigned short ) 512 )
5 #define configTOTAL_HEAP_SIZE ( 1024 * 1024 )
6 #define configUSE_APPLICATION_TASK_TAG 0
```

6.8 IPC

请参考 5 IPC。

6.9 接口说明

6.9.1 中断处理

在中断处理函数里面不要调用 freertos API 中不带 fromISR 接口的函数。

主要函数如下。

表 6-6 中断函数列表

函数原型	描述
int request_irq(unsigned int irq, gic_fn_ptr_t handle, void *arg)	申请中断且注册中断处理函数
void free_irq(unsigned int irq)	注销中断号
void gic_irq_config(unsigned int irq, unsigned int group,unsigned int cpu_target_mask, unsigned int trigger_type, gic_fn_ptr_t handle, void *arg)	详细配置中断及对应的中断号，触发类型等

request_irq 和 free_irq 使用方法和通用用法相同，这里特此说明下 gic_irq_config 的具体用法。

【描述】

中断配置函数。

【语法】

```
void gic_irq_config(
    unsigned int irq,
    unsigned int group,
    unsigned int cpu_target_mask,
    unsigned int trigger_type,
    gic_fn_ptr_t handle,
    void *arg)
```

【参数】

- irq: 中断号
- group: GIC group
- cpu_target_mask: CPU mask

- `trigger_type`: 触发类型
- `handler`: 中断处理函数
- `arg`: 传入的参数

【返回值】

无。

6.9.2 Task

FreeRTOS 支持两种创建 Task 的方式，动态创建和静态创建。

Task 在创建时，需要给一个指定的优先级。创建时，请使用 `tskIDLE_PRIORITY + 1`。

非必须情况下，请不要修改优先级。

6.9.2.1 动态创建

【描述】

动态创建 FreeRTOS 任务进程。

【语法】

```
BaseType_t xTaskCreate (
    TaskFunction_t pxTaskCode,
    const char * const pcName,
    const configSTACK_DEPTH_TYPE usStackDepth,
    void * const pvParameters,
    UBaseType_t uxPriority,
    TaskHandle_t *pxCreateTask
);
```

【参数】

- `pxTaskCode`: 任务函数指针，指向任务的入口函数，即任务将要执行的代码。
- `pcName`: 任务的名称，用于调试和识别任务。
- `usStackDepth`: 任务堆栈的深度（以字为单位），表示任务在执行时可以使用的堆栈空间大小。
- `pvParameters`: 任务参数的指针，可以传递给任务函数。
- `uxPriority`: 任务的优先级，值越高，优先级越高。

- `pxCreatedTask`: 指向任务句柄（Task Handle）的指针，用于获取创建的任务的句柄。

【返回值】

- `pdPASS`: 表明 task 创建成功
- `errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY`: 创建失败，且内存不足

【注意】

`configSUPPPORT_DYNAMIC_ALLOCATION` 必须打开。

6.9.2.1.1 示例

```
// 动态任务的堆栈大小
#define TASK_STACK_SIZE 100

// 任务函数
void vTaskFunction(void *pvParameters)
{
    // 任务的具体代码
    // ...
}

// 在 FreeRTOS 中创建动态任务的示例代码
void createDynamicTask()
{
    // 创建动态任务
    TaskHandle_t taskHandle;
    BaseType_t taskCreated = xTaskCreate(
        vTaskFunction,           // 任务函数
        "DynamicTask",           // 任务名称
        TASK_STACK_SIZE,         // 堆栈深度
        NULL,                     // 任务参数
        configMAX_PRIORITIES - 1, // 优先级
    );
}
```

```

        &taskHandle           // 任务句柄

    );

    if (taskCreated == pdPASS)
    {
        // 任务创建成功
        // ...
    }
    else
    {
        // 任务创建失败
        // ...
    }
}

```

6.9.2.2 静态创建

【描述】

创建 FreeRTOS 任务进程且使用的 RAM 由程序开发者来指定。

【语法】

```

BaseType_t xTaskCreateStatic (
    TaskFunction_t pxTaskCode,
    const char * const pcName,
    const uint32_t ulStackDepth,
    void * const pvParameters,
    UBaseType_t uxPriority,
    StackType_t* const puxStackBuffer,
    StackTask_t * const pxTaskBuffer
);

```

【参数】

- **pxTaskCode:** 任务函数指针，指向任务的入口函数，即任务将要执行的代码。

- **pcName:** 任务的名称，用于调试和识别任务。
- **ulStackDepth:** 任务堆栈的深度（以字为单位），表示任务在执行时可以使用的堆栈空间大小。
- **pvParameters:** 任务参数的指针，可以传递给任务函数。
- **uxPriority:** 任务的优先级，值越高，优先级越高。
- **puxStackBuffer:** 指向任务堆栈缓冲区的指针，用于存储任务的堆栈。
- **pxTaskBuffer:** 指向任务控制块（Task Control Block, TCB）的指针，用于存储任务的相关信息。

【返回值】

- **Non-NULL value:** 表明 task 创建成功
- **NULL:** 创建失败

【注意】

configSUPPPORT_STATIC_ALLOCATION 必须打开。

静态任务一旦创建，其优先级和堆栈大小等属性是固定的，无法在运行时更改。如果需要动态调整任务的属性，可以使用动态任务创建函数 `xTaskCreate`。

6.9.2.2.1 示例

```
// 静态任务的堆栈大小
#define TASK_STACK_SIZE 100

// 任务函数
void vTaskFunction(void *pvParameters)
{
    // 任务的具体代码
    // ...
}

// 静态任务的堆栈缓冲区和任务控制块
StackType_t taskStackBuffer[TASK_STACK_SIZE];
StaticTask_t taskControlBlock;
```

// 在 FreeRTOS 中创建静态任务的示例代码

```
void createStaticTask()
{
    // 使用 xTaskCreateStatic 函数创建静态任务
    BaseType_t taskCreated = xTaskCreateStatic(
        vTaskFunction,           // 任务函数
        "StaticTask",           // 任务名称
        TASK_STACK_SIZE,        // 堆栈深度
        NULL,                    // 任务参数
        configMAX_PRIORITIES - 1, // 优先级
        taskStackBuffer,         // 堆栈缓冲区
        &taskControlBlock        // 任务控制块
    );

    if (taskCreated == pdPASS)
    {
        // 任务创建成功
        // ...
    }
    else
    {
        // 任务创建失败
        // ...
    }
}
```

6.9.2.3 Task 退出

【描述】

删除之前创建的 task。

【语法】

```
void vTaskDelete(TaskHandle_t xTaskToDelete);
```

【参数】

xTaskToDelete: 要删除的任务的句柄（Task Handle）。

【返回值】

void

【注意】

所有的 task 退出时，必须调用此函数。

6.9.2.3.1 示例

```
#include <FreeRTOS.h>
#include <task.h>

// 任务函数
void vTaskFunction(void *pvParameters)
{
    // 任务的具体代码
    // ...

    // 删除当前任务
    vTaskDelete(NULL);
}

// 在 FreeRTOS 中删除当前任务的示例代码
void deleteCurrentTask()
{
    // 创建任务
    xTaskCreate(
        vTaskFunction,           // 任务函数
```

```

        "TaskToDelete",           // 任务名称
        configMINIMAL_STACK_SIZE, // 堆栈大小
        NULL,                     // 任务参数
        tskIDLE_PRIORITY,         // 优先级
        NULL                      // 任务句柄
    );

    // 启动调度器
    vTaskStartScheduler();
}
    
```

6.9.3 AT 接口说明

AT 模块涉及相关接口如下表，具体定义在：

source/vendor/autochips/proprietary/tinysys/hsm/os/include /at_driver.h

表 6-7 AT 接口说明

API	说明
ATP2V	将物理地址转化为虚拟地址
ATV2P	将虚拟地址转化为物理地址

6.9.3.1 ATP2V

【语法】

```
uint32_t at_phys_to_virt (uint64_t pa)
```

【描述】

物理地址转虚拟地址。

【参数】

pa: 需要转化的物理地址

【返回值】

- pa : 当前 AT 未映射
- 非 pa: 虚拟地址

【注意】

ATC 按照 1:1 映射，所以为固定映射。

6.9.3.2 ATV2P

【语法】

```
uint64_t at_virt_to_phys (uint32_t va)
```

【描述】

物理地址转虚拟地址。

【参数】

pa: 需要转化的物理地址

【返回值】

- va: 当前 AT 未映射
- 非 va: 物理地址

【注意】

ATC 按照 1:1 映射，所以为固定映射。

7 SecureBoot

7.1 简介

7.1.1 功能

Secure Boot 功能依据芯片中的 HW Key，对所用启动关键流程的镜像进行验证，保证平台上运行的软件是完整的、未被篡改的，以防止来自外部的一些攻击。Secure Boot 依赖 RSA private key/ public key 的签名与校验、AES 的加解密校验以及基于 SHA256 算法的哈希校验，通过这些方式来保证平台上运行软件的完整性。

- 编译系统中用 RSA private key 对镜像进行签名。
- 系统更新镜像时，使用芯片中 hw key (RSA public key)，对签名的镜像校验，当校验通过后再使用 hw key2 (AES key) 加密启动相关镜像。
- 系统正常启动时，使用 hw key2 对从 EMMC 中读出的镜像进行解密并校验。

7.1.2 代码结构及文件描述

表 7-1 secure boot 代码结构及描述

文件名	文件路径	文件描述
Crypto	libsec.a	提供加解密功能
AvbLib	source/vendor/autochips/proprietary/tinysys/hsm/loader/lib/avb	提供 hash 算法功能
Efuse	libefuse.a	存储 hash 值

7.2 软件设计

7.2.1 架构图

7.2.1.1 preloader 子模块

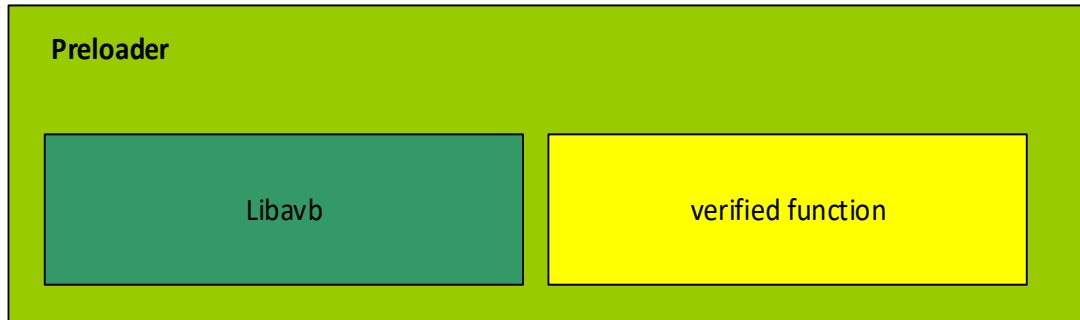


图 7-1 preloader 子模块

7.2.1.1.1 upgrade 过程中验证

upgrade 过程中该模块需要对必要的 image 进行校验，必要的校验对象为 trustzone image, hsm image, aah image 和 lk image。四个 image 的校验流程一致，校验步骤为：

1. 使用 hw crypto engine 计算 image body hash 值；
2. 签名认证，使用 public key 对 image 的签名解密，得到 image SHA-256 算法的 hash 摘要，跟第一步得到的 hash 值比较，如果内容相同，则证明了 image 没有被篡改，即完整性得到证明。如果验证通过则进行第 3 步，验证不通过则停止升级；
3. efuse 中保存了 RSA public key 的 hash 值，再使用 hw crypto engine 对存放在 image 中的 public key 做 hash，然后比较这两个 hash 值，确认 public key 的合法性。

7.2.1.1.2 正常启动过程中验证

正常启动中该模块负责对下一阶段启动需要的 image 进行校验，必要的校验对象为 trustzone image, hsm image, aah image 和 lk image。四个 image 的校验流程一致，校验步骤为：

1. preloader 对 Image 直接使用 AES 解密的方式来保证软件的合法性（因为 key 不同，解密失败），如果验证通过则进行第 2 步，验证不通过则停止启动流程；
2. 使用 public key 对 image 的签名解密，得到 image SHA-256 算法的 hash 摘要。
对 image 内容做 Hash，和第 2 步生成的 Hash 对比，如果内容相同，则证明了 image 没有被篡改，即完整性得到证明。

7.2.1.2 Image 签名子模块

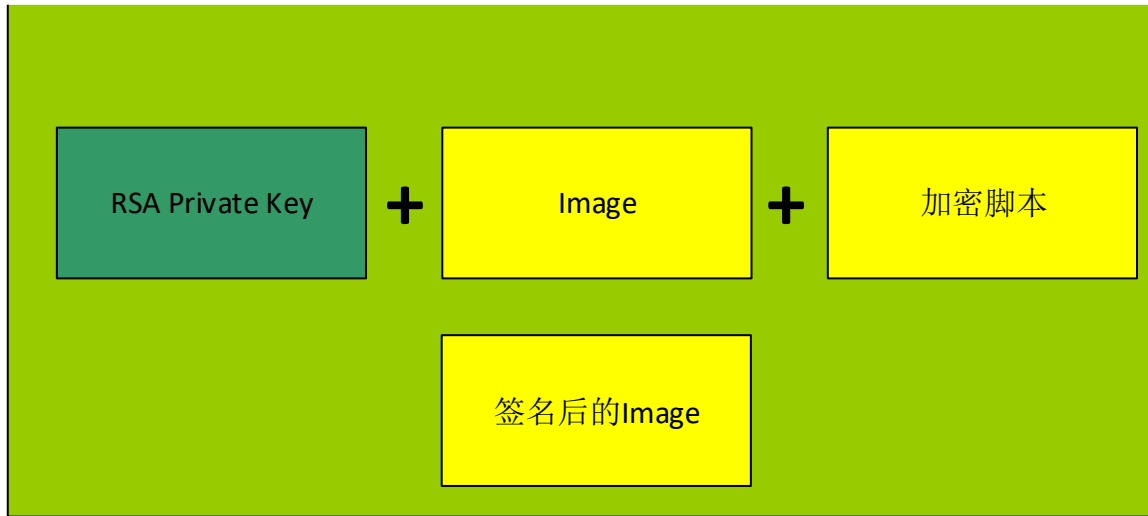


图 7-2 image 签名子模块

7.2.1.2.1 preloader image 签名

7.2.1.2.1.1 概述

系统编译整体框架中已经包含了 Preloader 的签名流程，当用户更新完 RSA Private Key（atc_rsa_prvk）并整体编译后即可生成对应的签名后的 Preloader（hl_avb_verified.img）。若用户不生成自己的 RSA Private Key，则编译时系统默认使用 test_key（文件名也叫做 atc_rsa_prvk）来生成签名 Preloader Image（hl_avb_verified.img）。

7.2.1.2.1.2 签名方式

使用签名脚本 generate_preloader_image_tool.py，脚本对应路径：

source/vendor/autochips/proprietary/tools/sign-image，该脚本支持 2 种加密 Preloader 的方式

RAW 类型

即不加密 Preloader，对应脚本命令：

```
./generate_preloader_image_tool.py generate_preloader_image --image <input_image> --out
<output_image> --binary_format bin --algorithm RAW
```

其中：<input_image>表示传入的未加密的 Preloader

<output_image>表示生成的签名后的 Preloader。

RSA 类型对应 RSA 签名脚本命令：

```
./generate_preloader_image_tool.py generate_preloader_image --image <input_image> --out
<output_image> --binary_format bin --algorithm RSA --key <rsa_prv_key>
```

其中：<input_image>表示传入的未加密的 Preloader

<output_image>表示生成的签名后的 Preloader

<rsa_prv_key>RSA 私钥

7.2.1.2.2 启动关键 image 签名

7.2.1.2.2.1 签名方式

启动关键流程的 image 签名，包括：hsm/aah/trustzone/lk/使用签名脚本完成 Image 签名，

签名脚本路径如下：source/vendor/autochips/proprietary/tools/sign-image/make_image.sh

脚本执行如下：

```
sh make_image.sh -in <input_image> -out <output_path>
```

例：sh make_image.sh -in lk.bin -out ./

其中：<input_image>表示传入的未签名的 Image

<output_path>表示生成的签名后的 Image 路径

7.2.1.3 流程图

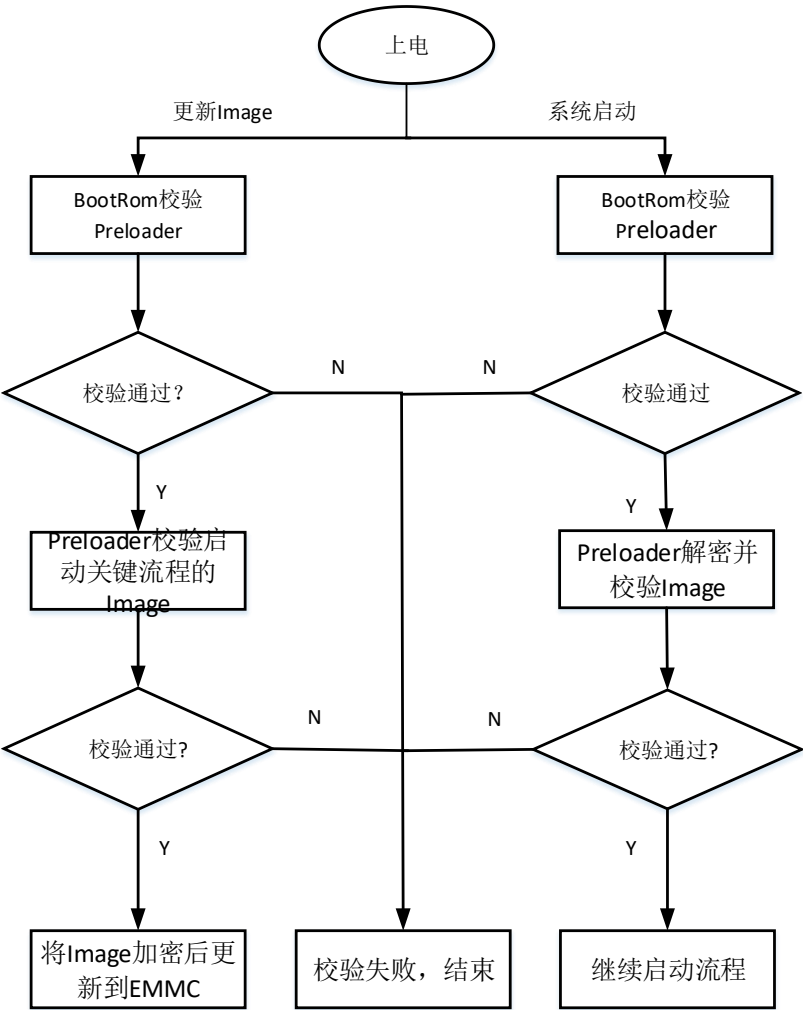


图 7-3 image 校验

7.3 接口描述

表 7-2 secure boot 接口描述

API	说明
verify_image_header	校验 image header
verify_image_body	校验 image body

7.3.1 verify_image_header

【描述】

verify image header function.

【语法】

```
int verify_image_header(void *p_header, u64 img_off, const char *name);
```

【返回值】

If success, return 0, otherwise, return nagative.

【参数】

p_header, img_off, name

【注意】

the user must call this function to verify image header

7.3.2 verify_image_body

【描述】

verify image body function.

【语法】

```
int verify_image_body(void *p_header, void *body_virt, u32 body_size, const char *name);
```

【返回值】

If success, return 0, otherwise, return nagative.

【参数】

p_header, body_virt, body_size , name

【注意】

the user must call this function to verify image body

7.4 使用说明和注意事项

7.4.1 平台 Key

7.4.1.1 key 的生成

使用 openssl 生成一对 RSA key，长度选择 2048，包括 private key 和 public key，可以用 generate_key.sh 脚本自动生成一对 RSA2048 的密钥对。generate_key.sh 脚本在 source/vendor/autochips/proprietary/tools/sign-image/ 中。

以下命令在该目录下执行，key 也会生成在该目录下，再按后面章节的步骤把 key 拷贝到相关的目录下。

生成 RSA key 命令：

```
sh generate_key.sh atc_rsa_pubk atc_rsa_prvk
```

generate_key.sh 实际使用的是 openssl 命令。

脚本执行，在当前目录会生成两个文件：atc_rsa_pubk 和 atc_rsa_prvk，分别表示 RSA2048 的公钥和私钥。

7.4.1.2 key 的存放

生成的对应密钥请放置在如下目录，并保持名称（atc_rsa_prvk）相同：

source/vendor/autochips/proprietary/tools/sign-image/cust_key/

该目录下已经存在一个 Test Key 用于支持 Secure boot。

7.4.2 Secure boot enable 配置

Secure Boot 功能取决于：

平台 efuse 是否烧写：平台需要烧写对应的 efuse bit 以支持 secure boot

对应的 bit 为 1 时打开 secure boot，否则关闭 secure boot。

当 efuse 中支持了 secure boot，则系统自动执行 secure boot 流程，所有不符合要求的 Image 都将无法引导系统启动。

当 efuse 未使能 secure boot 功能，则系统不对 Image 的完整性和合法性进行验证。

7.4.3 Preloader 签名

Preloader 的校验是由 Bootrom 完成的，因此校验流程和签名流程有别于其他启动相关 Image。

Preloader 签名使用 RSA2048，需要的秘钥参考 7.4.4.1 节生成即可。

7.4.3.1 签名方式一

系统编译整体框架中已经包含了 Preloader 的签名流程，当用户更新完 RSA Private Key (atc_rsa_prvk) 并整体编译后即可生成对应的签名后的 Preloader (hl_verified.img)。若用户不生成自己的 RSA Private Key，则编译时系统默认使用 7.4.4.1 节所述的 test_key (文件名也叫做 atc_rsa_prvk) 来生成签名 Preloader Image (hl_verified.img)。

同时为了支持非 Secure Boot 功能的平台，编译系统还会额外生成一个非 RSA 加密的 Preloader Image (hl.bin)。

实际两个 Preloader Image 均会通过脚本 generate_preloader_image_tool.py 进行签名，区别是所传参数不同。

7.4.3.2 签名方式二

使用 ATC 提供签名脚本 generate_preloader_image_tool.py，脚本对应路径：

source/vendor/autochips/proprietary/tools/sign-image

该脚本支持 2 种加密 Preloader 的方式：

1. RAW 类型

即不加密 Preloader，对应执行脚本命令：

```
./generate_preloader_image_tool.py generate_preloader_image -image <input_image> --out  
<output_image> --binary_format bin --algorithm RAW
```

其中：<input_image>表示传入的未加密的 Preloader

<output_image>表示生成的签名后的 Preloader

2. RSA 类型

ATC 推荐使用 RSA 类型加密，对应 RSA 签名脚本命令：

```
./generate_preloader_image_tool.py generate_preloader_image -image <input_image> --out  
<output_image> --binary_format bin --algorithm RSA --key <rsa_prv_key>
```

其中：<input_image>表示传入的未加密的 Preloader

<output_image>表示生成的签名后的 Preloader

<rsa_prv_key>使用 7.4.4.1 节生成的私钥即可。

7.4.4 启动关键 image 签名

本节介绍除 preloader 外的其他启动关键流程的 image 签名，包括：hsm /trustzone/LK，LK 之后的 Image 如 system、vendor 等默认使用 Android 原生 AVB（Android Verify Boot）流程，在本文不做描述。

7.4.4.1 Key 的生成

hsm/viss/trustzone/lk 的签名均使用 RSA2048 签名方式，其签名所用的 Key 与 Preloader RSA 签名方式使用的 Key 相同。

7.4.4.2 签名方式

可以参考 7.2.1.2.2.1 签名方式。

7.4.5 enable secure boot 步骤

1. key 的生成

生成 RSA key 命令：

```
sh generate_key.sh atc_rsa_pubk atc_rsa_prvk
```

generate_key.sh 脚本在 source/vendor/autochips/proprietary/tools/sign-image/中。

生成的对应密钥请放置在如下目录，并保持名称（atc_rsa_prvk）相同：

source/vendor/autochips/proprietary/tools/sign-image/cust_key/

2. 编译生成 secure boot 的镜像

现在默认编译出来的镜像就是带签名的，使用步骤 1 中生成的 key 对镜像签名。

3. 烧录 custom key

按照提供的 efuse tool 来烧录。

4. 烧录完 custom key，ic 已经开启了 secure boot，烧录相关的镜像系统可以启动成功。

8 Shutdown

8.1 简介

8.1.1 软硬件环境

8.1.1.1 软件环境

8.1.1.1.1 Andoird S Automotive 版本

Android Automotive 是 Google 为车机开发的车载专用嵌入式操作系统，专门为驾驶环境设计的。Android S Automotive 版本新增了许多与车载相关的服务，在 Shutdown 功能中用到的核心服务是 CarPowerManagementService。

CarPowerManagementService 用于接收 VMCU（Vehicle MCU，车载 MCU）的一些指令，例如：关机、重启、休眠等，并将这些指令依次推送到系统其他模块中。

此处应该注意，对于非标准 Shutdown 流程（非车机发送的 Shutdown 指令，而是由系统内部发送的，例如 recovery 应用需要重启平台）而言，无需走 Automotive 的相关车载服务流程，直接走原始 Tablet 版本的 Shutdown 流程。

8.1.1.1.2 Linux Kernel 5.10

AC8025 Android S 中使用 Linux kernel 5.10 版本，并且使用 ARM64 架构。Linux Kernel 提供了一个系统调用（reboot）专门用于接收上层发送的关机请求，当 Android 层完成 Shutdown 相关工作后，会通过系统调用将 Shutdown 请求发送至 Kernel，由 Kernel 完成剩余工作。

8.1.1.1.3 MCU 控制模块

AC8025 的 shutdown 流程最后阶段会进入到 Safety OS 中，在 Safety OS 中可以通过设置 GPIO 的电平来通知 MCU 完成断电工作。所以，AC8025 支持在外围电路中增加车载 MCU 用于控制系统电源的打开和关闭。

8.1.1.1.4 其他软件模块

在整个 Shutdown 流程中涉及到一些相关模块，下面将进行简单说明：

- Arm Trusted Firmware: Kernel 完成其工作后会进入 ATF 进行相关操作。
- IPC: AC8025 的 CPU 之间通信模块，在 AP 通知 Safety OS 关机事件时用到。

- Safety OS: 独立的安全子系统，提供基本的安全服务。在关机流程中，Safety OS 会在整体关机流程的最后将 AP 断电。

8.1.1.2 硬件环境

AC8025 Android Shutdown 功能的硬件环境包括：

- 看门狗——在系统重启过程中，可通过直接触发看门狗的方式来实现。
- IPC 通信模块——用于 CPU 间传递消息。
- 按键（GPIO）——用于接收用户的关机请求。
- 其他硬件——关机过程中需要进行相应处理的硬件模块，例如：Storage 相关。

8.1.2 功能

本模块的主要功能如下：

- 接收用户输入的关机行为，例如：按键、输入指令等方式
- 接收用户输入的重启行为，例如：指令输入
- 提供相关接口用于实现系统内部的关机或重启行为

8.1.3 代码结构以及文件描述

本模块相关的头文件、源文件、库文件、配置文件等的路径及文件描述如下表所示。

表 8-1 Shutdown 相关代码路径

文件名	文件路径	文件描述
gpio_keys.c	source/kernel/common/drivers/input/keyboard	Linux kernel 接收 GPIO 按键处理流程
powerkey.c	source/kernel/common/drivers/soc/autochips/powerkey	Linux Kernel 上报 Shutdown 事件到 Android 层
VehicleEmulator.cpp	source/android/hardware/interface/automotive/vehicle2.0/default/impl/vhal_v2_0/VehicleEmulator.cpp	Android 层接收 Kernel 上报（或 VMCU 发送）事件接口
reboot.c	source/kernel/common/kernel	Linux Kernel 提供的原生系统调用

文件名	文件路径	文件描述
pwrkey_task.c	hsm/os/task	Safety OS 接收关机相关事件
ac8x_shutdown_common.c	source/kernel/common/drivers/soc/autochips/shutdown_common	shutdown 流程控制接口

8.2 软件设计

8.2.1 架构图

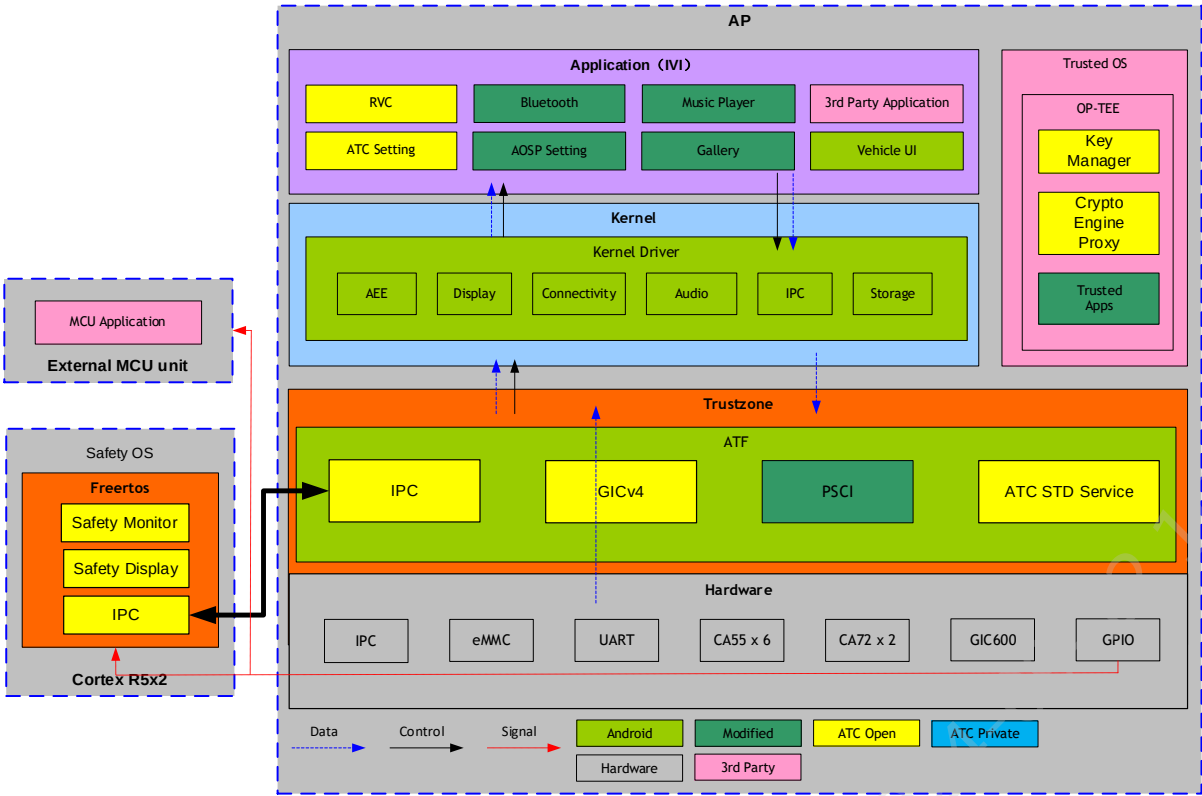


图 8-1 AC8025 Shutdown 整体架构图

8.2.2 流程图

8.2.2.1 Shutdown 流程图

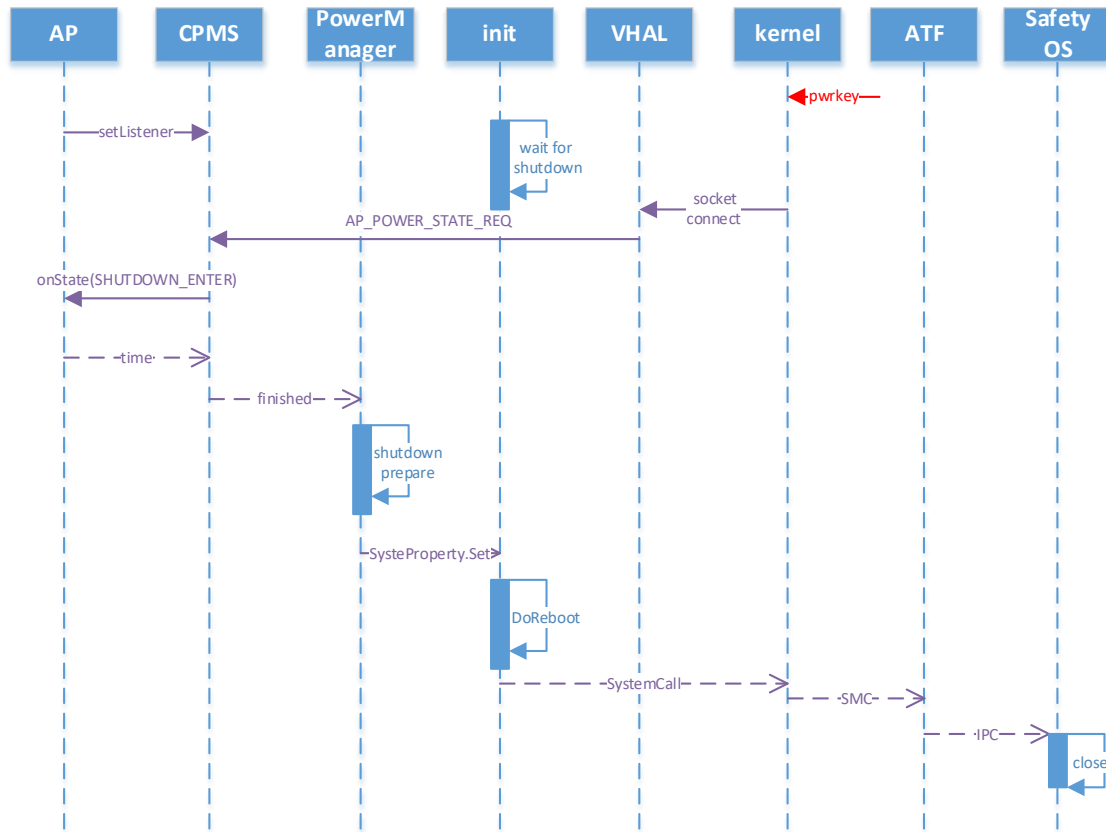


图 8-2 Shutdown 关键流程

对于 android 的系统关机流程，说明如下：

- kernel 接收到 powerkey 按键事件并将消息发送到 VHAL 层
- VHAL 发送 AP_POWER_STATE_REQ 消息到 CPMS（CarPowerManagementService）通知各注册相关事件的进程系统开始 Shutdown
- android 上层 Shutdown 流程完成之后，再次将 Shutdown 消息传到 Kernel 层
- 在 kernel 中首先进行 syscore shutdown、device shutdown 流程，最后将消息发送到 ATF 层
- 在 ATF 中将来自 kernel 中的 Shutdown 消息通过 IPC 发送给 Safety OS
- Safety OS 中对 AP 进行断电

另外，Android Automotive 版本关机流程中包括了两种关机：一种为立即关机，主要表现为 Android 层响应关机信号后直接关闭各个服务。而对于非立即关机的情况，Andorid 系统可以允许等待应用响应

关机事件，待操作完成后再行关机，其中上层 APP 可以通过注册电源状态变更的监听事件来获得关机事件，从而进行相应处理（注：此种注册监听事件的方式对于“立即关机模式”不可行）。当前 AC8025 中为了考虑关机时间，默认使用的是立即关机。其具体配置可以参考 [8.3.2 数据结构](#) 中的配置。

8.3 接口描述

8.3.1 API

由于 IVI 形态中只包含 android 系统，shutdown 功能涉及到相关接口如下表所示：

表 8-2 Shutdown 相关 API

API	说明
shutdown_work	Kernel 端发送关机消息到上层（VHAL 接收）
pwrkey_vhal_send	Kernel 端发送关机消息具体实现函数(VHAL 接收)
kernel_power_off	Kernel 端提供的源生 poweroff 流程
kernel_restart	Kernel 端提供的源生 restart 接口
plat_system_off	ATF 处理并发送 Shutdown 消息到 Safety OS 端接口
plat_system_reset	ATF 处理并发送 Reboot 消息到 Safety OS 端接口
android_reboot	上层系统 shutdown 接口
register_reboot_notifier	Kernel 层注册 reboot 回调接口
vPwrkeyTask	Safety OS 层处理 AP 端消息接口

8.3.1.1 shutdown_work

【描述】

用于在 Kernel 层面处理用户长按 Wakeup 键行为，具体行为是发送 Shutdown 消息到上层。

其接收端可索引同文件内的 socket 端口值。

【语法】

```
static void shutdown_work (void);
```

【返回值】

None

【参数】

None

【注意】

- 该接口为 ATC 提供，用于走 Android Automotive 的 Shutdown 流程，与 kernel 提供的原生 Shutdown 接口（kernel_power_off）的区别是，Kernel 原生接口仅仅在 Kernel 层面进行处理，并不通知上层。
- 该接口不对外开放，如客户想要在其他场景中使用，可以实现类似接口即可。
- 仅仅通过该接口的方式进入 Shutdown 才会真正进入 VHAL 层，走 Android Automotive 版本的 Shutdown。其他方式，如 android_reboot 等接口会跳过 VHAL 层，通过设置系统属性的方式直接走原 Tablet 版本的 Shutdown 流程。

8.3.1.2 pwrkey_vhal_send

【描述】

使用 socket 发送消息到 vhal 层。

【原型】

```
static int pwrkey_vhal_send ( void *msg, int size );
```

【返回值】

- 0：发送成功
- -1：发送失败

【参数】

- msg：具体的消息
- size：对应于 msg 的 size

【注意】

- 核心调用 kernel_sendmsg 接口，该接口不能用在中断上下文，所以对应的本接口也就不适用于中断上下文，所以 shutdown_work 是用在 delay work 机制中
- 对应 msg 的格式包括两个 iov[0]用于表示本次传输的具体 size，这样做的目的是适配 VHAL 层的处理
- 与 send 相关的其他接口本节不一一描述，例如：connect、open 等操作

8.3.1.3 kernel_power_off

【描述】

Kernel 层提供的原生 power off 接口。

【原型】

```
void kernel_power_off ( void );
```

【返回值】

None

【参数】

None

8.3.1.4 kernel_restart**【描述】**

Kernel 层提供的源生 restart 接口。

【原型】

```
void kernel_restart ( char *cmd );
```

【返回值】

None

【参数】

cmd: recovery 操作时需要传入的参数，当参数为“atcrecovery#”时，系统重启进入 recoverymode。

【注意】

无

8.3.1.5 plat_system_off**【描述】**

ATF 端通知 Safety OS 端 Shutdown 事件最后一步的接口。

【原型】

```
static void __dead2 plat_system_off ( void );
```

【返回值】

None

【参数】

Node

【注意】

进入此接口后，表明当前 Android 层和 Kernel 层面的 Shutdown 已经完成，基本的 Driver 均已结束，故应当尽量减少其内部操作。

8.3.1.6 plat_system_restart

【描述】

ATF 端通知 Safety OS 端 reset 事件最后一步的接口。

【原型】

```
static void __dead2 plat_system_reset ( void );
```

【返回值】

None

【参数】

Node

【注意】

接口名中的 reset 定义和 kernel 端的 restart 以及文中提到的 reboot 含义相同，均为重启系统的意义。

8.3.1.7 android_reboot

【描述】

上层提供的系统 shutdown 接口，并可以根据传入的参数类型决定具体的操作，包括关机和重启。

【原型】

```
int android_reboot ( int cmd, int flags, const char * arg );
```

【返回值】

- -1: 失败
- 0: 成功

【参数】

- cmd: 当前包括如下几个 cmd
 - ANDROID_RB_RESTART: 原有重启 cmd，现已废弃
 - ANDROID_RB_RESTART2: 新版重启 cmd
 - ANDROID_RB_POWEROFF: 关机 cmd
 - ANDROID_RB_THERMOFF: 因 Thermal 导致的关机
- flags: 未用到
- arg: 需要追加到 cmd 之后的额外参数，例如: recovery 重启

【注意】

此接口流程走原有 Android Tablet 版本的 Shutdown。

8.3.1.8 register_reboot_notifier**【描述】**

kernel 层提供的注册 reboot 通知事件回调。

【原型】

```
int register_reboot_notifier ( struct notifier_block *nb );
```

【返回值】

- -1: 失败
- 0: 成功

【参数】

nb: 注册的回调结构，详细参考 [8.3.2 数据结构](#)。

8.3.1.9 vPwrKeyTask**【描述】**

Safety OS 端会在启动时创建一个 Task，用来等待并接受 AP 端发送的 Shutdown 相关消息，该接口即为对应 Task 的实现。

【原型】

```
static void vPwrkeyTask ( void *parameter );
```

【返回值】

None

【参数】

无需关心

【注意】

接口内部通过消息类型区分具体的事件，用户可自定义其消息类型，具体类型参考 [8.3.2 数据结构](#)。

8.3.2 数据结构

表 8-3 Shutdown 相关数据结构

API	说明
notifier_block	Kernel 层通知事件结构
VehicleApPowerStateReq	Android 层电源请求类型
VehicleApPowerStateShutdownPrepare	Android 层请求 shutdown 具体事件类型

8.3.2.1 snotifier_block

```
struct notifier_block {
    notifer_fn_t  notifier_call;          /* 回调函数 */
    struct notifier_block __rcu *next;    /* rcu 锁 */
    int priority;                          /* 优先级 */
}
```

8.3.2.2 VehicleApPowerStateReq

```
enum VehicleApPowerStateReq:int32_t {
    OFF = 0;                               /* 关闭*/
    DEEP_SLEEP = 1;                       /* 深入睡眠 */
    ON_DISP_OFF = 2;                     /* AP 打开，但是请求 display 关闭 */
    ON_FULL = 3;                         /* AP 处于 display 显示状态 */
    SHUTDOWN_PREPARE = 4;                /* 请求 Shutdown */
}
```

8.3.2.3 VehicleApPowerStateShutdownPrepare

```
enum VehicleApPowerStateShutdownPrepare : int32_t {
    SHUTDOWN_IMMEDIATELY = 1;           /* 立即关闭 */
    CAN_SLEEP = 2;                      /* 允许睡眠 */
    SHUTDONW_ONLY = 3;                  /* 允许推迟关机 */
}
```

8.4 使用说明和注意事项

8.4.1 使用说明

- AC8025 Shutdown 模块支持系统的关机和重启功能，具体使用方式如下表所示。
- 在开机过程中不支持 CPU_Wakeup 键关机，只有当系统 Home 界面启动之后，按键生效。
- 按键按下超过 500ms 后松开，系统进入 Shutdown 流程。
- 当关机完成后，平台断电，CPU_Wakeup 键不再生效，如想重新启动平台，需要重新上电。
- 当前 AC8025 Shutdown 设置了按键超时 3 s，即长按超过 3 s 后即使不松开按键，系统仍然在第 3 秒开始进入 Shutdown 流程。

表 8-4 Shutdown 相关数据结构

触发方式	说明
长按CPU_Wakeup键 (> 500ms)	走Android Automotive版本的完整关机流程
通过ADB或者Shell的方式输入：“reboot”或者“echo reboot > /dev/shutdown0”	系统重启
通过ADB或者Shell的方式输入：“reboot -p”或者“echo shutdown > /dev/shutdown0”	系统关机

8.4.1.1 userspace 层广播使用方法

8.4.1.1.1 Android 系统接收 shutdown 信号广播

第一种情况，对于 android 系统中 java 开发的 app 可以通过注册 ACTION_SHUTDOWN 广播来获取关机消息，从而进行一些特殊的处理；使用方法如下：

1. 动态注册 ACTION_SHUTDOWN 监听

```
public class MainActivity extends Activity {
    ... private static final String TAG = "MainActivity";
    ... private final Context mContext = this;
    ... @Override
    ... protected void onCreate(Bundle savedInstanceState) {
    ...     super.onCreate(savedInstanceState);
    ...
    ... setContentView(R.layout.activity_main);
    ...     Log.e(TAG, "surfaceCreated.OnCreate.test.broadcast\n");
    ...     MyBroadcastReceiver mOwnerReceiver = new MyBroadcastReceiver();
    ...
    ...     IntentFilter ownerFilter = new IntentFilter();
    ...     ownerFilter.addAction(Intent.ACTION_SHUTDOWN);
    ...     mContext.registerReceiver(mOwnerReceiver, ownerFilter);
    ...
    ... setContentView(R.layout.activity_main);
    ...     ((TextView) findViewById(R.id.button1)).setText("test");
    ...     Button clone = findViewById(R.id.button1);
    ...     clone.setOnClickListener(new View.OnClickListener() {
    ...         @Override
    ...         public void onClick(View view) {
    ...             Log.e(TAG, "clone.button1");
    ...         }
    ...     });
    ... }
}
```

2. 实现 shutdown 信号处理函数

```
public class MyBroadcastReceiver extends BroadcastReceiver {
    ... final private String TAG = "MyBroadcastReceiver";
    ... @Override
    ... public void onReceive(Context context, Intent intent) {
    ...     String action = intent.getAction();
    ...     Log.e(TAG, "Device is shutting.getAction." + action);
    ...     if (action.equals(Intent.ACTION_SHUTDOWN)) {
    ...         Log.e(TAG, "Device is shutting.down.test");
    ...     }
    ... }
}
```

对于 android 系统，通过 CarPowerManager 来监控 shutdown 时电源的状态变化。当 ivi 系统 Shutdown 时，会回调执行 onStateChanged 函数。

第二种情况，当程序由 cpp 实现，需要通过 CarPowerManager 来监控 Shutdown 时电源的状态变化。如下图所示，当系统 Shutdown 时，会回调执行 onStateChanged 函数。

```
void onStateChanged(CarPowerManager::State state) {
    ALOGI(LOG_TAG "onStateChanged callback = %d", state);
    if (state == CarPowerManager::State::kShutdownPrepare) {
        // Stop the loop
        run = false;
    }
}

int main(int, char**)
{
    int retVal;

    sp<ProcessState> processSelf(ProcessState::self());
    processSelf->startThreadPool();
    ALOGI(LOG_TAG "started");

    std::unique_ptr<CarPowerManager> carPowerManager(new CarPowerManager());

    do {
        retVal = carPowerManager->setListener(onStateChanged);
    } while (retVal != 0);

    do {
        ALOGI(LOG_TAG "Waiting for CarPowerManager listener to initiate SHUTDOWN_PREPARE...");
        sleep(5);
    } while (run);

    ALOGI(LOG_TAG "Exited loop, shutting down");

    // Unregister the listener
    carPowerManager->clearListener();

    // Wait for threads to finish, and then exit.
    IPCThreadState::self()->joinThreadPool();
    return 0;
}
```

3.实现shutdown回调函数

1.实例化对象

2.注册监听函数

8.4.1.1.2 kernel 层广播使用方法

1. 方式一：通过在驱动中注册关机的回调函数。

Shutdown 流程中为设备提供了 device_shutdown 函数，当系统执行关机时此函数会回调驱动中注册的 shutdown 函数。此方法用起来比较简单，只需要在驱动中注册自己的 shutdown 函数即可。以 emmc 的驱动为例。

定义 shutdown 回调函数

```
void sdhci_atc_shutdown(struct platform_device *pdev)
{
    struct sdhci_host *host;
    struct device *dev = &pdev->dev;
    struct sdhci_pltfm_host *pltfm_host;
    struct sdhci_atc_host *host_atc;

    host = platform_get_drvdata(pdev);
    if (!host) {
        dev_err(dev, "sdhci_atc_shutdown gets no sdhci_host\n");
        return;
    }

    pltfm_host = sdhci_priv(host);
    host_atc = sdhci_pltfm_priv(pltfm_host);

    if (host_atc->host_id == SDIO_DEV_ID) {
        dev_warn(dev, "sdio need no shutdown\n");
        return;
    }

#ifdef CONFIG_MMC_VIRTUAL
    if (host_atc->host_id == EMMC_DEV_ID) {
        dev_warn(dev, "emmc(host%d) virtual shutdown\n", host_atc->host_id);
        return;
    }
#endif

    /* Disable all interrupts */
    free_irq(host->irq, host);
    sdhci_writel(host, 0, SDHCI_INT_ENABLE);
    sdhci_writel(host, 0, SDHCI_SIGNAL_ENABLE);

    cancel_delayed_work_sync(&host->mmc->detect);
    cancel_delayed_work_sync(&host->mmc->detect_sdcard);

    sdhci_atc_set_src_clk(host_atc, false);

    pr_info("===== sdhci SD4HC(host%d) shutdown =====\n", host_atc->host_id);

    return;
}
EXPORT_SYMBOL(sdhci_atc_shutdown);
```

填充 driver 结构体

```
static struct platform_driver sdhci_atc_driver = {
    .driver = {
        .name = "sdhci-atc",
        .owner = THIS_MODULE,
        .of_match_table = of_match_ptr(sdhci_atc_of_match),
        .probe_type = PROBE_PREFER_ASYNCHRONOUS,
    },
    .probe = sdhci_atc_probe,
    .remove = sdhci_atc_remove,
    .suspend = sdhci_atc_suspend,
    .resume = sdhci_atc_resume,
    .shutdown = sdhci_atc_shutdown,
};
```

- 方式二：通过 notify_block 的机制。对于有需要 shutdown 发生时收到消息通知的，只需关注 SYS_POWER_OFF 事件即可。在 kernel 中已经存在事件通知链 reboot_list，因此只需要定义自己的通知链回调函数，并且注册到 notifier chain 中即可，当发送 Shutdown 事件时，系统会自动调用所有注册到 reboot_notifier_list 的函数。

具体步骤如下：

- 1) 定义回调函数

```
int test_init_event(struct notifier_block *nb, unsigned long event, void *v)
{
    switch(event) {
        case SYS_POWER_OFF: // 1.shutdown时的事件
            pr_msg("i get the power off chain event"); // 2.do something
            break;
        default:
            pr_msg("no get the power off chain event");
    }
    return NOTIFY_DONE;
}
```

- 2) 初始化 notifier_block 结构体

```
static struct notifier_block test_init_notifier = {
    .notifier_call = test_init_event,
};
```

- 3) 注册回调函数到 reboot_notifier_list 中

```
blocking_notifier_chain_register(&reboot_notifier_list, &test_init_notifier);
```

8.4.2 注意事项

- 对于关机流程而言，需要在完全断开系统电源之间完成一些操作，以便于保留当前用户配置，其中以 storage 部分为主，若断电前没有进行相关处理，则数据会丢失。所以，以 storage 为例，通常的做法就是在关机时注册 shutdown callback，并在 callback 中完成数据的写入。
- 关机流程不可逆，不可暂停。

- AC8025 Shutdown 功能不开启用户确认界面，即当按键超过 500ms，则默认黑屏关机。
- 经测试得出：正常关机的时间不会超过 20s，若关机的整体时间超过了 20s，则可能存在系统软件方面的问题。

8.5 用户定制化

8.5.1 按键触发 Shutdown 流程

8.5.1.1 打开和关闭按键 Shutdown

AC8025 Android S 的按键 Shutdown 触发是在 Kernel 层完成的，其默认是打开的，用户关闭方式：配置 menuconfig，将 CONFIG_POWER_OFF 关闭即可。

8.5.1.2 按键超时设置

当前 AC8025 平台 Shutdown 流程涉及两个时间：

- 当按下超过 500ms 并小于 3s 后松开按键，则何时松开按键系统何时开始进入 Shutdown 流程；
- 当按下超过 3s 后，即使按键不松开系统也默认进入 Shutdown 流程。

这两个时间设置同在：source/kernel/common/drivers/soc/autochips/powerkey/powerkey.c 中

- g_pwrkey_current_time 用于记录按键按下松开的时间间隔，当前设置松开时间间隔大于 msecs_to_jiffies(500)即可触发；
- pwrkey_work 用于设置超过 3s 后的自动关机流程。

8.5.2 调用接口触发 Shutdown 流程

在 AC8025 中，提供命令去触发 Shutdown 流程，命令如下：

```
echo shutdown > /dev/shutdown0
```

对于调用接口触发 shutdown 流程的需求，可在 kernel 中调用函数 shutdown_work 函数实现。具体可以参考 8.3.1.1 节的说明。此函数的行为将 Shutdown 流程信号发送给上层 VHAL，从而开始整个系统由上层到底层的关机流程。

8.5.3 Shutdown 通信配置

当前 AC8025 Android S Shutdown 在多个 CPU 之间进行通信时采用 IPC 通信格式，其具体的通信信息沿用 Kernel 原生 reoot 系统调用的信息，通过具体的 CMD 值来区分操作行为，当前 CMD 值如下：

```
#define LINUX_REBOOT_CMD_RESTART    0x01234567
#define LINUX_REBOOT_CMD_HALT        0xCDEF0123
#define LINUX_REBOOT_CMD_CAD_ON      0x89ABCDEF
#define LINUX_REBOOT_CMD_CAD_OFF     0x00000000
#define LINUX_REBOOT_CMD_POWER_OFF   0x4321FEDC
#define LINUX_REBOOT_CMD_RESTART2    0xA1B2C3D4
#define LINUX_REBOOT_CMD_SW_SUSPEND  0xD000FCE2
#define LINUX_REBOOT_CMD_KEXEC       0x45584543
```

该值的源生定义在文件：source/kernel/common/include/uapi/linux/reboot.h

在 ATF 发送端的键值定义在：

source/trustzone/atf/v2.3/plat/autochips/ac8025/include/platform_def.h

在 Safety OSs 接收端的键值定义在：

source/vendor/autochips/proprietary/tinysys/hsm/os/tasks/pwrkey_tasks.c

用户自定义行为：

- 根据需要在发送端（ATF）和接收端（Safety Os）新增相关指令
- 根据需要变更指令格式

8.5.4 Safety OSs 配置 GPIO

当前 AC8025 Android S Shutdown 在运行到 Safety OS 后，可以通过设置相关 GPIO 的值来通知到 MCU。MCU 在检测到 GPIO 电平变化后进行平台断电工作。

当前 AC8025 选择 CPU LAUNCHER 作为信号，该 gpio 在开机时由 Kernel Driver 设置为高电平，并在 Shutdown 程序的最后由 Safety OS 拉低以通知 MCU。用户可根据自身需求，选择合适的 GPIO 口，设置 GPIO 的操作举例如下：

```
xGpioSetMode(gpio, 0);           // 设置 GPIO 的模式为普通 GPIO 功能
xGpioSetDirection(gpio, 1);       // 设置 GPIO 的输入输出方向，其中 1 为输出，0 为输入
xGpioSetPinValue(gpio, level);    // 设置 GPIO 对应的具体电平值，其中 1 为高电平，0 为低电平
```

同时，对应 GPIO 可以通过设置上拉的方式，使得对应 GPIO 可以驱动外接 LED 的点亮，从而实现肉眼观察 GPIO 高低的变化。设置上拉的方式如下：

```
xGpioSetPullUpDown(gpio, 1); // 设置 GPIO 上拉，其中参数 1 表示上拉，0 表示下拉
```

8.6 典型问题

8.6.1 输入关机指令系统重启

【问题描述】

输入关机指令后系统重启。

【原因】

在系统关机过程中出现了 Kernel Panic 等异常错误导致系统重启。

【解决方案】

- 通过查看 log 来确认是否存在 callstack，若有则可以确认问题所在；
- 通过查看 log 来确认是否存在 callstack，若无则需要确认是否有流程手动调用 system reboot 相关接口。

9 Thermal

9.1 简介

9.1.1 功能

主要功能：

- 通过 kernel 提供的节点获取 CPU 温度
- 按照既定 Thermal 温控策略，实现监控芯片的温度，并利用温控策略实现对芯片温升的控制
- 可按需进行 Thermal 温控策略配置
- 提供关闭 Thermal 的配置（默认保留高温上限重启，通常用于特定高温测试）

9.1.2 代码结构及文件描述

表 9-1 Thermal 代码结构及描述

文件名	文件路径	文件描述
thermal_ac8025.c	source/kernel/common/drivers/thermal/atc/ac8025/	注册的 8025 thermal module
thermal_core.c	source/kernel/common /drivers/thermal/thermal_core.c	Linux Thermal Framework 主要模块
thermal.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers	AC8025 平台 cpu thermal sensor 驱动

9.2 软件设计

9.2.1 架构图

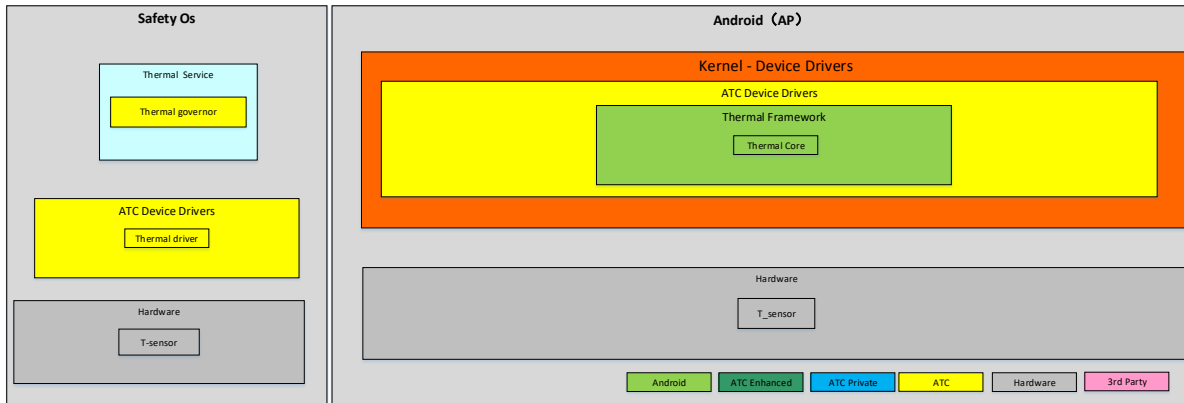


图 9-1 Thermal 架构图

- Thermal driver 做 thermal hw init，通过 Thermal Sensor 寄存器数值获取当前的温度。AC8025 有 2 对 T Sensor，现在选取的是靠近 CPU 的 T-sensor 读取温度。
- Thermal governor 监测高低温，当温度达到或者超过 target 温度时 Thermal service 会降低 CPU 频率，当温度达到或者超过 reboot 温度时 Thermal service 会重启所有系统。
- Thermal framework 提供一些列文件节点，给客户 获取实时温度跟 debug 使用。

9.2.2 流程图

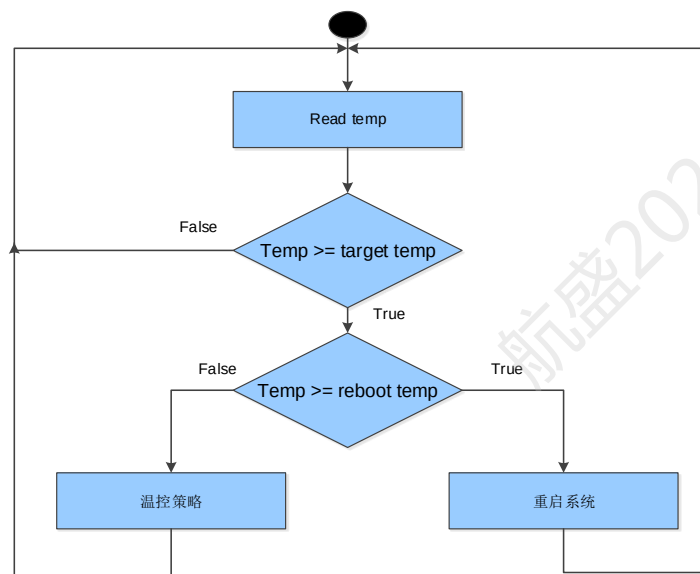


图 9-2 Thermal 流程图

9.3 接口描述

9.3.1 API

表 9-2 Thermal API

API	说明
thermal driver proc 节点	获取 CPU 实时温度 获取 thermal 配置策略
thermal driver debug 节点	控制 thermal 驱动 log 输出情况

9.3.1.1 Thermal driver proc 节点

【描述】

thermal driver 的 proc 目录 path 为：/proc/driver/thermal/，

kernel 使用 int read_temperature(void)接口获取温度

【语法】

- /proc/driver/thermal/tzcpu, cat 可输出当前 cpu thermal 策略配置
 - 【关闭 thermal 限频】可 pull 出系统中的/vendor/etc/.tp/thermal.conf，将所有温度值（单位毫摄氏度）统一修改为重启温度 125000，然后 push 回原位置，重启系统即可。
 - 【关闭 thermal 重启】可 pull 出系统中的/vendor/etc/.tp/thermal.conf，将默认最后一个最大的温度数值（默认为 125000）修改为一个不可能达到的温度值（比如 190000，驱动中默认忽略温度值高于 200 摄氏度的配置），然后 push 回原位置，重启系统即可
- /proc/driver/thermal/tzcpu_read_temperature, cat 可输出实时 cpu 温度，kernel 对应的获取温度接口为 int read_temperature（void）。

【参数】

NA

【返回值】

参考上述语法描述。

9.3.1.2 Thermal driver debug 节点

【描述】

Thermal driver 提供了属性节点，启用后可以 log 定期输出 CPU 实时温度。

【语法】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

217 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

/sys/devices/platform/soc/110b0000.tsensor/temp_debug，echo 写入 1 后启动 CPU 温度定期输出。

- 输出方式为 kernel log
- 固定格式：“[Thermal] CPU T=”
- 频率：2s 一次

【参数】

NA

【返回值】

参考上述语法描述。

9.3.1.3 Thermal dts 配置

thermal 在 kernel dts 中有一个默认配置（系统启动到 Android 阶段后会再由 init 进程调用 thermal_manager 将/vendor/etc/tp/thermal.conf 中的配置写入 thermal 驱动，从而覆盖 dts 中的这个默认配置）。

默认的 dts 配置位于：source/kernel/common/arch/arm64/boot/dts/autochips/ac802x.dtsi

参考默认配置如下：

```
thermal-zones {
    cls0:cls0{
        polling-delay=<2000>;
        polling-delay-passive=<200>;

        /* sensor ID */
        thermal-sensors=<&tsensor 0>;

        trips{
            cpu_alert:trip-point@0{
                temperature=<85000>;
                hysteresis=<1000>;
                type="passive";
            };
            cpu_target:trip-point@1{
                temperature=<100000>;
                hysteresis=<1000>;
                type="passive";
            };
            cpu_critical:trip-point@2{
                temperature=<125000>;
                hysteresis=<1000>;
            };
        };
    };
}
```

```

        type="critical";
    };
};

cooling-maps{
    map0{
        trip=<&cpu_target>;
        cooling-device=<&cpu0 0 4>;
        contribution = <1024>;
    };
};
};

```

其中，cpu_alert 温度为 CPU 限频策略启动的温度 trip point；cpu_target 温度为当前默认 thermal 策略用的参考目标温度，在 CPU 温度达到该温度后会增加限频的力度；cpu_critical 温度为系统重启温度 trip point。

9.3.1.4 Thermal 配置文件

thermal 配置文件（默认为/vendor/etc/.tp/thermal.conf）中存储了 thermal 策略方面配置。

其文件格式：

- 一行有效 kernel 文件节点
- 一行配置参数

上述格式在 thermal 支持多个策略配置时，可交替扩展下去。

当前 AC8025 只有 cpu 方面限频策略，其默认配置如下：

```

/proc/driver/thermal/tzcpu
85000 100000 123000

```

其中，第二行的三个参数，分别对应 9.3.1.3 Thermal dts 配置章节中的 cpu alert/target/critical 温度值（单位毫摄氏度）。

9.4 使用说明和注意事项

9.4.1 温控策略

step_wise 算法在计算 target cooling state 的过程中，根据 throttle 和 trend 来做计算。

trend 表示温升趋势，Linux Thermal Framework 定义了五种 trend type，见 enum thermal_trend，即稳定(THERMAL_TREND_STABLE), 上升(THERMAL_TREND_RAISING),

下降（THERMAL_TREND_DROPPING），最高温线（THERMAL_TREND_RAISE_FULL），最低温线（THERMAL_TREND_DROP_FULL）。

step_wise governor 对于 cooling_state 选择的策略：

- 当 throttle 发生且温升趋势为上升，使用更高一级的 cooling state；
- 当 throttle 发生且温升趋势为下降，不改变 cooling state；
- 当 throttle 解除且温升趋势为下降，不改变 cooling state；
- 当 throttle 解除且温升趋势为上升，使用更低一级的 cooling state。

9.5 典型问题

可以根据下面 log 判断是不是因为高温策略导致的重启：

```
"critical temperature reached(%d C),rebooting..."
```


10 LK 升级

10.1 简介

LK 升级是系统运行在 LK 阶段进行的升级，通过触发某一事件进入 LK 升级流程，把 FAT32 格式的 U 盘/SD 卡中特定目录下需要升级的升级文件（部分或全部 image）更新到 eMMC 指定位置。

10.1.1 软硬件环境

10.1.1.1 软件环境

本章所介绍的 LK 升级模块的设计基于 LK（Litter Kernel）流程，所有 Android 版本均支持。

- 外设（U 盘或者 SD 卡）需要是单分区的 FAT32 格式，如果多分区，只支持第一个分区是 FAT32 格式。
- 外设中根目录下面有 ATCUPG 目录。
- ATCUPG 目录下面有 atc.upd 文件。

10.1.1.2 硬件环境

本模块所依赖的硬件环境有：

- LK 拷贝升级
 - 平台插入外置 SD 卡或者 U 盘。
 - 串口或 adb。
- LK Fastboot 升级
 - LK Fastboot 功能依赖 usb cable 与 PC 的正常连接。

10.1.2 功能

LK Upgrade 包括 LK 拷贝升级和 LK Fastboot 升级。

- LK 拷贝升级功能
 - 全部 image 整体升级
 - 部分 image 升级
- LK Fastboot 支持 fastboot 协议自带的一些功能：

- 擦除分区
- 升级分区 image
- 重启系统
- 查看版本号

10.1.3 代码结构及文件描述

表 10-1 LK 拷贝升级代码结构和描述

文件名	文件路径	文件描述
atc_upg.c	source/vendor/autochips/proprietary/bootable/lk/app/upgrade/copy_upgrade/	LK 拷贝升级主程序
xml.c	source/vendor/autochips/proprietary/bootable/lk/lib/libxml/	LK 拷贝升级过程中负责解析 scatter.xml 文件
boot_flow.c	source/vendor/autochips/proprietary/bootable/lk/platform/autochips/ac8x/	LK 升级入口（包含 fastboot 升级、制卡升级和拷贝升级）
atc_upg.h	source/vendor/autochips/proprietary/bootable/lk/app/upgrade/include/	LK 拷贝升级头文件

表 10-2 LK Fastboot 升级代码结构和描述

文件名	文件路径	文件描述
fastboot.c	source/vendor/autochips/proprietary/bootable/lk/app/upgrade/fastboot_upgrade/	LK fastboot 升级主程序
fastboot.h	source/vendor/autochips/proprietary/bootable/lk/app/upgrade/include/	LK fastboot 升级头文件

10.2 软件设计

10.2.1 架构图

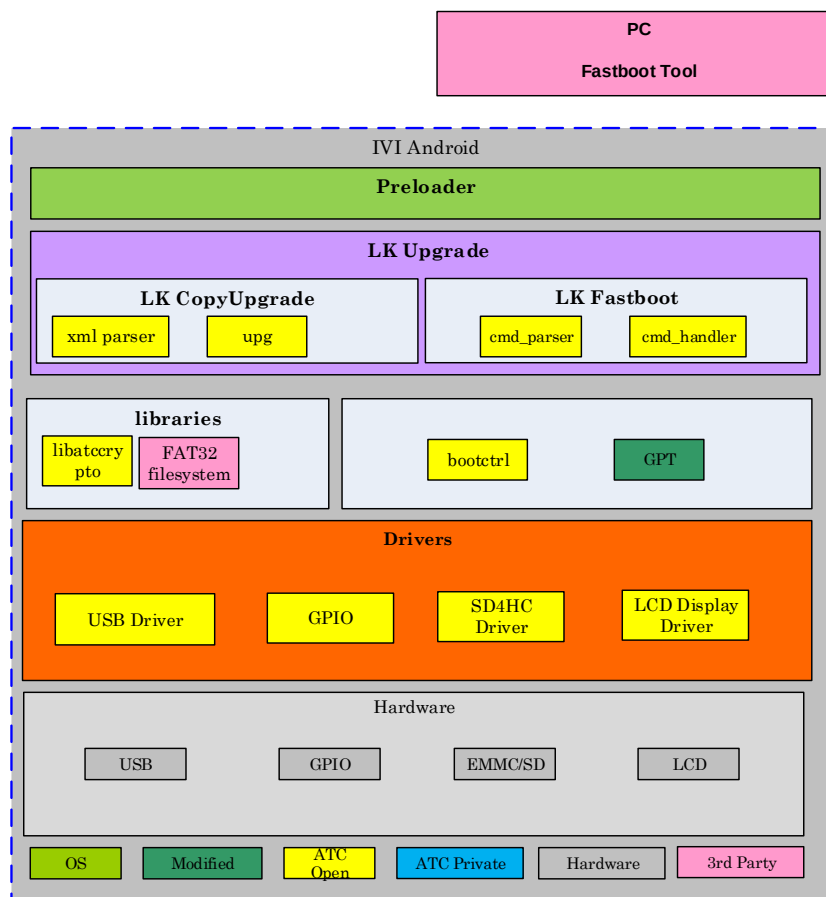


图 10-1 LK Upgrade 系统架构图

图 10-1 所示的架构图中共分为三层。

- 最上层为 LK 中实现的升级 Flow，可以称之为 LK App Layer；
- 中间层为 File System 和 driver layer，用于控制升级过程中使用到的硬件设备；
- 最下层为 Hardware Layer，主要是升级过程中用到的 Hardware。

10.2.2 流程图

10.2.2.1 LK 拷贝升级

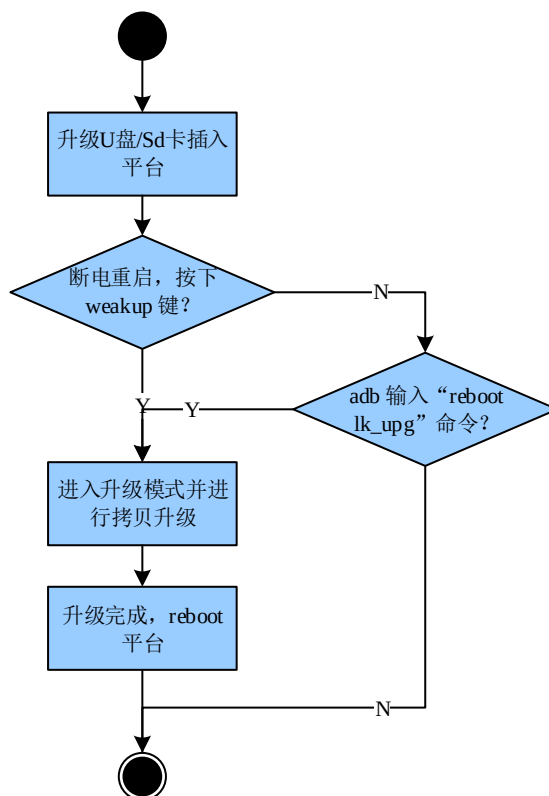


图 10-2 LK 拷贝升级流程

- 1) 系统启动，通过 adb 输入 “reboot lk_upg” 进入拷贝升级模式；
- 2) 检测升级文件存在则进入升级，否则提示升级失败；
- 3) 升级完成，reboot 系统（自动和非自动可以进行 config 文件配置）。

10.2.2.2 Fastboot 升级

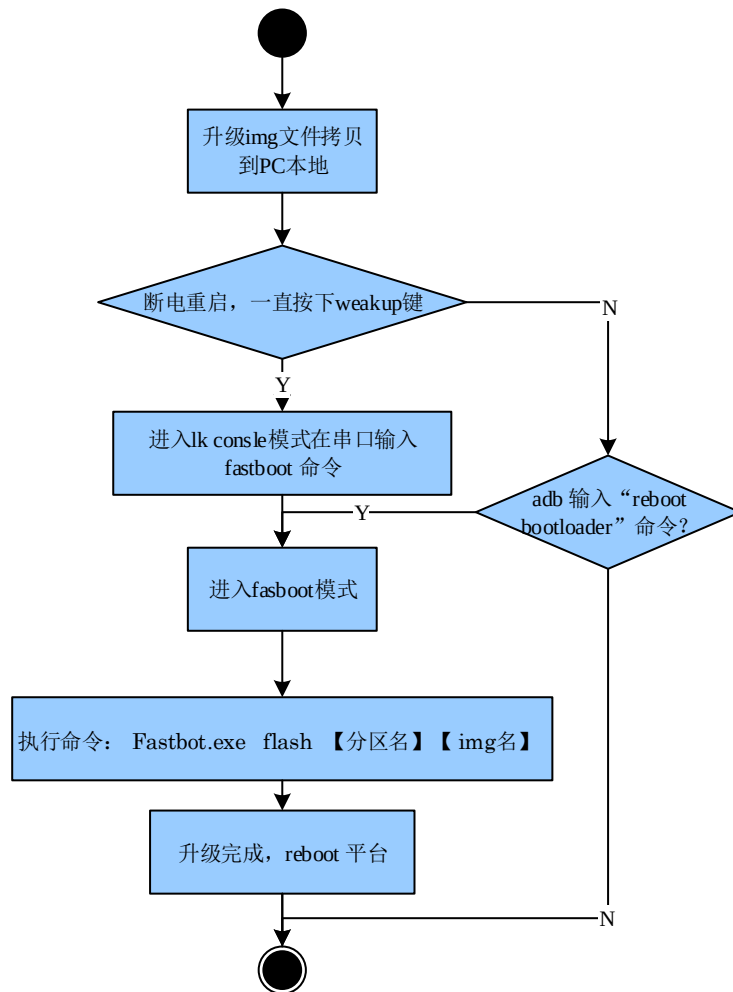


图 10-3 Fastboot 升级流程

- 1) 将需要升级的 img 文件拷贝到本地；
- 2) 可通过以下方式进入 fastboot 模式：
 - adb 输入 reboot bootloader 命令：
 - 连接 adb，执行命令：

```
adb root
adb shell
reboot bootloader
```

- 3) UI 显示 “enter fastboot mode” 字样，表示进入 fastboot 模式；
- 4) 执行 fastboot 的升级命令（见 10.3.2.2）；

5) 升级完，reboot 平台。

10.3 使用说明和注意事项

10.3.1 拷贝升级

10.3.1.1 分区发生改变说明

分区发生变化会触发 lk 全升级流程，分区发生变化包括四种变化，分别为：

- 分区数目发生变化
- 分区起始地址变化
- 分区 flag 发生变化
- 分区 size 发生改变

10.3.1.1.1 分区数目发生变化

分区数目发生变化，包括分区数目增加和删除。

分区数目删除变化如下所示（以删除 teedata 分区为例）：

原始 xml 文件：

```
<partition name="metazone" image_type="raw" fs_type="raw" uuid="D0AC6E2B-8F92-7815-B23B-4DC39CBD67A4" region="USER" startaddress="0x17f600000" size="0x200000"
imagename="metazone.bin" minium_system="Y" operation_type="update" flag="0xf" >
</partition>

<partition name="teedata" image_type="sparse" fs_type="ext4" uuid="CFC43833-C6D5-C854-7547-CF14AA31D0E9" region="USER" startaddress="0x17f800000" size="0x200000"
imagename="teedata.img" minium_system="N" operation_type="update" flag="0x4f" >
</partition>

<partition name="userdata" image_type="sparse" fs_type="ext4" uuid="C64B6FD6-5A07-08D4-74E4-3803C4CC385F" region="USER" startaddress="0x181800000"
size="0x140000000" imagename="userdata.img" minium_system="N"
operation_type="update" flag="0x4f" >
</partition>
```

变化后 xml 文件：

```
<partition name="metazone" type="raw" uuid="D0AC6E2B-8F92-7815-B23B-4DC39CBD67A4"
startaddress="0x17f800000" size="0x200000" imagename="metazone.bin" flag="0xf"
rw_attribute="RO" mount="0">

</partition>

<partition name="userdata" type="ext4" uuid="A233A921-8BA6-1C21-CB7A-469E3310E0EA"
startaddress="0x181800000" size="0x80000000" imagename="userdata.img" flag="0x4f"
rw_attribute="RO" mount="0">

</partition>
```

10.3.1.1.2 分区起始地址变化

分区起始地址发生变化，和分区 size 发生变化相似，因为分区 size 改变了，会导致下一个分区的起始地址改变。

原始 xml 文件:

```
<partition name=" metazone " image_type="raw" fs_type="raw" uuid=" D0AC6E2B-8F92-
7815-B23B-4DC39CBD67A4" region=" USER" startaddress="0x17f600000" size="0x200000"
imagename=" metazone.bin " minium_system="Y" operation_type="update" flag="0xf" >

</partition>

<partition name="teedata" image_type="sparse" fs_type="ext4" uuid=" CFC43833-C6D5-
C854-7547-CF14AA31D0E9" region="USER" startaddress="0x17f800000" size="0x2000000"
imagename="teedata.img" minium_system="N" operation_type="update" flag="0x4f" >

</partition>

<partition name="userdata" image_type="sparse" fs_type="ext4" uuid="C64B6FD6-5A07-
08D4-74E4-3803C4CC385F" region="USER" startaddress="0x181800000"
size="0x140000000" imagename="userdata.img" minium_system="N"
operation_type="update" flag="0x4f" >

</partition>
```

变化后 xml 文件:

```
<partition name=" metazone " image_type="raw" fs_type="raw" uuid=" D0AC6E2B-8F92-
7815-B23B-4DC39CBD67A4" region=" USER" startaddress="0x17f600000" size="0x200100"
imagename=" metazone.bin " minium_system="Y" operation_type="update" flag="0xf" >

</partition>

<partition name="teedata" image_type="sparse" fs_type="ext4" uuid=" CFC43833-C6D5-
C854-7547-CF14AA31D0E9" region="USER" startaddress="0x17f800100" size="0x2000000"
imagename="teedata.img" minium_system="N" operation_type="update" flag="0x4f" >

</partition>
```


downloadable 17 暂未支持

10.3.1.1.4 修改分区 Flag

为了方便修改 flag 的值，在 Partition Table 中新增如表 10-3 中 flag 属性，如果用户有对应的需求，就可以在分区表中设置对应的位值。



说明：

Flag 目前共有 18 个 bit 位，拷贝升级目前只判断了 copyupgrade 和 upgradeable 位。copyupgrade, upgradeable 值均设置为 1 则对应的分区会允许进行拷贝升级，否则不允许进行拷贝升级，则 LK 拷贝升级阶段不会升级该分区。

10.3.1.2 LK 升级操作流程

10.3.1.2.1 LK 升级触发方式

1. 通过 adb 输入命令：

```
adb root
adb shell
reboot lk_upg
```

2. UI 界面显示 “read gpt from flash...” “scan udisk...”等字样显示，则表示进入了 LK 升级模式；

3. 注意，在进入拷贝升级模式之前需提前在平台上插入升级 U 盘/SD 卡。

10.3.1.2.2 atc.upd 文件

在 img 文件夹中，有配置文件：atc.upd 文件，该文件主要是针对拷贝升级的属性进行配置的，内容如下：

```
< config reboot_after_upg="1" upg_all_slot="1" verify_data="0" log_level="2">
</config>
```

- reboot_after_upg 1 表示升级完自动重启 0 表示升级完成需要手动重启
- upg_all_slot 1 表示 A/B slot 都进行升级 0 表示只升级当前 slot
- verify_data 1 表示升级过程中对 raw 类型分区进行校验 0 表示不进行校验
- log_level 暂不支持

10.3.1.2.3 LK 升级的类型

LK 升级分为全升级和部分升级，升级的类型是根据升级使用的分区表是否变化来判断的。

LK 升级支持分区表不发生变化的升级（部分升级），操作步骤为：

1. 把 U 盘或 SD 卡格式化为 FAT32 的文件系统。在 U 盘或 SD 卡根目录下创建新目录并命名为 ATCUPG。Copy 对应需要升级的 image 文件和 atc.upd 到 U 盘或 SD 卡的 ATCUPG 目录，必须要包含 scatter file 文件，否则升级失败。然后再把 U 盘或 SD 卡插入到平台端。
2. 上电启动平台，通过 adb shell reboot lk_upg 命令进入拷贝升级模式，待进入到升级流程，即界面刷出升级的 UI 界面。
3. 进入升级流程后，会在屏幕上打印出相应的升级期间的信息，成功后，系统会自动重启。

LK 升级支持分区表发生变化的升级（全升级），操作步骤为：

1. 把 U 盘或 SD 卡格式化为 FAT32 的文件系统。在 U 盘或 SD 卡的根目录下创建新目录并命名为 ATCUPG。Copy 对应需要升级的全部 image 文件和 atc.upd 文件到 ATCUPG 目录，必须要包含变化的 scatter file 文件（scatter file 如果没有变化，进行部分升级）。把 U 盘或 SD 卡插入到平台。



注意：

如果各个分区 image file size 不超过分区 size 的，也会升级成功，可能会出现一些启动异常的现象。如果分区 image file size 超过分区 size 则会出现升级失败。（变化的 scatter file 是指和板子中烧写的 scatter file 在分区 layout 上有所不同）。

2. 上电启动平台，通过 adb shell reboot lk_upg 命令进入拷贝升级模式，待进入到升级流程，即界面刷出升级的 UI 界面后。
3. 进入升级流程后，会在屏幕上打印出相应的升级期间的信息，成功后，系统会自动重启。

10.3.1.2.4 LK 拷贝升级注意事项

- 只支持 FAT32 格式的 U 盘或者 SD 卡。
- 多分区的外设只支持第一个分区是 FAT32。
- 如果同时插入多个设备，例如 U 盘和 SD 卡都有插入，默认会先扫描 U 盘。如果 U 盘扫描到对应的升级文件，就会跳出扫描流程，然后开始升级。如果 U 盘多个口都有插入，会按照第一个扫描到的有升级文件的 U 盘为主，如果扫描到就退出扫描流程，扫描不到，继续下一个 U 盘。扫描顺序是 otg 口->xhci 口。如果没有扫描到 U 盘中相应的升级文件，会继续扫描 SD 卡，待扫描到对应的

升级文件后跳出，进行升级流程。部分升级也有进行分区表更新，主要是更新了 raw image 的 real size，该 real size 为系统启动期间 load image 时候使用。

- 如果外置存储卡（U 盘或者 SD 卡）里面没有放入相应的 scatter file 文件和 atc.upd，默认升级失败。
- 升级的擦除流程是升级到哪个分区就开始擦除哪一个分区，然后再升级该分区。

10.3.2 fastboot 升级

10.3.2.1 fastboot 升级触发方式

进入 fastboot 有以两种方式：

- reboot bootloader 命令：流程上类似于 reboot recovery，通过 lk 启动阶段读取 misc 分区的 BCB 判断是否进入 fastboot 模式。
- recovery 模式进入 fastboot：点击 reboot bootloader 选项，通过 SetProperty(ANDROID_RB_PROPERTY,cmd)函数传递 fastboot cmd，重启进入 fastboot 模式。

10.3.2.2 升级命令

在 fastboot 模式下，通过 cmd 面板输入命令：

```
fastboot flash 【分区名】 【img 名】
```

如升级 metazone，执行以下命令：

```
fastboot flash metazone metazone.bin
```

10.3.2.3 注意事项

升级完成的可以通过 log（“fastboot start flash img!” 和 “fastboot flash img success”）查看升级完成。当升级完成后，可以手动重启，也可以通过命令：fastboot reboot 进行重启。

AVB 版本在执行升级 img 命令之前，需要执行命令：fastboot flash unlock。

10.4 用户定制化

在 AC8025 Android S 中，目前不支持按键进行触发 LK 升级流程。

10.5 典型问题

10.5.1 拷贝升级

10.5.1.1 U 盘/SD 卡多分区问题

【问题描述】

U 盘/SD 卡多分区支持情况。

【解决方案】

对于多分区的外设，只支持如下情况：

- 第一个分区为 FAT32 格式；
- 所需升级的 image 存放在第一个分区。

10.5.1.2 升级期间擦除 eMMC 的操作问题

【问题描述】

有时用户在升级的时候，希望保留部分分区内容而擦除其他分区的内容。升级期间，是否有擦除 eMMC 的操作？如果有，如何擦除？

【解决方案】

升级期间，要检查是否有擦除 eMMC 的操作，只需要看是否该分区有 image 文件，该 image 文件是否被升级，若该 image 文件升级，则分区被擦除。

升级期间会对 eMMC 进行擦除，即升级每一个分区之前，会先对每一个分区进行 erase，然后再进行升级。如果部分升级，对于没有分区 image file 的分区，不会进行擦除。

10.5.1.3 如何判断已经进入到升级流程问题

【问题描述】

如何判断进入到 LK 拷贝升级流程？

【解决方案】

可以通过 LK 拷贝升级的 UI 界面来确认，进入升级流程的 UI 如下所示：

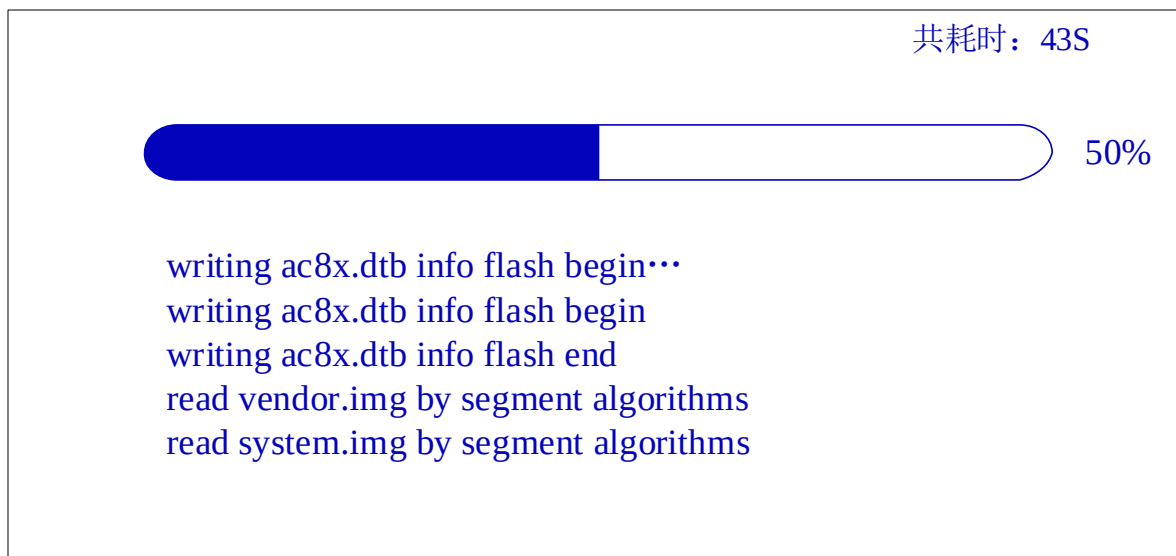


图 10-5 LK Copy Upgrade Display UI

该图中有具体的升级时间，进度百分比，还有升级期间升级分区信息的打印。升级失败后，会停在该界面，并打印升级失败信息。如果升级成功，会自动重启，并进入到正常系统流程中。

10.5.1.4 进入升级流程后，如何再次返回到正常系统？

【问题描述】

在 LK 升级期间，如果需要再次回到正常系统，该怎么操作？

【解决方案】

如果升级检测阶段，即没有进入到升级流程阶段，系统 hang 在升级 UI 界面，并打印出相应的失败信息和错误码，只需要重启后，即可回到正常系统界面。

如果在升级流程开始之后拔卡或者升级失败，hang 在升级 UI 界面拔卡，会概率性无法进入到正常系统。因此，为了保证升级的安全性，防止系统变砖，需要客户保证足够的电量，并避免升级期间异常情况的发生。

10.5.1.5 如何判断是否触发 LK 拷贝升级流程

【问题描述】

如何判断是否进入到 LK 拷贝升级流程？

【解决方案】

通过界面查看，通过 adb 输入 reboot bootloader 命令后，系统重启出现如图 10-5 所示的界面，说明已经触发进入到了 LK 拷贝升级流程。

10.5.1.6 如何判断部分升级还是全部升级

【问题描述】

前面章节介绍了支持的升级功能以及如何修改 xml 来进行相应的升级流程，如何判断是否进入了相应的升级流程中？

【解决方案】

- **方法一：**通过 log 查看

操作过程中通过接串口，保存相应的串口 log，然后通过关键字“UPG”，搜索 log，出现如下 log 说明是部分升级：

```
[UPG] Note:upg_main_task, scatter gpt and flash gpt ranges are the same, need to take partial upg.
```

```
[UPG] Info:upg_main_task, ---->take partial upg begin...
```

出现如下 log，说明是全部升级流程。

```
[UPG] Info:upg_main_task, begin to take full upg.
```

- **方法二：**通过升级 UI 界面可以查看

升级期间，在升级 UI 显示出来的靠前的时候，会通过屏幕打印相应的 log 在屏幕上，通过显示：

```
take partial upg begin
```

如果升级 UI 上有打出如下 log，说明是全部升级。

```
begin to take full upg.
```

10.5.1.7 如何判断 LK 升级是否成功

【问题描述】

如何判断 LK 升级是否成功？

【解决方案】

- **方法一：**从 log 看，如果 log 中有类似“upg_main_task upgrade successful!!!!!!” log，说明升级成功。
- **方法二：**从升级界面看，如果升级界面显示一直在升级，升级一段时间后，系统自动重启并能进入到正常系统，说明升级无异常。

10.5.2 Fastboot 升级

10.5.2.1 如何判断 fastboot 升级是否成功？

【问题描述】

如何判断 fastboot 升级是否成功？

【解决方案】

从 log 看，如果 log 中有类似如下 log，说明升级成功。

“fastboot start flash img!” 和 “fastboot flash img success” 的数量一致。

10.5.2.2 执行 fastboot 命令不成功怎么办？

【问题描述】

执行 fastboot 命令不成功怎么办？

【解决方案】

检查本地是否安装 fastboot 驱动，通过 fastboot -help 查看 fastboot 是否支持输入的命令。

11 OTA 升级

11.1 简介

11.1.1 功能

动态分区 OTA 升级。

注意事项：

- 升级方式：OTA 在线升级依赖 OTA 服务器，ATC 仅对 AC8025 系统升级框架实现和说明，不提供 OTA 服务器搭建方案。目前提到的 OTA 升级主要基于两种方式：一是通过 U 盘或 SD 卡进行升级，二是通过 PC 端 adb 命令进行升级。这两种方式均支持整包和差分包升级；
- 当前支持动态分区 OTA 升级，支持动态分区组内分区 size 的调整；
- 不能升级当前正在运行中的 Slot。

11.1.2 代码结构及文件描述

AB OTA 升级主要代码路径：

表 11-1 AB OTA 代码结构及描述

文件名	文件路径	文件描述
allmake_split.sh	ivi/	触发 OTA 包编译
ota_from_target_files.py	ivi/android/build/make/tools/releasetools/	打包 target_files 制作 OTA 包
generate_delta_main.cc	ivi/android/system/update_engine/payload_generator	制作 payload.bin
delta_performer.cc	ivi/android/system/update_engine/payload_consumer	解析 payload.bin 进行升级

11.2 软件设计

11.2.1 架构图

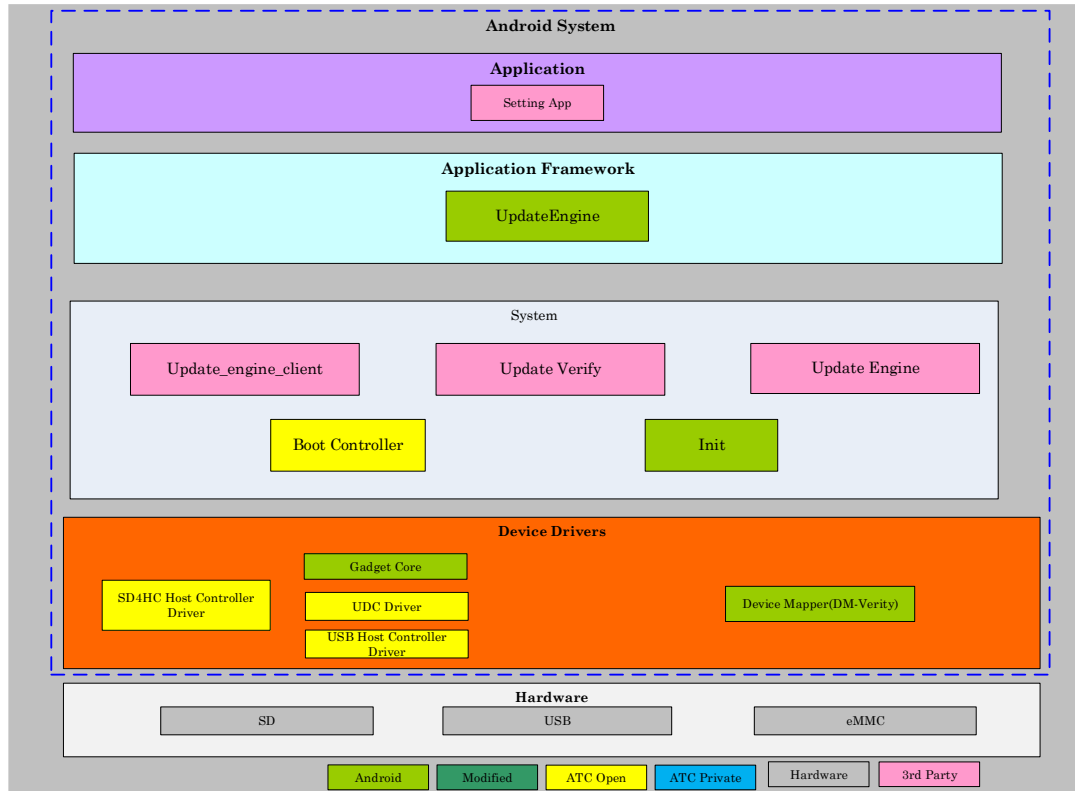


图 11-1 AB OTA 升级架构图

图 11-1 所示的架构图分为四层：

- 最上层为触发升级的入口 App，负责读取 U 盘中的升级文件，显示升级进度以及升级结束后通知 powermanager 重启平台；
- 中间的 framework 层负责创建 UpdateEngineCallback 的对象，开启 update_engine 升级服务，继续向下调用 update_engine 执行升级；
- update_engine 升级服务会依次执行 4 个升级动作，解析 payload.bin，将升级数据写到对应的分区；
- 第 4 层为 driver 层，负责处理上层下发的 block 读写请求，读写平台 eMMC/UFS 上的数据。

11.2.2 流程图

11.2.2.1 AB OTA 升级流程

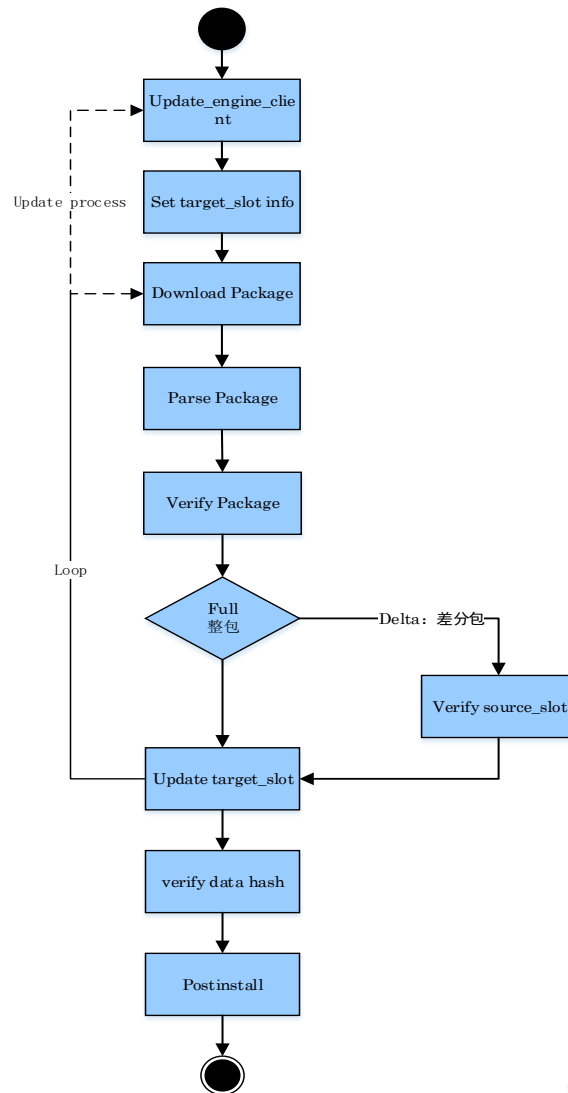


图 11-2 AB OTA 升级流程图

update_engine 是 A/B 升级的核心逻辑，用于执行下载和升级，路径：ivi/android/system/update_engine/，其中 update_engine_client 是客户端进程，用来解析命令行的各种操作(suspend/resume/cancel/reset_status/follow/update)，并将这些操作和参数通过 binder 机制，转发为对服务端进程 UpdateEngineService 相应操作的调用。

update_engine 升级会依次执行一下四个升级动作：

InstallAction, DownloadAction, FilesystemVerfierAction, PostinstallRunnerAction

其中：

- InstallAction 的任务就是构建记录升级参数和状态的对象给其它 Action 使用；
- DownloadAction 负责下载升级数据，并将其更新到对应的分区；
- FilesystemVerifierAction 会读取更新分区的所有数据计算 Hash，并和期望的 Hash 数据进行比较，若一致则下载更新成功，否则会报错；
- PostinstallRunnerAction 会执行单个分区的 post install script 脚本，若对应分区无此脚本，则跳过执行。

OTA 升级结束重启前会通过 bootctl hidl 层接口切换下一次的启动 slot，将升级的 target slot 设置成 active 状态。

11.2.3 AB 分区信息

11.2.3.1 AB 分区描述

当前 Android 系统采用 AB 双分区方案，部分分区会采用 A/B 分区存放在 eMMC/UFS 上，运行时分为 slotA 和 slotB，第一次烧写系统默认会从 slotA 启动。

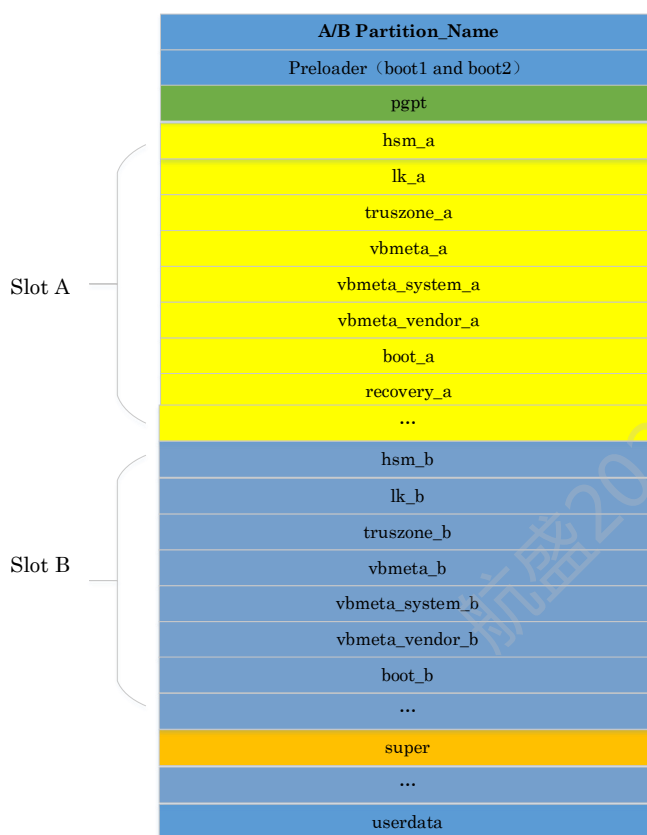


图 11-3 AB 分区示意图

11.2.3.2 Bootctl slot 信息

使用 metadata 参数来标识 Slot 分区的状态，该部分数据存放在 misc 分区中。

图 11-4 反映了各个场景下 Slot A 和 Slot B 中的 metadata 参数的变化。

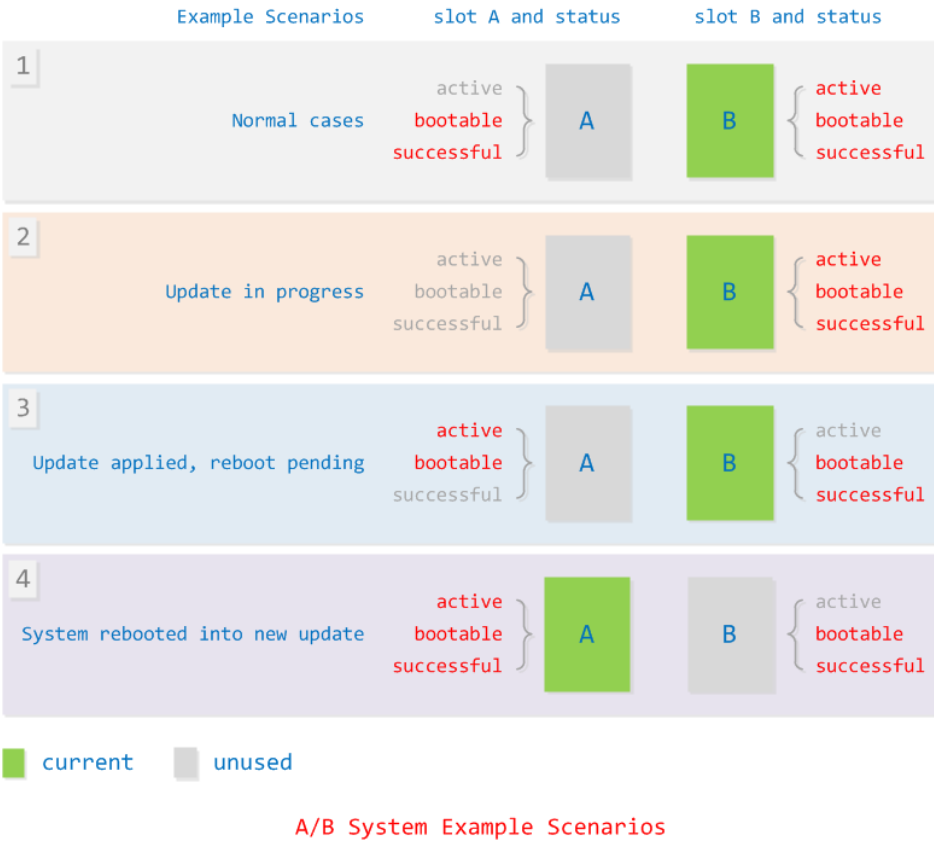


图 11-4 Slot 属性图

1. 在正常情况下（Normal case）当前系统运行的是 Slot B 中的系统，所以图中可以看到 Slot B 是 Active（激活）的，Bootable（可启动的），successful（已经成功启动）的。Slot A 中是一个旧版本的系统，可以看到该 Slot 处于不可用状态，Active 为 false。
2. AB 升级（升级到 Slot A）过程中，Slot A 的三个参数全部被置成 false 的状态。
3. AB 升级完成后，还未重启，此时 Slot A 中的 image 已经被更新，Slot A 的 bootable 与 active 被设置为 true，表示系统已经更新，下次系统重启时会从 Slot A 中启动。Slot B 的 active 被设置为 false，表示重启时不会从 Slot B 中启动。
4. 当系统从 Slot A 中成功启动后，Slot A 的 successful 被设置为 true。

当前 Active、Bootable、Successful 三种状态在 8025 实现时，会使用其他几个参数组合来实现。Successful 状态表明 successful_boot 属性为 1，retry_count 大于 0。Active 状态表明当前 Slot 的 priority 属性比另外 Slot 高，并且 normal_boot 为 1。Bootable 状态表明当前 priority 大于 0。

```
typedef struct slot_metadata {
```

```
uint8_t priority : 3
uint8_t retry_count : 3
uint8_t successful_boot : 1
uint8_t normal_boot : 1
} slot_metadata_t
```

举例说明，如果当前系统运行在 slot A 上，slot 信息如下图所示。

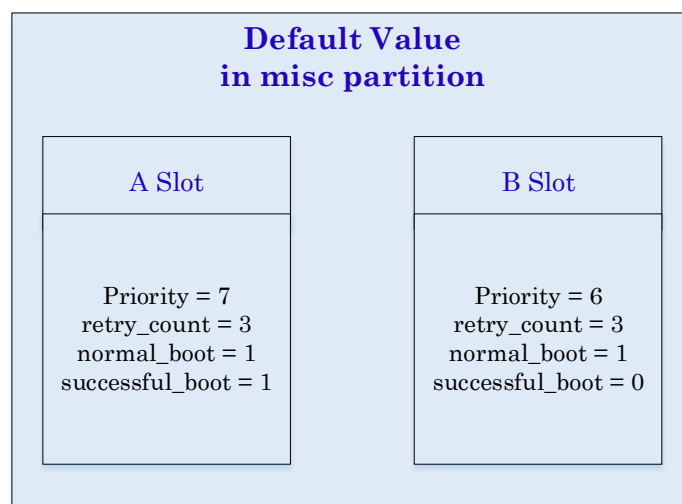


图 11-5 默认属性

在 OTA 升级完成后，平台将从 Slot B 启动。此时 Slot A 和 Slot B 均可以启动，但是 Slot B 比 Slot A 具有更高的优先级。

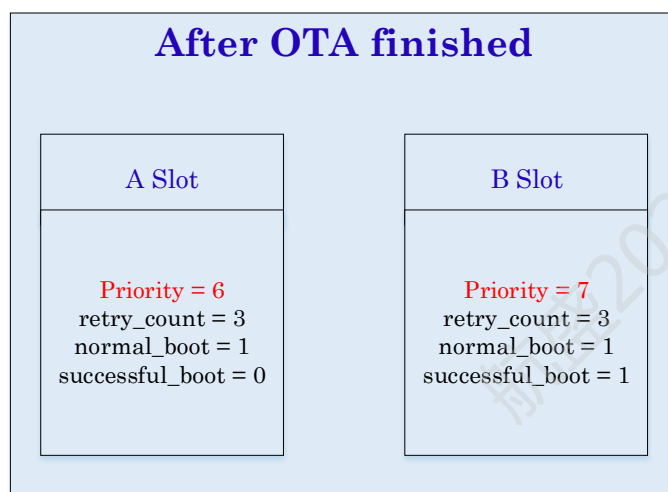


图 11-6 升级完成后的属性

11.3 使用说明和注意事项

11.3.1 AB OTA 分区添加

1. 当前加入 AB 分区的只读分区需要打包到 OTA 包进行 OTA 升级，并且 `imagename` 需要和分区名保持一致并且以 `.img` 后缀结尾。
2. 除了分区表中的 AB 分区需要配置正确的值，`source/android/device/autochips/system/assi_64/device.mk` 也需要添加对应的 AB 分区配置。

- a) 首先，为了区分不同产品形态，在 `build/projects/ivi_s_ac8025/storage_config.json` 和 `build/projects/cockpit_s_linux_ac8025/storage_config.json` 分别定义了除 Android 之外的 AB 分区 `list`，编译脚本会解析 `json` 文件中的 `"ab_ota_partitions"` 填充到环境变量 `AB_OTA_PARTITIONS_ADD` 中，如下所示：

`build/projects/ivi_s_ac8025/storage_config.json`

```
{
  "ab_update": "true",
  "ota_update": "false",
  "dynamic_partition": "yes",
  "ab_ota_partitions": "preloader,hsm,lk"
}
```

`build/projects/cockpit_s_linux_ac8025/storage_config.json`

```
{
  "ab_update": "true",
  "ota_update": "false",
  "dynamic_partition": "yes",
  "ab_ota_partitions": "preloader,hsm,lk,aah,aah-dtb,aah_recovery-dtb,cluster-boot,cluster-vbmeta,cluster-dtb,cluster-system,adas-boot,adas-vbmeta,adas-dtb,adas-system"
}
```

- b) 其次，`ivi/android/device/autochips/system/assi_64/device.mk` 中的 `AB_OTA_PARTITIONS` 会定义一些 Android 的 AB 分区，并且解析 `AB_OTA_PARTITIONS_ADD` 变量，最终编译 OTA 时根据 `AB_OTA_PARTITIONS` 生成 `ab_partitions.txt` 文件，完整记录当前项目需要加入 AB OTA 包的分区

```

123 ifeq ($(AB_OTA_UPDATER),true)
124     ATC_TABLET_EMMC_ACTIVE_BOOT_PART = yes
125     # default A/B OTA partitions
126     AB_OTA_PARTITIONS := \
127     vendor \
128     system \
129     boot \
130     vbmeta \
131     vbmeta_vendor \
132     vbmeta_system \
133     vendor_dkkm \
134     vendor_boot
135     ifdef BOARD_PRODUCTIMAGE_FILE_SYSTEM_TYPE
136         AB_OTA_PARTITIONS += product
137     endif
138
139     ifeq ($(ENABLE_OTA_UPDATE),yes)
140         AB_OTA_PARTITIONS += boot_misc
141         AB_OTA_PARTITIONS += dtbo
142         AB_OTA_PARTITIONS += trustzone
143         AB_OTA_PARTITIONS += recovery
144     ifneq ($(AB_OTA_PARTITIONS_ADD), )
145         AB_OTA_PARTITIONS += $(subst $(comma),$(space),$(AB_OTA_PARTITIONS_ADD))
146     endif
147     endif

```

- 打包 target_files.zip 之前，首先在 ivi/allmake_split.sh 中将除了 android 的 img 拷贝到 android 的 out/target/product/\$TARGET_DEVICE/topImages 目录下。

```

_option_before() {
    _info "_option_before"
    if [ x"$ENABLE_OTA_UPDATE" = x"yes" ]; then
        # prepare ab_ota partitions
        _info "#### AB_OTA_PARTITIONS_ADD = $AB_OTA_PARTITIONS_ADD ####"
        parts=(${AB_OTA_PARTITIONS_ADD//,/ })
        for part in ${parts[@]}
        do
            _info "#### ready to copy ${part} img to product_out ####"
            mkdir -p $PRODUCT_OUT/topImages
            if [ $part == "preloader" ]; then
                cp ${TOP_BSP_PATH}hl_avb_verified.img $PRODUCT_OUT/topImages/
                cp ${TOP_BSP_PATH}preloader.img $PRODUCT_OUT/topImages/
            elif [ $part == "cluster" ]; then
                cp ${TOP_CLUSTER_PATH}${part}.img $PRODUCT_OUT/topImages/
            elif [ $part == "adas" ]; then
                cp ${TOP_ADAS_PATH}${part}.img $PRODUCT_OUT/topImages/
            elif [ $part == "lk" ]; then
                cp ${TOP_BSP_PATH}${part}.img $PRODUCT_OUT/topImages/
            elif [ $part == "hsm" ]; then
                cp ${TOP_SAFETY_PATH}${part}.img $PRODUCT_OUT/topImages/
            elif [ $part == "aah-dtb" ]; then
                cp ${TOP_TOOLS_PATH}${part}.img $PRODUCT_OUT/topImages/
            elif [ $part == "aah_recovery-dtb" ]; then
                cp ${TOP_TOOLS_PATH}${part}.img $PRODUCT_OUT/topImages/
            elif [ $part == "aah" ]; then
                cp ${TOP_BSP_PATH}${part}.img $PRODUCT_OUT/topImages/
            elif [ $part == "datazone-ro" ]; then
                cp ${TOP_BSP_PATH}${part}.img $PRODUCT_OUT/topImages/
            fi
        done
        _info "#### cp Image done ####"
    fi
}

```

- 如果需要新增分区到 OTA 包，需要确保该分区的 img 拷贝到 target_files 目录下/IMAGES 目录，具体可以参考 ivi/android/device/autochips/build/tasks/tools/split_vendor.mk 中的拷贝文件操作。

```

28     $(ZIPSYNC) -d $(zip_root) $(BUILT_TARGET_FILES_PACKAGE)
29     rm -f $(zip_root)/BOOT/dtb $(zip_root)/BOOT/kernel $(zip_root)/RECOVERY/dtb $(zip_root)/RECOVER
)/IMAGES/boot.img $(zip_root)/IMAGES/system.img $(zip_root)/IMAGES/product.img $(zip_root)/IMAGES/syst
ot)/PREBUILT_IMAGES/dtbo-verified.img
30     #rm -rf $(zip_root)/PRODUCT $(zip_root)/SYSTEM $(zip_root)/SYSTEM_EXT $(zip_root)/SYSTEM_OTHER
31     rm -rf $(zip_root)/PRODUCT $(zip_root)/SYSTEM_EXT $(zip_root)/SYSTEM_OTHER
32     cp $(PRODUCT_OUT)/topImages/*.img $(zip_root)/IMAGES/
33     cp $(PRODUCT_OUT)/trustzone.img $(zip_root)/IMAGES/
34     cp $(PRODUCT_OUT)/dtb.img $(zip_root)/IMAGES/
35     cp $(PRODUCT_OUT)/boot.img $(zip_root)/IMAGES/
36     cp $(PRODUCT_OUT)/recovery.img $(zip_root)/IMAGES/

```

5. 新增定制化分区到 OTA 包中，需要按照步骤 1~4 的操作将准备打包的 img 提前拷贝到 target_files.zip 中，最终编译脚本 split_build.py 会根据 target_files.zip 中的 images 以及 AB_OTA_PARTITIONS 的配置制作 OTA 包。

6. 另外，新增分区到 OTA 包进行 OTA 升级时，需要配置一下对应分区的 sepolicy 规则。如果常规权限配置之后还有特殊的 selinux 权限问题，可以根据 avc denied 的 log 提示配置对应的权限。以 vendor_boot 分区的权限为例：

a) 配置 source/android/device/autochips/sepolicy/non_plate/device.te，新增 device type

```
222 type vendor_boot_block_device, dev_type;
```

b) 配置 source/android/device/autochips/sepolicy/non_plate/file_contexts，前面一系列的具体路径以实际平台访问的分区节点路径为准，一般为/dev/block/xxx 或者 /dev/block/platform/soc/12400000.sd0/by-name/xxx

```
203 /dev/block/platform/soc/12400000.sd0/by-name/vendor_boot(_[ab])? u:object_r:vendor_boot_block_device:s0
```

c) 配置 source/android/device/autochips/sepolicy/non_plate/update_engine.te，这个文件是 update_engine 升级服务更新具体分区时需要配置的 sepolicy 规则

```
117 allow update_engine vendor_boot_block_device:blk_file { open read write ioctl getattr };
```

11.3.2 AB OTA 包编译

在源代码根目录，制作 ivi_s_ac8025 的 OTA 包命令如下：

```
python3 allmake.py build -p ivi_s_ac8025 -a dynamic_partition =True,allota=True
```

其中-p 表示 project 为 ivi_s_ac8025，ab_update 为 True 表示编译 AB 版本，allota 为 True 表示编译 OTA 包。如果要编译 project 为 cockpit_s_linux_ac8025 的 OTA 包，将-p 后的参数修改成 cockpit_s_linux_ac8025 即可。

1. 当前，执行上述命令默认会编译两次 OTA 整包生成 otabase1.zip 和 otabase2.zip，然后通过脚本自动制作出 OTA 差分包。编译完成后，在 release_images/ivi_s_ac8025/目录下会有 OTA 整包和差分包同时生成，其中 OTA 包名称包含 full 的为 OTA 整包，otadiff.zip 是自动编译的 OTA 差分包。

2. 如果需要修改代码手动制作 OTA 差分包，需要通过以下步骤：

- 1) 按照上面的命令进行一次 OTA 编译，导出 release_images 路径的 otabase1.zip 为 V1.zip;
- 2) 修改需要更新的模块代码;
- 3) 重复步骤 1)，导出 otabase1.zip 为 V2.zip;
- 4) 将 V1.zip 和 V2.zip 移到 source/android/目录下，并且将通过如下命令制作 OTA 差分包:

```
i) source build/envsetup.sh
ii) lunch sys_assi_64-userdebug (以 userdebug 版本为例，具体根据 device mode)
iii) out_sys/host/linux-x86/bin/ota_from_target_files -v -p out_sys /host/linux-x86/ --metazone_path $path1 --datazone_path $path2 -i V1.zip V2.zip otadiff.zip
```

其中 path1 和 path2 分别为需要打包到 OTA 差分包中 metazone.bin 和 datazone-rw.img 的路径。

3. 当前代码实际区分 normal 平台和 AVB 平台是通过平台是否烧写了 efuse 实现的，因此软件需要烧写不同的 preloader 来区分。当前 OTA 升级会打包 preloader 分区 image，因此 OTA 编译打包需要区分 non-AVB 和 AVB 版本的 OTA 包。可以通过 OTA 的包名字是否包含“AVB”关键字来区分不同版本的 OTA 升级包。

11.3.3 AB 分区命名规则

A/B 所有分区应命名如下（插槽始终被命名为 a、b 等）：boot_a、boot_b、hsm_a、hsm_b、lk_a、lk_b 等。生成对应的 image 文件需要以分区名+.img 结尾，例如：boot_a 分区的 image 名字为 boot.img。

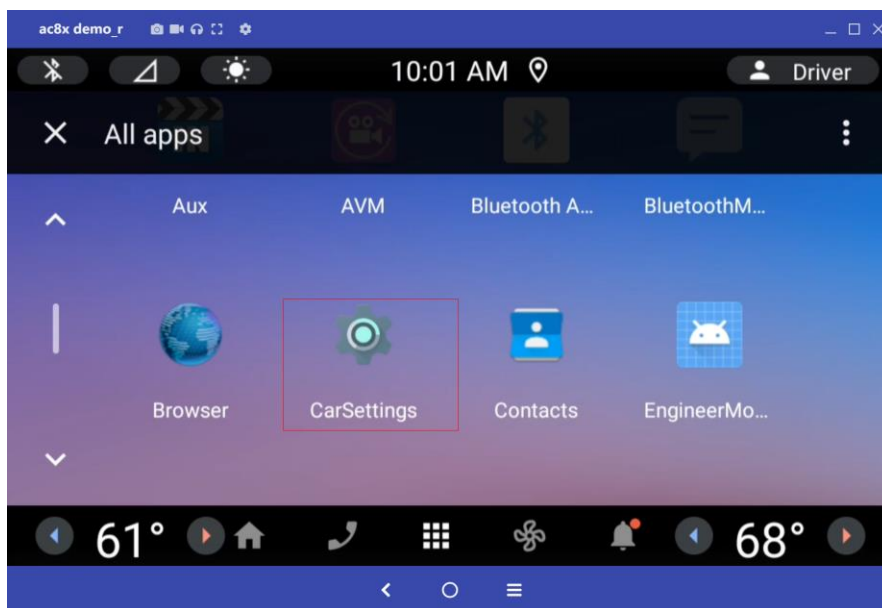
11.3.4 AB OTA 升级步骤

AB OTA 无感升级分为手动和命令两种触发方式，其中手动是通过 Android 原生的升级 apk SystemUpdater 调用 update_engine 服务提供的接口进行升级，命令触发是通过 update_engine_client 和 update_engine 升级服务间的通信进行升级。

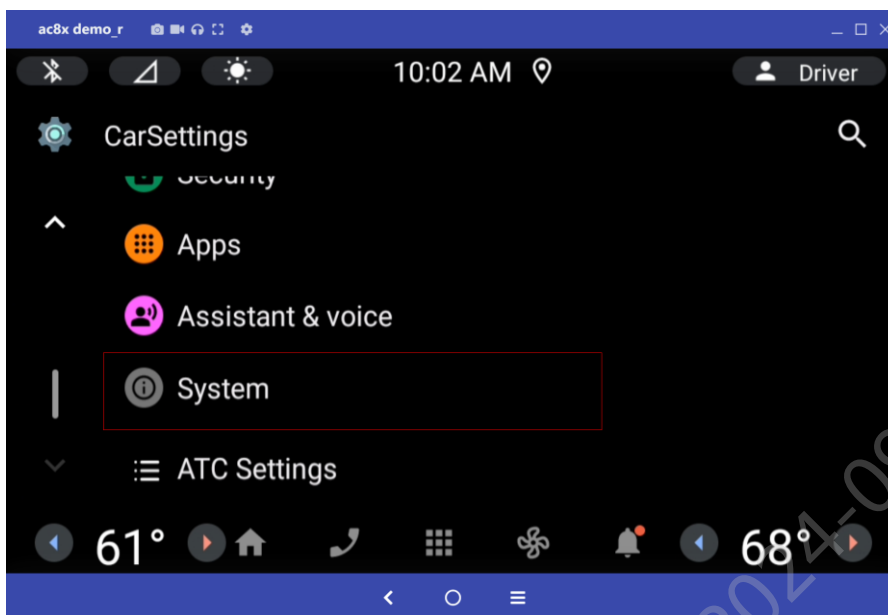
11.3.4.1 手动触发方式

通过选择“CarSettings->System->Local system Update”后进入到从 U 盘/SD 卡选择界面，选择对应的存储设备后找到对应 OTA 包，然后开始升级，升级成功后，系统自动重启，进入到 normal 系统，如果升级失败，会有相应的失败信息显示在屏幕上。AB OTA 无感升级时系统可以正常运行，待升级结束会自动重启运行新升级的系统。

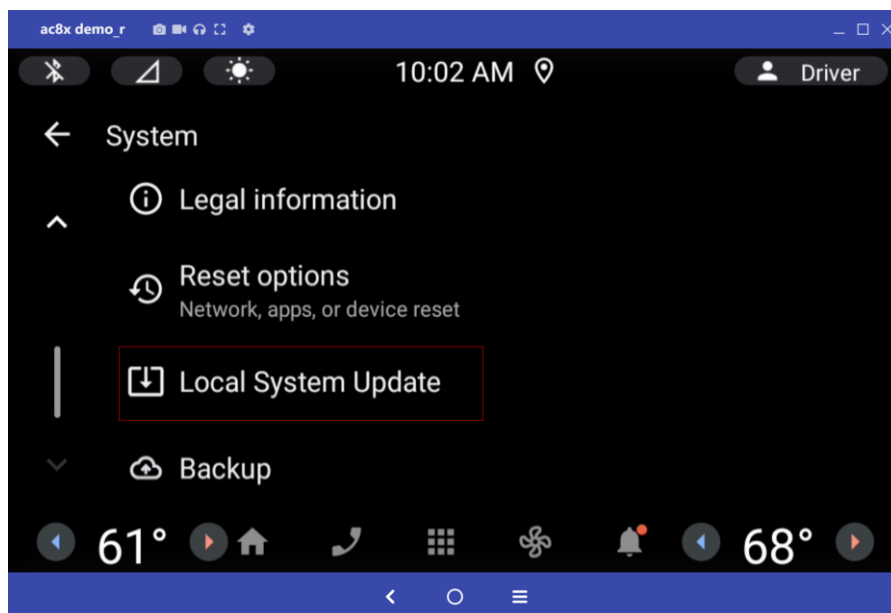
1. 系统启动到 home 界面，选择“CarSettings”App，然后点击：



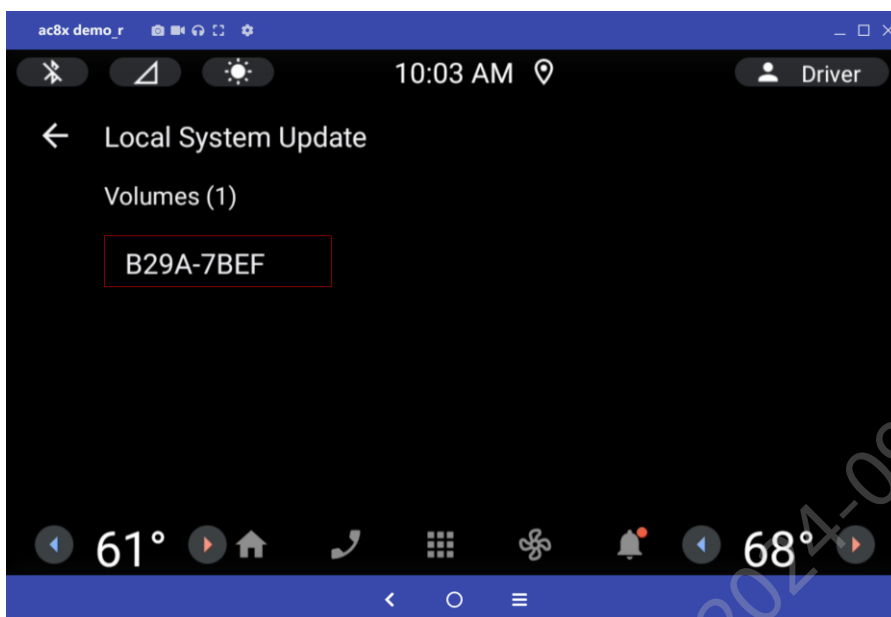
2. 选择“System”，点击：

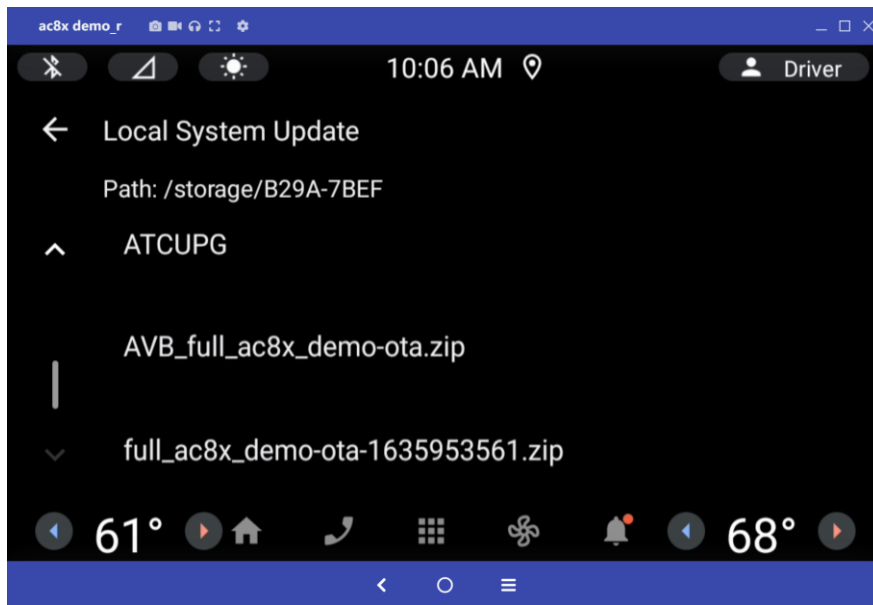


3. 选择“Local System Update”，点击：

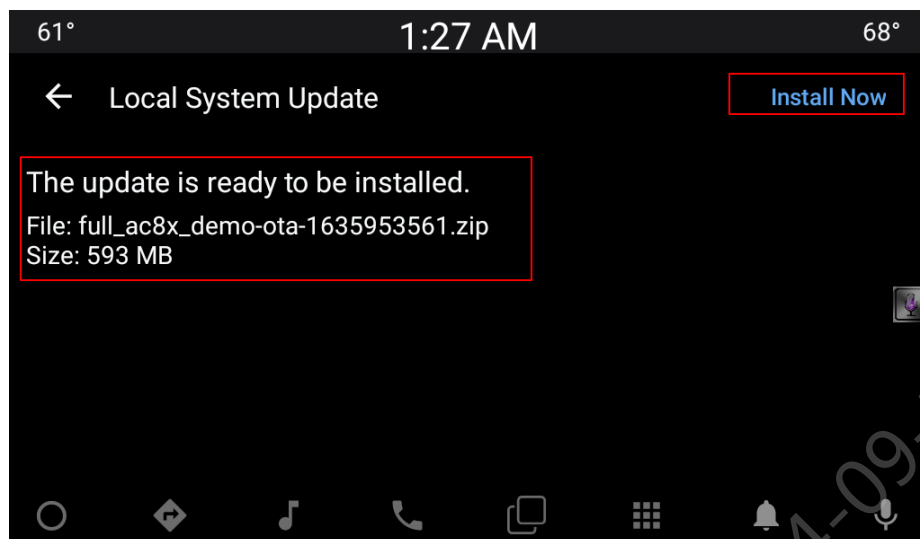


4. 进入存储盘选择界面，点击外置存储设备“B29A-7BEF”：

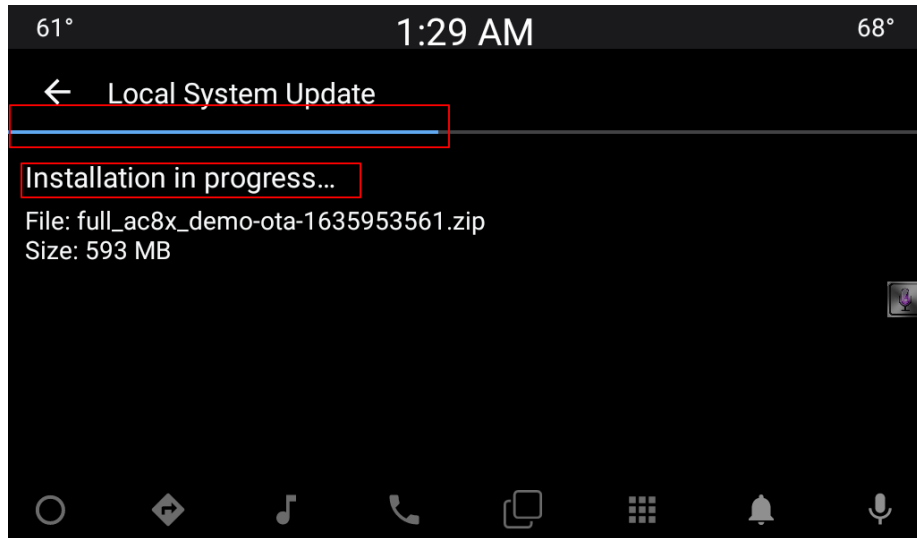




5. 进入到升级显示界面，点击“INSTALL NOW”：



6. 开始升级



7. 升级成功后自动重启进入到正常系统的 home 界面。

11.3.4.2 命令触发方式

1. 首先将 AB OTA zip 包解压，如下图所示：

名称	大小	压缩...
META-INF	3 027	2 295
apex_info.pb	444	444
care_map.pb	484	484
datazone_rw.img	50 856 960	1 166 ...
metazone.bin	1 048 576	2 034
payload.bin	737 145 552	737 1...
payload_properties.txt	154	154

简要说明一下 OTA 升级包中的文件：

- META-INF 存放了 OTA 升级校验的 metadata 和签名认证相关文件
- payload.bin：主要的升级文件，一些升级文件会进行拼接到该文件中，可以通过具体的工具进行查看，例如 payload_dumper 等工具。
- payload_properties.txt：升级文件附带的一些属性
- datazone_rw.img 和 metazone.bin 是通过写 zip 包的方式打包到 OTA 包中，通过 OTA 升级的途径实现更新 metazone 和 datazone 的目的

2. 升级包准备：将 payload.bin(OTA 升级包解压得到)文件 push 到本地存储设备路径，进入 adb shell 环境，升级命令如下：

```
update_engine_client \
--payload=file:///storage/ B29A-7BEF /payload.bin \
```

```
--update \
--headers="\
FILE_HASH=FZm0JuN+YB3oej1UQZ09YERZkq3oyT7B0cw2Xgg0MWc=
FILE_SIZE=608320848
METADATA_HASH=EyD0NykyR7w9972shbEjpS4UKFQgG8KfaeVULXStMqI=
METADATA_SIZE=60624
"
```

其中 FILE_HASH, FILE_SIZE, METADATA_HASH, METADATA_SIZE 均在 payload_properties.txt 文件中。



注意：

使用 update_engine_client 命令方式进行 AB OTA 升级结束，系统并不会自动重启，如果通过 adb 终端或者 log 观察到升级结束，可以手动重启平台。

11.4 典型问题

11.4.1 如何知道当前系统是哪一个 Slot

【问题描述】

如何知道当前系统运行的是哪一个 Slot?

【解决方案】

- 方式一：获取系统属性：ro.boot.slot_suffix，例如可以通过读取 /proc/cmdline 中的 androidboot.slot_suffix 字段。
- 方式二：通过 adb 命令 adb shell getprop ro.boot.slot_suffix，例如：

```
E:\tool\Fastboot\Android>adb shell getprop ro.boot.slot_suffix
_a
```

- 方式三：通过 adb 命令 adb shell bootctl get-current-slot，0 代表 slot A，1 代表 slot B

11.4.2 如何手动切换 slot

【问题描述】

如何手动切换系统运行的 Slot?

【解决方案】

通过 adb 命令，假设当前运行在 slot A（即 slot 0）

```
bootctl get-current-slot //获取当前运行 slot
```

```
bootctl set-active-boot-slot 1 //重启系统，切换到 slot B 运行
```



注意：

首次通过 flash tool 或者制卡升级完后的系统，请不要直接切换到 slot B，因为升级 Tool 默认不会升级 slot B 上的一些分区，所以直接切换到 slot B 会导致系统异常。

11.4.3 哪些分区需要做成 AB 分区

【问题描述】

什么分区需要做成 AB 分区？

【解决方案】

1. 该分区后期需要更新，设备出去以后如果存在 bug，需要通过 OTA 修复的；
2. 该分区数据比较重要，损坏后会变砖，这种分区建议增加 AB 分区；
3. 该分区必须只读，否则差分升级会失败，如果该分区需要读写，建议将只读的数据和读写数据进行隔离，存放在不同分区。

11.4.4 Slot 的 successful 状态设置

【问题描述】

系统升级启动成功后，哪里设置的 Slot 的 successful 状态？

【解决方案】

Android 系统 Home ready 之后会启动 update_verifier service，在 verify success 后，设置 boot_successfully flag。

源码位置：source/android/bootable/recovery/update_verifier/update_verifier.cpp

11.4.5 A/B 升级过程中异常终止

【问题描述】

AB 无感升级过程中若返回升级界面或断电重启后再进入无感升级，会接着上次未完成任务继续还是重新开始？

【解决方案】

A/B 升级过程中异常终止或断电，重新开始无感升级后会继续恢复升级。

11.4.6 A/B 无感升级反复升级同一个升级包

【问题描述】

A/B 无感升级反复升级同一个升级包，后面的升级过程很快就结束了，属于正常现象吗？

【解决方案】

A/B 无感升级完成后会记录当前升级包的 hash 值，如果同一个升级包再次进行 A/B 无感升级，则系统会认为是恢复升级，则后面的一次升级会从上一次升级包下载的断点处继续下载，下载完成后开始校验分区数据的 hash。如果上一次系统升级成功了，使用同一个 OTA 包升级，则后面的几次升级过程会跳过升级包下载阶段，直接开始 hash 校验环节，因此升级过程会较快结束，属于正常现象。

12 Recovery

12.1 简介

12.1.1 功能

Android 利用 Recovery 模式，进行恢复出厂设置，OTA 升级，patch 升级及 firmware 升级。ATC Recovery 是基于 Android Recovery 进行修改和增强。ATC Recovery 提供恢复出厂设置、本地 OTA Package 升级等功能。

Recovery 升级支持在升级期间断电或者其他异常导致升级中断，下一次重启后仍能继续进行升级和在系统启动异常后仍能进行系统恢复的功能。也即一直说的双分区备份功能。

12.1.2 代码结构及文件描述

表 12-1 recovery 代码结构及描述

文件名	文件路径	文件描述
Install	source/android/bootable/recovery/install	分区擦除、升级包安装 校验相关
otutil	source/android/bootable/recovery/otutil	OTA 升级包处理
bootloader_message	source/android/bootable/recovery/bootloader_message	recovery 和 main system 通信渠道
edify	source/android/bootable/recovery/edify	OTA 升级脚本解释器
updater	source/android/bootable/recovery/updater	OTA 升级相关

12.2 软件设计

12.2.1 架构图

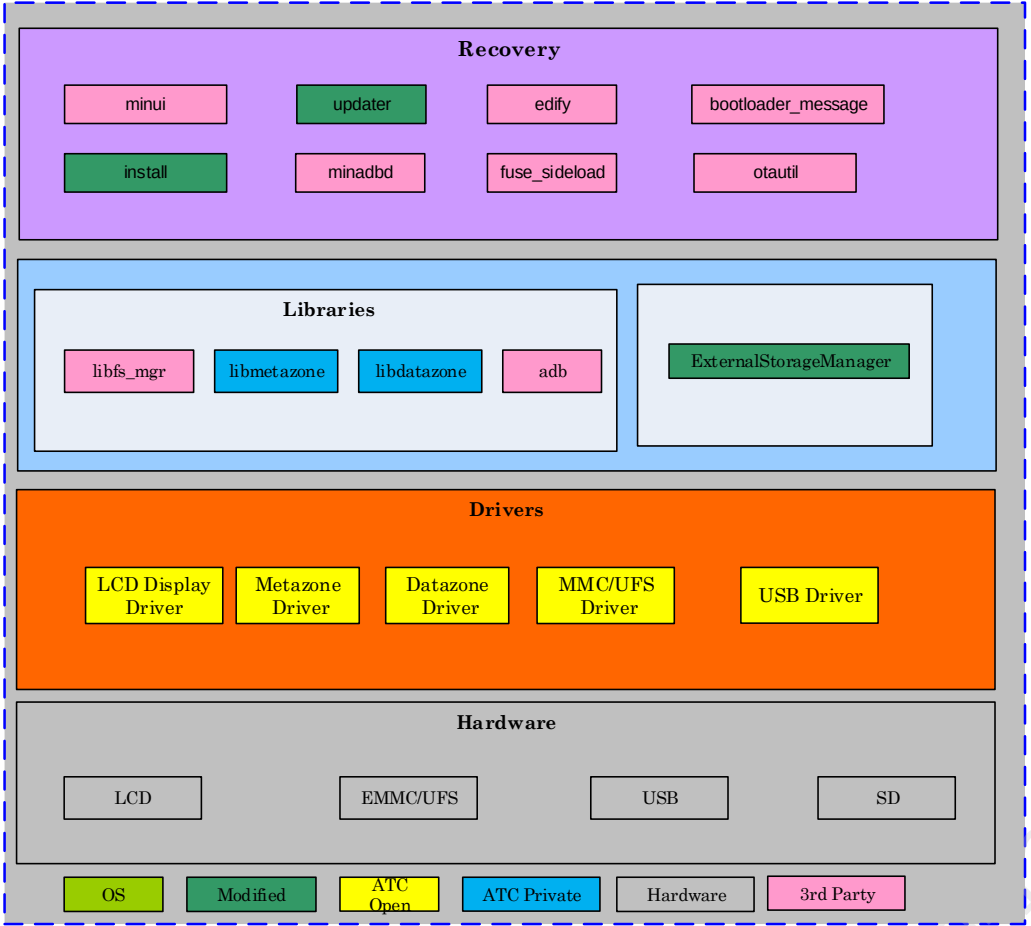


图 12-1 recovery 系统架构图

12.2.2 流程图

12.2.2.1 Recovery 数据流图

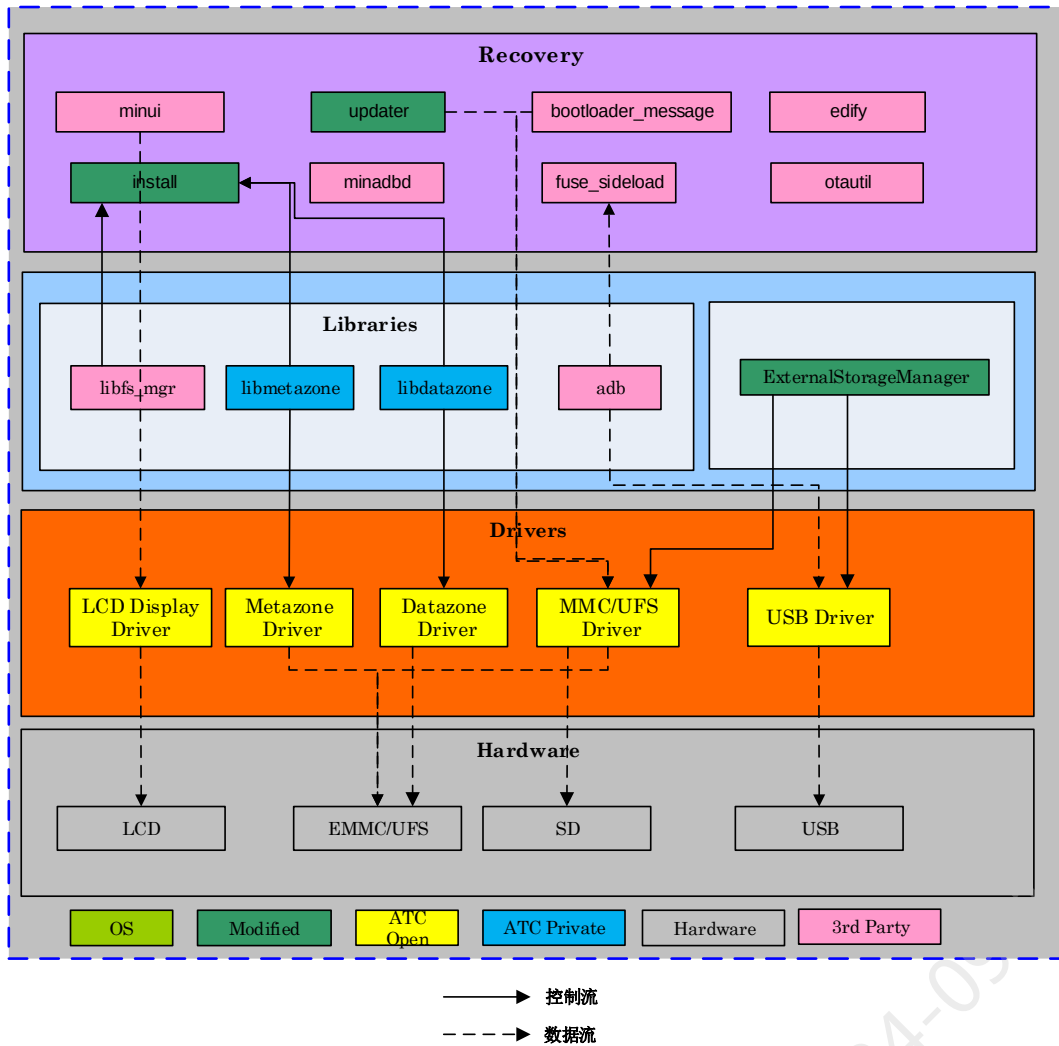


图 12-2 recovery 数据流图

- Recovery 调用 libfs_mgr 挂载 system、cache 等分区；
- Recovery OTA 升级期间调用 emmc 提供的接口对 emmc 进行读写；
- Recovery 恢复出厂设置会调用 metazone 和 datazone 提供的接口进行 factory reset 操作；

12.2.2.2 Recovery Main 数据流图

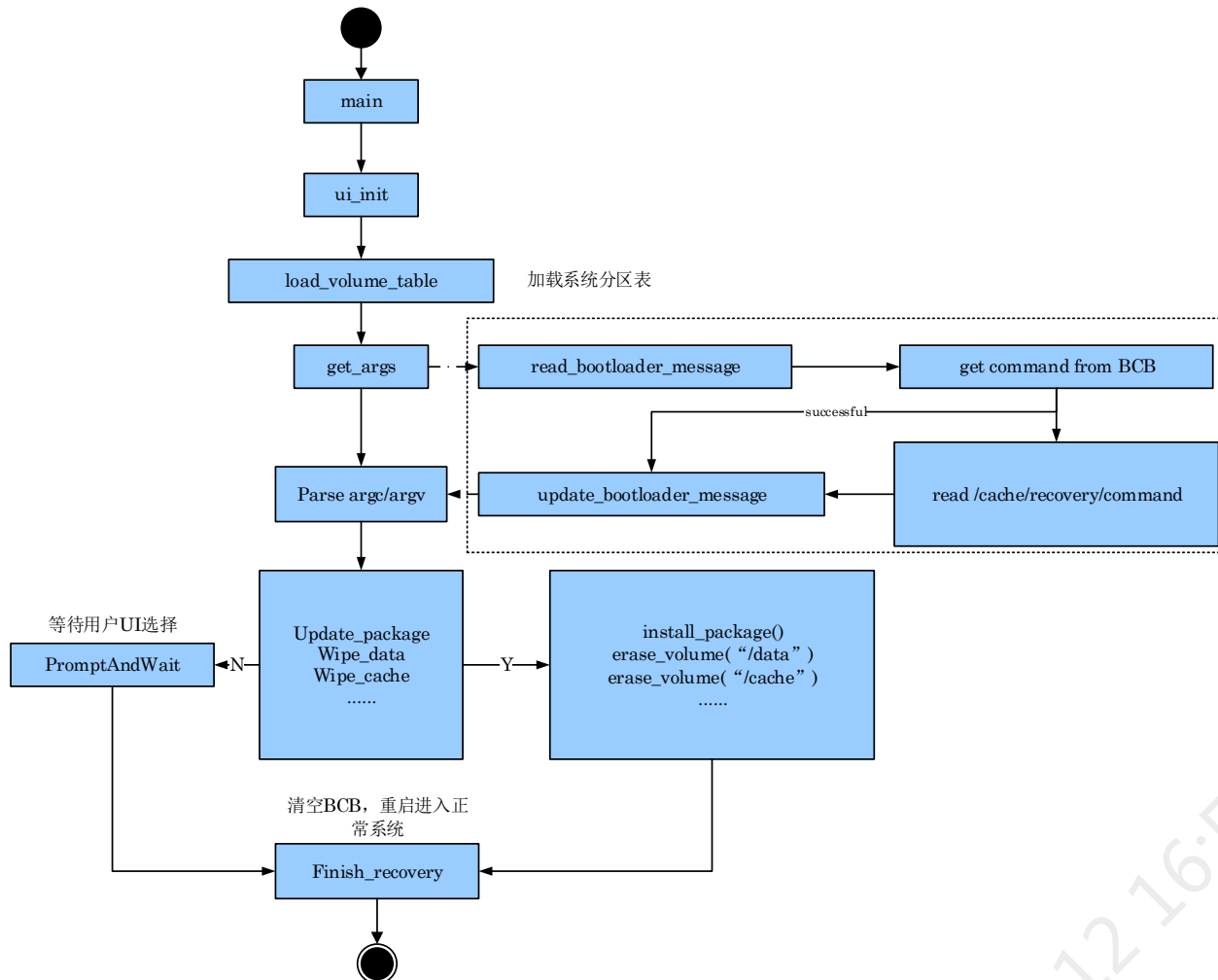


图 12-3 Recovery main 数据流图

- `get_args()` 负责检查和获取进入 Recovery mode 之前传递给 Recovery 的一些命令参数（虚线框中的流程）。
- `PromptAndWait()` 主要功能是等待 UI 的回调，返回用户操作，然后执行相应的操作。
- `Finish_recovery` 会在退出 Recovery mode 之前清空 BCB，这样保证重启后能够进入正常系统。

12.2.3 Recovery 基本流程

主系统通过 CACHE 分区中的三个文件和 MISC 分区控制 Recovery 所执行的功能和信息交换。

12.2.3.1 CACHE 分区

Recovery mode 通过 cache 分区和主系统打交道。主系统（包括恢复出厂设置和 OTA 升级）可以写入 recovery 所需的命令，读出 recovery 过程中的 log 和 intent。

12.2.3.1.1 系统支持的 recovery 命令

表 12-2 系统支持的 recovery 命令描述

命令	描述
--wipe_data	恢复出厂设置 Format /data & /cache & /metazone
--wipe_cache	擦除 cache (format /cache only).
--update_package=file name(including full path)	File name is update.zip: 用于升级的 OTA 包

其中，--wipe_data 功能和我们经常在 setting 页面的选择的恢复出厂设置功能一致，除了擦除 data，cache 分区内容外，还增加了恢复 metazone 分区部分内容。

- Recovery 的 command 写入文件位置：/cache/recovery/command
- Recovery 最后一次执行过程输出 log：/cache/recovery/last_log

log 输出格式是按照 last_log, last_log.1, last_log.2.....last_log.10 这种规律命名的，其中 last_log.x, x 数字越小表示输出的 log 越新，也即表示的是越前面一次的 log，最多可以保存 11 条 log，一次 log 文件是按照是否重启一次为标准，每重启一次，log 文件增加一个。

- 在 recovery 系统下面查看 log 有两种方式：

- 通过 recovery 的 UI 菜单项“View recovery log”，通过选中该选项，进行查看相对应的 log
- 通过进入到/tmp 目录下面查看对应的 recovery log

- 在 normal 系统下面查看 recovery log

进入到 cache/recovery 目录下面查看 recovery 相对应的 log。Recovery 输出的 intent 写入文件位置：/cache/recovery/intent。

12.2.3.2 MISC 分区

MISC 分区是文件系统的 RAW Data 分区，存放 BCB (Bootloader Control Block) 信息。定义如下：

```
struct    bootloader_message {
    char    command[32];
    char    status[32];
    char    recovery[768];
}
```

```
char    stage[32];

char    reserved[1184];

};
```

- command[32]: 表示接收的 recovery 的命令;

例如通过执行以下几个命令可以进入到 recovery mode 下进行相应的功能: (以 adb 命令为例)

```
adb root
adb shell
echo --update_package=$path/update.zip > /cache/recovery/command
reboot recovery
```



说明:

其中\$path 是对应的存储相应升级包的绝对路径, 和正常系统下相应的 storage 路径 mount 位置一致

重启后会自动进入到 ota 包的升级流程中, 升级成功后系统自动重启到正常系统, 如果升级失败, 界面停留在 recovery mode 下。

如果传入:

```
echo --wipe_data > /cache/recovery/command
或者 echo --wipe_cache > /cache/recovery/command
```

则重启后一个进入 recovery 的恢复出厂设置流程, 一个进入擦除 cache 分区流程。

- recovery[768]: system 和 recovery 之间的信息传递。
- status[32]: 用于返回执行结果, 表示恢复命令的执行状态, 可以是 "OK"、"FAIL" 或其他自定义状态。
- stage[32]: 表示恢复过程的阶段, 可以是 "1" (初始化阶段) 或 "2" (执行恢复命令阶段)。
- reserved[1184]: 预留字段, 暂未使用。

12.2.3.3 Recovery UI

Recovery 需实现一套包含“Reboot system now”、“Reboot to bootloader”、“Apply update from ADB”、“Apply update from OTA Package”、“Wipe data/factory reset”、“Wipe cache partition”、“Mount /system”、“View recovery logs”功能显示的 UI 菜单, 其余四个选项“Run graphics test”/“Run locale test”/“Enter rescue”/“Power Off”功能暂不维护。

各项功能描述如下表格：

表 12-3 RecoveryUI 及描述

Item	Description
Reboot system now	重启系统进入 Main System
Reboot to bootloader	重启进入 fastboot mode
Apply update from ADB	通过 adb 进行 OTA 升级
Apply update from OTA	进行 OTA 升级
Wipe data/factory reset	擦除 data&cache 和恢复部分 metazone 分区内容
Wipe cache partition	擦除 cache 分区
Mount /system	Mount /system，通过选中并执行该选项，可以 mount system，执行成功后可以在 recovery mode 下使用 adb 命令，进行相应的操作
View recovery logs	查看系统升级时输出的 log

UI 支持通过插入鼠标和通过触摸屏来进行相应菜单项的选中和执行，主界面如下所示：

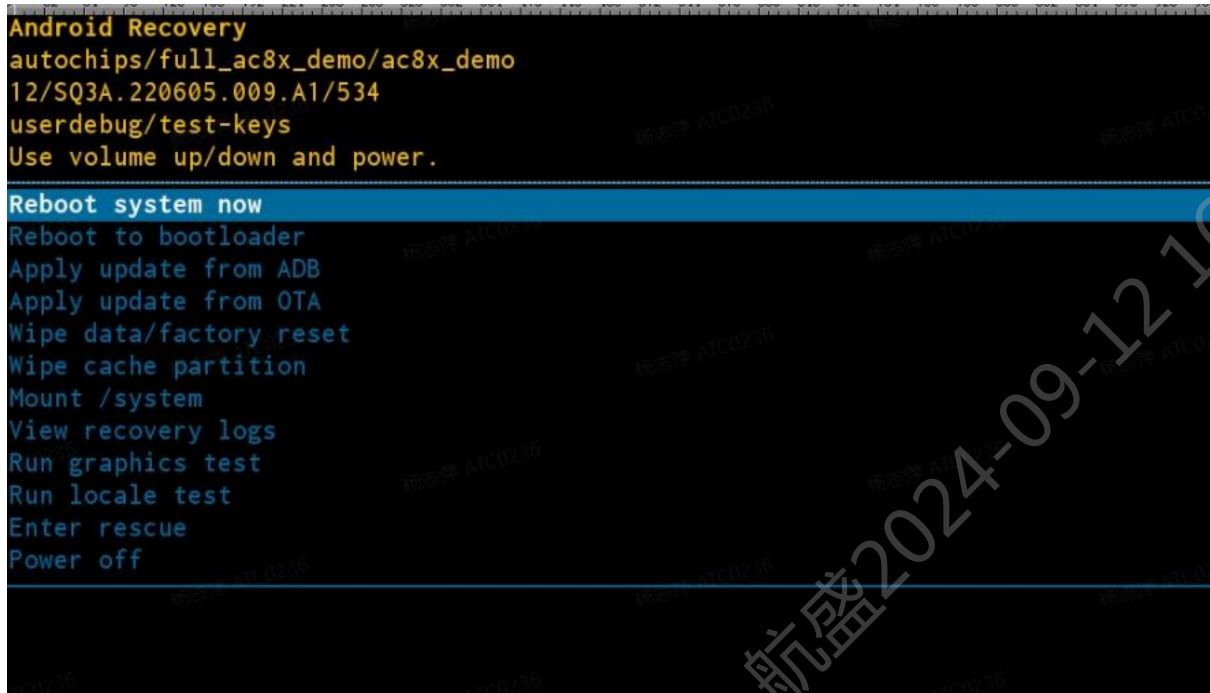


图 12-4 Recovery UI 显示

12.2.3.4 Recovery mouse

在 Recovery 的设计中，在该 project 中实现了对鼠标的支持，用户可以在进入 recovery 前插入鼠标，待系统重启进入 Recovery mode 后，通过对鼠标的操作，来实现对相应 Recovery UI 选项的选中和执行操作。

根据使用鼠标的习惯，设计的鼠标常见操作如下：

- 鼠标右键下翻
- 鼠标左键确定
- 滚轮上滚上翻，滚轮下滚下翻

12.2.3.5 Recovery touch panel

在 Recovery 的设计中，实现了对触摸屏的支持，用户可以通过使用触摸屏来进行 Recovery UI 上相应菜单的选中和执行操作，因原生触摸屏和实际使用触摸屏设计上有些差异，因此本文介绍的是根据实际使用的触摸屏改良后的一种算法。

考虑到用户实际使用的场景，采用根据 X、Y 轴的绝对值通过手指上下左右滑动来进行选中和执行的操作，这种能大大减少用户的误操作，使用起来也相对方便，精确度也有了大大的提升。

触摸屏使用规则：

- 上下滑动表示选中，即如果从上向下滑动，菜单相应的是从上向下选择；从下向上滑动，菜单相应的从下向上选择
- 左右滑动表示对选中的菜单进行执行操作



说明：

因此实际操作中，通过左右滑动和上下滑动配合来执行一次完整的操作。

12.2.3.5.1 OTA 升级包的种类

1. 完整包

OTA 完整包包含一个完整系统的文件。

2. 差分包

依据两个完整的基础包之间的差异，制作出来的 OTA 包。一般差分包的 Size 比完整包要小得多。

12.2.3.5.2 OTA 升级包的结构

从 Android 5.0 开始，完整 OTA 包，也是仿照增量包的方式进行打包。使用编译命令得到一个 xxxx.zip OTA 文件，下图就是 xxx.zip OTA 文件的结构

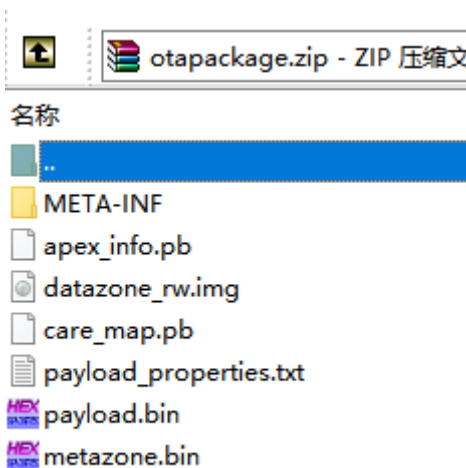


图 12-5 OTA 包文件结构

升级包的基本结构：

```

|— care_map.pb
|— META-INF
|   |— com
|   |   |— android
|   |       |— metadata
|   |       |— otacert
|— payload.bin          #主要的升级文件
|— payload_properties.txt #升级文件附带的属性
    
```

- payload.bin 是 Android OTA 镜像打包文件，包含了需要更新的数据文件；
- Payload_properties.txt: 升级内容的属性，在升级过程中会被用到；
- Care_map.pb: 文件是与系统更新相关的元数据文件之一。它包含了设备的硬件和软件信息的映射，用于确定哪些组件和分区需要进行更新；
- Metadata: 存储与系统更新相关的其他元数据文件；
- Otacert: 其中包含与数字签名相关的证书和密钥文件。这些文件用于验证 OTA 升级包的完整性和真实性，以确保系统更新的安全性；
- Metazone.bin: metazone 的升级文件；
- Datazone_rw.img:datazone 可读可写分区的升级文件。

12.2.3.6 Recovery Enhancement external storage manger 功能

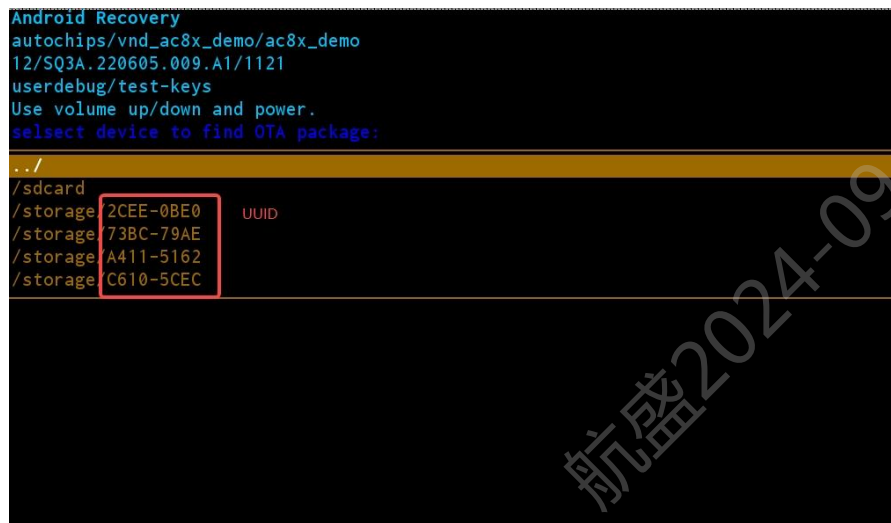
12.2.3.6.1 功能描述

- 分区正常解析，对于 VFAT/NTFS/exFAT 格式的文件系统分区，可正常挂载/卸载
- 可正常挂载/卸载 U 盘与 SD 卡
- USB HUB 上插入的 U 盘可正常挂载/卸载
- 串行处理外置存储设备（U 盘/SD 卡）的挂载卸载操作
- 接收到冗余的事件时（如插入事件还未处理，又接收到拔出事件），忽略冗余的事件
- 提供接口读取已挂载外置存储设备（U 盘/SD 卡）的挂载点信息
- 提供接口读取当前平台是否正在处理外置存储设备（U 盘/SD 卡）
- 挂载点名称与 normal mode 保持一致
- 挂载点名称通过 xml 配置文件（与 normal mode 共用一份配置文件“mountpoint.xml”）配置

12.2.3.6.2 挂载点名称

Recovery 模式下 SD 卡/U 盘挂载点名称为/storage/xxxx-xxxx，其中 xxxx-xxxx 是 uuid 编号。

如下图：插入了 3 个 U 盘和 SD 卡，U 盘和 SD 卡的路径都是/storage/xxxx-xxxx 的形式



```

Android Recovery
autochips/vnd_ac8x_demo/ac8x_demo
12/SQ3A.220605.009.A1/1121
userdebug/test-keys
Use volume up/down and power.
Select device to find OTA package:

../
/sdcard
/storage/2CEE-0BE0  UUID
/storage/73BC-79AE
/storage/A411-5162
/storage/C610-5CEC
    
```

图 12-6 recovery 下 U 盘/Sdcard 的路径示例

12.3 使用说明和注意事项

12.3.1 Recovery 菜单选项

进入 Recovery UI 目前有两种方式：

- 在 adb 中输入

```
adb shell reboot recovery
```

- 串口下面输入：

```
reboot recovery
```

显示菜单选项见图 12-4。

12.3.2 Recovery 功能

12.3.2.1 Wipe Data/Factory reset

ATC User data 分区为/data。Factory Reset 将擦除 data 分区，cache 分区，datazone 分区和部分 metazone 分区。

测试方法有以下两种：

- 点击“settings/System/Reset options/Erase all data(factory reset)/EARSE ALL DATA；
- 在 adb 输入如下命令：

```
adb root
adb shell
echo --wipe_data > /cache/recovery/command
reboot recovery;
```

12.3.2.2 Apply update from OTA

ATC OTA Update 支持本地 OTA Package (update.zip) 升级，需预先把 update.zip 放在本地 Storage。本地 Storage 支持 Sdcard 和 U 盘。也可将 update.zip 放在 emmc/ufs 的 data 分区，即 /sdcard 目录下；

Recovery mode 下测试方法：

1. 拷贝 OTA 升级包放在 Sdcard 的任意目录下；
2. 把 Sdcard 插入到 8025 的 demo 板上；

3. 在 mtktool 输入栏或通过 adb shell 输入如下命令：reboot recovery
4. 在 Menu Items 中选择“Apply update from OTA”，进入到对应的路径选择项，选中对应的需要升级的 OTA 包，开始进行升级。

Normal mode 下测试方法：

1. 拷贝 OTA 升级包放在 Sdcard 的任意目录下；
2. 把 Sdcard 卡插入到 8025 的 demo 板上；
3. 通过 adb shell 输入如下命令：

```
echo --update_package=&path/update.zip > /cache/recovery/command
reboot recovery
```



说明：

&path 即 update.zip 的存储路径，进入前先获取 root 权限，串口下输入“su”，adb 输入“adb root”。

12.3.2.2.1 U disk OTA Update 测试方法

Recovery mode 下测试方法：

1. 拷贝 OTA 升级包放在 U 盘的任意目录下；
2. 把 U 盘插入到 AC8025 的 demo 板上；
3. 在 mtktool 输入栏或通过 adb shell 输入如下命令：reboot recovery
4. 在 Menu Items 中选择“Apply Update from OTA”。选择对应的存储路径中的文件，选中后进行相应的升级。

Normal mode 下测试方法：

1. 拷贝 OTA 升级包放在 U 盘的任意目录下；
2. 把 U 盘插入到 AC8025 的 demo 板上；
3. 在 adb 输入如下命令：

```
adb root
adb shell
echo --update_package=&path/update.zip > /cache/recovery/command
```

reboot recovery

&path 即 update.zip 的存储路径（参照 12.2.3.6.2 小节中的挂载点对应），进入前先获取 root 权限，串口下输入“su”，adb 输入“adb root”。



注意：

本地 OTA 是指通过编译生成出来的 OTA 包，并非传统意义上的网络下载 OTA 包，然后 copy 到 U 盘/Sdcard 或者放在任意存储位置。

12.3.2.3 Reboot system now

ATC Recovery 支持进入 recovery mode 后能再次切换到正常系统模式下。

测试方法：

1. 在 adb 输入如下命令：

```
adb root
adb shell
reboot recovery
```

2. 在 Menu Items 中选择”Reboot system now”。

12.3.2.4 Reboot to bootloader

ATC Recovery 支持从 recovery mode 进入到 fastboot mode。

测试方法：

1. 在 adb 输入如下命令：

```
adb root
adb shell
reboot recovery
```

2. 在 Menu Items 中选择”Reboot to bootloader”。

3. 执行后系统重启，可以进入到 fastboot mode 下，通过串口 log 有类似的 fastboot mode 的关键字 log 打印。

12.3.2.5 Wipe cache partition

ATC Recovery 支持擦除 cache 分区数据。

测试方法：

- Recovery mode 下测试方法

1. 在 adb 输入如下命令：

```
adb root
adb shell
reboot recovery
```

2. 在 Menu Items 中选择 “Wipe cache partition”。

- Normal mode 下测试方法

在 adb 输入如下命令：

```
adb root
adb shell
echo --wipe cache > /cache/recovery/command
reboot recovery
```

12.3.2.6 Mount /system

通过 Menu UI 来进行 mount /system。

测试方法：（进入 recovery 前必须先获取 root 权限，adb 方式可以执行 adb root，串口可以执行 su，下同）

1. 通过 adb 输入如下命令：

```
adb root
adb shell
reboot recovery
```

2. 在 Menu Items 中选择 “Mount /system”。

12.3.2.7 View recovery logs

ATC Recovery 支持保存 recovery log 和查看 recovery log。

1. 测试方法：在 adb 中输入如下命令：

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

266 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

```
adb root
adb shell
reboot recovery
```

2. 在 Menu Items 中选择“View recovery logs”。

12.3.2.8 环境编译

如果本地环境已经 build 好，可以使用 njia 的环境来编译，这种编译非常快，例如如果修改 source/android/bootable/recovery/recovery.cpp 中的某些代码，可以进入到 source/android 使用如下几条命令：

1. source build/envsetup.sh;
2. lunch （选择合适的配置）；
3. prebuild/build-tools/linux-x86/bin/njia -f out/combinend-vnd_ac8x_demo.njia recovery （编译 recovery.img）

12.4 用户定制化

12.4.1 Recovery FS Table 的修改

Recovery 使用的 fstab 是自己的一份单独的 fstab 表，路径为 source/android/vendor/autochips/proprietary/hardware/fstab/ac8x_recovery/，定义了 Recovery 用到的 FS 的 mount point 及其设备节点和相关属性。

12.4.2 Recovery UI 的修改

ATC Recovery UIs 基于 Android Native Recovery UIs 进行修改，增加 Menu Items。Menu hear& menu help 定义于文件 source/android/bootable/recovery/screen_ui.cpp。

12.5 典型问题

12.5.1 Recovery 的 log 问题

【问题描述】

Recovery 下 log 如何查看？

【解决方案】

13 Debug Tool

13.1 DebugTool 简介

DebugTool 是 ATC 开发的用于系统 log 收集，系统运行状态监控、侦测和异常处理，以及 log、debug 信息整理、打包的一整套工具集合，主要包括 AEE 和 ATCLlogger 两大基本组件。

本章介绍的内容适用于 IVI 和 Cockpit 产品形态，其中 IVI 产品仅包含 IVI 系统，Cockpit 产品包含 IVI、ADAS 和 Cluster 系统。IVI 系统基于 Linux 内核运行 Android 系统，ADAS/Cluster 系统为纯粹的 Linux 系统。基于 IVI、ADAS 和 Cluster 系统在业务复杂程度及系统架构的差异，DebugTool 在不同系统上实现的功能也存在差异。整体上看，IVI 系统上实现的 DebugTool 功能相对比较复杂，支持 JE/KE/NE 等，以及这些功能的客户可定制化；ADAS/Cluster 系统的 DebugTool 功能类似，当前仅支持 KE/NE 功能。

13.2 AEE

13.2.1 简介

13.2.1.1 功能

AEE (Android Exception Engine) 是 ATC 基于 Android 原生机制优化的一套异常捕获和调试信息保存的方法。AEE 对于系统，类似于黑匣子 (Flight Recorder) 对于飞机的作用。主要包括：

- 监控系统运行，侦测和处理系统异常 (KE/NE/JE/HWT)。
- 当异常发生时产生 db 档。

AEE 也提供了其他工具，如 Dump Process、DAL 等。

同时，ATC 将 AEE 的概念扩展到 Linux 系统，在 Linux 系统(ADAS/Cluster)上也提供 KE/NE 异常监测功能，其中 linux NE 是指 UserSpace 发生的异常。

13.2.1.1.1 IVI 系统 AEE 功能

13.2.1.1.1.1 JEC

依赖 com.autochips.exceptionlog JAVA 包，可捕获以下异常。

表 13-1 JEC 可捕获类型表

异常类型	说明
------	----

异常类型	说明
Java Exception	Uncaught exception
	Occur wtf (LOG.wtf)
ANR	key dispatching timeout
	Broadcast timeout
	Bind service timeout
SWT	Software Watchdog Timeout

13.2.1.1.1.2 NEC

IVI 系统的 NEC 依赖于 Android 原生 debuggerd，用于捕获异常信号量，并增强备份系统信息和 coredump 的功能。

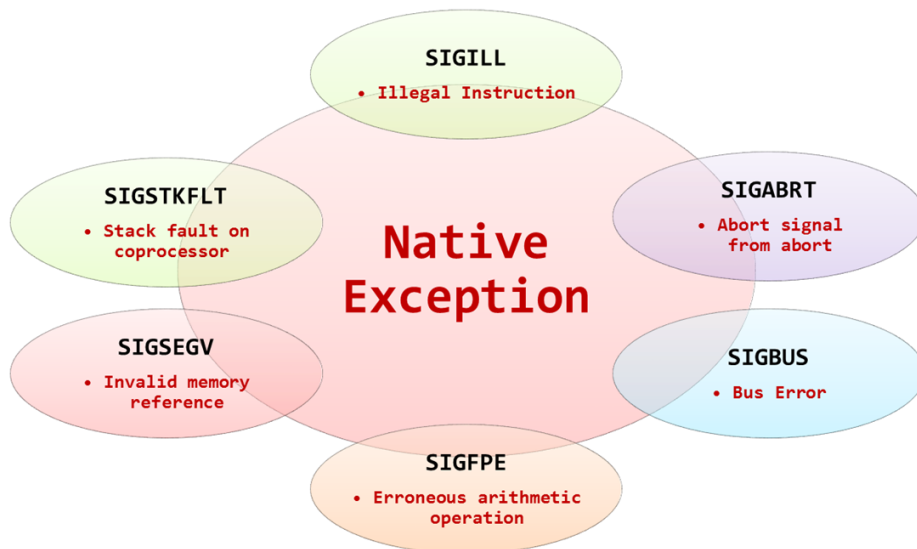


图 13-1 NE 可捕获异常信号量说明图

13.2.1.1.1.3 KEC

依赖于 aed 驱动以及 ipanic 模块。捕获 kernel 的 oops/die/panic，然后保存系统 log 及 minirdump，用于 RD 调试与分析。

13.2.1.1.2 ADAS/Cluster 系统 AEE 功能

ADAS/Cluster 系统本身业务比较简单，对应 AEE 实现的功能也比较简单，仅支持 NEC 和 KEC 功能，并且暂不支持类似 IVI 系统的定制化功能。

ADAS/Cluster 系统的 AEE 功能默认开启。

13.2.1.1.2.1 NEC

ADAS /Cluster 系统的 NEC 是指 UserSpace 发生的异常，依赖 glibc 的 segfault 子模块捕获异常信号量，并增强备份系统信息和 coredump 的功能。

13.2.1.1.2.2 KEC

ADAS /Cluster 系统与 IVI 系统使用同一份内核代码，KEC 实现方式与 IVI 系统一样，依赖于 aed 驱动以及 ipanic 模块。捕获 kernel 的 oops/die/panic，然后保存系统 log 及 minirdump，用于调试与分析。

13.2.1.2 代码结构及文件描述

表 13-2 代码路径及文件描述

文件名	文件路径	文件描述
*.c *.h	source/android/vendor/autochips/proprietary/external/aee	IVI 系统异常处理、信息收集、打包
*.cpp *.h	source/vendor/autochips/proprietary/external/aee	ADAS/Cluster 系统异常处理、信息收集、打包
*.c *.h	source/ kernel/common/drivers/soc/autochips/aee	kernel panic/die 时，dump register and process info , and set ram_console
*.c *.h	source/vendor/autochips/proprietary/boottable/lk/app/aee	根据 ram_console 内容 dump 内存信息到 expdb 分区
*.cpp, *.java	source/android/frameworks/base/core/java/com/autochips/aee	Java 和 jni 层 AEE log 接口

13.2.2 软件设计

13.2.2.1 架构图

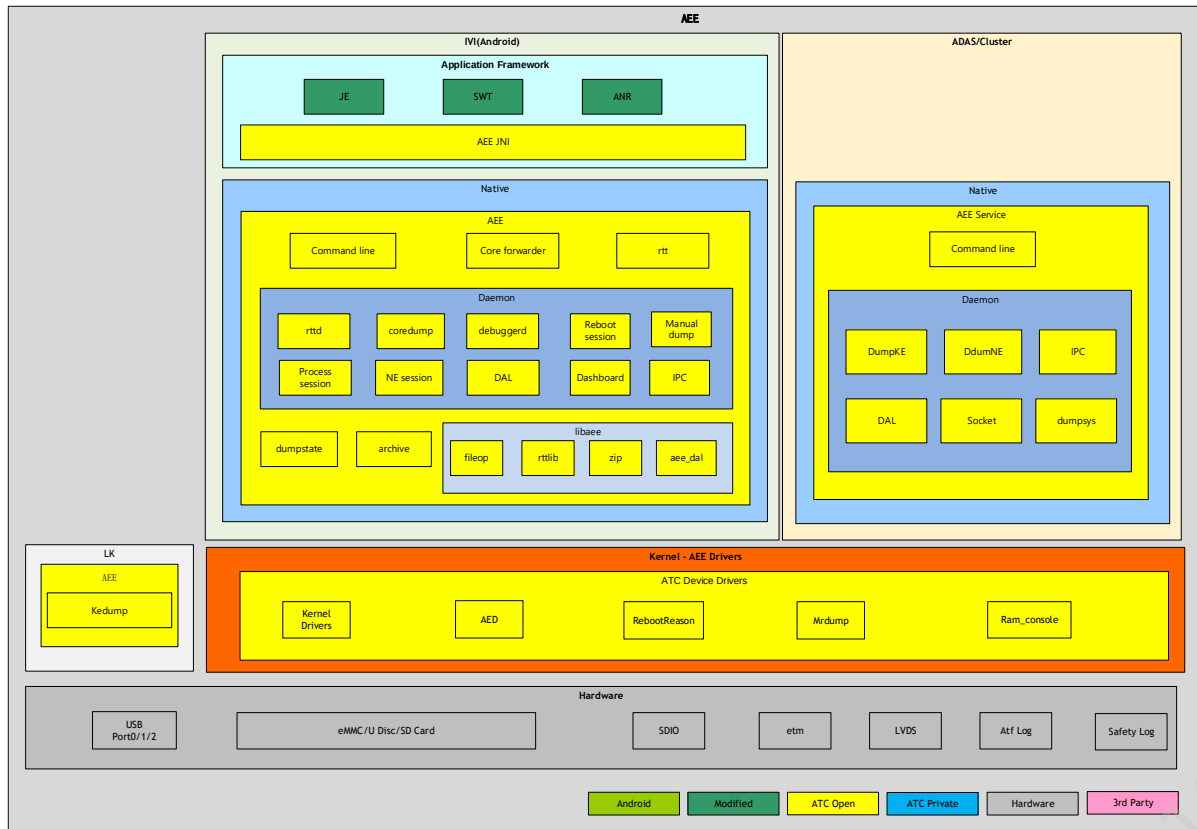


图 13-2 AEE 总体架构

13.2.2.1.1 IVI AEE 模块

IVI 系统 AEE 模块提供 Android native 层的一个 service，以及一些配合 Dump 信息的 Binary 和一个提供 Native 层对外接口的 so 库，AEE service 接收系统发生 Crash 时的 dump 请求。当系统发生 Crash，会通过 socket 发送请求给 AEE service，AEE Service 依据请求的 PID/TID 等信息，开始 Dump Debug Info 并打包保存。

如果系统发生异常重启，系统重启完成后，AEE service 解释重启前保存到 expdb 分区的内容，并打包到指定目录。

IVI AEE 在 Framework 层提供 Java 和 JNI API，APP 可以通过此 API 向 AEE service 主动报告异常。

13.2.2.1.2 ADAS/Cluster AEE 模块

ADAS/Cluster AEE 提供类似于 Android native 层的一个 aee service，进程 crash 时，通过 socket 发送请求给 AEE service，AEE Service 依据请求的 PID/TID 等信息，开始 Dump Debug Info 并打包保存到指定目录。

系统发生异常重启时，系统重启完成后，AEE service 解释重启前保存到 expdb 分区的内容，并生成异常 Log 信息打包到指定目录。

13.2.2.1.3 Kernel AEE Driver

AEE Kernel 驱动向 Kernel 注册 panic notify 回调函数，在发生 Kernel Exception（如 KE、HWT）时进入到 panic 流程，AEE Driver 里收集 Debug 问题需要的 CPU Register、Current Task PID、Backtrace 和 Kernel Log 等，保存到 Reserve Memory，然后重启系统。

13.2.2.1.4 LK AEE 模块

系统发生 KE、HWT 等异常时系统重启，Kernel AEE Driver 在重启前收集到 Reserve Memory 的内存重启到 LK 时不会丢失，LK AEE 负责将 Reserve Memory 的内存 dump 到 EMMC 的 expdb 分区，待系统启动完成后，Native 层的 AEE service 再从 expdb 分区解析异常 Log 信息并打包保存到指定目录。

13.2.2.2 流程图

13.2.2.2.1 JE Flow

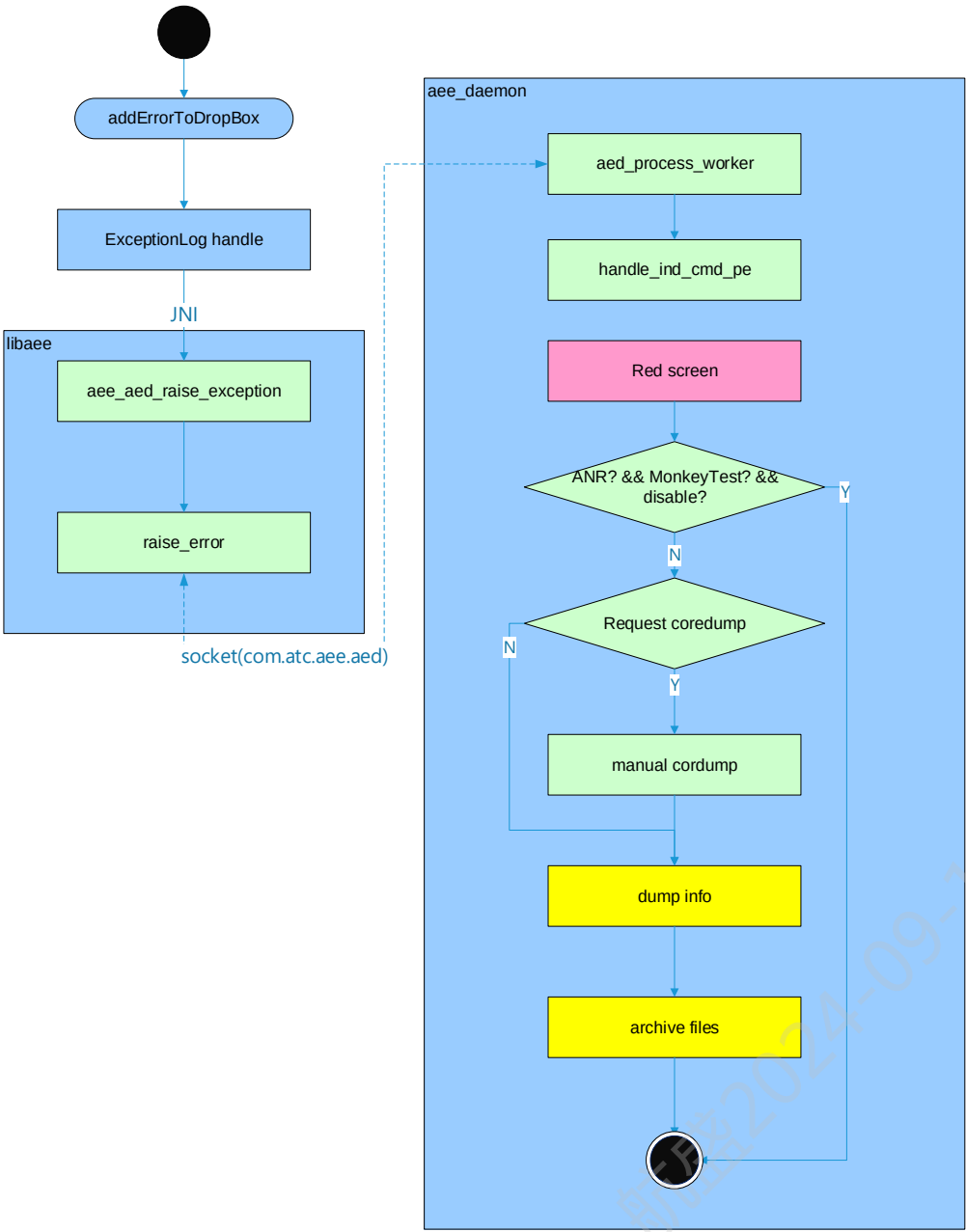


图 13-3 JE 流程图

13.2.2.2.2 NE Flow

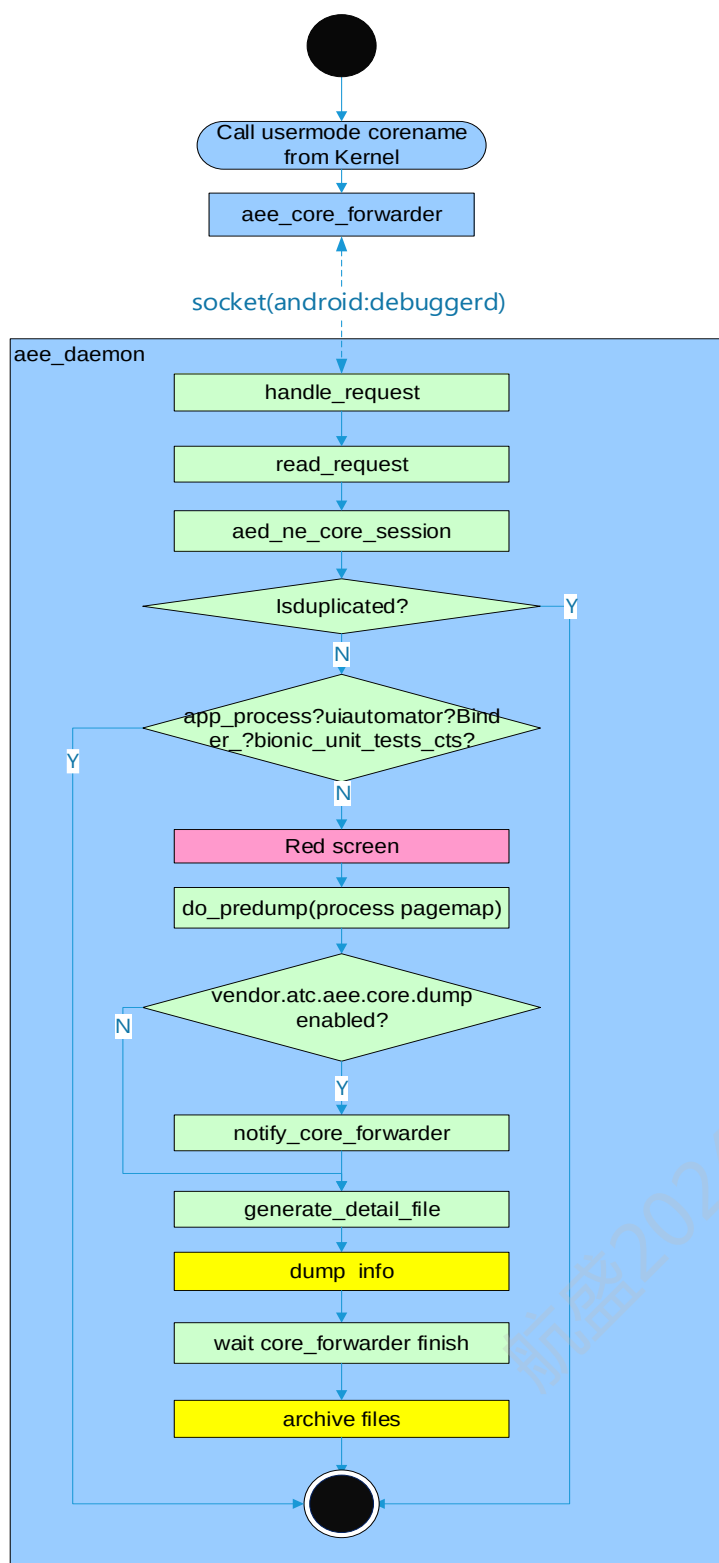


图 13-4 NE 流程图

13.2.2.2.3 KE/HWT Flow

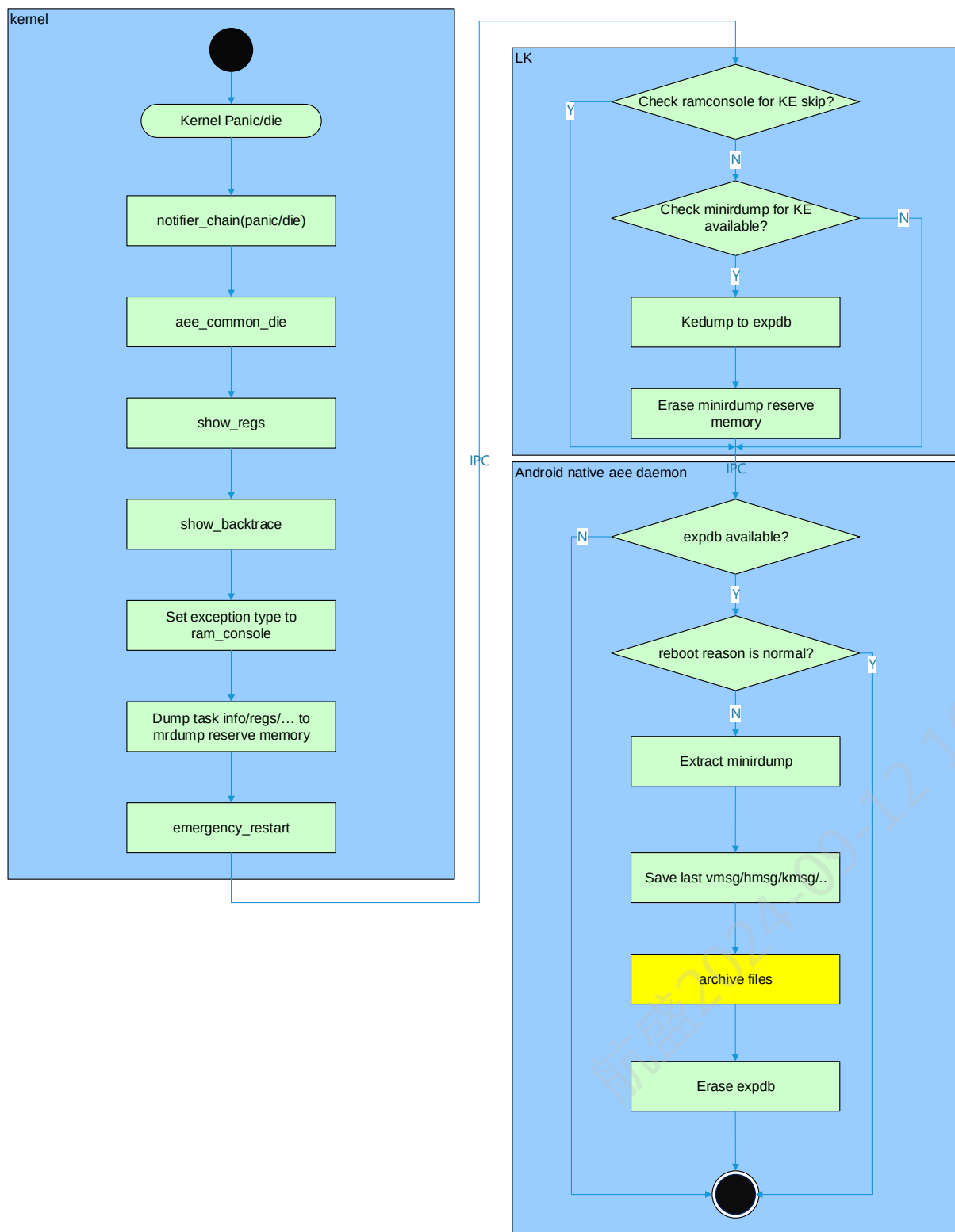


图 13-5 KE 流程图

13.2.3 接口描述

13.2.3.1 API

IVI 系统提供了一组 API 供系统调用。在系统出问题后，保留现场，包括 Kernel /Native /Java space，AEE API 在 ADAS/Cluster 系统上不提供。

13.2.3.1.1 Kernel Space

1. API

- aee_kernel_exception(module, msg, ...)
- aee_kernel_warning(module, msg, ...)
- aee_kernel_exception_api(__FILE__, __LINE__, db_opt, module, msg,...)
- aee_kernel_waring_api(__FILE__, __LINE__, db_opt, module, msg,...)

2. 参数说明

表 13-3 AEE Kernel 层 API 参数说明

参数	说明
module	Module Name
msg	描述异常内容的消息
db_opt	AEE Dump flag. Default AEE_DB_OPT
module	发生异常的模块名称

3. 头文件

\$(ANDROID_ROOT)/kernel/kernel-4.9/drivers/soc/autochips/include/aee.h

4. 行为

仅触发 AEE 并 dump 文件，不会重启。

13.2.3.1.2 Native Space

1. API

- aee_system_exception(const char *module, const char *path, unsigned int flag, const char *msg, ...);
- aee_system_warning(const char *module, const char *path, unsigned int flag, const char *msg, ...);

- aee_aed_raise_exception(AE_EXP_CLASS cls, pid_t pid, pid_t tid, const char* type, const char* process, const char* module, const char* backtrace, const char* detail_mem, const char* detail_file);
- aee_system_report_JNI(AE_DEFECT_ATTR attr, const char* module, const char* backtrace, const char* path, const char* msg, unsigned int flags);
- crash_dump_notify(const char* tombstone_path);

2. 头文件

\$(ANDROID_ROOT)/vendor/autochips/libs/\$(proj_name)/aee/binary/inc/aee.h

3. 参数说明

表 13-4 AEE Native 层 API 参数说明

参数	说明
module	Module Name
path	This file will dump to AEE DB
flag	AEE Dump flag. Default AEE_DB_OPT
msg	Output msg

13.2.3.1.3 Java Space

1. API

- Exceptionlog.systemReport(exceptionLog.AEE_WARNING_JNI, String module, String msg, String Path, ...)
- Exceptionlog.systemReport(exceptionLog.AEE_EXCEPTION_JNI, String module, String msg, String Path, ...)

2. 头文件

需要 import com.autochips.aee.ExceptionLog。

3. 参数说明

表 13-5 AEE Native 层 API 参数说明

参数	说明
module	Module Name
path	This file will dump to AEE DB.
msg	Output msg

13.2.3.2 数据结构

表 13-6 AEE 数据结构

数据结构	说明
AE_DEFECT_ATTR	异常级别
AE_EXP_CLASS	异常类型

13.2.3.2.1 AE_DEFECT_ATTR

```
enum AE_DEFECT_ATTR {
    AE_DEFECT_FATAL,          /* Fatal Error */
    AE_DEFECT_EXCEPTION,      /* Exception Error */
    AE_DEFECT_WARNING,        /* Warning Error */
    AE_DEFECT_REMINDING,      /* Remind Error */
    AE_DEFECT_END,
};
```

13.2.3.2.2 AE_EXP_CLASS

```
enum AE_EXP_CLASS {
    /* Fatal Error */
    AE_KE,                    /* Kernel Exception */
    AE_HWT,                   /* Hardward Watchdog Timeout Interrupt */
    AE_HWT_REBOOT,           /* Hardward Watchdog Timeout Reboot */
    AE_NE,                    /* Native Exception */
    AE_SWT,                   /* Soft Watchdog Timeout */
    AE_EE,                    /* External Exception */
    AE_EXP_ERR_END,
    /* Error or Warning or Defect */
    AE_ANR,                   /* Application Not Responding */
    AE_RESMON,                /* Resource Monitor */
    AE_MODEM_WARNING,
    AE_WTF,
    AE_WRN_ERR_END,
```

```

/* Manual Raise */
AE_MANUAL,          /* Manual Raise Exception */
AE_EXP_CLASS_END,
/* AEE API Report Exception */
AE_KERNEL_PROBLEM_REPORT = 1000, /* Kernel Space API Reprot Exception */
AE_SYSTEM_JAVA_DEFECT,          /* JAVA Space API Reprot Exception */
AE_SYSTEM_NATIVE_DEFECT,       /* Native Space API Reprot Exception */
AE_MANUAL_MRDUMP_KEY,
};

```

13.2.4 使用说明和注意事项

13.2.4.1 IVI 系统使用说明

13.2.4.1.1 AEE log 文件

通常，系统发生异常之后，触发 AEE 产生压缩 Log 文件，IVI 系统的压缩文件保存到文件系统的如下路径：

/data/aee_exp/*

此路径不能手动修改，删除。

开发人员通过拷贝等方式将 AEE 产生的 db 文件传输到本地，获取的是 zip 压缩包，用户只需要用 7z、rar、zip 等解压工具解压即可，解压出来的文件都是文本文件，直接打开查看即可。

例如，产生 NE 后会在/data/aee_exp 路径下产生 db.\$(ID).NE/db.\$(ID).NE.zip 压缩包，

```

assi_64:/data/aee_exp/db.00.NE # ls
ZZ_INTERNAL db.00.NE.zip
assi_64:/data/aee_exp/db.00.NE #

```

解压 zip 后会包含如下 log 文件：

```

assi_64:/data/aee_exp/db.00.NE # unzip *.zip
Archive:  db.00.NE.zip
  inflating: PROCESS_PAGEMAP.RAW-4152
  inflating: SYS_KERNEL_WAKELOCKS
  inflating: _exp_detail.txt
  inflating: DUMPSYS_GFXINFO
  inflating: SYS_FILE_SYSTEMS
  inflating: SYS_PROC_SCHED_DEBUG
  inflating: DUMPSYS_MEMINFO
  inflating: SYS_BACKLIGHTS
  inflating: PROCESS_OOM_ADJ
  inflating: THREAD_SCHED
  inflating: SYS_PROPERTIES
  inflating: PROCESS_SCHED
  inflating: SYS_KERNEL_LOG
  inflating: PROCESS_FILE_STATE
  inflating: SYS_CPU_INFO
  inflating: SYS_MOUNT_INFO
  inflating: DUMPSYS_SURFACEFLINGER
  inflating: SYS_PROCESSES_AND_THREADS
  inflating: _exp_main.txt
  inflating: PROCESS_ENVIRONMENT
  inflating: SYS_VMALLOC_INFO
  inflating: PROCESS_STATE
  inflating: PROCESS_OOM_SCORE
  inflating: SYS_MEMORY_INFO
  inflating: SYS_SLAB_INFO
  inflating: SYS_PACKAGE_SETTINGS
  inflating: PROCESS_CMDLINE
  inflating: SYS_ANDROID_RADIO_LOG
  inflating: SYS_VERSION_INFO
  inflating: THREAD_STATE
  inflating: SYS_BUDDY_INFO
  inflating: SYS_EVENT_LOG_TAGS
  inflating: SYS_ANDROID_EVENT_LOG
  inflating: SYS_ANDROID_LOG
  inflating: SYS_PACKAGE_UID_ERRORS
  inflating: PROCESS_COREDUMP
  inflating: SYS_BINDER_INFO
  inflating: PROCESS_MAPS
  inflating: SYS_ZONEINFO
  inflating: PROCESS_PAGEMAP_SUMMARY_KPAGEFLAGS-4152
  inflating: PROCESS_SHOWMAP
  inflating: SYS_VIRTUAL_MEMORY_STATS
  
```

KE/NE/JE 所产生的 zip 文件解压缩后的 log 文件内容及数量稍有差别。

13.2.4.1.2 AEE log 文件内容说明

表 13-7 AEE 文件内容说明

对应的系统中的文件以及如何输出	Files
/system/bin/logcat -d -v threadtime *:v 将 android log 输出	SYS_ANDROID_LOG
/system/bin/logcat -b events -v threadtime -d *:v 将 Android Event log 输出	SYS_ANDROID_EVENT_LOG
执行/system/bin/dmesg 将 Kernel Log 输出	SYS_KERNEL_LOG
将/sys/kernel/debug/tracing/trace 内容 Copy 到 SYS_FTRACE	SYS_FTRACE
执行/system/xbin/procmem 将指定 Process 的内存状态信息输出	SYS_PROCMEM
执行/system/bin/ls 将当前 Process 所拥有的文件句柄信息输出到	PROCESS_FILE_STATE
执行/system/xbin/showmap 将指定 Process 的内存占用情况输出到	PROCESS_SHOWMAP
将/data/anr/resmon_\$(pid).hprof 内容 Copy 到 PROCESS_HPROF	PROCESS_HPROF
将/data/anr/\$(pid).hprof 内容 Copy 到 PROCESS_OOME_HPROF	PROCESS_OOME_HPROF
将/proc/\$(pid)/status 内容 Copy 到 PROCESS_STATE	PROCESS_STATE
将/proc/\$(pid)/oom_score 内容 Copy 到 PROCESS_OOM_SCORE	PROCESS_OOM_SCORE
将/proc/\$(pid)/oom_adj 内容 Copy 到 PROCESS_OOM_ADJ	PROCESS_OOM_ADJ

将/proc/(pid)/cmdline 内容 Copy 到 PROCESS_CMDLINE	PROCESS_CMDLINE
将/proc/(pid)/sched 内容 Copy 到 PROCESS_SCHED	PROCESS_SCHED
将/proc/(pid)/maps 内容 Copy 到 PROCESS_MAPS	PROCESS_MAPS
将/proc/m4u/mva 内容 Copy 到	SYS_M4UMVA
将/proc/(pid)/task/(tid)/status 内容 Copy 到 THREAD_STATE	THREAD_STATE
将/proc/(pid)/task/(tid)/sched 内容 Copy 到 THREAD_SCHED	THREAD_SCHED
将/proc/interrupts 内容 Copy 到 SYS_INTERRUPTS	SYS_INTERRUPTS
将/sys/kernel/debug/mmpfile/buffer 内容 Copy 到 SYS_MMPROFILE	SYS_MMPROFILE
执行/system/bin/logcat -b radio -v threadtime -d *:v 将 Android Radio Log 输出到 SYS_ANDROID_RADIO_LOG	SYS_ANDROID_RADIO_LOG
将/etc/event-log-tags 内容 Copy 到 SYS_EVENT_LOG_TAGS	SYS_EVENT_LOG_TAGS
将/proc/meminfo 内容 Copy 到 SYS_MEMORY_INFO	SYS_MEMORY_INFO
将/sys/kernel/debug/tracing/buffer_total_size_kb 内容追加到 SYS_MEMORY_INFO	SYS_MEMORY_INFO
将/d/ged/gedlog 内容 Copy 到 SYS_GED_INFO	SYS_GED_INFO
将/proc/gpulong 内容 Copy 到 SYS_GPU_INFO	SYS_GPU_INFO
将/proc/sched_debug 内容 Copy 到 SYS_PROC_SCHED_DEBUG	SYS_PROC_SCHED_DEBUG
执行/system/xbin/librank 将系统所有 Process 内存使用信息输出到 SYS_LIBRANK	SYS_LIBRANK
执行/system/xbin/procrank 将系统所有 Process 的内存使用信息输出到 SYS_RPOCRANK	SYS_RPOCRANK
执行/system/bin/top -n 1 -d 1 -m 30 -t 将系统 CPU 使用最高的 30 个进程的 CPU 占用情况输出到 SYS_CPU_INFO	SYS_CPU_INFO
将/proc/vmstat 内容 Copy 到 SYS_VIRTUAL_MEMORY_STATS	SYS_VIRTUAL_MEMORY_STATS
将/proc/vmallocinfo 内容 Copy 到 SYS_VMALLOC_INFO	SYS_VMALLOC_INFO
将/proc/slabinfo 内容 Copy 到 SYS_SLAB_INFO	SYS_SLAB_INFO
将/proc/zoneinfo 内容 Copy 到 SYS_ZONEINFO	SYS_ZONEINFO
将/proc/buddyinfo 内容 Copy 到 SYS_BUDDY_INFO	SYS_BUDDY_INFO
执行/system/bin/netcfg 将 network 的 Log 输出到 SYS_NETWORK_STATE	SYS_NETWORK_STATE
将/proc/net/route 内容追加到 SYS_NETWORK_STATE	SYS_NETWORK_STATE
将系统所有的 Properties 输出到 SYS_PROPERTIES	SYS_PROPERTIES
将/sys/kernel/debug/wakeup_sources 内容 Copy 到 SYS_KERNEL_WAKELOCKS	SYS_KERNEL_WAKELOCKS
将/sys/devices/system/cpu/cpu0/cpufreq/stats/time_in_state 内容 Copy 到 SYS_KERNEL_CPUPREQ	SYS_KERNEL_CPUPREQ
执行/system/bin/ps -t -p -P -c --abi 将系统 Process State 输出到 SYS_PROCESS_AND_THREADS	SYS_PROCESS_AND_THREADS
将/data/anr/binderinfo 内容 Copy 到 SYS_BINDER_INFO	SYS_BINDER_INFO
将/sys/kernel/debug/binder/目录下的内容全部 Copy 到 SYS_BINDER_INFO	SYS_BINDER_INFO
将/data/binder/binder_native1.txt 内容 Copy 到 SYS_BINDER_BACKTRACE	SYS_BINDER_BACKTRACE

将/data/binder/binder_native2.txt 内容 Copy 到 SYS_BINDER_BACKTRACE2	SYS_BINDER_BACKTRACE2
执行/system/bin/df 将系统中各个挂在点的容量及使用量输出到 SYS_FILE_SYSTEMS	SYS_FILE_SYSTEMS
执行/system/bin/mount 将系统中各个磁盘类型、挂在点、读写情况等信息输出到 SYS_MOUNT_INFO	SYS_MOUNT_INFO
将/data/system/packages.xml 内容 Copy 到 SYS_PACKAGE_SETTINGS	SYS_PACKAGE_SETTINGS
将/data/system/uiderrors.txt 内容 Copy 到 SYS_PACKAGE_UID_ERRORS	SYS_PACKAGE_UID_ERRORS
将/sys/class/leds/目录中所有内容 Copy 到 SYS_BACKLIGHTS	SYS_BACKLIGHTS
将/proc/version、/proc/cmdline 等内容 Dump 到 SYS_VERSION_INFO	SYS_VERSION_INFO
执行/system/bin/dumpsys 将系统当前各种信息 Dump 到 DUMPSYS_*	DUMPSYS_*
执行/system/bin/ps -t -p -P -c 将系统 Threads State 输出到 SYS_ALL_THREADS	SYS_ALL_THREADS
Dump Backtrace of surfaceflinger to BT_SURFACEFLINGER	BT_SURFACEFLINGER
Dump Backtrace of system_server to BT_SYSTEM_SERVER	BT_SYSTEM_SERVER
Dump Backtrace of com.android.systemui to BT_SYSTEM_UI	BT_SYSTEM_UI
Dump Backtrace of surfaceflinger to BT_SURFACEFLINGER_1	BT_SURFACEFLINGER_1
Dump Backtrace of system_server to BT_SYSTEM_SERVER_1	BT_SYSTEM_SERVER_1
Dump Backtrace of com.android.systemui to BT_SYSTEM_UI_1	BT_SYSTEM_UI_1
将/data/anr/traces.txt 内容 Copy 到 NE_JBT_TRACES	NE_JBT_TRACES
将/data/anr/atc_traces.txt 内容追加到 NE_JBT_TRACES	NE_JBT_TRACES
将/data/anr/traces.txt 内容 Copy 到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/full_traces.txt 内容追加到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/traces_%x(1~11).txt 内容追加到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/atc_traces.txt 内容追加到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/atc_traces_%x(1~11).txt 内容追加到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/native1.txt 内容追加到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/native2.txt 内容追加到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/native3.txt 内容追加到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/native1_%x(1~4).txt 内容追加到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/native2_%x(1~4).txt 内容追加到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/native3_%x(1~4).txt 内容追加到 SWT_JBT_TRACES	SWT_JBT_TRACES
将/data/anr/SF_RTT/rtt_dump.txt 内容 Copy 到 SF_RTT_DUMP	SF_RTT_DUMP
将/data/anr/SF_RTT/rtt_dump_1.txt 内容 Copy 到 SF_RTT_DUMP1	SF_RTT_DUMP1
将/sys/module/lowmemorykiller/parameters/adj 内容 Copy 到 SYS_LOW_MEMORY_KILLER	SYS_LOW_MEMORY_KILLER
将/sys/module/lowmemorykiller/parameters/minfree 内容追加到 SYS_LOW_MEMORY_KILLER	SYS_LOW_MEMORY_KILLER
将/proc/mali/memory_usage 内容 Copy 到 SYS_MALI_MEM_USED	SYS_MALI_MEM_USED
将/sys/kernel/debug/binder/page_used 内容 Copy 到 SYS_BINDER_MEM_USED	SYS_BINDER_MEM_USED

将/proc/driver/io_logger/io_logger_display 内容 Copy 到 SYS_FS_IO_LOG	SYS_FS_IO_LOG
将/proc/driver/wmt_aee 内容 Copy 到 SYS_WCN_ISSUE_INFO	SYS_WCN_ISSUE_INFO
执行/system/bin/dmlog 将实时 Memory 使用情况 Dump 到 SYS_MEMORY_LOG	SYS_MEMORY_LOG
将/sys/kernel/debug/ion/ion_mm_heap 内容 Copy 到 SYS_ION_MM_HEAP	SYS_ION_MM_HEAP
将/sys/kernel/debug/m4u/pgtable 内容 Copy 到 SYS_M4U_PGTABLE	SYS_M4U_PGTABLE
将/sys/kernel/debug/page_recorder/Usage_rank 内容 Copy 到 SYS_PAGE_RECORDER	SYS_PAGE_RECORDER
将/sys/kernel/debug/ion/ion_debugger/pids 内容 Copy 到 SYS_ION_DEBUGGER_PIDS	SYS_ION_DEBUGGER_PIDS
将/sys/kernel/debug/ion/ion_debugger/buffers 内容 Copy 到 SYS_ION_DEBUGGER_BUFFERS	SYS_ION_DEBUGGER_BUFFERS
将/proc/atc_cmdq_debug/status 内容 Copy 到 CMDQ_STATUS	CMDQ_STATUS
将/proc/atc_cmdq_debug/record 内容 Copy 到 CMDQ_RECORD	CMDQ_RECORD
将/d/disp/dump 内容 Copy 到 SYS_DISP_DUMP	SYS_DISP_DUMP
将/proc/driver/usktrk_aee 内容 Copy 到 SYS_PROC_USKTRK	SYS_PROC_USKTRK
将/proc/pagetypeinfo 内容 Copy 到 SYS_PAGETYPE_INFO	SYS_PAGETYPE_INFO
将/proc/mtpprof/cputime 内容 Copy 到 SYS_MTPROF_CPUTIME	SYS_MTPROF_CPUTIME
将/sys/power/atcpasr/execstate 内容 Copy 到 SYS_ATCPASR_INFO	SYS_ATCPASR_INFO
将/sys/kernel/debug/atcfb 内容 Copy 到 SYS_DISPLAY	SYS_DISPLAY
将/sys/kernel/debug/cpuhvfs/dbg_repo 内容 Copy 到 SYS_CPUHVFS_REPO	SYS_CPUHVFS_REPO
将/proc/config.gz 内容 Copy 到 KERNEL_CONFIG.gz	KERNEL_CONFIG.gz
将/proc/atc_memcfg/slabtrace 内容 Copy 到 SYS_MEMCFG_SLABTRACE	SYS_MEMCFG_SLABTRACE
将/proc/log_much 内容 Copy 到 SYS_LOG_MUCH	SYS_LOG_MUCH
将/sys/kernel/debug/dynamic_debug/control 内容 Copy 到 SYS_DYNAMIC_DEBUG	SYS_DYNAMIC_DEBUG
将/sys/kernel/debug/shrinker 内容 Copy 到 SYS_SHRINKER	SYS_SHRINKER
将/sys/kernel/debug/blockio 内容 Copy 到 SYS_BLOCKIO	SYS_BLOCKIO
将/sys/kernel/debug/cisp_dbgtrig 内容 Copy 到 SYS_CISP_DBGTRIG	SYS_CISP_DBGTRIG
将/sdcard/ispdbg/imembufDBG.txt 内容 Copy 到 SYS_IMEMBUF_DBG	SYS_IMEMBUF_DBG
将/sdcard/ispdbg/imemleakDBG.txt 内容 Copy 到 SYS_IMEMLEAK_DBG	SYS_IMEMLEAK_DBG
将/proc/ccci_dump 内容 Copy 到 SYS_CCCI_DUMP	SYS_CCCI_DUMP

13.2.4.2 AEE Log 分析流程

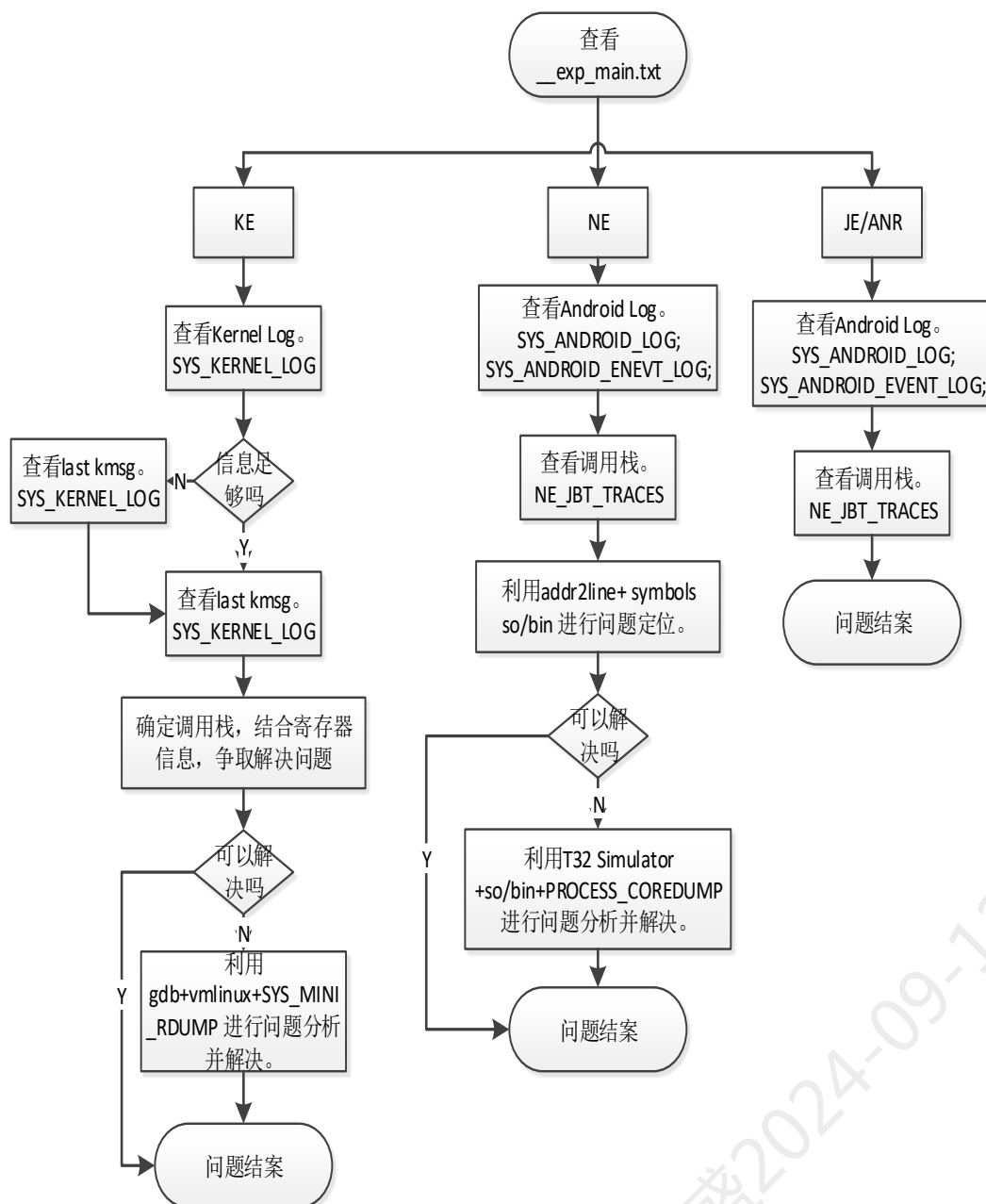


图 13-6 IVI 系统 AEE log 分析流程总结

13.2.4.3 ADAS/Cluster 系统使用说明

13.2.4.3.1 AEE Log 文件

系统发生异常之后，触发 AEE 产生压缩文件，Cluster 和 ADAS 在各自系统下有独立的文件系统，压缩文件保存到各自文件系统的如下路径：

/data/aee_exp/*

此路径不能手动修改，删除。

压缩文件使用方式同 IVI 系统。

13.2.4.4 AEE Log 分析流程

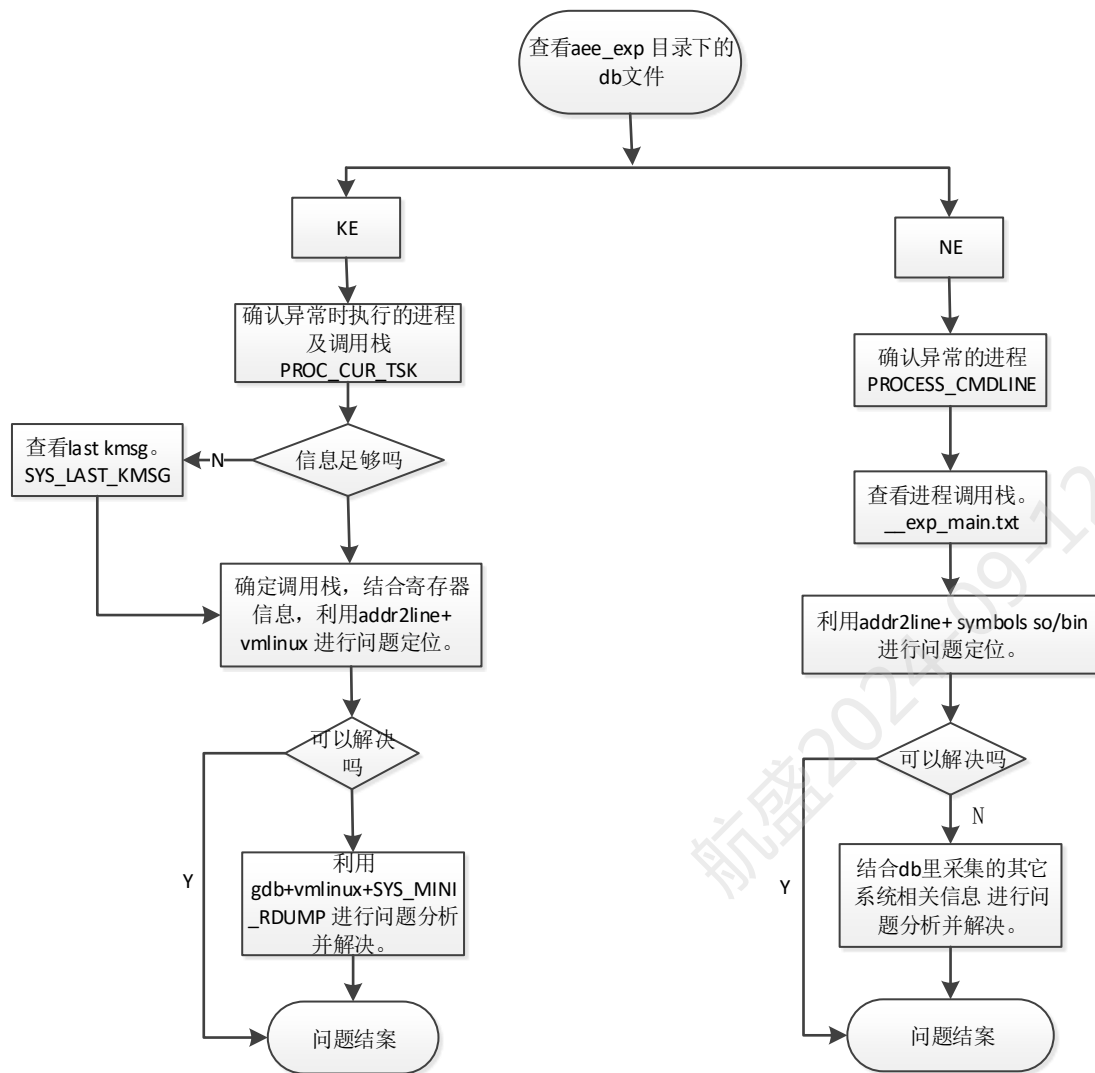


图 13-7 Adas/Cluster 系统 AEE Log 分析流程总结

13.2.4.5 注意事项

Java 层 AEE 以及 Kernel AEE 都依赖 Native 层的 AEE，所以 Native 层 AEE 关掉，就是整个 AEE 关掉，就不会生产 AEE 的压缩包。

13.2.5 用户定制化

13.2.5.1 IVI 系统 AEE 功能定制化

13.2.5.1.1 配置 AEE

IVI 系统在各层均可实现相应 AEE 功能的打开与关闭。

13.2.5.1.2 Kernel 部分

source/kernel/common/drivers/soc/autochips/Kconfig

```
config ATC_AEE_FEATURE
    tristate "Enable Android Exception Enhancement"
#    depends on HAVE_AEE_FEATURE="yes" && ATC_RAM_CONSOLE
    default y
    select FIQ_GLUE if !TRUSTONIC_TEE_SUPPORT && !ARM_PSCI && !TRUSTY
    help
```

Kernel 部分的 config 需要与 User Spaces 部分配置一致。

13.2.5.1.3 Native 部分

\$(ANDROID_ROOT)/device/autochips/vendor/\$(proj_name)/ProjectConfig.mk

```
HAVE_AEE_FEATURE = yes
```

13.2.5.1.4 Java 部分

\$(ANDROID_ROOT)/device/autochips/system/\$(proj_name)/SystemConfig.mk

```
HAVE_AEE_FEATURE = yes
```

以上为静态开关各层级的 AEE 功能，下面说明动态开关的方法。

13.2.5.1.5 动态配置 AEE 功能

13.2.5.1.5.1 Init.rc

AEE 模块有自己的 init rc 文件，

\$(ANDROID_ROOT)/vendor/autochips/proprietary/external/daemjon/init.aee.rc

```
27 setprop ro.vendor.aee.build.info atc
28 setprop vendor.atc.aee.mode 1
29 setprop vendor.atc.aee.dal 1
30 setprop vendor.atc.aee.filter 4
31 setprop vendor.atc.aeev.mode 1
32 setprop vendor.atc.aeev.dal 1
33 setprop vendor.atc.aeev.filter 4
```

此文件中有很多选项，但是不建议用户修改，一般来讲就是修改启动默认 mode 和是否打开 core dump 功能。

vendor.atc.aee.mode 的意义：

- 4 – 表示只有 Fatal Error 才会 dump，且 dump 次数限制 4 次，如果超过就循环覆盖；
- 3 – 表示只有 Fatal Error+Exception 会 dump，支持 NE 的 core dump；
- 2 – 表示所有的异常，包括 Fatal Error, Exception, ANR, WFT 等，但是不支持 NE 的 core dump；
- 1 – 功能和 2 一样的，除了支持 NE 的 core dump，3/2/1 所支持的 dump 次数限制是 20 次，超过了 20 次，就循环覆盖。循环覆盖的时候永远保留第一个 dump 文件。

13.2.5.1.5.2 AEE 命令动态修改功能配置

aee cmd 是 ATC 开发的用于动态配置 AEE 参数以及查看 AEE 参数状态的工具。

1. 命令行：adb shell aee

不带参数，可以显示如下命令帮助：

```
console:/ # aee
USAGE : aee [options...]
-c dal      : clear DAL layer
-s [on/off] : switch on/off the Red screen feature
-d db       : dump db.xx manually
-d [cnt]    : set db file count
-n [cnt]    : set fatal db file count
-d yes      : force db stored on sdcard
-d no       : store db on sdcard first. If fails, store it on data
-d coreon   : enable userspace coredump
-d coreoff  : disable userspace coredump
-d directon : enable one signal direct coredump
-d directoff : disable one signal direct coredump
-d info     : show the meaning of db files
-e [Level]  : switch caught exception level.0~4
-p pid      : Dump process information/core
-r          : Show current running exception
-k [Level]  : modify kernel console_loglevel(0~8)
-m [1/2/3/4] : 1(Internal Eng Build), 2(Internal user build), 3(custom Eng build), 4(custom user build)
-t [time]   : 0(disable hang detect trigger_hwt), time(hang_detect_trigger_hwt timeout)
-a [0/1/2/nnn] : [MT-RAMDUMP]: 0:allocation disable. 1:halfmem size. 2:fullmem size. nnn:user-defiend size(>256MB).
-z [outfile] : [MT-RAMDUMP]: fetch the ext4 coredump data to outfile.
```

图 13-8 IVI 系统 aee 命令参数

【举例】adb shell aee -c dal →清除屏幕红屏。

13.2.5.2 ADAS/Cluster 系统 AEE 功能定制化

ADAS /Cluster 系统 AEE 功能本身比较简单，默认开启，仅支持很少的动态配置功能。

13.2.5.2.1 配置 AEE

静态开关 AEE 功能

13.2.5.2.1.1 Kernel 部分

Kernel 部分 AEE 配置同 IVI，参考 13.2.5.1.2 节。

13.2.5.2.1.2 UserSpace 部分

```
source/vendor/autochips/proprietary/buildroot-configs/configs/configs/ac8025-
$(sys_name)_defconfig
```

```
BR2_PACKAGE_AEE_SERVICE=y
```

13.2.5.2.2 AEE 命令动态修改功能配置

ADAS /Cluster 系统支持如下 aee 配置命令：

```
sh-4.4# aee -h
USAGE : aee [options...]
-h/help      : Show this help.
-c dal       : clear DAL layer
-s [on/off]  : switch on/off the Red screen feature
```

【举例】adb shell aee -c dal →清除屏幕红屏。

【举例】adb shell aee -s on/off →打开/关闭对应系统的红屏功能



注意：

使用上述 adb 命令配置 ADAS /Cluster 系统的 aee 功能时，需要先使用 adb shell “echo adas/cluster > /proc/usb_debug” 命令切换到对应的系统。

13.3 ATCLogger

13.3.1 简介

13.3.1.1 功能

主要功能：

- ATCLogger 是负责 Android 运行时，所有 Log 的记录存储，包括 Android Log 和 Kernel Log。
- Android Log 包括如下几个部分：AP Log、GPS Log、Network Log 等。
- Log 默认保存在内置 Storage 中/data/atclog/目录下，也可以选择保存在外置 Storage 中/storage/emulator/0/atclog 目录下。
- 当 Taglog 功能开启，系统发生异常时，相关的异常 Log 保存在/data/atclog/taglog/目录中。
- ADAS/cluster 使用 syslog 它收集和存储系统产生的各种日志信息，包括内核消息、系统服务日志、应用程序日志等，默认保存路径是/data/atclog/syslog

13.3.1.2 代码结构及文件描述

表 13-8 ATCLogger 代码结构及描述

文件名	文件路径	文件描述
ATCLogger	source/android/vendor/autochips/proprietary/pacakges/app/ATCLogger	ATCLogger apk
syslog	systemd/syslog	提供 Log 输出控制

13.3.2 软件设计

13.3.2.1 架构图

13.3.2.1.1 ivi log 架构图

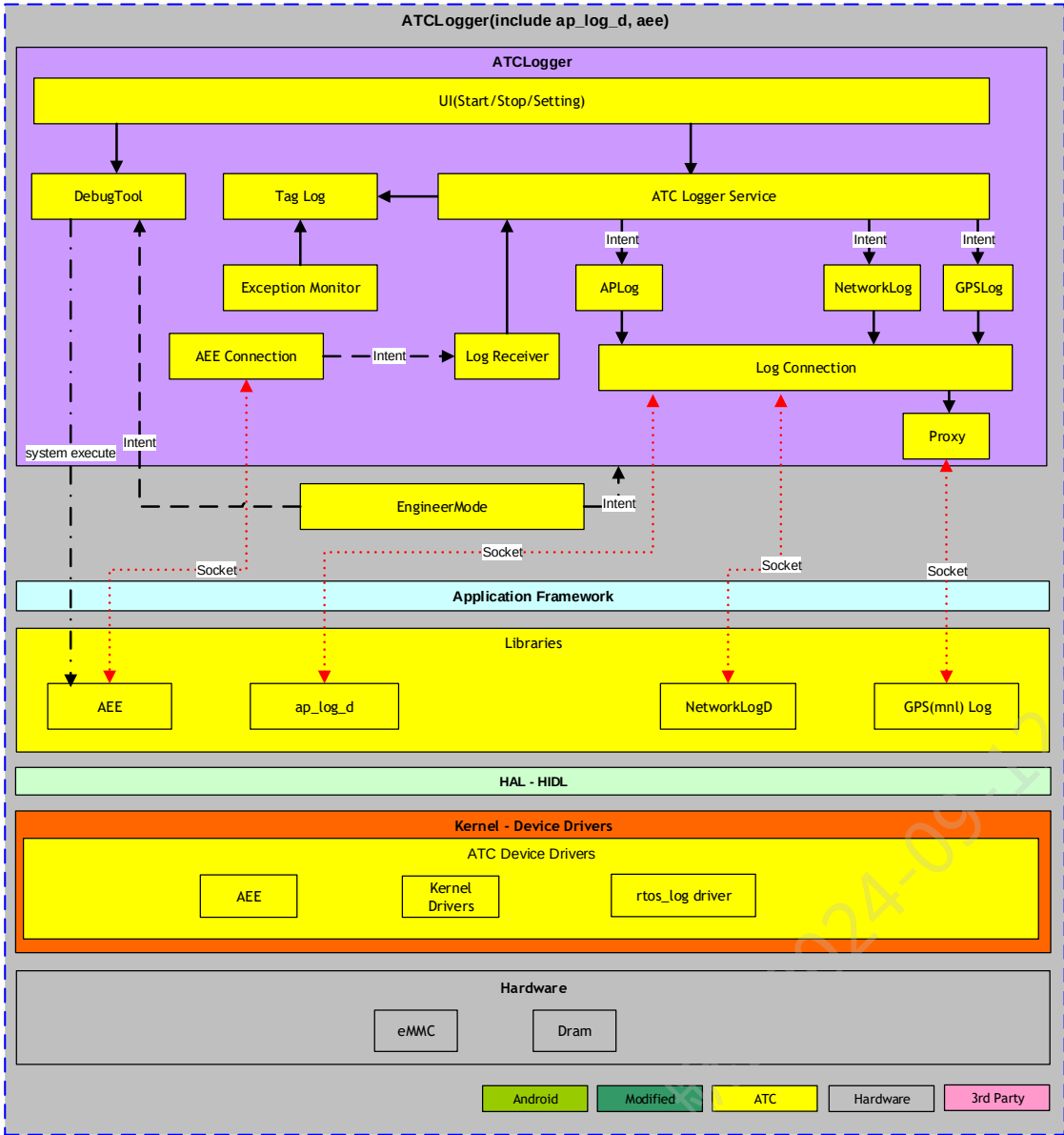


图 13-9 ivi log 架构图

- **UI**: 界面操作，主要负责发送命令，显示运行状态，Log Setting 等。
- **DebugTool**: 负责发送 Config information 到 AEE 模块。

- **TagLog:** 当系统发生异常，ATCLogger 接收到异常消息之后，保存当前出现异常时的 Android Log 到单独的 taglog 目录，就是把整个 atclog 目录中的内容压缩保存起来，然后重新启动 Log 录制。
- **Exception Monitor:** 当 ATCLogger 在 Recording 过程中，实时监控/data/aee_exp/db_history 和/data/vendor/atclog/aee_exp/db_history 这两个文件是否有修改动作，如果有修改动作立刻发送 Exception 消息给 ATCLogger，启动 Taglog 功能。
- **AEE Connection:** 建立 Socket server，接收来自 AEE 的 exception 消息，并把消息发送给 LogReceiver。
- **Log Receiver:** 接收系统消息，包括 BOOT_COMPLETE,SYSTEM_EXCEPTION 等系统消息，再把消息发送给 Service 统一处理分发。
- **ATCLogger Service:** ATCLogger 的核心子模块，负责处理 UI 的消息，创建各个 Log 实例的创建，发送消息给各个 Log 实例，接收从各个 Log 实例的消息并处理。
- **Log Connection:** 负责所有与 socket 创建以及通信相关的处理。
- **APLog:** APLog 实例，负责通过 Socket 转发 UI 的消息、配置给 Native 层的 ap_log_d。
- **Proxy:** GPSLog 通过 Proxy 建议 socket 与 Native 层的 server 端通信。
- **Log Config :** 根据 /system/etc/atclog_config.prop 文件的内容，初始化 /data/data.com.autochips.atclogger/shared_prefs/com.autochips.atclogger_preferences.xml。

13.3.2.1.2 adas/cluster log 架构图

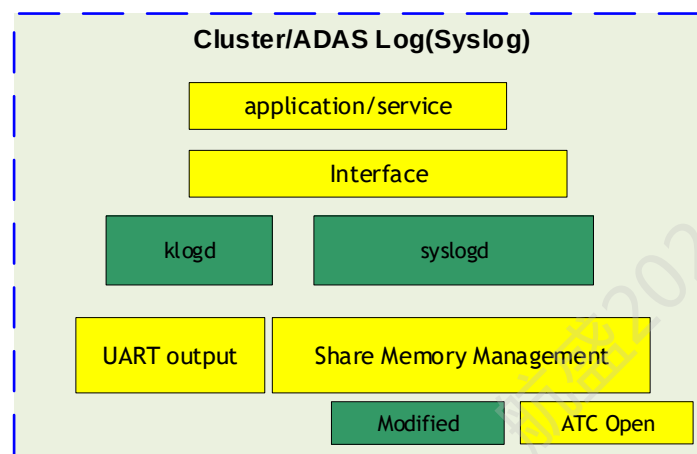


图 13-10 adas/cluster log 架构图

syslog 是一种系统日志记录服务。它负责收集和存储系统产生的各种日志信息，包括内核消息、系统服务日志、应用程序日志等。syslog 使用一个称为 syslogd 的守护进程来监听日志消息，并将其写入适当的日志文件。

13.3.2.2 流程图

13.3.2.2.1 ivi log 流程图

13.3.2.2.1.1 Starting Recording

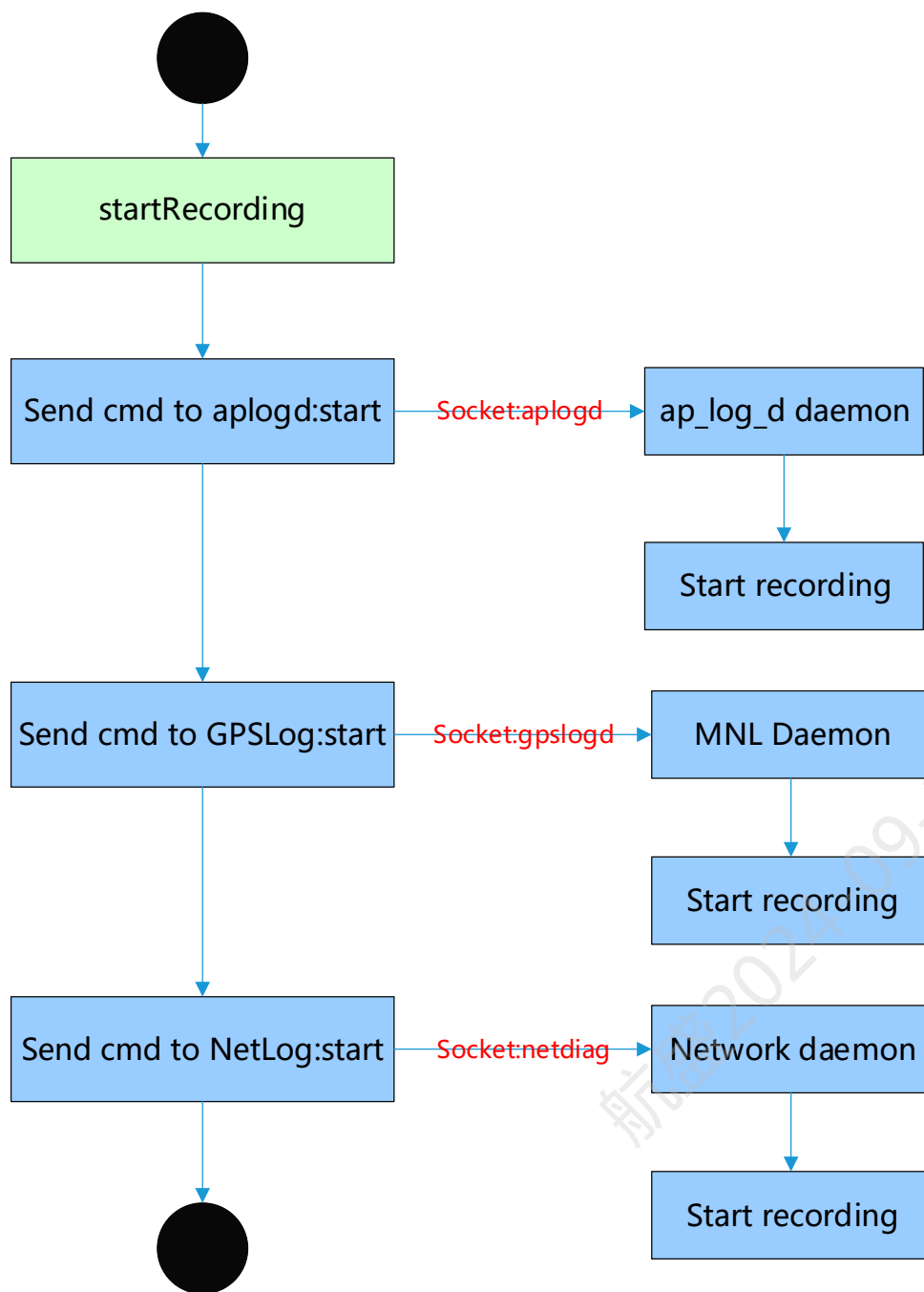


图 13-11 Starting recording 流程图

13.3.2.2.1.2 Stop Recording

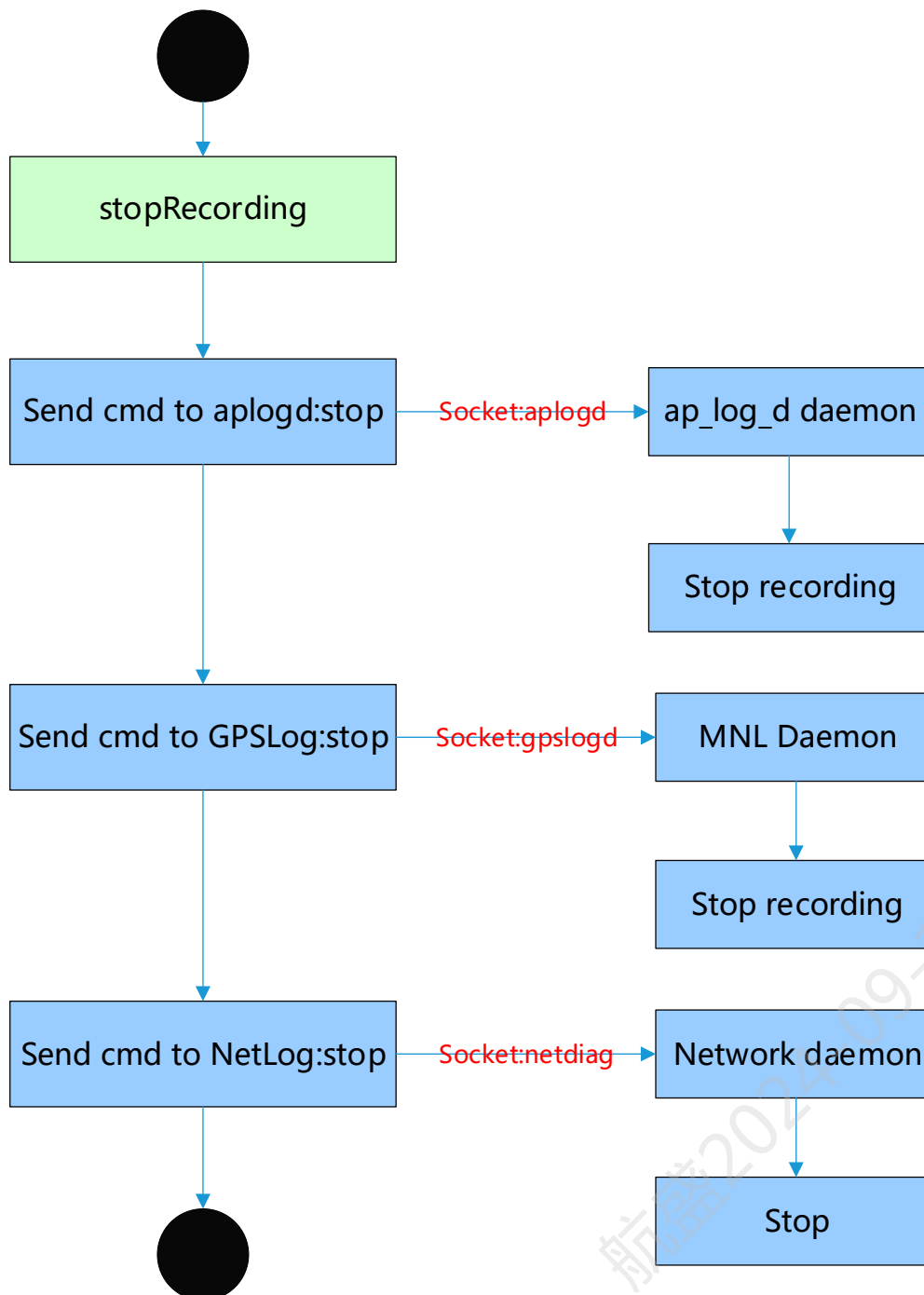


图 13-12 Stop recording 流程图

13.3.2.2.2 Adas/cluster log 流程图

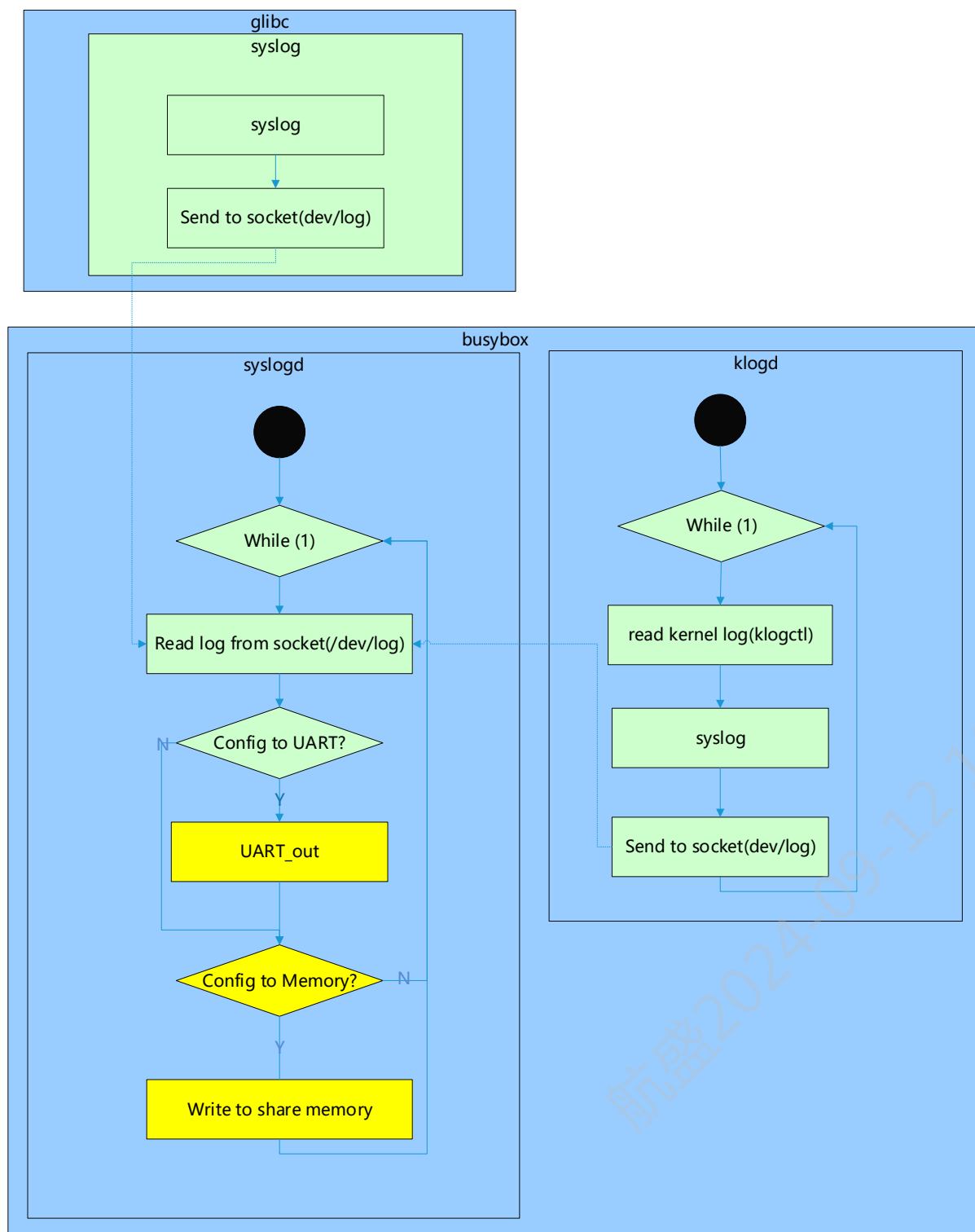


图 13-13 Stop recording 流程图

13.3.3 接口描述

13.3.3.1 ivi log 操作描述

表 13-9 ivi log 操作描述

操作描述	说明
starting recording 按钮	开始 atclog 录制
stop recording 按钮	停止 atclog 录制
settings 按钮	atclog 设置按钮

13.3.3.1.1 Starting recording

【描述】

在 Home 界面，点击 show more 进入 All apps 界面，再往下拉，找到 ATCLogger apk。

【操作】

点击 ATCLogger 选项，进入 ATCLogger 主界面：

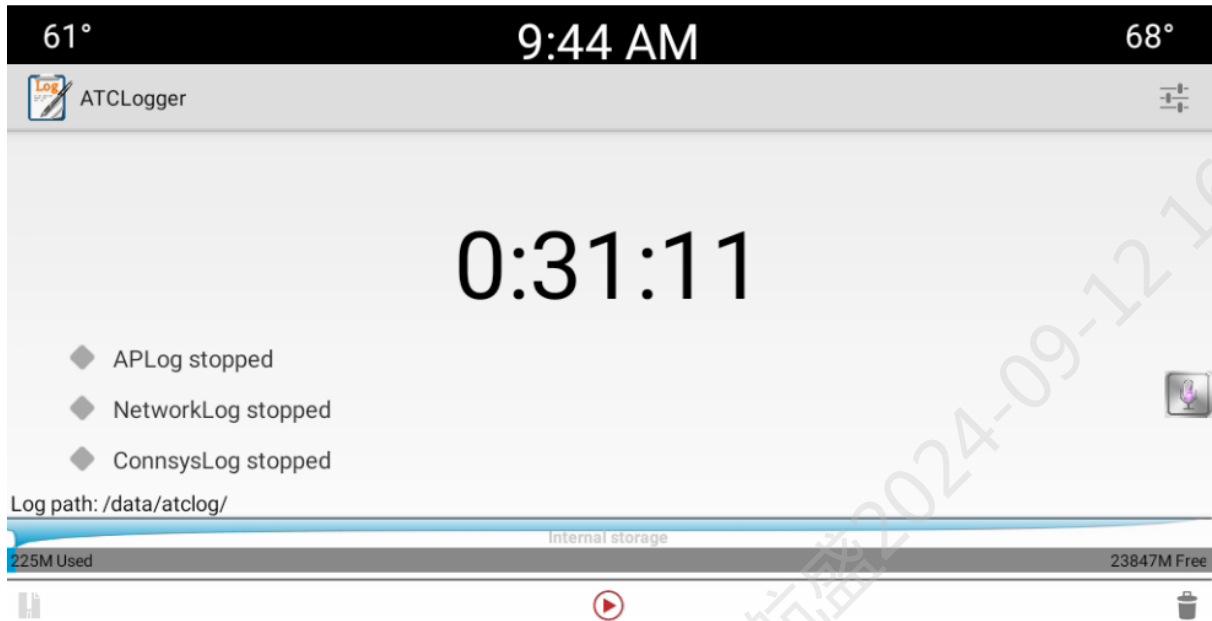


图 13-14 ATCLogger UI Starting

点击  这个按钮开始录制 atclogger。

13.3.3.1.2 Stop recording

【描述】

在 Home 界面，点击 show more 进入 All apps 界面，再往下拉，找到 ATCLogger apk。

【操作】

点击 ATCLogger 选项，进入 ATCLogger 主界面：

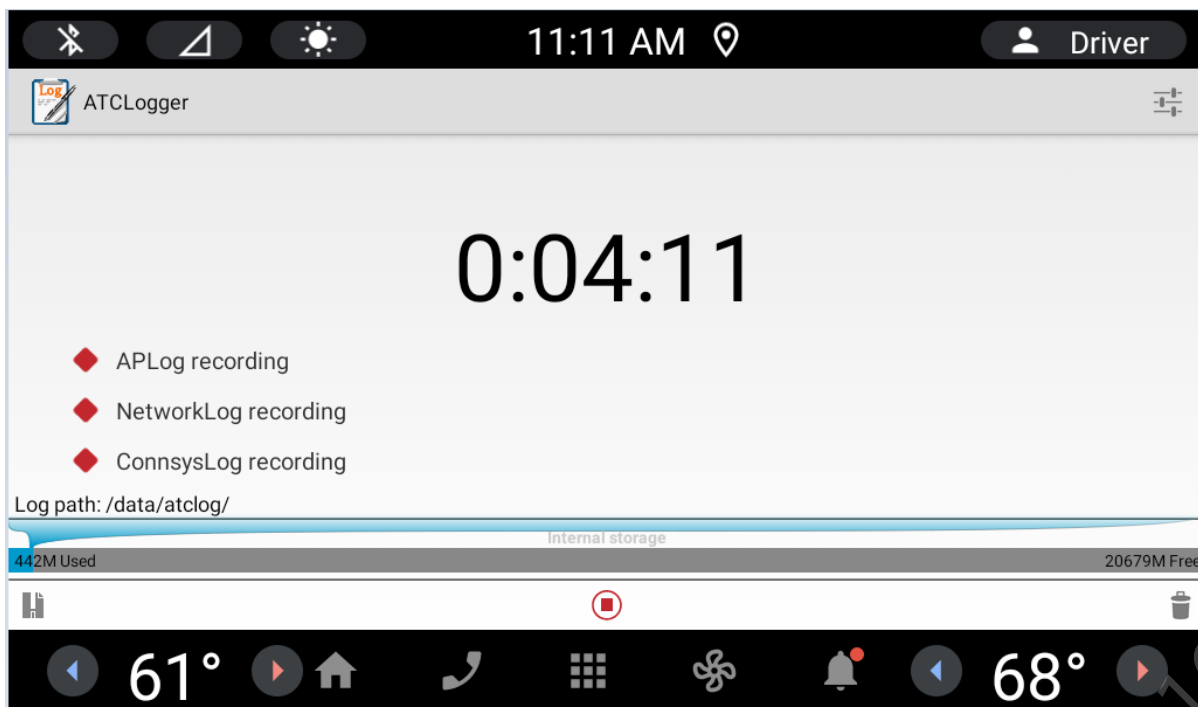


图 13-15 ATCLogger UI Stop

点击  这个按钮停止录制 atclogger。

13.3.3.1.3 settings

【描述】

在 Home 界面，点击 show more 进入 All apps 界面，再往下拉，找到 ATCLogger apk。

【操作】

点击 ATCLogger 选项，进入 ATCLogger 主界面：

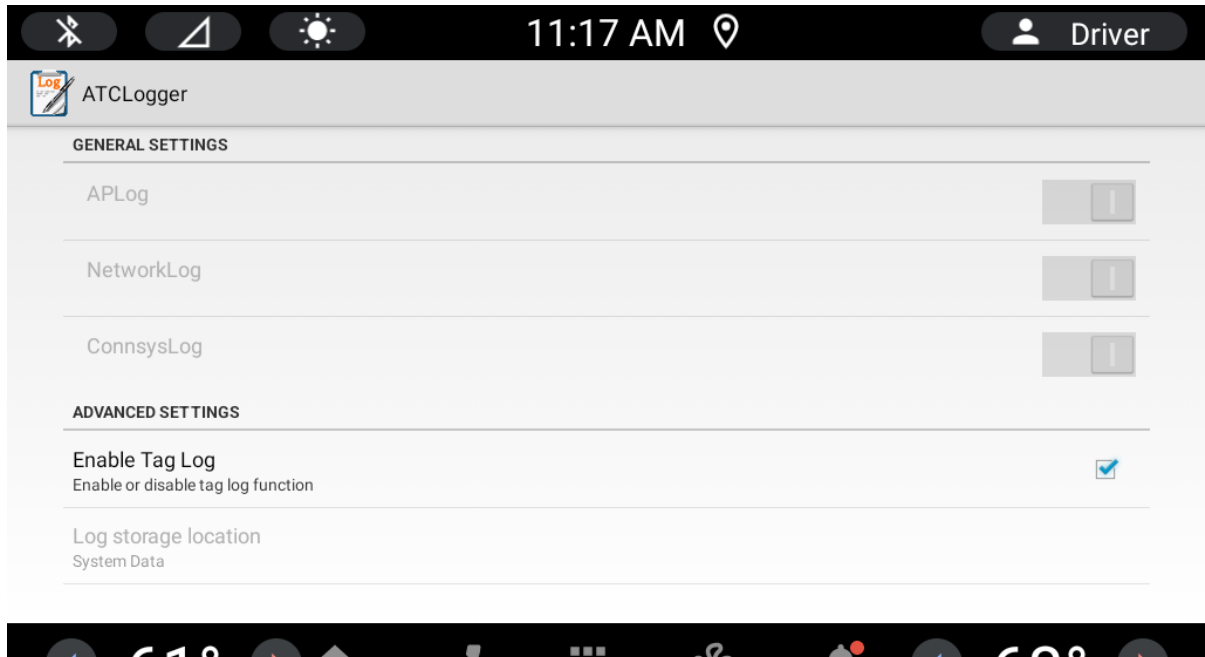


图 13-16 ATCLLogger UI Settings

点击  这个按钮配置 atcllogger 相关设置项。

1. 设置 log 储存路径

点击 Log storage location，可以选择"System Data"或者"Device Storage"。

System Data 表示录制的 Log 目录是 /data/atclog/，Device Storage 表示录制的 Log 目录是 /storage/emulated/0/atclog/

2. 删除 log 设置

点击 ATCLLogger 主界面右下角的垃圾桶按钮进入“删除 ATCLLogger 文件”界面。

屏幕右下方"全部删除"按钮可以删除全部的历史 Log。

屏幕中间的 Log List，点击进入可以选中删除单个 Log 文件。

3. 开关 tag log

勾选、取消高级设置中的 Tag Log 后的复选框。

4. 设置 log 容量

点击 APLog，进入 aplog 的设置界面，设置 Limit Current Log Size，这个是当前 log 目录的最大 size，设置 Limit Total Log Size，这个是整个 aplog 的最大 size。

13.3.3.2 adas/cluster 接口描述

- 接口文件：universal_utils/include/clog.h
- 封装 syslog 接口：syslog、openlog、closelog、setlogmask
 - 各个模块必须定义 Module_ID
 - 各个子模块建议定义 Sub_ID
 - 接口函数：ATCLOGD/ATCLOGI/ATCLOGW/ATCLOGE

以 atclogd 为例：

函数原型 void atclogd(fmt, ...)

参数意义：fmt,... -- 格式化输出 string

- 接口实现：

```
#define ATCLOGD(fmt, args...) \
    syslog(LOG_DEBUG, "[D][%s][%s]" fmt, MOD_ID, SUB_ID, ##args)
#define ATCLOGI(fmt, args...) \
    syslog(LOG_INFO, "[D][%s][%s]" fmt, MOD_ID, SUB_ID, ##args)
#define ATCLOGW(fmt, args...) \
    syslog(LOG_WARNING, "[D][%s][%s]" fmt, MOD_ID, SUB_ID, ##args)
#define ATCLOGE(fmt, args...) \
    syslog(LOG_ERR, "[E][%s][%s]%s,%s" fmt, MOD_ID, SUB_ID, __file__, __line__, \
    ##args)
```

14 Camera

14.1 简介

14.1.1 软硬件环境

本章介绍 ATC Camera 框架：基于不同操作系统，提供访问 Camera 硬件设备，配置 Camera 功能的软件框架，客户可以基于该框架在不同系统的不同层级开发满足需求的 Camera 应用。

14.1.1.1 硬件原理说明

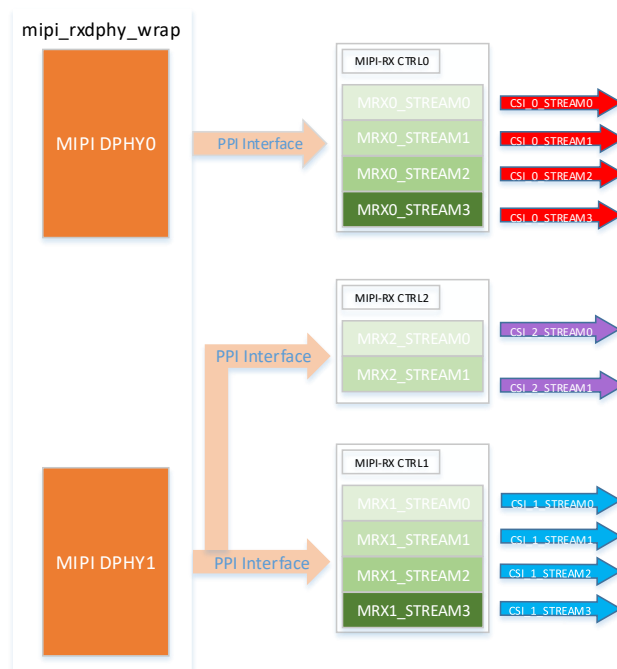


图 14-1 AC8025 MIPI 接口示意图

如上图所示，AC8025 支持 2 路 MIPI DPHY，其中 MIPI DPHY1 既可以独立接 1 个 Sensor 使用，硬件接 4 组 data lane 和 1 组 clock lane（4D1C Mode），后端 MIPI-RX Ctrl1 支持 4 个 Virtual Channel。也可以拆成 2 路接 2 个 Sensor，各自接 2 组 data lane 和 1 组 clock lane（2D1C Mode），后端 RX Ctrl1 仍然支持 4 个 Virtual Channel，但 Rx Ctrl2 只支持 2 个 Virtual Channel。拆分后，单个 Sensor 只有 2 组 data lane，单位时间数据传输量会下降，但总体支持的 Virtual Channel 数变多。客户可以根据自己的应用场景进行选择使用。

该配置硬件确定后，需要在 Metazone 或代码中进行相应配置，见 14.4.2 章节介绍。

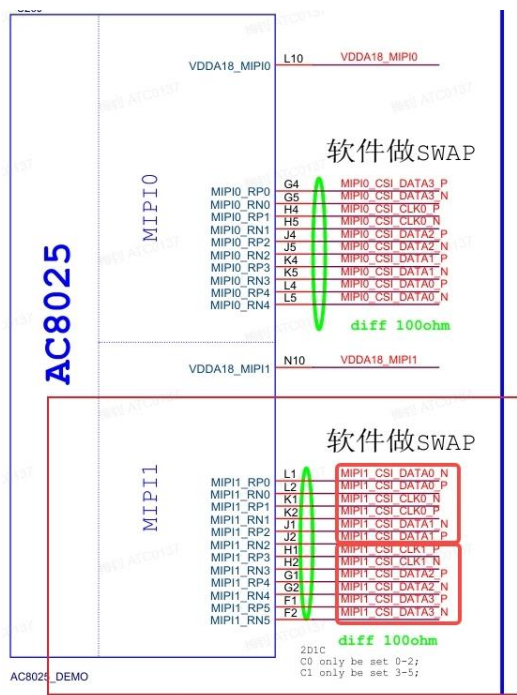


图 14-2 AC8025 MIPI 硬件原理图

以 ATC 开发板为例，MIPI0 接入一路 Sensor，可以支持 4 路 1080P@30 的 Camera 开发 AVM 等应用，MIPI1 被拆成 2 个 2D1C 模式，接入两个 Sensor，分别接入 1 路 Camera，进行 RVC 和 DVR 等不同的应用开发。

14.1.1.2 软件重要模块

- CameraSource Lib: Camera 驱动操作集合的 User 层封装库，并进行一些定制化功能的配置和扩展
- Videoin Driver: Camera 硬件访问的驱动程序
- Sensor Driver: Camera 外接 Sensor 的控制驱动程序

14.1.2 功能

AC8025 Camera 提供以下功能支持：

- 支持同一个系统下，一路 MIPI CSI 接入的多路 Camera 可被不同的应用独立控制

例如，Android 系统下，同一个 MIPI CSI 接入的两路 Camera 可以分别在 DMS 和 DVR 两个应用独立使用。

- 支持每路 Camera 输出图像数据的分流，且每路数据流的格式和分辨率可以不同，应用层可独立控制

例如，一个 1080P Camera 可以输出 2 路独立的数据流，可以将一路输出用于 DMS 应用，配置为 1080P NV21 格式，将另一路输出用于 DVR 应用，配置为 720P YV12 格式。

- 支持多路 Camera 输出图像的数据组合

例如，将 4 路 Camera 图像组合成 2x2 的田字格数据流输出，供 AVM 应用使用。

- 支持每路 Camera 图像质量和镜像等的独立调节

例如，亮度、对比度、Gamma 等图像质量的调节。

AC8025 Camera 提供以下接口支持：

- 可基于 Android Camera API1/API2 进行 Android Camera 应用开发
- 可基于 CameraSource Lib 接口，进行 Native 层 Camera 应用开发

14.1.3 代码结构及文件描述

14.1.3.1 CameraSource

表 14-1 CameraSource 代码结构及描述

文件名	文件路径	文件描述
ICameraManeger.h	ivi: source/android/vendor/autochips/libs/ac8025/ cockpit(ivi)/camerasource/ac8025/include adas: adas/prebuilts/ac8025_demo/camerasource/ac8025/include	CameraSource 对外头文件，包含相关功能接口
ICameraSource.h	ivi: source/android/vendor/autochips/libs/ac8025/cockpit(ivi) /camerasource/ac8025/include adas: adas/prebuilts/ac8025_demo /camerasource/ac8025/include	CameraSource 对外头文件，包含相关功能接口
libcamera.source.so	source/android/vendor/autochips/libs/ac8025/cockpit(ivi)/camerasource/lib source/android/vendor/autochips/libs/ac8025/ cockpit(ivi) /camerasource/lib64	IVI 域 Build CameraSource 生成的库文件
libcamerasource.so	adas/prebuilts/ac8025_demo/camerasource	ADAS 域 Build CameraSource 生成的库文件
atc_camera_config.xml	build/projects/cockpit_s-linux_ac8025/ac8025_demo/videoin/ build/projects/ivi_s_ac8025/ac8025_demo/videoin/	CameraSource 需要用到的配置文件，支持增加

文件名	文件路径	文件描述
		CameraDevice、 删除或修改某个 CameraDevice

14.1.3.2 Videoin Driver

表 14-2 Videoin Driver 代码结构及描述

文件名	文件路径	文件描述
Videoin Driver	source/kernel/atc_modules/videoin/ac8025/	Videoin 驱动代码

14.1.3.3 Sensor Driver

表 14-3 Sensor Driver 代码结构及描述

文件名	文件路径	文件描述
Sensor Driver	source/kernel/atc_modules/videoin/ac8025/sensor	Sensor 驱动代码

14.2 软件设计

14.2.1 架构图

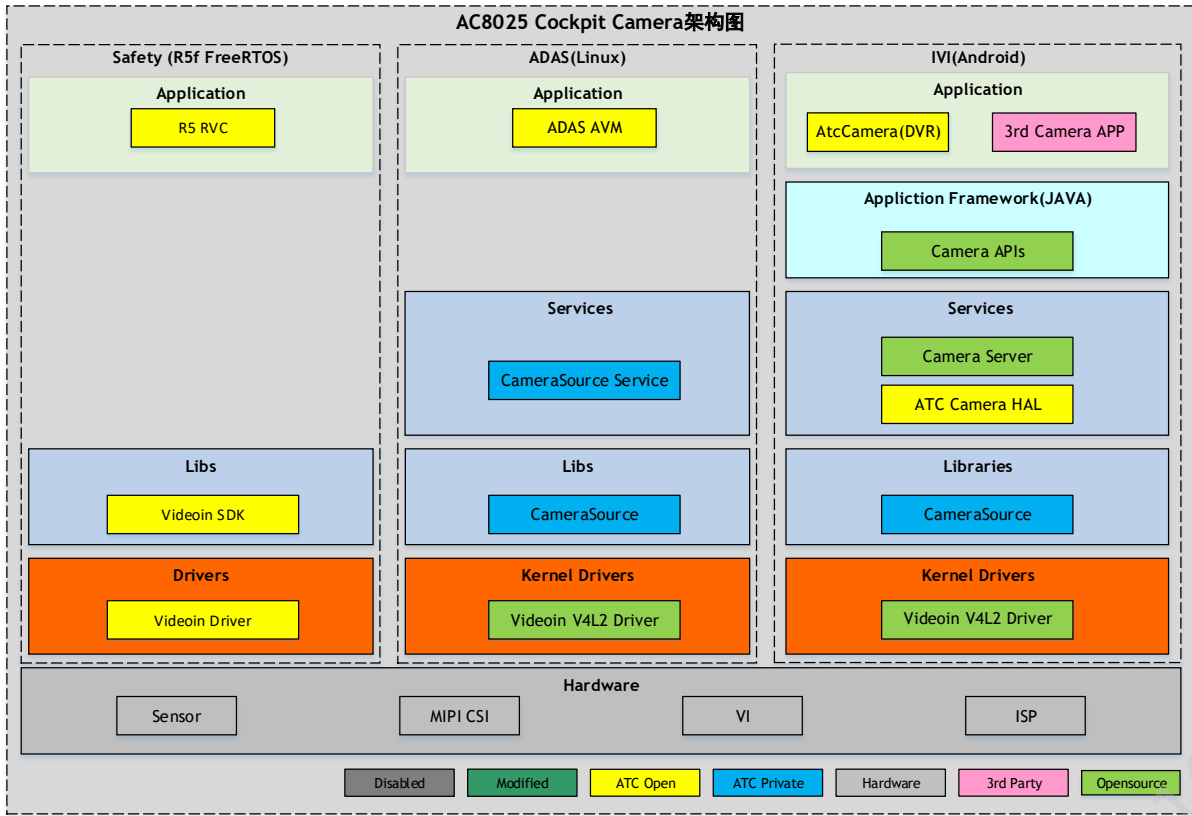


图 14-3 AC8025 Camera 架构图

14.3 接口描述

14.3.1 API

14.3.1.1 CameraSource

14.3.1.1.1 IEventCallback 类

Camera 事件变化的回调函数

表 14-4 IEventCallback 类

Public方法	说明
----------	----

IEventCallback()	构造函数
~IEventCallback()	析构函数
void onEvent()	Camera 事件变化处理函数

14.3.1.1.1 onEvent

【描述】

Camera 事件变化处理函数。当 camera 事件发生时，会调用该函数。

【语法】

```
public virtual void onEvent(enum CamEvent event, int arg1, int arg2);
```

【返回值】

None

【参数】

- event event ID
- arg1 for CAM_SIGNAL_CHANGED, means camera channel
- arg2 for CAM_SIGNAL_CHANGED, means camera signal status

【注意】

目前只支持 signal 事件处理

```
/** Camera event ID*/
enum CamEvent {
    CAM_SIGNAL_CHANGED = 1, /**< Camera evnet for signal change. onEvent's arg1
= channel number, arg2 = camSignal*/
    CAM_EVENT_MAX,
};
```

14.3.1.1.2 ICameraStream 类

表 14-5 ICameraStream 类

Public方法	说明
~ICameraStream()	析构函数
int getFormatList()	获取当前CameraStream对象支持的format列表
int config()	配置当前CameraStream对象的输出format; 以及选择用于CameraSource轮转buffer的申请方式及buffer个数。

int start()	启动当前CameraStream对象
int stop()	停止当前CameraStream对象
int dequeue()	从当前CameraStream对象获取一帧数据
int enqueue()	向当前CameraStream对象还一个buffer
int setMirror()	设置镜像
void dumpDebugInfo()	获取当前CameraStream对象的相关debug信息
int32_t queryCapabilities()	查询当前CameraStream对象支持的能力
int32_t setOsdParam()	设置当前CameraStream对象的OSD参数
int32_t getOsdParam()	获取当前CameraStream对象的OSD参数
int32_t setFbcState()	设置当前CameraStream对象的FBC使能状态
int32_t getFbcState()	获取当前CameraStream对象的FBC使能状态
int32_t getCameraPosition()	当DEVICE_TYPE是MULTI_DEVICE时，获取camera的位置信息
int32_t setCameraPosition()	当DEVICE_TYPE是MULTI_DEVICE时，设置camera的位置信息

14.3.1.1.2.1 getFormatList

【描述】

获取当前 CameraStream 对象支持的 format 列表

【语法 1】

```
public virtual int getFormatList(std::vector<videoin_format> &list __attribute__((unused)));
```

【语法 2】

```
public virtual int getFormatList(std::vector<VideoFormat> &list __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- list: videoin_format/ VideoFormat vector reference object

```
struct videoin_format {
    unsigned int pixelfmt;
    unsigned int width;
    unsigned int height;
    unsigned int fps;
```

```

}

struct videoin_VideoFormat {
    unsigned int pixelfmt;
    unsigned int width;
    unsigned int height;
    unsigned int fps;
    unsigned int yStride;
    unsigned int cStride;
}

```

14.3.1.1.2.2 config

【描述】

配置当前 CameraStream 对象的输出 format；以及选择用于 CameraSource 轮转 buffer 的申请方式及 buffer 个数。

【语法 1】

```

public virtual int config(const videoin_format &format __attribute__((unused)),
    ALLOCATE_BUFFER_FLAG __attribute__((unused)) flag =
    ALLOCATE_BUFFER_BY_USER,unsigned int __attribute__((unused)) bufCount = 0);

```

【语法 2】

```

public virtual int config(const VideoFormat &format __attribute__((unused)),
    ALLOCATE_BUFFER_FLAG __attribute__((unused)) flag =
    ALLOCATE_BUFFER_BY_USER,unsigned int __attribute__((unused)) bufCount = 0);

```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- format: specify the videoin_format/VideoFormat which will be used by CameraStream
- flag: specify the buffer is allocated by user or CameraStream internal
- bufCount: specify buffer count allocatd by CameraStream

【注意】

- 配置的 format 需要从支持的 format 列表（getFormatList 获取）里选择

- typedef enum

```
{
    ALLOCATE_BUFFER_INNER = 0,
    ALLOCATE_BUFFER_EXTERNAL,
    ALLOCATE_BUFFER_BY_USER,
    ALLOCATE_BUFFER_MAX,
} ALLOCATE_BUFFER_FLAG;
```

当 flag 值为 0 或 1 时：由 CameraSource 模块申请 buffer，同时，bufCount 不能为 0；

当 flag 值为 2 时：由 CameraSource 调用者申请 buffer，此时 CameraSource 不关心 bufCount 的值。

14.3.1.1.2.3 start

【描述】

启动当前 CameraStream 对象，将配置并启动相关的 camera 硬件。一旦成功，CameraStream 将更改为 STARTED 状态，并准备 dequeue/enqueue。

【语法】

```
public virtual int start(void);
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- None

【注意】

- 前提条件：CameraStream 已通过 config()成功配置；

14.3.1.1.2.4 stop

【描述】

停止当前 CameraStream 对象，相关的 camera 硬件将被停止。一旦成功，CameraStream 将变更为 STOPPED 状态。

【语法】

```
public virtual int stop(void);
```


【返回值】

- 非 0: fail
- 0: success.

【参数】

None

【注意】

- 前提条件: CameraStream 已经处于 STARTED 状态;
- 当前 CameraStream 申请的 buffer 都将在此函数中释放。

14.3.1.1.2.5 dequeue**【描述】**

从当前 CameraStream 对象获取一帧数据

【语法 1】

```
public virtual int dequeue(bufferInfo *buffer, int timeout = 2000);
```

【语法 2】

```
public virtual int dequeue(bufferInfo *videoBuffer __attribute__((unused)), osdBufferInfo  
*osdBuffer __attribute__((unused)), fbcBufferInfo *fbcBuffer __attribute__((unused)), int  
__attribute__((unused)) timeout = 2000);
```

【返回值】

- 11:dequeue timeout
- <0:dequeue fail
- 0: success.

【参数】

- buffer/ videoBuffer: a pointer to BufferInfo. This pointer can't be nullptr.

```
/** Video Buffer information*/  
typedef struct _bufferInfo {  
    unsigned int bufIdx;      /**< Buffer index*/  
    int fd;                   /**< Buffer fd*/  
    unsigned int size;        /**< Buffer size in byte*/  
    /** if the stride is set, the videoin dirver uses the set stide.
```

```

        * If the stide value is zero, the allocated buffer width must be
        64 bytes aligned
        */

        unsigned int yStride;          /**< Data y stride*/
        unsigned int cStride;          /**< Data c stride*/
        unsigned int flags;            /**< Buffer flags. IS_BUFFER_ERROR or
not*/

        SIGNAL_STATUS_ENUM signalStatus;

        void *va;                      /**< IO Virtual memory address.*/
        void *pa;                      /**< Buffer physical memory address.
Reserved*/

        /**Handle of buffer memory. it used by allocator*/
        union {
            void *bufHandle;           /**< Reserved for buffer
allocator*/

            int intHandle;              /**< Reserved for buffer allocator*/
        };

        int64_t timestamp;             /**< Data timestamp in */
        std::vector<SIGNAL_STATUS_ENUM> signals;
    } bufferInfo;

```

- timeout: specify timeout in ms.
- osdBuffer: a pointer to osdBufferInfo

```

/** osd Buffer information*/
typedef struct _osdBufferInfo {
    int fd;                            /**< Buffer fd*/
    unsigned int size;                 /**< Buffer size in byte, must be
aligned*/
    unsigned int stride;               /**< Data stride*/
    void* va;                          /**< IO Virtual memory address.*/
    int64_t timestamp;                /**< Data timestamp in */
}osdBufferInfo;

```

- fbcBuffer: a pointer to fbcBufferInfo

```

/** fbc Buffer information*/
typedef struct _fbcBufferInfo {
    int    fd;                /**< Buffer fd*/
    unsigned int size;        /**< Buffer size in byte, must be
aligned*/
    void* va;                /**< IO Virtual memory address.*/
    bool fbc_bypass;
    struct FbcParam fbc_params[DATA_FMT_MAX_PLANE];
}fbcBufferInfo;

```

【注意】

- 前提条件：CameraStream 已经处于 STARTED 状态；
- 当未使用 OSD 和 FBC 时：语法 1 可直接使用；语法 2 使用时需将 osdBuffer、fbcBuffer 赋值为 nullptr；
- 当使用 OSD 或 FBC 时：只能使用语法 2；
- 多路流数据组合和 USB Camera 不支持语法 2。

14.3.1.1.2.6 enqueue

【描述】

向当前 CameraStream 对象还一个空闲 buffer

【语法 1】

```
public int enqueue(bufferInfo *buffer);
```

【语法 2】

```
public int enqueue(bufferInfo *videoBuffer __attribute__((unused)), osdBufferInfo *osdBuffer
__attribute__((unused)), fbcBufferInfo *fbcBuffer __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- buffer/ videoBuffer: the bufferInfo address. This pointer can't be nullptr. Va and size are recommended to be assigned, otherwise dump data will fail.
- osdBuffer: the osdBufferInfo address

- fbcBuffer: the fbcBufferInfo address

【注意】

- 前提条件: CameraStream 已经处于 STARTED 状态;
- 当未使用 OSD 和 FBC 时: 语法 1 可直接使用; 语法 2 使用时需将 osdBuffer、fbcBuffer 赋值为 nullptr;
- 当使用 OSD 或 FBC 时: 只能使用语法 2;
- 多路流数据组合和 USBCamera 不支持语法 2。

14.3.1.1.2.7 setMirror

【描述】

设置镜像值。

【语法】

```
public virtual int setMirror(int value __attribute__((unused)), int __attribute__((unused)) channel = 0);
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- value: the mirror value
- channel: e channel value

【注意】

设置镜像是针对 channel 的, 即如果一个 CameraStream 设置了镜像, 则同一个 channel 的其他 CameraStream 也会跟着一起镜像。

14.3.1.1.2.8 dumpDebugInfo

【描述】

获取当前 CameraStream 对象的相关 debug 信息

【语法】

```
public virtual void dumpDebugInfo();
```

【返回值】

None

【参数】

None

14.3.1.1.2.9 queryCapabilities

【描述】

查询当前 CameraStream 对象支持的能力

【语法】

```
public virtual int32_t queryCapabilities(struct Capability &capability __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- capability: e capabilities of CameraStream

```
struct Capability {
    bool  supportHue;    /**< true:support;false:unsupport*/
    bool  supportBrightness; /**< true:support;false:unsupport*/
    bool  supportSaturation; /**< true:support;false:unsupport*/
    bool  supportContrast; /**< true:support;false:unsupport*/
    bool  supportMirror;  /**< true:support;false:unsupport*/
    bool  supportCrop;    /**< true:support;false:unsupport*/
    bool  supportOsd;     /**< true:support;false:unsupport*/
    bool  supportFbc;     /**< true:support;false:unsupport*/
};
```

14.3.1.1.2.10 setOsdParam

【描述】

设置当前 CameraStream 对象的 OSD 相关参数

【语法】

```
public virtual int32_t setOsdParam(const OsdParam &osd_param __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- `osd_param`: specify the `OsdParam` which will be used by `CameraStream`

```
struct OsdParam {
    bool osd_en;                /**< true:enable;false:disable */
    uint32_t global_a_data;     /**< global alpha data */
    uint32_t format;            /**< osd data format */
    uint32_t x_start;           /**< osd x start position */
    uint32_t y_start;           /**< osd y start position */
    uint32_t stride;            /**< osd rgb data stride */
    uint32_t width;             /**< osd data width */
    uint32_t height;            /**< osd data height */
};
```

【注意】

- 如果使用 OSD 功能，则不支持 `CameraSource` 模块本身分配 `video_buffer`，需由 `CameraSource` 调用者分配 `buffer`，即调用 `config` 接口时 `flag` 必须设置为 `ALLOCATE_BUFFER_BY_USER`;
- `buffer/video_buffer` 如果是由 `CameraSource` 分配，则不支持配置 OSD;
- `STARTED` 状态下不允许再配置 OSD。

14.3.1.1.2.11 `getOsdParam`

【描述】

获取当前 `CameraStream` 对象的 OSD 相关参数

【语法】

```
public virtual int32_t getOsdParam(struct OsdParam &osd_param __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- `osd_param` the `OsdParam` of current `CameraStream`

【注意】

与 `setOsdParam` 配合使用。

14.3.1.1.2.12 `setFbcState`

【描述】

设置当前 `CameraStream` 对象的 FBC 使能状态

【语法】

```
public int32_t setFbcState(bool value __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- `value`: specify the `Fbc` state of current `CameraStream`; true:enable fbc; false:disable fbc

【注意】

- 此接口调用要在调用 `config` 接口之后，`start` 接口之前；
- 开启了 `waterMark` 时不能使用 FBC 功能。

14.3.1.1.2.13 `getFbcState`

【描述】

获取当前 `CameraStream` 对象的 FBC 使能状态

【语法】

```
public virtual int32_t getFbcState(bool &value __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- `value`: the `Fbc` state of current `CameraStream`; true: fbc is enable; false: fbc is disable

【注意】

与 setFbcState 配合使用。

14.3.1.1.2.14 getCameraPosition

【描述】

当 DEVICE_TYPE 是 MULTI_DEVICE 时，获取 camera 的位置信息

【语法】

```
public virtual int32_t getCameraPosition(std::vector<std::vector<uint32_t>>
&cameraCh2position __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- cameraCh2postion: avmPort vector for save avmPort info
- 里层 vector 存储的是某行的 virtual channel numbers，size 大小表示拼接列数。
- 外层 vector 存储的是每行的 virtual channel numbers，size 大小表示拼接行数。

【注意】

- 当 DEVICE_TYPE 不是 MULTI_DEVICE 时，无需使用此接口；
- 与 14.3.1.1.2.15 setCameraPosition 配合使用。

14.3.1.1.2.15 setCameraPosition

【描述】

当 DEVICE_TYPE 是 MULTI_DEVICE 时，设置 camera 的位置信息

【语法】

```
public virtual int32_t setCameraPosition(const std::vector<std::vector<uint32_t>>
&cameraCh2position __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- cameraCh2postion : avmPort vector to be set

- 里层 vector 存储的是某行的 virtual channel numbers，size 大小表示拼接列数。
- 外层 vector 存储的是每行的 virtual channel numbers，size 大小表示拼接行数。

【注意】

- 当 DEVICE_TYPE 不是 MULTI_DEVICE 时，无需使用此接口；
- 必须要在 start 或 enqueue 之前（即 setPosition->start->enqueue 或 setPosition->enqueue->start）。否则，此接口返回 fail。
- 如果切换了拼接模式，则要在调用 setPosition 之后需要重新调用 getFormatList->config 接口，即 setPosition->getFormatList->config->start/enqueue，否则可能会 start 或 enqueue 失败。
- 当 atc_camera_config.xml 文件中的 DEVICE_TYPE 不是 MULTI_DEVICE 且 CameraKind 属性值是 PUBLIC 时，此 CameraDevice 的每路 CameraStream 的拼接模式需保持一致。

14.3.1.1.3 ICameraSource 类

表 14-6 ICameraSource 类

Public方法	说明
~ICameraSource()	析构函数
unsigned int getStreamNum()	获取当前CameraSource对象的CameraStream元素的个数
ICameraStream *acquireCameraStream()	获取一个CameraStream对象
int releaseCameraStream()	释放acquireCameraStream返回的某个CameraStream对象
int open()	打开camera device
int close()	关闭camera device
int registerCallback()	设置事件回调
int getCameraPosition()	当DEVICE_TYPE是MULTI_DEVICE时，获取camera的位置信息
int setPosition()	当DEVICE_TYPE是MULTI_DEVICE时，设置camera的位置信息
int setColorValue()	设置camera画面的颜色参数
int switchChannel()	切换显示对应channel的camera画面

14.3.1.1.3.1 getStreamNum

【描述】

获取当前 CameraSource 对象 CameraStream 元素的个数

【语法】

```
public virtual unsigned int getStreamNum(void);
```

【返回值】

number of CameraStream of current CameraSource

【参数】

None

14.3.1.1.3.2 acquireCameraStream

【描述】

获取一个 CameraStream 对象

【语法】

```
public virtual ICameraStream *acquireCameraStream(void);
```

【返回值】

- a camera stream object pointer: success
- nullptr: fail

【参数】

None

【注意】

调用 open 接口之后调用该接口。

14.3.1.1.3.3 releaseCameraStream

【描述】

释放acquireCameraStream返回的某个CameraStream对象

【语法】

```
public virtual int releaseCameraStream(ICameraStream *stream);
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

318 /1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

- stream: pointer to ICameraStream object which returned by acquireCameraStream()

【注意】

- stream 必须为当前 CameraSource 调用 acquireCameraStream 返回的;
- 调用 close 接口之前调用该接口
- 当 CameraStream 对象不再使用时, 调用此接口释放 CameraStream 对象, 与 acquireCameraStream 配合使用。

14.3.1.1.3.4 open

【描述】

打开 camera device

【语法】

```
public virtual int open(void);
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

None

14.3.1.1.3.5 close

【描述】

关闭 camera device

【语法】

```
public virtual int close(void);
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

None

14.3.1.1.3.6 registerCallback

【描述】

设置当前 CameraSource 对象的事件回调

【语法】

```
public virtual int registerCallback(IEventCallback *pCallback);
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- pCallback: a pointer to IEventCallback object

14.3.1.1.3.7 getCameraPosition

【描述】

当 DEVICE_TYPE 是 MULTI_DEVICE 时，获取 camera 的位置信息

【语法】

```
public virtual int getCameraPosition(std::map<CAMERA_CHANNEL_ENUM,  
POSITION_ENUM> &cameraCh2postion __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

cameraCh2postion: avmPort vector for save avmPort info

【注意】

- 当 DEVICE_TYPE 不是 MULTI_DEVICE 时，无需使用此接口；
- 与 14.3.1.1.3.8 setCameraPosition 配合使用。
- 与 14.3.1.1.2.14 对比：（1）14.3.1.1.2.14 是针对 CameraStream 的接口，每个 CameraStream 的此接口调用是独立的，更灵活，支持多路 camera 一字格拼接、田字格拼接多种拼接模式；（2）而此接口是针对 CameraSource 的，无法针对 CameraStream 进行获取。另外此接口只支持四路田字格拼接，其他拼接模式均不支持。

14.3.1.1.3.8 setCameraPosition

【描述】

当 DEVICE_TYPE 是 MULTI_DEVICE 时，设置 camera 的位置信息

【语法】

```
public virtual int setCameraPosition(const std::map<CAMERA_CHANNEL_ENUM,  
POSITION_ENUM> &cameraCh2postion __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- cameraCh2postion: avmPort vector to be set

```
/** virtual channel enum*/  
typedef enum  
{  
    CAMERA_CHANNEL_ZERO = 0,      /**< virtual channel 0*/  
    CAMERA_CHANNEL_ONE,           /**< virtual channel 1*/  
    CAMERA_CHANNEL_TWO,           /**< virtual channel 2*/  
    CAMERA_CHANNEL_THREE,         /**< virtual channel 3*/  
    CAMERA_CHANNEL_MAX,           /**< channel max*/  
} CAMERA_CHANNEL_ENUM;  
  
/** camera position enum*/  
typedef enum  
{  
    POSITION_TOP_LEFT = 0,  
    POSITION_TOP_RIGHT,  
    POSITION_BOTTOM_LEFT,  
    POSITION_BOTTOM_RIGHT,  
    POSITION_MAX,  
} POSITION_ENUM;
```

【注意】

- 当 DEVICE_TYPE 不是 MULTI_DEVICE 时，无需使用此接口；
- enqueue 之后不支持调用此接口，否则，此接口返回 fail。
- 与 14.3.1.1.2.15 对比：（1）14.3.1.1.2.15 是针对 CameraStream 的接口，每个 CameraStream 的此接口调用是独立的，更灵活，支持多路 camera 的一字格拼接、田字格拼接多种拼接模式；（2）而此接口是针对 CameraSource 的，无法针对 CameraStream 进行设置，即同一个 CameraSource 的所有 CameraStream 会一起生效。另外此接口只支持四路田字格拼接，其他拼接模式均不支持。

14.3.1.1.3.9 setColorValue

【描述】

设置 camera 画面的颜色参数

【语法】

```
public virtual int setColorValue(IMG_COLOR_TYPE_ENUM colorType __attribute__((unused)),
int value __attribute__((unused)),int __attribute__((unused)) channel = 0);
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

- colorType: the color type

```
typedef enum
{
    IMG_COLOR_TYPE_HUE = 0,          /**< value range:[-128,127],default
value:0*/
    IMG_COLOR_TYPE_BRIGHTNESS,      /**< value range:[-128,127],default
value:0*/
    IMG_COLOR_TYPE_SATURATION,       /**< value range:[0,255],default
value:128*/
    IMG_COLOR_TYPE_CONTRAST,         /**< value range:[0,255],default
value:128*/
    IMG_COLOR_TYPE_MAX,
} IMG_COLOR_TYPE_ENUM;
```

- value: the color value
- channel: the channel value

【注意】

此接口是针对 channel 的，同一个 CameraSource 的所有 CameraStream 会一起生效。不支持 HUE 设置。

14.3.1.1.3.10 switchChannel

【描述】

切换显示对应 channel 的 camera 画面

【语法】

```
public virtual int switchChannel(int __attribute__((unused)) channel = 0);
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

channel: the channel value

14.3.1.1.4 ICameraCallback 类

表 14-7 ICameraCallback 类

Public方法	说明
~ICameraCallback ()	析构函数
void cameraStatusChange()	camera状态改变处理函数

14.3.1.1.4.1 cameraStatusChange

【描述】

camera状态改变处理函数，主要用于热插拔设备

【语法】

```
public virtual void cameraStatusChange(uint32_t cameraId, CameraStatus cameraStatus);
```

【返回值】

None

【参数】

- cameraId: logical Camera ID
- cameraStatus: the camera status

```
/** Camera device status*/
enum CameraStatus {
    /** The camera device is not currently connected*/
    NOT_PRESENT = 0,

    /** The camera device is connected, and opening it is possible, as
    long as
    * sufficient resources are available. */
    PRESENT = 1,
};
```

【注意】

- 当 camera 状态发生变化时，会调用此函数；
- 目前只有热插拔设备（DeviceType 为 EXTERNAL_DEVICE）使用。

14.3.1.1.5 ICameraManager 类

表 14-8 ICameraManager 类

Public方法	说明
~ICameraManager()	析构函数
ICameraManager *getCameraManager()	获取一个CameraManager对象
bool getCameraList()	获取当前支持的camera device（cameraId）列表
bool getCameraInfo()	获取指定camera device的静态信息
ICameraSource *getCameraInstance()	获取指定cameraId的CameraSource对象
int freeCameraInstance()	销毁getCameraInstance返回的某个CameraSource对象
int registerCallback()	设置camera状态变化的回调函数

14.3.1.1.5.1 getCameraManager

【描述】

获取一个 CameraManager 对象

【语法】

```
public static ICameraManager *getCameraManager();
```

【返回值】

一个静态的 CameraManager 对象

【参数】

None

14.3.1.1.5.2 getCameraList

【描述】

获取当前支持的 camera device (cameraId) 列表

【语法】

```
public virtual bool getCameraList(std::vector<unsigned int> &cameraList);
```

【返回值】

- false: fail
- true: success.

【参数】

cameraList: vector<unsigned int> object for saving the list of camera device

【注意】

获取到的 camera device (cameraId) 列表不包含热插拔设备 (DeviceType 为 EXTERNAL_DEVICE)。

14.3.1.1.5.3 getCameraInfo

【描述】

获取指定 camera device 的静态信息

【语法】

```
public virtual bool getCameraInfo(unsigned int cameraId, CameraStaticInfo &cameraInfo);
```

【返回值】

- false: fail
- true: success.

【参数】

- cameraId: camera id of camera device
- cameraInfo: parameter for saving CameraStaticInfo

【注意】

cameraId 需要是 getCameraList 其中的一个或者是热插拔设备（DeviceType 为 EXTERNAL_DEVICE）。

14.3.1.1.5.4 getCameraInstance

【描述】

获取指定 cameraId 的 CameraSource 对象

【语法】

```
public virtual ICameraSource *getCameraInstance(unsigned int cameraId);
```

【返回值】

- CameraSource object pointer: success
- NULL: fail

【参数】

cameraId: camera id of camera device

【注意】

cameraId 需要是 getCameraList 其中的一个或者是热插拔设备（DeviceType 为 EXTERNAL_DEVICE）。

14.3.1.1.5.5 freeCameraInstance

【描述】

销毁getCameraInstance返回的某个CameraSource对象

【语法】

```
public virtual int freeCameraInstance(ICameraSource *instance);
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

instance: pointer of CameraSource object which returned by getCameraInstance ()

【注意】

- instance 必须为当前 CameraManager 调用 getInstance 返回的；
- 调用 close 接口之后且 CameraSource 对象不再使用时，调用此接口释放 CameraSource 对象，与 getInstance 配合使用。

14.3.1.1.5.6 registerCallback

【描述】

设置 camera 状态变化的回调函数

【语法】

```
virtual int registerCallback(ICameraCallback *pCameraDeviceCallback __attribute__((unused)));
```

【返回值】

- 非 0: fail
- 0: success.

【参数】

pCameraDeviceCallback: a pointer to ICameraDeviceCallback object

【注意】

目前只有热插拔设备（DeviceType 为 EXTERNAL_DEVICE）使用。

14.3.2 数据结构

14.3.2.1 CameraSource

表 14-9 CameraSource 数据结构

数据结构	说明
CameraStaticInfo	Camera 静态信息

14.3.2.1.1 CameraStaticInfo

```
struct CameraStaticInfo {
    unsigned int id;    /**< ID of camera devcie */
    unsigned int type; /**< Camera device type (0:normal, 1: 2x2 mode) */
    int streamNum;     /**< Max streams of camera device supports */
    int facing;        /**< Facing (-1: invalid)*/
}
```

```
int rotation;      /**< Rotation of camera device*/

bool systemCamera; /**< indicate the camera is system camera or not*/

int width;         /**< Width (in pixel) of camera device*/

int height;        /**< Height (in pixel) of camera device*/

int fps;           /**< frame per seconds of camera device*/

unsigned int domainType; /**< Camera domain type (0:adas domain, 1:ivi domain, 2:cross
domain) */

std::vector<unsigned int> conflictCameraVec; /**< Cameras that conflict with the video
node of the current camera*/

};
```

14.4 使用说明和注意事项

14.4.1 Android Camera 框架定制

在 Android Camera 框架中增加了如下几种定制化 vendor tag，具体说明如下：

表 14-10 定制化 Vendor Tag 说明

Tag 名称	Tag 类型	说明
com.atc.sourceinf	TYPE_INT32	表征信号状态，其中 int[0]、int [1]分别对应 output 宽高，当均为非 0 值时，表示有信号，其他均表示无信号
com.atc.mirror	TYPE_INT32	表征镜像参数，其中 0 表示无镜像；1 表示：FLIP_H；2 表示 FLIP_V；3 表示 FLIP_H FLIP_V
com.atc.color_process.brightness	TYPE_INT32	表示亮度调节参数，范围在 0~255 之间
com.atc.color_process.saturation	TYPE_INT32	表示色调调节参数，范围在 0~255 之间
com.atc.color_process.contrast	TYPE_INT32	表示饱和度调节参数，范围在 0~255 之间

14.4.2 Sensor 使用的预配置

AC8025 SoC 支持三路 MIPI CSI In，每个 Sensor 又可以接入多路 Virtual Channel Camera，系统端又存在多系统域运行的场景，为了更加合理的分配 ISP 等硬件资源，提升 Camera 出图性能和工作稳定性，需要根据平台使用场景将 Sensor 的一些使用配置进行预设。

公版为了方便兼容性测试，默认使用 Metazone 进行配置动态修改和保存，各个系统域在启动阶段读取使用，Metazone 模块的详细介绍请参考第 33 章相关章节。客户也可以直接在 Sensor 的代码中修改默认值，而不使用 Metazone。

14.4.2.1 工作域预配置

为了保证 Sensor 工作的稳定性，目前设计实现一个 MIPI CSI 和接入的 Sensor 只能工作在一个系统域下，不支持多系统域的访问和控制。所以需要提前配置该 Sensor 工作的系统域，以便不同的系统域启动时只初始化各自的 Sensor，以及绑定分配的 ISP 等硬件资源的工作域。以下是预存 Sensor 工作域使用的 Metazone 节点及配置说明。

表 14-11 Sensor 工作域配置 Metazone 说明

Metazone Name	说明
METAZONE_MIPI0_WORKING_OS	- 配置MIPI0的工作域 - 值说明： 0: IVI_Android 1: ADAS_Linux 2: CLUSTER_Linux 3: IVI_Linux 5: Safety (R5F)
METAZONE_MIPI1_WORKING_OS	配置MIPI1的工作域

	值说明，同MIPI0
METAZONE_MIPI2_WORKING_OS	- 配置MIPI2的工作域 - 值说明，同MIPI0

例如，MIPI0 接入的 Sensor 如果是工作在 R5F 端，用于 R5F RVC 应用，则“METAZONE_MIPI0_WORKING_OS”写值为十进制：5。

14.4.2.2 各通道使能预配置

为了 Sensor 初始化时进行相应配置，合理分配 ISP 等硬件资源，需要提前获取各 Sensor 可能会使能并接入 Camera 的通道情况，以下是预存通道使能使用的 Metazone 节点及配置说明。

表 14-12 Sensor 通道使能配置 Metazone 说明

Metazone Name	说明
METAZONE_MIPI0_CH_ENABLE	- 配置MIPI0的各通道的使能 - 值说明： byte0: channel0: 0 Disable, 1 Enable byte1: channel1: 0 Disable, 1 Enable byte2: channel2: 0 Disable, 1 Enable byte3: channel3: 0 Disable, 1 Enable
METAZONE_MIPI1_CH_ENABLE	- 配置MIPI1的各通道的使能 - 值说明，同MIPI0
METAZONE_MIPI2_CH_ENABLE	- 配置MIPI2的各通道的使能（MIPI2只支持2个通道） - 值说明，同MIPI0

例如，MIPI0 VC0~3 使能了 4 个通道，则“METAZONE_MIPI0_CH_ENABLE”写值为十进制：16843009（0x01010101）。

14.4.2.3 各通道制式预配置

为了缩短 Sensor 初始化时间，加快 Camera 应用出图速度，设计使用了在 Metazone 中预存摄像头制式的方式，以删减 Sensor 每次启动检测插入摄像头制式的初始化流程耗时。以下是预存通道制式使用的 Metazone 节点及配置说明：

表 14-13 Sensor 通道制式配置 Metazone 说明

Metazone Name	说明
METAZONE_MIPI0_CH_RESOLUTION	MIPI0 Sensor接入的各摄像头预存的制式： - Total 4 Byte - byte 0 ~3分别表示virtual channel 0~3 接

	入的摄像头制式。 ■ 值说明： 0: Invalid 1: Ntsc 2: Pal 3: HD_25 4: HD_30 5: FHD_25 6: FHD_30; (可扩展)
METAZONE_MIPI1_CH_RESOLUTION	■ 配置MIPI1的各通道的制式 值说明，同MIPI0
METAZONE_MIPI2_CH_RESOLUTION	■ 配置MIPI2的各通道的制式（MIPI2只支持2个通道） ■ 值说明，同MIPI0

例如，MIPI0 VC0~3 分别接入了 4 个 HD_30 的摄像头，则“METAZONE_MIPI0_CH_RESOLUTION”写值为十进制：67372036（0x04040404）。

如果需要扩展分辨率枚举，需要配合修改 Sensor 驱动，修改步骤请参考 14.5.1 章节的相关内容。

14.4.2.4 MIPI1 工作模式配置

如 14.1.1 章节介绍，MIPI1 可以工作在 4D1C 和 2D1C 两种模式下，当硬件接法确定后，需要将配置传给软件。

表 14-14 MIPI1 工作模式配置 Metazone 说明

Metazone Name	说明
METAZONE_MIPI1_LANE_MODE	MIPI1 工作模式： ■ 值说明： 0: 4D1C Mode 1: 2D1C Mode

14.4.3 Camera 数据流的使用

在本章简介部分介绍了 Camera 的相关功能，提到了一路 MIPI CSI 接入的多路 Camera 的独立使用，一路 Camera 的多路数据流的独立使用，多路数据流的组合等，本小节会结合以下图示做出说明，便于客户理解和开发，为 14.4.4 Camera 配置文件的使用说明做铺垫。

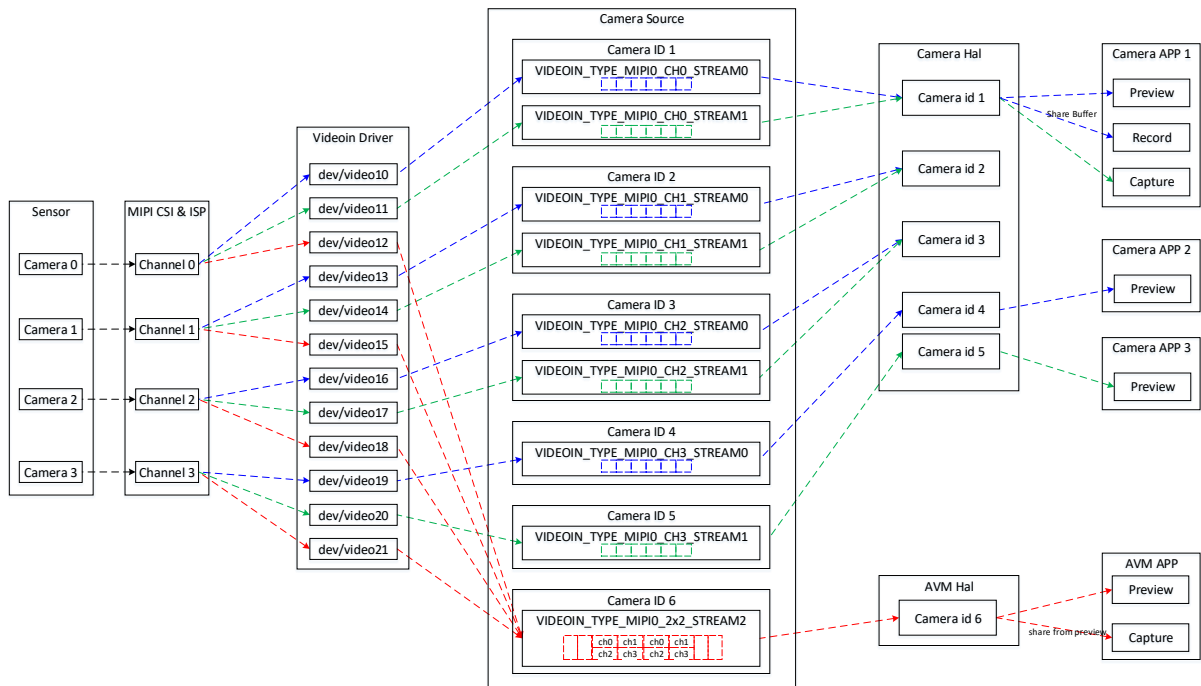


图 14-4 AC8025 Camera 数据流使用

上图中通过一路 MIPI CSI 接入的 4 路 Camera，经过 ISP 硬件处理后，每路 Camera 都可以出 3 路独立的数据流（分别用不同颜色标识不同的使用流向）：

- **Videoin Driver:** 将每一路数据流独立成一个设备节点供 User 层访问，并进行必要的虚拟化，保证所有设备节点的数据流的独立性。

例如，图中的 dev/video10~dev/video21 每个节点提供的数据流都是独立的，开关配置等控制相互之间没有影响。

- **CameraSource:** 根据使用场景，通过配置文件自由组合数据流（详见 14.4.4 Camera 配置文件），并上报组合出的“虚拟”Camera ID，供 CameraSource 使用者操作。

例如，图中通过配置文件，组合出 Camera ID 1~6：其中 Camera ID1~3，分别对应接入的物理 Camera0~2，每个 CameraID 可以出 2 路数据流；Camera ID4~5，对应接入的物理 Camera3，每个 CameraID 可以出 1 路数据流；Camera6 由 4 个物理 Camera 分别提供一路数据流组成 1 路 2x2 田字格数据流

- **Camera Hal / AVM Hal:** 通过 CameraSource 接口，访问其上报的各个 Camera ID。如果数据流个数存在不足，还可以使用其他软件方式，扩展数据流，如 Android Camera 框架提供的 Surface Sharing 机制等（参考谷歌 Camera 相关介绍）。

14.4.4 Camera 配置文件

使用 `atc_camera_config.xml` 文件对平台的 Camera 进行定制化配置，以满足不同场景应用的开发需要。示例如下：

```
<CameraDevice>
    <Enable>TRUE</Enable>
    <CameraId>10</CameraId>
    <CameraFacing>BACK</CameraFacing>
    <CameraRotation>0</CameraRotation>
    <SystemCamera>FALSE</SystemCamera>
    <CameraKind>PUBLIC</CameraKind>
    <DeviceType>NORMAL_DEVICE</DeviceType>
    <CameraStream>
        <DomainType>IVI_DOMAIN</DomainType>
        <VideoSource>video30</VideoSource>
    </CameraStream>
    <CameraStream>
        <DomainType>IVI_DOMAIN</DomainType>
        <VideoSource>video31</VideoSource>
    </CameraStream>
</CameraDevice>
```

配置文件字段说明：

表 14-15 `atc_camera_config.xml` 配置字段说明

字段	字段说明
Enable	表示当前 CameraDevice 的可用状态，可选范围：TRUE（此可用状态）、FALSE（不可用状态）。
CameraId	表示当前 CameraDevice 的枚举值（大于等于 0 的整数）。可通过此 Id 从 CameraSource 中获取到此 CameraDevice。
CameraFacing	表示当前 CameraDevice 为前置或后置，可选范围：FRONT、BACK、EXTERNAL（用于 external cameras）。
CameraRotation	表示当前 CameraDevice 的旋转角度，可选范围：顺时针旋转 0、90、180、270。
SystemCamera	表示当前 CameraDevice 是否可以由第三方应用打开，可选范围：TRUE（不能

字段	字段说明
	由第三方应用打开）、FALSE（可以由第三方应用打开）。
CameraKind	表示当前 CameraDevice 是否对所有应用可见，可选范围：PUBLIC（对所有应用可见）、HIDDEN_SECURE_CAMERA（仅对 hal 用户可见）。
DeviceType	表示当前 CameraDevice 的类型，可选范围：NORMAL_DEVICE（单 camera 场景）、MULTI_DEVICE（多路 camera 拼接场景）、VIRTUAL_DEVICE（虚拟设备）、EXTERNAL_DEVICE（外部设备，如 USBCamera）。
DomainType	表示当前 CameraStream 在哪个域工作，可选范围：ADAS_DOMAIN（仅在 ADAS 域可用）、IVI_DOMAIN（仅在 IVI 域可用）、CROSS_DOMAIN（挂载在 ADAS 域，但在 IVI 域使用）。
Row	当 CameraDevice 是 MULTI_DEVICE 时的可选属性，表示拼接行数
Column	当 CameraDevice 是 MULTI_DEVICE 时的可选属性，表示拼接列数
VideoSource	表示当前 CameraStream 使用到的设备节点，可以填当前设备中实际存在的 video 节点（eg: video30），也可以填 null（用于 EXTERNAL_DEVICE）。 Video 节点和实际数据流对应关系如下： video10 => MIPI0_CH0_PORT0, //MIPI0 的 Channel 0 的第 1 路数据流 video11 => MIPI0_CH0_PORT1, //MIPI0 的 Channel 0 的第 2 路数据流 video12 => MIPI0_CH0_PORT2, //MIPI0 的 Channel 0 的第 3 路数据流 video13 => MIPI0_CH1_PORT0, //MIPI0 的 Channel 1 的第 1 路数据流 video14 => MIPI0_CH1_PORT1, //MIPI0 的 Channel 1 的第 2 路数据流 video15 => MIPI0_CH1_PORT2, //MIPI0 的 Channel 1 的第 3 路数据流 video16 => MIPI0_CH2_PORT0, //MIPI0 的 Channel 2 的第 1 路数据流 video17 => MIPI0_CH2_PORT1, //MIPI0 的 Channel 2 的第 2 路数据流 video18 => MIPI0_CH2_PORT2, //MIPI0 的 Channel 2 的第 3 路数据流 video19 => MIPI0_CH3_PORT0, //MIPI0 的 Channel 3 的第 1 路数据流 video20 => MIPI0_CH3_PORT1, //MIPI0 的 Channel 3 的第 2 路数据流 video21 => MIPI0_CH3_PORT2, //MIPI0 的 Channel 3 的第 3 路数据流 video22 => MIPI1_CH0_PORT0, //MIPI1 的 Channel 0 的第 1 路数据流 video23 => MIPI1_CH0_PORT1, //MIPI1 的 Channel 0 的第 2 路数据流 video24 => MIPI1_CH0_PORT2, //MIPI1 的 Channel 0 的第 3 路数据流 video25 => MIPI1_CH1_PORT0, //MIPI1 的 Channel 1 的第 1 路数据流 video26 => MIPI1_CH1_PORT1, //MIPI1 的 Channel 1 的第 2 路数据流 video27 => MIPI1_CH1_PORT2, //MIPI1 的 Channel 1 的第 3 路数据流 video28 => MIPI1_CH2_PORT0, //MIPI1 的 Channel 2 的第 1 路数据流 video29 => MIPI1_CH2_PORT1, //MIPI1 的 Channel 2 的第 2 路数据流 video30 => MIPI1_CH2_PORT2, //MIPI1 的 Channel 2 的第 3 路数据流 video31 => MIPI1_CH3_PORT0, //MIPI1 的 Channel 3 的第 1 路数据流 video32 => MIPI1_CH3_PORT1, //MIPI1 的 Channel 3 的第 2 路数据流 video33 => MIPI1_CH3_PORT2, //MIPI1 的 Channel 3 的第 3 路数据流

字段	字段说明
	video34 => MIPI2_CH0_PORT0, //MIPI2 的 Channel 0 的第 1 路数据流 video35 => MIPI2_CH0_PORT1, //MIPI2 的 Channel 0 的第 2 路数据流 video36 => MIPI2_CH0_PORT2, //MIPI2 的 Channel 0 的第 3 路数据流 video37 => MIPI2_CH1_PORT0, //MIPI2 的 Channel 1 的第 1 路数据流 video38 => MIPI2_CH1_PORT1, //MIPI2 的 Channel 1 的第 2 路数据流 video39 => MIPI2_CH1_PORT2, //MIPI2 的 Channel 1 的第 3 路数据流

注意事项:

- 检查 CameraId 是否重复: 所有 CameraDevice 的 CameraId 不可重复;
- 检查各 CameraId 的 VideoSource 是否冲突: 有冲突的多个 CameraId 不可同时运行;
- 检查每个 CameraDevice 中 StreamNum 的值和 CameraStream 元素的个数是否一致;
- DeviceType 是 MULTI_DEVICE 时, 支持将多路 Camera 数据拼接成一字格、田字格模式。一个 CameraStream 元素需要对应多个 VideoSource 元素;
- 示例 1: (无 Row 和 Column 两个属性)

```

<CameraDevice>
    <Enable>TRUE</Enable>
    <CameraId>14</CameraId>
    <CameraFacing>BACK</CameraFacing>
    <CameraRotation>0</CameraRotation>
    <SystemCamera>TRUE</SystemCamera>
    <CameraKind>PUBLIC</CameraKind>
    <DeviceType>MULTI_DEVICE</DeviceType>
    <CameraStream>
        <DomainType>IVI_DOMAIN</DomainType>
        <VideoSource>video30</VideoSource>
        <VideoSource>video32</VideoSource>
        <VideoSource>video34</VideoSource>
        <VideoSource>video36</VideoSource>
    </CameraStream>
    <CameraStream>
        <DomainType>IVI_DOMAIN</DomainType>
        <VideoSource>video31</VideoSource>

```

```

        <VideoSource>video33</VideoSource>

        <VideoSource>video35</VideoSource>

        <VideoSource>video37</VideoSource>

    </CameraStream>
</CameraDevice>

```

以上示例无 Row 和 Column 两个属性，只支持四路田字格拼接模式。与 14.3.1.1.3.7 和 14.3.1.1.3.8 配合使用。

- 示例 2: (有 Row 和 Column 两个属性)

```

<CameraDevice>

    <Enable>TRUE</Enable>

    <CameraId>14</CameraId>

    <CameraFacing>BACK</CameraFacing>

    <CameraRotation>0</CameraRotation>

    <SystemCamera>TRUE</SystemCamera>

    <CameraKind>PUBLIC</CameraKind>

    <DeviceType>MULTI_DEVICE</DeviceType>

    <CameraStream>

        <Row>1</Row>

        <Column>4</Column>

        <DomainType>IVI_DOMAIN</DomainType>

        <VideoSource>video30</VideoSource>

        <VideoSource>video32</VideoSource>

        <VideoSource>video34</VideoSource>

        <VideoSource>video36</VideoSource>

    </CameraStream>

    <CameraStream>

        <Row>2</Row>

        <Column>2</Column>

        <DomainType>IVI_DOMAIN</DomainType>

        <VideoSource>video31</VideoSource>

        <VideoSource>video33</VideoSource>

```

```
<VideoSource>video35</VideoSource>
<VideoSource>video37</VideoSource>
</CameraStream>
```

```
</CameraDevice>
```

以上示例有 Row 和 Column 两个属性，支持多路 camera 的一字格、田字格拼接模式。示例中第一个 CameraStream 拼接模式是一行四列的横一字；第一个 CameraStream 拼接模式是两行两列的田字格。与 14.3.1.1.2.14 和 14.3.1.1.2.15 配合使用。

- 如果 DeviceType 不是 EXTERNAL_DEVICE，VideoSource 的值不能填 null，需填具体的 video 节点。

14.4.5 Camera 图像质量调节

AC8025 SoC 端对 MIPI 的视频输入图像质量具有一定的硬件调节能力，主要包括：2D NR，GAMMA，Sharpen，ColorProcess(Brightness，Contrast，Saturation)。

其中，Brightness，Contrast，Saturation 的调节，V4l2 标准接口、CameraSource、Android Camera 框架各层级都有暴露接口供开发调节使用（详见接口描述等章节介绍），其他图像质量的调节可借助提供的 Camera 配套调节工具进行 Tuning（详见 Tool 的介绍，此工具后续提供）。

14.4.6 CameraSource 使用

14.4.6.1 接口调用顺序

ICameraSource、ICameraStream 基本流程接口调用顺序如下表所示。

表 14-16 接口调用顺序表

	open	config	start	enqueue	dequeue	stop	close
open 后	√	√	×	×	×	×	√
config 后	×	√	√	√	×	×	√
start 后	×	×	√	√	√	√	√
stop 后	×	√	√	×	×	√	√
close 后	√	×	×	×	×	×	√

以上表格中所列举的，所有不合理的调用顺序都会引发接口返回 fail。

以 config 接口为例：config 接口可以在调用 open 或 config 或 stop 接口后调用；不可以在 start 或 close 后再调用 config 接口，否则，config 接口会返回 fail。

14.4.6.2 Buffer 申请

CameraSource 所使用的 buffer 分配有两种情况：CameraSource 调用者分配或 CameraSource 模块自己分配。具体说明详见 14.3.1.1.2.2 章节。

注意事项：

- 当选择由 CameraSource 模块自己分配时，config 接口中 bufCount 不能设置为 0；否则，config 接口将返回 fail；
- 当选择由 CameraSource 调用者分配时，在 dequeue 前必须调用 enqueue 接口先将 buffer 给 CameraSource，否则 dequeue 接口将返回 fail。

14.4.7 Camera Debug 方法

Videoin 提供了配套的 Debug Tool，供分析问题使用，包括各层级图像数据 dump，log 等级调整等，具体见 DebugTool 文档的 Videoin 章节说明。

14.5 用户定制化

14.5.1 Sensor 移植和定制化

14.5.1.1 添加新 Sensor 代码

14.5.1.1.1 代码路径

以 N4 为例：

- 代码源文件存放路径为
source/kernel/atc_modules/videoin/ac8025/sensor/sensor_list_src/n4_yuv.c
- 头文件存放路径为 source/kernel/atc_modules/videoin/ac8025/sensor/sensor_list_inc/n4_yuv.h

14.5.1.1.2 i2c address、sensor id 等 sensor 相关信息配置

需要定义如下结构体，

```
static struct sensor_common_params n4_params = {  
    .sensor_dp = VIN_MIPI_MAX,  
    .is_need_wait_hv_lock = false,  
    .is_support_independent_ctrl = true,  
}
```

```
.i2c_info = {
    .i2c_bus      = 0,
    .i2c_address  = 0x30,
    .sensor_id    = 0xb000,
    .i2c_mode     = SENSOR_I2C_MODE_0,
    .i2c_adapternum = 0, /* config by dts,not used */
    .i2c_speed    = 40000, /* config by dts,not used */
    .i2c_client   = NULL,
}
};
```

其中 i2c_info 的结构体内容需要按照真实的硬件信息填充,否则会有异常。其中特别注意以下三个成员的值,如下

```
.i2c_address = 0x30,
.sensor_id   = 0xb000,
.i2c_mode    = SENSOR_I2C_MODE_0,
```

i2c_address 代表设备的地址, sensor_id 代表了此 IC 的 id 号,用于识别设备, i2c_mode 用于表示不同 i2c 地址和数据 bit 位,目前平台中支持的 i2c 读写可以选择如下:

- SENSOR_I2C_MODE_0 : 代表 8bit addr, 8bit data
- SENSOR_I2C_MODE_1 : 代表 8bit addr, 16bit data
- SENSOR_I2C_MODE_2 : 代表 16bit addr, 8bit data
- SENSOR_I2C_MODE_3 : 代表 16bit addr, 16bit data

14.5.1.1.3 上下电配置

上电配置时序表,如下所示, pin name 是在 dtsi 中的配置, delay 单位为毫秒,该数组是在 open 操作时默认会被调用执行上电操作时,上电时序就按照数组所示,可以理解为:

- reset pin 拉低,然后执行 10 ms 的延时
- reset pin 拉高,然后执行 10 ms 的延时
- 供电引用拉高,然后执行 10ms 延时

```
static const struct sensor_power_setting n4_power_on_setting[] = {
    {
        .pin_name = "mipi1_reset_0",
```

```

        .delay    = 10,
    },
    {
        .pin_name = "mipi1_reset_1",
        .delay    = 10,
    },
    {
        .pin_name = "mipi1_pdn_1",
        .delay    = 10,
    },
};

```

下电配置时序表如下，解释同上。

```

static const struct sensor_power_setting n4_power_down_setting[] = {
    {
        .pin_name = "mipi1_reset_0",
        .delay    = 10,
    },
    {
        .pin_name = "mipi1_pdn_0",
        .delay    = 10,
    },
};

```

需要特别注意但是，power_on 和 power done setting 需要手动添加在 n4_yuv_s_ctrl 的结构体中才能正常工作，具体如下所示：

```

/*need customer to fill related parameters*/
struct sensor_vendor_ctrl n4_yuv_s_ctrl = {
    .power_on_settings = {
        .setting = n4_power_on_setting,
        .pin_num = ARRAY_SIZE(n4_power_on_setting),
    },
    .power_down_settings = {

```



```

        .setting = n4_power_down_setting,
        .pin_num = ARRAY_SIZE(n4_power_down_setting),
    },
};

```

14.5.1.1.4 csi rx 配置

```

static struct sensor_common_info n4_common_info = {
    .mipi_mode = VIN_MIPI_MODE_4D1C,
    .mr_x_pix_clk = VI_MR_X_PIXEL_CLK_267M,
    .mr_x_sys_clk = VI_MR_X_SYS_CLK_320M,
    .os_type = VIN_OS_IVI,
    .csi_info = {
        .lane_num = 4,
        .dl0_index = DPHY_LAN4,
        .dl1_index = DPHY_LAN3,
        .dl2_index = DPHY_LAN2,
        .dl3_index = DPHY_LAN0,
        .cl0_index = DPHY_LAN1,
        .cl1_index = DPHY_LAN5,
        .dl0_dpdn_swap = false,
        .dl1_dpdn_swap = false,
        .dl2_dpdn_swap = false,
        .dl3_dpdn_swap = false,
        .cl0_dpdn_swap = false,
        .cl1_dpdn_swap = false,
        .ch_info[VIN_CHANNEL_0] = {
            .fifo_mode = FULL_LINE_BUF,
            .fifo_level = 0,
            .ls_le_mode = 0
            .fmt = VIN_DATA_FMT_UYVY,
        },
    },
};

```

```

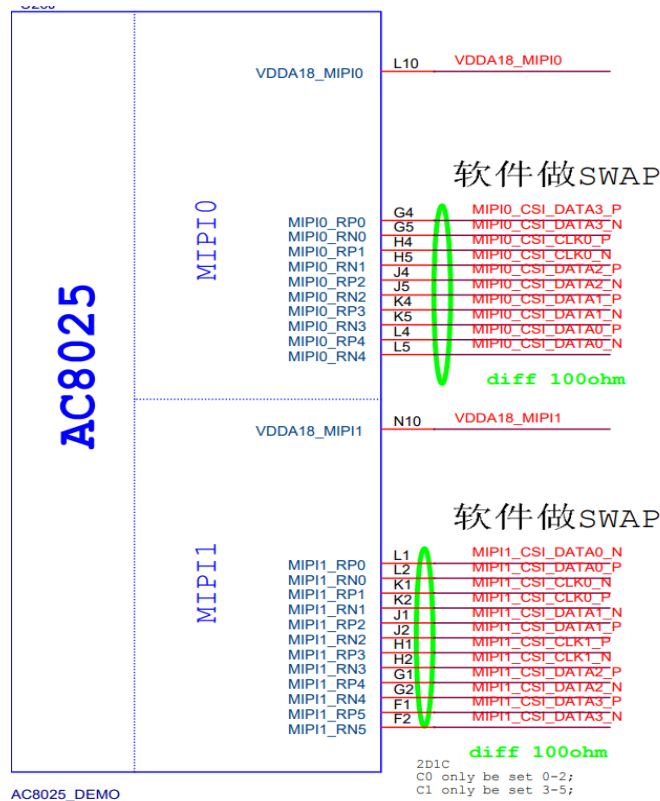
        .ch_info[VIN_CHANNEL_1] = {
            .fifo_mode = FULL_LINE_BUF,
            .fifo_level = 0,
            .ls_le_mode = 0
            .fmt = VIN_DATA_FMT_UYVY,
        },
        .ch_info[VIN_CHANNEL_2] = {
            .fifo_mode = FULL_LINE_BUF,
            .fifo_level = 0,
            .ls_le_mode = 0
            .fmt = VIN_DATA_FMT_UYVY,
        },
        .ch_info[VIN_CHANNEL_3] = {
            .fifo_mode = FULL_LINE_BUF,
            .fifo_level = 0,
            .ls_le_mode = 0
            .fmt = VIN_DATA_FMT_UYVY,
        },
    },
};

```

sensor_common_info 包含如下配置：

- mipi_mode 选择为 4D1C 或者 2D1C，4D1C 和 2D1C 的配和 8025 的 MIPI PHY 特性有关，MIPI0 是不可以拆分的，所以是固定的 4D1C，而 MIPI1 是可以拆分的，当不拆分的时候，其和 MIPI0 完全一致，当拆分的时候，MIPI1 和 MIPI2 均是 2D1C，当前 8025 公版的默认设计 MIPI1 是拆分模式。
- mrx_pix_clk 和 mrx_sys_clk，代表选择的平台端 mipi csirx 的 pixel clk 和 system clk 档位，默认使用最高档位，即 pixel_clk 使用 267M 的档位，system_clk 使用 320M 的档位。
- os_type 代表 sensor 工作的 os，如 IVI 或者 Safety 等。
- csi_info 的代表了当前 data lane 和 clock lane 的连接，要依据硬件的连接关系来进行设置。如 lane_num 表示 sensor 实际配置输出的 lane 数量。Index 和 swap 的属性是因为 DPHY 支持 Lane swap 和 DPDN swap，需要根据 MIPI DPHY 的连接关系来配置 Lane swap 和 DPDN swap。配置方法如下：

以 8025 公版的硬件设计为例:



Lane swap 配置:

只需要关注 Sensor 实际的 lane 的 index 对应的 AC8025 平台上的 PAD ID，平台的 PAD ID 从上到下的 index 为 0/1/2/3/4/5，根据 sensor 和 8025 的实际连接方式，按上述的原理图对应的 index 配置如下:

MIPI0 上: 对应的 data lane 和 clock lane 的 index 分别为 4/3/2/0/1/5

MIPI1 上: 对应的 data lane 和 clock lane 的 index 分别为 0/2/4/5/1/3

DPDN swap 配置:

按照上述原理图，可以看出 MIPI0 的 DPDN 均没有 swap，因此全部配置为 false，而 MIPI1 上部分做了 swap，部分没有做 swap，需要注意的是，DPDN 的 swap 的 Index 是 Lane swap 后的 index,如 0/1/2 需要做 DPDN swap，但是经过 Lane swap 后需要变成了 0/2/4，对应 0/2/4 的 Lane Swap 配置为 true，其他的配置为 false。

ch_info 的配置每个 channel 互相独立，主要有以下几个参数:

fifo_mode: 配置 csirx fifo 的属性，默认是 FULL_LINE_BUF，一般不需要更改。只有当输入源为 RGB888，并且一行的 Pixel 达到 1920 时，才需要配置为 LARGE_BUF，或者前端同一个 channel 相邻两行数据的 hblanking 很短不符合标准要求的 hblanking 时间可能会导致出现 stream fifo overflow，此时也可以使用 large buffer mode 或者提高 data rate 以增大前端 hblanking 时间。

`fifo_fill_level`: 当配置为 `FULL_LINE_BUF` 时该值为 0，当配置为 `LARGE_BUF` 模式时，该值和输入源的大小有关系，常见可以配置为 200、500 等。

`fs_le_mode`: 默认为 0，一般不需要更改。

`fmt`: 输入源的格式，根据实际输入源可以配置为 `YUV422/RGB565/RGB888` 等。

14.5.1.1.5 不同 channel camera 的信息配置

```
static const struct sensor_channel_info n4_channel_default_info = {
    .enable          = true,
    .detect_resolution = true,
    .resolution       = VIN_INPUT_HD_30,
    .position         = 0,
    .mount_angle      = 0,
    .fps              = 30,
    .drop_frames      = 0,
    .input_fmt        = VIN_DATA_FMT_YUYV,
    .width            = 1280,
    .height           = 720,
    .h_offs           = 0,
    .v_offs           = 0,
    .vi_h_total       = 2560,
    .vi_v_total       = 750,
    .is_interlace     = false,
    .csi_info = {
        .fifo_mode      = FULL_LINE_BUF,
        .fifo_fill_level = 0,
        .ls_le_mode     = 0,
        .vc_sel         = 0xf,
    },
};
```

上述代表了一个摄像头输入时的参数，其中我们需要特别关心的参数如下：

- `drop_frames`：代表了驱动接收到数据后需要掉丢的帧的个数

- `input_fmt`: 代表了当前通道的输入格式, 支持 YUYV、UYVY、RGB888、RGB565
- `h_offs`: 代表行抓图的起始地址点
- `v_offs`: 代表从第几行开始抓取
- `vi_h_total`: 代表了抓图的行整体大小
- `vi_v_total`: 代表抓了多少行

对于其他参数, 除了 `csi_info` 的参数, 其他的会在不同的地方被赋值, `csi_info` 的相关参数使用默认值就行。

14.5.1.1.6 set channel 更改配置

对于帧率、是否为交错模式以及 pixel clock 会在 `n4_set_channel_params` 函数中重新赋值, 为了支持同一个 channel 支持不同的 `fmt`, 因此需要在使用的時候根据传入的 `fmt` 进行将参数替换为当前实际 `fmt` 的参数, 以 `n4_set_channel_params` 为例, 支持 `VIN_INPUT_HD_25` 和 `VIN_INPUT_FHD_25` 两个不同的 `fmt`, 然后根据传入参数替换, 需要特别注意的是 `N4_IMAGE_SENSOR_HD25_WIDTH`、`N4_IMAGE_SENSOR_HD25_HEIGHT`、`N4_IMAGE_SENSOR_HD25_PIX_CLK` 需要在 `n4_yuv.h` 中定义。

```
static int n4_set_channel_params(enum videoin_channel_id ch, enum
videoin_input_resolution fmt)
{
    struct sensor_channel_info *ch_info = &n4_channel_info[ch];

    switch (fmt) {
    case VIN_INPUT_HD_25:
        ch_info->fps = 25;
        ch_info->is_interlace = false;
        ch_info->resolution = VIN_INPUT_HD_25;
        ch_info->width = N4_IMAGE_SENSOR_HD25_WIDTH;
        ch_info->height = N4_IMAGE_SENSOR_HD25_HEIGHT;
        ch_info->pix_clk = N4_IMAGE_SENSOR_HD25_PIX_CLK;
        break;
    case VIN_INPUT_FHD_25:
        ch_info->fps = 25;
        ch_info->is_interlace = false;
        ch_info->resolution = VIN_INPUT_FHD_25;
```

```

        ch_info->width = N4_IMAGE_SENSOR_FHD25_WIDTH;
        ch_info->height = N4_IMAGE_SENSOR_FHD25_HEIGHT;
        ch_info->pix_clk = N4_IMAGE_SENSOR_FHD25_PIX_CLK;
        break;

    default:
        vin_log(LOG_E, "failed, not support this fmt(%d)", fmt);
        return -1;
    }

    ch_info->enable = true;
    return 0;
}

```

14.5.1.1.7 init setting 配置通道寄存器

```

static int n4_init_setting(enum videoin_channel_id ch, enum
videoin_input_resolution fmt)

```

在这个函数在，根据传入的 Channel 和 fmt 信息选择不同的 setting 进行配置，因为有些 IC 是不能感觉 channel 配置的，这个时候对于该接进行统一配置即可。

14.5.1.1.8 clock enable 、disable 寄存器配置

```

static int n4_clock_enable(void)
static int n4_clock_disable(void)

```

需要实现以上 clock 功能，控制 mipi 的输出和关闭，对于连续的 mipi clock mode,我们需要有 SOC 和前端 IC 的开关有时序要求，因此该功能需要时序，一般的 IC 都支持该功能。

该接会默认被调用，在 start 、stop 接口中会分别调用这两个接口。

14.5.1.1.9 hv clock 前端 sensor 输出是否稳定检测

```

static int n4_init_setting(enum videoin_channel_id ch, enum
videoin_input_resolution fmt)

```

14.5.1.1.10 信号检测功能

需要实现如下接口进行对当前通道的 sensor 输出是否稳定进行检测，如果不稳定，则输出的数据内容可能出现异常，并且 blanking 不稳定可能导致出现了 SOC 上报异常，导致接收数据异常。

```
static int n4_hvlock_detect(bool *ch, bool *status)
```

14.5.1.1.11 制式检测功能

有些 IC 是支持对当前插入的 channel 的摄像头检测制式的，具体的可以查看 spec 是否支持，如果支持并且希望使用 IC 的功能，则需要实现如下接口，根据传入 channel 的不同来实现制式检测

```
static enum videoin_input_resolution n4_format_detect(enum videoin_channel_id ch)
```

14.5.1.1.12 sensor_function_table 结构体数组

```
static struct sensor_function_table n4_func_table = {
    .sensor_init = n4_sensor_init,
    .sensor_uninit = n4_sensor_uninit,
    .sensor_start = n4_sensor_start,
    .sensor_stop = n4_sensor_stop,
    .sensor_open = n4_sensor_open,
    .sensor_close = n4_sensor_close,
    .sensor_pre_open = n4_sensor_pre_open,
    .format_detect = n4_format_detect,
    .init_setting = n4_init_setting,
    .hvlock_detect = n4_hvlock_detect,
    .signal_check = n4_signal_plug_detect,
    .sensor_get_id = n4_get_id,
    .mipi_clock_enable = n4_mclk_enable,
    .mipi_clock_disable = n4_mclk_disable,
    .sensor_dump_all_page = n4_dump_all_page,
    .sensor_set_ch_params = n4_set_channel_params,
};

/*need customer to fill related parameters*/
```

```
struct sensor_vendor_ctrl n4_yuv_s_ctrl = {  
    .param = &n4_params,  
    .func = &n4_func_table,  
    .power_on_settings = {  
        .setting = n4_power_on_setting,  
        .pin_num = ARRAY_SIZE(n4_power_on_setting),  
    },  
    .power_down_settings = {  
        .setting = n4_power_down_setting,  
        .pin_num = ARRAY_SIZE(n4_power_down_setting),  
    },  
    .common_info = &n4_common_info,  
    .channel_info = n4_channel_info,  
};
```

需要将上述的一些接口添加到对应 结构体成员变量中。

14.5.1.2 修改 Sensor DTS 配置

代码路径: source/kernel/common/arch/arm64/boot/dts/autochips/ac8x_demo.dts

整体的 dtsi 如下所示, 在 dtsi 中需要修改三个部分, i2c、i2c address、pin number 的修改。

```
&i2c0 {  
    sensor1@44 {  
        compatible = "sensor,mipi1";  
        reg = <0x44>;  
        dp_idx = <1>;  
        pinctrl-name = "mipi1_reset_0","mipi1_reset_1", mipi1_pdn_0,mipi1_pdn_1;  
        pinctrl-0 = <&mipi1_reset0>;  
        pinctrl-1 = <&mipi1_reset1>;  
        pinctrl-2 = <&mipi1_pdn0>;  
        pinctrl-3 = <&mipi1_pdn1>;  
    };  
};
```



```
&pio {
    mipi1_reset0 = mipi1reset0 {
        mipi1_reset {
            pinmux = <PINMUX_GPIO78_FUNC_GPIO00_17>;
            output-low;
        };
    };
    mipi1_reset1 = mipi1reset1 {
        mipi1_reset {
            pinmux = <PINMUX_GPIO78_FUNC_GPIO00_17>;
            output-high;
        };
    };
    mipi1_pdn0 = mipi1_pdn 0 {
        mipi1_pdn {
            pinmux = <PINMUX_GPIO69_FUNC_GPIO00_8>;
            output-low;
        };
    };
};
```

14.5.1.2.1 i2c 选择

```
&i2c0 {
```

MIPI1 的 camera 是挂载到 I2C0 上，i2c 的选择需要和硬件匹配，否则会出现 i2c 不通。

14.5.1.2.2 i2c address

```
sensor1@44 {
    compatible = "sensor,mipi1";
    reg = <0x44>;
```

由于支持动态适配多个不同的 sensor。此处可以默认的 reg 的 value 不修改，但是一定要确保驱动中的 i2c address 是正确的，否则会出现 i2c 无法读写。

14.5.1.2.3 pin number 的选择

AC8025 默认提供了控制上下电和 复位的引用控制，使用了内核中 `pinmux` 的方式。已经提供了默认 `pin number`，需要根据实际的硬件连接来修改 `pin number`，以下面的举例：

```
mipi1_reset0 = mipi1reset0 {
    mipi1_reset {
        pinmux = <PINMUX_GPIO78_FUNC_GPIO00_17>;
        output-low;
    };
};
```

`pinmux = <PINMUX_GPIO78_FUNC_GPIO00_17>;`reset 的 `pin numbe` 为 `PINMUX_GPIO78_FUNC_GPIO00_17`，`output-low`;代表输出为低。

在使用时需要主要根据实际的硬件连接来确认选择的 `pin number`。

14.5.1.3 Sensor 的兼容性配置

为了使同一版本镜像可以做到兼容多款 `Sensor`，又不会由于兼容的 `Sensor` 列表过多导致代码编译时冗余和运行时遍历耗时，所以在项目顶层的编译控制中添加了 `Sensor` 列表宏，并在 `Sensor` 驱动中根据顶层的宏生成对应的 `Sensor` 支持列表数组（在添加 `Sensor` 前，必须明确该 `sensor` 硬件接在哪个 `MIPI CSI` 上）。

以 `N4` 接在 `MIPI1` 上，工作在 `ivi` 产品形态下进行配置为例：

1. 在 `build/project/ivi_s_ac8025/xxx/videoin_config.json` 的 `ATC_MIPI1_SENSOR_SUPPORT` 宏中增加 `n4` 的名字，下面的含义代表 `MIPI1` 支持兼容 `n4` 和 `max9286` 两款转换芯片

```
“ATC_MIPI1_SENSOR_SUPPORT”: “n4,max9286”,
```

该宏会经过解析后，会自动生成 `ATC_MIPI1_SENSOR_N4_SUPPORT`，`ATC_MIPI1_SENSOR_MAX9286_SUPPORT` 两个宏。

2. 修改 `sensor_list.h`，使用上面生成的宏控制添加新增的 `Sensor` 到对应 `MIPI` 列表中
`source/kernel/atc_modules/videoin/ac8025/sensor/sensor_list.h`

```
#ifdef ATC_MIPI1_SENSOR_N4_SUPPORT
#include “n4_yuv.h”
#endif
...
struct sensor_name_ctrl sensor_list_mipi1[] = {
#ifdef ATC_MIPI1_SENSOR_N4_SUPPORT
```

```
{
    .sensor_name = "n4_yuv",
    .sensor_ctrl = &n4_yuv_s_ctrl,
},
#endif
}
```

为了支持一条 MIPI 兼容不同的转换芯片，增加了 `sensor_list_mipi1[]` 这个结构体数组保存了这条 MIPI 可能会存在的 Sensor，只有增加在此数组中，才能保证在启动的时候，`sensor` 会被遍历到。

3. 如客户没有兼容性需求，一版软件固定使用一款 Sensor，则可以直接修改“`sensor_list.h`”，在对应 MIPI 列表数组中添加 Sensor，将其他 Sensor 移除，不需要添加宏控制。

14.5.1.4 Sensor 添加客制化制式

可按如下步骤添加 Sensor 客制化制式，以 N4 需要添加一个 1920x1080@30 的制式为例：

1. 在 `videoin_common.h` 中扩展 `enum videoin_input_resolution`。

`source/kernel/atc_modules/videoin/ac8025/common/videoin_common.h`

```
/**
 * @brief only used for sensor to detect, report and config input camera resolution.
 * ----- it need map to real width and height for camera stream using.
 */
enum videoin_input_resolution {
    >> VIN_INPUT_ERROR = 0,
    >> VIN_INPUT_NTSC,
    >> VIN_INPUT_PAL,
    >> VIN_INPUT_HD_25,
    >> VIN_INPUT_HD_30,
    >> VIN_INPUT_FHD_25,
    >> VIN_INPUT_FHD_30,
    >> VIN_INPUT_MAX
};
```

2. 在 N4 的实现代码中定义该制式翻译出的宽高分辨率信息

`source/kernel/atc_modules/videoin/ac8025/sensor/sensor_list_src/n4_yuv.c`

```
#define N4_IMAGE_SENSOR_HD30_WIDTH----- (1280)
#define N4_IMAGE_SENSOR_HD30_HEIGHT----- (720)
#define N4_IMAGE_SENSOR_HD30_PIX_CLK----- (37125000)

#define N4_IMAGE_SENSOR_FHD25_WIDTH----- (1920)
#define N4_IMAGE_SENSOR_FHD25_HEIGHT----- (1080)
#define N4_IMAGE_SENSOR_FHD25_PIX_CLK----- (74250000)

#define N4_IMAGE_SENSOR_FHD30_WIDTH----- (1920)
#define N4_IMAGE_SENSOR_FHD30_HEIGHT----- (1080)
#define N4_IMAGE_SENSOR_FHD30_PIX_CLK----- (74250000)
```

3. 在 N4 的实现代码的 xxx_set_channel_params 接口中新增分辨率枚举的信息赋值

```
static int n4_set_channel_params(enum videoin_channel_id ch, enum videoin_input_resolution fmt)
{
    struct sensor_channel_info *ch_info = &n4_channel_info[ch];

    switch (fmt) {
        case VIN_INPUT_HD_25: ...
        case VIN_INPUT_HD_30: ...
        case VIN_INPUT_FHD_25: ...
        case VIN_INPUT_FHD_30:
            ch_info->fps = 30;
            ch_info->is_interlace = false;
            ch_info->resolution = VIN_INPUT_FHD_30;
            ch_info->width = N4_IMAGE_SENSOR_FHD30_WIDTH;
            ch_info->height = N4_IMAGE_SENSOR_FHD30_HEIGHT;
            ch_info->pix_clk = N4_IMAGE_SENSOR_FHD30_PIX_CLK;
            break;
        case VIN_INPUT_NTSC: ...
        case VIN_INPUT_PAL: ...
        default:
            vin_log(LOG_E, "failed, not support this fmt(%d)", fmt);
            return -1;
    } // end switch fmt

    ch_info->enable = true;

    return 0;
} // end n4_set_channel_params
```

- 在 N4 的实现代码的 xxx_channel_setting 接口中添加新增分辨率的 sensor 配置

```
void n4_channel_setting(enum videoin_channel_id ch, enum videoin_input_resolution fmt)
{
    n4_i2c_write(0xFF, 0x00);
    n4_i2c_write(0x00+ch, 0x10);
    if ((VIN_INPUT_HD_25 == fmt) || (VIN_INPUT_HD_30 == fmt) || (VIN_INPUT_FHD_25 == fmt) || (VIN_INPUT_FHD_30 == fmt)) {
        n4_i2c_write(0x04+ch, 0x00);
    } else if (VIN_INPUT_PAL == fmt) {
        n4_i2c_write(0x04+ch, 0x05);
    } else if (VIN_INPUT_NTSC == fmt) {
        n4_i2c_write(0x04+ch, 0x04);
    }
    if (VIN_INPUT_HD_25 == fmt) {
        n4_i2c_write(0x08+ch, 0x00);
    } else if (VIN_INPUT_HD_30 == fmt) {
        n4_i2c_write(0x08+ch, 0x0C);
    } else if (VIN_INPUT_FHD_25 == fmt) {
        n4_i2c_write(0x08+ch, 0x03);
    } else if (VIN_INPUT_FHD_30 == fmt) {
        n4_i2c_write(0x08+ch, 0x02);
    } else {
        n4_i2c_write(0x08+ch, 0x00);
    }
    n4_i2c_write(0x0c+ch, 0x00);
    if ((VIN_INPUT_HD_25 == fmt) || (VIN_INPUT_HD_30 == fmt)) {
        n4_i2c_write(0x10+ch, 0x20);
    } else if (VIN_INPUT_PAL == fmt) {
        n4_i2c_write(0x10+ch, 0xdd);
    } else if (VIN_INPUT_NTSC == fmt) {
        n4_i2c_write(0x10+ch, 0xa0);
    }
}
```

15 R5 RVC

15.1 简介

15.1.1 软硬件环境

15.1.1.1 软件环境

R5 RVC 分为 R5 RVC 和 RVC Setting App 两部分：

- R5 RVC 基于 FreeRTOS 系统开发，运行在 HSM 系统上，提供倒车信号检测、Video/UI 显示、Audio 播放等倒车核心功能；
- RVC Setting App 基于 Android 系统开发，提供人机交互、Android App 交互等倒车辅助功能；
- R5 RVC 和 RVC Setting App 之间使用一套私有框架接口进行核间通信。

15.1.1.2 硬件环境

- AC8025 平台 RVC 方案：AC8025 + CVBS/AHD 转 MIPI CSI Bridge IC + 倒车开关 + CVBS 或 AHD 摄像头。
- AC8025 响应倒车开关事件，控制是否采集来自 Bridge IC 的摄像头数据与视频显示。

15.1.2 功能

在 FreeRTOS 上实现的倒车影像功能如下：

- 在开机阶段快速响应倒车信号。
- 实时显示后视摄像头视频画面。
- 显示倒车辅助信息，如倒车雷达及倒车轨迹线等。
- 当倒车雷达检测到障碍物时进行警报音的播放。
- 实时调节倒车图像的 PQ “效果”，如：亮度、饱和度、对比度。

15.1.3 代码结构及文件描述

表 15-1 RVC 代码结构及描述

文件名	文件路径	文件描述
*.aidl *.cpp *.h	source/android/vendor/autochips/proprietary/hardware/interfaces/rvc source/android/vendor/autochips/proprietary/hardware/rvc	RVC Service 接口及代码， 提供给 RVC Setting App 调用
*.c *.h	source/vendor/autochips/proprietary/tinysys/hsm/os/apps/rvc	R5 RVC MiddleWare 及 App

15.2 软件设计

15.2.1 架构图

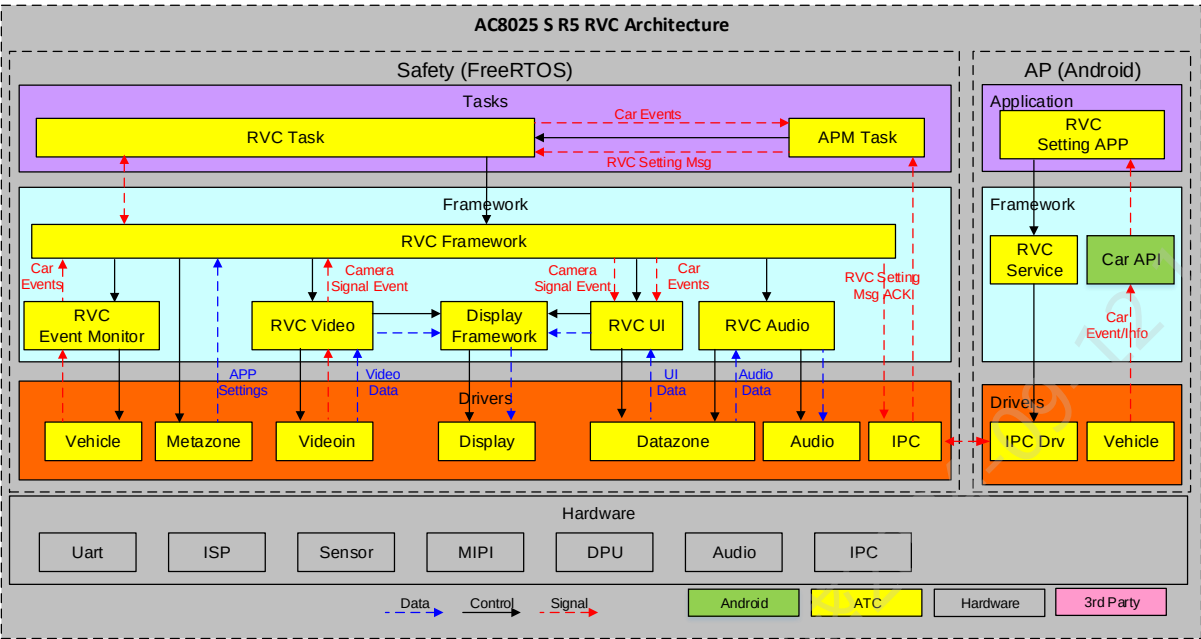


图 15-1 R5 RVC 架构图

15.2.2 流程图

15.2.2.1 R5 RVC 和 RVC Setting App 交互

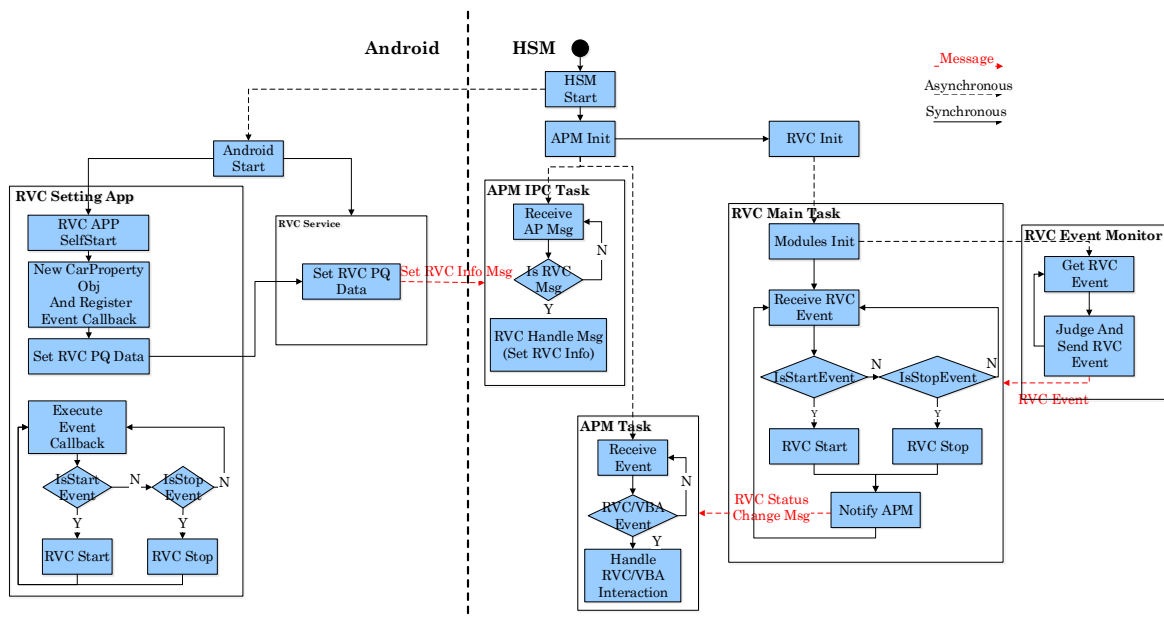


图 15-2 R5 RVC 和 RVC Setting App 交互流程

R5 RVC 启动后就不再退出，在系统运行过程中一直监听和处理倒车事件。同时，APM 模块会一直监听所有从 IVI 域发送到 HSM 的 IPC 消息，如果有获取或设置倒车信息的信息，则 R5 RVC 会进行相应的处理和反馈。

RVC Setting App 通过注册到 CarProperty 的回调函数监听并响应倒车状态变化消息，并通过 RVC Service 提供的接口获取或设置 R5 RVC 的相关配置信息，例如通过 Android 的人机交互界面配置 R5 RVC 视频的亮度、对比度等。

R5 RVC 通过 RVC Event Monitor 模块获取倒车状态变化消息。当倒车状态变化时，RVC 会通知 APM 模块，用来处理倒车画面与开机动画的交互。

15.2.2.2 Videoin 预初始化

由于 Sensor 桥接芯片在初始化后往往需要一定的时间之后才能信号稳定并正常出图。因此在 HSM 系统尚未进行 RVC 的初始化时，需要先对 Videoin 模块进行预初始化，从而加速 RVC 出图的时间。

此外，由于 RVC 需要的 UI、Audio 等资源需要从 eMMC 搬移到 DRAM 中，此阶段较为耗时，所以也在预初始化阶段进行操作。

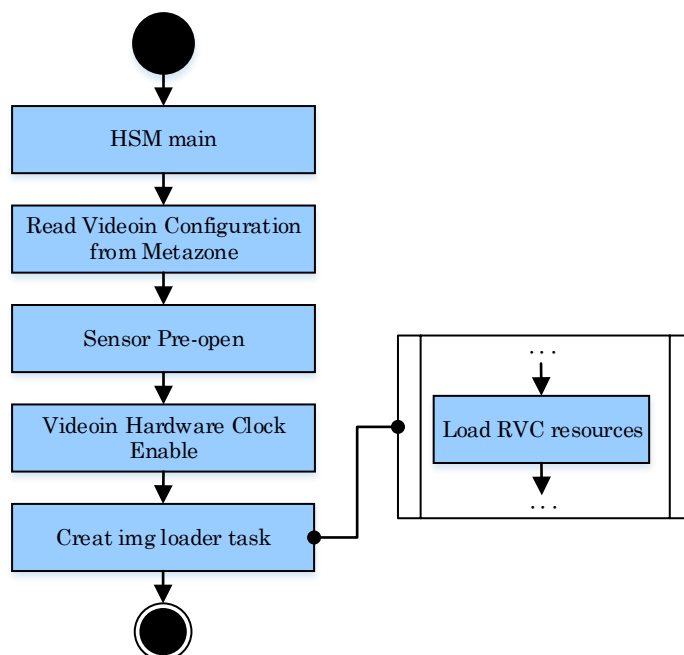


图 15-3 Videoin 预初始化流程

15.2.2.3 R5 RVC 初始化

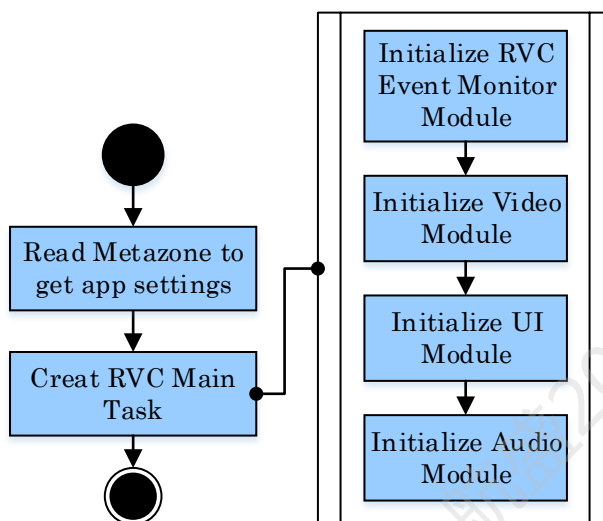


图 15-4 R5 RVC 初始化流程

R5 RVC 初始化函数在 APM 初始化时调用。初始化内容包括：从 Metazone 中读取倒车配置信息，并创建 RVC Main Task。

在 RVC Main Task 中，将继续进行各子模块初始化工作，包括倒车事件监测模块初始化、倒车 Video 模块初始化、倒车 UI 模块初始化、倒车 Audio 模块初始化。初始化完成后，RVC Main Task 会进入等待和处理倒车消息的循环。

15.2.2.4 R5 RVC 开启/关闭

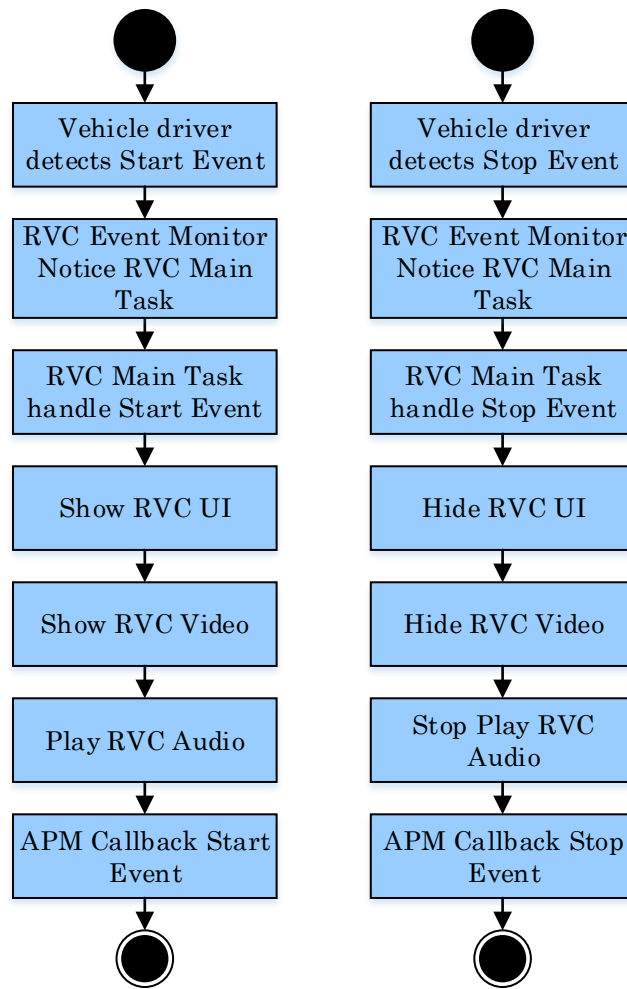


图 15-5 R5 RVC 开启流程及关闭流程

RVC Event Monitor 在检测到倒车事件发生时，会将消息发送到 RVC Main Task，由该 Task 实际处理倒车事件。如果消息为倒车开始，则显示倒车 UI、显示倒车 Video、播放倒车 Audio、通知 APM 倒车开始状态。如果消息为倒车结束，则隐藏倒车 UI、隐藏倒车 Video、停止播放倒车 Audio、通知 APM 倒车结束状态。

15.2.2.5 R5 RVC 拔插信号通知

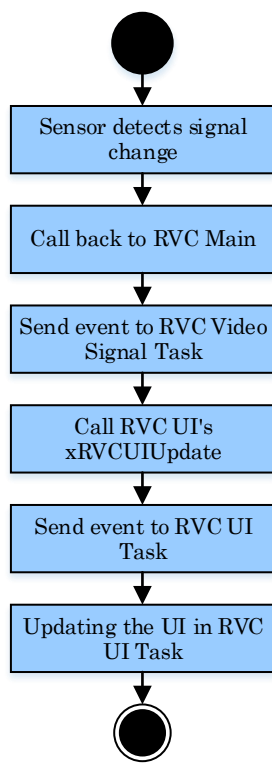


图 15-6 R5 RVC 拔插信号通知流程流程

R5 RVC Signal Notify 是实现倒车 Video 插拔事件的通知。底层 Sensor 会有 Task 一直监测摄像头的插拔状态，在获取到状态变化后会通知到上层 Video 模块，再告知 UI 模块更换显示的 UI 图片。

15.2.2.6 RVC Setting App 设置及获取参数流程

为了解决 RVC Setting App 在实现一些人机交互等辅助功能时，需要设置及获取 R5 RVC 的可配信息，RVC Service 提供了一系列 set/get 接口。这些接口的调用流程如下图所示。

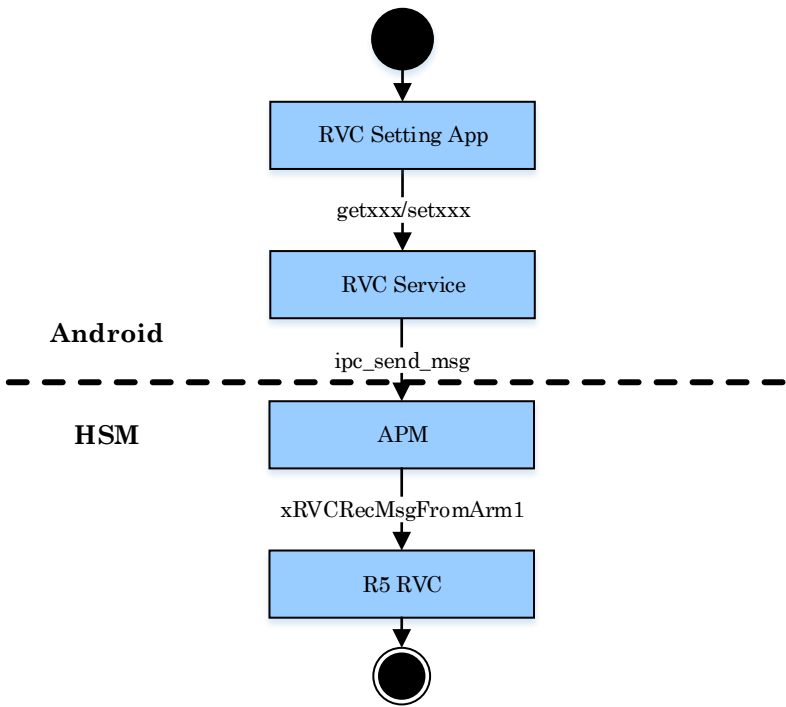


图 15-7 RVC Setting App Set/Get 流程

15.3 接口描述

15.3.1 API

15.3.1.1 RVC Framework API

15.3.1.1.1 概述

RVC Framework 对外提供的接口，主要供 APM 完成 R5 RVC 下模块状态的管控，定义在 source/vendor/autochips/proprietary/tinysys/hsm/os/include/rvc.h。

以下是各 API 接口的功能说明和简要的使用说明，例程请参考：
source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/apm/apm.c。

表 15-2 RVC Framework API

API	接口说明
void(*RVC_EVENT_CALLBACK)(RVCEventType_t)	回调函数，用来通知APM倒车的状态，做相应的处理
xRVCInit()	初始化相关倒车资源，注册APM 回调函数，创建相关线程
xRVCDeinit()	去初始化，清理倒车相关资源

xRVCGetsolutionType()	获取倒车类型
xRVCGetUIResourceType()	获取倒车UI类型
xRVCRespondToDispPowerOnOff()	当屏幕上电或掉电时，调用此接口，处理屏幕上电或掉电的消息
xRVCSendEvent()	控制开启倒车、停止倒车及发送其他相关倒车事件
xRVCRecMsgFromArm1	当APM接收到RVC Service的消息后，R5 RVC需要解析消息，并进行相应的处理

15.3.1.1.2 xRVCInit

【描述】

Initialize R5 RVC.

【语法】

```
BaseType_t xRVCInit( RVC_EVENT_CALLBACK pvCallback );
```

【参数】

pvCallback callback for APM to notify RVC status

【返回值】

- pdPASS means success
- pdFAIL means failed

15.3.1.1.3 xRVCDeinit

【描述】

Deinitialize R5 RVC

【语法】

```
BaseType_t xRVCDeinit();
```

【参数】

None

【返回值】

- pdPASS means success
- pdFAIL means failed

15.3.1.1.4 xRVCGetsolutionType

【描述】

Get RVC solution type.

【语法】

```
RVCsolution_t xRVCGetsolutionType();
```

【参数】

None

【返回值】

- eRVCSolutionArm2Only means R5 RVC
- eRVCSolutionAVM means AVM, in this case R5 RVC will not work
- eRVCSolutionNative means Native RVC, in this case R5 RVC will not work

15.3.1.1.5 xRVCGetUIResourceType

【描述】

Get RVC UI resource type.

【语法】

```
RVCUIResType_t xRVCGetUIResourceType();
```

【参数】

None

【返回值】

- eUIResTypePngOrBmp means png or bmp type UI resource
- eUIResTypeJpeg means jpeg type UI resource

15.3.1.1.6 xRVCRespondToDispPowerOnOff

【描述】

When display power on or off, call this function to make RVC handle this status.

【语法】

```
void xRVCRespondToDispPowerOnOff(BaseType_t status);
```

【参数】

status pdTURE means display power on, pdFalse means display power off

【返回值】

None

15.3.1.1.7 xRVCSendEvent

【描述】

This interface controls starting and stopping RVC and sends other relevant RVC events.

【语法】

BaseType_t xRVCSendEvent (RVCEventType_t xRVCEvent);

【参数】

xRVCEvent eRVCSstartEvent means start RVC, eRVCSstopEvent means stop RVC

【返回值】

- pdPASS means success
- pdFAIL means failed

15.3.1.1.8 xRVCSecMsgFromArm1

【描述】

After APM receives the message from RVC Service, R5 RVC needs to parse the message and process it accordingly.

【语法】

BaseType xRVCSecMsgFromArm1 (char *msg, int len);

【参数】

- msg: the message received from RVC Service.
- len: the length of the message.

【返回值】

- pdPASS means success
- pdFAIL means failed

15.3.1.2 R5 RVC Modules API

15.3.1.2.1 概述

RVC Modules 是各模块对内提供的功能接口，主要供 RVC Framework 和 RVC Main 调用，完成 RVC 各功能的实现，可参考框架图和流程图，定义在 source/vendor/autochips/proprietary/tinysys/hsm/os/apps/rvc/目录下。

以下是各 API 接口的功能说明和简要的使用说明，例程请参考 source/vendor/autochips/proprietary/tinysys/hsm/os/apps/rvc/rvc_main.c。

15.3.1.2.2 RVC Main

RVC Main 主要用于 RVC 总体工作流程的控制。

表 15-3 RVC Main 接口

接口名称	接口说明
xRVCReadAppSetting	从Metazone中读取倒车相关配置信息
xRVCReadResourceInfo	从Datazone中读取倒车资源文件信息
xRVCModulesInit	初始化倒车模块，包括Videoin、Audio、GPIO等
xRVCModulesDeinit	去初始化倒车模块
xRVCShow	处理倒车开始消息，显示倒车画面
xRVCHide	处理倒车停止消息，隐藏倒车画面
vRVCMainTask	线程函数，进行倒车模块初始化后，循环等待和处理倒车消息
xRVCUpdate	通知倒车车身变化信息，调用此接口来更新倒车UI
xRVC_suspend	调用此接口使倒车功能挂起，通常用于系统休眠前
xRVCResume	调用此接口使倒车功能恢复，通常用于系统唤醒后
xRVCResetVideo	重置倒车的Video硬件
xRVCSendMsgToArm1	实现向RVC Service发送消息的功能，当处理完RVC Service的消息后，调用此接口向RVC Service发送消息告知处理结果

15.3.1.2.3 RVC Video

RVC Video 主要用于 RVC 摄像头画面的获取及显示。

表 15-4 RVC Video 接口

接口名称	接口说明
xRVCVideoInit	初始化Video、Display资源，完成相关配置，并start video硬件
xRVCVideoDeinit	去初始化video、display资源
xRVCVideoShow	显示Video画面

xRVCVideoHide	隐藏Video画面
xRVCVideoWaitUntilFirstFrame	调用此函数会阻塞直到第一帧Video buffer送显完毕或者超时
xRVCVideoSwitchChannel	切换当前虚拟通道实际使用的Sensor通道
XRVCVideoSuspend	挂起Video模块，通常用于系统休眠前
xRVCVideoResume	恢复Video模块，通常用于系统唤醒后
xRVCVideoSetBrightness	设置Video画面亮度
xRVCVideoSetHue	设置Video画面色度，此接口在AC8025上无效
xRVCVideoSetSaturation	设置Video画面饱和度
xRVCVideoSetContrast	设置Video画面对比度
xRVCVideoSetFlip	设置Video画面翻转
XRVCVideoSwitchDisplayId	切换倒车Video显示屏
xVideoBufferDeqCallback	回调函数，供底层Videoin Driver线程从Display获取Free Buffer
xVideoBufferEnqCallback	回调函数，供底层Videoin Driver线程将Ready Buffer送往Display显示
vVideoSignalTask	线程函数，接收并处理Video插拔信号消息

15.3.1.2.4 RVC UI

RVC UI 主要用于处理车身信息并进行倒车 UI 显示。

表 15-5 RVC UI 接口

接口名称	接口说明
xRVCUIInit	初始化Display资源，创建UI线程
xRVCUIDeinit	去初始化UI资源，销毁线程
xRVCUIUpdate	更新显示UI，将更新信息发送至UI线程
xRVCUIShow	显示UI画面
xRVCUIHide	隐藏UI画面
xRVCUIWaitUntilFirstFrame	调用此函数会阻塞直到第一帧UI buffer送显完毕或者超时
vRVCUITask	线程函数，接收并处理UI更新消息，读取并送显需要更新的UI

15.3.1.2.5 RVC Audio

RVC Audio 用于播放倒车警示音。

表 15-6 RVC Audio 接口

接口名称	接口说明
xRVCAudioInit	初始化Audio资源
xRVCAudioDeinit	去初始化Audio资源

15.3.1.2.6 RVC Event Monitor

表 15-7 RVC Event Monitor 接口

15.3.2 数据结构

表 15-8 R5 RVC 数据结构

15.3.2.1 RVCEventType

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

365 / 1254

```
eRVC_suspendEvent,          /* R5 RVC 挂起 */
eRVC_resumeEvent            /* R5 RVC 恢复工作 */
eRVC_resetVideoEvent,       /* 重置倒车 Video 模块事件 */
eRVC_switchScreenEvent,     /* 切换倒车显示屏事件 */
} RVCEventType_t;
```

15.3.2.2 rvc_data_type

此数据结构仅用于 RVC Setting App 与 R5 RVC 的通信，如果不需要 RVC Setting App，则无需关注。

```
typedef enum rvc_data_type RVCDatatype_t;
enum rvc_data_type
{
    RVC_DATA_NONE = 0;
    RVC_DATA_RVC_STATUS,          /* 倒车当前状态信息 */
    RVC_DATA_CAMERA_VIDEO_PARAMS, /* 倒车视频的属性信息 */
    RVC_DATA_MAX,
};
```

15.3.2.3 rvc_data

此数据结构仅用于 RVC Setting App 与 R5 RVC 的通信，如果不需要 RVC Setting App，则无需关注。

```
typedef struct rvc_data RVCDatat_t;
struct rvc_data
{
    char ipc_key[12];
    enum rvc_data_type type;
    int32_t data[4];
};
```

- ipc_key: 用以识别发送的ipc信息内容，限制长度为12个char（受IPC通信传递的数据长度限制）。目前定义和使用如下四种key字符串：

1. #define RVC_IPC_KEY_GET_DATA “rvc_getdata”

表示RVC Setting App获取R5 RVC数据的IPC消息；

```
2. #define RVC_IPC_KEY_SET_DATA "rvc_setdata"
```

表示RVC Setting App设置R5 RVC数据的IPC消息；

```
3. #define RVC_IPC_KEY_ARM2_RESPOND "rvc_respond"
```

表示R5 RVC返回给RVC Setting App的响应消息；

```
4. #define RVC_IPC_KEY_ARM2_NOTIFY "rvc_notify"
```

表示R5 RVC主动通知RVC Service的消息。

- type: 传递的rvc data类型
- data: 传递的rvc data数据，限制长度为4个int长度（受IPC通信传递的数据长度限制）

15.3.2.4 RVCDataset

```
typedef struct RVCDataset
{
    int32_t lRVCStatus;          /* 倒车状态 */
    int32_t lRVCVideoHue;        /* 倒车视频画面色度值（未使用） */
    int32_t lRVCVideoBrightness; /* 倒车视频画面亮度值 */
    int32_t lRVCVideoContrast;   /* 倒车视频画面对比度值 */
    int32_t lRVCVideoSaturation; /* 倒车视频画面饱和度值 */
}RVCDataset_t;
```

15.4 使用说明和注意事项

15.4.1 倒车响应时间

倒车响应时间分为倒车冷启动时间及倒车非冷启动时间两种。倒车冷启动时间是指从在断电情况下，从硬件上电到倒车第一帧图像显示到屏幕上所花费的时间；倒车非冷启动时间指，在系统不断电情况下，从操作倒挡到倒车第一帧图像显示到屏幕上所花费的时间。由于冷启动倒车会首先对硬件进行初始化，因此往往慢于非冷启动响应时间。影响冷启动倒车响应时间的因素有：

1. 冷启动时间主要受所使用的 Sensor 和 Camera 本身上电稳定时间特性影响，不同的硬件环境可能会导致出图时间相差较大。
2. 倒车 UI 资源在系统启动时会从 EMMC 中读取 UI、Audio 等资源，过多的资源可能会拖慢出图时间。

R5 RVC 方案中，关闭倒车并不会实际关闭硬件，而只是使倒车画面不显示在屏幕上。因此可以做到非常快速的非冷启动倒车。

15.4.2 倒车资源冲突

15.4.2.1 Videoin 资源

R5 RVC 所使用的 Videoin 资源需要一直独占，即 R5 RVC 选择使用的硬件模块，Android 的所有应用将无法使用，主要有以下一些硬件冲突：

1. Videoin 硬件通路和 Sensor

R5 RVC 方案下，R5 RVC 选择使用的硬件通路将被 HSM 一直占用，Android 只能使用剩下的硬件通路。8025 平台的每条硬件通路都会配置工作域的 Metazone，R5 RVC 使用的硬件 Path 将被配置为 HSM 使用，其他域将无法使用。

15.4.2.2 Display 资源

由于 Display 硬件资源有限，只有 4 层 overlay 和 1 层 scaling module。且 Android 会尽可能地使用 overlay，降低 GPU loading，即大多数时间 Android 会占用 4 层 overlay（如 video 全屏播放等场景 Android 只使用 1 层 overlay）；而 R5 RVC 会占用至少两层 overlay，故在 Android 显示过程中，启动倒车会出现 display 资源不够用的冲突问题。

为解决 HSM 和 Android 之间 Display 资源冲突问题，Display 驱动中有加入资源分配策略，资源分配在 HSM Display 驱动中进行。HSM 中可以拿到所有的显示需求（Android & HSM），并会根据优先级分配有限的硬件资源。

目前优先级设定如下：priority 值越大优先级越高，获取离人眼更近的 overlay。

表 15-9 Display 优先级设定

Type	Priority	Type	Priority
RVC PQ UI	152	Video bootanimation	144
RVC UI	148	Android UI	100-103
RVC Video	145		

Display 驱动根据如上优先级进行资源分配，在复杂场景会出现低优先级显示需求无法获取资源，导致无法被显示出的结果。目前约定高优先级显示，如 RVC Setting App 需要满屏显示，保证用户无法感知到其他 Android UI 未真正显示。

为满足 Android 启动完成后，RVC Setting App 有显示到 R5 RVC Video&UI 之上的需求，ATC 提供定制机制如下：

1. device.mk 中设定标记，如 ATC Android RVC Setting App Demo 中设定标记为 arm2pqview

```
PRODUCT_PRODUCT_PROPERTIES += persist.sf.mark_layer=arm2pqview
```

2. RVC Setting App 设定 view 的 name 和标记一致，如 ATC Android RVC Setting App Demo 中如下：

```
mNativeLayout.setTitle(System.get(Contants.MARK_LAYER_KEY, "arm2pqview"))
```

Framework 会将该特殊标记图层显示到 R5 RVC Video & UI 之上，目前只支持最多标记一个 Android 图层。

15.5 用户定制化

客户在定制化 RVC 功能时时，应参考本章节内容，且需要额外注意多线程的并发与互斥。当需要定制化的功能较为复杂时，例如需要增删数据结构或者 Task 时，请联系 ATC 协助完成定制化功能开发。

15.5.1 倒车触发方式修改

目前 R5 RVC 倒车开关实现方式是通过向 vehicledrv 注册回调来监听倒车事件，而 vehicledrv 内部则是通过监听 GPIO 高低电平的变化来检测倒车事件。如果客户需要定制化 Uart、MCU 等方式触发，需要修改对应的实现。

客户需参考 rvc_event_monitor.h 和 rvc_event_monitor.c 的实现，将通过其他方法检测到的倒车事件通过 xRVCSendEvent 接口，传入事件类型参数（事件类型参数详见接口说明章节），发送即可。RVC Main Task 即可接收并处理倒车事件。

15.5.2 倒车显示配置

倒车显示支持一些用户配置项，如亮度、对比度、色度、饱和度、视频裁剪、视频镜像、视频翻转等等，这些配置项保存在 Metazone 中，在每次重启平台后，R5 RVC 将读取 Metazone 中最新的数值生效。

表 15-10 R5 RVC 显示配置相关 Metazone

Metazone Name	说明
METAZONE_RVC_DISP_ROTATION	R5 倒车显示翻转角度
METAZONE_RVC_DISP_MIRROR	R5 倒车显示镜像
METAZONE_RVC_DISP_RECT_*	R5 倒车显示区域相关配置，（X，Y，W，H）设定了一个左上角坐标为（X，Y），宽为W，高为H的矩形框，倒车视频将显示在屏幕上这个矩形框内
METAZONE_RVC_CROP_RECT_*	R5 倒车视频裁剪相关配置，（X，Y，W，H）设定了一个左上角坐标为（X，Y），宽为W，高为H的矩形框，倒车视频将被裁剪掉矩形框以外的部

	分，只保留矩形框内部的区域显示到屏幕上
METAZONE_RVC_PQ_BRIGHTNESS	R5倒车视频亮度
METAZONE_RVC_PQ_SATURATION	R5倒车视频饱和度
METAZONE_RVC_PQ_CONTRAST	R5倒车视频对比度

用户可以通过 adb shell 来修改 Metazone 的值，以修改亮度为例：

```
adb shell "echo dword_write METAZONE_RVC_PQ_BRIGHTNESS 127 > /proc/mtz_debug"
```

即可将亮度值修改为 128，重启后生效。

需要注意的是，这些 Metazone 是为公版测试方便设计的，客户并非必须使用。客户可以在读取这些 Metazone 的地方以硬编码的方式固定所使用的倒车显示配置。

同时，PQ 相关参数支持运行过程中实时修改，实现方式参考 RVC Setting App Set/Get Flow。

15.5.3 RVC 的输入源设置

由于 AC8025 平台支持多硬件通路、多通道的视频输入，为了方便对每一种输入的测试，需要选择 RVC 使用的输入类型。配置 R5 RVC 输入源的方式也是通过 Metazone，相关的 Metazone 配置如下表所示。

表 15-11 R5 RVC 输入源配置相关 Metazone

Metazone Name	说明
METAZONE_R5_RVC_DEV_ID	R5 倒车使用的 Camera ID，每个 Camera ID 对应一个硬件通路通道
METAZONE_MIPI*_CH_ENABLE	配置 MIPI* 硬件通路的通道是否使能
METAZONE_MIPI*_CH_RESOLUTION	配置 MIPI* 硬件通路的视频输入格式

MIPI 相关配置具体内容，可参考 Camera 章节介绍。需要注意的是，这些 Metazone 是为公版测试方便设计的，客户并非必须使用。客户可以在读取这些 Metazone 的地方以硬编码的方式固定所使用的输入源。

15.5.4 R5 RVC UI 定制

R5 RVC UI 既可以使用描点绘图或算法生成等方式直接对 UI Buffer 内容进行绘制填充，也可以使用贴图的方式：将图片资源使用 MRFTool 打包并烧写到 Datazone 分区，系统启动时进行资源的搬运、解析和使用。

15.5.4.1 UI 更新流程

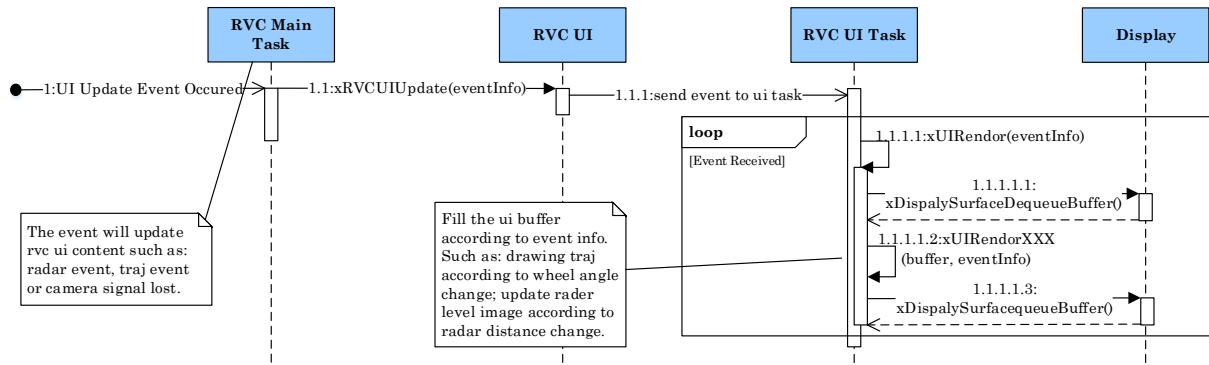


图 15-8 R5 RVC UI 更新流程

R5 RVC UI Task 在接受到需要更新 UI 的事件后，会从 Display 申请一块空闲 Buffer，根据事件信息内容填充这块 Buffer：可以在不同的区域填充不同的内容（例如在左半边区域填充轨迹线，右半边填充雷达），或将不同的内容进行叠加并进行透明度的融合（例如在雷达背景之上叠加半透明显示雷达警告标识），但需要注意整体填充的时间性能问题，过多的像素融合计算会影响 UI 更新的时效性。最后将完成填充后的 Buffer 进行送显。

更新 UI 的事件类型和填充 UI Buffer 的方式，客户可以根据实际使用进行定制化。

15.5.4.2 UI 更新事件类型扩展

以添加汽车方向盘角度事件为例说明如下：

(1) 扩展结构体：source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/rvc/rvc_ui.h

```

typedef enum UIUpdateEventType {
    eUnknowUpdate,
    eRVCEnvirUpdate,
    eVideoSignalUpdate
}UIUpdateEventType_t;

typedef struct UIUpdateEvent {
    UIUpdateEventType_t xUIUpdateEventType;
    void * xUIUpdateEventData;
}UIUpdateEvent_t;
    
```

source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/rvc/rvc_main.h

```
typedef struct RVCEnvirInfo {
    uint32_t ulDistance[4];

    int32_t lSteeringAngle;

    BaseType_t xIsTurnRight;
}UIUpdateData_t;
```

(2) 添加更新事件的发送和接收:

source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/rvc/rvc_ui.c

```
/* send event in this function */
BaseType_t xRVCUIUpdate( UIUpdateEvent_t xUIUpdateEvent )
{
    ...;

    switch( xUIUpdateEvent.xUIUpdateEventType ) {
        case eRVCEnvirUpdate:
            xQueueOverwrite( xUIEnvirEventQueue, (RVCEnvirInfo_t
*)xUIUpdateEvent.xUIUpdateEventData);

            break;

        ...;
    }
}
```

```
/* receive and handle event in this function */
void vRVCUITask( void *pvParameters )
{
    ...;

    if(pdPASS != xQueueReceive(xUIEnvirEventQueue, &xRVCEnvirInfo, 0))
    {
        /* do error handle */
    }
    else
    {
        vHandleEnvirEvent(&xRVCEnvirInfo);
    }
}
```



```

    ...;
}

```

(3) 添加更新事件的 UI 处理:

source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/rvc/rvc_ui.c

```

void vHandleEnvirEvent(RVCEnvirInfo_t *pRVCEnvirInfo)
{
    ...;
    xUITestData.wheelAngle = pRVCEnvirInfo->lSteeringAngle;
    if (pdPASS != xUIRenderDynamic(xUITestData))
    {
        /* do error handle */
    }
    ...;
}

```

```

static BaseType_t xUIRenderDynamic(UIUpdateData_t xData)
{
    GraphicBufferHandle_t xUIBuffer = NULL;
    ...;
    xDisplaySurfaceDequeueBuffer(xUISurface, &xUIBuffer, xDeqBuffWait);
    ...; /* render other ui content */
    xUIRenderDynamicImpl(xUIBuffer, &xData);
    ...;
    xDisplaySurfaceQueueBuffer(xUISurface, xUIBuffer);
    ...;
}

```

source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/rvc/rvc_ui_render_impl.c

```

Basetype_t xUIRenderDynamicImpl(raphicBufferHandle_t pxUIBuffer,
                                UIUpdateData_t *pxData)
{

```

```

...;

/* fill the buffer according to the wheel angle
 * in the xUIRenderTraj.
 */

xUIRenderTraj(pxUIBuffer, pData->wheelAngle);

...;
}

```

(4) 更新事件发生时调用 UI 更新接

source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/rvc/rvc_main.c

```

BaseType_t xRVCUpdate(RVCEnvirInfo xEnvirInfo)
{
    ...;

    UIUpdateEvt_t xUIUpdateEvent = {eRVCEnvirUpdate,
                                     (void*)&xEnvirInfo};

    xRVCUIUpdate(xUIUpdateEvent);
}

```

15.5.4.3 UI Buffer 填充方式

(1) 算法或绘图接口填充

以上一节中绘制轨迹线为例，在函数 xUIRenderTraj 中实现算法或者绘图，对形参 pxUIBuffer 进行填充即可。

(2) 贴图方式填充

以贴图方式绘制倒车提示语 UI 为例说明如下：

- (a) 使用 MRF Tool，将提示语图片加入倒车模块资源群中，和其他模块资源一起打包生成 MRF 文件。

需要注意的是，打包时倒车资源的序列是不能随意改变的，倒车运行时，会根据约定的序号访问和使用对应的资源，如果出现不匹配则会出现功能的紊乱。

此外，每个倒车图片显示在屏幕中的起始坐标可以在 MRF 打包时进行填写保存，实际使用时进行解析设定（本方案默认使用方式），也可以直接在代码里使用数组等方式保存。

如下图示例，倒车提示语图片打包的序号是 0，将会显示在屏幕的起始位置为（312， 10）：

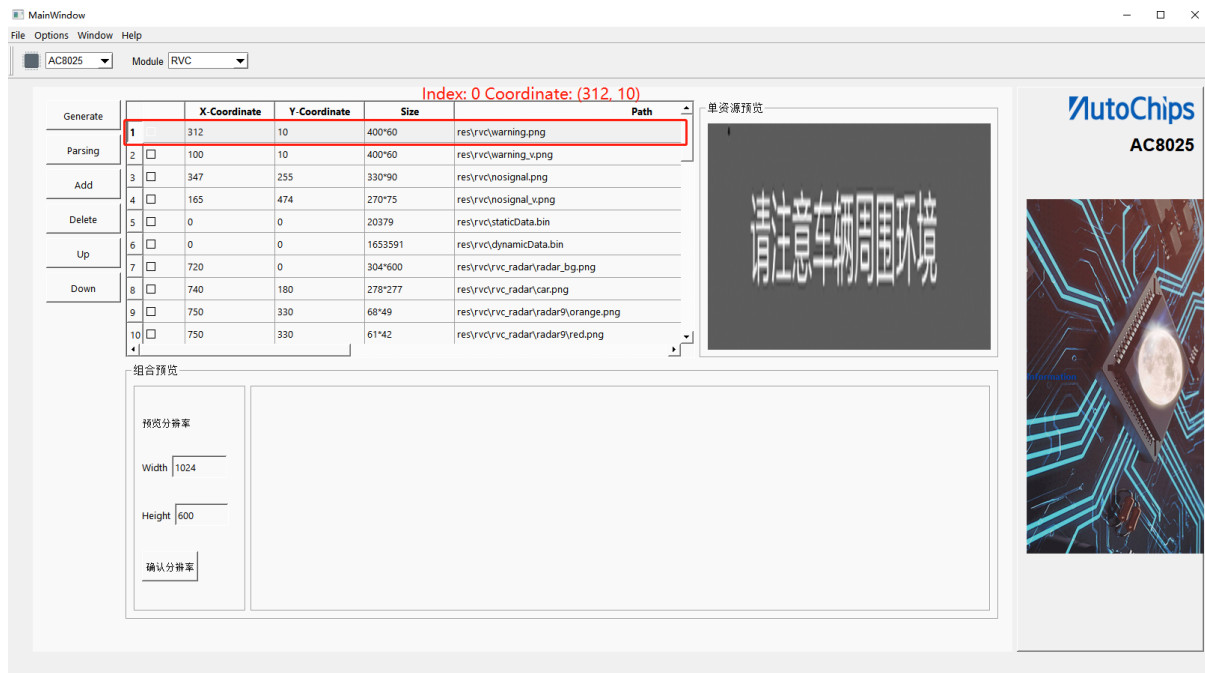


图 15-9 MRF Tool RVC 界面

- (b) 将生成的资源文件添加到代码路径 `build/projects/ivi_s_ac8025/datazone` 下，重命名为 `rvc_resource.bin`，并修改 `build/projects/ivi_s_ac8025` 路径下的 `datazone_config.json`，将 `rvc_resource` 的 `size` 字段修改为大于等于 `rvc_resource.bin` 实际字节数的值。这样修改之后，最新编译的 `img` 中就会包含最新的资源。
- (c) R5 RVC 实际使用时，先定义新加入的图片资源的序号宏（需要和实际打包的 MRF 资源序号严格一致），在需要使用该图片对 UI Buffer 进行填充时，调用 `xUILoadImage` 接口，传入序号和 Buffer 即可。

15.5.4.4 UI 资源打包限制

使用 MRF Tool 打包并使用 UI 资源，目前有以下一些限制：

- 支持图片格式为 `bmp`、`png` 或 `bin` 类型的 UI 资源打包；
- 仅支持 `RGB565` 和 `ARGB8888` 两种数据格式的 UI 图片，且不能混搭。R5 RVC 会根据 MRF Tool 存储的第一张倒车图片资源的数据格式自动适配；
- 打包各个模块资源的最终 MRF 文件不能超过 `Datazone` 分区的大小；
- RVC 资源的大小不能超过预分配的 `RVC Reserve Memory` 大小，目前设定的阈值为 8M。修改阈值需要修改 `build/projects/ivi_s_ac8025/ac8x_reserved_memory.csv` 表格中的“`rvc_resource`”节点的大小。

15.5.4.5 Audio 资源定制

R5 RVC 支持 Audio 播放，方便客户定制化倒车警报音等需求。Audio 数据的获取和 UI 完全相同，都是提前打包在 Datazone 中，R5 RVC 在使用时进行解析和获取，最后将 Reserve Memory 地址直接传给 Audio 接口，即可进行声音的播放。

具体使用方式和支持的 Audio 文件格式等内容，请参考 [AudioIO](#) 相关内容。

15.5.5 R5 RVC 模块开关

R5 RVC 为了单独调试 Video、UI 或 Audio 等模块，避免其他模块的影响，或者倒车不需要某一个功能，提供了单独开关功能模块的宏定义。

修改文件：source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/rvc/rvc_test.h

```
#define ENABLE_VIDEO_FUNC      1
#define ENABLE_UI_FUNC        1
#define ENABLE_AUDIO_FUNC     1
```

15.6 典型问题

15.6.1 R5 RVC 显示异常

【问题描述】

R5 RVC 的 Video/UI 显示出现不显示或花屏等异常。

【解决方案】

针对 RF RVC 的图像显示异常问题，提供一种基于 CLI 的图像 dump Debug 方式来定位 Bug。使用步骤介绍如下。

1. 确认需要 Dump 的内存地址与大小

在 log 中查找关键字“display buffer info”：

```
[RVC][UI] xUISurfaceBufferAlloc(line:326): display buffer info: sWidth(1024), sHeight(600),
lFormat(0x34325241), lSize(2457600), sPlaneCount(1), pcHandle(0x0000000811000000).
```

log 前缀为[RVC][UI]的行，对应 UI 使用的 buffer，pcHandle 后的地址 0x0000000811000000 即为要 Dump 的内存地址，lSize 后的数字 2457600 即为 Buffer 内存的大小。

```
[RVC][VIDEO] xVideoSurfaceBufferAlloc(line:452): display buffer info: sWidth(1280),
sHeight(720), lFormat(0x3231564e), lSize(1382400), sPlaneCount(2),
pcHandle(0x00000008118ca000).
```

log 前缀为 [RVC][VIDEO] 的行，对应 Video 使用的 buffer，pcHandle 后的地址 0x00000008118ca000 即为要 Dump 的内存地址，lSize 后的数字 1382400 即为 Buffer 内存的大小。

- 以 root 用户使用 adb shell 输入命令行，跳转到 mem block

```
echo cd mem > /sys/cli/commands
```

- 使用如下命令完成 dump 功能

```
echo dump <phys_address> <size> [number] [time] > /sys/cli/commands
```

- phys_address: 必选参数，需要 dump 的物理地址，使用步骤 1 中所述的 Video 或 UI 的物理地址。
- size: 必选参数，Dump 的 Buffer 内存大小，使用步骤 1 中所述的 Video 或 UI 的 Buffer 内存大小。
- number: 可选参数，需要连续 dump 的次数，默认为 1
- time: 可选参数，连续 Dump 的时间间隔，默认为 0，单位为 ms。

- 将/data/backup/中 Dump 的图像 pull 到 PC 端，使用 7YUV 等工具检查图片是否有异常。

对于 UI 图像而言，若 Dump 出的图像无异常，则需要进一步定位 Display 问题。若 Dump 出的图像有异常，则需要排查倒车资源文件打包是否有异常。

对于 Video 图像而言，若 Dump 出的图像无异常，则需要进一步定位 Display 问题。若 Dump 出的图像有异常，则需要进一步排查 Videoin Driver 相关问题。

16 Watermark

16.1 简介

16.1.1 软硬件环境

Watermarkcore 是水印的核心，它实现在 Native 层，代码与平台无关。使用 libpng 库解析 png 图像，使用 libtinyxml2 库进行 xml 文件的解析。当前位置信息（经纬度）以及车速信息的获取依赖于 Android 系统，由 WatermarkService 实现。

16.1.2 功能

水印的主要功能如下：

- WatermarkService 通过 Android 位置服务获得当前位置信息（经纬度）以及车速信息；利用 IPC 通信实现多 OS 之间的位置、车速信息传递；
- Watermarkcore 通过加载 xml 文件中预先设置的水印 Icon 图像，可将 Icon 图像叠加到源图像的指定位置；水印素材与叠加的目标图像无需格式一致。目前水印素材支持的格式有：NV12、NV21、YV12、BGRA8888 和 ARGB8888；
- WatermarkWrapper 对 Watermarkcore 进行封装，提供包含时间、位置、车速等信息的固定格式水印，便于用户使用。
- 水印定制化：用户可以通过更改配置文件中的水印素材和调用相关接口，对任意的素材进行任意位置的叠加。如：调整时间水印的字体可以通过替换含字号较大数字的图片来达到效果。

16.1.3 代码结构及文件描述

Watermark 可分为 WatermarkService、Watermarkcore 和 WatermarkWrapper 三部分，各部分的代码结构及描述可见表 16-1。

Watermarkcore 是水印功能的核心实现，代码结构可见表 16-2。

表 16-1 Watermark 代码结构及描述

文件名	文件路径	文件描述
Watermark Wrapper	source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkwrapper	供上层 APP 调用，提供时间和 GPS 信息水印

文件名	文件路径	文件描述
Watermark Core	source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkcore	叠加水印的底层实现
Watermark Service	source/vendor/autochips/proprietary/hardware/videoin/services/WatermarkService	水印服务端，获取 GPS 信息

表 16-2 Watermarkcore 代码结构及描述

文件名	文件路径	文件描述
WatermarkManagerImpl.cpp	source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkcore /libs/src	水印管理模块，用于管理水印资源及 MergeExcutor 实例
MergeExcutor.cpp	source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkcore /libs/src	提供水印叠加、水印设置相关接口
WatermarkConfig.cpp	source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkcore /libs/src	封装对 xml 配置文件进行解析的逻辑
MergeImpl.cpp	source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkcore /libs/src	实现水印叠加的功能
WatermarkUtils.cpp	source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkcore /libs/src	实现像素格式转换及相关逻辑
BufferManager.cpp	source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkcore /libs/src	封装水印文件读取及 buffer 管理的逻辑
png_utils.cpp	source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkcore /libs/src	实现 png 图像解码逻辑

16.2 软件设计

16.2.1 架构图

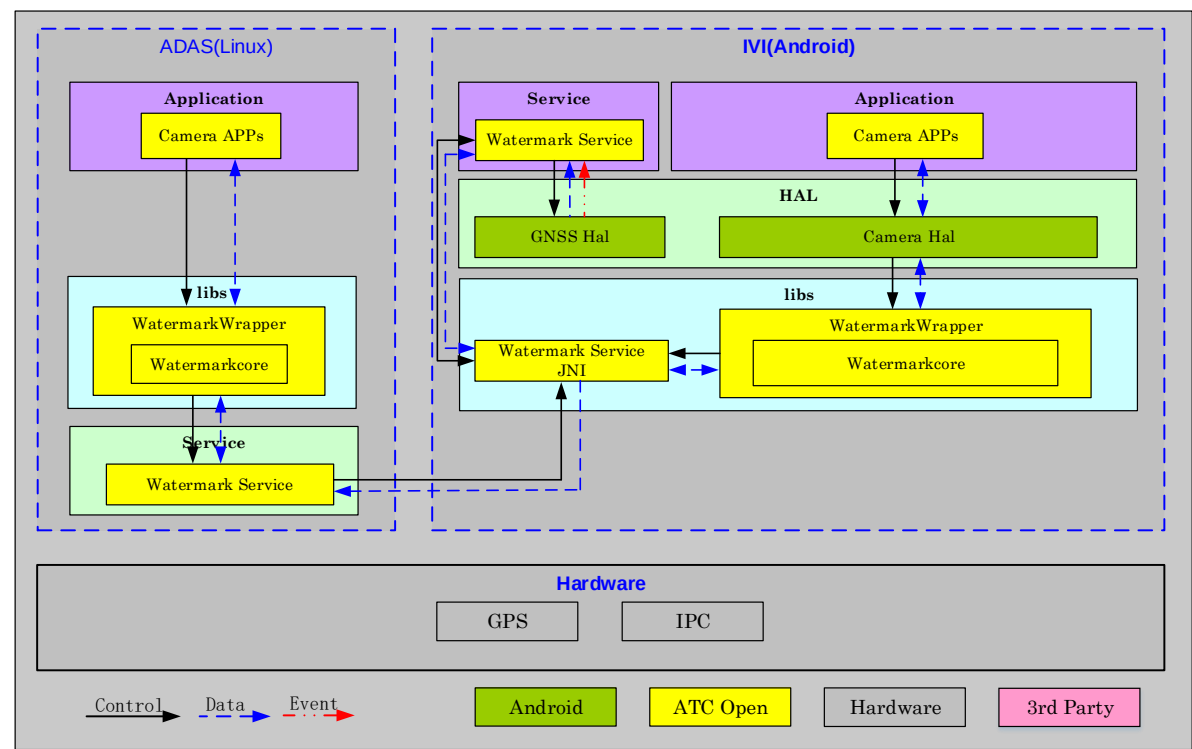


图 16-1 Watermark 整体架构图

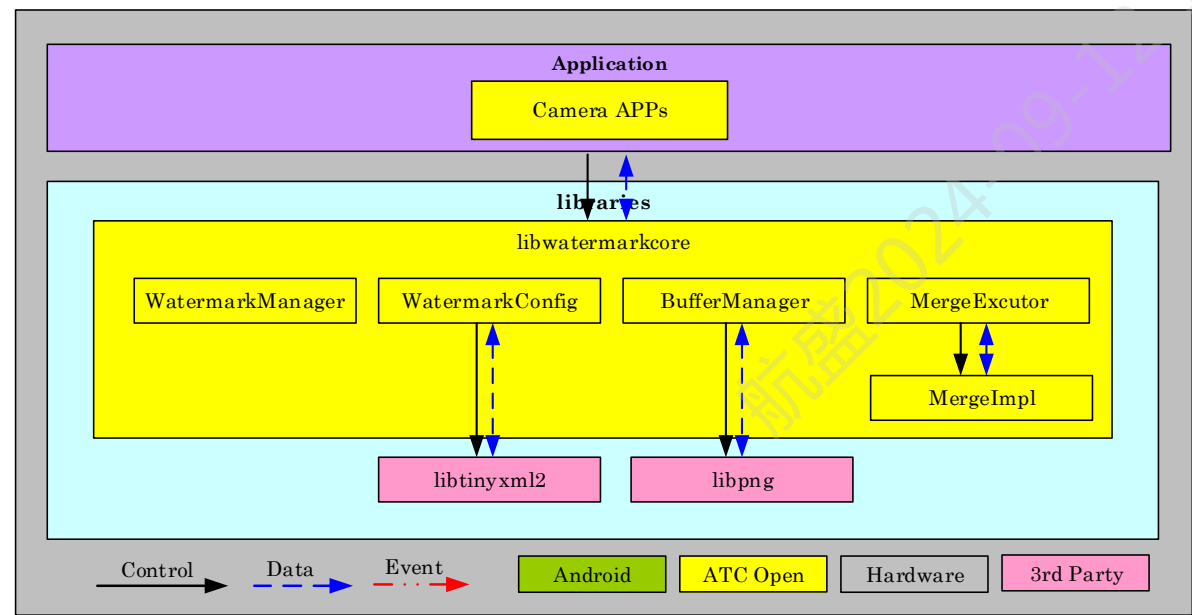


图 16-2 Watermarkcore 架构图

16.2.2 流程图

16.2.2.1 Watermarkcore 工作流程

Watermarkcore 是水印功能的核心实现，工作流程如下：

1. 调用者首先通过 `getWatermarkManger` 接口获取 `WatermarkManager` 的实例，该实例为单例，在进程结束时自动销毁。
2. 然后，指定所需的 `Watermark id` 及待叠加水印的图像像素格式，调用 `getMergeInst` 接口，即可创建 `IMergeExcutor` 实例。在此过程中，将会把所有所需的水印 `Icon` 图像读取到内存中，并转化为叠加水印的图像像素格式。如果水印 `Icon` 图像以 `png` 格式存储，则先通过 `libpng` 将图像解析为 `RGBA8888` 格式的图像，再转换为目标格式的图像。
3. `MergeExcutor` 实例创建完毕后，即可对其中持有的 `Watermark id` 对应的水印进行配置。可设置的参数有：
 - 是否打开水印
 - 水印是否闪烁，以及闪烁的周期
 - 水印在待叠加水印的图像上的位置
 - `Watermark type` 为 `TYPE_MULTI_ICON` 时，可配置多个水印 `Icon` 在水印中的布局。以上参数可以对每个 `Watermark id` 单独配置，每个 `Watermark id` 之间互相不影响。
4. 配置完毕后即可进行水印的叠加操作，调用一次叠加接口即可将所有打开的水印叠加到待叠加水印的图像上。每次叠加操作之前可以重新进行配置，并在叠加时生效配置；
5. 最后，调用 `freeMergeInst` 接口完成对 `IMergeExcutor` 实例的销毁。

具体使用示例可见 [16.4.3](#) 节。

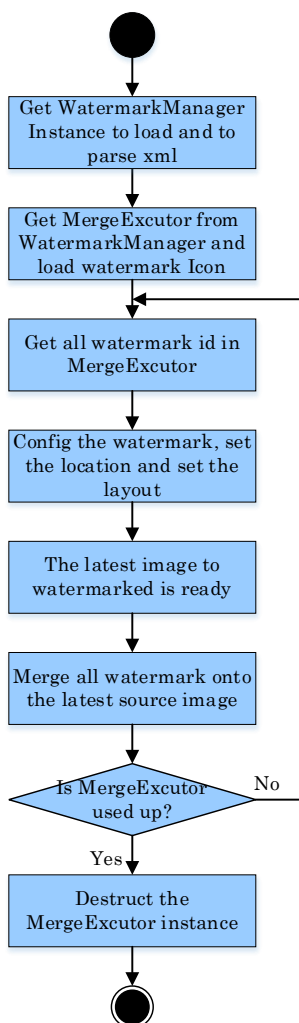


图 16-3 Watermarkcore 工作流程图

16.2.2.2 WatermarkWrapper 工作流程

下图为 WatermarkWrapper 的工作流程，ADAS 域 (Linux) 下与 IVI 域 (Android) 下 WatermarkWrapper 工作流程相同：

1. 当调用者需要叠加一帧水印图像时，先创建 WatermarkWrapper 实例；
2. 创建该实例时，WatermarkWrapper 内部会创建异步线程，定时从共享内存中读取当前的位置信息。当用户调用 merge 接口进行叠加水印时，根据线程所获取的位置信息，调用 libwatermarkcore 中相应接口，配置好当前水印的布局，完成水印的叠加；
3. 最后，通过 freeInstance 接口对 WatermarkWrapper 实例进行销毁。

具体使用示例可见 16.4.4 节。

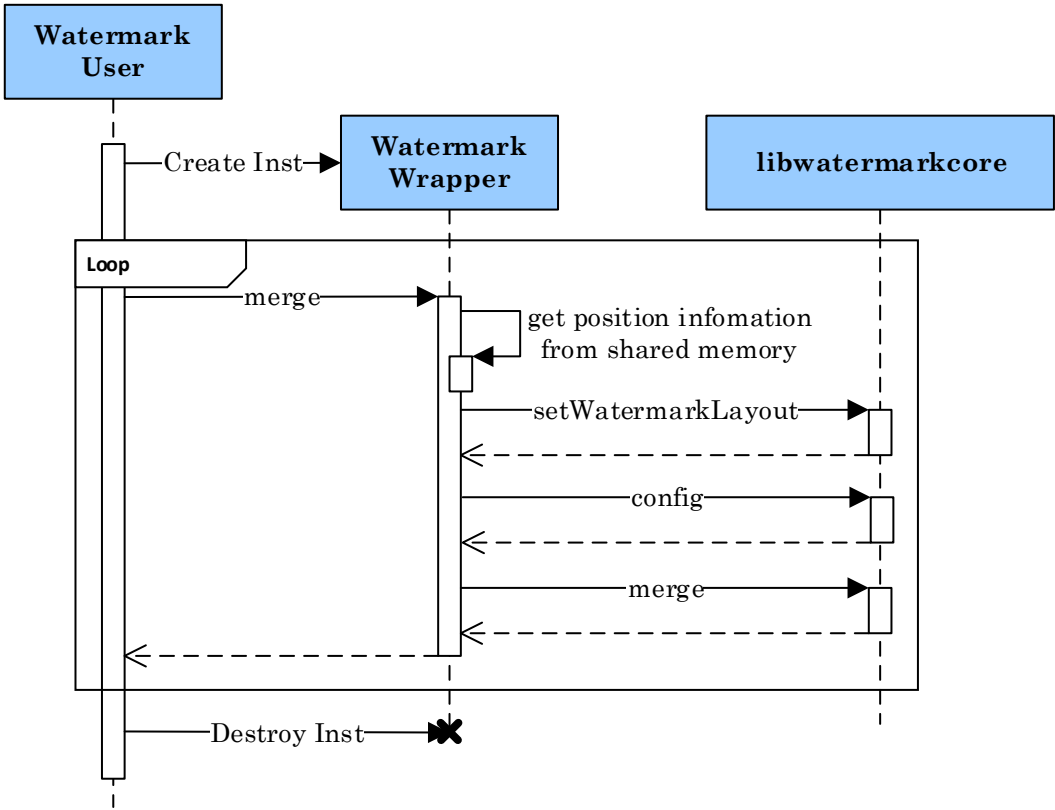


图 16-4 WatermarkWrapper 工作流程

16.2.2.3 WatermarkService 工作流程

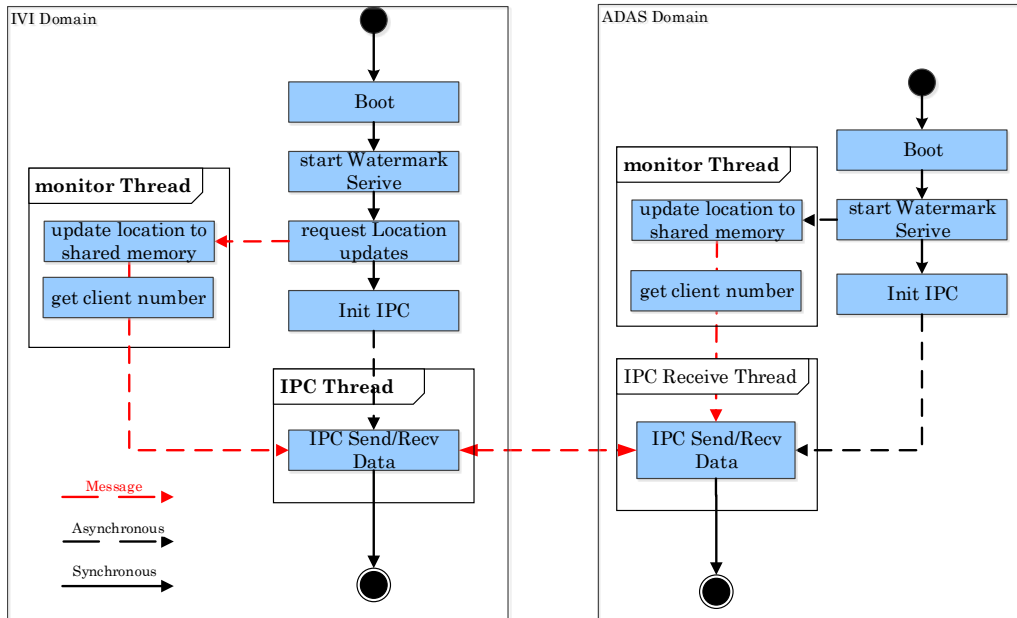


图 16-5 Watermark Service 位置信息获取流程图

由于 ADAS 域(Linux)下无法直接获取位置信息，因此必须通过 IPC 通信从 IVI 域(Android)间接获取。如上图所示，IVI 域与 ADAS 域下各部署一个 Watermark Service。

在 IVI 域内部以及 ADAS 域内部，需要依赖共享内存完成 Watermark Service 与 Watermark Wrapper 之间的通信，Watermark Service 将 GPS 信息更新到共享内存中，Watermark Wrapper 从共享内存中读取 GPS 信息。此外，共享内存中保存一把互斥锁，能够使多进程能够互斥访问该区域，共享内存还维护了一个 client number 的计数器，Watermark Service 根据此变量的数目来决定是否开启位置信息的更新。

IVI 域下，通过 JAVA 接口注册位置信息的监听，当位置信息发生变化时，会通过回调得到此信息。然后将位置信息更新到共享内存中，并通过 IPC 将位置信息发送给 ADAS 域。

ADAS 域下，也存在一个自启的 Watermark Service。该进程中，IPC Receive Thread 始终阻塞式的接收 IPC 消息，并在位置信息有更新时通知 updateThread 将位置信息更新到共享内存中。

16.3 接口描述

16.3.1 WatermarkWrapper API

在叠加水印前，用户需要调用 Watermarkcore 中相应接口指定水印的布局和叠加位置。WatermarkWrapper 将这些前置接口调用合并为一个接口。它的主要功能提供固定格式的时间、位置、车速等信息的水印，对如 Camera APP 等具有等同需求的调用者更易用。用户可以参考本代码，根据需求进行定制化封装开发，也可直接调用 3.2 中 Watermarkcore API 使用。

16.3.1.1 IWatermarkWrapper 类

表 16-3 IWatermarkWrapper 类

Public方法	说明
getInstance()	获取IWatermarkWrapper实例，每个instance仅支持一种格式的水印
freeInstance()	释放IWatermarkWrapper实例
getWatermarkInfo()	获取IWatermarkWrapper实例的水印信息
merge()	进行水印叠加

16.3.1.1.1 getInstance

【描述】

Get the IWatermarkWrapper instance.

【语法】

```
static IWatermarkWrapper *getInstance(unsigned int pixelfmt);
```

【参数】

the pixel format of dest video image

【返回值】

- a IWatermarkWrapper instance pointer means success
- nullptr means failure

16.3.1.1.2 freeInstance

【描述】

Free the IWatermarkWrapper instance.

【语法】

```
static int freeInstance(IWatermarkWrapper *inst);
```

【参数】

the instance that need to free

【返回值】

- WATERMARK_SUCCESS means success
- WATERMARK_INVALID_PARAM means failed

16.3.1.2 getWatermarkInfo

【描述】

Get the watermark information of this IWatermarkWrapper instance.

【语法】

```
virtual videoin_format getWatermarkInfo() = 0;
```

【参数】

None

【返回值】

Supported formats, including pixel format and width and height. The type is videoin_format.

16.3.1.3 Merge

【描述】

Merge watermark on the videoin img.

【语法】

```
virtual int merge(const videoin_buf_info &img, int startX, int startY) = 0;
```

【参数】

- img: the information of image which need to merge watermark
- startX: default X-coordinate of the merge start point
- startY: default Y-coordinate of the merge start point

【返回值】

- WATERMARK_SUCCESS means success
- WATERMARK_STATUS means merging in an invalid state

- WATERMARK_INVALID_PARAM means invalid params

16.3.2 Watermarkcore API

Watermarkcore 是水印的核心实现，可读取配置文件的 icon 水印素材，提供 setWatermarkPosition、setWatermarkLayout 等接口灵活控制水印布局 and 位置等。当 WatermarkWrapper API 中提供固定格式 GPS 信息和时间的水印不满足需求时，用户可以参照 WatermarkWrapper 对 Watermarkcore 进行封装。

16.3.2.1 IWatermarkManager 类

IWatermarkManger 类提供创建与删除 IMergeExcutor 实例的接口，其内部管理所有 Watermark 资源。

表 16-4 IWatermarkManger 类

Public方法	说明
getWatermarkManager()	获取IWatermarkManger单例
getMergeInst()	获取IMergeExcutor实例，即：水印实际执行者
freeMergeInst()	释放IMergeExcutor实例
getAllWatermarkIdList()	获取水印配置文件中的所有水印id
getFormatAbilities()	获取某水印id所有支持的图像格式，可根据该格式来获取MergeExcutor实例

16.3.2.1.1 getWatermarkManager

【描述】

Get IWatermarkWrapper singleton instance.

【语法】

```
static IWatermarkManager *getWatermarkManager(const std::string &configPath
= "/vendor/etc/atcwatermark_config.xml");
```

【参数】

configPath: the xml config file path. Default is atcwatermark_config.xml.

【返回值】

- A IWatermarkManager singleton instance pointer means success
- nullptr means failure

16.3.2.1.2 getMergeInst

【描述】

Get IMergeExcutor instance.

【语法】

```
virtual IMergeExcutor *getMergeInst(const std::vector<int> &ids, unsigned int pixFmt) = 0;
```

【参数】

- ids: a set of watermark ids held by the new IMergeExcutor instance
- pixelFmt: the target pixel format which the watermark icon need to be converted

【返回值】

- A IMergeExcutor instance pointer means success
- nullptr means failure

【注意】

- The instance must free by freeMergeInst.
- The pixel format of image to be watermarked must be the same as pixFmt.

16.3.2.1.3 freeMergeInst

【描述】

Free IMergeExcutor instance.

【语法】

```
virtual int freeMergeInst(IMergeExcutor *mergeInst) = 0;
```

【参数】

mergeInst: a IMergeExcutor instance pointer which returned by getMergeInst()

【返回值】

- 0 means success
- Non 0 means failure

16.3.2.1.4 getAllWatermarkIdList

【描述】

Get all watermark ids in the xml configuration file.

【语法】


```
virtual std::vector<int> getAllWatermarkIdList() = 0;
```

【参数】

None

【返回值】

All the watermark ids in the xml configuration file

16.3.2.1.5 getFormatAbilities

【描述】

Get all the supportive pixel formats of the IMergeExcutor for the param watermark id.

【语法】

```
virtual std::vector<int> getFormatAbilities(int id) = 0;
```

【参数】

id: the watermark id that need to get its supportive pixel format

【返回值】

All supportive pixel format of source image to be watermarked

16.3.2.2 IMergeExcutor 类

IMergeExcutor 实例持有一组 Watermark id 及其设置好像素格式的水印，支持通过接口对这些水印进行配置，并能通过接口将所有持有的水印按照配置叠加到待叠加水印的图像上。

表 16-5 IMergeExcutor 类

Public方法	说明
int merge(img)	将所有在该MergeExcutor中的水印都叠加到图像上
int merge(img, ...)	将指定的水印素材叠加到图像上
getWatermarkList()	获取所有支持的水印id
int config()	对水印进行配置，可以打开或关闭水印，设置水印是否闪烁，指定闪烁周期等
int setWatermarkPosition()	设置水印位置
int setWatermarkLayout()	设置水印素材的布局
int setHWMerge()	设置是否采用硬件叠加水印
int getFormat()	获取该IMergeExcutor实例的图像格式。被叠加的目标图像需与对应调用的IMergeExcutor的图像格式一致

16.3.2.2.1 merge(one-param)

【描述】

Merge all enabled watermark in IMergeExcutor instance onto the image.

【语法】

```
virtual int merge(const ImgInfo &img) = 0;
```

【参数】

img: the information of the image which need to watermarked

【返回值】

- 0 means success
- non 0 means failure

16.3.2.2.2 merge(multi-params)

【描述】

Merge custom watermark on to the img.

【语法】

```
virtual int merge(const ImgInfo &img, const ImgInfo &watermark, int startX, int startY,  
unsigned char alpha = 0) = 0;
```

【参数】

- img: the information of image which need to be watermarked
- watermark: the information of custom watermark
- startX: coordinate X on image of watermark top left
- startY: coordinate Y on image of watermark top left
- alpha: the alpha value of the watermark, in order to filter the background of the watermark material in some formats that do not have it

【返回值】

- 0 means success
- non 0 means failure

16.3.2.2.3 getWatermarkList

【描述】

Get all watermark id in this IMergeExcutor instance.

【语法】

```
virtual std::vector<int> getWatermarkList() = 0;
```

【参数】

None

【返回值】

The id vector in this instance.

16.3.2.2.4 config

【描述】

Configure watermark.

【语法】

```
virtual int config(int id, bool enable, bool isBlink = false, unsigned int blinkPeriod = 0, unsigned char alpha = 0) = 0;
```

【参数】

- id: the id that the watermark to be configured
- enable: enable this watermark or not
- isBlink: make this watermark blink or not
- blinkPeriod: watermark blink period, valid when isBlink is true
- alpha: the value of the watermark alpha channel

【返回值】

- 0 means success
- non 0 means failure

16.3.2.2.5 setWatermarkPosition

【描述】

Set watermark position.

【语法】

```
virtual int setWatermarkPosition(int id, int startX, int startY) = 0;
```

【参数】

- id: the id that the watermark to be set position
- startX: coordinate X on image of watermark top left
- startY: coordinate Y on image of watermark top left

【返回值】

- 0 means success
- non 0 means failure

16.3.2.2.6 setWatermarkLayout

【描述】

Set watermark layout for TYPE_MULTI_ICON.

【语法】

```
virtual int setWatermarkLayout(int id, const std::vector<int> &layout) = 0;
```

【参数】

- id: the id that the watermark to be set position
- layout: the layout of icon in xml ImgFileName field, -1 in layout vector means no icon in this location. For example: ImageFileName field is "0.yuv,1.yuv,2.yuv". layout {0, 2, -1, 1} means the watermark icons from left to right are: 0.yuv 2.yuv empty 1.yuv

【返回值】

- 0 means success
- non 0 means failure

【注意】

This interface is valid only when type of watermark is TYPE_MULTI_ICON, in other words, Type of this watermark in xml is 2.

16.3.2.2.7 setHWMerge

【描述】

Set hardware merge or software merge. Currently only the software merge is supported.

【语法】

```
virtual int setHWMerge(bool isHWMerge) = 0;
```

【参数】

isHWMerge: true means use hardware, or false means software merge

【返回值】

- 0 means success
- non 0 means failure

16.3.2.2.8 getFormat

【描述】

Get watermark pixel format in this instance, the pixel format of image to be watermarked must be the same as this pixel format.

【语法】

```
virtual unsigned int getFormat() = 0;
```

【参数】

None

【返回值】

watermark pixel format in this instance

16.3.3 数据结构

表 16-6 Watermark 数据结构

数据结构	说明
WatermarkType	水印类型。 ▪ 0：表示单个图标的水印； ▪ 1：表示图标组成的时间水印，格式为2023-05-19 15:50:33； ▪ 2：表示自由组合图标的水印，如组合成含车速、GPS等信息的水印
ImgInfo	图像信息，含虚拟地址、物理地址、像素格式、宽高和步长等信息
videoin_buf_info	Buffer信息，包含像素格式、步长、buffer大小、buffer handle、虚拟地址、video状态和时间戳

16.3.3.1 WatermarkType

```
enum class WatermarkType
{
    TYPE_SINGLE_ICON = 0; // 单 Icon 图像组成的水印，对应 xml 中 Type 0
    TYPE_TIME_ICON    = 1; // 当前时间的水印，对应 xml 中 Type 1
}
```

```
TYPE_MULTI_ICON = 2; // 多个 Icon 自由组合的水印，对应 xml 中 Type 2
};
```

16.3.3.2 ImgInfo

```
struct videoin_buf_info
{
    void *va {nullptr};
    void *pa {nullptr};
    unsigned int pixelfmt {VIDEOIN_PIXEL_FMT_INVALID}; /* pixel format of input */
    int width {0}; /* width of pixels */
    int height {0}; /* height of pixels */
    int stride[3] {0, 0, 0};
};
```

- va: 存放图像内存的虚拟地址
- pa: 存放图像内存的物理地址，保留参数，目前可以置为 nullptr
- pixelfmt: 图像的像素格式，使用 videoin 图像格式的定义，见 [16.3.3.4](#)
- width: 图像的像素宽度
- height: 图像的像素高度
- stride: 数组的三个元素分别表示三个 plane 的步长

16.3.3.3 videoin_buf_info

```
struct videoin_buf_info
{
    struct videoin_format fmt;
    int stride[MAX_BUF_PLANE_CNT]; /*buffer stride for buffer plane. Max count
is BUF_PLANE_CNT */
    int size; /* buffer size */
    int fd; /* buffer handle */
    void *va; /* Virtual start address of buffer */
    enum signal_status status; /* videoin source status */
    uint64_t timestamp; /* Data timestamp in */
};
```

```
};
```

16.3.3.4 videoin_format

```
struct videoin_format
{
    unsigned int pixelfmt;    /* pixel format */
    unsigned int width;      /* width of pixels */
    unsigned int height;     /* height of pixels */
    unsigned int fps;        /* frames per second */
};
```

pixelfmt 包含如下格式:

```
#define VIDEOIN_PIXEL_FMT_YUYV    videoin_pixel_type('Y', 'U', 'Y', 'V')
表示 YUYV 格式的图像

#define VIDEOIN_PIXEL_FMT_UYVY    videoin_pixel_type('U', 'Y', 'V', 'Y')
表示 UYVY 格式的图像

#define VIDEOIN_PIXEL_FMT_YYUV    videoin_pixel_type('Y', 'Y', 'U', 'V')
表示 YYUV 格式的图像

#define VIDEOIN_PIXEL_FMT_NV12    videoin_pixel_type('N', 'V', '1', '2')
表示 NV12 格式的图像

#define VIDEOIN_PIXEL_FMT_NV21    videoin_pixel_type('N', 'V', '2', '1')
表示 NV21 格式的图像

#define VIDEOIN_PIXEL_FMT_YUV420  videoin_pixel_type('Y', 'U', 'I', '2')
表示 YU12/YUV420 格式的图像

#define VIDEOIN_PIXEL_FMT_YU12    VIDEOIN_PIXEL_FMT_YUV420
表示 YU12/YUV420 格式的图像

#define VIDEOIN_PIXEL_FMT_YVU420  videoin_pixel_type('Y', 'V', '1', '2')
表示 YV12/YVU420 格式的图像

#define VIDEOIN_PIXEL_FMT_YV12    VIDEOIN_PIXEL_FMT_YVU420
表示 YV12/YVU420 格式的图像

#define VIDEOIN_PIXEL_FMT_BGRA8888 videoin_pixel_type('B', 'A', '2', '4')
```

表示 BGRA8888 格式的图像

```
#define VIDEOIN_PIXEL_FMT_INVALID 0xFFFFFFFF
```

表示无效格式的图像

以上宏定义中，videoin_pixel_type 的定义如下：

```
#define videoin_pixel_type(a, b, c, d) (((unsigned int)(a) | ((unsigned int)(b) << 8) | ((unsigned int)(c) << 16) | ((unsigned int)(d) << 24))
```

16.4 使用说明和注意事项

16.4.1 库名及头文件

WatermarkWrapper:

- 动态库: libwatermarkwrapper.so
- 头文件:
source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkwrapper/Interface

Watermarkcore:

- 动态库: libwatermarkcore.so
- 头文件: source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkcore/Interface



注意：

使用时需引入对应的库，并 include 相应头文件。

16.4.2 水印资源配置文件

表 16-7 水印配置文件各字段说明

字段	说明
Id	水印唯一标识
Type	水印类型。 ▪ 0：表示单个图标的水印； ▪ 1：表示图标组成的时间水印，格式为2023-05-19 15:50:33； ▪ 2：表示自由组合图标的水印，如组合成含车速、GPS等信息的水印
Width	单个水印图像的宽度

Height	单个水印图像的高度
DefaultX	水印图像叠加的默认起始位置，即：左上角的X坐标
DefaultY	水印图像叠加的默认起始位置，即：左上角的Y坐标
ImageFormat	表示水印图像的像素格式
ImagePath	表示水印图像的路径
ImageFileName	当Type为0时，为单个水印图像的文件名； 当Type为1时，该字段表示多个时间图标的文件名，时间水印图像Icon的顺序必须为[0,1,2,3,4,5,6,7,8,9,;-]，时间水印图像Icon文件名之间以“,”间隔，例如0.yuv, 1.yuv, 2.yuv, 3.yuv, 4.yuv, 5.yuv, 6.yuv, 7.yuv, 8.yuv, 9.yuv, colon.yuv, line.yuv。注意，这些水印图像Icon的图像宽高及像素格式必须与字段Width、Height及ImageFormat中数值一致； 当Type为2时，表示多个图标的文件名。注意，多个文件的图像宽高及像素格式必须完全相同

atcwatermark_config.xml 示例如下：

```
<?xml version="1.0" encoding="UTF-8" ?>
<WatermarkConfig>
  <Common>
    <MergeMode>soft</MergeMode>
  </Common>
  <Watermark>
    <Id>0</Id>
    <Type>2</Type>
    <Width>20</Width>
    <Height>36</Height>
    <DefaultX>0</DefaultX>
    <DefaultY>32</DefaultY>
    <ImageFormat>PNG</ImageFormat>
    <ImagePath>/vendor/etc/watermark</ImagePath>
    <ImageFileName>0.png,1.png,2.png,3.png,4.png,5.png,6.png,7.png,8.png,9.png,degree.
png,minute.png,second.png,E.png,N.png</ImageFileName>
  </Watermark>
  <Watermark>
    <Id>2</Id>
    <Type>2</Type>
```

```
<Width>20</Width>

<Height>36</Height>

<DefaultX>0</DefaultX>

<DefaultY>32</DefaultY>

<ImageFormat>PNG</ImageFormat>

<ImagePath>/vendor/etc/watermark</ImagePath>
<ImageFileName>0.png,1.png,2.png,3.png,4.png,5.png,6.png,7.png,8.png,9.png,degree.
png,minute.png,second.png,E.png,N.png</ImageFileName>

</Watermark>
</WatermarkConfig>
```

16.4.3 Watermarkcore 使用示例

示例代码路径见：

source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkcore/libs/test/test.cpp



注意：

- getMergeInst 接口传入的 ids 需为水印的配置文件中所设定的 ids，具体可通过调用 getAllWatermarkIdList 接口获得；
- 每个 IMergeExcutor 实例只具有叠加一种 pixel format 的能力；
- IMergeExcutor 的 merge 接口传入的图像的 pixel format 需与本实例一致，否则叠加失败。

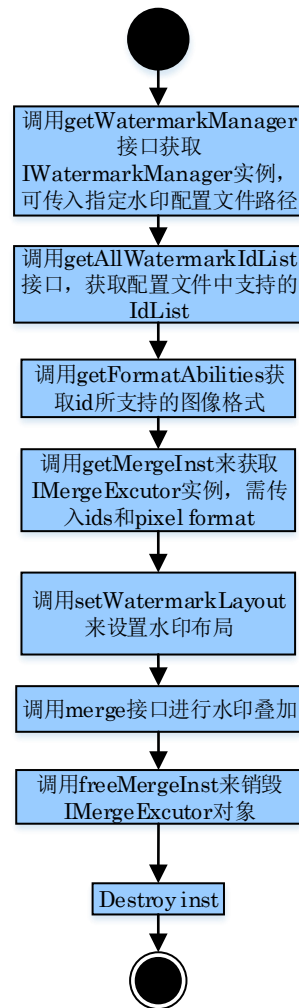


图 16-6 Watermarkcore 示例程序调用流程

16.4.4 WatermarkWrapper 使用示例

示例代码路径见：

source/vendor/autochips/proprietary/hardware/videoin/libs/watermarkwrapper/test/test.cpp

用户通过 getInstance 接口来获取 IWatermarkWrapper 实例，然后传入 videoin_buf_info 类型的图像信息，再调用 merge 接口并传入 startX 和 startY 即可完成水印叠加。



注意：

IWatermarkWrapper 的实例与 pixel format 是一一对应的。每个实例只能叠加水印到与该 format 对应的目标图像上。



图 16-7 WatermarkWrapper 水印叠加效果示例

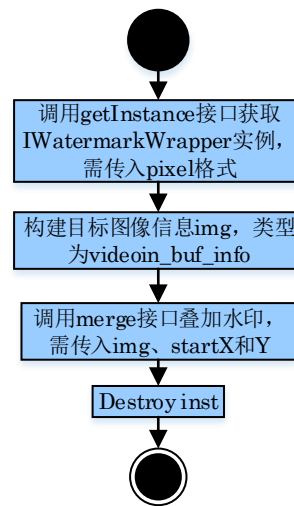


图 16-8 WatermarkWrapper 示例程序调用流程

16.5 用户定制化

16.5.1 概述

当图 16-7 中的含时间、速度和 GPS 信息的水印已经满足需求时，用户可直接调用 WatermarkWrapper 的相关接口，并可参照 16.5.2 和 16.5.3，通过更换配置文件中的水印素材来达到字体大小、颜色和样式更换。定制化的本质是组合任意图片素材进行任意叠加，直至达到预期效果。

若 WatermarkWrapper 中 GPS 等固定布局的水印不满足需求时，用户也可参照 16.5.3，调用 WatermarkWrapper 相关接口进行水印叠加。为了简化调用过程，也可参照 watermarkwrapper 对其进行封装，使得对外接口更简单易用。

16.5.2 水印的起始点

目前用户可以通过在水印配置文件中设置默认起始点来控制叠加位置，也可以通过 16.3.2.2.5 中的 setWatermarkPosition 接口来设置。

16.5.3 水印的内容

水印的内容定制化，可以通过修改水印的配置文件，添加不同的水印素材组合来达到目的。

需注意的是，同一个 id 下的水印素材叠加的 Y 坐标是相同的，X 坐标是按照 layout 的布局从左到右依次叠加的。用户可以把自己想要叠加在同一行的素材用同一个 id 来组织。

例如：2023-05-19 15:50:33 是时间水印，显示在同一行。用户可以把该时间水印需要的素材放在同一 id 下。（不同 id 可包含重复的素材）。

16.5.3.1 车速水印实现

本节举例说明如何调用 libwatermarkcore 实现车速水印。

在 xml 中添加如下配置：

```
<Watermark>
    <Id>3</Id>
    <Type>0</Type>
    <Width>108</Width>
    <Height>36</Height>
    <DefaultX>500</DefaultX>
    <DefaultY>0</DefaultY>
    <ImageFormat>NV12</ImageFormat>
    <ImagePath>/data/vendor/avm/other/</ImagePath>
    <ImageFileName>km.yuv</ImageFileName>
</Watermark>
<Watermark>
    <Id>4</Id>
    <Type>2</Type>
    <Width>16</Width>
    <Height>32</Height>
    <DefaultX>510</DefaultX>
    <DefaultY>0</DefaultY>
    <ImageFormat>NV12</ImageFormat>
    <ImagePath>/data/vendor/avm/speed/</ImagePath>
    <ImageFileName>0.yuv,1.yuv,2.yuv,3.yuv,4.yuv,5.yuv,6.yuv,7.yuv,8.yuv,
```

```
9.yuv, -.yuv</ImageFileName>
```

```
</Watermark>
```

打开上述两个 id 的水印。其中，id 为 3 的水印 Type 为 0，其水印图像为为“km/h”的字样，以及整个显示车速区域的背景；id 为 4 的水印 Type 为 2，其水印图像为为组成车速的数字，通过组合这些数字图像就可以组成车速数值图像。

实际使用时，以下列步骤完成车速水印的实现：

- 1) 在 xml 的 WatermarkConfig 字段中添加上述两个 Watermark 相关配置；
- 2) 将相应的水印图像文件放入 xml 中配置的路径中；
- 3) 调用 IWatermarkManager::getMergeInst()，并传入 id3 与 4；
- 4) 依次调用 IMergeExcutor::config，IMergeExcutor::SetWatermarkPosition,将 id 为 3 和 4 的水印打开，并设置两个水印的位置，将 id 为 4 的车速数值水印覆盖到 id 为 3 的车速单位水印之上；
- 5) 调用者获取当前车速，并对 id 为 4 的水印调用 IMergeExcutor::setLayout，如当前车速为 120km/h，则 layout 为{1,2,0}；如当前车速为 8km/h，则 layout 为{0,0,8}，以此类推；
- 6) 调用 IMergeExcutor::merge，完成水印的叠加，效果如下图所示。



图 16-9 车速水印效果



注意：

- 同一个 id 下的水印素材大小，需与 Width 和 Height 一致；
- 同一个 id 下的水印素材格式，需与 ImageFormat 一致；
- 同一个 id 下的水印素材只能叠加在同一行，因为 setWatermarkPosition 接口是按照 id 来进行设置的。

16.6 典型问题

16.6.1 水印显示异常

【问题描述】

水印显示不合预期，应如何处理？

【解决方案】

针对水印叠加异常，提供开放 log 和 dump merge 前后图像的方式来定位 Bug。

使用步骤介绍如下。

1. 创建 watermark_log.cmd 文件，用来调整开放的 log 等级

包含两个字段，示例如下：

```
LOG_LEVEL 5
```

表 16-8 示例 watermark_log.cmd 各字段含义

字段	含义
LOG_LEVEL	固定标识
10 进制数值	0: 表示只开放 error 等级 log 1: 表示开放 warning 和 error 等级 log 2: 表示开放 info、warning 和 error 等级 log 4: 表示开放 debug 及以上等级的 log 5: 输出全部 log

2. 创建 watermark_dump.cmd 文件，用来指定 dump 图像命令

示例及各字段解释如下：

```
CAM_DUMP 2 /data/vendor/camera_dump
0 1 10
```

表 16-9 示例 watermark_dump.cmd 各字段含义

字段	含义
CAM_DUMP	命令标识
2	dump id，可任意数字，用于区分多次 dump
/data/vendor/camera_dump	指定 dump 后图片存放的路径
0	接口标识，0 表示 dump 的为 one-param 的 merge 接口，1 表示 dump 的为 multi-params 的 merge 接口
1	暂未设定含义，数值任意
10	指定 dump 的帧数

17 Audio

17.1 Audio 简介

本章介绍 AC8025 Audio 模块。从该模块支持的功能上看，支持灵活的上下行通路配置、支持为更高识别率而定制的语音识别通路、支持 Android 系统未启动之前快速出声、支持灵活的 FM 接入等等功能；从该模块对音频数据的处理来看，结合内置 HIFI3 DSP，支持常见 EQ、Reverb 等多种音效处理，支持降噪消回音处理，支持 VolumePolicy 处理等；从该模块对硬件的支持情况看，支持 I2S、PCM 等常见音频接口。

AC8025 Android 系统融合了对 HIFI3 DSP 的高度模块化集成，对车机上的多音源同时输出、语音识别等常见场景提供了客户定制化方案，提供了支持 Carplay 认证的降噪消回音和时延通路解决方案，支持第三方音效如 CSAUTO 的集成等。结合上述功能说明，AC8025 Android 系统 Audio 模块为车机的音频环境提供了完善的需求解决方案。

下面几个章节将从通路、音量、音效、蓝牙/Carplay 电话、FM/Auxin、语音识别、Vehicle 快速出声、debug 几大方面一一进行介绍具体功能和使用方法。Audio 模块暂不涉及编解码任何介绍，编解码相关请详见 22 Multimedia 章节。

17.1.1 内容介绍

- 通路部分：
 - 框架介绍；
 - Framework 音频分离，此处的音频分离指的是控制某些音频数据可混音，便于后续将混音后的数据控制某些喇叭输出，形成音区的效果；
 - ALSA device 节点动态配置，结合 framework 层音频分离，此处通过动态配置输出的 ALSA device 节点，灵活适配硬件输出环境；
 - 输出喇叭映射配置，可通过配置输出数据的声道映射，灵活适配硬件接线环境。
- 音量部分：
 - 音量设置接口；
 - 音量交互实现，重点介绍 ATC VolumePolicy 方案；
 - 典型问题介绍。
- 音效部分：
 - 框架介绍；

- Java 接口，基于 HIDL 接口，提供 APP 直接调用的 Java 接口，包含更简便的基于音效 feature 的接口，详细请查阅对应章节；
- kernel 接口，支持 tinymix、CLI debug 方式。
- 蓝牙/Carplay 电话部分：
 - 框架实现等介绍；
 - AP ECNR，即降噪消回音在 AP 端实现；
 - DSP ECNR，即降噪消回音在 DSP 端实现，默认采用第三方提供的算法；由于 DSP code 不公开 release，该部分使用请先咨询 ATC 商务。
- FM/AuxIn 部分：
 - direct/indirect 模式介绍，两种模式都能够实现外部音频信号在车机端直接输出，区别主要在通路上；
 - 配置方法，说明两种模式的配置方式，包括 APP 的使用、输入源的配置等；
- 语音识别 support 部分：
 - DSP 四声道介绍，DSP 将 MIC 和 REF 数据进行对齐拼接，上层获取拼接好的数据供语音识别模块使用。
- FastAudio 部分：
 - 音频文件设计，可以自定义开机音频、倒车音频等；
 - 音量设置，支持开机过程中的多音源输出，支持音源的优先级定义和音量的自适应等功能。
- 通用 debug 方法等。

17.1.2 软硬件环境

17.1.2.1 软件环境

Audio 模块整体基于 Android 的分层软件模型。Userspace 基于 HIDL 架构，Audio framework 保持 Android 原生音频框架，kernel 内核版本为 kernel-5.10，驱动代码遵循 ALSA Soc 架构。

17.1.2.2 硬件环境

基于 2 颗内置 HIFI3 DSP（支持浮点操作、最高 640MCPS）的 AC8025 芯片，一颗 DSP 用于音效和解码，另一颗用于 ECNR，外围搭配 2 组 PCM、4 组 I2S SC、1 组 I2S MC 进行开发。

17.2 通路部分

本章主要描述 AC8025 Android 系统的 Audio 通路，包含输入和输出通路。AC8025 公版默认配置 CarAudio，即支持多音区（Mutil-Zone）（各音区（Audio Zone）的音量、AudioFocus 均独立使用，音频通路也是独立使用）。如下图中的中控和副驾驶区域（可理解为中控音区和副驾驶音区），中控若在全车喇叭输出，副驾驶外接一个小喇叭或者有线耳机（采用 I2S 进行对接），实现中控在导航和多媒体播放时，副驾驶播放三方视频，二者调节音量互不影响的场景。当配置为 PhoneAudio 时，一般不支持多音区，额外需求需要定制化开发。CarAudio 的配置参考[如何配置和查看 CarAudio 还是 PhoneAudio](#) 章节。

在输出通路上，PhoneAudio 配置时，ATC 通过将 AudioTrack 的 StreamType 和输出线程 Flag 进行多对一映射，在 Framework 层达到音频分离的效果，该音频分离可通过配置 audioCustomConfig.xml 自定义映射方式；基于 CarAudio 配置时，采用 Android 原生的 usage 与 address 匹配的方式达到音频分离的效果，配置文件为原生 car_audio_configuration.xml。另外，通过自定义输出线程与 ALSA device 节点的一对一映射关系，实现灵活的输出配置，配置文件为 audioCustomConfig.xml。CarAudio 和 PhoneAudio 配置时音频通路分离方式如下示意图描述。

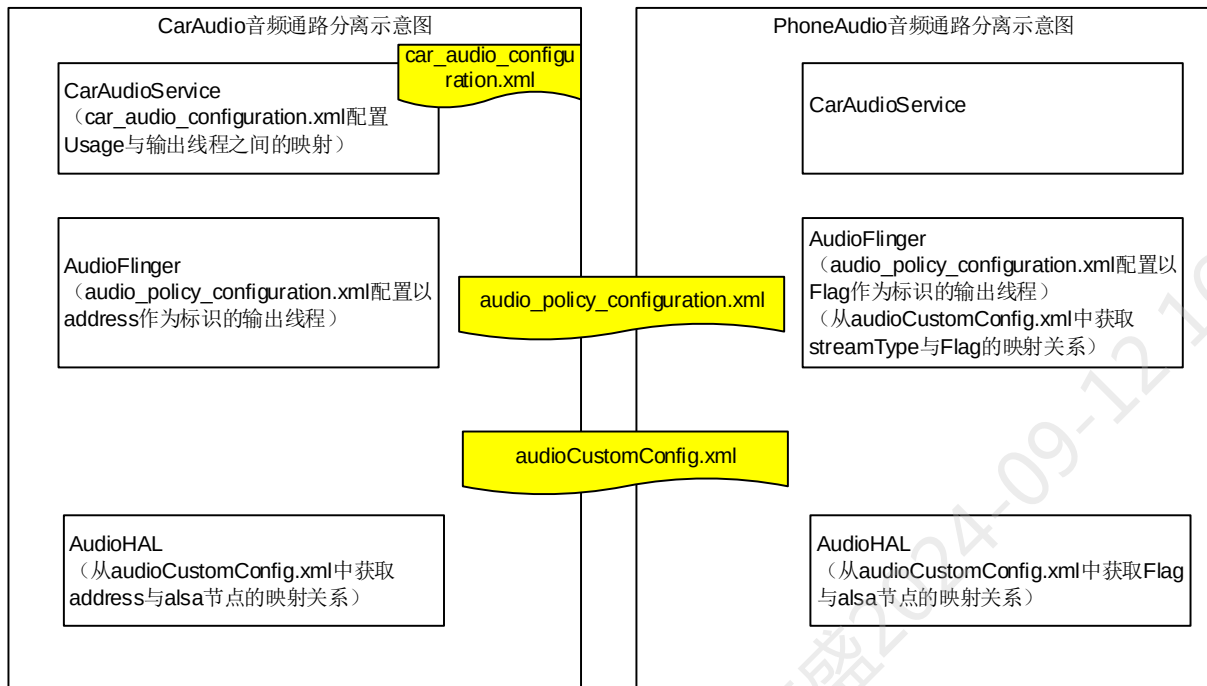


图 17-1 CarAudio VS PhoneAudio 音频通路分离简化图

在输入通路上，ATC 通过将 HAL 层录制时的 source 和 ALSA device 进行可选择性的一对一映射，实现依据硬件设计和需求的输入配置。目前的功能实现包含 FM/Auxin 输入配置，MIC 配置等，后文 FM/Auxin 章节详细介绍 FM/Auxin 输入配置。

17.2.1 框架

17.2.1.1 系统框图

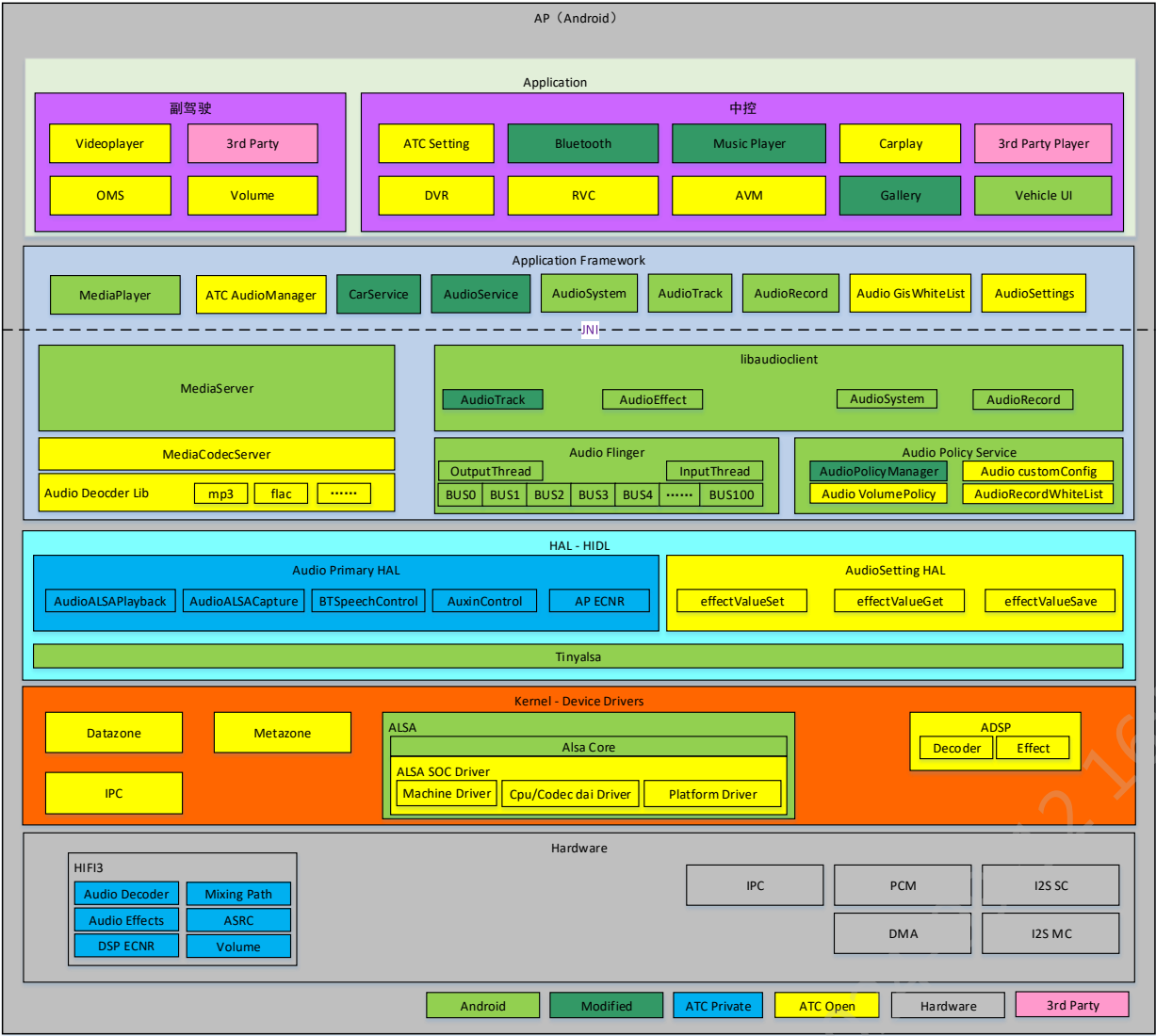


图 17-2 Audio 系统框图

17.2.1.2 音频数据流介绍

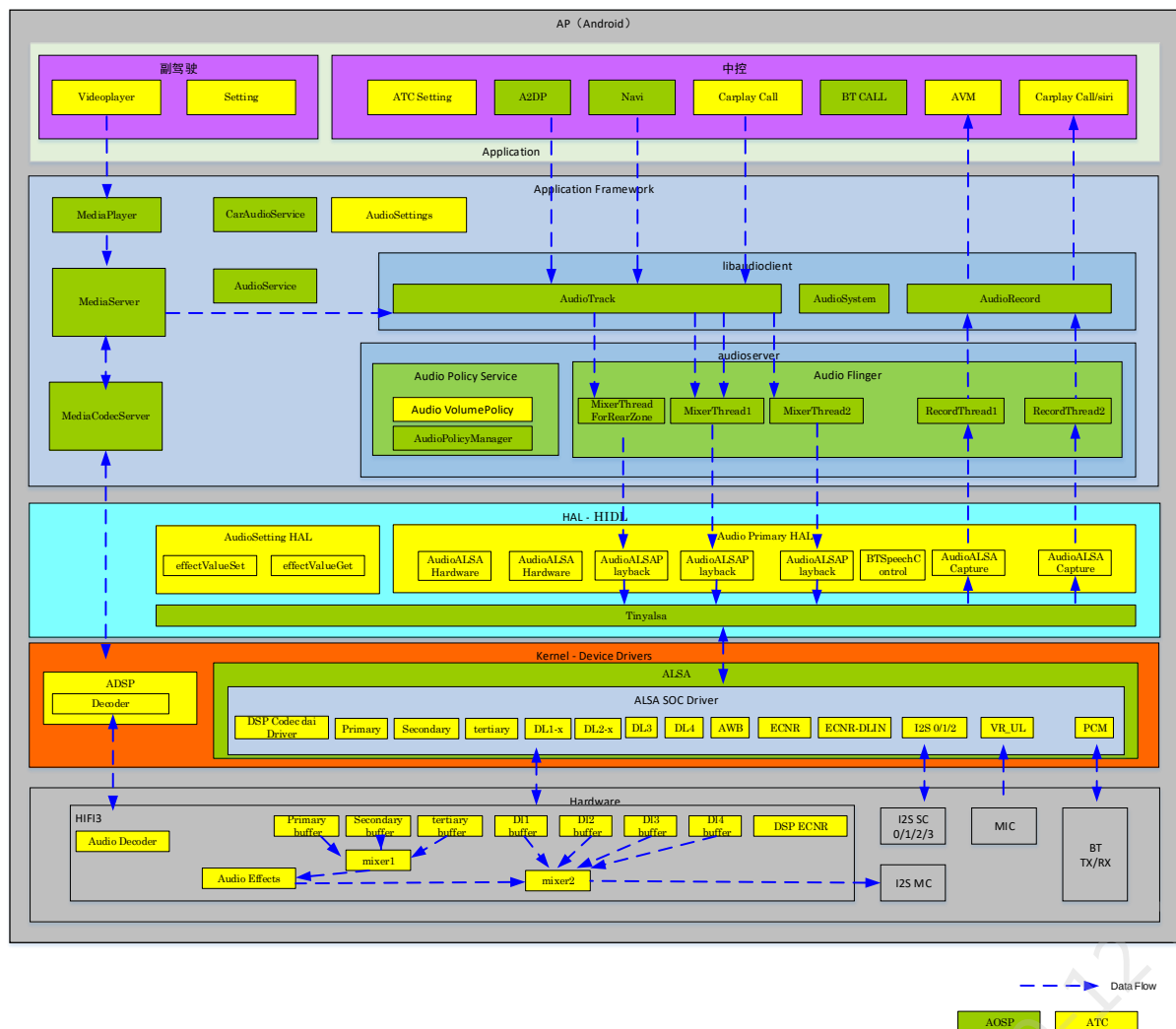


图 17-3 Audio 数据流图

本小节针对采用 DSP 输出的 path 进行说明，DSP 中共 7 条下行 path，分别为 primary、second、Tertiary、DL1（与 alsa 的 DL1-x 对应）、DL2（与 alsa 的 DL2-x 对应）、DL3、DL4，其中前三条 path 的数据在混音后会做音效处理，因此这三条 path 常用作媒体类播放，后四条 path 会直接与过完音效的媒体类音频数据进行混音操作，之后将混音后的数据送往 I2S MC 进行输出播放。Framework 层的音频通路如何与 ALSA 层的节点关联，由 audioCustomConfig.xml 文件进行配置，可查看 [ALSA device 节点配置](#) 章节。

不论 CarAudio 还是 PhoneAudio，当 APP 播放音频数据时，其通用数据流为：AudioTrack——AudioFlinger（MixerThread）——Audio HAL（AudioALSAStrout——AudioALSAPlaybackHandler）——Driver——DSP。在车机场景中，一般设置四个喇叭，对于车主较为关注的如导航、车机 Beep 音、电话类声音一般从前左前右喇叭播放，而媒体类音频一般在全部喇叭中播放。针对实际存在的上述需求场景，ATC 设计了 Framework 层音频分离和 ALSA device 节点

配置方案，可从音源选择开始定制化车机的音频输出方案。另外，针对声道输出喇叭定制化，ATC 也可修改极少 code 达成目的，Framework 音频分离小节分别进行说明。

17.2.1.3 DSP 内部输出 path 框图

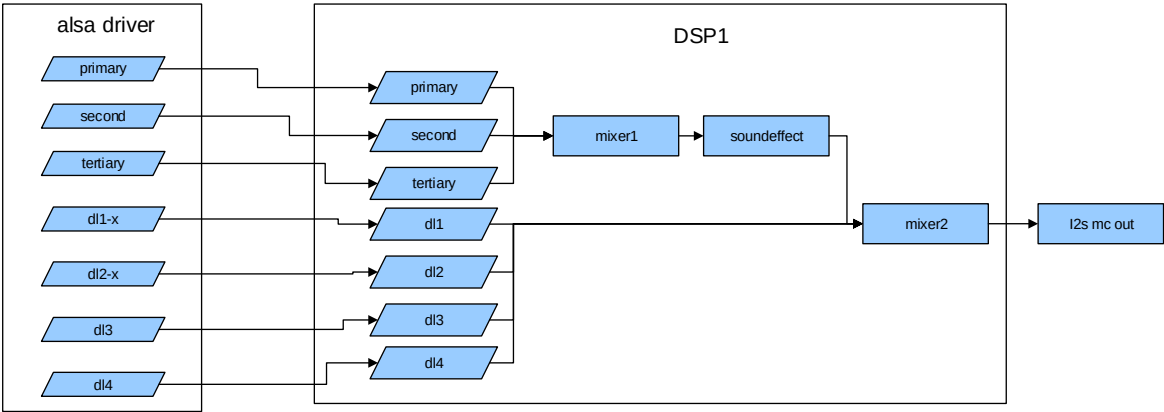


图 17-4 DSP 输出框图

17.2.1.4 I2S 模块图

详见模块图章节。

17.2.2 文件描述

表 17-1 配置文件描述

文件名	代码路径	平台路径	文件描述
audio_policy_configuration.xml	source/android/device/autochips/ vendor/ac8x/common/audio	/vendor/etc	Android 原生 Audio 配置文件
car_audio_configuration.xml	source/android/device/autochips/ vendor/ac8x/common/audio/caraudio	/vendor/etc	Android CarAudio 配置下，Car Audio 配置文件
audioCustomConfig.xml	source/android/device/autochips/ vendor/ac8025_demo/audio	/vendor/etc	ATC 音频分离和 alsa 节点配置文件

17.2.3 Framework 音频分离

Framework 音频分离，此处的音频分离指的是控制某些音频数据可混音，便于后续将混音后的数据控制某些喇叭输出，形成音区的效果。

17.2.3.1 车机环境多喇叭场景需求

下图为导航和音乐同时播放时的设计播出示例。图中 driver 为车主位置，导航声音可以控制只在前左、前右喇叭出声，Music 声音在全车四个喇叭出声。

GIS+Music (driver)	GIS+Music
Music	Music

图 17-5 多喇叭输出示例

17.2.3.2 音频分离实现方式

根据 ATC_CARAUDIO 的配置，当 ATC_CARAUDIO 配置为 yes 时，表示当前系统支持 Car Audio，支持多 Audio Zone；当 ATC_CARAUDIO 配置为 no 时，表示当前系统支持 Phone Audio，即只支持单屏 Audio。

17.2.3.2.1 Car Audio

AC8025 公版默认支持 Car Audio，在音频分离实现上，依赖 car_audio_configuration.xml 和 audio_policy_configuration.xml 两个配置文件。

car_audio_configuration.xml 配置 usage（即 APP 播放 AudioTrack 时，设置的 AudioAttribute 中的 usage）与 device（AUDIO_DEVICE_OUT_BUS 类型的 device，Car Audio 的输出逻辑 device，以 address 作为区分）的映射关系。

audio_policy_configuration.xml 依然是由 AudioPolicyService 读取，用来配置 AudioFlinger 中输出输入线程，在 CarAudio 中，输出逻辑 device 变为 AUDIO_DEVICE_OUT_BUS，各个输出线程一一路由到不同 address 的 device，即可理解为 AudioFlinger 的输出线程以 address 作为区分标识。结合两个 xml 的配置，即可将某 usage 配置到某个 AudioFlinger 的输出线程中。

<!-- mixPort 表示 AudioFlinger 中的一个输出线程 -->

```
<mixPort name="mixport_bus0_media_out" role="source"
flags="AUDIO_OUTPUT_FLAG_PRIMARY">
```

```

<profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
samplingRates="96000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>

</mixPort>

...

...

<!-- devicePort 表示一个逻辑的 device-->

<devicePort tagName="bus0_media_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
address="bus0_media_out">

<profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="48000"
channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>

    <gains>

        <gain name="" mode="AUDIO_GAIN_MODE_JOINT" minValueMB="-5000"
maxValueMB="0" defaultValueMB="-1350" stepValueMB="50"/>

    </gains>

</devicePort>

...

...

<!-- route 表示 mixPort 与 devicePort 之间的连接关系，即数据由 source 流向 sink，这里
source 中包含 name 为 mixport_bus0_media_out 的输出线程，sink 为 address 为
bus0_media_out 的 device，即可看做 AudioFlinger 的 mixport_bus0_media_out 输出线程与
address 为 bus0_media_out 的 device 绑定，即一条音频通路-->

<!-- AUX Digital In,AUX Line In,mmap_no_irq_out 是 createAudioPatch 和 Aaudio 使用，
此处不做关注 -->

<route type="mix" sink="bus0_media_out" sources="mixport_bus0_media_out,AUX
Digital In,AUX Line In,mmap_no_irq_out"/>

```

如上是 audio_policy_configuration.xml 部分内容，devicePort 中的增加了 address 字段，该字段用于唯一区分一个 AUDIO_DEVICE_OUT_BUS 类型的 device。而每个 device 与 mixPort（用于表示 AudioFlinger 的输出线程）存在一一对应的映射关系，即通过不同 address 可以识别不同的通路。

```

<carAudioConfiguration version="2">

    <zones>

        <zone name="primary zone" isPrimary="true">

            <volumeGroups>

                <group>

                    <device address="bus0_media_out">

```

```

        <context context="music"/>
    </device>
    <device address="bus0_smix_out">
        <context context="smix"/>
    </device>
</group>
<group>
    <device address="bus3_call_ring_out">
        <context context="call_ring"/>
        <context context="notification"/>
        <context context="alarm"/>
    </device>
</group>
<group>
    <device address="bus1_navigation_out">
        <context context="navigation"/>
    </device>
</group>
<group>
    <device address="bus2_voice_command_out">
        <context context="voice_command"/>
        <context context="system_sound"/>
        <context context="emergency"/>
        <context context="safety"/>
        <context context="vehicle_status"/>
        <context context="announcement"/>
    </device>
</group>
<group>
    <device address="bus4_call_out">
        <context context="call"/>

```



```

        </device>
    </group>
    <group>
        <device address="bus5_alternate_out">
            <context context="alternate"/>
        </device>
    </group>
</volumeGroups>
</zone>
</zones>
</carAudioConfiguration>

```

如上为 car_audio_configuration.xml 的示例，其中：

- “zone” TAG 表示音区；
- “group” TAG 表示音量组（位于同一个音量组的 device，其音量相同）；
- “device” TAG 表示 AUDIO_DEVICE_OUT_BUS 类型的 device，用 address 进行区分，该 address 需要与 audio_policy_configuration.xml 中的 address 字段一致；
- “context” TAG 表示 usage 概念，即表示播放的声音是什么内容，什么含义。

按照上述的配置，music 类的声音，将从 address 为“bus0_media_out”的 device 输出。navigation 类的声音，将从 address 为“bus1_navigation_out”的 device 输出，两个 device 在 AudioFlinger 中是两个输出线程，这样在 Framework 层，达到将 music 类的声音和 navigation 类的声音通路分离的目标。下图为 CarAudio 音频通路分离的实现流程。

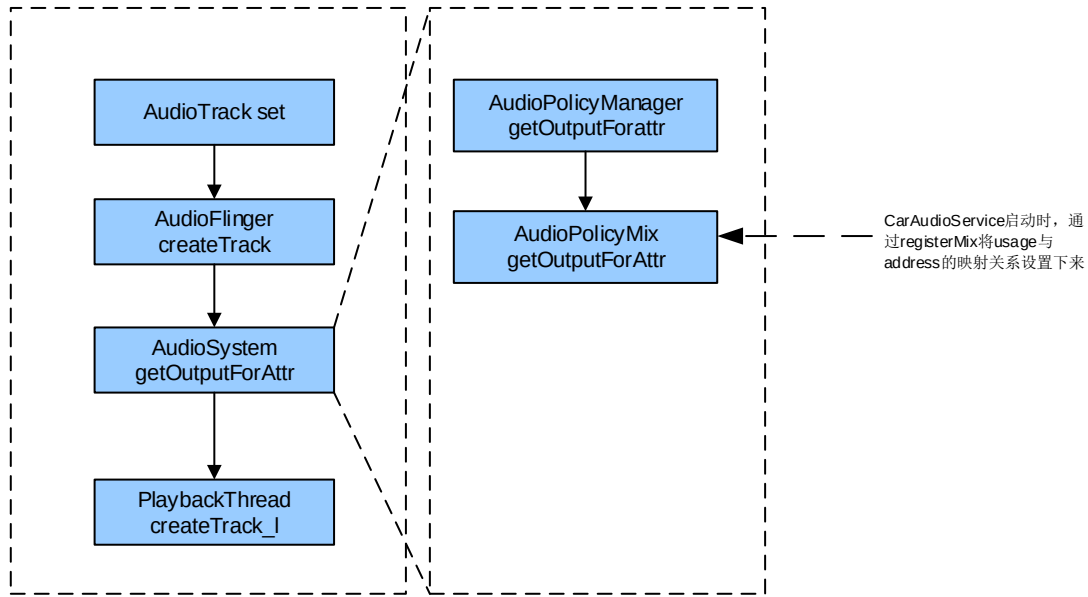


图 17-6 CarAudio 音频通路分离实现流程

17.2.3.2.2 Phone Audio

ATC 以 Android 原生的音频框架为基础，从 AudioTrack 的创建流程入手，设计了基于 streamtype 和 MixerThread 的 Flag 映射方式实现音频分离。下图为 AudioTrack 的创建流程图，在 AudioTrack 选择 output 时依据 Audio Attribute（与 streamtype 含义类似，都是用作标识声音的）来选定，在原有 getOutputForDevice 流程的基础上，新增 AudioCustomConfig 模块，该部分直接根据 audioCustomConfig.xml 中的配置来进行 streamType 与 Flag 的映射。而 audioserver 进程在启动后，根据 audio_policy_configuration.xml 中的配置信息，已经创建对应的 MixerThread，并且每个 MixerThread 采用特定的 Flag 进行标识。

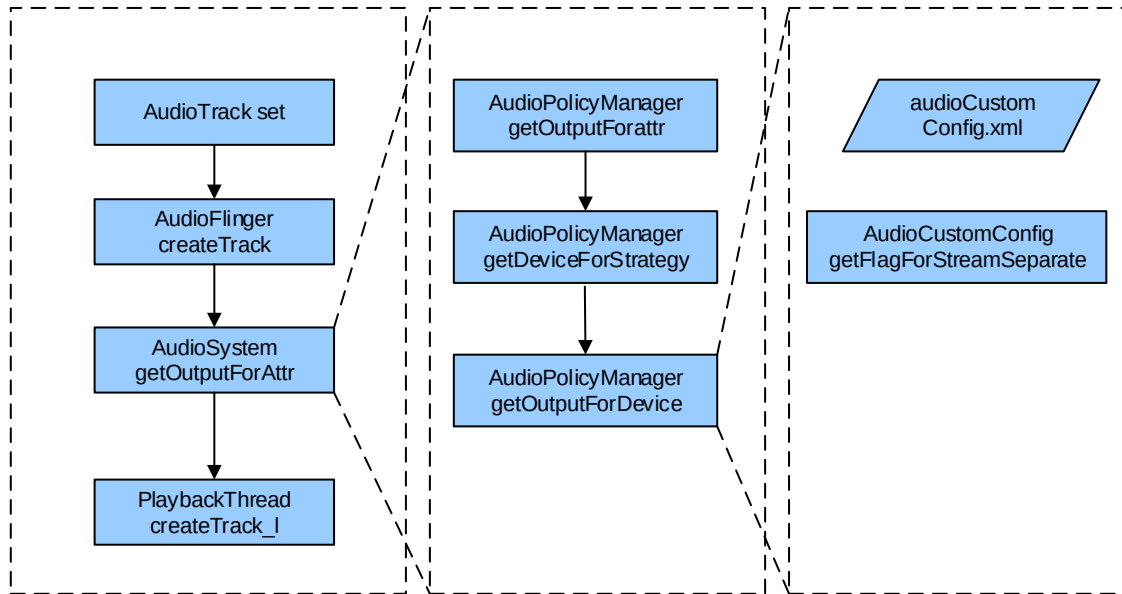


图 17-7 音频分离实现流程图

下面为 AC8025 demo 板 audio_policy_configuration.xml 中对输出线程的配置，从 mixport 可以看到，每个 mixport 有设置 flag、profile 等信息，如 AUDIO_OUTPUT_FLAG_TO_ADSP 标识该混音输出音频数据为媒体类型，需要进行音效处理；AUDIO_OUTPUT_FLAG_BEEP 标识该混音输出音频数据一般为车机 BEEP 音；AUDIO_OUTPUT_FLAG_QUICK 标识该混音输出音频数据为电话声音，如 carplay 电话、Hicar 电话等。同时也可以注意到有两个 mixport 的 flag 类型均为 AUDIO_OUTPUT_FLAG_TO_ADSP，此处是 AC8025 为支持 5.1 声道数据混音所做的处理，两个 mixport 的 profile 支持是有声道区别的。

```

<mixPort name="primary output" role="source"
flags="AUDIO_OUTPUT_FLAG_PRIMARY|AUDIO_OUTPUT_FLAG_FAST">
    <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="44100,48000"
channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
</mixPort>

<mixPort name="softmix output1" role="source" flags="AUDIO_OUTPUT_FLAG_TO_ADSP">
    <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="44100,48000"
channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
</mixPort>

<mixPort name="softmix output2" role="source" flags="AUDIO_OUTPUT_FLAG_TO_ADSP">
    <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="44100,48000"
channelMasks="AUDIO_CHANNEL_OUT_5POINT1"/>
</mixPort>

<mixPort name="softmix output3" role="source" flags="AUDIO_OUTPUT_FLAG_BEEP">
    <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="44100,48000"
channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>

```

```
</mixPort>

<mixPort name="softmix output4" role="source" flags="AUDIO_OUTPUT_FLAG_QUICK">
    <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="44100,48000"
channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
</mixPort>
```

17.2.3.3 音频分离配置方式

17.2.3.3.1 Car Audio

Car Audio 下音频分离在系统下的配置文件为 car_audio_configuration.xml，只需将不同 Usage 对应的 context 配置到 device tag 中，具体 Usage 与 context 的对应关系见 CarAudioContext.java 中的 CONTEXT_TO_USAGES 和 USAGES_TO_CONTEXT 这 2 个映射表。

若想增加或者减少 AudioFlinger 的输出线程，联合 audio_policy_configuration.xml 一起修改即可，即 audio_policy_configuration.xml 中确定输出线程个数和 address 名称，car_audio_configuration.xml 中确定 address 与 usage 之间的映射关系。

因音频适配更多的与硬件相关，在与公版项目差别较大时，可直接联系 ATC 进行沟通。

17.2.3.3.2 Phone Audio

Phone Audio 下音频分离配置仅需在 audioCustomConfig.xml 配置文件中“stream_separate”块标识下修改即可，该文件在项目中的位置为：/device/autochips/ac8x/，打包进 image 后，该文件位于：vendor/etc/。该文件修改后重启平台即可生效。下面介绍如何通过修改 xml 配置音频分离。

下面的示例是 AC8025 中默认的音频分离配置，通过“flag_to_adsp”标识可配置某一 StreamType 的音频数据需要混音到 Flag 为 AUDIO_OUTPUT_FLAG_TO_ADSP 的输出线程中，后续经 DSP 做音效处理；同理，标识“flag_beep”、“flag_quick”分别对应与其名称对应的 Flag，默认未配置的 StreamType 将从 Flag 为 primary 的输出线程混音输出。在配置文件中，StreamType 采用 int 数值标识，其值需要与当前系统中设置的 StreamType 对应，下表为常见 StreamType 与 int 值的对应表格，也可直接查看 code 中的 audio-base.h 文件。客户在自定义音频分离配置时，注意两点即可：

- flag 标识无法在 xml 中新增，默认只支持“flag_to_adsp”、“flag_beep”、“flag_quick”；
- flag 标识与 int 值是一对一形式，不支持一对多形式。

```
<stream_separate>
    <flag_to_adsp>3</flag_to_adsp>
    <flag_to_adsp>16</flag_to_adsp>
    <flag_beep>14</flag_beep>
    <flag_quick>0</flag_quick>
```

</stream_separate>

表 17-2 streamType 和 Int 值映射关系表

StreamType	int值
AUDIO_STREAM_VOICE_CALL	0
AUDIO_STREAM_SYSTEM	1
AUDIO_STREAM_RING	2
AUDIO_STREAM_MUSIC	3
AUDIO_STREAM_ALARM	4
AUDIO_STREAM_NOTIFICATION	5
AUDIO_STREAM_GIS	12

17.2.3.4 音频分离 debug 方法

Phone Audio 配置下，当配置好需要分离的 StreamType 后，可以通过如下步骤 check 是否分离成功：

- 使用 APP 播放需要 check 的 StreamType；
- 使用 adb shell dumpsys media.audio_flinger 命令，看对应的 StreamType 是否在对应 Flag 的线程中有对应的 Track，如下图示例，StreamType 3 即 AUDIO_STREAM_MUSIC 的数据是在 AUDIO_OUTPUT_FLAG_TO_ADSP 这个 Flag 对应的线程中进行 Mix 的。若存在，即配置成功，若不存在，请继续下一步；
- 查看 vendor/etc/audioCustomConfig.xml 文件中的配置是否满足要求；
- 若配置 xml 文件没有错误，音频正常播放，可实际运行时依旧没有按照配置进行分离，请提供完整的 log（atclog 即可）和 audioCustomConfig.xml 文件给到 ATC。

```
Effect buffer: 0xef148000
Fast track availMask=0xfe
Standby delay ns=3000000000
AudioStreamOut: 0xef949f10 flags 0x20000 <AUDIO_OUTPUT_FLAG_TO_ADSP>
Frames written: 389120
Suspended frames: 0
Hal stream dump:
Signal power history:
01-01 00:01:20.008: [-103.2 -102.4 -102.4 -102.3 -99.7 -100.7 -99.7 -99.8 -98.8 -98.7
01-01 00:01:20.370: -97.7 -97.0 -97.0 -80.2 -74.3 -70.2 -67.2 -66.8 -67.1 -67.3
01-01 00:01:20.854: -67.5 -67.4 -67.5 -67.3 -67.9 -67.4 -67.9 -68.0 -68.2 -67.6
01-01 00:01:21.362: -66.7 -67.2 -68.0 -67.3 -67.2 -67.4 -67.3 -67.3 -66.9 -67.4
01-01 00:01:21.858: -66.9 -66.9 -67.2 -67.5 -67.1 -67.2 -67.3 -66.9 -67.1 -68.0
01-01 00:01:22.342: -68.0 -67.5 -67.4 -67.5 -67.1 -67.7 -68.1 -68.2 -67.7 -67.9
01-01 00:01:22.858: -67.7 -68.1 -67.7 -67.7 -68.2 -67.9 -67.7 -66.9 -67.2 -67.6
01-01 00:01:23.358: -67.9 -67.0 -67.4 -67.7 -67.8 -68.1 -68.6 -68.3 -67.9 -67.6
01-01 00:01:23.870: -67.2 -67.6 -66.7 -66.5 -66.6 -66.4 -65.7 -65.3 -65.5 -66.2
01-01 00:01:24.350: -66.4 -66.9 -66.5 -67.0 -67.4 -67.5 -66.8 -66.5 -66.7 -66.4
01-01 00:01:24.866: -64.7 -64.6 -65.1 -66.0 -65.9 -66.6 -67.5 -66.8 -67.5 -67.2
01-01 00:01:25.350: -67.5 -67.6 -67.4 -67.8 -67.0 -67.8 -67.1 -67.6 -67.4 -67.2
01-01 00:01:25.846: -67.2 -67.1 -67.2 -66.5 -67.4 -67.0 -67.1 -66.4 -68.5 -67.1
01-01 00:01:26.358: -67.4 -66.7 -66.9 -67.2 -67.1 -67.7 -66.7 -67.7 -67.0 -66.7
01-01 00:01:26.854: -37.1 -35.5 -36.2 -37.5 -38.4 -39.5 -40.7 -35.2 -33.6 -34.3
01-01 00:01:27.366: -35.4 -36.1 -36.3 -37.0 -31.8 -25.4 -26.3 -28.1 -30.3 -31.9
Thread throttle time (msecs): 178
AudioMixer tracks: 0
Master mono: off
No FastMixer
Stream volumes in dB: 0:-4.7, 1:-11, 2:-8.1, 3:-11, 4:-6, 5:-8.1, 6:-2.8, 7:-11, 8:-11, 9:0, 10:-1
Normal mixer raw underrun counters: partial=0 empty=0
1 Tracks of which 1 are active
I Name Active Client Session S Flags Format Chn mask SRate ST L dB R dB US dB Server P
0 yes 3319 17 A 0x000 00000001 00000003 44100 3 0 0 0 00057480
```

图 17-8 debug 音频分离

17.2.4 ALSA device 节点配置

本小节介绍 ALSA device 节点的配置方式。即 HAL 层输入输出通道与 ALSA 输入输出 device 节点之间的一对一关系，一般作用于客户定制化通路需求中，如经 I2S 外接 DSP 输出、经 I2S 输入 FM 信号、MIC 信号经 I2S TDM 输入等。

17.2.4.1 功能说明

为尽量清晰描述 ALSA device 节点配置的功能，将其划分为输入和输出两部分。输入部分，从常用场景上看，分为 MIC、FM/Auxin 这两种，其中 MIC 配置为 I2S 接入方式，该部分实现及扩展描述请参阅 [FM/AuxIn](#) 部分和 [Audio IO](#)。输出部分，从常见的适配方案上区分，分为使用内置 DSP 和不使用内置 DSP 两种，使用内置 DSP 时，DL1、DL2、DL3、DL4 节点以及 I2S 节点可以灵活对接 AudioFlinger 的输出通路，不使用内置 DSP 时，也可通过映射方式将 AudioFlinger 的输出通路灵活对接至 I2S 节点。该部分描述的配置功能依赖具体的硬件框架设计和需求整理，基于 xml 的配置方式可减少 code 修改，便于调试和适配。由于该部分一对一选择部分的逻辑位于 Audio HAL 层，该 code 是 ATC 保护 code，暂不详述其实现方式。

17.2.4.2 ALSA 节点介绍

根据目前 AC8025 已有的 PCM 节点，分别对每个 PCM 节点做简要介绍。

表 17-3 ALSA 节点介绍

pcm节点名称	含义
machine-stream-dsp-primary	输出通路，对接 DSP 音效 path，分别对应 DSP 内部输出 path 框图中的 primary、secondary、tertiary
machine-stream-dsp-secondary	
machine-stream-dsp-tertiary	
machine-stream-dsp-dl-1-1	输出通路，对接 DSP 非音效 path，所有的 1-x 通路在 alsa 进行混音，混音后输出到 DSP，对应 DSP 内部输出 path 框图中的 dl1。 每条 1-x 通路在 alsa 有单独的音量 Kcontrol、声道配置 Kcontrol 接口可供设置。
machine-stream-dsp-dl-1-2	
machine-stream-dsp-dl-1-3	
machine-stream-dsp-dl-1-4	
machine-stream-dsp-dl-1-5	
machine-stream-dsp-dl-1-6	
machine-stream-dsp-dl-1-7	
machine-stream-dsp-dl-1-8	
machine-stream-dsp-dl-2-1	输出通路，对接 DSP 非音效 path，所有的 2-x 通路在 alsa 进行混音，混音后输出到 DSP，对应 DSP 内部输出 path 框图中的 dl2。 每条 2-x 通路在 alsa 有单独的音量 Kcontrol、声道配置 Kcontrol 接口可供设置。
machine-stream-dsp-dl-2-2	
machine-stream-dsp-dl-2-3	
machine-stream-dsp-dl-2-4	
machine-stream-dsp-dl-3	输出通路，对接 DSP 非音效 path，对应 DSP 内部输出 path 框图中的 dl3。
machine-stream-dsp-dl-4	输出通路，对接 DSP 非音效 path，对应 DSP 内部输出 path 框图中的 dl4。
machine-stream-awb	参考音数据输入通路
machine-stream-i2s-sc0	对接 I2S0 的输入输出通路
machine-stream-i2s-sc1	对接 I2S1 的输入输出通路
machine-stream-i2s-sc2	对接 I2S2 的输入输出通路
machine-stream-i2s-sc3	对接 I2S3 的输入输出通路
machine-stream-pcm	对接 PCM0 的输入输出通路
machine-stream-pcm1	对接 PCM1 的输入输出通路
machine-stream-dsp-ecnr-dlin	DSP ECNR 下行通路
machine-stream-dsp-ecnr	DSP ECNR 上行通路
machine-stream-dsp-vr-ul	DSP 4 声道（2 声道 MIC、2 声道 REF，拼接格式查阅 拼接格式说明 ）拼接数据输入通路，上层使用的 MIC 数据可从此节点抽离获取
machine-stream-dsp-vr2-ul	DSP 4 声道（2 声道 MIC、2 声道 REF，拼接

	格式查阅 拼接格式说明) 拼接数据输入通路，上层使用的 MIC 数据可从此节点抽离获取
machine-stream-dsp-ll-dl	用于定制化快速出声通路
machine-stream-ap-vr	ALSA 4 声道 (2 声道 MIC、2 声道 REF，拼接格式查阅 拼接格式说明) 拼接数据输入通路
machine-stream-multi-os-primary	仅供 debug
machine-stream-multi-os-secondary	
machine-stream-multi-os-tertiary	

17.2.4.3 ALSA Path 相关 Kcontrol 介绍

表 17-4 ALSA Patg 相关 Kcontrol

功能分类	参数描述	tinymix 名称	功能描述
音量	音量的参数描述： 0dB 的音量对应为 0x20000 正常设置范围是 0~0x20000 特殊情况下，可能需要增益，则可以设置更大音量，真正设置范围是 0~7FFFFFFF，注意增益设置下谨防爆音削顶。 Note:使用 tinymix 时，请将音量转换为 int 值后输入，否则输入不成功或设置错误。	Media_Master_Vol	Media_Pri/Media_Sec/Media_Ter 混音后的音量
		Media_Pri_Vol	DSP Primary 的音量
		Media_Sec_Vol	DSP Secondary 的音量
		Media_Ter_Vol	DSP Tertiary 的音量
		Sys_DL1_Vol	DSP DL1 的音量 (DL1_1~DL1_8 混音后的音量)
		Sys_DL2_Vol	DSP DL2 的音量 (DL2_1~DL2_4 混音后的音量)
		Sys_DL3_Vol	DSP DL3 的音量
		Sys_DL4_Vol	DSP DL4 的音量
		ALSA_DSP_DL1_1_Volume	分别对应 DSP DL1_1~DL1_8 的音量
		ALSA_DSP_DL1_2_Volume	
		ALSA_DSP_DL1_3_Volume	
		ALSA_DSP_DL1_4_Volume	

		ALSA_DSP_DL1_5_Volume	
		ALSA_DSP_DL1_6_Volume	
		ALSA_DSP_DL1_7_Volume	
		ALSA_DSP_DL1_8_Volume	
		ALSA_DSP_DL2_1_Volume	分别对应 DSP DL2_1~DL2_4 的音量
		ALSA_DSP_DL2_2_Volume	
		ALSA_DSP_DL2_3_Volume	
		ALSA_DSP_DL2_4_Volume	
		I2S0_TX_Volume	分别对应 I2S0~I2S3 的 Boost 音量，比如外接 MIC 音量较小可以进行增益，默认是 0x20000
		I2S0_RX_Volume	
		I2S1_TX_Volume	
		I2S1_RX_Volume	
		I2S2_TX_Volume	
		I2S2_RX_Volume	
		I2S3_TX_Volume	
		I2S3_RX_Volume	
		pri_volume	
		sec_volume	
		thd_volume	
Mute / unMute	设置范围：0/1 0: unMute 1: Mute	Media_Master_Mute	Media_Pri/Media_Sec/Media_Ter 混音后的 Mute 控制
		Media_Pri_Mute	DSP Primary 的 Mute 控制
		Media_Sec_Mute	DSP Secondary 的 Mute 控制
		Media_Ter_Mute	DSP Tertiary 的 Mute 控制
		Sys_DL1_Mute	DSP DL1 的 Mute 控制（DL1_1~DL1_8 混音后的 Mute 控制）
		Sys_DL2_Mute	DSP DL2 的 Mute 控制（DL2_1~DL2_4 混音后的 Mute 控制）
		Sys_DL3_Mute	DSP DL3 的 Mute 控制

		Sys_DL4_Mute	DSP DL4 的 Mute 控制
		ALSA_DSP_DL1_1_Mute	分别对应 DSP DL1_1~DL1_8 的 Mute 控制
		ALSA_DSP_DL1_2_Mute	
		ALSA_DSP_DL1_3_Mute	
		ALSA_DSP_DL1_4_Mute	
		ALSA_DSP_DL1_5_Mute	
		ALSA_DSP_DL1_6_Mute	
		ALSA_DSP_DL1_7_Mute	
		ALSA_DSP_DL1_8_Mute	
		ALSA_DSP_DL2_1_Mute	分别对应 DSP DL2_1~DL2_4 的 Mute 控制
		ALSA_DSP_DL2_2_Mute	
		ALSA_DSP_DL2_3_Mute	
		ALSA_DSP_DL2_4_Mute	
Channel Config	CHCFG: CH8~CH1 指定最终的 CH 上输出音频源对应的声道。 一共是 32bit 控制，每 4bit 控制一个声道(CH)的配置，一共是 8 声道，当前 DSP 为 6 声道输出，故目前仅需设置 CH6~CH1。 Note:使用 tinymix 时，请将音量转换为 int 值后输入，否则输入不成功或设置错误。 注意当前 DL1 配置 6 声道输出，DL2 配置 2 声道输出。所以 CHCFG 对于 DL1~DL8 最多可以配置 6 声道， DL2_1~DL2_4 最多可以配置 2 声道，请结合时间情况使用配置	ALSA_DSP_DL1_1_CH_CFG	分别对应 DL1_1~DL1_8 的声道配置，最多可配置 6 声道输出
		ALSA_DSP_DL1_2_CH_CFG	
		ALSA_DSP_DL1_3_CH_CFG	
		ALSA_DSP_DL1_4_CH_CFG	
		ALSA_DSP_DL1_5_CH_CFG	
		ALSA_DSP_DL1_6_CH_CFG	
		ALSA_DSP_DL1_7_CH_CFG	分别对应 DL2_1~DL2_4 的声道配置，最多可配置 2 声道输出
		ALSA_DSP_DL1_8_CH_CFG	
		ALSA_DSP_DL2_1_CH_CFG	
		ALSA_DSP_DL2_2_CH_CFG	
		ALSA_DSP_DL2_3_CH_CFG	
		ALSA_DSP_DL2_4_CH_CFG	

Debug	设置完成后下一次开启生效。 1:master 0:slave,	dbg_i2s0_master_mode	设置 master/slave
	设置 I2S 标准模式及其模式的传输方式，设置完成后下一次开启生效。 0: I2S 标准模式 1: I2S 右对齐模式 2: I2S 左对齐模式 3: DSP 模式	dbg_i2s0_trans_mode	设置传输模式（I2S/LF/RF/DSP）
	设置 I2S Main Clock 输出配置，设置完成后下一次开启生效。 单位是 fs，即采样率的倍数。比如采样率是 48K，设置 Main Clock 为 256fs 即是 12.288MHz，那直接设置为 256。	dbg_i2s0_mclk	i2s main clock 的设置
	i2s 支持全双工设置，设置完成后下一次开启生效。 0: 支持 I2S TX/RX，两者不能同时工作 1: 支持 I2S TX 和 RX 同时工作	dbg_i2s0_full_duplex	i2s 全双工设置
	i2s TDM 模式的支持设置，设置完成后下一次开启生效。 0: 不支持 TDM 模式 1: 支持 TDM 模式	dbg_i2s0_tdm_mode	i2s TDM 模式设置
	设置 I2S TDM 的 bitclk，设置完成后下一次开启生效。如设置 256fs 则设置 256 即可。	dbg_i2s0_tdm_bclk	i2s TDM 模式下设定 tdm 的 bitclk
	声道数，可以设置 2~16	dbg_i2s0_tdm_ch_nums	i2s TDM 模式下设定 tdm 的声道数
	可以设置的值为 8/12/16/18/20/24/28/32，其他为无效设置，一般设置 16/32	dbg_i2s0_ch_width	一个声道占用的位宽
	分辨率，设置范围 1~32，表	dbg_i2s0_resolution	数据分辨率，全双工模式下的 TX 或单工模

	明 i2s 每声道需要接收或发送的位数，一般设置 16/24/32		式下的 TX/RX
	分辨率，设置范围 1~32，全双工下，表明 i2s 每声道需要接收的位数，一般设置 16/24/32	dbg_i2s0_rresolution	数据分辨率，全双工模式下的 RX
	loop 模式，该模式下，i2s 内部将 TX 与 RX 自动短接，无需外部 PIN 短接。设置完成后下一次开启生效。 0: 关闭 loop 模式 1: 打开 loop 模式	dbg_i2s0_loop_mode	i2s 的 loopmode 模式
	同 I2S0 的设定，请参考 I2S0	dbg_i2s1_master_mode	同 I2S0 的设定，请参考 I2S0
		dbg_i2s1_trans_mode	
		dbg_i2s1_mclk	
		dbg_i2s1_full_duplex	
		dbg_i2s1_tdm_mode	
		dbg_i2s1_tdm_bclk	
		dbg_i2s1_tdm_ch_nums	
		dbg_i2s1_ch_width	
		dbg_i2s1_resolution	
		dbg_i2s1_rresolution	
		dbg_i2s1_loop_mode	
		dbg_i2s2_master_mode	
		dbg_i2s2_trans_mode	
		dbg_i2s2_mclk	
		dbg_i2s2_full_duplex	
		dbg_i2s2_tdm_mode	
		dbg_i2s2_tdm_bclk	
		dbg_i2s2_tdm_ch_nums	

		dbg_i2s2_ch_width	
		dbg_i2s2_resolution	
		dbg_i2s2_rresolution	
		dbg_i2s2_loop_mode	
		dbg_i2s3_master_mode	
		dbg_i2s3_trans_mode	
		dbg_i2s3_mclk	
		dbg_i2s3_full_duplex	
		dbg_i2s3_tdm_mode	
		dbg_i2s3_tdm_bclk	
		dbg_i2s3_tdm_ch_nums	
		dbg_i2s3_ch_width	
		dbg_i2s3_resolution	
		dbg_i2s3_rresolution	
		dbg_i2s3_loop_mode	
	设置完成后下一次开启生效。 0: 短帧 1: 长帧	dbg_pcm0_short_long_frame_mode	PCM 长短帧设定
	帧长度 1~9, 表示 1~9 个 bitclk 长度	dbg_pcm0_long_frame_legth	PCM 长帧长度
	设置完成后下一次开启生效。 0: 关闭翻转 1: 翻转 clock	dbg_pcm0_sck_invert	bitclk 翻转
	设置完成后下一次开启生效。 0: 关闭翻转 1: 翻转 clock	dbg_pcm0_ws_invert	sync clk 翻转
	loop 模式, 该模式下, pcm 内部将 TX 与 RX 自动短接, 无需外部 PIN 短接。设置完成后下一次开启生效。 0: 关闭 loop 模式	dbg_pcm0_loop_mode	loop 模式

	1: 打开 loop 模式		
	同 PCM0 的设定, 请参考 PCM0	dbg_pcm1_short_long_frame_mode	同 PCM0 的设定, 请参考 PCM0
		dbg_pcm1_long_frame_legth	
		dbg_pcm1_sck_invert	
		dbg_pcm1_ws_invert	
		dbg_pcm1_loop_mode	
	多系统之间的简单互动, 用于测试	ADAS_TEST_FLAG	单元测试和集成测试专用
		CLUSTER_TEST_FLAG	
	只读, 用于时间校准, 主要用于 debug	IVI_DIFF_TIME	IVI 系统相对于主输出 DMA 时间的 diff 值

关于表格中的 tinymix, 都只有 1 个参数。

关于表格中的 Channel Config, 举例说明如下:

- 如果音频源是 2ch, 想从 CH1 和 CH2 输出, 则设置为 0x21 即可, 如果想 4 声道输出, CH3 CH4 Copy CH1 CH2 输出, 则可以设定为 0x2121。其中 1/2 指的是音频源的声道。
- 如果音频源是 1ch, 想从 CH1 输出, 则设置为 0x1 即可, 想从 CH2 输出, 设置 0x10 即可, 想从 CH3 输出, 设置 0x100 即可, 想从 CH4 输出, 设置 0x1000 即可; 想从 CH1CH2 一起输出, 设置 0x11 即可, 想从 CH3CH4 一起输出, 设置 0x1100 即可。
- 如果音频源是 2ch, 想左声道 (第 1 声道) 从 CH1 输出, 则设置 0x1 即可; 想右声道 (第 2 声道) 从 CH1 输出, 则设置 0x2 即可。

17.2.4.4 配置方式

17.2.4.4.1 Phone Audio 配置

ALSA device 节点配置 xml 文件为 audioCustomConfig.xml, 与前一小节 xml 文件一致。如下为示例输出 config。输出配置以 “outConfig” 进行块标识, 其中的每一个一对一映射均以 “out” 作为标识, “sink” 为 Audio HAL 层输出 path 的标识, 如 adsp、primary、quick、beep 分别对应于 AudioFlinger 中对应名称的输出线程, machine-stream-dsp-dl-1 为 ALSA 中某 device 的完整名称。如需查看当前 ALSA driver 提供的所有 device 节点, 可采用 adb shell cat /proc/asound/pcm 进行查看, 如下图 “PCM 节点” 所示为当前的 ALSA device 信息, 如 “00-00: machine-stream-dsp-

primary atc-soc-codec-dai-dspse-0 : : playback 1”表示声卡 0，device 0，device 名称为 machine-stream-dsp-primary，补充描述为 atc-soc-codec-dai-dspse-0，仅支持 playback。

```
<outConfig>
    <!--this adsp path use i2s must setprop vendor.audiohal.playback.i2s.support
1-->

    <!--out sink="adsp">machine-stream-i2s-sc0</out-->

    <out sink="primary">machine-stream-dsp-dl-1</out>

    <out sink="quick">machine-stream-dsp-dl-2</out>

    <out sink="beep">machine-stream-dsp-dl-3</out>
</outConfig>
```

如下表格描述 AC8025 当前系统中可配置的 Audio HAL 输出 path 标识及其对应关系。

表 17-5 sink 标识与含义关系表

Sink 标识	含义
adsp	对应 Flag 为 AUDIO_OUTPUT_FLAG_TO_ADSP 的 2 声道混音线程
primary	对应 Flag 包含 AUDIO_OUTPUT_FLAG_PRIMARY 的混音线程
quick	对应 Flag 为 AUDIO_OUTPUT_FLAG_QUICK 的混音线程
beep	对应 Flag 为 AUDIO_OUTPUT_FLAG_BEEP 的混音线程

如输出配置中的注释说明，若当前系统媒体类输出无需经过内置 DSP 的音效处理，需要经 I2S 输出，除了在 xml 中配置外，还需设置 property “vendor.audiohal.playback.i2s.support” 为 1。

例，如需将 Music 输出 2 声道，可以将 Music 这个 StreamType 配置移除 flag_to_adsp，此时 Music 将在 AudioFlinger 中 Flag 为 Primary 的 MixerThread 中进行混音；经 DSP 的 DL1 进行 2 声道输出，若通过 ALSA 节点配置 primary 为 I2S 节点，此时，Music 声音将从 I2S 输出。

17.2.4.4.2 Car Audio 配置

Car Audio 下 outConfig 配置示例如下：

```
<outConfig>

    <out sink="bus0_media_out">machine-stream-dsp-primary</out>

    <out sink="bus0_smix_out">machine-stream-dsp-secondary</out>

    <out sink="bus1_navigation_out">machine-stream-dsp-dl-1</out>

    <out sink="bus2_voice_command_out">machine-stream-dsp-dl-4</out>

    <out sink="bus3_call_ring_out">machine-stream-dsp-dl-3</out>

    <out sink="bus4_call_out">machine-stream-dsp-dl-2</out>
```

```
<out sink="bus5_alternate_out">machine-stream-dsp-dl-1</out>

<out sink="bus100_audio_zone_1">machine-stream-i2s-sc1</out>

</outConfig>
```

其中的 sink 标识对应了各个 device 的 address。

17.2.4.4.3 输入配置

VR 输入配置如下所示，主要配置为 sampleRate 和 channel，其中 sampleRate 表示打开 machine-stream-dsp-vr-ul 节点时的采样率，channel 表示支持的声道数，默认为 4 声道。

```
<vrConfig>

  <vr attribute="sampleRate">48000</vr>

  <vr attribute="channel">4</vr>

</vrConfig>
```

Auxin 输入配置如下，其中 aux_mode 有 2 种取值，分别为 direct mode 和 indirect mode，取值分别为 1 和 0。Direct mode 数据不经过 Hal 层及之上，通过 createAudioPatch 创建 device 到 device 的 audiopatch；indirect mode 使用 AudioRecord 将数据录制到 App 层再使用 AudioTrack 进行播放。digital_i2s_index 表示从哪个 I2S 输入 digital in 数据，分别为 I2S0、I2S1、I2S2、I2S3，取值分别为 0、1、2、3。

```
<auxConfig>

  <!-- aux_mode: 1->direct mode, 0->indirect mode; digital_i2s_index(direct and
  indirect mode): 0~3->i2s0/i2s1/i2s2/i2s3; -->

  <aux name="aux_mode">0</aux>

  <aux name="digital_i2s_index">0</aux>

</auxConfig>
```

MIC Gain 配置如下，其中 MIC_Gain 即表示了通过 kcontrol 设置的 MIC_Gain 的数值。

```
<gainConfig>

  <!--MIC_Gain-->

  <gain name="MIC_Gain">4</gain>

</gainConfig>
```

配置 ALSA device 节点时，需要注意标识 Audio HAL 层输入输出 path 的名称不要随意变动，Audio HAL 层需要依据该名称获取映射的 ALSA device 名称。


```
ac8025_demo:/ # cat /proc/asound/pcm
cat /proc/asound/pcm
00-00: machine-stream-dsp-primary atc-soc-codec-dspse-0 : : playback 1
00-01: machine-stream-dsp-secondary atc-soc-codec-dspse-1 : : playback 1
00-02: machine-stream-dsp-tertiary atc-soc-codec-dspse-2 : : playback 1
00-03: machine-stream-dsp-dl-1-1 atc-soc-codec-dspse-3 : : playback 1
00-04: machine-stream-dsp-dl-1-2 atc-soc-codec-dspse-4 : : playback 1
00-05: machine-stream-dsp-dl-1-3 atc-soc-codec-dspse-5 : : playback 1
00-06: machine-stream-dsp-dl-1-4 atc-soc-codec-dspse-6 : : playback 1
00-07: machine-stream-dsp-dl-1-5 atc-soc-codec-dspse-7 : : playback 1
00-08: machine-stream-dsp-dl-1-6 atc-soc-codec-dspse-8 : : playback 1
00-09: machine-stream-dsp-dl-1-7 atc-soc-codec-dspse-9 : : playback 1
00-10: machine-stream-dsp-dl-1-8 atc-soc-codec-dspse-10 : : playback 1
00-11: machine-stream-dsp-dl-2-1 atc-soc-codec-dspse-11 : : playback 1
00-12: machine-stream-dsp-dl-2-2 atc-soc-codec-dspse-12 : : playback 1
00-13: machine-stream-dsp-dl-2-3 atc-soc-codec-dspse-13 : : playback 1
00-14: machine-stream-dsp-dl-2-4 atc-soc-codec-dspse-14 : : playback 1
00-15: machine-stream-dsp-dl-3 atc-soc-codec-dspse-15 : : playback 1
00-16: machine-stream-dsp-dl-4 atc-soc-codec-dspse-16 : : playback 1
00-17: machine-stream-awb snd-soc-dummy-dai-17 : : capture 1
00-18: machine-stream-i2s-sc0 snd-soc-dummy-dai-18 : : playback 1 : capture 1
00-19: machine-stream-i2s-sc1 snd-soc-dummy-dai-19 : : playback 1 : capture 1
00-20: machine-stream-i2s-sc2 snd-soc-dummy-dai-20 : : playback 1 : capture 1
00-21: machine-stream-i2s-sc3 snd-soc-dummy-dai-21 : : playback 1 : capture 1
00-22: machine-stream-pcm snd-soc-dummy-dai-22 : : playback 1 : capture 1
00-23: machine-stream-pcm1 snd-soc-dummy-dai-23 : : playback 1 : capture 1
00-24: machine-stream-dsp-ecnr-dlin atc-soc-codec-dspse-24 : : playback 1
00-25: machine-stream-dsp-ecnr snd-soc-dummy-dai-25 : : capture 1
00-26: machine-stream-dsp-vr-ul snd-soc-dummy-dai-26 : : capture 1
00-27: machine-stream-dsp-vr2-ul snd-soc-dummy-dai-27 : : capture 1
00-28: machine-stream-dsp-ll-dl atc-soc-codec-dspse-28 : : playback 1
00-29: machine-stream-ap-vr snd-soc-dummy-dai-29 : : capture 1
00-30: machine-stream-multi-os-primary snd-soc-dummy-dai-30 : : playback 1
00-31: machine-stream-multi-os-secondary snd-soc-dummy-dai-31 : : playback 1
00-32: machine-stream-multi-os-tertiary snd-soc-dummy-dai-32 : : playback 1
```

图 17-9 PCM 节点

17.2.4.5 Debug 方法

对于输出 device 的配置验证，可使用如下方式：

- 播放能够输出到该输出线程的 streamtype 音频数据，该部分采用 dumphys media.audio_flinger 查看；
- 正常播放声音后，通过 `lsf | grep pcm` 查看当前打开的 ALSA device 节点信息，通过 device ID 与名称的对比确定配置是否成功。

对于输入 device 的配置验证，可使用如下方式：

- 采用 APP 录音或者蓝牙电话方式，预设对应的 ALSA device 节点通路处于 open 状态；如验证 digital_in 可采用 Source 为 AUDIO_SOURCE_DIGITAL_IN 的 AudioRecord 进行录制，如验证 mic_in 可采用 Source 为 AUDIO_SOURCE_MIC 的 AudioRecord 进行录制；ref_in 可采用蓝牙电话方式进行验证；

- 通过 `lsdf | grep pcm` 查看当前打开的 ALSA device 节点信息，通过 device ID 与名称的对比确定配置是否成功。

17.2.5 输出声道配置

AC8025 结合内置 DSP 支持客户定制输出声道配置，即内置 DSP 输出为 5.1 声道，分别为前左（FL），前右（FR），后左/左环绕（LS），后右/右环绕（RS），中置（C），低音（Sub），ALSA 的不同输出 path 可以通过控制 channel MAP 的方式，将 userspace write 进 ALSA 的数据进行定制化从 DSP 的哪个声道进行输出。

17.2.5.1 配置说明

配置 ALSA 输出声道的参数为 `outhw_channel_mode`，在 code 中的位置为 `source/kernel/common/sound/soc/autochips/common/aud_dsphw`。

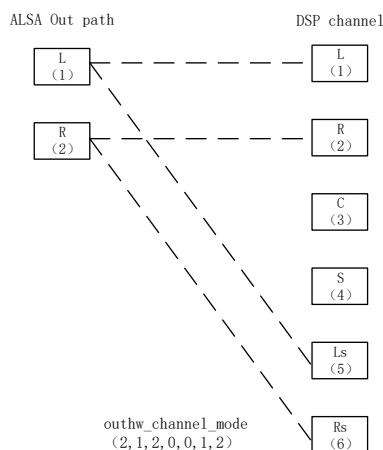


图 17-10 声道映射示意图

如图所示，通过配置 `outhw_channel_mode` 即将输出 2 声道的 path 经内置 DSP 处理成多声道数据后，再经 I2S TDM 或者 MC 对应声道进行播放。`outhw_channel_mode` 配置共 7 个值，第一个值表示当前 ALSA 输出 path 的声道数，第二个值到第七个值分别表示 ALSA 输出数据的第几声道输出到当前 DSP 的对应 index 的声道。上图表示 ALSA 输出 path 为 2 声道，分别将左右声道数据 copy 进 DSP 的前左、前右、左环绕、右环绕声道，配置的 `outhw_channel_mode` 为 (2,1,2,0,0,1,2)，其中第一个 2 表示 ALSA 输出的声道数，后面的 6 位分别表示 DSP 的 L、R、C、S、Ls、Rs 声道，如在第一位上的数字为 1 表示 ALSA 输出的第一声道数据 copy 进 DSP 的 L 声道，第二位上的数字为 2 表示 ALSA 输出的第二声道数据 copy 进 DSP 的 R 声道，第三位上的数字为 0 表示无数据 copy，以此类推，这种配置即可将 ALSA 的双声道数据 copy 进 DSP 的 L、R、Ls、Rs 四个声道进行输出。

17.3 音量部分

在车机环境中，音量是非常敏感及重要的功能，本章节主要描述音量设置、音量的交互等功能。

17.3.1 音量框架

在 AC8025 的产品形态中，分为 IVI 产品和 Cockpit 产品。传统的 IVI 产品包含 Android 系统和 Safety 系统，Cockpit 产品包含 Android 系统、ADAS 系统、Cluster 系统和 Safety 系统，音量的设置均交由 IVI 系统处理，交互控制交由 ATC VolumePolicy 处理，系统与系统之间采用 ATC 实现的 IPC 进行通信。CarAudio 或者 PhoneAudio 是 IVI 系统中的概念，与 IVI 产品或者 Cockpit 产品无关。

航盛2024-09-12 16:50:32

17.3.1.1 IVI 产品音量框架

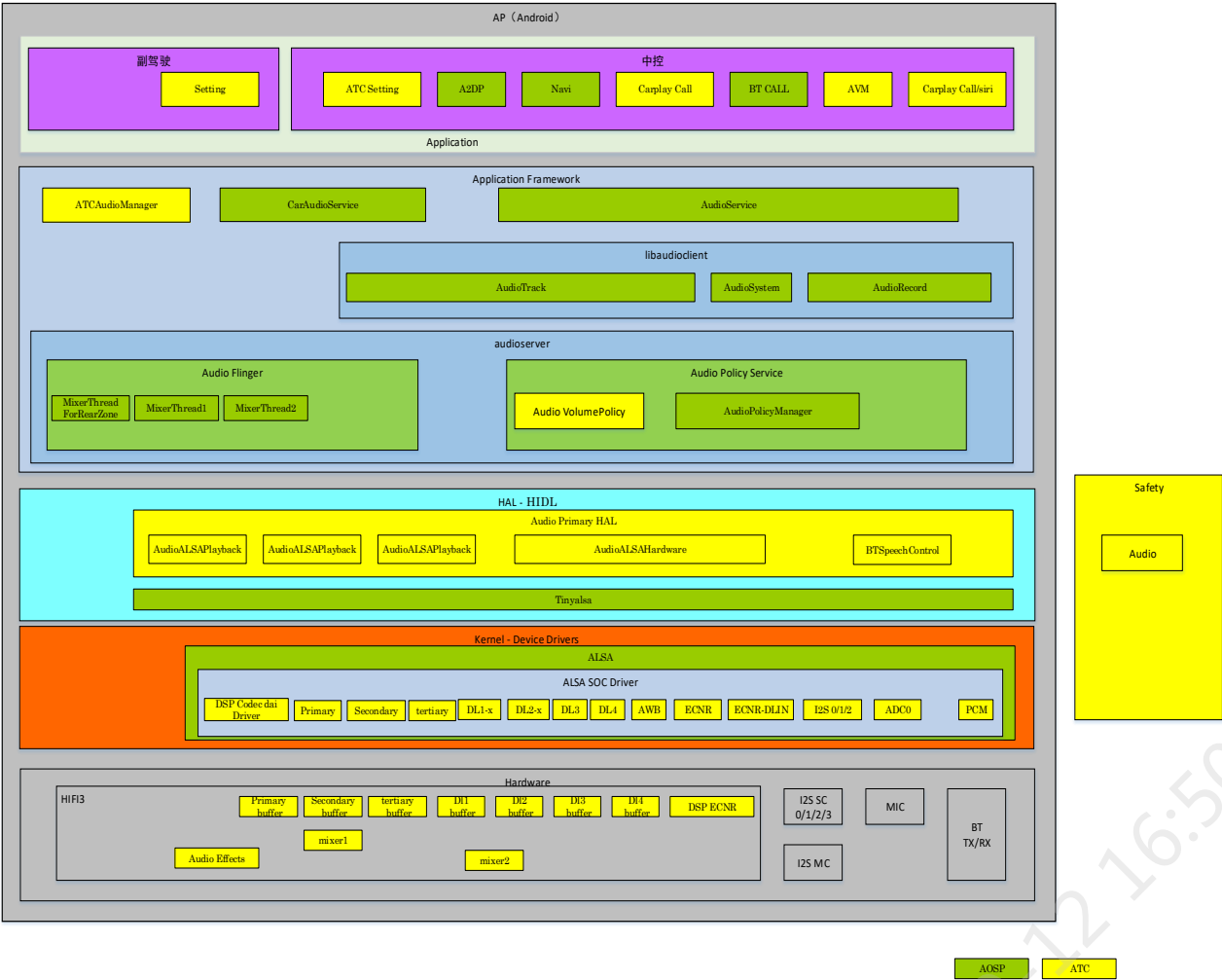


图 17-11 IVI 音量框架图

17.3.1.2 Cockpit 产品音量框架

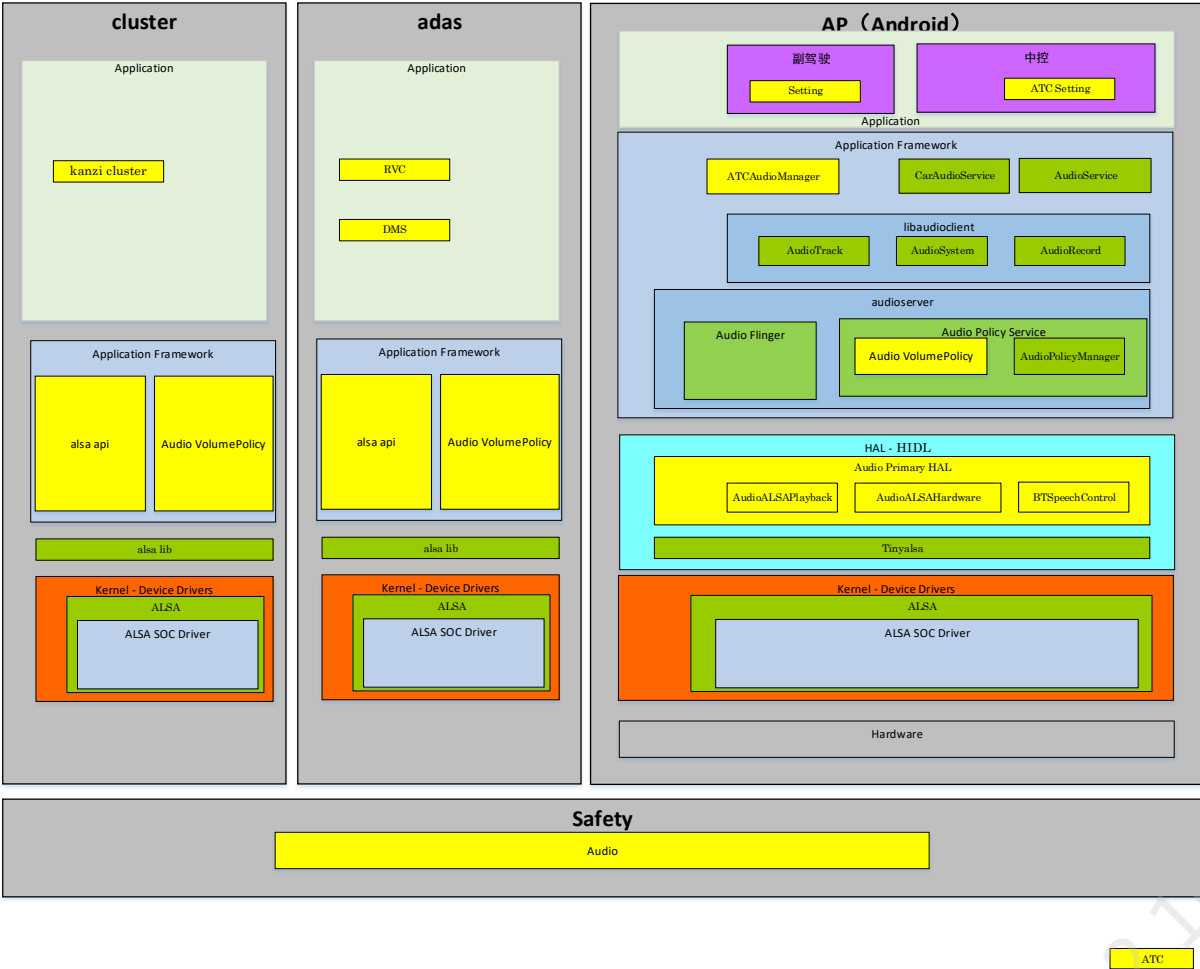


图 17-12 Cockpit 音量框架

17.3.2 音量接口介绍

17.3.2.1 CarAudioManager

17.3.2.1.1 类概述

CarAudio 配置时，App 层只能通过 CarAudioManager 类中的相关接口进行音量设置，AudioManager 的音量接口将不再生效，CarAudioManager 的相关接口如下：

表 17-6 CarAudioManager 接口

Public方法	说明
List<Integer> getAudioZoneIds()	获取当前配置的Audio Zone

int getVolumeGroupCount()	获取Primary Audio Zone的VolumeGroup个数
int getVolumeGroupCount(int zoneId)	获取zoneId对应的Audio Zone中的VolumeGroup个数
int getVolumeGroupIdForUsage(int usage)	获取Primary Audio Zone中给定usage对应的volume group id
int getVolumeGroupIdForUsage(int zoneId, int usage)	获取zoneId对应的Audio Zone中给定usage对应的volume group id
int getGroupMaxVolume(int groupId)	获取Primary Audio Zone中groupId的最大音量值
int getGroupMaxVolume(int zoneId, int groupId)	获取zoneId对应的Audio Zone中groupId的最大音量值
int getGroupMinVolume(int groupId)	获取Primary Audio Zone中groupId的最小音量值
int getGroupMinVolume(int zoneId, int groupId)	获取zoneId对应的Audio Zone中groupId的最小音量值
int getGroupVolume(int groupId)	获取Primary Audio Zone中groupId的当前音量值
int getGroupVolume(int zoneId, int groupId)	获取zoneId对应的Audio Zone中groupId的当前音量值
void setGroupVolume(int groupId, int index, int flags)	设置Primary Audio Zone中groupId的音量值为index
void setGroupVolume(int zoneId, int groupId, int index, int flags)	设置zoneId对应的Audio Zone中groupId的音量值为index

17.3.2.1.2 getAudioZoneIds

【描述】

获取当前配置的 Audio Zone。

【语法】

```
public List<Integer> getAudioZoneIds();
```

【返回值】

Audio ZoneId 数组

【参数】

无

17.3.2.1.3 getVolumeGroupCount

【描述】

获取某个 Audio Zone 的 VolumeGroup 个数。

【语法】

```
public int getVolumeGroupCount();  
public int getVolumeGroupCount(int zoneId)
```

【返回值】

某个 Audio Zone 的 VolumeGroup 个数

【参数】

zoneId: Audio Zone ID，表示声音区域

17.3.2.1.4 getVolumeGroupIdForUsage**【描述】**

获取某个 Audio Zone 中给定 usage 对应的 volume group id。

【语法】

```
public int getVolumeGroupIdForUsage(int usage);  
public int getVolumeGroupIdForUsage(int zoneId, int usage);
```

【返回值】

某个 Audio Zone 中给定 usage 对应的 volume group id

【参数】

- usage: AudioAttribute 中的 usage，表示声音的用意
- zoneId: Audio Zone ID，表示声音区域

17.3.2.1.5 getGroupMaxVolume**【描述】**

获取某个 Audio Zone 中 groupId 的最大音量值。

【语法】

```
public int getGroupMaxVolume(int groupId);  
public int getGroupMaxVolume(int zoneId, int groupId);
```

【返回值】

某个 Audio Zone 中 groupId 的最大音量值

【参数】

- groupId: volume group id，处于一个 volume group 的多种 usage 声音，其音量相同
- zoneId: Audio Zone ID，表示声音区域

17.3.2.1.6 getGroupMinVolume

【描述】

获取某个 Audio Zone 中 groupId 的最小音量值。

【语法】

```
public int getGroupMinVolume(int groupId);
public int getGroupMinVolume(int zoneId, int groupId);
```

【返回值】

某个 Audio Zone 中 groupId 的最小音量值

【参数】

- groupId: volume group id, 处于一个 volume group 的多种 usage 声音, 其音量相同
- zoneId: Audio Zone ID, 表示声音区域

17.3.2.1.7 getGroupVolume

【描述】

获取某个 Audio Zone 中 groupId 的当前音量值。

【语法】

```
public int getGroupVolume(int groupId)
public int getGroupVolume(int zoneId, int groupId)
```

【返回值】

某个 Audio Zone 中 groupId 的当前音量值

【参数】

- groupId: volume group id, 处于一个 volume group 的多种 usage 声音, 其音量相同
- zoneId: Audio Zone ID, 表示声音区域

17.3.2.1.8 setGroupVolume

【描述】

设置某个 Audio Zone 中 groupId 的音量值为 index。

【语法】

```
public void setGroupVolume(int groupId, int index, int flags)
```



```
public void setGroupVolume(int zoneId, int groupId, int index, int flags);
```

【返回值】

无

【参数】

- groupId:设置音量的 volume group id，处于一个 volume group 的多种 usage 声音，其音量相同
- zoneId: Audio Zone ID，表示声音区域
- index:音量 index
- flags:可选 FLAG_SHOW_UI/FLAG_PLAY_SOUND，分别表示显示 UI 和播放提示音，一般写 0 即可

17.3.2.2 AudioManager

17.3.2.2.1 类概述

当 CarAudio 配置时，如下 AudioManager 的音量接口将不再生效。

当 PhoneAudio 配置时，APP 层除了可以通过 CarAudioManager 的接口设置音量（其实现也是调用 AudioManager 的音量接口）外，还可以通过 AudioManager 类中的相关接口进行音量设置，相关接口如下：

表 17-7 AudioManager 类音量设置接口

Public方法	说明
void adjustStreamVolume()	调节某一个streamType对应的音量
void adjustSuggestedStreamVolume()	调节某一个streamType音量，但实际调节的streamType音量可能与建议调节的streamType不一致，实际调节的streamType由当前active的音频流决定
void adjustVolume()	调节当前正在播放的音频音量
void setStreamVolume()	设置某一个streamType对应的index音量

17.3.2.2.2 adjustStreamVolume

【描述】

调节对应 streamType 的音量，每次调节 step 为 1，即音量 index 值在当前基础上加 1 或减 1。

【语法】

```
public void adjustStreamVolume(int streamType, int direction, int flags);
```

【返回值】

None

【参数】

- streamType, 将要调节的音频对应的 streamType;
- direction, 取值 ADJUST_LOWER/ADJUST_RAISE/ADJUST_SAME 分别对应调小/调大/不变;
- flags, 一般设置为 0

17.3.2.2.3 adjustSuggestedStreamVolume

【描述】

调节相关 streamType 的音量。当 suggestStreamType 为确定的某个 streamType, 则调节对应的音量; 否则 suggestStreamType 为 USE_DEFAULT_STREAM_TYPE 时, 则调节当前 active 的音频, 没有 active 的音频则不进行任何调节。

【语法】

```
public void adjustSuggestedStreamVolume(int direction, int suggestedStreamType, int flags);
```

【返回值】

None

【参数】

- direction, 取值
ADJUST_LOWER/ADJUST_RAISE/ADJUST_SAME/ADJUST_MUTE/ADJUST_UNMUTE/
ADJUST_TOGGLE_MUT, ADJUST_LOWRE/ADJUST_RAISE 分别将 index 值调大 1 和调小 1, ADJUST_SAME 不调节大小, ADJUST_MUTE/ADJUST_UNMUTE 分别将 index 调为 0 和恢复原来 index 值, ADJUST_TOGGLE_MUTE 则以开关方式实现 mute 和 unmute;
- suggestStreamType, 建议调节的音频类型;
- flags, 一般设置为 0

17.3.2.2.4 adjustVolume

【描述】

调节当前 active 音频的音量。

【语法】

```
public void adjustVolume(int direction, int flags);
```

【返回值】

None

【参数】

- direction, 取值
ADJUST_LOWER/ADJUST_RAISE/ADJUST_SAME/ADJUST_MUTE/ADJUST_UNMUTE/
ADJUST_TOGGLE_MUT, ADJUST_LOWRE/ADJUST_RAISE 分别将 index 值调大 1 和调小 1, ADJUST_SAME 不调节大小, ADJUST_MUTE/ADJUST_UNMUTE 分别将 index 调为 0 和恢复原来 index 值, ADJUST_TOGGLE_MUTE 则以开关方式实现 mute 和 unmute;
- flags, 一般设置为 0

17.3.2.2.5 setStreamVolume

【描述】

设置对应 streamType 音量。

【语法】

```
public void setStreamVolume(int streamType, int index, int flags);
```

【返回值】

None

【参数】

- streamType, 设置的音频类型;
- index, 音量值;
- flags, 一般设置为 0。

17.3.2.3 ATCAudioManager

17.3.2.3.1 类概述

ATCAudioManager 是 ATC 私有接口, 位于 autochips.jar 包中, 源码路径为:
source/android/vendor/autochips/proprietary/frameworks/base/atcaudiomanager。

提供对非 IVI 系统中的音源音量控制。相关接口如下:

表 17-8 ATCAudioManager 接口

Public方法	说明
Map<Integer, String> getMulSystemStreams()	获取当前非IVI系统的streamType类型和名称
void setStreamVolume(int	设置某一个streamType对应的音量

streamType, int volume)	
int getStreamVolume(int streamType)	获取某一个streamType对应的音量
int getMinVolume()	获取可调节的最小音量值
int getMaxVolume()	获取可调节的最大音量值

表 17-9 非 IVIstreamType 说明

非IVI系统StreamType类型	使用场景
STREAM_VEHICLE	一般用于kanzi cluster
STREAM_ADAS_SYSTEM	一般用于DMS
STREAM_VBA	Reserved
STREAM_RVC	一般用于ADAS 倒车
STREAM_SAFETY_RVC	一般用于R5 only倒车
STREAM_SAFETY_VBA	一般用于R5开机音效
STREAM_SAFETY_CHIME	一般用于safety chime

17.3.2.3.2 getMulSystemStreams

【描述】

获取非 IVI 系统 StreamType 类型和名称。

【语法】

```
public Map<Integer, String> getMulSystemStreams();
```

【返回值】

非 IVI 系统 StreamType 类型和名称

【参数】

无

17.3.2.3.3 setStreamVolume

【描述】

设置某一个 streamType 对应的音量。

【语法】

```
public void setStreamVolume(int streamType, int volume);
```

【返回值】

无

【参数】

- streamType: 非 IVI 系统 StreamType 类型;
- volume: 要设置的音量值

17.3.2.3.4 getStreamVolume**【描述】**

获取某一个 streamType 对应的音量。

【语法】

```
public int getStreamVolume(int streamType);
```

【返回值】

音量值

【参数】

streamType: 非 IVI 系统 StreamType 类型;

17.3.2.3.5 getMinVolume**【描述】**

获取非 IVI 系统音源可设置的最小音量值。

【语法】

```
public int getMinVolume();
```

【返回值】

最小音量值

【参数】

无

17.3.2.3.6 getMaxVolume**【描述】**

获取非 IVI 系统音源可设置的最大音量值。

【语法】

```
public int getMaxVolume();
```

【返回值】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

441 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

最大音量值

【参数】

无

17.3.3 ATC volumepolicy

ATC volumepolicy 主要提供在多个音频源交互时自动触发音量压低策略的功能， 可通过简单的配置 xml 文件实现自定义的音频源交互行为。支持多域交互和单域交互。

17.3.3.1 AudioVolumePolicy 架构图

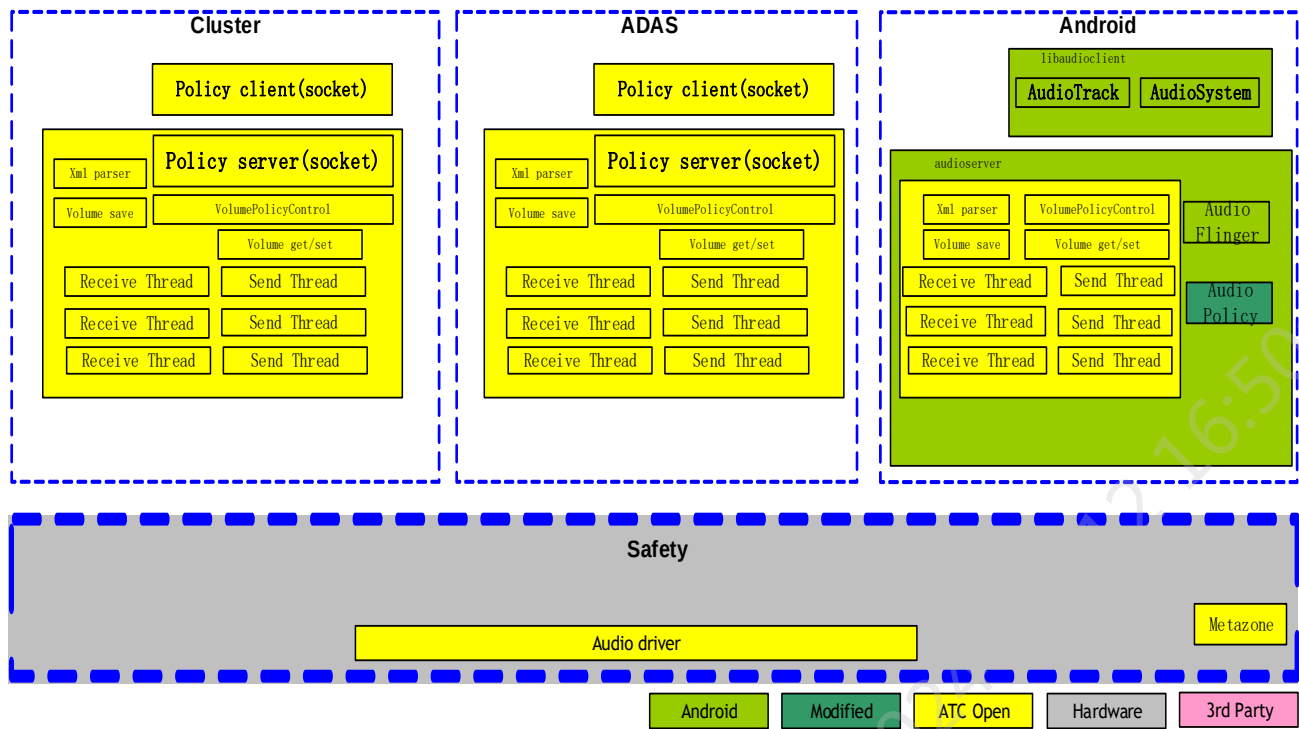


图 17-13 AudioVolumePolicy 架构图

17.3.3.2 代码结构及文件描述

表 17-10 代码接口和文件描述

文件名	文件路径	文件描述
VolumePolicyControl.cpp	/source/vendor/autochips/proprietary/frameworks/atcaudio/	VolumePolicy

文件名	文件路径	文件描述
	audiovolumepolicy/volumepolicycontrol/	主体逻辑部分
XmlParser.cpp	/source/vendor/autochips/proprietary/frameworks/atcaudio/audiovolumepolicy/xmlparser	Xml 文件和 datazone 文件解析
VolumeImpl.cpp VolumeImplInAF.cpp VolumeImplInAudioPort.cpp VolumeImplUseKControl.cpp VolumeImplUseSafetyControl.cpp VolumeImplDummy.cpp	/source/vendor/autochips/proprietary/frameworks/atcaudio/audiovolumepolicy/volumeutils	音量设置、获取接口
PolicySound.cpp VolumeConfig.h VolumePolicyConfig.h	/source/vendor/autochips/proprietary/frameworks/atcaudio/audiovolumepolicy/config	Policy 优先级、压低信息、非 IVI 系统音量信息封装
PolicyServerUseSocket.cpp main.cpp	/source/vendor/autochips/proprietary/frameworks/atcaudio/audiovolumepolicy/policyserver	Policy server 端实现
PolicyClientImplUseSocket.cpp PolicyClientWrapper.cpp	/source/vendor/autochips/proprietary/frameworks/atcaudio/audiovolumepolicy/policyclient	Policy client 端封装接口
VolumePolicyTransferIPC.cpp	/source/vendor/autochips/proprietary/frameworks/atcaudio/audiovolumepolicy/volumepolicytransfer	跨系统 message 传递
volumepolicy.xml	/source/vendor/autochips/proprietary/frameworks/atcaudio/audiovolumepolicy/car_mode	CarAudio 配置下 volume policy 配置文件
	/source/vendor/autochips/proprietary/frameworks/atcaudio/audiovolumepolicy/phone_mode	PhoneAudio 配置下 volume policy 配置文件
atc_volume.txt	/build/projects/ivi_s_ac8025/datazone /build/projects/cockpit_s_linux_ac8025/datazone	非 ivi 系统音量配置及存储文件

- 仓库名称: vendor/autochips/proprietary/framework/atcaudio/
- 目录名称: source/vendor/autochips/proprietary/framework/atcaudio/audiovolumepolicy
- Android 库名称: libaudiovolumepolicy.so
- Linux 模块名称: audiovolumepolicy libaudiovolumepolicyclient.so libaudiovolumeclient.so

17.3.3.3 流程图

17.3.3.3.1 接入方式

- Android 系统

AudioVolumePolicy 是基于 Android 的播放框架进行设计，依赖 AudioTrack 的 start、stop 操作。

AudioVolumePolicy 要求 startVolumePolicy、stopVolumePolicy 是成对操作的。

- ADAS/Cluster 系统

在 ADAS、Cluster 系统中，VolumePolicy 基于 C/S 架构，client 和 server 端采用 socket 进行通信。为方便播放 APP 的接入方便，目前已将 volumepolicy client 端接入 alsa api 的 start、stop、pause 接口中。

17.3.3.3.2 Start VolumePolicy

下图为 startVolumePolicy，其中 set Volume 根据低优先级与当前 start 的优先级之间的比例关系，按照：

应当设置的音量 = 当前 start volume policy 的音频源应设置音量 * 压低比例

进行计算，若应当设置的音量小于实际音量时，设置该音量。

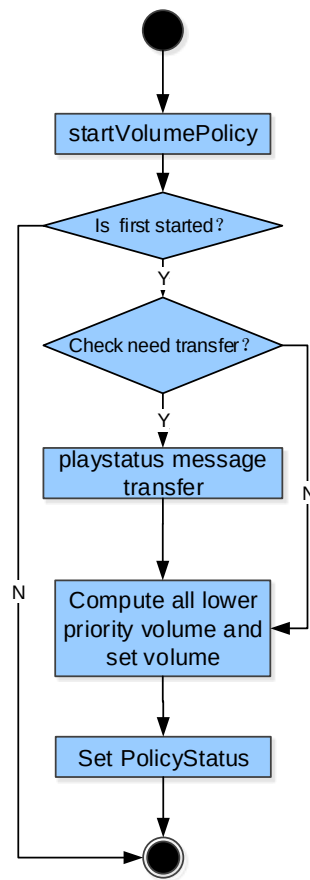


图 17-14 startVolumePolicy 调用流程图

17.3.3.3.3 Stop VolumePolicy

下图为 stopVolumePolicy，其中 reset Volume 根据当时的实际情况，可能存在：

- 若待停止音频源优先级更高，recover 当前（待停止音频源）优先级与处于 start 状态的最高优先级之间的音频源音量。之后，按照处于 Start 状态的所有优先级的音源压低比例，逐一对低优先级音源进行音量计算和设置，计算方式为当前处于 Start 状态的高优先级应设置音量*压低比例，按照最小音量优先原则进行设置；当高优先级对该低优先级的压低比例为 100 时，则恢复低优先级的音量。
- 若当前处于 Start 状态中的最高优先级更高，逐一对低于这个优先级的音源进行音量计算和设置，计算方式为当前处于 Start 状态的高优先级应设置音量*压低比例，按照最小音量优先原则进行设置；当高优先级对该低优先级的压低比例为 100 时，则恢复低优先级的音量。

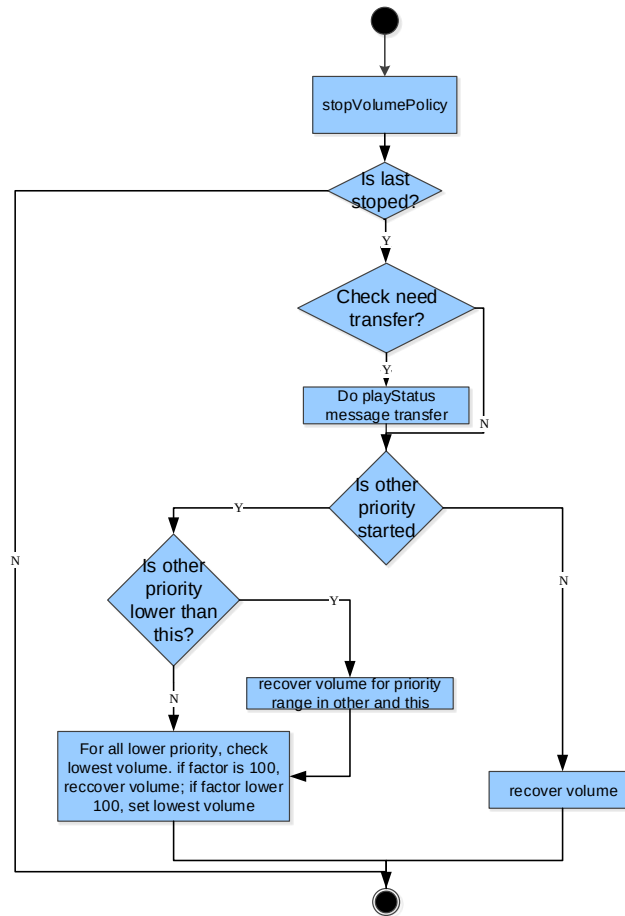


图 17-15 stopVolumePolicy 调用流程图

17.3.3.3.4 设置音量流程

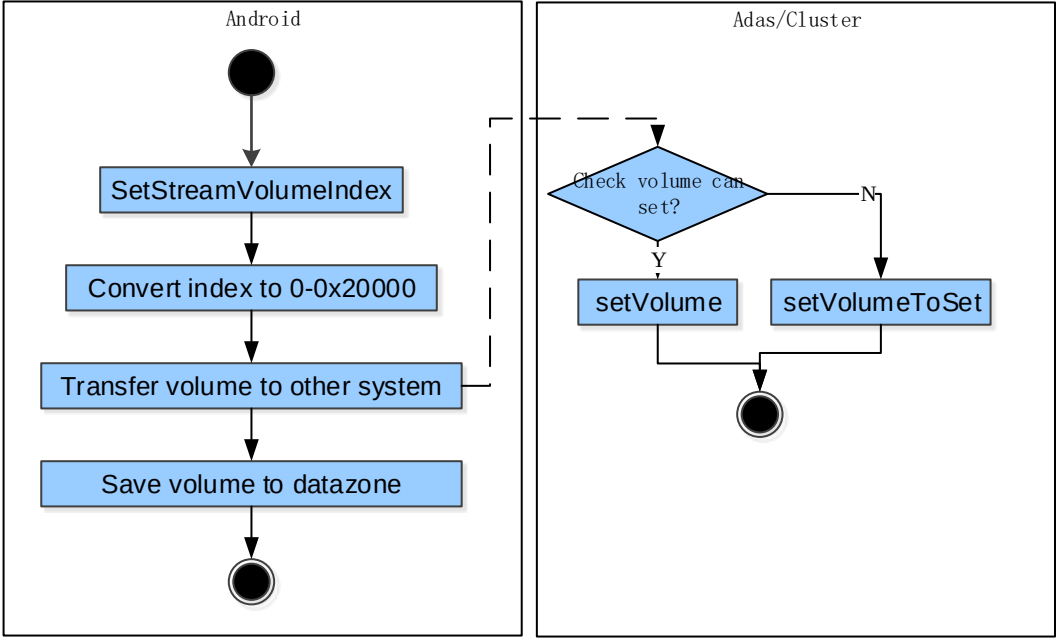


图 17-16 设置音量流程图

17.3.3.4 接口描述

此处描述的为 VolumePolicyControl 对外提供的主要接口，便于理解接入方式。

如下为 AudioVolumePolicy 模块的主要 API。

表 17-11 AudioVolumePolicy 接口

Public方法	说明
static VolumePolicyControl* getInstance();	获取VolumePolicyControl的单例对象
bool handleVolumePolicy(int streamType, bool enable);	PhoneAudio配置下， handleVolumePolicy接口
bool handleVolumePolicy(string address, bool enable);	CarAudio配置下， handleVolumePolicy接口
float getMaxVolumeForStreamType(int streamType, float volumeToSet);	PhoneAudio配置下， 获取可设置的最大音量
float getMaxVolumeForAudioPort(string address, float volumeToSet);	CarAudio配置下， 获取可设置的最大音量

17.3.3.4.1 getInstance

【描述】

获取 VolumePolicyControl 的单例对象。

该对象解析 volumepolicy.xml 和 atc_volume.xml，获取优先级和压低比例，以及待控制音源的当前音量。

【语法】

```
public static VolumePolicyControl* getInstance();
```

【参数】

无

【返回值】

VolumePolicyControl 对象

17.3.3.4.2 handleVolumePolicy

17.3.3.4.2.1 in PhoneAudio

【描述】

触发或停止某 StreamType 的 volumepolicy 行为

【语法】

```
bool handleVolumePolicy(int streamType, bool enable);
```

【参数】

- streamType: 触发或停止 volumepolicy 行为的音频源类型
- enable: true 表示触发，false 表示停止

【返回值】

操作结果，true 为成功，false 为 fail

17.3.3.4.2.2 in CarAudio

【描述】

触发或停止某 AudioPort 的 volumepolicy 行为

【语法】

```
bool handleVolumePolicy(string address, bool enable);
```

【参数】

- address: 触发或停止 volumepolicy 行为的音频源 address 标识
- enable: true 表示触发, false 表示停止

【返回值】

操作结果, true 为成功, false 为 fail

17.3.3.4.3 getMaxVolumeForStreamType

场景描述: 如在倒车场景下, Music 被自动压低。

此时在 Setting APP 中手动调节 Music 音量, 为保证倒车时的安全性, Music 只能被调小, 不能调大。Music 音量调节的较大音量值, 在倒车结束后, 自动应用。

17.3.3.4.3.1 in PhoneAudio

【描述】

获取某 StreamType 当前可设置的最大音量

【语法】

```
float getMaxVolumeForStreamType(int streamType, float volumeToSet);
```

【参数】

- streamType: 音频源类型
- volumeToSet: 预设置音量 float 值

【返回值】

该 streamType 类型可设置的最大音量

17.3.3.4.3.2 in CarAudio

【描述】

获取某 AudioPort 当前可设置的最大音量

【语法】

```
float getMaxVolumeForStreamType(string address, float volumeToSet);
```

【参数】

- address: 音频源 address 标识
- volumeToSet: 预设置音量 float 值

【返回值】

该 AudioPort 类型可设置的最大音量

17.3.3.4.3.3 setStreamVolumeIndex/getStreamVolumeIndex**【描述】**

设置和获取非 IVI 系统音源的音量

【语法】

```
bool setStreamVolumeIndex(int streamType, int valueIndex);
```

```
int getStreamVolumeIndex(int streamType);
```

【参数】

- streamType: 非 IVI 系统音源类型
- valueIndex: 音量 index

【返回值】

- bool, 设置音量成功与否
- int, 当前音量 index 值

17.3.3.5 配置方法

配置方法指的是 volumepolicy 交互功能的配置方法，在不同的交互需求中，可通过配置 volumepolicy.xml 中不同音源的优先级，及优先级之间的比例，达到不同的交互行为。

17.3.3.5.1 配置规则

volumepolicy 的交互规则使用 xml 文件进行配置，xml 文件中包含如下字段，其含义如下表。

配置注意点：

- 整个 xml 中，priority 需 0 开始，step 为 1 累加；priority 值越大，优先级越大；
- 配置 Car Audio 或者 Phone Audio 时，注意配置对应的 volumepolicy.xml 文件；
- type 的可选值为：stream, kcontrol, audioport；暂不支持其他值；
- system 的可选值为：ivi, cluster, adas, safety；暂不支持其他值。

表 17-12 volumepolicy.xml 字段及其含义

字段名称	Value 及其含义
sound	表示一种音源类型，内含 name、type、priority、system 字段，以 name 为唯一标识
name	标识一种音源类型，在 CarAudio 中，以 address 字符串作为标识；在 PhoneAudio 中，以 streamType 名称作为标识
type	标识该音源类型的音量调节方式，可选值 “stream”\ “kcontrol” \ “audioport” ,分别对应 audioflinger 中调节 streamType 的音量、kernel 中调节底层音量（8025 中指代 DSP 音量）、audio hal 层中调节 audioport 音量
priority	标识该音源类型的优先级，优先级的值由 0 开始累加（step 为 1），值越小表示优先级越低。（备注：整个 xml 中，priority 需 0 开始，step 为 1 累加）
system	标识该音源类型所在的系统，可选值 “ivi” \ “cluster” \ “adas” \ “safety” ,分别表示 Android、Cluster、Adas、Safety 系统
interaction	作为 sound 的子节点，表示父 sound 节点作为高优先级音源，压低低优先级的音源名称和压低方式
lower	标识低优先级音源名称，与 sound 节点中 name 字段相同，且已在之前出现
interaction 的 value	表示压低比例，含义为压低到高优先级当前应设置音量的 x%，x 为 0-100,0 表示 mute，100 表示不压低（无影响）

非 IVI 系统音源的名称及对应的 StreamType 匹配关系如下表：

表 17-13 非 IVI 系统音源 name 标识与 streamType 对应关系表

name标识	对应streamType
safety_vba	STREAM_SAFETY_VBA
adas	STREAM_ADAS_SYSTEM
rvc	STREAM_RVC
vehicle	STREAM_VEHICLE
safety_rvc	STREAM_SAFETY_RVC
vba	STREAM_VBA

17.3.3.5.2 Car Audio

当 CarAudio 配置时，volumepolicy.xml 的配置示例如下：

```
<?xml version="1.0" encoding="UTF-8"?>
<volumepolicy>
<sound name="bus0_media_out" type="kcontrol" priority="0" system="ivi">
</sound>
<sound name="bus3_call_ring_out" type="kcontrol" priority="1" system="ivi">
  <interaction lower="bus0_media_out">20</interaction>
</sound>
</volumepolicy>
```

```

</sound>
<sound name="bus1_navigation_out" type="audioport" priority="2" system="ivi">
    <interaction lower="bus0_media_out">20</interaction>
    <interaction lower="bus3_call_ring_out">20</interaction>
</sound>
<sound name="bus2_voice_command_out" type="kcontrol" priority="3" system="ivi">
    <interaction lower="bus0_media_out">0</interaction>
    <interaction lower="bus3_call_ring_out">0</interaction>
    <interaction lower="bus1_navigation_out">20</interaction>
</sound>
<sound name="bus4_call_out" type="kcontrol" priority="4" system="ivi">
    <interaction lower="bus0_media_out">0</interaction>
    <interaction lower="bus3_call_ring_out">0</interaction>
    <interaction lower="bus1_navigation_out">0</interaction>
    <interaction lower="bus2_voice_command_out">100</interaction>
</sound>
<sound name="bus5_alternate_out" type="audioport" priority="5" system="ivi">
    <interaction lower="bus0_media_out">20</interaction>
    <interaction lower="bus3_call_ring_out">20</interaction>
    <interaction lower="bus1_navigation_out">100</interaction>
    <interaction lower="bus2_voice_command_out">20</interaction>
    <interaction lower="bus4_call_out">20</interaction>
</sound>
<sound name="safety_vba" type="kcontrol" priority="6" system="safety">
    <interaction lower="bus0_media_out">20</interaction>
    <interaction lower="bus3_call_ring_out">20</interaction>
    <interaction lower="bus1_navigation_out">20</interaction>
    <interaction lower="bus2_voice_command_out">20</interaction>
    <interaction lower="bus4_call_out">20</interaction>
    <interaction lower="bus5_alternate_out">20</interaction>
</sound>

```



```

<sound name="adas" type="kcontrol" priority="7" system="adas">
  <interaction lower="bus0_media_out">20</interaction>
  <interaction lower="bus3_call_ring_out">20</interaction>
  <interaction lower="bus1_navigation_out">20</interaction>
  <interaction lower="bus2_voice_command_out">20</interaction>
  <interaction lower="bus4_call_out">20</interaction>
  <interaction lower="bus5_alternate_out">20</interaction>
  <interaction lower="safety_vba">0</interaction>
</sound>

<sound name="vehicle" type="kcontrol" priority="8" system="cluster">
  <interaction lower="bus0_media_out">20</interaction>
  <interaction lower="bus3_call_ring_out">20</interaction>
  <interaction lower="bus1_navigation_out">20</interaction>
  <interaction lower="bus2_voice_command_out">20</interaction>
  <interaction lower="bus4_call_out">20</interaction>
  <interaction lower="bus5_alternate_out">20</interaction>
  <interaction lower="adas">50</interaction>
  <interaction lower="safety_vba">50</interaction>
</sound>

<sound name="rvc" type="kcontrol" priority="9" system="adas">
  <interaction lower="bus0_media_out">20</interaction>
  <interaction lower="bus3_call_ring_out">20</interaction>
  <interaction lower="bus1_navigation_out">20</interaction>
  <interaction lower="bus2_voice_command_out">20</interaction>
  <interaction lower="bus4_call_out">20</interaction>
  <interaction lower="bus5_alternate_out">20</interaction>
  <interaction lower="adas">20</interaction>
  <interaction lower="safety_vba">50</interaction>
  <interaction lower="vehicle">20</interaction>
</sound>

<sound name="safety_rvc" type="kcontrol" priority="10" system="safety">

```

```

<interaction lower="bus0_media_out">20</interaction>
<interaction lower="bus3_call_ring_out">20</interaction>
<interaction lower="bus1_navigation_out">20</interaction>
<interaction lower="bus2_voice_command_out">20</interaction>
<interaction lower="bus4_call_out">20</interaction>
<interaction lower="bus5_alternate_out">20</interaction>
<interaction lower="adas">20</interaction>
<interaction lower="safety_vba">20</interaction>
<interaction lower="rvc">100</interaction>
</sound>
<sound name="safety_chi" type="kcontrol" priority="11" system="safety">
  <interaction lower="bus0_media_out">20</interaction>
  <interaction lower="bus3_call_ring_out">20</interaction>
  <interaction lower="bus1_navigation_out">20</interaction>
  <interaction lower="bus2_voice_command_out">20</interaction>
  <interaction lower="bus4_call_out">20</interaction>
  <interaction lower="bus5_alternate_out">20</interaction>
  <interaction lower="adas">20</interaction>
  <interaction lower="safety_vba">20</interaction>
  <interaction lower="rvc">100</interaction>
  <interaction lower="vehicle">20</interaction>
  <interaction lower="safety_rvc">100</interaction>
</sound>
</volumepolicy>

```

其中，sound 的 name 标识是 deviceport 中的 address 名称，可直接在 audio_policy_configuration.xml 中查询，例如：

```
<devicePort tagName="bus1_navigation_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
  address="bus1_navigation_out">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
    samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
  <gains>
    <gain name="" mode="AUDIO_GAIN_MODE_JOINT"
      minValueMB="-5000" maxValueMB="0" defaultValueMB="-1350" stepValueMB="50"/>
  </gains>
</devicePort>
<devicePort tagName="bus2_voice_command_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
  address="bus2_voice_command_out">
  <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
    samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
  <gains>
    <gain name="" mode="AUDIO_GAIN_MODE_JOINT"
      minValueMB="-5000" maxValueMB="0" defaultValueMB="-1350" stepValueMB="50"/>
  </gains>
</devicePort>
```

图 17-17 deviceport address 示例

17.3.3.5.3 Phone Audio

当 PhoneAudio 配置时，配置文件可参考
/source/vendor/autochips/proprietary/frameworks/atcaudio/audiovolumepolicy/phone_mode 下的配置
文件，name 名称表示对应的 streamType，其名称与 streamType 的对应关系如下：

表 17-14 PhoneAudio IVI 系统音源 name 标识及含义

name标识	对应streamType
voice	AUDIO_STREAM_VOICE_CALL
ring	AUDIO_STREAM_RING
music	AUDIO_STREAM_MUSIC
alarm	AUDIO_STREAM_ALARM
notification	AUDIO_STREAM_NOTIFICATION
btsco	AUDIO_STREAM_BLUETOOTH_SCO
enforced_audible	AUDIO_STREAM_ENFORCED_AUDIBLE
tts	AUDIO_STREAM_TTS
assistant	AUDIO_STREAM_ASSISTANT
gis	AUDIO_STREAM_GIS
auxin	AUDIO_STREAM_AUXIN
alternate	AUDIO_STREAM_ALTERNATE

17.3.3.6 Debug 方法

17.3.3.6.1 Dump 当前 volumepolicy 状态

adb shell dumpsys media.audio_policy

得到的如下显示，可实时查看某音源当前的 started policy 次数，具体见 “policyed num”，以及实时的音量值，具体见 “volume now”。

```
[VolumePolicy][AC8025]volumepolicy dump:
VolumePolicyControl Dump: 0xb4000071c1af5820
name: bus0_media_out; streamType: -1; policyed num: 0; volume now: 0.81
name: bus3_call_ring_out; streamType: -1; policyed num: 0; volume now: 0.73
name: bus1_navigation_out; streamType: -1; policyed num: 0; volume now: 0.73
name: bus2_voice_command_out; streamType: -1; policyed num: 0; volume now: 0.73
name: bus4_call_out; streamType: -1; policyed num: 0; volume now: 0.73
name: bus5_alternate_out; streamType: -1; policyed num: 0; volume now: 0.73
name: safety_vba; streamType: 87; policyed num: 0; volume now: 0.06
name: adas; streamType: 85; policyed num: 0; volume now: 0.24
name: vehicle; streamType: 84; policyed num: 0; volume now: 0.06
name: rvc; streamType: 81; policyed num: 0; volume now: 0.06
name: safety_rvc; streamType: 86; policyed num: 0; volume now: 0.06
name: safety_chi; streamType: 88; policyed num: 0; volume now: 0.06
VolumePolicyControl Dump done
```

图 17-18 volumepolicy dump 信息

17.3.4 导航白名单说明

通常导航音播放时需要压低媒体等音量，而 android 没有定义统一的用于导航的 streamType，故自定义了 AUDIO_STREAM_GIS 用于识别导航音，同时通过 xml 配置车机端常见的导航 app 包名来匹配识别导航 app 进程，将导航 app 进程创建的 AudioTrack 时通过 AudioSystem.isNaviProcType 方法查询当前进程是否为导航 App 进程，如果是则将 streamType 替换为 AUDIO_STREAM_GIS 来标识导航音，这样在 ATC volumepolicy 中实现了导航音的交互行为。例如：

```
<GISName>navi</GISName>
```

高德导航 App 进程名 com.autonavi.amapauto 中包含 navi 关键字，则高德导航 App 数据导航类 App，当其进行语音播放时 streamType 将替换为 AUDIO_STREAM_GIS，通过该 streamType 在 volumepolicy 中作用可实现导航播报时音乐音量自动压低。

17.3.4.1 导航白名单配置

导航白名单的 xml 文件放置于工程中的

source/android/device/autochips/vendor/ac8x/common/audio/audioGisWhitelist.xml，在平台系统中的路径为/vendor/etc/audioGisWhitelist.xml。

其中 GISName 属性的值为导航 app 的包名或包名的部分关键字。

17.3.5 典型问题

17.3.5.1 如何配置和查看 CarAudio 还是 PhoneAudio

17.3.5.1.1 配置方式

公版默认配置为 CarAudio。

配置方式：source/android/device/autochips/vendor/和 source/android/device/autochips/system/对应的 device 目录下，ProjectConfig.mk 和 SystemConfig.mk 文件中的 ATC_CARAUDIO 置为 yes 时，表示当前为 CarAudio，置为 no 时，表示当前为 PhoneAudio。

配置为 CarAudio 时，支持多 Audio Zone（即多音区）；配置为 PhoneAudio 时，仅支持主音区。

17.3.5.1.2 查看方式

- 方式 1：通过 `dumpsys media.audio_flinger` 查看输出线程的 device 是否为 `AUDIO_DEVICE_OUT_BUS` 判定，若是 `AUDIO_DEVICE_OUT_BUS`，表示当前配置的为 CarAudio，否则，为 PhoneAudio

```
Output thread 0xb40000738fcf19e0, name AudioOut_D, tid 474, type 0 (MIXER):
I/O handle: 13
Standby: no
Sample rate: 96000 Hz
HAL frame count: 2048
HAL format: 0x3 (AUDIO_FORMAT_PCM_32_BIT)
HAL buffer size: 16384 bytes
Channel count: 2
Channel mask: 0x00000003 (front-left, front-right)
Processing format: 0x3 (AUDIO_FORMAT_PCM_32_BIT)
Processing frame size: 8 bytes
Pending config events: none
Output devices: 0x1000000 (AUDIO_DEVICE_OUT_BUS)
Input device: 0 (AUDIO_DEVICE_NONE)
Audio source: 0 (AUDIO_SOURCE_DEFAULT)
Timestamp stats: n=1735731 disc=78 cold=0 nRdy=0 err=36 rate=1 jitterMs(ave=-0.000669101 std=0.109778)
Timestamp corrected: no
Last write occurred (msecs): 16
Process time ms stats: ave=1.41498 std=0.570786 min=0.951815 max=9.81411
Hal write jitter ms stats: ave=-4.56837 std=15.0416 min=-20.226 max=25.1813
Threadloop write latency stats: ave=88.7985 std=5.27503 min=49.2865 max=110.785
Master volume: 1.000000
Master mute: off
Mixer channel Mask: 0x3 (front-left, front-right)
```

图 17-19 CarAudio output device 显示

- 方式 2：通过 ATC 新增的 property 进行查询，`getprop ro.atc_caraudio`，若显示 yes，表示当前配置的为 CarAudio，若显示 no，表示当前配置的为 PhoneAudio。

17.3.5.2 如何 disable ATC VolumePolicy

当客户已经有自己的交互实现时，可通过如下方式 disable ATC VolumePolicy 功能。将 AudioPolicyManager.h 中的 `AUDIO_VOL_POLICY` define 注释掉即可，如图：

```

82 // Default minimum length allowed for offloading a compressed track
83 // Can be overridden by the audio.offload.min.duration.secs property
84 #define OFFLOAD_DEFAULT_MIN_DURATION_SECS 60
85
86 #ifdef ATC AOSP ENHANCEMENT_AUDIO
87 // #define AUDIO_VOL_POLICY
88 #define NUM_POLICY_STREAM 11
89
90 #define BACKCAR_VOICE_COEXIT_HANDLE
91 #endif
    
```

图 17-20 disable volumepolicy 方式

17.3.5.3 查看当前音量

在一些问题的查看中，如无声等，及时查看音量是否为 0 是非常快速的判定排查方法。如下为 Car Audio 和 Phone Audio 配置下，如何查看当前音量的方法。

17.3.5.3.1 Car Audio

Car Audio 的音量大部分使用的是 DSP 中的 volume，按照如下步骤查看其音量：

1. 确认通路，首先确认当前播放的 APP 所使用的音频输出通路

- 1) 在 adb 中，使用 `dumpsys media.audio_policy`，查看当前播放的通路 address；如下示例

```

Outputs dump:
- Output 13 dump:
  Latency: 21
  Flags 00000002
  ID: 1
  Sampling rate: 96000
  Format: 00000003
  Channels: 00000003
  Devices: {type:0x10000000,0:bus0_media_out}
  Global active count: 1
  Product Strategy id: 27 - ActivityCount: 1, StopTime: 67911292467280,
  Volume Activities id: 4 - ActivityCount: 1, StopTime: 67911292467280, , Volume: nan, MuteCount: 00
  Volume Activities id: 10 - ActivityCount: 0, StopTime: 0, , Volume: nan, MuteCount: 01
  AudioTrack Clients:
    Client 1:
      - Port Id: 40 Session Id: 153 UID: 1002
      - Format: 00000005 Sampling rate: 44100 Channels: 00000003
      - Attributes: { Content type: AUDIO_CONTENT_TYPE_MUSIC Usage: AUDIO_USAGE_MEDIA Source: AUDIO_SOURCE_DEFAULT Flags: 0x0 Tags: }
      - Preferred Device Id: 00000000
      - State: Active
      - Stream: 3 flags: 00000000
      - Refcount: 1
      - DAP Primary Mix: 0xb400007181aeecf0
      - DAP Secondary Outputs:
  AudioTrack active (stream) clients:
    Client 1:
      - Port Id: 40 Session Id: 153 UID: 1002
      - Format: 00000005 Sampling rate: 44100 Channels: 00000003
      - Attributes: { Content type: AUDIO_CONTENT_TYPE_MUSIC Usage: AUDIO_USAGE_MEDIA Source: AUDIO_SOURCE_DEFAULT Flags: 0x0 Tags: }
      - Preferred Device Id: 00000000
      - State: Active
      - Stream: 3 flags: 00000000
    
```

图 17-21 当前 active 通路示例

- 2) 找到 address 之后，按照配置的 `audioCustomConfig.xml` 确定 ALSA 层节点，此处会决定音量使用的是 DSP 层音量还是 Audio HAL 层音量；

```
<outConfig>
  <!--this adsp path use i2s should setprop vendor.audiohal.playback.i2s.support 1-->
  <!--out sink="adsp">machine-stream-i2s-sc0</out-->
  <!--out sink="primary">machine-stream-dsp-dl-1</out-->
  <!--out sink="quick">machine-stream-dsp-dl-2</out-->
  <!--out sink="beep">machine-stream-dsp-dl-3</out-->
  <out sink="bus0_media_out">machine-stream-dsp-primary</out>
  <out sink="bus0_smix_out">machine-stream-dsp-secondary</out>
  <out sink="bus1_navigation_out">machine-stream-dsp-dl-1-1</out>
  <out sink="bus2_voice_command_out">machine-stream-dsp-dl-4</out>
  <out sink="bus3_call_ring_out">machine-stream-dsp-dl-3</out>
  <out sink="bus4_call_out">machine-stream-dsp-dl-2-1</out>
  <out sink="bus5_alternate_out">machine-stream-dsp-dl-1-1</out>
  <out sink="bus100_audio_zone_1">machine-stream-i2s-sc1</out>
  <out sink="bt-tx">machine-stream-pcm1</out>
</outConfig>
```

图 17-22 audioCustomConfig.xml 中节点映射示例

2. 之后，若通路使用的 DSP 音量，是根据下表中各条通路与其音量 Kcontrol 的对应关系，确认 Kcontrol 名称，然后采用 tinymix 方式，拿到当前的音量值（音量值的范围是 0~0x20000）；若通路使用的是 Audio HAL 层音量，暂不支持命令方式查看音量，若支持 ATC VolumePolicy，可从 VolumePolicy 的 dump 间接查看（音量范围 0-1，详见 17.3.3.6.1 Dump 当前 volumePolicy 状态小节）。

表 17-15 alsa device 与音量 Kcontrol 映射表

alsa device名称	音量Kcontrol
machine-stream-dsp-primary	Media_Master_Vol/Media_Pri_Vol，公版默认使用 Media_Master_Vol
machine-stream-dsp-secondary	Media_Master_Vol/Media_Sec_Vol，公版默认使用 Media_Master_Vol
machine-stream-dsp-tertiary	Media_Master_Vol/Media_Ter_Vol，公版默认使用 Media_Master_Vol
machine-stream-dsp-dl-1-1	ALSA_DSP_DL1_1_Volume
machine-stream-dsp-dl-1-2	ALSA_DSP_DL1_2_Volume
machine-stream-dsp-dl-1-3	ALSA_DSP_DL1_3_Volume
machine-stream-dsp-dl-1-4	ALSA_DSP_DL1_4_Volume
machine-stream-dsp-dl-1-5	ALSA_DSP_DL1_5_Volume
machine-stream-dsp-dl-1-6	ALSA_DSP_DL1_6_Volume
machine-stream-dsp-dl-1-7	ALSA_DSP_DL1_7_Volume
machine-stream-dsp-dl-1-8	ALSA_DSP_DL1_8_Volume
machine-stream-dsp-dl-2-1	ALSA_DSP_DL2_1_Volume
machine-stream-dsp-dl-2-2	ALSA_DSP_DL2_2_Volume
machine-stream-dsp-dl-2-3	ALSA_DSP_DL2_3_Volume
machine-stream-dsp-dl-2-4	ALSA_DSP_DL2_4_Volume
machine-stream-dsp-dl-3	Sys_DL3_Vol
machine-stream-dsp-dl-4	Sys_DL4_Vol
machine-stream-dsp-ecnr-dlin	DLIN_Vol

machine-stream-dsp-ll-dl

DSP_LL_DL_Volume

17.3.5.3.2 Phone Audio

当 PhoneAudio 配置时，当前 streamType 的音量可直接通过 `dumpsys media.audio_flinger` 查看。

Stream volumes in dB: 0:0, 1:0, 2:0, 3:0, 4:0, 5:0, 6:-inf, 7:0, 8:0, 9:0, 10:0, 11:0, 12:0, 13:0, 14:0, 15:0, 16:0, 17:0, 18:0, 19:-inf, 20:0, 21:0, 22:0

17.3.5.4 音量曲线如何配置

17.3.5.4.1 IVI 系统音源

17.3.5.4.1.1 Car Audio

Car Audio 配置时，其音量在如下 gain 配置中，minValueMB 表示该 devicePort 支持的最小音量（-50000 表示 -50dB），maxValueMB 表示该 devicePort 支持的最大音量，defaultValueMB 表示默认音量，stepValueMB 表示 UI 调节时，每次调节的步长（此处为 0.5dB）。

```
<devicePort tagName="bus0_media_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
address="bus0_media_out">
    <profile name="" format="AUDIO_FORMAT_PCM_16_BIT" samplingRates="48000"
channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
    <gains>
        <gain name="" mode="AUDIO_GAIN_MODE_JOINT" minValueMB="-5000"
maxValueMB="0" defaultValueMB="-1350" stepValueMB="50"/>
    </gains>
</devicePort>
```

若需要更细致的音量曲线，需要定制化开发，请联系 ATC 沟通。

17.3.5.4.1.2 Phone Audio

通过 AudioManager 设置的音量值都是 index 值（范围通常为 0~100），但最终在 AudioFlinger 进行混音前设置的音量值通常为 amplifier volume（范围为 0.0~1.0），所以在中间会有一个 index 值与 amplifier volume 的映射转换，音量曲线就定义了它们之间的映射关系。通常音量曲线由 3 段连续的折线组成，而这 3 段折线是通过定义 4 个点来进行描述。device/autochips/ac8x/audio_policy_config/audio_policy_volumes.xml 文件中定义了各个 streamtype 对应不同 device 的音量曲线，例如：

```
<volume stream="AUDIO_STREAM_VOICE_ALL" deviceCategory="DEVICE_CATEGORY_SPEAKER">
```



```
<point>0, -2400</point>
<point>33, -1600</point>
<point>66, -800</point>
<point>100, 0</point>
</volume>
```

属性 stream 表示调节的 streamtype，即音频类型；属性 deviceCategory 表示的是输出设备，输出设备 speaker 和 headset 的音量曲线可设置为不一样。属性 point 中的第一个坐标值为 index 值，第二个为 mb 值（100mb=1db），例如当 index 值为 0 时对应为 -2400mb，即 -24db。调用 VolumeCurve::VolIndexToDb 可将 index 根据音量曲线转换成 db 值，在该方法中首先将判断输入参数的 index 值落在哪个折线段上，再根据该折线段的比例关系计算出对应的 db 值，计算公式为 $y = (y_1 + (x - x_1) * ((y_2 - y_1) / (x_2 - x_1))) / 100$ ，其中 (x1,y1) 和 (x2,y2) 分别代表所在线段的起点和终点。得到 db 值后再转换成 amplifier volume 值，转换公式为 $\exp(db * 0.115129)$ 。另由 amplifier volume 转换成 db 值的公式为 $10 * \log(\text{ampl})$ 。

17.3.5.4.2 非 IVI 系统音源

目前非 IVI 系统音源的音量曲线，直接在 code 中定义，位于 VolumePolicyControl.cpp 中。如下：

```
//volume curve
//<reference name="DEFAULT_DEVICE_CATEGORY_SPEAKER_VOLUME_CURVE">
//    <!-- Default is Speaker Media Volume Curve -->
//        <point>1,-4950</point>
//        <point>33,-3350</point>
//        <point>66,-1700</point>
//        <point>100,0</point>
//    </reference>
static int AttenuationInMb[4] = {-4950, -3350, -1700, 0};
static int INDEX[4] = {1, 33, 66, 100};
```

图 17-23 非 IVI 系统音源音量曲线

若需要修改非 IVI 系统音源的音量曲线，可结合项目 Spec 修改 code。

17.3.5.5 非 IVI 系统音源音量默认值如何配置

非 IVI 系统音源的音量记录在 datazone 的 atc_volume.txt 中，其默认值可根据需要在 atc_volume.txt 中修改，编译出 datazone image 即可。

atc_volume.txt 文件路径：/build/projects/ivi_s_ac8025/datazone/atc_volume.xml

atc_volume.txt 文件格式: Key 值部分(包含“:”), 占据 12byte;value 部分, 占据 8byte。若修改 atc_volume 中的排列位置, 对应的解析 code 也需要对应修改!

音量名称含义参考“非 IVI 系统音源 name 标识与 streamType 对应关系表”, 音量值的范围为 0~0x20000, step 为 1。

```
safety_rvc :0x10000
safety_vba :0x10000
safety_chi :0x10000
adas       :0x10000
vba        :0x10000
rvc        :0x10000
vehicle    :0x10000
```

17.3.5.6 如何控制高频发声音源的压低时序

场景:

- Kanzi cluster 中的左转和右转灯提示音, 采用 alsa api 播放, 会频繁 start、stop, 按照 volumePolicy 的流程, 会频繁压低和恢复媒体声音, 导致整体听感不佳;
- 倒车场景, 只有在障碍物距离较近时, 才会有提示音发出, 提示音触发媒体声音的压低; 若交互的需求修改为倒车开启时, 媒体声音需要降低;

针对上述两种交互场景, 由于 ATC volumepolicy 对接的是音源的 start、stop 时机, 若交互中触发时机与音源的播放时机不对应, 均需要 APP 主动控制 volumepolicy 的 start、stop 时机。

在 Android 系统中, 提供如下接口对接某音源的 volumepolicy start、stop:

```
mAudioManager.setParameters("AudioStreamStartPolicy=xx");
mAudioManager.setParameters("AudioStreamStopPolicy=xx");
```

其中 xx 表示音源的 streamType 值。

在 ADAS 或者 Cluster 系统中, APP 直接调用 policyclient 端提供的 API 控制某音源的 volumepolicy start、stop; 可参考 Kanzi cluster。

17.4 音效部分

本章节介绍内置 DSP 提供的各类音效, 音效主要包含 Tone (Bass、Middle、Treble)、Loudness、Balance、SpeakerLayout、EQ、Reverb、Channel Delay、AutoSurround、Biquad IIR 等。

17.4.1 音效框架

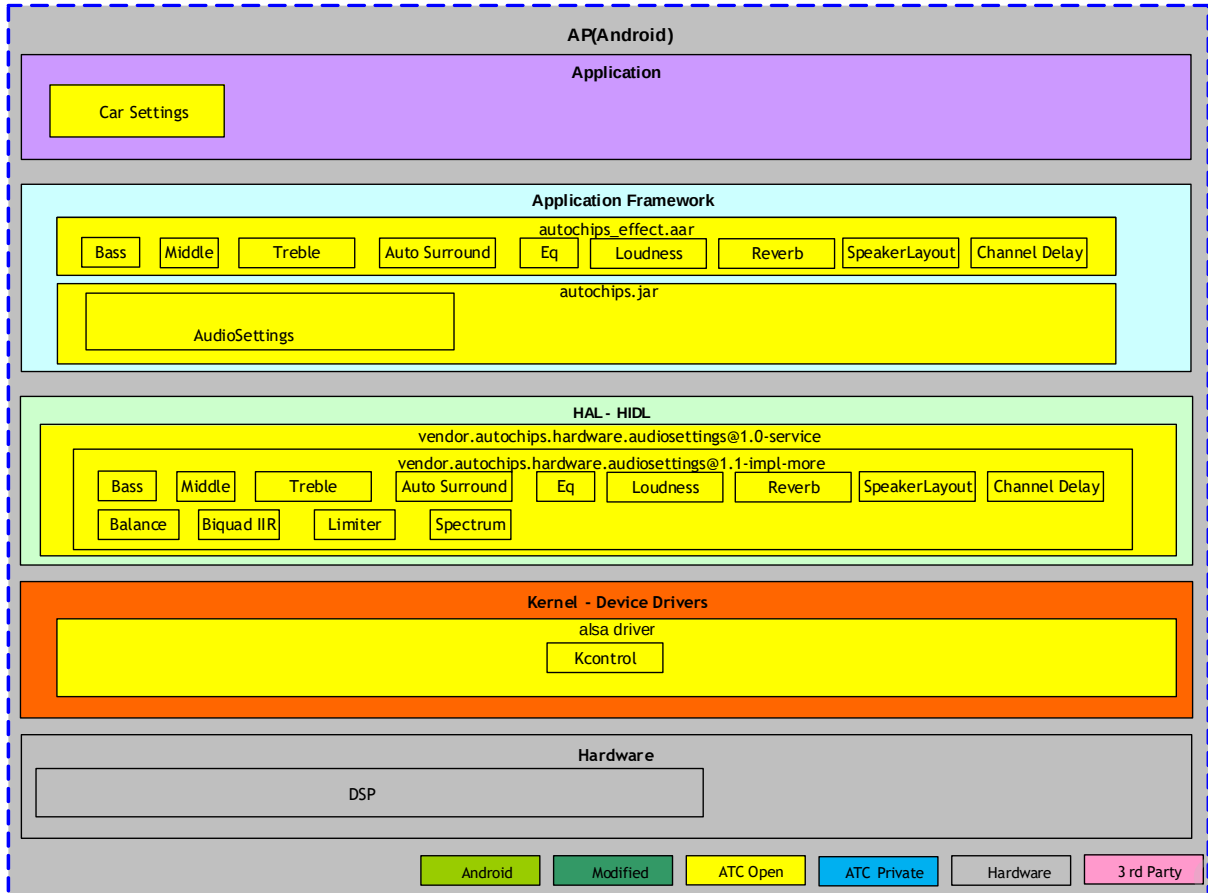


图 17-24 Android 音效框图

17.4.2 音效类型说明

17.4.2.1 Bass

17.4.2.1.1 Common Bass

低音加强（Bass Boost），对特定低频部分的声音进行加强，只针对四个低频段(60Hz, 80Hz, 100Hz, 200Hz)的增益值进行调节。

17.4.2.1.2 Custom Bass

自定义低音加强，用户可自定义低频频率、滤波器类型、Q 值和 gain 值。滤波器类型可选 peak 或者 low filter，中心频点支持 20-20KHz，Q 值支持 0.01-10，gain 值支持 -15.0dB-15.0dB。

17.4.2.2 Middle

17.4.2.2.1 Common Middle

中音加强（Medium Boost），对中频部分的声音进行加强，只针对四个中频段(500Hz, 1KHz, 1.5KHz, 2.5KHz)的增益值进行调节。

17.4.2.2.2 Custom Middle

自定义中音加强，用户可自定义低频频率、Q 值和 gain 值。中心频点支持 20-20KHz，Q 值支持 0.01-10，gain 值支持-15.0dB-15.0dB。

17.4.2.3 Treble

17.4.2.3.1 Common Treble

高音加强（Treble Boost），对高频部分的声音进行加强，只针对四个高频段(10KHz, 12.5KHz, 15KHz, 17.5KHz)的增益值进行调节。

17.4.2.3.2 Custom Treble

自定义高音加强，用户可自定义低频频率、滤波器类型、Q 值和 gain 值。滤波器类型可选 peak 或者 high filter，中心频点支持 20-20KHz，Q 值支持 0.01-10，gain 值支持-15.0dB-15.0dB。

17.4.2.4 Balance

平衡（Balance）是指把一边的声音减小（增大），而把另一边的声音增大（减小）。平衡做法是调节各个声道的权重(trim)值。

17.4.2.5 Loudness

17.4.2.5.1 Loudness V2

新版本的 Loudness 音效是一种基于音量的动态 boost 低音和高音的音效。

17.4.2.6 SpeakerLayout

支持 5.1 声道输出。其中 5.1 声道的前左(L)、前右(R)、后左(Ls)、后右(Rs)、中置(Center)可以独立配置输出高通 HPF 截止频率，范围为 0~300Hz, 以 1Hz 为单位。比如前左(L)的输出频率设成 100 表示其输出 100Hz~24KHz 的声音。

5.1 声道的重低音喇叭(就是所说的点 1 喇叭), 可以设置它的低通 LPF 截止频率, 范围为: 0~300Hz, 以 1Hz 为单位。比如设置成 100, 输出频率为 0~100Hz 的音频信号, 并将其他喇叭的 0~100Hz 的音频信号改道到重低音喇叭输出。

17.4.2.7 Equalizer

均衡器(Equalizer)简称 EQ, 通过增益和衰减音频信号的不同频段来达到一种理想的听觉效果。ATC 支持 2 类 EQ 音效, 一种为可以调节高达 31 个频段的 EQ 类型, 各个 band 的中心频率 (20Hz~20KHz 之间) 可以自定义设置, 一般与其他音效组合形成某自定义音效方案; 另一种为 5-10 段的 EQ 类型, 一般单独用作主机的 EQ 音效调节。

目前 demo 板中的音效 APP 采用第一种类型, 可以设置 10 段 EQ, 各个频段的中心频率分别为: 31.25Hz, 62.5Hz, 125Hz, 250Hz, 500Hz, 1KHz, 2KHz, 4KHz, 8KHz, 16KHz, 每个频段增益或衰减的幅度为 -15dB~15dB, 10 段 EQ 中心频率点的曲线如图所示。对于默认支持的 10 段 EQ, 其 preset (预设) 类别如下: Voice、Folk、Rock、Jazz、Pop、Dance、Classic、Treble Boost、BassBoost 一共九种效果, 当 APP 设置上述 preset 时, DSP 将根据此 preset 设置对应的 gain 值。当预设类型设置为 PRESET_FLAT 时, 每段的 gain 值采用 APP 设置的值。

一般而言, EQ 音效, 一旦确定 band 数, 每段的中心频率随之设置, 而 Q 值一般是根据 gain 值进行匹配的, 所以针对不同的 EQ 音效, 设置每段的 gain 值即可。

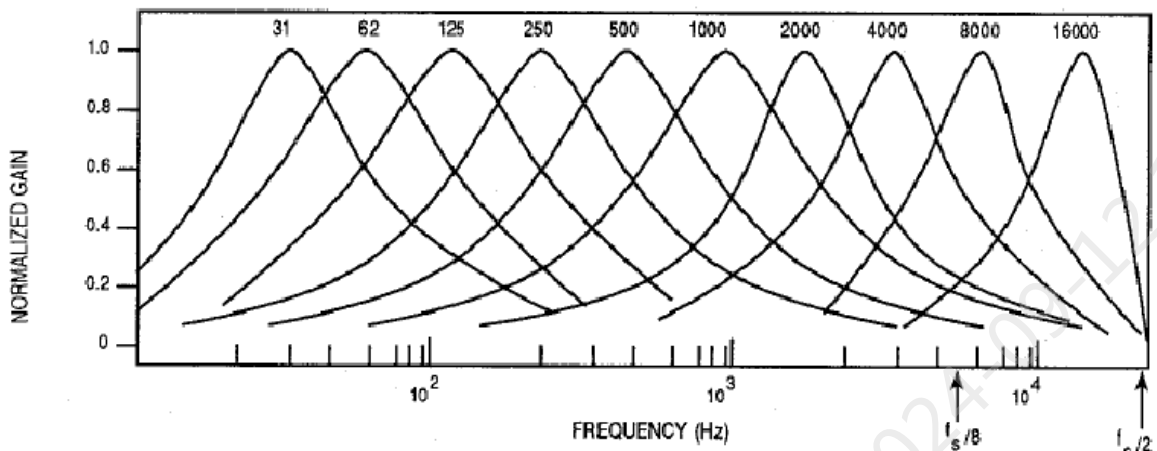


图 17-25 均衡器中心频率曲线

17.4.2.8 Reverb

混响 (Reverb) 是一种模拟声场的音效处理, 如我们在音乐厅听音乐时, 会觉得回音很大, 这就是混响的一种效果。它的作用就是用来模拟不同的环境音场, 会给听者带来大空间的感觉。

目前 AC8025 支持的混响的类别: 现场(Living), 大厅(Hall), 音乐厅(Concert), 洞穴(Cave), 浴室(Bathroom), 竞技场(Arena), 教堂(Church)。混响 (Reverb) 在 DSP 端通过延时、反馈、叠加实现, 混响的实现方式如下图所示。

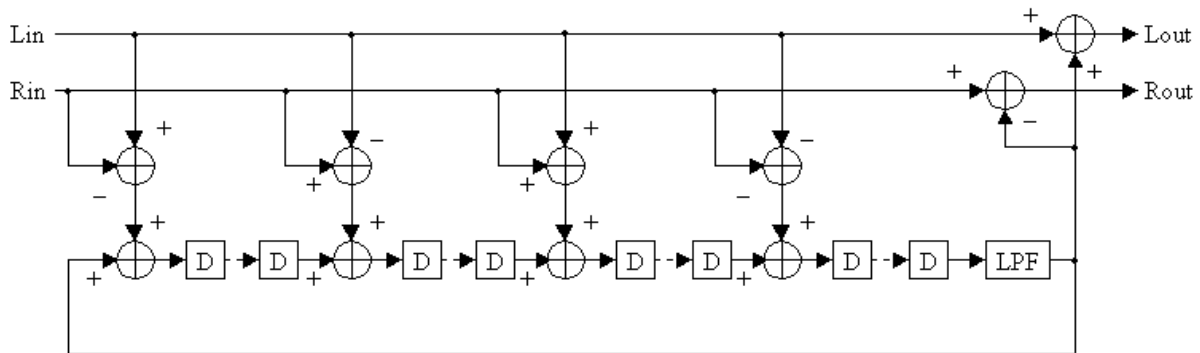


图 17-26 混响反馈叠加

17.4.2.9 ChannelDelay

因人的位置距离每个喇叭的位置都不是固定的，通道延迟就是为了优化各个声道输出的声音到达人耳时间，使音乐的听感更加和谐。

17.4.2.10 AutoSurround

AutoSurround 主要用于支持扩展环绕音。AutoSurround 音效不仅支持将 mono 或 stereo 音源扩展成 4 声道或 5 声道的输出，而且针对车机系统可以调节出一些特定的效果。通过增加的环绕声信号，可重现出原曲目中携带的环绕声信息，还原现场的混响声场及包围感。通过对比开关 AutoSurround 前后，通过增加的环绕声喇叭，中置喇叭，及重低音喇叭的输出，可以明显感觉到开启功能后，有较丰富的层次感，现场感和包围感。通过调整适当的环绕声道延迟，避免环绕声道影响听众对前置声道产生的声像的定位。

17.4.3 音效参数持久化实现方式

在各个 Android 版本中，APP 均无需保存用户断电之前设置的音效参数，音效参数由 AudioSettings Hal 层进行保存。其实现依赖 tinyxml2，在 APP 设置音效参数时，AudioSettings Hal 层调用 tinyxml2 提供的 write 接口将设定好的 key-value 值进行更新。当下一系统重新启动时，AudioSettings Hal Service 通过读取已经保存好的 xml 进行初始音效设置（即上一次用户设置的音效值）。

17.4.3.1 音效 xml

如下介绍的 xml 文件，在系统运行中的位置位于 data/vendor/audiohal 目录下；在代码结构中，该音效 xml 文件位于 source/android/device/autochips/vendor/ac8x/common/audio/audio_dsp_setting/effect 目录下。

17.4.3.1.1 atc_audio_effect.xml

atc_audio_effect.xml 主要负责 Bass、Middle、Treble、Loudness V1、Balance、EQ、Reverb、Channel Delay、Auto Surround 这些常见音效 feature 的保存。

如下为当前的 xml 形式：

```
<?xml version="1.0" encoding="UTF-8"?>
<effects>
  <Feature name="Bass" isSupported="1" enable="1" para="1" tag="">
    <paras para_0="2"/>
  </Feature>
  <Feature name="Middle" isSupported="1" enable="1" para="1" tag="">
    <paras para_0="-1"/>
  </Feature>
  <Feature name="Treble" isSupported="1" enable="1" para="1" tag="">
    <paras para_0="-5"/>
  </Feature>
  <Feature name="Balance" isSupported="1" enable="1" para="6" tag="">
    <paras para_0="6" para_1="5" para_2="-2" para_3="0" para_4="0" para_5="0"/>
  </Feature>
  <Feature name="SpeakerLayout" isSupported="1" enable="1" para="1" tag="">
    <paras para_0="51"/>
  </Feature>
  <Feature name="Eq" isSupported="1" enable="1" para="13" tag="">
    <paras para_0="1" para_1="10" para_2="0" para_3="-12" para_4="-2" para_5="-9"
para_6="-15" para_7="-3" para_8="-10" para_9="-2" para_10="-14" para_11="4" para_12="14"/>
  </Feature>
  <Feature name="Reverb" isSupported="1" enable="1" para="1" tag="">
    <paras para_0="2"/>
  </Feature>
  <Feature name="ChannelDelay" isSupported="1" enable="1" para="6" tag="">
    <paras para_0="68" para_1="48" para_2="61" para_3="42" para_4="15" para_5="0"/>
  </Feature>
  <Feature name="AutoSurround" isSupported="1" enable="1" para="2" tag="">
    <paras para_0="1" para_1="1"/>
  </Feature>
</effects>
```

```
</Feature>
</effects>
```



说明：

atc_audio_effect.xml 支持两种版本的 Bass、Middle、Treble 音效配置，如下为 V2 版本的 Bass、Middle、Treble 音效的配置示例：

```
<Feature name="Bass" isSupported="1" enable="1" para="7" tag="v2">
    <!-- band id; enable; type; frequency; Q ; gain; reserved-->
    <paras para_0="2" para_1="0" para_2="1" para_3="50" para_4="1" para_5="0"
para_6="0"/>
</Feature>

<Feature name="Middle" isSupported="1" enable="1" para="7" tag="v2">
    <paras para_0="3" para_1="0" para_2="1" para_3="1000" para_4="1" para_5="0"
para_6="0"/>
</Feature>

<Feature name="Treble" isSupported="1" enable="1" para="7" tag="v2">
    <paras para_0="4" para_1="0" para_2="1" para_3="15000" para_4="1" para_5="0"
para_6="0"/>
</Feature>
```

参数说明：

- band id，一般固定为 bass 为 2，middle 为 3，treble 为 4；
- enable，0 表示 disable，2 表示 enable；
- type；0~8 // 0:none; 1:peak; 2:low pass; 3:high pass; 4:low shelf; 5:high shelf; 6:band pass; 7:norch; 8:all pass;
- frequency; 20~20KHz，step 1Hz;
- Q 值，在 xml 中 1-1000，含义 0.01-10;
- gain 值，在 xml 中-150 到 150，含义 -15dB 到 15dB，step 0.1dB。

17.4.3.1.2 loudness.xml

loudness.xml 为 Loudness V2 版本的音效参数文件，支持自定义音量范围和音效参数。如下为 Music 音量范围为 0-32 的 loudness 参数表，在每个音量 index 下，分别可设置 bass 和 treble 的滤波器类型、中心频率、Q 值和 gain 值。音量可对接 AudioFlinger 音量或者 DSP 的音量。

各字段参数范围含义如下：

- type; 0~8 // 0:none;1:peak; 2:low pass; 3:high pass; 4:low shelf; 5:high shelf; 6:band pass; 7:norch; 8:all pass;
- frequency; 20~20KHz, step 1Hz;
- Q 值, 在 xml 中 1-1000, 含义 0.01-10;
- gain 值, 在 xml 中-150 到 150, 含义 -15dB 到 15dB, step 0.1dB

```
<?xml version="1.0" encoding="UTF-8"?>
<Loudness indexMax="32" enable="1" index="1">
  <Bass>
    <bass index="0" type="1" frequency="400" qfactor="1" gain="0"/>
    <bass index="1" type="1" frequency="400" qfactor="1" gain="10"/>
    <bass index="2" type="1" frequency="400" qfactor="1" gain="10"/>
    <bass index="3" type="1" frequency="400" qfactor="1" gain="10"/>
    <bass index="4" type="1" frequency="400" qfactor="1" gain="10"/>
    <bass index="5" type="1" frequency="400" qfactor="1" gain="10"/>
    <bass index="6" type="1" frequency="400" qfactor="1" gain="10"/>
    <bass index="7" type="1" frequency="400" qfactor="1" gain="9.9"/>
    <bass index="8" type="1" frequency="400" qfactor="1" gain="9.7"/>
    <bass index="9" type="1" frequency="400" qfactor="1" gain="9.3"/>
    <bass index="10" type="1" frequency="400" qfactor="1" gain="8.8"/>
    <bass index="11" type="1" frequency="400" qfactor="1" gain="8.3"/>
    <bass index="12" type="1" frequency="400" qfactor="1" gain="7.7"/>
    <bass index="13" type="1" frequency="400" qfactor="1" gain="7.0"/>
    <bass index="14" type="1" frequency="400" qfactor="1" gain="6.0"/>
    <bass index="15" type="1" frequency="400" qfactor="1" gain="5.0"/>
    <bass index="16" type="1" frequency="400" qfactor="1" gain="4.0"/>
    <bass index="17" type="1" frequency="400" qfactor="1" gain="3.0"/>
    <bass index="18" type="1" frequency="400" qfactor="1" gain="2.2"/>
    <bass index="19" type="1" frequency="400" qfactor="1" gain="1.3"/>
    <bass index="20" type="1" frequency="400" qfactor="1" gain="0.8"/>
    <bass index="21" type="1" frequency="400" qfactor="1" gain="0.3"/>
    <bass index="22" type="1" frequency="400" qfactor="1" gain="0"/>
    <bass index="23" type="1" frequency="400" qfactor="1" gain="0"/>
```

```

<bass index="24" type="1" frequency="400" qfactor="1" gain="0"/>
<bass index="25" type="1" frequency="400" qfactor="1" gain="0"/>
<bass index="26" type="1" frequency="400" qfactor="1" gain="0"/>
<bass index="27" type="1" frequency="400" qfactor="1" gain="0"/>
<bass index="28" type="1" frequency="400" qfactor="1" gain="0"/>
<bass index="29" type="1" frequency="400" qfactor="1" gain="0"/>
<bass index="30" type="1" frequency="400" qfactor="1" gain="0"/>
<bass index="31" type="1" frequency="400" qfactor="1" gain="0"/>
<bass index="32" type="1" frequency="400" qfactor="1" gain="0"/>
</Bass>
<Treble>
  <treble index="0" type="1" frequency="10000" qfactor="1" gain="0"/>
  <treble index="1" type="1" frequency="10000" qfactor="1" gain="4"/>
  <treble index="2" type="1" frequency="10000" qfactor="1" gain="4"/>
  <treble index="3" type="1" frequency="10000" qfactor="1" gain="4"/>
  <treble index="4" type="1" frequency="10000" qfactor="1" gain="4"/>
  <treble index="5" type="1" frequency="10000" qfactor="1" gain="4"/>
  <treble index="6" type="1" frequency="10000" qfactor="1" gain="4"/>
  <treble index="7" type="1" frequency="10000" qfactor="1" gain="3.9"/>
  <treble index="8" type="1" frequency="10000" qfactor="1" gain="3.8"/>
  <treble index="9" type="1" frequency="10000" qfactor="1" gain="3.6"/>
  <treble index="10" type="1" frequency="10000" qfactor="1" gain="3.4"/>
  <treble index="11" type="1" frequency="10000" qfactor="1" gain="3.1"/>
  <treble index="12" type="1" frequency="10000" qfactor="1" gain="2.8"/>
  <treble index="13" type="1" frequency="10000" qfactor="1" gain="2.5"/>
  <treble index="14" type="1" frequency="10000" qfactor="1" gain="2.1"/>
  <treble index="15" type="1" frequency="10000" qfactor="1" gain="1.7"/>
  <treble index="16" type="1" frequency="10000" qfactor="1" gain="1.3"/>
  <treble index="17" type="1" frequency="10000" qfactor="1" gain="0.9"/>
  <treble index="18" type="1" frequency="10000" qfactor="1" gain="0.7"/>
  <treble index="19" type="1" frequency="10000" qfactor="1" gain="0.4"/>
  <treble index="20" type="1" frequency="10000" qfactor="1" gain="0.2"/>
  <treble index="21" type="1" frequency="10000" qfactor="1" gain="0.1"/>

```

```
<treble index="22" type="1" frequency="10000" qfactor="1" gain="0"/>
<treble index="23" type="1" frequency="10000" qfactor="1" gain="0"/>
<treble index="24" type="1" frequency="10000" qfactor="1" gain="0"/>
<treble index="25" type="1" frequency="10000" qfactor="1" gain="0"/>
<treble index="26" type="1" frequency="10000" qfactor="1" gain="0"/>
<treble index="27" type="1" frequency="10000" qfactor="1" gain="0"/>
<treble index="28" type="1" frequency="10000" qfactor="1" gain="0"/>
<treble index="29" type="1" frequency="10000" qfactor="1" gain="0"/>
<treble index="30" type="1" frequency="10000" qfactor="1" gain="0"/>
<treble index="31" type="1" frequency="10000" qfactor="1" gain="0"/>
<treble index="32" type="1" frequency="10000" qfactor="1" gain="0"/>

</Treble>

</Loudness>
```

17.4.3.1.3 eq.xml

eq.xml 是 5~10 段 EQ 的配置和保存文件，根据客户的一般使用方式，设计如下的 xml：

```
<?xml version="1.0" encoding="UTF-8"?>

<!-- band 填写使用的 band 数（支持 5~10 段），preset 为当前的 preset 类型，enable 为当前的开关状态 -->
<Eq band="10" preset="voice" enable="0">

<!--frequency 对应填写各个 band 的中心频率，范围 20-20K Hz-->
<frequency para0="640" para1="1400" para2="4000" para3="9800" para4="13000" para5="320"
para6="1000" para7="2000" para8="6000" para9="17000"/>

<!--q 对应填写各个 band 的 Q 值 范围 0.01-10，step 0.01-->
<q para0="1" para1="1" para2="1" para3="1" para4="1" para5="1" para6="1" para7="1"
para8="1" para9="1"/>

<!--preset 对应填写不同 preset 种类，对应的各个 band 的 gain 值，范围-15 到 15dB，step 1dB -->
<preset preset_mode="voice" para0="0" para1="2" para2="1" para3="2" para4="0" para5="0"
para6="2" para7="1" para8="2" para9="4"/>

<preset preset_mode="classic" para0="0" para1="2" para2="1" para3="2" para4="0" para5="0"
para6="2" para7="1" para8="2" para9="3"/>

<preset preset_mode="folk" para0="0" para1="2" para2="1" para3="2" para4="0" para5="0"
para6="2" para7="1" para8="2" para9="2"/>
```

```
<preset preset_mode="user" para0="0" para1="2" para2="1" para3="2" para4="0" para5="0"
para6="2" para7="1" para8="2" para9="1"/>
</Eq>
```



说明：

- preset 时，preset_mode 的名称用户可以自定义，framework 和 HAL 层对于 preset_mode 是通过 index 去识别的，例这里 voice 是 index 0，framework 层只需要告知 HAL 层，设置 preset 为 index 0，HAL 层即可将对应的 gain 值设置到 DSP
- 除 user mode 外，其他 mode 类型的 gain 值只可读，不能写

17.4.4 Java 接口

17.4.4.1 代码结构及文件描述

表 17-16 音效代码结构和文件描述

文件名	文件路径	文件描述
AudioSettings.java	source/android/vendor/autochips/proprietary/frameworks/base/audiosettings/java/com/autochips/audiosettings	DSP 音效功能接口
Bass.java Middle.java Treble.java Loudness.java BalanceFader.java EQ.java SpeakerLayout.java ChannelDelay.java	source/android/vendor/autochips/proprietary/frameworks/base/atcaudioeffect/java/com/autochips/atcaudioeffect	常用音效 feature 的集成接口

17.4.4.2 常用音效封装接口

这部分接口对接在 AudioSettings 之上，将常用的音效 feature 进行了封装，可减少 APP 的逻辑部分。该接口采用 autochips_effect.aar 包的形式提供。下面对该 aar 包中的几个类分别作接口说明。

17.4.4.2.1 Bass 类

表 17-17 Bass 类接口说明

Public方法	说明
----------	----

Bass getInstance()	获取Bass音效单例对象
int[] getGains()	获取当前Bass音效支持的所有gain值
boolean setGain(int gainIndex)	设置当前Bass音效的gain值对应的index
int getCurrentGainIndex()	获取当前Bass音效的gain值对应的index

17.4.4.2.2 Loudness 类

表 17-18 Loudness 类接口说明

Public方法	说明
Loudness getInstance()	获取Loudness音效单例对象
void setEnable(boolean enable)	设置Loudness音效开关状态
boolean isEnable()	获取当前Loudness音效开关状态

17.4.4.2.3 Balance 类

表 17-19 Balance 类接口说明

Public方法	说明
BalanceFader getInstance()	获取BalanceFader音效单例对象
int[] getCount()	获取当前BalanceFader音效支持的横纵坐标范围
void setIndex(int x, int y)	设置当前BalanceFader音效的横纵坐标
int[] getIndex()	获取当前BalanceFader音效的横纵坐标
int[] getGains()	获取当前前左、前右、后左、后右四个声道的Gain值
void registerListener(GainChangedListener listener)	注册gain值监听listener，该listener中实现onGainChanged(int[] gains)方法
void unregisterListener(GainChangedListener listener)	注销之前已经注册过的gain值监听listener

17.4.4.2.4 SpeakerLayout 类

表 17-20 SpeakerLayout 类接口说明

Public方法	说明
SpeakerLayout getInstance()	获取SpeakerLayout音效单例对象
void reset()	将当前SpeakerLayout音效重置为默认状态，即默认输出的声道数。 8015R，默认输出的声道为四声道。

boolean setSpeakerOn(int channelLayout, boolean on)	将channelLayout对应的声道设置为on或者off状态。详细见下方接口说明
boolean isSpeakerOn(int channelLayout)	获取channelLayout对应声道的输出状态，true为输出，false为不输出

17.4.4.2.4.1 setSpeakerOn 接口

【描述】

将 channelLayout 对应的声道设置为 on 或者 off 状态

【语法】

```
public boolean setSpeakerOn(int channelLayout, boolean on)
```

【返回值】

- true, 该声道成功设置状态
- false, 该声道状态设置失败

【参数】

- channelLayout, 按照如下声道参数(位于 AtcAudioEffectManager.java 中)进行配置

```
public static final int CHANNEL_FRONT_LEFT;
public static final int CHANNEL_FRONT_RIGHT;
public static final int CHANNEL_REAR_LEFT;
public static final int CHANNEL_REAR_RIGHT;
public static final int CHANNEL_CENTER;
public static final int CHANNEL_SUB;
```

- on, true 表示该声道允许输出；false 表示该声道关闭输出

【注意】

- 该接口设置后，最长 20ms 即生效；
- 一般对于 5.1 声道的电影原声，人声位于中置声道；若当前配置的 6 声道输出，人声位于中置喇叭，若当前配置的四声道，人声混在前左前右喇叭。

17.4.4.2.5 AutoSurround 类

表 17-21 AutoSurround 类接口说明

Public方法	说明
----------	----

AutoSurround getInstance()	获取AutoSurround音效单例对象
string[] getModes(Context context)	获取AutoSurround音效支持的mode，一般分为单声道、立体声
boolean setMode(int modeIndex)	设置AutoSurround音效的modeIndex
int getCurrentModeIndex()	获取当前AutoSurround音效的ModeIndex
boolean setEnable(boolean enable)	设置AutoSurround音效开关状态，true表示开，false表示关
boolean isEnable()	获取AutoSurround音效的开关状态

17.4.4.2.6 ChannelDelay 类

表 17-22 ChannelDelay 类接口说明

Public方法	说明
ChannelDelay getInstance()	获取ChannelDelay音效单例对象
int getChannelsMask()	获取支持Channel Delay的声道Mask
int[] getDelayRange()	获取ChannelDelay的最小值、step、最大值
boolean setDelayIndex(int channel, int delayIndex)	设置channel对应的声道的delayIndex值
int getCurrentDelayIndex(int channel)	获取channel对应的声道当前的DelayIndex

17.4.4.2.7 EQ 类

表 17-23 EQ 类接口说明

Public方法	说明
EQ getInstance()	获取EQ音效单例对象
int getBandsCount()	获取当前设置的band数，公版默认为10段
String[] getFrequencyOfBands()	获取默认10段EQ的每段中心频率
int[] getGainRange()	获取EQ音效支持的每段gain值调节范围，返回可调节的所有gain值列表
String[] getPresets(Context context)	获取EQ音效支持的所有预置类型，返回对应的预置类型名称
boolean setGainIndexs(int[] gainsIndex)	设置10段的gain值
boolean setGainIndex(int bandIndex, int gainIndex)	设置第bandIndex段的gainIndex值
int getCurrentPresetIndex()	获取当前EQ的preset类型
boolean setPresetIndex(int presetIndex)	设置EQ音效的preset类型

int[] getCurrentGainsIndex()	获取当前10段EQ的所有gain值index
int getCurrentGainIndex(int bandIndex)	获取第bandIndex段EQ的gain值index
boolean setEnable(boolean enable)	设置EQ音效的开关状态
boolean isEnable()	获取EQ音效的开关状态
boolean reset()	重置EQ音效，即gain值全部恢复为0
void registerListener(GainChangedListener listener)	注册gain值监听listener，listener中实现onGainChanged(int[] gainsOfBands)方法
void unRegisterListener(GainChangedListener listener)	注销已经注册过的gain值监听listener

17.4.4.3 AudioSettings 接口

所有音效 java 接口均集成至 autochips.jar 中，类名为 AudioSettings.java。用户可以通过该接口进行各类音效的封装，对接 APP 行为。

表 17-24 AudioSettings 类

Public方法	说明
int SetValue(int effect_id, int[] variable, int variable_size)	设置音效参数接口
int[] GetValue(int effect_id, int variable_size)	获取音效参数接口

17.4.4.3.1 SetValue

【描述】

通过设置不同的 effect_id 进行不同音效 feature 的参数设置。

【语法】

```
public static int SetValue(int effect_id, int[] variable, int variable_size)
```

【返回值】

- 0 : success
- -1: fail

【参数】

- effect_id: 表示需要设置的音效 feature 标识
- variable: 需要设置的音效参数
- variable_size: 需要设置的音效参数个数

【注意】

请明确音效 feature 的 id 及其参数表示后，通过该接口设置音效参数。

17.4.4.3.2 GetValue

【描述】

通过不同的 effect_id 获取不同音效 feature 的当前参数。

【语法】

```
public static int[] GetValue(int effect_id, int variable_size)
```

【返回值】

当前音效 feature 的所有参数

【参数】

- effect_id: 表示需要获取参数的音效 feature 标识
- variable_size: 需要获取的音效参数个数

【注意】

请明确音效 feature 的 id 及其参数顺序，处理获取的返回值。

ATC_AUDIO_SETTING_GET_SPECTRUM

【示例 code】

```
int[] spectrum = AudioSettings.GetValue(ATC_AUDIO_SETTING_GET_SPECTRUM, mode); //mode 为 fft 的点数
```

17.4.4.3.3 effect_id 对应参数说明

表 17-25 Effect ID 列举

Effect ID	说明
ATC_AUDIO_SETTING_SET_SPEAKER_LAYOUT	输出声道配置
ATC_AUDIO_SETTING_SET_EQ	31 段 EQ 音效设置
ATC_AUDIO_SETTING_SET_BASS	Bass 音效设置
ATC_AUDIO_SETTING_SET_MID	Middle 音效设置
ATC_AUDIO_SETTING_SET_TRE	Treble 音效设置
ATC_AUDIO_SETTING_SET_REVERB	Reverb 音效设置
ATC_AUDIO_SETTING_SET_UPMIX	Upmix 音效设置
ATC_AUDIO_SETTING_SET_AUTOSURROUND	AutoSurround 音效设置（ATC 自研）
ATC_AUDIO_SETTING_SET_BALANCE	BalanceFader 音效设置
ATC_AUDIO_SETTING_SET_CHANNEL_DELAY	ChannelDelay 音效设置

Effect ID	说明
ATC_AUDIO_SETTING_SET_CSAUTO	CsAuto 音效设置（第三方合作音效）
ATC_AUDIO_SETTING_SET_SPECTRUM_MODE	设置 Spectrum mode
ATC_AUDIO_SETTING_GET_SPECTRUM	获取当前的实时输出频谱
ATC_AUDIO_SETTING_LOUDNESS_V2_ENABLE	Loudness 音效设置（可自定义 Bass、Treble boost）
ATC_AUDIO_SETTING_LOUDNESS_V2_INDEX	Loudness 音效 index 设置
ATC_AUDIO_SETTING_EQ_V2	EQ V2 音效设置（可自定义 5/10 段 EQ）

▪ ATC_AUDIO_SETTING_SET_SPEAKER_LAYOUT

【variable】

variable[0]: spk_layout 配置打开 5.1 声道的哪些声道

public final static int AUDIO_CHANNEL_OUT_FRONT_LEFT = 0x1;

public final static int AUDIO_CHANNEL_OUT_FRONT_RIGHT=0x2;

public final static int AUDIO_CHANNEL_OUT_FRONT_CENTER=0x4;

public final static int AUDIO_CHANNEL_OUT_LOW_FREQUENCT=0x8;

public final static int AUDIO_CHANNEL_OUT_BACK_LEFT=0x10;

public final static int AUDIO_CHANNEL_OUT_BACK_RIGHT=0x20;

variable[1~4]: front_size, center_size, rear_size, sub_size

配置各声道的截止频率，范围：0~300Hz，四个 size 都可以设置不同的值。

variable[5]: sub_boost, boost gain, 0: -5dB; 1: 0dB。

variable[6]: sub_force_out, 是否打开 sub channel，默认需要打开；

【示例 code】

```
int[] mSpLayoutValue = new int[7]; //variable_size = 7;
mSpLayoutValue[0] = (AUDIO_CHANNEL_OUT_FRONT_LEFT | AUDIO_CHANNEL_OUT_FRONT_RIGHT
| AUDIO_CHANNEL_OUT_FRONT_CENTER |
AUDIO_CHANNEL_OUT_LOW_FREQUENCT | AUDIO_CHANNEL_OUT_BACK_LEFT |
AUDIO_CHANNEL_OUT_BACK_RIGHT); //all channel out
mSpLayoutValue[1] = 0; // front_size : 0Hz
mSpLayoutValue[2] = 0; // center_size : 0Hz
mSpLayoutValue[3] = 0; // rear_size : 0Hz
mSpLayoutValue[4] = 100; // sub_size : 100Hz
```

```
mSpLayoutValue[5] = 0; // 公版默认-5dB
mSpLayoutValue[6] = 1; //open sub channel
SetValue(ATC_AUDIO_SETTING_SET_SPEAKER_LAYOUT, mSpLayoutValue, 7);
```

▪ ATC_AUDIO_SETTING_LOUDNESS_V2_ENABLE

【variable】

variable[0]: enable, 1: enable; 0: disable

【示例 code】

```
int[] enable = new int [1];
enable[0] = 1;
SetValue(ATC_AUDIO_SETTING_LOUDNESS_V2_ENABLE, enable,1);
```

▪ ATC_AUDIO_SETTING_LOUDNESS_V2_INDEX

【variable】

variable[0]: index

【示例 code】

```
//设置当前 Music 音量 index
int[] index = new int [1];
index[0] = xx;
SetValue(ATC_AUDIO_SETTING_LOUDNESS_V2_ INDEX, index,1);
//获取当前 Loudness 音效的 index 最大值
int[] indexMax = AudioSettings.GetValue(ATC_AUDIO_SETTING_LOUDNESS_V2_ INDEX, 1);
```

【备注】

index 的设置已经对接到 AudioFlinger 的 setParameters 方法中，客户在实现自定义 Loudness 音效时可选 AudioManager 的 setParameter API 设置 key 值 “MusicIndex”、value 值为当前媒体的 UI 值进行对接。

▪ ATC_AUDIO_SETTING_SET_EQ_V2

【variable】

variable[0]: u4_enable, 1: enable; 0: disable

variable[1]: u4_preset, preset 类型。每个 preset 类型的参数配置客户自定义配置

variable[2~2+band]: 用户自定义设置时, 每个 band 的 GAIN 值, gain 值范围-15dB~15dB

【示例 code】

```
//eq.xml 如 eq.xml
//打开 EQ, 并配置 preset 为 0 的 EQ 参数
int[] eqParam= new int[2];
eqParam [0] = 1; //enable
eqParam[1] = 0;//preset index 为 0, 即配置 voice 的参数
SetValue(ATC_AUDIO_SETTING_SET_EQ_V2, eqParam,2);
//关闭 EQ
int[] eqParam= new int[1];
eqParam [0] = 0; //disable
SetValue(ATC_AUDIO_SETTING_SET_EQ_V2, eqParam,1);
//设置自定义 EQ 参数
int[] eqParam= new int[12];
eqParam [0] = 1; //enable
eqParam [1] = 3; //user preset mode
eqParam [2] = 3; //第一段 band 的 gain 值为 3dB
...
eqParam [11] = -4; //第十段 band 的 gain 值为-4dB
SetValue(ATC_AUDIO_SETTING_SET_EQ_V2, eqParam,12);
```

▪ ATC_AUDIO_SETTING_SET_BASS

【variable 方式 1】

即 common Bass 的音效配置方式。

variable[0]: u4_enable, 1: enable; 0: disable

variable[1]: u4_gain, 低音 gain 值, 范围: -15dB~15dB, 即 G_M15DB~G_15DB

variable[2]: u4_CFreq, 中心频率:

```
public final static int BB_CF_60HZ = 0;
```

```
public final static int BB_CF_80HZ = 1;
```

```
public final static int BB_CF_100HZ = 2;

public final static int BB_CF_200HZ = 3;

variable[3]: u4_QFactor, Q 值:
```

```
public final static int BB_Q_1_00 = 0;

public final static int BB_Q_1_25 = 1;

public final static int BB_Q_1_50 = 2;

public final static int BB_Q_2_00 = 3;
```

【示例 code1】

```
int[] mBassValue = new int[4];
mBassValue[0] = 1; //enable
mBassValue[1] = G_15DB; // gain
mBassValue[2] = BB_CF_100HZ; //公版默认设置中心频率为 100Hz，客户可定制
mBassValue[3] = BB_Q_1_50; //公版默认设置 Q 值为 1.5，客户可定制
setValue(ATC_AUDIO_SETTING_SET_BASS, mBassValue, 4);
```

【variable 方式 2】

即 custom Bass 的音效配置方式。

```
variable[0]: u4_bandId, 固定为 2

variable[1]: u4_enable, 2 表示 enable, 0 表示 disable

variable[2]: u4_FilterType, 1:peak;4:low shelf;

variable[3]: u4_CFreq, 中心频率: 20-20K

variable[4]: u4_QFactor, Q 值, 1~1000, 表示 0.01~10

variable[5]: u4_Gain, gain 值, -150~150, 表示-15.0dB~15.0dB

variable[6]: reserved
```

【示例 code2】

```
int[] mBassValue = new int[7];
mBassValue[0] = 2; //band id
mBassValue[1] = 2; // enable
mBassValue[2] = 1; //peak filter
mBassValue[3] = 300; //300Hz
mBassValue[4] = 100; // Q 值为 1
```

```
mBassValue[5] = 13; //1.3dB
mBassValue[6] = 0; //reserved
setValue(ATC_AUDIO_SETTING_SET_BASS, mBassValue, 7);
```

▪ ATC_AUDIO_SETTING_SET_MID

【variable 方式 1】

即 common Middle 的音效配置方式。

variable[0]: u4_enable, 1: enable; 0: disable

variable[1]: u4_gain, 中音 gain 值, 范围: -15dB~15dB, 即 G_M15DB~G_15DB

variable[2]: u4_CFreq, 中心频率:

```
public final static int MB_CF_500HZ = 0;
public final static int MB_CF_1KHZ = 1;
public final static int MB_CF_1K5HZ = 2;
public final static int MB_CF_2K5HZ = 3;
```

variable[3]: u4_QFactor, Q 值:

```
public final static int MB_Q_0_50 = 0;
public final static int MB_Q_0_75 = 1;
public final static int MB_Q_1_00 = 2;
public final static int MB_Q_1_25 = 3;
```

【示例 code1】

```
int[] mMidValue = new int[4];
mMidValue[0] = 1; //enable
mMidValue[1] = G_15DB; // gain
mMidValue[2] = MB_CF_1K5HZ; //公版默认设置中心频率为 1.5KHz, 客户可定制
mMidValue[3] = MB_Q_1_00; //公版默认设置 Q 值为 1.0, 客户可定制
setValue(ATC_AUDIO_SETTING_SET_MID, mMidValue, 4);
```

【variable 方式 2】

即 custom Middle 的音效配置方式。

variable[0]: u4_bandId, 固定为 3

variable[1]: u4_enable, 2 表示 enable, 0 表示 disable

variable[2]: u4_FilterType, 1:peak;

variable[3]: u4_CFreq, 中心频率: 20-20K

variable[4]: u4_QFactor, Q 值, 1~1000, 表示 0.01~10

variable[5]: u4_Gain, gain 值, -150~150, 表示-15.0dB~15.0dB

variable[6]: reserved

【示例 code】

```
int[] mMidValue = new int[7];
mMidValue [0] = 3; //band id
mMidValue [1] = 2; // enable
mMidValue [2] = 1; //peak filter
mMidValue [3] = 1300; //1300Hz
mMidValue [4] = 100; // Q 值为 1
mMidValue [5] = 13; //1.3dB
mMidValue [6] = 0; //reserved
setValue(ATC_AUDIO_SETTING_SET_MID, mMidValue, 7);
```

■ ATC_AUDIO_SETTING_SET_TRE

【variable 方式 1】

即 common Treble 的音效配置方式。

variable[0]: u4_enable, 1: enable; 0: disable

variable[1]: u4_gain, 中音 gain 值, 范围: -15dB~15dB, 即 G_M15DB~G_15DB

variable[2]: u4_CFreq, 中心频率:

```
public final static int TB_CF_10KHZ = 0;
public final static int TB_CF_12K5HZ = 1;
public final static int TB_CF_15KHZ = 2;
public final static int TB_CF_17K5HZ = 3;
```

variable[3]: u4_QFactor, Q 值:

```
public final static int TB_Q_0_50 = 0;
public final static int TB_Q_0_75 = 1;
public final static int TB_Q_1_00 = 2;
```

```
public final static int TB_Q_1_25 = 3;
```

【示例 code1】

```
int[] mTreValue = new int[4];
mTreValue[0] = 1; //enable
mTreValue[1] = G_15DB; // gain
mTreValue[2] = TB_CF_15KHZ; //公版默认设置中心频率为 15KHz，客户可定制
mTreValue[3] = TB_Q_1_00; //公版默认设置 Q 值为 1.0，客户可定制
setValue(ATC_AUDIO_SETTING_SET_TRE, mTreValue, 4);
```

【variable 方式 2】

即 custom Treble 的音效配置方式。

variable[0]: u4_bandId, 固定为 4

variable[1]: u4_enable, 2 表示 enable, 0 表示 disable

variable[2]: u4_FilterType, 1:peak; 5:high shelf

variable[3]: u4_CFreq, 中心频率: 20-20K

variable[4]: u4_QFactor, Q 值, 1~1000, 表示 0.01~10

variable[5]: u4_Gain, gain 值, -150~150, 表示-15.0dB~15.0dB

variable[6]: reserved

【示例 code】

```
int[] mTreValue = new int[7];
mTreValue [0] = 4; //band id
mTreValue [1] = 2; // enable
mTreValue [2] = 1; //peak filter
mTreValue [3] = 1300; //1300Hz
mTreValue [4] = 100; // Q 值为 1
mTreValue [5] = 13; //1.3dB
mTreValue [6] = 0; //reserved
setValue(ATC_AUDIO_SETTING_SET_TRE, mTreValue, 7);
```

■ ATC_AUDIO_SETTING_SET_REVERB

【variable】

variable[0]: 混响的类型

```
public final static int REVERB_OFF = 0;
public final static int REVERB_LIVE = 1;
public final static int REVERB_HALL = 2;
public final static int REVERB_CONCERT = 3;
public final static int REVERB_CAVE = 4;
public final static int REVERB_BATHROOM = 5;
public final static int REVERB_ARENA = 6;
public final static int REVERB_CHURCH = 7;
public final static int REVERB_MAX = 8;
```

【示例 code】

```
int[] mReverbType = new int[1];
mReverbType[0] = REVERB_CAVE;
SetValue(ATC_AUDIO_SETTING_SET_REVERB, mReverbType, 1);
```

- ATC_AUDIO_SETTING_SET_UPMIX

【variable】

Upmix 默认打开，若用于非车机场景，可使用 API 关闭

variable[0]: u4_enable, 1 表示打开，0 表示关闭

【示例 code】

```
int[] mUpmixGain = new int[1];
mUpmixGain[0] = 1; // enable upmix
SetValue(ATC_AUDIO_SETTING_SET_UPMIX, mUpmixGain, 1);
```

- ATC_AUDIO_SETTING_SET_AUTOSURROUND

【variable】

variable[0]: enable, 是否打开 auto Surround 功能, 1 为打开, 0 为关闭;

variable[1]: mode, 即单双声道, 1 表示 mono 单声道, 0 表示 stereo 双声道;

variable[2]: filter, 滤波器开关, 1 表示打开滤波器, 0 表示关闭滤波器;

variable[3~10]: 输入输出的 gain 值, 范围均为 0~0x20000, 0x20000 对应 0db;

variable[3]: 总的输入 gain 值;

variable[4]: 环绕喇叭的输入 gain 值;

variable[5]: 中置喇叭的输入 gain 值;

variable[6]: 低音的输入 gain 值;

variable[7]: 左右喇叭的输出 gain 值;

variable[8]: 环绕喇叭的输出 gain 值;

variable[9]: 中置喇叭的输出 gain 值;

variable[10]: 低音喇叭的输出 gain 值;

variable[11~13]: 声道 mix 的 gain 值, 范围均为 0~0x7FFFFFFF;

variable[11]: 中置声道 mix 到左右声道的 gain 值;

variable[12]: 中置声道 mix 到环绕声道的 gain 值;

variable[13]: 左右声道 mix 到环绕声道的 gain 值;

variable[14]: 环绕喇叭的延迟值, 范围 0~20;

【示例 code】

```
int[] mAutoSurround = new int[15];
mAutoSurround[0] = 1;//enable
mAutoSurround[1] = 0;//stereo
mAutoSurround[2] = 0;//disable filter
mAutoSurround[3] = 0x1A000;//数组第 3~10 成员为输入输出 gain 值
mAutoSurround[4] = 0x169FB;
mAutoSurround[5] = 0x20000;
mAutoSurround[6] = 0x20000;
mAutoSurround[7] = 0x20000;
mAutoSurround[8] = 0x20000;
mAutoSurround[9] = 0x20000;
mAutoSurround[10] = 0x20000;
mAutoSurround[11] = 0;//数组第 11~13 成员为 mix 声道 gain 值
mAutoSurround[12] = 0;
mAutoSurround[13] = 0;
mAutoSurround[14] = 0x10;//环绕喇叭 delay 值
```

```
SetValue(ATC_AUDIO_SETTING_SET_AUTOSURROUND, mAutoSurround, 15);
```

■ ATC_AUDIO_SETTING_SET_CHANNEL_DELAY

【variable 方式 1】

variable[0]: chDelay_L, 左声道 delay 的时间。

variable[1]: chDelay_R, 右声道 delay 的时间。

variable[2]: chDelay_C, 中置声道 delay 的时间。

variable[3]: chDelay_Sub, 低音声道 delay 的时间。

variable[4]: chDelay_Ls, 后左声道 delay 的时间。

variable[5]: chDelay_Rs, 后右声道 delay 的时间。

各个声道 delay 的时间范围为 0ms~10ms, 0.1ms 为一单位, 参数的范围为 0~100。

【variable 方式 2】

此种方式 Delay 的精度可以达到 1 帧, 若输出采样率为 48000Hz, 即为 1/48ms, 参数的范围为 0~512。

Code 配置方式: 在

source/vendor/autochips/proprietary/hardware/audiosettings/new_aud_settings/ChannelDelay.cp
p 中将#define USE_SAMPLE 解除注释即可

variable[0]: chDelay_L, 左声道 delay 的帧数。

variable[1]: chDelay_R, 右声道 delay 的帧数。

variable[2]: chDelay_C, 中置声道 delay 的帧数。

variable[3]: chDelay_Sub, 低音声道 delay 的帧数。

variable[4]: chDelay_Ls, 后左声道 delay 的帧数。

variable[5]: chDelay_Rs, 后右声道 delay 的帧数。

【示例 code】

```
int[] mChannelDelay = new int[6];
mChannelDelay[0] = 15; // L channel delay 1.5ms
mChannelDelay[1] = 23; // R channel delay 2.3ms
.....
mChannelDelay[5] = 100; // RS channel delay 10ms
SetValue(ATC_AUDIO_SETTING_SET_CHANNEL_DELAY, mChannelDelay, 6);
```

- ATC_AUDIO_SETTING_SET_SPECTRUM_MODE

【variable】

variable[0]: u4_mode, 可设置 0,32,64,128,256,512,1024, 表示 fft 的点数

【示例 code】

```
int[] mSpectrum = new int[1];
mSpectrum[0] = 512;
SetValue(ATC_AUDIO_SETTING_SET_SPECTRUM_MODE, mSpectrum, 6);
```

17.4.5 Kernel 接口

17.4.5.1 Kcontrol 支持

Kcontrol 是按照标准的 tinymix 方式开发一套接口。

Kcontrol 相关会涉及到的功能如下：

- 下行通路
- 音量控制
- 音效控制

17.4.5.1.1 代码结构及文件描述

表 17-26 ALSA SoC driver 代码结构及描述

文件名	文件路径	文件描述
all	source/kernel/common/sound/soc/autochips/alsa_soc/	ALSA SoC 架构驱动代码
ac8x_dspse_codec.c ac8x_dspse_api.c ac8x_dspse_table.c	source/kernel/common/sound/soc/autochips/alsa_soc/codec	音效相关 kcontrol 实现
aud_dspse.h	source/kernel/common/sound/soc/autochips/alsa_soc/codec	音效相关 kcontrol 头文件

17.4.5.1.2 DSP 架构图

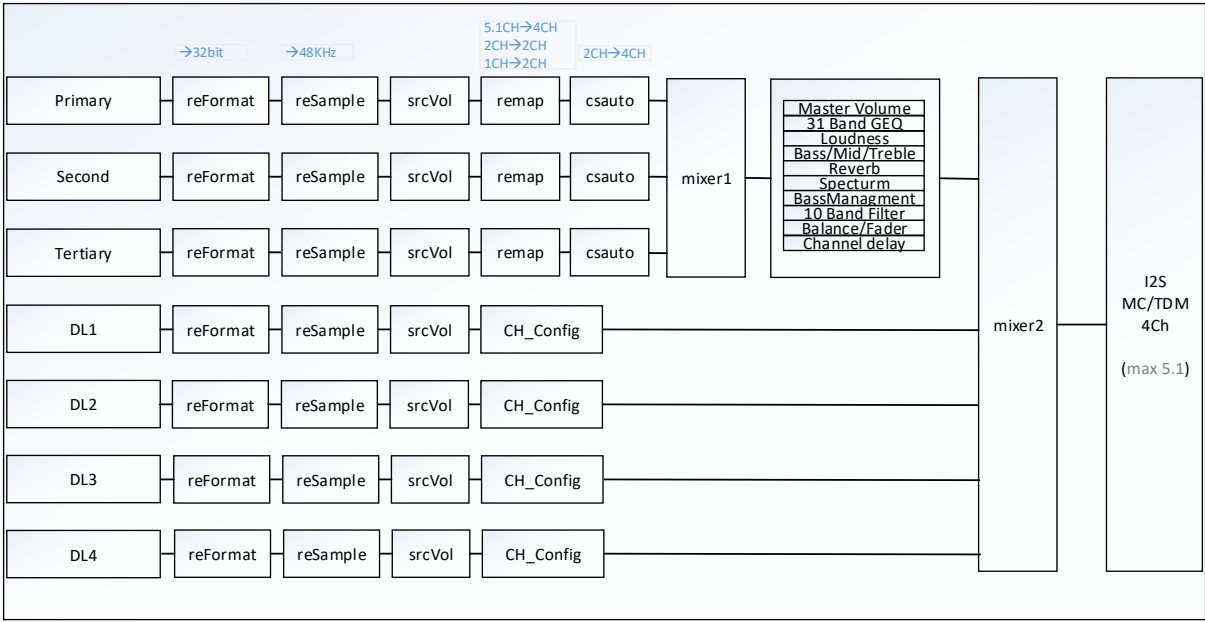


图 17-27 DSP 架构图

DSP 内部当前支持 Primary\Second\Tertiary\DL1\DL2\DL3\DL4 共 7 路下行，这 7 条下行 Path 由 ALSA driver 驱动，并向用户层提供声卡设备供用户层调用：

- Primary\Secondary\Tertiary 属于媒体类音频通路，支持音效；
- DL1\DL2\DL3\DL4 属于系统类音频通路，不支持音效；
- DSP 当前支持 I2S MC 和 TDM 两种模式输出 MAX 5.1ch，常用配置是 4ch 输出；
- CH_Config 用于指定该数据通路(mono/stereo)任意混音到 4ch 的哪些喇叭中去；
- 音量调节包括 srcVol, Master_Volume, Balance_Fader:
 - SrcVol 指调节每个路的音量.
 - Master_Volume 用于多媒体音量曲线.
 - Balance_Fader 用于 4CH 的声场调节.

17.4.5.1.3 Kontrol 接口

表 17-27 DSP 音效 Kcontrol API

API	说明
Media_Master_Vol	媒体主音量
Media_Pri_Vol	媒体 primary 通路音量

Media_Sec_Vol	媒体 secondary 通路音量
Media_Ter_Vol	媒体 tertiary 通路音量
Sys_DL1_Vol	系统 DL1 通路音量
Sys_DL2_Vol	系统 DL2 通路音量
Sys_DL3_Vol	系统 DL3 通路音量
Sys_DL4_Vol	系统 DL4 通路音量
Media_Master_Mute	媒体总静音开关
Media_Pri_Mute	媒体 primary 通路静音开关
Media_Sec_Mute	媒体 secondary 通路静音开关
Media_Ter_Mute	媒体 tertiary 通路静音开关
Sys_DL1_Mute	系统 DL1 通路静音开关
Sys_DL2_Mute	系统 DL2 通路静音开关
Sys_DL3_Mute	系统 DL3 通路静音开关
Sys_DL4_Mute	系统 DL4 通路静音开关
CH_L_Vol	前左喇叭音量设置
CH_R_Vol	前右喇叭音量设置
CH_LS_Vol	后左喇叭音量设置
CH_RS_Vol	后右喇叭音量设置
CH_C_Vol	中置喇叭音量设置
CH_Sub_Vol	低音喇叭音量设置
CH_L_Delay	前左喇叭延迟设置
CH_R_Delay	前右喇叭延迟设置
CH_LS_Delay	后左喇叭延迟设置
CH_RS_Delay	后右喇叭延迟设置
CH_C_Delay	中置喇叭延迟设置
CH_Sub_Delay	低音喇叭延迟设置
CH_L_Delay_sample	前左喇叭延迟设置
CH_R_Delay_sample	前右喇叭延迟设置
CH_LS_Delay_sample	后左喇叭延迟设置
CH_RS_Delay_sample	后右喇叭延迟设置
CH_C_Delay_sample	中置喇叭延迟设置
CH_Sub_Delay_sample	低音喇叭延迟设置
Speaker_Layout	喇叭配置
Front_Speaker_Size	前置(前左前右)喇叭尺寸
Center_Speaker_Size	中置喇叭尺寸
Rear_Speaker_Size	后置(后左后右)喇叭尺寸
Subwoofer_Speaker_Size	低音喇叭尺寸

Upmix_Enable	声道扩展开关
CH_Reverse_Phase	声道反相设置
ATS_Enable	Auto Surround 环绕声道扩展开关
ATS_Mode	Auto Surround 模式选择
ATS_Filter	Auto Surround 滤波开关
ATS_PARAM	Auto Surround 参数设置
Tone_Bass	低音增益设置
Common_Bass	低音音效参数设置
Tone_Middle	中音增益设置
Common_Middle	中音音效参数设置
Tone_Treble	高音增益设置
Common_Treble	高音音效参数设置
Reverb_Mode	混响设置
Media_Master_Vol_Step	媒体音量档位等级设置
Media_Master_Vol_Speed_Step	媒体音量随车速变化等级设置
Sys_DL1_Vol_Step	系统 DL1 通路音量档位等级设置
Sys_DL2_Vol_Step	系统 DL2 通路音量档位等级设置
Sys_DL3_Vol_Step	系统 DL3 通路音量档位等级设置
Sys_DL4_Vol_Step	系统 DL4 通路音量档位等级设置
Balance_Fader_FL	前左喇叭音量档位等级设置
Balance_Fader_FR	前右喇叭音量档位等级设置
Balance_Fader_RL	后左喇叭音量档位等级设置
Balance_Fader_RR	后右喇叭音量档位等级设置
Set_Sound_Field	声场调节(balance, fader)
Set_31EQ	31 段 EQ 设置
GEQ_Enable	均衡器开关
GEQ_Preset	均衡器模式设置
GEQ_Band_Gain	均衡器段增益设置
GEQ_All_Gain	均衡器所有段增益设置
Loudness_Enable	Loudness 等响开关
PEQ_Tuning	7 段参量均衡器设置
Set_Biquad_IIR	Biquad IIR 音效设置
Set_iir_4ch	4 声道（前左、前右、后左、后右）IIR 音效设置
Set_iir_5ch	5 声道（前左、前右、中置、后左、后右）IIR 音效设置
lmtr_enable	Limiter 开关
lmtr_gain	Limiter gain 值设置
lmtr_pre_gain	Limiter pre gain 值设置

lmtr_post_gain	Limiter post gain 值设置
spectrum_mode	Spectrum mode 设置
spectrum_data	Spectrum data 获取

17.4.5.1.3.1 Media_Master_Vol

【描述】

媒体主音量设置，Primary\Second\Linein(Tertiary)均受其控制

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

如 0x20000 对应 0dB, 0 对应 mute, 方便客户定制音量曲线.

17.4.5.1.3.2 Media_Pri_Vol

【描述】

媒体 primary 通路音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.3 Media_Sec_Vol

【描述】

媒体 secondary 通路音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.4 Media_Ter_Vol

【描述】

媒体 linein(Tertiary) 通路音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.5 Sys_DL1_Vol

【描述】

系统音 Downlink1 通路音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.6 Sys_DL2_Vol

【描述】

系统音 Downlink2 通路音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.7 Sys_DL3_Vol

【描述】

系统音 Downlink3 通路音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.8 Sys_DL4_Vol

【描述】

系统音 Downlink4 通路音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.9 Media_Master_Mute

【描述】

媒体总静音开关，Primary\Second\Linein(Tertiary)均受其控制

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示 unmute；1 表示 mute

17.4.5.1.3.10 Media_Pri_Mute

【描述】

媒体 primary 通路静音开关

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示 unmute；1 表示 mute

17.4.5.1.3.11 Media_Sec_Mute

【描述】

媒体 secondary 通路静音开关

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示 unmute；1 表示 mute

17.4.5.1.3.12 Media_Ter_Mute

【描述】

媒体 linein(Tertiary) 通路静音开关

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示 unmute；1 表示 mute

17.4.5.1.3.13 Media_DL1_Mute

【描述】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

494 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

系统音 Downlink1 通路静音开关

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示 unmute；1 表示 mute

17.4.5.1.3.14 Media_DL2_Mute

【描述】

系统音 Downlink2 通路静音开关

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示 unmute；1 表示 mute

17.4.5.1.3.15 Media_DL3_Mute

【描述】

系统音 Downlink3 通路静音开关

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示 unmute；1 表示 mute

17.4.5.1.3.16 Media_DL4_Mute

【描述】

系统音 Downlink4 通路静音开关

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示 unmute；1 表示 mute

17.4.5.1.3.17 CH_L_Vol

【描述】

前左喇叭音量设置

【参数】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

495 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.18 CH_R_Vol

【描述】

前右喇叭音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.19 CH_LS_Vol

【描述】

后左喇叭音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.20 CH_RS_Vol

【描述】

后右喇叭音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.21 CH_C_Vol

【描述】

中置喇叭音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.22 CH_Sub_Vol

【描述】

低音喇叭音量设置

【参数】

一个参数：音量值；类型是 unit32；范围是 0 ~ 0x20000；默认值是 0x20000

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

17.4.5.1.3.23 CH_L_Delay

【描述】

前左喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 100；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 0.1ms

17.4.5.1.3.24 CH_R_Delay

【描述】

前右喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 100；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 0.1ms

17.4.5.1.3.25 CH_LS_Delay

【描述】

后左喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 100；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 0.1ms

17.4.5.1.3.26 CH_RS_Delay

【描述】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

497 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

后右喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 100；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 0.1ms

17.4.5.1.3.27 CH_C_Delay

【描述】

中置喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 100；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 0.1ms

17.4.5.1.3.28 CH_Sub_Delay

【描述】

低音喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 100；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 0.1ms

17.4.5.1.3.29 CH_L_Delay_sample

【描述】

前左喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 512；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 1 frame

17.4.5.1.3.30 CH_R_Delay_sample

【描述】

前右喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 512；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 1frame

17.4.5.1.3.31 CH_LS_Delay_sample

【描述】

后左喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 512；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 1frame

17.4.5.1.3.32 CH_RS_Delay_sample

【描述】

后右喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 512；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 1frame

17.4.5.1.3.33 CH_C_Delay_sample

【描述】

中置喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 512；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 1frame

17.4.5.1.3.34 CH_Sub_Delay_sample

【描述】

低音喇叭延迟设置

【参数】

一个参数：延迟值；类型是 unit32；范围是 0 ~ 512；默认值是 0

延迟值与时间的对应关系：0 ~ 10ms, 单位是 1frame

17.4.5.1.3.35 Speaker_Layout

【描述】

喇叭配置：配置输出的喇叭个数和布局

【参数】

一个参数：喇叭；类型是 unit32；范围是 b0 ~ b5 有效；默认值是 0x3F (5.1CH)

表 17-28 bit 与声道对应表

B31 ~ B6	B5	B4	B3	B2	B1	B0
Reserved	RS	LS	Sub	C	R	L

如车上通常 4 个喇叭的布局是 L,R,LS,RS. 那么其对应的喇叭配置为 0x33。

17.4.5.1.3.36 Front_Speaker_Size

【描述】

前置(前左前右)喇叭尺寸：如果不是全尺寸喇叭，则需要通过 HPF 来滤掉低频部分

【参数】

一个参数：HPF 截止频率；类型是 unit32；范围是 0 ~ 300(Hz)；默认值是 0

17.4.5.1.3.37 Center_Speaker_Size

【描述】

中置喇叭尺寸：如果不是全尺寸喇叭，则需要通过 HPF 来滤掉低频部分

【参数】

一个参数：HPF 截止频率；类型是 unit32；范围是 0 ~ 300(Hz)；默认值是 0

17.4.5.1.3.38 Rear_Speaker_Size

【描述】

后置(后左后右)喇叭尺寸：如果不是全尺寸喇叭，则需要通过 HPF 来滤掉低频部分

【参数】

一个参数：HPF 截止频率；类型是 unit32；范围是 0 ~ 300(Hz)；默认值是 0

17.4.5.1.3.39 Subwoofer_Speaker_Size

【描述】

低音喇叭尺寸：通过 LPF 来滤掉高频部分

【参数】

一个参数：LPF 截止频率；类型是 unit32；范围是 20 ~ 300(Hz)；默认值是 100

17.4.5.1.3.40 Upmix_Enable

【描述】

声道扩展开关：把 mono/stereo 的音源扩展成多声道输出

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 1

0 表示关闭；1 表示打开

17.4.5.1.3.41 CH_Reverse_Phase

【描述】

声道反相配置：配置输出的声道是否反相

【参数】

一个参数：类型是 unit32；范围是 b0 ~ b5 有效，当该 bit 位为 1 时，表示对应声道配置反相；默认值是 0（均不反相）

表 17-14 bit 与声道对应表

B5	B4	B3	B2	B1	B0
RS	LS	Sub	C	R	L

17.4.5.1.3.42 ATS_Enable

【描述】

Auto Surround 环绕扩展开关：把 mono/stereo 的音源扩展成多声道输出，带有环绕声场效果

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示关闭；1 表示打开

【注意】

ATS 与 Upmix 是同类功能，两者同时只能二选一，不能同时打开。

17.4.5.1.3.43 ATS_Mode

【描述】

Auto Surround 模式选择

【参数】

一个参数：模式；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示输入源是 stereo 模式；1 表示输入源是 mono 模式

17.4.5.1.3.44 ATS_Filter

【描述】

Auto Surround 滤波开关

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示滤波功能关闭；1 表示滤波功能打开

17.4.5.1.3.45 ATS_PARAM

【描述】

Auto Surround 滤波参数设置

【参数】

共 15 个，分别表示：

- 开关状态：1 表示打开，0 表示关闭
- 设置模式：1 表示 mono，0 表示 stereo
- 滤波器开关：1 表示打开，0 表示关闭
- 总的输入 gain 值：范围 0-0x20000
- 环绕喇叭的输入 gain 值：范围 0-0x20000
- 中置喇叭的输入 gain 值：范围 0-0x20000
- 低音喇叭的输入 gain 值：范围 0-0x20000
- 左右喇叭的输出 gain 值：范围 0-0x20000
- 环绕喇叭的输出 gain 值：范围 0-0x20000
- 中置喇叭的输出 gain 值：范围 0-0x20000

- 低音喇叭的输出 gain 值：范围 0-0x20000
- 中置 Mix 到左右的 gain 值：范围 0-0x7FFFFFFF
- 中置 Mix 到环绕的 gain 值：范围 0-0x7FFFFFFF
- 左右 Mix 到环绕的 gain 值：范围 0-0x7FFFFFFF
- 环绕喇叭的延迟值：范围 0-20，1 表示延迟 1.33ms

17.4.5.1.3.46 Reverb_Mode

【描述】

混响设置

【参数】

一个参数：模式；类型是 unit32；范围是 0 ~ 7；默认值是 0

0：混响关闭；1: Live; 2: Hall; 3: Concert; 4: Cave; 5: Bathroom; 6: Arena; 7: Church;

17.4.5.1.3.47 Tone_Bass

【描述】

Bass 低音增益设置

【参数】

一个参数：增益；类型是 unit32；范围是 0 ~ 14；默认值是 7

增益 0 ~ 14 对应 -7 ~ 7dB。

表 17-29 Bass Gain 值与 dB 值对应表

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
dB	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7

17.4.5.1.3.48 Common_Bass

【描述】

Bass 低音音效参数设置，包含开关状态、中心频率、Q 值、Gain 值

【参数】

- 开关状态：类型为 uint32_t; 0 表示关，1 表示开
- 中心频率：类型为 uint32_t，取值及含义见下方“[Bass 中心频率 value 表](#)”，默认取值为 2
- Q 值：类型为 uint32_t，取值及含义见下方“[Bass Q 值 value 表](#)”，默认取值为 0

- Gain 值：类型为 uint32_t，取值及含义见下方“Bass Gain 值与 dB 映射表”，默认取值为 15

表 17-30 Bass 中心频率 value 表

Index	0	1	2	3
Frequency	60Hz	80Hz	100Hz	200Hz

表 17-31 Bass Q 值 value 表

Index	0	1	2	3
Q Factor	1.0	1.25	1.5	2.0

表 17-32 Bass Gain 值与 dB 映射表

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
dB	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	0
Index	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	
dB	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	

17.4.5.1.3.49 Tone_Middle

【描述】

Middle 中音增益设置

【参数】

一个参数：增益；类型是 unit32；范围是 0 ~ 14；默认值是 7

增益 0 ~ 14 对应 -7 ~ 7dB。

表 17-33 Middle Gain 值与 dB 值对应表

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
dB	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7

17.4.5.1.3.50 Common_Middle

【描述】

Middle 中音音效参数设置，包含开关状态、中心频率、Q 值、Gain 值

【参数】

- 开关状态：类型为 uint32_t; 0 表示关，1 表示开
- 中心频率：类型为 uint32_t，取值及含义见下方“[Middle 中心频率 value 表](#)”，默认取值为 1
- Q 值：类型为 uint32_t，取值及含义见下方“[Middle Q 值 value 表](#)”，默认取值为 2
- Gain 值：类型为 uint32_t，取值及含义见下方“[Middle Gain 值与 dB 映射表](#)”，默认取值为 15

表 17-34 Middle 中心频率 value 表

Index	0	1	2	3
Frequency	500Hz	1000Hz	1500Hz	2500Hz

表 17-35 Middle Q 值 value 表

Index	0	1	2	3
Q Factor	0.5	0.75	1.00	1.25

表 17-36 Middle Gain 值与 dB 映射表

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
dB	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	0
Index	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	
dB	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	

17.4.5.1.3.51 Tone_Treble

【描述】

Treble 高音增益设置

【参数】

一个参数：增益；类型是 unit32；范围是 0 ~ 14；默认值是 7

增益 0 ~ 14 对应 -7 ~ 7dB。

表 17-37 Treble Gain 值与 dB 值对应表

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
dB	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7

17.4.5.1.3.52 Common_Treble

【描述】

Treble 高音音效参数设置，包含开关状态、中心频率、Q 值、Gain 值

【参数】

- 开关状态：类型为 uint32_t; 0 表示关，1 表示开
- 中心频率：类型为 uint32_t，取值及含义见下方“Treble 中心频率 value 表”，默认取值为 0
- Q 值：类型为 uint32_t，取值及含义见下方“Treble Q 值 value 表”，默认取值为 2
- Gain 值：类型为 uint32_t，取值及含义见下方“Treble Gain 值与 dB 映射表”，默认取值为 15

表 17-38 Treble 中心频率 value 表

Index	0	1	2	3
Frequency	10KHz	12.5KHz	15KHz	17.5KHz

表 17-39 Treble Q 值 value 表

Index	0	1	2	3
Q Factor	0.5	0.75	1.00	1.25

表 17-40 Treble Gain 值与 dB 映射表

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
dB	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	0
Index	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	
dB	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	

17.4.5.1.3.53 Media_Master_Vol_Step

【描述】

媒体音量档位等级设置(客制化)

【参数】

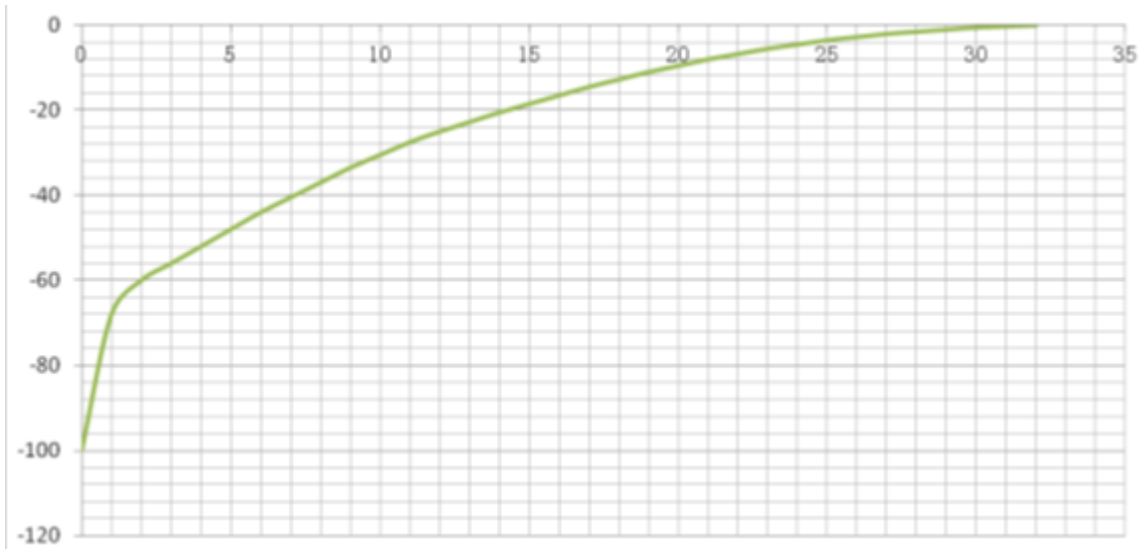
一个参数：音量档位；类型是 unit32；范围是 0 ~ 32；默认值是 32

音量档位与 dB 对应如下：

表 17-41 Mater Volume dB

Step	0	1	2	3	4	5	6	7	8	9	10
dB	-100	-68	-60	-56	-52	-48	-44	-40.5	-37	-33.5	-30.5
Step	11	12	13	14	15	16	17	18	19	20	21
dB	-27.5	-25	-22.75	-20.5	-18.5	-16.5	-14.5	-12.75	-11	-9.5	-8
Step	22	23	24	25	26	27	28	29	30	31	32
dB	-6.75	-5.5	-4.5	-3.5	-2.75	-2	-1.5	-1	-0.5	-0.25	0

音量曲线图：



音量值与 dB 的换算: $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

ac8x_dspse_table.c 的 VOL_SPEED_CTRL 数组左边第一列描述了从 dB 换算成的 33 档音量值。

如下: 如果客户需要修改音量曲线, 可以通过修改此数组表格来实现。

```
const uint32_t VOL_SPEED_CTRL[MEDIA_VOL_STEP_MAX][SPEED_STEP_MAX] = {
/* 0dB      1      2      3      4      5      6      7      8      9      10      11dB */
{0x00001, 0x00001, 0x00002, 0x00002, 0x00002, 0x00002, 0x00003, 0x00003, 0x00003, 0x00004, 0x00004, 0x00005},
{0x00034, 0x0003B, 0x00042, 0x0004A, 0x00053, 0x0005D, 0x00068, 0x00075, 0x00083, 0x00093, 0x000A5, 0x000B9},
{0x00083, 0x00093, 0x000A5, 0x000B9, 0x000D0, 0x000E9, 0x00106, 0x00125, 0x00149, 0x00171, 0x0019E, 0x001D1},
{0x000D0, 0x000E9, 0x00106, 0x00125, 0x00149, 0x00171, 0x0019E, 0x001D1, 0x0020A, 0x00249, 0x00291, 0x002E1},
{0x00149, 0x00171, 0x0019E, 0x001D1, 0x0020A, 0x00249, 0x00291, 0x002E1, 0x0033B, 0x003A0, 0x00411, 0x00490},
{0x0020A, 0x00249, 0x00291, 0x002E1, 0x0033B, 0x003A0, 0x00411, 0x00490, 0x0051F, 0x005BF, 0x00672, 0x0073B},
{0x0033B, 0x003A0, 0x00411, 0x00490, 0x0051F, 0x005BF, 0x00672, 0x0073B, 0x0081D, 0x0091B, 0x00A37, 0x00B76},
{0x004D5, 0x0056C, 0x00616, 0x006D4, 0x007A9, 0x00898, 0x009A5, 0x00AD2, 0x00C24, 0x00D9F, 0x00F49, 0x01126},
{0x0073B, 0x0081D, 0x0091B, 0x00A37, 0x00B76, 0x00CDC, 0x00EE6, 0x01031, 0x0122B, 0x01462, 0x016DF, 0x019A9},
{0x00AD2, 0x00C24, 0x00D9F, 0x00F49, 0x01126, 0x0133E, 0x01597, 0x0183A, 0x01B2E, 0x01E7F, 0x02238, 0x02665},
{0x00F49, 0x01126, 0x0133E, 0x01597, 0x0183A, 0x01B2E, 0x01E7F, 0x02238, 0x02665, 0x02B14, 0x03056, 0x0363C},
{0x01597, 0x0183A, 0x01B2E, 0x01E7F, 0x02238, 0x02665, 0x02B14, 0x03056, 0x0363C, 0x03CDA, 0x04447, 0x04C9B},
{0x01CCB, 0x0204E, 0x0243F, 0x028AB, 0x02DA2, 0x03333, 0x03973, 0x04075, 0x04852, 0x05126, 0x05B0C, 0x06628},
{0x0254E, 0x029DB, 0x02EF7, 0x034B2, 0x03B20, 0x04257, 0x04A6F, 0x05384, 0x05DB5, 0x06924, 0x075F8, 0x0845D},
{0x03056, 0x0363C, 0x03CDA, 0x04447, 0x04C9B, 0x055F4, 0x06071, 0x06C36, 0x0796A, 0x0883B, 0x098DA, 0x0AB81},
{0x03CDA, 0x04447, 0x04C9B, 0x055F4, 0x06071, 0x06C36, 0x0796A, 0x0883B, 0x098DA, 0x0AB81, 0x0C06E, 0x0D7E9},
{0x04C9B, 0x055F4, 0x06071, 0x06C36, 0x0796A, 0x0883B, 0x098DA, 0x0AB81, 0x0C06E, 0x0D7E9, 0x0F241, 0x10FD0},
{0x06071, 0x06C36, 0x0796A, 0x0883B, 0x098DA, 0x0AB81, 0x0C06E, 0x0D7E9, 0x0F241, 0x10FD0, 0x130FB, 0x15631},
{0x075F8, 0x0845D, 0x09484, 0x0A6A3, 0x0BAF8, 0x0D1C8, 0x0EB61, 0x1081A, 0x12853, 0x14C7C, 0x1750D, 0x1A292},
{0x0904D, 0x0A1E9, 0x0B5AA, 0x0C8D5, 0x0E4B4, 0x1009C, 0x11FEB, 0x1430D, 0x16A78, 0x196B2, 0x1C852, 0x20000},
{0x0AB81, 0x0C06E, 0x0D7E9, 0x0F241, 0x10FD0, 0x130FB, 0x15631, 0x17FF2, 0x1AECB, 0x1E35C, 0x20000, 0x20000},
{0x0C8D5, 0x0E4B4, 0x1009C, 0x11FEB, 0x1430D, 0x16A78, 0x196B2, 0x1C852, 0x20000, 0x20000, 0x20000, 0x20000},
{0x0EB61, 0x1081A, 0x12853, 0x14C7C, 0x1750D, 0x1A292, 0x1D5A5, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000},
{0x10FD0, 0x130FB, 0x15631, 0x17FF2, 0x1AECB, 0x1E35C, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000},
{0x130FB, 0x15631, 0x17FF2, 0x1AECB, 0x1E35C, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000},
{0x15631, 0x17FF2, 0x1AECB, 0x1E35C, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000},
{0x1750D, 0x1A292, 0x1D5A5, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000},
{0x196B2, 0x1C852, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000},
{0x1AECB, 0x1E35C, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000},
{0x1C852, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000},
{0x1E35C, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000},
{0x1F179, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000},
{0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000, 0x20000}
};
```


17.4.5.1.3.54 Media_Master_Vol_Speed_Step

【描述】

媒体音量随车速变化等级设置, 即随车速的变化来自动调节以抵消高速风噪的影响。

【参数】

一个参数：速度档位；类型是 unit32；范围是 0 ~ 11；默认值是 0

速度档位同样使用上面的 VOL_SPEED_CTRL 数组。

当速度档位为 0 时，其 33 档音量曲线为第一列，与 Media_Master_Vol_Step 是一致的。

当速度档位为 1 时，采用第二列，其整体比第一列增加 1dB。

以此类推，当速度越高，档位越大，最大档位比第一列增加 11dB。最大补偿值不超过最大音量，以避免失真和功放增益饱和。

【注意】

该功能会根据当前车速对应的增益值，结合当前的音量档位，在表中找到当前应该设置的媒体音量。

17.4.5.1.3.55 Sys_DL1_Vol_Step

【描述】

系统 DL1 通路音量档位等级设置, 用于传输导航、铃声、提示音、通话、报警音等无需过音效的声音

【参数】

一个参数：音量档位；类型是 unit32；范围是 0 ~ 8；默认值是 8

音量档位与 dB 对应如下：

表 17-42 DL1 volume dB

Step	0	1	2	3	4	5	6	7	8
dB	-100	-52	-37	-25	-16.5	-9.5	-4.5	-1.5	0

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

ac8x_dspse_table.c 的 SYS_VOL_STEP 数组描述了从 dB 换算成的 9 档音量值。

如下：如果客户需要修改音量曲线，可以通过修改此数组表格来实现。

```
const uint32_t SYS_VOL_STEP[SYS_VOL_STEP_MAX] = {

    0x00001, // -100dB
    0x00149, // -52dB
    0x0073B, // -37dB
    0x01CCB, // -25dB
    0x04C9B, // -16.5dB
    0x0AB81, // -9.5dB
    0x130FB, // -4.5dB
    0x1AECB, // -1.5dB
    0x20000 // 0dB
};
```

17.4.5.1.3.56 Sys_DL2_Vol_Step

【描述】

系统 DL2 通路音量档位等级设置, 作用同 DL1

【参数】

一个参数: 音量档位; 类型是 unit32; 范围是 0 ~ 8; 默认值是 8

详细同 DL1

17.4.5.1.3.57 Sys_DL3_Vol_Step

【描述】

系统 DL3 通路音量档位等级设置, 作用同 DL1

【参数】

一个参数: 音量档位; 类型是 unit32; 范围是 0 ~ 8; 默认值是 8

详细同 DL1

17.4.5.1.3.58 Sys_DL4_Vol_Step

【描述】

系统 DL4 通路音量档位等级设置, 作用同 DL1

【参数】

一个参数: 音量档位; 类型是 unit32; 范围是 0 ~ 8; 默认值是 8

详细同 DL1

17.4.5.1.3.59 Balance_Fader_FL

【描述】

前左喇叭音量档位等级设置

【参数】

一个参数：音量档位；类型是 unit32；范围是 0 ~ 14；默认值是 7

音量档位与 dB 对应如下：

表 17-43 Balance Fader index dB 对应表

Step	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
坐标	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7
dB	-65	-45	-28	-15	-9	-5	-2	0	0	0	0	0	0	0	0

音量值与 dB 的换算： $x \text{ dB} = 20 * \log(\text{音量值}/0x20000)$

ac8x_dspse_table.c 的 TRIM_TABLE 数组描述了从 dB 换算成的 15 档音量值。

如下：如果客户需要修改音量曲线，可以通过修改此数组表格来实现

```
const uint32_t TRIM_TABLE[TRIM_STEP_MAX] = {

    0x0004A, // -65dB
    0x002E1, // -45dB
    0x01462, // -28dB
    0x05B0C, // -15dB
    0x0B5AA, // -9dB
    0x11FEB, // -5dB
    0x196B2, // -2dB
    0x20000, // 0dB
    0x20000, // 0dB
    0x20000, // 0dB
    0x20000, // 0dB
    0x20000, // 0dB
    0x20000, // 0dB
    0x20000, // 0dB
    0x20000, // 0dB
    0x20000 // 0dB
};
```

17.4.5.1.3.60 Balance_Fader_FR

【描述】

前右喇叭音量档位等级设置

【参数】

一个参数：音量档位；类型是 unit32；范围是 0 ~ 14；默认值是 7

详细同 Balance_Fader_FL

17.4.5.1.3.61 Balance_Fader_RL

【描述】

后左喇叭音量档位等级设置

【参数】

一个参数：音量档位；类型是 unit32；范围是 0 ~ 14；默认值是 7

详细同 Balance_Fader_FL

17.4.5.1.3.62 Balance_Fader_RR

【描述】

后右喇叭音量档位等级设置

【参数】

一个参数：音量档位；类型是 unit32；范围是 0 ~ 14；默认值是 7

详细同 Balance_Fader_FL

17.4.5.1.3.63 Set_Sound_Field

【描述】

声场调节(balance, fader)设置

【参数】

两个参数：balance, fader；类型是 unit32；范围均是 0 ~ 14；默认值均是 7

Balance 相当于横坐标，fader 相当于纵坐标；0 ~ 14 映射成坐标-7 ~ 7

中心点：Balance, Fader(7, 7) 对应坐标(0, 0)

左上角(前左喇叭)：Balance, Fader(0, 0) 对应坐标(-7, 7)

右上角(前右喇叭)：Balance, Fader(14, 0) 对应坐标(7, 7)

左下角(后左喇叭)：Balance, Fader(0, 14) 对应坐标(-7, -7)

右下角(后右喇叭)：Balance, Fader(14, 14) 对应坐标(7, -7)

Balance&Fader Control					
Balance			Fader		
Step	Gain(dB)	Gain(dB)	Step	Gain(dB)	Gain(dB)
R7	-65	0	R7	-65	0
6	-45	0	6	-45	0
5	-28	0	5	-28	0
4	-15	0	4	-15	0
3	-9	0	3	-9	0
2	-5	0	2	-5	0
1	-2	0	1	-2	0
0	0	0	0	0	0
1	0	-2	1	0	-2
2	0	-5	2	0	-5
3	0	-9	3	0	-9
4	0	-15	4	0	-15
5	0	-28	5	0	-28
6	0	-45	6	0	-45
L7	0	-65	F7	0	-65

15 档音量档位与 dB 对应关系同 Balance_Fader_FL，也同样可以修改 `ac8x_dspse_table.c` 的 `TRIM_TABLE` 数组调整客制化音量值。

17.4.5.1.3.64 Set_31EQ

【描述】

31 段 EQ 设置

【参数】

variable[0]: u4_enable, 1: enable; 0: disable

variable[1]: u4_bandnum, EQ 的 band 数。

variable[2]: u4_preset, 0~9: 预设的 EQ。仅支持 10 段 EQ，使用此预设 EQ 后，后面使用的 400 多个参数将不会生效（但需要设置）。当 band 数不为 10 时，该预设类型无效，DSP 会按照各个 band 的 gain 值、Q 值、中心频率进行音效处理。

```
public final static int PRESET_FLAT = 0;
public final static int PRESET_VOICE = 1;
public final static int PRESET_CLASSICAL = 2;
public final static int PRESET_DANCE = 3;
public final static int PRESET_FOLK = 4;
```

```
public final static int PRESET_JAZZ = 5;

public final static int PRESET_POP = 6;

public final static int PRESET_ROCK = 7;

public final static int PRESET_TREBLE_BOOST = 8;

public final static int PRESET_BASS_BOOST = 9;
```

variable[3~157]: u4_CFreq[5][31], 5 个 channel, 每个 channel 对应 31 段的中心频率。

```
public static int default_EQCFreq[10] = {31,62,125,250,500,1000,2000,4000,8000,16000};
//默认十段 EQ 的中心频率, 中心频率不能设置 0。
```

variable[158~312]: u4_QFactor[5][31], 5 个 channel, 每个 channel 对应 31 段的 Q 值, Q 值不能设置为 0。

```
public static int default_PositiveQ[10] = {200,200,200,200,200,200,200,200,200,200};//默认
十段 EQ, gain 在 0~15DB 时的 Q 值。
```

```
public static int default_NegativeQ[10] = {50,50,50,50,50,50,50,50,50,50};//默认十段 EQ,
gain 在-1~-15DB 时的 Q 值。
```

variable[313~467]: u4_gain[5][31], 5 个 channel, 每个 channel 对应 31 段的 gain 值:

```
public final static int G_M15DB = 0;

public final static int G_M14DB = 1;

public final static int G_M13DB = 2;

public final static int G_M12DB = 3;

public final static int G_M11DB = 4;

public final static int G_M10DB = 5;

public final static int G_M9DB = 6;

public final static int G_M8DB = 7;

public final static int G_M7DB = 8;

public final static int G_M6DB = 9;

public final static int G_M5DB = 10;

public final static int G_M4DB = 11;

public final static int G_M3DB = 12;

public final static int G_M2DB = 13;

public final static int G_M1DB = 14;

public final static int G_0DB = 15;
```

```
public final static int G_1DB = 16;
public final static int G_2DB = 17;
public final static int G_3DB = 18;
public final static int G_4DB = 19;
public final static int G_5DB = 20;
public final static int G_6DB = 21;
public final static int G_7DB = 22;
public final static int G_8DB = 23;
public final static int G_9DB = 24;
public final static int G_10DB = 25;
public final static int G_11DB = 26;
public final static int G_12DB = 27;
public final static int G_13DB = 28;
public final static int G_14DB = 29;
public final static int G_15DB = 30;
```

u4_gain [5][31]表示均衡器 10 个频段 DB 值对应的增益值, 范围: -15dB ~ 15dB, 上层 APP 只需要传入对应的定义值即可, 即 0~30 (G_M15DB~G_15DB)。

```
public static int default_EQGain[10][10] = {
{15, 15, 15, 15, 15, 15, 15, 15, 15, 15}, //flat preset
{10, 15, 14, 8, 18, 21, 23, 22, 14, 10}, // voice preset
{22, 20, 15, 14, 13, 13, 14, 16, 20, 22}, // classical preset
{18, 22, 19, 13, 19, 10, 10, 24, 11, 18}, // dance preset
{20, 21, 18, 9, 18, 18, 9, 20, 23, m}, // folk preset
{12, 20, 20, 15, 8, 16, 15, 18, 20, 16}, // jazz preset
{17, 21, 20, 11, 18, 23, 11, 19, 20, 19}, // pop preset
{19, 19, 22, 10, 16, 10, 12, 19, 21, 19}, // rock preset
{18, 19, 11, 17, 11, 19, 15, 23, 21, 24}, // treble boost preset
{19, 22, 20, 10, 9, 13, 15, 14, 11, 11} // bass boost preset
};// 对应各个 dB 值的宏定义
```


如果参数一的段 大于当前总段数，则会设置无效。

比如当前 10 段 EQ, 如果想要设置第 11 段的增益，则为无效。

17.4.5.1.3.68 GEQ_All_Gain

【描述】

均衡器所有段增益设置, 同时设置所有段数的 gain 值

【参数】

N 个参数: (N 指当前 EQ 的总段数)

参数为增益; 类型是 unit32; 范围是 0 ~ 30; 对应-15~15dB. 默认值是 15(0dB)

【注意】

比如当前是 10 段 EQ, 那么需要设置 10 参数, 每个参数顺序对第 1 到第 10 段..

17.4.5.1.3.69 Loudness_Enable

【描述】

Loudness 等响: 媒体主音量调小, Loudness 按比例增强低频和高频, 两者抵消以达到等响的效果。

【参数】

一个参数: 开关; 类型是 unit32; 范围是 0 ~ 1; 默认值是 0

0 表示关闭; 1 表示打开

【注意】

等响的参数设置可根据实际需求, 如音效框架、音量曲线设计、等响 Spec 等, 选择在 kernel 层或者 userspace 层进行配置。

Kernel 层配置:

VOL	Loudness		
	Treble Gain(dB)	Bass Gain(dB)	
0	0	0	const int32_t LOUD_TABLE[][LOUD_2_BAND] = { /* bass_100Hz, treble_1kHz */ { 0, 0}, /* vol 0 */ {100, 40}, /* vol 1 */ {100, 40}, /* vol 2 */ {100, 40}, /* vol 3 */ {100, 40}, /* vol 4 */ {100, 40}, /* vol 5 */ {100, 40}, /* vol 6 */ { 99, 39}, /* vol 7 */ { 97, 38}, /* vol 8 */ { 93, 36}, /* vol 9 */ { 88, 34}, /* vol 10 */ { 83, 31}, /* vol 11 */ { 77, 28}, /* vol 12 */ { 70, 25}, /* vol 13 */ { 60, 21}, /* vol 14 */ { 50, 17}, /* vol 15 */ { 40, 13}, /* vol 16 */ { 30, 9}, /* vol 17 */ { 22, 7}, /* vol 18 */ { 13, 4}, /* vol 19 */ { 8, 2}, /* vol 20 */ { 3, 1}, /* vol 21 */ { 0, 0}, /* vol 22 */ { 0, 0}, /* vol 23 */ { 0, 0}, /* vol 24 */ { 0, 0}, /* vol 25 */ { 0, 0}, /* vol 26 */ { 0, 0}, /* vol 27 */ { 0, 0}, /* vol 28 */ { 0, 0}, /* vol 29 */ { 0, 0}, /* vol 30 */ { 0, 0}, /* vol 31 */ { 0, 0}, /* vol 32 */ };
1	4	10	
2	4	10	
3	4	10	
4	4	10	
5	4	10	
6	4	10	
7	3.9	9.9	
8	3.8	9.7	
9	3.6	9.3	
10	3.4	8.8	
11	3.1	8.3	
12	2.8	7.7	
13	2.5	7	
14	2.1	6	
15	1.7	5	
16	1.3	4	
17	0.9	3	
18	0.7	2.2	
19	0.4	1.3	
20	0.2	0.8	
21	0.1	0.3	
22	0	0	
23	0	0	
24	0	0	
25	0	0	
26	0	0	
27	0	0	
28	0	0	
29	0	0	
30	0	0	
31	0	0	
32	0	0	

等响的 Gain 设置可参考 [ac8x_dspse_table.c](#) 的 **LOUD_TABLE** 数组(如上)做修改.

Loudness bass boost 频点是 100Hz, treble boost 是 1KHz. Q 为 1.0. 若需要修改可以参考 [ac8x_dspse_table.c](#) 的 **FILTER_TABLE** 数组(如下):

```
int32_t FILTER_TABLE[202] = {
    /* enable */
    1,
    /* bar 1~2 for loudness, 3~9 for PEQ, 10: reserved */
    9,
    /* filter type: 1:PEAK;2:LPF;3:HPF;4:LSF;5:HSF */
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, /* L CH */
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, /* R CH */
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, /* C CH */
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, /* LS CH */
    1, 1, 1, 1, 1, 1, 1, 1, 1, 1, /* RS CH */
    /* freq */
    100, 1000, 31, 62, 125, 250, 500, 2000, 4000, 8000, /* L CH */
    100, 1000, 31, 62, 125, 250, 500, 2000, 4000, 8000, /* R CH */
    100, 1000, 31, 62, 125, 250, 500, 2000, 4000, 8000, /* C CH */
    100, 1000, 31, 62, 125, 250, 500, 2000, 4000, 8000, /* LS CH */
    100, 1000, 31, 62, 125, 250, 500, 2000, 4000, 8000, /* RS CH */
    /* Q: 0.01~10 -> 1~1000 */
    100, 100, 100, 100, 100, 100, 100, 100, 100, 100, /* L CH */
    100, 100, 100, 100, 100, 100, 100, 100, 100, 100, /* R CH */
    100, 100, 100, 100, 100, 100, 100, 100, 100, 100, /* C CH */
    100, 100, 100, 100, 100, 100, 100, 100, 100, 100, /* LS CH */
    100, 100, 100, 100, 100, 100, 100, 100, 100, 100, /* RS CH */
    /* Gain: -15.0~15.0dB -> -150~150 */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* L CH */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* R CH */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* C CH */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* LS CH */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* RS CH */
};
```

Userspace 层配置:

可参考 [loudness.xml](#) 进行配置。

17.4.5.1.3.70 PEQ_Tuning

【描述】

7 段参量均衡器设置：用于微调较准输出的幅频响应

【参数】

21 个参数：类型是 int32；band_1:freq/Q/Gain, band_2:freq/Q/Gain,...band_7:freq/Q/Gain;

- Freq: 范围是 20 到 20000(Hz); 调整单位 1Hz
- QFactor: 范围是 1 到 1000; 对应实际的 0.01~10. 调整单位 0.01
- Gain: 范围是-150 到 150; 对应-15dB 到 15dB, 调整单位 0.1dB

```
int32_t FILTER_TABLE[202] = {
    /* enable */
    1,
    /* bar 1~2 for loudness, 3~9 for PEQ, 10: reserved */
    9,
    /* filter type: 1:PEAK;2:LPF;3:HPF;4:LSF;5:HSF */
    1, 1, 1, 1, 1, 1, 1, 1, 1, /* L CH */
    1, 1, 1, 1, 1, 1, 1, 1, 1, /* R CH */
    1, 1, 1, 1, 1, 1, 1, 1, 1, /* C CH */
    1, 1, 1, 1, 1, 1, 1, 1, 1, /* LS CH */
    1, 1, 1, 1, 1, 1, 1, 1, 1, /* RS CH */
    /* freq */
    100, 1000, 31, 62, 125, 250, 500, 2000, 4000, 8000, /* L CH */
    100, 1000, 31, 62, 125, 250, 500, 2000, 4000, 8000, /* R CH */
    100, 1000, 31, 62, 125, 250, 500, 2000, 4000, 8000, /* C CH */
    100, 1000, 31, 62, 125, 250, 500, 2000, 4000, 8000, /* LS CH */
    100, 1000, 31, 62, 125, 250, 500, 2000, 4000, 8000, /* RS CH */
    /* Q: 0.01~10 -> 1~1000 */
    100, 100, 100, 100, 100, 100, 100, 100, 100, 100, /* L CH */
    100, 100, 100, 100, 100, 100, 100, 100, 100, 100, /* R CH */
    100, 100, 100, 100, 100, 100, 100, 100, 100, 100, /* C CH */
    100, 100, 100, 100, 100, 100, 100, 100, 100, 100, /* LS CH */
    100, 100, 100, 100, 100, 100, 100, 100, 100, 100, /* RS CH */
    /* Gain: -15.0~15.0dB -> -150~150 */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* L CH */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* R CH */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* C CH */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* LS CH */
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, /* RS CH */
};
```

17.4.5.1.3.71 Set_Biquad_IIR

【描述】

Biquad IIR 音效参数设置，目前支持最多 20 段 Biquad IIR，每段 Biquad IIR 配置 7 个参数，分别为如下参数描述。Loudness V2 版本依靠其中的第 0 段和第 1 段实现，custom Bass 依靠其中的第 2 段实现，custom Middle 依靠其中的第 3 段实现，custom Treble 依靠其中的第 4 段实现，5/10 段 EQ 依靠其中的第 5~9/5~14 段实现。

【参数】

每段包含 7 个参数，分别为：

para[0]: band id, 0~19

para[1]: enable, 0 表示 disable, 1 表示使用 b0、b1、b2、a1、a2 作为 IIR 的参数, 2 表示使用 type、frequency、Q、gain 作为 IIR 的参数

para[2]: Coef[0], 当 enable 为 1 时, 表示 b0; 当 enable 为 2 时, 表示 filter type, 可选

0:none;1:peak;2:low pass;3:high pass;4:low shelf;5:high shelf;6:band pass;7:norch;8:all pass;

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

521 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

para[3]: Coef[1], 当 enable 为 1 时, 表示 b1; 当 enable 为 2 时, 表示 frequency, 20~20KHz, step 为 1Hz

para[4]: Coef[2], 当 enable 为 1 时, 表示 b2; 当 enable 为 2 时, 表示 Q 值, 1~1000, 含义为 0.01~10

para[5]: Coef[3], 当 enable 为 1 时, 表示 a1; 当 enable 为 2 时, 表示 gain 值, -150~150, 含义为 -15.0dB~15.0dB

para[6]: Coef[4], 当 enable 为 1 时, 表示 a2; 当 enable 为 2 时, reserved

17.4.5.1.3.72 Set_iir_4ch

【描述】

4 声道 2 阶 IIR 音效配置, 该接口与下方 Set_iir_5ch 在使用时是二选一, 不能同时使用。目前支持 32 段 2 阶 IIR 配置, 每段包含 22 个参数, 参数含义如下描述。

【参数】

每段包含 22 个参数, 支持 b0、b1、b2、a1、a2 作为 IIR 的参数或者使用 type、frequency、Q、gain 作为 IIR 的参数。

1. 当使用 b0、b1、b2、a1、a2 作为 IIR 的参数时, 参数含义如下:

para[0]: band id; 0~31

para[1]: enable; 0: disable; 1: enable

para[2]: CH_L->B0 (fixed point Q28 Format: 4.28, area[-8,8])

para[3]: CH_L->B1 (fixed point Q28 Format: 4.28, area[-8,8])

para[4]: CH_L->B2 (fixed point Q28 Format: 4.28, area[-8,8])

para[5]: CH_L->A1 (fixed point Q28 Format: 4.28, area[-8,8])

para[6]: CH_L->A2 (fixed point Q28 Format: 4.28, area[-8,8])

para[7]~para[11]: CH_R->type/frequency/Q/Gain/reserved

para[12]~para[16]: CH_LS->type/frequency/Q/Gain/reserved

para[17]~para[21]: CH_RS->type/frequency/Q/Gain/reserved

2. 当使用 type、frequency、Q、gain 作为 IIR 的参数, 参数含义如下:

para[0]: band id; 0~31

para[1]: enable; 0: disable; 2: enable

para[2]: CH_L->type; 0~8: (UI Show:0:none;1:peak;2:low pass;3:high pass;4:low shelf;5:high shelf;6:band pass;7:norch;8:all pass)

para[3]: CH_L->frequency; 20~20000

para[4]: CH_L->Q; 1~1000 , 含义为 0.01 ~ 10.00

para[5]: CH_L->gain; -150~150, 含义为-15.0dB ~ 15.0dB

para[6]: reserved. write to 0

para[7]~para[11]: CH_R->type/frequency/Q/Gain/reserved

para[12]~para[16]: CH_LS->type/frequency/Q/Gain/reserved

para[17]~para[21]: CH_RS->type/frequency/Q/Gain/reserved

17.4.5.1.3.73 Set_iir_5ch

【描述】

5 声道 2 阶 IIR 音效配置，该接口与上方 Set_iir_4ch 在使用时是二选一，不能同时使用。目前支持 32 段 2 阶 IIR 配置，每段包含 27 个参数，参数含义如下描述。

【参数】

每段包含 27 个参数，支持 b0、b1、b2、a1、a2 作为 IIR 的参数或者使用 type、frequency、Q、gain 作为 IIR 的参数。

1. 当使用 b0、b1、b2、a1、a2 作为 IIR 的参数时，参数含义如下：

para[0]: band id; 0~31

para[1]: enable; 0: disable; 1: enable

para[2]: CH_L->B0 (fixed point Q28 Format: 4.28, area[-8,8])

para[3]: CH_L->B1 (fixed point Q28 Format: 4.28, area[-8,8])

para[4]: CH_L->B2 (fixed point Q28 Format: 4.28, area[-8,8])

para[5]: CH_L->A1 (fixed point Q28 Format: 4.28, area[-8,8])

para[6]: CH_L->A2 (fixed point Q28 Format: 4.28, area[-8,8])

para[7]~para[11]: CH_R->type/frequency/Q/Gain/reserved

para[12]~para[16]: CH_C->type/frequency/Q/Gain/reserved

para[17]~para[21]: CH_LS->type/frequency/Q/Gain/reserved

para[22]~para[26]: CH_RS->type/frequency/Q/Gain/reserved

2. 当使用 type、frequency、Q、gain 作为 IIR 的参数，参数含义如下：

para[0]: band id; 0~31

para[1]: enable; 0: disable; 2: enable

para[2]: CH_L->type; 0~8: (UI Show:0:none;1:peak;2:low pass;3:high pass;4:low shelf;5:high shelf;6:band pass;7:norch;8:all pass)

para[3]: CH_L->frequency; 20~20000

para[4]: CH_L->Q; 1~1000 , 含义为 0.01 ~ 10.00

para[5]: CH_L->gain; -150~150, 含义为-15.0dB ~ 15.0dB

para[6]: reserved. write to 0

para[7]~para[11]: CH_R->type/frequency/Q/Gain/reserved

para[12]~para[16]: CH_C->type/frequency/Q/Gain/reserved

para[17]~para[21]: CH_LS->type/frequency/Q/Gain/reserved

para[22]~para[26]: CH_RS->type/frequency/Q/Gain/reserved

17.4.5.1.3.74 lmtr_enable

【描述】

Limiter 开关

【参数】

一个参数：开关；类型是 unit32；范围是 0 ~ 1；默认值是 0

0 表示关闭；1 表示打开。

17.4.5.1.3.75 lmtr_gain

【描述】

Limiter gain 值配置

【参数】

一个参数：类型是 unit32；范围是 0 ~ 120；含义为 0~ -12dB

17.4.5.1.3.76 lmtr_pre_gain

【描述】

Limiter pre gain 值配置

【参数】

一个参数：类型是 unit32；范围是 0 ~ 6；含义如下表

表 17-45 limiter pre gain 对应表

Value	0	1	2	3	4	5	6
dB	0	-6	-12	-18	-24	-30	-36

17.4.5.1.3.77 lmtr_post_gain

【描述】

Limiter post gain 值配置

【参数】

一个参数：类型是 unit32；范围是 0 ~ 6；含义如下表

表 17-46 limiter post gain 对应表

Value	0	1	2	3	4	5	6
dB	0	6	12	18	24	30	36

17.4.5.1.3.78 spectrum_mode

【描述】

Spectrum mode 设置

【参数】

一个参数：类型是 unit32；支持 0,32,64,128,256,512,1024

当配置为 0 时，即关闭 spectrum

17.4.5.1.3.79 spectrum_data

【描述】

Spectrum 频谱数据

【参数】

无，该接口仅支持 get

17.4.5.1.4 使用说明

Kcontrol 参数分为单个 int 类型和 int 数组类型，userspace 层在调用 tinyalsa 提供的标准接口设置两类 Kcontrol 时需要区分参数是单个值还是数组。ATC 提供了 demo code，便于客户自定义音效接口，code 路径为 source/vendor/autochips/proprietary/frameworks/atcaudio/utls/AudioEffectUtils.cpp。如下为参数分别为单个值和数组时的调用方式：

```
//通过 tinyalsa 提供的 mixer_ctl_set_value 接口设置单个 int32_t 类型的值
bool AudioEffectUtils::setEffectValue(const char *kControlName, int32_t effectValue) {
    ALOGD("setEffectValue %s, %d", kControlName, effectValue);
}
```

```
    if (mixer_ctl_set_value(mixer_get_ctl_by_name(mMixer, kControlName), 0,
effectValue))
    {
        ALOGE("mixer_ctl_set_value error");
        return false;
    }
    return true;
}
```

//通过 tinyalsa 提供的 mixer_ctl_set_array 接口设置 int32_t 类型、大小为 size 的数组

```
bool AudioEffectUtils::setEffectValue(const char *kControlName, int32_t *effectValues,
int32_t size) {
    int32_t *tlv = NULL;
    int32_t tlv_size = 0;
    char *buf = NULL;
    int sizeInByte = size * sizeof(int32_t);
    tlv_size = sizeInByte + 2 * sizeof(int32_t);
    buf = (char *)calloc(1, tlv_size);
    if (buf == NULL) {
        ALOGE("failed to alloc mem for bytes %d", tlv_size);
        return false;
    }
    tlv = (int32_t *)buf;
    tlv[0] = 0;
    tlv[1] = sizeInByte;
    memcpy(&tlv[2], effectValues, sizeInByte);
    if (mixer_ctl_set_array(mixer_get_ctl_by_name(mMixer, kControlName), (const void
*)tlv, tlv_size)) {
        ALOGE("mixer_ctl_set_value error");
        free(buf);
        return false;
    }
    free(buf);
    return true;
}
```

17.4.6 Debug 方法

在设置完音效参数之后，可以通过 CLI 方式查看音效的值是否正确。若设置音效参数直接采用的是 Kcontrol 方式，也可以直接通过 tinymix 查看当前的音效值是否正确。推荐采用 CLI，该值显示的是当前 DSP 使用的音效参数。

17.4.6.1 CLI

基本命令：echo 0xXXX X X > /sys/audio/adsp，其中 0xXXX 表示当前需要查看的音效参数寄存器地址，后面第一个 X 表示当前的操作（如读数据、test），在 test 操作时，第二个 X 表示需要设置的音效参数。具体不同音效读操作 CLI 详见下表，对于某一个音效，采用下表中的 CLI 读取参数后，可以通过 Kcontrol 中的参数进行对比 check 具体含义。

表 17-47 音效参数读取方式

音效feature	读操作CLI	tinymix方式
Reverb	echo 0x305 0x2C11C0 1 > /sys/audio/adsp	tinymix Reverb_Mode（推荐方式）
31段EQ	echo 0x211 0 0 > /sys/audio/adsp	tinymix Set_31EQ
Upmix	已移除	tinymix Upmix_Enable
Common Bass	已移除	tinymix Tone_Bass tinymix Common_Bass
Common Middle	已移除	tinymix Tone_Middle tinymix Common_Middle
Common Treble	已移除	tinymix Tone_Treble tinymix Common_Treble
filter	echo 0x506 0 0 > /sys/audio/adsp	tinymix Media_Master_Vol tinymix Media_Pri_Vol tinymix Media_Sec_Vol tinymix Sys_DL1_Vol tinymix Sys_DL2_Vol tinymix Sys_DL3_Vol tinymix Sys_DL4_Vol
volume	echo 0x504 0 0 > /sys/audio/adsp	tinymix Speaker_Layout tinymix Front_Speaker_Size tinymix Center_Speaker_Size tinymix Rear_Speaker_Size tinymix Subwoofer_Speaker_Size
speakerLayout	echo 0x305 0x2C1180 15 > /sys/audio/adsp	tinymix ATS_PARAM
Autosurround	echo 0x210 0 0 > /sys/audio/adsp	tinymix Sound_3D_Effect
CSAuto	echo 0x212 0 0 > /sys/audio/adsp	tinymix CH_L_Vol tinymix CH_R_Vol tinymix CH_LS_Vol

		tinymix CH_RS_Vol tinymix CH_C_Vol tinymix CH_Sub_Vol
Channel gain	已移除	tinymix Tone_Bass tinymix Common_Bass
Balance	echo 0x212 0 0 > /sys/audio/adsp	tinymix Balance_Fader_FL tinymix Balance_Fader_FR tinymix Balance_Fader_RL tinymix Balance_Fader_RR
Channel Delay	echo 0x213 0 0 > /sys/audio/adsp	tinymix CH_L_Delay tinymix CH_L_Delay_sample tinymix CH_R_Delay tinymix CH_R_Delay_sample tinymix CH_LS_Delay tinymix CH_LS_Delay_sample tinymix CH_RS_Delay tinymix CH_RS_Delay_sample tinymix CH_C_Delay tinymix CH_C_Delay_sample tinymix CH_Sub_Delay tinymix CH_Sub_Delay_sample
Loudness V2	Loudness Bass : echo 0x305 0x2c2004 6 > /sys/audio/adsp Loudness Treble : echo 0x305 0x2c2024 6 > /sys/audio/adsp	tinymix Loudness_Enable 具体参数推荐使用CLI查看
Custom Bass	echo 0x305 0x2c2044 6 > /sys/audio/adsp	具体参数推荐使用CLI查看
Custom Middle	echo 0x305 0x2c2064 6 > /sys/audio/adsp	具体参数推荐使用CLI查看
Custom Treble	echo 0x305 0x2c2084 6 > /sys/audio/adsp	具体参数推荐使用CLI查看
5/10段EQ	echo 0x305 0x2c20A0 40 > /sys/audio/adsp	具体参数推荐使用CLI查看
输出level	echo 0x401 1 0 > /sys/audio/adsp	tinymix out_level

17.4.7 用户定制化

17.4.7.1 如何从 kernel 层进行音效对接

- 确定音效 Spec 是否满足需求定义，可通过查看 Kcontrol 进行确认；
- 明确是否无需 ATC 音效 framework 层方案，若无需可通过如下移除 AudioSetting HIDL 进程

在 source/android/device/autochips/vendor/ac8x/common/device.mk 中将如下命令移除即可

```
# Audio settings hal
PRODUCT_PACKAGES += vendor.autochips.hardware.audiosettings@1.0-service \
                    vendor.autochips.hardware.audiosettings@1.0-impl \
                    vendor.autochips.hardware.audiosettings@1.0
```

- 依靠 Kcontrol 接口进行适配开发

若存在无法支持的音效，或者 Spec 不满足，请及时联系 ATC。

17.4.7.2 如何从 java 层进行音效对接

- 确认 java 接口中的音效定义和实现方式，满足需求定义；
- 依靠 java 接口进行适配开发。

若存在无法支持的音效，或者 Spec 不满足，请及时联系 ATC。

17.4.7.3 三方音效对接支持

根据音效的种类和对接方式不同，暂无法统一对接方式，请直接联系 ATC。

17.4.7.4 音效参数如何快速调节

ATC 提供电脑端 Tuning Tool，可实现音效参数的实时调节、参数保存功能，Tool 的使用详见 DebugTool 文档的 Audio Tuning 章节。

17.5 蓝牙/Carplay 电话部分

17.5.1 框架说明

AC8025 蓝牙通话，默认采用的是 AutoChips HAL 降噪消回音算法（运行在 ARM CPU 上）。

蓝牙通话(HAL ECNR)整体数据通路如下图所示：

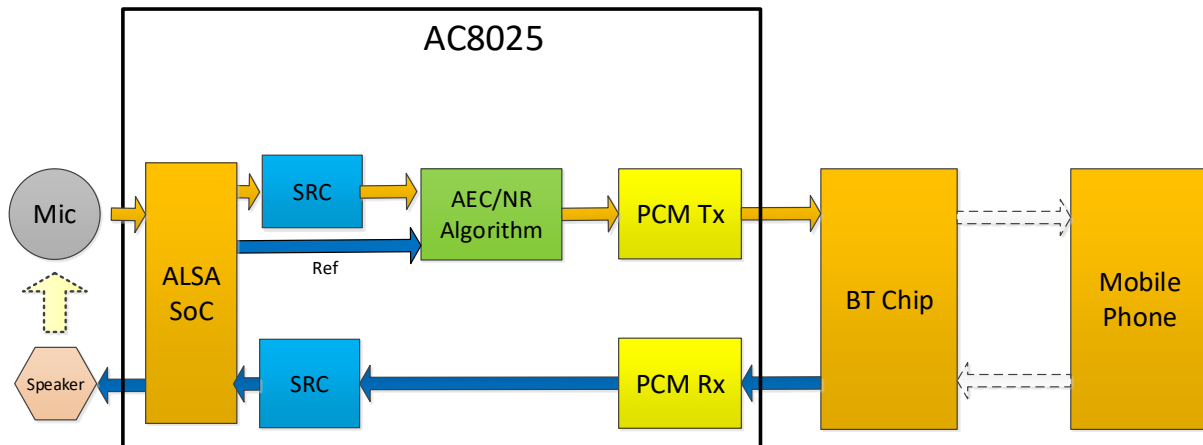


图 17-28 蓝牙通话数据通路

17.5.2 接口描述

ATC BT Speech 完整上下行通路 Audio 关键 API（API 的调用顺序不能改变）：

先获取 AudioManager 对象 `mAudioManager`,

1) SCO 连接 Enable Audio 流程:

```

mAudioManager.setMode(AudioManager.MODE_IN_COMMUNICATION);

mAudioManager.setParameters("atc_hfp_set_sampling_rate=8000"); //宽带 16000，窄带 8000

mAudioManager.setParameters("atc_hfp_enable=true");
    
```

2) SCO 断开 Disable Audio 流程:

```

mAudioManager.setParameters("atc_hfp_enable=false");

mAudioManager.setMode(AudioManager.MODE_NORMAL);
    
```

ATC audio hal 和 kernel 对 宽带 16000/窄带 8000 均支持，请蓝牙模块根据手机灵活支持并动态设置 `atc_hfp_set_sampling_rate`，现在多数手机支持宽带 16000。

17.5.3 HAL ECNR

17.5.3.1 蓝牙通话/Carplay 语音质量测试步骤

17.5.3.1.1 Audio Dump

要调整 BT 通话相关参数，首先需要将数据 dump 出来。

如下 2 个脚本内容，请使用者先保存到后缀为.bat 脚本文件中，脚本 1 和 2 均需执行，不是 2 选 1（脚本 1 在系统烧录后只需执行 1 次，脚本 2 每次系统重启均需再执行）。

说明：

- 步骤 1：脚本 1 仅第一次开机需运行 1 次，需重启生效（脚本已带执行完毕自动重启功能）（主要 dump aurisys ECNR 相关 pcm 文件和 bin 文件(类似 EPL)）
- 步骤 2：脚本 2 每次重新开机需运行 1 次，（与 P 类似，开启 steam in/ out / TX /RX dump）。

dump 文件产生位置： data/vendor/audiohal/audio_dump, 并提供 atclog。

脚本 1 内容：

```
adb root
adb shell setprop persist.vendor.aurisys.enable_raw_dump 1
adb shell setprop persist.vendor.aurisys.enable_lib_dump 1

::delay 3s to make sure property setting finish and reboot
ping -n 3 127.1>nul

adb reboot

pause
```

脚本 2 内容：

```
adb root

adb shell setprop vendor.streamout.pcm.dump 1
adb shell setprop vendor.streamin.pcm.dump 1

pause
```

17.5.3.1.2 配置方法

支持使用 system prop 的方式配置部分重要参数。



说明：

不建议自行调整 ECNR 参数，若有必要，请在 ATC 指导下调整。

调试相关参数，首先要看录音，根据实际情况，决定是否对参数做出调整。

请开发者在项目初期测试通话质量，并提供 Audio Dump 和 atclog 给到 ATC，以便确认 MIC IN 数据是否符合要求并提供建议。

17.5.3.2 主要参数介绍

17.5.3.2.1 降噪

AC8025 使用单 Mic 降噪。近端的 Mic 采集到的声音，通过降噪算法后再送到远端。

根据 dump 的文件，做出相应的调整。如果觉得噪音较大，可以调整 parameter8，值越大，控制力度越大，其中 bit6~8 为 Tx 方向的消噪参数，如下表：NR 力度控制。

表 17-48 NR 力度控制

Para8																
NR Strength	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Value					DL NR suppression strength			UL NR suppression strength								
Note					Level 0 ~ 7 000:variable alpha, original 001 ~ 110: decade suppression strength 111:NR off			Level 0 ~ 7 000: variable alpha, original 001 ~ 110: decade suppression strength 111: NR off								

17.5.3.2.2 消回音

回音相关的参数为 para0/para2/para12。

1) Para0: NLP Control

该值需要调试的是 bit0~bit7，值的范围为 0~255。如果看到有残留的一点回音，则可以将该值调大。

表 17-49 NLP 控制

Para0																
NLP	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Value	Manual clipping										NLP threshold and strength					
Note	0:off										Nonlinear processing threshold					

1: on	Value: 0 ~ 255
-------	----------------

2) Para2: ES Control

6212、5956、5444、5158、5256、5000、4232、2184、2218、1962。

以上参数表示消回音效果越来越强。

表 17-50 ES 控制

Para2	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
ES	ES suppression gain lower bound								Residual echo weighting -- nonlinear part					Residual echo weighting -- linear part		
Value		1	0	0	0	0	0	0	0	0	0	0	0	1	0	0
Note	Value: 0 ~ 127								Value: 0 ~ 15					Value: 0 ~ 15		

3) Para12: AES Control

Bit 0 ~ Bit 4: 舒适噪声的 lower bound 值，舒适噪声可以使背景噪声更平滑。该值越大，产生舒适噪声的 dB 值越大。

Bit 8 ~ Bit 14: AES lower bound，该值为 AES 的最小值。如果有部分回音无法消去，可以减小该值。

表 17-51 AES 控制

Para12	Bit 15	Bit 14	Bit 13	Bit 12	Bit 11	Bit 10	Bit 9	Bit 8	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
AES	AES gain lower bound												Minimum comfort noise value			
Value		1	0	1	0	0	0	1				0	0	1	0	1
Note	Value: 0 ~ 127												Value: 0 ~ 31			

17.5.3.3 注意事项

MIC IN 幅值注意事项:



说明:

当匹配不同硬件 MIC 时，由于 MIC 物理特性存在差异，需至少保证 MIC IN 数据不能削顶。

判别削顶的方法：

1. 距离 MIC 30cm 正常说话，用录音软件或 tinycap dump MIC 的录音数据；

Tinycap 录制 MIC 数据的方法：

```
adb root
```

```
cat /proc/asound/pcm -> 查看 ADC0 对应的节点
```

```
adb shell
```

```
tinycap sdcard/mic.wav -D 0 -d (ADC0 节点) -c 2 -r 48000 -b 16 -p 960 -n 6
```

2. 查看 MIC 录音数据，如图 17-29 所示为削顶。

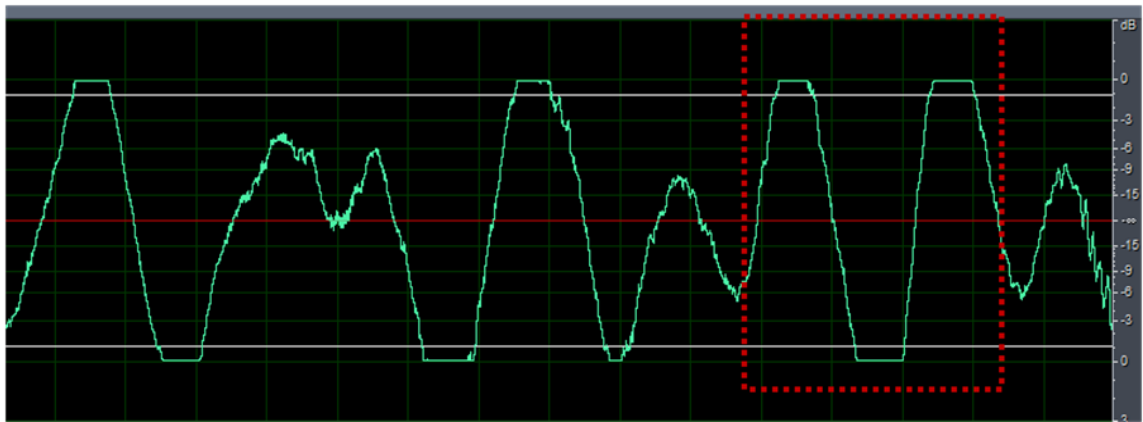


图 17-29 削顶的 MIC 录音数据

3. 为达到良好的 ECNR 效果，MIC IN 的数据不能削顶失真

若达不到此要求，可通过以下几种方法调整：

- 调节 analog gain；
- HW 添加分压电阻等方式。

请在项目 HW 定版前期，即需关注通话质量，并提供 audio dump 和 atclog 给到 ATC 进行 Double Check。

17.5.4 DSP ECNR

对于内置 DSP，ATC 提供 DSP ECNR 功能，其主要优势在于可以配合 Carplay 过苹果认证。有此方面需求的客户请咨询 ATC 商务。

17.6 FM/AuxIn 部分

AC8025 对于 FM/Auxin 功能上，支持两种方式：

- **Direct Mode:** 即数据直接经 Kernel 层输入和输出，打开对应的 ALSA device 节点即可。该种方式延迟低（60ms-80ms），但音量控制需要采用 ATC 私有接口进行控制；
- **Indirect Mode:** 即数据经 Framework 层输入和输出，通过 AudioRecord 输入，经 AudioTrack 输出。该种方式音量设置对应的 StreamType 音量即可，但由于通路 path 变长，延迟在 200-400ms。

两种方式均被 ATC 支持，并采用 AudioFocus 进行源控制，音量接入 VolumePolicy 进行控制。

17.6.1 硬件说明

FM/Auxin 输入经 I2S 数字接入，可以配置 I2S0-I2S3。

17.6.2 Direct mode

17.6.2.1 数据通路和控制通路说明

Direct mode 下的 auxin 功能是通过 Android 原生 createAudioPatch API 实现的。

根据 APP 传入的 fmt 选择对应的声卡设备 ATC_MACHINE_STREAM_I2Sx 进行操作，实现从模拟或者数字进行信号采集，声卡设备会将采集的数据填写到 DSP Buffer 从车机端喇叭播放出来。

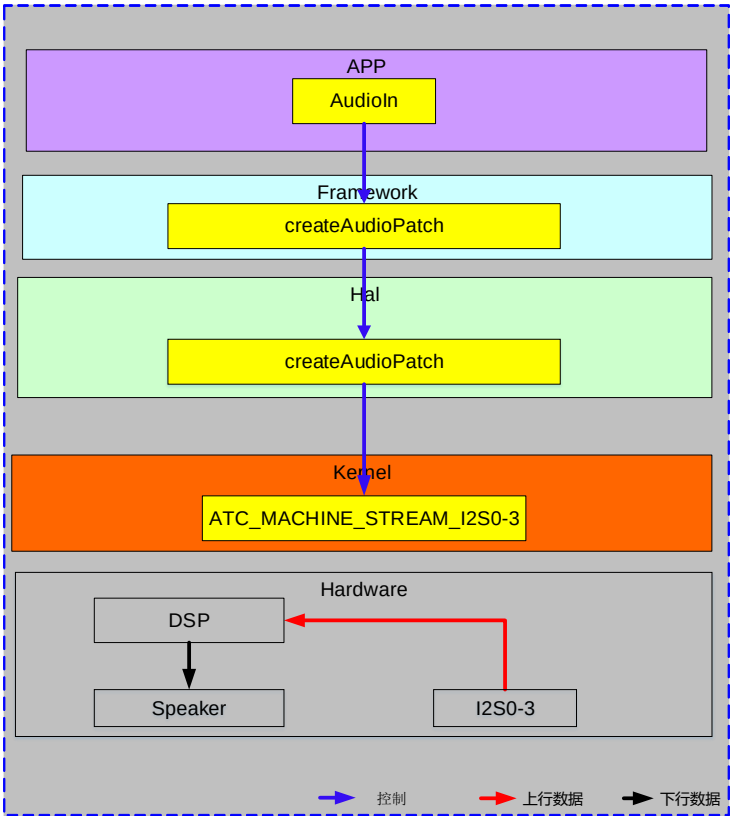


图 17-30 Direct Mode 数据通路图

17.6.2.2 接口说明

17.6.2.2.1 Java 部分

Java 代码使用 android.media.AudioManager 类的 createAudioPatch

【描述】

创建 audiopatch，在 Hal 层打开相关数据通路。

【语法】

```
public static int createAudioPatch(AudioPatch[] patch, AudioPortConfig[] source,
AudioProtConfig[] sinks);
```

【返回值】

- SUCCESS, 创建成功
- ERROR_BAD_VALUE, 不支持的参数
- ERROR_INVALID_OPERATION, 不支持的操作

- `ERROR_PERMISSION_DENIED`, 未授予相关权限
- `ERROR_DEAD_OBJECT`, `audioserver` 进程 dead
- `ERROR`, 其他原因导致的错误

【参数】

- `patch`, 输出参数, 代表创建好的 `AudioPatch`
- `sources`, `type` 为 `device` 的输入源
- `sinks`, `type` 为 `device` 的输出源

17.6.2.3 配置说明

默认情况下 `auxin` 是 `direct mode`, 无需额外配置。

17.6.2.4 Debug 方法

- 通过 `lsdf | grep pcm` 查看当前打开的 `ALSA device` 节点信息, 通过 `device ID` 与名称的对比确定配置是否成功。
- 如果没有对应的 `device` 信息则表示没有打开成功, 则需要通过 `App` 和 `audio` 相关 `log` 查看具体出错的原因。

17.6.3 Indirect mode

17.6.3.1 数据通路和控制通路说明

`Indirect mode` 下的 `auxin` 是通过 `AudioTrack` 和 `AudioRecord` 由 `AudioIn App` 实现的, 在 `audio-base.h` 中自定义了 `AUDIO_SOURCE_DIGITAL_IN` `audio_source_t` 类型来表示 `auxin` 输入源, 在 `AudioIn App` 中通过选择选择其中一种 `source` 由 `AudioRecord` 将输入数据录制到 `app` 中, 再通过 `AudioTrack` 将录制的数据进行播放出来。

切换不同的通路只需要选择对应的 `source` 重新创建 `AudioRecord` 即可, 详细请看 `AudioIn APP` 的源码, 这样便实现了使用不同输入源的数据从车机端喇叭播放出来。

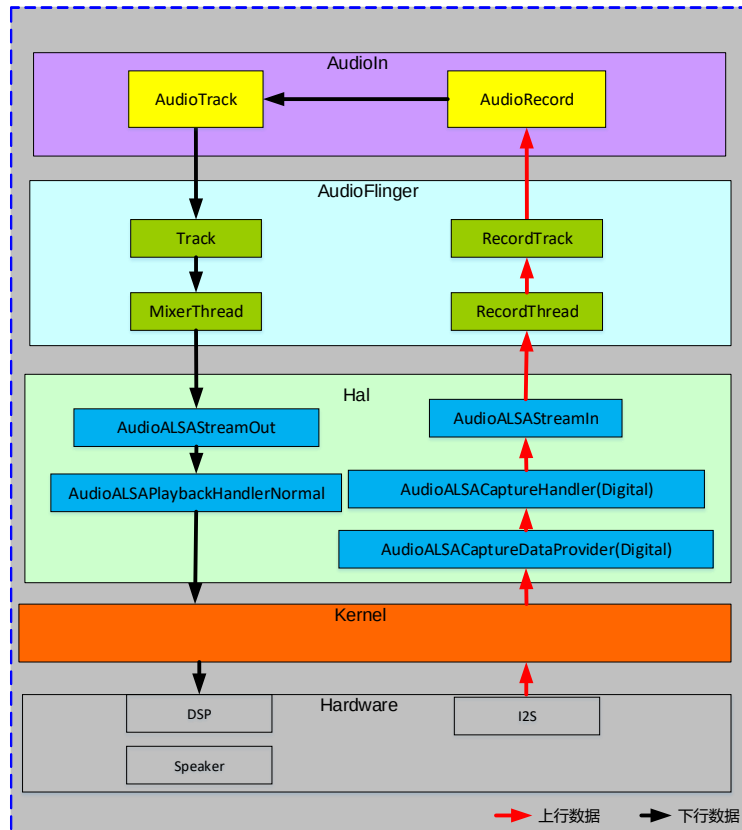


图 17-31 Indirect Mode 数据通路图

17.6.3.2 接口说明

由于 indirect mode 使用的是 AudioRecord 和 AudioTrack 进行录制和播放，只需参考标准接口用法即可，需要注意的是传入 AudioRecord 的 source type 需要选择自定义 source 中的一种（在 MediaRecorder.java 中定义了 DIGITAL_IN）。

17.6.3.3 配置说明

默认 auxin 配置即为 direct mode，修改为 indirect mode 需要修改 audioCustomConfig.xml，Auxin 输入配置按照[输入配置](#)进行配置，并重启系统即可生效。

17.6.3.4 Debug 方法

首先可通过 adb shell dumpsys media.audio_flinger 查看当前有对应的 active track，Track 数据如下：

1 Tracks of which 1 are active

```

Type   Id Active Client Session Port Id S Flags  Format Chn mask SRate ST Usg CT  G db
L dB  R dB  VS dB  Server FrmCnt  FrmRdy F Underruns  Flushed  Laten
cy

      57  yes  3623   25   19 A 0x000 00000001 00000003 48000 17 17 a -32  0  0
0 00036000 15360 15360 A      0      0 unava
il

```

Active 表示当前 Track 是否正在播放，yes 表示正在播放，no 表示不在播放。SRate 表示 samplerate，此处为 48000Hz，ST 表示 streamType，analog in 为 AUDIO_STREAM_AUXIN。

RecordTrack 数据如下：

```

1 Tracks of which 1 are active

Active   Id Client Session Port Id S Flags  Format Chn mask SRate Source  Server FrmCnt
FrmRdy Sil  Latency

    yes   56  3623   17   18 A 0x000 00000001 0000000C 48000   B 0003A5C0 15360
1216 n  unavail

```

SRate 为 48000，表示 AudioRecord 录制的是 48000Hz 采样率的数据。Source 表示对应的 sourceType，此处为 B，即 11，表示 AUDIO_SOURCE_DIGITAL_IN。

如果有对应的 active track 在录制和播放则进一步通过 dump framework 层和 hal 层的数据看看是在哪一部分出现问题。如果没有对应的 active track 则表示没有录制或播放成功，则需要通过 App 和 audio 相关 log 查看具体出错的原因。

17.7 语音识别 support 部分

本节将对 AC8025 系统上对语音识别的支持做详细说明。语音识别算法不由 ATC 提供，ATC 主要提供 MIC 数据和参考音数据。为更好的支持语音识别效果，根据当前的硬件情况，ATC 提供基于 DSP 拼接或者基于 I2S TDM 方式的组合数据，即将 MIC 和参考音按照一定的方式进行组合，通过组合数据的方式避免 MIC 和参考音数据的对齐不稳定或者时延相差比较大的异常情况，保证语音识别的效果。

17.7.1 DSP 四声道拼接

DSP 四声道拼接方式指的是在 DSP 中将 ADC 拿到的 MIC 数据和当前的参考音数据（即车机播放的数据）进行对齐并按照预定格式进行拼接，对接 ALSA driver，userspace 通过 tinypalsa 的标准接口将拼接好的数据 read 上去。二者对齐的标准是参考信号和 MIC 录制到的参考信号抖动范围为 5ms 以内。

17.7.1.1 拼接格式说明

四声道的拼接格式为：

Mic (左)	Mic (右)	Ref (左)	Ref (右)
16bit	16bit	16bit	16bit

图 17-32 四声道拼接格式

17.7.1.2 数据流图

四声道拼接数据流图如下所示，分别从 MIC 和输出端拿到 MIC 数据和参考音数据，格式分别为 48KHz、2 声道、16bit，在 sync 完成后，根据 Audio HAL 层配置的数据格式进行 reformat、SRC 操作，并且将二者分别放在对应的声道位置上。Audio HAL 直接通过 pcm_read 方法将四声道数据读取。

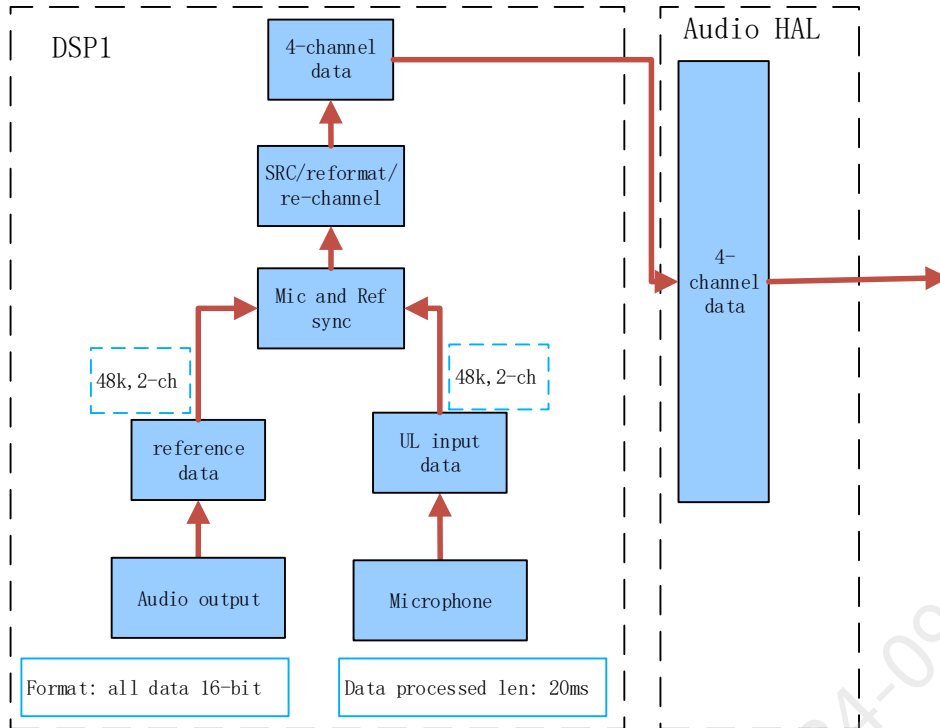


图 17-33 DSP 四声道拼接数据流图

17.7.1.3 支持情况

目前的 DSP 四声道拼接支持采样率为：8k、16k、24k、32k、48k。

支持 MIC 和参考音之间的时延抖动为 5ms。

17.7.1.4 使用方式

在使用四声道拼接数据对接语音识别模块时，若采用 AudioRecord 对接方式，配置方式满足以下几点即可：

- AudioRecord 创建的参数为 2 声道、需要的采样率、16bit；source 为 MediaRecorder.AudioSource.VOICE_RECOGNITION；
- 配置 property：setprop vendor.audiodsp_mixMicRef.support 1
- 配置 vendor/etc/audioCustomConfig.xml 中的采样率为需要的采样率，对于该功能，xml 中的可修改部分如下：

```
<vrConfig>
    <vr attribute="sampleRate">16000</vr>
    <vr attribute="channel">4</vr>
</vrConfig>
```

17.8 FastAudio

本节所述的 FastAudio 功能特指依赖 AC8025 Safety 系统的硬件与 FreeRTOS 软件实现的 Audio 快速出声的功能。

FastAudio 功能主要包含：

- Safety OS 上 Audio 出声，支持倒车、开机以及 Chime 音（转向灯、报警声和提示音等用户定制声音）等声音；
- 平台可以支持 I2S Out 模式输出，支持 I2S MC/I2S SC 2 声道模式或 TDM 模式输出；
- 支持 Channel Map，可以对音频源声道进行扩展（如 1 声道扩展为 2/4/6 声道）或指定某声道在指定喇叭输出（如指定音频源的左声道在前左和后左喇叭输出，音频源的右声道在前右和后右喇叭输出）；
- Safety OS 上播放的音频文件格式为 Wav，支持的音频文件 16-bit 48k，1/2/4/6-channel；
- 将音频资源通过 MRF TOOL 制作到 Datazone 中，MRF TOOL 的使用请参考 MRFTool 文档；
- 可以根据不同的应用场景，更换音频 WAV 文件；注意，FastAudio 不支持增益，所以尽量使用 0dB 大小的音频文件；
- 统计从冷启动开机到 Fast Audio 声音开始播放，能够做到 0.5s 内出声；
- 支持 Audio Substream Type 优先级；
- 支持音频文件单次或循环播放，支持占空比形式循环播放；
- 支持静态及动态调节音量功能，音量掉电保存功能；

- 支持多路 Mix 功能；
- 支持 Volume Policy 功能。

17.8.1 硬件环境

Fast Audio 涉及硬件操作，包括 I2S MC、I2S SC、DMA 的 AC8025 芯片。

17.8.2 代码结构及文件描述

表 17-52 Fast Audio 代码结构及描述

文件名	文件路径	文件描述
all	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/audio	Fast Audio 核心代码
audio_driver.h	hsm/os/drivers/include	客户使用接口头文件
audio_customer_config.h	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/audio/customer	客户硬件相关定制化配置
audio_customer_config.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/audio/customer	客户硬件相关定制化配置
audio_volume_policy_settings.h	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/audio/customer	Volume Policy 定制化功能

17.8.3 框架图

17.8.3.1 Fast Audio 整体框架图

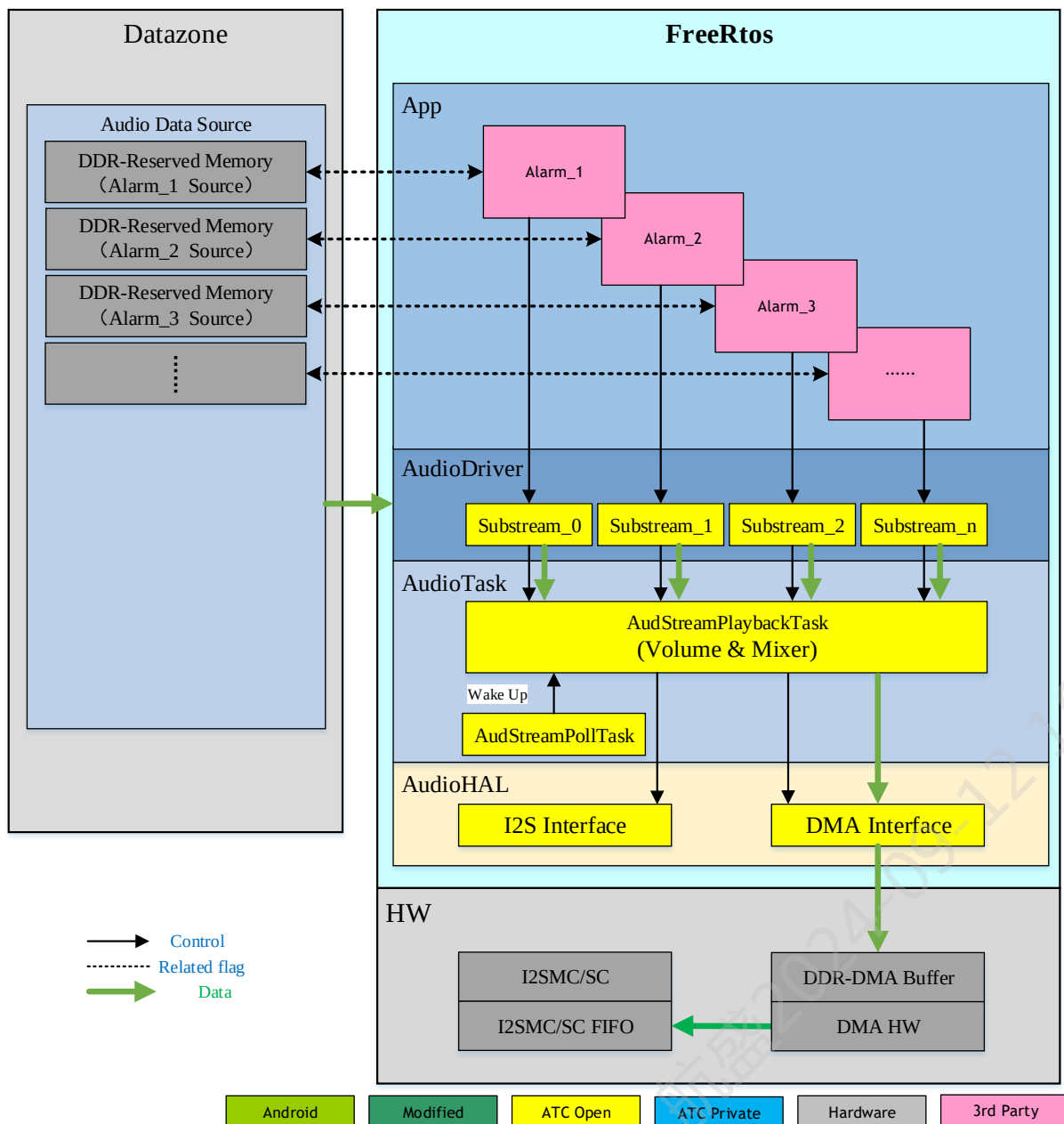


图 17-34 Fast Audio 整体框架图

17.8.3.2 Safety & IVI OS 架构图

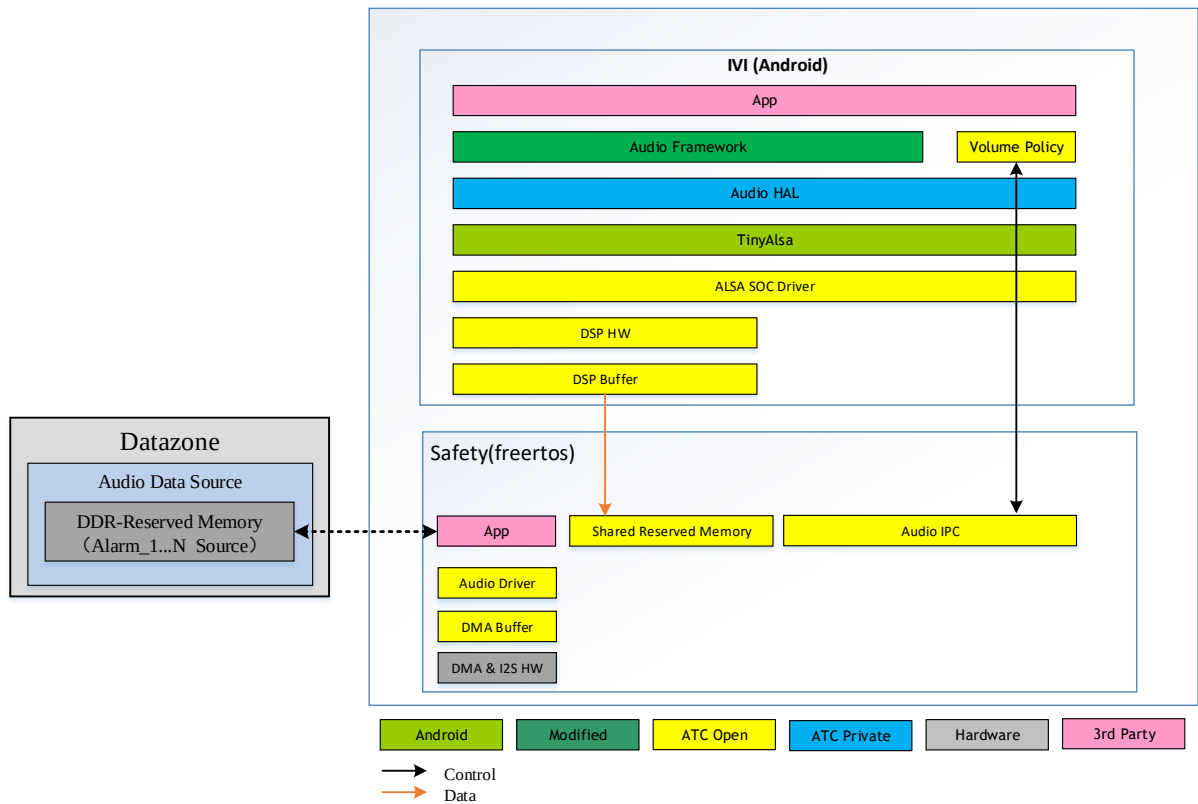


图 17-35 Safety & IVI OS 架构图

17.8.4 功能描述

17.8.4.1 快速出声

从冷启动开机到 Fast Audio 开始出声，支持 0.5 秒内出声。

17.8.4.2 多路出声

该快速出声模块支持多路音频数据同时出声，如倒车和开机音等同时出声。

17.8.4.3 Volumepolicy

Fast Audio 支持多路 Substream 同时播放，并支持多路播放时对 Volume 进行管控，如客户需要定制化 Volume 管控，修改 g_AudioPolicyArray 数组即可。具体看参考 17.8.6.3 章节。

17.8.5 接口描述

17.8.5.1 vAudioOpen

【描述】

用于告知 Safety Audio Driver，标记该 APP 需要使用 Safety Audio。该函数与 vAudioClose 必须成对使用，且这两个函数不必频繁调用，APP 只需在 init 时调用 vAudioOpen，在不再需要 Safety Audio 资源后，进行 close 即可。

【语法】

```
void vAudioOpen(uint32_t priority)
```

【参数】

priority: 已定义的优先级，用于区别上层 APP，目前支持的优先级如下所示：

```
#define PRIORITY_BC    0    (RVC 声音使用)
#define PRIORITY_VBA   1    (开机动画声音使用)
#define PRIORITY_CHIME 2    (Chime 声音使用)
```

【示例代码】

```
vAudioOpen(PRIORITY_BC);
```

17.8.5.2 vAudioClose

【描述】

用于告知 Safety Audio Driver，该 APP 不再使用 Fast Audio 资源。

【语法】

```
void vAudioClose(uint32_t priority)
```

【参数】

priority: 已定义的优先级，用于区别上层 APP，目前支持的优先级如下所示：

```
#define PRIORITY_BC    0    (RVC 声音使用)
#define PRIORITY_VBA   1    (开机动画声音使用)
#define PRIORITY_CHIME 2    (Chime 声音使用)
```

【示例代码】

```
vAudioClose(PRIORITY_BC);
```

17.8.5.3 vAudioConfigInit

【描述】

在配置参数前务必调用该函数。

【语法】

```
void vAudioConfigInit(aud_cfg_t *aud_cfg)
```

【示例代码】

```
aud_cfg_t aud_cfg;
vAudioConfigInit(&aud_cfg);
```

17.8.5.4 xAudioStart

【描述】

用于开始播放音频数据。

【语法】

```
BaseType_t xAudioStart(uint32_t priority, aud_cfg_t *aud_cfg)
```

【参数】

- priority: 已定义的优先级;
- aud_cfg_t 的参数:
 - mrf_id: 音频类型定义, Chime 音频类型定义从 0x0~0xF0, 用户自行定义, 其值与 Datazone 中的 Chime 音频数据源的放置顺序要保持一致。非 Chime 音频类型, 则统一使用如下宏即可。

```
#define EXT_MRF_ID 0xFF
```

当为 Chime 音类型时, 无需配置 phys_addr 和 size 两个参数, 非 Chime 音频类型, Datazone 需要自行管理 (如 RVC 和 VBA), 需要配置 phys_addr 和 size 两个参数。

- offset: 需要播放的音频源物理地址的起始偏移地址, 默认为 0 是从头开始播放;
- cha_map: 控制喇叭输出, 使用的宏如下 (其他情况请自行参考定义)

MONO_FL 源 WAV 为单声道, 输出到前排左声道的喇叭

MONO_FR 源 WAV 为单声道, 输出到前排右声道的喇叭

MONO_F_STEREO 源 WAV 为单声道, 输出到前排左右声道的喇叭

MONO_RL 源 WAV 为单声道, 输出到后排左声道的喇叭

MONO_RL 源 WAV 为单声道, 输出到后排左声道的喇叭

MONO_R_STEREO 源 WAV 为单声道，输出到后排左右声道的喇叭

MONO_4_CH 源 WAV 为单声道，输出到前排和后排左右声道的喇叭

- volume: 自定义播放音量，取值为（0x0~0x20000，其中 0x20000 表示原声播放，其余音量都是降低声音播放，所以 wav 文件尽量取 0dB 的音频源）；
- loop: 是否循环播放的标志位，为 1 时进行循环播放，APP 需要调用 xAudioStop 停止播放，为 0 时则单次播放，播放完成后，将回调 cb 告知 APP 播放完成；
- duty_enable: 是使能占空比方式输出，1 使能，0 不使能
- period: 周期时长，单位是 ms，这里需要注意，传入的 wav 文件时长就是 active 的时间，如果需要设定某种特定占空比，需要提供相应的 wav 文件；
- cb: 回调函数，用于告知 APP 当前播放状态，其定义为

```
typedef void (*AUDIO_CB)(const uint32_t);
```

其中 const uint32_t 是常数 0，表示成功播放完成或 stop 完成；

如使用 mrf 为 EXT_MRF_ID，则需要配置以下两个参数

- phys_addr: 需要播放的音频源物理地址的起始地址（包括 wav header）
- size: 需要播放的音频源的长度（单位是 bytes）

【返回值】

- 操作成功返回 pdPASS
- 操作失败返回 pdFAIL

【示例代码】

```
aud_cfg_t aud_cfg;
vAudioConfigInit(&aud_cfg);
aud_cfg.mrf_id = CHIME_CLACK;
aud_cfg.volume = 0x10000;
aud_cfg.ch_map = MONO_F_STEREO;
aud_cfg.phys_addr = XXXX;
aud_cfg.size = XXXX;
ret = xAudioStart (EXT_MRF_ID, aud_cfg);
if (pdFAIL == ret) {
    printf("xAudioStart Failed!\n");
}
```

17.8.5.5 xAudioStop

【描述】

用于停止播放音频资源，对于以 loop = 1 start 的，必须以该函数结束播放，对于 loop = 0 start 的，也可以调用该函数停止播放，也可以等待回调函数被调用而获知是否播放完成，在播放完成后，不需要调用该函数，若调用则会返回 pdFAIL，不特殊处理即可，如 retry 等。

【语法】

BaseType_t xAudioStop(uint32_t priority)

【参数】

priority: 已定义的优先级，用于区别上层 APP

【返回值】

- 操作成功返回 pdPASS
- 操作失败返回 pdFAIL

【示例代码】

```
ret = xAudioStop (PRIORITY_BC);  
if (pdFAIL == ret) {  
    printf("xAudioStop Failed \n", ret);  
}
```

17.8.5.6 xAudioMuteOn

【描述】

用于 Mute 音频资源，该命令下发后，会使得对应优先级的音频静音播放，直到 APP 调用 xAudioMuteOff 后才会恢复播放。

【语法】

BaseType_t xAudioMuteOn(uint32_t priority)

【返回值】

- 操作成功返回 pdPASS
- 操作失败返回 pdFAIL

【示例代码】

```
ret = xAudioMuteOn (PRIORITY_VBA);  
if (pdFAIL == ret) {
```



```
printf("xAudioMuteOn Failed \n", ret);  
}
```

17.8.5.7 xAudioMuteOff

【描述】

用于恢复被 xAudioMuteOn 静音的通路。

【语法】

BaseType_t xAudioMuteOff(uint32_t priority)

【返回值】

- 操作成功返回 pdPASS
- 操作失败返回 pdFAIL

【示例代码】

```
ret = xAudioMuteOff (PRIORITY_VBA);  
if (pdFAIL == ret) {  
    printf("xAudioMuteOff Failed \n", ret);  
}
```

17.8.5.8 xAudioVolume

【描述】

用于动态调节音频资源的音量。

【语法】

BaseType_t xAudioVolume (uint32_t priority, uint32_t volume)

【参数】

- priority: 已定义的优先级，用于区别上层 APP
- volume: 自定义播放音量，取值为（0x0~0x20000，其中 0x20000 表示原声播放，其余音量都是降低声音播放，所以 wav 文件尽量取 0dB 的音频源）；

【返回值】

- 操作成功返回 pdPASS
- 操作失败返回 pdFAIL

【示例代码】

```
ret = xAudioVolume(PRIORITY_VBA, 0x20000);
if (pdFAIL == ret) {
    printf("xAudioVolume Failed \n", ret);
}
```

17.8.6 定制方法

定制硬件输出格式等，请直接修改定制化头文件 `audio_customer_config.h`，默认的路径为 `source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/audio/customer`。

17.8.6.1 硬件定制

- 设定 Safety OS 是否支持 Audio 功能
 - 设置 `VISS_NOT_USE_AUDIO` 为 0 表示 Safety OS 支持 Fast Audio 输出功能；
 - 设置 `VISS_NOT_USE_AUDIO` 为 1 表示 Safety OS 不支持 Fast Audio 输出功能。

- 设定输出设备的宏

`HW_DEVICE_OUTPUT` 用于定制何种硬件设备输出：

本头文件定义了四种可选的输出设备，分别是宏

`USE_I2S_SC0_AS_OUTPUT`（选择 I2S SC0 作为 Safety OS 最终硬件设备输出）

`USE_I2S_SC1_AS_OUTPUT`（选择 I2S SC1 作为 Safety OS 最终硬件设备输出）

`USE_I2S_SC2_AS_OUTPUT`（选择 I2S SC2 作为 Safety OS 最终硬件设备输出）

`USE_I2S_SC3_AS_OUTPUT`（选择 I2S SC3 作为 Safety OS 最终硬件设备输出）

`USE_I2S_SC4_AS_OUTPUT`（选择 I2S SC4（PCM0）作为 Safety OS 最终硬件设备输出）

`USE_I2S_MC_AS_OUTPUT`（选择 I2S MC 作为 Safety OS 最终硬件设备输出）

当前可支持使用 `dtb` 来配置该输出设备，在代码中修改该宏为

`g_device_dtb_info->audio_info.i2s_main_output` 即可，然后在 `dtb` 中找到 `dev_info` 节点，配置 `i2s_main_output` 属性的值即可，该属性的值同上述宏，在 `debug` 时，可以直接以宏赋值给 `HW_DEVICE_OUTPUT`，这样可以只需要编译 `safety` 即可。

- 可以直接修改部分
 - `HW_SAMPLE_RATE` 用于定制采样率，如 48K 则直接设置为 48000 即可；
 - `HW_MCLK_FS` 用于设置 main clock 大小，如设置 256，即是 256fs；

- HW_MODE 用于设置 master/slave，如 1 代表 master，0 代表 slave；
- HW_TRANS_MODE 用于设置 i2s 的传输模式，如 i2s 标准模式，设定为 0，其他模式参考 code 实现；
- TDM_BCLK_FS 用于设定 i2s tdm 模式时的 bit clock，如 256fs 则设定为 256；
- 如下为当前默认配置，若需修改需要反馈至 ATC 评估
 - HW_CHANNELS 用于设定输出声道数，当前默认为 6 声道输出；
 - HW_RESOLUTION 用于设定输出精度，当前默认为 24bit；
 - SUPPORT_TDM_FUNC 用于设定 i2s 模式还是 TDM 模式；
- 其他
 - 宏 SUPPORT_FADE_WHEN_LOOP 用于支持音频循环播放时的 fade out & in 功能；
 - 宏 SUPPORT_RESATRTR_FUNCTION 用于使用 xAudioStart 函数实现 Restart 功能，省去 stop 操作，快速生效，但注意本次调用 xAudioStart 的参数必须同前一次，如果参数改变，则请 Disable 该功能。

17.8.6.2 Chime 音定制

- SUPPORT_CHIME 用于设定是否支持 Chime 音；
- CHIME_RESOURCE_NUMS 用于表明 Datazone 中 Chime 音频源的数量；
- SUPPORT_CHIME_TEST 仅供测试使用，Release 版本应 Disable 掉；
- CHIME_XXX 是用户自定义的宏，用于表明音频源文件在 Datazone 中的放置顺序，该类宏将作为 xAudioStart 函数的参数。

17.8.6.3 Volumepolicy 定制

- SUPPORT_VOLUME_POLICY 用于设定是否应用 audio volume policy；
- SUPPORT_MAX_SUBSTREAM 表示 Fast Audio 总共支持的 Substream 数，当前默认是 3，如要增加 Substream，需要做对应修改。具体修改参考 17.8.6.4 章节使用方法。

当用户需要整体调节音量时，可调用 xAudioVolume 函数，其设置范围是 0~0x20000，其中 0x20000 表示以 wav 文件的原声播放，其他值则是音量衰减。

代码路径：source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/audio/customer/
audio_volume_policy_settings.h

```
/**
```

```
* Priority : substream_0 >= substream_1 >= substream_2
```

```
* policy rules:
*   value stands for percent of min(max priority volume, self volume).
*   NO_PLAY stands for no use.
*   When tune volume, just use max priority substream rules.
*/

static const int32_t
g_AudioPolicyArray[SUPPORT_MAX_SUBSTREAM_MIX][SUPPORT_MAX_SUBSTREAM_MIX] = {
    /* substream_0, substream_1, substream_2 */
    {   100, 0, 0},    /* substream 0 rules */
    {   NO_PLAY, 100, 50},    /* substream 1 rules */
    {   NO_PLAY, NO_PLAY, 100},    /* substream 2 rules */
};
```

请注意，优先级用 Level0~2 来表示，实际上对应到 substream0~2，它们是一一对应的，每个 substream 对应一个优先级。

当前默认支持 3 路音频同时 Mix，其中 NO_PLAY 表示没有使用的元素，使用的值是 0~100，0 表示静音，100 表示音量不受规则控制，0~99 的定义是当前最高优先级音源的 0%~99%。

当应用 AudioPolicy 规则时，只取用该规则的一行，具体取哪一行要看当前最高优先级，比如当前最高优先级是 1，则应用 substream 1 rules 的这一行。

g_AudioPolicyArray[0]代表的是当前播放最高优先级为 Level0 播放时的音量控制规则，此时，Level1 和 Level2 都要静音。

g_AudioPolicyArray[1]代表的是当前播放最高优先级为 Level1 播放时的音量控制规则，此时，Level2 需降低音量至 Level1 音量的 50%；如果 Level2 的音量原本就小于 Level1 的 50%，则保持其本身的音量（调小不调大原则）。

g_AudioPolicyArray[2]代表的是当前播放最高优先级为 Level2 播放时的音量控制规则，此时，由于没有比它优先级还低的音频源，所以没有有用的数值。

17.8.6.4 增加多路音频

需要修改的代码路径：(以下路径都相对于 source/vendor/autochips/proprietary/tinysys/hsm/os)

drivers/audio/customer/audio_customer_config.h

drivers/audio/customer/audio_volume_policy_settings.h

当前系统支持 3 路音频，每一路音频拥有一个 priority，如果需要配置大于 3 路音频，可以修改 SUPPORT_MAX_SUBSTREAM_MIX 的值，当额外增加音频 Path 时，请一定也要更新

AudioPolicy 的规则表格。例如，修改 `SUPPORT_MAX_SUBSTREAM_MIX = 5` 时，则 AudioPolicy 规则应该参考如下修改：

```
static const int32_t
g_AudioPolicyArray[SUPPORT_MAX_SUBSTREAM_MIX][SUPPORT_MAX_SUBSTREAM_MIX] = {
    /* substream_0, substream_1, substream_2, substream_3, substream_4,*/
    { 100, 0, 0, 0, 0 }, /* substream 0 rules */
    { NO_PLAY, 100, 50, 30, 20 }, /* substream 1 rules */
    { NO_PLAY, NO_PLAY, 100, 50, 40 }, /* substream 2 rules */
    { NO_PLAY, NO_PLAY, NO_PLAY, 100, 50 }, /* substream 3 rules */
    { NO_PLAY, NO_PLAY, NO_PLAY, NO_PLAY, 100 }, /* substream 4 rules */
};
```

请注意，此处所说 Level0~4，实际上对应到 substream0~4，它们是一一对应的，每个 substream 对应一个优先级。

`g_AudioPolicyArray[0]`代表的是当前播放最高优先级为 Level0 播放时的音量控制规则，此时，Level1~Level4 都要静音。

`g_AudioPolicyArray[1]`代表的是当前播放最高优先级为 Level1 播放时的音量控制规则，此时，Level2 需降低音量至 Level1 音量的 50%，Level3 需降低音量至 Level1 音量的 30%，Level4 需降低音量至 Level1 音量的 20%，遵守调小不调大原则。

`g_AudioPolicyArray[2]`代表的是当前播放最高优先级为 Level2 播放时的音量控制规则，此时，Level3 需降低音量至 Level2 音量的 50%，Level4 需降低音量至 Level2 音量的 40%，遵守调小不调大原则。

`g_AudioPolicyArray[3]`代表的是当前播放最高优先级为 Level3 播放时的音量控制规则，此时，Level4 需降低音量至 Level3 音量的 50%，遵守调小不调大原则。

`g_AudioPolicyArray[4]`代表的是当前播放最高优先级为 Level4 播放时的音量控制规则，此时，由于没有比它优先级还低的音频源，所以没有有用的数值。

其中，AudioPolicy 的值可以根据实际情况进行调节。

17.9 Audio IO

本章所述的 Audio IO 章节主要描述 AC8025 的 I2S/PCM 硬件环境及软件支持说明，AC8025 芯片共有 2 组 PCM、4 组 I2S SC、1 组 I2S MC。

17.9.1 硬件环境

17.9.1.1 I2S

表 17-53 I2S 资源

Audio IO	Numbers	描述
I2S_MC	1	Pins: MCLK/BCLK/LRCLK/SDO0/SDO1/SDO2/SDO3 支持 8~192K, 支持 Master, 支持 TX, 4 Data Pin 8 声道输出 支持 I2S 标准模式/I2S 左对齐/I2S 右对齐 支持 16/24/32bit Resolution
I2S_SC	4	Pins: MCLK/BCLK/LRCLK/SDIO/SDO/SDO2(Only I2S0) 支持 8~192K, 支持 Master/Slave, Slave mode 要求 Master Clock 常开 支持 TX/RX 及 TX RX 同时工作 支持 I2S 标准模式/I2S 左对齐/I2S 右对齐/I2S DSP 模式 支持 TDM 模式, 最大支持 16 声道, 其中 I2S SC0 支持 TDM 2Pin 输出 支持 16/24/32bit Channel Width 支持 16/24/32bit Resolution

17.9.1.2 PCM

表 17-54 PCM 资源

Audio IO	Numbers	描述
PCM	2	Pins: MCLK(Only PCM0)/BCLK/LRCLK/SDIO/SDO 支持 8~192K, 支持 Master/Slave, Slave Mode 要求 Master Clock 常开 支持 TX/RX 及 TX RX 同时工作 支持短帧/长帧模式 支持 16/24/32bit Channel Width 支持 16/24/32bit Resolution PCM0 支持 I2S 与 PCM 切换

17.9.2 软件支持说明

本节介绍 Demo Board 上可以直接使用的功能项, 其中 ALSA Driver 基于 Kernel 5.10 开发。

17.9.2.1 硬件模块框图

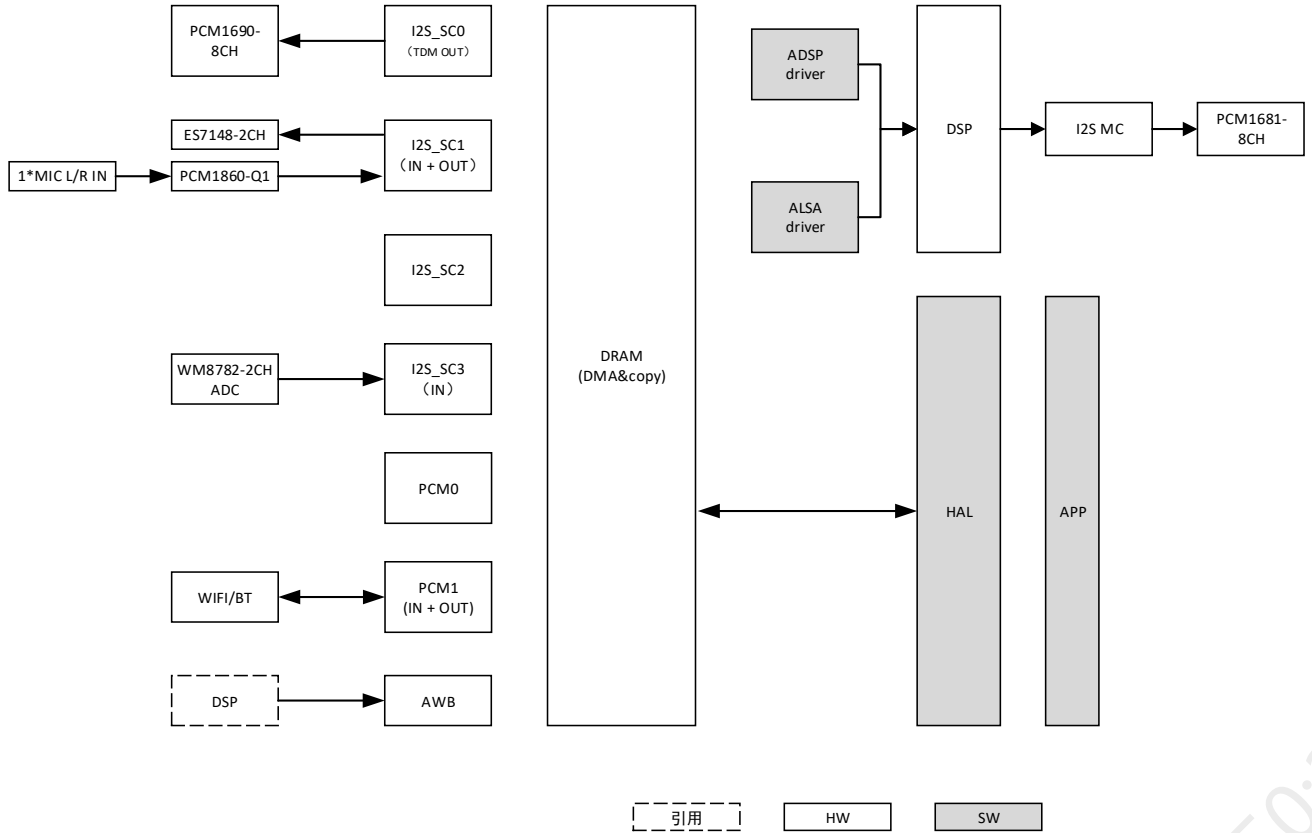


图 17-36 Audio IO 模块框图

17.9.2.2 功能支持情况

AC8025 Demo Board 上可以直接使用的功能项:

- 支持一路最多 6 声道音频从 I2S MC 输出，只支持 Master Mode。（如需要支持 8 声道需要进一步开发）



说明:

I2S-MC 被默认分配给内置 DSP 使用，不在 ALSA 架构中，I2S MC 固定成 48K Sample Rate 输出。

- 支持 I2S SC0 TDM 输出，4CH~8CH，需要 Rework 硬件，可替换 I2S MC 作为 6 声道主输出。
- 支持 I2S SC1 全双工，支持 2 声道 MIC 经过外置 ADC 转换后的输入，支持 2 声道输出，输入输出的采样率同为 48K。
- 支持 I2S SC3 输入，支持一路 2 声道 Line in 经过外置 ADC 转换后的输入，48K 采样率。

- 支持 PCM1 全双工，支持 8K/16/24/32K 采样率的长帧/短帧的模式输入与输出，连接外置 WIFI/BT 芯片

ALSA SoC Driver 当前支持的可配置的功能：

- 4 路 I2S_SC 均可支持 TDM 模式，最大支持到 16 Channel；
- 4 路 I2S_SC 均可支持全双工模式，单工输出模式或单工输入模式；
- 4 路 I2S_SC 均可支持 Master/Slave 模式；
- I2S_SC0 可以配置 TDM 双线模式
- PCM0 可以切换为 I2S SC 功能

17.9.2.3 代码结构及文件描述

表 17-55 Audio ALSA SOC Driver 代码结构及描述

文件名	文件路径	文件描述
all	source/kernel/atc_modules/audalsa/alsa_soc	Alsa 架构相关代码
all	source/kernel/atc_modules/audalsa/common	Common 代码
all	source/kernel/atc_modules/audalsa/hw	HW 相关代码

17.9.2.4 软件框图

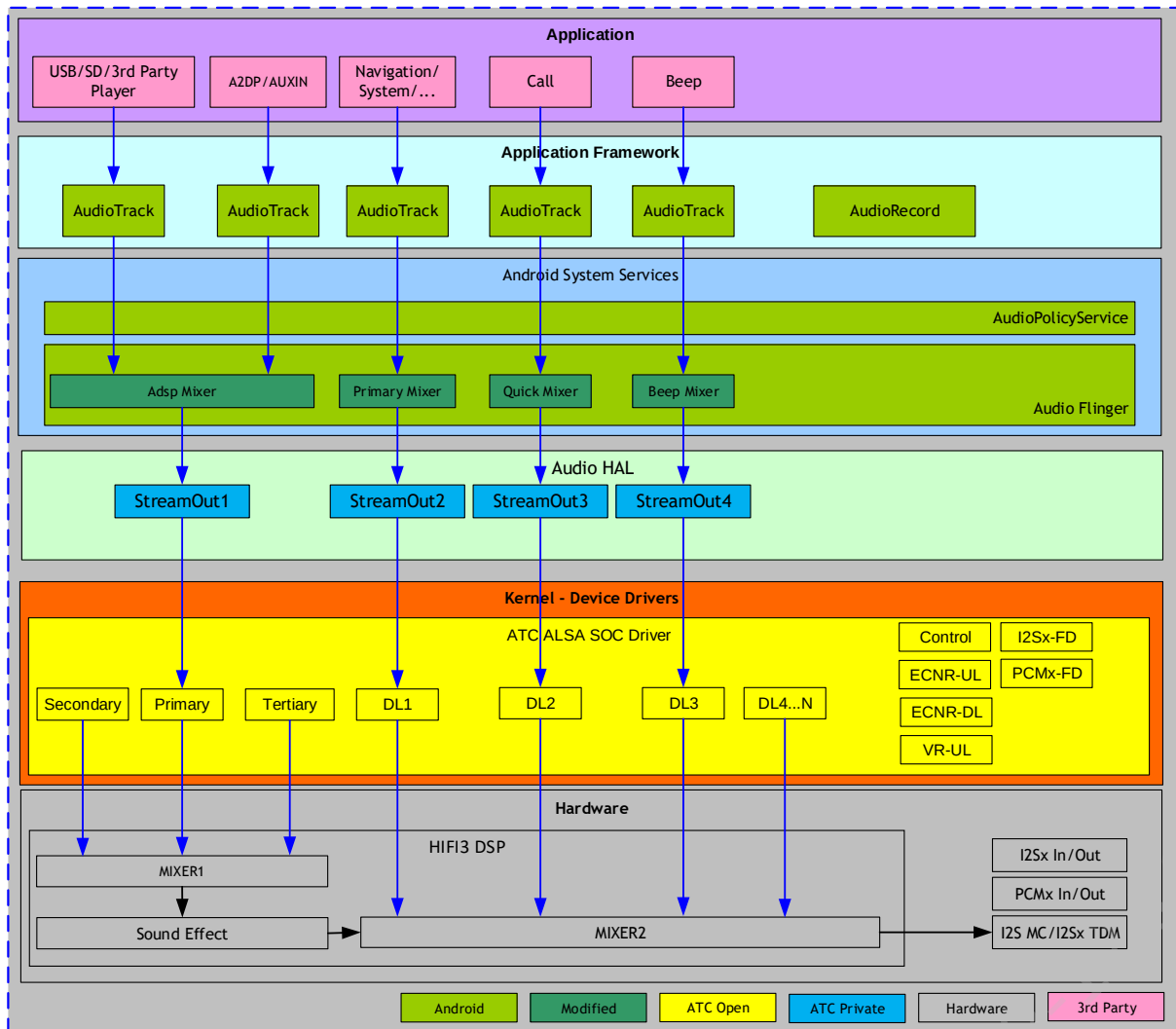


图 17-37 Audio IO 软件框图

ALSA SOC Driver 中，所支持的 PCM 设备节点，主要分为 DSP Path 节点、I2S 节点、PCM 节点。

其中：

- DSP Path 节点包括 Primary、Secondary、Tertiary、DL1_X（包含多个节点）、DL2_X（包含多个节点）、DL3、DL4、DLn(后续可以增加)、ECNR-UL、ECNR-DL、VR-UL、VR2-UL、DSP LL 等；
- I2S 节点包括 I2S0~3 的 TX 及 RX；
- PCM 节点包括 PCM0~1 的 TX 及 RX。

17.9.3 接口描述

17.9.3.1 Tinyalsa

Android OS，用户层使用 Audio 通过 tinyalsa 接口，这些接口主要为 Audio HAL 层提供，一般不对其他模块开放，其他模块如需要使用，请自行参考 tinyalsa 等相关官方文档。

17.9.3.2 Kcontrol

Kcontrol 是按照标准的 tinymix 方式开发一套接口。

Kcontrol 相关会涉及到的功能如下：

- 通路控制

表 17-56 Kcontrol API

Name	说明	参数取值
DigIn_Direct_Mode	AuxIn 数字输入，选择 Direct Mode 或 Indirect Mode	一个参数 0/1 0: Indirect Mode 1: Direct Mode
DigIn_Direct_I2S_Index	AuxIn 数字输入，Direct Mode 下选择哪个 I2S 输入	一个参数：0~3 对应到 I2S SC0~3

17.9.3.3 AlsaApi

17.9.3.3.1 功能描述

本节所述的 AlsaApi 功能应用于 Linux 系统（例如 AC8025 的仪表系统等）中，且特指基于 alsa-lib 而开发的一套 ATC 私有接口 alsa-api，内含 ATC 的 Volume Policy 功能，其旨在简化其他模块使用 Audio 资源的流程，alsa-api 功能主要包含：

- 直接指定 wav 文件即可播放；
- 可以将 wav 文件进行单次或循环播放；
- 单次播放 wav 文件可以设定播完后继续播放 silence 的时间（ms 级），有效规避短音频（时间较短的音频，时长小于 100ms）被 fade out 的风险；
- 可以将 wav 文件循环播放时，设定两次循环播放的 silence 间隔（ms 级）；
- 可以将 2 个 wav 文件进行轮换播放，且 2 文件间的间隔时间(ms 级)可以设定。

17.9.3.3.2 代码结构及文件描述

表 17-57 Alsa Api 代码结构及描述

文件名	文件路径	文件描述
atcAlsaApi.c	source/vendor/autochips/proprietary/framework/atcaudio/alsa-api/src/	对接 alsa-lib 的核心代码，将 alsa-lib 的一些调用进行简化，类似于 tinyalsa 的接口
atcAlsaWav.c	source/vendor/autochips/proprietary/framework/atcaudio/alsa-api/src/	直接操作并播放 wav 文件的接口
atcAlsaWavHeader.h	source/vendor/autochips/proprietary/framework/atcaudio/alsa-api/src/	仅供 atcAlsaWav.c 使用，定义 wav 文件的相关信息
atcAlsaApi.h	source/vendor/autochips/proprietary/framework/atcaudio/alsa-api/src/	供调用者使用的头文件
main.c	source/vendor/autochips/proprietary/framework/atcaudio/alsa-api/test/	测试使用或作为客户使用的 demo code

17.9.3.3.3 API Overview

表 17-58 Alsa Api API Overview

API	说明
类似 tinyalsa 的接口：	
int alsa_pcm_init (const char *card, int stream)	初始化一条 substream，主要是为了支持 alsa-lib 的 soft volume，所以需要在开机的时间点进行 init。
int alsa_pcm_open (snd_pcm_t **pcm, const char *card, int stream, struct pcm_config *config)	打开一条 substream。
int alsa_pcm_close (snd_pcm_t *pcm)	关闭一条 substream。
int alsa_pcm_start (snd_pcm_t *pcm)	开始播放 substream，一般情况下不主动调用，其会在 alsa_pcm_read/ alsa_pcm_write 中被调用。
snd_pcm_sframes_t alsa_pcm_read (snd_pcm_t *pcm, void *buffer, snd_pcm_uframes_t frames)	读取数据到 user 层
snd_pcm_sframes_t alsa_pcm_write (snd_pcm_t *pcm, const void *buffer, snd_pcm_uframes_t frames)	写入数据到 kernel 中
int alsa_mixer_kcontrol_set (const char *name, int value)	kcontrol 的 set 命令
int alsa_mixer_kcontrol_get (const char *name, int *value)	kcontrol 的 get 命令

int alsa_kcontrol_set_arrays (int card_num, const char *control_name, const unsigned int *value, uint32_t num_bytes)	kcontrol的arrays设置命令
Wav文件的操作 API:	
wav_info_t * alsa_wav_init (enum WAV_DEVICE dev_id, int stream, const char *file_name)	解析一个wav文件，并返回wav_info_t类型的指针，该指针是操作WAV所有API的参数
void alsa_wav_deinit (wav_info_t *wav)	释放wav_info_t指针
int alsa_wav_callback (wav_info_t *wav, void (*cb)(int))	设置播放完成的回调函数
int alsa_wav_bind_2_files (wav_info_t *wav1, wav_info_t *wav2, int interval_ms)	绑定两个WAV文件，用于轮换循环播放
int alsa_wav_unbind_2_files (wav_info_t *wav1, wav_info_t *wav2)	对两个WAV文件解除绑定
int alsa_wav_set_mode (wav_info_t *wav, enum WAV_MODE mode, int interval_ms)	设置播放模式，与alsa_wav_bind_2_files是互斥的操作
int alsa_wav_open (wav_info_t *wav)	打开一条audio通路
int alsa_wav_close (wav_info_t *wav)	关闭一条audio通路
int alsa_wav_start (wav_info_t *wav)	开始播放wav文件
int alsa_wav_stop (wav_info_t *wav)	中途停止wav的播放，是阻塞的，等待播放完成
int alsa_wav_sync_playback (wav_info_t *wav)	单次播放wav文件并阻塞直至播放完成

17.9.3.3.1 struct pcm_config

PCM 流的相关配置项。

```
/* Configuration for a stream */
struct pcm_config {
    unsigned int channels;
    unsigned int rate;
    unsigned int period_size;
    unsigned int period_count;

    /**
     * Format:
     * MACRO defined in alsa/pcm.h, we copy them here. Refer & Use them:
     *
     * SND_PCM_FORMAT_UNKNOWN    // Unknown
     *
     * SND_PCM_FORMAT_S8         // Signed 8 bit
     *
     * SND_PCM_FORMAT_U8         // Unsigned 8 bit
     */
};
```

```

*   SND_PCM_FORMAT_S16_LE    // Signed 16 bit Little Endian
*   SND_PCM_FORMAT_S16_BE    // Signed 16 bit Big Endian
*   SND_PCM_FORMAT_U16_LE    // Unsigned 16 bit Little Endian
*   SND_PCM_FORMAT_U16_BE    // Unsigned 16 bit Big Endian
*   SND_PCM_FORMAT_S24_LE    // Signed 24 bit Little Endian using low three bytes in 32-bit word
*   SND_PCM_FORMAT_S24_BE    // Signed 24 bit Big Endian using low three bytes in 32-bit word
*   SND_PCM_FORMAT_U24_LE    // Unsigned 24 bit Little Endian using low three bytes in 32-bit word
*   SND_PCM_FORMAT_U24_BE    // Unsigned 24 bit Big Endian using low three bytes in 32-bit word
*   SND_PCM_FORMAT_S32_LE    // Signed 32 bit Little Endian
*   SND_PCM_FORMAT_S32_BE    // Signed 32 bit Big Endian
*   SND_PCM_FORMAT_U32_LE    // Unsigned 32 bit Little Endian
*   SND_PCM_FORMAT_U32_BE    // Unsigned 32 bit Big Endian
*/

snd_pcm_format_t format;

/* Values to use for the ALSA start, stop and silence thresholds, and
 * silence size. Setting any one of these values to 0 will cause the
 * default tinyalsa values to be used instead.
 * Tinyalsa defaults are as follows.
 *
 * start_threshold   : period_count * period_size
 * stop_threshold    : period_count * period_size
 * silence_threshold : 0
 * silence_size      : 0
 */
unsigned int start_threshold;
unsigned int stop_threshold;
unsigned int silence_threshold;
unsigned int silence_size;

/* Minimum number of frames available before pcm_mmap_write() will actually
 * write into the kernel buffer. Only used if the stream is opened in mmap mode
 * (pcm_open() called with PCM_MMAP flag set). Use 0 for default.
 */
int avail_min;

```

```
};
```

17.9.3.3.2 enum WAV_DEVICE

播放的设备定义。

```
enum WAV_DEVICE {  
    /* ADAS Devices */  
    RVC_DEVICE,  
    VBA_DEVICE,  
    ADAS_DEVICE,  
  
    /* Cluster Devices */  
    CLUSTER_DEVICE,  
  
    MAX_DEVICE_NUM  
};
```

在 ADAS 上，则对应模块使用对应的 Device，Cluster 系统上则使用 CLUSTER_DEVICE。

17.9.3.3.3 enum WAV_MODE

文件播放模式选择。

```
enum WAV_MODE {  
    NO_LOOP_MODE,                //单次播放，播放完成后立即结束  
    NO_LOOP_DELAY_STOP_MODE,     //单次播放并在播放完成后继续播放 0 数据  
    LOOP_MODE,                   //循环播放  
};
```

17.9.3.3.4 enum WAV_STATE

播放 wav 文件时的状态信息。

```
enum WAV_STATE {  
    WAV_CLOSE,  
    WAV_OPEN,  
    WAV_START,  
    WAV_STOP,  
};
```

17.9.3.3.5 typedef void wav_info_t

wav_info_t 是操作各个 wav 接口的句柄。

17.9.3.3.4 接口详细介绍

本文主要介绍直接播放 wav 文件的接口，对简化 alsa-lib 操作的接口不作详细说明，具体用法可以参考 Android 上面的 tinyalsa。涉及到的接口如下：

```
int alsa_pcm_init(const char *card, int stream);
int alsa_pcm_open(snd_pcm_t **pcm, const char*card, int stream, struct pcm_config *config);
int alsa_pcm_close(snd_pcm_t *pcm);
int alsa_pcm_start(snd_pcm_t *pcm);

snd_pcm_sframes_t alsa_pcm_read(snd_pcm_t *pcm, void *buffer, snd_pcm_uframes_t frames);
snd_pcm_sframes_t alsa_pcm_write(snd_pcm_t *pcm, const void *buffer, snd_pcm_uframes_t frames);

int alsa_mixer_kcontrol_set(const char *name, int value);
int alsa_mixer_kcontrol_get(const char *name, int *value);

int alsa_kcontrol_set_arrays(int card_num, const char *control_name,
                             const unsigned int *value, uint32_t num_bytes);
```

17.9.3.3.4.1 alsa_wav_init

【描述】

用于解析 wav 文件，并返回 wav_info_t 指针，该指针用于操作 wav 相关的所有函数接口。

建议程序在初始化的时候就调用该函数进行解析，不用使用时都解析，在确认后续不再使用后则调用 alsa_wav_deinit 进行资源释放。

【语法】

```
wav_info_t *alsa_wav_init(enum WAV_DEVICE dev_id, int stream, const char *file_name)
```

【参数】

- dev_id: 参照 enum WAV_DEVICE 的相关定义，正确使用 device
- stream:

SND_PCM_STREAM_PLAYBACK （播放使用）

SND_PCM_STREAM_CAPTURE （录音使用，当前不支持）

- file_name: 完整的文件路径及文件名称

【返回值】

- wav_info_t 类型的指针，用于操作所有 wav 相关的接口。

【示例代码】

```
wav_info_t *info = alsa_wav_init(CLUSTER_DEVICE, SND_PCM_STREAM_PLAYBACK,
                                "./test.wav");
```

17.9.3.3.4.2 alsa_wav_deinit**【描述】**

用于释放相关内存，与 alsa_wav_init 成对使用。

【语法】

```
void alsa_wav_deinit(wav_info_t *wav)
```

【参数】

- wav: wav_info_t 类型的指针，alsa_wav_init 返回的指针

【返回值】

- 无

17.9.3.3.4.3 alsa_wav_callback**【描述】**

在 start 前可以设置播放完成时的回调函数，当播放完成后，回调函数将被调用。注意，回调函数中禁止调用 wav 相关的所有接口，一般只用于更新程序自己的播放状态。

【语法】

```
int alsa_wav_callback(wav_info_t *wav, void (*cb)(int))
```

【参数】

- wav: wav_info_t 类型的指针，alsa_wav_init 返回的指针
- cb: 参数为 int 返回值为 void 的函数指针，当播放结束后，cb 将会被回调

【返回值】

- 无操作成功返回 0
- 操作失败返回错误码

17.9.3.3.4.4 `alsa_wav_bind_2_files`

【描述】

用于绑定 2 个 wav 文件，使其可以进行循环播放。与 `alsa_wav_set_mode` 是互斥的接口。

以 `wav1 -> silence (时长 interval_ms) -> wav2 -> silence (时长 interval_ms) -> wav1 -> ...` 的次序进行循环播放。其中 `wav1` 和 `wav2` 谁先播放，需要看 `alsa_wav_start` 中的参数，当两个 wav 文件绑在一起以后，在使用 `alsa_wav_open/alsa_wav_close/alsa_wav_start/alsa_wav_stop` 这几个函数时，其参数只能同时是 `wav1` 或 `wav2`，即参数需要保证一致，参数所指向的 wav 是最先播放的，后续就是循环播放。

【语法】

```
int alsa_wav_bind_2_files(wav_info_t *wav1, wav_info_t *wav2, int interval_ms)
```

【参数】

- `wav1`: `wav_info_t` 类型的指针，`alsa_wav_init` 返回的指针
- `wav2`: `wav_info_t` 类型的指针，`alsa_wav_init` 返回的指针
- `interval_ms`: 两个文件循环播放时，中间播放 0 数据的时间，单位是 ms

【返回值】

- 无操作成功返回 0
- 操作失败返回错误码

17.9.3.3.4.5 `alsa_wav_unbind_2_files`

【描述】

用于解除 2 个 wav 文件的绑定播放，是 `alsa_wav_bind_2_files` 的反向操作，解除后，2 个文件即可单独操作和单独播放。

【语法】

```
int alsa_wav_unbind_2_files(wav_info_t *wav1, wav_info_t *wav2)
```

【参数】

- `wav1`: `wav_info_t` 类型的指针，`alsa_wav_init` 返回的指针
- `wav2`: `wav_info_t` 类型的指针，`alsa_wav_init` 返回的指针

【返回值】

- 无操作成功返回 0
- 操作失败返回错误码

17.9.3.3.4.6 `alsa_wav_set_mode`

【描述】

单个文件播放时，用于设置播放模式。与 `alsa_wav_bind_2_files` 是互斥的接口。

【语法】

```
int alsa_wav_set_mode(wav_info_t *wav, enum WAV_MODE mode, int interval_ms)
```

【参数】

- `wav`: `wav_info_t` 类型的指针，`alsa_wav_init` 返回的指针
- `mode`: 文件播放模式选择，详细见 enum `WAV_MODE` 的定义
- `interval_ms`:
 - `NO_LOOP_MODE`, 该参数无效循环播放时，中间播放 0 数据的时间，单位是 `ms`
 - `NO_LOOP_DELAY_STOP_MODE`, 文件单次播放完成后继续播放 0 数据的时间 `ms`
 - `LOOP_MODE`, 文件循环播放时，2 次播放间隔时间 `ms`，间隔时间即是播放 0 数据

【返回值】

- 无操作成功返回 0
- 操作失败返回错误码

17.9.3.3.4.7 `alsa_wav_open`

【描述】

用于打开一条 Audio 数据通路，对接 `alsa-lib` 接口的 `open`。

【语法】

```
int alsa_wav_open(wav_info_t *wav)
```

【参数】

- `wav`: `wav_info_t` 类型的指针，`alsa_wav_init` 返回的指针

【返回值】

- 无操作成功返回 0
- 操作失败返回错误码

17.9.3.3.4.8 `alsa_wav_close`

【描述】

用于关闭一条 Audio 数据通路，对接 alsa-lib 接口的 close。

【语法】

```
int alsa_wav_close(wav_info_t *wav)
```

【参数】

- wav: wav_info_t 类型的指针，alsa_wav_init 返回的指针

【返回值】

- 无操作成功返回 0
- 操作失败返回错误码

17.9.3.3.4.9 alsa_wav_start**【描述】**

用于开启播放。

【语法】

```
int alsa_wav_start(wav_info_t *wav);
```

【参数】

- wav: wav_info_t 类型的指针，alsa_wav_init 返回的指针

【返回值】

- 无操作成功返回 0
- 操作失败返回错误码

17.9.3.3.4.10 alsa_wav_stop**【描述】**

用于中途关闭播放，对于 LOOP 和 NO_LOOP 的播放，都可以中途停止，该接口是阻塞的，真正停止后才会返回。针对 NO_LOOP 模式的，其播放完成后自动调用 stop，并回调注册的回调函数，所以 NO_LOOP 中可以主动调用也可以不调用。

【语法】

```
int alsa_wav_stop(wav_info_t *wav)
```

【参数】

- wav: wav_info_t 类型的指针，alsa_wav_init 返回的指针

【返回值】

- 无操作成功返回 0
- 操作失败返回错误码

17.9.3.3.4.11 **alsa_wav_sync_playback**

【描述】

单次播放模式才能使用该函数，该函数功能与 `alsa_wav_start` 基本相同，但其是阻塞的，等待文件播放完成后才会返回。`alsa_wav_start` 仅仅是 `start`，`alsa_wav_sync_playback` 是 `start` 并等待其结束。

【语法】

```
int alsa_wav_sync_playback(wav_info_t *wav)
```

【参数】

- `wav`: `wav_info_t` 类型的指针，`alsa_wav_init` 返回的指针

【返回值】

- 无操作成功返回 0
- 操作失败返回错误码

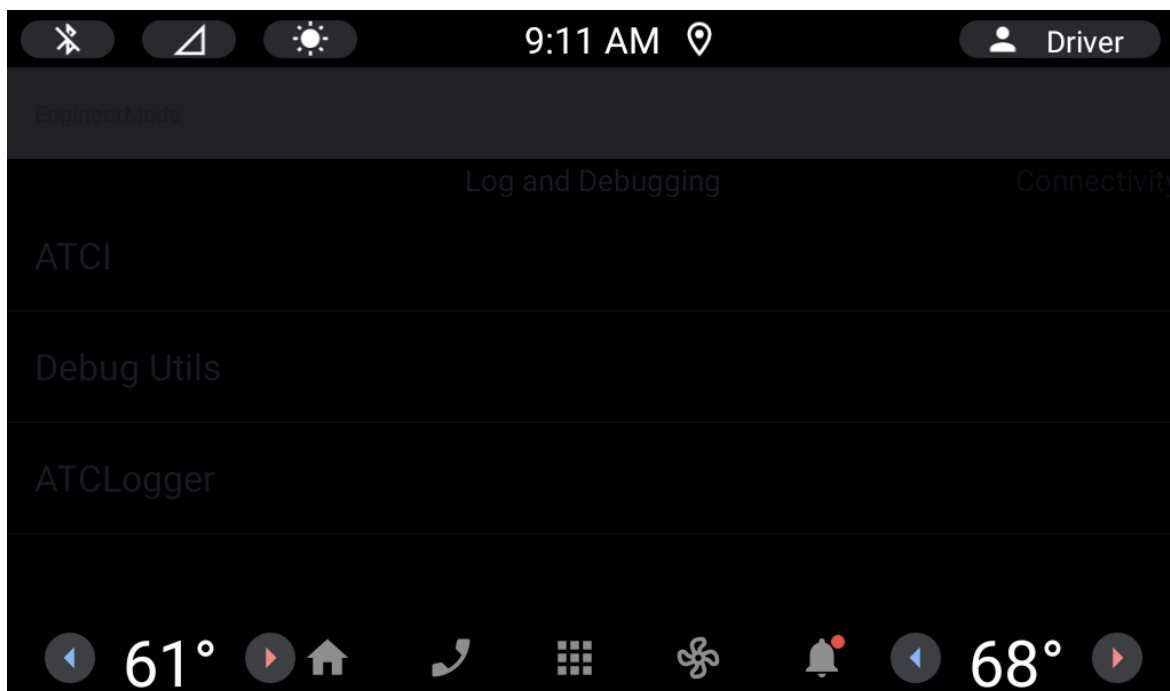
17.9.3.3.5 使用说明和注意事项

- 对于只播放一次的 `wav`，使用 `alsa_wav_start` 后，可以让其自行播完停止，无需再调用 `alsa_wav_stop` 和 `alsa_wav_close`，当然程序调用了也不会出错。下一次播放重新调用 `alsa_wav_open` 和 `alsa_wav_start` 即可。
- 对于只播放一次的 `wav`，使用 `alsa_wav_sync_playback` 后，阻塞等到其播放完成，无需再调用 `alsa_wav_stop` 和 `alsa_wav_close`，当然程序调用了也不会出错。下一次播放重新调用 `alsa_wav_open` 和 `alsa_wav_sync_playback` 即可。
- 接口的具体方法请参考 [test](#) 目录的 [main.c](#)。

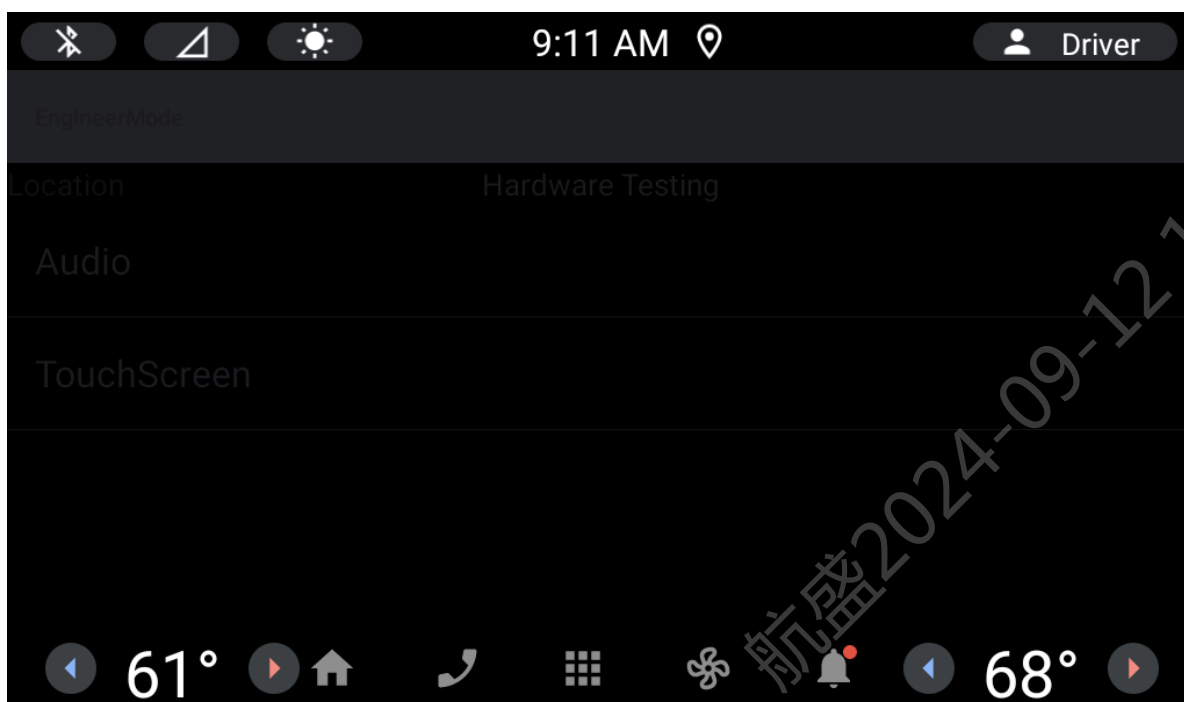
17.10 通用 debug 方法

17.10.1 **AtcEngineerMode Audio 使用说明**

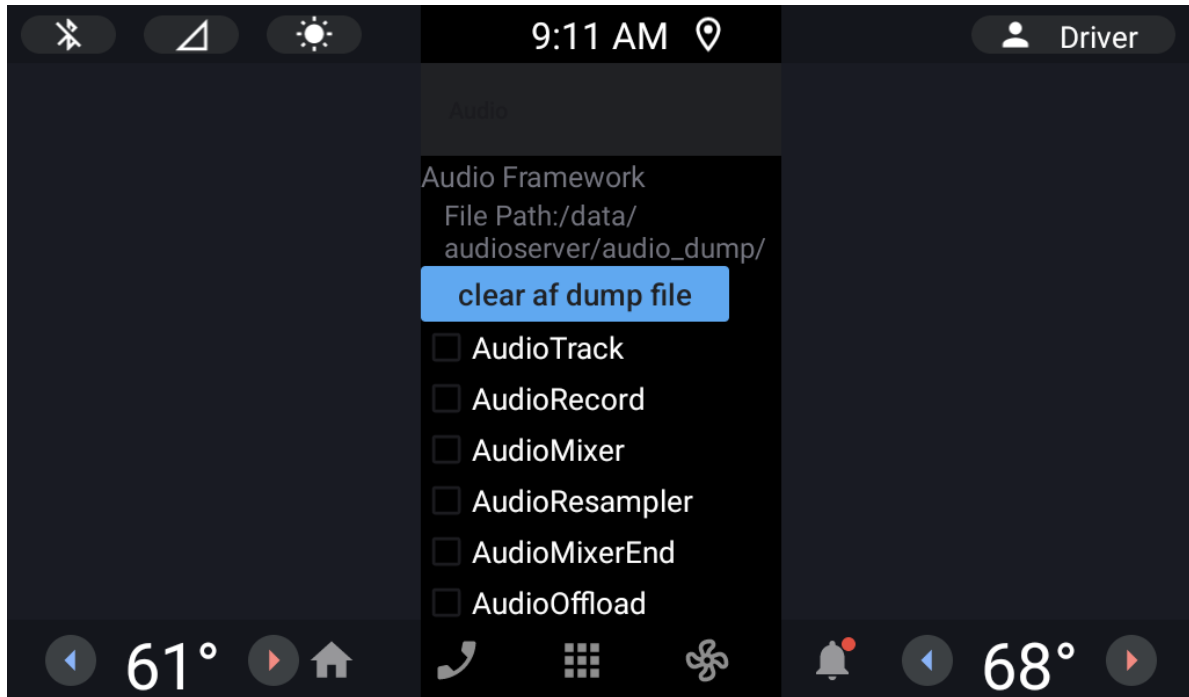
打开 Audio 模块 UI 如下， 首先打开 AtcEngineerMode App 界面如下：



接着向左滑动直到以下页面：



点击 Audio，打开界面如下：



在相关标签的 CheckBox 中打上勾即可是相关 dump 生效，反之关闭 dump。

AtcEngineerMode 下 Audio 模块主要功能为：

- 动态开启/关闭 audio dump 功能；
- 一键删除 dump 文件；
- 调用 setParameters 方法设置 audio 参数。
 - Audio 的 dump 功能分为 framework 层和 hal 层 dump，其中 framework 层的 dump 文件主要存放在/data/audioserver 目录下，hal 层的 dump 文件主要存放在/data/vendor/audiohal 目录下。而这两层的 dump 功能又分别同时存在两种 dump，普通 audio dump 和 cycle audio dump，两者的区别在于 cycle audio dump 会循环删除之前录制的文件，而普通 dump 不会自动删除 dump 文件，并且普通 dump 使用临时属性控制，重启自动失效，而 cycle dump 使用 persist 属性控制。
 - 另外，还设置了 aurisys 的开关及 dump 功能，使用 persist 属性控制，但需重启才可生效。
 - dump 文件删除功能无需使用 adb shell 进行删除，可一键进行删除，注意在播放音频时一键删除无法删除正在 dump 的文件。



说明：

设置 cycle dump 时最后的 file num 和 duration 需要输入相应数字，且输入之后需要勾选任一模块 dump 功能（AudioTrack/AudioMixer 等）方可生效，不输入任何数字则默认为 0。

17.10.2 Dump 文件说明

17.10.2.1 普通 dump 文件

Framework 层的 dump 文件都存放在/data/audioserver/audio_dump 中，文件命名含义如下：

表 17-59 framework 层普通 dump

dump 类型	文件命名	文件说明
AudioTrack	af_track_pcm_xxx_yyy_out.pcm.wav	xxx 表示 track id，yyy 表示 Track 对象地址。该文件为 AudioFlinger 从 cblk 共享 buffer 中取出来的数据，是 AudioTrack 写去 cblk 中的原始数据
AudioRecord	af_track_pcm_xx_yy_in.pcm.wav	xx 表示 track id，yy 表示 RecordTrack 对象地址。该文件是 AudioFlinger 写入 cblk 共享 buffer 中的数据，也即 AudioRecord 从共享 buffer 中获取的数据
AudioMixer	af_mixer_pcm.pcm.xxx.pidyyy.tidxxx.wav	xxx 表示 MixerThread 的 id，yyy 表示当前进程号，zzz 表示当前线程号。前者表示混音之后的数据，后者表示写入 MessageQueue（framework 层与 hal 层的共享 buffer）中的数据

Hal 层的文件存放在/data/vendor/audihal/audio_dump 中，文件命名如下：

表 17-60 HAL 层普通 dump

dump 类型	文件命名	文件说明
StreamIn	StreamIn_Dump.pcmxxx.pcm 及 StreamIn_Dump.pcmxxx.YYY.pcm	xxx 表示 StreamIn id，YYY 是 DataProvider 类名。前者表示从 hal 层送到 MessageQueue 中数据，后者表示从 pcm 节点获取的数据
StreamOut	streamout.pcm.xxx.MMM.pidyyy.tidzzz.pcm	xxx 表示 id，MMM 表示类

dump 类型	文件命名	文件说明
		名，yyy 表示当前进程号，zzz 表示当前线程号。MMM 为 AudioALSAStreamOut 时表示 hal 层从 MessageQueue 中取出的数据，MMM 为 PlaybackHandler 时一般表示写入 pcm 节点的数据
EPL	SPE_EPLxxx.EPL	xxx 表示 id，该数据为降噪消回音相关数据

17.10.2.2 Cycle dump 文件

Framework 层的 dump 文件都存放在/data/audioserver/audio_cycledump 中，文件命名含义如下（以下 mmm 表示 formatchangecount, nnn 表示 removecount）：

表 17-61 Framework 层 cycledump

dump 类型	文件命名	文件说明
AudioTrack	audiotrack_xxx_yyy.pcm.mmm.nnn.wav	xxx 表示 streamType，yyy 表示 Track 对象地址。该文件为 AudioFlinger 从 cblk 共享 buffer 中取出来的数据，是 AudioTrack 写去 cblk 中的原始数据
AudioFlinger_Output	af_write_xxx.mmm.nnn.wav	xxx 表示 MixerThread 对象地址，该数据表示混音之后写到 MessageQueue 中的数据
AudioRecord	record_xxx.pcm.wav	xxx 表示 RecordTrack 对象地址。该文件是 AudioFlinger 写入 cblk 共享 buffer 中的数据，也即 AudioRecord 从共享 buffer 中获取的数据
AudioFlinger_Input	af_record_xxx.pcm.wav	xxx 表示 RecordThread 对

dump 类型	文件命名	文件说明
		象地址，该数据为 AudioFlinger 从 MessageQueue 中读取的数据

Hal 层数据存放在 /data/vendor/audiohal/audio_cycledump 中，文件命名如下：

表 17-62 HAL 层 cycledump

dump 类型	文件命名	文件说明
PlaybackHandler	hal_playbackhandler_xxx.mmm.nnn.wav	xxx 表示 playbackhandler 后缀。该数据一般表示写到 pcm 节点的数据
StreamIn	hal_streamin_handler(xxx).mmm.nnn.wav	xxx 表示 capturehandler 对象的地址。该数据为写入 MessageQueue 中的数据
CaptureDataProvider	hal_dataprovider_mmm.nnn.wav	该数据为从 pcm 节点获取的数据

17.10.3 Aurisys 相关 dump

Aurisys 的 dump 设置无法即时生效，需要重启平台后才能生效。其中 Log Enable 表示是否开启 aurisys 的 log，Raw Dump 和 Lib Dump 则是 dump aurisys 相关数据，该数据文件存放在 /data/vendor/audiohal/audio_dump。

17.10.4 设置参数

获取某个参数可在输入框中直接输入相关 key 即可，结果将通过 Toast 进行弹出显示。设置参数则需要区分该参数是在 framework 层生效还是在 hal 层生效，如果在在输入框中输入“key=value”的形式，否则输入“SetProp=key=value”的形式。常用 key 值如下：

表 17-63 常用 Key 值

Key	作用处	说明
atc_hfp_set_sampling_rate	hal	设置蓝牙电话采样率
atc_hfp_enable	hal	设置蓝牙电话开关
atc_hfp_volume_policy	hal	设置蓝牙电话音量

航盛2024-09-12 16:50:35

18 Bluetooth

18.1 简介

18.1.1 软硬件环境

18.1.1.1 软件环境

原生 Android 系统已经全面支持 Carkit 端蓝牙应用，包括 Hfp Client，A2dp Sink，Avrcp Controller，Pbap Client，Map Client 等功能，并且这些 Carkit 端蓝牙功能的接口，已经对接上原生 Android Framework 接口，车机系统与手机系统使用的是同一套 Framework 接口，因此，手机系统上的电话、电话本、音乐播放器等应用，几乎无需修改，就可以移植到车机上正常运行。

ATC 建议用户使用 Android 原生的 Telecom、Contacts、MediaBrowser 接口来实现车机的蓝牙电话、蓝牙电话本、蓝牙音乐播放器应用，因为公版 SQC 测试均以这种应用方式进行，应用场景经反复 SQC 测试，质量更有保障，并且接口更通用，随后续 Android 版本升级更方便，移植工作更小。

ATC 车机蓝牙基于 Android 原生 Bluedroid 协议栈，配置作为 Carkit 端协议运行。系统能够支持 AAC Decoder。

18.1.1.2 硬件环境

AC8025 蓝牙方案需要使用以下硬件环境：

- 蓝牙 RF IC：QCA6595/CYW88373



说明：

芯片上电时序需参考 DataSheet 的对应说明

18.1.2 功能

表 18-1 蓝牙功能列表

基本功能	子功能
基本蓝牙电话功能	拨打指定电话号码
	通话状态通知
	获取通话号码
	接听来电

基本功能	子功能
	拒接来电
	挂断电话
	通话声音切换回手机/车机
	发送 DTMF Code
	播放手机来电铃声（前提是手机也支持）
三方通话功能	接听第三方来电，并保持其他通话
	接听第三方来电，并挂断其他通话
	保持当前通话，进入某一方通话
	通话过程中，再拨出一通电话
	挂断某一方通话
	将两方通话合并，即多方会议功能
多手机电话	可支持同时连接两部手机的蓝牙电话
	两部手机同时通话时，可以选择或者切换其中一部手机声音在车机上输出
蓝牙音乐功能	播放蓝牙音乐
	调节蓝牙音乐音量
	获取播放状态
	获取播放进度
	获取当前歌曲 ID3 信息，包括：标题、艺术家、专辑
	播放控制，包括：播放、暂停、停止、下一曲、上一曲、快进、快退等
	播放列表，包括获取播放列表，从列表选择曲目播放
	播放循环模式，获取和设置播放循环模式（repeat mode）
	获取专辑图片
蓝牙电话本、通话记录	手机电话本同步
	Sim 卡电话本同步
	电话本记录的人名分别获取 first name、middle name、given name
	手机通话记录同步
	通话记录的时间
	可同时连接两部手机的蓝牙电话，两部手机电话本、通话记录均可以同步到车机上。
	可控制电话本/通话记录的下载与停止下载
	下载 favorite 联系人
蓝牙 SSP 功能	作为 SPP Client 端连接
	作为 SPP Service 端连接
	接收/发送数据
蓝牙 OPP	可通过蓝牙分享媒体文件给其他设备
	可接收其他设备的媒体文件分享

基本功能	子功能
蓝牙 LE	支持 LE 广播及扫描
	支持 GATT，包括作为 Service、Client
蓝牙 PAN 上网	支持作为 Pan User，连接蓝牙热点上网
蓝牙短信功能	接收新短信通知，当连接的蓝牙手机收到新短信时，车机能收到新短信通知
	读取未读短信，车机端能显示手机新收到的未读短信，并能从手机同步新收到的未读短信的内容
	发送短信，车机端能通过连接的蓝牙手机，发送短信
	短信同步，车机端从连接的蓝牙手机，同步最近 N 条短信内容
	读取彩信功能
其他功能	支持通过 OOB 配对
	支持获取手机端电量、信号强度、漫游状态、运营商
	检测远端设备是否在附近



说明：

三方通话功能需要手机端及运营商支持此功能；
蓝牙音乐的播放列表功能需要手机端及手机播放器支持此功能。

18.1.3 代码结构及文件描述

表 18-2 BT 代码结构及描述

文件名	文件路径	文件描述
蓝牙音乐应用代码	source/android/vendor/autochips/proprietary/packages/apps/AtcMedia	蓝牙音乐应用代码，基于原生 CarMedia 修改
电话应用代码	source/android/vendor/autochips/proprietary/packages/apps/CarDialer	蓝牙电话应用代码，基于原生 packages/apps/Car/Dialer 代码修改
CarSetting 应用代码	source/android/vendor/autochips/proprietary/packages/apps/AtcCarSetings	CarSetting 应用代码，基于原生 packages/apps/Car/Settings 修改，蓝牙开关、配对、连接的界面也在此

文件名	文件路径	文件描述
Settingslib 的代码	source/android/frameworks/base/packages/Settingslib/src/com/android/settingslib/bluetooth	原生 SetingsLib 代码，com.android.settingslib.bluetooth 这个包的代码均在这里实现
Android 原生蓝牙 Framework 接口	source/android/frameworks/base/core/java/android/bluetooth	Android 原生蓝牙 Framework 接口
ATC 私有蓝牙 Framework 接口	source/android/vendor/autochips/proprietary/frameworks/base/bluetooth	ATC 添加的私有蓝牙接口
蓝牙短信应用代码	source/android/vendor/autochips/proprietary/packages/apps/BluetoothMsm	蓝牙短信应用代码

18.2 软件设计

18.2.1 架构图

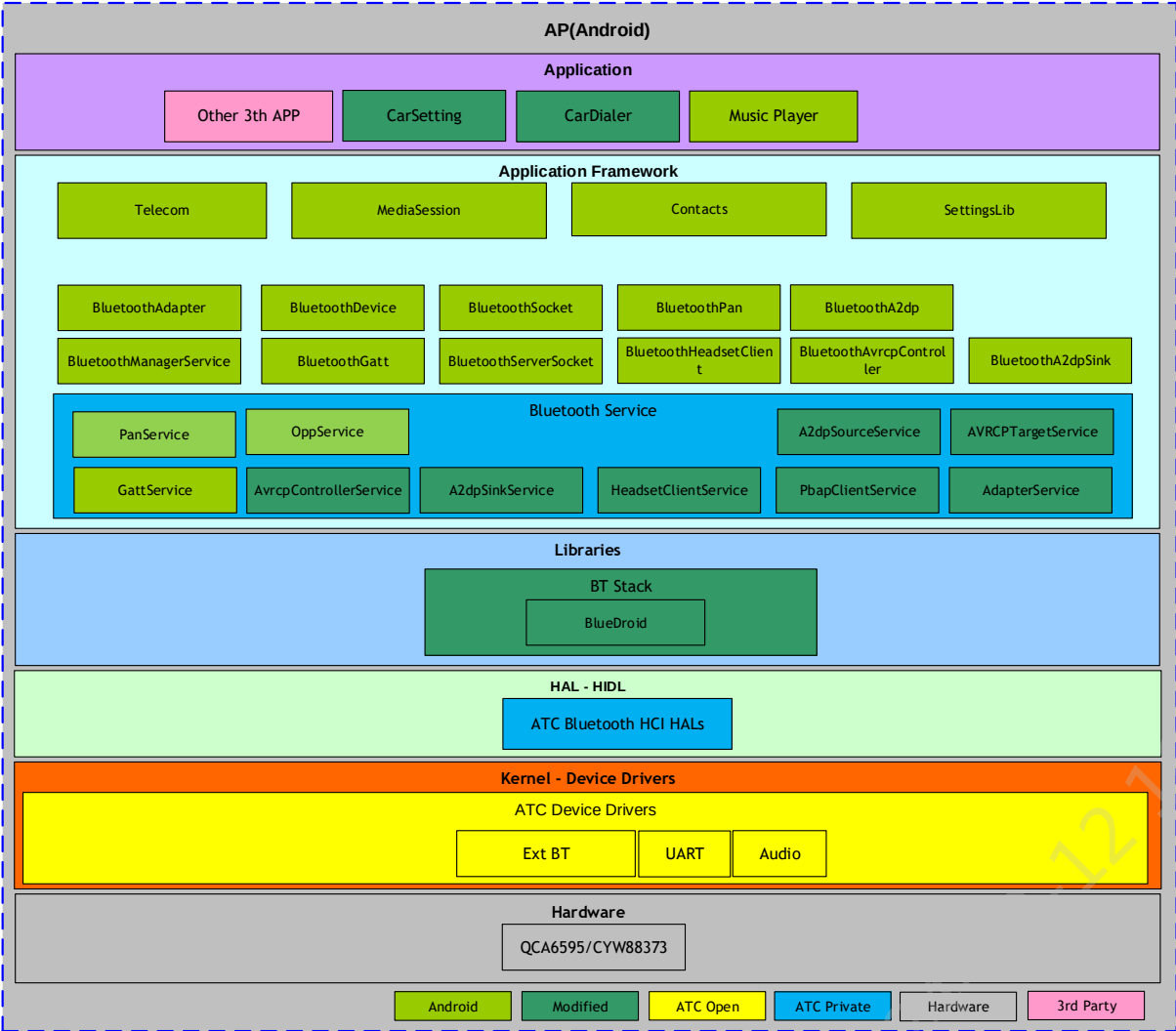


图 18-1 蓝牙框架图

在以上蓝牙框架图中，应用层通过 SettingsLib、Telecom、MediaSession、Contacts 接口来实现蓝牙 Carkit 应用，而非直接访问蓝牙接口。

18.2.2 流程图

18.2.2.1 蓝牙 Settings 相关流程

1. 蓝牙 Settings 初始化流程

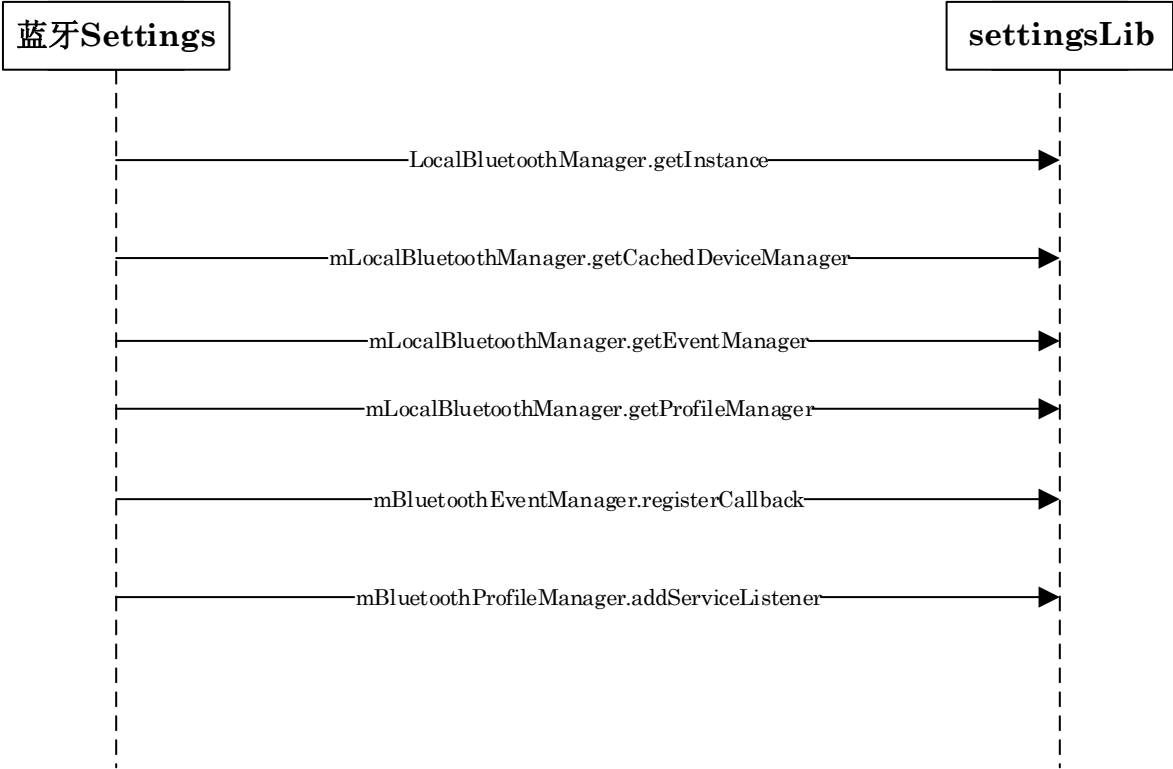


图 18-2 蓝牙 Settings 初始化流程

LocalBluetoothManager 是所有接口的入口，通过 LocalBluetoothManager，可以获取到 LocalBluetoothAdapter、CachedBluetoothDeviceManager、BluetoothEventManager、LocalBluetoothProfileManager。通过 BluetoothEventManager.registerCallback 注册回调，就可以监听蓝牙状态变化、设备搜索、连接状态等信息。

2. 打开蓝牙流程

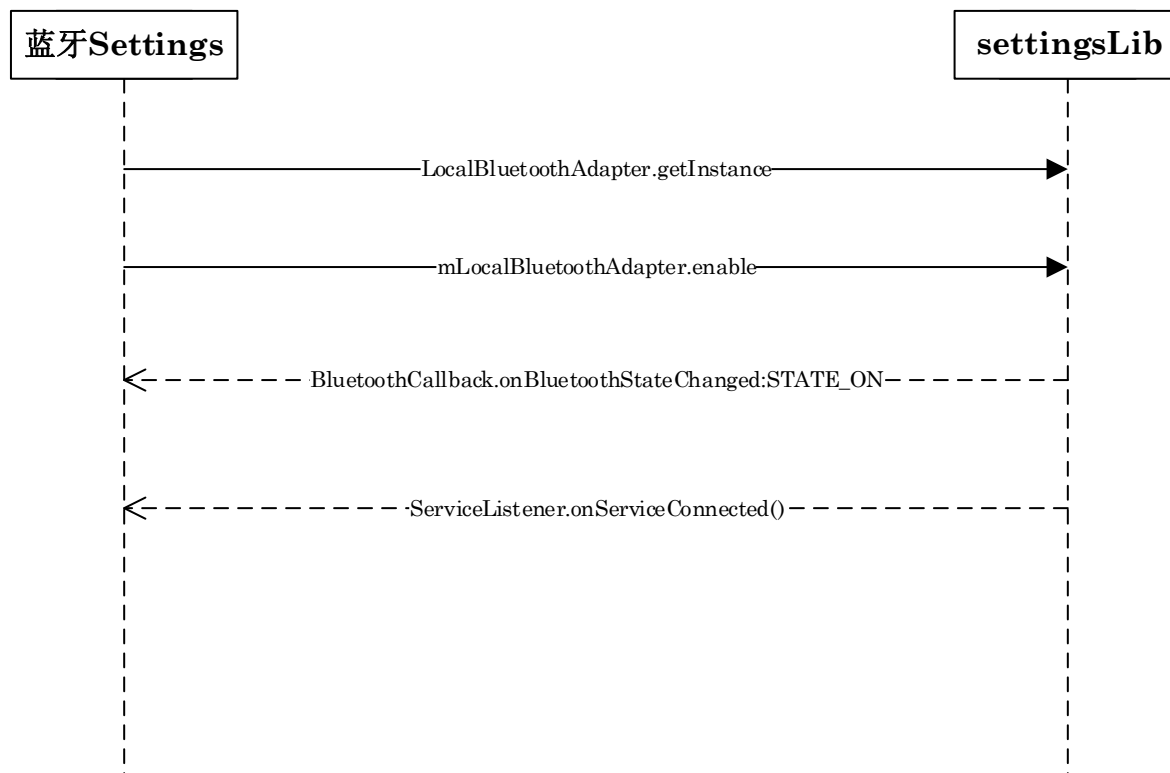


图 18-3 打开蓝牙流程

通过 `LocalBluetoothAdapter.enable` 蓝牙，蓝牙打开成功后，App 会收到 `BluetoothCallback.onBluetoothStateChanged` 回调，通知状态变为 `STATE ON`，接着当所有 Profile 接口都 Bind 上 Service 后，App 会收到 `ServiceListener.onServiceConnected` 回调，在此以后，App 才可以正常调用连接或者获取设备连接状态的接口。

关闭蓝牙流程与打开流程类似。

3. 搜索设备流程

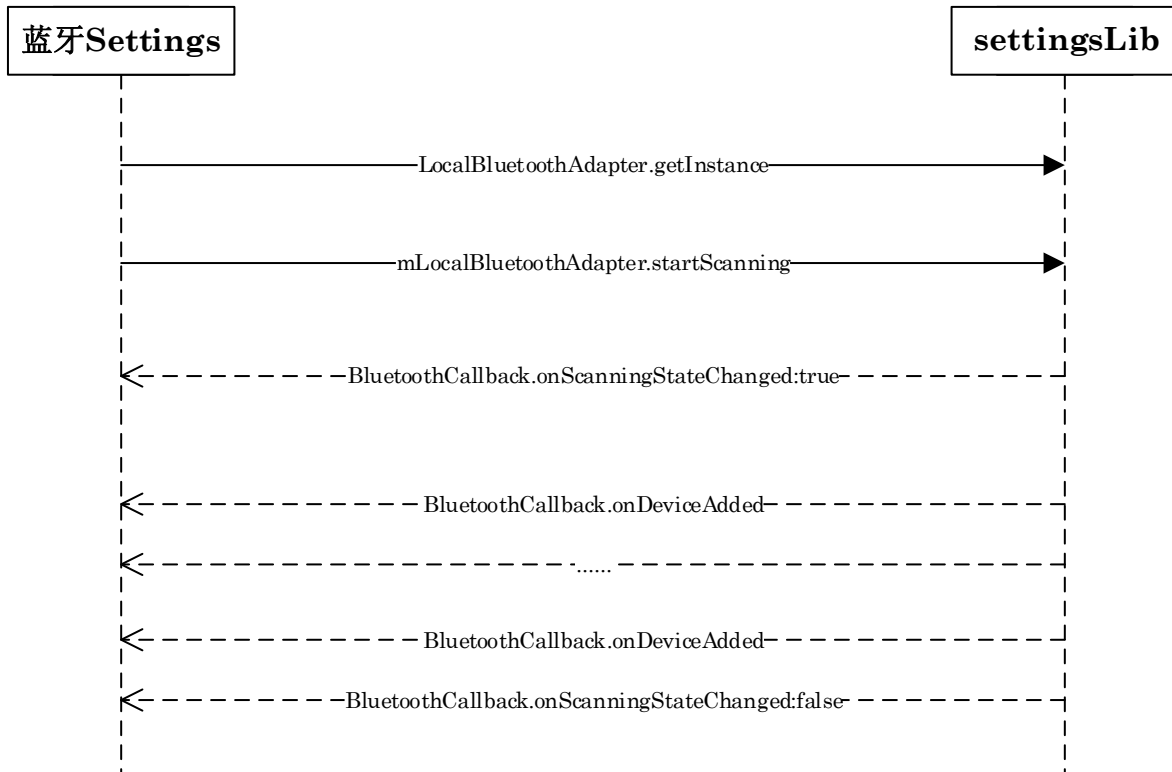


图 18-4 搜索设备流程

通过 `LocalBluetoothAdapter.startScanning` 开始搜索，接着 app 会收到 `BluetoothCallback.onScanningStateChanged` 通知搜索开始，搜索到的设备通过 `BluetoothCallback.onDeviceAdded` 回调给 App。

4. 蓝牙连接流程

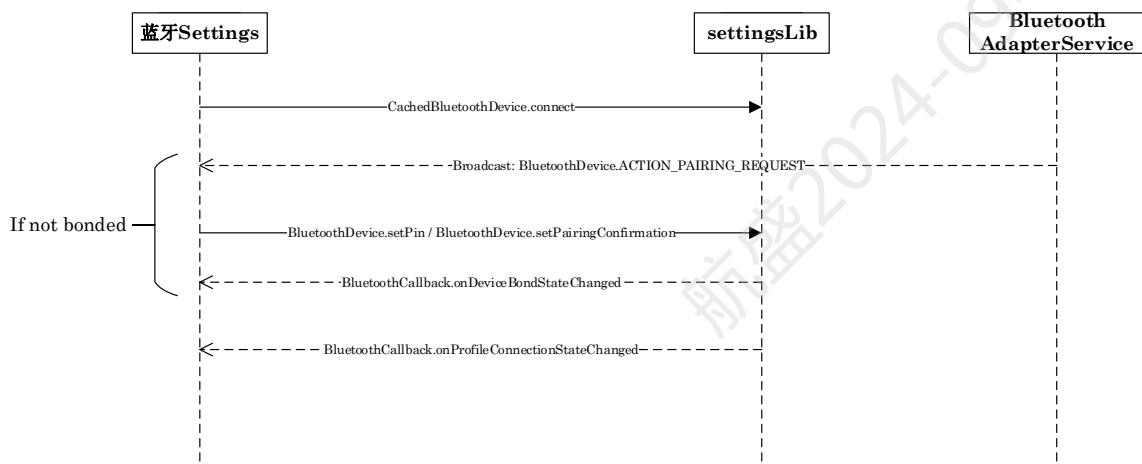


图 18-5 蓝牙连接流程

通过 `CachedBluetoothDevice.connect` 主动发起连接会连接该设备支持的所有 Profile。

18.2.2.2 蓝牙电话相关流程

蓝牙电话与 Telephony 电话实现接口一样，可以参考 Android Developers 中关于 Telecom 的介绍开发蓝牙电话应用，详见：<https://developer.android.com/guide/topics/connectivity/telecom/>。

该章节以流程图的方式，展示一个电话应用常见流程。

1. 电话应用初始化准备流程

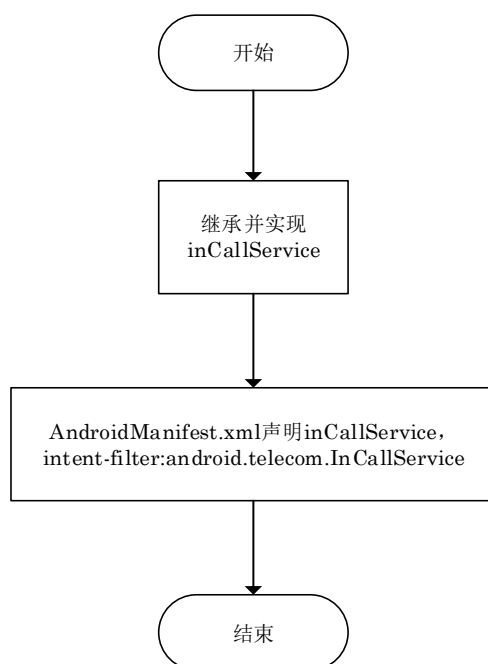


图 18-6 电话应用初始化准备

需要注意：系统中只可以存在唯一一个电话应用，因为 Framework 的 `TelecomService` 是通过“`android.telecom.InCallService`”找到电话应用实现的 `inCallService` 来回调通知的，后面通话状态的变化通过 `inCallService` 通知到电话应用。

2. 拨打及挂断电话流程

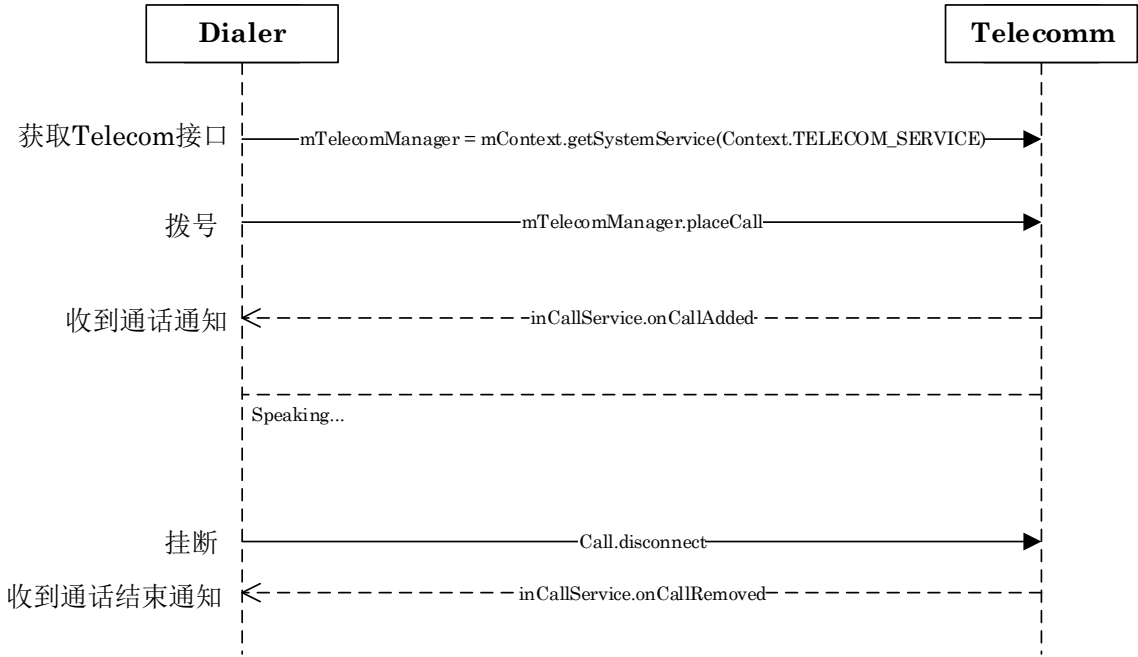


图 18-7 拨打及挂断电话流程

来电接听及挂断与拨打及挂断电话流程类似。

3. 蓝牙通话声音切换流程

蓝牙通话时，可以将通话声音在车机与手机之间切换，此功能无法通过 **Telecom** 接口实现，因此当蓝牙电话时，必须使用蓝牙 **HeadsetClientProfile** 的接口实现通话声音从车机与手机之间切换。

本章第 18.4 节会介绍电话应用如何区分当前 **Call** 是否是蓝牙 **Call**，如何获取蓝牙 **HeadsetClientProfile** 接口，这里侧重介绍蓝牙通话时声音切换流程。

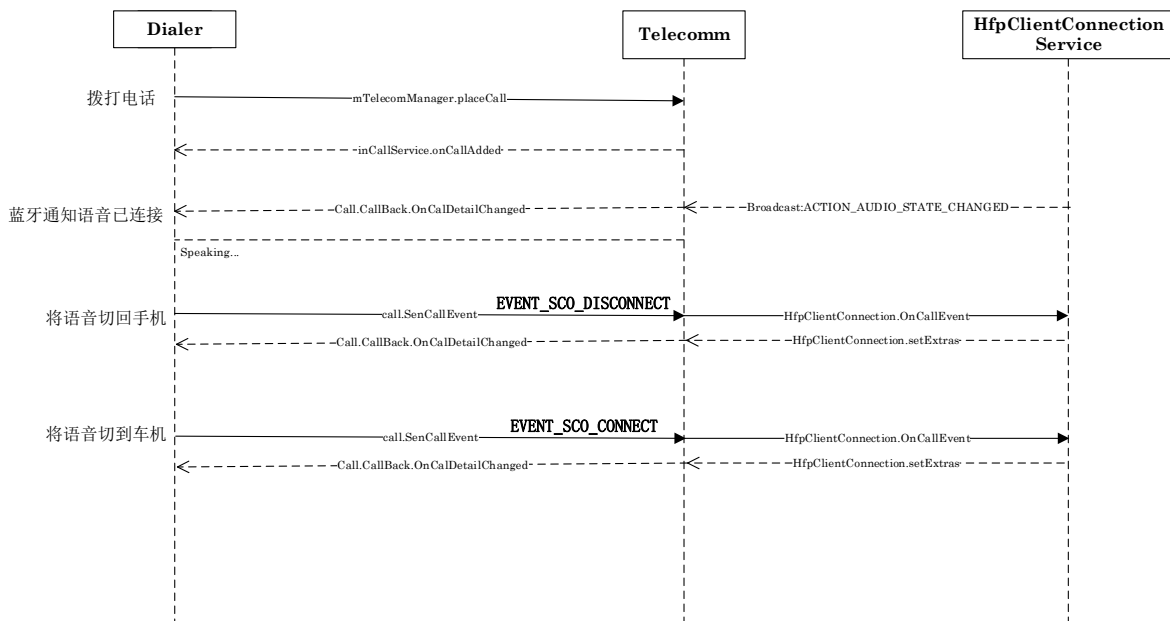


图 18-8 蓝牙通话语音切换流程

18.2.2.3 电话本相关流程

18.2.2.3.1 原生 Android 电话本功能

如果使用原生 Android 接口实现蓝牙电话本功能，用户不需要关心蓝牙电话本如何从手机同步。当蓝牙电话功能连接上手机后，系统自动在后台同步手机电话本及通话记录，当蓝牙手机功能连接断开后，系统自动删除该手机的电话本及通话记录。

蓝牙电话本应用只需监听系统电话本及通话记录变化，从系统电话本及通话记录里面加载数据即可。系统电话本及通话记录的实现可参考 Android Developers:

<https://developer.android.com/guide/topics/providers/contacts-provider>

18.2.2.3.2 ATC 基于原生功能的扩展

ATC 在 Android S 原生功能的基础上，添加了可主动下载/停止下载电话本和通话记录的接口，同时，也将当前下载的状态机进度通过广播的形式报告给 APP，其流程如下：

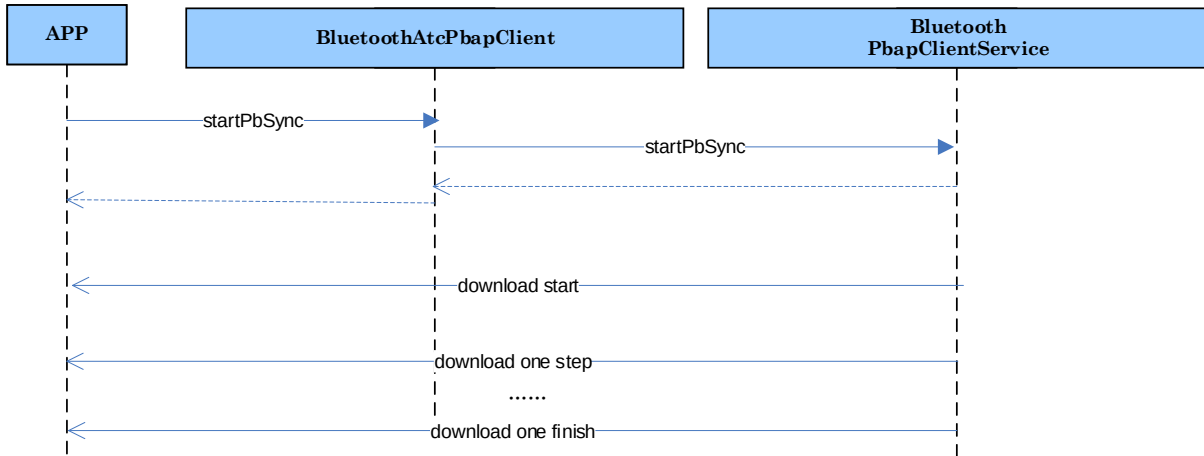


图 18-9 主动下载电话本的流程

18.2.2.4 蓝牙音乐相关流程

原生蓝牙音乐播放器使用 MediaBrowser 框架实现，详细可参考 Android Developers：
<https://developer.android.com/reference/android/media/browse/MediaBrowser>

这里侧重介绍使用 MediaBrowser 播放蓝牙音乐的常见流程。

1. 音乐播放器初始化流程

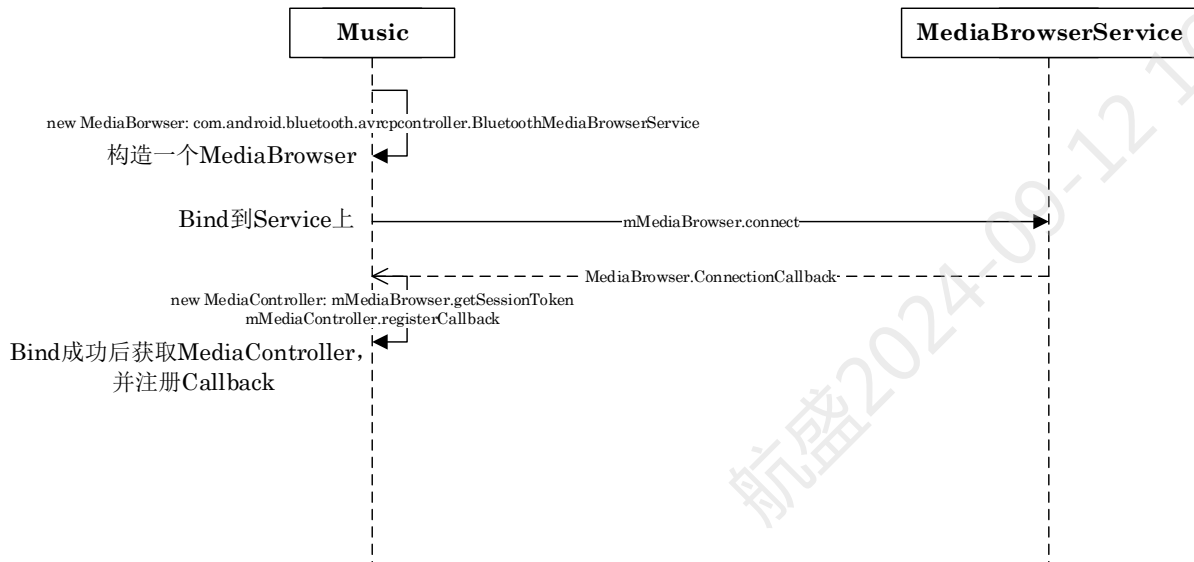


图 18-10 音乐播放器初始化流程

构造一个用于 Bind BluetoothMediaBrowserService 的 MediaBrowser，然后通过 MediaBrowser.connect 来 Bind 到 BluetoothMediaBrowserService 上。当 Bind 成功后，根据

mMediaBrowser.getSessionToken 构造一个播放控制 MediaController，并注册 Callback，以后则通过 MediaController 控制播放，通过 Callback 监听播放状态及媒体信息变化。

2. 音乐播放流程

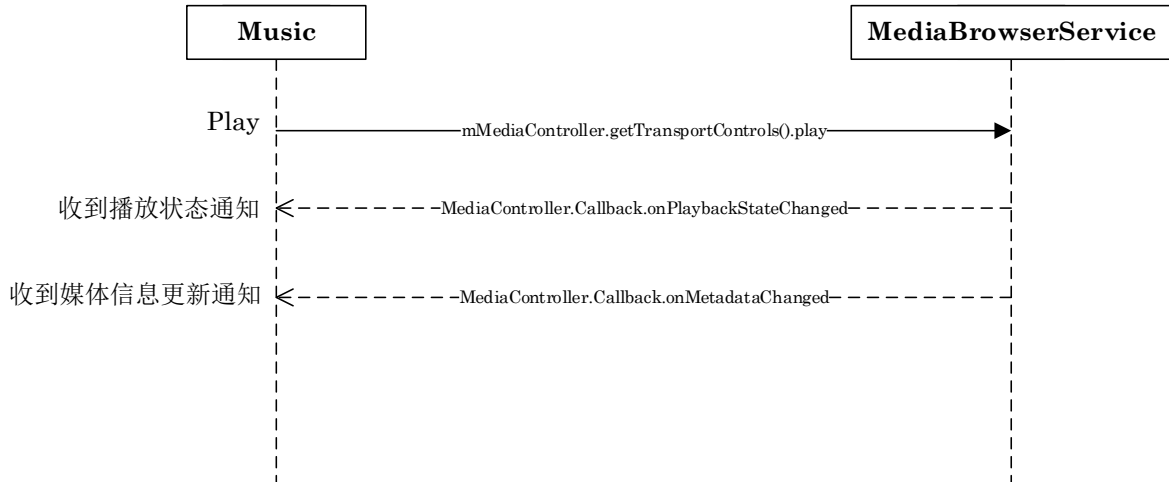


图 18-11 音乐播放流程

3. 播放列表获取流程

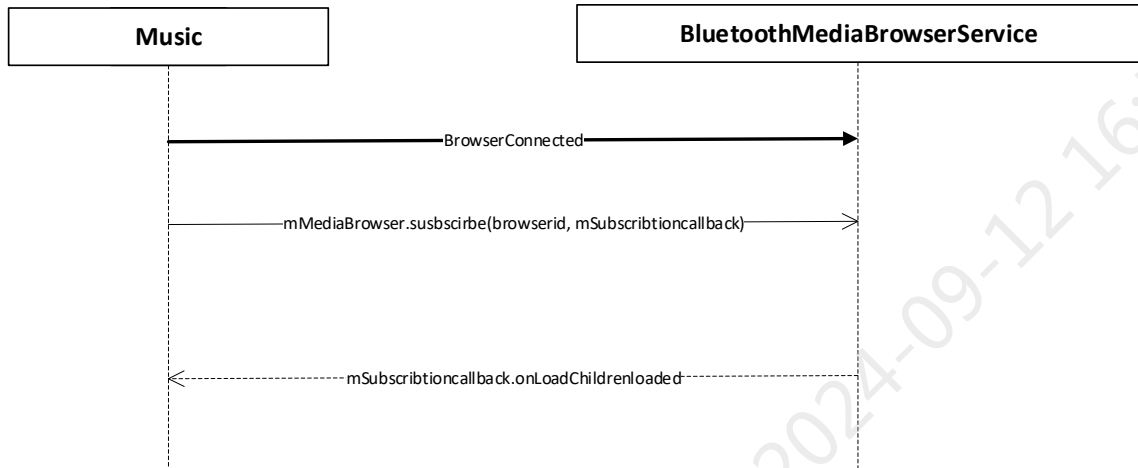


图 18-12 列表浏览流程

18.2.2.5 蓝牙短信相关流程

手机收到新短信时，会通知车机端的 BluetoothMapClientService，BluetoothMapClientService 就会主动获取该新短信内容，获取到后通过广播通知 App 收到了新短信。

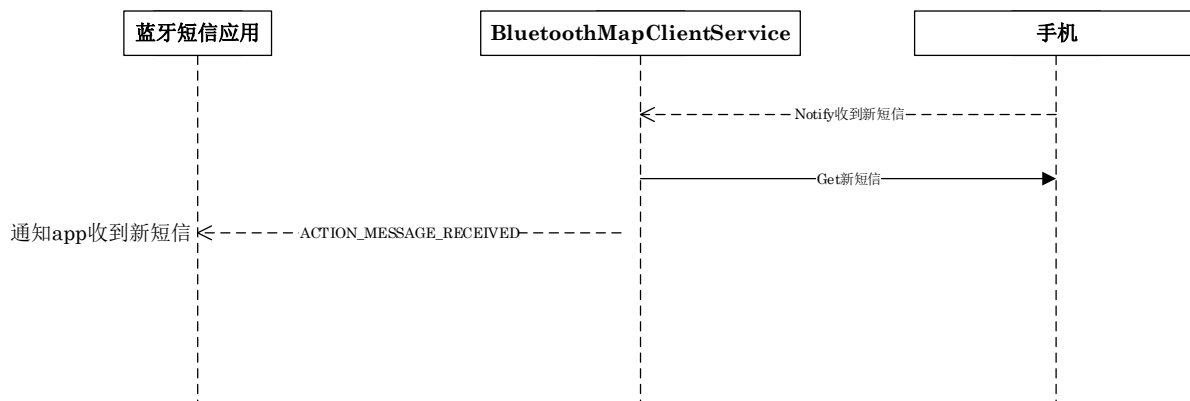


图 18-13 接收新短信流程

同步短信时，App 调用 get Message 相关接口，BluetoothMapClientService 则会从手机端同步符合条件的短信过来，每同步到一条新短信，就会广播 ACTION_MESSAGE_RECIVED 通知 App。

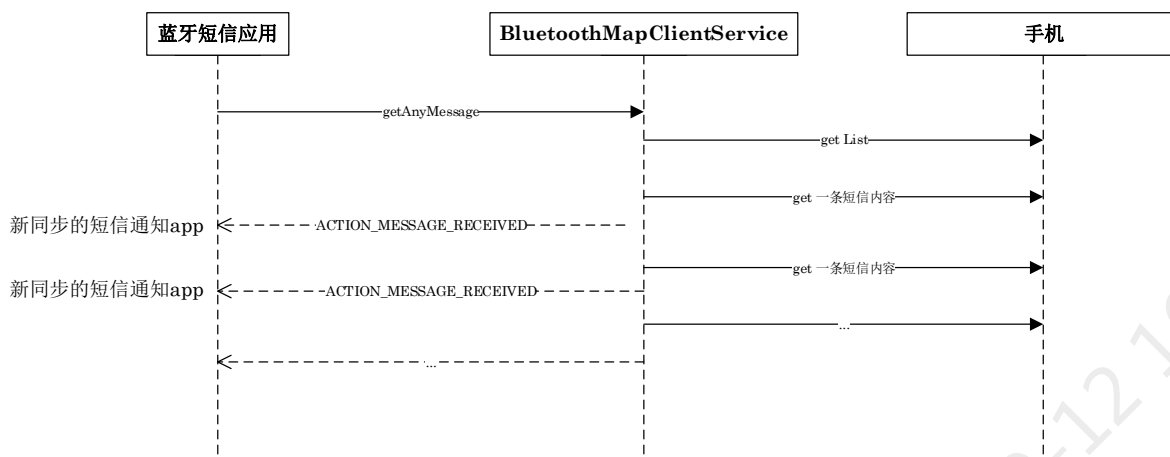


图 18-14 短信同步流程

App 发送新短信时，最终可通过接收广播判定该短信是否发送成功。

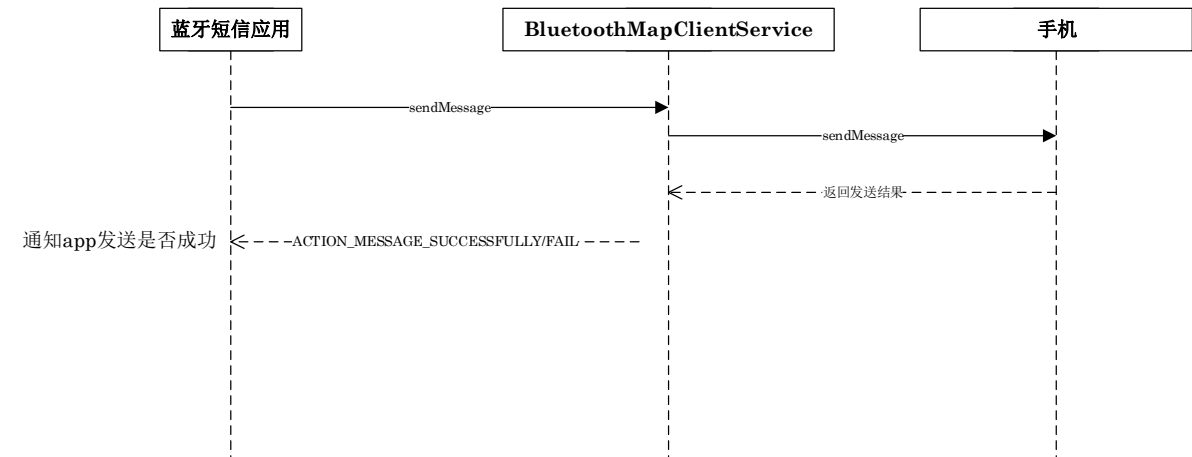


图 18-15 短信发送流程

18.3 接口描述

18.3.1 Android 原生接口

蓝牙的大部分功能可以通过原生 Android 接口实现，可以通过 Android Developers 获得帮助：
<https://developer.android.com/>

18.3.2 ATC 特有接口

18.3.2.1 GAP

18.3.2.1.1 通用蓝牙接口

com.autochips.bluetooth.BluetoothAtcGap。

表 18-3 BluetoothAtcGap API

API	说明
getRemoteConnectable	获取远端设备是否在可连接范围内
setDeviceOutOfBandData	OOB 配对时，设置远端设备的 OOB Data
readOutOfBandData	OOB 配对时，获取本地设备的 OOB Data

表 18-4 BluetoothAtcGap 广播参数

API	说明
ACTION_CONNECTABLE	调用 getRemoteConnectable 的检测结果返回广播
EXTRA_RESULT	ACTION_CONNECTABLE 的参数， CONNECTABLE(0)表示检测的设备在可连接范围内，NOT_CONNECTABLE(-1)表示检测的设备不在可连接范围内

18.3.2.1.1.1 BluetoothAtcGap.getRemoteConnectable

【描述】

获取远端设备是否在可连接范围内，结果会通过广播 ACTION_CONNECTABLE 返回。

【语法】

```
boolean getRemoteConnectable(BluetoothDevice device);
```

【返回值】

- true: 调用成功
- false: 调用失败

【参数】

device: 远端蓝牙设备

18.3.2.1.1.2 BluetoothAtcGap.setDeviceOutOfBandData

【描述】

OOB 配对时，设置远端设备的 OOB Data。

【语法】

```
boolean setDeviceOutOfBandData(BluetoothDevice device, byte[] hash, byte[] randomizer);
```

【返回值】

- true: 调用成功
- false: 调用失败

【参数】

- device: 远端蓝牙设备
- hash: OOB Data 的 hash 序列

- randomizer: OOB Data 的 random key

18.3.2.1.1.3 BluetoothAtcGap.readOutOfBandData

【描述】

OOB 配对时，获取本地设备的 OOB Data。

【语法】

```
Pair<byte[], byte[]> readOutOfBandData();
```

【返回值】

OOB Data 的 Hash 和 Randomizer 对

18.3.2.2 Map Client 和 Pbap Client

18.3.2.2.1 蓝牙短信

com.autochips.bluetooth.BluetoothAtcMapClient。

表 18-5 BluetoothAtcMapClient API

API	说明
getAnyMessages	请求从手机端获取短信

表 18-6 BluetoothAtcMapClient 广播参数

API	说明
EXTRA_MESSAGE_STATUS	ACTION_MESSAGE_RECEIVED 广播新增参数，标识短信是“已读”或者“未读”
EXTRA_MESSAGE_DATE_TIME	ACTION_MESSAGE_RECEIVED 广播新增参数，表示短信的日期时间

18.3.2.2.1.1 BluetoothAtcMapClient.getAnyMessages

【描述】

请求从手机端获取短信，若手机响应并返回短信内容，则会广播 ACTION_MESSAGE_RECEIVED。

【语法】

```
boolean getAnyMessages(BluetoothDevice device, int limit);
```

【返回值】

- true: 请求发送成功
- false: 请求发生失败

【参数】

- device: 远端蓝牙设备
- limit: 请求获取短信的个数

18.3.2.2.2 蓝牙电话本

com.autochips.bluetooth.BluetoothAtcPbapClient。

表 18-7 BluetoothAtcPbapClient API

API	说明
startPbSync	当关闭系统自动同步电话本功能时，调用这个接口可以主动开始电话本同步
stropPbSync	当关闭系统自动同步电话本功能时，调用这个接口可以主动停止当前电话本同步

表 18-8 电话本广播

API	说明
ACTION_PHONEBOOK_DOWNLOAD_STATE_IND	电话本同步状态的广播。 参数： ▪ EXTRA_DEVICE, 同步的设备 ▪ EXTRA_PATH， 同步的路径，如 PBMGR_PHONEBOOK. ▪ EXTRA_STATE， 同步的状态，如 PB_IND_START. ▪ EXTRA_LISTSIZE， 同步列表的总记录个数
ACTION_PHONEBOOK_DOWNLOAD_ONESTEP	电话本同步进度的广播。 参数： ▪ EXTRA_DEVICE, 同步的设备 ▪ EXTRA_PATH， 同步的路径，如 PBMGR_PHONEBOOK. ▪ EXTRA_INDEX， 当前同步到的 Index.

18.3.2.2.2.1 BluetoothAtcPbapClient.startPbSync

【描述】

当关闭系统自动同步电话本功能时，调用这个接口可以主动开始电话本同步。

【语法】

```
boolean startPbSync(BluetoothDevice device, int path);
```

【返回值】

- true: 请求发送成功
- false: 请求发生失败

【参数】

- device: 远端蓝牙设备
- path: 同步的路径，可选：PBMGR_PHONEBOOK, PBMGR_COMBINED_CALLED_HISTORY, PBMGR_FAVORATE 之一，不可复合。

【注意】

必须在 PBAP 连接上后，才能调用，否则调用无效。

18.3.2.2.2.2 BluetoothAtcPbapClient.stopPbSync

【描述】

当关闭系统自动同步电话本功能时，调用这个接口可以主动停止当前电话本同步。

【语法】

```
boolean stopPbSync(BluetoothDevice device);
```

【返回值】

- true: 请求发送成功
- false: 请求发生失败

【参数】

device: 远端蓝牙设备

18.4 使用说明和注意事项

本章节将会一步一步帮助用户，如何使用 Android 原生接口构建一套 CarKit 蓝牙应用。

首先请熟悉 18.2.2 节各过程的流程。其次，需要特别注意的是，Carkit 蓝牙应用必须是 System App，也就是必须是预打包在 System 的 App，否则可能会遇到权限问题。

18.4.1 构建蓝牙 Settings

这里介绍的蓝牙 Settings，是 Carkit 蓝牙应用里面，处理蓝牙开关、搜索、配对、连接、自动连接等与 Settings 相关的功能。注意，这里使用的接口为 com.android.settingslib.bluetooth 里面的接口。

【步骤 1】在 AndroidManifest.xml 里面声明操作蓝牙所需要的权限。

AndroidManifest.xml 里添加：

```
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"/>
```

【步骤 2】蓝牙 Settings 相关接口获取及初始化。

Setting 相关接口初始化：

```
mLocalBluetoothManager = LocalBluetoothManager.getInstance(context,
mOnInitCallback);
mCachedDeviceManager = mLocalBluetoothManager.getCachedDeviceManager();
mBluetoothEventManager = mLocalBluetoothManager.getEventManager();
mBluetoothProfileManager = mLocalBluetoothManager.getProfileManager();

mBluetoothEventManager.registerCallback(mBluetoothCallback);
mBluetoothProfileManager.addServiceListener(mServiceListener);
```

【步骤 3】实现监听的回调接口，以处理各种状态变化。

实现 ServiceListener，以监听 Profile 相关接口是否可用。

```
LocalBluetoothProfileManager.ServiceListener mServiceListener = new
LocalBluetoothProfileManager.ServiceListener() {
    void onServiceConnected() {
        //与Bluetooth Service bind 上了，可以调用各Profile 的接口了，比如可以获取连接
        状态，可以发起连接了。
    }

    void onServiceDisconnected() {
        //与Bluetooth Service unbind 了，此后不能再调用Profile 相关的接口了。
    }
};
```

实现 BluetoothCallback，以监听蓝牙各状态变化：

```
BluetoothCallback mBluetoothCallback = new BluetoothCallback() {
    void onBluetoothStateChanged(int bluetoothState) {
```

```
//todo. 蓝牙开关状态变化
}

void onScanningStateChanged(boolean started) {
    //todo. 搜索状态变化
}

void onDeviceAdded(CachedBluetoothDevice cachedDevice) {
    //todo. 搜索到新设备
}

void onDeviceDeleted(CachedBluetoothDevice cachedDevice) {
    //todo. 设备被移除
}

void onDeviceBondStateChanged(CachedBluetoothDevice cachedDevice, int bondState) {
    //todo. 设备配对状态变化
}

void onConnectionStateChanged(CachedBluetoothDevice cachedDevice, int state) {
    //todo. 设备连接状态变化
}

void onActiveDeviceChanged(CachedBluetoothDevice activeDevice, int bluetoothProfile) {
    //todo. 活动设备变化
}

void onProfileConnectionStateChanged(CachedBluetoothDevice cachedDevice) {
    //todo. 协议连接状态变化
}
};
```

至此，CarKit 蓝牙应用 Setting 部分已经基本完成，后续通过 LocalBluetoothAdapter 接口控制本地车机蓝牙，通过 CachedBluetoothDevice 连接或者断开远端蓝牙设备，通过 BluetoothCallback 监听状态变化，更新 UI 即可。

18.4.2 构建蓝牙电话 Dialer

需要注意，Android 系统中只允许使用唯一的 Dialer，也就是 Telephony 电话与蓝牙电话必须要使用相同的 Dialer 处理。用户可以从 Android Developers 中获得帮助：

<https://developer.android.com/guide/topics/connectivity/telecom/selfManaged>

【步骤 1】 在 AndroidManifest.xml 里面声明操作电话所需要的权限。

声明使用电话的权限

```
<uses-permission android:name="android.permission.MANAGE_OWN_CALLS"/>
<uses-permission android:name="android.permission.READ_CALL_LOG"/>

<!-- Needed only if your calling app reads numbers from the `PHONE_STATE`
     intent action. -->
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
```

【步骤 2】继承并实现 `android.telecom.InCallService`，以监听通话状态变化。

继承并实现 `InCallService`

```
import android.telecom.Call;
import android.telecom.InCallService;
import android.telecom.TelecomManager;

public class InCallServiceImpl extends InCallService {
    private TelecomManager mTelecomManager;

    @Override
    public void onCreate() {
        super.onCreate();

        // 获取 TelecomManager 接口
        mTelecomManager =
getApplicationContext().getSystemService(TelecomManager.class);
    }

    @Override
    public void onCallAdded(Call telecomCall) {
        super.onCallAdded(telecomCall);

        // 当有新电话添加时，注册监听该电话状态变化
        telecomCall.registerCallback(mCallListener);
    }

    @Override
    public void onCallRemoved(Call telecomCall) {
        // 当电话结束时，注销该电话状态变化监听
        telecomCall.unregisterCallback(mCallListener);

        super.onCallRemoved(telecomCall);
    }

    private final Call.Callback mCallListener = new Call.Callback() {
        @Override
```



```
        public void onStateChanged(Call call, int state) {  
            // 当该通话状态发生变化  
        }  
    };  
}
```

【步骤 3】在 AndroidManifest.xml 里面声明 InCallService，以便 Framework 层的 TelecomService 能够找到 App 的 InCallService。

intent-filter 必须是："android.telecom.InCallService"。

```
<service android:name=".InCallServiceImpl"  
    android:permission="android.permission.BIND_INCALL_SERVICE"  
    android:exported="true">  
    <meta-data android:name="android.telecom.IN_CALL_SERVICE_UI"  
android:value="true" />  
    <intent-filter>  
        <action android:name="android.telecom.InCallService" />  
    </intent-filter>  
</service>
```

至此，Dialer 应用基本构建完成，后续通过 mTelecomManager 接口拨打电话，通过 InCallService 的回调监听通话变化，通过 Call 控制当前通话，即可完成 Dialer 应用。

由于 Dialer 需要同时处理 Telephony Call 和 Bluetooth Call，通过如下方法可以区分是否是 Bluetooth Call。

通过 Call 判定是否是 Bluetooth Call:

```
final String HFP_CLINET_CONNECTION_SERVICE_CLASS_NAME  
    = "com.android.bluetooth.hfpclient.connserv.HfpClientConnectionService";  
  
public boolean isBluetoothCall(Call telecomCall) {  
    if (telecomCall == null) {  
        return false;  
    }  
    PhoneAccountHandle phoneAccountHandle =  
    telecomCall.getDetails().getAccountHandle();  
    if (phoneAccountHandle != null && phoneAccountHandle.getComponentName() !=  
    null) {  
        return HFP_CLINET_CONNECTION_SERVICE_CLASS_NAME.equals(  
            phoneAccountHandle.getComponentName().getClassName()  
        );  
    }  
  
    return false;  
}
```

如果判定当前电话是 Bluetooth Call，用户还可以通过如下方法获取到这个 Bluetooth Call 是来自哪个蓝牙设备的电话：

根据 Call 获取到该 Call 来自于哪个蓝牙设备（手机）：

```
public BluetoothDevice getDevice(Call telecomCall) {  
    BluetoothDevice device = null;  
    PhoneAccountHandle phoneAccountHandle;  
    PhoneAccount account;  
    String btAddr;  
  
    if (telecomCall == null) return null;  
  
    phoneAccountHandle = telecomCall.getDetails().getAccountHandle();  
    if (phoneAccountHandle == null) return null;  
  
    account = mTelecomManager.getPhoneAccount(phoneAccountHandle);  
    if (account == null) return null;  
  
    btAddr = account.getAddress().getSchemeSpecificPart();  
    if (BluetoothAdapter.checkBluetoothAddress(btAddr)) {  
        device = BluetoothAdapter.getDefaultAdapter().getRemoteDevice(btAddr);  
    }  
  
    return device;  
}
```

根据 Call 获取到蓝牙 Device 后，即可根据自己需求，对 Bluetooth Call 做一些特殊操作，比如蓝牙通话声音从车机切回手机，就可以根据蓝牙 Device，断开该 Call 的 Bluetooth Audio 连接，声音就回到手机端了。

18.4.3 构建蓝牙音乐播放器

蓝牙音乐，使用的是 Media session and media controller 的应用框架，用户可以从 Android Developers 了解这种 Media app 框架：

<https://developer.android.com/guide/topics/media-apps/media-apps-overview>。

【步骤 1】 构造用于 bind BluetoothMediaBrowserService 的 MediaBrowser，并连接。



注意：

构造 MediaBrowser，Context 必须是 Application，不能是 Activity，否则会 connect 失败。

```
mMediaBrowser = new MediaBrowser(mContext, //注意这个Context 必须是Application 的
Context
    new ComponentName("com.android.bluetooth",
        "com.android.bluetooth.avrcpcontroller.BluetoothMediaBrowserService"),
        mConnectionCallback,
        null);
mMediaBrowser.connect();
```

【步骤 2】实现 ConnectionCallback，以监听 bind 结果。

监听 bind 成功后，获取到 MediaController，并注册媒体状态信息变化监听。

```
private MediaBrowser.ConnectionCallback mConnectionCallback = new
MediaBrowser.ConnectionCallback() {
    @Override
    public void onConnected() {
        //连接上 BrowserService 了

        //构造一个MediaController
        mMediaController = new MediaController(mContext,
mMediaBrowser.getSessionToken());
        //注册媒体状态信息监听
        mMediaController.registerCallback(mMediaControllerCallback);
    }

    @Override
    public void onConnectionSuspended() {
        //连接 suspend 时
    }
};
```

【步骤 3】实现媒体状态信息变化的监听。

根据媒体信息变化，更新 UI 即可。

```
private MediaController.Callback mMediaControllerCallback = new
MediaController.Callback() {
    @Override
    public void onPlaybackStateChanged(PlaybackState state) {
        //更新播放状态
    }

    @Override
    public void onMetadataChanged(MediaMetadata metadata) {
        //更新媒体信息，ID3 等
    }
};
```

```
//专辑图片通过 MediaMetadata.METADATA_KEY_ALBUM_ART_URI 获取更新
}

@Override
public void onSessionEvent(String event, Bundle extras) {
    //定制化的其他 Event 通知
}
};
```

至此，蓝牙音乐播放器已经基本构造完成，后续即可通过 `MediaController` 的接口控制播放（play/pause/next/pre 等），媒体状态变化后，`MediaControllerCallback` 会通知更新 UI。

18.5 用户定制化

18.5.1 关闭 in-band ring tone

In-band ring tones 功能是指蓝牙来电的时候，能够播放手机端传过来的来电彩铃声。

手机和车机蓝牙都支持 in-band ring tones 才能实现此功能。

ATC 蓝牙默认打开此功能。

【关闭方法】

设置系统属性：“persist.vendor.bt.inbandringtone” 值为 “disable”；

设置后，下一次打开蓝牙生效。

【读写系统属性的方法】

1. adb shell

读系统属性：adb shell getprop [key]

写系统属性：adb shell setprop [key] [value]

例如，关闭蓝牙 in-band ring tones 功能：

```
adb shell setprop persist.vendor.bt.inbandringtone disable
```

2. SystemProperties

在 Java 层，android 提供读写系统属性的封装 `SystemProperties`。这个类是隐藏的，在 `android.os` 里面，这里就不详细讲使用方法。

3. 系统定制化的时候设置属性

例如：在 `/device/` 对应设备 `/device.mk`，添加想要设置的系统属性。

示例：关闭蓝牙 in-band ring tones 功能，在文件里面添加一项：

```
PRODUCT_DEFAULT_PROPERTY_OVERRIDES +=
persist.vendor.bt.inbandringtone=disable
```

18.5.2 蓝牙地址

QCA6595 模块内置了蓝牙地址。

CYW88373 固件初始化时，需要从 host 端将地址下发到固件，如下说明适用 CYW88373。

蓝牙地址会从 Metazone 中读取，如果 Metazone 读不到有效蓝牙地址，则会随机生成一个蓝牙地址，使用此蓝牙地址并写入 Metazone 中保存起来，下一次即可使用 Metazone 中地址。也就是若升级时没有写入有效蓝牙地址，第一次开机则会随机生成一个蓝牙地址，后续使用该蓝牙地址。

蓝牙地址使用流程如下图：

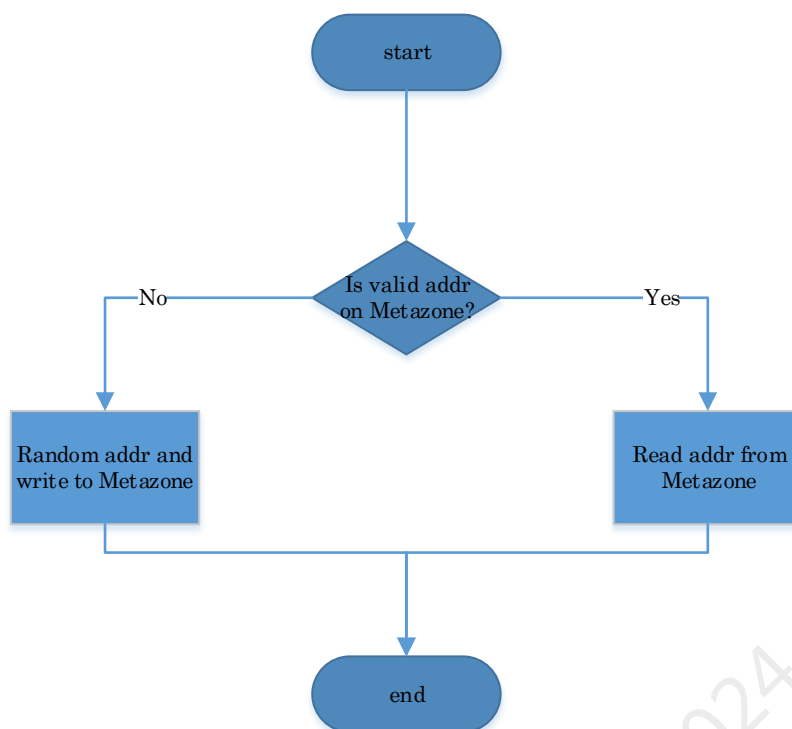


图 18-16 蓝牙地址使用流程

【配置蓝牙地址方法】

用户可以在升级的时候，往 Metazone 里面写入一个有效的蓝牙地址。

在 Native, LK, Kernel 等均可以操作 Metazone，详细 Metazone 操作方法，请参考 Metazone 部分的文档内容。

如下是在 Native 层写入蓝牙地址的 demo，需要链接 libmetazone 这个动态链接库。

```
#include <metazone_index.h>
```

```

#define METAZONE_BTADDR_LEN (6)
#define MTZ_WRITE_FLAG_NONE (0)
#define MTZ_WRITE_FLAG_FACTORYREST (1)

extern unsigned int MetaZone_Init();
extern unsigned int MetaZone_Deinit();
extern unsigned int MetaZone_ReadBinary(unsigned int u4Idx, char *pbData, unsigned int u4Siz);
extern unsigned int MetaZone_SpecWriteBinary(unsigned int u4Idx, char *pbData, unsigned int u4Siz, unsigned int u4Flag);
extern unsigned int MetaZone_Flush(int fgblock);

int write_bt_addr(unsigned char *bt_mac_addr) {
    unsigned int ret = 0;
    ret = MetaZone_Init();
    if (ret == 0) {
        ret = MetaZone_SpecWriteBinary(METAZONE_BT, (char *)bt_mac_addr, METAZONE_BTADDR_LEN, MTZ_WRITE_FLAG_FACTORYREST);
        LOG_DBG("metazone write (%d)", ret);
        MetaZone_Flush(FALSE);
        MetaZone_Deinit();
    } else {
        LOG_ERR("metazone init fail! (%d)", ret);
    }

    return 0;
}

```

【车机蓝牙地址查看方法】

设置->关于车载平台->状态信息->蓝牙地址

必须要将蓝牙打开后才可以查看。

18.5.3 微信语音、语音识别通话界面弹框配置

手机上微信语音、语音识别软件启用的时候，手机会模拟一个蓝牙电话给车机，ATC 蓝牙默认会过滤掉这类无效的蓝牙通话，用户依然可以通过配置系统属性关闭此次过滤功能。

【关闭方法】

设置系统属性：“persist.vendor.bt.callfilter” 值为 “disable”。

设置后，下一次打开蓝牙生效。

如果用户不是很了解系统属性如何配置，可以参考 18.5.1 节。

18.5.4 关闭蓝牙音乐服务端控制 audioFocus 功能

如果用户需要在 APP 中控制 a2dp 的 audio Focus，则可以关闭此功能。这样蓝牙音乐 audio Focus 的申请和释放就需要用户 APP 去控制，用户可以通过配置系统属性关闭此功能。

【关闭方法】

设置系统属性：“persist.vendor.bt.a2dpaudiofocus” 值为 “disable”。

设置后，下一次打开蓝牙生效。

如果用户不是很了解系统属性如何配置，可以参考 18.5.1 节。

18.5.5 关闭蓝牙通话音量同步功能

在蓝牙通话时，从车机或者手机端调节音量，车机与手机都会相互同步通话音量设置。这就是蓝牙通话音量同步功能，用户可以通过配置系统属性关闭此功能。

【关闭方法】

设置系统属性：“persist.vendor.bt.scovolumesync” 值为 “disable”。

设置后，下一次打开蓝牙生效。

如果用户不是很了解系统属性如何配置，可以参考 18.5.1 节。

18.5.6 A2dp source 功能配置

在蓝牙音乐中，默认配置 A2DP sink 端。如果客户需要将 A2DP 配置为 source 端，用户可以通过配置系统属性来进行切换。但是不支持在运行过程中切换，切换后需要重启机器。

【配置方法】

设置如下两个系统属性值：

设置系统属性：“persist.vendor.bt.a2dp.role” 值为 “source”

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

603 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

设置系统属性：“persist.bluetooth.enablenewavrcp”值为“true”

设置后，重启机器生效。

如果用户不是很了解系统属性如何配置，可以参考 [18.5.1 节](#)。

18.5.7 蓝牙短信同步时间限制配置

在蓝牙短信同步时，会限制手机短信时间，默认只同步手机最近 30 天内的短信，可以通过下面系统属性配置，更改这个限制。

【配置方法】

设置如下系统属性值：

设置系统属性：“persist.vendor.bt.mapclient.downloadperiod”

设置的值代表限制的天数，比如限制只同步手机最近 7 天内的短信，则设置该系统属性值为 7。

假如此时 APP 调用同步 20 条短信，而最近 7 天只有 10 条短信，则只会同步 10 条短信。

设置后，下一次打开蓝牙生效。

如果用户不是很了解系统属性如何配置，可以参考 [18.5.1 节](#)。

18.5.8 关闭电话本联系人头像同步

同步联系人时，若联系人设置的有头像功能，则用户可以选择是否同步联系人头像。联系人头像通常比较大，会降低电话本同步速度。

ATC 默认同步联系人头像，客户可配置不同步联系人头像。

【配置方法】

设置如下系统属性值：

设置系统属性：“persist.vendor.bt.pb.photo”，值为 disable，则不同步联系人头像。

如果用户不是很了解系统属性如何配置，可以参考 [18.5.1 节](#)。

18.5.9 关闭原生电话本自动同步功能

有些用户不喜欢原生电话本在连接上 Pbap Client 后，自动在后台同步，可通过配置关闭自动同步功能，当调用下载接口才开始同步，接口参考 [18.3.2.2.2 接口描述部分](#)。

ATC 默认开启自动同步功能。

【关闭方法】

设置如下系统属性值：

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

604 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

设置系统属性：“persist.vendor.bt.pb.autosync” 值为 disable，则不会自动同步电话本

如果用户不是很了解系统属性如何配置，可以参考 [18.5.1 节](#)。

18.5.10 蓝牙网络评分配置

Android 下有多种网络存在，连接管理通过一个评分机制来实现不同接入方式的选择，当有更高分数的网络就绪时就将当前分值低的连接断开。在实际运行中，Android 会根据网络的实时状态调整各网络评分。

ATC 默认蓝牙网络评分 55、WIFI 网络评分 60，用户可以通过配置系统属性修改蓝牙网络评分。

【配置方法】

设置如下系统属性值：

设置系统属性：“persist.vendor.bt.pan.score”，值为 int 类型，以设置蓝牙网络评分。

如果用户不是很了解系统属性如何配置，可以参考 [18.5.1 节](#)。

18.5.11 蓝牙 HFP 连接上限配置

蓝牙 HFP 最大连接设备数是 2，也就是可以同时连接两个设备的 HFP，用户可以配置将 HFP 最大连接设备数限制为 1。

【配置方法】

设置如下系统属性值：

设置系统属性：“persist.vendor.bt.maxhfpdev”，设置的值代表限制连接数。

设置后，下一次打开蓝牙生效。

如果用户不是很了解系统属性如何配置，可以参考 [18.5.1 节](#)。

18.5.12 蓝牙通话记录同步条数上限设置

蓝牙通话记录同步条数上限默认为 6000 条，也就是最多同步 6000 条通话记录，用户可以通过如下配置方法配置上限值。

【配置方法】

设置如下系统属性值：

设置系统属性：“persist.vendor.bt.limit.calllog”，设置的值代表限制条数。

设置后，下一次打开蓝牙生效。

如果用户不是很了解系统属性如何配置，可以参考 [18.5.1 节](#)。

18.5.13 关闭蓝牙 A2DP Sink AAC Codec 功能

有些手机蓝牙音乐 AAC Codec 支持不好，若要兼容这种类型的手机，可以选择关闭车机蓝牙 A2DP Sink AAC Codec 功能。

【配置方法】

设置如下系统属性值：

设置系统属性：“persist.vendor.bt.sink.sbc.only”，值为 1，则代表 A2DP Sink 只支持 sbc codec，不支持 aac codec。默认值或者是其他值，则支持 aac codec。

设置后，下一次打开蓝牙生效。

如果用户不是很了解系统属性如何配置，可以参考 18.5.1 节。

18.5.14 配置蓝牙芯片

AC8025 可以支持使用蓝牙芯片：QCA6595/ CYW88373。

【配置方法】

默认蓝牙芯片选择为 CYW88373，如需配置 QCA6595 可在编译时添加参数 bt_wifi_chip=qca6595_pcie，即可编译为使用 QCA6595 的 image。

18.6 典型问题

18.6.1 如何获取蓝牙 HCI log

【问题描述】

如何获取蓝牙 HCI log？

【解决方案】

为了快速澄清用户遇到的蓝牙兼容性相关问题，通常需要获取到蓝牙 HCI log，下面介绍如何打开并获取蓝牙 HCI log。

【步骤 1】打开 Atclogger。

打开 Atclogger 的入口很多，下面介绍几种：

方法一：直接 adb 命令打开：

```
adb shell am start -n com.autochips.atclogger/com.autochips.atclogger.MainActivity
```

方法二：依次点击 Engineer mode → Log and Debugging → AtcLogger 打开。

【步骤 2】在 Atclogger 里面勾选打开 Hci log。

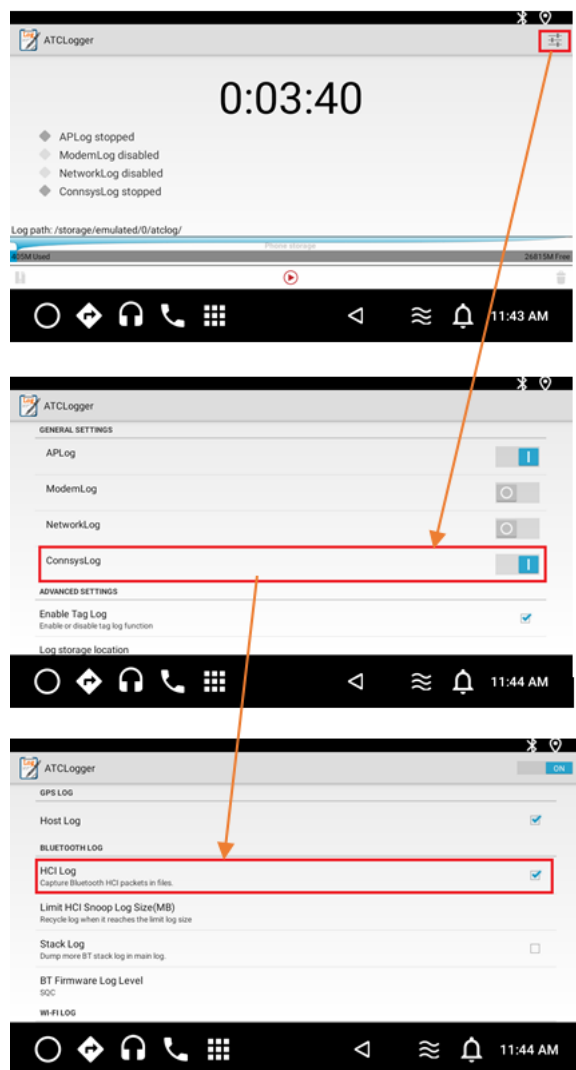


图 18-17 打开 Hci log

【步骤 3】重新关闭、打开蓝牙。

Hci log 打开后，存放在/data/atclog/hci 文件夹下面。

18.6.2 如何进入蓝牙测试模式

【问题描述】

如何进入蓝牙测试模式？

【解决方案】

在做蓝牙 BQB 认证或者各国家无线相关认证时，需要测试蓝牙 RF，这就需要蓝牙进入测试模式。

蓝牙测试默认进入方法：

【步骤 1】进入工程模式。

打开工程模式的入口很多，下面介绍几种：

方法一：直接 adb 命令打开。

```
adb shell am start -n  
com.autochips.engineermode/com.autochips.engineermode.EngineerMode
```

方法二：点击 EngineerMode App

【步骤 2】在工程模式应用下点击 Connectivity → Bluetooth。

选择需要的测试项测试即可。

18.6.3 蓝牙通话声音质量差或者近距离下 A2DP 卡顿

【问题描述】

车机连接手机普遍存在 A2DP 拉几米距离就卡顿或者通话声音质量差问题。

【解决方案】

若存在此问题，请按如下步骤操作：

- 尝试更换比较好的蓝牙天线，看是否有改善。
- 若更换天线无改善，请帮忙先测试蓝牙 RF 性能是否达标。如果 RF 性能不达标，请找 HW 帮忙或者发测试报告给我司，我司会帮忙分析怀疑点。
- 如果以上尝试都无改善，请提供机器给我司调试。

如果是个别手机存在声音质量差问题，请直接提供手机及 Issue 车机给我司，我司会协助用户澄清。

18.6.4 如何知道蓝牙走远了信号丢失断开

【问题描述】

如何知道蓝牙走远了，信号丢失断开？

【解决方案】

ACL 连接断开的时候，app 可以收到 BluetoothDevice.ACTION_ACL_DISCONNECTED 这个广播，带参数“com.autochips.bluetooth.LocalBluetoothProfile.extra.EXTRA_HCI_REASON”里面可以获取到 ACL 断开原因，其中 HCI_ERR_CONNECTION_TIMEOUT(0x08)表示信号丢失而断开。

18.6.5 如何知道蓝牙设备重新回到可连接范围了

【问题描述】

蓝牙走远了，信号丢失断开后，如何知道设备重新回到可连接范围了

【解决方案】

ATC 新增私有接口：18.3.2.1.1.1。

通过该接口，应用可定时检测设备是否已经回到可连接范围内了，建议检测间隔不小于 20 秒，最好 30 秒检测一次。

18.6.6 如何控制蓝牙音乐是否可以播放出声音

【问题描述】

如何控制蓝牙音乐是否可以播放出声音？

【解决方案】

蓝牙音乐使用原生 Audio Focus 方式进行交互管理，这部分逻辑在 Bluetooth Service 层，用户无法通过修改代码来客制化。

蓝牙音乐 Audio Focus 逻辑如下：

1. 如果蓝牙音乐无 Audio Focus，如下条件下才会申请 Audio Focus 播放：
 - 应用层主动调用播放接口，蓝牙音乐服务则申请 Audio Focus 播放。
 - 蓝牙音乐应用在前台，手机端主动播放，蓝牙音乐服务也会申请 Audio Focus 播放。
2. 在蓝牙音乐无 Audio Focus 的其他情况下，蓝牙音乐服务不会申请 Audio focus，并尝试阻止手机播放。比如：蓝牙音乐无 Audio Focus，并且蓝牙音乐应用不在前台，手机端主动播放音乐，车机端不会播出蓝牙音乐声音，车机同时会尝试暂停手机端播放。

也就是，用户可以通过控制蓝牙音乐应用是否在前台，来控制蓝牙应用是否允许播放出声音。蓝牙音乐应用可参考如下代码，实现控制是否在前台：

蓝牙音乐应用如何设置是否在前台：

```
private final String CUSTOM_ACTION_A2DP_PLAYER_FOREGROUND_CHANGED =
    "autochips.bluetooth.a2dp.CUSTOM_ACTION_PLAYER_STATE_FOREGROUND";
private final String FOREGROUND_EXTR_STATE =
    "autochips.bluetooth.a2dp.foreground.extra.CUSTOM_STATE";

public void setPlayerForeground(boolean isForeground) {
    Bundle extData = new Bundle();
    extData.putBoolean(FOREGROUND_EXTR_STATE, isForeground);

    mMediaController.getTransportControls().sendCustomAction(
        CUSTOM_ACTION_A2DP_PLAYER_FOREGROUND_CHANGED,
```

```
    extData);  
}
```

18.6.7 如何打开 Bluedroid 协议栈 log

【问题描述】

如何打开 Bluedroid 协议栈 log?

【解决方案】

分析协议栈问题时，通常需要打开协议栈的 log，可以通过 Atclogger 来打开蓝牙协议栈的 log。

【步骤 1】进入 Atclogger，进入 Atclogger 的方法可参考 18.6.1 节。

【步骤 2】打开 Stack log。

Stack log 打开后，会打到 mobilelog 中，则拉出 atclog 文件夹即可。

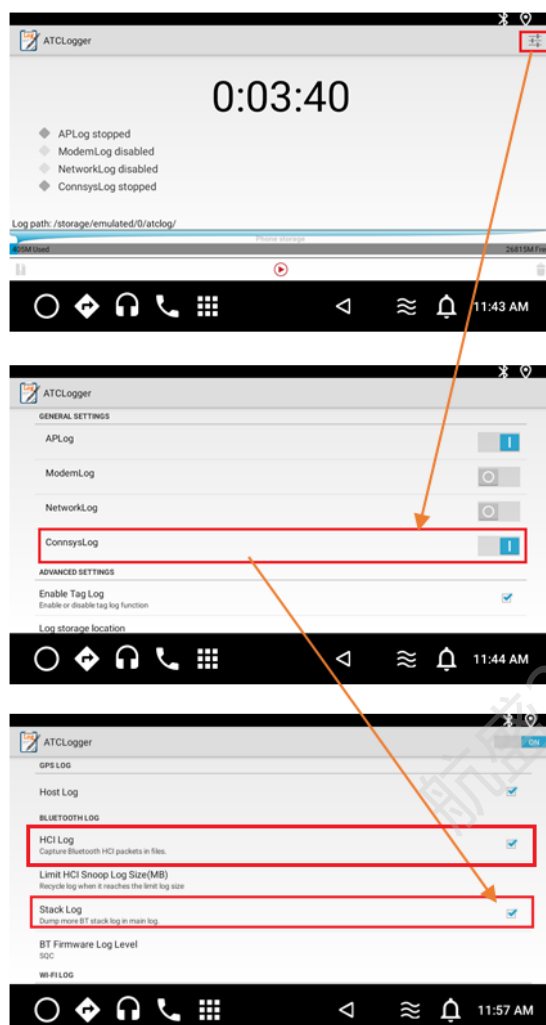


图 18-18 打开 Stack log

18.6.8 如何打开蓝牙通话 audio dump

【问题描述】

分析蓝牙通话声音相关问题，需要打开蓝牙通话 audio dump，如何打开蓝牙通话 audio dump？

【解决方案】

可以通过如下工程模式 APP，来打开蓝牙通话相关的 audio dump：

打开工程模式的入口很多，下面介绍几种：

方法一：直接 adb 命令打开。

```
adb shell am start -n  
com.autochips.engineermode/com.autochips.engineermode.EngineerMode
```

方法二：点击 EngineerMode App

【步骤 1】在工程模式应用下点击 Hardware Testing → Audio。

【步骤 2】勾选如图 18-19 中的选项，并下拉到页面最下方，选择 set 后生效。

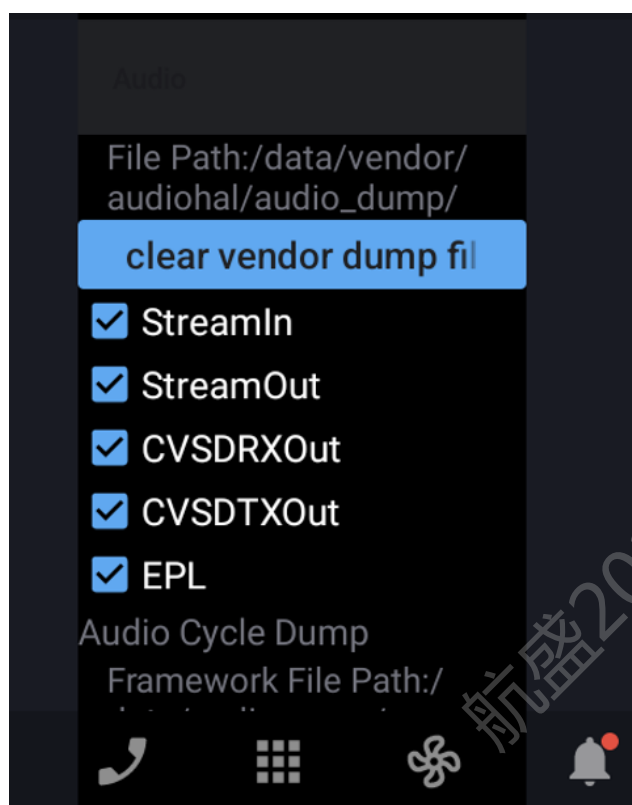


图 18-19 打开 Audio Dump

生成文件路径：/data/vendor/audiohal/audio_dump。

18.6.9 当多 Dialer 存在时，如何配置系统默认电话 Dialer

【问题描述】

当多 Dialer 存在时，如何配置系统默认电话 Dialer？

【解决方案】

- 配置文件 `source/android/packages/services/Telecomm/res/values/config.xml` 中修改 `"ui_default_package"` 和 `"incall_default_class"` 为默认电话 app 的包名和回调电话状态的 `InCallService`；
- 修改后重新编译 Telecom，替换 Telecom.apk 即可生效。
- 若只需要临时测试，亦可通过 `adb shell telecom` 命令临时改变默认 Dialer。

比如：

```
adb shell telecom set-default-dialer com.autochips.dialer
```

即可临时修改默认 Dialer 包为 `"com.autochips.dialer"`，修改后立刻生效。

可通过 `adb shell telecom` 命令获取更多帮助。

18.6.10 使用其他第三方蓝牙模块时，如何关闭 ATC 蓝牙的相关代码

【问题描述】

当客户需要使用其他第三方蓝牙模式时，如何关闭 ATC 蓝牙的相关代码？

【解决方案】

修改 `source/android/device/autochips/${project_name}/ProjectConfig.mk`

ATC_BT_SUPPORT 设置为 no，则会关闭所有 ATC 蓝牙相关的代码，还原回 Android 原生代码。

19 Wi-Fi

19.1 简介

19.1.1 软硬件环境

19.1.1.1 软件环境

该系统集成了 Station/AP/P2P 的功能，APP 使用 Framework 提供的 Wi-Fi 接口，即可完成对应功能的实现，诸如 Station 的开关、搜索、连接/断开；AP 的开关、连接/断、，网络共享；P2P 的搜索、连接等。

该系统使用开源的 wpa_supplicant/hostapd 完成 802.11 协议规定的管理控制功能，使用 Android 原生的 netd 和 iptables 功能完成诸如网络地址转换、配置路由表等网络的相关功能。

19.1.1.2 硬件环境

AC8025 Wi-Fi 方案需要使用如下硬件环境：

- Wi-Fi IC: CYW88373/QCA6595
- CYW8x373 支持 PCIE 和 SDIO 传输，QCA6595 支持 PCIE 传输。

19.1.2 功能

19.1.2.1 Wi-Fi Station

- 打开
- 关闭
- Scan
- 连接 2.4G/5G AP
- 支持 Open/WPA/WPA2/WPA3-SAE/WPA3-Transition 类型的加密方式
- 可支持 20M/40M/80M 带宽
- 断开
- 删除网络
- 添加可见网络

- 添加隐藏网络

19.1.2.2 Wi-Fi Hotspot

- 打开 2.4G/5G Hotspot
- 关闭 Hotspot
- 支持 2.4G/5G 信道
- 可被远端 Wi-Fi 设备搜索到
- 可被远端 Wi-Fi 设备连接
- 设置空闲时 Wi-Fi Hotspot 保持打开的时间
- 修改 Hotspot 配置，如名字、密码等
- 最大连接数
- Block user
- Unblock user
- AP 主动断开 Station
- 支持 20M/40M/80M 带宽
- 支持 Open/WPA/WPA2/WPA3-SAE/WPA3-Transition 类型的加密方式

19.1.2.3 Wi-Fi P2P

- 查找直连设备
- 连接
- 断开

19.1.3 代码结构及文件描述

表 19-1 Wi-Fi 代码结构及描述

文件名	文件路径	文件描述
ATCSettings	source/android/vendor/autochips/proprietary/packages/apps/ATCSettings source/android/vendor/autochips/proprietary/packages/apps/AtcCarSettings source/android/frameworks/packages/SettingsLib	ATC 提供的 Wi-Fi demo APP

文件名	文件路径	文件描述
framework-wifi.jar	source/android/packages/modules/Wifi/framework	Wi-Fi 的源生 java Framework 代码，供上层 APP 调用。
autochips.jar	vendor/autochips/proprietary/frameworks/base	ATC Wi-Fi 新增 java Framework 代码，供上层 APP 调用。
ConnectivityManager.java	source/android/packages/modules/Connectivity/frameworks/ src/android/net/ConnectivityManager.java	Connectivity 的统管接口，包括 Wi-Fi AP 的管理接口，供上层 APP 调用。
service-wifi.jar	source/android/packages/modules/Wifi/service	wifiservice，Wi-Fi 控制逻辑的集中处理中心。它实现了 Wi-Fi framework 接口所提供的功能。
libwifi-hal.so	source/android/frameworks/opt/net/wifi/libwifi_hal	加载 wifi 芯片对应的 wifihal，完成 insmod driver，网卡 up 等工作。
atc-wifi-service.jar	source/android/vendor/autochips/proprietary/frameworks/opt/net/wifi	atc-wifiservice，ATC 客制化功能的逻辑控制中心，实现 ATC Framework 接口的功能。
ServiceWifiResources.apk	source/android/packages/modules/Wifi/service/ServiceWifiResources	Android 原生的资源文件，在这里 Android 会给出需要配置项，如是否支持 5G AP 等
NetworkStackResOverlay.apk ServiceConnectivity	source/android/vendor/autochips/proprietary/packages/overlay/vendor	ATC 客制化资源文件，在这里配

文件名	文件路径	文件描述
ResourceOverlay.apk TetheringResOverlay.apk WifiResOverlay.apk		置的属性，会覆盖掉 Android 原生的同名配置项。
wificond	source/android/system/connectivity/wificond	Wificond 进程，用来实现 Station 的搜索，Softap 连接状态的上报等。
supplicant/hostapd	source/android/external/wpa_supplicant_8	wpa_supplicant 进程 hostapd 进程，分别用来配置 Station network 和 AP network。
wifi_vendor_hidl	source/android/hardware/interface/wifi/1.5	wifi_vendor_hidl 进程，用来加载 wifihal，获取 wifi 芯片特性，装载 KO，up 网卡等。
WifiBoardConfig.mk Wifidevice.mk	source/android/vendor/autochips/proprietary/hardware/connectivity/wlan/common	Wi-Fi 编译配置信息，芯片配置信息等

19.2 软件设计

19.2.1 架构图

Wi-Fi 架构如图 19-1 所示。

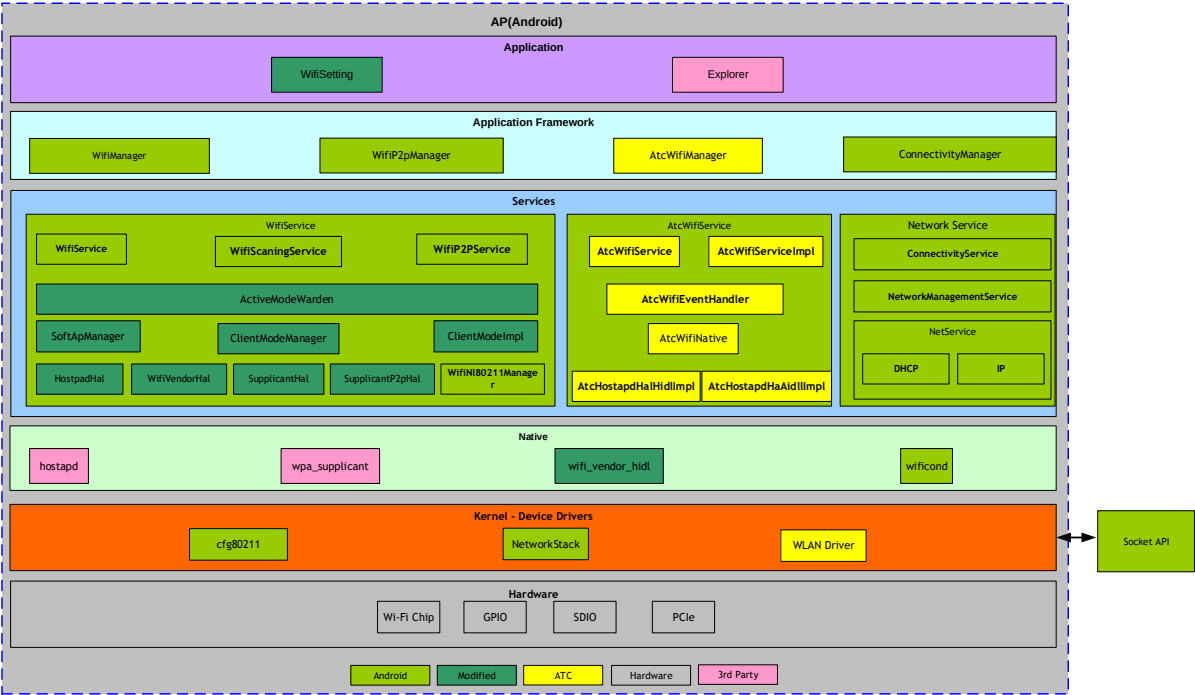


图 19-1 Wi-Fi 架构图

ATC Android 相较于 Android 原生架构来说增加了 AtcWifiManager 接口，其被 AtcWifiService 所实现，用来实现客制化需求，比如 AP 主动断开 Station 的连接，获取 AP 所使用的信道等。

由于 Wi-Fi 功能总体上分为 3 大类，因此子模块分别分成以下 3 大类分别描述：

- Wi-Fi Station
- Wi-Fi AP
- Wi-Fi P2P

19.2.1.1 Wi-Fi Station

Wi-Fi Station 整体框架图如下：

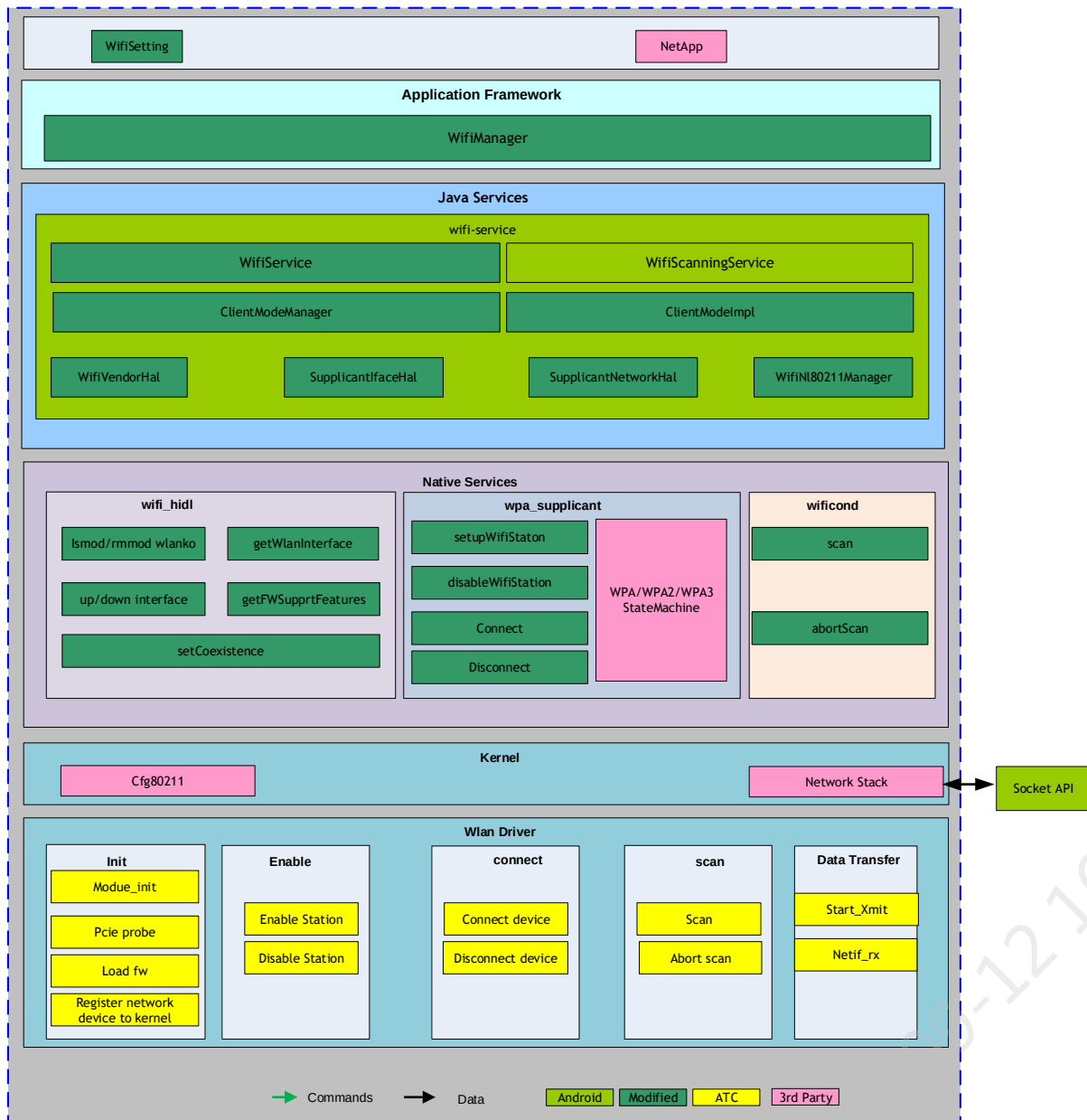


图 19-2 Wi-Fi Station 架构图

1: Android Framework 接口

Android Framework 提供了操作 Wi-Fi 的总接口，比如打开/关闭 Wi-Fi Station，搜索设备等。APP 可以通过 **WifiManager** 对象调用这些接口，该对象通过 AIDL 与 **wifi-service** 进行通信。

2: wifi-service

该服务是一个 Java Service，运行在 **system_server** 进程中。各模块分别描述如下：

- **WifiService** 用来实现 **WifiManager** 对应的接口。
- **WifiScanningService** 用来处理 **Station** 的搜索逻辑，搜索相关的状态，会通过广播通知 APP。

- ClientModeManager 用来处理 Station 的开关逻辑，开关相关的状态，会通过广播通知 APP。
- ClientModeImpl 是用来处理 Station 的连接逻辑，连接相关的状态，会通过广播通知 APP。
- WifiVendorHal 是 wifi_hidl 进程在 wifi-service 中的一个代理，它通过 HIDL 调用到 vendor_hidl 进程，让其装载/卸载 KO，UP/Down 网卡等。
- SupplicantIfaceHal 是 wpa_supplicant 进程在 wifi-service 中的代理，它通过 HIDL 启动 wpa_supplicant 进程，同时也会通过 HIDL 调用 wpa_supplicant，让其配置 Wi-Fi Station，打开/关闭 Station。
- SupplicantIfaceHal 也是 wpa_supplicant 进程在 wifi-service 中的代理，它会通过 HIDL 调用 wpa_supplicant，连接/断开设备等。
- WifiNl80211Manager 是 wificond 进程在 wifiservice 中的代理，它通过 AIDL 调用到 wificond，让其进行设备搜索。

3: Native Services

(1) wifi_hidl 进程

主要用来进行 wifi 打开前的 init 动作。各模块分别描述如下：

- insmod/rmmod wlan ko，即通过系统调用装载/卸载 wlan driver
- getWlanInterface，获取 wifi driver 的 interface
- up/down interface，up/down 网卡
- getFWSupportFeatures，获取芯片支持的特性
- setCoexistence，设置共存模式，即 AP/STA/P2P 如何共存。

该进程会通过 nl80211/cfg80211 以及 ioctl 与 wlan driver 进行通信。

(2) wpa_supplicant 进程

用来配置 Wi-Fi 参数并打开 Station。各模块分别描述如下：

- SetupWifiStation，配置 Station 参数并打开 Station，比如 Station 是否支持 WPA3，Station 要搜索哪些信道等。开关的状态，会通过 HIDL callback 通知给 wifi-service。
- DisableWifiStation，关闭 Wi-Fi Station。开关的状态，会通过 HIDL callback 通知给 wifi-service
- Connect，配置连接参数，连接对端设备。连接的状态，会通过 HIDL callback 通知给 wifi-service。
- Disconnect，断开对端设备。
- WPA/WPA2/WPA3 StateMachine，加密认证相关的状态机，用来处理认证流程。

该进程会通过 nl80211/cfg80211 以及 ioctl 与 wlan driver 进行通信

(3)wificond 流程

用来处理 Station 的搜索流程，各模块分别描述如下：

- Scan，开始搜索，搜索的状态，会通过 AIDL callback 通知给 wifi-service，wifi-service 拿到状态后，决定是否主动从 wificond 获取搜索结果。
- AbortScan，停止搜索。

该进程会通过 nl80211/cfg80211 以及 ioctl 与 wlan driver 进行通信。

4: Kernel

(1)Cfg80211 模块

对接到 wlan driver，且配合 nl80211，让用户态的 wpa_supplicant 等 Native 进程得以访问内核态。

(2)NetworkStack

网络协议栈，wlan driver 从 FW 取到的数据会送给网络协议栈进一步解析分发，网络协议栈的数据也会交给 Wlan driver 并最终送给 FW。同时，应用程序直接通过 Socket 套接字可以向网络协议栈收发数据。

5: Wlan driver

处理上层应用程序下发的命令，包括搜索，连接等，同时负责 FW 和网络协议栈之间数据的转发，wlan driver 和上述的几个 Native 进程之间的通讯，均采用 cfg80211 和 ioctl。

模块可以细分为如下几个：

(1) Init 模块

负责驱动的初始化，包括 Bus probe，load FW 等，具体描述如下：

- Module_init，驱动的入口。驱动各模块初始化，GPIO 使能，触发 device 和 driver 的匹配等。
- Pcie_probe，device 与 driver 匹配成功，开始在设备树中获取资源
- Load FW，加载固件
- Register network device to kernel，向内核注册网络设备

(2) Enable 模块

负责 Wi-Fi 的开关，具体描述如下：

- Enable Station，根据 wpa_supplicant 配置的参数，打开 wifi station
- Disable Station，关闭 wifi station

(3) Connect 模块

- Connect device，根据 wpa_supplicant 配置的连接参数，与对算设备协商 Assoc，确定 Association ID，信道带宽等，最终连接对端设备。

- Disconnect device, 断开对端设备

(4) Scan 模块

- Scan, 搜索设备, 搜索的状态, 会先返回给 wificond, wificond 再来决定是否通过 nl80211 来取搜索结果。
- Abort scan, 停止搜索

(5) Data Transfer 模块

- Start_Xmit, 从网络协议栈接收数据并发送给 FW
- Netif_rx, 从 FW 获取数据并将其送给网络协议栈

19.2.1.2 Wi-Fi AP

Wi-Fi AP 整体框架图如下:

航盛2024-09-12 16:50:36

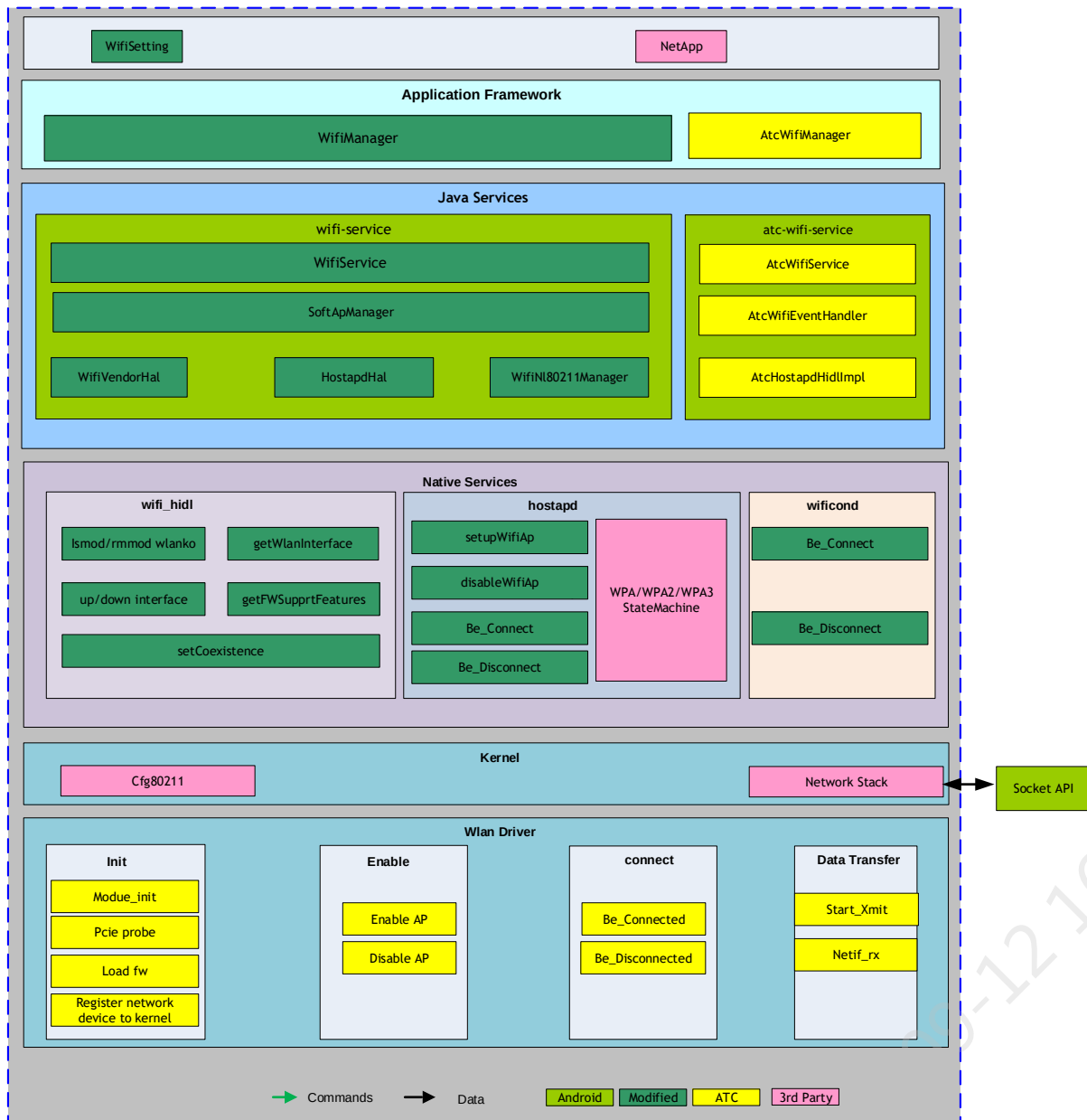


图 19-3 Wi-Fi AP 架构图

1: Android Framework 接口

Android Framework 提供了操作 Wi-Fi 的总接口，比如打开/关闭 Wi-Fi AP 等。APP 可以通过 **WifiManager** 对象调用这些接口，该对象通过 AIDL 与 **wifiservice** 进行通信。

同时，为了扩展 Wi-Fi AP 功能，我们新增了一个 **AtcWifiManager** 的接口，提供了如下功能：

- AP 主动断开 Station 的连接
- 获取 AP 当前已连接设备的信息
- 获取 AP 的 channel

- 获取 AP 的网卡名
- 获取 AP 被断开的 reasonCode
- 获取 AP 的 BSSID

AtcWifiManager 通过 AIDL 接口与 AtcWifiService 通信。

2: wifi-service

该服务是一个 Java Service，运行在 system_server 进程中。

- WifiService 用来实现 wifimanager 对应的接口。
- SoftApManager 用来处理 AP 的开关逻辑，开关相关的状态，会通过广播通知 APP
同时 SoftApManager 也用来处理 AP 的连接逻辑，连接相关的状态，会通过广播通知 APP。
- WifiVendorHal 是 wifi_hidl 进程在 wifiservice 中的一个代理，它通过 HIDL 调用到 vendor_hidl 进程，让其装载/卸载 KO，UP/Down 网卡等。
- Hostapdhal 是 hostapd 进程在 wifiservice 中的代理，它通过 HIDL 启动 hostapd 进程，同时也会通过 HIDL 调用 hostapd，让其配置 Wi-Fi AP，打开/关闭 AP。
- WifiNl80211Manager 是 wificond 进程在 wifiservice 中的代理，它通过 AIDL 调用到 wificond，并注册 callback，以便接收 wificond 上报的连接状态信息。

3: atc-wifi-service

该服务是一个 Java Service，同样运行在 system_server 进程中。

- AtcWifiService 用来实现 AtcWifiManager 对应的接口。
- AtcWifiEventHandler 用来实现如下逻辑：AP 随系统自动打开，获取 AP 使用的 channel，发送连接断开的 reasonCode（通过广播），获取网卡名，获取 BSSID 等。
- AtcHostapdHidlImpl 是 hostapd 进程在 atc-wifi-service 中的代理，它通过 HIDL 调用到 hostapd，并注册 callback，以便接收 hostapd 上报的连接状态信息（包含 reasonCode）。

3: Native Services

(1)wifi_hidl 进程

这部分与 Station 基本一致，不再赘述。

(2)hostapd 进程

用来配置 Wi-Fi AP 参数并打开 AP。具体如下：

- SetupWifiAp，配置 AP 参数并打开 AP，比如 AP 是否支持 WPA3，AP 使用的信道，带宽等。
- DisableWifiAP，关闭 Wi-Fi AP
- Be_Connected，被对端设备连接

- Be_Disconnected, 被对端设备断开
- WPA/WPA2/WPA3 StateMachine, 加密认证相关的状态机, 用来处理认证流程。

该进程会通过 nl80211/cfg80211 以及 ioctl 与 wlan driver 进行通信。

(3)wificond 流程

用来处理 AP 的连接断开消息。

4: Kernel

这部分与 Station 并无区别, 不再赘述。

5: Wlan driver

处理上层应用程序下发的命令, 同时负责 FW 和网络协议栈之间数据的转发。

wlan driver 和上述的几个 Native 进程之间的通讯, 均采用 cfg80211 和 ioctl。具体内容如下:

(1) Init 模块

这部分与 Station 并无区别, 不再赘述。

(2) Enable 模块

负责 Wi-Fi AP 的开关

- Enable AP, 根据 hostapd 配置的参数, 打开 Wi-Fi AP
- Disable AP, 关闭 Wi-Fi AP

(3) Connect 模块

- Be_Connected, 根据 hostapd 配置的参数, 与对端设备协商 Assoc, 确定 Association ID, 信道带宽等, 最终完成连接。
- Be_Disconnected, 被对端设备断开。

(4) Data Transfer 模块

与 Station 无区别, 不再赘述。

19.2.1.3 Wi-Fi P2P

Wi-Fi P2P 整体框架图如下:

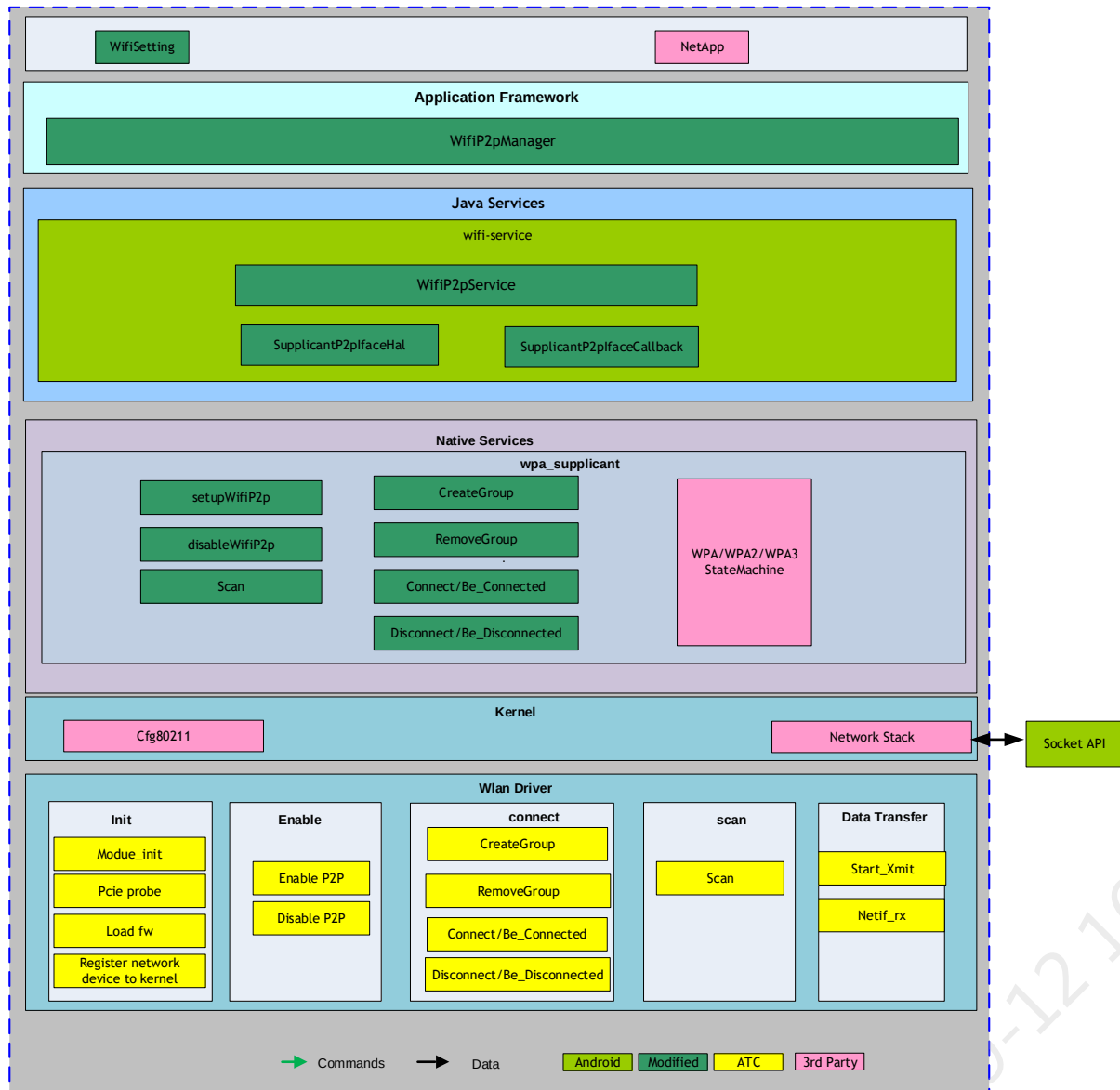


图 19-4 Wi-Fi 架构图

1: Android Framework 接口

Android Framework 提供了操作 Wi-Fi P2P 的总接口，比如打开/关闭 Wi-Fi P2P，搜索设备，创建 Group 等。APP 可以通过 `WifiP2pManager` 对象调用这些接口，该对象通过 AIDL 与 `wifi-service` 进行通信。

2: wifi-service

该服务是一个 Java Service，跑在 `system_server` 进程中。

- `WifiP2pService` 用来实现 `WifiP2pManager` 对应的接口。

- SupplicantP2pIfaceHal 是 wpa_supplicant 进程在 wifi-service 中的代理，它通过 HIDL 调用 wpa_supplicant，让其配置 Wi-Fi P2P，打开/关闭 P2P，连接/断开设备等。

3: Native Service

(1)wpa_supplicant 进程

用来配置 Wi-Fi 参数并打开 P2P。具体如下：

- SetupWifiP2P，配置 P2P 参数(比如 P2P 的 intent，P2P 的 operating class 等)并打开 P2P。
- DisableWifiP2p，关闭 Wi-Fi P2p。
- Scan，P2P 搜索，其中会包含 listen 和 find 的阶段。
- CreateGroup，创建 P2P 群组
- RemoveGroup，移除 P2P 群组
- Connect/Be_Connected，连接对端设备/被对端设备连接，这部分与 station 和 AP 差异不大。
- Disconnect/Be_Disconnected，断开对端设备/被对端设备主动断开，这部分与 station 和 AP 差异不大。
- WPA/WPA2/WPA3 StateMachine，加密认证相关的状态机，用来处理认证流程，这部分与 station 和 AP 差异不大。

该进程会通过 nl80211/cfg80211 以及 ioctl 与 wlan driver 进行通信。

4: Kernel

与 Station 无差异，不再赘述。

5: Wlan driver

处理上层应用程序下发的命令，包括搜索，连接等，同时负责 FW 和网络协议栈之间数据的转发。具体内容如下：

(1) Enable 模块

负责 Wi-Fi P2P 的开关

- Enable P2P，使用 wpa_supplicant 下发的参数，打开 Wi-Fi P2P
- Disable P2P，关闭 Wi-Fi P2P

(2) Connect 模块

- CreateGroup，根据 wpa_supplicant 参数，创建 P2P 群组
- RemoveGroup，移除 P2P 群组
- Connect/Be_Connected，连接对端设备/被对端设备连接，这部分与 station 和 AP 差异不大。

- Disconnect/Be_Disconnected，断开对端设备/被对端设备主动断开，这部分与 station 和 AP 差异不大。

(3) Scan 模块

Scan，搜索设备。

(4) Data Transfer 模块

与 Station 无差异，不再赘述。

19.2.2 流程图

Wi-Fi 整体的数据流向见下图：

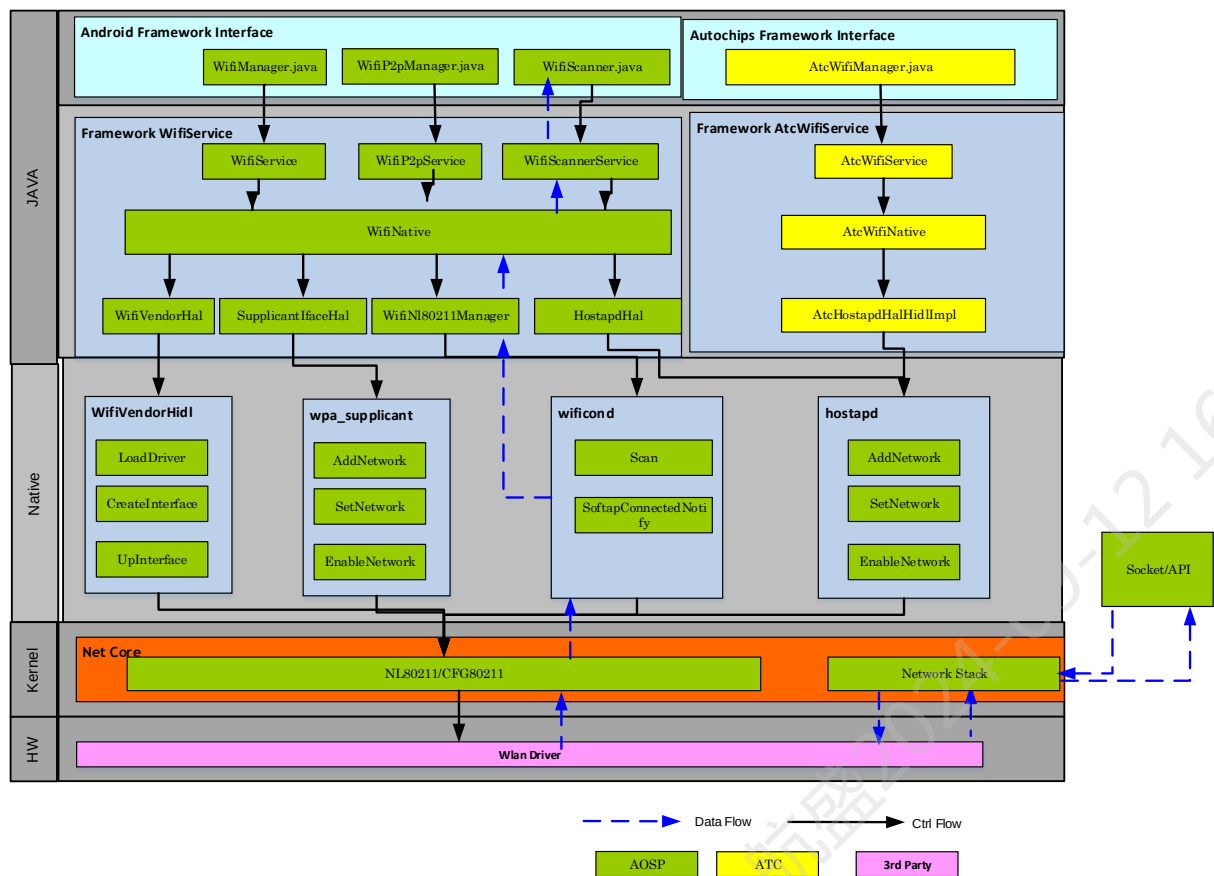


图 19-5 Wi-Fi 整体流程图

19.2.2.1 Wi-Fi Station

19.2.2.1.1 Wi-Fi Enable Flow

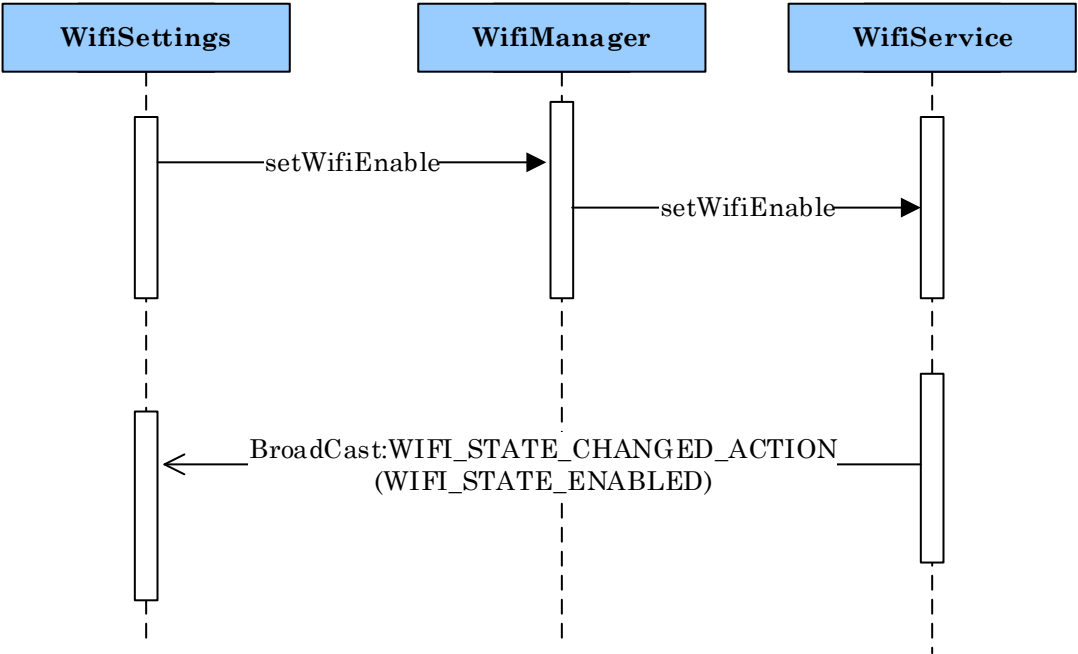


图 19-6 Wi-Fi Enable Flow

19.2.2.1.2 Wi-Fi Scan Flow

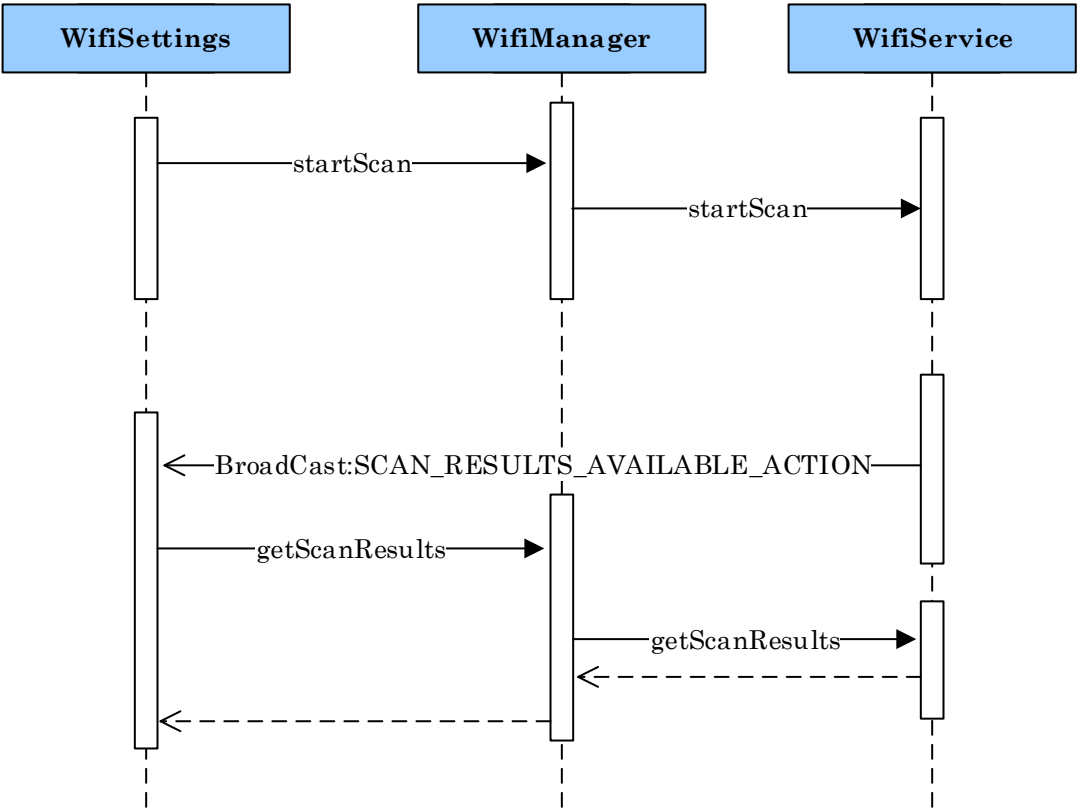


图 19-7 Wi-Fi Scan Flow

19.2.2.1.3 Wi-Fi Connect Flow

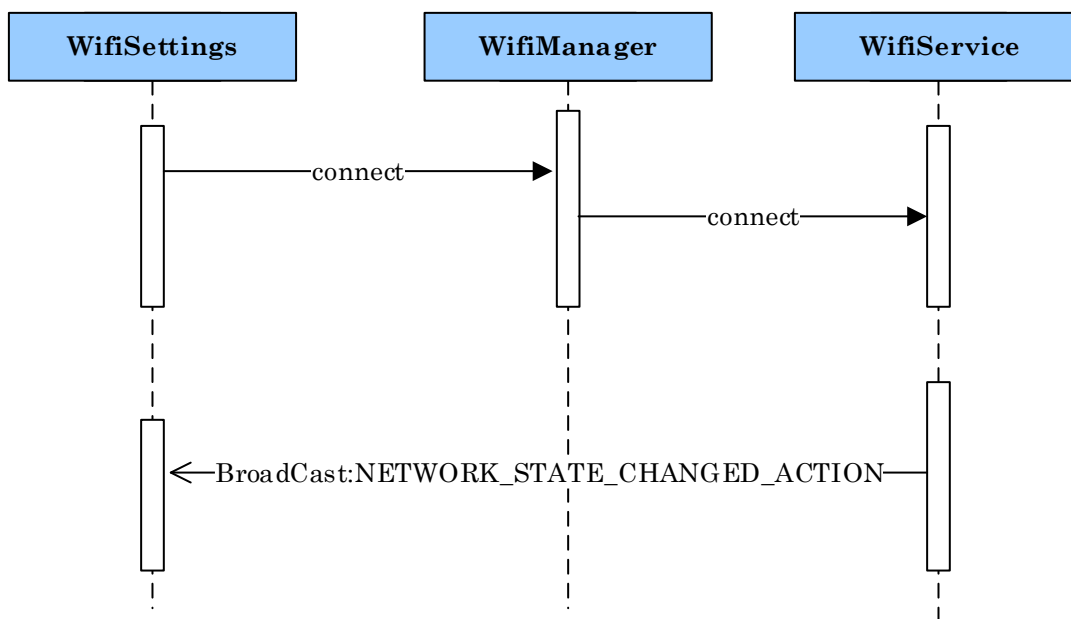


图 19-8 Wi-Fi Connect Flow

19.2.2.1.4 Wi-Fi Disconnect

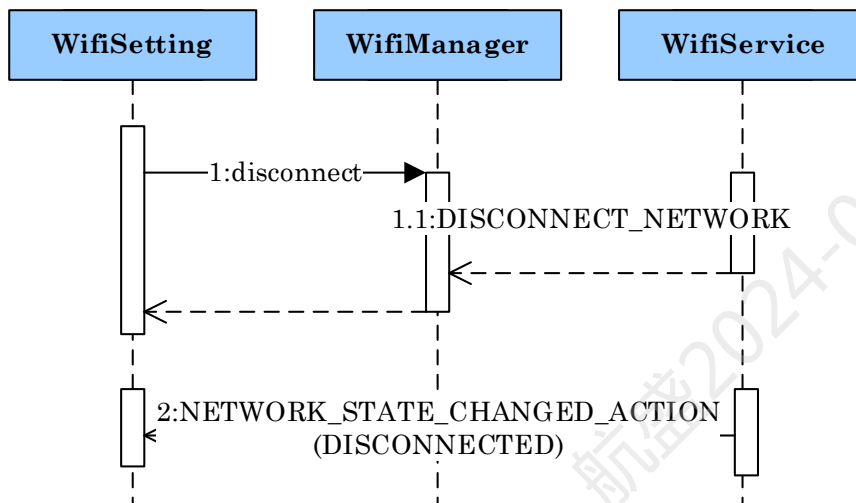


图 19-9 Wi-Fi Disconnect Flow

19.2.2.2 Wi-Fi AP

19.2.2.2.1 Wi-Fi AP Enable

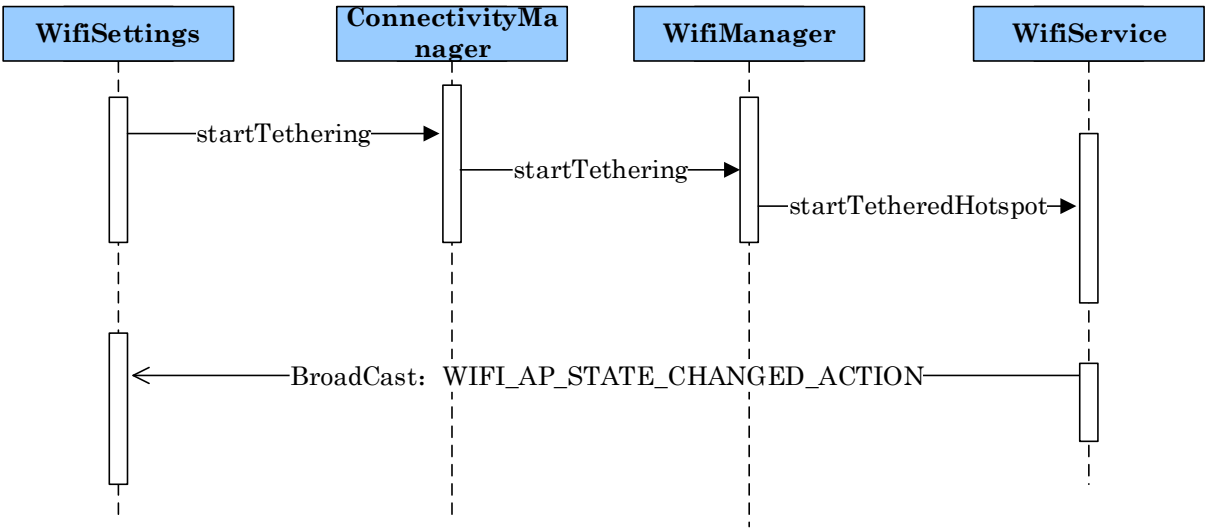


图 19-10 AP Enable Flow

19.2.2.2.2 Wi-Fi AP Connect

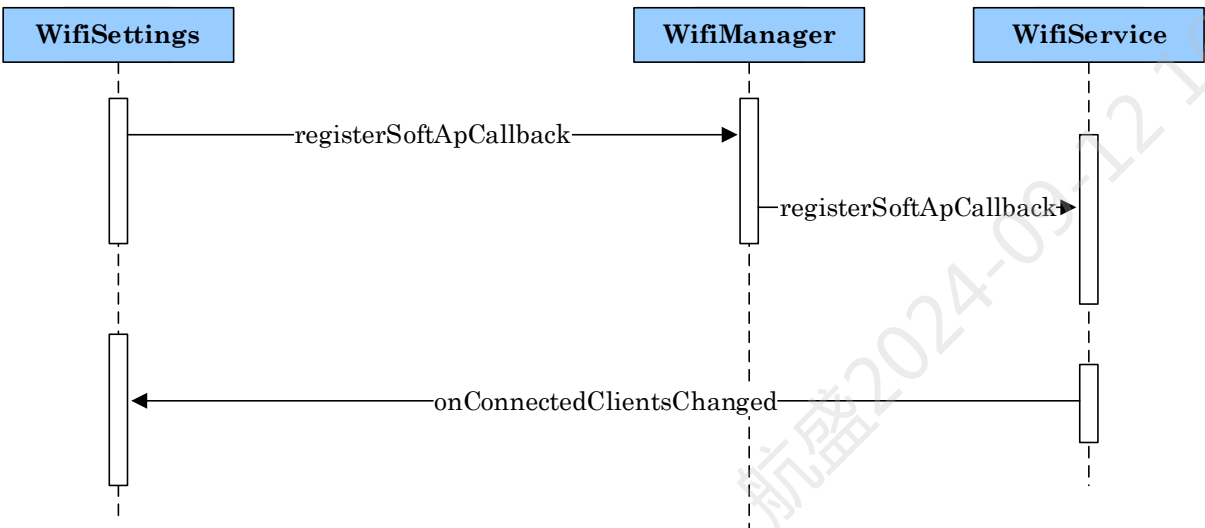


图 19-11 AP connected Flow

19.2.2.3 Wi-Fi P2P

19.2.2.3.1 Wi-Fi P2P Enable

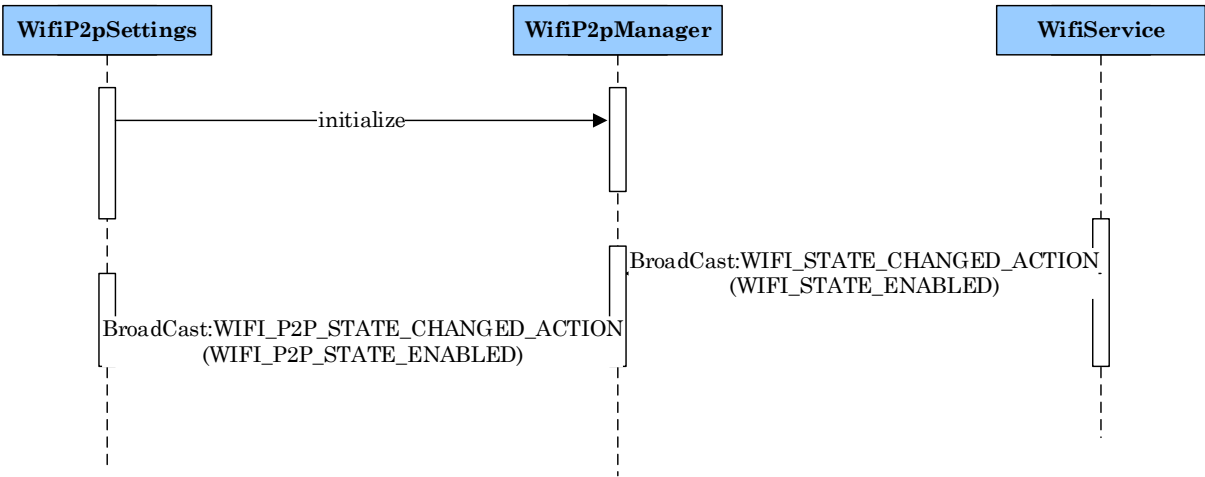


图 19-12 Enable P2P Flow

19.2.2.3.2 Wi-Fi P2p Discover

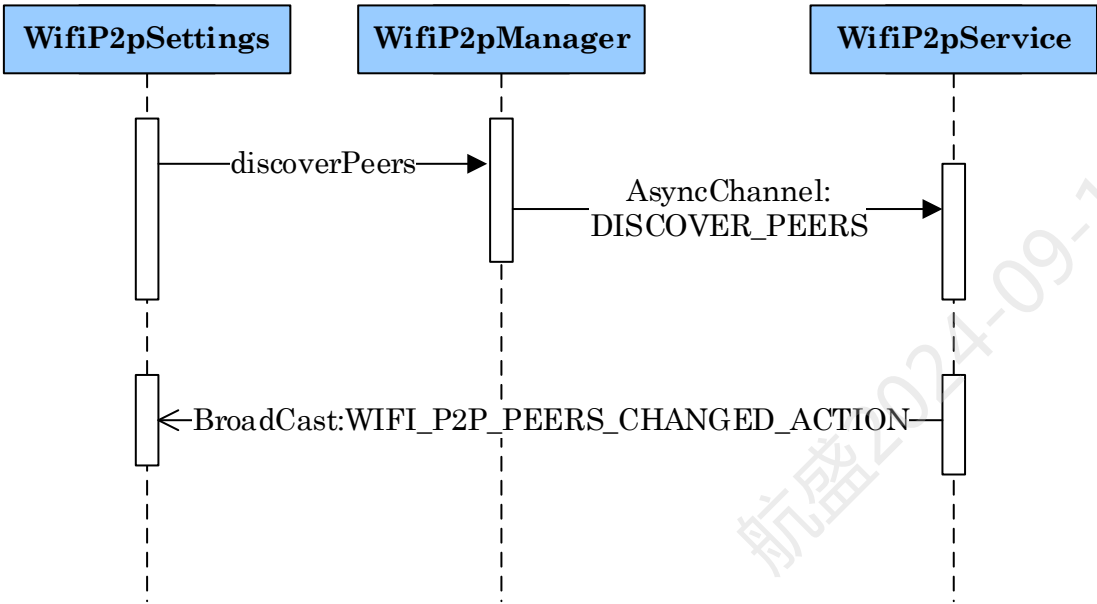


图 19-13 P2P Discovery Flow

19.2.2.3.3 Wi-Fi P2P Connect

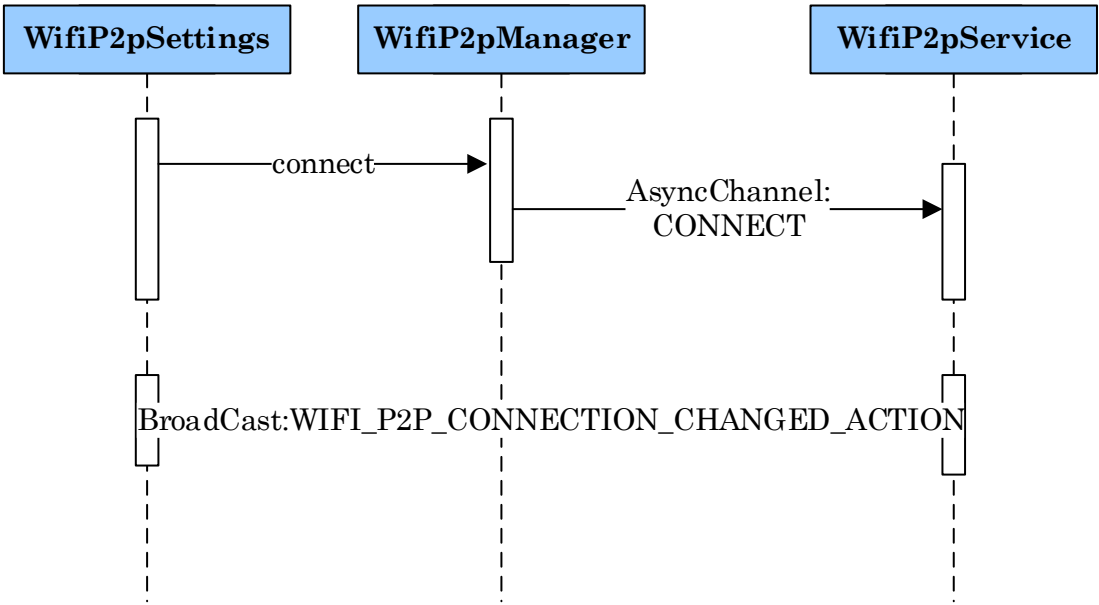


图 19-14 P2P Connect Flow

19.2.2.4 Wi-Fi MAC

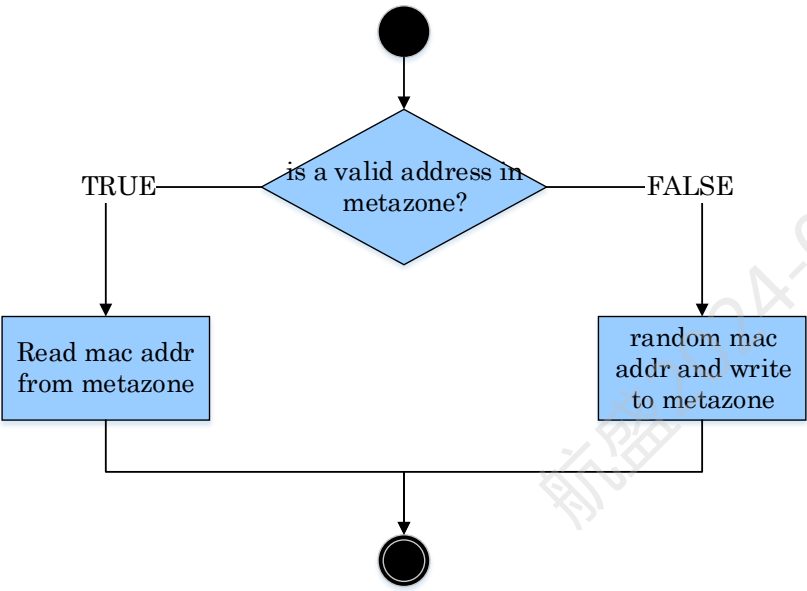


图 19-15 Wi-Fi MAC Address handle in WLAN Driver

Wi-Fi MAC Address WLAN Driver 的处理算法如上图所示：

- 1. WLAN Driver 先从 metazone 读取是否有有效地址；

2. 如果 metazone 有有效地址，则使用 metazone 的地址；
3. 如果 metazone 无有效地址，则随机一个 mac addr，然后写入到 metazone，并使用此地址。

19.3 接口描述

19.3.1 原生接口

1. 原生接口请参考安卓开发者官网

<https://developer.android.com/reference/android/net/wifi/package-summary>

<https://developer.android.com/reference/android/net/package-summary>

2. 原生 HIDL 接口可参考 Google open source

<https://android.googlesource.com/platform/frameworks/base>

3. 代码目录：

source/android/packages/modules/Wifi/framework/

19.4 使用说明和注意事项

19.4.1 如何使用 Wi-Fi

19.4.1.1 构建 Station 功能

1. 获取 WifiManger 和 ConnectivityManger 的接口

```
// 获取 ConnectivityManager 对象
mCm = (ConnectivityManager)
mContext.getSystemService(Context.CONNECTIVITY_SERVICE);

// 获取 WifiManager 的接口对象
mWifiManager = (WifiManager) context.getSystemService(Context.WIFI_SERVICE);
```

2. 通过 WifiManager 提供的接口完成 Wifi 的打开/关闭，连接/断开，搜索等功能

```
// 打开 Wi-Fi station
mWifiManager.setWifiEnabled(true);

// 搜索设备
mWifiManager.startScan();
```

```
// 连接设备
```

```
mWifiManager.connect(config, wifiManagerListener);
```

3. 注册广播的监听

```
//添加需要关注的 intent 消息
```

```
IntentFilter filter = new IntentFilter();
```

```
// Station 开关状态监测
```

```
filter.addAction(WifiManager.WIFI_STATE_CHANGED_ACTION);
```

```
// 搜索状态监测
```

```
filter.addAction(WifiManager.SCAN_RESULTS_AVAILABLE_ACTION);
```

```
// 网络配置状态监测
```

```
filter.addAction(WifiManager.CONFIGURED_NETWORKS_CHANGED_ACTION);
```

```
// Wi-Fi 连接状态监测
```

```
filter.addAction(WifiManager.NETWORK_STATE_CHANGED_ACTION);
```

```
// 信号强度状态监测
```

```
filter.addAction(WifiManager.RSSI_CHANGED_ACTION);
```

```
mContext.registerReceiver(mBroadcastReceiver, filter);
```

4. 接收广播后进行处理

```
private final BroadcastReceiver mReceiver = new BroadcastReceiver() {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        String action = intent.getAction();  
        if (WifiManager.WIFI_STATE_CHANGED_ACTION.equals(action)) {  
            // Wi-Fi 开关状态变化了  
        } else if (WifiManager.SCAN_RESULTS_AVAILABLE_ACTION.equals(action)) {  
            // Wi-Fi 搜索状态变化了，可以去向服务获取搜索结果了  
        } else if  
(WifiManager.CONFIGURED_NETWORKS_CHANGED_ACTION.equals(action)) {  
            // 网络配置发生了变化了  
        } else if (WifiManager.NETWORK_STATE_CHANGED_ACTION.equals(action)) {  
            // 网络连接状态变化了  
        }  
    }  
};
```

```
        } else if (WifiManager.RSSI_CHANGED_ACTION.equals(action)) {  
            // 信号强度变化了  
        }  
    }  
};
```

19.4.1.2 构建 AP 功能

1. 获取 WifiManger 和 ConnectivityManger 的接口

```
// 获取 ConnectivityManager 对象  
mCm = (ConnectivityManager)  
mContext.getSystemService(Context.CONNECTIVITY_SERVICE);  
  
// 获取 WifiManager 对象  
mWifiManager = (WifiManager) context.getSystemService(Context.WIFI_SERVICE);
```

2. 通过 WifiManager 提供的接口完成 AP 的打开/关闭, 注册 softApCallback, 设置 AP 配置等功能

```
// 打开 Wi-Fi AP  
mWifiManager.startTetheredHotspot(softApConfig);  
  
// 注册回调  
mWifiManager.registerSoftApCallback(callback);  
  
// 设置 AP 配置, 比如 AP 的名称, 密码等内容  
mWifiManager.setSoftApConfiguration(softApConfig);
```

3. 注册广播的监听

```
//添加需要关注的 intent 消息  
IntentFilter filter = new IntentFilter();  
  
// AP 开关状态监测  
filter.addAction(WifiManager.WIFI_AP_STATE_CHANGED_ACTION);  
  
// Tethering 状态监测  
filter.addAction(ConnectivityManager.ACTION_TETHER_STATE_CHANGED);
```

4. 接收广播后进行处理

```
private final BroadcastReceiver mReceiver = new BroadcastReceiver() {  
    @Override
```



```

public void onReceive(Context content, Intent intent) {
    String action = intent.getAction();
    if (action.equals(ACTION_TETHER_STATE_CHANGED)) {
        // Tethering 状态发生了变化
    } else if (action.equals(WIFI_AP_STATE_CHANGED_ACTION)) {
        // AP 开关状态发生了变化
    }
}
};

```

5. 注册 softAPCallback

```

mWifiManager.registerSoftApCallback(new HandlerExecutor(mHandler), mSoftApCallback)
mApCallback = new WifiApStateCallback();
public class WifiApStateCallback implements WifiManager.SoftApCallback {
    default void onStateChanged(@WifiApState int state, @SapStartFailure int
failureReason) {
        // AP 状态发生变化
    }
    default void onConnectedClientsChanged(@NonNull SoftApInfo softApInfo ,
@NonNull List<WifiClient> clients) {
        // 当前连接至 AP 的所有 Station 的信息
    }
    default void onInfoChanged(@NonNull List<SoftApInfo> softApInfoList) {
    }
    default void onCapabilityChanged(@NonNull SoftApCapability
softApCapability) {
    }
    default void onBlockedClientConnecting(@NonNull WifiClient client,
@sapClientBlockedReason int blockedReason) {
    }
}
}

```

19.4.1.3 构建 P2P 功能

1. 获取 WifiP2pManager 的接口

```
// 获取 WifiP2pManager 的对象  
mWifiP2pManager =  
(WifiP2pManager)mContext.getSystemService(Context.WIFI_P2P_SERVICE);
```

2. 通过 WifiP2pManager 提供的接口完成 P2P 的搜索，连接等功能

```
// 向 Wi-Fi framework 注册应用程序。在执行任何 P2P 操作之前，必须首先调用此函数  
WifiP2pManager.Channel mChannel = mWifiP2pManager.initialize(context, looper,  
listener);  
  
// 搜索  
mWifiP2pManager.discoverPeers(mChannel, listener);  
  
// 连接  
mWifiP2pManager.connect(mChannel, mConfig, listener);
```

3. 注册广播的监听

```
//添加需要关注的 intent 消息  
IntentFilter filter = new IntentFilter();  
  
// P2P 的开关状态监测  
filter.addAction(WifiP2pManager.WIFI_P2P_STATE_CHANGED_ACTION);  
  
// P2P 连接状态监测  
filter.addAction(WifiP2pManager.WIFI_P2P_CONNECTION_CHANGED_ACTION);  
  
// P2P 搜索状态发生变化  
filter.addAction(WifiP2pManager.WIFI_P2P_DISCOVERY_CHANGED_ACTION);  
  
// P2P 连接设备列表  
filter.addAction(WifiP2pManager.WIFI_P2P_PEERS_CHANGED_ACTION);  
mContext.registerReceiver(mBroadcastReceiver, filter);
```

4. 接收广播后进行处理

```
private final BroadcastReceiver mReceiver = new BroadcastReceiver() {  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        String action = intent.getAction();
```

```
        if (WifiP2pManager.WIFI_P2P_STATE_CHANGED_ACTION.equals(action)) {  
            // P2P 开关状态  
        } else if (WifiP2pManager.WIFI_P2P_PEERS_CHANGED_ACTION.equals(action))  
{  
            // P2P 连接设备的列表  
        } else if  
(WifiP2pManager.WIFI_P2P_CONNECTION_CHANGED_ACTION.equals(action)) {  
            // P2P 连接状态的变化  
        } else if  
(WifiP2pManager.WIFI_P2P_DISCOVERY_CHANGED_ACTION.equals(action)) {  
            // P2P 搜索状态的变化  
        }  
    }  
};
```

19.4.2 Wi-Fi 芯片的配置规则

19.4.2.1 ATC Wi-Fi 的配置框架

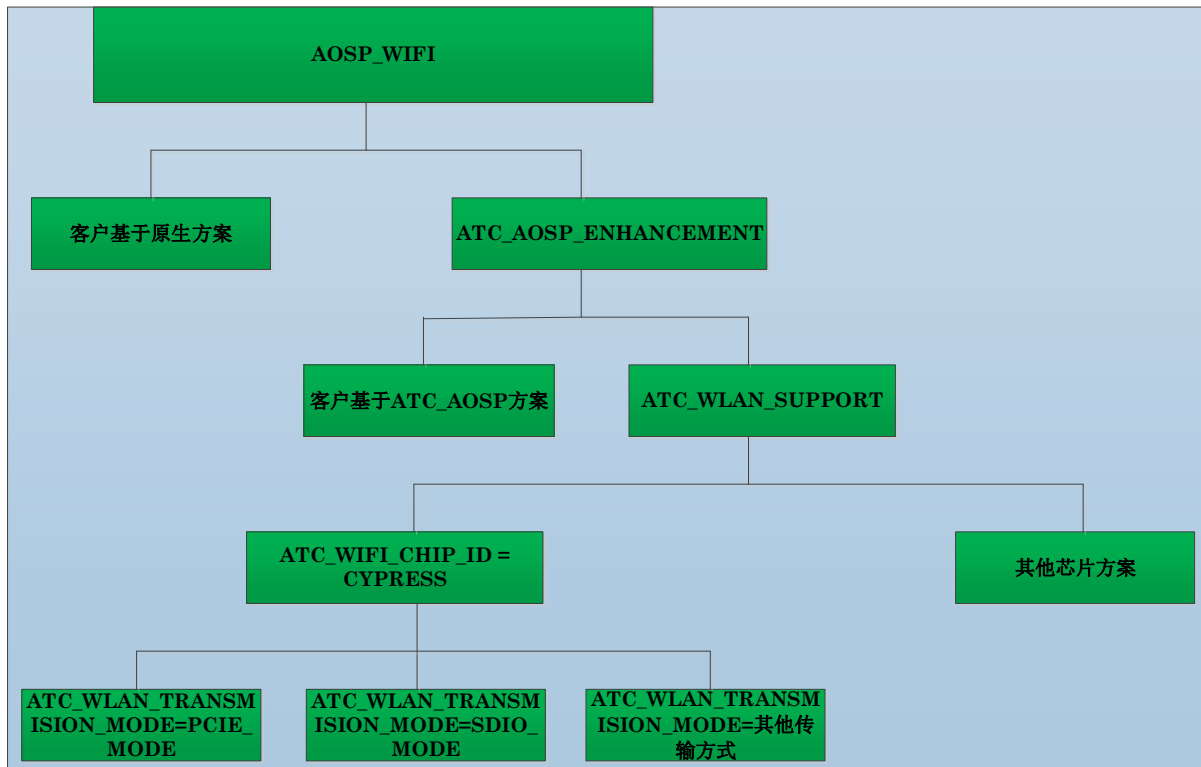


图 19-16 ATC Wi-Fi 配置框架

ATC Wi-Fi 的配置规则如上图所示，其中：

(1) ATC_AOSP_ENHANCEMENT 代表 ATC 系统提供的总开关

在 Wi-Fi 相关的 Native 代码中，有宏 ATC_AOSP_ENHANCEMENT 与之对应，受其控制。在 Wi-Fi 相关的 Java 层代码中，有系统属性 ro.atc.aosp_enhancement 与之对应，受其控制。

(2) ATC_WLAN_SUPPORT 代表 ATC 方案的 Wi-Fi

在 Wi-Fi 相关的 Native 代码中，有宏 ATC_WLAN_SUPPORT 与之对应，受其控制。在 Wi-Fi 相关的 Java 层代码中，有系统属性 persist.atc.wifi.wlansupport 与之对应，受其控制。

(3) ATC_WIFI_CHIP 代表 ATC 方案的 Wi-Fi 使用的 Wi-Fi 芯片是什么，目前只有 CYPRESS

在 Wi-Fi 相关的 Native 代码中，根据 chip 的信息，目前 AC8025 上只使用了 cypress 芯片，定义了宏 ATC_AOSP_ENHANCEMENT_CYPRESS。在 Wi-Fi 相关的 Java 代码中，定义了系统属性 persist.atc.wifi.chip，受其控制。

(4) ATC_WLAN_TRANSMISSION_MODE 代表 CYPRESS 方案的通讯方式，目前包括 PCIE_MODE 和 SDIO_MODE

目前，Wi-Fi Java 和 Native 层代码，无需关心 PCIE 和 SDIO，故无需做特殊的逻辑处理。

19.4.2.2 客户在 ATC 代码基础上，如何适配其他方案的芯片

19.4.2.2.1 客户基于 ATC_WLAN_SUPPORT 适配新的方案

1. 客户只需添加新的 ATC_WIFI_CHIP_ID，并适配新的 driver 即可。
2. ATC WifiService 和 Wi-Fi HIDL 中所有的补丁、配置、新增功能均可生效，理论上，WifiService 和 Wi-Fi HIDL 的代码无需再做修改。

19.4.2.2.2 客户基于 ATC_AOSP_ENHANCEMENT 适配新的方案

1. 若客户基于 ATC_AOSP_ENHANCEMENT 适配方案，需要把 ATC_WLAN_SUPPORT 关掉。
2. 这样，ATC 适配的 Wi-Fi 功能便不可用了（比如 hostapd 都不会再参与整编了）。同时，ATC 在 AOSP 基础上新增的功能，也不再可用了，如 AP 为了支持 carplay，添加的一些逻辑。
3. 但 WifiService 和 Wi-Fi HIDL 中，ATC 修复的原生 bug，比如空指针，多线程安全等问题，依旧是生效的。

19.4.2.2.3 客户基于 AOSP 适配新的方案

1. 若客户基于 AOSP 适配新的方案，需要把 ATC_AOSP_ENHANCEMENT 关掉。
2. 这样，ATC 在 AOSP 上所有的改动，便不再生效了，包括解决的一些空指针和多线程 Bug，也不再生效了。

19.4.3 Wi-Fi 配置

【步骤 1】在 ac8x_demo 中，配置 ATC_WLAN_SUPPORT

代码路径: source/android/device/autochips/ac8x_demo/ProjectConfig.mk

```
#ATC WIFI config
ifeq ($(strip $(ATC_AOSP_ENHANCEMENT)), yes)
ATC_WLAN_SUPPORT = yes
endif
```

【步骤 2】在 ac8x_demo 中，配置芯片定义

代码路径: source/android/device/autochips/ac8x_demo/ProjectConfig.mk

```
# WIFI config begin
```

```
# Note: CYPRESS_CONNECTIVITY_PCIE must after include ProjectConfigGen.mk
ifeq ($(strip $(ATC_WLAN_SUPPORT)), yes)
ifeq ($(filter yes, \
    $(strip $(CYPRESS_CONNECTIVITY_PCIE)) \
    $(strip $(CYPRESS_CONNECTIVITY_SDIO)) \
), yes)
ATC_WIFI_CHIP = CYPRESS
ifeq ($(strip $(CYPRESS_CONNECTIVITY_PCIE)), yes)
ATC_WLAN_TRANSMISSION_MODE = PCIE_MODE
endif
ifeq ($(strip $(CYPRESS_CONNECTIVITY_SDIO)), yes)
ATC_WLAN_TRANSMISSION_MODE = SDIO_MODE
endif
endif #filter yes
endif #ATC_WLAN_SUPPORT

# WIFI config end
```

【步骤 3】 在 device.mk 中，根据芯片的选择，添加要编译的模块

代码路径：source/android//device/autochips/ac8x/device.mk

```
# WIFI begin

$(call inherit-product-if-exists,
vendor/autochips/proprietary/hardware/connectivity/wlan/common_wlan/ac8x_config/wi
fidevice.mk)

#WIFI end
```

代码路径：

source/android/vendor/autochips/proprietary/hardware/connectivity/wlan/common_wlan/ac8x_con
fig/wifidevice.mk

```
ifeq ($(strip $(ATC_WLAN_SUPPORT)), yes)
PRODUCT_PACKAGES += android.hardware.wifi@1.0-service-autochips
PRODUCT_PACKAGES += hostapd
PRODUCT_PACKAGES += hostapd_cli
PRODUCT_PACKAGES += wpa_supplicant
```

```

PRODUCT_PACKAGES += wpa_cli
PRODUCT_PACKAGES += wpa_supplicant.conf
PRODUCT_PACKAGES += wpa_supplicant_overlay.conf
PRODUCT_PACKAGES += p2p_supplicant_overlay.conf
PRODUCT_PACKAGES += atc-wifi-service
PRODUCT_PROPERTY_OVERRIDES += persist.atc.wifi.wlansupport=1
PRODUCT_PROPERTY_OVERRIDES += wifi.interface=wlan0
WIFI_HIDL_UNIFIED_SUPPLICANT_SERVICE_RC_ENTRY:=true
#Tethering RRO
PRODUCT_PACKAGES += TetheringResOverlay
PRODUCT_PACKAGES += InProcessTetheringResOverlay
PRODUCT_PACKAGES += GoogleTetheringResOverlay
#Wi-Fi RRO
PRODUCT_PACKAGES += WifiResOverlay
PRODUCT_PACKAGES += GoogleWifiResOverlay
PRODUCT_PROPERTY_OVERRIDES += ro.boot.wificountrycode=CN
#ConnectivityService RRO
PRODUCT_PACKAGES += ServiceConnectivityResourcesOverlay
PRODUCT_PACKAGES += GoogleServiceConnectivityResourcesOverlay
#NetworkStack RRO
PRODUCT_PACKAGES += NetworkStackResOverlay
PRODUCT_PACKAGES += NetworkStackInProcessResOverlay
PRODUCT_PACKAGES += GoogleNetworkStackResOverlay
else
PRODUCT_PACKAGES += android.hardware.wifi@1.0-service-autochips
endif

ifeq ($(strip $(ATC_WIFI_CHIP)), CYPRESS)
PRODUCT_PROPERTY_OVERRIDES += persist.atc.wifi.chip=cypress
PRODUCT_PROPERTY_OVERRIDES += wifi.direct.interface=p2p-dev-wlan0
PRODUCT_PROPERTY_OVERRIDES += persist.atc.wifi.fwpathparam=/sys/class/net/wlan0

```

```
$(call inherit-product-if-exists,
vendor/autochips/proprietary/hardware/connectivity/wlan/cypress_wlan/product_packa
ge.mk)

endif
```

【步骤 4】配置芯片信息

代码路径：

source/android/vendor/autochips/proprietary/hardware/connectivity/wlan/common_wlan/ac8x_con
fig/WifiBroadConfig.mk:

```
ifeq ($(strip $(ATC_WIFI_CHIP)), CYPRESS)

BOARD_WLAN_DEVICE := bcmhdhd

WPA_SUPPLICANT_VERSION := VER_0_8_X

BOARD_WPA_SUPPLICANT_DRIVER := NL80211

BOARD_HOSTAPD_DRIVER := NL80211

BOARD_HOSTAPD_PRIVATE_LIB := lib_driver_cmd_bcmhdhd

BOARD_WPA_SUPPLICANT_PRIVATE_LIB := lib_driver_cmd_bcmhdhd

#ifeq ($(strip $(ATC_WLAN_TRANSMISSION_MODE)), PCIE_MODE)

WIFI_DRIVER_MODULE_NAME := bcmhdhd

WIFI_DRIVER_MODULE_PATH := /vendor/lib/modules/bcmhdhd.ko

WIFI_DRIVER_MODULE_ARG := firmware_path=/vendor/firmware/firmware_pcie.bin \
nvram_path=/vendor/firmware/I1433-SP.txt clm_path=/vendor/firmware/ \
4373_cypress_auto.clm_blob

#ifeq ($(strip $(ATC_WLAN_TRANSMISSION_MODE)), PCIE_MODE)

WIFI_DRIVER_MODULE_NAME := bcmhdhd_sdio

WIFI_DRIVER_MODULE_PATH := /vendor/lib/modules/bcmhdhd_sdio.ko

WIFI_DRIVER_MODULE_ARG := firmware_path=/vendor/firmware/firmware_sdio.bin \
nvram_path=/vendor/firmware/I1433-SP.txt clm_path=/vendor/firmware/ \
4373_cypress_auto.clm_blob\dhd_slpauto=0\ dhd_idletime=0

endif

endif
```


19.4.4 Wi-Fi MAC 的读取和修改

19.4.4.1 ATC 方案的 WLAN Driver 中，如何从 Metazone 中读取 MAC 地址

【步骤 1】引用头文件

```
#include <generated/storage/metazone_index.h>
```

【步骤 2】从 metazone 中读取 Wi-Fi 地址

```
#define ETHER_ADDR_LEN 6  
Uinit32 metaIndex = METAZONE_WIFI;  
// 从 metazone 中读取的地址放入这个 buffer 中  
char readData[ETHER_ADDR_LEN] = {0};  
Metazone_ReadBinary(metaIndex, readData, ETHER_ADDR_LEN);
```

19.4.4.2 ATC 方案的 WLAN Driver 中，如何通过代码向 Metazone 中写入 MAC 地址

【步骤 1】引用头文件

```
#include < generated/storage/metazone_index.h >
```

【步骤 2】向 metazone 中 Wi-Fi 对应的 index 位置，写入 Wi-Fi 地址

```
#define ETHER_ADDR_LEN 6  
Uinit32 metaIndex = METAZONE_WIFI;  
// 从 metazone 中读取的地址放入这个 buffer 中  
char macAddress[ETHER_ADDR_LEN] = {0};  
Metazone_SpecWriteBinary(metaIndex, macAddress, ETHER_ADDR_LEN,  
MTZ_WRITE_FLAG_FACTORYREST);
```

19.5 用户定制化

19.5.1 热点开机自动打开功能

开机时注册 AtcWifiService 服务，服务启动时会检查是否要打开热点，如果需要此功能，需要将此系统属性置为 true，如果是 APP 修改需要配置 selinux 权限：

```
private final static boolean ATC_AUTO_ENABLE_AP_BY_SYSTEMPROPERTIES =  
    SystemProperties.getBoolean("persist.atc.wifi.autoenableap", false);
```

或者通过 settingProvider 将该配置置为 1，不需要配 selinux 权限：

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

645 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

```
int result = Settings.System.getInt(context.getContentResolver(),  
"atc_wifi_auto_enableap")
```

打开此功能后，如果关机前热点是打开状态，重启后服务会自动打开热点。

19.5.2 热点主动断开对端 station 功能

通过调用 com.autochips.wifi 包中的 AtcWifiManager.disconnectStation()接口可断开对端 Station 连接。

```
/**  
 * Return true or false  
 * Disconnect the specified device on the first softap  
 * @param device The mac address of the hotspot client,  
 * @param reasonCode disconnect reason  
 * @see {WifiManager.SAP_CLIENT_BLOCK_REASON_CODE_BLOCK_BY_USER,  
 * WifiManager.SAP_CLIENT_BLOCK_REASON_CODE_NO_MORE_STAS,  
 * WifiManager.SAP_CLIENT_DISCONNECT_REASON_CODE_UNSPECIFIED}  
 * @return if success, return true, other, return false  
 */  
  
@RequiresPermission(android.Manifest.permission.NETWORK_SETTINGS)  
public boolean disconnectStation(@NonNull String device, int reasonCode) {  
    boolean ret = false;  
    try {  
        if (mService != null) {  
            ret = mService.disconnectStation(device, reasonCode);  
        } else {  
            Log.e(TAG, "atc wifi service is not init");  
        }  
    } catch (RemoteException e) {  
        Log.e(TAG, "Failed to disconnectStation."+e);  
    }  
  
    Return ret;  
}
```

```
}
```

19.5.3 根据 MAC 地址获取指定设备的连接状态

通过调用 com.autochips.wifi 包中的 AtcWifiManager.getConnectionState()接口获取指定设备的连接状态

```
/**
 * Return connection state of a MacAddress
 * Get the connection state of the specified device on the first softap
 * @param device The mac address of the hotspot client
 * @return the connection state.
 */
@RequiresPermission(android.Manifest.permission.NETWORK_SETTINGS)
public boolean getConnectionState(@NonNull String device) {
    int state = AtcWifiConstants.AP_DISCONNECTED_WITH_THE_STATION;
    try {
        if (mService != null) {
            state = mService.getConnectionState(device);
        } else {
            Log.e(TAG, "atc wifi service is not init");
        }
    } catch (RemoteException e) {
        Log.e(TAG, "Failed to getConnectionState."+e);
    }

    return state;
}
```

19.5.4 获取对端 station 断开时的 reasonCode

可通过注册广播接收器来获取对端 station 断开时的 reasonCode。

```
private void registerBroadcastReceiver() {
    WifiBroadcastReceiver wifiBroadcastReceiver;
```

```
IntentFilter intentFilter = new IntentFilter();
IntentFilter.addAction(AtcWifiConstants.WIFI_HOTSPOT_CLIENTS_CHANGED_ACTION);
mContext.registerReceiver(wifiBroadcastReceiver, intentFilter)
}

private class WifiBroadcastReceiver extends BroadcastReceiver() {
    @Override
    public void onReceive(Context context, Intent intent) {
        String action = intent.getAction();
        if (AtcWifiConstants.WIFI_HOTSPOT_CLIENTS_CHANGED_ACTION
.equals(action)) {
            // 热点连接设备变了
            boolean connected = intent.getBooleanExtra(AtcWifiConstants.
                EXTRA_STA_CONNECTED); //当前设备是连接还是断开
            String deviceAddress = intent.getStringExtra(AtcWifiConstants.
                EXTRA_DEVICE_ADDRESS); //当前设备的 MAC 地址
            String apInterface = intent.getStringExtra(AtcWifiConstants.
                EXTRA_SOFT_AP_INTERFACE); //当前 ap 的节点名称
            if (!connected) {
                int reasonCode = intent.getIntExtra(AtcWifiConstants.
                    EXTRA_STA_DISCONNECT_REASON); //设备断开的 reasonCode
            }
        }
    }
};
```

19.5.5 获取当前热点的信道

通过调用 com.autochips.wifi 包中的 AtcWifiManager.getSoftapChannel()接口获取当前热点的信道

```
/**
 * Return the softap's channel
 * Get the channel used by the specified softap
 * @param apInterface The softap's interface
```

```
* @return the channel
*/
@RequiresPermission(android.Manifest.permission.NETWORK_SETTINGS)
public boolean getSoftapChannel(@NonNull String apInterface) {
    int channel = 0;
    try {
        if (mService != null) {
            ret = mService.getSoftapChannel(apInterface);
        } else {
            Log.e(TAG, "atc wifi service is not init");
        }
    } catch (RemoteException e) {
        Log.e(TAG, "Failed to getSoftapChannel"+e);
    }

    return channel;
}
```

19.6 典型问题

19.6.1 Wi-Fi 难以搜索到周边设备

【问题描述】

车机 Wi-Fi 搜索周围 AP 只能搜索到少数或搜不到设备的问题。

【解决方案】

以下可排查的方向：

1. 请帮忙先测试 Wi-Fi RF 性能是否达标，如果 RF 性能不达标，请检查硬件设计是否有问题；
2. 如果 RF 性能达标，请测试天线性能是否 OK，比如将 Wi-Fi 天线牵出来或者更换更好的天线是否有改善；
3. 如果以上尝试都无改善，请提供机器给 ATC 调试。

19.6.2 Wi-Fi Mac 地址说明

19.6.2.1 Wi-Fi MAC 地址低 2 位有什么特殊含义

【问题描述】

MAC 地址低 2 位有什么特殊含义？

【解决方案】

MAC 地址的第 1 个字节的 8 个 BIT 从低位到高位分别用 BIT1~BIT8 表示。

1: BIT2:

- (1) 0 代表 Universal administered;
- (2) 1 代表 locally administered;
- (3) 所有的厂商这个 BIT 都是 0。

2: BIT1:

- (1) 0 代表是单播 MAC。接收方的 MAC 与这一帧的目的 MAC 一样的话，就会接收这一帧，否则会丢掉；
- (2) 1 代表是多播 MAC。接收方会检查自己允许接收的多播 MAC 列表，如果接收的这一帧的目的 MAC 在列表中才接收，否则就丢掉。

19.6.2.2 什么样的 Wi-Fi MAC 地址是合法的

【问题描述】

什么样的 MAC 地址是合法的？

【解决方案】

MAC 地址第 1 个字节的低 2 bit 必须为 0，这两个有特殊含义，具体可参考 19.6.2.1 节。其余部分，没有特殊的要求。

比如：00:12:22:33:44:78（这个是合法的）

03:12:22:33:44:78（下面这些都是不合法的）

01:12:22:33:44:78

02:12:22:33:44:78

0e:12:22:33:44:78

19.6.2.3 用户若想按需配置 MAC 地址时，该如何配置

【问题描述】

用户若想按需配置 MAC 地址时，该如何配置？

【解决方案】

Wi-Fi Mac Address 的策略如下图所示。

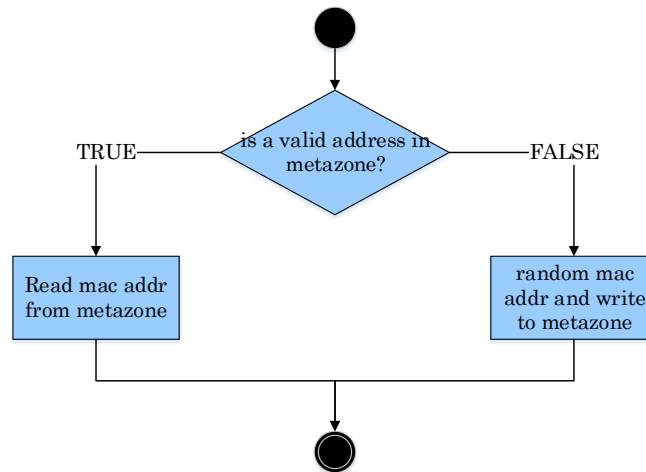


图 19-17 Wi-Fi MAC Address 策略

Wi-Fi MAC Address WLAN Driver 的处理算法如图 19-17 所示：

1. WLAN Driver 先从 Metazone 读取是否有有效地址。
2. 如果 Metazone 有有效地址，则使用 Metazone 的地址。
3. 如果 Metazone 无有效地址，则随机一个 mac addr，然后写入到 Metazone，并使用此地址。

由于优先从 Metazone 中读取 Wi-Fi MAC 地址，因此用户若想配置固定的 Wi-Fi Mac 地址，则可用以下操作配置 Metazone。

19.6.2.3.1 用户也可以直接通过编写代码将 Wi-Fi 地址写入 Metazone 中

具体方法见 19.4.4 节。

19.6.2.3.2 确认修改的 Mac 地址有效

在 Setting-->System→About Car → Wi-Fi Mac address 中，查看当前平台的 MAC 地址。

19.6.3 开发 Wi-Fi APP 时需要添加的权限

【问题描述】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

651 /1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

开发 Wi-Fi 时需要注意添加 Wi-Fi 或 Network 相关的权限，否则可能接口调用无效或功能异常。

【解决方案】

1. 在 Wi-Fi APP 的 AndroidManifest.xml 中添加如下权限：

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.CHANGE_NETWORK_STATE" />
<uses-permission android:name="android.permission.WRITE_SETTINGS" />
<uses-permission android:name="android.permission.WRITE_SECURE_SETTINGS" />
<uses-permission android:name="android.permission.CONNECTIVITY_INTERNAL" />
<uses-permission android:name="android.permission.TETHER_PRIVILEGED" />
<uses-permission android:name="android.permission.WAKE_LOCK" />
<uses-permission android:name="android.permission.DEVICE_POWER" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.INTERACT_ACROSS_USERS_FULL" />
<uses-permission android:name="android.permission.INTERACT_ACROSS_USERS" />
    <uses-permission android:name="android.permission.INJECT_EVENTS" />
    <uses-permission android:name="android.permission.DEVICE_POWER" />
    <uses-permission android:name="android.permission.NETWORK_STACK" />
    <uses-permission android:name="android.permission.OVERRIDE_WIFI_CONFIG" />
    <uses-permission android:name="android.permission.NETWORK_SETTINGS" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.NETWORK_SETUP_WIZARD" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

19.6.4 如何开启 Wi-Fi 的 Debug log

【问题描述】

如何开启 Wi-Fi 的 debug log，以便获取更多的 Wi-Fi log 信息。

【解决方案】

1. 在 Setting 中连续点击版本号，并确认进入开发者模式。
2. 在 Setting 中进入开发者模式，将 Enable Wi-Fi Verbose Logging 选项打开即可，

打开 Wi-Fi Debug log 后，在 Station 的搜索界面，可以看到 AP 的 Mac 地址、信道等相关信息，同时 wpa_supplicant, hostapd 的 debug 级别 log 也会打开。

19.6.5 如何获取 TCP dump 文件

【问题描述】

如何获取 TCP dump 文件？

【解决方案】

【步骤 1】打开 ATCLogger

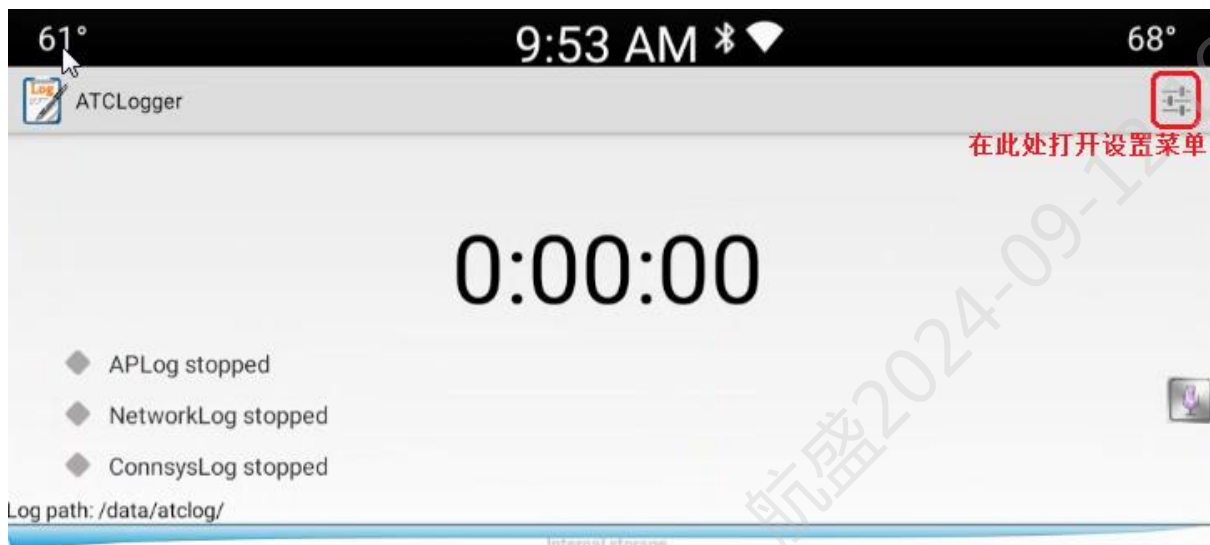
打开 ATCLogger 的入口很多，下面介绍几种：

方法一：直接 adb 命令打开：

```
adb shell am start -n com.autochips.atclogger/com.autochips.atclogger.MainActivity
```

方法二：依次点击 Car Settings → Other → Engineer mode → Log and Debugging → ATCLogger 打开。

【步骤 2】在 ATCLogger 里面勾选打开。





注意：

在把 TcpDump Pull 出来时，一定要确保先停止 AtcLog。

图 19-18 打开 TCP Dump 功能

19.6.6 Wi-Fi P2P GO 和 GC 如何决定

【问题描述】

在 P2P 连接后，如何判断谁做 GO，如何修改 GO intent 值？

【解决方案】

1. 如何判断谁做 GO

(1) 使用 GO Intent 值来判断谁做 GO，Device 1 的 GO Intent 小于 Device 2 的 GO Intent，则 Device 2 将成为 GO。

(2) 若 Device 1 和 Device 2 的 GO Intent 相同，则比较 Tie Breaker，该字段值为 1 的 Device 将成为 GO。由于 Tie Breaker 为随机值，所以两个设备的 Tie Breaker 取值相同的几率非常低。

(3) 如果两个设备都想扮演 GO(例如 GO Intent 都为 15，或者说正好 Tie Breaker 正好随机生成的值都为 1)，则 Group Owner Negotiation 失败。

2. 如何修改 GO intent 值

(1) Android framework 设置的默认值

```
source/android/packages/modules/Wifi/service/java/com/android/server/wifi/p2p/WifiP2pServiceI  
mpl.java
```

```
private static final int DEFAULT_GROUP_OWNER_INTENT = 6;
```

(2) Supplicant 中设置的默认值

```
source/android/vendor/autochips/proprietary/hardware/connectivity/wlan/cypress_wlan/cypress_s  
upplicant/wpa_supplicant/config.h
```

```
#define DEFAULT_P2P_GO_INTENT 7
```

3. 如何查看 GO Negotiation 协商结果

(1) 对端做 GO 时的 log 参考

```
Line 57: 11-03 14:59:39.252937 3598 3598 D wpa_supplicant: P2P: Request to start group  
negotiation - peer=06:b1:67:88:1c:91 GO Intent=6
```

```
Line 93: 11-03 14:59:39.983148 3598 3598 D wpa_supplicant: P2P: GO Negotiation with  
06:b1:67:88:1c:91 completed (peer will be GO)
```

```
Line 98: 11-03 14:59:39.983299 3598 3598 I wpa_supplicant: P2P-GO-NEG-SUCCESS  
role=client freq=5805 ht40=0 peer_dev=06:b1:67:88:1c:91 peer_iface=06:b1:67:88:1c:91  
wps_method=PBC
```

(2) 本地做 GO 时的 log 参考

```
Line 334: 11-03 15:05:35.074789 3598 3598 I wpa_supplicant: P2P-GO-NEG-SUCCESS role=GO  
freq=5745 ht40=0 peer_dev=06:b1:67:88:1c:91 peer_iface=06:b1:67:88:1c:91 wps_method=PBC
```

19.6.7 如何修改 P2P 配置将其投屏在 5G 信道

【问题描述】

2.4G 投屏效果不佳，需要将投屏信道固定在 5G 信道。

【解决方案】

目前投屏信道选择优先支持 5G channel，对于大部分支持 5G WiFi 手机来说，和车机投屏协商最终都是工作在 5G channel，不需要额外配置。

投屏连接成功后，确认如下 log 中 freq 值：

```
Line 334: 11-03 15:05:35.074789 3598 3598 I wpa_supplicant: P2P-GO-NEG-SUCCESS role=GO  
freq=5745 ht40=0 peer_dev=06:b1:67:88:1c:91 peer_iface=06:b1:67:88:1c:91 wps_method=PBC
```

19.6.8 如何打开 AP 的随机 MAC 地址

【问题描述】

AP 打开后，如何使用随机 MAC 地址？

【解决方案】

通过设置

source/android/packages/modules/Wifi/service/ServiceWifiResources/res/values/config.xml 中的 config_wifi_ap_mac_randomization_supported 属性来控制其是否支持 AP 随机 MAC。若其为 true，代表支持 AP 随机 MAC，否则不支持。

修改后，编译替换 com.android.wifi.capex，重启系统生效。

19.6.9 若修改了接口或者 wifiserver 中的内容，如何编译，单独 push 什么可以生效

可在 source/android 目录下单独编译模块：

```
source/android/prebuilts/build-tools/linux-86/bin/ninja -f out/combined-full_ac8x_demo.ninja com.android.wifi
```

替换 com.android.wifi.cpaex 到 system/apex/下重启生效。

19.6.10 需要根据 Vendor 厂商客制化的系统属性，以及这些系统属性的意义是什么？

```
[persist.atc.wifi.autoenableap]: [false]
```

该系统属性默认关闭，打开后，每次会保存热点状态，开机时会根据保存的热点状态决定是否打开热点

```
[persist.atc.wifi.chip]: [cypress]
```

该系统属性表示芯片厂商，在使用 cypress 芯片时为 cypress

```
[wifi.direct.interface]: [p2p-dev-wlan0]
```

该系统属性表示当前 p2p 默认合法节点名称，在使用 cypress 芯片时为 p2p-dev-wlan0

```
[persist.atc.wifi.wlansupport]: [1]
```

该系统属性表示 ATC Wi-Fi 客制化修改，默认为 1

```
[wifi.interface]: [wlan0]
```

该系统属性表示当前 wifi 默认合法节点名称，默认为 wlan0

```
[ro.boot.wificountrycode]: [CN]
```

该系统属性默认为 CN，为开机时可以快速开启 5G 热点。

19.6.11 如何设置 country code，本地若要修改 country code，怎么处理？

本地在 source/android/vendor/autochips/proprietary/hardware/connectivity/wlan/common_wlan/ac8x_config/wifidevice.mk 修改后整编：

```
PRODUCT_PROPERTY_OVERRIDES += ro.boot.wificountrycode = CN
```

或者拉出 vendor/build.prop，修改里面的 ro.boot.wificountrycode，再替换进去生效。

19.6.12 如何通过 overlay 机制客制化 Vendor Feature

1. 目前 ATC Wi-Fi 客制化内容参考

source/android/vendor/autochips/proprietary/hardware/connectivity/wlan/common_wlan/ac8x_config/wifidevice.mk

```
#Tethering RRO
PRODUCT_PACKAGES += TetheringResOverlay

# TetheringResOverlay 覆盖的是
source/android/packages/modules/Connectivity/Tethering/res/values/config.xml 中的内容

PRODUCT_PACKAGES += InProcessTetheringResOverlay
PRODUCT_PACKAGES += GoogleTetheringResOverlay

#Wi-Fi RRO
PRODUCT_PACKAGES += WifiResOverlay

# WifiResOverlay 覆盖的是
source/android/packages/modules/Wifi/Service/ServiceWifiResources/res/values/config.xml 中的内容

PRODUCT_PACKAGES += GoogleWifiResOverlay

PRODUCT_PROPERTY_OVERRIDES += ro.boot.wificountrycode=CN

#ConnectivityService RRO
PRODUCT_PACKAGES += ServiceConnectivityResourcesOverlay

# ServiceConnectivityResourceOverlay 覆盖的是
source/android/packages/modules/Connectivity/ServiceConnectivityResources/res/values/config.xml 中的内容

PRODUCT_PACKAGES += GoogleServiceConnectivityResourcesOverlay

#NetworkStack RRO
PRODUCT_PACKAGES += NetworkStackResOverlay
```

```
PRODUCT_PACKAGES += NetworkStackInProcessResOverlay
```

```
PRODUCT_PACKAGES += GoogleNetworkStackResOverlay
```

Wi-Fi 和 Network 相关的客制化，均在 source/android/vendor/autochips/proprietary/packages/overlay/vendor 下。修改后可通过编译该模块 com.android.xxx，替换 com.android.xxx.capex 重启后生效。

19.6.13 如何查看当前 AP 使用的信道和带宽

#查看信道

```
adb shell
```

```
cat data/vendor/wifi/hostapd/hostapd_wlan.conf
```

```

v Tagged parameters (172 bytes)
  > Tag: SSID parameter set: AndroidAP_8945
  > Tag: Supported Rates 6(B), 9, 12(B), 18, 24(B), 36, 48, 54, [Mbit/sec]
  > Tag: Traffic Indication Map (TIM): DTIM 1 of 0 bitmap
  > Tag: HT Capabilities (802.11n D1.10)
  > Tag: HT Information (802.11n D1.10)
  > Tag: Extended Capabilities (8 octets)
  > Tag: Vendor Specific: Microsoft Corp.: WMM/WME: Parameter Element
  > Tag: RSN Information
  > Tag: VHT Capabilities
  v Tag: VHT Operation
    Tag Number: VHT Operation (192)
    Tag length: 5
    v VHT Operation Info
      Channel Width: 20 MHz or 40 MHz (0x00)
      Channel Center Segment 0: 0
      Channel Center Segment 1: 0

```

20 Ethernet

20.1 简介

20.1.1 软硬件环境

20.1.1.1 软件环境

- DNS 服务 – 提供域名服务（Android 自带）
- DHCP 服务 – 提供动态 IP 配置（Android 自带）
- MAC Driver – MAC 驱动（ATC Driver）
- PHY Driver – PHY 芯片驱动（不同的 PHY 芯片需要调试不同的 PHY Driver）

20.1.1.2 硬件环境

Ethernet 功能包含 MAC（AC8025 内置）+ PHY 两部分：

- 主芯片：AC8025（内置 MAC）
- PHY 芯片：DP83TG720R-Q1、TJA1101（公板贴这两个，其他适配过的型号见 [20.5.1 已适配过的 PHY 芯片型号](#)）

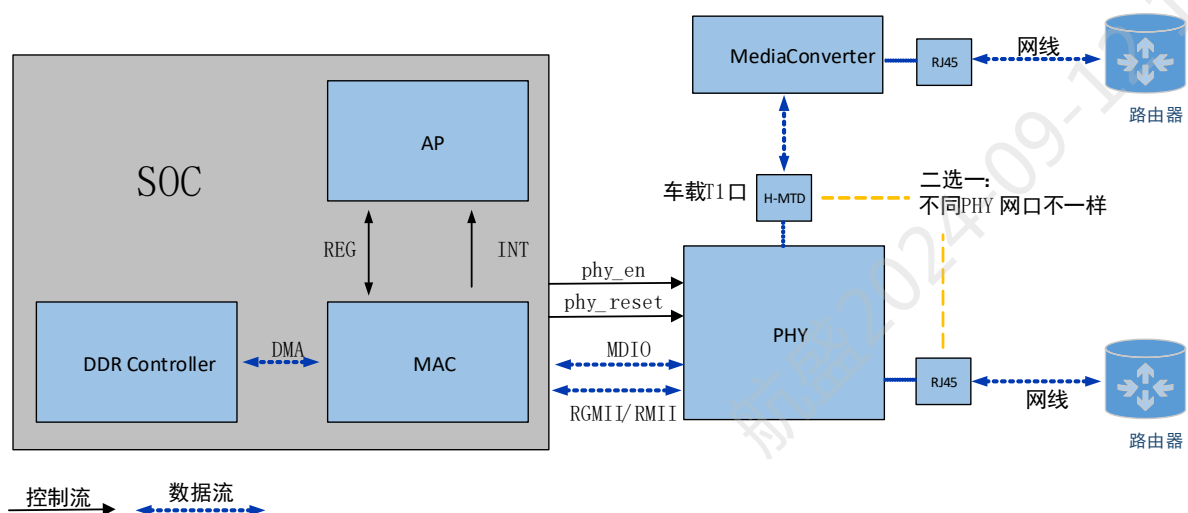


图 20-1 Ethernet HW 连接图

- AP 通过 REG 控制 MAC，通过 MAC+MDIO 配置 PHY 芯片
- MAC 通过 DMA 访问 DDR，通过 RGMII(1000M)、RMII(100M)与 PHY 进行大数据传输

- PHY 与外部的连接，根据支持不同的网口，连接方式不同：
 - 1000BASE-T1、100BASE-T1，车规接口，直接与 T1 口对端连接，或用 MediaConverter 转为普通网口与对端连接
 - 1000BASE-T、100BASE-TX，直接用普通网线与对端连接

20.1.2 功能

20.1.2.1 MAC Features

- 支持 10/100/1000Mbps Ethernet
- 支持支持 8 对 TX/RX Queue
- 支持 RGMII/RMII
- 支持 Clause 22 MDIO
- 支持 TX/RX Offload
- 支持 RX Filter
- 支持 TX 调度
- 支持高精度时钟 TSU
- 支持 Pause Frame
- 支持 DMA

20.1.3 代码结构及文件描述

表 20-1 Ethernet 代码结构及描述

文件名	文件路径	文件描述
ac802x.dtsi	source/kernel/common/arch/arm64/boot/dts/autochips/	macb0 节点是 MAC & PHY driver 的配置
ac8x_demo.dts	source/kernel/common/arch/arm64/boot/dts/autochips/	macb_pins_x macb0 节点，进行 pinctrl & phy_en phy_reset pin 配置
整个目录	source/kernel/common/drivers/net/ethernet/cadence/	MAC 驱动，会注册 eth0 网卡设备供 user 使用
整个目录	source/kernel/common/drivers/net/phy/	PHY core 驱动和不同 PHY 芯片的驱动，对接 PHY 芯片和 MAC 驱动

文件名	文件路径	文件描述
整个目录	source/android/packages/modules/NetworkStack/ src/android/net/dhcp/	DHCP 服务，用于动态获取 IP 地址
整个目录	source/android/packages/modules/DnsResolver/	DNS 服务，用于域名服务

20.2 软件设计

20.2.1 架构图

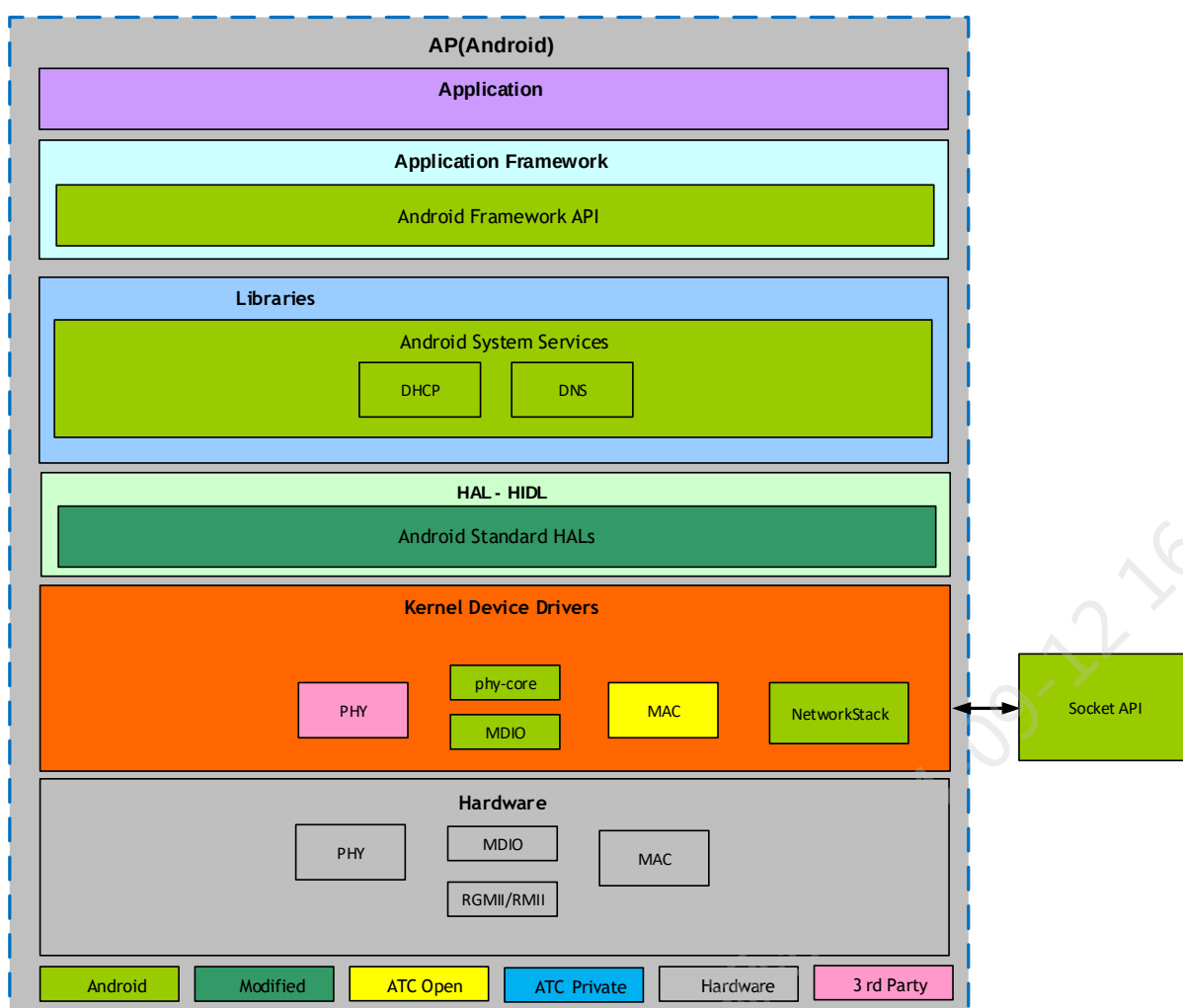


图 20-2 Ethernet 架构图

20.2.2 流程图

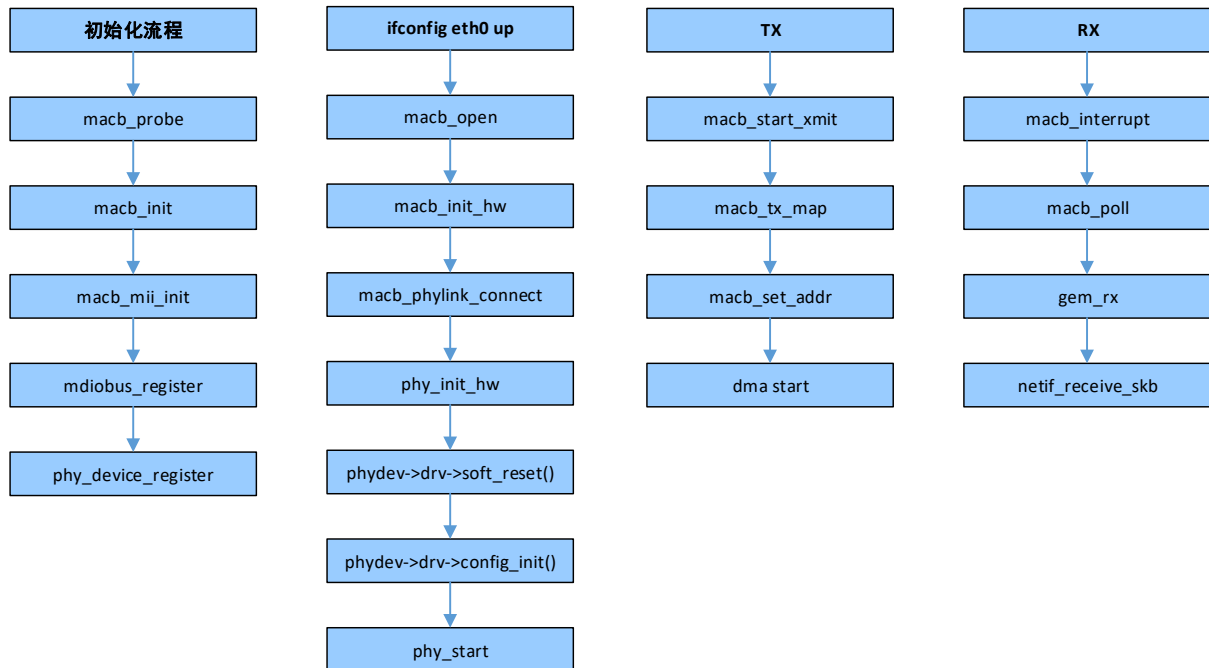


图 20-3 Ethernet 流程图

20.3 接口描述

Ethernet 控制流是标准的网卡操作接口，数据流是标准的 socket 接口。

20.4 使用说明和注意事项

20.4.1 如何确认 Ethernet 是否已通

1. Android 启动后 MAC 驱动会注册 eth0 网卡设备，Android 网络服务会将 eth0 up 起来；
2. DHCP 服务会自动获取 IP 地址，然后就可以 ping 通路由器了；
3. 如果路由器连接了网络，用浏览器就可以直接访问网络；
4. 状态栏图标：<...>|（只能访问局域网）或<...>（可以访问外网）

20.4.2 如何用命令测试 Ethernet

如果 Android 网络服务、DHCP 服务等没有 ready，或者是在 linux 下调试，则需要手动 up 网卡：
ifconfig eth0 [ip] up # [ip]必须跟连接的路由器同一网段，然后就可以 ping 通路由器 ip 了。

20.5 用户定制化

20.5.1 已适配过的 PHY 芯片型号

表 20-2 Ethernet 已适配过的 PHY 芯片型号

厂商	型号	速率	网线接口
TI	DP83TG720R-Q1	1000M	1000BASE-T1
TI	DP83867	1000M	1000BASE-T
Broadcom	BCM89884	1000M	1000BASE-T1 / 100BASE-T1
NXP	TJA1100	100M	100BASE-T1
NXP	TJA1101	100M	100BASE-T1
JLSemi	JL11x1	100M	100BASE-TX

其他未调试过的只要符合 Ethernet Spec 都可以支持，需要调试 PHY driver。

20.5.2 PHY 芯片适配流程

适配一款新的 PHY 芯片，需要根据 PHY Spec 确认硬件电路和软件配置，可按以下确认项来逐项核对。

20.5.2.1 datasheet 确认

表 20-3 Ethernet PHY datasheet 确认项

检查项	支持的 Features	Comment
MDIO 版本	Clause 22	
MII 接口	RGMII/RMII	1000M 需要 RGMII, 10/100M 可以用 RGMII/RMII
速率	10/100/1000Mbps	
REF_CLK	50M 输入	RMII 模式，必须由外部或 PHY 输出 50M REF_CLK 供给 MAC RX_CLK pin

20.5.2.2 硬件确认

表 20-4 Ethernet PHY 硬件确认项

检查项	子项	正常值	可能的异常	Comment
MAC	RMII_SELECT	100M - LOW - RMII 1000M - HIGH - RGMII		需要与 dts 对应
供电	供电电压	依赖 Spec	供电电压不符合 spec	
	上拉电压	依赖 Spec	EN & RESET & IO 要求 3.3V 上拉，实际 1.8V 上	

检查项	子项	正常值	可能的异常	Comment
			拉	
	上电时序	依赖 Spec	HW 控制部分上电时序不符合 spec	通常 power 由 HW 控制，EN & RESET 由 SW 控制
CLK	晶振	25M	晶振不合格，PHY 状态异常	比较难发现，有时候要 FAE 才能看出来
	REF_CLK	RMII: 50M	有些 PHY 是 TX_CLK pin 出来的，HW 没接对	通常是 PHY 供出来给 MAC RX_CLK pin
PHY Strapping	MII Mode	100M - RMII 1000M - RGMII		需要与 dts 对应
	REF_CLK	RMII: 50M 输出		JL11x1 有此项
	PHY ADDR	需要与 dts 对应		
	Master/Slave	需要与对端 PHY (Converter) 相反		T1 口的有此项设置
	Managed/Autonomous	Autonomous		TJA1100/TJA1101 有此项
	LDO MODE	依赖 Spec		TJA1100/TJA1101/JL11x1 有此项
	其他	依赖 Spec		对一下 Spec 看还有没有其他 strapping 需要设置的
连接线	T1 双绞线	顺序正确	双绞线接反了	

20.5.2.3 软件确认

表 20-5 Ethernet PHY 软件确认项

检查项	子项	正常值	代码位置	Comment
dts	MII Mode	100M: phy-mode = "rmii"; 1000M: phy-mode = "rgmii-id";	ac802x.dtsi	
	phy node: phy@x	status = "okay"; compatible = "厂商,型号","ethernet-phy-ieee802.3-c22";	ac802x.dtsi	与 strapping 对应，或者注掉
	PHY ADDR	reg = <xxx>;	ac802x.dtsi	
	macb_pins_15 /phyen_pins macb_pins_16 /phyreset_pins	与 HW 对应	ac8x_demo.dts	
defconfig	defconfig	开启以下配置 CONFIG_MACB CONFIG_PHYLINK CONFIG_XXX_PHY	ac8025_gki.fragment	默认没有开启的在 ac8025_gki.fragment 添加，确认 out/.../.config 最终是否生效
driver	phy driver	有些有专用 phy driver，有些是通用的 genphy_driver	source/kernel/common/drivers/net/phy/xxx.c	
	mac driver	1、如果上电时序不一样，需要修改 macb_IOConfig 上电部分 2、如果识别不到 phy，尝试拉一下 reset	source/kernel/common/drivers/net/ethernet/cadence/macb.c	

20.6 典型问题

20.6.1 PHY link up 成功的参考 log

1. MAC driver ready

```
[ 5.208576][ T1][I] macb 11020000.ethernet eth0: Cadence GEM rev 0x01070109 at
0x11020000 irq 37 (3a:0e:03:04:05:06)
```

2. PHY link up 成功

```
[ 12.245323][ T113][I] macb 11020000.ethernet eth0: PHY [11020000.ethernet-ffff:00]
driver [NXP TJA1100] (irq=POLL)

[ 12.246621][ T113][I] macb 11020000.ethernet eth0: configuring for phy/rmii link mode

[ 184.307408][ T114][I] macb 11020000.ethernet eth0: Link is Up - 100Mbps/Full - flow
control off

/# ifconfig

eth0    Link encap:Ethernet HWaddr 3A:0E:03:04:05:06

        inet addr:192.168.2.33 Bcast:192.168.2.255 Mask:255.255.255.0

        UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1

        RX packets:0 errors:0 dropped:0 overruns:0 frame:0

        TX packets:3 errors:0 dropped:0 overruns:0 carrier:0

        collisions:0 txqueuelen:1000

        RX bytes:0 (0.0 B) TX bytes:278 (278.0 B)

        Interrupt:32
```

20.6.2 PHY 芯片工作异常

【问题描述】

无法识别到 PHY 芯片，或无法 link up。

【解决方案】

1. 如果是已经适配 ok 的 PHY 不工作，可检查以下几点：

- 路由器工作是否正常（连接 PC 测试）；
- MediaConverter 是否正常（重新断电上电）；
- 检查网线连接；

- PHY 芯片供电、晶振、EN、RESET 是否正常；
- PHY 芯片是否损坏。

2. 如果是新的 PHY 芯片适配遇到问题，按 [20.5.2 PHY 芯片适配流程](#) 进行检查；

20.6.3 PHY link up 但 ping 失败

【问题描述】

link up 成功，但 ping 失败。

【解决方案】

1. ifconfig eth0，如果没有 IP 地址则为 DHCP 服务异常，用 tcpdump 分析 DHCP 协议包；
2. 有 IP 地址 ping 不通路由器，检查路由器是否正常，用 PC 连接测试一下；
3. 可以 ping 通路由器，ping 不通外网，检查路由器与外网的连接。

航盛2024-09-12 16:50:37

21 GNSS

21.1 简介

21.1.1 软硬件环境

21.1.1.1 软件环境

本章所述的 Location 采用 Android 原生的 Location 框架提供 Location 服务。

21.1.1.2 硬件环境

GNSS IC: UC6226

21.1.2 功能

GNSS 的基本功能:

- 打开及关闭
- 搜星定位
- 获取时间信息
- 获取经纬度及高度信息
- 获取当前可视卫星信息
- 获取辅助星历定位数据

21.1.3 代码结构及文件描述

1. Source code 及路径

表 21-1 GNSS 代码路径及描述

路径	作用
source/android/frameworks/base/location/	原生 java Framework 代码, 供 APP 调用
source/android/frameworks/base/services/core/java/com/ android/server/location/LocationManagerService.java	Location Manager Service 的实现
source/android/frameworks/base/services/core/java/com/	

路径	作用
android/server/location/ source/android/vendor/autochips/proprietary/hardware/ connectivity/gps/	GPS HAL 层及定位运算 进程的实现

2. Build 生成的库文件以及路径

表 21-2 GNSS 动态库及描述

动态库名称	源代码路径	作用
gps.default.so	/source/android/vendor/autochips/proprietary/ hardware/connectivity/gps/atc_gnss_hal	该库主要用于实现 GPS HAL 的所有功能

3. Configure 文件以及路径

表 21-3 GNSS 配置文件及描述

文件名	文件路径	文件描述
gnssconfig.h	/source/android/vendor/autochips/proprietary/ hardware/connectivity/gps/atc_gnss_hal/config/	GNSS config definitions

21.2 软件设计

21.2.1 架构图

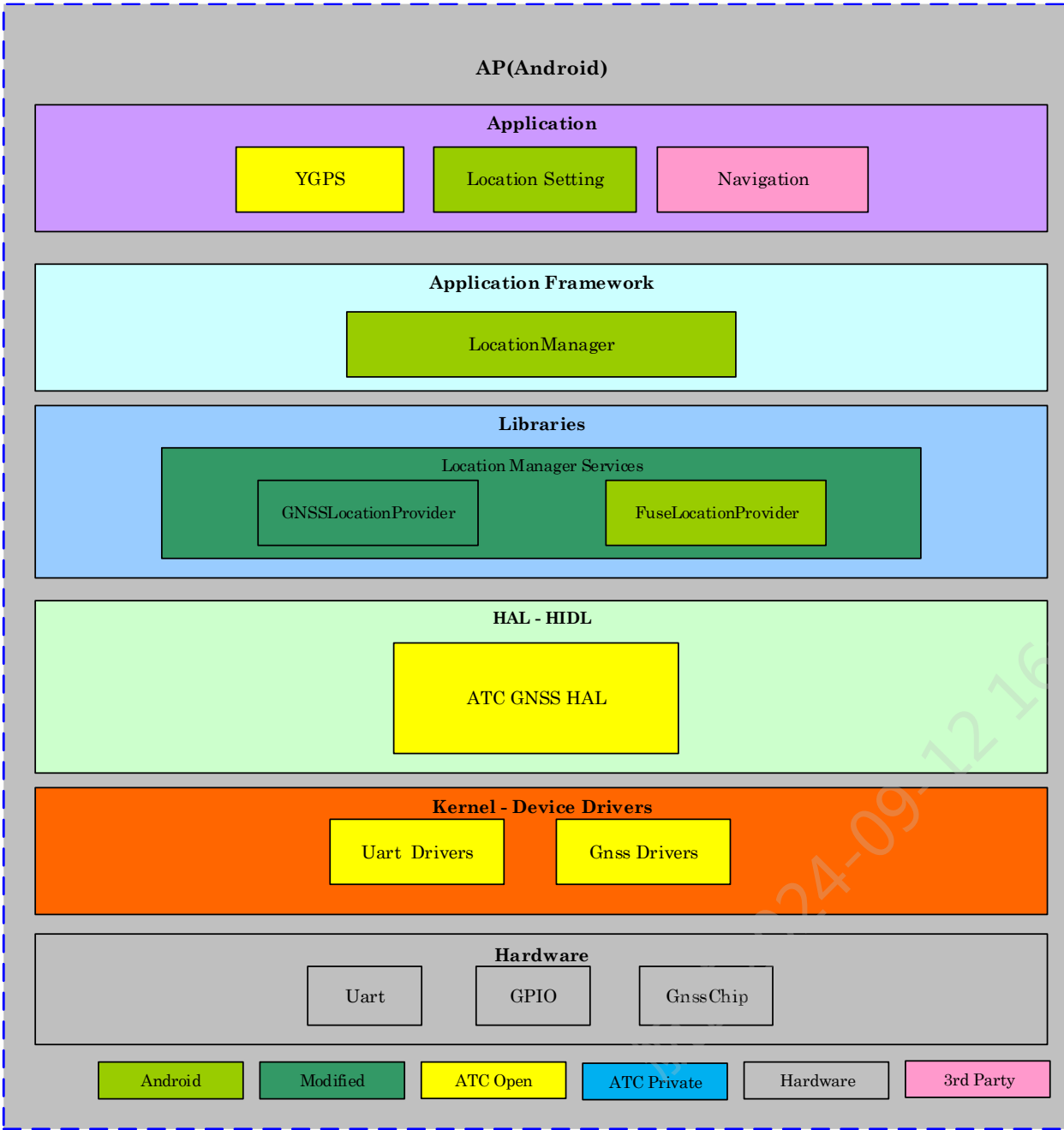


图 21-1 GNSS 架构图



说明：

使用 UC6226 芯片方案，通过 UART 进行通信。

21.2.2 数据流

GNSS 的数据流是通过 Uart driver 在 kernel 态与芯片进行交互后，然后将 Nmea 数据通知给用户态 GNSS HAL，最终通过回调的方式传送给 HAL 层的服务。

如图 21-2 所示，数据通知给 HAL 层后，通过 callback 的方式回调给应用。应用只需要注册 listener 给 locationManagerService，当有位置信息的时候即可得到需要的回调信息。

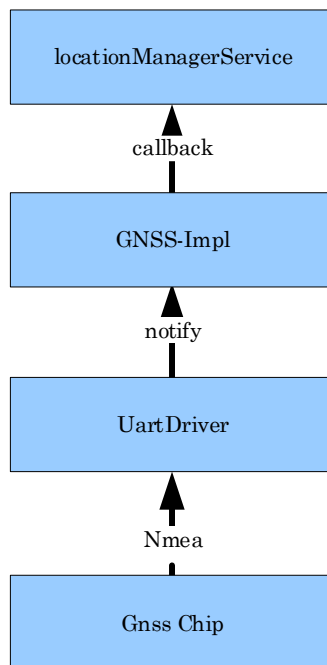


图 21-2 UC6226 数据流示意图

21.3 使用说明和注意事项

使用由 Android 原生定义的 LocationManager 类，详细请参见：
<https://developer.android.com/reference/android/location/LocationManager>
<https://developer.android.com/training/location>

使用注意事项如下：

- 若要使用 GNSS 服务获取信息，需要选择性导入如下包：

```
import android.location.GpsSatellite;    //需要卫星信息
import android.location.GpsStatus;       // GPS 状态定义包
import android.location.GpsStatus.Listener; //GPS status 的监听接口包
import android.location.Location;        //位置信息的包
import android.location.LocationListener; //位置信息的监听接口包
import android.location.LocationManager;
```

- 获取 LocationManger 的接口

```
//获取 LocationManager 的接口对象
```

```
mLocationManager = (LocationManager)getSystemService(Context.LOCATION_SERVICE);
```

- 实现 location listener （以需要获取 location 为例子）

```
LocationListener mLocationListener = new LocationListener() {
    @Override
    public void onStatusChanged(String provider, int status, Bundle extras) {
        // TODO Auto-generated method stub
        //This callback will never be invoked on Android Q and above
    }

    @Override
    public void onProviderEnabled(String provider) {
        // TODO Auto-generated method stub
    }

    @Override
    public void onProviderDisabled(String provider) {
        // TODO Auto-generated method stub
    }

    @Override
    public void onLocationChanged(Location location) {
        // TODO Auto-generated method stub
    }
}
```

```
//if fixed , it will get the method callback
if(location.hasAltitude()) {
    // location.getLatitude();
    // location.getLongitude();
} else {
    //Log.d("GPS", "not have info");
}
// location.getTime();
}
};
```

- 向 locationManager 注册监听，等待有位置信息的回调。

```
mLocationManager.requestLocationUpdates(LocationManager.GPS_PROVIDER, 1000, 0,
    mLocationListener);
```

21.4 用户定制化

21.4.1 GNSS Mode 配置

目前默认设置的值，同时支持 GPS、Beidou、Galileo，用户可以不用更改。如果需要更改，修改文件：

source/android/vendor/autochips/proprietary/hardware/connectivity/gps/atc_gnss_hal/protocol/vendor_cmd/uc6226vendorecmd.cpp 中：

```
int Uc6226VendorCmd::getDefaultGnssMode() const
{
    return (config::GNSS_MODE_GPS_SUPPORT_BIT |
            config::GNSS_MODE_BEIDOU_SUPPORT_BIT |
            config::GNSS_MODE_GALILEO_SUPPORT_BIT);
}
```

此值定义如下：

```
enum
{
    GNSS_MODE_GPS_SUPPORT_BIT = (1<<0),    //config support GPS bit
    GNSS_MODE_BEIDOU_SUPPORT_BIT = (1<<1),  //config support BEIDOU bit
```

```
GNSS_MODE_GLONASS_SUPPORT_BIT = (1<<2),    //config support GLONASS bit
GNSS_MODE_GALILEO_SUPPORT_BIT = (1<<3),      //config support GALILEO bit
};
```



说明：

根据上述定义，当前可以配置的值如下：

- 1 - GNSS_MODE_GPS_SUPPORT_BIT //support GPS only
- 2 - GNSS_MODE_BEIDOU_SUPPORT_BIT //support BEIDOU only
- 11 - (GNSS_MODE_GPS_SUPPORT_BIT | GNSS_MODE_BEIDOU_SUPPORT_BIT |
GNSS_MODE_GALILEO_SUPPORT_BIT) //support GPS & Beidou & Galileo
- 13 - (GNSS_MODE_GPS_SUPPORT_BIT | GNSS_MODE_GLONASS_SUPPORT_BIT |
GNSS_MODE_GALILEO_SUPPORT_BIT) //support GPS & Glonass & Galileo

21.5 典型问题

21.5.1 如何进行 GPS 测试

【问题描述】

如何进行 GPS 测试？

【解决方案】

1. 直接使用地图进行测试，查看搜星定位情况；
2. 使用 YGPS 进行查看搜星定位情况。adb 输入：

```
am start -n com.autochips.ygps/.YgpsActivity
```

21.5.2 如何打开 GPS debug log

【问题描述】

如何打开 GPS debug log？

【解决方案】

进入到工程模式（在 adb 中敲命令：`am start -n com.autochips.engineermode/com.autochips.engineermode.EngineerMode`） >> Log and Debugging >> ATCLogger >> 勾选上 ConnsysLog recording >> start。

当 GNSS 开始工作后默认就会将 log 保存在/data/atclog/connsyslog/gpshost/下。

21.5.3 3D 定位和 2D 定位的区别

【问题描述】

3D 定位和 2D 定位的区别？

【解决方案】

- 3D 一般是指使用至少 4 颗以上卫星完成的定位。
- 2D 一般使用 3 颗卫星完成的定位过程。
- 3D 相比于 2D 定位，即定位精度更高。

21.5.4 各个卫星系统的卫星编号是多少

【问题描述】

各个卫星系统的卫星编号是多少？

【解决方案】

表 21-4 卫星编号

Satellite System	SVID
GPS	1 ~ 32
WAAS	120 ~ 151 183 ~ 192
GLONASS	1 ~ 24 93 ~ 106
GALILEO	1 ~ 54
QZSS	193 ~ 200
BEIDOU	1 ~ 37



说明：

SVID 应用层收到的卫星编号(这些编号是符合 GNSS 各大系统官方设计的编号)。如果需要区分是那种卫星，其定义请直接参考：

<https://android.googlesource.com/platform/frameworks/base/+/-/master/location/java/android/location/GnssStatus.java>

21.5.5 硬件传导条件要求

【问题描述】

硬件传导条件要求。

【解决方案】

从天线插入口打入-130dBm 的信号，要求搜到卫星的信号值在 38dB-Hz ~ 55dB-Hz 之间。

如果无法达到此条件，请用户从硬件上检查板间的干扰情况（如走线、信号传出的距离是否过长、走线周边是否存在其它干扰源等）。



说明：

使用天线搜索实际空中信号进行定位，要求搜索卫星值需要小于 55dB-Hz，防止由于信号饱和导致定位漂移。

21.5.6 晶体要求

【问题描述】

晶体要求。

【解决方案】

目前 clock 指标数据，可以参考如下：

- 没有热干扰的情况下，建议 clock drift 小于 2.5ppb/s。
- 在有热干扰的情况下，建议 clock drift 小于 10ppb/s。

21.5.7 各种启动方式的 TTFF 是多少

【问题描述】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

675 /1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

各种启动方式的 TTFF 是多少？

【解决方案】

TTFF 的结果和测试环境、测试的手法、硬件的 GPS 性能强相关。

给出的数据是基于在 open sky 的环境下，有 6 颗卫星 SNR $\geq 40\text{db}$ 条件下标准如下表。

表 21-5 TTFF 标准

CHIP	HOT start	WARM Start	COLD Start
UC6226	$\leq 5\text{s}$	$\leq 30\text{s}$	$\leq 30\text{s}$

21.5.8 各种启动方式 TTFF 的测试方法

【问题描述】

各种启动方式的 TTFF 的测试方法？

【解决方案】

参考前提条件及标准（各种启动方式的 TTFF 是多少），测试步骤如下：

1. adb 中执行 `am start -n com.autochips.ygps/.YgpsActivity` 打开 YGPS；
2. 点击进入 INFORMATION 栏；
3. Hot start 按钮对应测试 HOT start TTFF

Warm start 按钮对应测试 WARM start TTFF

Cold start 按钮对应测试 Cold start TTFF



说明：

- 测试需要保证满足对应的测试条件要求，如果无法满足，对应测试的 TTFF 值可能偏大；
- 测试前保持定位 15 分钟以上，两次测试间隔建议大于 1 分钟。

21.5.9 各种启动方式的条件要求

【问题描述】

各种启动方式的条件要求。

【解决方案】

每种启动方式条件如下：“√”表示必须的条件，“×”表示不需要的条件，optional 表示可选。

表 21-6 TTFF 启动条件

Restart type	Last postion	Reference time	ephemeris	almanac	Clock drift
Hot	√	√	√	optional	√
Warm	√	√	×	√	√
Cold	×	√	×	√	√

21.5.10 如何快速获取 GNSS 软件版本

【问题描述】

如何快速获取 GNSS 软件版本？

【解决方案】

1. 打开 gps debug log，详细见 [21.5.2](#)
2. 进入工程模式下 YGPS >> INFORMATION，点击下 HOT start
3. 抓取 GPS debug log，使用 notepad++打开，在此 log 文件中搜索“PDTINFO”，找到语句中内容类似于“R3.4.0.0Build7258”的字符串即为软件版本号

21.5.11 如何从 NMEA 数据中看是否定位

【问题描述】

如何从 NMEA 数据中看是否定位？

【解决方案】

查看 GSA 语句的第二个字段，如果是 2 表示 2D 定位，如果是 3 表示 3D 定位，其它值为不定位。

```
$GPGSA,A,3,193,24,15,20,21,13,18,05,29,,1.14,0.67,0.93*30
```

21.5.12 如何修改 framework 代码，强制输出某个地区经纬度的内容

【问题描述】

如何修改 framework 代码，强制输出某个地区经纬度的内容？

【解决方案】

添加固定的经纬度信息到 location 中，让其模拟上报：

修改

/source/android/frameworks/base/services/core/java/com/android/server/location/gnss/GnssLocationProvider.java 的 handleReportLocation 方法。

如下红色部分为增加需要测试的数据即可。

```
private void handleReportLocation(boolean hasLatLong, Location location){  
  
    location.setLatitude(22.55534); //set the latitude valude need test  
    location.setLongtitude(113.54443); //set the longtitude value need test  
    location.setExtras(mLocationExtras.getBundle());  
    reportLocation(LocationResult.wrap(location).validate());  
  
    .....  
}
```

21.5.13 如何关闭在线获取辅助星历功能

【问题描述】

如何关闭在线获取辅助星历数据功能？

【解决方案】

客户配置文件位置：

/source/android/vendor/autochips/proprietary/hardware/connectivity/gps/atc_gnss_hal/config/atcnssconfig.xml 如下：

```
<?xml version="1.0" encoding="UTF-8"?>  
<atcgnss>  
    <AgnssEnable>1</AgnssEnable>  
</atcgnss>
```

将 AgnssEnable 字段配置为 0，则关闭在线获取辅助星历数据的功能。

21.5.14 如何配置位置更新频率

【问题描述】

如何配置位置更新频率？

【解决方案】

UC6226 没有提供芯片相关的配置位置更新频率的接口，目前是 1HZ，就是 1 秒更新一次定位信息。

软件上客户如果想调节应用接收到位置信息的频率，可以参考官方 LocationManager 的 requestLocationUpdates 接口进行调节。

21.5.15 第三方 GNSS 芯片如何适配

【问题描述】

第三方 GNSS 芯片如何适配？

【解决方案】

- 若第三方芯片厂商有提供对应的 HAL 和 driver，可以直接用第三方厂商提供的代码。
- 若第三方芯片厂商未提供对应的 GNSS HAL，建议尽量使用厂商提供的 hal 层，需要厂商支持。
- 若客户打算自己写 HAL 层，ATC 写了一个 GNSS HAL 框架，可以以此框架为基础，快速适配，类图如下：

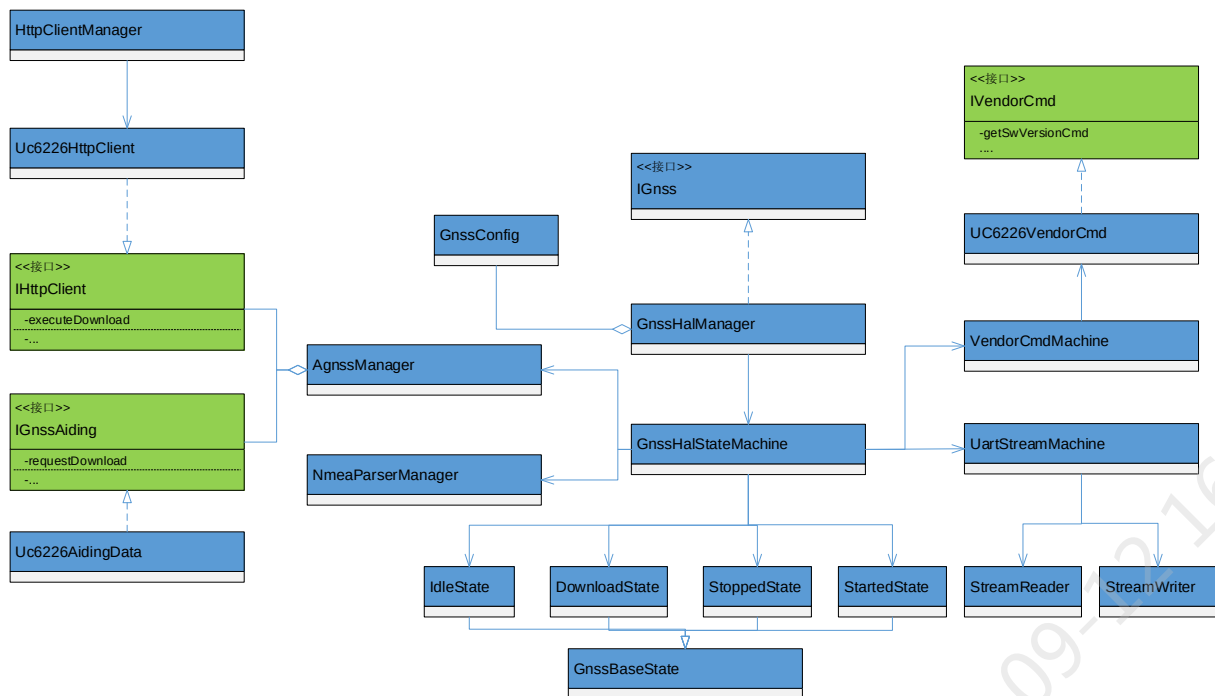


图 21-3 GNSS HAL 类图

上述类图中绿色部分为接口，其他第三方芯片需要适配时，只需要对应的芯片实现上述接口并加入到对应的 manager 中，即可快速方便适配。

- IVendorCmd 为 vendor 相关命令获取的接口。
- IHttpClient 接口为通过 http 协议从芯片厂商服务器拿辅助定位数据，如果其它芯片也使用的是 http 方式获取，可以实现此接口。
- IGnssAidingData 为辅助数据的接口定义，对于无格式要求的，当前实现的 Uc6226AidingData 可以直接复用，如果有格式要求可以实现对应接口，然后在对应对象中封装对应的数据。

上述类图蓝色部分为接口实现类，不对外暴露接口。主要分为状态机管理模块、Vendor 命令管理模块、Stream 数据管理模块、数据 Parser 管理模块、AGNSS 管理模块及 GNSS 配置模块。

- 状态机管理模块，主要负责对 GNSS 的状态进行管理及 error recovery 恢复流程处理。
- Vendor 命令管理模块主要抽象出 IVendorCmd 接口，管理各个厂商的 Vendor 命令（其他厂商实现 IVendorCmd 接口进行简单适配即可）。
- Stream 数据管理模块主要负责数据的读写，提供 AP 与 GNSS 芯片间数据的读写管理服务。
- 数据 Parser 管理模块，主要分析对 NMEA 数据的 parser 及 Vendor 数据的 parser。
- AGNSS 管理模块，主要负责对辅助定位数据的下载及管理。
- GNSS 配置模块，主要负责 GNSS 配置参数的管理。

22 Multimedia

22.1 简介

22.1.1 软硬件环境

22.1.1.1 软件环境

本章所述 Multimedia 播放采用原生的 MediaPlayer 框架，底层使用 NuPlayer。由于需要增强对多媒体文件播放能力的支持，ATC 对原生框架做了相应的改动，如 NuPlayer、ACodec 等。

本章所述 MediaScanner 功能，Android S 使用 ModernMediaScanner。

22.1.1.2 硬件环境

本章所述的多媒体播放基于以下硬件设备：

- 视频编码器---用于文件编码
- 视频解码器---用于文件解码
- Audio DSP---用于 Audio 文件解码

22.1.2 功能

22.1.2.1 多媒体的播放功能

本章所述的多媒体播放功能支持：

- Play

点击播放视频文件，视频文件 3s 内起播，音频文件 1s 内起播。

- Stop

停止播放，做 reset 操作，释放底层资源。

- Pause

暂停播放，当前播放流程处于暂停状态。

- Resume

从暂停播放状态继续播放。

▪ Seek

跳转到指定位置进行播放。

▪ Audio Switch

系统最大支持 8 路 Audio。播放有多路 Audio 媒体文件时，默认播放第一路，可以手动切换，如果该路 Audio 格式不支持，则给出相应提示信息。

▪ Subtitle Switch

系统支持内置字幕和外置字幕，详情如下：

- 内置字幕最多支持 8 路，外置字幕没有限制
- 字幕支持 subpic 格式和 subtext 格式
- 播放时默认不显示字幕

22.1.2.2 MediaRecorder 功能

录制功能支持格式：

- mp4
- ts

本文档所述 MediaRecorder, 支持如下：

表 22-1 MediaRecorder 支持列表

文件格式	Video Codec	Audio Codec
TS	H.264 6x1080P@30@50Mbps Baseline@L5.1;Main@L5.1;High@L5.1 H.265 6x1080P@30@50Mbps Main Profile@Level 5.1	AAC HE-AAC AAC-ELD
MP4	H.264 6x1080P@30@50Mbps Baseline@L5.1;Main@L5.1;High@L5.1 H.265 6x1080P@30@50Mbps Main Profile@Level 5.1	AAC, HE-AAC, AAC-ELD, AMR

22.1.2.3 MediaScanner 功能

22.1.2.3.1 扫描时机

接收系统发出的特定消息，执行多媒体文件扫描或删除数据库的操作，具体消息列表如下：

- 系统启动后的 Boot_Complete 消息触发 mediascanner 扫描；

- 插入 SD 卡或 U 盘后的 Mount 消息触发 mediascanner 扫描；
- 新增文件后的 Scan_File 消息触发 mediascanner 扫描（目前在文件浏览 APK 中增删文件，MediaRecord 创建文件会发送 Scan_File，触发扫描，adb push 文件到板子上不会触发）；
- 拔出 SD 卡或 U 盘后的 Unmount 消息不触发删除数据库的操作，重新插入同一张 SD 卡或 U 盘，会触发更新数据库。

22.1.2.3.2 扫描性能

存储设备中存放 500 个视频文件、1500 个图片文件、3000 个音频文件，扫描耗时如下：

表 22-2 存储设备扫描性能

Storage	Test Item	MediaScanner 时间（秒）
SD 卡	启动系统后插入 SD 卡	152
USB2.0 OTG	启动系统后插入 U 盘(USB2.0 OTG)	106
USB2.0 Host	启动系统后插入 U 盘(USB2.0 Host)	106
USB3.0 TypeC	启动系统后插入 U 盘(USB3.0 TypeA)	98

22.1.2.3.3 扫描限制

单个分区支持最大扫描 500 个视频文件，1500 个图片文件，3000 个音频文件。

22.1.2.4 支持的 Container 和 Codec

详细的支持情况请参考 Release 的 File Capability 文档：
《ATC_AC8025_Android_S_Multimedia_File_Capability》。

22.1.2.5 性能说明

- 本文档所述多媒体播放，视频文件支持情况参考 Release 的 File Capability 文档：
《ATC_AC8025_Android_S_Multimedia_File_Capability》
- MediaScanner 性能参见 22.1.2.3.2。

22.1.2.6 代码结构及文件描述

22.1.2.6.1 源代码

开发 Multimedia，修改或添加的源文件如表 22-3 所示。

表 22-3 Multimedia 源文件

文件名	文件路径	文件描述
NuPlayer.cpp	source/android/frameworks/av/media/libmediaplayerservice/nuplayer	负责流程控制
NuPlayerDecoder.cpp	source/android/frameworks/av/media/libmediaplayerservice/nuplayer	负责送入 Au 到解码模块以及从解码模块接收解码后的数据
NuPlayerRenderer.cpp	source/android/frameworks/av/media/libmediaplayerservice/nuplayer	负责 AV 同步
GenericSource.cpp	source/android/frameworks/av/media/libmediaplayerservice/nuplayer	负责向 Extractor 获取文件数据
MediaReciever.java	source/android/packages/providers/mediaprovider/src/com/android/providers/media/	负责接收需要扫描的广播消息，启动 MediaService
MediaService.java	source/android/packages/providers/mediaprovider/src/com/android/providers/media/	触发 MediaScanner 扫描
MediaProviders.java	source/android/packages/providers/mediaprovider/src/com/android/providers/media/	负责扫描过程中有关数据库的相关操作
ModernMediaScanner.java	source/android/packages/providers/mediaprovider/src/com/android/providers/media/scan	负责扫描的整体控制，通过 MediaMetadataRetriever 获取每个文件信息
StagefrightMetadataRetriever.cpp	source/android/frameworks/av/media/libstagefright/	负责从 Extractor 获取文件的详细信息
MMExtractor.cpp	source/android/vendor/autochips/proprietary/frameworks/av/media/libstagefright/mmextractor	负责文件解析
MediaCodec.cpp	source/android/frameworks/av/media/libstagefright/	用于访问底层媒体编解码器，以便开发者更加灵活的处理音

文件名	文件路径	文件描述
		视频的编解码
StagefrightRecoder.cpp	source/android/frameworks/av/media/libmediaplayerservice/	负责 recorder 流程控制
MediaCodecSource.cpp	source/android/frameworks/av/media/libstagefright/	负责编码 source 流程控制
MediaRecorder.cpp	source/android/frameworks/av/media/libmedia/	Media recorder 框架流程
ACodec.cpp	source/android/frameworks/av/media/libstagefright/	对接解码模块

22.1.2.7 动态库

本文档 release 的动态库如表 22-4 所示。

表 22-4 Multimedia 动态库

名称	路径	作用
libatcmultimedia.so	source/android/vendor/autochips/libs/ac8025_demo/media/	用于文件解析
libtswriter.so	source/android/vendor/autochips/libs/ac8025_demo/tswriter/	用于 ts 文件封装
libAtcOmxAdec.so	source/android/vendor/autochips/libs/ac8025_demo/media/	DSP Audio Decoder 对应的 OMX Component
libAtcOmxAdpcmDec.so	source/android/vendor/autochips/libs/ac8025_demo/media/	Adpcm 软解对应的 OMX Component
libAtcOmxApeDec.so	source/android/vendor/autochips/libs/ac8025_demo/media/	Ape 软解对应的 OMX Component
libAtcOmxMp3Dec.so	source/android/vendor/autochips/libs/ac8025_demo/media/	Mp3 软解对应的 OMX Component
libAtcOmxRawDec.so	source/android/vendor/autochips/libs/ac8025_demo/media/	pcm 软解对应的 OMX Component
libstagefrighthw.so	source/android/vendor/autochips/libs/ac8025_demo/media/	Video Component、Audio Component 对应的 plugin
libmmextractor.so	source/android/vendor/autochips/proprietary/frameworks/av/media/libstagefright/mmextractor/	AutoChips 的 extractor 插件，调用 libatcmultimedia.so 实现文件解析功能

名称	路径	作用
libatcomxvdec.so	source/android/vendor/autochips/libs/ac8025_demo/media/	Video decoder 对应的 OMX Component
libatcvdec.so	source/android/vendor/autochips/libs/ac8025_demo/media/	用于 video 解码
libatcomxvenc.so	source/android/vendor/autochips/libs/ac8025_demo/media/	Video encoder 对应的 OMX Component
libatcvenc.so	source/android/vendor/autochips/libs/ac8025_demo/media/	用于 video 编码

22.2 软件设计

22.2.1 架构图

Application Framework 层的模块为 APP 提供接口，Libraries 层的模块实现具体的功能，Device Drivers 层的模块对上提供硬件的驱动。

22.2.1.1 MediaPlayer 框架图

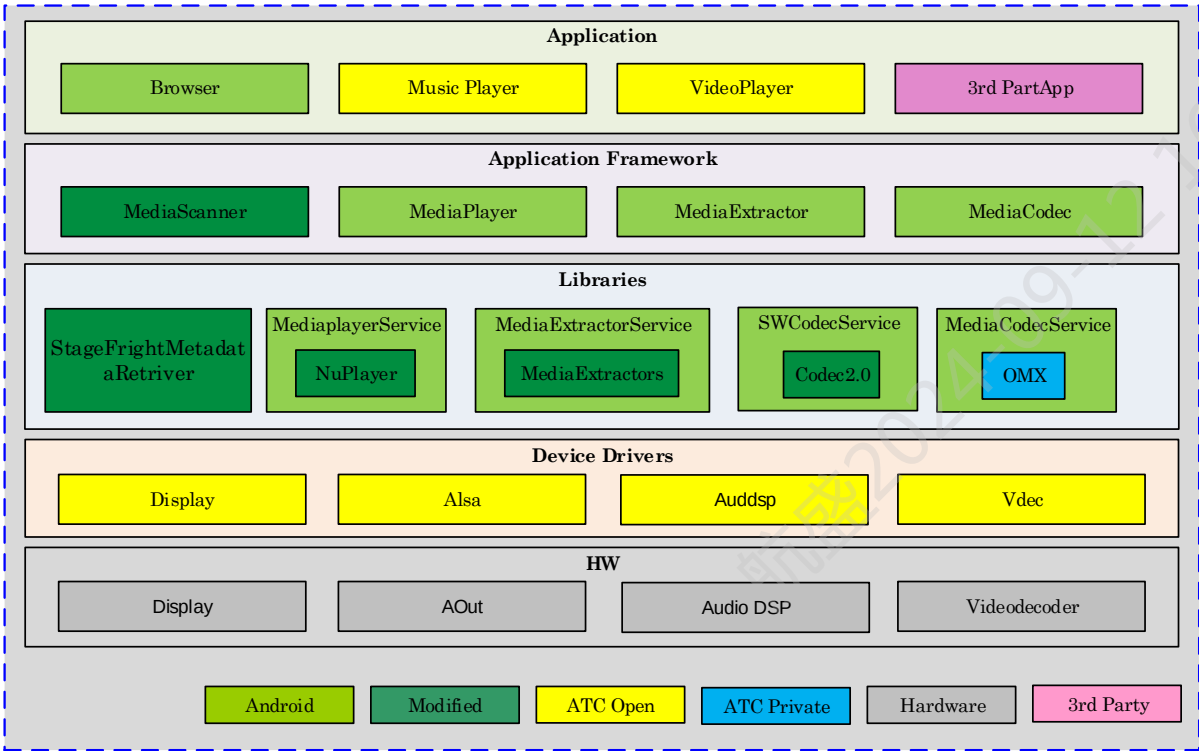


图 22-1 MediaPlayer 架构图

22.2.1.2 MediaRecorder 框架图

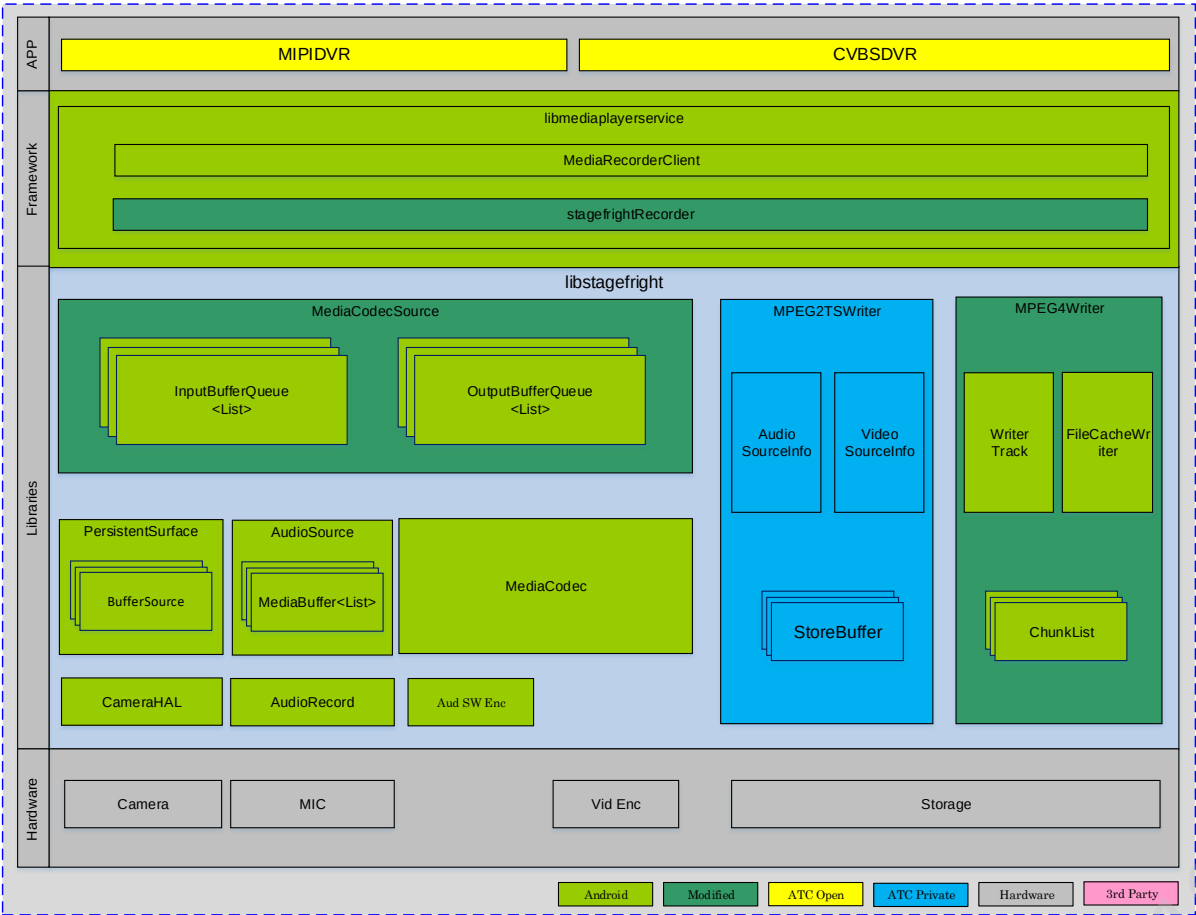


图 22-2 MediaRecorder 框架图

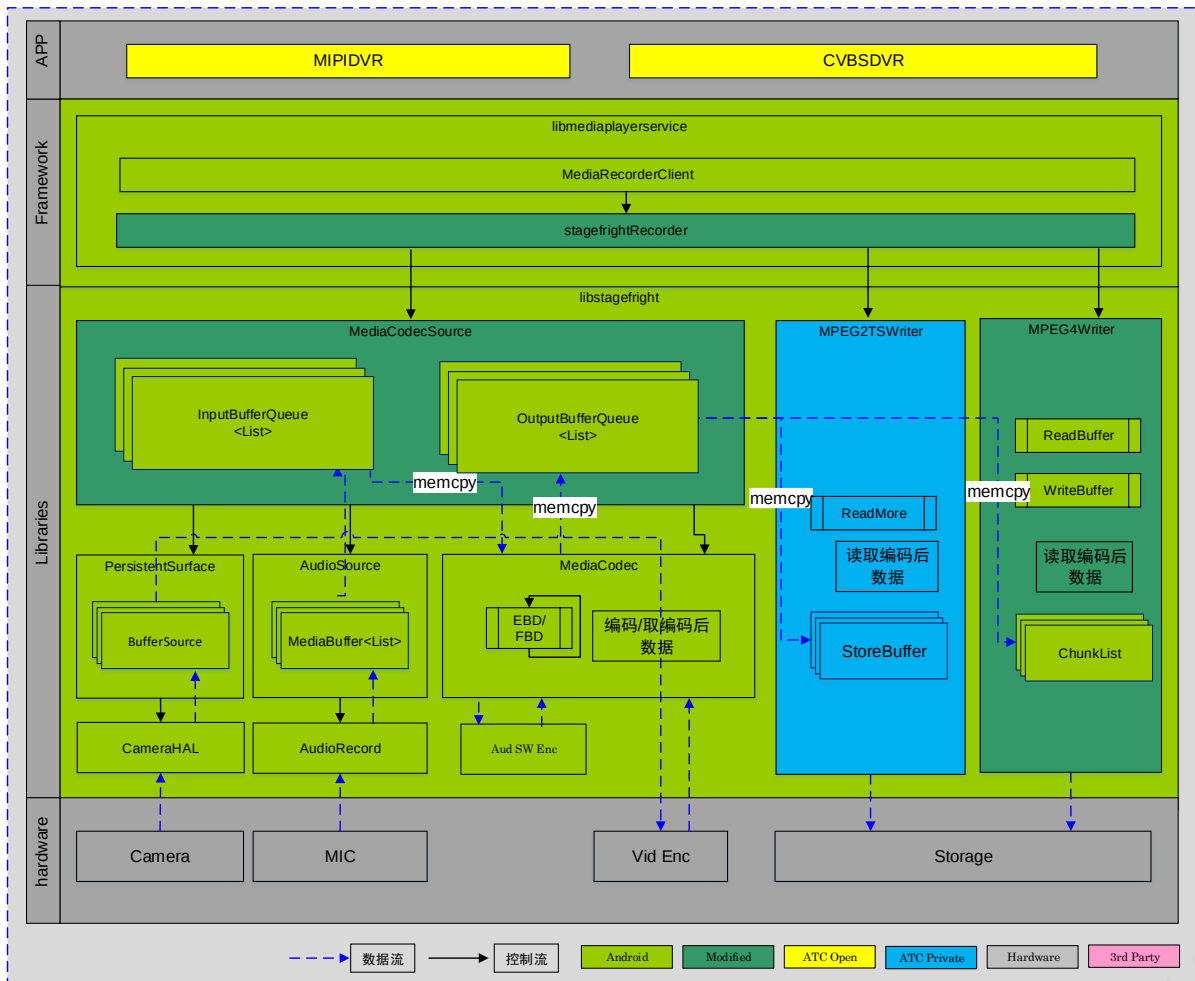


图 22-3 MediaRecorder 数据流控制流图

模块框架介绍如下：

- APP: 初始化 MediaRecorder，设置相关参数，并启动 MediaRecorder，开始/结束录制功能。
- StagefrightRecorder: 主要负责逻辑控制，对上接收 APP 下来的命令，对下控制整个 Recorder 框架流程。
- AudioSource: 提供编码前 Audio 数据。
- PersistentSurface: 保存 camera 抓取的编码前 Video 数据的容器。
- MediaCodecSource: 主要负责接收编码前的数据，并对接编码模块，最终可以为 MPEG2TSWriter 提供编码后的 A/V 数据。
- MediaCodec: 主要用来编码，将 audio 编码为 AAC，将 video 编码为 AVC。
- MPEG2TSWriter: 主要负责将编码后的 A/V 数据打包为 TS 格式，并写入文件。

22.2.1.3 MediaScanner 框架图

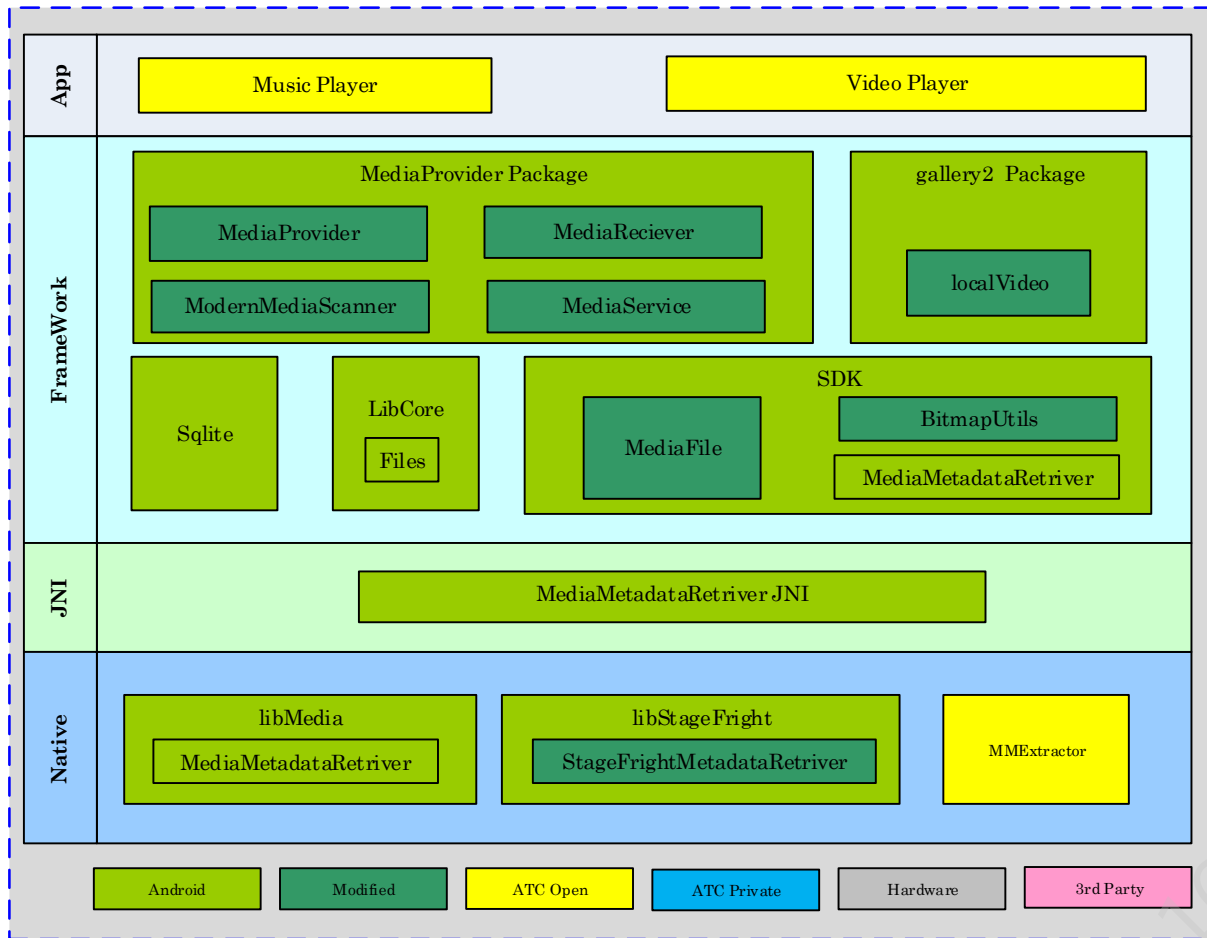


图 22-4 MediaScanner 框架图

模块架构介绍如下：

1. Framework 层：

- **MediaProvider Package:** 扫描及数据库管理模块，负责整个扫描流程及对外提供数据库操作接口
 - **MediaReceiver:** 接收广播（mount 广播、扫描广播），并 start MediaService
 - **MediaService:** 处理广播消息，根据不同的广播调用 ModernMediaScanner 开始扫描动作
 - **ModernMediaScanner:** 处理整个扫描流程
 - **MediaProvider:** 处理与数据库的交互动作，并对外提供了一系列数据库操作接口
- **LibCore、Files 模块:** ModernMediaScanner 中，使用这两个模块来遍历获取卡中的每个文件。

- Sqlite: 数据库模块
 - SDK: 提供 image、audio、video 解析接口
2. Native 层: MediaMetadataRetriever 提供一套公共对外接口，获取文件的基本信息，通过一系列调用最终会使用私有的 MMExtractor 解析 video 和 audio 文件。

22.2.2 流程图

22.2.2.1 MediaRecorder 流程图

22.2.2.1.1 设置编码数据源

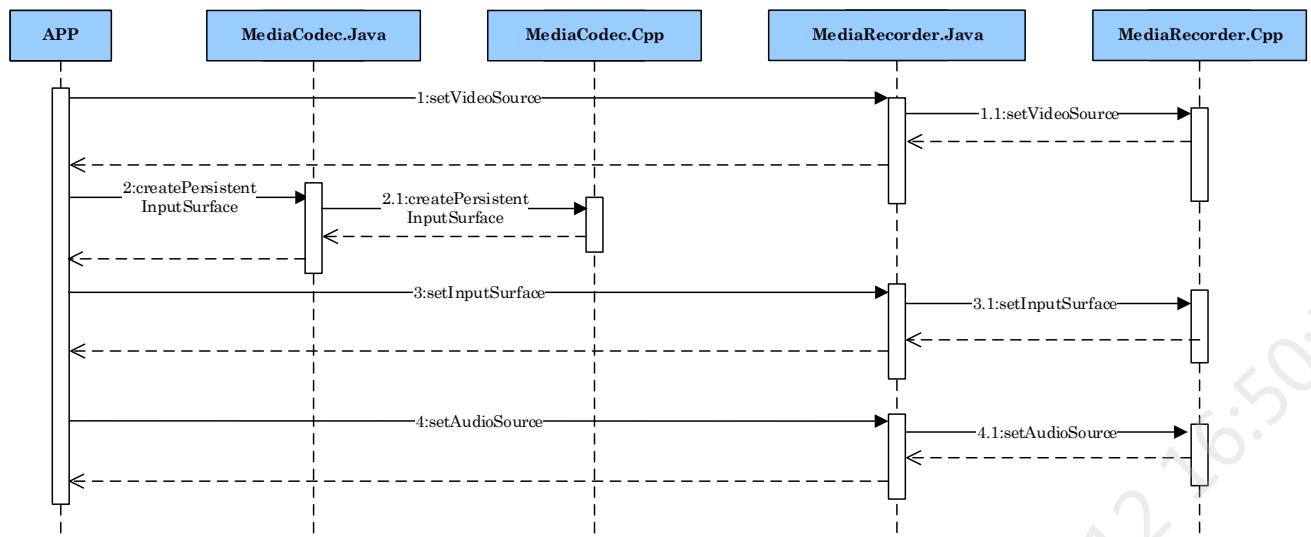


图 22-5 设置编码数据源时序图_Surface 模块

Video 编码数据源的设置方式有三种：

- DEFAULT
- CAMERA
- SUFACE

DEFAULT 与 CAMERA 模式会创建 CameraSource, SUFACE 模式不会创建 CameraSource, 会从 MediaCodec 创建 InputSurface,然后将该 InputSurface 设置给 MediaRecorder。

22.2.2.1.2 设置录制参数

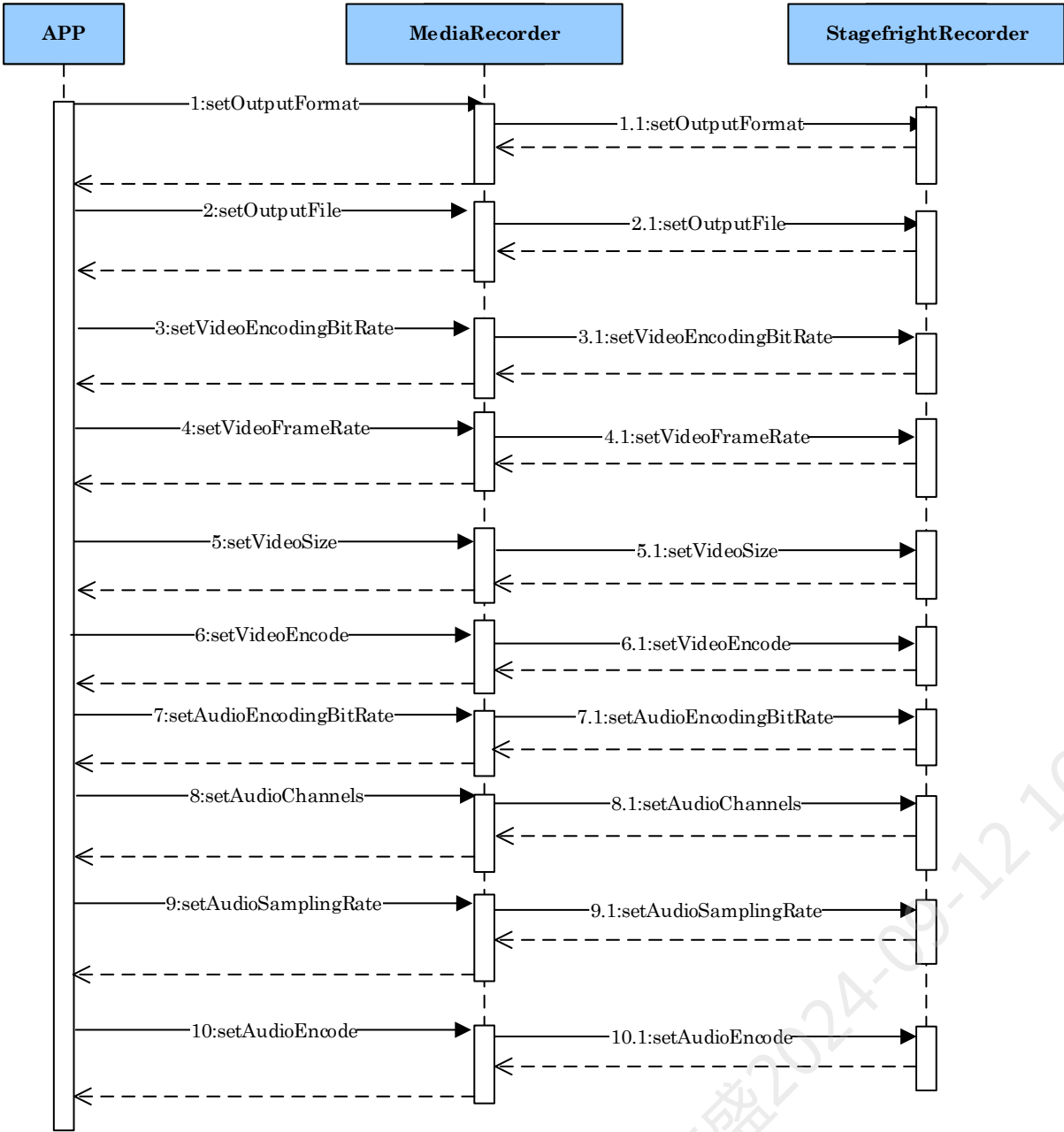


图 22-6 设置录制参数时序图

设置录制时需要的相关信息,具体支持的参数参见表 22-1。

22.2.2.2 MediaScanner 流程图

22.2.2.2.1 消息处理

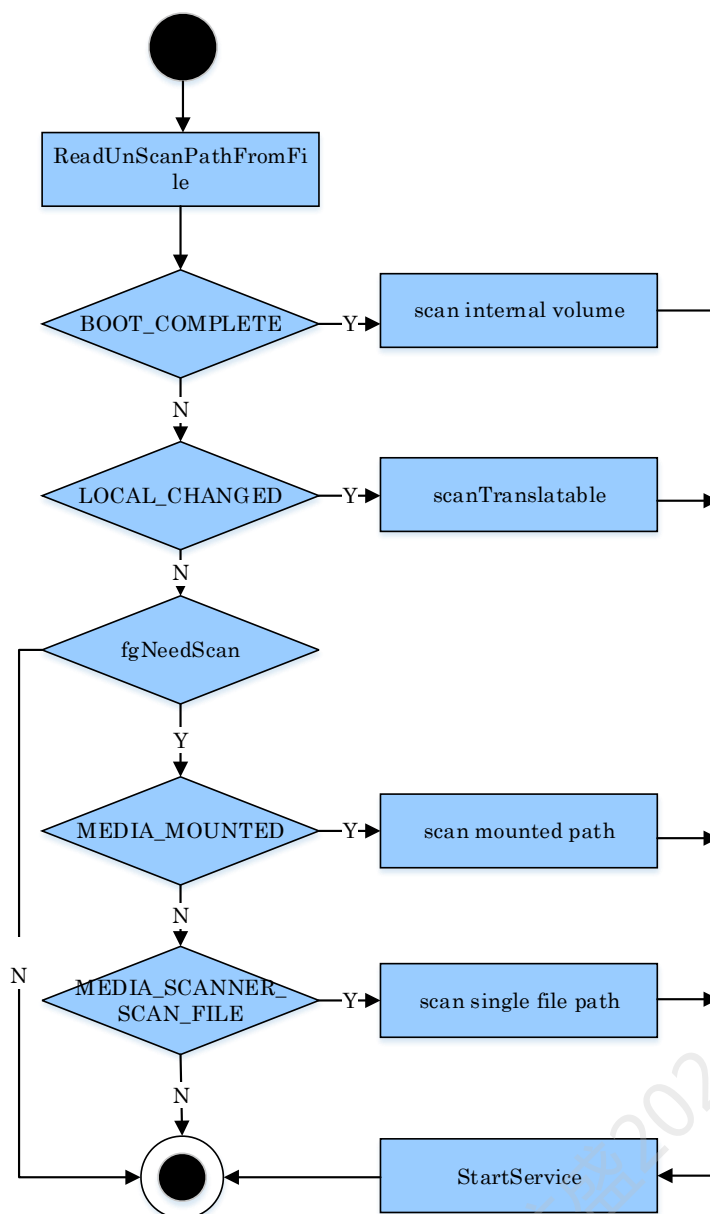


图 22-7 消息处理

22.2.2.2.2 获取 Video Thumbnail

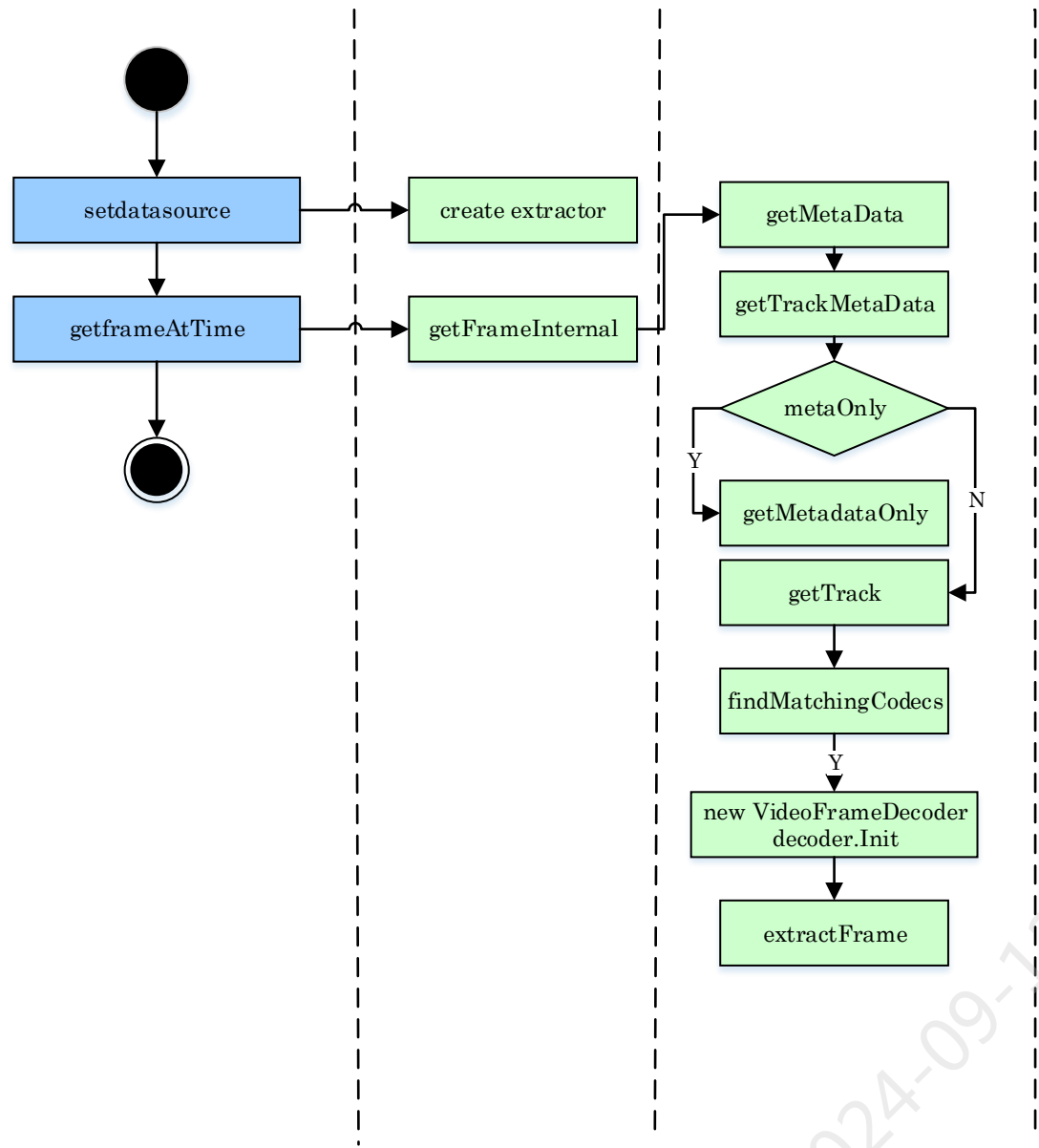


图 22-8 获取 Video Thumbnail

22.3 接口描述

1. MediaPlayer API
<https://developer.android.com/reference/android/media/MediaPlayer>
2. MediaCodec API

<https://developer.android.com/reference/android/media/MediaCodec>

3. MediaRecorder API

<https://developer.android.com/reference/android/media/MediaRecorder>

4. MediaScanner API

<https://developer.android.com/reference/android/media/MediaScannerConnection>

<https://developer.android.com/reference/android/media/MediaMetadataRetriever>

22.4 使用说明和注意事项

22.4.1 示例代码

22.4.1.1 文件播放

22.4.1.1.1 准备播放

```
MediaPlayer mMediaPlayer = new MediaPlayer(); /* 创建播放器 */

private final OnPreparedListener mPreparedListener = new OnPreparedListener() {
    @Override
    public void onPrepared(MediaPlayer mp) {
        /* 异步 Prepare 完成处理代码 */
    }
};

private final OnCompletionListener mCompletedListener = new OnCompletionListener() {
    @Override
    public void onCompletion(MediaPlayer mp) {
        /* 播放结束完成处理代码 */
    }
};

private final OnInfoListener mInfoListener = new OnInfoListener() {
    @Override
    public boolean onInfo(MediaPlayer mp, int what, int extra) {
        /* Info 消息处理代码 */
        return true;
    }
};
```

```
    }  
};  
  
private final OnErrorListener mErrorListener = new OnErrorListener() {  
    @Override  
    public boolean onError(MediaPlayer mp, int what, int extra) {  
        /* Error 消息处理代码 */  
        return true;  
    }  
};  
  
mMediaPlayer.setOnPreparedListener(mPreparedListener); /* 设置 prepared 监听 */  
mMediaPlayer. setOnCompletionListener(mCompletedListener); /* 设置结束监听 */  
mMediaPlayer.setOnInfoListener(mInfoListener); /* 设置信息监听 */  
mMediaPlayer.setOnErrorListener(mErrorListener); /* 设置错误监听 */  
  
String path = "xxx"; /* 文件路径 */  
mMediaPlayer.setDataSource(path); /* 设置文件路径 */  
  
SurfaceView surfaceView = (SurfaceView) findViewById(R.id.surfaceView); /* id.surfaceView 是  
控件的 id*/  
  
mMediaPlayer.setDisplay(surfaceView.getHolder()); /* 设置 video surface ，只有播放 video 时才  
需要*/  
  
mMediaPlayer.setAudioStreamType(AudioManager.STREAM_MUSIC); /* 设置音频流类型 */  
  
mMediaPlayer.prepareAsync (); /* 准备播放 */
```

MediaPlayer 的 prepareAsync 是异步接口，在 MediaPlayer Prepare 完成后，MediaPlayer 会回调到 APP 设置的 mPreparedListener 的 OnPrepared 接口。

22.4.1.1.2 开始播放

```
mMediaPlayer.start(); /* 开始播放 */
```

如果 APP 之前调用的是 prepareAsync，则应在 OnPrepared 被回调后才能调用 MediaPlayer 的 start。

如果 APP 之前调用的是 prepare 接口，在调用完 prepare 后可直接调用 MediaPlayer 的 start。

播放结束，MediaPlayer 会回调 APP 设置的 OnCompletionListener 的 onCompletion 接口。

22.4.1.1.3 暂停播放

```
mMediaPlayer.pause(); /* 暂停播放 */
```

22.4.1.1.4 恢复播放

```
mMediaPlayer.start(); /* 恢复播放 */
```

22.4.1.1.5 Seek 播放位置

```
int seekTimeMs = 100; /* seek time 的单位为 milliseconds */  
mMediaPlayer.seekTo(seekTimeMs);
```

22.4.1.1.6 停止播放

```
mMediaPlayer.reset(); /* 停止播放流程 */
```

22.4.1.1.7 释放资源

```
mMediaPlayer.release(); /* 释放资源 */
```

22.4.1.1.8 获取文件基本播放信息

prepare 成功后即可获取文件基本信息。

```
int durationMSec = mMediaPlayer.getDuration(); /*获取 Duration, 单位 milliseconds */  
  
int positionMSec = mMediaPlayer.getCurrentPosition(); /* 获取当前的 Position, 单位  
milliseconds */  
  
TrackInfo trackInfo[] = mMediaPlayer.getTrackInfo(); /* 获取 Tracks 信息 */  
for (int i = 0; i < trackInfo.length; i++) {  
    if (trackInfo[i].getTrackType() == TrackInfo.MEDIA_TRACK_TYPE_TIMEDTEXT) {  
        /* 该 Track 为 TimedText */  
    } else if (trackInfo[i].getTrackType() == TrackInfo.MEDIA_TRACK_TYPE_AUDIO) {  
        /* 该 Track 为 Audio */  
    } else if (trackInfo[i].getTrackType() == TrackInfo.MEDIA_TRACK_TYPE_VIDEO) {  
        /* 该 Track 为 Video */  
    }  
}
```

```
}  
}
```

22.4.1.2 文件录制

22.4.1.2.1 准备录制

```
MediaRecorder mRecorder = new MediaRecorder(); /* 创建 MediaRecorder */  
  
private MediaRecorder.OnErrorListener mRecorderErrorListener = new  
MediaRecorder.OnErrorListener() {  
    @Override  
    public void onError(MediaRecorder mr, int what, int extra) {  
        /* Error 消息处理代码 */  
    }  
};  
  
private MediaRecorder.OnInfoListener mRecorderInfoListener = new  
MediaRecorder.OnInfoListener() {  
    @Override  
    public void onInfo(MediaRecorder mr, int what, int extra) {  
        /* Info 消息处理代码 */  
    }  
};  
  
mRecorder.setOnInfoListener(mRecorderInfoListener); /* 设置信息监听 */  
mRecorder.setOnErrorListener(mRecorderErrorListener); /* 设置错误监听 */  
  
/* 设置 Video 编码数据源 */  
mRecorder.setVideoSource(MediaRecorder.VideoSource.SURFACE);  
Surface mRecorderSurface = MediaCodec.createPersistentInputSurface();  
mRecorder.setInputSurface(mRecorderSurface);  
  
/* 设置 Audio 编码数据源 */  
mRecorder.setAudioSource(MediaRecorder.AudioSource.MIC);  
  
/* 设置录制的文件格式 */  
mRecorder.setOutputFormat(MediaRecorder.OutputFormat.MPEG_2_TS);
```

```
/*设置录制的文件*/
```

```
File file = xxx;
```

```
mRecorder.setOutputFile(file);
```

```
mRecorder.setVideoEncodingBitRate(bitrate); /*设置 Video 编码的 bitrate*/
```

```
mRecorder.setVideoFrameRate(framerate); /*设置 Video 编码的 framerate*/
```

```
mRecorder.setVideoSize(width, height); /*设置 Video 编码的宽高信息*/
```

```
mRecorder.setVideoEncoder(videocodec); /*设置 Video 编码的 codec*/
```

```
mRecorder.setAudioEncodingBitRate(bitrate); /*设置 Audio 编码的 bitrate*/
```

```
mRecorder.setAudioChannels(channels); /*设置 Audio 编码的 channels*/
```

```
mRecorder.setAudioSamplingRate(samplerate); /*设置 Audio 编码的 sample rate*/
```

```
mRecorder.setAudioEncoder(audiocodec); /*设置 Audio 编码 codec*/
```

```
mRecorder.prepare();
```

当前录制写文件时，只会调用 `fwrite()` 进行写文件，并没有将数据同步到存储设备上，该情况将会导致断电或者拔卡时丢数据，最大为 5s，因为 `storage` 每隔 5s 会将数据同步到存储设备上。

如果录制时需要将数据同步到存储设备上，可通过系统属性设置：

```
SystemProperties.set("persist.sys.media.recorder.setsync.storage",  
    String.valueOf(1));
```

设置过该属性，数据写存储设备时会同时进行同步操作，该情况断电或者拔卡时，丢失数据最大为 500ms，因为 500ms 写一次卡。

去除该属性设置：

```
SystemProperties.set("persist.sys.media.recorder.setsync.storage",  
    String.valueOf(0));
```

22.4.1.2.2 开始录制

```
mRecorder.start();
```

22.4.1.2.3 结束录制

```
mRecorder.stop();
```

```
mRecorder.reset();
```

```
mRecorder.release();
```

22.4.1.3 使用 MediaCodec 编解码

22.4.1.3.1 创建编码/解码器

```
MediaCodec codec = MediaCodec.createByCodecName(name);  
MediaFormat mOutputFormat; // member variable
```

22.4.1.3.2 设置 event 异步回调

```
codec.setCallback(new MediaCodec.Callback() {  
    @Override  
    void onInputBufferAvailable(MediaCodec mc, int inputBufferId) {  
        ByteBuffer inputBuffer = codec.getInputBuffer(inputBufferId);  
        // fill inputBuffer with valid data  
        ...  
        codec.queueInputBuffer(inputBufferId, ...);  
    }  
  
    @Override  
    void onOutputBufferAvailable(MediaCodec mc, int outputBufferId, ...) {  
        ByteBuffer outputBuffer = codec.getOutputBuffer(outputBufferId);  
        MediaFormat bufferFormat = codec.getOutputFormat(outputBufferId); // option A  
        // bufferFormat is equivalent to mOutputFormat  
        // outputBuffer is ready to be processed or rendered.  
        ...  
        codec.releaseOutputBuffer(outputBufferId, ...);  
    }  
  
    @Override  
    void onOutputFormatChanged(MediaCodec mc, MediaFormat format) {  
        // Subsequent data will conform to new format.  
        // Can ignore if using getOutputFormat(outputBufferId)  
        mOutputFormat = format; // option B  
    }  
});
```

```

    }

    @Override
    void onError(...) {
        /* Error 消息处理代码 */
    }
});

```

22.4.1.3.3 编解码流程

```

codec.configure(format, ...);
//启动编解码，开始数据的循环处理
codec.start();
// wait for processing to complete
//停止编解码
codec.stop();
//释放资源
codec.release();

```

MediaCodec 提供了同步和异步两种数据处理方式，上面示例使用的是异步处理方式，推荐使用这种处理方式；更多 MediaCodec 介绍参考：

<https://developer.android.com/reference/android/media/MediaCodec>

22.4.1.3.4 平台支持的 Codec 信息

本平台支持的 Codec 见下表，当本平台硬件（HW）与 Google 原生（SW）同时支持某一 Codec 时，对于 Video 会优先选择平台硬件进行编解码，对于 Audio 可以通过 setting 设置 HW 和 SW 的优先级。

Audio 设置解码方式的方法：在 CarSetting->More->Audio Dec Mode 中点击切换 Audio 解码方式，默认是 CPU Decoder（ARM Audio）。

表 22-5 MediaCodec 编解码支持情况

Mime Type	Component Name	SW/HW	Decoder/Encoder
video/mpeg2	OMX.hantro.VC9000D.video.decoder.mpeg2	HW	decoder
video/hevc	OMX.hantro.VC9000D.video.decoder.hevc	HW	decoder
video/mp4v-es	OMX.hantro.VC9000D.video.decoder.mpeg4	HW	decoder

Mime Type	Component Name	SW/HW	Decoder/Encoder
video/3gpp	OMX.hantro.VC9000D.video.decoder.h263	HW	decoder
video/avc	OMX.hantro.VC9000D.video.decoder.avc	HW	decoder
video/rm	OMX.hantro.VC9000D.video.decoder.rm	HW	decoder
video/x-vnd.on2.vp8	OMX.hantro.VC9000D.video.decoder.vp8	HW	decoder
video/x-vnd.on2.vp9	OMX.hantro.VC9000D.video.decoder.vp9	HW	decoder
video/vc1	OMX.hantro.VC9000D.video.decoder.vc1	HW	decoder
video/wmv3	OMX.hantro.VC9000D.video.decoder.wmv3	HW	decoder
video/x-motion-jpeg	OMX.hantro.VC9000D.video.decoder.mjpeg	HW	decoder
video/flv1	OMX.hantro.VC9000D.video.decoder.s263	HW	decoder
video/divx3	OMX.hantro.VC9000D.video.decoder.divx3	HW	decoder
video/divx	OMX.hantro.VC9000D.video.decoder.divx	HW	decoder
video/avc	OMX.hantro.VC8000E.video.encoder.avc	HW	encoder
video/hevc	OMX.hantro.VC8000E.video.encoder.hevc	HW	encoder
audio/mpeg	OMX.atc.audio.decoder.mp3	HW	decoder
	OMX.ATC.AUDIO.SWDECODER.MP3	SW	decoder
audio/raw	OMX.atc.audio.decoder.raw	HW	decoder
	OMX.ATC.AUDIO.SWDECODER.RAW	SW	decoder
audio/x-adpcm-ms	OMX.atc.audio.decoder.adpcm.ms	HW	decoder
	OMX.ATC.AUDIO.SWDECODER.ADPCM.MS	SW	decoder
audio/x-adpcm-dvi-ima	OMX.atc.audio.decoder.adpcm.dvi	HW	decoder
	OMX.ATC.AUDIO.SWDECODER.ADPCM.DVI	SW	decoder
audio/ape	OMX.atc.audio.decoder.ape	HW	decoder
	OMX.ATC.AUDIO.SWDECODER.APE	SW	decoder
audio/vorbis	OMX.atc.audio.decoder.vorbis	HW	decoder
audio/flac	OMX.atc.audio.decoder.flac	HW	decoder
audio/mp4a-latm	OMX.atc.audio.decoder.aac	HW	decoder
audio/cook	OMX.atc.audio.decoder.cook	HW	decoder
audio/mp2	OMX.atc.audio.decoder.mp2	HW	decoder
audio/opus	OMX.atc.audio.decoder.opus	HW	decoder

22.5 典型问题

22.5.1 Debug 方法

1. Nuplayer 框架调试方法

1) 动态打开 NuPlayer 播放流程的 Log

```
adb root
adb shell dumpsys media.player -logv
```

2) dump 当前播放状态

```
adb root
adb shell dumpsys media.player>state.txt
```

2. ACodec 框架调试方法

1) 打开 ACodec audio buffer 的 log

```
adb root
adb shell setprop acodec.audio.buflogging 1
```

2) dump audio 解码的 input buffer

```
adb root
adb shell setenforce 0
adb shell setprop acodec.audio.bstrdump 1
```

Dump 位置:

/data/media/ACodec.ComponentName.HHMM, 其中 ComponentName 指音频对应的 OMX Component, 比如 OMX.ATC.AUDIO.SWDECODER.MP2;HHMM 表示 Dump 的时间

3) dump audio 解码的 output buffer

```
adb root
adb shell setenforce 0
adb shell setprop acodec.audio.rawdump 1
```

Dump 位置:

/data/media/ACodecRaw.HHMMSS.PCM, 其中 HHMMSS 表示 Dump 的时间

4) 打开 ACodec video buffer 的 log

```
adb root
adb shell setprop acodec.video.buflogging 1
```

5) dump video 解码的 input buffer

```
adb root
adb shell setenforce 0
adb shell setprop acodec.video.bstrdump 1
```

Dump 位置:

/data/media/ACodec.ComponentName.HHMM, 其中 ComponentName 指视频对应的 OMX Component, 比如 OMX.hantro.VC8000E.video.encoder.avc;HHMM 表示时间

6) dump video 解码的 output buffer

```
adb root
adb shell setenforce 0
adb shell setprop acodec.video.rawdump 1
```

Dump 位置:

/data/media/ACodecRaw.HHMMSS, 其中 HHMMSS 表示 Dump 的时间

3. 从 OMX dump Video 解码前和解码后的数据

1) Dump Video 解码前的数据

```
adb root
adb shell setenforce 0
echo omx.dump.vdec.inputdata 1 > /dev/dbg
```

Dump 位置:

/data/media/xxx.input, xxx 随 dump 的 decoder 的时间的不同而不同。

2) Dump Video 解码后的数据

```
adb root
adb shell setenforce 0
echo omx.dump.vdec.outputdata 1 > /dev/dbg
```

Dump 结果:

/data/media/xxx.yuv, xxx 随 dump 的时间不同而不同。

3) 动态打开 OMX 编解码 Log

```
adb root
adb shell setenforce 0
adb shell
```

```
echo omx.log.vdec loglevel > /dev/dbg //设置 omx 解码 log 等级, 其中 loglevel 为
debug/info/warn/error/fatal

echo omx.log.venc loglevel > /dev/dbg //设置 omx 编码 log 等级, 其中 loglevel 如
下

debug/d/1
info/i/2
warn/w/3
error/e/4
fatal/f/5
```

4) 动态打开 Codec 模块 Log

```
adb root
adb shell setenforce 0
adb shell

echo vdec.log.sdk loglevel > /dev/dbg //设置 vdec 模块 sdk 的 log 等级

echo venc.log.sdk loglevel > /dev/dbg //设置 venc 模块 sdk 的 log 等级, 其中
loglevel 如下

debug/d/1
info/i/2
warn/w/3
error/e/4
fatal/f/5
```

4. Dump Audio 的输出状态

```
adb root
adb shell dumpsys media.audio_flinger
```

5. Dump MediaPlayer 的播放状态

```
adb root
adb shell dumpsys media.player
```

6. 检查 Mediascanner 扫描结果

扫描开始日志:

MediaScannerService: start scanning directories:/storage/6EEA-F3D8

扫描结束及扫描时间日志:

MediaScannerService: done scanning directories:/storage/6EEA-F3D8, cost:245ms

导出数据库:

```
adb root
```

```
adb pull /data/user_de/0/com.android.providers.media.module/databases
```

利用数据库工具查看导出的 db 文件。

7. 动态修改 MediaScanner 最大支持的文件扫描个数

```
adb root
```

```
adb shell setprop "persist.mediascanner.video.cnt" 50//设置 mediascanner 最大支持 video 扫描个数为 50，设置的值超过 500 不生效
```

```
adb shell setprop "persist.mediascanner.image.cnt" 50//设置 mediascanner 最大支持 image 扫描个数为 50，设置的值超过 500 不生效
```

```
adb shell setprop "persist.mediascanner.audio.cnt" 50//设置 mediascanner 最大支持 audio 扫描个数为 50，设置的值超过 500 不生效
```

```
adb reboot//重启后设置生效
```

8. 调整 VideoThumbnail 解码方式

```
adb root
```

```
adb shell setprop "media.stagefright.thumbnail.prefer_hw_codecs" true/false
```

true: 硬件优先（默认） false: 软解优先

22.5.2 常见问题分析

22.5.2.1 视频花屏、黑屏和卡顿问题

【问题描述】

视频花屏、黑屏和卡顿。

【解决方案】

1. Dump 解码前和解码后的数据;
2. 检查解码前和解码后数据是否正常:
 - 1) 如果解码前数据异常, 则进一步 Check 文件本身是否正常。
 - 2) 如果解码前正常, 解码后异常, 则是解码有问题。

22.5.2.2 音频卡顿

【问题描述】

音频卡顿。

【解决方案】

- 不断地 Dump Audio 的输出状态。
- 搜索“underrun”，查看下面的数值：
 - 如果数值在增大，说明 Audio 数据跟不上，则进一步确认 audio 的写入速率。
 - 如果数值没有增大，则是 Audio 输出有问题或者 audio 数据有问题。

22.5.2.3 播放停止或退出

【问题描述】

播放停止或退出。

【解决方案】

- Log 中搜索“MediaPlayer”，查看是否有 Error。

```
2742 3138 E MediaPlayerService: [0xece35f80] setVideoSurfaceTexture failed: -22
3587 3665 I audioWhiteList: [whiteList] procName:com.autochips.testMediaPlayer
3587 3665 I audioWhiteList: [whiteList] process name is com.autochips.testMediaPlayer, ret 0 [1:GIS,0:other]
3587 3665 E MediaPlayer-JNI: call MediaPlayer error:-38, (java/io/IOException, Prepare failed.)
```

- Log 中搜索“crash”和“runtime”，查看是否有 NE 和 JE。

```
----- beginning of crash
01-01 01:30:53.926 2564 6908 F libc : Fatal signal 6 (SIGABRT), code -6 (SI_TRILL) in tid 6908 (generic), pid 2564 (mediaserver)
01-01 01:30:53.971 6962 6962 I crash_dump32: obtaining output fd from tombstoned, type: kDebuggerdTombstone
01-01 01:30:53.972 1819 1819 I /system/bin/tombstoned: received crash request for pid 6908
01-01 01:30:53.972 6962 6962 I crash_dump32: performing dump of process 2564 (target tid = 6908)
01-01 01:30:53.982 6962 6962 F DEBUG : *** *** *** *** *** *** *** *** *** ***
01-01 01:30:53.982 6962 6962 F DEBUG : Build fingerprint: 'autochips/full_ac8x_car/ac8x_car:9/PPR1.180610.011/1615160097:userdebug/test-keys'
01-01 01:30:53.982 6962 6962 F DEBUG : Revision: '0'
01-01 01:30:53.982 6962 6962 F DEBUG : ABI: 'arm'
01-01 01:30:53.982 6962 6962 F DEBUG : pid: 2564, tid: 6908, name: generic >>> /system/bin/mediaserver <<<
01-01 01:30:53.982 6962 6962 F DEBUG : signal 6 (SIGABRT), code -6 (SI_TRILL), fault addr: -----
01-01 01:30:53.982 6962 6962 F DEBUG : Abort message: 'frameworks/av/media/libstagefright/mpeg2ts/AnotherPacketSource.cpp:342 CHECK(mLatestEnqueuedMeta->findint64("time
01-01 01:30:53.983 6962 6962 F DEBUG : r0 00000000 r1 00001afc r2 00000006 r3 00000008
01-01 01:30:53.983 6962 6962 F DEBUG : r4 00000a04 r5 00001afc r6 ef203264 r7 0000010c
01-01 01:30:53.983 6962 6962 F DEBUG : r8 efd1a23c r9 00000000 r10 00000002 r11 00000000
01-01 01:30:53.983 6962 6962 F DEBUG : ip 00000073 sp ef203250 lr f386e69 pc f3860cb6
01-01 01:30:54.007 6962 6962 F DEBUG :
01-01 01:30:54.007 6962 6962 F DEBUG : backtrace:
01-01 01:30:54.007 6962 6962 F DEBUG : #00 pc 0001ccb6 /system/lib/libc.so (abort+58)
01-01 01:30:54.007 6962 6962 F DEBUG : #01 pc 00006ecd /system/lib/liblog.so (_android_log_assert+156)
01-01 01:30:54.007 6962 6962 F DEBUG : #02 pc 0002a777 /system/lib/libstagefright_httplive.so (android::AnotherPacketSource::queueAccessUnit(android::sp<android::AF
01-01 01:30:54.007 6962 6962 F DEBUG : #03 pc 00070b3f /system/lib/libmediaplayerservice.so (android::NuPlayer::GenericSource::readBuffer(android::media_track_type,
01-01 01:30:54.007 6962 6962 F DEBUG : #04 pc 0006fc5f /system/lib/libmediaplayerservice.so (android::NuPlayer::GenericSource::onMessageReceived(android::sp<android
01-01 01:30:54.007 6962 6962 F DEBUG : #05 pc 0000d2b5 /system/lib/libstagefright_foundation.so (android::AHandler::deliverMessage(android::sp<android::AMessage> cc
01-01 01:30:54.007 6962 6962 F DEBUG : #06 pc 0000fb15 /system/lib/libstagefright_foundation.so (android::AHandler::deliverMessage(android::sp<android::AMessage> cc
01-01 01:30:54.007 6962 6962 F DEBUG : #07 pc 0000df59 /system/lib/libstagefright_foundation.so (android::AHandler::deliverMessage(android::sp<android::AMessage> cc
01-01 01:30:54.007 6962 6962 F DEBUG : #08 pc 0000c107 /system/lib/libutils.so (android::Thread::threadLoop(void*)+290)
01-01 01:30:54.007 6962 6962 F DEBUG : #09 pc 000632e5 /system/lib/libc.so (_pthread_start(void*)+22)
01-01 01:30:54.007 6962 6962 F DEBUG : #10 pc 0001de79 /system/lib/libc.so (__start_thread+24)
```

- Log 中搜索“kill”和“die”，看是否有进程死掉。

```
1787 1823 W libprocessgroup: kill(-2340, 9) failed: No such process
1787 1823 W libprocessgroup: kill(-2340, 9) failed: No such process
1787 1823 I libprocessgroup: Successfully killed process cgroup uid 1000 pid 2340 in 44ms
1787 1823 W libprocessgroup: kill(-3652, 9) failed: No such process
1787 1823 W libprocessgroup: kill(-3652, 9) failed: No such process
1787 1823 W libprocessgroup: kill(-3652, 9) failed: No such process
```

4. Log 中搜索“anr”，查看是否有 ANR。

```
AEE      : ExceptionLog: notify aed, process:com.autochips.testMediaPlayer pid:3652 cause:data_app_anr
AEE/DAEMON64: Exception 'ANR' raised
AEE/DAEMON64: Dumping EXP/ANR
AEE/DAEMON64:   Got data:ANR
AEE/DAEMON64: ANR
AEE/DAEMON64:   Type:data_app_anr
AEE/DAEMON64: data_app_anr
AEE/DAEMON64: anr filepath: /data/anr/anr_2020-09-16-04-18-14-262
AEE/DAEMON64: anr filepath: /data/anr/anr_2020-09-16-04-18-14-262
AEE/DAEMON64: anr filepath: /data/anr/anr_2020-09-16-04-18-14-262
AEE/DAEMON64: aed_report_dumpstate: filepath /data/aee_exp/temp/db.xKi8P3, command is /system/bin/aee_dum
AEE/DAEMON64: aed_report_dumpstate: filepath /data/aee_exp/temp/db.xKi8P3, command is /system/bin/aee_dum
AEE/DUMPSTATE: dump prv anr file: (null).
```

5. Dump MediaPlayer 的播放状态，搜索'executingMsg'，查看其后的数值：

- 1) 如果数值不为 0，说明卡在 MediaPlayer 的消息处理过程中了。
- 2) 如果数值为 0，需要根据 Log 进行具体分析。

```
--NuPlayer:
state(5), atEOS(0), looping(0), autoLoop(0)
path(/storage/udisk3/new_Xvid_Simple@L0_1920x1080_8717Kbps_30p_MP3.avi)

generic:
durationUs(90144000), audioTimeUs(4584000), videoTimeUs(-1)
executingMsg(0), executingLine(1834), httpBufferingPercent(0)

renderer:
paused(0), anchorTimeMediaUs(3072000), videoLateByUs(-21248)
numFramesWritten(147488), audioPendingNum(0)
executingMsg(0), executingLine(1306)
audioQueueSize(4), videoQueueSize(3)

mime(video/mp4v-es):
decoder(OMX.ARM.video.v5xx.decoder.MPEG4)
resolution(1920 x 1080)
numFramesTotal(88), numFramesDropped(0), percentageDropped(0.00%)
numInputFramesDropped(0)
executingMsg(0), executingMsgCbID(-1), executingLine(1465), paused(0)
auRequested(97), auFilled(97), frameDecoded(88), frameReleased(85), frameDiscarded(0)
inputTimeUs(-1), outputTimeUs(2899971), renderTimeUs(2799972)

mime(audio/mpeg):
decoder(OMX.ATC.AUDIO.SWDECODER.MP3)
executingMsg(0), executingMsgCbID(-1), executingLine(1465), paused(0)
auRequested(136), auFilled(136), frameDecoded(132), frameReleased(128), frameDiscarded(0)
inputTimeUs(3240000), outputTimeUs(3144000), renderTimeUs(-1)
```

22.5.2.4 支持的后缀文件扫描不出来

【问题描述】

支持的后缀文件扫描不出来。

【解决方案】

Check source/android/frameworks/base/mime/java-res/android.mime.type、vendor.mime.type 文件中的后缀映射是否包含。

22.5.2.5 ID3 乱码

【问题描述】

ID3 字符串显示乱码。

【解决方案】

目前 ID3 信息的解析默认采用 google 原生的方式，首先获取这个文件的所有 ID3 信息，将这些 ID3 信息的字符串连接，成为一个长的字符串，再将这个长的字符串用于解析获取当前字符串的编码方式。字符串长度越长，解析的正确率越高。当文件的 ID3 信息较少时，解析字符串编码方式的正确率会降低，可能会导致编码方式识别错误，从而导致 ID3 乱码。

支持通过设置系统属性 `persist.system.codepage` 来指定 ID3 使用何种 `codepage` 来解析。

23 VBA

23.1 简介

23.1.1 软硬件环境

23.1.1.1 软件环境

本章所述快速开机动画（VBA）功能，运行在基于 FreeRTOS 的 Vehicle 上。

23.1.1.2 硬件环境

VC9000D 解码器。

23.1.2 功能

系统开机自动运行快速开机动画。快速开机动画使用视频硬件解码器进行视频解码，调用显示接口进行全屏显示，调用Audio 接口进行音频输出。

启动过程中会检查datazone 文件：

1. datazone中没有VBA 资源，无法显示开机画面。
2. datazone中有VBA 资源，且boot mode 为normal启动，VBA 资源为图片，则进入快速开机图片播放流程。
3. datazone中有VBA 资源，且boot mode 为normal启动，VBA 资源为视频，则进入快速开机动画播放流程。
4. datazone中有VBA 资源，且boot mode 为recovery启动，则进入快速开机图片播放流程。
5. datazone中有VBA 资源，且boot mode 为升级模式，则显示升级界面。

交互设计：

开机动画播放期间存在与快速倒车的交互，以及启动完成后与Android系统的交互。快速开机动画支持多屏显示。Android系统负责通知开机动画退出，当开机动画退出时，系统各屏幕分别进入相应的界面。

表 23-1 VBA 交互行为表

交互的应用	与快速开机动画交互说明	备注
快速倒车	1. 正常启动，则进入开机动画；	快速倒车包含音频

交互的应用	与快速开机动画交互说明	备注
	2. 启动倒车，显示快速倒车界面，播放倒车提示音； 3. 关闭倒车 – 开机动画未结束，则继续播放开机动画视频和声音 – 开机动画已结束，直接进入安卓系统	

23.1.3 代码结构及文件描述

表 23-2 VBA 代码结构及描述

文件名	文件路径	文件描述
animation.h	source /vendor/autochips/proprietary/tinysys/hsm/os/include	VBA 功能接口
animation_main.c	source /vendor/autochips/proprietary/tinysys/hsm/os/apps/vba	VBA 主体控制播放和响应交互流程

23.2 软件设计

23.2.1 架构图

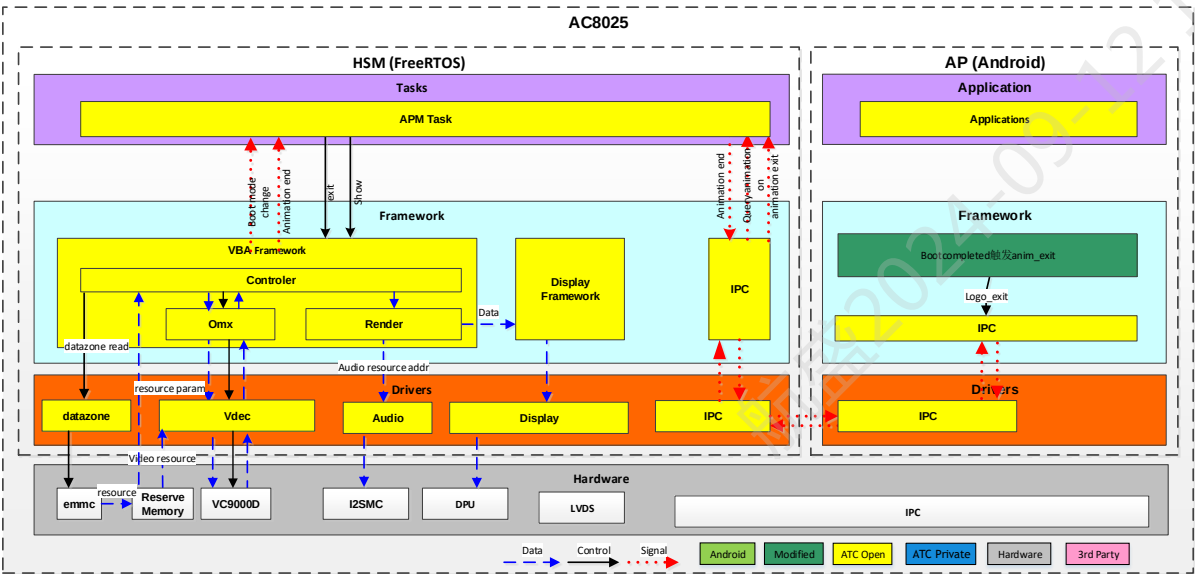


图 23-1 IVI 系统 VBA 架构图

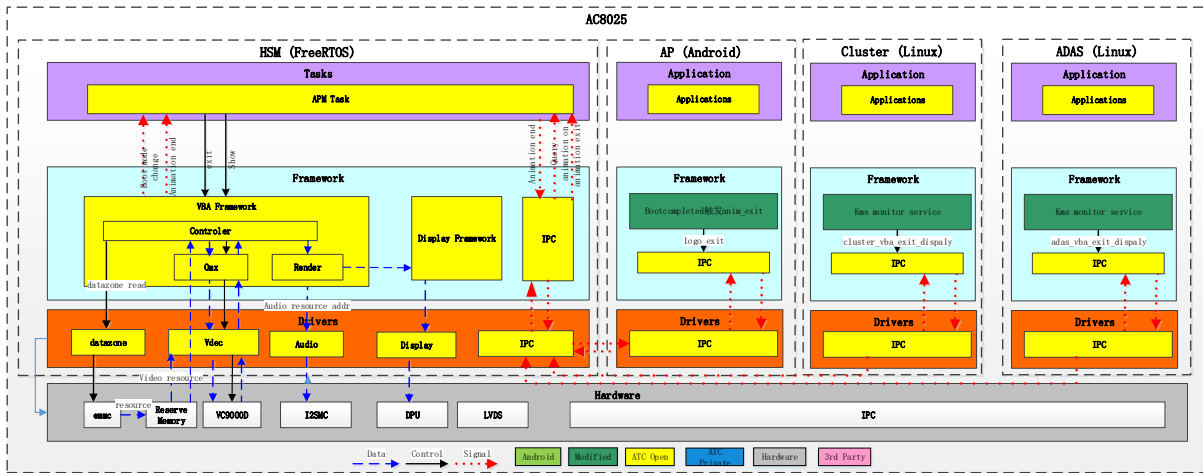


图 23-2 cockpit 系统 VBA 架构图

23.2.2 流程图

23.2.2.1 VBA 的启动时机

VBA 在 FreeRTOS 中是一个 task，HSM 系统初始化完成后，调用 xAPMInit() 函数，xAPMInit() 创建 vApmInitTaskFunction 任务。vApmInitTaskFunction 中进行 VBA 初始化，其会查询 boot mode，若是 normal 启动模式或者 recovery 模式则进入开机动画，若是其他 upgrade 模式则会进入相应的 Logo。

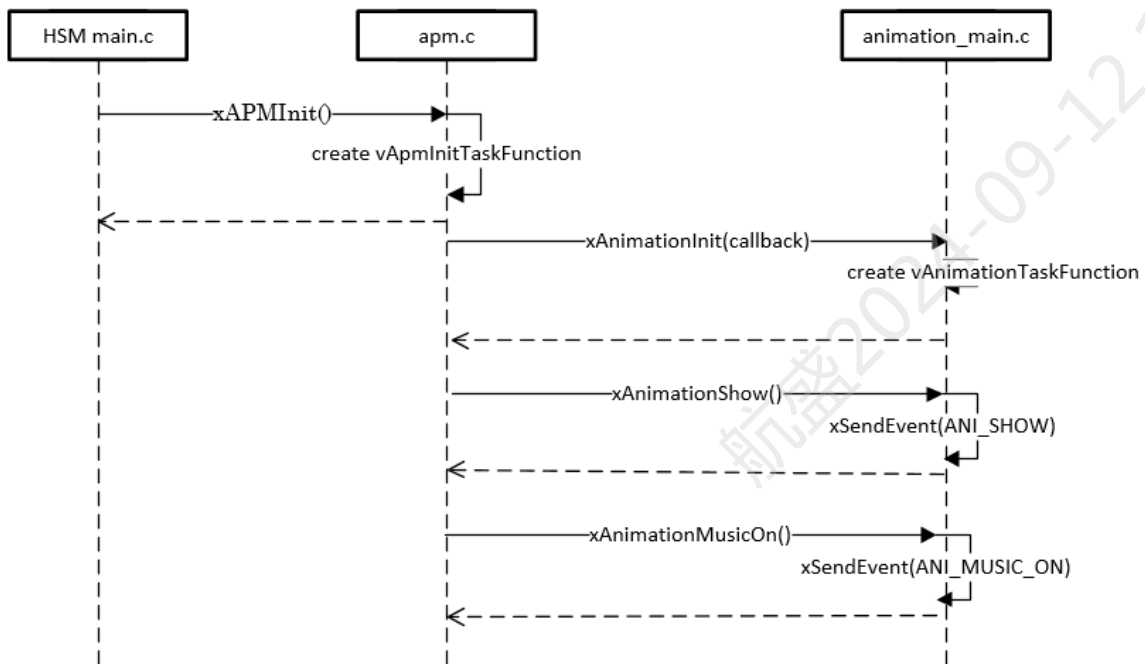


图 23-3 VBA 初始化调用流程图

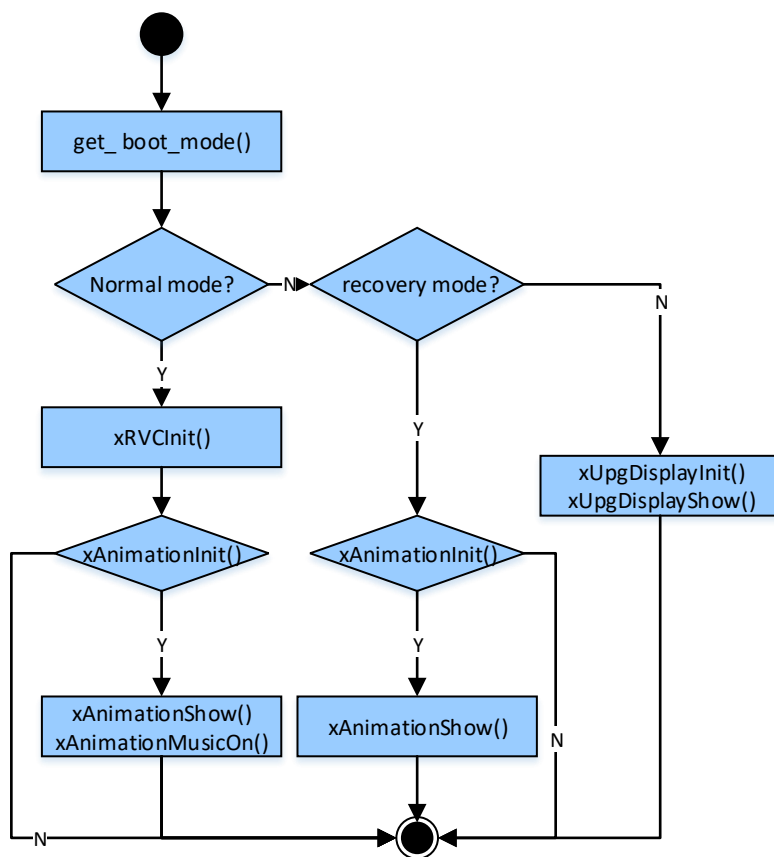


图 23-4 APM 初始化 VBA 流程图

23.2.2.2 VBA 资源读取

VBA 资源包含配置文件资源、图片资源、视频资源和音频资源，可通过 MRFTTool 制作。

HSM 初始化时调用 VBA 接口 xAnimationLoadSource()将 VBA 资源加载至内存，
xAnimationLoadSource()调用 datazone 接口 datazone_read_source 将 VBA 资源读取至内存。

Recovery 模式时，音频无法播放，因此不会加载 audio 资源。

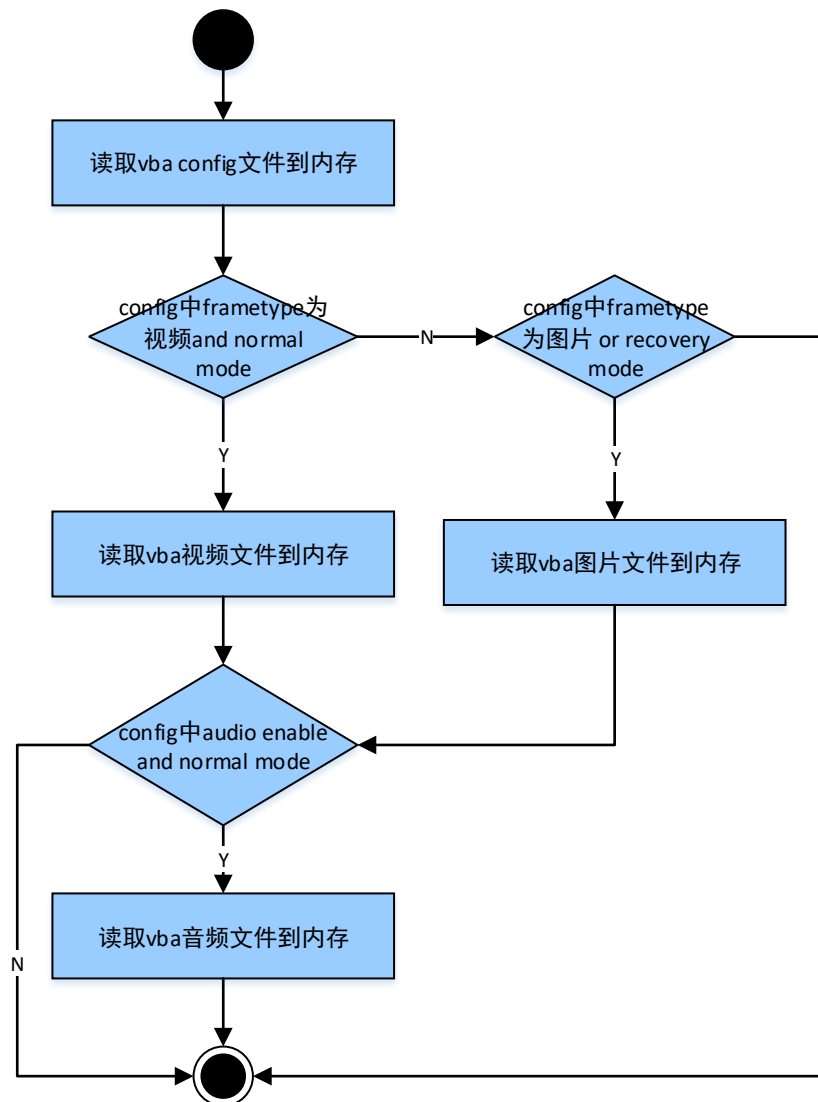


图 23-5 VBA 资源读取流程图

23.2.2.3 VBA Init

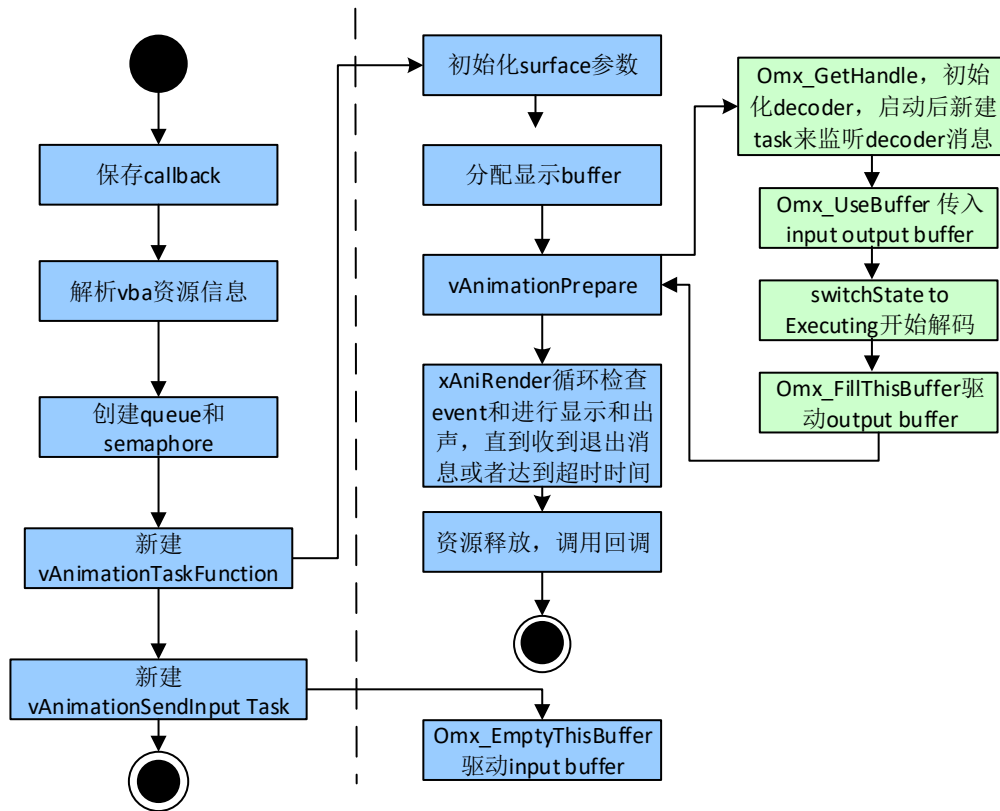


图 23-6 VBA 初始化流程图

23.2.2.4 VBA 退出流程

Android 系统启动到 home 界面显示时，触发 anim_exit 发送 IPC 消息给 HSM 系统，APM 接收到退出 VBA 消息后，调用 xAniHideDisplay()通知 VBA 进行退出 VBA 显示，当所有显示 VBA 的屏幕都被 hide 后，xAniHideDisplay() 调用 xAnimationExitEnter() 退出 VBA 播放，导致 vAnimationTaskFunction()退出，其调用回调函数通知 VBA 退出完成，AP 可以监听来自 HSM 的 IPC 消息，以获取 VBA 退出完成的消息。

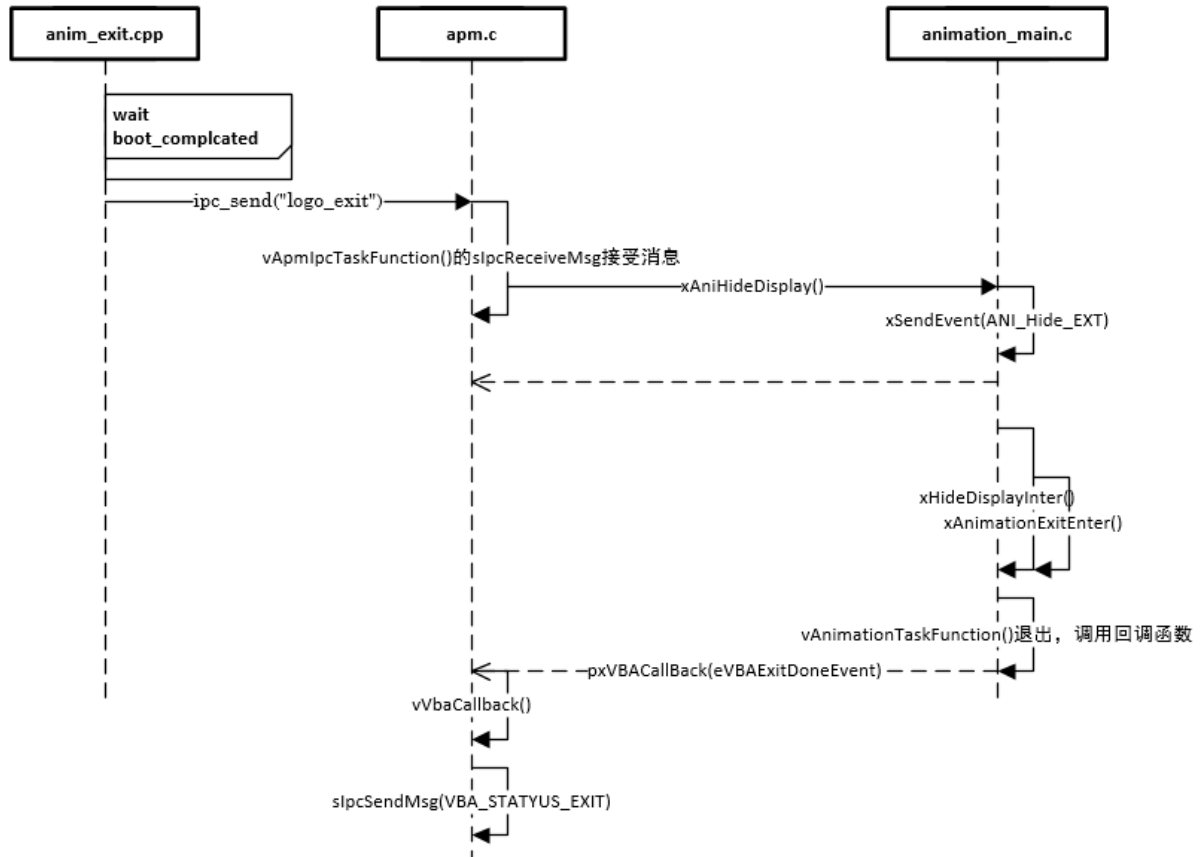


图 23-7 VBA 退出消息流程图

23.2.2.5 VBA 单屏退出流程

Cluster 和 ADAS 系统启动完成后, kms monitor service 检测到 drm 第一帧后发送 IPC 消息给 HSM 系统, APM 的 `vApmIpcTaskFunctionAdas` 接收来自 ADAS 系统的 IPC 消息, APM 的 `vApmIpcTaskFunctionCluster` 接收来自 Cluster 系统的 IPC 消息, APM 接收到消息后, 调用 `xAniHideDisplay()` 通知对应屏幕的 VBA 退出播放。

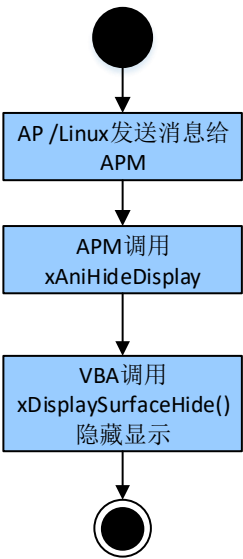


图 23-8 VBA 单屏退出流程图

23.3 接口描述

23.3.1 API

VBA 模块详细 API 参见表 23-3。

表 23-3 VBA API

API	说明
xAnimationLoadSource	Load VBA resource
xAnimationInit	Init VBA to start work.
xAnimationShow	To show VBA on display
xAnimationHide	To hide VBA on display
xAnimationExit	Exit VBA
xAnimationVideoExit	While the video loop count is 0xff,we can exit the video playback by using this interface
xAnimationShowInter	While the VBA is set not to play immediately upon startup,we can use this interface to start VBA play
xAnimationOnEnter	Start VBA
xAniHideDisplay	To hide VBA on the specified display and release the surface

23.3.1.1 xAnimationLoadSource

【描述】

The first API called before using VBA, which is used to Load VBA resource.

【语法】

```
BaseType_t xAnimationLoadSource (void);
```

【参数】

None

【返回值】

- pdFALSE: fail
- pdTRUE: success.

【注意】

The interface can only use before ivi kernel launch.

23.3.1.2 xAnimationInit

【描述】

The second API called to use VBA, which is used to start VBA task.

【语法】

```
BaseType_t xAnimationInit (VBA_EVENT_CALLBACK pvCallback);
```

【参数】

pvCallback: callback function receives the notification from VBA

【返回值】

- pdFALSE: fail
- pdTRUE: success.

【注意】

The interface is used at boot time. When we want to start VBA after the system is started, we need use interface xAnimationOnEnter.

23.3.1.3 xAnimationShow

【描述】

The interface is used to show VBA on display.

【语法】

```
BaseType_t xAnimationShow ();
```

【参数】

None

【返回值】

- pdFALSE: fail
- pdTRUE: success.

23.3.1.4 xAnimationHide

【描述】

The interface is used to hide VBA on display.

【语法】

```
BaseType_t xAnimationHide ();
```

【参数】

None

【返回值】

- pdFALSE: fail
- pdTRUE: success.

23.3.1.5 xAnimationExit

【描述】

The interface is used to exit VBA.

【语法】

```
BaseType_t xAnimationShow ();
```

【参数】

None

【返回值】

- pdFALSE: fail
- pdTRUE: success.

23.3.1.6 xAnimationVideoExit

【描述】

While the video loop count is 0xFF, we can exit the video playback by using this interface

【语法】

```
BaseType_t xAnimationVideoExit ();
```

【参数】

None

【返回值】

- pdFALSE: fail
- pdTRUE: success.

23.3.1.7 xAnimationShowInter

【描述】

While the VBA is set not to play immediately upon startup, we can use this interface to start VBA play.

【语法】

```
BaseType_t xAnimationShowInter ();
```

【参数】

None

【返回值】

- pdFALSE: fail
- pdTRUE: success.

23.3.1.8 xAnimationOnEnter

【描述】

The interface is used when we want to start VBA After the system is started.

【语法】

```
BaseType_t xAnimationOnEnter (VBA_EVENT_CALLBACK pvCallback);
```

【参数】

pvCallback: callback function receives the notification from VBA

【返回值】

- pdFALSE: fail

- pdTRUE: success.

23.3.1.9 xAniHideDispaly

【描述】

The interface is used to hide VBA on the specified display and release the surface.

【语法】

BaseType_t xAniHideDispaly (unsigned short display_id);

【参数】

display_id: specified display need to be hided and released

【返回值】

- pdFALSE: fail
- pdTRUE: success.

23.3.2 数据结构

表 23-4 VBA 数据结构

数据结构	说明
VBAConfigInfo_t	VBA config file struct
VBAVideoInfo_t	VBA video file header struct
VBALogoInfo_t	VBA logo file header struct

23.3.2.1 VBAConfigInfo_t

```
typedef struct {
    unsigned char szConfigFileMagic [PARTITION_NAME_SIZE]; // Identify VBA conf
file
    unsigned int u4RealSize; //data region real size
    unsigned char VBAAudioEnable;
    unsigned int Res1TimeOut;//the max time to exit vba after vba play to end
    unsigned int Res2TimeOut;
    unsigned int BackupVidCount;
    unsigned int BackupAudCount;
    unsigned int BackupLogoCount;
```

```

    unsigned int Res1DisplayMask;
    unsigned int Res2DisplayMask;
    unsigned int PictureMask;
    unsigned char Res1VideoFileName[VBA_FILE_NAME_SIZE];
    unsigned char Res1AudioFileName[VBA_FILE_NAME_SIZE];
    unsigned char Res2VideoFileName[VBA_FILE_NAME_SIZE];
    unsigned char Res2AudioFileName[VBA_FILE_NAME_SIZE];
    unsigned char PictureFileName[VBA_FILE_NAME_SIZE];
    unsigned char ImmdeiatStart;
    unsigned char frameTypeMask;//to identify which kind of frame to play,1-
video,2-logo
    unsigned char VBAAudioEnable;
    unsigned char VBAAudioLoop;
    unsigned char reserved[CONFIG_RESERVE_LENGTH];
} VBAConfigInfo_t

```

23.3.2.2 VBAVideoInfo_t

```

typedef struct {
    unsigned char szVideoFileMagic [PARTITION_NAME_SIZE];// Identify VBA video
file
    unsigned int u4RealSize; //data region real size
    unsigned int u4Offset;
    uint16_t u2Width;
    uint16_t u2Height;
    uint8_t codecId;
    uint8_t u1FrameRate;
    uint8_t u1DisplayCount;
    uint8_t u1VideoCount;
    uint8_t reserved[VIDEO_RESERVE_LENGTH];
} VBAVideoInfo_t;
typedef struct {
    uint16_t cropX;

```

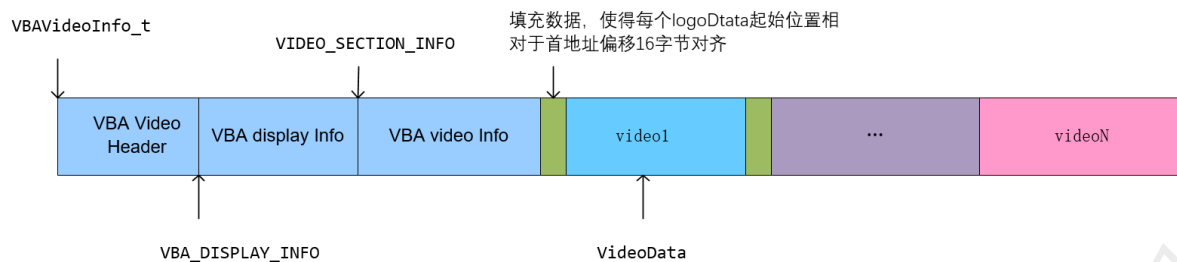
```
uint16_t cropY;
uint16_t cropW;
uint16_t cropH;
uint8_t displayId;
```

```
}VBA_DISPLAY_INFO;
```

```
typedef struct {
    unsigned int SectionTotalLength;
    unsigned __int8 loopcount;
    unsigned char ImmdeiatExit;
```

```
}VIDEO_SECTION_INFO;
```

VBA 视频资源内容，首先是 VBAVideoInfo_t，紧接着是 VBA_DISPLAY，其个数为 VBAVideoInfo_t 中的 u1DisplayCount，VBA_DISPLAY 之后是 VIDEO_SECTION_INFO，其个数为 VBAVideoInfo_t 中的 u1VideoCount，最后依次是各个视频文件数据 VideoData。数据结构之间可能会有一些填充数据，用于 video 数据起始地址对齐，因为解码器对原始视频数据地址有对齐要求。



23.3.2.3 VBALogoInfo_t

```
typedef struct {
    unsigned char szLogoFileMagic[PARTITION_NAME_SIZE]; // Identify the VBA logo
file
    unsigned int u4RealSize; //data region real size
    unsigned int u4Offset;
    uint8_t u1LogoCount;
} VBALogoInfo_t;
typedef struct {
    uint16_t u2Width;
    uint16_t u2Height;
    uint8_t u1DisplayCount;
```

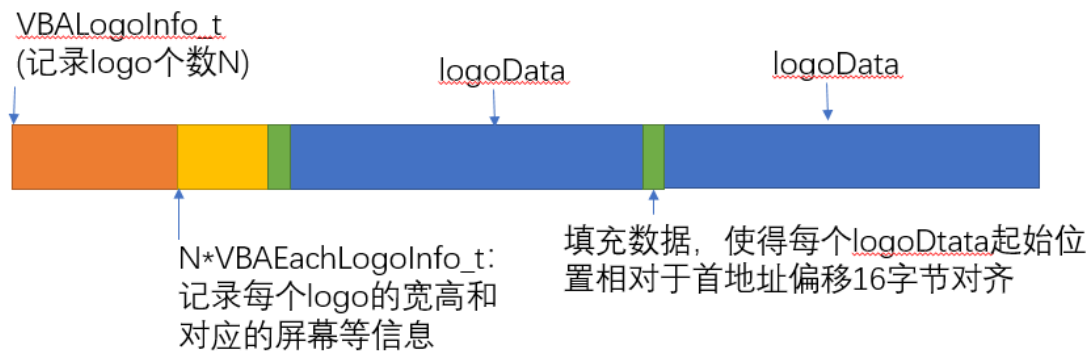
```
uint8_t u1DisplayId[ANI_MAX_DISPLAY];

unsigned int u4DisplayMask;

unsigned int SectionTotalLength;

} VBAEachLogoInfo_t;
```

VBA Logo 资源内容，首先是 VBALogoInfo_t，接着是 VBAEachLogoInfo_t，其个数为 VBALogoInfo_t 中的 VBAEachLogoInfo_t，最后依次为各个图片的数据。每个图片数据部分 logoData，其长度记录在 VBAEachLogoInfo_t 的 SectionTotalLength 中，其有效数据长度记录在 VBAEachLogoInfo_t 的 SectionRealLength 中。



23.4 用户定制化

23.4.1 开机资源客制化

- ATC 提供 MRFTool 工具用于导入用户定制的开机资源，支持开机时显示开机图片或者开机动画，支持生成开机音频。
- 系统启动完成后，可以通过 Carsetting 中的 ATC Settings 进行 Logo Setting，修改开机时的 Logo 图片，重启生效。

24 Backlight

24.1 简介

24.1.1 软硬件环境

24.1.1.1 软件环境

依赖 metazone。

24.1.1.2 硬件环境

- PWM 控制模块
- 使用 PWM 波开启和关闭 LED 来改变正向电流的导通时间以达到亮度调节的效果
- eMMC 用来存储 backlight 的 backlight value 值

24.1.2 功能

- 点亮屏幕
- 调节屏幕亮度
- 系统启动过程背光强度保持一致
- 支持灭屏功能

24.2 代码结构及文件描述

表 24-1 BackLight 代码结构及描述

文件名	文件路径	文件描述
PowerManagerService.java	source/android/frameworks/base/services/core/java/com/ android/server/power/	Android 电源 管理服务设置 背光
DisplayManagerService.java	source/android/frameworks/base/services/core/java/com/ android/server/power/	Android 显示 管理服务设置 背光

文件名	文件路径	文件描述
LightsService.java	source/android/frameworks/base/services/core/java/com/ android/server/lights/	Android 光系 统用来控制系 统中的各个光 源
com_android_server_lights _LightsService.cpp	source/android/frameworks/base/services/core/jni/ com_android_server_lights_LightsService.cpp	Android 对接 LightsService 和 light hidl 层的 jni
lights.c	source/android/vendor/autochips/proprietary/hardware /liblights/	Android 设置 主屏背光的 HAL 实现
atc_lights_lib.c	source/android/vendor/autochips/proprietary/hardware /display/libs/liblight	设置主副屏背 光对接 driver 实现
ac8x_bl.c	source/kernel/atc_modules/display/	Kernel driver 背光设置接口 的实现
backlight.h	source/kernal/common/include/linux	Kernel driver 背光设置接口 和相关结构体 的声明
AtcDisplay.java	source/android/vendor/autochips/proprietary/framework/base/ atcdisplay/java/com/autochips/display	ATC framework 私 有 display 功 能相关的接口 (包含背光功 能)
atcdisplay_client.cpp	source/android/vendor/autochips/proprietary/framework/base/ atcdisplay/jni	ATC 私有的 display 功能 对接 hidl 实现 jni 接口
atcdisplay_hidl.cpp	source/android/vendor/autochips/proprietary/hardware/atcpq/	ATC 私有的 display 功能 hidl 实现
atc_bl.c	source/vendor/autochips/proprietary/tinysys/hsm/os/ drivers/display/driver/backlight/	HSM 读取 AP 的设置值来设 置背光

24.3 软件设计

24.3.1 架构图

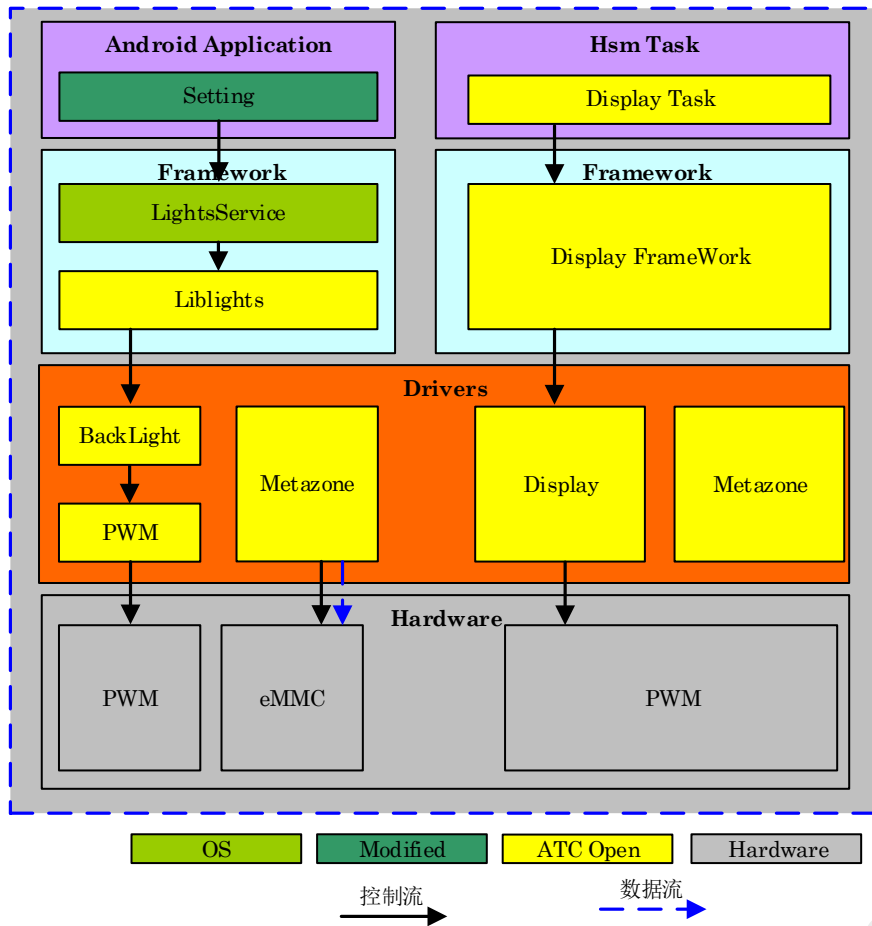


图 24-1 Backlight 架构图

Backlight 设计采用 AP 设置背光，并将背光值通过 metazone 存储到 eMMC 中，然后重启系统，HSM 将存在 eMMC 中的数据搬运到 metazone reserve memory 中，HSM 从 metazone reserve memory 中读取背光值，并使用读取的背光值设置背光，保证系统启动时和系统启动后背光一致性。后文所述的背光一致性都是指系统启动时和系统启动后，不再赘述。

24.3.2 流程图

系统启动进入 Android Home 界面，通过 Setting 或第三方应用设置背光值，App 可以通过 AtcDisplay 或者 lightservices 控制 backlight 的值。

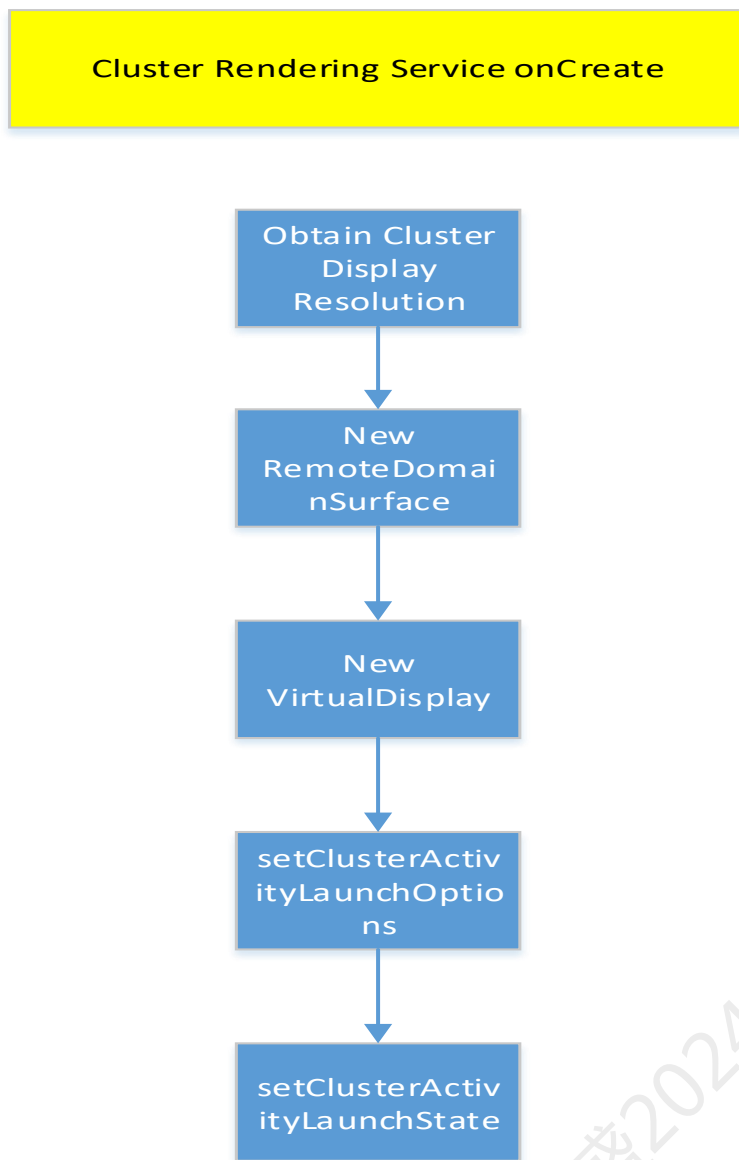


图 24-2 Android 保存背光值流程图

当 AP 设置好背光并将背光值保存到 metazone 中，系统重启后，HSM 读取 metazone 中的背光值，作为设置背光的参数。如果 metazone 中的背光值不存在，则使用默认的背光值设置背光。

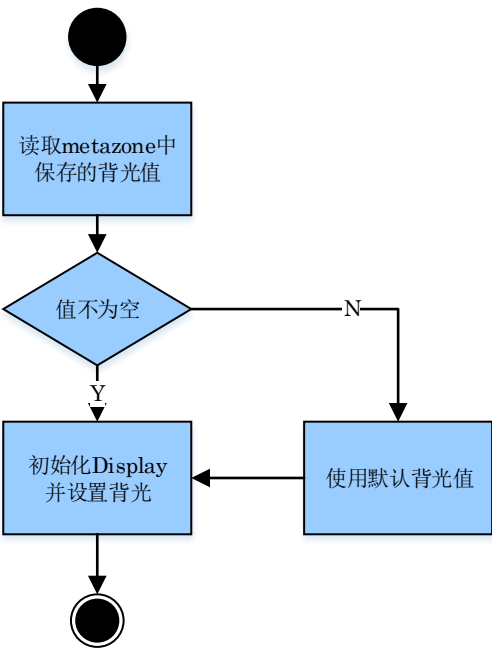


图 24-3 HSM 读取背光值并设置

24.4 接口描述

24.4.1 API

Backlight 相关的通用接口如下表所示。

表 24-2 BackLight API

API	说明
vSetBacklightIntensity	Hsm 读取 metazone 中保存的背光值并设置背光
bkl_shutdown	Hsm 关闭背光
set_light_backlight	主屏背光 native 层接口。Setting app 中设置背光会使用这个接口
set_backlight_brightness	副屏背光 native 层接口。Car Launcher 中设置背光会使用这个接口
setDisplayMute	开关指定 displayId 物理屏背光，TP 功能
getDisplayMuteStatus	获取指定 displayId 物理屏 DisplayMute 的状态

24.4.1.1 vSetBacklightIntensity

【描述】

HSM 读取 metazone 中保存的对应 pwm_num 背光值并设置背光。

【语法】

```
void vSetBacklightIntensity(unsigned int pwm_num)
```

【返回值】

无

【参数】

pwm_num: 主屏背光或副屏背光的 pwm 编号, 例如: 主屏为 PWM0, 副屏为 PWM1。

24.4.1.2 bkl_shutdown

【描述】

HSM 关闭对应 pwm_num 号的屏幕背光。

【语法】

```
void bkl_shutdown(unsigned int pwm_num)
```

【返回值】

无

【参数】

pwm_num: 主屏背光或副屏背光的 pwm 编号, 主屏为 PWM0, 副屏为 PWM1。

24.4.1.3 set_light_backlight

【描述】

主屏背光 Native 层接口, Setting app 中设置背光会通过这个接口传参数给内核, 该接口是 AP 原生接口, 参数可不关注。但如果需要做主屏背光值的重映射, 则需更改此函数的实现。

【语法】

```
static int set_light_backlight(struct light_device_t* dev, struct light_state_t const * state)
```

【返回值】

如果出错, 返回 error number, 否则返回 0

【参数】

- dev: 目前无用, 可不用关注
- state: 通过 rgb_to_brightness 转化为 brightness, 再映射为 PWM 值后设置给背光 kernel driver 节点。

24.4.1.4 set_backlight_brightness

【描述】

副屏背光 Native 层接口, Car Launcher app 中设置背光会通过这个接口传参数给内核, 该接口是 ATC 新增接口。如果需要做副屏背光值的重映射, 则需更改此函数的实现。

【语法】

```
int set_backlight_brightness(int displayID, int slider_val)
```

【返回值】

如果出错, 返回 error number, 否则返回 0

【参数】

- displayID: 屏幕 ID, 目前无用
- slider_val: Car Launcher app slider bar value, 转化为 PWM duty 值后设置给背光 kernel driver 节点。

24.4.1.5 setDisplayMute

【描述】

开关指定 displayId 物理屏背光, TP 功能

【语法】

```
int setDisplayMute (int displayID, boolean isMute)
```

【返回值】

如果出错, 返回-1, 否则返回 0

【参数】

- @param displayID [IN]: 物理 displayID
- @param isMute [IN]: true, enable displaymute 功能 (关闭物理屏背光, TP 功能); false, disable displaymute 功能 (开启物理屏背光, TP 功能)

24.4.1.6 getDisplayMuteStatus

【描述】

设置指定 displayId 物理屏 DisplayMute 的状态

【语法】

int getDisplayMute (int displayID)

【返回值】

返回 1 是 enable DisplayMute 状态，0 是 disable DisplayMute 状态，-1 is invalid

【参数】

@param displayID [IN]: 物理 displayID。

24.4.2 数据结构

表 24-3 BackLight 数据结构

数据结构	说明
backlight_device	backlight背光子系统的设备结构体 backlight_device
backlight_properties	背光属性
backlight_ops	背光操作函数结构体

```

struct backlight_device{
    struc backlight_properties props;//背光属性
    struct mutex update_lock;
    struct backlight_ops *ops;    //背光操作函数，类似于 file_operations
    struct notifier_block fb_notif;
    struct device dev;  //内嵌设备
}

struct backlight_properties {
    int brightness;  //当前背光值
    int max_brightness;  //最大背光值
    int power; //power mode 0 full on ,1 to 3 power saving modes 4 full off
    int fb_blank;
    unsigned int state;//背光的状态
    
```

```

    }
struct backlight_ops {
    unsigned int options;

    int (*update_status)(struct backlight_device *);    //改变背光状态
    int (*get_brightness)(struct backlight_device *);  //获取背光值
    int (*check_fb)(struct fb_info *);

};

//metazone 背光值 index:
METAZONE_BACKLIGHT_NUM1 = 0x10000,
METAZONE_BACKLIGHT_NUM2 = 0x10001,
METAZONE_BACKLIGHT_NUM3 = 0x1003f,
METAZONE_BACKLIGHT_NUM4 = 0x10040,

//metazone 背光频率 index:
METAZONE_BACKLIGHT_FREQ1 = 0x1001a,
METAZONE_BACKLIGHT_FREQ2 = 0x1001b,
METAZONE_BACKLIGHT_FREQ3 = 0x10041,
METAZONE_BACKLIGHT_FREQ4 = 0x10042,

```

24.5 使用说明和注意事项

目前已经支持背光一致性，即系统启动时和系统启动后的背光亮度是一致。

- 第一次烧写 image 成功，并启动至 home 界面；
- 进入 setting 界面，点击 Brightness level，调节背光至合适值；

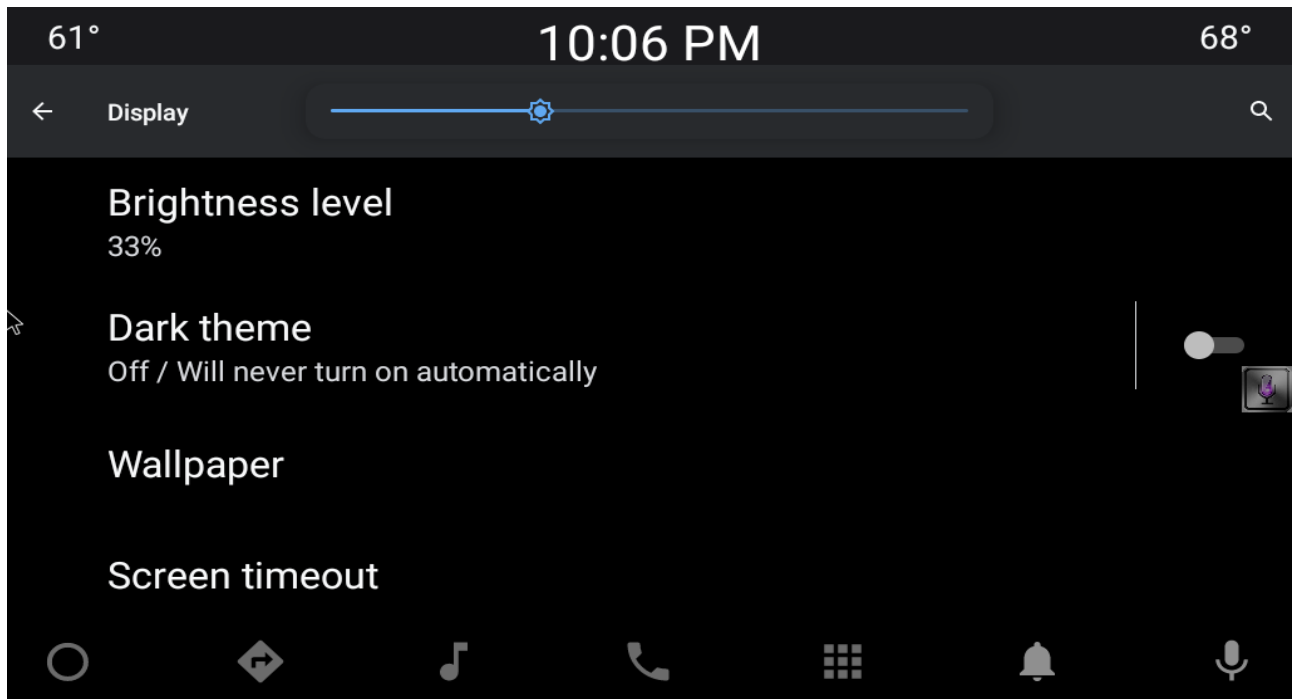


图 24-4 AP 背光设置操作界面图

- 重启系统，Hsm 采用刚刚设置的背光值设置背光，并且系统刚启动时的背光亮度和系统启动完后的背光亮度一致，即背光一致性的功能。

24.6 用户定制化

24.6.1 AP 保存背光

用户在 App 层设置背光后，分别调用 HAL 层如下接口进行设置：

- `set_light_backlight()`：主屏，代码位于
`source/android/vendor/autochips/proprietary/hardware/liblights/lights.c`
- `set_backlight_brightness()`：副屏，代码位于
`source/android/vendor/autochips/proprietary/display/libs/liblight/atc_lights_lib.c`

上述两个 HAL 层接口最终是调用背光 driver 的 `ioctl`，触发 kernel 层的驱动设置背光，详见 `source/kernel/common/drivers/video/backlight/ac8x_bl.c`。

Driver 层在设置背光的同时将背光值写入到 `metazone` 中，详见如下代码。

```
MetaZone_Init();
err = (int)MetaZone_Write(BACKLIGHT_INDEX, g_backlight);
ALOGD("write_int backlight %d %x level=%d\n", __LINE__, err, g_backlight);
```

```
if (0 != err) {
    MetaZone_Deinit();
    goto FAIL;
}
MetaZone_Flush(1);
MetaZone_Deinit();
```

该部分代码用于保存用户设置的背光值到 metazone 中，metazone index 为：

- 第一块屏 index 为 0x10000
- 第二块屏 index 为 0x10001

当不需要背光一致性功能时，可以将该部分注释。

24.6.2 HSM 设置背光

HSM 设置背光直接通过调用 `vSetBacklightIntensity()` 或 `bkl_shutdown()` 打开背光或关闭背光。在 `vSetBacklightIntensity()` 函数中会通过读取 metazone 保存的背光值，然后设置背光，如下所示。

```
#ifdef ATC_METAZONE_SUPPORT
    ret = ulMetazone_Read(meta_duty, &meta_duty);
    ...
#endif /* ATC_METAZONE_SUPPORT */
```

ATC_METAZONE_SUPPORT 宏使能，使用 metazone 中保存的值设置背光。

ATC_METAZONE_SUPPORT 宏未使能，使用默认值设置背光。

从 metazone 中相应的 index（index 参见 24.4.2 数据结构的 metazone 背光值 index 介绍）处读取背光值。如果没有使用 metazone，则通过定义的 `default_level` 来设置背光。

可参考文件路径为： `source/kernel/common/drivers/video/backlight/ac8x_bl.c`

24.6.3 开关背光一致性功能

目前 AC8025 通过 `displayId` 传给 `dpim` 模块的 `dpim_disp_get_bkl_info` 找到相对应的 `pwmId` 来设置 backlight。

对于主屏和副屏，当前背光保存及一致性的实现方式有部分差异。

24.6.3.1 主屏

24.6.3.1.1 用户未手动设置背光

在用户未手动设置背光前，默认共有三处（HSM、Android PowerManagerService、Android DisplayManagerService）来设置背光，其中 HSM 读取的背光值存于 metazone，PowerManagerService、DisplayManagerService 读取的背光值存于 Android xml 文件中。为保证启动过程中背光同步，要保证各处背光值的一致性。

HSM:

读取的背光值位于 Metazone Dword 区

- 第一块屏 index 为 0x10000，默认值 22000
- 第二块屏 index 为 0x10001，默认值 22000

Android PowerManagerService（ATC 已通过修改配置文件将其背光设置功能关闭）：

程序文件路径：

source/android/frameworks/base/services/core/java/com/android/server/power/PowerManagerService.java

配置文件路径：source/android/frameworks/base/core/res/res/values/config.xml

```
<integer name="config_screenBrightnessSettingDefaultFloat">-2</integer>;
```

ATC 通过将 xml 文件中 default 值为-2，可阻止 PowerManagerService 每次启动自动配置背光值。

source/android/frameworks/base/packages/SettingsProvider/res/values/defaults.xml

```
<!-- Default screen brightness, from 0 to 255. 22==slider:38==pwm:31. -->  
<!-- ATC_AOSP_ENHANCEMENT, change value from 102 to 22 -->  
<integer name="def_screen_brightness">22</integer>;
```

ATC 通过将 xml 文件中 default 值由 102 改为 22，可与 metazone 中的 PWM 默认值 31 相匹配。

24.6.3.1.2 用户手动设置背光后

当用户手动调整了背光值以后，保存在 Metazone 的默认值会被更新，同时 Android 也会在会数据库中保存设置值。下次启动过程中，HSM 和 Android 会分别读取 Metazone 和数据库中的保存值进行设置，从而达到 HSM 和 Android 阶段背光值的一致。

24.6.3.2 副屏

目前副屏的背光的保存采用 ATC 独立设计的 path，在保存背光 brightness 值时，下次进来的时候，副屏获取 brightness，gamma 值来，来换算得到 backlightprocess。

背光 PWM 频率修改:

目前背光 PWM 的频率默认值为 1,000,000Hz，默认值保存于 Metazone Dword 区域中，Metazone index 分别为:

- 第一块屏: METAZONE_BACKLIGHT_FREQ1, 0x1001a
- 第二块屏: METAZONE_BACKLIGHT_FREQ2, 0x1001b

如果需要修改 PWM 默认频率，只需更改 metazone 中默认值即可。

24.6.3.2.1 在 HSM 中

代码路径:

source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/display/driver/backlight/atc_bl.c

在 vSetBacklightIntensity()中设置 struct pwm_config conf 的成员 freq 和 duty 的值。freq 是频率，最高可设置为 200MHz。duty 是背光 PWM 占空比，duty 范围是 1~99。

结构体 pwm_config conf 如下:

```
struct pwm_config {  
    unsigned int pwm_no;  
    unsigned int duty;  
    unsigned int freq;  
    enum pwm_mode_emum mode;  
    unsigned int vsync_mode_edge;  
};
```

24.6.3.2.2 在 kernel 中

代码路径: source/kernel/common/drivers/video/backlight/ac8x_bl.c

在 bkl_pwm_init()中设置 struct pwm_config conf 的成员 freq 和 duty 的值。结构体 pwm_config conf 与 HSM 中相同。

24.6.3.3 更换背光 PWM

如果不使用 AC8025 SoC 的 PWM，而又需要使用原生的方式调节背光，此时我们就可以更改 driver 中的接口来实现。

24.6.3.3.1 在 HSM 中

代码路径：

source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/display/driver/backlight/atc_bl.c

更改 vSetBacklightIntensity() 的函数实现即可（在此函数中实现 PWM 的调节）。

24.6.3.3.2 在 kernel 中

代码路径：source/kernel/common/drivers/video/backlight/ac8x_bl.c

上层最终会调用 ac8x_bl.c 中的 atc_bkl_ioctl() 函数，而 atc_bkl_ioctl() 函数里面调用的是 set_backlight()，所以我们只需要更改此函数实现即可（在此函数中实现 PWM 的调节）。

24.7 典型问题

24.7.1 设置背光常见问题

【问题描述】

重启背光一致性功能，未生效。

【解决方案】

1. 检查是否使能了背光一致性功能，即检查 ATC_AOSP_ENHANCEMENT 和 ATC_METAZONE_SUPPORT 宏是否定义。如果没有请打开，如果有请继续往后查。
2. 通过在 setting 界面设置背光，检查 log 中是否有下列 log，如果有，则说明写入失败，若没有说明写入成功。

“metazone init fail:”

“metazone write fail:”

3. 写入 Metazone 失败，则检测 AP Metazone 的问题。
4. 写入 Metazone 成功，则检查启动 log 中是否有下列 log。如果有，则读取 Metazone 成功，并且设置背光。

“VISS BL read metazone success,value:”

5. 如果读取 metazone 失败，则检查 HSM metazone 的问题。
6. 如果写入的值和读出的值不一致，检查写入的位置和读取的位置是否一致。

25 PQ

25.1 简介

25.1.1 软硬件环境

AC8025 PQ 处理是在 DPU 硬件上完成的，DPU display path 在 composition 之后，有集成在 YUV 色域调节算法或通过 3D LUT 对色彩进行校正。UDPU 中已集成色温调节，FDPU 当前没有集成针对色温的调节。

25.1.2 功能

提供调节显示效果相关参数的功能，可调节的参数包括亮度、对比度、饱和度和色温等。

针对 PQ 的调节主要目的是赋予图像更丰富的色彩、赋予图像更受用户喜爱的色彩。

25.1.2.1 亮度

色彩的亮度，又称为色彩明度。色彩亮度是色彩的三要素（色相、明度和饱和度）之一。不同颜色会有明暗的差异，相同颜色也有明暗深浅的变化。

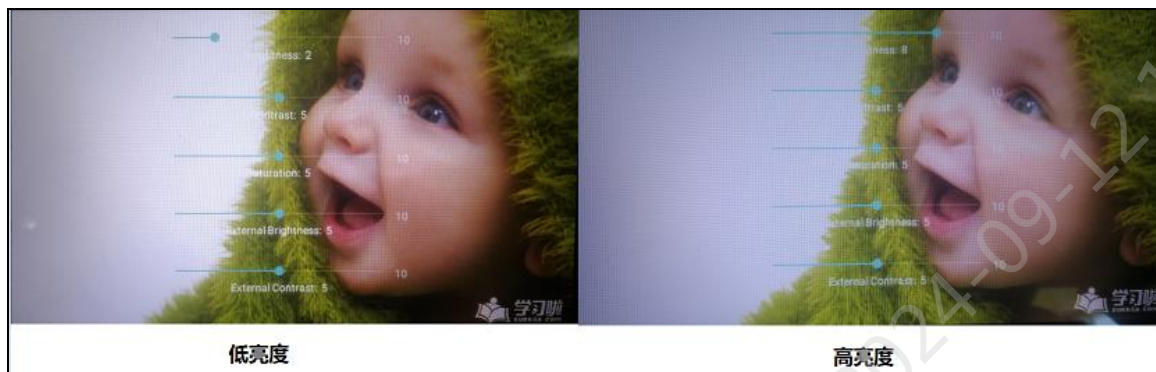


图 25-1 亮度对比图

25.1.2.2 对比度

对比度指的是一幅图像中明暗区域最亮的白和最暗的黑之间不同亮度层级的测量，差异范围越大代表对比越大，差异范围越小代表对比越小。即对比度越高，照片中暗的部分就越暗，亮的部分就越亮。对比度越低，照片中暗的部分就会偏亮，而亮的部分则会偏暗。

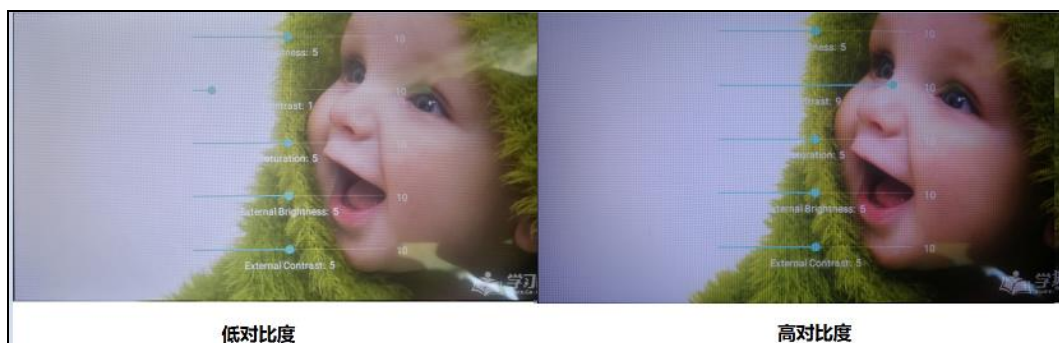


图 25-2 对比度对比图

25.1.2.3 饱和度

饱和度是指色彩的纯度，饱和度越高，色彩越纯，色彩越鲜艳，越低的饱和度意味着越接近灰白色。

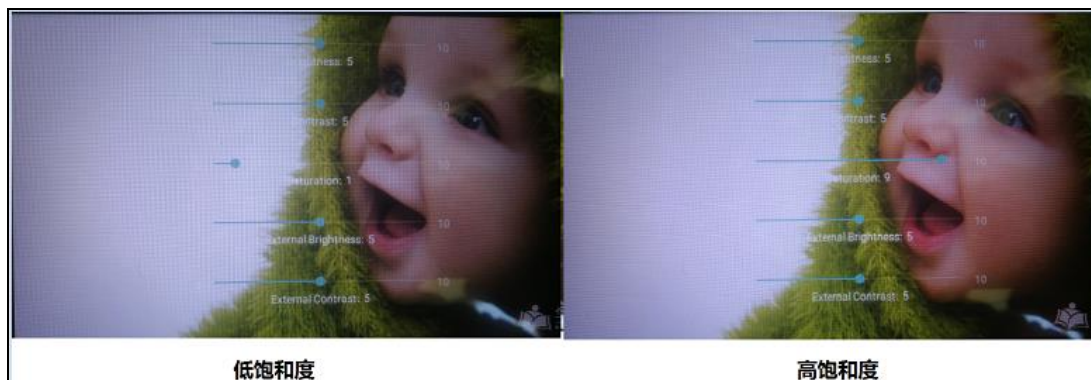


图 25-3 饱和度对比图

25.1.2.4 色温

色温是指颜色偏暖或偏冷程度的一种度量。一般情况下，较低的色温值（约 2700K 至 3000K）被认为是暖色调；中等色温值（约 3500K 至 4500K）被认为是中性色调，类似于日光或阴天的光线；较高的色温值（约 5000K 至 6500K 或以上）被认为是冷色调。



图 25-4 色温对比图

25.1.3 代码结构及文件描述

表 25-1 PQ 代码结构及描述

文件名	文件路径	文件描述
com.autochips.pq.atcpq	source/android/vendor/autochips/proprietary/frameworks/base/atcdisplay	为 APP 提供的 Java 层调节接口
metazone.csv	build/projects/cockpit_s_linux_ac8025/metazone/ 或 build/projects/ivi_s_ac8025/metazone/	Metazone 配置文件，其中包含 PQ 相关参
HIDL-service: vendor.autochips.hardware .atcdisplay@1.0-service	source/android/vendor/autochips/proprietary/hardware/diisplay/hidl/atcdisplay	hidl service，用于启动 PQ 服务并响应 PQ 调节需求
Libatcdisplay	source/android/vendor/autochips/proprietary/hardware/display/libs/libpq	提供接口给 pq_service 调用，完成 PQ 设置
DPU（包含 PQ）Driver	Hsm:safety/hsm/drivers/display/driver/ac8025 Kernel:source/kernel/atc_modules/drm_autochips	Hsm 中会对 PQ 初始化，并完成 PQ 硬件设置。Kernel 中会向应用层提供 ioctl，配置 pq 参数
Gamma driver	source/kernel/atc_modules/display/	提供 sysfs 供 tool 操作 gamma

25.2 软件设计

25.2.1 架构图

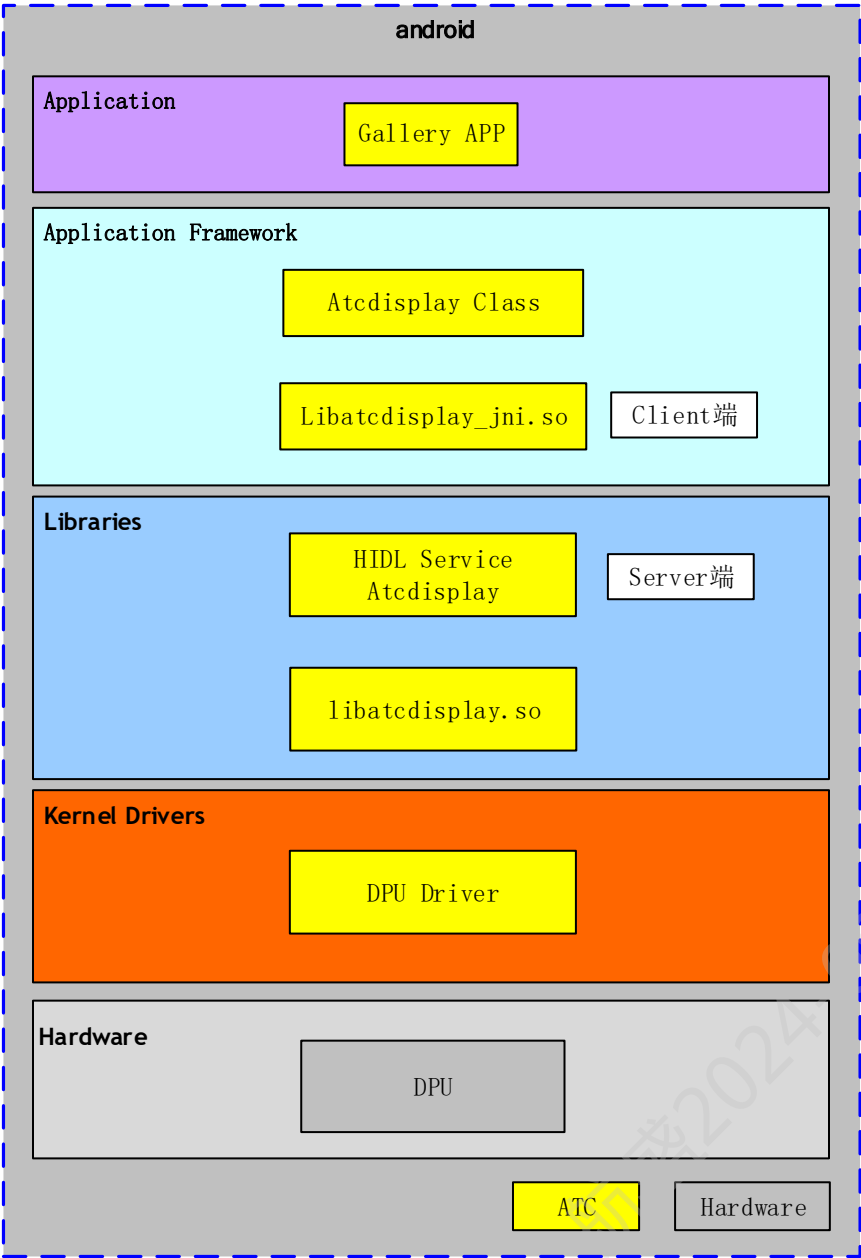


图 25-5 Android PQ 架构图

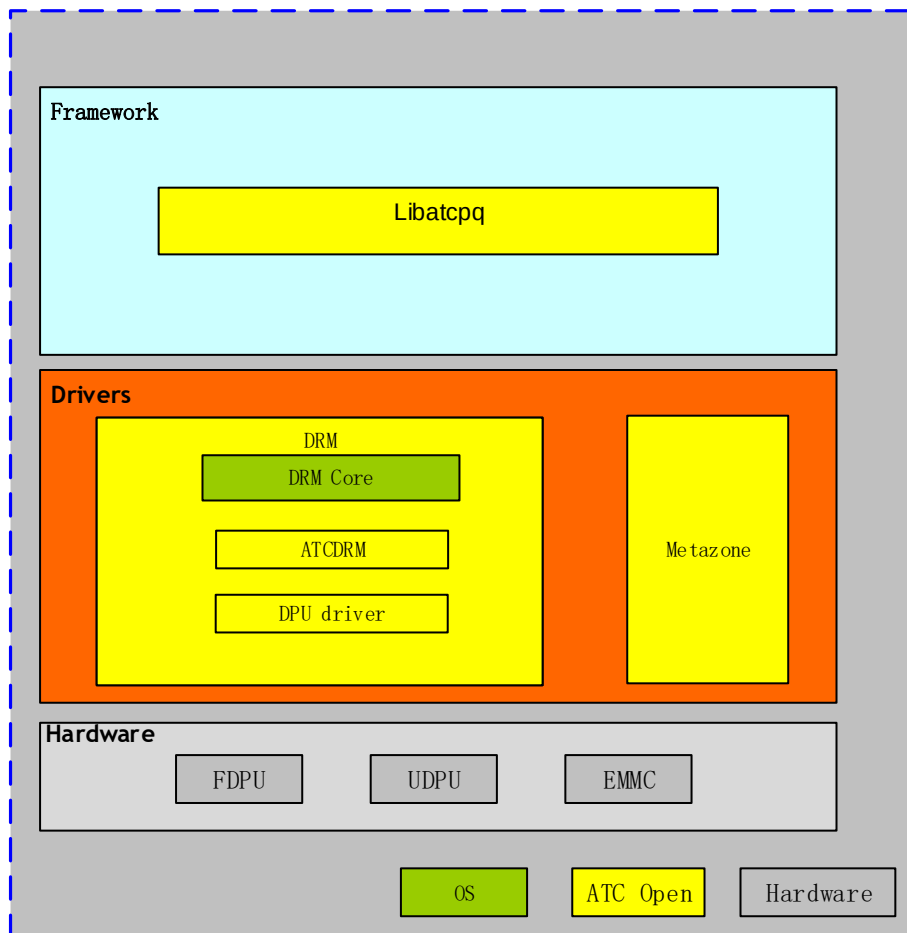


图 25-6 Cluster PQ 架构图

25.2.2 流程图

系统启动过程中，前后共有两处设置 PQ 参数，分别是 Hsm 阶段、Kernel 阶段。

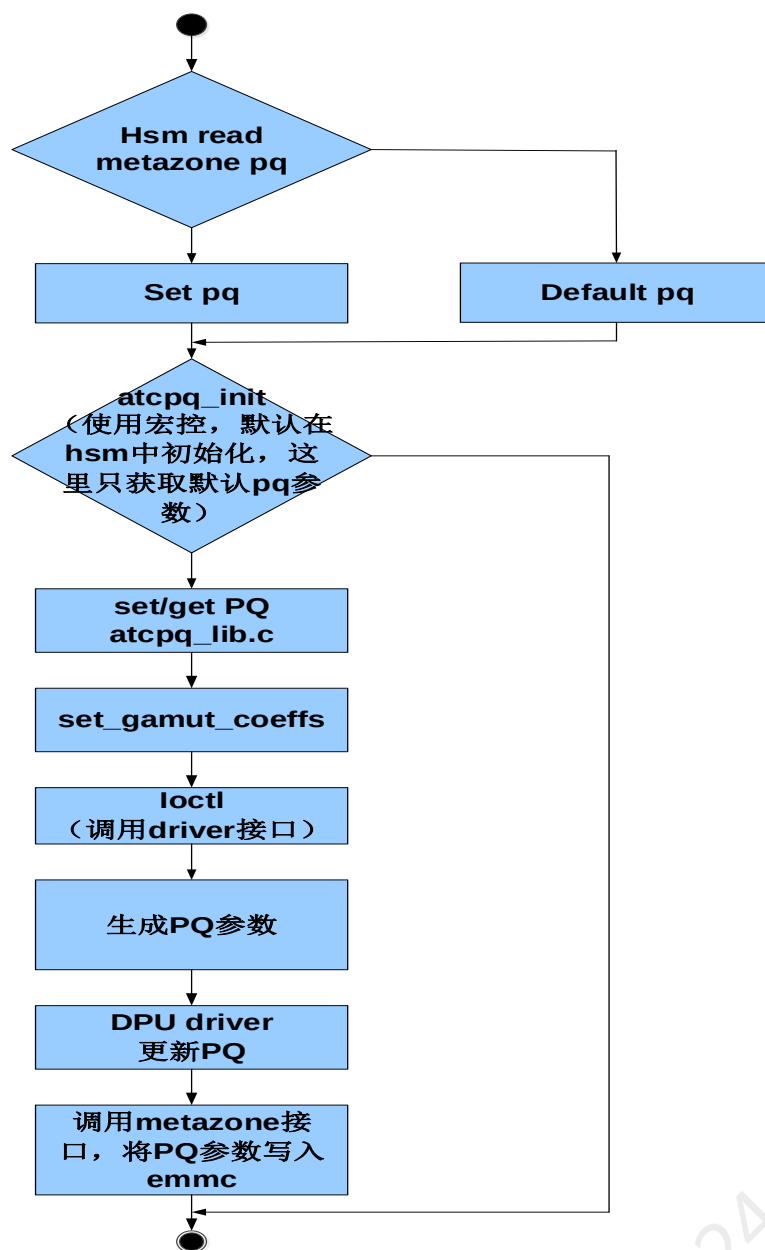


图 25-7 PQ 初始化流程图

25.2.2.1 Hsm 阶段 PQ 初始化

在 Hsm Display Driver 初始化的过程中会调用函数 `fdpu_pq_set/udpu_hw_pq_cfg`，完成单屏或多屏 PQ 初始化。

函数 `fdpu_pq_set/udpu_hw_pq_cfg` 位置： `/safety/hsm/drivers/display/driver/ac8025`。

25.2.2.2 Kernel 阶段 PQ 初始化

在 PQ service 的初始化函数 AtcPq 中调用了函数 atcpq_init。因为在 Hsm 阶段已经初始化了 PQ，所以 service 中不会再次初始化。

函数 AtcPq 位置：

source/android/vendor/autochips/proprietary/hardware/hidl/atcdisplay/atcpq_hidl.cpp。

25.3 接口描述

25.3.1 java 层接口

包名：com.autochips.display.AtcDisplay。

表 25-2 Java 层接口

API	说明
getMaxSaturationIndex	获取饱和度最大的 index
getMinSaturationIndex	获取饱和度最小的 index
getCurrentSaturationIndex	获取饱和度当前的 index
setSaturationIndex	设置饱和度的 index
getMaxContrastIndex	获取对比度最大的 index
getMinContrastIndex	获取对比度最小的 index
getCurrentContrastIndex	获取对比度当前的 index
setContrastIndex	设置对比度的 index
getMaxBrightnessIndex	获取亮度最大的 index
getMinBrightnessIndex	获取亮度最小的 index
getCurrentBrightnessIndex	获取亮度当前的 index
setBrightnessIndex	设置当前亮度的 index
getColorTemperature	获取当前色温的 index
setColorTemperature	设置当前色温的 index
getMinColorTemperatureIndex	获取最小色温的 index
getMaxColorTemperatureIndex	获取最大色温的 index

25.3.1.1 getMaxSaturationIndex

【描述】

获取饱和度最大的 index

【语法】

```
public native static int getMaxSaturationIndex(int displayID);
```

【返回值】

- 如果成功，则返回饱和度可设置的最大 index
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16

25.3.1.2 getMinSaturationIndex

【描述】

获取饱和度最小的 index

【语法】

```
public native static int getMinSaturationIndex(int displayID);
```

【返回值】

- 如果成功，则返回饱和度可设置的最小 index
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16

25.3.1.3 getCurrentSaturationIndex

【描述】

获取饱和度当前的 index

【语法】

```
public native static int getCurrentSaturationIndex(int displayID);
```

【返回值】

- 如果成功，则返回饱和度当前 index
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16

25.3.1.4 setSaturationIndex

【描述】

设置饱和度的 index

【语法】

```
public native static int setSaturationIndex(int displayID, int index);
```

【返回值】

- 如果成功，则返回 0
- 如果失败，则返回 error number

【参数】

- displayID: [IN] 0-16
- index: [IN] 设置的饱和度 index

25.3.1.5 getMaxContrastIndex

【描述】

获取对比度最大的 index

【语法】

```
public native static int getMaxContrastIndex(int displayID);
```

【返回值】

- 如果成功，则返回对比度可设置的最大 index
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16

25.3.1.6 getMinContrastIndex

【描述】

获取对比度最小的 index

【语法】

```
public native static int getMinContrastIndex(int displayID);
```

【返回值】

- 如果成功，则返回对比度可设置的最小 index
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16

25.3.1.7 getCurrentContrastIndex

【描述】

获取对比度当前的 index

【语法】

```
public native static int getCurrentContrastIndex(int displayID);
```

【返回值】

- 如果成功，则返回对比度当前 index
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16

25.3.1.8 setContrastIndex

【描述】

设置对比度的 index

【语法】

```
public native static int setContrastIndex(int displayID, int index);
```

【返回值】

- 如果成功，则返回 0
- 如果失败，则返回 error number

【参数】

- displayID: [IN] 0-16。
- index: [IN] 设置的对比度 index

25.3.1.9 getMaxBrightnessIndex

【描述】

获取亮度最大的 index

【语法】

```
public native static int getMaxBrightnessIndex(int displayID);
```

【返回值】

- 如果成功，则返回亮度可设置的最大 index
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16

25.3.1.10 getMinBrightnessIndex

【描述】

获取亮度最小的 index

【语法】

```
public native static int getMinBrightnessIndex(int displayID);
```

【返回值】

- 如果成功，则返回亮度可设置的最小 index
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16

25.3.1.11 getCurrentBrightnessIndex

【描述】

获取亮度当前的 index

【语法】

```
public native static int getCurrentBrightnessIndex(int displayID);
```

【返回值】

- 如果成功，则返回亮度当前 index

- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16。

25.3.1.12 setBrightnessIndex

【描述】

设置亮度的 index

【语法】

```
public native static int setBrightnessIndex(int displayID, int index);
```

【返回值】

- 如果成功，则返回 0
- 如果失败，则返回 error number

【参数】

- displayID: [IN] 0-16
- index: [IN] 设置的亮度 index

25.3.1.13 getColorTemperature

【描述】

获取当前色温的 index

【语法】

```
public native static int getColorTemperature(int displayID);
```

【返回值】

- 如果成功，则返回当前色温值
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16

25.3.1.14 setColorTemperature

【描述】

设置色温的 index

【语法】

```
public native static int setColorTemperature(int displayID, int index);
```

【返回值】

- 如果成功，则返回当前色温值
- 如果失败，则返回 error number

【参数】

- displayID: [IN] 0-16.
- index:[IN], 20-80.

25.3.1.15 getMinColorTemperature

【描述】

获取最小色温的 index

【语法】

```
public native static int getMinColorTemperature(int displayID);
```

【返回值】

- 如果成功，则返回当前色温值
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16.

25.3.1.16 getMaxColorTemperature

【描述】

获取最大色温的 index

【语法】

```
public native static int getMaxColorTemperature(int displayID);
```

【返回值】

- 如果成功，则返回当前色温值
- 如果失败，则返回 error number

【参数】

displayID: [IN] 0-16.

25.3.2 hal 层接口

cluster/adas domain 直接调用 hal 层接口调节 PQ。

表 25-3 Java 层接口

API	说明
get_MaxSaturationIndex	获取饱和度最大的 index
get_MinSaturationIndex	获取饱和度最小的 index
get_CurrentSaturationIndex	获取饱和度当前的 index
set_SaturationIndex	设置饱和度的 index
get_MaxContrastIndex	获取对比度最大的 index
get_MinContrastIndex	获取对比度最小的 index
get_CurrentContrastIndex	获取对比度当前的 index
set_ContrastIndex	设置对比度的 index
get_MaxBrightnessIndex	获取亮度最大的 index
get_MinBrightnessIndex	获取亮度最小的 index
get_CurrentBrightnessIndex	获取亮度当前的 index
setBrightnessIndex	设置当前亮度的 index
get_ColorTemperature	获取当前色温的 index
set_ColorTemperature	设置当前色温的 index
get_MinColorTemperatureIndex	获取最小色温的 index
get_MaxColorTemperatureIndex	获取最大色温的 index

25.3.2.1 get_MaxSaturationIndex

【描述】

获取饱和度最大的 index。

【语法】

```
int get_MaxSaturationIndex(int displayID);
```

【返回值】

返回饱和度可设置的最大 index

【参数】

displayID: [IN] 0-16。

25.3.2.2 get_MinSaturationIndex

【描述】

获取饱和度最小的 index。

【语法】

```
int get_MinSaturationIndex(int displayID);
```

【返回值】

返回饱和度可设置的最小 index

【参数】

displayID: [IN] 0-16。

25.3.2.3 get_CurrentSaturationIndex

【描述】

获取饱和度当前的 index。

【语法】

```
int get_CurrentSaturationIndex(int displayID);
```

【返回值】

- 如果成功，则返回饱和度当前 index
- 如果失败，则返回-1

【参数】

displayID: [IN] 0-16。

25.3.2.4 set_SaturationIndex

【描述】

设置饱和度的 index。

【语法】

```
int set_SaturationIndex(int displayID, int index);
```

【返回值】

- 如果成功，则返回 0
- 如果失败，则返回-1

【参数】

- displayID: [IN] 0-16。
- index: [IN] 0-10。

25.3.2.5 get_MaxContrastIndex**【描述】**

获取对比度最大的 index。

【语法】

```
int get_MaxContrastIndex(int displayID);
```

【返回值】

返回对比度可设置的最大 index

【参数】

displayID: [IN] 0-16。

25.3.2.6 get_MinContrastIndex**【描述】**

获取对比度最小的 index。

【语法】

```
int get_MinContrastIndex(int displayID);
```

【返回值】

返回对比度可设置的最小 index

【参数】

displayID: [IN] 0-16。

25.3.2.7 get_CurrentContrastIndex**【描述】**

获取对比度当前的 index。

【语法】

```
int get_CurrentContrastIndex(int displayID);
```

【返回值】

- 如果成功，则返回对比度当前 index
- 如果失败，则返回-1

【参数】

displayID: [IN] 0-16。

25.3.2.8 set_ContrastIndex**【描述】**

设置对比度的 index。

【语法】

```
int set_ContrastIndex(int displayID, int index);
```

【返回值】

- 如果成功，则返回 0
- 如果失败，则返回-1

【参数】

- displayID: [IN] 0-16。
- index: [IN] 0-10。

25.3.2.9 get_MaxBrightnessIndex**【描述】**

获取亮度最大的 index。

【语法】

```
int get_MaxBrightnessIndex(int displayID);
```

【返回值】

返回亮度可设置的最大 index

【参数】

displayID: [IN] 0-16。

25.3.2.10 get_MinBrightnessIndex

【描述】

获取亮度最小的 index。

【语法】

```
int get_MinBrightnessIndex(int displayID);
```

【返回值】

返回亮度可设置的最小 index

【参数】

displayID: [IN] 0-16。

25.3.2.11 get_CurrentBrightnessIndex

【描述】

获取亮度当前的 index。

【语法】

```
int get_CurrentBrightnessIndex(int displayID);
```

【返回值】

如果成功，则返回亮度当前 index

如果失败，则返回-1

【参数】

displayID: [IN] 0-16。

25.3.2.12 set_BrightnessIndex

【描述】

设置亮度的 index。

【语法】

```
int set_BrightnessIndex(int displayID, int index);
```

【返回值】

- 如果成功，则返回 0
- 如果失败，则返回-1

【参数】

- displayID: [IN] 0-16。
- index: [IN] 0-10。

25.3.2.13 get_ColorTemperature

【描述】

获取当前色温的 index。

【语法】

```
int get_ColorTemperature(int displayID);
```

【返回值】

- 如果成功，则返回当前色温值
- 如果失败，则返回-1

【参数】

displayID: [IN] 0-16。

25.3.2.14 set_ColorTemperature

【描述】

设置色温的 index。

【语法】

```
int set_ColorTemperature(int displayID, int index);
```

【返回值】

- 如果成功，则返回当前色温值
- 如果失败，则返回-1

【参数】

- displayID: [IN] 0-16。
- index:[IN], 20-80。

25.3.2.15 get_MinColorTemperature

【描述】

获取最小色温的 index。

【语法】

```
int get_MinColorTemperature(int displayID);
```

【返回值】

- 如果成功，则返回当前色温值
- 如果失败，则返回-1

【参数】

displayID: [IN] 0-16.

25.3.2.16 get_MaxColorTemperature

【描述】

获取最大色温的 index。

【语法】

```
int get_MaxColorTemperature(int displayID);
```

【返回值】

- 如果成功，则返回当前色温值
- 如果失败，则返回-1

【参数】

displayID: [IN] 0-16.

25.3.3 Hsm 接口

Hsm 中只是从 metazone 中获取 pq 参数，然后直接调用接口初始化 pq，用于 pq 效果一致性。一般不提供对外客制化。

表 25-4 Hsm 接口

API	说明
fdpu_pq_set	Hsm 中 FDPU 设置 PQ 参数
udpu_hw_pq_cfg	Hsm 中 UDPU 设置 PQ 参数

25.3.3.1 fdpu_pq_set

【描述】

Hsm 中 fdpu 设置 PQ 参数

【语法】

```
int fdpu_pq_set(enum tg_id id, struct pq_config *pq_cfg);
```

【返回值】

0

【参数】

```
enum tg_id {
    TG_ID_UDPU,
    TG_ID_FDPU0_0,
    TG_ID_FDPU0_1,
    TG_ID_FDPU1_0,
    TG_ID_FDPU1_0,
    TG_ID_MAX,
};

struct pq_config {
    uint8_t brightness;
    uint8_t contrast;
    uint8_t saturation;
    int kelvin;

    struct gamma_config gamma;
    struct sharpness_config sharp;
};
```

- id: [IN] 1-4。
- pq_cfg: [IN] 。

25.3.3.2 udpu_hw_pq_cfg

【描述】

Hsm 中 udpu 设置 PQ 参数

【语法】

```
void udpu_hw_pq_cfg(int id, struct color_adjust *pq);
```

【返回值】

none

【参数】

```
struct color_adjust {  
    u16 pq_updated;  
    struct pq_index pq;  
    u16 gamma_updated;  
    struct gamma_info gamma;  
};
```

- id: [IN] 0。
- pq: [IN] 。

25.4 使用说明和注意事项

25.4.1 PQ 调节工具介绍

PQ 调节是在 Gallery APP 中，可以分别独立调节显示 path 输出到屏幕的亮度、对比度、饱和度和色温，而 Gamma 调节使用专用的 gamma tool 来调节，调试效果都是实时显示。

25.4.2 Android 如何进入 PQ 调节界面

方式一：

1. 点击进入桌面上的 CarSetting APP，如下图：

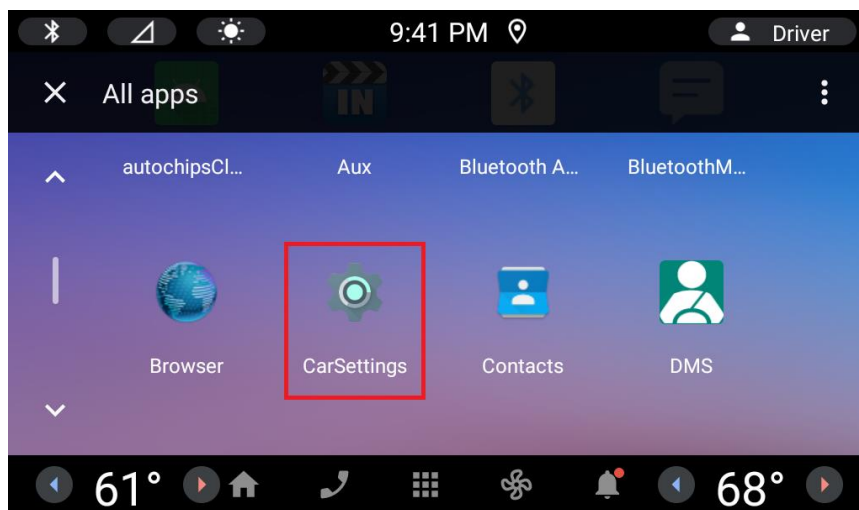


图 25-8 CarSetting APP

2. 下拉找到 ATC Settings，如下图：

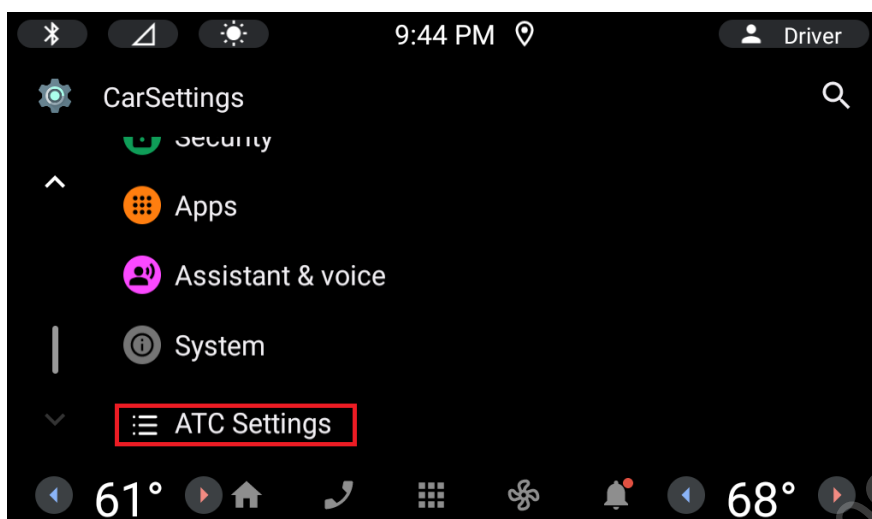


图 25-9 ATC Settings

3. 点击 Display

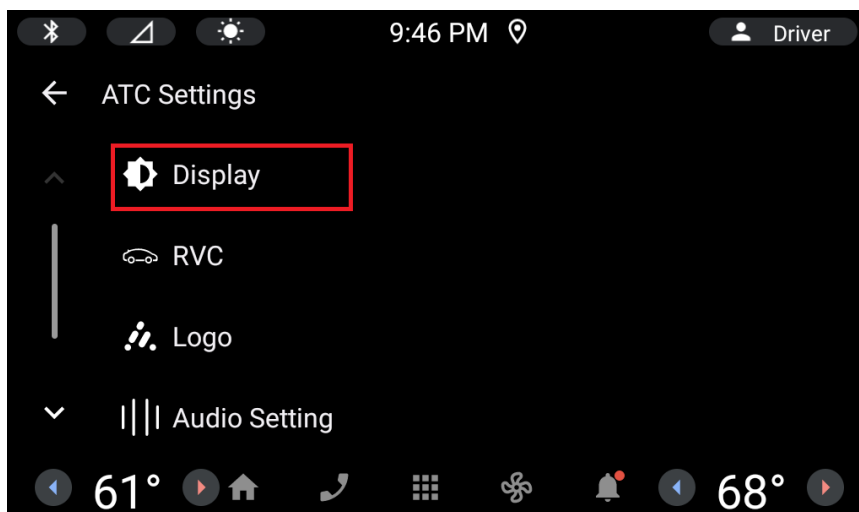


图 25-10 Display

4. 拖动进度条即可调节 PQ

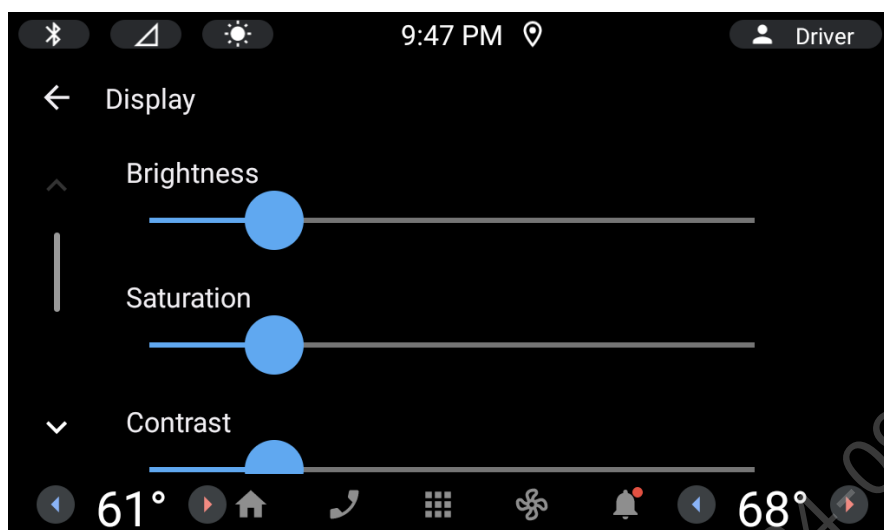


图 25-11 PQ 调节

方式二：

1. 点击进入桌面上的 Gallery APP，如下图：

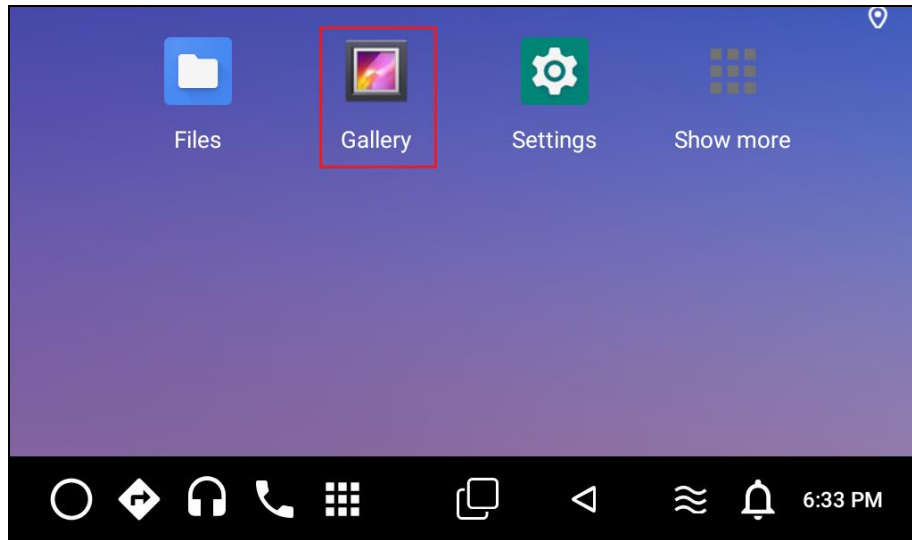


图 25-12 gallery APP

2. 选定需要调节的图片，点击右上角扩展功能键，在下拉列表中选择 PQ 进入调节界面（必须使用触摸屏），如下图：

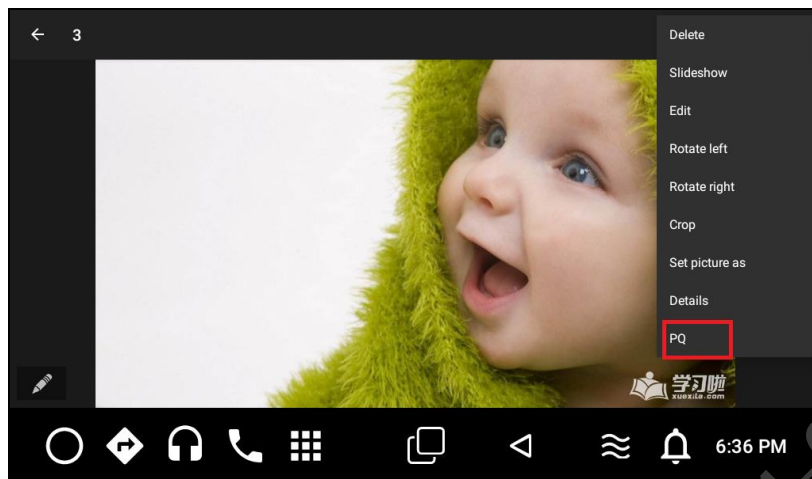


图 25-13 进入 PQ 调节界面

25.4.3 Android 调节界面介绍

25.4.3.1 UI 整体界面

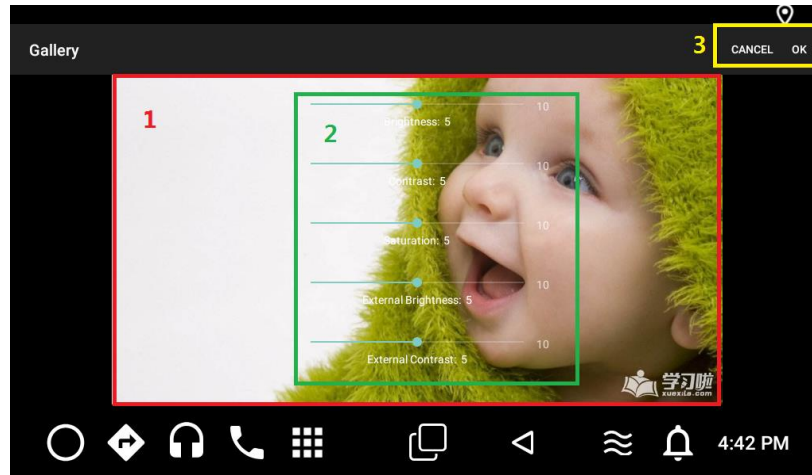


图 25-14 UI 整体界面

1. 图片效果显示区

显示处于调节状态下的图片，可以选择比较有代表性的图片，其可以实时显示 PQ 调试效果。



说明：

此种方式只支持 JPEG 图片格式。

2.PQ 功能调节区

拖动相关拖动条可以调节对应 PQ 效果，Brightness、Contrast、Saturation 分别代表 primary display path 的亮度、对比度、饱和度调节，External Brightness、External Contrast、External Saturation（如果系统有注册 Secondary display，否则不会有相关拖动条）分别代表 secondary display path 的亮度、对比度、饱和度调节。

3.按键

- OK：确认将当前调节好的参数应用到系统并返回至 Gallery 界面；
- CANCEL：取消将当前调节好的参数应用到系统并返回至 Gallery 界面。

25.4.3.2 Brightness

Primary display path（主屏）亮度调节项包含 11 个等级 (0~10)：

- 5：关闭此增强功能（原始状态），即没有调节亮度；

- 0~4: 与原始状态相比减弱亮度;
- 6~10: 与原始状态相比增强亮度, 10 为最大强度。

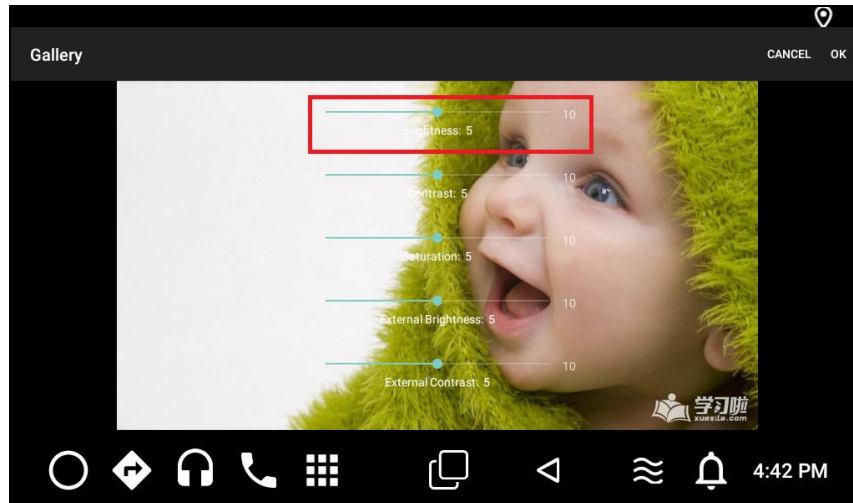


图 25-15 主屏亮度

25.4.3.3 Contrast

Primary display path（主屏）对比度调节项包含 11 个等级 (0~10)：

- 5: 关闭此增强功能（原始状态），即没有增强对比度;
- 0~4: 与原始状态相比减弱对比度;
- 6~10: 与原始状态相比增强对比度, 10 为最大强度。

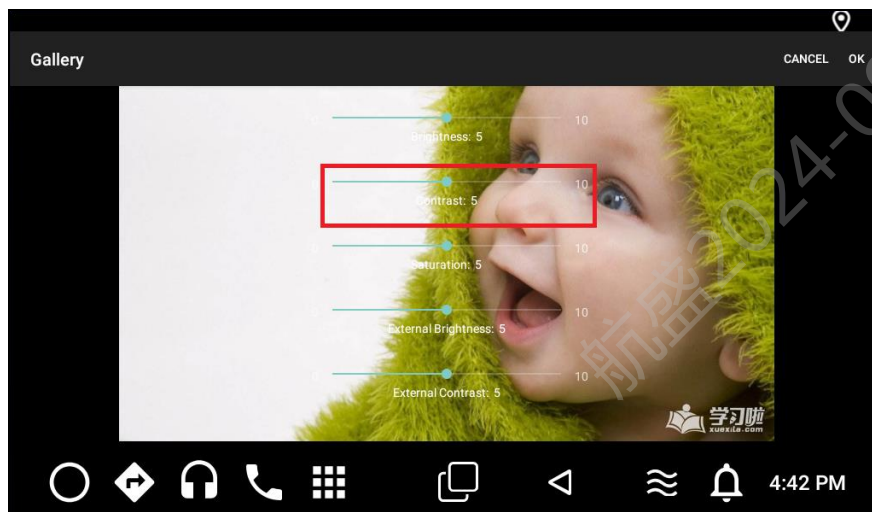


图 25-16 主屏对比度

25.4.3.4 Saturation

Primary display path（主屏）饱和度调节项包含 11 个等级 (0~10)：

- 5：关闭此增强功能（原始状态），即没有增强饱和度；
- 0~4：与原始状态相比减弱饱和度；
- 6~10：与原始状态相比增强饱和度, 10 为最大强度。

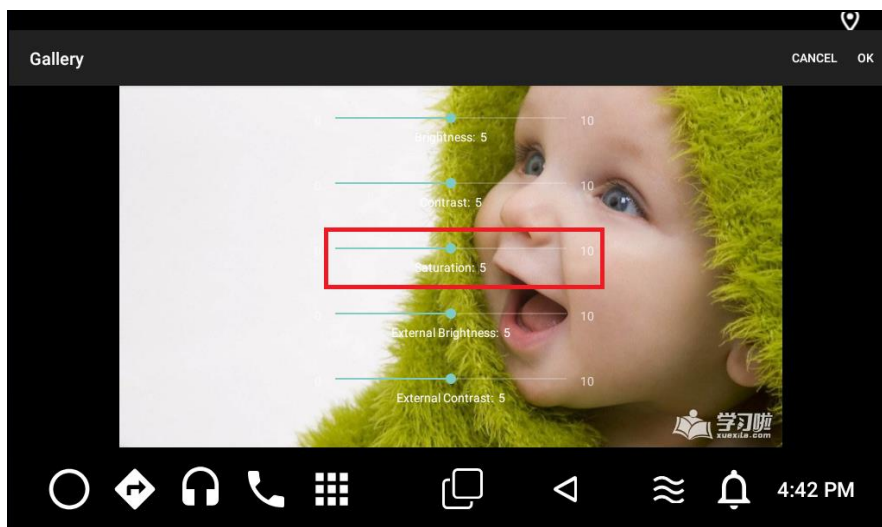


图 25-17 主屏饱和度

25.4.3.5 ColorTemperature

Primary display path（主屏）色温调节项包含 7 个等级 (20~80)：

- 50：关闭此增强功能（原始状态），即没有调节色温；
- 20~40：与原始状态相比调节为暖色调；
- 60~80：与原始状态相比调节为冷色调, 80 为最大强度。

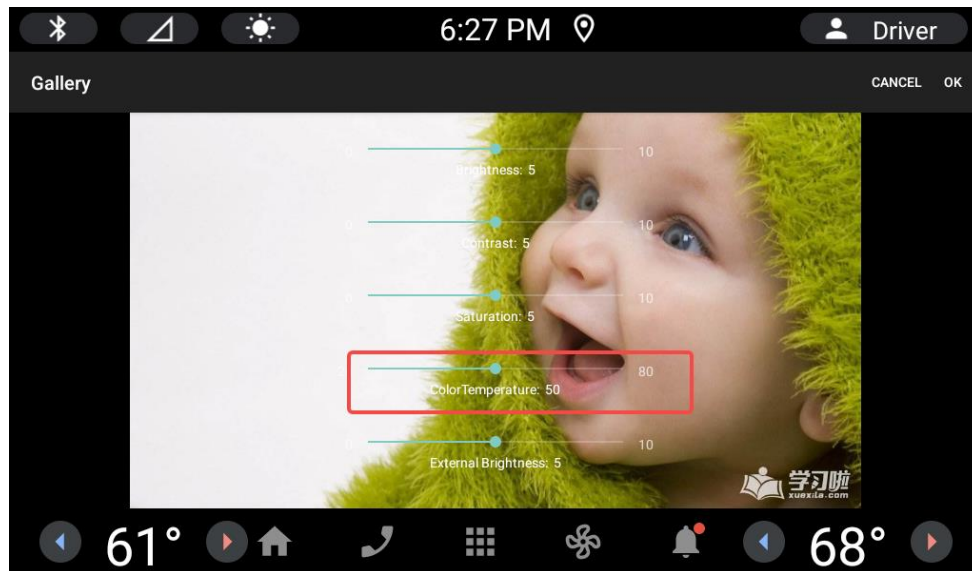


图 25-18 主屏色温

25.4.3.6 External Brightness

Secondary display path（副屏）亮度调节项包含 11 个等级 (0~10)：

- 5：关闭此增强功能（原始状态），即没有调节亮度；
- 0~4：与原始状态相比减弱亮度；
- 6~10：与原始状态相比增强亮度, 10 为最大强度。

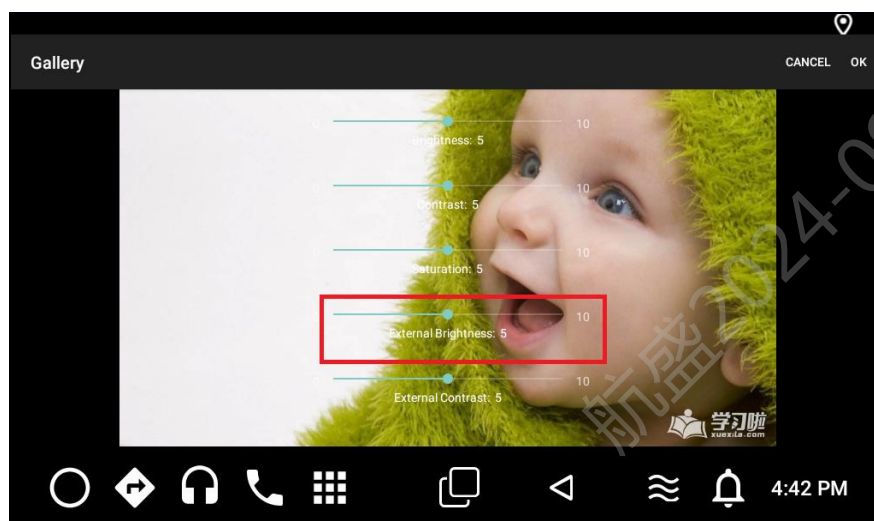


图 25-19 副屏亮度

25.4.3.7 External Contrast

Secondary display path（副屏）对比度调节项包含 11 个等级 (0~10)：

- 5：关闭此增强功能（原始状态），即没有增强对比度；
- 0~4：与原始状态相比减弱对比度；
- 6~10：与原始状态相比增强对比度, 10 为最大强度。

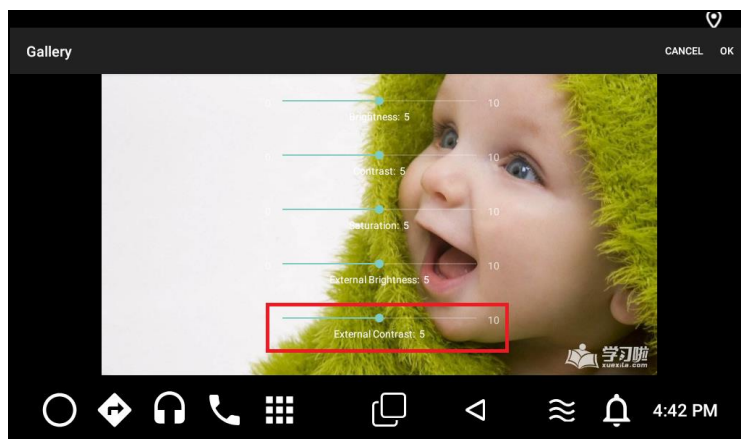


图 25-20 副屏对比度

25.4.3.8 External Saturation

Secondary display path（副屏）饱和度调节项包含 11 个等级 (0~10)：

- 5：关闭此增强功能（原始状态），即没有增强饱和度；
- 0~4：与原始状态相比减弱饱和度；
- 6~10：与原始状态相比增强饱和度, 10 为最大强度。

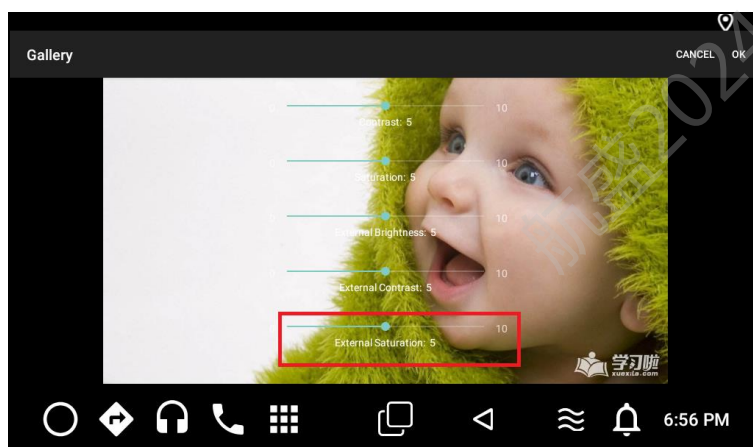


图 25-21 副屏饱和度

25.4.3.9 External ColorTemperature

Secondary display path（扩展屏）色温调节项包含 7 个等级 (20~80):

- 50: 关闭此增强功能（原始状态），即没有调节色温；
- 20~40: 与原始状态相比调节为暖色调；
- 60~80: 与原始状态相比调节为冷色调, 80 为最大强度。

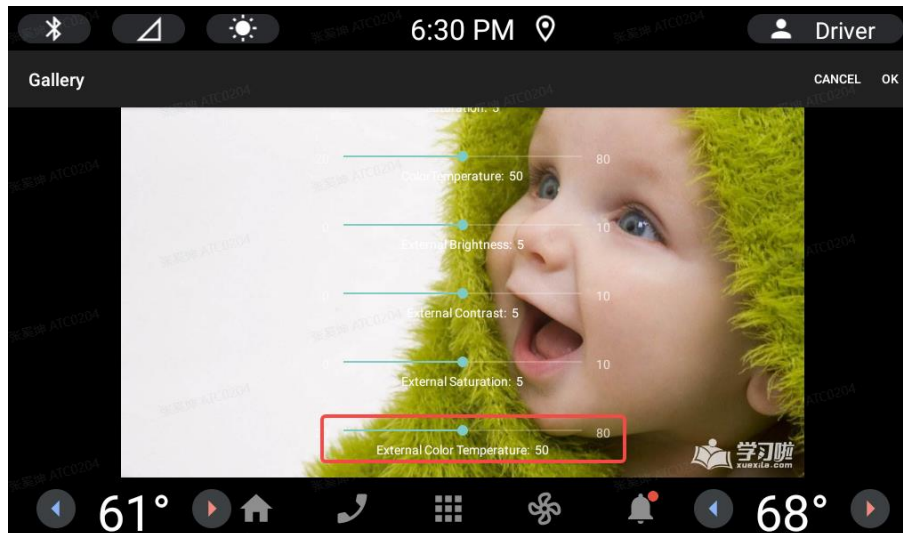


图 25-22 主屏色温

25.4.4 Cluster/adas 如何调节 PQ

使用 displayctrl 命令进行调节，具体 命令如下:

adb 进入 shell 后，使用 displayctrl displayid index value 进行调节

displayed: 0-16

index: kelvin/saturation/contrast/brightness

value: kelvin(20-80), saturation(0-10), contrast(0-10),brightness(0-10)

25.5 用户定制化

25.5.1 应用参数

Tuning 的 PQ 参数值会实时保存至 Metazone 中，下一次重启依然生效。但系统首次启动时，为保证系统第一次启动的 PQ 效果值即为 tuning 的结果应该将 tuning 的 PQ 参数值在 Metazone 里面设置为默认参数。

步骤 1： 记录调节后的 PQ 参数值。

例如：

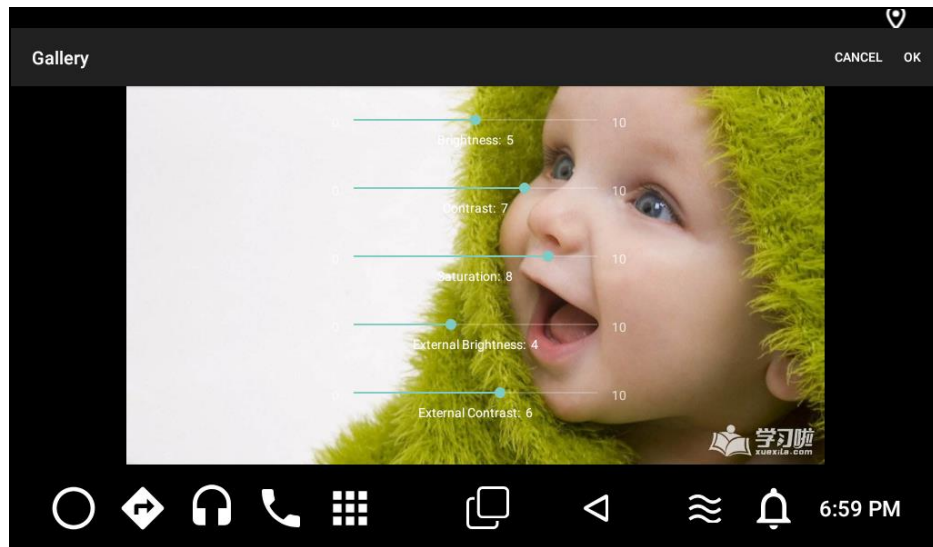


图 25-23 参数应用

如上图：

primary display path PQ 参数值为 Brightness: 5, Contrast: 7, Saturation: 8

secondary display path PQ 参数值为 Brightness: 4, Contrast: 6, Saturation: 7

步骤 2： 将参数填到 metazone.csv(build/projects/cockpit_s_linux_ac8025/metazone/ 或 build/projects/ivi_s_ac8025/metazone/)

	A	B	C	D	E	F	G	H	I	J
108	Dword	METAZONE_SAFETY_DISPO_ICON30_X_Y	4	1	1	0	0	0		
109	Dword	METAZONE_SAFETY_DISPO_ICON31_X_Y	4	1	1	0	0	0		
110	Dword	METAZONE_AVM_TEST	4	1	1	0	0	0		
111	Dword	METAZONE_PQ_TG0_CONTRAST	4	1	1	0	5	5		
112	Dword	METAZONE_PQ_TG0_SATURATION	4	1	1	0	5	5		
113	Dword	METAZONE_PQ_TG0_BRIGHTNESS	4	1	1	0	5	5		
114	Dword	METAZONE_PQ_TG0_KELVIN	4	1	1	0	50	50		
115	Dword	METAZONE_PQ_TG1_CONTRAST	4	1	1	0	5	5		
116	Dword	METAZONE_PQ_TG1_SATURATION	4	1	1	0	5	5		
117	Dword	METAZONE_PQ_TG1_BRIGHTNESS	4	1	1	0	5	5		
118	Dword	METAZONE_PQ_TG1_KELVIN	4	1	1	0	50	50		
119	Dword	METAZONE_PQ_TG2_CONTRAST	4	1	1	0	5	5		
120	Dword	METAZONE_PQ_TG2_SATURATION	4	1	1	0	5	5		
121	Dword	METAZONE_PQ_TG2_BRIGHTNESS	4	1	1	0	5	5		
122	Dword	METAZONE_PQ_TG2_KELVIN	4	1	1	0	50	50		
123	Dword	METAZONE_PQ_TG3_CONTRAST	4	1	1	0	5	5		
124	Dword	METAZONE_PQ_TG3_SATURATION	4	1	1	0	5	5		
125	Dword	METAZONE_PQ_TG3_BRIGHTNESS	4	1	1	0	5	5		
126	Dword	METAZONE_PQ_TG3_KELVIN	4	1	1	0	50	50		
127	Dword	METAZONE_PQ_TG4_CONTRAST	4	1	1	0	5	5		
128	Dword	METAZONE_PQ_TG4_SATURATION	4	1	1	0	5	5		
129	Dword	METAZONE_PQ_TG4_BRIGHTNESS	4	1	1	0	5	5		
130	Dword	METAZONE_PQ_TG4_KELVIN	4	1	1	0	50	50		
131	Dword	METAZONE_RVC_ENABLE_SWITCH_CHANNEL	4	1	1	0	0	0		
132	Dword	METAZONE_SENSOR_PATTERN_MODE	4	1	1	0	0	0		
133	Dword	METAZONE_OTA_TEST	4	1	1	1	1	0		
134	Dword	METAZONE_MIP10_WORKING_OS	4	1	1	0	5	5		
135	Dword	METAZONE_MIP11_WORKING_OS	4	1	1	0	0	0		
136	Dword	METAZONE_MIP12_WORKING_OS	4	1	1	0	0	0		

图 25-24 Metazone 配置

步骤 3:

- 重编 Metazone,生成 metazone.bin

```
cd ivi/android
prebuild/build-tools/linux-x86/bin/ninja -f out/combined-full_ac8x_car.ninja
out/target/product/ac8x_demo/metazone.bin
```

目标文件生成目录: ivi/android/out/target/product/ac8x_demo/metazone.bin

(2) 将生成的 metazone.bin 通过烧写工具更新平台端 Metazone 分区

步骤 4: 重启 demo 板平台, 查看显示效果。

25.5.2 Android PQ 接口

用户可使用 com.autochips.pq.Atcpq 中的接口在其它的 app 中动态设置 PQ。

可参考如下路径中的文件:

source/android/vendor/autochips/proprietary/packages/apps/AtcGallery2/autochips/pqtool/

25.5.3 linux PQ 接口

用户可直接使用 libatcpq 库中的接口。

可参考如下路径中的文件：

source/android/vendor/autochips/proprietary/hardware/display/libs/libpq

25.5.4 Hsm 调用 PQ 接口

用户可以在 Hsm 其它模块通过 extern 25.3.3.1 章节描述的接口 fdpu_pq_set/udpu_hw_pq_cfg，来设置 PQ 参数。

25.6 典型问题

25.6.1 拖动某一调节项，显示没有变化

【问题描述】

出现具体的调节项界面后，拖动某一调节项，显示看起来没有变化，设置没有生效。

【解决方案】

1. 选择合适的测试 pattern，尽量选择色彩丰富的 pattern 来测试饱和度、色度和对比度等；
2. 检查 code 执行通路是否通，包含 APP 中调用、JNI 层 so、driver 处理及返回等。

25.6.2 PQ tool 中调节好一组参数后，重启平台，PQ 效果又恢复到原始状态

【问题描述】

在 PQ tool 中调节好一组参数后点击 OK，重启平台，PQ 效果又恢复到原始状态。

【解决方案】

1. 检查 PQ 参数是否正常写入 Metazone 参数

例如，以关键字“libatcpq”搜索 android framework 层 log，查看搜索结果中是否有 error 的 log，比如 log 中包含[E] [libatcpq]，如果有错误信息请联系 FAE 分析具体失败的原因。

2. 检查 PQ 初始化阶段读取 Metazone 参数是否正常？

比如以关键字“[PQ]”搜索 Hsm 中 log，查看搜索结果中是否有类似于“[PQ]: Get METAZONE_PQ_PRIMARY_BRIGHTNESS failed”的信息，如果有错误信息请联系 FAE 分析具体失败的原因。

26 FreeRTOS Display

26.1 简介

26.1.1 软硬件环境

本文档描述的 AC8025 项目中的 FreeRTOS display feature 基于 FreeRT OS 系统。

26.1.2 功能

FreeRTOS Display 提供如下功能：

- FreeRTOS application 图像显示和隐藏功能
- FreeRTOS application 图像显示区域，像素格式，alpha 等参数的设置
- 冻屏检测回掉注册，冻屏检测使能
- 显示区域完整性校验回调注册，完整性校验使能
- FreeRTOS application 申请绘制 buffer
- 提供 display power on / off 功能

26.1.3 代码结构及文件描述

表 26-1 Display Framework 代码结构及描述

文件名	文件路径	文件描述
display_fw.c	safety/hsm/drivers/display/framework/	FreeRTOS Display 接口实现
display_fw.h	safety/hsm/drivers/display/framework/	FreeRTOS Display 数据结构定义
display.h	safety/hsm/include/	FreeRTOS Display 接口头文件
display_fourcc.h	safety/hsm/include/	支持的 pixel format 以及旋转和 Flip 格式

26.2 软件设计

26.2.1 架构图

在AC8025方案中，ATC开发了FreeRTOS display用于支持快速显示机制。用户基于Display framework 开发，相关接口和主流程在26.2.2章节以及第26.3 节接口说明中有详细描述。

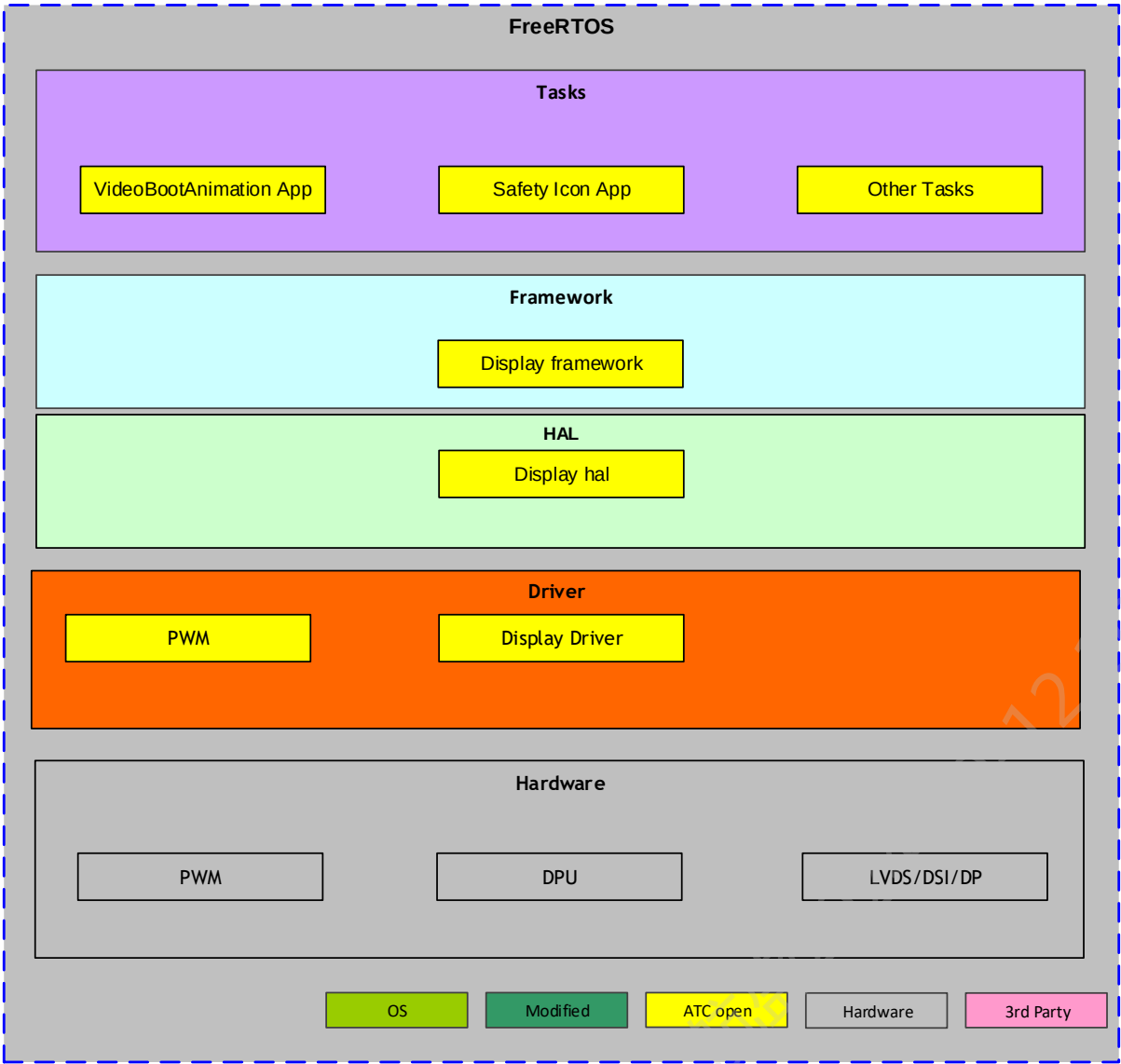


图 26-1 FreeRtos Display 架构图

26.2.2 流程图

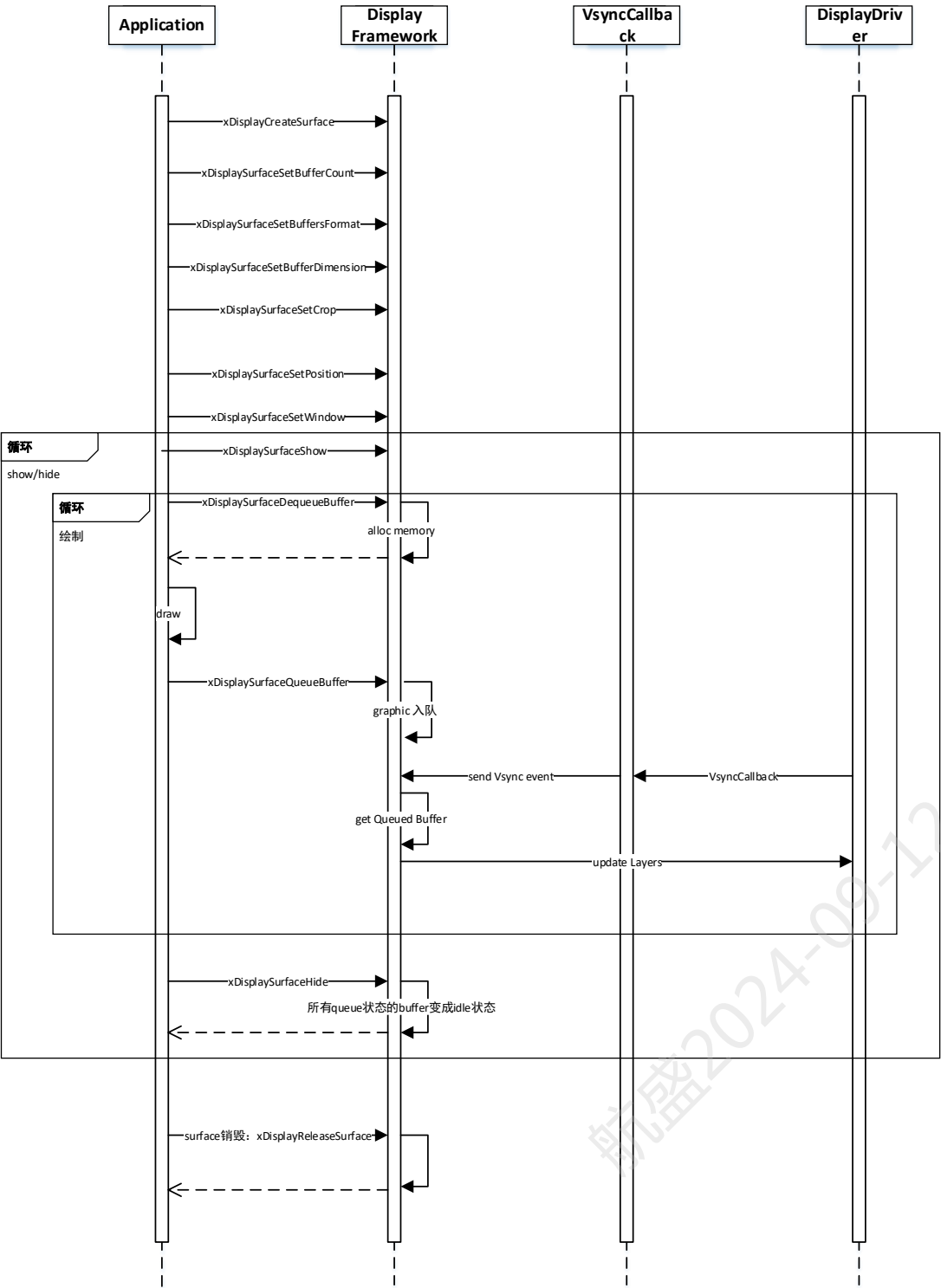


图 26-2 FreeRTOS application 流程图

26.3 接口描述

26.3.1 API

表 26-2 FreeRTOS Display Framework API

API	说明
xFreeRtosDisplayInit	FreeRtos Display 总初始化接口
xDisplayGetCount	获取所有可显示的屏 ID
xDisplayGetResolution	获取某个屏的分辨率
xDisplayCreateSurface	创建 surface 对象
xDisplaySurfaceSetBufferCount	设置 buffer 数量
xDisplaySurfaceSetBuffersFormat	设置 buffer pixel format 格式
xDisplaySurfaceSetBuffersFlags	设置 gamma 使能, SRGB 等 flag
xDisplaySurfaceSetBuffersDimension	设置 buffer 大小
xDisplaySurfaceSetCrop	设置 buffer 裁剪区域
xDisplaySurfaceSetPosition	设置 application 在屏幕显示的起始点
xDisplaySurfaceSetWindow	设置 application 在屏幕上显示区域宽和高
xDisplaySurfaceSetTransform	设置旋转角度
xDisplaySurfaceSetColorkey	设置 colorkey
xDisplaySurfaceSetCapability	申请使用 colorkey / 全局 alpha 功能
xDisplaySurfaceSetGlobalAlpha	设置全局 Alpha
xDisplaySurfaceSetColorRange	设置 color full or limit range type
xDisplaySurfaceSetColorEncoding	设置 color 编码格式, BT601 / BT709 / BT2020
xDisplaySurfaceDequeueBuffer	从 Display Framework 申请一块 buffer
xDisplaySurfaceQueueBuffer	将 Application 绘制的 buffer 送给 Display Framework
xDisplaySurfaceCancelBuffer	将 dequeue 的 buffer 归还给 Display Framework
xDisplaySurfaceShow	显示 surface
xDisplaySurfaceHide	隐藏 surface
xDisplayReleaseSurface	Release surface
xDisplayAlloc	申请 buffer
xDisplayFree	释放 buffer
xDisplaySurfaceSetBufferAllocType	设置 surface 对应的 buffer 是有由 application 自己申请, 还是 display framework 替代申请
xDisplayRsvHeaderDump	Dump Rsv memory debug Header 信息
xDisplayRsvNotFree	Application 设置申请的 rsv size 不释放

xDisplayRegisterFrzDetCb	注册冻屏异常回掉
xDisplayFrzDetEnable	使能冻屏检测功能
xDisplayCreateCrcRegion	创建完整性校验区域
xDisplayCrcCheckEnable	使能完整性校验功能
xDisplayCrcRegionUdate	更新完整性校验区域信息
xDisplayCrcGoldenUpdate	更新完整性校验区域 golden 值
xDisplayDestoryCrcRegion	销毁完整性校验区域
xDisplayModeSet	对指定 tg 进行 power on/off
xDisplayRsvMemRelease	Display Rsv memory 释放

26.3.1.1 xFreeRtosDisplayInit

【描述】

FreeRtos Display Init Func, Must be invoked before create display surface

【语法】

```
BaseType_t xFreeRtosDisplayInit(void);
```

【参数】

none

【返回值】

[OUT] success: return pdTRUE, otherwise return pdFALSE.

26.3.1.2 xDisplayGetCount

【描述】

Get display id mask you can show. Must be invoked before create display surface

【语法】

```
int xDisplayGetCount(unsigned int *displays_mask, unsigned int *display_num);
```

【参数】

- [OUT]display_mask: The display id mask.
- [OUT] display_num: The display id nums.

【返回值】

[OUT] success: return > 0, otherwise return 0.

【注意】

FreeRTOS 支持的 display ID 如下所示：

```
enum display_id {
    IVI_Display_0,
    IVI_Display_1,
    IVI_Display_2,
    IVI_Display_3,
    IVI_Display_4,
    CLUSTER_Display_0,
    CLUSTER_Display_1,
    CLUSTER_Display_2,
    CLUSTER_Display_3,
    CLUSTER_Display_4,
    ADAS_Display_0,
    ADAS_Display_1,
    ADAS_Display_2,
    ADAS_Display_3,
    ADAS_Display_4,
    DISPLAY_MAX,
    TOUCH_Display_0 = ADAS_Display_0,
}
```

display_mask 可以是上述的任意组合，比如 display_mask = 1 << IVI_Display_1 | 1 << IVI_Display_0。

xDisplayGetCount 的具体使用可参考如下示例：

```
unsigned int uiDispMask = 0, uiDispNum = 0;
unsigned int i = 0;
unsigned short displayId;

if (!xDisplayGetCount(&uiDispMask, &uiDispNum)) {
    printf("get Display Count error\n");
    return error;
}
```

```
for (i = 0; i < DISPLAY_MAX; i++) {
    if (!(uiDispMask & (0x1 << i))) {
        continue;
    }
    displayId = i; // displayId 就对应可上屏显示的 Id，也就是 xDisplayCreateSurface
    的入参
}
```

26.3.1.3 xDisplayGetResolution

【描述】

Get display resolution(width and height). Not must be invoked

【语法】

```
BaseType_t xDisplayGetResolution(unsigned short sDisplayId, int *Width, int *Height);
```

【参数】

- [IN] sDisplayId: The display id
- [OUT] Width: The display's width.
- [OUT] Hidth: The display's height.

【返回值】

[OUT]success:return pdTRUE, otherwise, return pdFALSE.

26.3.1.4 xDisplayCreateSurface

【描述】

Create a surface object for display. You must invoke this interface before show something

【语法】

```
SurfaceHandle_t xDisplayCreateSurface(unsigned short sZorder, unsigned short sDisplayId);
```

【参数】

- [IN]sZorder: The surface Zorder.
- [IN] sDisplayId: The display id.

【返回值】

[OUT] success: return surface object handle, otherwise return NULL.

【注意】

关于 sZorder 的值请参考 [26.3.2.2 SurfaceZorder](#) 描述。

26.3.1.5 xDisplaySurfaceSetBufferCount

【描述】

Set buffer count of the specified surface object. You should set buffer count before dequeueBuffers;

【语法】

```
BaseType_t xDisplaySurfaceSetBufferCount(SurfaceHandle_t xSurf, unsigned short sCount);
```

【参数】

- [IN]xSurf: The surface object.
- [IN]sCount: Buffer count you want to set.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.6 xDisplaySurfaceSetBuffersFormat

【描述】

Set buffer format of the specified surface object. You should set buffer format before dequeueBuffers.

【语法】

```
BaseType_t xDisplaySurfaceSetBuffersFormat(SurfaceHandle_t xSurf, unsigned long lFormat);
```

【参数】

- [IN]xSurf: The surface object.
- [IN]lFormat: Buffer format you want to set.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

【注意】

支持的 Pixel format 可参考 `safety/hsm/include/display_fourcc.h` 里的定义。

26.3.1.7 xDisplaySurfaceSetBuffersFlags

【描述】

Set buffer flags of the specified surface object. Not Must be Invoked

【语法】

BaseType_t xDisplaySurfaceSetBuffersFlags(SurfaceHandle_t xSurf, unsigned int flags);

【参数】

- [IN]xSurf: The surface object.
- [IN]flags: Buffer flags you want to set.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

【注意】

Flags 描述: buffer 特殊格式设置, 当前系统软件未使用此 flags, 无需调用。

26.3.1.8 xDisplaySurfaceSetBuffersDimension

【描述】

DO NOT Support Resize the Buffer Size.

Set buffer Dimension of the specified surface object. You should set buffer dimension before dequeueBuffers;

【语法】

BaseType_t xDisplaySurfaceSetBuffersDimension(SurfaceHandle_t xSurf, unsigned short sWidth, unsigned short sHeight);

【参数】

- [IN]xSurf: The surface object.
- [IN]sWidth: Buffer width you want to set.
- [IN]sHeight: Buffer height you want to set.

【返回值】

[OUT] success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.9 xDisplaySurfaceSetCrop

【描述】

Set Crop of the specified surface object. Not Must be Invoked

【语法】

BaseType_t xDisplaySurfaceSetCrop(SurfaceHandle_t xSurf, unsigned short sX, unsigned short sY, unsigned short sWidth, unsigned short sHeight);

【参数】

- [IN]xSurf: The surface object.
- [IN]sX: Crop x you want to set.
- [IN]sY: Crop y you want to set.
- [IN]sWidth: Crop width you want to set.
- [IN]sHeight: Crop height you want to set.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.10 xDisplaySurfaceSetPosition

【描述】

Set position on screen of the specified surface object. Not Must be Invoked

【语法】

BaseType_t xDisplaySurfaceSetPosition(SurfaceHandle_t xSurf, unsigned short sX, unsigned short sY);

【参数】

- [IN]xSurf: The surface object.
- [IN]sX: Distination x you want to set.
- [IN]sY: Distination y you want to set.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.11 xDisplaySurfaceSetWindow

【描述】

Set region on screen of the specified surface object. Not Must be Invoked

【语法】

BaseType_t xDisplaySurfaceSetWindow(SurfaceHandle_t xSurf, unsigned short sX, unsigned short sY, unsigned short sWidth, unsigned short sHeight);

【参数】

- [IN]xSurf: The surface object.
- [IN]sX: Distination x you want to set.
- [IN]sY: Distination y you want to set.
- [IN]sWidth: Distination width you want to set.
- [IN]sHeight: Distination height you want to set.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.12 xDisplaySurfaceSetTransform

【描述】

Set transform of the specified surface object. Firstly Flip, Seconedly Rotate.

Not Must be Invoked;

【语法】

```
BaseType_t xDisplaySurfaceSetTransform(SurfaceHandle_t xSurf, unsigned short sTransform);
```

【参数】

- [IN]xSurf: The surface object.
- [IN]sTransform: Transform you want to set, such as rotate,flip.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

【注意】

支持的 Transform 格式可参考 safety/hsm/include/display_fourcc.h 里 enum SurfaceTransform_t 的定义。

26.3.1.13 xDisplaySurfaceSetColorkey

【描述】

Set colorkey of the specified surface object. Not Must be Invoked;

【语法】

```
BaseType_t xDisplaySurfaceSetColorkey(SurfaceHandle_t xSurf, unsigned long lColor);
```

【参数】

- [IN]xSurf: The surface object.
- [IN]sTransform: Colorkey you want to set.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.14 xDisplaySurfaceSetCapability

【描述】

Set Capability of the specified surface object. Not Must be Invoked;

【语法】

BaseType_t xDisplaySurfaceSetCapability(SurfaceHandle_t xSurf, unsigned long lCapability);

【参数】

- [IN]xSurf: The surface object.
- [IN]lCapability: Capability you want to set. Such as CAPS_LOCAL_ALPHA(Default) or CAPS_COLOR_KEY.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.15 xDisplaySurfaceSetGlobalAlpha

【描述】

Set global alpha of the specified surface object. Not Must be Invoked;

【语法】

BaseType_t xDisplaySurfaceSetGlobalAlpha(SurfaceHandle_t xSurf, unsigned char cGlobalAlpha);

【参数】

- [IN]xSurf: The surface object.
- [IN]sTransform: global alpha you want to set.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.16 xDisplaySurfaceSetColorRange

【描述】

Set color range type . Not Must be Invoked;

【语法】

```
BaseType_t xDisplaySurfaceSetColorRange(SurfaceHandle_t xSurf, enum drm_color_range
color_range);
```

【参数】

[IN]xSurf: The surface object.

[IN]color_range: color range type.

【返回值】

[OUT]success:return pdTRUE, otherwise, return pdFALSE.

【注意】

```
enum drm_color_range {
    DRM_COLOR_YCBCR_LIMITED_RANGE,
    DRM_COLOR_YCBCR_FULL_RANGE,
    DRM_COLOR_RANGE_MAX
}
```

26.3.1.17 xDisplaySurfaceSetColorEncoding

【描述】

Set color encoding type . Not Must be Invoked;

【语法】

```
BaseType_t xDisplaySurfaceSetColorEncoding(SurfaceHandle_t xSurf, enum
drm_color_encoding color_encoding);
```

【参数】

[IN]xSurf: The surface object.

[IN]color_range: color encoding type.

【返回值】

[OUT]success:return pdTRUE, otherwise, return pdFALSE.

【注意】

```
enum drm_color_range {  
    DRM_COLOR_YCBCR_BT601,  
    DRM_COLOR_YCBCR_BT709,  
    DRM_COLOR_YCBCR_BT2020,  
    DRM_COLOR_ENCODING_MAX  
}
```

26.3.1.18 xDisplaySurfaceDequeueBuffer

【描述】

Dequeue Buffer from the specified surface object.

【语法】

```
BaseType_t xDisplaySurfaceDequeueBuffer(SurfaceHandle_t xSurf, GraphicBufferHandle_t  
*pxBuffer, TickType_t xWait);
```

【参数】

- [IN]xSurf: The surface object.
- [IN]pxBuffer: GraphicBufferHandle_t ptr
- [IN]xWait: Tick you want to wait. If you want dequeueBuffer with block mode, please set portMAX_DELAY.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.19 xDisplaySurfaceQueueBuffer

【描述】

Queue Buffer to the specified surface object.

【语法】

```
BaseType_t xDisplaySurfaceQueueBuffer(SurfaceHandle_t xSurf, GraphicBufferHandle_t  
xBuffer);
```

【参数】

- [IN]xSurf: The surface object.
- [IN]xBuffer: GraphicBuffer you want to queue in.

【返回值】

[OUT]success:return pdTRUE, otherwise, return pdFALSE.

26.3.1.20 xDisplaySurfaceCancelBuffer

【描述】

Cancel Buffer to the specified surface object.Buffer canceled will not be displayed on screen.

【语法】

```
BaseType_t xDisplaySurfaceCancelBuffer(SurfaceHandle_t xSurf, GraphicBufferHandle_t xBuffer);
```

【参数】

- [IN]xSurf: The surface object.
- [IN]xBuffer: GraphicBuffer you want to cancel.

【返回值】

[OUT]success:return pdTRUE, otherwise, return pdFALSE.

26.3.1.21 xDisplaySurfaceShow

【描述】

Show the surface object on screen.

【语法】

```
BaseType_t xDisplaySurfaceShow(SurfaceHandle_t xSurf);
```

【参数】

[IN]xSurf: The surface object.

【返回值】

[OUT] success:return pdTRUE, otherwise, return pdFALSE.

26.3.1.22 xDisplaySurfaceHide

【描述】

Hide the surface object on screen.

【语法】

```
BaseType_t xDisplaySurfaceHide(SurfaceHandle_t xSurf);
```

【参数】

[IN]xSurf: The surface object.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.23 xDisplayReleaseSurface

【描述】

Release the surface object.

【语法】

```
BaseType_t xDisplayReleaseSurface(SurfaceHandle_t xSurf);
```

【参数】

[IN]xSurf: The surface object.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.24 xDisplayAlloc

【描述】

Alloc memory from reserve memory region. Not Must be Invoked

【语法】

```
BaseType_t xDisplayAlloc(unsigned int iSize, unsigned long long *lPhyAddr, unsigned int iAlign);
```

【参数】

- [IN]iSize: Buffer size you want to set.
- [IN]lPhyAddr: allocated buffer address ptr.
- [IN]iAlign: Alignment you want to set.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.25 xDisplayFree

【描述】

Alloc memory from reserve memory region. Not Must be Invoked

【语法】

BaseType_t xDisplayFree(unsigned long long lMemBase);

【参数】

[IN]lMemBase: Memory address you want to free.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.26 xDisplaySurfaceSetBufferAllocType

【描述】

Alloc memory from reserve memory region. Not Must be Invoked

【语法】

BaseType_t xDisplaySurfaceSetBufferAllocType(SurfaceHandle_t xSurf, BaseType_t sAllocWithDequeue);

【参数】

- [IN]xSurf: The surface object.
- [IN]sAllocWithDequeue: True will alloc from framework, false will form application-self.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.27 xDisplayRegisterFrzDetCb

【描述】

Register the display Froze Detect callback. Must invoke the function after display init

【语法】

BaseType_t xDisplayRegisterFrzDetCb(unsigned short sDisplayId, ForzeCallback cb, void *pvdata);

【参数】

- [IN] sDisplayId: specified the display id that will be queried.
- [IN] cb: specified the callback function of register.
- [IN] pvdata: private data.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

【注意】

```
typedef unsigned int (*xFrozeCallback)(void *pvdata, int event)
```

26.3.1.28 xDisplayFrzDetEnable

【描述】

Enable or disable the display Froze Detect feature. Must invoke the function after xDisplayRegisterFrzDetCb

【语法】

```
BaseType_t xDisplayFrzDetEnable(unsigned short sDisplayId, unsigned short enable, unsigned short x, unsigned short y, unsigned short w, unsigned short h, int type, unsigned region_id, unsigned int frame_interval);
```

【参数】

- [IN] sDisplayId: specified the display id that will be queried.
- [IN] enable: enable or disable feature.
- [IN] x: specified the horizontal origin point of region.
- [IN] y: specified the vertical origin point of region.
- [IN] w: specified the horizontal width of region.
- [IN] h: specified the vertical height of region.
- [IN] frame_interval: specified the frame interval fo freezing screen detection.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.29 xDisplayCreateCrcRegion

【描述】

Create the display Crc Region to validate the region data onscreen whether meeting expection, Must invoke the function after xDisplayInit.

【语法】

```
CrcRegionHandle_t xDisplayCreateCrcRegion(unsigned short sDisplayId, unsigned short sIdx, xCrcCallback cb, void *pvdata, unsigned short x, unsigned short y, unsigned short w, unsigned short h, unsigned int golden);
```

【参数】

- [IN]sID: specified the display id that will be queried..

- [IN]cb: specified the callback function of register, will be invoke when crc validate failure.
- [IN]pvdata: specified the private data of callback function.
- [IN]x: specified the horizontal origin point of region.
- [IN]y: specified the vertical origin point of region.
- [IN]w: specified the horizontal width of region.
- [IN]h: specified the vertical height of region.
- [IN]golden: specified the crc golden value of the region.

【返回值】

[OUT]success: return a valid handle, otherwise, return Null.

【注意】

CrcRegionHandle_t 数据结构见 26.3.2.3 章节

26.3.1.30 xDisplayCrcCheckEnable

【描述】

Enable or disable the display Crc check of region. Must invoke the functions after xDisplayInit.

【语法】

```
BaseType_t xDisplayCrcCheckEnable(CrcRegionHandle_t CrcRegion, unsigned short enable);
```

【参数】

- [IN]CrcRegion: specified the display region that will be operate.
- [IN]enable: specified enable or disable the crc check.

【返回值】

[OUT] success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.31 xDisplayCrcRegionUpdate

【描述】

Update the position and size of the region. Must invoke the functions after xDisplayCreateCrcRegion.

【语法】

```
BaseType_t xDisplayCrcRegionUpdate(CrcRegionHandle_t CrcRegion, unsigned short x, unsigned short y, unsigned short w, unsigned short h);
```

【参数】

- * [IN]CrcRegion: specified the display region that will be operate.
- * [IN]x: specified the horizontal origin point of region.
- * [IN]y: specified the vertical origin point of region.
- * [IN]w: specified the horizontal width of region.
- * [IN]h: specified the vertical height of region.

【返回值】

[OUT] success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.32 xDisplayCrcGoldenUpdate

【描述】

Update the crc golden value of the region. Must invoke the functions fater xDisplayCreateCrcRegion.

【语法】

```
BaseType_t xDisplayCrcGoldenUpdate(CrcRegionHandle_t CrcRegion, unsigned int golden);
```

【参数】

- [IN]CrcRegion: specified the display region that will be operate.
- [IN]golden: specified the golden value of the crc of region.

【返回值】

[OUT] success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.33 xDisplayRsvHeaderDump

【描述】

Dump debug info of the reserve memory

【语法】

```
BaseType_t xDisplayRsvHeaderDump(void);
```

【参数】

None

【返回值】

[OUT] success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.34 xDisplayRsvNotFree

【描述】

The size applied for is not released

【语法】

```
BaseType_t xDisplayRsvNotFree(unsigned int iSize);
```

【参数】

[IN]iSize: buffer size

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.35 xDisplayModeSet

【描述】

Set tg all display power on/off status

【语法】

```
BaseType_t xDisplayModeSet(unsigned short tg_id, unsigned int power_on);
```

【参数】

- [IN]tg_id: tg id.
- [IN]power_on: pdTRUE or pdFALSE.

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.1.36 xDisplayRsvMemRelease

【描述】

Display Rsv memory release.

【语法】

```
BaseType_t xDisplayRsvMemRelease(void);
```

【参数】

None

【返回值】

[OUT]success: return pdTRUE, otherwise, return pdFALSE.

26.3.2 数据结构

表 26-3 FreeRTOS Display 数据结构

数据结构	说明
GraphicBufferHandle_t	Graphic buffer handle structure
SurfaceZorder	Surface Zorder
CrcRegionHandle_t	Crc Region structure

26.3.2.1 GraphicBufferHandle_t

```
typedef struct GraphicBuffer {
    unsigned short sWidth;
    unsigned short sHeight;
    unsigned long lFormat;
    unsigned long lSize;
    unsigned short sPitch[MAX_GRAPHIC_BUFFER_PLANE_COUNT];
    unsigned short sPlaneCount;
    unsigned long long pcHandle[MAX_GRAPHIC_BUFFER_PLANE_COUNT];/*64bit address, we
    should shift it for display/avin hardware.*/
    char cReserved[];
} *GraphicBufferHandle_t;
```

26.3.2.2 SurfaceZorder

Zorder 越大，离人眼越近。如果画面有重合，重合部分 zorder 高的会覆盖 zorder 低的画面。

一些使用场景对应的 Zorder 值，系统已经帮忙定义好，使用如下宏即可：

safety/hsm/include/display.h

```
typedef enum {
    SurfaceZorderSafety = Z_SAFETY_DEFAULT;
    SurfaceZorderAee = Z_AEE_DEFAULT;
    SurfaceZorderRVCUI = Z_RVC_UI_PQ;
    SurfaceZorderRVCStaticUI = Z_RVC_UI_NORMAL;
    SurfaceZorderRVCVideo = Z_RVC_VIDEO;
    SurfaceZorderLogo = Z_LOGO_DEFAULT;
}
```

上面具体 Zorder 值的大小可参考 `safety/hsm/include/display_zorder.h` 的定义，如下为截取的信息，例如 Logo 的 zorder 值为 11。

```
#define Z_TYPE_BASE_LEN 10

#define NEXT_BASE(x)    (x + Z_TYPE_BASE_LEN)

enum disp_zorder {
    Z_TYPE_DEFAULT,
    Z_TYPE_KMS_BASE,
    Z_TYPE_LOGO_BASE = NEXT_BASE(Z_TYPE_KMS_BASE),
    Z_TYPE_VBA_BASE = NEXT_BASE(Z_TYPE_LOGO_BASE),
    Z_TYPE_FAST_DISP_BASE = NEXT_BASE(Z_TYPE_VBA_BASE),
    Z_TYPE_RVC_BASE = NEXT_BASE(Z_TYPE_FAST_DISP_BASE),
    Z_TYPE_AVM_BASE = NEXT_BASE(Z_TYPE_RVC_BASE),
    Z_TYPE_AEE_BASE = NEXT_BASE(Z_TYPE_AVM_BASE),
    Z_TYPE_TEMP_BASE = NEXT_BASE(Z_TYPE_AEE_BASE),
    Z_TYPE_SAFETY_BASE = NEXT_BASE(Z_TYPE_TEMP_BASE),
    Z_TYPE_MAX,
};

enum disp_zorder_logo {
    Z_LOGO_DEFAULT = Z_TYPE_LOGO_BASE,
};

enum disp_zorder_vba {
    Z_VBA_DEFAULT = Z_TYPE_VBA_BASE,
};

enum disp_zorder_safety {
    Z_SAFETY_DEFAULT = Z_TYPE_SAFETY_BASE,
};

enum disp_zorder_aee {
    Z_AEE_DEFAULT = Z_TYPE_AEE_BASE,
};

enum disp_zorder_rvc {
    Z_RVC_VIDEO = Z_TYPE_RVC_BASE,
    Z_RVC_UI_NORMAL,
    Z_RVC_UI_PQ,
```

```
};
enum disp_zorder_avm {
    Z_AVM_VIDEO = Z_RVC_VIDEO,
    Z_AVM_UI,
};
```

26.3.2.3 CrcRegionHandle_t

```
typedef struct CrcRegion {
    unsigned short sID;
    unsigned short sEn;
    unsigned short sIdx;
    unsigned short sX;
    unsigned short sY;
    unsigned short sWidth;
    unsigned short sHeight;
    unsigned int CrcGolden;
    xCrcCallback cb;
    void *pvData;
} *CrcRegionHandle_t;
```

26.4 使用说明和注意事项

26.4.1 Normal 显示

关键代码调用流程可以参考 [26.2.2](#) 章节，具体的使用示例可以参考 FreeRtos 升级显示 demo（drivers/upg_display/upg_display.c）。

26.4.2 冻屏检测 / 完整性校验

具体使用方法参考 [27 Safety Display](#)。

26.5 用户定制化

26.5.1 冻屏以及完整性校验定制化

FreeRTOS Display 提供相关接口，用户可以注册冻屏检测以及完整性校验出现异常时候的 Callback，由用户自己决定如果处理冻屏/校验失败事件。具体使用方法请参考 [27 Safety Display](#)。

26.5.2 QT/QD 显示适配说明

26.5.2.1 API 调用说明

关键代码调用流程可以参考 [26.2.2](#) 章节，具体的使用示例可以参考 FreeRtos 升级显示 demo（drivers/upg_display/upg_display.c）。

26.5.2.2 Buffer 管理

26.5.2.2.1 Buffer 数目

考虑到帧率效果，建议至少要有两块 buffer 参与绘制显示，调用如下接口：

```
xDisplaySurfaceSetBufferCount(xSurf, buffer_count);
```

26.5.2.2.2 Buffer 轮转

为了防止显示撕裂，FreeRTOS Display Framework 有 Buffer 管理策略。

QT/QD 在绘制一帧之前，需要调用 xDisplaySurfaceDequeueBuffer API 去获取 idle 状态的 buffer，如果当前 Display Framework 层没有 idle 状态的 buffer，API 调用会阻塞。

绘制一帧完成之后，需要调用 xDisplaySurfaceQueueBuffer API 将 buffer 送到 Display Framework，Framework 会在 vsync 周期来之后获取 queue 状态的 buffer，上屏显示。

27 Safety Display

27.1 简介

27.1.1 软硬件环境

27.1.1.1 软件环境

本文档描述的 AC8025 项目中的 Safety Display feature 基于 FreeRTOS 系统。

27.1.1.2 硬件环境

Safety Display 模块的安全图标的 CRC 检测依赖于 ATC-DPU 硬件，ATC-DPU 支持对指定区域进行 CRC 计算，通过与预先写入寄存器的 Gloden CRC 进行比较，不一致，则会产生 CRC 中断提示。冻结检测依赖于 LVDS 硬件，Hardware 会对指定的区域进行连续 N 帧检测，若连续 N 帧检测出不发生变化，则会产生冻结中断提示。

27.1.2 功能

- 支持 32 个安全图标显示并且支持图标完整性校验；
- 支持 16 个区域屏幕冻屏检测。

27.1.3 代码结构及文件描述

表 27-1 Safety Display 代码结构及描述

文件名	文件路径	文件描述
safety_display.c	safety/hsm/drivers/safety_display/safety_display.c	提供关键图标显示、图标完整性检查和冻屏检测功能
safety_display.h	safety/hsm/include/common/safety_display.h	Safety Display 数据结构
safety_display_driver.c	safety/hsm/drivers/display/framework/safety_display_driver.c	解析 safety icon 以及冻屏/校验的检测周期以及开关等配置信息

文件名	文件路径	文件描述
safety_display_demo.c	safety/hsm/apps/safety_display/safety_display_demo.c	提供关键图标显示、图标完整性检查和冻屏检测功能 demo

27.2 软件设计

27.2.1 架构图

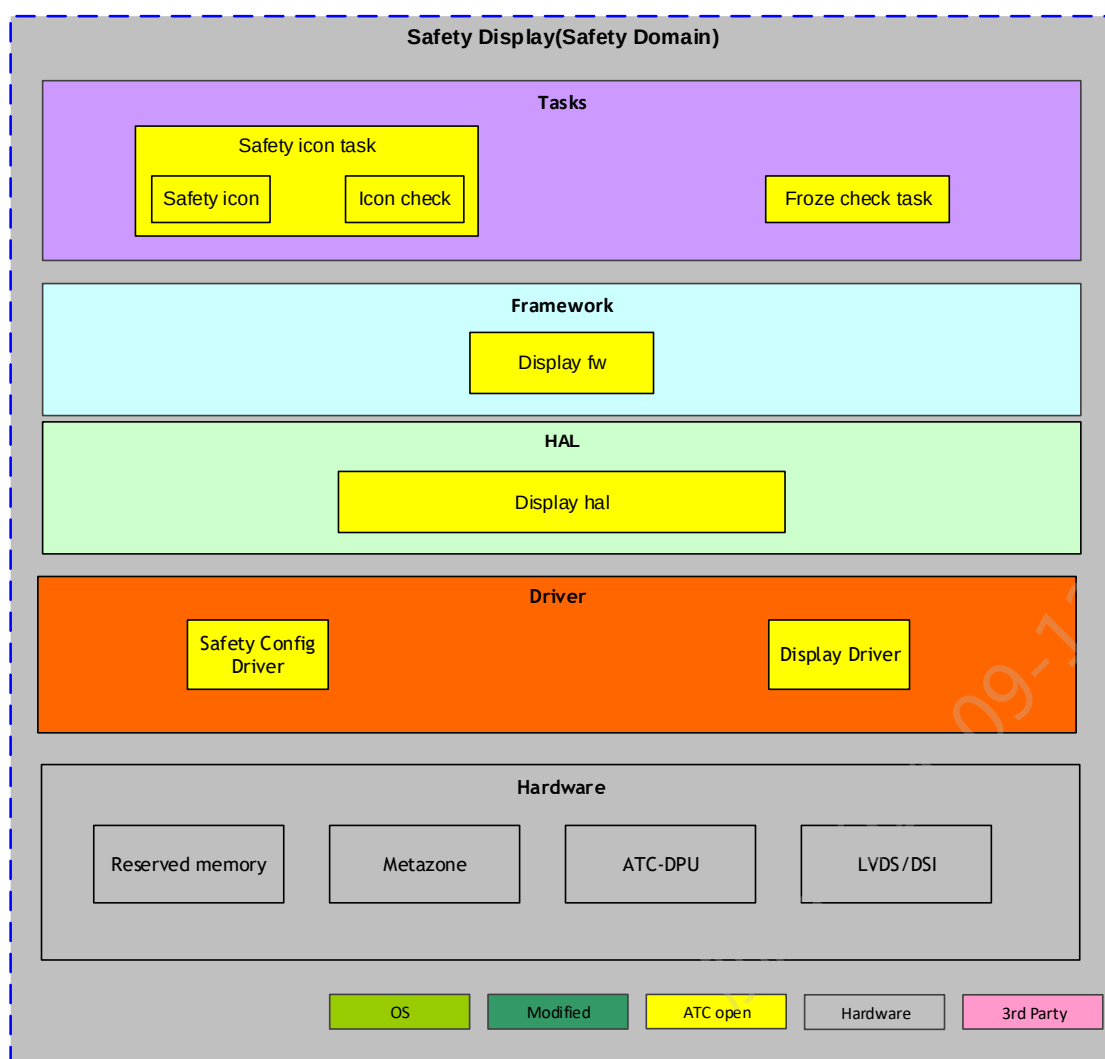


图 27-1 Safety Display 架构图

27.2.2 流程图

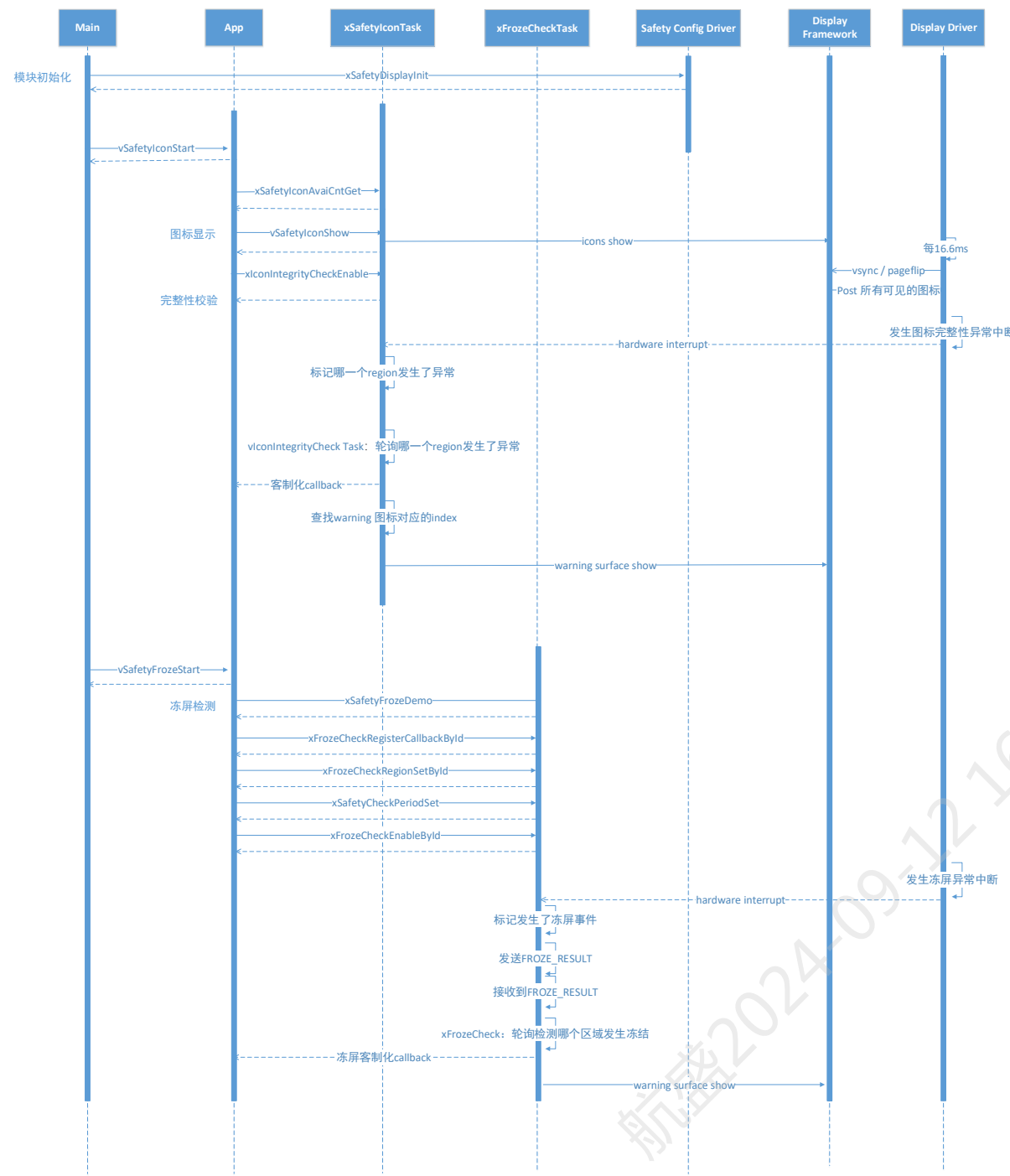


图 27-2 Safety Display 流程图

27.3 接口描述

27.3.1 API

表 27-2 Safety Display API

API	说明
xSafetyDisplayInit	safety display init
xSafetyDisplayDeInit	safety display deinit
xSafetyCheckPeriodSet	set froze check interval
xSafetyIconInfoGet	get safety icon information
xSafetyIconAvaiCntGet	get safety icon count
xSafetyIconCrcGoldenSet	get icon crc
xSafetyIconPositionSet	set safety icon (x, y) postion
xSafetyIconShowAll	show all icons
xSafetyIconHideAll	hide all icons
xSafetyIconShow	icon show
xSafetyIconHide	icon hide
xIconIntegrityCheckEnable	integrity check enable
xFrozeCheckInit	froze init
xFrozeCheckDeinit	froze deinit
xFrozeCheckEnable	froze check enable
xFrozeCheckEnableById	froze check enable by id
xFrozeCheckRegisterCallback	register froze check callback
xFrozeCheckRegisterCallbackById	register froze check callback by id
xFrozeCheckRegionSet	set region
xFrozeCheckRegionSetById	set region by id

27.3.1.1 xFrozeCallback

【描述】

For customer callback

【语法】

```
typedef int(* xFrozeCallback)(void *pvdata, unsigned int region_id, int result);
```

【参数】

- @param pvdata [in] : callback data
- @param index [in] : the region id
- @param result [out] : 0 not freeze, 1 freeze

【返回值】

- 0: success
- -1: failed

27.3.1.2 xCrcCallback

【描述】

For customer callback

【语法】

```
typedef int(* xCrcCallback)(void *pvdata, unsigned int icon_index);
```

【参数】

- @param pvdata [in] : callback data
- @param index [in] : the region id

【返回值】

- pdTRUE: success
- pdFALSE: failed

27.3.1.3 xSafetyDisplayInit

【描述】

Safety Display Init

【语法】

```
BaseType_t xSafetyDisplayInit(unsigned short sID)
```

【参数】

@param sID [in] : specified the display id that will be queried

【返回值】

- pdTRUE: success
- pdFALSE: failed

27.3.1.4 xSafetyDisplayDeInit

【描述】

Safety Display Deinit

【语法】

BaseType_t xSafetyDisplayDeInit(unsigned short sID);

【参数】

@param sID [in] : specified the display id that will be queried

【返回值】

- pdTRUE: success
- pdFALSE: failed

27.3.1.5 xSafetyCheckPeriodSet

【描述】

Set froze check interval to metazone(just available when reboot)

【语法】

xSafetyCheckPeriodSet (unsigned short sID, enum check_type type, unsigned long period)

【参数】

- @param sID [in] : specified the display id that will be queried
- @param type [in] : force or icon integrity check
- @param period [in] : check period

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.1.6 xSafetyIconInfoGet

【描述】

Get all Safety Icon info.

【语法】

BaseType_t xSafetyIconInfoGet (unsigned short sID, unsigned int uiIconIndex, struct SafetyIconInfo *pxSafetyIconInfo)

【参数】

- @param sID [in] : specified the display id that will be queried
- @param uiIconIndex [in] : specified the icon index that will be queried

- @param pxSafetyIconInfo [out] : Safetyicon information

【返回值】

- pdTRUE: successd
- pdFALSE: failed

27.3.1.7 xSafetyIconAvaiCntGet

【描述】

Get remain available Safety Icon count for show

【语法】

unsigned int xSafetyIconAvaiCntGet (unsigned short sID)

【参数】

@param sID [in] : specified the display id that will be queried

【返回值】

the available safety Icon count

27.3.1.8 xSafetyIconCrcGoldenSet

【描述】

Update safety icon crc golden

【语法】

BaseType_t xSafetyIconCrcGoldenSet (unsigned short sID, unsigned int uiIconIndex, unsigned int uiGolden)

【参数】

- @param sID [in] : specified the display id that will be queried
- @param uiIconIndex [in] : icon index
- @param uiGolden [in] : icon crc golden

【返回值】

the available safety Icon count

27.3.1.9 xSafetyIconPositionSet

【描述】

Set Safety Icon (x, y) position

【语法】

BaseType_t xSafetyIconPositionSet (unsigned short sID, unsigned int uiIconIndex, unsigned short x, unsigned short y)

【参数】

- @param sID [in] : specified the display id that will be queried
- @param uiIconIndex [in] : icon index
- @param x [in] : icon x position
- @param y [in] : icon y position

【返回值】

- pdTRUE: success
- pdFALSE: failed

27.3.1.10 xSafetyIconShowAll

【描述】

Show all icons.

【语法】

BaseType_t xSafetyIconShowAll(unsigned short sID)

【参数】

- @param sID [in] : specified the display id that will be queried

【返回值】

- pdTRUE: success
- pdFALSE: failed

27.3.1.11 xSafetyIconHideAll

【描述】

Hide all icons.

【语法】

BaseType_t xSafetyIconHideAll(unsigned short sID)

【参数】

- @param sID [in] : specified the display id that will be queried

【返回值】

- pdTRUE: success
- pdFALSE: failed

27.3.1.12 xSafetyIconShow

【描述】

Safety Icon show

【语法】

BaseType_t xSafetyIconShow(unsigned short sID, unsigned int uiIconIndex)

【参数】

- @param sID [in] : specified the display id that will be queried
- @param uiIconIndex [in] : icon index

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.1.13 xSafetyIconHide

【描述】

Safety Icon hide

【语法】

BaseType_t xSafetyIconHide(unsigned short sID, unsigned int uiIconIndex)

【参数】

- @param sID [in] : specified the display id that will be queried
- @param uiIconIndex [in] : icon index

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.1.14 xIconIntegrityCheckEnable

【描述】

Enable Integrity Check function

【语法】

BaseType_t xIconIntegrityCheckEnable(unsigned short sID, xCrcCallback cb, void *pvData, unsigned int uiIconIndex, unsigned short usEnable)

【参数】

- @param sID [in] : specified the display id that will be queried
- @param cb [in] : customized callback
- @param pvdata[in] : specified the parameter for callback
- @param uiIconIndex [in] : icon index in metazone
- @param usEnable [in] : enable crc region check or not

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.1.15 xFrozeCheckInit

【描述】

Init Safety Froze Detect Function

【语法】

BaseType_t xFrozeCheckInit (unsigned short sID)

【参数】

@param sID [in] specified the display id that will be queried

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.1.16 xFrozeCheckDeInit

【描述】

DeInit Safety Froze Detect Function

【语法】

BaseType_t xFrozeCheckDeInit (unsigned short sID)

【参数】

@param sID [in] : specified the display id that will be queried

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.1.17 xFrozeCheckEnable

【描述】

Froze Check enable.

【语法】

BaseType_t xFrozeCheckEnable(unsigned short sID, unsigned short usEnable)

【参数】

- @param sID [in] : specified the display id that will be queried
- @param enable [in] : enable or disable feature

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.1.18 xFrozeCheckEnableById

【描述】

Frozen Function Enable By Region Id.

【语法】

BaseType_t xFrozeCheckEnableById(unsigned short sID, unsigned short usEnable, unsigned short sRegionId);

【参数】

- @param sID [in] : specified the display id that will be queried
- @param usEnable [in] : 0: disbale, 1:enable
- @param sRegionId [in] : region id

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.1.19 xFrozeCheckRegisterCallback

【描述】

Register customized callback.

【语法】

BaseType_t xFrozeCheckRegisterCallback (unsigned short sID, xFrozeCallback cb, void *pvdata)

【参数】

- @param sID [in] : specified the display id that will be queried
- @param cb [in] : customized callback
- @param pvdata [in] : specified the parameter for callback customized private data

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.1.20 xFrozeCheckRegisterCallbackById

【描述】

Register customized callback by id.

【语法】

BaseType_t xFrozeCheckRegisterCallbackById(unsigned short sID, xFrozeCallback cb, void *pvData, unsigned short sRegionId);

【参数】

- @param sID [in] : specified the display id that will be queried
- @param cb [in] : customized callback
- @param pvdata [in] : specified the parameter for callback customized private data
- @param sRegionId [in] : region id

【返回值】

- pdTRUE: success,

- pdFALSE: failed

27.3.1.21 xFrozeCheckRegionSet

【描述】

Set Frozen Region.

【语法】

BaseType_t xFrozeCheckRegionSet(unsigned short sID, unsigned short usX, unsigned short usY, unsigned short usWidth, unsigned short usHeight);

【参数】

- @param sID [in] : specified the display id that will be queried
- @param usX [in] : x offset.
- @param usY [in] : y offset.
- @param usWidth [in] : width
- @param usHeight [in] : height.

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.1.22 xFrozeCheckRegionSetById

【描述】

Set Frozen Region By Id.

【语法】

BaseType_t xFrozeCheckRegionSetById(unsigned short sID, unsigned short usX, unsigned short usY, unsigned short usWidth, unsigned short usHeight, unsigned short sRegionId);

【参数】

- @param sID [in] : specified the display id that will be queried
- @param usX [in] : x offset.
- @param usY [in] : y offset.
- @param usWidth [in] : width
- @param usHeight [in] : height.

- @param sRegionId [in] : region id

【返回值】

- pdTRUE: success,
- pdFALSE: failed

27.3.2 数据结构

表 27-3 Safety Display 数据结构

数据结构	说明
icon_type	Icon type, such as: safety icon, crc warning icon, froze warning icon.
check_type	Force or icon integrity check type.
SafetyIconInfo	Safety icon information.

27.3.2.1 icon_type

```
enum icon_type
{
    SAFETY_ICON_UNKNOWN = 0,
    SAFETY_ICON,
    SAFETY_CRC,
    SAFETY_FROZE,
    SAFETY_ICON_MAX,
};
```

27.3.2.2 check_type

```
enum check_type
{
    SAFETY_CHECK_UNKNOWN = 0,
    SAFETY_CHECK_ICON,
    SAFETY_CHECK_FROZE,
    SAFETY_CHECK_MAX,
};
```

27.3.2.3 SafetyIconInfo

```
typedef struct SafetyIconInfo
{
    unsigned short usX;
    unsigned short usY;
    unsigned short usIconWidth;
    unsigned short usIconHeight;
    unsigned char ucIconIndex;
    unsigned long long ulStartAddr;
    unsigned int uiCrcGolden;
    unsigned short usVisable;
    CrcRegionHandle_t pxCrcRegion;
} *SafetyIconInfo_t;
```

27.4 使用说明和注意事项

关键图标显示和校验功能，本身较为简单，客户应用时，可以 demo 作为参考，进行客制化，客制化流程可参考第 27.5 章节。

1、实际应用场景中，关键图标显示应该与实际车辆信号管理，demo 只做了固定显示，客户应用时需结合车辆信号对图标进行 show 和 hide，且应该只在图标 show 的状态进行校验完整性。

2、关于关键图标显示限制：

图标本身为矩形块（安全图标打包时所有像素 alpha 均会被强制设置为 0xFF），检测时检测 alpha 为 0xFF 的整个矩形区域，但此时需要考虑图标和仪表整体风格的契合度。比如如下 P 图标为黑色背景，建议和仪表整体背景保持统一风格，以免观感上的突兀。



图 4-3 safety display(icon)示意图

上述方案中，可确保矩形区完整性异常被检测到，不会出现漏检测，但需要考虑背景和仪表整体风格的契合度，不同仪表风格的客制化工作量相对较大。

27.5 用户定制化

27.5.1 安全图标完整性校验客制化

- ATC 提供 Tool 工具用于导入用户定制的安全图标，导入的过程中会计算每一个图标的 Golden CRC 用于 ATCDPU 后期完整性检验，安全图标仅支持 PNG 和 BMP 格式。Tool 工具请参考《AC8025 SWIP Tool MPTool 用户使用指南》。

- ATC 提供 Metazone 开关安全图标完整性检测：

Type	Name	Size	Limits
Dword	METAZONE_SAFETY_ICON_CRC_ON_OFF	4	0:OFF 1:ON

提供如下 adb 命令调试（重启生效）：

```
adb shell "echo dword_write METAZONE_SAFETY_ICON_CRC_ON_OFF 1 > /proc/mtz_debug"
```

- ATC 提供 Metazone 设置安全图标完整性检测周期：

Type	Name	Size	Limits
Dword	METAZONE_SAFETY_ICON_CRC_INTERVAL	4	>=500(ms)

提供如下 adb 命令调试（重启生效）：

```
adb shell "echo dword_write METAZONE_SAFETY_ICON_CRC_INTERVAL 1000 > /proc/mtz_debug"
```

- ATC 提供 Metazone 设置安全图标显示开关：

Type	Name	Size	Limits
Dword	METAZONE_SAFETY_ICON_SHOW_ON_OFF	4	0:OFF 1:ON

提供如下 adb 命令调试（重启生效）：

```
adb shell "echo dword_write METAZONE_SAFETY_ICON_SHOW_ON_OFF 1 > /proc/mtz_debug"
```

- 安全图标的 demo code 参考 Safety/hsm/apps/safety_display/safety_display_demo.c


```

47:
48: struct TestData {
49:     uint16_t display_id;
50:     uint16_t index;
51: };
52:
53: int xIconCheckCallback(void *pvdata, unsigned int icon_index)
54: {
55:     pr_info("Icon%d crc err, please check !\n", icon_index);
56:
57:     return 0;
58: }
59:
60: void xSafetyIconsDemo(void)
61: {
62:     int i, icon_cnt = 0;
63:     unsigned short display_id = CLUSTER_Display_0;
64:     struct TestData *pxData = (struct TestData *)pvPortMalloc(sizeof(struct TestData));
65:     if (!pxData) {
66:         pr_err("pvdata malloc failed !\n");
67:         return;
68:     }
69:
70:     pxData->display_id = display_id;
71:     icon_cnt = xSafetyIconAvaiCntGet(display_id);
72:
73:     for (i = 0; i < icon_cnt; i++) {
74:         if (!xSafetyIconShow(display_id, i)) {
75:             pr_err("display %d show icon%d failed! \n", display_id, i);
76:             continue;
77:         }
78:
79:         pxData->index = i;
80:         if (!xIconIntegrityCheckEnable(display_id, xIconCheckCallback, (void *)pxData, i, 1)) {
81:             pr_err("display %d enable icon%d check failed! \n", display_id, i);
82:             continue;
83:         }
84:     }
85:
86:     pr_info("xSafetyIconsDemo success !");
87: } < end xSafetyIconsDemo >
88:

```

→ 客制化pvData数据结构，自定义

→ 客制化pvData

→ 显示安全图标

→ 指定图标使能图标完整性检测开关

27.5.2 冻屏检测客制化

- ATC 提供 Meatzone 开关冻屏检测：

Type	Name	Size	Limits
Dword	METAZONE_SAFETY_FROZE_ON_OFF	4	0:OFF 1:ON

提供如下 adb 命令调试（重启生效）：

adb shell “echo dword_write METAZONE_SAFETY_FROZE_ON_OFF 1 > /proc/mtz_debug”

- ATC 提供 Meatzone 设置冻屏检测周期：

Type	Name	Size	Limits
Dword	METAZONE_SAFETY_FROZE_INTERVAL	4	>=500(ms)

提供如下 adb 命令调试（重启生效）：

adb shell “echo dword_write METAZONE_SAFETY_FROZE_INTERVAL 1000 > /proc/mtz_debug”

- 冻结检测 demo code 参考 Safety/hsm/apps/safety_display/safety_display_demo.c。

```

90: int xFrozeCheckCallback(void *pvdata, unsigned int region_id, int result)
91: {
92:     struct TestData *pxData = (struct TestData *)pvdata;
93:
94:     if (result > 0) {
95:         pr_info("The current cluster screen is frozen, please check! display=%d id=%d\n", pxData->display_id, region_id);
96:         /*for MCU customization*/
97:     } else {
98:         pr_info("region%d is ok\n", region_id);
99:     }
100:     return 0;
101: }
102:
103:
104:
105:

```

冻结功能如果需要定制化与 MCU 关联，可以在冻结的 callback 中增加相关操作，如上图 demo 中“for MCU customization”位置。

```

103: void xSafetyFrozeDemo(void)
104: {
105:     int width = 1024;
106:     int height = 600;
107:     unsigned short usX = 0;
108:     unsigned short usY = 0;
109:     unsigned short region_id = 0;
110:     unsigned short display_id = CLUSTER_Display_0;
111:
112:     struct TestData *pxData = (struct TestData *)vPortMalloc(sizeof(struct TestData));
113:     if (!pxData) {
114:         pr_err("pvdata malloc failed!\n");
115:         return;
116:     }
117:     pxData->display_id = display_id;
118:     pxData->index = region_id;
119:
120:     if (!xDisplayGetResolution(display_id, &width, &height)) {
121:         pr_err("get screen resolution failed!\n");
122:         vPortFree(pxData);
123:         return;
124:     }
125:
126:     if (!xFrozeCheckRegisterCallbackById(display_id, xFrozeCheckCallback, (void *)pxData, region_id)) {
127:         pr_err("froze check register failed! region=%d\n", region_id);
128:         vPortFree(pxData);
129:         return;
130:     }
131:
132:     if (!xFrozeCheckRegionSetById(display_id, usX, usY, width, height, region_id)) {
133:         pr_err("froze check set region failed!\n", region_id);
134:         vPortFree(pxData);
135:         return;
136:     }
137:
138:     if (!xSafetyCheckPeriodSet(display_id, SAFETY_CHECK_FROZE, 500)) {
139:         vPortFree(pxData);
140:         return;
141:     }
142:
143:     if (!xFrozeCheckEnableById(display_id, 1, region_id)) {
144:         pr_err("froze check enable failed! region=%d\n", region_id);
145:         vPortFree(pxData);
146:         return;
147:     }
148:
149:     pr_info("xSafetyFrozeDemo success!");
150: }
151:
152:
153:

```

指定区域冻结范围，注意：不可以超过屏幕分辨率范围

指定DisplayID

定制化pvData

注册定制化callback

设置检测区域

设置检测周期，单位毫秒

使能冻结开关

27.6 典型问题

1. Safety_Display 机制依赖于 AC8025 R5F 下 Display 框架的初始化，若客户想通过 SOC 外部模块控制 Safety_Display 的图标显示，冻结检测功能，需要保证 R5F 下 Display 框架初始化完成后才可以操作。
2. Safety Display 功能会有一些宏的配置，如下：
 - (1) ATC_SAFETY_ICON_SUPPORT
安全图标显示功能开关。
 - (2) ATC_SAFETY_FROZE_CHECK_SUPPORT
冻结检测功能开关。

(3) ATC_SAFETY_ICON_INTEGRITY_CHECK_SUPPORT

图标完整性校验功能开关。

以上三个宏开关的使能在如下目录文件下设置：

Cockpit project:

build/projects/cockpit_s_linux_ac8025/ac8025_demo/with_adas/project_config.json

build/projects/cockpit_s_linux_ac8025/ac8025_demo/without_adas/project_config.json

Ivi project:

build/projects/ivi_s_ac8025/ac8025_demo/project_config.json

```

"Advanced_Options" : {
    "CONFIG_ATC_OS" : "android",
    "ATC_AOSP_ENHANCEMENT" : "yes",
    "ATC_MACRO_CTS" : "no",
    "ATC_MACRO_VTS" : "no",
    "ATC_DTBO_FEATURE" : "no",
    "ATC_NPU_SUPPORT" : "yes",
    "ATC_NPU_LITE_SUPPORT" : "yes",
    "ATC_NPU_ANDROID_INS" : "yes",
    "annotate": "metazone support",
    "ATC_METAZONE_SUPPORT" : "yes",
    "annotate": "adb change support",
    "ATC_ADB_CHANGE_SUPPORT" : "no",
    "ATC_CARPLAY_SUPPORT" : "no",
    "ATC_AVM_PARTITION_SUPPORT" : "no",
    "ATC_AVM_SUPPORT" : "yes",
    "ATC_AVM_SURFACE_FLINGER_BUFFER_ENABLE" : "yes",
    "ATC_AVM_FAST_DISPLAY_2_ENABLE" : "no",
    "ATC_AVM_FAST_DISPLAY_3_ENABLE" : "yes",
    "ATC_AVM_FAST_RECORD_ENABLE" : "no",
    "ATC_AVM_REMOTE_INVOKE_ENABLE" : "no",
    "ATC_AVM_DATAZONE_SUPPORT" : "no",
    "ATC_AVM_VIDEO_EMU_SUPPORT" : "yes",
    "ATC_LOGO_SUPPORT" : "yes",
    "annotate": "fastdisplay 2.0",
    "ATC_FASTDISP_VERSION" : "2.0",
    "DIS_FASTBOOT_IN_USER" : "no",
    "BOOTING_ENABLE" : "yes",
    "ATC_DMS_SUPPORT" : "no",
    "CONFIG MMC_CMDQ" : "yes",
    "ATC_CLUSTER_SUPPORT" : "no",
    "ATC_COCKPIT_MODE" : "no",
    "CONFIG_ATC_DYNAMIC_TP" : "no",
    "SECURE_STORAGE_RPM_SUPPORT" : "yes",
    "CONFIG_AVM_PARTITION_ENABLE" : "yes",
    "ATC_SAFETY_DISPLAY_SUPPORT" : "yes",
    "ATC_SAFETY_ICON_SUPPORT" : "no",
    "ATC_SAFETY_ICON_INTEGRITY_CHECK_SUPPORT" : "no",
    "ATC_SAFETY_FROZE_CHECK_SUPPORT" : "yes",
    "ATC_SAFETY_INTEGRITY_CHECK_ENHANCED" : "yes",
    "CONFIG_AAH_ENABLE" : "n",
    "ATC_PARTITION_PLUGIN_SUPPORT" : "yes",
    "GIC_BASE_ADDR" : "0x12800000",
    "TEE_UART_BASE_ADDR" : "0x11220000",
    "ATC_VEHICLE_GEARBOX_STATE_R_FROM_GPIO_ENABLE" : "yes",
    "ATC_VEHICLE_AVM_STATE_FROM_GPIO_ENABLE" : "yes",

```

目前 8025 Cockpit 和 Ivi 项目宏配置情况如下：

Cockpit: 默认开启以上三个宏；

Ivi: 默认仅开启了 ATC_SAFETY_FROZE_CHECK_SUPPORT

3. Safety Display 安全图标和 Warning 图标（冻结提示/图标完整性异常提示）目前都打包存放在 datazone-ro.img，针对 Safety 资源的安全性，模块做了如下的操作：

- (1) 系统初始化解析 Binary 资源时，若 CRC 计算不匹配，Safety OS 会弹紫色透明全屏弹窗告警，提示安全图标资源已损坏，请检查；
- (2) 若开启了 SafetyDisplay 全功能，但 datazone 无定制的图标资源，Safety OS 会弹紫色透明全屏弹窗告警，提示安全图标资源已损坏，请检查。

航盛2024-09-12 16:50:39

28 Atcsurface

28.1 简介

28.1.1 功能

AtcSurface 模块主要提供私有图层显示接口，其主要功能如下：

- 控制图层的显示和隐藏
- 设置 source 的属性，包括大小、格式、crop、Z-序、优先级、亮度、色域、压缩信息等
- 设置输出窗口的位置、大小、自适应、transform 等
- 支持同一个应用多屏显示
- 支持不同进程下调用同一 atcsurface 进行相关控制功能
- 支持多应用同时使用（按照优先级进行资源抢占）

另外 atcsurface 可以支持快速显示，因此需要快速显示的应用可以调用 atcsurface 模块进行显示（如 rvc、avm 等）。

28.1.2 代码结构及文件描述

表 28-1 Atcsurface 代码结构及描述

文件名	文件路径	文件描述
surface.c	source/vendor/autochips/proprietary/hardware/display/libs/libatcsurface/src	提供创建 surface 的功能
v4l2overlay.c	source/vendor/autochips/proprietary/hardware/display/libs/libatcsurface/src	提供对 surface 的控制功能
atcsurface_ext.h	source/vendor/autochips/proprietary/hardware/display/libs/libatcsurface/include	Atcsurface 对外提供的接口和数据结构
surface_test.c	source/vendor/autochips/proprietary/samplecode/surface_test	Atcsurface 使 Demo

28.2 软件设计

28.2.1 架构图

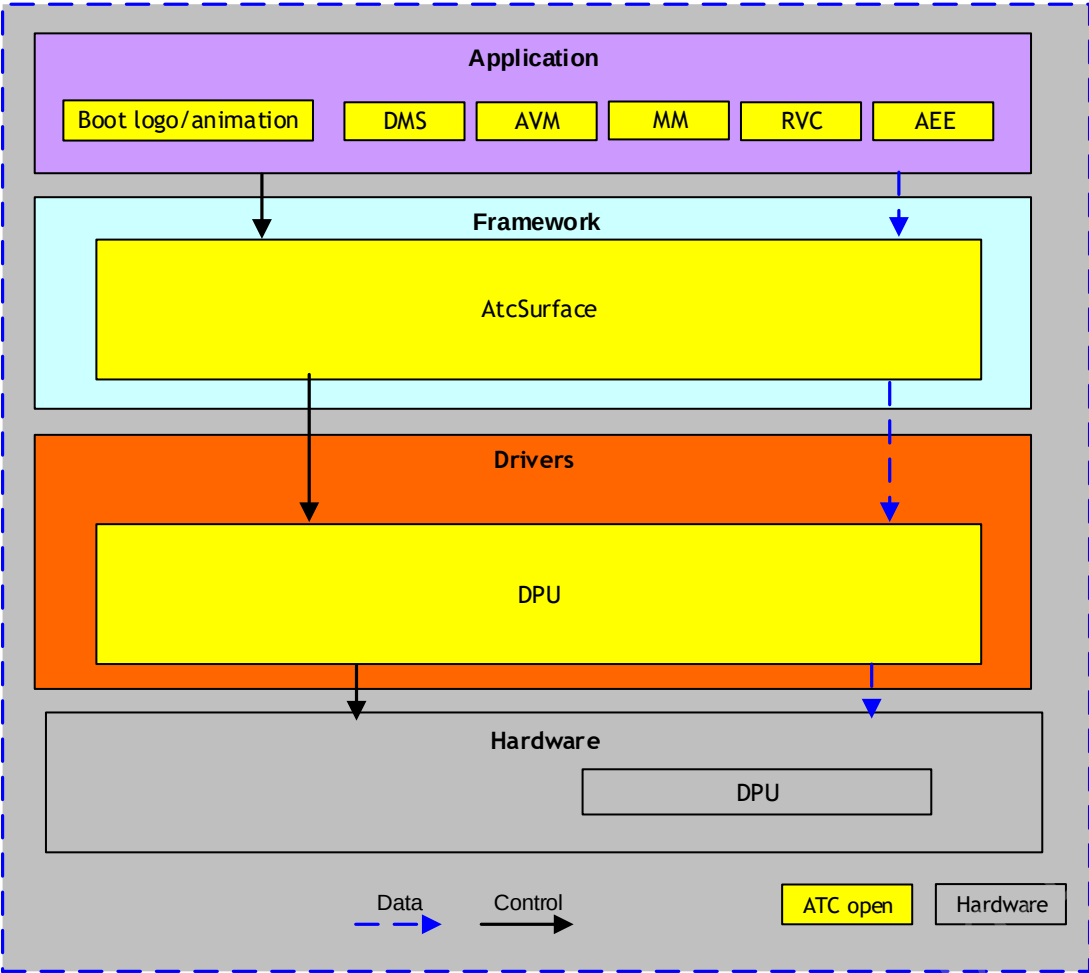


图 28-1 Atcsurface 架构图

28.2.2 流程图

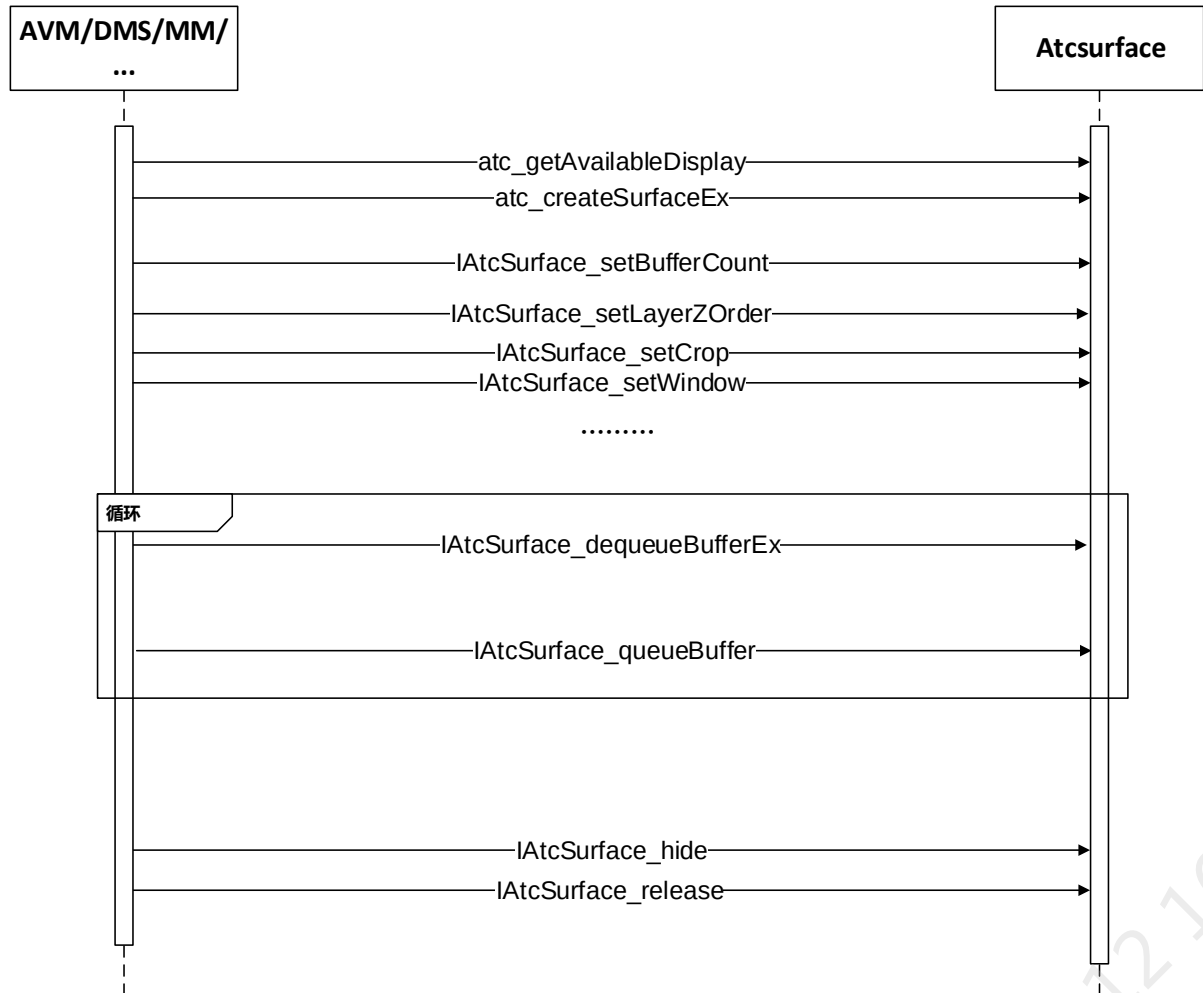


图 28-2 Atcsurface 流程图

1. 获取当前 domain 支持的 display id;
2. 根据需求创建 Surface 对象，可以根据应用创建多个 atcsurface，每个 atcsurface 可以显示在多个 displayId 上;
3. 配置 buffer count、优先级、source crop window、旋转角度、显示 window 等属性;
4. Dequeue buffer;
5. 应用针对 buffer 的真实 fd 地址，填入数据;
6. Queue buffer 至 driver 显示，循环此过程，直到显示的数据流结束;
7. 结束显示，释放 buffer 及 Surface 对象。

28.3 接口描述

28.3.1 API

表 28-2 Atcsurface API

API	说明
atc_getAvailableDisplay	Get available display in this domain
atc_createsurfaceEx	Create a surface
atc_createsurface_from_nativedata	Create a surface from a nativedata
atc_getScreenResolutionEx	Get display resolution
IAtcSurface_addRef	Add reference for a surface
IAtcSurface_release	Release a surface
IAtcSurface_dequeueBufferEx	Dequeue buffer
IAtcSurface_cancelBuffer	Cancel a buffer
IAtcSurface_queueBuffer	Queue a buffer
IAtcSurface_resetBuffers	Reset all buffers for a surface
IAtcSurface_setBuffersSize	Set buffer size
IAtcSurface_setBuffersFormat	Set buffer format
IAtcSurface_setBufferCount	Set buffer count number
IAtcSurface_showEx	Show a surface
IAtcSurface_hide	Hide a surface
IAtcSurface_setCropEx	Set crop rect
IAtcSurface_setPosition	Set destination x and y position
IAtcSurface_setWindowEx	Set destination window rect
IAtcSurface_setCapability	Set capalibity
IAtcSurface_setColorEncoding	Set color encoding
IAtcSurface_setLayerZOrder	Set layer z-order
IAtcSurface_blank	Set blank frame for surface
IAtcSurface_getNativeData	Get a native data from a surface
IAtcSurface_setTransformEx	Set transform
IAtcSurface_setWindowAdaptionEx	Set window adaption
IAtcSurface_setColorRange	Set color range
updateDisplayIds	Update display ids

28.3.1.1 atc_getAvailableDisplay

【描述】

从当前域中获取可用的 display 集合

【语法】


```
int32_t atc_getAvailableDisplay(u_int32_t *display_mask, u_int8_t *display_count)
```

【参数】

- @param display_mask [IN OUT]: 可用的 display mask, 值的意义参考 [display_id_mask](#) 枚举值
- @param display_count [IN OUT]: 可用的 display count

【返回值】

- 0: success
- other: failed

28.3.1.2 atc_createsurfaceEx

【描述】

根据 config 信息创建 surface 对象

【语法】

```
IAtcSurface atc_createsurfaceEx (struct SurfaceConfig config);
```

【参数】

- @param config [IN]: surface 配置例如 width, height, format 等信息

【返回值】

- IAtcSurface 对象: success
- null: failed

28.3.1.3 atc_createsurface_from_nativedata

【描述】

利用已有的 surface 对象的 nativedata 创建 surface

【语法】

```
IAtcSurface atc_createsurface_from_nativedata (int *fds, int n_fds, u_int8_t *native_data,  
u_int32_t data_size);
```

【参数】

- @param fds [IN]: 打开的 video 设计 fd 集合
- @param n_fds [IN]: fd 集合中的第几个 fd
- @param native_data[IN]:用于指向多进程共享的数据地址

- @param data_size [IN]:nativedata 的 size 大小

【返回值】

- IAtcSurface instance: success
- null: failed

28.3.1.4 atc_getScreenResolutionEx

【描述】

从当前域中获取可用的 display 集合

【语法】

```
int32_t atc_getScreenResolutionEx (enum DisplayId displayId, int *w, int *h)
```

【参数】

- @param displayId [IN]: 指定的 display id, 具体值参考下一节数据结构中 DisplayId 枚举值
- @param w [IN OUT]: display 的 width
- @param h [IN OUT]: display 的 height

【返回值】

- 0: success
- other: failed

28.3.1.5 IAtcSurface_addRef

【描述】

Surface 对象增加引用计数

【语法】

```
int32_t IAtcSurface_addRef (IAtcSurface *surf)
```

【参数】

@param surf [IN]: surface 对象

【返回值】

- Surface 对象的引用计数: success
- 负数: failed

28.3.1.6 IAtcSurface_release

【描述】

Release surface 对象

【语法】

```
void IAtcSurface_release (IAtcSurface *surf)
```

【参数】

@param surf [IN]: surface 对象

【返回值】

None

28.3.1.7 IAtcSurface_dequeueBufferEx

【描述】

从 surface 对象中 dequeue 一块空闲的 buffer

【语法】

```
int32_t IAtcSurface_dequeueBufferEx (IAtcSurface *surf, atc_buffer_t *buf, int nonblocking);
```

【参数】

- @param surf [IN]: surface 对象
- @param buf [OUT]: dequeue 到的 buffer 结构体
- @param nonblocking [IN]: dequeue 操作是否为 non-blocking 模式

【返回值】

- 0: success
- 负数: failed

28.3.1.8 IAtcSurface_cancelBuffer

【描述】

Cancel 一块 buffer 到 surface 中

【语法】

```
int32_t IAtcSurface_cancelBuffer (IAtcSurface *surf, atc_buffer_t *buf);
```

【参数】

- @param surf [IN]: surface 对象
- @param buf [IN]: queue 的 buffer

【返回值】

- 0: success
- 负数: failed

28.3.1.9 IAtcSurface_queueBuffer

【描述】

Queue 一块 buffer 到 surface 中

【语法】

```
int32_t IAtcSurface_queueBuffer (IAtcSurface *surf, atc_buffer_t *buf);
```

【参数】

- @param surf [IN]: surface 对象
- @param buf [IN]: queue 的 buffer

【返回值】

- 0: success
- 负数: failed

28.3.1.10 IAtcSurface_resetBuffer

【描述】

Reset surface 对象 bufferqueue 中所有的 buffer

【语法】

```
int32_t IAtcSurface_resetBuffer (IAtcSurface *surf);
```

【参数】

@param surf [IN]: surface 对象

【返回值】

- 0: success
- 负数: failed

28.3.1.11 IAtcSurface_setBufferSize

【描述】

设置 Surface 对象 buffer 的大小

【语法】

```
int32_t IAtcSurface_setBuffersSize (IAtcSurface *surf, int32_t w, int32_t h);
```

【参数】

- @param surf [IN]: surface 对象
- @param w [IN]: 设置 Surface 对象的指定 buffer 宽度
- @param h [IN]: 设置 Surface 对象的指定 buffer 高度

【返回值】

- 0: success
- 负数: failed

28.3.1.12 IAtcSurface_setBufferFormat

【描述】

设置 Surface 对象 buffer 的 format

【语法】

```
int32_t IAtcSurface_setBuffersFormat (IAtcSurface *surf, int32_t format, u_int32_t fourcc);
```

【参数】

- @param surf [IN]: surface 对象
- @param format [IN]: 设置指定 buffer 的格式
- @param fourcc [IN]: 默认设置为 0

【返回值】

- 0: success
- 负数: failed

28.3.1.13 IAtcSurface_setBufferCount

【描述】

设置 Surface 对象 buffer 的数量

【语法】

```
int32_t IAtcSurface_ setBufferCount (IAtcSurface *surf, u_int32_t count);
```

【参数】

- @param surf [IN]: surface 对象
- @param count [IN]: buffer 的数量

【返回值】

- 0: success
- 负数: failed

28.3.1.14 IAtcSurface_showEx

【描述】

将 surface 对象进行显示

【语法】

```
int32_t IAtcSurface_ showEx (IAtcSurface *surf, int32_t option);
```

【参数】

- @param surf [IN]: surface 对象
- @param option [IN]: 1 代表需要显示 last 帧, 0 代表不需要显示 last 帧

【返回值】

- 0: success
- 负数: failed

28.3.1.15 IAtcSurface_hide

【描述】

将 surface 对象进行 hide

【语法】

```
int32_t IAtcSurface_ hide (IAtcSurface *surf);
```

【参数】

@param surf [IN]: surface 对象

【返回值】

- 0: success
- 负数: failed

28.3.1.16 IAtcSurface_setCropEx

【描述】

设置 surface 对象的裁剪参数

【语法】

```
int32_t IAtcSurface_setCropEx (IAtcSurface *surf, enum DisplayId displayID, u_int32_t x,
u_int32_t y, u_int32_t w, uint32_t h);
```

【参数】

- @param surf [IN]: surface 对象
- @param displayID [IN]: 指定的 display
- @param x [IN]: 裁剪区域的左上角 x 坐标
- @param y [IN]: 裁剪区域的左上角 y 坐标
- @param w [IN]: 裁剪区域的宽度
- @param h [IN]: 裁剪区域的高度

【返回值】

- 0: success
- 负数: failed

28.3.1.17 IAtcSurface_setPosition

【描述】

设置 surface 对象显示的目标窗口位置

【语法】

```
int32_t IAtcSurface_setPosition (IAtcSurface *surf, int32_t x, int32_t y);
```

【参数】

- @param surf [IN]: surface 对象
- @param x [IN]: 指定目标区域的左上角 x 坐标
- @param y [IN]: 指定目标区域的左上角 y 坐标

【返回值】

- 0: success
- 负数: failed

28.3.1.18 IAtcSurface_setWindowEx

【描述】

设置 surface 对象显示的目标窗口

【语法】

```
int32_t IAtcSurface_setWindowEx (IAtcSurface *surf, enum DisplayId displayID, int32_t x,
int32_t y, int32_t w, int32_t h);
```

【参数】

- @param surf [IN]: surface 对象
- @param displayID [IN]: 指定的 display
- @param x [IN]: 指定目标区域的左上角 x 坐标
- @param y [IN]: 指定目标区域的左上角 y 坐标
- @param w [IN]: 指定目标区域的宽度
- @param h [IN]: 指定目标区域的高度

【返回值】

- 0: success
- 负数: failed

28.3.1.19 IAtcSurface_setCapability

【描述】

设置 surface 对象的 capability 能力

【语法】

```
int32_t IAtcSurface_setCapability (IAtcSurface *surf, int32_t capability);
```

【参数】

- @param surf [IN]: surface 对象
- @param capability [IN]: surface 支持的能力

【返回值】

- 0: success
- 负数: failed

28.3.1.20 IAtcSurface_setColorEncoding**【描述】**

设置 surface 对象的 color encoding 信息（BT601/BT709 等）

【语法】

```
int32_t IAtcSurface_setColorEncoding (IAtcSurface *surf, u_int32_t encoding);
```

【参数】

- @param surf [IN]: surface 对象
- @param encoding [IN]: color encoding 信息, 其中 encoding 的值参考如下枚举

```
enum atc_drm_color_encoding {  
    ATC_DRM_COLOR_YCBCR_BT601,  
    ATC_DRM_COLOR_YCBCR_BT709,  
    ATC_DRM_COLOR_YCBCR_BT2020,  
    ATC_DRM_COLOR_ENCODING_MAX,  
};
```

【返回值】

- 0: success
- 负数: failed

28.3.1.21 IAtcSurface_setLayerZOrder**【描述】**

设置 surface 对象的 z 序

【语法】

```
int32_t IAtcSurface_setLayerZOrder (IAtcSurface *surf, u_int32_t order);
```

【参数】

- @param surf [IN]: surface 对象

- @param order [IN]: 设置的 z-order 值，具体的值建议参考下一节的 display_zorder 枚举值

【返回值】

- 0: success
- 负数: failed

28.3.1.22 IAtcSurface_blank

【描述】

设置黑帧效果

【语法】

```
int32_t IAtcSurface_blank (IAtcSurface *surf);
```

【参数】

@param surf [IN]: surface 对象

【返回值】

- 0: success
- 负数: failed

28.3.1.23 IAtcSurface_getNativeData

【描述】

获取 surface 对象的 natedata 信息

【语法】

```
u_int8_t * IAtcSurface_getNativeData(IAtcSurface *surf, int *fds, int *n_fds, u_int32_t *data_size);
```

【参数】

- @param surf [IN]: surface 对象
- @param fds [OUT]: 指向当前已经打开 video 设备的 fd 集合的指针
- @param n_fds [IN OUT]: 指定检索到的 fd 资源的编号，初始值指定检索到的最大 fd 编号
- @param data_size [OUT]: 指向检索到的 natedata 大小的指针

【返回值】

- 指向 natedata 缓冲区的指针: success

- null: failed

28.3.1.24 IAtcSurface_setTransformEx

【描述】

设置 surface 对象的 gain 值

【语法】

```
int32_t IAtcSurface_set setTransformEx (IAtcSurface *surf, enum DisplayId displayID,
u_int32_t angle, u_int32_t flip);
```

【参数】

- @param surf [IN]: surface 对象
- @param displayID [IN]: 指定的 display
- @param angle [IN]: 设置的旋转角度值
- @param flip [IN]: 设置的 flip 值

【返回值】

- 0: success
- 负数: failed

28.3.1.25 IAtcSurface_setWindowAdaptionEx

【描述】

设置 surface 对象的 gain 值

【语法】

```
int32_t IAtcSurface_setWindowAdaptionEx (IAtcSurface *surf, enum DisplayId displayID,
u_int32_t adaption);
```

【参数】

- @param surf [IN]: surface 对象
- @param displayID [IN]: 指定的 display
- @param adaption [IN]: 指定 Surface 对象的目标窗口是否打开全屏自适应，0 表示关闭，1 表示打开

【返回值】

- 0: success

- 负数: failed

28.3.1.26 IAtcSurface_setColorRange

【描述】

设置 surface 对象的 color range

【语法】

```
int32_t IAtcSurface_setColorRange (IAtcSurface *surf, u_int32_t range);
```

【参数】

- @param surf [IN]: surface 对象
- @param range [IN]: 设置的 range 值，其中 range 的值参考如下枚举

```
enum atc_drm_color_range {  
    ATC_DRM_COLOR_YCBCR_LIMITED_RANGE,  
    ATC_DRM_COLOR_YCBCR_FULL_RANGE,  
    ATC_DRM_COLOR_RANGE_MAX,  
};
```

【返回值】

- 0: success
- 负数: failed

28.3.1.27 IAtcSurface_updateDisplayIds

【描述】

更新 surface 对象需要显示的 display

【语法】

```
int32_t IAtcSurface_updateDisplayIds (IAtcSurface *surf, u_int32_t displayIDs);
```

【参数】

- @param surf [IN]: surface 对象
- @param displayIDs [IN]: 更新的 display mask 值，具体 mask 值参考下一小节的 display_id_mask 枚举，即用户需要更新显示的 display id 集合。

【返回值】

- 0: success

- 负数: failed

28.3.2 数据结构

表 28-3 Atcsurface 数据结构

数据结构	说明
display_id_mask	Display id mask
DisplayId	Display id
SurfaceConfig	Config for create a surface
Atc_buffer_t	Dequeuer buffer struct from atcsurface

28.3.2.1 display_id_mask

```
enum display_id_mask {
    IVI_Display_0_mask = 1 << IVI_Display_0,
    IVI_Display_1_mask = 1 << IVI_Display_1,
    IVI_Display_2_mask = 1 << IVI_Display_2,
    IVI_Display_3_mask = 1 << IVI_Display_3,
    IVI_Display_4_mask = 1 << IVI_Display_4,
    CLUSTER_Display_0_mask = 1 << IVI_Display_0,
    CLUSTER_Display_1_mask = 1 << IVI_Display_1,
    CLUSTER_Display_2_mask = 1 << IVI_Display_2,
    CLUSTER_Display_3_mask = 1 << IVI_Display_3,
    CLUSTER_Display_4_mask = 1 << IVI_Display_4,
    ADAS_Display_0_mask = 1 << ADAS_Display_0,
    ADAS_Display_1_mask = 1 << ADAS_Display_1,
    ADAS_Display_2_mask = 1 << ADAS_Display_2,
    ADAS_Display_3_mask = 1 << ADAS_Display_3,
    ADAS_Display_4_mask = 1 << ADAS_Display_4,
    TOUCH_Display_0_mask = ADAS_Display_0_mask,
    ALL_Display_mask = 0xffffffff,
};
```

28.3.2.2 DisplayId

```
enum DisplayId {  
    IVI_Display_0,  
    IVI_Display_1,  
    IVI_Display_2,  
    IVI_Display_3,  
    IVI_Display_4,  
  
    CLUSTER_Display_0,  
    CLUSTER_Display_1,  
    CLUSTER_Display_2,  
    CLUSTER_Display_3,  
    CLUSTER_Display_4,  
  
    ADAS_Display_0,  
    ADAS_Display_1,  
    ADAS_Display_2,  
    ADAS_Display_3,  
    ADAS_Display_4,  
  
    DISPLAY_MAX,  
    TOUCH_Display = ADAS_Display_0,  
};
```

28.3.2.3 SurfaceConfig

```
struct SurfaceConfig {  
    char *name;  
    int32_t width;  
    int32_t height;  
    int32_t format;  
    uint32_t DisplayIds;
```

```
bool needAlloc;
#define CROSS_OS 1
uint32_t flags;
}
```

该 SurfaceConfig 结构体是用户在创建 atcsurface 时需要传入的参数，其中需要注意的是：

- DisplayIds: 是传入需要创建的 atcsurface 所在的 display，可以是多个 display 的 mask。具体通过 atc_getAvailableDisplay 接口获取当前域可用的 display，再根据用户的应用场景设置具体的 display id。Display Id 的具体值参照上一节的定义。
- NeedAlloc: 该参数代表用户是否需要 atcsurface 模块分配真实的 buffer fd，值 true 的情况 dequeue buffer 时会带出真实的 fd，值 false 的情况代表用户自己分配 buffer，dequeue buffer 时只是带出对应的 buffer 结构体。
- Flags: 该参数配置成 CROSS_OS，代表是否需要跨 OS，该宏可以让用户进行配置。

28.3.2.4 atc_buffer_t

```
typedef struct _atc_buffer {
    int32_t width;
    int32_t height;
    int32_t stride;
    int32_t v_stride;
    u_int32_t format;
    u_int32_t fmt_fourcc;
    void *bits;
    int32_t fd;
    int32_t tile_fd;
    u_int8_t tile_mode[3]; //每个 plane 的 tile mode 信息
    u_int8_t tile_size[3]; //每个 plane 的压缩单元信息
    bool fbc_avail; //fbc 是否可用
    int32_t fbc_enable; //1:USE fbc 0: not use fbc
    u_int64_t timestamp_in; /*time start*/
    u_int64_t timestamp_out; /*time cfg to hw*/
    int32_t reserved[6];
}
```

```
} atc_buffer_t;
```

该结构体是用户调用 `dequeueBuffer` 时获取的 `buffer` 结构体，同时在 `queueBuffer` 时需要传入该结构体，其中需要注意的是：

- `bits`: 是 `atcsurface` 和 `vout` 共享内存的地址，为了同步 `buffer index` 和信息传递。用户不需要设置该地址。
- `fd`:
 - 在用户需要分配真实 `buffer` 的情况（即上一小节 `SurfaceConfig` 中 `NeedAlloc` 为 `true` 的情况），该 `fd` 是 `atcsurface` 调用 `disp_device_mem` 分配出来的有效的 `fd` 数值；
 - 反之，`NeedAlloc` 为 `false` 的情况，在 `dequeue buffer < buffer count` 的时候 `fd` 值为 0，这种情况需要用户在 `queueBuffer` 时传入真实的 `fd`；当 `dequeue buffer` 大于 `buffer count` 的时候，`fd` 是之前用户传入的 `fd`。
- `tile_fd`: 代表的是 `tile status buffer` 的 `fd`，对于多个 `plane` 的情况，需要根据分辨率和 `format` 等信息计算 `offset`。
- `tile_mode`: 分别代表每个 `plane` 的 `tile mode`
- `tile_size`: 分别代表每个 `plane` 的 `tile size` (128 / 256 / 512)
- `fbc_avail`: 这个值代表 `atcsurface` 决策后返回给用户的 `fbc` 是否可以使用的信息，值分别为 `true` 和 `false`
- `fbc_enable`: 是用户在 `queueBuffer` 时填入的是否需要 `enable fbc` 的信息，值分别为 1 和 0，对应 `enable` 和 `disable`

28.3.2.5 display_zorder

```
enum disp_zorder {
    Z_TYPE_DEFAULT,
    Z_TYPE_KMS_BASE,
    Z_TYPE_LOGO_BASE = NEXT_BASE(Z_TYPE_KMS_BASE),
    Z_TYPE_VBA_BASE = NEXT_BASE(Z_TYPE_LOGO_BASE),
    Z_TYPE_FAST_DISP_BASE = NEXT_BASE(Z_TYPE_VBA_BASE),
    Z_TYPE_RVC_BASE = NEXT_BASE(Z_TYPE_FAST_DISP_BASE),
    Z_TYPE_AVM_BASE = NEXT_BASE(Z_TYPE_RVC_BASE),
    Z_TYPE_AEE_BASE = NEXT_BASE(Z_TYPE_AVM_BASE),
    Z_TYPE_SAFETY_BASE = NEXT_BASE(Z_TYPE_AVM_BASE),
    Z_TYPE_MAX,
```



```
};
```

28.4 使用说明和注意事项

Atcsurface 的使用，参考提供的 sample code，路径：
source/vendor/autochips/proprietary/samplecode/surface_test



注意：

用户不能跨域调用 atcsurface。

28.5 典型问题

28.5.1 用户使用不当，导致内存泄露

【问题描述】

Avm 使用 atcsurface 进行显示，不停进行开关操作即不停地创建和销毁 atcsurface，导致内存泄露。

【解决方案】

用户销毁 atcsurface 之前，需要对已经 map 的 buffer 进行 unmap。

28.5.2 用户如何设置 atcsurface 的显示层级/优先级

【问题描述】

Atcsurface 支持多用户多进程调用，调用者需要根据应用的实际情况，设置合理的 z-order 即优先级，确保能得到合理的显示。

【解决方案】

调用 setLayerZOrder 接口按照 28.3.2.5 display_zorder 枚举值进行设置。

29 LCM

29.1 简介

29.1.1 软硬件环境

29.1.1.1 软件环境

依赖的内部模块：display 子系统的 DPU/LVDS/DSI/DP driver。

29.1.1.2 硬件环境

外围器件：液晶显示模组（ATC demo/evb 平台公版所用 LVDS 接口 LCM，型号为 WD090SHL40AD-C0）。

Display subsystem 包含 DPU/LVDS/DSI/DP/Panel 等模块，其中和 LCM 强相关的是 LVDS/DSI/DP/Panel 模块。

29.1.2 功能

AC8025 平台为方便用户快速调试新的 LCM，LVDS/DSI/DP/Panel driver 可以通过读取由 MRF 工具软件生成的包含屏幕信息的配置文件，来完成相关参数配置。其实现的主要思想就是把 LCM 常用的屏幕调试参数，统一在 MRF 工具软件中配置，然后由该工具软件生成包含屏幕信息的配置文件 (datazone_ro.img)，最后把配置文件通过烧录工具烧写到 eMMC，在系统起来的时候系统会从 eMMC 把相关内容加载到指定物理地址的内存，然后由 DPIM Driver 完成对相关数据的读取、解析处理、然后初始化为对应的数据结构，最后 display driver 根据对应的数据结构完成对应硬件的初始化。

29.1.3 代码结构及文件描述

表 29-1 LCM 代码结构及描述

文件名	文件路径	文件描述
Lvds driver	safety/hsm/drivers/display/driver/ac8025/lvds/	LVDS driver，主要提供 LVDS HW 初始化接口和初始化 LVDS
Mipi dsi driver	safety/hsm/drivers/display/driver/ac8025/mipi/	DSI driver，主要提供 DSI HW 初始化接口和初始化 dsi
Dp driver	safety/hsm/drivers/display/driver/ac8025/dp/	DP driver，主要提供 DP HW 初始化接口和初始化 DP

文件名	文件路径	文件描述
Panel driver	safety/hsm/drivers/display/driver/ac8025/ panel	Panel driver, 主要用于从 dpim 中获取初始化序列去初始化 mipi dsi 屏或桥接芯片

表 29-2 Kernel 部分代码结构及描述

文件名	文件路径	文件描述
Lvds driver	source/kernel/atc_modules/drm_autochips/platforms/ac8025/lvds	LVDS driver, 向 drm 核心层注册 encoder 和 connector, 初始化 LVDS (通过判断 init flag, hsm 中初始化后, kernel 中不会再重复初始化)
Mipi dsi driver	source/kernel/atc_modules/drm_autochips/platforms/ac8025/mipi	DSI driver, 向 drm 核心层注册 encoder 和 onnector, DSI HW 初始化(同 LVDS)
Dp driver	source/kernel/atc_modules/drm_autochips/platforms/ac8025/dp	DP driver, 向 drm 核心层注册 encoder 和 onnector, DP HW 初始化(同 LVDS)
Panel driver	source/kernel/atc_modules/drm_autochips/panel	Panel driver, 主要用于从 dpim 中获取初始化序列去初始化 mipi dsi 屏或桥接芯片

29.2 软件设计

29.2.1 架构图

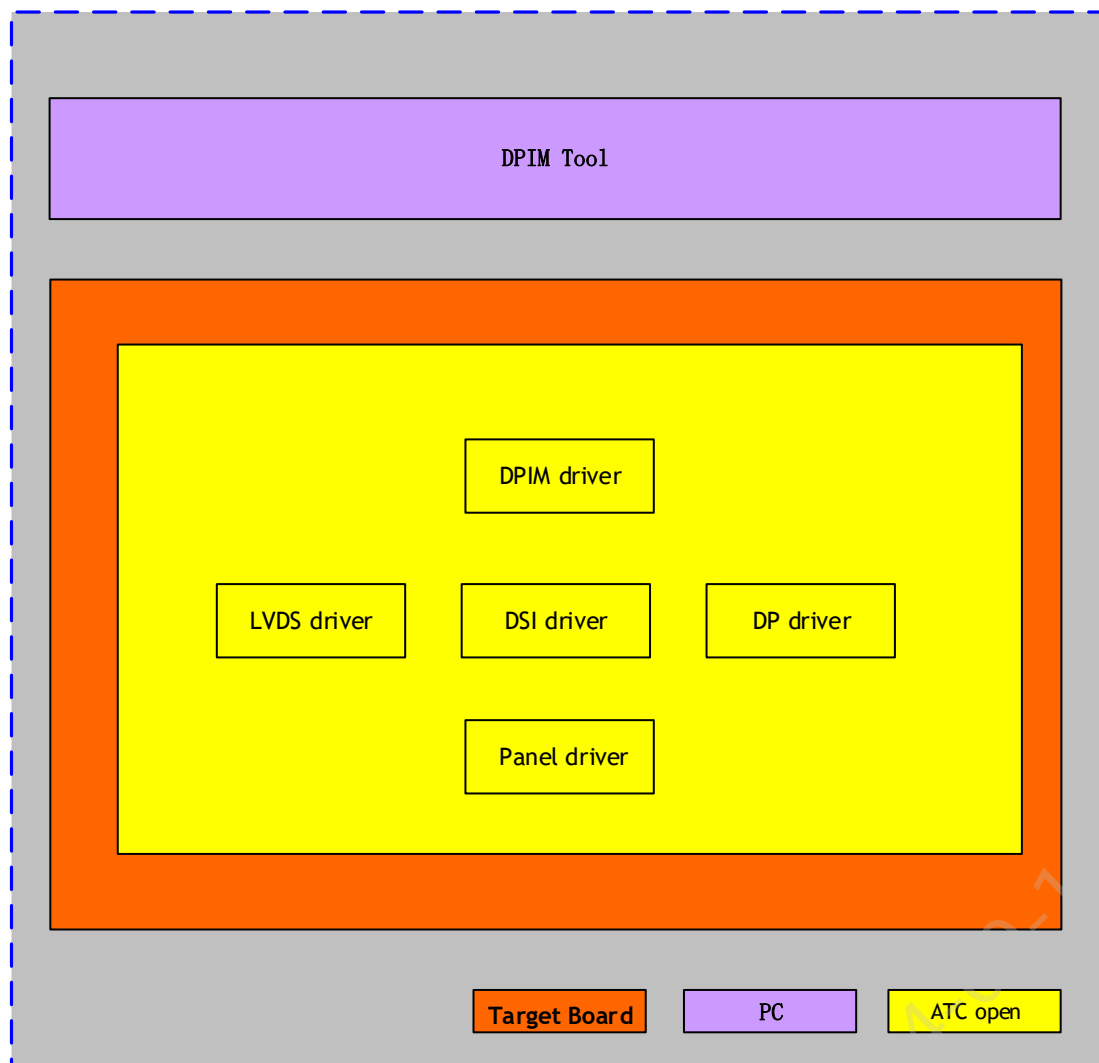


图 29-1 LCM 架构图

29.2.2 流程图

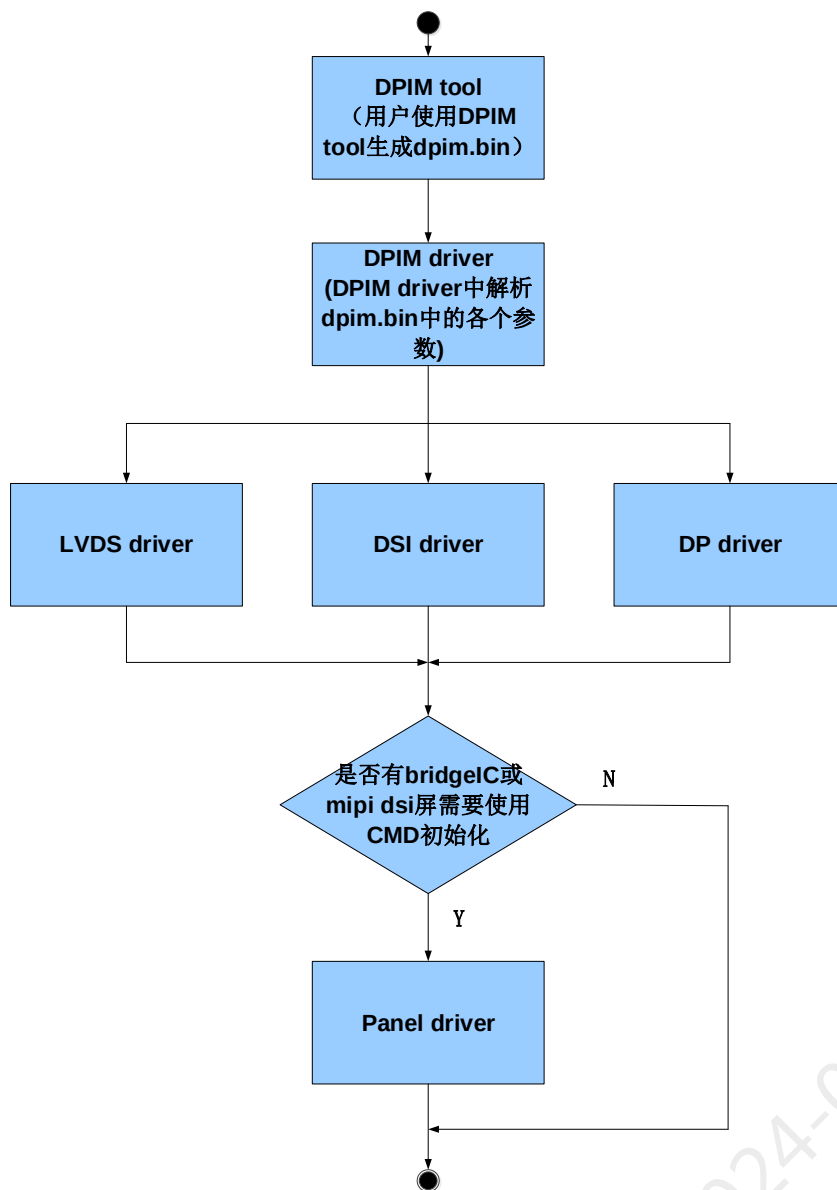


图 29-2 LCM 流程图

29.3 使用说明和注意事项

MRF tool 会单独 release，请在 atc release 的 tool kit 中获取。

MRF tool 使用说明请参看文档：《ATC-SWIP_Tool_MPTool_User_Guide.pdf》

将 MRF tool 生成的 dpim.bin 打包成 datazone 说明文档《ATC-SWIP_Tool_User_Guide_Datazone.docx》

本章节不再重复描述。

29.4 用户定制化

29.4.1 新屏幕调试步骤

这里以公版使用的 LVDS 接口屏幕为例，说明如何从屏幕 spec 获取屏参，DSI/DP 获取屏参方式类似。



注意：

HFP、HSW、VFP、VSW、VBP 要大于 0，不能填 0，HBP 要大于等于 4。

29.4.1.1 获取屏幕相关设置参数

29.4.1.1.1 屏幕分辨率、显示区域物理尺寸设置

Item		Specification	Unit
Outline Dimension		211.1(H) x 126.5(V) x 5.5(Typ.)	mm
Display area		196.608(H) x 114.15(V) —— 屏幕物理尺寸	mm
Number of Pixel		1024 RGB (H) x 600(V) —— 屏幕分辨率	pixels
Pixel pitch		0.192(H) x 0.19025(V)	mm
Pixel arrangement		RGB Vertical stripe	
Display mode		Normally Black	
Surface treatment		Antiglare, Hard-Coating (3H)	
Weight		240 (Typ.)	g
Back-light		Single LED (Side-Light type)	
Power Consumption	Logic System	0.3(Max)	W
	B/L System	2.769 (Max.)	W

图 29-3 屏幕分辨率、显示区域物理尺寸信息

29.4.1.1.2 Timing、Clk、同步模式设置

Parameter	Symbol	Value			Unit
		Min.	Typ.	Max.	
DCLK frequency @Frame rate=60hz	fclk	40.8	51.2	67.2	Mhz
Horizontal display area	thd		1024		DCLK
HSYNC period time	th	1114	1344	1400	DCLK
HSYNC blanking	thb+thfp	90	320	376	DCLK
Vertical display area	tvd		600		H
VSYNC period time	tv	610	635	800	H
VSYNC blanking	tvb+tvfp	10	35	206	H

图 29-4 屏幕 Timing、Clk、同步模式信息 (Interface Timing-DE mod)

表 29-3 屏参填写说明

名称	参数	说明
HPW	100	如果 LCM spec 上面没有单独的 HPW、HFP、HBP 只有 HSYNC blanking，只需要保持 HPW+HFP+HBP = HSYNC blanking 即可
HFP	110	
HFP	110	
VPW	11	如果 LCM spec 上面没有单独的 VPW、VFP、VBP 只有 VSYNC blanking，只需要保持 VPW+VFP+VBP = VSYNC blanking 即可
VFP	12	
VBP	12	
Clk	51200	LVDS clock 计算公式 $H_{total} = h_{blank} + h_{active}$ $V_{total} = v_{blank} + v_{active}$ $clk = h_{total} * v_{total} * fps$ (single link) $clk = h_{total} * v_{total} * fps / 2$ (dual link) DSI clock 计算公式 $clk = h_{total} * v_{total} * fps * 3$

29.4.1.1.3 LCM 接口类型、数据格式设置

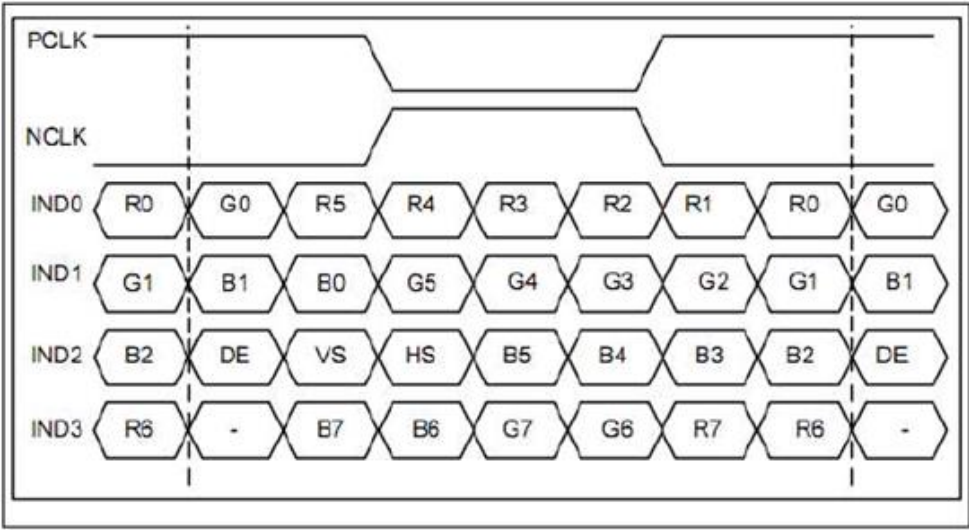


图 29-5 屏幕接口类型、数据格式信息（8Bit LVDS input）

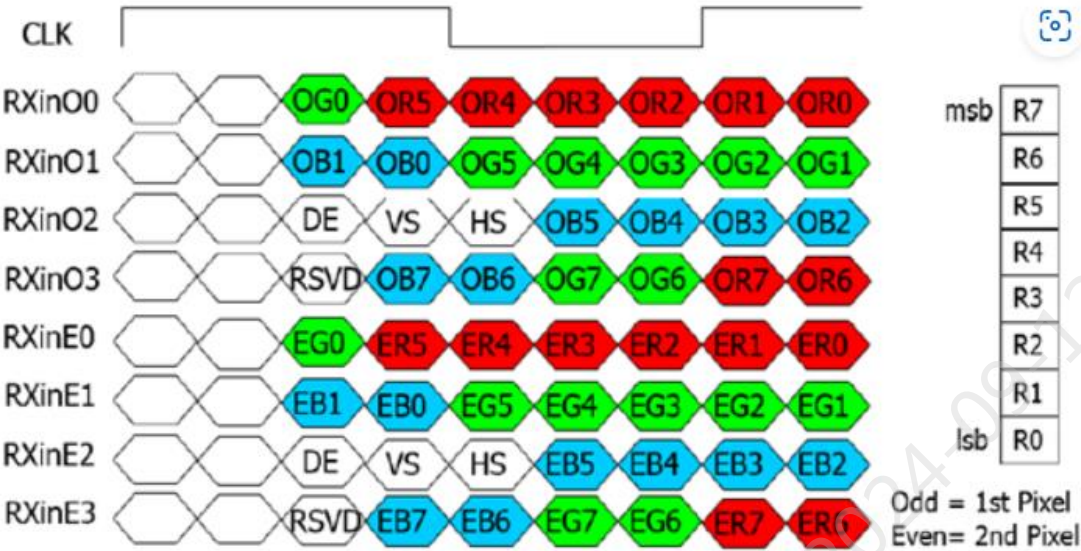


图 29-6 VESA format（24.1）



图 29-7 JEIDA format（24.0，也叫做 DISM）

表 29-4 数据格式填表说明

名称	参数	说明
Mode	DE	目前 LVDS 屏一般都是 DE mode；MIPI DSI 界面下的 mode 为 dsi mode 选择，可选择的有 sync_pulse_vdo/sync_event_vdo /cmd/burst_vdo，这个可在屏或桥接芯片的 spec 中获取。
Link Mode	SINGLE	图 29-5 输入数据只有 4 条 LVDS data 数据，所以可以判定是 single-link，dual-link 一般是 8 条 data，如图 29-6 和图 29-7 所示；
RGB Format	RGB888	图 29-5 4 条 data lane 输入数据 R0~R7、G0~G7、B0~B7 所以可知是 RGB888
Data Format	24.1	图 29-5 4 条 data lane 输入数据 R0~R7、G0~G7、B0~B7 符合标准 24.1（也叫 VESA）数据格式的排列

29.4.1.1.4 BridgeIC/DSI cmd 参数填写说明

针对有 bridge IC 或需要使用 dsi cmd 初始化的 DSI 屏，我们可以在 BridgeIC 文本框中填写相关初始化序列。

LVDS 序列填写示例：

```
[Lvds_Init_S]
BS, 0x1
AS, 0x2C
0x09, 0x1, 0x00
MM, 0x5
0x0A, 0x1, 0x05
UU, 0x10
0x0D, 0x1, 0x01
0xFC, 0x1, 0x0A
0x09, 0x1, 0x01
[Lvds_Init_E]
```

MIPI DSI 序列填写示例：

```
[Mipi_Init_S]
BS, 0x1
AS, 0x2C
0x09, 0x1, 0x00
MM, 0x5
0x0A, 0x1, 0x05
UU, 0x10
0x0D, 0x1, 0x01
0xFC, 0x1, 0x0A
0x09, 0x1, 0x01
[Mipi_Init_E]
```

DP 序列填写示例：

```
[Dp_Init_S]
```

```
BS, 0x1
AS, 0x2C
0x09, 0x1, 0x00
MM, 0x5
0x0A, 0x1, 0x05
UU, 0x10
0x0D, 0x1, 0x01
0xFC, 0x1, 0x0A
0x09, 0x1, 0x01
[Dp_Init_E]
```



说明:

- 第一列除 BS/AS/MM/UU 外表示 reg，第二列除 BS/AS/MM/UU 外表示 data 个数，第三列表示 data（注意 data 个数和实际填写的 data 个数要一致）
- BS 表示 I2C bus 选择，BS 后面的参数表示 I2C bus number；AS 表示 salve addr 选择，AS 后面的参数表示 salve addr；MM 表示 delay ms，MM 后面的参数表示要 delay 多少 ms；UU 表示 delay us，UU 后面的参数表示要 delay 多少 us。

将上述获取到的屏参填入 MRF tool 中，生成 dpim.bin，再打包生成 datazone_ro.img 烧录进板子即可。

29.5 典型问题

29.5.1 生成 datazone 烧录后屏幕没有正常点亮

【问题描述】

客户在调试新屏期间，自己做好的 datazone_ro.img 烧录后屏幕没有正常点亮。

【解决方案】

1. 查看串口 log，搜索关键字“DPIM”，看 dpim driver 初始化过程中是否正确解析了 dpim.bin 中配置参数；主要查看屏参和预期是否一致即可。

```
\par [ 0.907552][ T1][I] DISP[DPIM]:[dpim_parse_dts 2338] find tx type:1. res count:2
\par [ 0.911325][ T1][I] DISP[DPIM]:[disp_path_info_print 1670] ==display path info start===
\par [ 0.913439][ T1][I] DISP[DPIM]:[disp_path_info_print 1671] path cnt:2
\par [ 0.915375][ T1][I] DISP[DPIM]:[disp_path_info_print 1672] path mask:0x3
\par [ 0.916250][ T1][I] DISP[DPIM]:[disp_path_info_print 1674] path[0].tg:0
\par [ 0.918605][ T1][I] DISP[DPIM]:[disp_path_info_print 1675] path[0].tx:0x0
\par [ 0.919484][ T1][I] DISP[DPIM]:[disp_path_info_print 1676] path[0].req_layers_mask:0x3
\par [ 0.922010][ T1][I] DISP[DPIM]:[disp_path_info_print 1677] path[0].disp_id:0
\par [ 0.924431][ T1][I] DISP[DPIM]:[disp_path_info_print 1678] path[0].need_layers:2
\par [ 0.925386][ T1][I] DISP[DPIM]:[disp_path_info_print 1679] path[0].smmu_support:1
\par [ 0.927859][ T1][I] DISP[DPIM]:[disp_path_info_print 1680] path[0].fusa_support:0
\par [ 0.928843][ T1][I] DISP[DPIM]:[disp_path_info_print 1681] path[0].pixel_h:600
\par [ 0.931338][ T1][I] DISP[DPIM]:[disp_path_info_print 1682] path[0].pixel_w:1024
\par [ 0.933800][ T1][I] DISP[DPIM]:[disp_path_info_print 1683] path[0].phy_h:99
\par [ 0.934701][ T1][I] DISP[DPIM]:[disp_path_info_print 1684] path[0].phy_w:176
\par [ 0.937236][ T1][I] DISP[DPIM]:[disp_path_info_print 1685] path[0].pwm_id:0
\par [ 0.939623][ T1][I] DISP[DPIM]:[disp_path_info_print 1686] path[0].min_duty:10
\par [ 0.940564][ T1][I] DISP[DPIM]:[disp_path_info_print 1687] path[0].max_duty:43
\par [ 0.942993][ T1][I] DISP[DPIM]:[disp_path_info_print 1688] path[0].addr:0x80
\par [ 0.943912][ T1][I] DISP[DPIM]:[disp_path_info_print 1674] path[1].tg:1
\par [ 0.946345][ T1][I] DISP[DPIM]:[disp_path_info_print 1675] path[1].tx:0x1
\par [ 0.948730][ T1][I] DISP[DPIM]:[disp_path_info_print 1676] path[1].req_layers_mask:0x3
\par [ 0.949750][ T1][I] DISP[DPIM]:[disp_path_info_print 1677] path[1].disp_id:1
\par [ 0.952180][ T1][I] DISP[DPIM]:[disp_path_info_print 1678] path[1].need_layers:2
\par [ 0.953135][ T1][I] DISP[DPIM]:[disp_path_info_print 1679] path[1].smmu_support:1
\par [ 0.955686][ T1][I] DISP[DPIM]:[disp_path_info_print 1680] path[1].fusa_support:0
\par [ 0.958173][ T1][I] DISP[DPIM]:[disp_path_info_print 1681] path[1].pixel_h:600
\par [ 0.960273][ T1][I] DISP[DPIM]:[disp_path_info_print 1682] path[1].pixel_w:1024
\par [ 0.962250][ T1][I] DISP[DPIM]:[disp_path_info_print 1683] path[1].phy_h:99
\par [ 0.964259][ T1][I] DISP[DPIM]:[disp_path_info_print 1684] path[1].phy_w:176
\par [ 0.964265][ T1][I] DISP[DPIM]:[disp_path_info_print 1685] path[1].pwm_id:1
\par [ 0.967419][ T1][I] DISP[DPIM]:[disp_path_info_print 1686] path[1].min_duty:10
\par [ 0.968386][ T1][I] DISP[DPIM]:[disp_path_info_print 1687] path[1].max_duty:43
\par [ 0.969319][ T1][I] DISP[DPIM]:[disp_path_info_print 1688] path[1].addr:0x80
RIG" (18 hits in 1 file of 1 searched)
```

图 29-8 LOG 信息

2. 系统起来后使用 adb 命令，可以查看更详细的屏参信息，具体命令如下：

```
adb root
adb shell
echo show > ./sys/kernel/debug/disp-dpim/dpim
```

命令输入后串口 log 截图如下：

```

\par [ 369.651750][ T122][ I] [AEE]UTC [2023-05-30 17:13:40] kernel is running...
\par [ 389.356040][ T465][ I] [drm] [ATC_DRM:FRAMEWORK][atc_crtc_enable_vblank:694]IN
\par [ 394.089818][ T2592][ I] DISP[DPIM]:[dpim_dbg_write 2452] argc = 2.
\par [ 394.090609][ T2592][ I] DISP[DPIM]:[dpim_dbg_write 2467] dpim show
\par [ 394.091376][ T2592][ I] DISP[DPIM]:[tx_info_print 1569] ==type:0 start==
\par [ 394.092305][ T2592][ I] DISP[DPIM]:[tx_info_print 1570] panel_num:1
\par [ 394.093078][ T2592][ I] DISP[DPIM]:[tx_info_print 1572] bkl_info.pwm_id:0
\par [ 394.093911][ T2592][ I] DISP[DPIM]:[tx_info_print 1573] bkl_info.min_duty:10
\par [ 394.094778][ T2592][ I] DISP[DPIM]:[tx_info_print 1574] bkl_info.max_duty:43
\par [ 394.095644][ T2592][ I] DISP[DPIM]:[tx_info_print 1575] bkl_info.addr:0x80
\par [ 394.096528][ T2592][ I] DISP[DPIM]:[tx_info_print 1578] bkg_info[0].bridge_en:0
\par [ 394.097401][ T2592][ I] DISP[DPIM]:[tx_info_print 1580] bkg_info[0].init_block_idx:0
\par [ 394.098382][ T2592][ I] DISP[DPIM]:[tx_info_print 1582] panel[0].clk:51200
\par [ 394.099227][ T2592][ I] DISP[DPIM]:[tx_info_print 1583] panel[0].disp_id:0
\par [ 394.100086][ T2592][ I] DISP[DPIM]:[tx_info_print 1584] panel[0].vc_id:0
\par [ 394.100914][ T2592][ I] DISP[DPIM]:[tx_info_print 1585] panel[0].fps:60
\par [ 394.101726][ T2592][ I] DISP[DPIM]:[tx_info_print 1586] panel[0].bit_fmt:1
\par [ 394.102571][ T2592][ I] DISP[DPIM]:[tx_info_print 1587] panel[0].h_active:1024
\par [ 394.103459][ T2592][ I] DISP[DPIM]:[tx_info_print 1588] panel[0].hbp:110
\par [ 394.104296][ T2592][ I] DISP[DPIM]:[tx_info_print 1589] panel[0].hfp:110
\par [ 394.105122][ T2592][ I] DISP[DPIM]:[tx_info_print 1590] panel[0].hsw:100
\par [ 394.105946][ T2592][ I] DISP[DPIM]:[tx_info_print 1591] panel[0].hsync_pol:0
\par [ 394.106813][ T2592][ I] DISP[DPIM]:[tx_info_print 1592] panel[0].v_active:600
\par [ 394.107703][ T2592][ I] DISP[DPIM]:[tx_info_print 1593] panel[0].vbp:12
\par [ 394.108519][ T2592][ I] DISP[DPIM]:[tx_info_print 1594] panel[0].vfp:12
\par [ 394.109331][ T2592][ I] DISP[DPIM]:[tx_info_print 1595] panel[0].vsw:11
\par [ 394.110144][ T2592][ I] DISP[DPIM]:[tx_info_print 1596] panel[0].vsync_pol:0
\par [ 394.111011][ T2592][ I] DISP[DPIM]:[tx_info_print 1597] panel[0].phy_hz:99
\par [ 394.111857][ T2592][ I] DISP[DPIM]:[tx_info_print 1598] panel[0].phy_w:176
\par [ 394.112705][ T2592][ I] DISP[DPIM]:[tx_info_print 1602] lvds.mode:0
\par [ 394.113474][ T2592][ I] DISP[DPIM]:[tx_info_print 1603] lvds.link_mode:1
\par [ 394.114298][ T2592][ I] DISP[DPIM]:[tx_info_print 1604] lvds.data_fmt:1
\par [ 394.115110][ T2592][ I] DISP[DPIM]:[tx_info_print 1605] lvds.vod:0
\par [ 394.115911][ T2592][ I] DISP[DPIM]:[tx_info_print 1608] lvds.ssc_enable:0
\par [ 394.116717][ T2592][ I] DISP[DPIM]:[tx_info_print 1609] lvds.ssc_depth:1
\par [ 394.117541][ T2592][ I] DISP[DPIM]:[tx_info_print 1610] lvds.ssc_freq:10
\par [ 394.118363][ T2592][ I] DISP[DPIM]:[tx_info_print 1611] lvds.ssc_dir:0
\par [ 394.119165][ T2592][ I] DISP[DPIM]:[tx_info_print 1613] lvds.lane_swap[0]:0
\par [ 394.120034][ T2592][ I] DISP[DPIM]:[tx_info_print 1615] lvds.lane_invert[0]:0
\par [ 394.120915][ T2592][ I] DISP[DPIM]:[tx_info_print 1613] lvds.lane_swap[1]:0
\par [ 394.121770][ T2592][ I] DISP[DPIM]:[tx_info_print 1615] lvds.lane_invert[1]:0
\par [ 394.122647][ T2592][ I] DISP[DPIM]:[tx_info_print 1613] lvds.lane_swap[2]:0
\par [ 394.123503][ T2592][ I] DISP[DPIM]:[tx_info_print 1615] lvds.lane_invert[2]:0
\par [ 394.124393][ T2592][ I] DISP[DPIM]:[tx_info_print 1613] lvds.lane_swap[3]:0
\par [ 394.125253][ T2592][ I] DISP[DPIM]:[tx_info_print 1615] lvds.lane_invert[3]:0
\par [ 394.126130][ T2592][ I] DISP[DPIM]:[tx_info_print 1613] lvds.lane_swap[4]:0
\par [ 394.126986][ T2592][ I] DISP[DPIM]:[tx_info_print 1615] lvds.lane_invert[4]:0
\par [ 394.127876][ T2592][ I] DISP[DPIM]:[tx_info_print 1613] lvds.lane_swap[5]:0
\par [ 394.128735][ T2592][ I] DISP[DPIM]:[tx_info_print 1615] lvds.lane_invert[5]:0
\par [ 394.129612][ T2592][ I] DISP[DPIM]:[tx_info_print 1613] lvds.lane_swap[6]:0
\par [ 394.130468][ T2592][ I] DISP[DPIM]:[tx_info_print 1615] lvds.lane_invert[6]:0
\par [ 394.131345][ T2592][ I] DISP[DPIM]:[tx_info_print 1613] lvds.lane_swap[7]:0
\par [ 394.132214][ T2592][ I] DISP[DPIM]:[tx_info_print 1615] lvds.lane_invert[7]:0
\par [ 394.133094][ T2592][ I] DISP[DPIM]:[tx_info_print 1613] lvds.lane_swap[8]:0
\par [ 394.133950][ T2592][ I] DISP[DPIM]:[tx_info_print 1615] lvds.lane_invert[8]:0
\par [ 394.134828][ T2592][ I] DISP[DPIM]:[tx_info_print 1613] lvds.lane_swap[9]:0
\par [ 394.135695][ T2592][ I] DISP[DPIM]:[tx_info_print 1615] lvds.lane_invert[9]:0

```

图 29-9 DPIM 详细 log

- 如果步骤 1/2 中 log 都正确，tx driver 初始化也正常的情况，可以用示波器测量 tx(LVDS/DSI/DP) 输出端是否有波形输出；
- 如果步骤 3 正确，说明 SOC 出来的数据是正常的，主要是屏幕接收问题，可以和硬件一起排查屏幕硬件和芯片连接、屏幕电源供电、背光是否正确。

29.5.2 出现巨齿纹



图 29-10 锯齿纹

如果出现如上图所示的巨齿纹路，则是客户的板子 datalane 的奇偶 lane 接反了，只需要软件 swap 一下奇偶 lane 即可。

29.5.3 黑屏

如果出现屏参都是按照屏幕 spec 配置，并且确认 code 中打印的屏参和预设一致，且无桥接芯片的情况下，可以 check 原理图，查看每条 lane 之间的 P/N 是否接反，屏和 soc 之间的 lane 线序是否一致。

29.5.4 镜像显示

如果出现显示的内容是镜像（显示内容反掉了），一般是扫描方向设置的不对，可以查看屏幕 spec，查看设置扫描方向的 pin 脚或 reg 是否正确。

航盛2024-09-12 16:50:40

30 Display Debug

30.1 简介

30.1.1 功能

本章主要针对 display 常见问题，提供对应的分析方法和 debug 手段，主要包括以下内容：

- UI debug
- Video debug

遇到问题涉及到 display module 主要有：

- Display framework
- HWC
- Atcsurface
- DPIM
- DPU
- LVDS/DSI/DP

30.2 软件设计

该章节主要介绍当前的 debug 方法，具体对应不同的问题场景（参见第 30.3 章场景分析，不局限于所列场景），可以初步分析和排查。

30.2.1 架构图

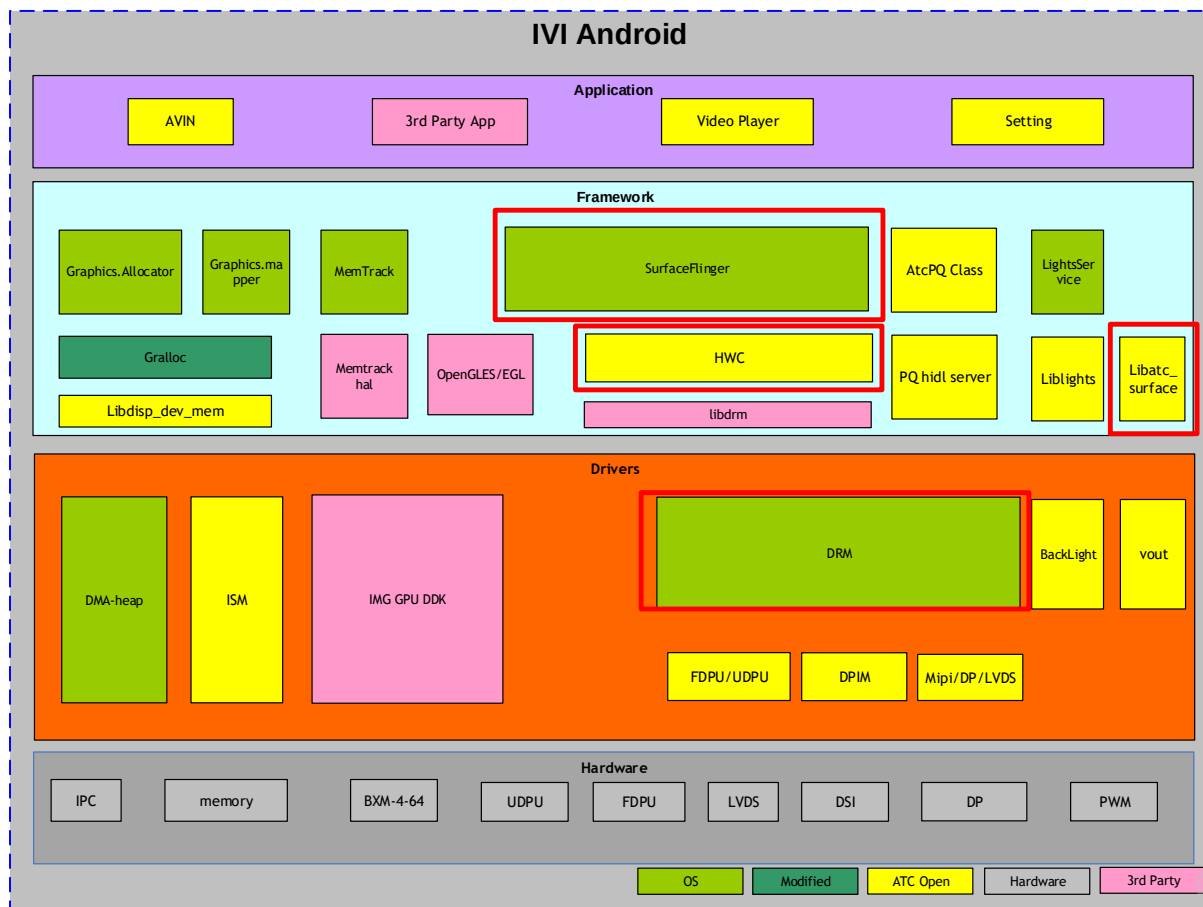


图 30-1 Android 显示架构图

本章主要针对显示的一些异常问题做初步的排查，经常需要 debug 和 dump 的模块用红框进行标识。

30.2.2 Android System Debug 功能

Android 提供了很多功能，可以获取当前系统服务的信息。针对 display 部分，我们经常需要用到的 debug 手段主要有如下几类。

30.2.2.1 Dump Surfaceflinger

对于很多 Android 的显示问题，我们需要 Dump SurfaceFlinger 信息来获取当前 display 和图层的相关信息。

具体命令：

```
adb shell dumpsys SurfaceFlinger > sf.txt
```


根据 dump 的文件，首先可以查看当前显示的 display，display 包括主屏，副屏和 virtual display。示例如下，当前显示的三个 display，分别是 primary display、external display 和 virtual display：

Displays (3 entries)

```
+ DisplayDevice{0, internal, primary, "Internal display"}
+ DisplayDevice{1, external, "External display"}
+ DisplayDevice{11529215048202569052, virtual, "stdout"}
```

根据 dump 出来的信息可以查看每个 display 当前显示的 layer，各个 layer 的合成方式（查看 Comp Type，Client 代表走 GPU，Device 代表硬件 overlay），以及 layer 的坐标和大小等信息，示例如下：

Display 0 (active) HWC layers:										
Layer name	Z	Window Type	Comp Type	Transform	Disp Frame (LTRB)	Source Crop (LTRB)	Frame Rate (Explicit)	(Seamlessness)	[Focused]	
Wallpaper BBQ wrapper#1	rel 0	2013	DEVICE	0	0 0 1024 600	99.0 58.0 2086.0 1222.0				[]
com.android.car.carlauncher/com.android.car.carlauncher.CarLauncher#0	rel 0	1	CLIENT	0	0 0 1024 600	0.0 0.0 1024.0 600.0				[*]
TopCarSystemBar#0	rel 0	2041	CLIENT	0	0 0 1024 50	0.0 0.0 1024.0 50.0				[]
BottomCarSystemBar#0	rel 0	2024	CLIENT	0	0 530 1024 600	0.0 0.0 1024.0 70.0				[]
Display 1 (inactive) HWC layers:										
Layer name	Z	Window Type	Comp Type	Transform	Disp Frame (LTRB)	Source Crop (LTRB)	Frame Rate (Explicit)	(Seamlessness)	[Focused]	
Wallpaper BBQ wrapper#0	rel 0	2013	DEVICE	0	0 0 1024 600	99.0 58.0 2086.0 1222.0				[]
com.android.car.carlauncher/com.andr[...]launcher.SecondaryAppGridActivity#0	rel 0	1	DEVICE	0	0 0 1024 600	0.0 0.0 1024.0 600.0				[*]

图 30-2 dump SF

30.2.2.2 截屏

很多场景下，我们可以通过截屏来辅助分析，确认前端数据是否 ok。

```
adb shell /system/bin/screencap -p /sdcard/screenshot.png
adb pull /sdcard/screenshot.png d:\
```

通过查看截屏出来的图片，确认送往显示设备之前的数据是否正常。

30.2.2.3 Virtual Display

很多场景下，我们需要确认前端数据是否正常，可以通过 virtual display 手段来排查。

Virtual display 主要的方法有：vysor，录屏。

1. Vysor

Vysor 是 Chrome 的一个应用，能够通过 usb 直接显示和控制 android 设备。使用前需要连接上 adb。

可以通过 `vysor` 查看显示是否正常。

2. 录屏

针对有些动态操作或者视频等问题，可以通过录屏操作，做初步排查。

命令：

```
adb shell screenrecord /sdcard/test.mp4
adb pull /sdcard/test.mp4
```

30.2.2.4 Vsync 节拍

有些场景，我们可以通过 check vsync 节拍，来辅助确认硬件是否工作异常。

命令：`adb shell cat /proc/interrupts > interrupts.txt`

Check 的中断名为 “DPU_TGx”，Check 方法如下：

1. 可以通过 log 查看当前 kernel 启动的时间，如当前对应启动了约 13400s。

另：无 UART 环境查看 kernel 启动时间可以 `cat /proc/uptime`，获取的第一个时间就是 kernel 启动时间。

2. 按照 1s 60 帧的频率，即 $13400 \times 60 = 804000$ ，即理论值
3. 在 cat 出来的文件中，将 4 个核的中断数目相加，即实际值
4. 比较理论值和实际值是否一致，如一致证明 vsync 在驱动层处理正常。

30.2.2.5 OpenGL ES 支持版本

在 Debug GPU 问题，有时候需要确认 OpenGL ES/EGL 的版本信息，如 EGL 创建 Context，Choose Config 时需要确认 OpenGL ES version；以及确认当前 OpenGL ES 是否支持的 RGBA/sRGB 压缩、texture buffer 等 feature 时，可以通过 `dumpsys SurfaceFlinger` 来获取 OpenGL ES 版本信息：

```
-----RE-----
EGL implementation : 1.5 Android META-EGL
GL ES: Imagination Technologies, PowerVR B-Series BXM-4-64, OpenGL ES 3.2 build
1.16@6099580
```

表示当前 OpenGL ES 版本为 3.2，EGL 版本为 1.5。

30.2.2.6 GPU loading dump

执行命令：

```
adb shell cat /sys/kernel/debug/pvr/status
```

可以 dump 当前 GPU 状态， GPU Utilisation 项表示 GPU 利用率：

```
Driver Status: OK
Device ID: 0:128
Firmware Status: OK
Server Errors: 0
HWR Event Count: 2
CRR Event Count: 0
FWF Event Count: 0
APM Event Count: 3
SLR Event Count: 0
WGP Error Count: 0
TRP Error Count: 0
GPU Utilisation: 20%
```

如上图， 当前 GPU 利用率为 20%。

30.2.3 HWC Dump Debug 功能

Hwcomposer 负责将系统中需要显示的每一层 layer 能够合理并流畅的显示到屏幕上。HWC 提供 buffer dump 和 config dump 的 debug 功能，可以将送给 display 硬件之前的数据和信息进行 dump。具体如下。

30.2.3.1 Dump layer buffer 功能

Dump 命令如下：

```
adb root
adb shell setenforce 0 //关闭 selinux
adb shell setprop debug.hwc.dump_buffer 1 //开始 dump
adb shell setprop debug.hwc.dump_buffer 0 //停止 dump
```

dump 的内容为：每次刷新时，hwc 收到的 layer，以及部分 layer 走 gpu 合成得到的 fbtarget layer

dump 的路径为：/data/hwc/，文件名命名规则为：

count_displayId_format_widthxheight(stridexheight)_layer(compositionType_zorder)_isFBC.bin

如图例所示，以-----分界共有 3 帧，每帧由数个 layer 合成：



图 30-3 dump HWC

其中 fbtarget 的 zorder 设为-1, hwc dump 可以选择使用 DebugTool display 工具, 可以直接显示 dump 的图像。

30.2.3.2 Dump layer config 功能

Dump 命令如下:

```
adb shell setprop debug.hwc.dump_config 1 //开始 dump
```

```
adb shell setprop debug.hwc.dump_config 0 //停止 dump
```

dump 的内容显示在 logcat 中，包括：

- 每次 vsync，hwc 收到的各 layer、fbtarget layer 的 config
- 每层 layer 和 overlay 硬件资源的映射关系

```
- Display on: Display-50
zorder|  l,  t,  r,  b|  l,  t,  r,  b|  trans|type|blend|alpha|marked|line
5|  0,  0,1024, 50|  0, 550,1024, 600|  0|  1|  1|1.000|  0| 208|
4|  0,  0,1024, 50|  0,  0,1024, 50|  0|  1|  1|1.000|  0| 208|
3|  0,  0,1024, 70|  0, 530,1024, 600|  0|  1|  1|1.000|  0| 208|
2|  0,  0,1024, 50|  0,  0,1024, 50|  0|  1|  1|1.000|  0| 208|
1|  0,  0,1024, 600|  0,  0,1024, 600|  0|  1|  1|1.000|  0| 208|
0| 99, 58,2086,1222|  0,  0,1024, 600|  0|  2|  0|1.000|  0| 181|
* -1|  0,  0,1024, 600|  0,  0,1024, 600|  0|  0|  1|1.000|  0| -1|

- Display on: Display-70
zorder|  l,  t,  r,  b|  l,  t,  r,  b|  trans|type|blend|alpha|marked|line
1|  0,  0,1024, 600|  0,  0,1024, 600|  0|  2|  1|1.000|  0| 181|
0| 99, 58,2086,1222|  0,  0,1024, 600|  0|  2|  0|1.000|  0| 181|
* -1|  0,  0,1024, 600|  0,  0,1024, 600|  0|  0|  1|1.000|  0| -1|

Display-50:
hwc_zorder|plan id|plane zorder
0          |31      | 0
-1         |39      | 1

Display-70:
hwc_zorder|plan id|plane zorder
0          |51      | 0
1          |59      | 1
```

图 30-4 dump HWC layer config

关于 hwc 的 dump 和图片查看，可以选择使用 DebugTool display 工具。

30.2.4 Atcsurface debug 功能

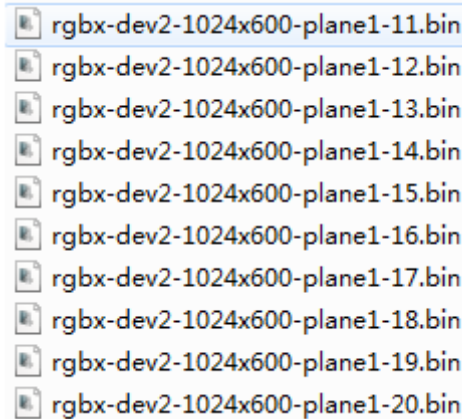
Atcsurface 是我司实现的快速显示框架。可以支持 buffer dump 功能来 check 送显数据是否存在异常。

Dump 命令: `echo dumpbuffer flag area count > /sys/kernel/debug/vout-debug`, 其中 flag 代表开关（1 代表打开 dump, 0 代表关闭 dump），area 是指 dump 的区域（0 代表整个区域, 1/2/3/4 代表从上到下的 1/4 区域），count 代表需要 dump 的帧数。示例如下：

```
adb root
adb shell setenforce 0 //关闭 selinux
adb shell "echo dumpbuffer 1 0 10 > /sys/kernel/debug/vout-debug"
```

dump 出来的文件存放在/data/vout 目录下，如下图所示。

命名方式为 format-devNum-WidthxHeight-planeNumber-count.bin



另外,可以选择使用 DebugTool display 工具直接进行 dump 和查看图像。

30.2.5 Drm screenshot 功能

针对linux系统，截屏功能可以使用drm的screenshot功能来进行截图。

命令如下：

```
adb root
adb shell screenshot
adb pull /data/picture_1024x600_rgba.bin d:\
```

图片的命名方式为 picture_widthxheight_format.bin

pull 出来的 picture.bin 使用常规的 bmpYUV tool 即可查看。

30.3 常见场景分析

30.3.1 UI 问题

Display UI 类问题可以总结为如下几部分，对照第 30.2.2 节的各类 debug 手段，可以初步进行排查。

30.3.1.1 UI 图像撕裂

针对 Android 系统：

- 1) 可以 dump SurfaceFlinger 确认该层 UI 是走 GPU 还是 overlay，方法参见 [Dump Surfaceflinger](#)。

- 2) Dump HWC layer, 方法参见 [hwc dump](#), 确认 HWC 收到的 layer 数据是否有撕裂, 如果收到的 layer 数据就有问题, 需要 check 前端送显模块。
- 3) 如果 HWC 收到的 layer 数据正常, 合成方式是 GPU, check HWC dump 出来的 fbtarget layer 是否异常。如果 fbtarget layer 撕裂, 需要 check GPU 合成的操作; 如果 fbtarget layer 数据正常, 需要继续 check DPU 模块。
- 4) 如果 HWC 收到的 layer 数据正常, 合成方式是 overlay, 需继续 check DPU 模块。
- 5) 同时, 需要关注 log 中是否有 fence 相关异常, 考虑是否 fence 同步问题。

针对 linux 系统:

- 1) 针对使用 atcsurface 送显的, 可以 dump atcsurface 的 buffer, 方法参见 [atcsurface debug](#)。如果 atcsurface dump 的 layer 数据存在撕裂, 需要前端送显模块 check; 如果 atcsurface dump 的 layer 数据正常, 需要 check vout 和 dpu 模块。
- 2) 针对非 atcsurface 送显的, 可以通过 drm screenshot 功能进行截屏, 方法参见 [screenshot](#)。如果截屏图像存在撕裂, 需要前端送显模块 check, 反之需要 check dpu 模块。

30.3.1.2 UI 卡顿

- 1) 可以通过 cat proc 的 interrupt 信息, check 硬件 vsync 节拍是否正常, 方法参见 [vsync check](#)。
- 2) Check 当前 CPU loading 状态。
- 3) 查看 log 中有没有模块耗时过多的 warning log。

30.3.1.3 UI 花屏、黑屏、白屏、绿屏等显示异常

针对 Android 系统:

- 1) 先可以通过 screenshot 截屏、vysor 查看、hwc dump 各种手段 check source 数据是否正常, 方法参见 [截屏 / virtual display / hwc dump](#)。
- 2) 如果 source 数据就有问题, 从前端模块开始 check。同时需要注意的是, 如果 source 数据异常, 需要客户确认渲染工具是通过软件 CPU 还是 GPU 渲染, 如果 CPU 操作的话需要确认是否有 clean cache 操作。
- 3) 如果碰到字体显示异常, 可以优先怀疑同步问题, 可以通过录屏来 debug, 方法参见 [录屏](#)。
- 4) 如果前端数据 ok, 如果合成方式是 GPU, check gpu 合成操作是否有异常, 否则需要往后继续 check DPU 模块; 如果合成方式是 device, 需要 check DPU 模块。

针对 linux 系统:

- 1) 针对使用 atcsurface 送显的，可以 dump atcsurface 的 buffer，方法参见 [atcsurface dump](#)。如果 atcsurface dump 的 layer 数据存在异常，需要前端送显模块 check；如果 atcsurface dump 的 layer 数据正常，需要 check vout 和 dpu 模块。
- 2) 针对非 atcsurface 送显的，可以通过 drm screenshot 功能进行截屏，方法参见 [screenshot](#)。如果截屏图像存在异常，需要前端送显模块 check，反之需要 check dpu 模块。

30.3.1.4 UI 回帧

针对 Android 系统：

- 1) 可以通过录屏来 debug，check 前端数据是否有问题，方法参见[录屏](#)。
- 2) 如果 source 数据就有回帧问题，从前端模块开始 check。
- 3) 如果前端没有回帧，结合合成方式继续 check GPU 或 DPU。

针对 linux 系统：

- 1) 针对使用 atcsurface 送显的，可以 dump atcsurface 的 buffer，方法参见 [atcsurface dump](#)。如果 atcsurface dump 的 layer 数据存在异常，需要前端送显模块 check；如果 atcsurface dump 的 layer 数据正常，需要 check vout 和 dpu 模块。
- 2) 针对非 atcsurface 送显的，可以通过 drm screenshot 功能进行截屏，方法参见 [screenshot](#)，如果截屏图像存在异常，需要前端送显模块 check，反之需要 check dpu 模块。

30.3.1.5 屏幕闪烁

- 1) 通过更换 panel，交叉实验，确认是否是屏的问题
- 2) 考虑上电时序问题

30.3.1.6 色彩失真、颜色/灰阶、过渡等异常

- 1) 先可以通过 screenshot 截屏、vysor 查看、hwc dump、atcsurface dump 各种手段 check source 数据是否正常，方法参见[截屏](#) / [virtual display](#) / [hwc dump/atcsurface dump/screenshot](#)。
- 2) 考虑 PQ 问题。

30.3.2 Video 问题

Display Video 类问题可以总结为如下几部分，对照上一章节的各类 debug 手段，可以初步进行排查。

30.3.2.1 Video 撕裂

针对 Android 系统：

- 1) Dump SurfaceFlinger 确认 video layer 走 GPU 还是 overlay，方法参见 [Dump Surfaceflinger](#)。
- 2) Dump HWC，方法参见 [hwc dump](#)。确认 HWC 收到的 layer 数据是否有撕裂，如果收到的 layer 数据就有问题，需要 check 前端模块。
- 3) 如果 HWC 收到的 layer 数据正常，合成方式是 GPU，check HWC dump 出来的 fbtarget layer 是否异常。如果 fbtarget layer 撕裂，需要 check GPU 合成的操作；如果 fbtarget layer 数据正常，需要继续 check DPU 模块。
- 4) 如果 HWC 收到的 layer 数据正常，合成方式是 overlay，需继续 check DPU 模块。
- 5) 同时，需要关注 log 中是否有 fence 相关异常，考虑是否 fence 同步问题。

针对 linux 系统：

- 1) 针对使用 atcsurface 送显的，可以 dump atcsurface 的 buffer，方法参见 [atcsurface dump](#)。
- 2) 如果 atcsurface dump 的 layer 数据存在撕裂，需要前端送显模块 check。
- 3) 如果 atcsurface dump 的 layer 数据正常，需要 check vout 和 dpu 模块。

30.3.2.2 Video 卡顿

- 1) Check log 中是否有模块耗时过多的 warning log；
- 2) 可以 check 当前 vsync 节拍；
- 3) 针对 android 系统可以使用 android System trace 进行 debug 分析。

30.3.2.3 Video 花屏、黑屏、白屏、绿屏等显示异常

针对 Android 系统：

- 1) Dump HWC，确认 video layer 数据是否正常，方法参见 [hwc dump](#)。
- 2) 如果 HWC dump video 数据异常，从前端开始 check。
- 3) 如果 HWC dump video 数据正常，交给 DPU 去继续 check。

针对 linux 系统：

- 1) 针对使用 atcsurface 送显的，可以 dump atcsurface 的 buffer，方法参见 [atcsurface dump](#)。
- 2) 如果 atcsurface dump 的 layer 数据存在异常，需要前端送显模块 check。
- 3) 如果 atcsurface dump 的 layer 数据正常，需要 check vout 和 dpu 模块。

30.3.2.4 Video 回帧

针对 Android 系统：可以通过 dump hwc 的 video layer 确认是前端还是后端问题，方法参见 [hwc dump](#)。

针对 linux 系统：可以通过 dump atcsurface 的 video layer 确认是前端还是后端问题，方法参见 [atcsurface dump](#)。

30.3.3 开机黑屏

有时候会遇到开机就黑屏的场景，首先通过日志确认系统是正常启动，同时确认背光是否点亮。

然后可以按如下方法进行排查：

- 优先考虑 DPIM 与屏参数是否匹配，可以通过如下命令查看当前使用的 dpim：

```
adb root
adb shell
echo dword_read METAZONE_DPIM_INDEX > /proc/mtz_debug
cat /proc/mtz_debug
```

- 确认当前使用的 dpim pipeline 信息

```
adb root
adb shell
echo show > /sys/kernel/debug/dpim
```

详细的 display pipeline 信息会在串口日志中输出。

- 针对 android 系统，查看 log，如果能启动到 android，可以通过 adb 截图或者 vssor 查看分辨率是否正常，方法见[截屏 / virtual display](#)，同时可以 check DPU 中断是否正常，方法见 [vsync check](#)。

如果以上都正常，可以根据屏的使用的情况继续 check LVDS/DSI/DP 硬件相关是否异常。

31 Projection

31.1 简介

31.1.1 功能

AC8025 Cluster 投屏显示功能是将 AC8025 IVI 系统上的导航地图界面或其他定制信息投屏到仪表屏上显示的功能，本功能涉及 Cluster 和 IVI 系统上的多个功能模块，通过这些功能模块可以完成如下功能。

- Android 系统上应用可以通过标准 Android Car Framework 接口指定特定 Activity 投屏到仪表屏；
- Android Native 层或 Cluster 系统可以透过本设计中提供的 API 将存放数据的内存零复制共享到其他系统；
- Android Native 层或 Cluster 系统可以同步或异步发送消息到其他系统处理，其他系统处理完消息后可以将处理完的结果反馈给 Android Native 层或 Cluster 系统；
- Android/Cluster 系统显示驱动层或其他驱动模块可以调用本设计提供的 Driver 层接口将需要共享的内存零复制共享到其他系统。

31.1.2 代码结构及文件描述

表 31-1 Projection 代码结构及描述

文件名	文件路径	文件描述
ATCClusterRenderingService.java	source/vendor/autochips/proprietary/package/ATCRenderingClusterService/	被 CarServices 启动的服务，主要作用创建虚拟屏和 Rds
RemoteDomainSurface.java	source/vendor/autochips/proprietary/frameworks/base/rds/java/com/autochips/rds/	完成不同系统之间的图形渲染共享 surface
com_autochips_idc_RemoteDomainSurface.cpp	source/vendor/autochips/proprietary/frameworks/base/rds/jni/	Rds 的 jni 模块，用来接受生产者数据，将数据发送给消费者

文件名	文件路径	文件描述
client.cpp	source/vendor/autochips/proprietary/hardware/libidc/src/	提供不同系统之间用户空间层通信客户端接口
server.cpp	source/vendor/autochips/proprietary/hardware/libidc/src/	提供不同系统之间用户空间层通信服务端接口

31.2 软件设计

31.2.1 架构图

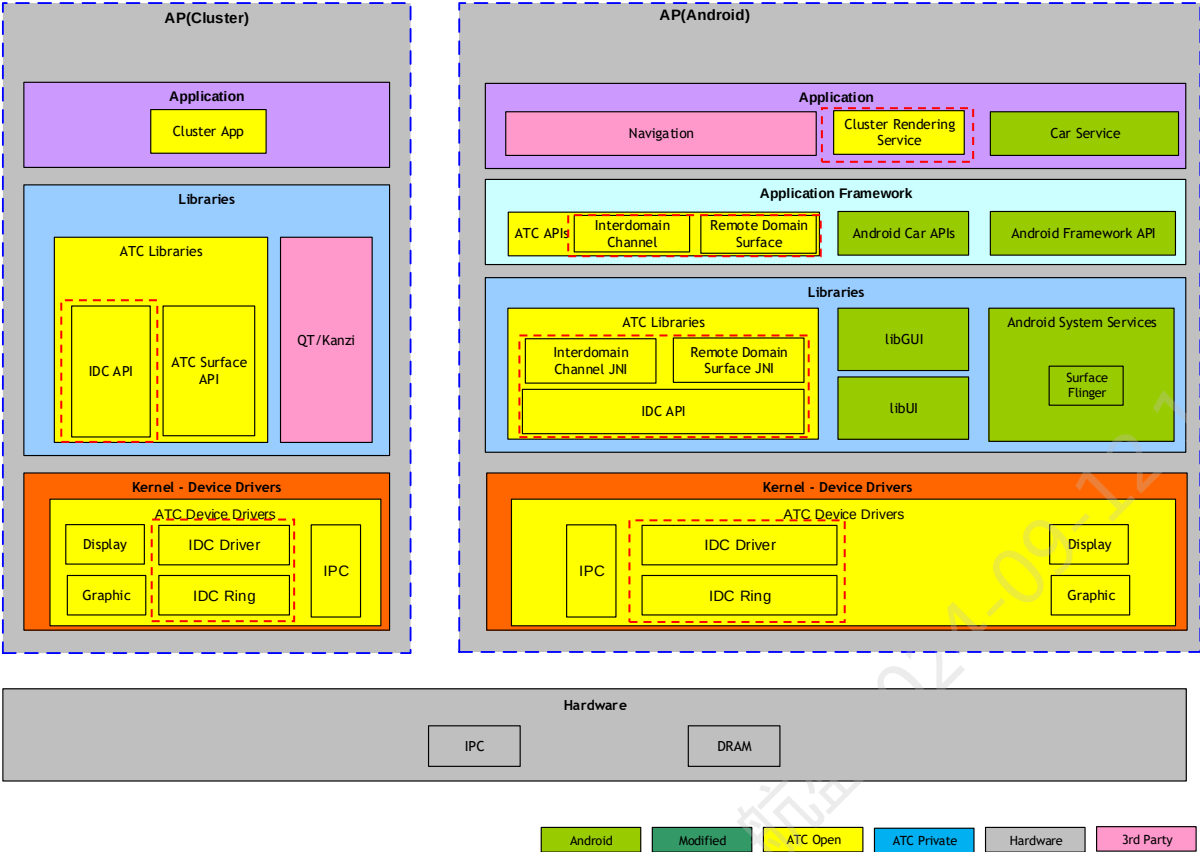


图 31-1 Projection 架构图

CarServices 把 ATCClusterRendering Service 拉起来之后创建 VirtualDisplay，Navigation 会把 Activity 启动到 VirtualDisplay 上利用 GPU 进行渲染。

SurfaceFlinger 会对 Virtualdisplay 上 VirtualDisplaySurface 进行合成，通过 IPC 进程间通信把生产好的 buffer 发送给 RDS 消费者，RDS 通过 IDC 跨系统传送给 Cluster 域的 cluster app 进行显示。

31.2.2 流程图

31.2.2.1 Cluster Rendering Service onCreate 流程

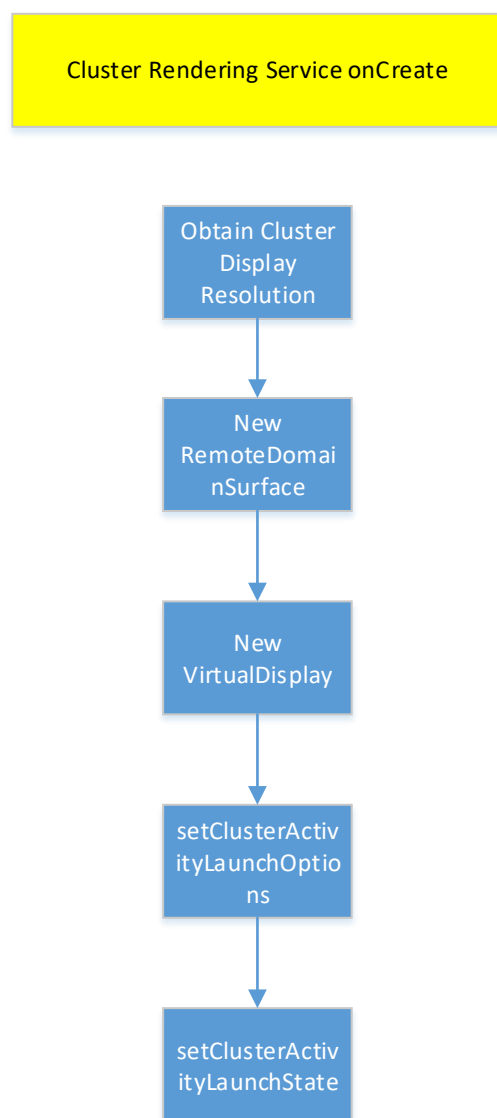


图 31-2 Cluster Rendering Service onCreate 流程图

31.2.2.2 投屏 Buffer 同步流程图

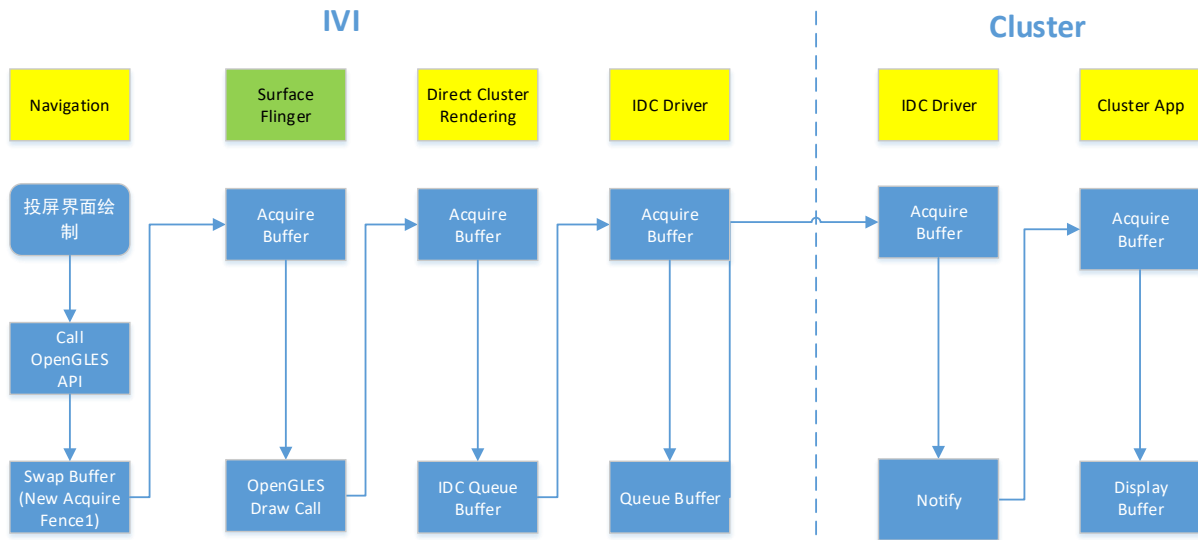


图 31-3 投屏 Buffer 同步流程图

IVI 系统端数据流说明:

- 导航软件调用 OpenGL ES 接口将数据送给 GPU
- GPU 处理完后通过 EGL 模块将处理完的数据送到 Surface Flinger
- Surface Flinger 将要投屏的数据合成到由 Cluster Rendering Service 创建的 Cluster 虚拟屏上, 因 Cluster 虚拟屏绑定了 Remote Domain Surface 对象, 这导致数据最终流向 Remote Domain Surface JNI 层创建的 Buffer Queue.
- Remote Domain Surface JNI 层实现了 IDC Consumer, 并绑定到 Buffer Queue 的消费者, 这导致投屏数据最终流向了 IDC Consumer
- IDC Consumer 通过调用 IDC API 接口将投屏数据送给 IDC Driver
- IDC Driver 透过 IDC Ring 将数据共享给 Cluster 系统

Cluster 系统端数据流说明:

- IDC Driver 收到 IVI 系统端通知后会从 IDC Ring 中获取共享的投屏数据并唤醒 IDC API 模块创建的 Remote Domain Monitor 线程来读取共享数据
- IDC API 模块获取到共享投屏数据后通过 Cluster APP 注册的 callback 通知 Cluster APP
- Cluster APP 获取到数据后直接调用显示相关接口对接 Display Driver, 最终由 Display Driver 将投屏数据送给 DPU HW

31.3 接口描述

31.3.1 API

31.3.1.1 CarAppFocusManager

表 31-2 CarAppFocusManager API

API	说明
addFocusListener	注册监听监测 app 的焦点变化
requestAppFocus	请求应用程序焦点。通过请求这个，应用程序将成为焦点的所有者，
removeFocusListener	停止对于某种 focustype 的监听
getActiveAppTypes()	返回系统中当前活动的应用程序类型。
abandonAppFocus	放弃给定的焦点，即将其标记为不活动。这还涉及释放焦点的所有权。

31.3.1.1.1 addFocusListener

【描述】

注册监测 app 的焦点变化。

【语法】

```
void addFocusListener(OnAppFocusChangedListener listener, AppFocusType int appType)
```

【参数】

- @param OnAppFocusChangedListener [IN]: 注册的 focues 变化的回调
- @param AppFocusType [IN]: Focus 的类型 默认 APP_FOCUS_TYPE_NAVIGATION

【返回值】

无

31.3.1.1.2 requestAppFocus

【描述】

请求应用程序焦点。通过请求这个，应用程序将成为焦点的所有者。

【语法】

```
int requestAppFocus(int appType, OnAppFocusOwnershipCallback ownershipCallback)
```

【参数】

- @param appType [IN]: Focus 的类型 默认 APP_FOCUS_TYPE_NAVIGATION
- @param OnAppFocusOwnershipCallback [IN]: focus lost 和 focus granted 的回调提示

【返回值】

- 0 : failed
- 1 : success

31.3.1.1.3 removeFocusListener**【描述】**

停止对于某种 focus type 的监听。

【语法】

```
void removeFocusListener(OnAppFocusChangedListener listener, AppFocusType int appType)
```

【参数】

- @param appType [IN]: Focus 的类型 默认 APP_FOCUS_TYPE_NAVIGATION
- @param OnAppFocusOwnershipCallback [IN]: focus lost 和 focus granted 的回调提示

【返回值】

无

31.3.1.1.4 getActiveAppTypes**【描述】**

返回系统中当前活动的应用程序类型。

【语法】

```
int[] getActiveAppTypes()
```

【参数】

无

【返回值】

int[] 返回 int 类型的数组，为当前获取 focus 有哪些，例如 navigation focus = 1, voice command focus = 2。

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

868 /1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

31.3.1.1.5 abandonAppFocus

【描述】

放弃给定的焦点，即将其标记为不活动

【语法】

```
void abandonAppFocus(OnAppFocusOwnershipCallback ownershipCallback,AppFocusType int appType)
```

【参数】

- @param appType [IN]: Focus 的类型 默认 APP_FOCUS_TYPE_NAVIGATION
- @param OnAppFocusOwnershipCallback [IN]: focus lost 和 focus granted 的回调提示

【返回值】

无

31.3.1.2 InterdomainChannel

表 31-3 InterdomainChannel API

API	说明
IInterdomainChannel	IInterdomainChannel 构造函数
connect	与其他系统建立通信连接
disconnect	断开与其他系统之间的连接
get	该方法为静态方法，返回缺省的 Inter-Domain Channel 对象
create	该方法为静态方法，用来创建新的 Inter-Domain Channel 对象
setName	设置 Inter-Domain Channel 对象的名字
getName	获取 Inter-Domain Channel 对象的名字
postEvent	发送事件到其他系统，如 BT 电话号码，导航左转/右转信息等。发送事件的类型可以通过事件 ID 来指定，可以发送 raw buffer 数据对应的事件，也可以发送多个 key/value 键值对字符串对应的事件。
postBuffer	发送共享 DMA buffer 到其他系统
postGfxBuffer	发送共享 Graphics Buffer 到其他系统，Graphics Buffer 是带有宽,高,数据格式等信息的共享 DMA buffer
destroy	销毁 Inter-Domain Channel 对象

31.3.1.2.1 create

【描述】

该方法为静态方法, 用来创建新的 Inter-Domain Channel 对象

【语法】

```
*IInterdomainChannel create(void)
```

【参数】

无

【返回值】

- IInterdomainChannel 对象

31.3.1.2.2 connect

【描述】

与其他系统建立通信连接。

【语法】

```
*connect (const char* domain_name)
```

【参数】

- @param domain_name [IN]: 系统的 name

目前有如下字段可供选择:

```
#define IDC_CLUSTER_DOMAIN NAME "cluster"
```

```
#define IDC_ADAS_DOMAIN NAME "adas"
```

```
#define IDC_IVI_DOMAIN NAME "ivi"
```

【返回值】

- true : success
- false: failed

31.3.1.2.3 disconnect

【描述】

与其他系统建立通信断开连接。

【语法】

void connect (void)

【参数】

无

【返回值】

无

31.3.1.2.4 setName

【描述】

设置 Inter-Domain Channel 对象的名字。

【语法】

void setName(const char* name)

【参数】

- @param name [IN] : Inter-Domain Channel 对象的名字
例如: map.projection

【返回值】

无

31.3.1.2.5 getName

【描述】

获取 Inter-Domain Channel 对象的名字

【语法】

void getName(char* name)

【参数】

@param name [OUT] : Inter-Domain Channel 对象的名字

【返回值】

- false: failed
- true: success

31.3.1.2.6 postBuff

【描述】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

871 /1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

发送共享 DMA buffer 到其他系统

【语法】

```
bool postBuffer(idc_buffer_t *buf,IIDCEventCallback *cb,void *extra)
```

【参数】

- @param buf [IN] : 发送的 idc_buffer_t 数据类型的 buff
- @param cb [IN] : IIDCEventCallback 事件 response 回调
- @param extra [IN] : 传递用户自定义信息

【返回值】

- false: failed
- true: success

31.3.1.2.7 postGfxBuff

【描述】

发送共享 Graphics Buffer 到其他系统, Graphics Buffer 是带有宽, 高, 数据格式等信息的共享 DMA buffer。

【语法】

```
bool postBuffer(idc_gfx_buffer_t *buf,IIDCEventCallback *cb,void *extra)
```

【参数】

- @param buf [IN] : 发送的 idc_gfx_buffer_t 数据类型的 buff
- @param cb [IN] : IIDCEventCallback 事件 response 回调
- @param extra [IN] : 传递用户自定义信息

【返回值】

- false: failed
- true: success

31.3.1.2.8 postEvent

【描述】

发送事件到其他系统, 如 BT 电话号码, 导航左转/右转信息等. 发送事件的类型可以通过事件 ID 来指定, 可以发送 raw buffer 数据对应的事件, 也可以发送多个 key/value 键值对字符串对应的事件。

【语法】

```
bool postEvent(idc_event_t *event,IIDCEventCallback *cb,void *extra)
```

【参数】

- @param event [IN] : 发送的 event 事件, 其中包含 raw data, message, event parcel
- @param cb [IN] : IIDCEventCallback 事件 response 回调
- @param extra [IN] : 传递用户自定义信息

【返回值】

- false: failed
- true: success

31.3.1.2.9 destroy

【描述】

销毁 Inter-Domain Channel 对象

【语法】

```
void destroy(void)
```

【参数】

无

【返回值】

无

31.3.1.3 IInterdomainChannelProxy

表 31-4 IInterdomainChannelProxy API

API	说明
addListener	注册事件通知回调函数
removeListener	移除事件通知回调函数
getName	获取 Inter-Domain Channel 对象的名字

31.3.1.3.1 addListener

【描述】

注册事件通知回调函数

【语法】

```
void addListener(IIDCProxyListener *listener)
```

【参数】

@param listener [IN] : 用于接受的 event 事件，其中包含 idc_event, reply 等数据

【返回值】

无

31.3.1.3.2 removeListener

【描述】

移除事件通知回调函数

【语法】

```
void removeListener(IIDCProxyListener *listener)
```

【参数】

@param listener [IN] : 用于接受的 event 事件，其中包含 idc_event, reply 等数据

【返回值】

无

31.3.1.3.3 getName

【描述】

获取 Inter-Domain Channel 对象的名字

【语法】

```
void getName(char*name)
```

【参数】

@param name[OUT] : 用于接受 Inter-Domain Channel 对象的名字

【返回值】

无

31.3.1.4 IInterdomainChannelMonitor

表 31-5 IInterdomainChannelMonitor API

API	说明
get	获取 Inter-Domain Channel Monitor 对象实例

addListener	注册事件通知回调函数
removeListener	移除事件通知回调函数

31.3.1.4.1 get

【描述】

获取 Inter-Domain Channel Monitor 对象实例。

【语法】

```
IDCMonitorImpl get(void)
```

【参数】

无

【返回值】

IDCMonitorImpl 对象实例

31.3.1.4.2 addListener

【描述】

注册事件通知回调函数

【语法】

```
void addListener(IIDCMonitorListener *listener)
```

【参数】

@param listener[IN]：用于接受跨域传过来的 event 事件的监听，其中包含 domain，channel 信息

【返回值】

无

31.3.1.4.3 removeListener

【描述】

移除事件通知回调函数

【语法】

```
void removeListener(IIDCMonitorListener *listener)
```

【参数】

@param listener[IN]：用于接受跨域传过来的 event 事件的监听，其中包含 domain，channel 信息

【返回值】

无

31.3.2 数据结构

表 31-6 Projection 数据结构

数据结构	说明
idc_gfx_buffer	跨系统传输的graphicbuffer结构体
idc_buffer	跨系统传输的buffer结构体
idc_event	跨系统传输的msg 原始的字节流 结构体

```
typedef struct idc_gfx_buffer {  
    uint32_t width;  
    uint32_t height;  
    uint32_t stride;  
    uint32_t format;  
    int fd;  
    int fence;  
  
    uint32_t priv[8];  
} idc_gfx_buffer_t;  
  
struct idc_buffer {  
    int fd;  
    int size;  
    int fence;  
};  
  
struct idc_event {  
    uint32_t id;  
    uint64_t param1;  
    uint64_t param2;  
};
```


31.4 使用说明和注意事项

31.4.1 InstrumentClusterApp 开发注意事项

31.4.1.1 Android car sdk 部署

文件：android.car.jar 放在 libs 下面：



android.car.jar

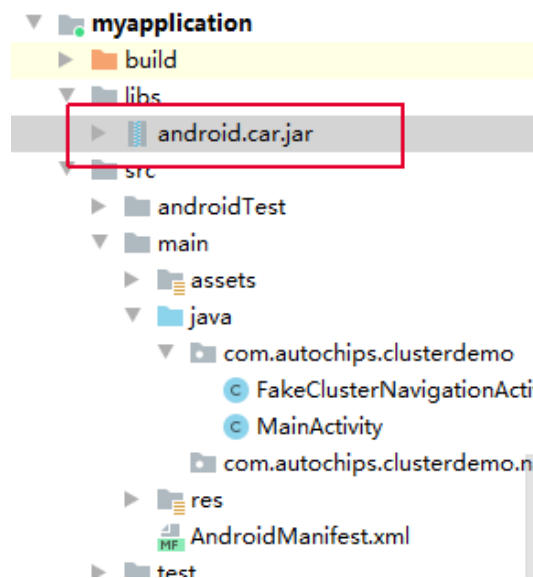


图 31-4 nav map demo 投屏示例图 1

引用：

在 build.gradle 文件的 dependencies {} 加入 implementation files('libs\\android.car.jar')

31.4.1.2 编译成系统应用

在 AndroidManifest.xml 中加入 android:sharedUserId="android.uid.system"

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.autochips.clusterdemo"
    android:sharedUserId="android.uid.system">
```

图 31-5 nav map demo 投屏示例图 2

在 build.gradle 文件 android {} 中加入

```
signingConfigs {
    debug {
        storeFile file('D:\\system_keystore\\platform.keystore')
        storePassword 'android'
        keyPassword 'android'
        keyAlias = 'platform'
    }
    release {
        storeFile file('D:\\system_keystore\\platform.keystore')
        storePassword 'android'
        keyAlias = 'platform'
        keyPassword 'android'
    }
}
```

31.4.1.3 Code 开发

1. 在 AndroidManifest.xml 申请以下权限

```
<uses-permission android:name="android.car.permission.CAR_NAVIGATION_MANAGER" />
<uses-permission android:name="android.car.permission.CAR_DISPLAY_CLUSTER" />
<uses-permission android:name="android.car.permission.CAR_DISPLAY_IN_CLUSTER" />
```

2. 需要投屏的 activity 需要在 AndroidManifest.xml

定义以下 category `<category android:name="android.car.cluster.NAVIGATION" />`

添加 `android:resizeableActivity="true",`

`android:permission="android.car.permission.CAR_INSTRUMENT_CLUSTER_CONTROL"`

3. 标签:

```
<activity
    android:name=".FakeClusterNavigationActivity"
    android:allowEmbedded="true"
    android:launchMode="singleInstance"
    android:permission="android.car.permission.CAR_INSTRUMENT_CLUSTER_CONTROL"
    android:resizeableActivity="true"
    android:theme="@android:style/Theme.NoTitleBar.Fullscreen">
    <intent-filter android:priority="-1">
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.car.cluster.NAVIGATION" />
    </intent-filter>
</activity>
```

图 31-6 nav map demo 投屏示例图 3

4. 初始化 initCarApi

```
private void initCarApi() {
    mCarApi = Car.createCar( context: this, /* handler= */ handler: null,
        Car.CAR_WAIT_TIMEOUT_WAIT_FOREVER, mCarServiceLifecycleListener);
}
```

图 31-7 nav map demo 投屏示例图 4

5. 跳转 navActivity

```
private void startNavActivity() {
    CarInstrumentClusterManager.clusterManager;
    try {
        clusterManager = (CarInstrumentClusterManager) mCarApi.getCarManager(
            android.car.Car.CAR_INSTRUMENT_CLUSTER_SERVICE);
    } catch (CarNotConnectedException e) {
        Log.e(TAG, "Failed to get CarInstrumentClusterManager", e);
        // Toast.makeText(getContext(), "Failed to get CarInstrumentClusterManager",
        // Toast.LENGTH_LONG).show();
        return;
    }

    // Implicit intent
    Intent intent = new Intent(Intent.ACTION_MAIN);
    intent.addCategory(CarInstrumentClusterManager.CATEGORY_NAVIGATION);
    try {
        clusterManager.startActivity(intent);
    } catch (android.car.CarNotConnectedException e) {
        Log.e(TAG, "Failed to startActivity in cluster", e);
        // Toast.makeText(getContext(), "Failed to start activity in cluster",
        // Toast.LENGTH_LONG).show();
        return;
    }
}
```

图 31-8 nav map demo 投屏示例图 5

6. 开启 Cluster 投屏

```
private void initCluster() {
    if (hasFocus()) {
        Log.i(TAG, msg: "Already has focus");
        return;
    }
    mCarAppFocusManager.addFocusListener(mOnAppFocusChangeListener,
        CarAppFocusManager.APP_FOCUS_TYPE_NAVIGATION);
    mCarAppFocusManager.requestAppFocus(CarAppFocusManager.APP_FOCUS_TYPE_NAVIGATION,
        mFocusCallback);
    Log.i(TAG, msg: "Focus requested");
}
```

图 31-9 nav map demo 投屏示例图 6

7. 关闭 cluster 投屏

```
private void stopCluster() {
    stopSendTurn();
    mCarAppFocusManager.removeFocusListener(mOnAppFocusChangeListener,
        CarAppFocusManager.APP_FOCUS_TYPE_NAVIGATION);
    mCarAppFocusManager.abandonAppFocus(mFocusCallback,
        CarAppFocusManager.APP_FOCUS_TYPE_NAVIGATION);
}
```

图 31-10 nav map demo 投屏示例图 7

31.4.2 Android 异常仪表保留最后一帧验证方法

31.4.2.1 Android 端系统异常出现重启

- 切换 cluster 域，adb root , adb shell "echo cluster > /proc/usb_debug"
- 切换 cluster theme，输入 echo b56,1e > sys/devices/platform/vehicle-dev/vehicle_info
- 打开 autochipsAutoClusterTest 应用，点击 RequestFocus 按钮
画面如下：



图 31-11 nav map demo 投屏示例图 8

- 在 cluster 域重启 android 端系统，输入 `echo reboot ivi >/dev/shutdown0,android` 端重启，仪表投屏区域保留最后一帧



图 31-12 nav map demo 投屏示例图 9

31.4.2.2 Android 端 ATCRenderService 进程异常触发

- 切换 cluster 域，adb root ， adb shell "echo cluster > /proc/usb_debug"
- 切换 cluster theme，输入 echo b56,1e > sys/devices/platform/vehicle-dev/vehicle_info，可以通过 adb 模拟 uart 的输入指令来模拟车身属性信息，指令映射意思如下：

枚举 NUM	车身属性	数值类型	数值枚举	实例
--------	------	------	------	----

56	CLUSTER_THEME（仪表盘主题）	int	THEME_0 = 0, THEME_1 = 1, THEME_2 = 2, THEME_3 = 3, THEME_4 = 4,	b56,1e
----	----------------------	-----	--	--------

- 打开 autochipsAutoClusterTest 应用，点击 RequestFocus 按钮，如 [nav map demo 投屏示例图 8](#)
- adb root ， adb shell "echo ivi > /proc/usb_debug"，切换到 ivi 域，然后搜索 cluster service 进程

```
llac8025_demo:/ # ps -A | grep cluster
system      2712      1 10923144  5480 binder_ioctl_write_read 0 S vendor.autochips.hardware.atcluster@1.0-service
u0_a9       5578    332 15079528 100388 do_epoll_wait      0 S com.autochips.clusterservice
llac8025_demo:/ #
```

- kill 2712 进程，仪表画面保留最后一帧，如 [nav map demo 投屏示例图 9](#)

31.5 用户定制化

31.5.1 投屏显示配置

用户可以通过配置 atc_projection.xml 文件来配置投屏区域的显示宽高和最多几块 buffer 轮转。

```
<?xml version="1.0" encoding="UTF-8"?>
<atcprojection>
  <Width>640</Width>
  <Height>480</Height>
  <maxBuffer>7</maxBuffer>
</atcprojection>
```

31.5.2 QT/QD /Kancuster app 集成 idc 功能步骤

1. 引入 libidc 的库，导入头文件<idc_server.h>

```
1 #include <idc_server.h>
2
3 #include <stdio.h>
4 #include <idc_server.h>
5
6 #include <unistd.h>
7 #include <sys/mman.h>
```



```
CPFLAGS += -std=c++11
LOCAL_SHARE_LIBRARIES += -lidl -lstdc++ -lpthread -lsurface_atc -lCommonAPI -lCommonAPI-DBus -ldbus-1
```

- 根据不同的 channel，获取 IInterdomainChannelProxy 对象注册 evnet 监听回调，目前有如下 3 种 channel：MAP_CHANNEL_NAME "map.projection" 监听投屏数据的，IVI_CHANNEL_NAME "cluster.transport" 监听 ivi 发送过来的 message 数据包的，ADAS_CHANNEL_NAME "idc.dms_server_cluster" adas 域发过来的 dms 信息的。

```
{
    KANZILOGD(TAG, "start()");
    pProjectionCallback = callback;
    IInterdomainChannelMonitor *idc_monitor = NULL;

    idc_monitor = IInterdomainChannelMonitor::get();
    if (!idc_monitor) {
        KANZILOGE(TAG, "ChannelMonitor::get() fail");
        return;
    }

    pProjectionChannel = IInterdomainChannelProxy::get(MAP_CHANNEL_NAME);
    if (!pProjectionChannel) {
        KANZILOGE(TAG, "Failed to get map.projection channel proxy!\n");
        return;
    }
    pIviTransportChannel = IInterdomainChannelProxy::get(IVI_CHANNEL_NAME);
    if (!pIviTransportChannel) {
        KANZILOGE(TAG, "Failed to get cluster.transport channel proxy!\n");
        return;
    }
    pAdasTransportChannel = IInterdomainChannelProxy::get(ADAS_CHANNEL_NAME);
    if (!pAdasTransportChannel) {
        KANZILOGE(TAG, "Failed to get idc.dms_server_cluster channel proxy!\n");
        return;
    }
    pProjectionChannel->addListener(this);
    pIviTransportChannel->addListener(this);
    pAdasTransportChannel->addListener(this);
}
```

- 实现回调方法 onEvent，并根据不同的 channelName，和 message 类型来解析不同的数据 data，比如 IDC_EVENT_MESSAGE 传输过来的一般是当前播放的音乐，专辑图片，蓝牙电话的状态，联系人头像 IDC_EVENT_GFX_DMA_BUFFER，android 投屏的数据流 IDC_EVENT_MESSAGE:解析出来文本信息，图像信息。


```

} else if (IDC_EVENT_MESSAGE == event->id) {
    char channelName[64] = {0};
    proxy->getName(channelName);
    KANZILOGI(TAG, " received idc message event, channelName:%s\n", channelName);
    if (0 == strcmp(channelName, ADAS_CHANNEL_NAME)) {
        KANZILOGI(TAG, " ADAS_CHANNEL_NAME\n");
        if (pProjectionCallback)
            pProjectionCallback->onDmsRespond();
    } else if (0 == strcmp(channelName, IVI_CHANNEL_NAME)) {
        KANZILOGI(TAG, " IVI_CHANNEL_NAME\n");
        IDCMessage *message = (IDCMessage*)event->param2;
        KANZILOGI(TAG, " message->m_msg.what:%d\n", message->m_msg.what);
        switch (message->m_msg.what) {
            case RECEIVE_PLAYBACK_STATE: {
                int state = message->getIntExtra(ARG1);

                if (state == MEDIA_STATE_PAUSED || state == MEDIA_STATE_STOPPED) {
                    pProjectionCallback->onPlayStateChanged(state);
                }
            }
            break;
            case RECEIVE_MEDIA_METADATA: {
                std::string mediaName = message->getStringExtra(ARG2);
                int action = message->getIntExtra(ARG4);
                unsigned int dataLength = 0;
                const unsigned char* data = message->getArrayExtra(ARG5, &dataLength);
                KANZILOGD(TAG, "handleAsyncMessage, RECEIVE_MEDIA_METADATA dataLength:%d", dataLength);

                pProjectionCallback->onCurrentSongNameChanged(mediaName);
                pProjectionCallback->onCurrentAlbumPixmapChanged(data, dataLength);
                pProjectionCallback->onPlayStateChanged(MEDIA_STATE_PLAYING);

                if (action == MEDIA_ACTION_NEXT) {
                    pProjectionCallback->onPlayActionChanged(0);
                } else if (action == MEDIA_ACTION_PREVIOUS) {
                    pProjectionCallback->onPlayActionChanged(1);
                }
            }
            break;
            case RECEIVE_ALBUM_IMAGE: {
                std::string mediaId = message->getStringExtra(ARG1);
                unsigned int dataLength = 0;
                const unsigned char* data = message->getArrayExtra(ARG2, &dataLength);
            }
        }
    }
}

```

IDC_EVENT_GFX_DMA_BUFFER:解析出来 fd, 通过 fd, mmap 出来 buffer 指针地址, 来显示

```

} else if (IDC_EVENT_GFX_DMA_BUFFER == event->id) {
    idc_gfx_buffer_t *gfx_buf = (idc_gfx_buffer_t *)event->param1;
    KANZILOGI(TAG, " %s -> gfx buffer event, fd: %d, width: %u, height: %u\n",
        __func__, (int)gfx_buf->fd, gfx_buf->width, gfx_buf->height);
    KANZILOGI(TAG, "DRM_FORMAT_NV12:%d, DRM_FORMAT_RGBA8888:%d, DRM_FORMAT_RGB888:%d", DRM_FORMAT_NV12, DRM_FORMAT_RGBA8888, DRM_FORMAT_RGB888);
    int size = gfx_buf->width * gfx_buf->height * ARG8888CHANNELS;
    if (DRM_FORMAT_NV12 == gfx_buf->format) {
        size = gfx_buf->width * gfx_buf->height * 1.5;
        KANZILOGI(TAG, "DRM_FORMAT_NV12");
    }
    KANZILOGI(TAG, "gfx_buf->format:%d, size:%d", gfx_buf->format, size);
    if (pProjectionCallback && size == PROJECTION_WIDTH * PROJECTION_HEIGHT * ARG8888CHANNELS) {
        unsigned char *data_buf = (unsigned char *)mmap(NULL, size, PROT_READ,
            MAP_SHARED, gfx_buf->fd, 0);
        KANZILOGI(TAG, " %s -> data buf: %p\n", __func__, data_buf);
        pProjectionCallback->onProjectionPixmapChanged(data_buf, gfx_buf->width, gfx_buf->height, gfx_buf->format);
        munmap(data_buf, size);
    }
}

```

停止 idc 监听的时候需要 remove 掉对应的监听。

31.6 典型问题

- 投屏依赖 CarServices 框架，ATCRenderingService 需要 CarServices 启动，所以系统需要导入 CarServices 框架，才能正常使用 ATC 投屏功能；
- 创建一个 channel 是只能单向传输发送或者接受 buffer 或者 event 事件，不支持既能接受数据，也能发送数据。

航盛2024-09-12 16:50:40

32 Filesystem

32.1 简介

32.1.1 软硬件环境

32.1.1.1 软件环境

模块软件运行环境，包括：

- kernel 5.10
- e2fsck 1.45.4
- sgdisk 1.0.4
- blkid 1.0.0
- ntfsfix 2017.3.23
- libfuse 3.8.0
- ntfs-3g 2017.3.23
- exfatprogs 1.2.0

32.1.1.2 硬件环境

模块需要的硬件环境，包括：

- 两个 USB 端口
 - OTG 口支持 host 和 device 模式的切换，支持 USB2.0 协议
 - Host30 仅支持 host 模式，支持 USB2.0 & USB3.0
- 一个 SD 卡端口

32.1.2 功能

- FAT32/NTFS/EXT4/EXFAT/EROFS 文件系统支持

NTFS 并不是开源文件系统，它的代码和设计都归微软独自私藏，而且还在不断更新。所以 ntfs-3g 项目并没有办法获得官方的一手的而且是实时更新的关于 NTFS 的设计和代码，ntfs-3g 是通过种种逆向手段反推的 NTFS 的文件系统结构，以及如何访问 NTFS 的逻辑。所以 ntfs-3g 对微软的 NTFS 的兼容性是没有微软官方保证的，难免会碰到问题。我们目前提供的版本仅是演示版本，提供针对 ntfs 文

件系统基本的读写功能，对于客户端反馈的关于 ntfs-3g 工具自身的问题，是不 Support 的。如果要避免被微软控诉，建议采用商用版本，比如 Tuxera 公司。

- 支持动态分区功能
- SD 卡/U 盘挂载/卸载
- 支持 Auto Recovery
- 支持 Adoptable Storage
- 支持 Shared Sdcard
- 低存储保护
- FBE

32.1.3 代码结构及文件描述

表 32-1 Filesystem 代码结构及描述

模块名	路径	描述
e2fsck	source/android/external/e2fsprogs/e2fsck	Check & Repair EXT4 partition
sgdisk	source/android/external/gptfdisk	It is used for creation and manipulation of partition tables. In AC8025 Android S, we generally use it's functionalities as follows: ▪ convert disk between MBR and GPT ▪ wipe disk's MBR and GPT data structure ▪ get partition's information
ntfs-3g	source/android/vendor/autochips/proprietary/external/ntfs-3g	It's the realization of NTFS filesystem.
ntfsfix	source/android/vendor/autochips/proprietary/external/ntfs-3g	Check & Repair partition with NTFS filesystem.
fsck_msdos	source/android/external/fsck_msdos	Check & Repair partition with FAT filesystem.
exfatprogs	source/android/vendor/autochips/proprietary/external/exfat	Provide Filesystem Consistance Check & Create Exfat Filesystem function
FuseDaemon	source/android/packages/provider/MediaProvider/jni/	Support the permission control of external storage
Libfuse	source/android/vendor/autochips/proprietary/external/libfuse	对 fuse 功能封装的库，提供一系列的 api，使用户可以更方便、更简单的使用 fuse 功能

32.2 软件设计

32.2.1 架构图

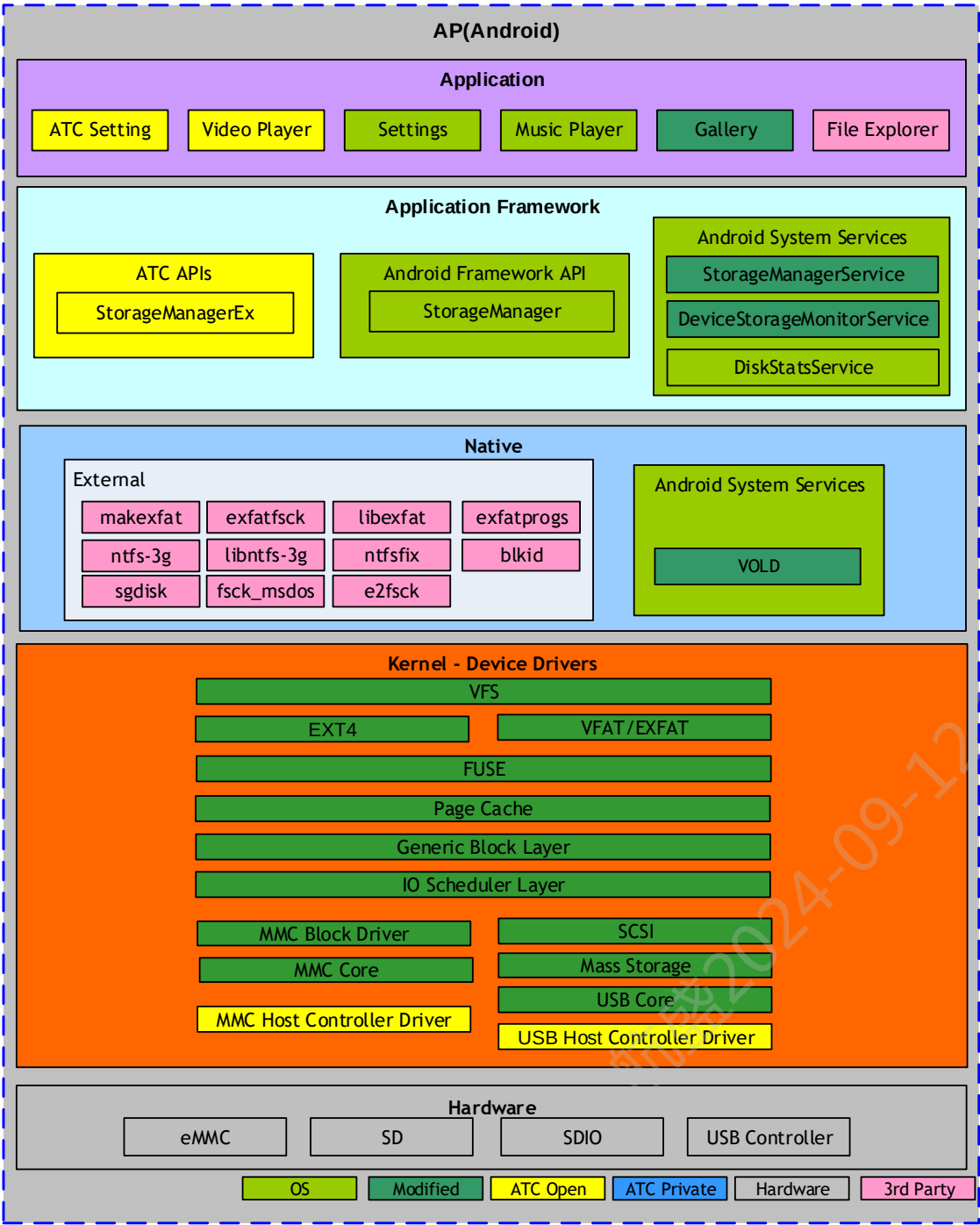


图 32-1 文件系统架构图

32.2.2 流程图

本节简要的介绍可插拔设备（SD 卡、U 盘）的挂载流程。

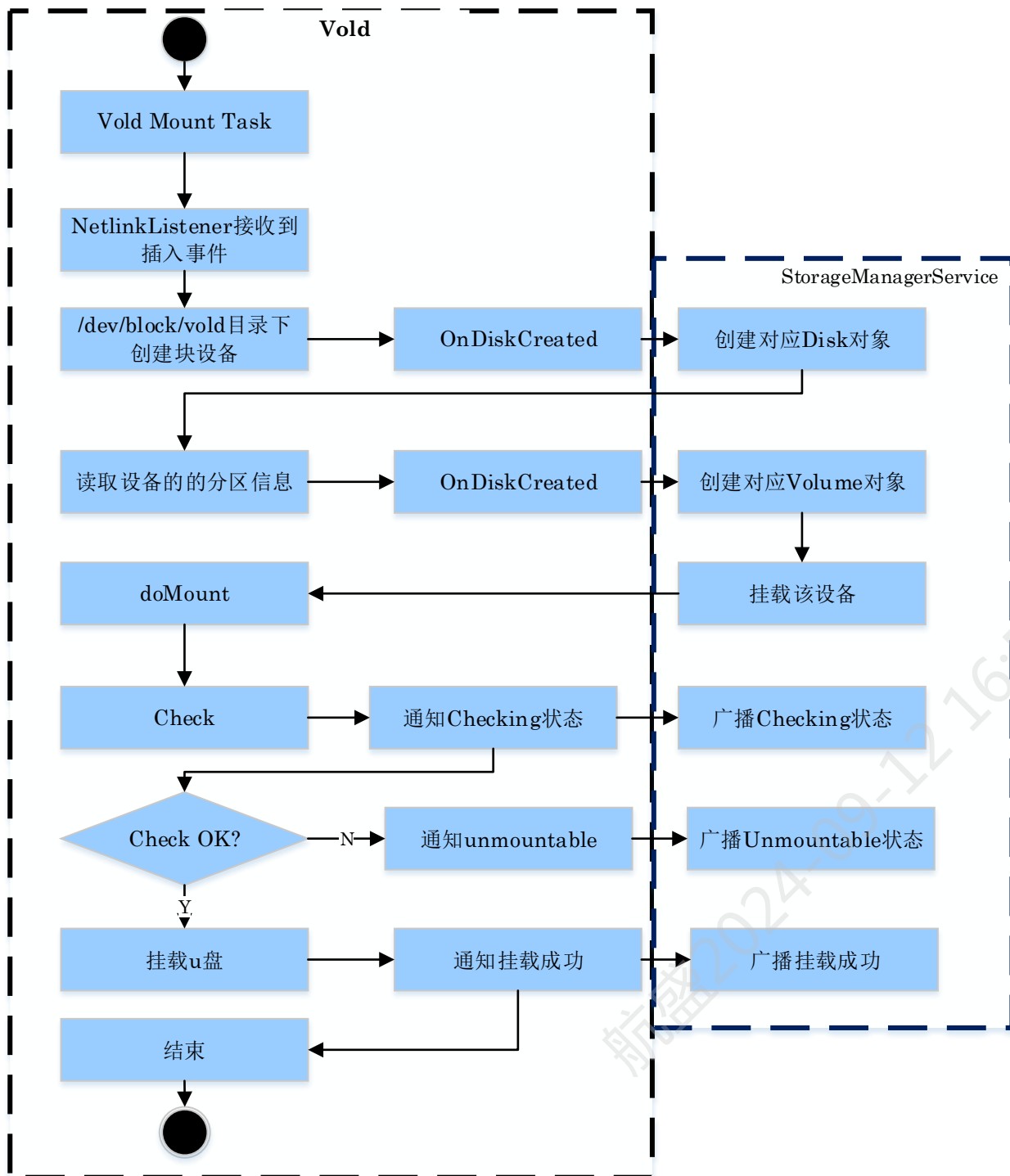


图 32-2 文件系统流程图

- Vold Mount 的主要流程如下：
 - 1) VOLD 接收到设备插入 uevent，解析 uevent，获取需要的消息然后通知 StorageManagerService；
 - 2) StorageManagerService 接收到 VOLD 通知的消息后下发 mount command 给 VOLD；
 - 3) VOLD 接收到 command 后执行 mount 操作，并把操作结果发给 StorageManagerService；
 - 4) StorageManagerService 广播设备插入的相关消息。
- 设备的卸载 flow 如下图所示：

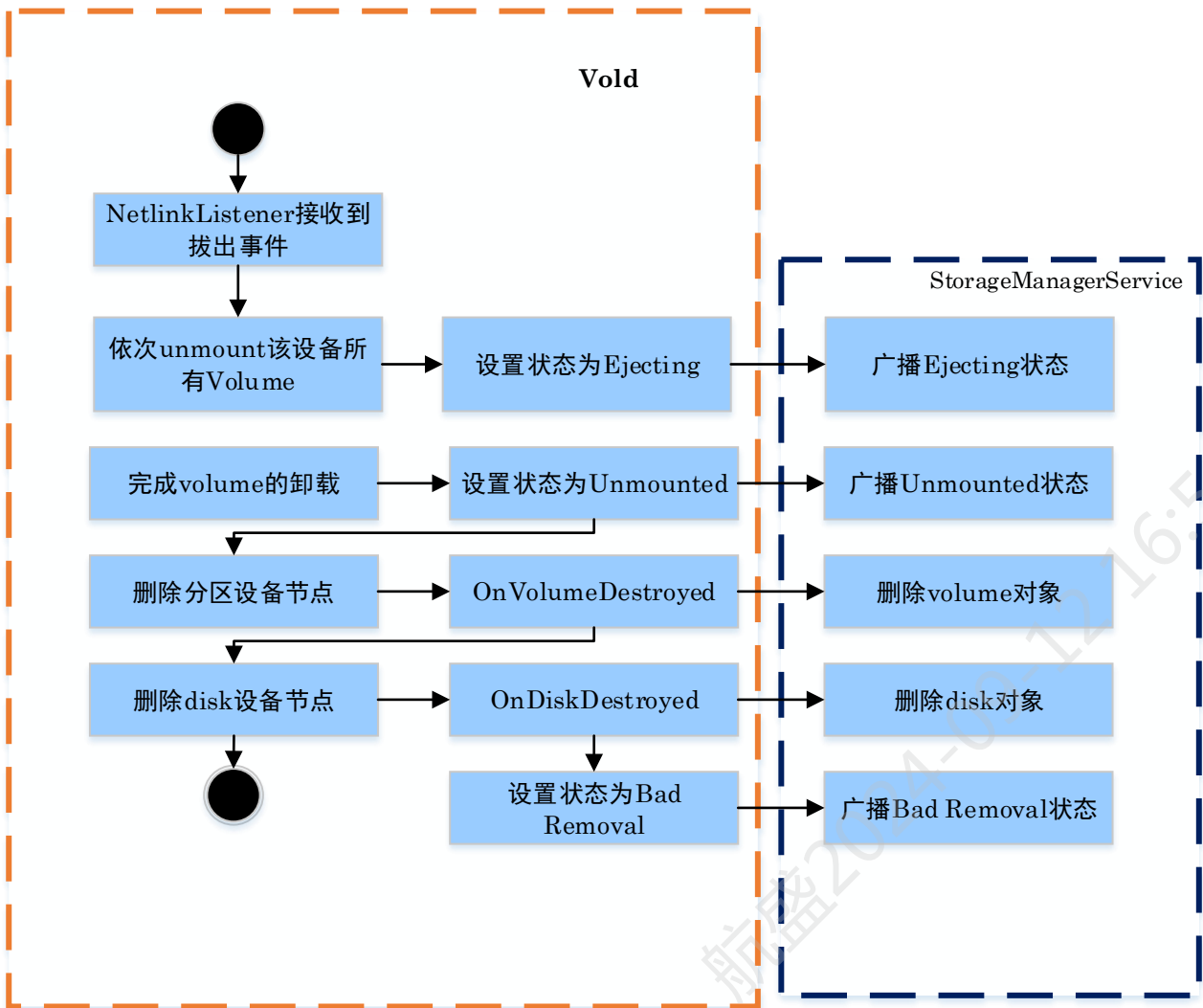


图 32-3 U 盘/SD 卡卸载流程

Unmount 的主要流程如下：

- 1) VOLD 接收到 Kernel 传过来的设备拔出 uevent，解析该 uevent，并执行 unmount 操作，然后把操作结果通知 StorageManagerService；
- 2) StorageManagerService 把设备拔出的相关消息广播出去。

32.3 接口描述

StorageManager 与 StorageManagerEX 提供了接口供用户使用，本节对这些接口进行描述。这些接口都是通过调用 StorageManagerService 提供的接口来实现，其实现方式主要如下所述：

- 调用 StorageManagerService 提供的接口来获取相应信息（下发挂载等相应命令）；
- 根据接口的需求对 StorageManagerService 提供的接口进行组合&封装；
- 根据接口的需求对 StorageManagerService 接口的结果进行过滤。

32.3.1 StorageManager 接口

Path: source/android/frameworks/base/core/java/android/os/storage/StorageManager.java

该类提供系统存储服务的相关操作接口，包括获取已挂载分区，获取默认存储路径，格式化分区，弹出外置存储设备，获取分区状态等操作。

32.3.2 StorageManagerEx 接口

表 32-2 Filesystem StorageManagerEx 接口

Class	Description
com.autochips.storage.StorageManagerEx	此类提供一些接口给上层APP使用。上层APP可以通过调用这些接口获取存储设备的挂在路径和指定路径的挂载状态（mount state/unmount state）

表 32-3 Filesystem Methods Summary

Public Methods	Description
getUsbAllPaths()	获取所有U盘的路径
getUsbMountedPaths()	获取所有处于mount状态的U盘路径
getSdAllPaths()	获取所有SD卡（卡槽）的路径
getSdMountedPaths()	获取所有已经处于mount状态的SD卡的路径
getStorageAllPaths()	获取所有存储设备的路径
getStorageMountedPaths()	获取所有已经处于mount状态的存储设备的路径
getStorageMountedPaths(ArrayList<String>)	获取所有已经处于mount状态的存储设备的路径

arrayList)	
getStorageState(String mountpoint)	获取指定路径的存储设备的挂载/卸载状态

32.3.2.1 getUsbAllPaths

【描述】

获取所有 U 盘的路径，包括挂载状态和卸载状态的 U 盘，通过返回数组的长度可以得到系统支持 U 盘的个数

【语法】

```
public String[] getUsbAllPaths ();
```

【返回值】

string，此字符串包含了获取的 U 盘路径

【参数】

None

32.3.2.2 getUsbMountedPaths

【描述】

获取只包含已经挂载上的 U 盘路径

【语法】

```
public String[] getUsbMountedPaths ();
```

【返回值】

String，此字符串包含了获取的 U 盘路径

【参数】

None

32.3.2.3 getSdAllPaths

【描述】

获取所有 SD 卡的路径，包括挂载状态和卸载状态的 SD 卡，通过返回数组的长度可以得到系统支持 SD 卡的个数

【语法】

```
public String[] getSdAllPaths ();
```

【返回值】

String，此字符串包含了获取的路径

【参数】

None

32.3.2.4 getSdMountedPaths

【描述】

获取只包含已经挂载上的 SD 卡路径

【语法】

```
public String[] getSdMountedPaths ();
```

【返回值】

String，此字符串包含了获取的路径

【参数】

None

32.3.2.5 getStorageAllPaths

【描述】

获取所有存储设备（U 盘/外置 SD 卡/内置 SD 卡）的路径，包括挂载状态和卸载状态。

【语法】

```
public String[] getStorageAllPaths ();
```

【返回值】

String：此字符串包含了获取的路径

【参数】

None

32.3.2.6 getStorageMountedPaths

【描述】

获取只包含已经挂载上的存储设备(U 盘/外置 SD 卡/内置 SD 卡)的路径

【语法】

```
public String[] getStorageMountedPaths ();
```

【返回值】

String，此字符串包含了获取的路径

【参数】

None

32.3.2.7 `getStorageMountedPaths`

【描述】

获取只包含已经挂载上的存储设备(U 盘/外置 SD 卡/内置 SD 卡)的路径

【语法】

```
public void getStorageMountedPaths (ArrayList<String> arrayList);
```

【返回值】

Void

【参数】

arrayList，函数返回时，获取的路径保存在 arrayList 中。

32.3.2.8 `getStorageState`

【描述】

获取指定路径的当前挂载状态（挂载/卸载）

【语法】

```
public String[] getStorageState(String mountPoint);
```

【返回值】

String，此字符串包含了获取的当前状态信息

【参数】

String，指定的路径字符串



注意：

上述描述的接口在 Cockpit 系统中 cluster/adas 域是不支持的，且 cluster/adas 域不支持外设的插拔，主要是由于 cluster/adas 域无支持 u 盘/sdcard 等外设的需求。

32.4 使用说明和注意事项

32.4.1 如何使用 Adoptable Storage

原生 adoptable storage 功能包括：是把设备进行加密，使之无法再被其他平台识别，从而保护存储设备中的内容。

- Settings -> Storage -> SD Card -> Storage settings -> Format as internal -> Erase & FORMAT ->完成。

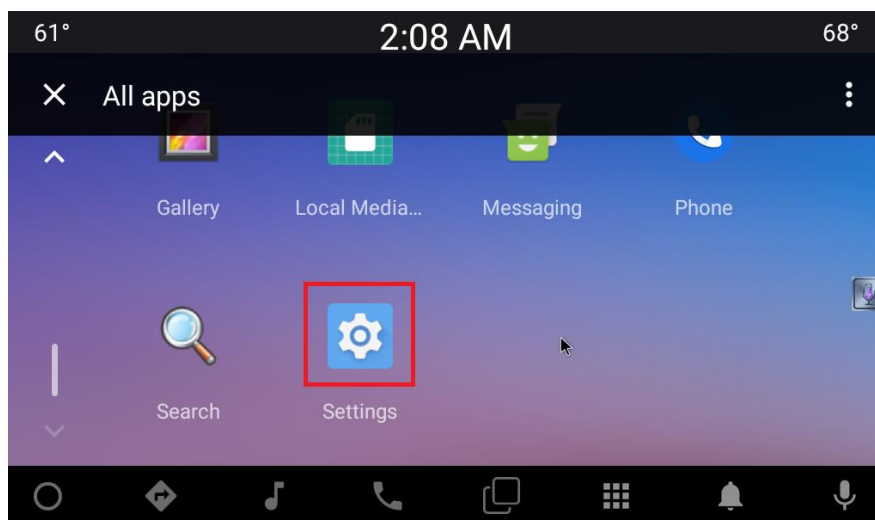


图 32-4 Settings

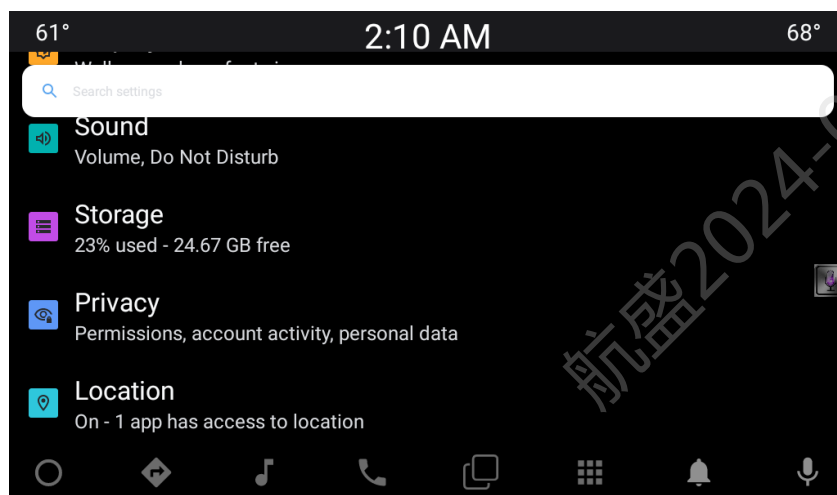


图 32-5 Setting Storage 显示界面

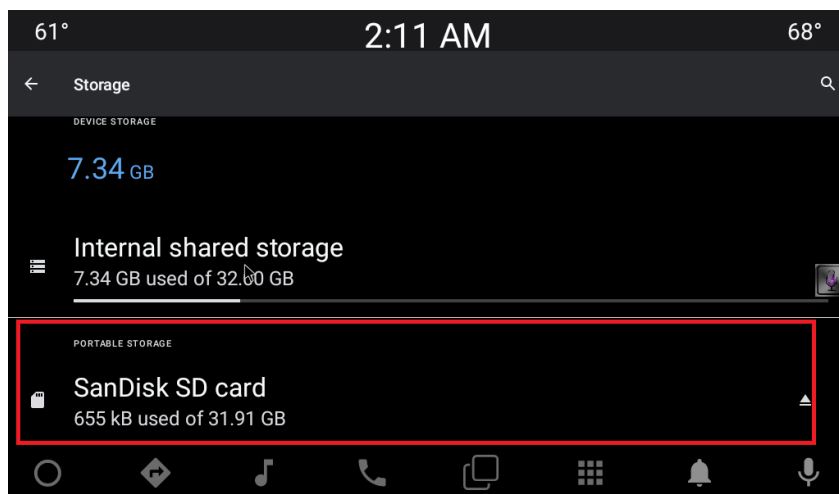


图 32-6 进入 Storage

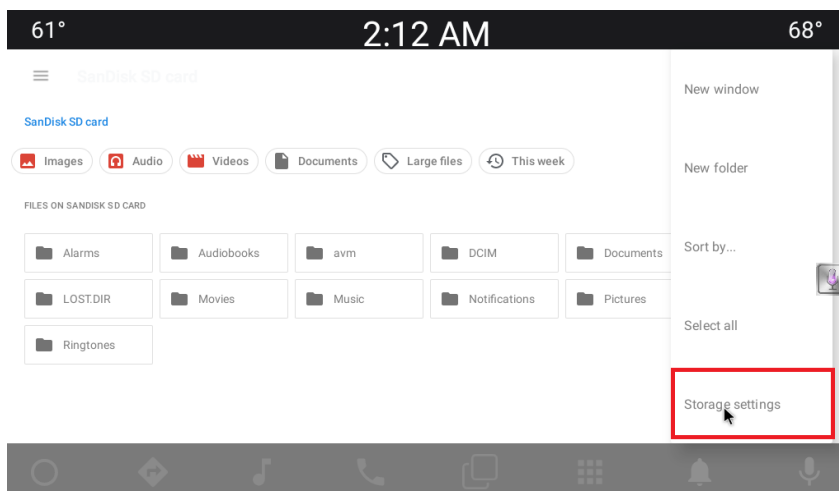


图 32-7 Storage settings

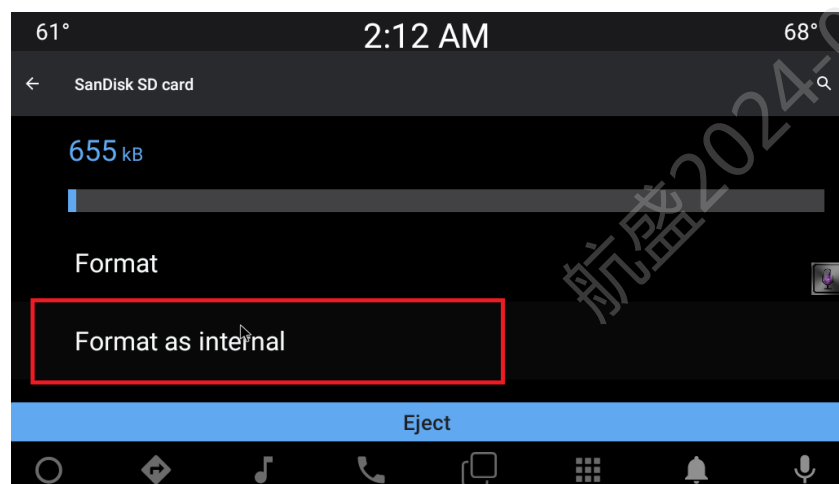


图 32-8 格式化

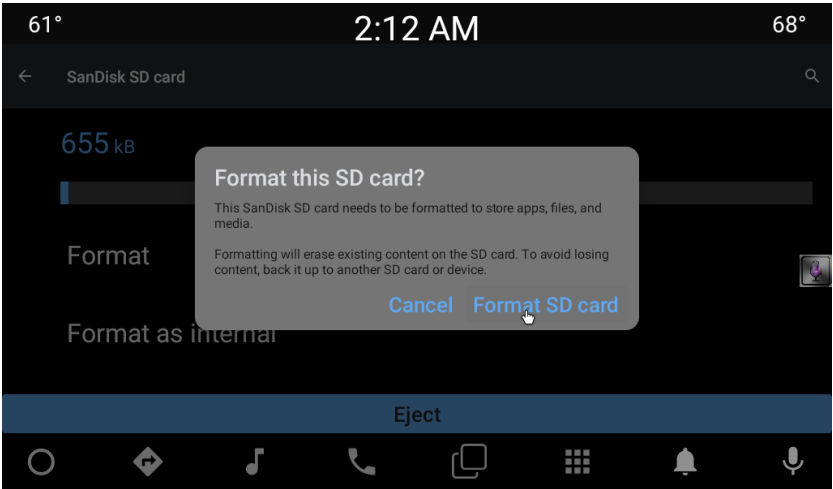


图 32-9 确认

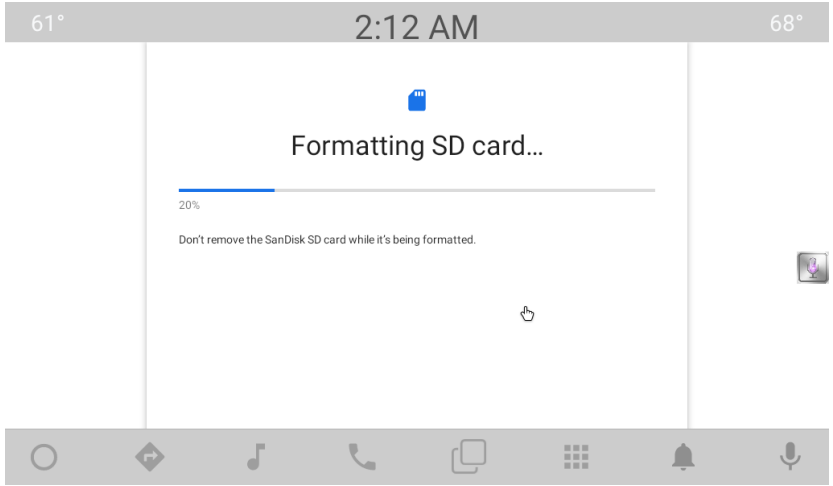


图 32-10 进度

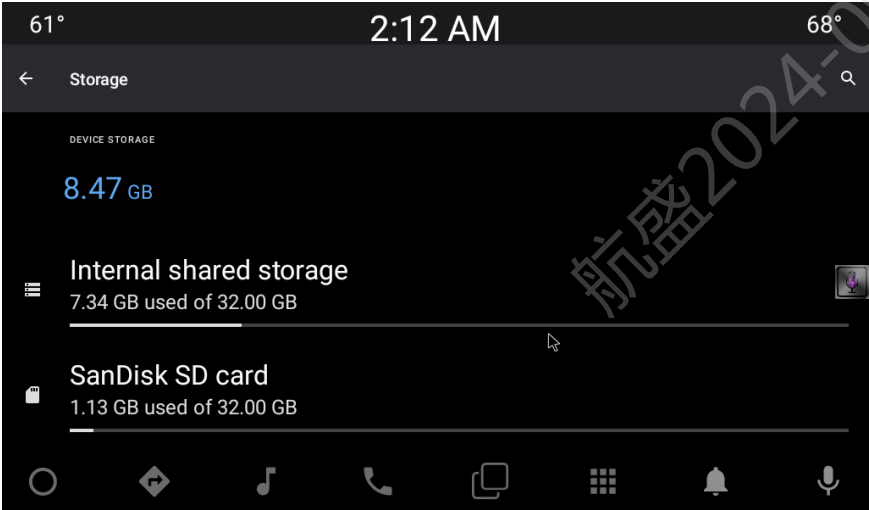


图 32-11 结束

Adoptable Storage 设置成功后，重新进入 settings->storage 界面，UI 上会显示 adoptable storage 的 size 等信息。若需要将 Adoptable Storage 作为 Primary Storage 则需要进入 Setting 选择格式化为 adoptable storage 的设备，点击 Migrate data -> move content 即可。

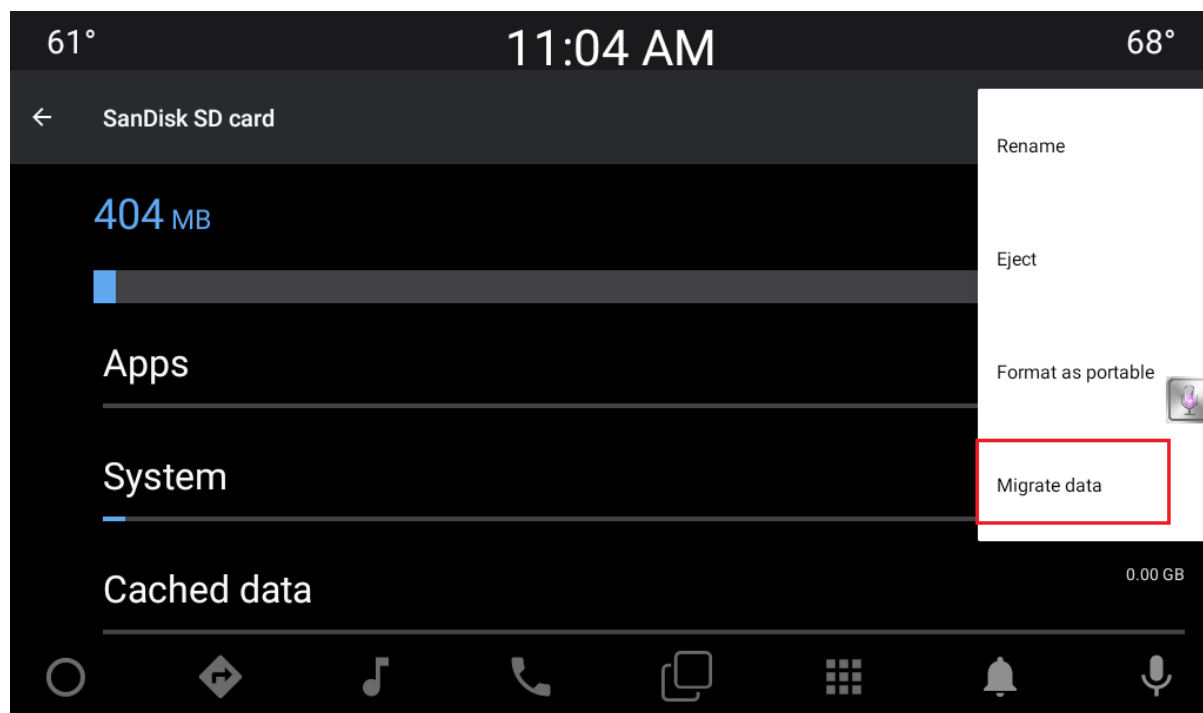


图 32-12 Migrate Data

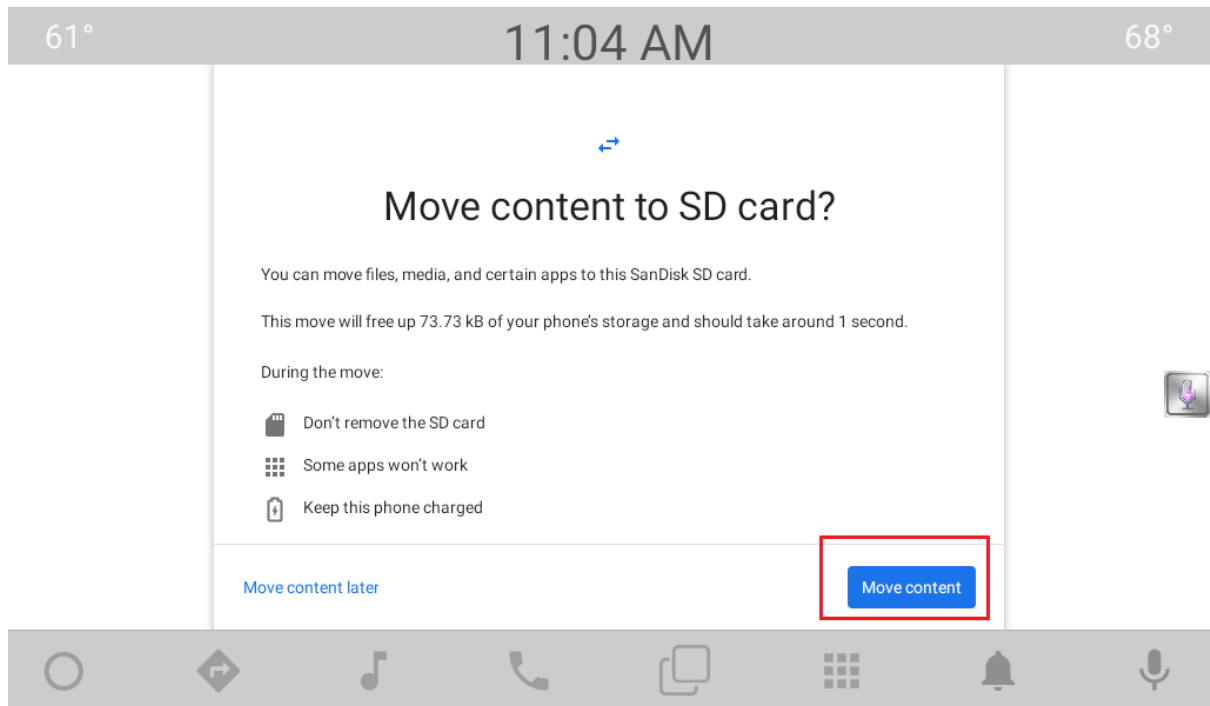


图 32-13 Move Content

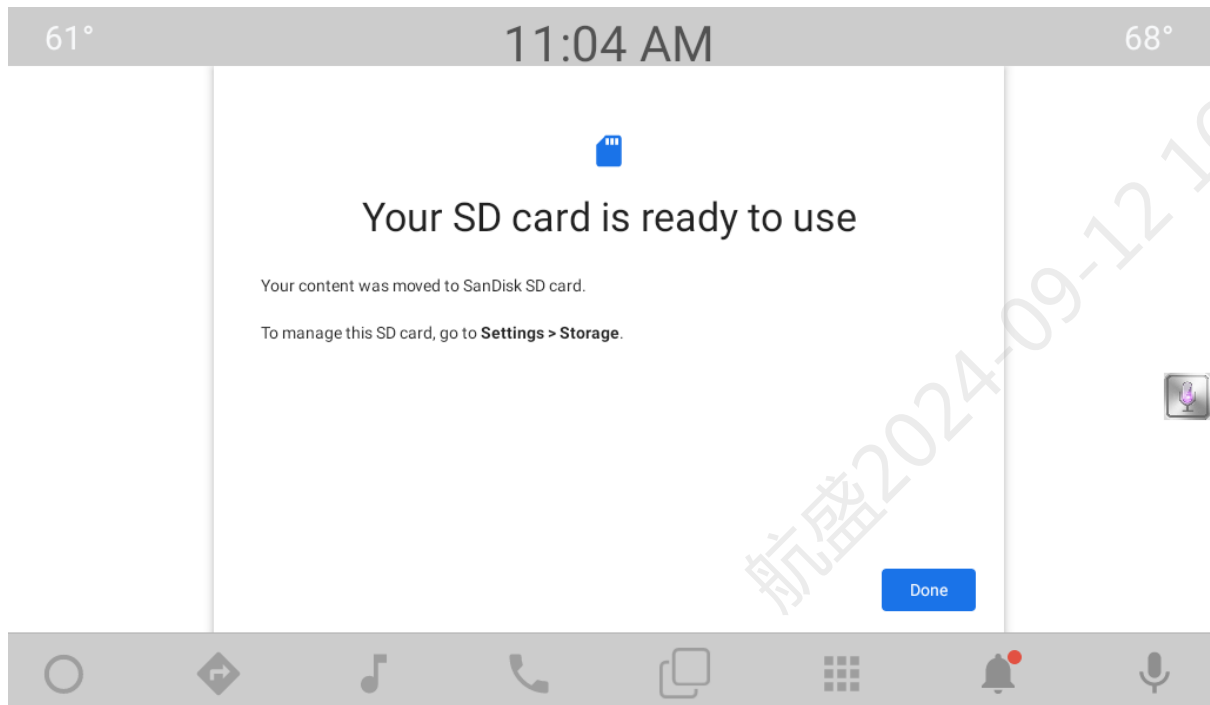


图 32-14 adoptable storage 完成

可以通过同样的方式通过 Setting -> Internal Storage -> Migrate data -> Move content 将内置存储作为 primary storage。若作为 primary storage 的外置设备被拔出，则需要通过 Setting -> 选择作为 primary storage 外置设备 -> forget 即可重新将内置 sdcard 作为 primary storage。

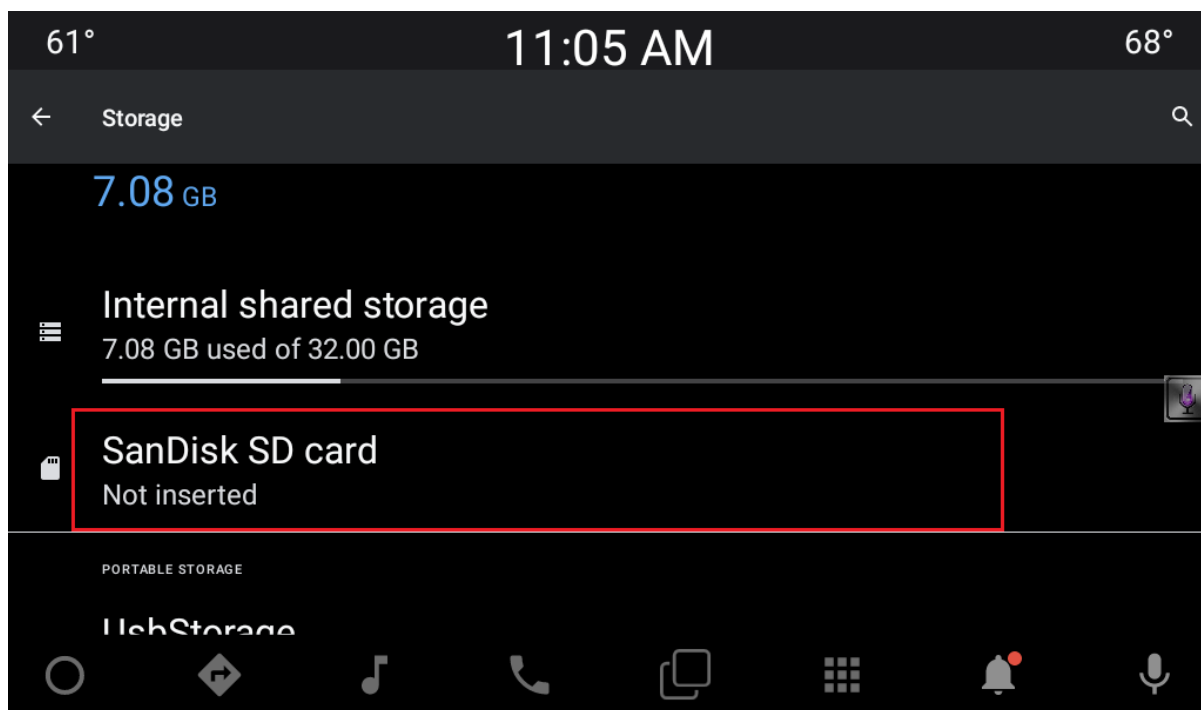


图 32-15 点击作为 Primary Storage 的外置存储设备

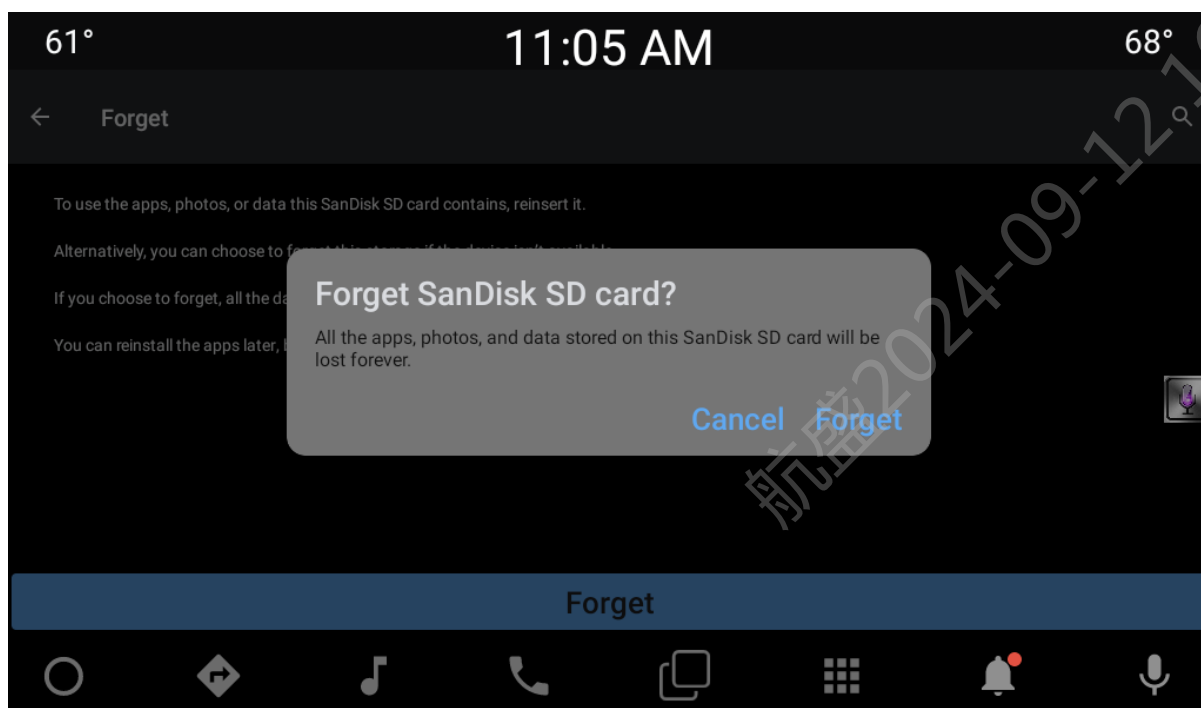


图 32-16 点击 forget 并确认

32.4.1.1 取消 Adoptable Storage

当一个存储设备（U 盘/SD 卡）被设置成 adoptable storage 之后，依然可以通过 settings app 把其恢复成正常存储设备，从而可以被其他平台识别。

- 设置 adoptable storage 成功；
- Settings -> Storage -> SD Card -> Format as internal -> FORMAT ->完成。

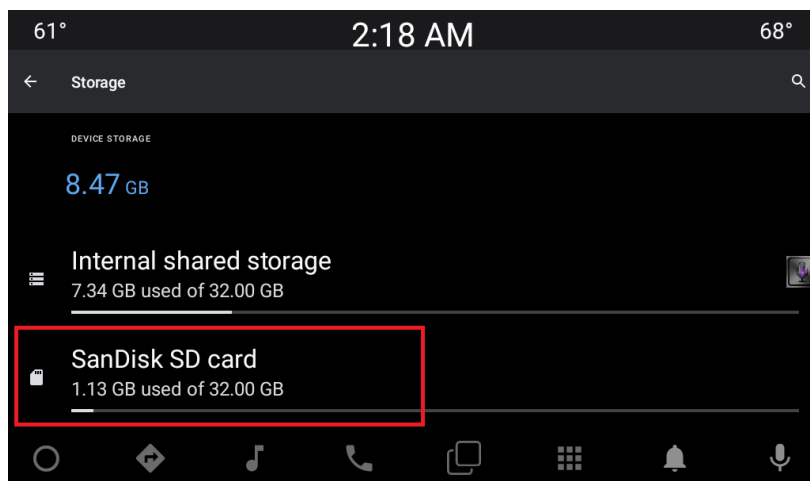


图 32-17 Storage 1

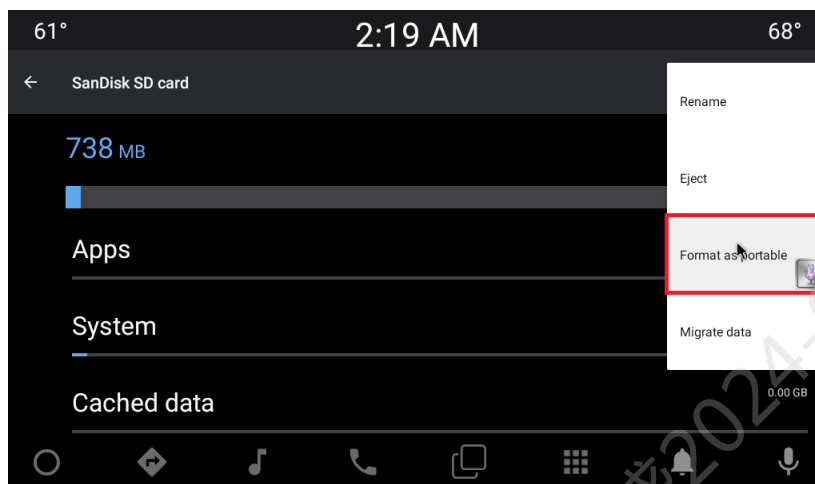


图 32-18 把 Adoptable Storage 格式化成外置存储设备

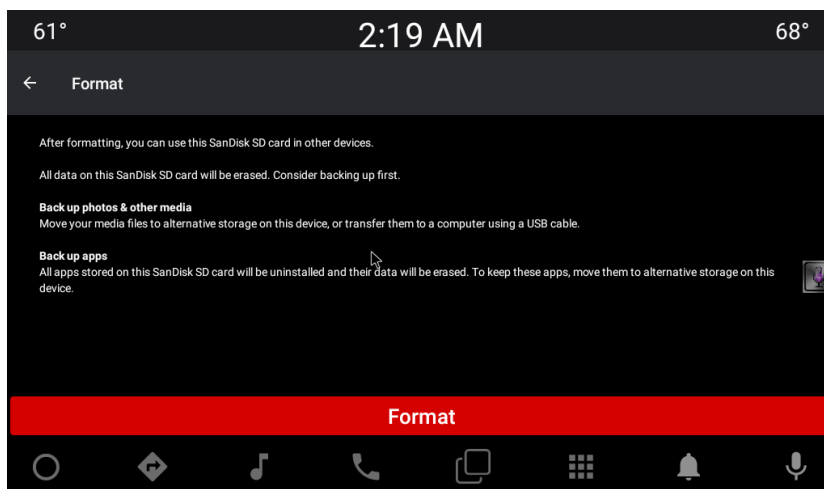


图 32-19 格式化

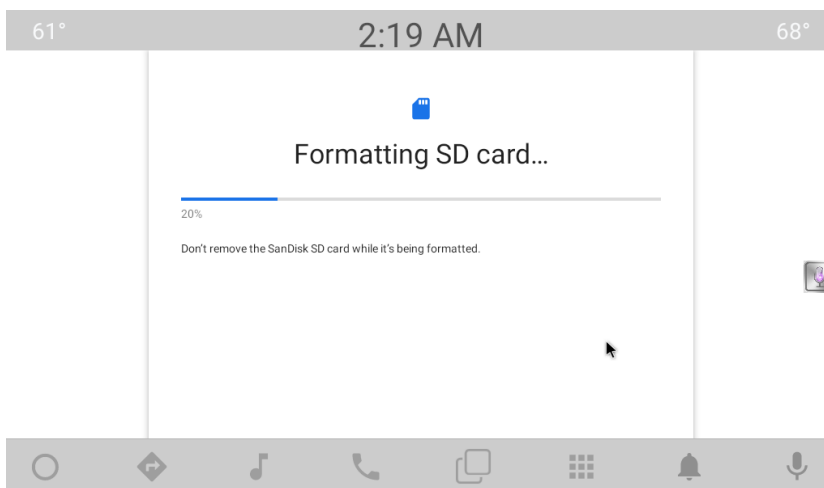


图 32-20 进度

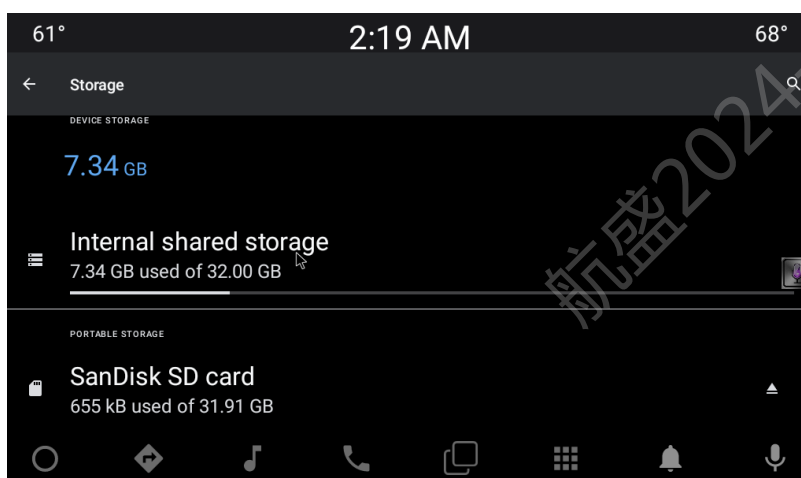


图 32-21 查看外置存储设备状态

32.4.2 如何使用 Auto Recovery

32.4.2.1 访问流程

init 进程在挂载分区时，若发现分区的 fstab 项中存在 autoformat 选项时，会在 e2fsck 和挂载时触发出现致命 fail 时触发格式化操作

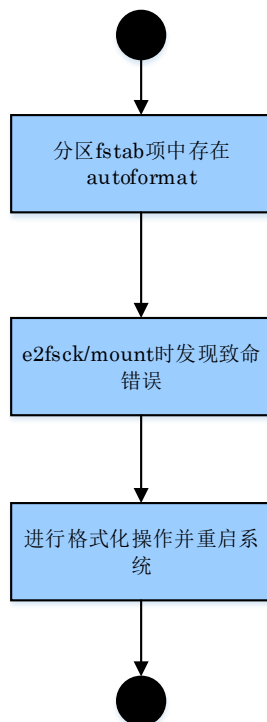


图 32-22 Auto Recovery 分区格式化流程

若发现分区的 fstab 项中存在 autoformat 选项且配置 autorestore 选项时，会在 e2fsck 和挂载出现致命 fail 时触发格式化操作后重启进入 recovery 模式，在 recovery 模式下对配置的 autorestore 路径下的内容做备份操作，并在完成后重启系统，可以在系统启动后/data 查找到 autorestore 备份的内容。

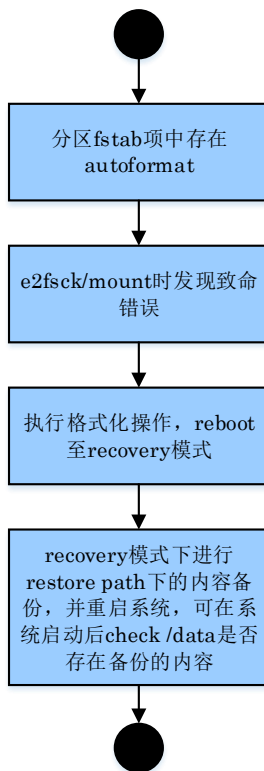


图 32-23 Auto Restore 分区格式化流程

32.4.2.2 使用方法

Fstab path: source/android/vendor/autochips/proprietary/hardware/fstab/ac8x/fstab.in.ac8x.

1. 若分区发生损坏, 可以将分区内容进行擦除, 请添加 `autoformat` 挂载参数:

```
DEVPATH(customer) /customer ext4 FS_FLAG_COMMIT FSMGR_FLAG_FMT FS_FLAG_AUTOFORMAT
```

2. 若分区发生损坏, 可以将分区内容进行擦除且使用备份数据对分区进行恢复, 请添加

`autorestore` 挂载参数:

```
DEVPATH(customer) /customer ext4 FS_FLAG_COMMIT FSMGR_FLAG_FMT FS_FLAG_AUTORESTORE
```

3. 对于 RAW 分区, 请添加 RAW 分区定义:

```
FSTAB_RAW(customer)
```

注意:

- 在 `recovery` 中提供了两种接口, 用于分区数据恢复, `restore_volume` 和 `restore_files`, 其中:
 - `restore_volume` 接口是将 `backup` 分区的数据搬到原分区, 因此需要保证原分区和 `backup` 分区的大小一致。当前默认以 1M 为单位进行数据拷贝, 若要提升单次拷贝数据大小, 请调整 `bootable/recovery/recovery.cpp` 中 `BUFFERSIZE` 的值。

- `restore_files` 接口是将 `backup` 目录中的文件一一拷贝至原分区挂载目录中，注意确保 `backup` 目录中有文件存在，否则 `restore` 将会失败。
- 备份分区内容请用户自行添加，可以在升级时将备份分区内容烧录到 eMMC 中，择时机将备份分区挂载，然后将备份内容拷贝的挂载目录中。
- `fstab` 中对于 RAW 格式分区，请务必添加客制化分区的信息注册：`FSTAB_RAW (customer)`，否则格式化该分区动作将会执行失败！
- RW 的分区在异常断电过程中可能存在分区数据被损坏的可能性，如果是文件系统数据损坏可能会触发到 `autorecovery` 流程修复，如果是非文件系统的数据而是关键文件的内容被损坏则不会触发到 `autorecovery` 流程，但仍会影响到系统启动。故在使用过程中，建议走关机流程，以免出现异常掉电导致的数据损坏。

32.4.3 如何使用添加/删除/修改分区

AC8025 Android S 平台中分区信息定义表格位置：

IVI 系统：

`/build/projects/ivi_s_ac8025/ac8025_demo/partition_table/emmc_ab/ivi_s_ac8025_partition.csv`

Cockpit 系统：

`/build/projects/cockpit_s_linux_ac8025/ac8025_demo/partition_table/emmc_ab/cockpit_s_linux_ac8025_partition.csv`

32.4.3.1 如何删除分区

若需要删除分区，则在分区表文件中删除相应分区的定义即可。

32.4.3.2 添加带有分区系统的分区

分区添加主要涉及到如下流程：

1. 分区 Image 的打包
 - 分区表的修改
 - Image 编译与打包
2. Selinux 权限添加
3. 分区的挂载
 - 挂载点权限的配置

32.4.3.2.1 修改分区表

1. 修改分区表

以新增 customer 分区，分区类型为 ext4，分配 100M 空间为例：

partname	image_type	fs_type	region	size	imagename	dynamic_partition_group	minium_system	operation_type	upgradable	erasable	fastbootable	copyupgradable	emmc_hw_wp	emmc_sw_wp	mountable	recoverable	dualpartition	downloadable
preloader	raw	raw	BOOT	512KB	preloader.img		Y	bootload	Y	Y	Y	Y	Y	Y	Y	N	Y	Y
pgpt	raw	raw	USER	1MB			N	invisible	N	Y	N	N	Y	Y	N	N	N	N
hsm_a	raw	raw	USER	6MB	hsm.img		Y	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
lk_a	raw	raw	USER	2MB	lk.img		Y	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
trustzone	raw	raw	USER	4MB	trustzone.img		Y	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
vendor_b	raw	raw	USER	96MB	vendor_boot.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
boot_a	raw	raw	USER	64MB	boot.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
recovery	raw	raw	USER	70MB	recovery.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
vbmeta_a	raw	raw	USER	1MB	vbmeta.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
vbmeta_sys	raw	raw	USER	1MB	vbmeta_system.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
vbmeta_ver	raw	raw	USER	1MB	vbmeta_vendor.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
dtb_a	raw	raw	USER	1MB	dtb.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
vendor_a	sparse	ext4	USER	512MB	vendor.img	main	N	update	Y	Y	Y	Y	Y	Y	Y	Y	N	Y
system_a	sparse	ext4	USER	3GB	system.img	main	N	update	Y	Y	Y	Y	Y	Y	Y	Y	N	Y
vendor_dl	sparse	ext4	USER	512MB	vendor_dlkm.img	main	N	update	Y	Y	Y	Y	Y	Y	Y	Y	N	Y
product	sparse	ext4	USER	1GB, 512MB	product.img	main	N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
datazone	raw	raw	USER	100MB	datazone-ro.img		N	update	Y	Y	Y	Y	Y	Y	N	N	N	Y
dtbo_a	raw	raw	USER	1MB	dtbo.img		N	update	Y	Y	Y	Y	Y	Y	N	N	N	Y
hsm_b	raw	raw	USER	6MB	hsm.img		Y	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
lk_b	raw	raw	USER	2MB	lk.img		Y	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
trustzone	raw	raw	USER	4MB	trustzone.img		Y	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
vendor_b	raw	raw	USER	96MB	vendor_boot.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
boot_b	raw	raw	USER	64MB	boot.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
recovery	raw	raw	USER	70MB	recovery.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
vbmeta_b	raw	raw	USER	1MB	vbmeta.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
vbmeta_sys	raw	raw	USER	1MB	vbmeta_system.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
vbmeta_ver	raw	raw	USER	1MB	vbmeta_vendor.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
dtb_b	raw	raw	USER	1MB	dtb.img		N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
vendor_b	sparse	ext4	USER	512MB	vendor.img	main	N	update	Y	Y	Y	Y	Y	Y	Y	Y	N	Y
system_b	sparse	ext4	USER	3GB	system.img	main	N	update	Y	Y	Y	Y	Y	Y	Y	Y	N	Y
vendor_dl	sparse	ext4	USER	512MB	vendor_dlkm.img	main	N	update	Y	Y	Y	Y	Y	Y	Y	Y	N	Y
product	sparse	ext4	USER	1GB, 512MB	product.img	main	N	update	Y	Y	Y	Y	Y	Y	N	Y	N	Y
dtbo_b	raw	raw	USER	1MB	dtbo.img		N	update	Y	Y	Y	Y	Y	Y	N	N	N	Y
super	raw	ext4	USER	10GB	super.img		N	update	Y	Y	Y	Y	Y	Y	N	N	N	Y
datazone	raw	raw	USER	100MB	datazone-ro.img		N	update	Y	Y	Y	Y	Y	Y	N	N	N	Y
metazone	raw	raw	USER	1MB	metazone		N	update	Y	Y	Y	Y	Y	Y	N	N	N	Y
datazone	raw	raw	USER	100MB	datazone-rw.img		N	update	Y	Y	Y	Y	N	N	N	N	N	Y
datazone	raw	raw	USER	100MB	datazone-rw.img		N	update	Y	Y	Y	Y	N	N	N	N	N	Y
expdb	raw	raw	USER	30MB			N	update	Y	Y	Y	Y	N	N	N	N	N	N
cache	sparse	ext4	USER	400MB	cache.img		N	update	Y	Y	Y	Y	N	N	Y	N	N	Y
fastdata	sparse	ext4	USER	100MB	fastdata.img		N	update	Y	Y	Y	Y	N	N	Y	N	N	Y
customer	sparse	ext4	USER	100MB	customer.img		N	update	Y	Y	Y	Y	N	N	Y	N	N	Y
nisc	raw	raw	USER	2MB			N	update	Y	Y	Y	Y	N	N	N	N	N	N
metazone	raw	raw	USER	1MB	metazone.bin		N	update	Y	Y	Y	Y	N	N	N	N	N	Y

图 32-24 添加 customer 分区

分区表中各列代表含义如下：

- partname: 分区名称；
- image_type: image 类型；
- fs_type: 文件系统类型；
- region: 分区在 emmc 中区域；
- size: 分区大小，形如"XX GB, XX MB, XX KB" 的形式；
- imagename: 代表分区默认烧写镜像文件，如 customer.img；
- dynamic_partition_group: 动态分区所属于的组；
- minium_system: 暂未使用，预留；
- operation_type: 暂未使用，预留；
- Flag: 属性值，具体属性如下：

upgradable	erasable	fastbootable	copyupgradable	emmc_hw_wp	emmc_sw_wp	mountable	recoverable	dualpartition	downloadable
Y	Y	Y	Y	N	N	Y	N	N	Y

图 32-25 添加 customer 分区 flag

属性值各列表含义如下：

- upgradable: 默认置 Y;
- erasable: 代表分区是否可被擦除;
- fastbootable: 当前未使用，默认置 Y;
- copyupgradable: 代表分区是否可以通过拷贝升级方式升级，需要 upgradable 和 copyupgradable 同时为 Y，才可以生效;
- emmc_hw_wp: 代表分区是否有硬件写保护;
- emmc_sw_wp: 代表分区是否有软件写保护;
- recoveryable: 代表分区是否要打包到 OTA 包，需要 upgradable 和 copyupgradable 同时为 Y，才可以生效;
- dualpartition: 该字段已被废弃，未来可能会被移除;

cockpit 系统 ivi 域/cluster 域/adas 域添加分区同以方式，直接添加在分区表即可。添加分区时需要注意分区位置，需要在 userdata 分区之前。

32.4.3.2.2 Image 编译与打包

Image 的新增按照如下提供的方式进行：

客户先需要确认在/build/projects/\$(PROJECT_NAME)/\$(TARGET_DEVICE)/project_config.json 中 ATC_PARTITION_PLUGIN_SUPPORT 被正确配置为 yes。其次，需要在 source/android/device/autochips/vendor/\$(TARGET_DEVICE)目录下新建 partition 文件夹，存在则不需要再新建，partition 目录下的文件结构如下(以\$(TARGET_DEVICE)为 ac8025_demo 为例)：

source\android\device\autochips\vendor\ac8025_demo\partition				
Name ^	Date modified	Type	Size	
 customer	2023/8/8 17:10	Makefile	2 KB	
 image	2023/8/7 14:22	Makefile	1 KB	

图 32-26 Customer 目录内容

partition 目录包含 image.mk、和 customer.mk 文件，image.mk 的内容如下：


```
# include the mk to create image
ifndef BOARD_CUSTOMERIMAGE_FILE_SYSTEM_TYPE
include $(TOP)/device/autochips/vendor/ac8025_demo/partition/customer.mk
endif
```

图 32-27 image.mk 目录内容

image.mk 目的是 include 新增的 customer 分区 mk 文件参与编译，customer.mk 是包含编译 customer image 的相关内容，具体内容如下所示 (customer.mk 可参考 source/android/device/autochips/common/image/partition/teedata.mk 做修改):

```
PROP_DICTONARY_IMAGES += customer

TARGET_OUT_CUSTOMER := $(PRODUCT_OUT)/customer

INTERNAL_CUSTOMER_FILES := \
$(filter $(TARGET_OUT_CUSTOMER)/%, $(ALL_DEFAULT_INSTALLED_MODULES))

# generate the intermedia file path for partition config file
customerimage_intermediates := \
$(call intermediates-dir-for, PACKAGING, customer)
BUILT_CUSTOMER_IMAGE_TARGET := $(PRODUCT_OUT)/customer.img

# We just build this directly to the install location.
BUILT_CUSTOMER_IMAGE_TARGET := $(BUILT_CUSTOMER_IMAGE_TARGET)

FSTAB_FILE := $(TOP)/device/autochips/vendor/$(CUSTOMER_TARGET_DEVICE)/partition/pt_partition.fstab
# it's ok to add some logic before call atc-generic-create-image
define build-customerimage-target
mkdir -p $(TARGET_OUT_CUSTOMER)
mkdir -p $(TARGET_ROOT_OUT)/customer
$(hide) if [ -d $(FSTAB_FILE) ]; then \
cp -rf $(FSTAB_FILE) $(TARGET_ROOT_OUT)/vendor/etc; \
fi
$(call atc-generic-create-image, customer, $(TARGET_OUT_CUSTOMER), $(BUILT_CUSTOMER_IMAGE_TARGET), $(customerimage_intermediates))
endef

$(BUILT_CUSTOMER_IMAGE_TARGET): $(INTERNAL_USERIMAGES_DEPS) $(INTERNAL_CUSTOMER_FILES)
$(build-customerimage-target)

.PHONY: customerimage-nodexps
customerimage-nodexps: | $(INTERNAL_USERIMAGES_DEPS)
$(build-customerimage-target)

.PHONY: customerimage
customerimage: $(BUILT_CUSTOMER_IMAGE_TARGET)

ATC_INSTALLED_IMAGES += customerimage
```

图 32-28 customer.mk 内容

分区的挂载时需要在 fstab 添加挂载的信息，以确保系统启动之后可以正常的挂载 customer 分区。

fstab 的路径: source/android/vendor/autochips/proprietary/hardware/fstab/ac8x/fstab.in.ac8x, 具体的添加方式如下:

```
#ifdef ATC_FDE_TYPE_FILE
#define FSWGR_FLAG_DATA wait,check,quota,reservedsize=128M,formatable,resize,FLAG_FDE_TYPE
#else
#define FSWGR_FLAG_DATA wait,check,quota,reservedsize=128M,formatable,resize
#endif

#define FS_FLAG_DISCARD noatime,nosuid,nodex,nowait,da_alloc,discard
#define FS_FLAG_NO_DISCARD noatime,nosuid,nodex,nowait,da_alloc
#define FSWGR_FLAG_FMT wait,check,formatable

#ifdef USERDATA_USE_F2FS
#define FS_FLAG_DATA FS_FLAG_NO_DISCARD,errors=remount-ro
#else
#define FS_FLAG_DATA noatime,nosuid,nodex,discard,noflush_merge
#endif

DEVPATH(cache) /cache ext4 FS_FLAG_DISCARD FSWGR_FLAG_FMT

#ifdef USERDATA_USE_F2FS
DEVPATH(userdata) /data __ATC_DATAIMG_FSTYPE FS_FLAG_DATA,autoformat,autorestore=/vendor/restore_dir/data FSWGR_FLAG_DATA
#else
DEVPATH(userdata) /data __ATC_DATAIMG_FSTYPE FS_FLAG_DATA FSWGR_FLAG_DATA,autoformat
#endif

DEVPATH(teedata) /teedata ext4 FS_FLAG_DATA wait,first_stage_mount
DEVPATH(metadata) /metadata ext4 FS_FLAG_DATA FSWGR_FLAG_FMT
DEVPATH(customer) /customer ext4 FS_FLAG_DATA wait,check

/devices/platform/soc/ext-gdcard* auto auto defaults voldmanaged=sdcard1:auto,encryptable=userdata
/devices/platform/soc/13030000.usb* auto auto defaults voldmanaged=usb0tg:auto,encryptable=userdata
:~
```

图 32-29 fstab 添加 customer 分区挂载

编译生成的 customer.img 需要导出到 release_image/\$(PROJECT_NAME)下，才能确保 customer.img 被正确烧写。需要在 ivi/allmake_split.sh 添加 customer.img，如下所示：

```
diff --git a/allmake_split.sh b/allmake_split.sh
index 2ba0199..69182a9 100755
--- a/allmake_split.sh
+++ b/allmake_split.sh
@@ -478,6 +478,7 @@ _release() {
    ndata.img \
    resdata.img \
    update.img \
+   customer.img \
    scatter.mmcboot.ext4.xml"

    rm -rf $DIR
```

图 32-30 allmake_split.sh 添加 customer.img

cockpit 的分区挂载 adas 域/cluster 域系统需要按照以下方式修改：

■ adas 域

修改 source/vendor/autochips/proprietary/buildroot-configs/board/autochips/ac8025-adas/fstab 路径下的 fstab 文件，如下图所示：

```
# <file system> <mount pt> <type> <options> <dump> <pass>
/dev/root / ext2 ro,noauto 0 1
proc /proc proc defaults 0 0
devpts /dev/pts devpts defaults,gid=5,mode=620,ptmxmode=0666 0 0
tmpfs /dev/shm tmpfs mode=0777 0 0
tmpfs /tmp tmpfs mode=1777 0 0
tmpfs /run tmpfs mode=0755,nosuid,nodev 0 0
sysfs /sys sysfs defaults 0 0
debugfs /sys/kernel/debug debugfs defaults 0 0
/dev/adas-data /data ext4 defaults 0 0
customer /customer ext4 defaults 0 0
```

图 32-31 8025 adas fstab 内容

■ cluster 域

修改 source/vendor/autochips/proprietary/buildroot-configs/board/autochips/ac8025-cluster/fstab 路径下的 fstab 文件，如下图所示：

```
# <file system> <mount pt> <type> <options> <dump> <pass>
/dev/root / ext2 ro,noauto 0 1
proc /proc proc defaults 0 0
devpts /dev/pts devpts defaults,gid=5,mode=620,ptmxmode=0666 0 0
tmpfs /dev/shm tmpfs mode=0777 0 0
tmpfs /tmp tmpfs mode=1777 0 0
tmpfs /run tmpfs mode=0755,nosuid,nodev 0 0
sysfs /sys sysfs defaults 0 0
debugfs /sys/kernel/debug debugfs defaults 0 0
/dev/cluster-data /data ext4 defaults 0 0
customer /customer ext4 defaults 0 0
```

图 32-32 8025 cluster fstab 内容

通过上述操作配置步骤，整体编译即可完成分区 customer image 的生成与设备节点的挂载。



注意：

新增的分区需要挂载在第二阶段挂载。

32.4.3.2.3 Selinux 权限的添加

SELinux 是一个安全体系结构，它通过 LSM(Linux Security Modules)框架被集成到 Linux Kernel 2.6.x 中。提供了一种灵活的强制访问控制(MAC)系统，且内嵌于 Linux Kernel 中。SELinux 定义了系统中每个用户、进程、应用和文件的访问和转变的权限，然后它使用一个安全策略来控制这些实体(用户、进程、应用和文件)之间的交互，安全策略指定如何严格或宽松地进行检查。Android L 以及之后的版本普遍性开启 SELinux Enforce mode。所以添加完了分区，必须要配置 SELinux 相关权限设置，否则将无法添加分区。

添加 selinux 权限需要修改 source/android/device/autochips/sepolicy 目录下 te 与 file_contexts 文件，具体添加方法如下所示：

```

index 7e4f568..8a656d1 100644
--- a/non_plat/device.te
+++ b/non_plat/device.te
@@ -179,6 +179,7 @@ type misc2_block_device, dev_type;
 type logo_block_device, dev_type;
 type para_block_device, dev_type;
 type tee_block_device, dev_type;
+type customer_block_device, dev_type;
 type seccfg_block_device, dev_type;
 type secro_block_device, dev_type;
 type preloader_block_device, dev_type;
diff --git a/non_plat/file.te b/non_plat/file.te
index 50ebb4e..4ebaf6d 100644
--- a/non_plat/file.te
+++ b/non_plat/file.te
@@ -50,7 +50,7 @@ type nvdata_file, file_type, data_file_type;
 type nvcfg_file, file_type, data_file_type;
 type atolog_file, file_type, data_file_type, core_data_file_type;
 type car_file, file_type, data_file_type, core_data_file_type;
-
+type customer_file, file_type;

# debugfs file
type debugfs_blockio, fs_type, debugfs_type;
diff --git a/non_plat/file_contexts b/non_plat/file_contexts
index 62b8ced..eb6c98a 100644
--- a/non_plat/file_contexts
+++ b/non_plat/file_contexts
@@ -411,6 +411,10 @@
/dev/block/teedata          u:object_r:tee_block_device:s0
/teedata(/.*)?             u:object_r:tee_data_file:s0

+# add for teedata partition
+/dev/block/customer        u:object_r:customer_block_device:s0
+/customer(/.*)?           u:object_r:customer_file:s0
+
# Fast DMS service
/(system/vendor|vendor)/bin/hw/vendor/.autochips/.hardware/.fastdms@1.0-service u:object_r:hal_fastdms_daemon_exec:s0

diff --git a/non_plat/init.te b/non_plat/init.te
index 3cf84da..d29f806 100644
--- a/non_plat/init.te
+++ b/non_plat/init.te
@@ -37,10 +37,15 @@ allow userdata_block_device labeledfs:filesystem { associate };
# Date WK21.39, add for teedata
allow init tee_data_file:dir mounton;

+allow init customer_file:dir mounton;
+
allow init debugfs_wakeup_sources:file relabelfrom;

allow init tee_block_device:blk_file relabelto;

+allow init customer_block_device:lnk_file { relabelto };
+allow init customer_block_device:blk_file relabelto;

```

图 32-33 selinux 添加方法内容

32.4.3.2.4 修改挂载点权限

修改挂载点权限需要修改 config.fs 文件(source/android/device/autochips/common/ac8x/config.fs), 修改的方式如下:

```
[vendor/etc/ppp/ip-up]
mode: 0777
user: system
group: system
caps: BLOCK_SUSPEND

[vendor/etc/ppp/ip-down]
mode: 0777
user: system
group: system
caps: BLOCK_SUSPEND

[teedata/]
mode: 0771
user: root
group: system
caps: 0

[teedata/*]
mode: 0771
user: root
group: system
caps: 0

[avm/]
mode: 0771
user: root
group: system
caps: 0

[customer/*]
mode: 0771
user: root
group: system
caps: 0
```

图 32-34 config.fs 文件添加 customer 挂载点权限

32.4.3.2.5 检查修改是否成功

- 执行 adb mount 查看是否有 mount 成功

```
/dev/block/customer /customer ext4 rw,seclabel,nosuid,nodev,noatime,discard,noauto_da_allo
c,data=ordered 0 0
```

- ls -Z/ 查看 customer 目录权限是否设置成功

```
drwxrwx--x system system          u:object_r:customer_file:s0 customer
```

32.4.3.3 添加 raw 分区

Raw 格式的分区添加方法类似于带文件系统的分区。首先需要修改分区表，但是因为 raw 分区不需要进行挂载，则不需要修改 fstab，也不需要创建挂载点。接下来是打包烧写文件，将 raw 分区的文件打包到镜像中，最后还需要对 raw 分区添加 selinux 的相关权限。

32.4.4 FBE

32.4.4.1 FBE 介绍

FBE 全称为 File-Based Encryption，即文件级加密，在 kernel 4.14 版本前，FBE 基于 fscrypt 特性实现，在 kernel 4.14 版本后，FBE 可基于 inlinecrypto 实现。由于 FBE 是使用时进行加密，因此首次启动加密耗时极低，此外 FBE 基于目录进行加密配置，更加灵活。

FBE 包含凭证加密(Certificant Encryption) 以及设备加密(Device Encryption)。

32.4.4.2 如何打开 FBE 功能

在 AC8025 S 上，可以打开 build/project/<product_type>/<project_name>/project_config.json 文件，并将 ATC_FBE_ENABLE 配置为 yes 以打开 FBE 功能。

xref: /build/projects/ivi_s_ac8025_tmec/ac8025_hsa/project_config.json (revision c64d247f)

[Home](#) | [History](#) | [Annotate](#) | [Line#](#) | [Scopes#](#) | [Navigate#](#) | [Raw](#) | [Download](#)

```

1 {
2     "annotate": "follow is overall config for all sub system.",
3     "system_config": {
4         "Common_Options" : {
5             "annotate": "support ATCLogger feature for ivi subsystem.",
6             "HAVE_ATCLOG_FEATURE" : "yes",
7             "annotate": "support AEE feature for ivi subsystem.",
8             "HAVE_AEE_FEATURE" : "yes",
9             "ATC_OPTEE_SUPPORT" : "yes",
10            "ATC_TEE_DATA_PARTITION_SUPPORT" : "yes",
11            "SECURE_BOOT_ENABLE" : "yes",
12            "ATC_EMMC_SUPPORT" : "yes",
13            "ATC_EMMC_HW_WP_SUPPORT" : "yes",
14            "BOARD_AVE_ENABLE" : "true",
15            "annotate": "support dump crash info in lk while system reboot from KE.",
16            "KEDUMP_MINI" : "yes",
17            "DEVICE_TYPE": "ac8025",
18            "PROJECT_TYPE": "ivi",
19            "TARGET_BOARD_PLATFORM" : "ac8x",
20            "TARGET_DEVICE": "ac8025_hsa",
21            "CUSTOMER_TARGET_DEVICE": "ac8025_hsa",
22            "HIBES": "true",
23            "ATC_FBE_ENABLE" : "yes"
24        },

```

图 32-35 project_config.json 中 FBE 的配置项

将打开 FBE 功能的 image 烧写入平台并启动后，输入 adb shell getprop ro.crypto.state 后可以发现加密状态为 encrypted，并且输入 adb shell getprop ro.crypto.type 后可以查看到加密类型为 file:

```

[ro.crypto.state]: [encrypted]
[ro.crypto.type]: [file]

```

图 32-36 FBE 打开后的属性

32.4.4.3 如何为定制分区打开 FBE 功能

客户定制分区可以支持使能 FBE 功能, 但对分区有如下要求:

1. 分区必须可读可写
2. 分区中待加密目录必须为空目录
3. 在密钥安装以及待加密目录被配置为加密前, 不能在待加密目录中创建文件
4. Factory Reset 时, 该分区必须被格式化, 否则该分区中已加密目录无法访问

以 customer 分区为例, 使能定制化分区 FBE 功能的流程如下:

i. 为 customer 分区打开 FBE 功能

- a) 打开/source/android/vendor/autochips/proprietary/hardware/fstab/ac8x/fstab.in.ac8x, 在 fs_mgr flag 字段中添加 fileencryption=aes-256-xts

```
...
DEVPATH(metadata) /metadata ext4 FS_FLAG_DISCARD FSMGR_FLAG_FMT
DEVPATH(customer) /customer ext4 FS_FLAG_DISCARD FSMGR_FLAG_FMT,fileencryption=aes-256-xts
...
```

ii. 为 customer 分区安装密钥

- a) 打开/source/android/device/autochips/vendor/ac8x/common/init.rc 文件, 在 mount_all 完成后添加如下密钥安装命令

```
on fs
    write /proc/bootprof "INIT:Mount_START"
    # Sync with device.mk config
    mount_all /vendor/etc/fstab.ac8x
    write /proc/bootprof "INIT:Mount_END"
    write /proc/mmc_debug "emmc_enable_all_wp"
    install_fs_key /customer
```

- b). 密钥安装后, 可以在上述 init.rc 中使用如下命令将目录配置为加密

```
mkdir /customer/test 0777 shell shell encryption=Attempt
```

encryption 后可以配置如下行为:

表 32-4 mkdir encryption 行为

行为	描述
Require	目录配置加密失败, 则进入 recovery 模式提示用户进行恢复出厂设置

行为	描述
Attempt	忽略目录加密配置失败，继续启动流程
DeleteIfNecessary	若待加密目录非空，则尝试删除待加密目录并重新尝试加密
None	默认行为，不将该目录配置为加密

完成配置，烧写并系统启动后，密钥将会被安装到如下路径：

```
/ <mount_point> /unencrypted
```

```
ac8025_hsa:/extdata # ls unencrypted/ -l
total 12
drwx----- 2 root root 4096 2024-01-05 07:36 key
-rw----- 1 root root 26 2024-01-05 07:36 mode
-rw----- 1 root root 16 2024-01-05 07:36 ref
```

图 32-37 密钥安装路径

32.4.4.4 如何验证 FBE 功能已经打开

【方法 1】 利用 debugfs 工具查看目录是否配置为加密

注意，该方法仅适用于 ext4 文件系统。

Autochips 默认不将 debugfs 工具打包到镜像中，因此需要使用 ninja 编译 debugfs_static 目标，并将 debugfs_static 工具 push 到平台，并为其增加可执行权限。

执行如下命令进入 debugfs 命令行：

```
./debugfs_static /dev/block/by-name/<partition>
```

在 debugfs 命令行中输入如下命令：

```
debugfs: stat <待查看目录>
```

可以在 Extended Attributes 中找到 c (40)，即已经为该目录配置了 FBE。


```
debugfs: stat test
Inode: 53985  Type: directory  Mode: 0777  Flags: 0x80800
Generation: 3295032473  Version: 0x00000000:00000002
User: 2000  Group: 2000  Project: 0  Size: 4096
File ACL: 262629
Links: 2  Blockcount: 16
Fragment: Address: 0  Number: 0  Size: 0
  ctime: 0x6597b4a4:dc4c7404 -- Fri Jan 5 07:49:56 2024
  atime: 0x6597b16d:8c30abac -- Fri Jan 5 07:36:13 2024
  mtime: 0x6597b4a4:dc4c7404 -- Fri Jan 5 07:49:56 2024
  crtime: 0x6597b16d:8c30abac -- Fri Jan 5 07:36:13 2024
Size of extra inode fields: 32
Extended attributes:
  security.selinux (27) = "u:object_r:extdata_file:s0\000"
  c (40)
EXTENTS:
(0):262628
debugfs: █
```

图 32-38 debugfs 查看目录是否被配置了 FBE

在 debugfs 命令行中输入如下命令：

```
debugfs: ls -l <待加密目录>
```

可以发现相关目录下的文件均为 encrypted：

```
debugfs: ls -l test
53985 40777 (2) 2000 2000 4096 5-Jan-2024 07:49 .
2 40771 (2) 0 1000 4096 5-Jan-2024 07:36 ..
53986 100666 (1) 0 0 0 5-Jan-2024 07:49 <encrypted (16)>
debugfs: █
```

图 32-39 使用 debugfs 根据查看目录是否加密

【方法 2】删除密钥并重启系统

此方法要求配置目录加密时，encryption 模式需要配置为 Attempt。

方法如下：

- 1. 删除<mount_point>/unencrypted/ 目录；
- 2. 重启启动，并查看待加密目录，会发现该目录下的文件为一串 base64 编码。

```
1|ac8025_hsa:/extdata/test # ls -lh
total 4.0K
-rw-rw-rw- 1 root root 0 2024-01-05 07:49 2f9hIVQns9e1LLo6XIeZITH565SjaSWR
ac8025_hsa:/extdata/test # █
```

图 32-40 查看加密目录

32.5 用户定制化

32.5.1 定制化低存储保护机制

32.5.1.1 实现

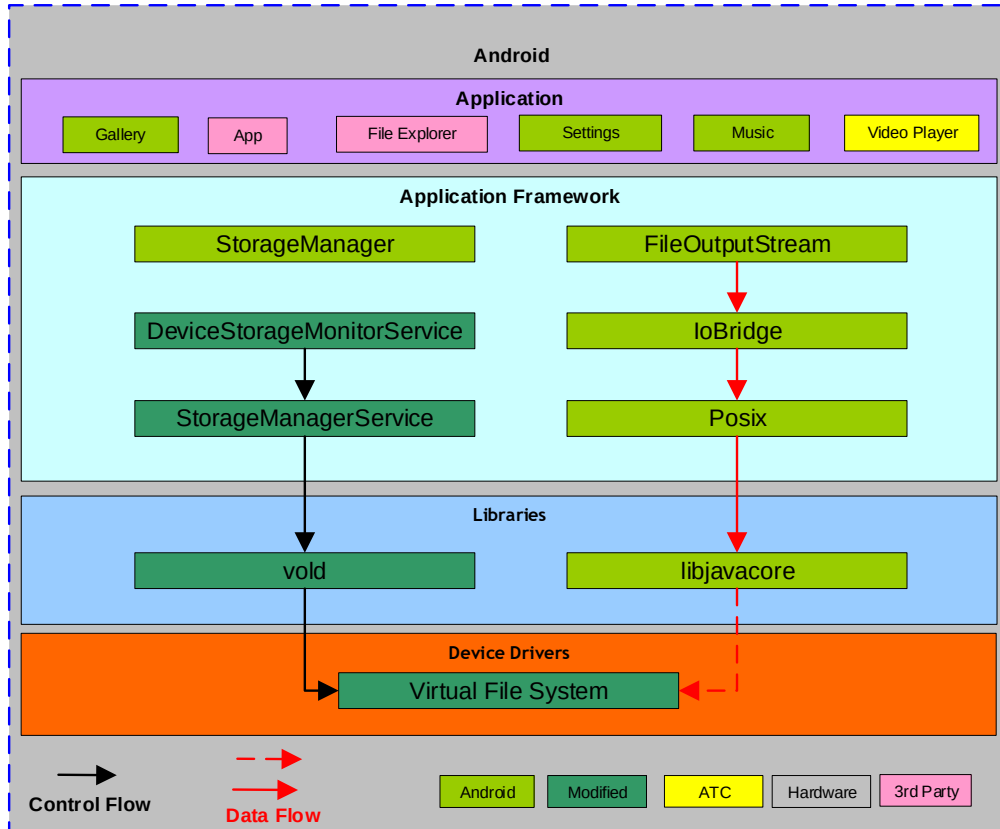


图 32-41 低存储保护机制

图中黑色箭头串联的 flow 是本 feature 基于原有 DSMS 基础上增加的策略。利用 DSMS 实时对需要保护的分区进行 check，当剩余存储空间低于某个配置的阈值（storage space full threshold）时，通过 StorageManagerService、VOLD、VFS 把文件系统状态置为存储空间已满状态。

红色箭头串联的 flow 是 AC8025 Android S 上文件系统写操作流程。其中红色虚线箭头部分为本 feature 在原有写操作实现基础上作出的修改，VFS 中写操作之前增加存储空间是否已满的 check，当存储空间已满时，根据配置屏蔽非系统进程在该分区上的写操作，并且返回存储空间不足状态信息。

如此，在 VFS 中可以对文件所属文件系统进行统一管理，当剩余存储空间较低时置文件系统于存储空间已满状态，写操作执行前 check 文件系统状态，屏蔽非系统进程对存储空间不足的文件系统中文件的写操作，从而保护分区。



注意:

本 feature 尽量避免分区被写满，但是由于 DSMS 对分区的 check 并不是时时进行，存在时间间隔（源生 DSMS storage check 时间间隔为 30 秒），两次 storage check 间隔期间把存储空间写满的情形无法兼顾。

32.5.1.2 状态转换

Low, critical low 和 full 三种状态监测时状态转换示意图如下：

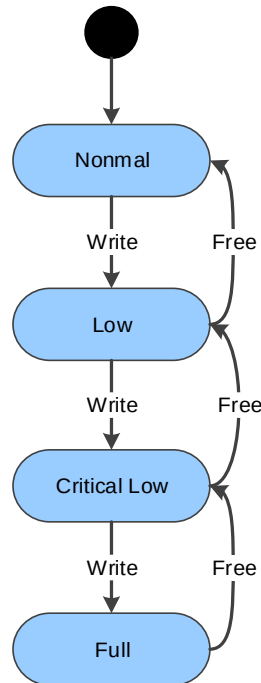


图 32-42 状态转换流程图

- Write，向受保护分区进行写操作，该步操作会使受保护分区可用存储空间减少；
- Free，对受保护分区进行删除操作，可以释放部分存储空间。

32.5.1.3 使用方式

32.5.1.3.1 查看当前 Low Storage 状态

在 adb 下(非 root)可以通过 cmd devicestorage monitor get-status 命令查看当前 low storage 状态：

```
ac8x_demo:/ # cmd devicestoragemonitor get-status
NORMAL
```

图 32-43 Low Storage Get Status

32.5.1.3.2 修改 Low Storage 阈值

1. 该 feature 可以通过 pull 出平台中的 /vendor/etc/MonitoredPathes.json，查看 MonitoredPartitions 的 internal 成员，如下图所示：

```
assi_64:/ # cat vendor/etc/MonitoredPathes.json
{
  "overall": {
    "uncovered_apps": [
      "com.android.documentsui",
      "com.android.settings",
      "com.autochips.backcarapp"
    ],
    "interval": 30
  },
  "monitoredPartitions": {
    "internal": {
      "enable": "yes",
      "storage_low": "600",
      "critical_low": "400",
      "storage_full": "200"
    },
    "adoptable": {
      "enable": "no",
      "storage_low": "600",
      "critical_low": "400",
      "storage_full": "200"
    }
  }
}
```

图 32-44 MonitoredPathes.json 文件内容

其中：

- overall 表示 Low Storage 配置的相关信息
 - uncovered_apps 代表这些应用不能进行 Low Storage 弹窗信息提醒
 - interval 表示 Low Storage 监测的时间间隔
- MonitoredPartition 表示监测的分区
 - Internal 表示内置分区
 - enable 表示是否监控内置分区
 - storage_low 对应触发内置分区 storage_low 状态的阈值，单位为 MB，不支持小数，注意不要漏掉引号
 - critical_low 对应触发内置分区 critical_low 状态的阈值，单位为 MB，不支持小数，注意不要漏掉引号
 - storage_full 对应触发内置分区 storage_full 状态的阈值，单位为 MB，不支持小数，注意不要漏掉引号

- adoptable 表示 adoptable storage 分区
 - enable 表示是否监控 adoptable storage 分区
 - storage_low 对应触发 adoptable storage 分区 storage_low 状态的阈值，单位为 MB，不支持小数，注意不要漏掉引号
 - critical_low 对应触发 adoptable storage 分区 critical_low 状态的阈值，单位为 MB，不支持小数，注意不要漏掉引号
 - storage_full 对应触发 adoptable storage 分区 storage_full 状态的阈值，单位为 MB，不支持小数，注意不要漏掉引号
- 2. 调试以及测试的过程可以方便 pull MonitoredPathes.json 出来修改，实际的产品无法通过 pull 的方式来修改，则需要修改 MonitoredPathes.json 文件。该文件位于以下路径：

source/android/device/autochips/vendor/ac8x/common/MonitoredPathes.json

通过直接修改 internal 或 adoptable key 中 storage_low、critical_low、storage_full 对应的值来设置其阈值。

32.5.2 统计文件写入频率与写入量的方法

针对客户需要统计写入类似 EMMC 等存储设备的写入频率和写入量，设计了以下统计工具。工具使用方式如下：

```
xref: /source/kernel/common/fs/Kconfig (revision e9ab3474)
```

[Home](#) | [History](#) | [Annotate](#) | [Line#](#) | [Scopes#](#) | [Navigate#](#) | [Raw](#) | [Download](#)

☐ current directory

```

232 ramfs. for architectures that support it, say Y here and read
233 <file:Documentation/admin-guide/mm/hugetlbpage.rst> for details.
234
235 If unsure, say N.
236
237 config HUGETLB_PAGE
238     def_bool HUGETLBFS
239
240 config AUTO_RECOVERY
241     bool "Autochips Auto Recovery Feature"
242     default y
243     depends on ATC_AOSP_ENHANCEMENT
244
245 config LOW_STORAGE_PROTECTION
246     bool "Autochips Low Storage Protection Feature"
247     default y
248     depends on ATC_AOSP_ENHANCEMENT
249
250 config WRITE_COUNT
251     bool "Autochips Write Statistics"
252     default n
253     depends on ATC_AOSP_ENHANCEMENT
254
255 config MEMFD_CREATE
256     def_bool TMPFS || HUGETLBFS
257
258 config ARCH_HAS_GIGANTIC_PAGE
259     bool
260
261 source "fs/configfs/Kconfig"
262 source "fs/efivarfs/Kconfig"
263
264 endmenu
    
```

图 32-45 打开写入频率与写入量统计工具

修改以上 source/kernel/common/fs 目录下的 kconfig 文件 WRITE_COUNT 配置选项，将 default 修改为 Y 即可默认打开。此时写入的文件可以在 vfs 层统计，统计的结果使用以下指令导出：

adb shell cat /proc/write_count > your_path/write_count.csv

write_count.csv 文件最终的统计结果展示如下：

	A	B	C	D	E	F	G	H	I	J	K	L
	File Name	Process Name	PID	Thread Name	Write Size(Bytes)	Write Times	File Sync Times	Write Min Freq(n)	Write Max Freq	Average Write Freq(ms)		
1	/data/system/packages.list.tmp	system_server	510	system_server	24300	4	4	8765	8765	5843		
2	/data/system/package_cache/52abfb2b10e208e	package-parsing	585	package-parsing	4112	1	1	0	0	0		
3	/data/system/package_cache/52abfb2b10e208e	package-parsing	585	package-parsing	2056	1	1	0	0	0		
4	/data/system/package_cache/52abfb2b10e208e	package-parsing	585	package-parsing	2980	1	1	0	0	0		
5	/data/system/recoverablekeystore.db-shm	system_server	510	system_server	8	8	8	0	0	0		
6	/data/system/package_cache/52abfb2b10e208e	package-parsing	585	package-parsing	1988	1	1	0	0	0		
7	/data/system/package_cache/52abfb2b10e208e	package-parsing	585	package-parsing	4148	1	1	0	0	0		
8	/data/system/package_cache/52abfb2b10e208e	package-parsing	584	package-parsing	9680	1	1	0	0	0		
9	/data/system/package_cache/52abfb2b10e208e	package-parsing	585	package-parsing	1860	1	1	0	0	0		
10	/data/resource-cache/data@resource-cache@Binder.652.4		696	Binder.652.4	452	1	1	0	0	0		
11	/data/misc_de/0/apexdata/com.android.permissioncontrol		558	android.bg	535980	68	68	1	8771	131		
12	/data/system/package_cache/52abfb2b10e208e	package-parsing	585	package-parsing	8776	1	1	0	0	0		
13	/data/misc/aplog/aplog_history	ap_log_d	654	ap_log_d	295	6	6	2	132	26		
14	/data/system/package_cache/52abfb2b10e208e	package-parsing	585	package-parsing	1872	1	1	0	0	0		
15	/dev/bkio/background/bkio.bfg.weight	init	1	init	2	1	1	0	0	0		
16	/data/system/package_cache/52abfb2b10e208e	package-parsing	583	package-parsing	1960	1	1	0	0	0		
17	/data/misc/bluetooth/bt.conf	bt_stack_manage	1042	bt_stack_manage	150	1	1	0	0	0		
18	/data/system/battery-history/0.bin	new	558	android.bg	16	1	1	0	0	0		
19	/data/system/package_cache/52abfb2b10e208e	package-parsing	584	package-parsing	78920	1	1	0	0	0		
20	/data/user_de/0/com.android.systemui/shared_queueued-work-100		1242	queued-work-100	112	1	1	0	0	0		
21	/data/system/package_cache/52abfb2b10e208e	package-parsing	584	package-parsing	147128	1	1	0	0	0		
22	/data/property/persistent_properties.tmp	init	201	init	25792	16	16	540	12692	882		
23	/data/system/users/0/wallpaper_info.xml.tmp	Binder.510.3	616	Binder.510.3	14732	2	2	270	270	270		
24	/data/user_de/0/com.android.permissioncontrol	ssioncontroller	1374	ssioncontroller	257	3	3	2	4	3		

图 32-46 打开写入频率与写入量统计工具

用户可通过上图查看写入的文件名、文件进程和写入 size 等信息。图中各列依次代表的是写入文件的文件名、写入文件的进程名、写入文件的进程号、写入文件的线程名，写入文件的大小、写入文件的 sync 次数、写入文件的最大间隔和写入文件的最小间隔。

32.6 典型问题

32.6.1 拔出外置设备，导致使用外置设备内文件的进程被 kill 掉

【问题描述】

拔出外置设备，导致使用外置设备内文件的进程被 kill 掉。

【解决方案】

当自定义开发的进程需要用到外置存储设备路径时，需要注意接收 android.intent.action.MEDIA_EJECT 广播。在接收到该广播时，占用到 eject 外置设备路径的进程需要停止使用该外置路径的文件，否则可能会因为占用挂载点导致挂载点卸载失败，挂载点卸载失败时会发送信号给占用进程，对应进程会被 kill。如果进程不停占用挂载点，导致最终挂载点卸载失败，后续重新插入外置设备时可能存在无法挂载成功的问题。

33 Metazone

33.1 简介

33.1.1 软硬件环境

本模块需要的硬件环境，包括：

- eMMC/UFS
- DRAM

33.1.2 功能

Metazone 是一个提供数据读写的驱动模块，它主要的功能包括：

- 从存储设备读写 Dword 区域数据
- 从存储设备读写 Binary 区域数据
- 从存储设备读写 Reserved 区域数据

33.1.3 代码结构及文件描述

表 33-1 metazone 代码结构和描述

文件名	文件路径	文件描述
AtcMetazone.java	source/android/vendor/autochips/proprietary/ frameworks/base/metazone /java/com/autochips/metazone/	提供 framework 层读写 metazone 接口
metazone_client.h metazone_client.cpp	source/android/vendor/autochips/proprietary/ frameworks/base/metazone/ metazonejni	提供 HIDL 用户 端接口
com_autochips_metazone_ AtcMetazone.h AtcMetazone.cpp	source/android/vendor/autochips/proprietary/ frameworks/base/metazone/metazonejni	提供 metazone 在 framework 层 的 JNI 接口
Metalogo.h Metalogo.cpp service.cpp	source/android/vendor/autochips/proprietary /hardware/metalogo/	提供 framework 层 HIDL server 端接口
IMetalogo.hal types.hal	source/android/vendor/autochips/proprietary/ hardware/interfaces/metalogo/1.0/	提供 metalogo HIDL 接口

文件名	文件路径	文件描述
metazone.c	source/android/vendor/autochips/proprietary/frameworks/native/libs/metazone/	提供 native 层 metazone 的读写接口
metazone_drv.c	source/kernel/common/drivers/soc/autochips/metazone	提供 metazone kernel driver
metazone.h	source/kernel/common/drivers/soc/autochips/include/metazone	定义 metazone 数据项 index 和 driver 功能函数
metazone_inter.h	source/kernel/common/drivers/soc/autochips/include/metazone	定义 metazone 内部接口和宏
metazone_ioctl.h	source/kernel/common/ drivers/soc/autochips/include/metazone	定义 metazone ioctl 函数
metazone_inter.c	source/vendor/autochips/proprietary/bootable/lk/platform/autochips/ac8x/	提供 lk 中 metazone 读写接口
metazone_inter.h	source/vendor/autochips/proprietary/bootable/lk/platform/autochips/ac8x/include	定义 lk 中 metazone 函数接口和宏
metazone_driver.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/misc	提供 safety os metazone 读写接口
metazone_driver.h	source/vendor/autochips/proprietary/tinysys/hsm/os/include/common	定义 safety os 中 metazone 函数接口和宏

33.2 软件设计

33.2.1 架构图

33.2.1.1 Metazone 应用框架

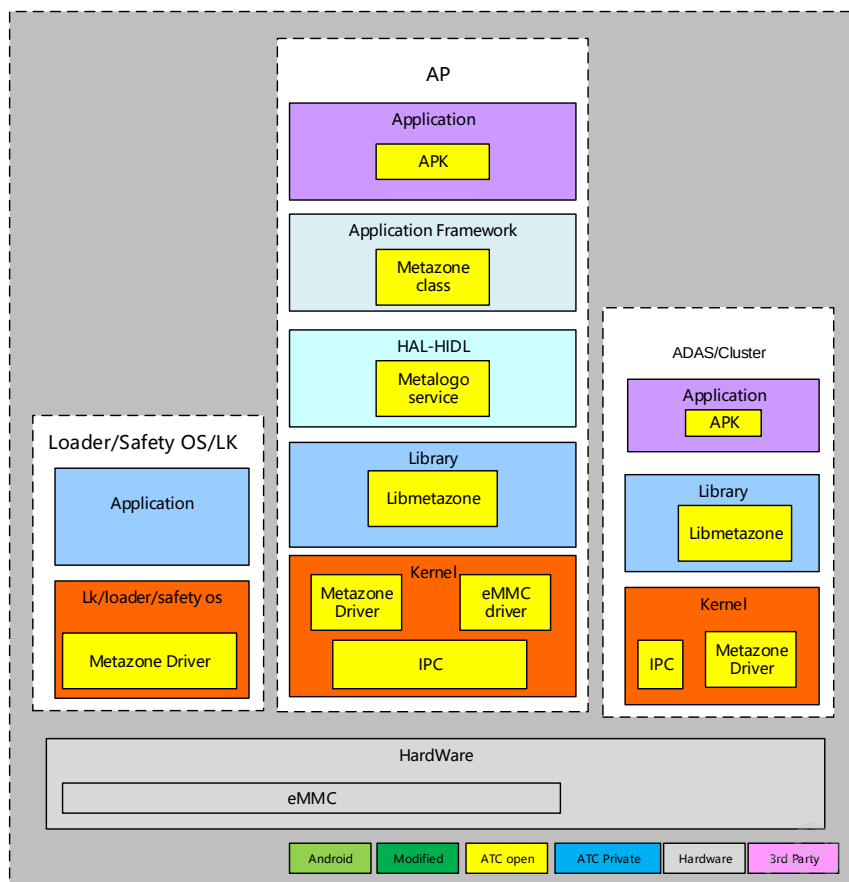


图 33-1 metazone 系统架构图

Metazone 在 Android 四层框架中展开，主要功能是在 Framework 层、Library 层、Kernel 层及 LK 中提供读/写 API，另外在 Adas/Cluster/Safety 中也提供了读 API。

33.2.1.2 Metazone 结构图

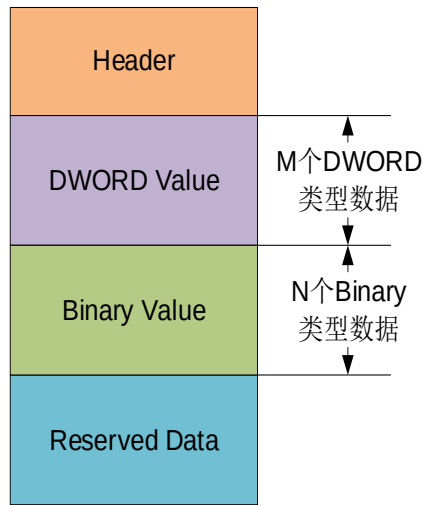


图 33-2 metazone.bin 结构图

原始数据 metazone.bin 的最大 size 默认设计为 1024KB，由 Header、DWORD Value 、 Binary Value 和 Reserved Data 四部分组成。

33.2.1.2.1 Header

如图 33-2 所示，metazone 头部是固定 Size（512 bytes）的 Header。其中关键内容包含：

- DWORD dwVersion（metazone 的版本信息）
- DWORD dwSignature（metazone 的识别标示）
- DWORD dwMetaSize（metazone 的大小）
- DWORD dwDwordNum（metazone DWORD 类型数据的个数）
- DWORD dwDwordOffset（metazone DWORD 数据区域偏移）
- DWORD dwBinaryNum（metazone Binary 类型数据的个数）
- DWORD dwBinaryOffset（metazone Binary 数据区域偏移）
- DWORD dwReservedOffset（metazone Reserved 数据区域偏移）
- DWORD dwReservedSize（metazone Reserved 数据区域 size）

33.2.1.2.2 DWORD 数据

DWORD 数据区，由 Header 给定数目的 DWORD 数据项组成。每个 DWORD 类型数据项大小为 4 bytes，目前默认最大支持 500 个 DWORD 类型 item。使用时指定 index 值来确定具体读写数据项的序号。



说明：

DWORD 区的 index 与 Binary 区的 index 分开处理，仅仅针对各自数据区，均从 0 开始计算，index 的范围与各数据区实际提供的 item 数量对应。

DWORD item 的数据结构如下：

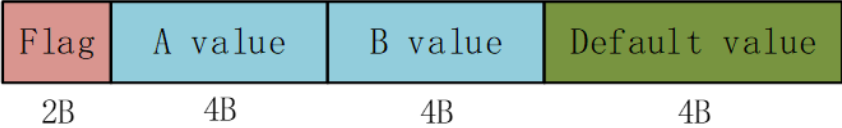


图 33-3 Dword item 数据结构

每个 Dword Item 包含了标志位 Flag（2 个 bytes）、存储数据 A Value（当系统 slot 为 A 时，metazone 使用 A_value, 占 4 个 bytes）和 B Value（当系统 slot 为 B 时，metazone 使用 B Value, 占 4 个 bytes）以及默认值 Default（4 个 bytes）。

其中标志位 Flag 定义如下：

表 33-2 Dword Flag 标志位定义

bit[0]	0: 无数据 1: 有数据
bit[1]	0: 不 factory reset 1: factory reset
bit[2]	0: 不加密 1: 加密
bit[3]	0: 不升级 1: 升级
bit[4]	0: A Value 无效 1: A Value 有效
bit[5]	0: B Value 无效 1: B Value 有效
bit[6]~ bit[15]	预留



注意:

bit[0]/bit[1]/bit[3]分别对应 33.4.1 中 metazone.csv 表格中的 Have Data/FactoryReset/Update。

33.2.1.2.3 Binay 数据

Binary 数据区，由 header 给定数目的 binary data items 组成。目前默认最大支持 200 个 Binary 类型数据，每个 Binary item 数据项的结构如下：

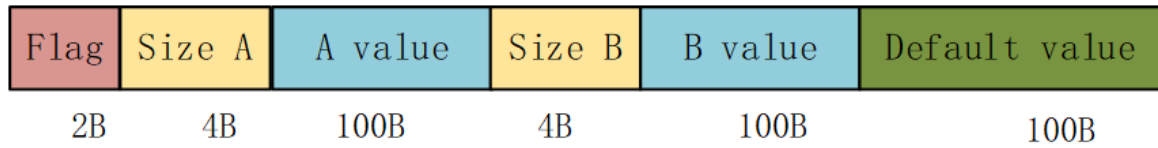


图 33-4 Binary item 数据结构

每个 Binary Item 包含了标志位 Flag（2 个 bytes）、存储 Binary 的 A value 和 B value 的实际 size（各 4 bytes），存储数据 A value（100 bytes）和 B value（100 bytes）以及默认值 Default（100 个 bytes）。

其中标志位 Flag 定义如下：

表 33-3 Binary Flag 标志位定义

bit[0]	0: 无数据 1: 有数据
bit[1]	0: 不 factory reset 1: factory reset
bit[2]	0: 不加密 1: 加密
bit[3]	0: 不升级 1: 升级
bit[4]	0: A Value 无效 1: A Value 有效
bit[5]	0: B Value 无效 1: B Value 有效
bit[6]~ bit[15]	预留



注意：
bit[0]/bit[1]/bit[3]分别对应 33.4.1 中 metazone.csv 表格中的 Have Data/FactoryReset/Update。

33.2.1.2.4 Reserved 数据

Reserved 数据为预留区域，可用于保存较大 size 的数据，可以通过 API 整体读写或从某一 offset 位置读写指定 size 数据。

Reserved Data 数据区域，由一个 Reserved Header 和 6 个 64KB 数据块组成。整个 Reserved Data 数据区域结构如下：

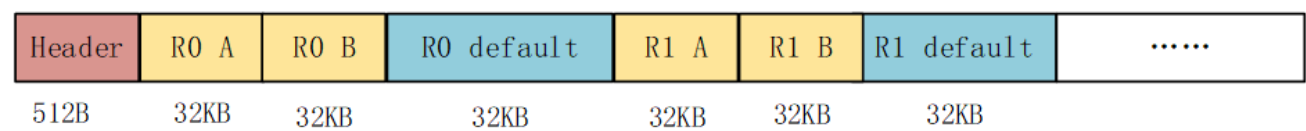


图 33-5 Reserved item 数据结构

每个 Reserved item 包含了标志位 Flag（2 个 bytes），存储数据 A value（32kb）和 B value（32kb）以及默认值 Default（100 个 bytes）。

其中标志位 Flag 定义如下：

表 33-4 Reserved Flag 标志位定义

bit[0]	0: 无数据 1: 有数据
bit[1]	0: 不 factory reset 1: factory reset
bit[2]	0: 不加密 1: 加密
bit[3]	0: 不升级 1: 升级
bit[4]	0: A Value 无效 1: A Value 有效
bit[5]	0: B Value 无效 1: B Value 有效
bit[6]~ bit[15]	预留



注意:

bit[0]/bit[1]/bit[3]分别对应 33.4.1 中 metazone.csv 表格中的 Have Data/FactoryReset/Update。

33.2.1.3 Metazone 的 RO 备份 Metazonebk

Metazone 会有 metazonebk 分区并且是 RO 类型的，该分区的作用是 metazone 数据的备份，当 metazone 分区数据被损坏时，会从 metazonebk 中获取数据进行备份，恢复 Metazone 中的数据；

虽然该分区类型的 RO 的，但是为了防止备份数据的太旧，我们会周期性（默认更新周期是 100 次重启）进行更新，此更新频率可以自定义修改。

33.2.2 流程图

33.2.2.1 Metazone read flow

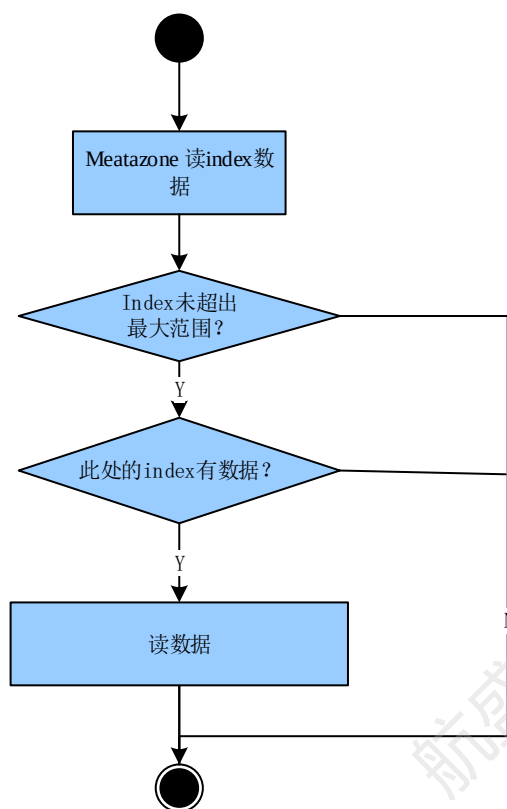


图 33-6 Dword read flow

33.2.2.2 Metazone write flow

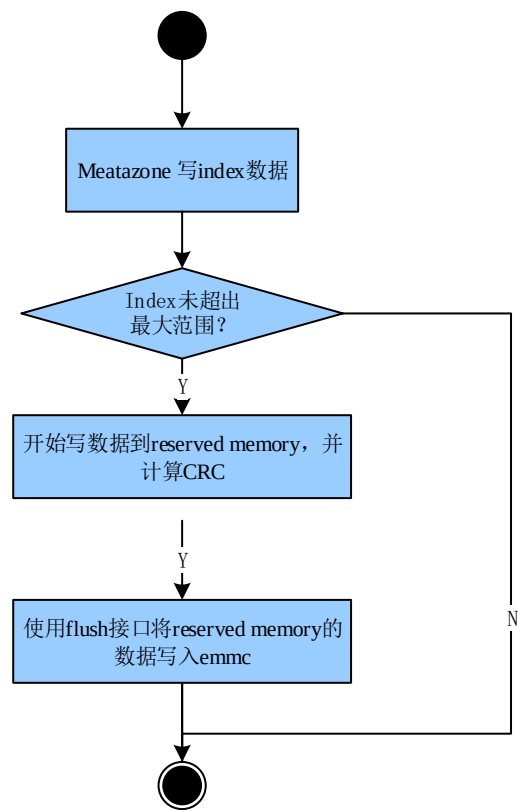


图 33-7 Dword write flow



注意：
metazone write 操作执行完之后，需要执行 metazone 的 flush 操作。

33.3 接口描述

33.3.1 AtcMetazone Class

表 33-5 AtcMetazone class

Class	说明
com.autochips.metazone.AtcMetazone	Provide interfaces to read/write Metazone data
Public方法	说明
readval	Read a DWORD item value
writeval	Write a DWORD item value
specwriteval	Special write a DWORD item value for factory

	reset.
readbinary	Read a Binary item data
writebinary	Write a Binary item data
specwritebinary	Special write a DWORD item value for factory reset.
specreadreserved	Read Reserved data for enhancement
specwritereserved	Write Reserved data for enhancement
flush	Trigger Metazone write data to storage

33.3.1.1 readval

【描述】

Read value of given DWORD item.

【语法】

```
public native static int readval (int index, int [] data);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- index: the index of DWORD item.
- data: buffer for storing read DWORD value

【注意】

DWORD item 的 index 值从 0x10000 开始（index 0—0xFFFF 保留）。实际第一个 DWORD item 使用 0x10000 来访问，以此类推，最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 DWORD Value 的数目，目前默认为 500)。

33.3.1.2 writeval

【描述】

Write value of given DWORD item.

【语法】

```
public native static int writeval (int index, int data);
```


【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- index: the index of DWORD item.
- data: the write value of DWORD item

【注意】

DWORD item 的 index 值 从 0x10000 开始 (index 0—0xFFFF 保留)。实际第一个 DWORD item 使用 0x10000 来访问, 以此类推, 最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 DWORD Value 的数目, 目前默认为 500)。

33.3.1.3 specwriteval

【描述】

Write special value to metazone of given index and flag.

【语法】

```
public native static int specwriteval (int index, int [] data, int flag);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- index: the index of DWORD item.
- data: DWORD value to be written.
- flag : write flag. flag = 0 (no operate), flag = 1 (write to default value)

33.3.1.4 readbinary

【描述】

Read the data of given Binary item.

【语法】

```
public native static int readbinary (int index, byte [] data, int size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- index: the index of Binary item.
- data: the buffer to store the Binary data
- size: buffer size in bytes

【注意】

Binary data item 的 index 从 0x10000 开始（index 0—0xFFFF 保留）。第一个 Binary data item 的 index 为 0x10000，以此类推，最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 Binary Data 的数目，目前默认为 200)。

33.3.1.5 writebinary

【描述】

Write the data of given Binary item.

【语法】

```
public native static int writebinary (int index, byte [] data, int size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- index: the index of Binary item.
- data: the buffer to store the Binary data
- size: buffer size in bytes

【注意】

Binary data item 的 index 从 0x10000 开始（index 0—0xFFFF 保留）。第一个 Binary data item 的 index 为 0x10000，以此类推，最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 Binary Data 的数目，目前默认为 200)。

33.3.1.6 specwritebinary

【描述】

Write special value to metazone of given index and flag.

【语法】

```
public native static int specwritebinary (int index, byte [] data, int size, int flag);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- index: the index of Binary item.
- data: buffer of data to be written.
- size: data size in bytes.
- flag: write flag. flag = 0 (no operate), flag = 1 (write to default value)

33.3.1.7 specreadreserved

【描述】

Read Reserved data.

【语法】

```
public native static int specreadreserved (int areaNum, byte [] data, int offset, int size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- areaNum: the number of read reserved area.

- data: buffer to store the reserved data.
- offset: the offset of read.
- size: buffer size in bytes.

33.3.1.8 specwritereserved

【描述】

Write reserved data.

【语法】

```
public native static int specwritereserved (int areaNum, byte [] data, int offset, int size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- areaNum: the number of read reserved area.
- data: buffer to store the reserved data.
- offset: the offset of read.
- size: buffer size in bytes.

【注意】

writereserved 无法改变 size of reserved data, size 由 metazone.bin Header 给出。写入的数据 size 最好与 reserved data size 一致。如果写入数据的 Size 小于 reserved data size, 则只修改前面 Size 的数据; 如果写入数据的 size 大于 reserved data size, 则多余数据被丢弃。

33.3.1.9 flush

【描述】

清除 metazone 缓存, 将修改的数据写入 metazone 分区。writeval & writebinary 和 writereserved 只是把数据写入到 metazone 的 memory, 调用 flush 后才把修改的结果写到 metazone 分区。

【语法】

```
public native static int flush(bool sync);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

Sync

- 为 TRUE(1)时，本函数在数据写入 metazone 分区完成后才返回；
- 为 FALSE(0)，本函数立即返回，metazone 由 driver 中 flush thread 写入 metazone 数据。

【注意】

调用 writeval & writebinary & specwritereserved 修改 Metazone 数据后，一定要调用 flush 把数据写入到 metazone 分区。否则系统重启后，修改结果将丢失（回到上次 flush 的状态）。

33.3.2 Native APIs



说明：

adas/cluster/ivi 的 Native 接口是复用的，以下接口的说明适用 adas/custer/ivi。

表 33-6 Native APIs

API	说明
MetaZone_Init	Initialize metazone
MetaZone_Deinit	De-initialize metaZone
MetaZone_Read	Read a DWORD item value
MetaZone_Write	Write a DWORD item value.
MetaZone_SpecWrite	Special Write a DWORD item value for factory reset.
MetaZone_ReadBinary	Read a Binary item data
MetaZone_WriteBinary	Write a Binary item data
MetaZone_SpecWriteBinary	Special write a Binary item data for factory reset
MetaZone_SpecReadReserved	Read Reserved data
MetaZone_SpecWriteReserved	Write Reserved data
MetaZone_Flush	Trigger metaZone write data to storage media.
MetaZone_AB_Upgrade	Metazone OTA upgrade

33.3.2.1 MetaZone_Init

【描述】

The first API called before use metazone, which is used to initialize metazone.

【语法】

```
u32 MetaZone_Init(void);
```

【返回值】

- 0x0: succeed
- 0x80000000: fail

【参数】

None

33.3.2.2 MetaZone_Deinit

【描述】

De-initialize metazone.

【语法】

```
u32 MetaZone_Deinit (void);
```

【返回值】

- 0x0: succeed
- 0x80000000: fail

【参数】

None

33.3.2.3 MetaZone_Read

【描述】

Read a DWORD item value.

【语法】

```
u32 MetaZone_Read(u32 u4Idx, u32 *pu4Data);
```

【返回值】

- 0x0: succeed

- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- pu4Data: for storing read DWORD value

【注意】

DWORD item 的 index 值 从 0x10000 开始 (index 0—0xFFFF 保留)。实际第一个 DWORD item 使用 0x10000 来访问, 以此类推, 最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 DWORD Value 的数目, 目前默认为 500)。

33.3.2.4 MetaZone_Write

【描述】

Write a DWORD item value.

【语法】

```
u32 MetaZone_Write(u32 u4Idx, u32 u4Data);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- u4Data: the write value of DWORD item

【注意】

DWORD item 的 index 值 从 0x10000 开始 (index 0—0xFFFF 保留)。实际第一个 DWORD item 使用 0x10000 来访问, 以此类推, 最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 DWORD Value 的数目, 目前默认为 500)。

33.3.2.5 MetaZone_SpecWrite

【描述】

Write a DWORD item value.

【语法】

```
u32 MetaZone_SpecWrite(u32 u4Idx, u32 u4Data, u32 flag);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- u4Data: the write value of DWORD item
- flag: write flag. flag = 0 (no operate), flag = 1 (write to default value)

【注意】

DWORD item 的 index 值 从 0x10000 开始（index 0—0xFFFF 保留）。实际第一个 DWORD item 使用 0x10000 来访问，以此类推，最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 DWORD Value 的数目，目前默认为 500)。

33.3.2.6 MetaZone_ReadBinary

【描述】

Read a Binary item data.

【语法】

```
u32 MetaZone_ReadBinary(u32 u4Idx, u32 *pbData , u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pbData: the buffer to store the Binary data
- u4Size: buffer size in bytes

【注意】

Binary data item 的 index 从 0x10000 开始（Index 0—0xFFFF 保留）。第一个 Binary data item 的 index 为 0x10000，最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 Binary Data 的数目，目前默认为 200)。

33.3.2.7 MetaZone_WriteBinary

【描述】

Write a Binary data.

【语法】

```
u32 MetaZone_WriteBinary(u32 u4Idx, u32 *pbData , u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pu4Data: the buffer of Binary data to be written.
- u4Size : buffer size in bytes

【注意】

Binary data item 的 index 从 0x10000 开始（Index 0—0xFFFF 保留）。第一个 Binary data item 的 index 为 0x10000，最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 Binary Data 的数目，目前默认为 200)。

33.3.2.8 MetaZone_SpecWriteBinary

【描述】

Write a Binary data.

【语法】

```
u32 MetaZone_SpecWriteBinary(u32 u4Idx, u32 *pbData , u32 u4Size, u32 flag);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error

- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pu4Data: the buffer of Binary data to be written.
- u4Size: buffer size in bytes
- flag: write flag. flag = 0 (no operate), flag = 1 (write to default value)

【注意】

Binary data item 的 index 从 0x10000 开始（Index 0—0xFFFF 保留）。第一个 Binary data item 的 index 为 0x10000，最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 Binary Data 的数目，目前默认为 200)。

33.3.2.9 MetaZone_SpecRead

【描述】

Read Reserved data.

【语法】

```
u32 MetaZone_SpecReadReserved(u32 u4AreaNum, char *pbData, u32 u4Offset, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- pbData: the buffer of Reserved data to be written.
- u4AreaNum: the number of read reserved area.
- u4Offset: the offset of read.
- u4Size: buffer size in bytes

33.3.2.10 MetaZone_SpecWriteReserved

【描述】

Write Reserved data.

【语法】

```
u32 MetaZone_SpecWriteReserved(u32 u4AreaNum, char *pbData, u32 u4Offset, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- pbData: the buffer of Reserved data to be written.
- u4AreaNum: the number of write reserved area.
- u4Offset: the offset of write
- u4Size: buffer size in bytes

33.3.2.11 MetaZone_Flush**【描述】**

清除 metazone 缓存，将在内存中修改的数据写入 emmc 实际存储中。MetaZone_Write & MetaZone_WriteBinary & MetaZone_SpecWriteReserved 只是把数据写入到 metazone 的 memory，调用 MetaZone_Flush 后才把修改的结果写到 metazone 分区。

【语法】

```
u32 MetaZone_Flush(int fgblock);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

fgblock

- 为 TRUE(1)时，本函数在数据写入 metazone 分区完成后才返回；
- 为 FALSE(0)，本函数立即返回，metazone 另开 thread 写入 metazone 数据。

【注意】

当调用 MetaZone_Write & MetaZone_WriteBinary & MetaZone_SpecWriteReserved 等修改 metazone 数据后一定要调用 MetaZone_Flush 把数据写入到 metazone 分区。否则系统重启后，修改结果将丢失（回到上次 flush 的状态）。

33.3.2.12 MetaZone_AB_Upgrade

【描述】

Metazone OTA upgrade

【语法】

```
u32 MetaZone_AB_Upgrade(const char *path, u32 upgradeflag);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- Path: the path of metazone upgrade file
- Upgradeflag: 1 represents there is a metazone upgrade file in the OTA package;
0 represents there is not a metazone upgrade file in the OTA package

33.3.3 LK APIs

表 33-7 LK APIs

API	说明
_MetaZone_Read	Read a DWORD item value
_MetaZone_Write	Write a DWORD item value.
_MetaZone_SpecWrite	Special Write a DWORD item value for factory reset.
_MetaZone_ReadBinary	Read a Binary item data
_MetaZone_WriteBinary	Write a Binary item data
_MetaZone_SpecWriteBinary	Special Write a Binary item value for factory reset.
_MetaZone_SpecReadReserved	Read Reserved data
_MetaZone_SpecWriteReserved	Write Reserved data
_MetaZone_Flush	Trigger metazone write data to storage media.

33.3.3.1 _MetaZone_Read

【描述】

Read a DWORD item value.

【语法】

```
u32 _MetaZone_Read(u32 u4Idx, u32 *pu4Data);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- pu4Data: for storing read DWORD value

【注意】

DWORD item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 DWORD item index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 DWORD Value 的数目, 目前默认为 500)。

33.3.3.2 _MetaZone_Write

【描述】

Write a DWORD item value.

【语法】

```
u32 _MetaZone_Write(u32 u4Idx, u32 u4Data);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- u4Data: the write value of DWORD item

【注意】

DWORD item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 DWORD item index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 DWORD Value 的数目, 目前默认为 500)。

33.3.3.3 _MetaZone_SpecWrite**【描述】**

Write a DWORD item value.

【语法】

```
u32 _MetaZone_SpecWrite(u32 u4Idx, u32 u4Data, u32 flag);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- u4Data: the write value of DWORD item
- flag: write flag. flag = 0 (no operate), flag = 1 (write to default value)

【注意】

DWORD item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 DWORD item index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 DWORD Value 的数目, 目前默认为 500)。

33.3.3.4 _MetaZone_ReadBinary**【描述】**

Read a Binary item data.

【语法】

```
u32 _MetaZone_ReadBinary(u32 u4Idx, u32 *pbData, u32 u4Size);
```

【返回值】

- 0x0: succeed

- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pu4Data: the buffer to store the Binary data
- u4Size: buffer size in bytes

【注意】

Binary data item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 Binary data item 的 index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 Binary Data 的数目, 目前默认为 200)。

33.3.3.5 _MetaZone_WriteBinary

【描述】

Write a Binary data.

【语法】

```
u32 _MetaZone_WriteBinary(u32 u4Idx, u32 *pbData, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pu4Data: the buffer of Binary data to be written.
- u4Size: buffer size in bytes

【注意】

Binary data item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 Binary data item 的 index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 Binary Data 的数目, 目前默认为 200)。

33.3.3.6 _MetaZone_SpecWriteBinary

【描述】

Write a Binary data.

【语法】

```
u32 _MetaZone_SpecWriteBinary(u32 u4Idx, u32 *pbData, u32 u4Size, u32 flag);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pu4Data: the buffer of Binary data to be written.
- u4Size: buffer size in bytes
- flag: write flag. flag = 0 (no operate), flag = 1 (write to default value)

【注意】

Binary data item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 Binary data item 的 index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 Binary Data 的数目, 目前默认为 200)。

33.3.3.7 _MetaZone_SpecReadReserved

【描述】

Read Reserved data.

【语法】

```
u32 MetaZone_SpecReadReserved(u32 u4AreaNum, char *pbData, u32 u4Offset, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- pbData: the buffer of Reserved data to be written.
- u4AreaNum: the number of read reserved area.
- u4Offset: the offset of read
- u4Size: buffer size in bytes

33.3.3.8 _MetaZone_SpecWriteReserved

【描述】

Write Reserved data.

【语法】

```
u32 _MetaZone_SpecWriteReserved(u32 u4AreaNum, char *pbData, u32 u4Offset, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- pbData: the buffer of Reserved data to be written.
- u4AreaNum: the number of read reserved area.
- u4Offset: the offset of read
- u4Size: buffer size in bytes

33.3.3.9 _MetaZone_Flush

【描述】

_MetaZone_Write & _MetaZone_WriteBinary & _MetaZone_SpecWriteReserved 只是把数据写入到 metazone 的 memory，调用 flush 后才把修改的结果写到 metazone 分区。

【语法】

```
u32 _MetaZone_Flush(BOOL fgSync);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error

- 0x80000001: metazone not init before read or write

【参数】

fgSync: not used here, just keep same with the flush API in other's layer .

【注意】

调用 _MetaZone_Write & _MetaZone_WriteBinary & _MetaZone_SpecWriteReserved 修改 metazone 数据后一定要调用 _MetaZone_Flush 把数据写入到 metazone 分区。否则系统重启后，修改结果将丢失（回到上次 flush 的状态）。

33.3.4 Safety OS APIs

表 33-8 Safety OS APIs

API	说明
ulMetazoneRead	Read a DWORD item value
ulMetazoneWrite	Write a DWORD item value.
ulMetazoneSpecWrite	Special Write a DWORD item value for factory reset.
ulMetazoneReadBinary	Read a Binary item data
ulMetazoneWriteBinary	Write a Binary item data
ulMetazoneSpecWriteBinary	Special Write a Binary item value for factory reset.
ulMetaZoneSpecReadReserved	Read Reserved data
ulMetazoneSpecWriteReserved	Write Reserved data
ulMetazoneFlush	Trigger metazone write data to storage media.

33.3.4.1 ulMetazoneRead

【描述】

Read a DWORD item value.

【语法】

```
u32 ulMetazoneRead(u32 u4Idx, u32 *pu4Data);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.

- pu4Data: for storing read DWORD value

【注意】

DWORD item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 DWORD item index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 DWORD Value 的数目, 目前默认为 500)。

33.3.4.2 ulMetazoneWrite

【描述】

Write a DWORD item value.

【语法】

```
u32 ulMetazoneWrite(u32 u4Idx, u32 u4Data);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- u4Data: the write value of DWORD item

【注意】

DWORD item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 DWORD item index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 DWORD Value 的数目, 目前默认为 500)。

33.3.4.3 ulMetazoneSpecWrite

【描述】

Write a DWORD item value.

【语法】

```
u32 ulMetazoneSpecWrite(u32 u4Idx, u32 u4Data, u32 flag);
```

【返回值】

- 0x0: succeed

- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- u4Data: the write value of DWORD item
- flag: write flag. flag = 0 (no operate), flag = 1 (write to default value)

【注意】

DWORD item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 DWORD item index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 DWORD Value 的数目, 目前默认为 500)。

33.3.4.4 ulMetazoneReadBinary

【描述】

Read a Binary item data.

【语法】

```
u32 ulMetazoneReadBinary(u32 u4Idx, u32 *pbData, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pu4Data: the buffer to store the Binary data
- u4Size: buffer size in bytes

【注意】

Binary data item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 Binary data item 的 index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 Binary Data 的数目, 目前默认为 200)。

33.3.4.5 ulMetazoneWriteBinary

【描述】

Write a Binary data.

【语法】

```
u32 ulMetazoneWriteBinary(u32 u4Idx, u32 *pbData , u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pu4Data: the buffer of Binary data to be written.
- u4Size: buffer size in bytes

【注意】

Binary data item 的 index 从 0x10000 开始（Index 0—0xFFFF 保留）。第一个 Binary data item 的 index 为 0x10000，最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 Binary Data 的数目，目前默认为 200)。

33.3.4.6 ulMetazoneSpecWriteBinary

【描述】

Write a Binary data.

【语法】

```
u32 ulMetazoneSpecWriteBinary(u32 u4Idx, u32 *pbData , u32 u4Size, u32 flag);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.

- pu4Data: the buffer of Binary data to be written.
- u4Size: buffer size in bytes
- flag: write flag. flag = 0 (no operate), flag = 1 (write to default value)

【注意】

Binary data item 的 index 从 0x10000 开始 (Index 0—0xFFFF 保留)。第一个 Binary data item 的 index 为 0x10000, 最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 Binary Data 的数目, 目前默认为 200)。

33.3.4.7 ulMetazoneSpecReadReserved

【描述】

Read Reserved data.

【语法】

```
u32 MetaZone_SpecReadReserved(u32 u4AreaNum, char *pbData, u32 u4Offset, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- pbData: the buffer of Reserved data to be written.
- u4AreaNum: the number of read reserved area.
- u4Offset: the offset of read
- u4Size: buffer size in bytes

33.3.4.8 ulMetazoneSpecWriteReserved

【描述】

Write Reserved data.

【语法】

```
u32 MetaZone_SpecWriteReserved(u32 u4AreaNum, char *pbData, u32 u4Offset, u32 u4Size);
```

【返回值】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

954 / 1254

本文档包含信息归杰发科技所有, 未经许可, 严禁转载或披露

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- pbData: the buffer of Reserved data to be written.
- u4AreaNum: the number of read reserved area.
- u4Offset: the offset of read
- u4Size: buffer size in bytes

33.3.4.9 ulMetazoneFlush

【描述】

ulMetazoneWrite & ulMetazoneWriteBinary & ulMetazoneSpecWriteReserved 只是把数据写入到 metazone 的 memory，调用 flush 后才把修改的结果写到 metazone 分区。

【语法】

```
u32 ulMetazoneFlush(BOOL fgSync);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

fgSync: not used here, just keep same with the flush API in other's layer

【注意】

调用 ulMetazoneWrite & ulMetazoneWriteBinary & ulMetazoneSpecWriteReserved 修改 metazone 数据后一定要调用 ulMetazoneFlush 把数据写入到 metazone 分区。否则系统重启后，修改结果将丢失（回到上次 flush 的状态）。

33.3.5 Kernel Driver APIs



说明：

adas/cluster/ivi 的 Native 接口是复用的，以下接口的说明适用 adas/custer/ivi 。

表 33-9 kernel driver APIs

API	说明
MetaZone_Read	Read a DWORD item value
MetaZone_Write	Write a DWORD item value.
MetaZone_SpecWrite	Special Write a DWORD item value for factory reset.
MetaZone_ReadBinary	Read a Binary item data
MetaZone_WriteBinary	Write a Binary item data
MetaZone_SpecWriteBinary	Special Write a Binary item value for factory reset.
MetaZone_SpecReadReserved	Read Reserved data
MetaZone_SpecWriteReserved	Write Reserved data
MetaZone_Flush	Trigger metazone write data to storage media.
MetaZone_AB_Upgrade_For_Kernel	Metazone OTA upgrade

33.3.5.1 MetaZone_Read

【描述】

Read a DWORD item value.

【语法】

```
u32 MetaZone_Read(u32 u4Idx, u32 *pu4Data);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- pu4Data: for storing read DWORD value

【注意】

DWORD item 的 index 从 0x10000 开始（Index 0—0xFFFF 保留）。第一个 DWORD item index 为 0x10000，最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 DWORD Value 的数目，目前默认为 500)。

33.3.5.2 MetaZone_Write

【描述】

Write a DWORD item value.

【语法】

```
u32 MetaZone_Write(u32 uIdx, u32 u4Data);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- u4Data: the write value of DWORD item

【注意】

DWORD item 的 index 从 0x10000 开始（Index 0—0xFFFF 保留）。第一个 DWORD item index 为 0x10000，最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 DWORD Value 的数目，目前默认为 500)。

33.3.5.3 MetaZone_SpecWrite

【描述】

Write a DWORD item value.

【语法】

```
u32 MetaZone_SpecWrite(u32 uIdx, u32 u4Data, u32 flag);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of DWORD item.
- u4Data: the write value of DWORD item

- flag: write flag. flag = 0 (no operate), flag = 1 (write to default value)

【注意】

DWORD item 的 index 从 0x10000 开始（Index 0—0xFFFF 保留）。第一个 DWORD item index 为 0x10000，最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 DWORD Value 的数目，目前默认为 500)。

33.3.5.4 MetaZone_ReadBinary

【描述】

Read a Binary item data.

【语法】

```
u32 MetaZone_ReadBinary(u32 u4Idx, u32 *pbData, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pbData: the buffer to store the Binary data
- u4Size: buffer size in bytes

【注意】

Binary data item 的 index 从 0x10000 开始（Index 0—0xFFFF 保留）。第一个 Binary data item 的 index 为 0x10000，最后一个 item 的 index 为 0x10000 + number - 1 (number 为 Header 给出 Binary Data 的数目，目前默认为 200)。

33.3.5.5 MetaZone_WriteBinary

【描述】

Write a Binary data.

【语法】

```
u32 MetaZone_WriteBinary(u32 u4Idx, u32 *pbData, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pu4Data: the buffer of Binary data to be written.
- u4Size: buffer size in bytes

【注意】

Binary data item 的 index 从 0x10000 开始（Index 0—0xFFFF 保留）。第一个 Binary data item 的 index 为 0x10000，最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 Binary Data 的数目，目前默认为 200)。

33.3.5.6 MetaZone_SpecWriteBinary

【描述】

Write a Binary data.

【语法】

```
u32 MetaZone_SpecWriteBinary(u32 u4Idx, u32 *pbData, u32 u4Size, u32 flag);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- u4Idx: the index of Binary item.
- pu4Data: the buffer of Binary data to be written.
- u4Size: buffer size in bytes
- flag: write flag. flag = 0 (no operate), flag = 1 (write to default value)

【注意】

Binary data item 的 index 从 0x10000 开始（Index 0—0xFFFF 保留）。第一个 Binary data item 的 index 为 0x10000，最后一个 item 的 index 为 0x10000 + number -1 (number 为 Header 给出 Binary Data 的数目，目前默认为 200)。

33.3.5.7 MetaZone_SpecReadReserved

【描述】

Read Reserved data.

【语法】

```
u32 MetaZone_SpecReadReserved(char *pbData , u32 u4AreaNum, u32 u4Offset, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- pbData: the buffer of Reserved data to be written.
- u4AreaNum: the number of read reserved area.
- u4Offset: the offset of read
- u4Size: buffer size in bytes

33.3.5.8 MetaZone_SpecWriteReserved

【描述】

Write Reserved data.

【语法】

```
u32 MetaZone_SpecWriteReserved(char *pbData , u32 u4AreaNum, u32 u4Offset, u32 u4Size);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- pbData: the buffer of Reserved data to be written.
- u4AreaNum: the number of writing reserved area.

- u4Offset: the offset of writing.
- u4Size: buffer size in bytes

33.3.5.9 MetaZone_Flush

【描述】

清除 metazone 缓存，将修改的数据写入 metazone 分区。MetaZone_Write & MetaZone_WriteBinary 和 MetaZone_SpecWriteReserved 只是把数据写入到 metazone 的 memory，调用 flush 后才把修改的结果写到 metazone 分区。

【语法】

```
u32 MetaZone_Flush(int fgblock);
```

【返回值】

- 0x0: succeed
- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

fgblock

- 为 TRUE(1)时，本函数在数据写入 metazone 分区完成后才返回；
- 为 FALSE(0)，本函数立即返回，metazone 另开 thread 写入 metazone 数据。

【注意】

调用 MetaZone_Write & MetaZone_WriteBinary 和 MetaZone_SpecWriteReserved 修改 metazone 数据后一定要调用 MetaZone_Flush 把数据写入到 metazone 分区。否则系统重启后，修改结果将丢失（回到上次 flush 的数据）。

33.3.5.10 MetaZone_AB_Upgrade_For_Kernel

【描述】

Metazone OTA Upgrade

【语法】

```
u32 MetaZone_AB_Upgrade_For_kernel(char *buf, u32 u4flag);
```

【返回值】

- 0x0: succeed

- 0x80000000: metazone option error
- 0x80000001: metazone not init before read or write

【参数】

- Buf: the buffer of metazone data
- U4flag: 1 represents there is a metazone upgrade file in the OTA package;
0 represents there is not a metazone upgrade file in the OTA package.

33.3.6 注意事项

Metazone 读写接口都是在内存中的操作，故在任何一层中调用 metazone 写接口后，只有调用了对应层次的 flush 接口，数据才能被真正同步到物理存储中，否则修改的数据容易丢失。

33.4 使用说明和注意事项

33.4.1 Metazone 生成方式

AC8025 Android S 上使用 perl 脚本来生成 metazone.bin，在 build/projects/ivi_s_ac8025/ 和 build/projects/cockpit_s_linux_ac8025/ 下有 metazone.csv 文件，metazone.pl 根据 metazone.csv 生成 metazone_index.h 和 metazone.bin。

在修改 metazone.bin 的值时，仅需要修改 metazone.csv，在编译时就可以动态生成 metazone_index.h 和 metazone.bin。

metazone_index.h 和 metazone.bin 编译生成后路径：source/android/out/soong/.intermediates/
vendor/autochips/proprietary/external/metazone/metazone_header/gen/

其他模块使用 metazone 时，无需再记住 index 具体数值，只需要知道 metazone_index 中枚举名称即可。metazone.csv 格式如下图所示。

```

1 Type,Name,Size,Have_Data,Factory_Reset,Update,Data,Default_Data
2 Dword,METAZONE_BACKLIGHT_NUM1,4,1,1,0,22000,22000
3 Dword,METAZONE_BACKLIGHT_NUM2,4,1,1,0,22000,22000
4 Dword,METAZONE_PQ_PRIMARY_CONTRAST,4,1,1,0,5,5
5 Dword,METAZONE_PQ_PRIMARY_SATURATION,4,1,1,0,5,5
6 Dword,METAZONE_PQ_PRIMARY_BRIGHTNESS,4,1,1,0,5,5
7 Dword,METAZONE_PQ_SECONDARY_CONTRAST,4,1,1,0,5,5
8 Dword,METAZONE_PQ_SECONDARY_SATURATION,4,1,1,0,5,5
9 Dword,METAZONE_PQ_SECONDARY_BRIGHTNESS,4,1,1,0,5,5
10 Dword,METAZONE_BACKCAR_VDO_SOURCE,4,1,1,0,33685504,33685504
    
```

图 33-8 metazone.csv 格式

其中：

- **Type:** 表示需使用 metazone 类型，Dword、Binary 和 Reserved
- **Name:** 即生成 metazone_index.h 中枚举名称
- **Size:** 实际数据大小，Dword 固定 4 字节，Binary 最大 100 字节，Reserved 最大 32KB
- **Have Data:** 表明当前 index 是否有初始值，0 表示无值，1 表示有值
- **Factory Reset:** 表明是否支持 factory reset，0 表示不支持（factory reset 时不进行操作），1 表示支持（factory reset 时会恢复默认值）
- **Update:** 表明当前 index 是否支持升级，0 表示不支持，1 表示支持
- **Data:** 初始值，如果无初始值则可填 0。（初始值若为 0，则 Have Data 区域需为 1）
- **Default Data:** 默认值，factory reset 恢复的值，不支持 factory 时可填 0

Metazone.bin 生成的方式有两种：

- 全编，使用 allmake.sh 脚本
- 使用 ninja 编译，需要进入到 source/android 目录下，执行具体命令：`./prebuild/build-tools/linux-x86/bin/ninja -f out/combined-xx.ninja out/target/product/${PROJECT}/metazone.bin`

33.4.2 各层 Metazone 接口使用注意

为了防止 metazone reserved memory 中的数据被毁导致其他模块功能异常。建议各模块在 code 逻辑中读写 metazone fail 的时候，根据各模块特性给定合适的默认值。这样即使 metazone 使用异常，也能保证各模块功能正常。

33.4.3 Java 层使用 Metazone

在 java 层使用 metazone index,需要在 Android.mk 中添加如下语句：

```
ifeq ( $(ATC_METAZONE_SUPPORT), yes)
    LOCAL_JAVA_LIBRARIES += libmetazone_header_jar
endif
```

然后在 java 文件中 import com.autochips.metazone.index.MetazoneIndex;

初始化变量时可参考使用 `private final int USB_PROTOCOL = MetazoneIndex.DwordIndex.METAZONE_USB_PROTOCOL;`

如果是 binray 类型，则是 `MetazoneIndex.BinaryIndex.xxxx。`

若是 APK 访问 Metazone，需要编译成 system 或 platform 属性的 app，而 untrusted app 访问 Metazone 会被 SELinux 阻止，Java 层的 sample code 如下：

```
import com.autochips.metazone.AtcMetazone;    //导入 AtcMetazone 包
import com.autochips.metazone.index.MetazoneIndex;    //导入 MetazoneIndex 包

int dwordIndex=0x10000+0x2;    //选取 dword 类型的 index 值
int binaryIndex=0x10000+0x10;    //选取 Binary 类型的 index 值
int binarySize=100;    //选取 Binary 类型读写的 size
int reservedSize=512;    //选取 Reserved 类型读写的 size
int reservedOffset=0x100;    //选取 Reserved 类型读写的 offset
int reservedAreaNum= 1;    //选取 Reserved 类型写的区域 areanum
int data=10;    // 定义 dword 写测试的 data 值
int[] dwordBuffer=new int[1];    //创建 dword 读测试的 buffer
byte[] binaryBuffer=new byte[binarySize];    //创建 Binary 读写测试的 buffer
byte[] reservedBuffer=new byte[reservedSize]; //创建 Reserved 读写测试的 buffer
boolean flushFlag=true;    //定义 flush 操作的 flag

/*for metazone Dword RW test*/
AtcMetazone.readval(dwordIndex, dwordBuffer); //将 dword 类型 dwordIndex 位置存储值读到 dwordBuffer
AtcMetazone.writeval(dwordIndex,data); //将 data 值写到 dword 类型的 dwordIndex 位置
AtcMetazone.flush(flushFlag); //调用 flush，将内存中的数据保存到 emmc 中，待操作完成即返回

/*for metazone Binary RW test*/
AtcMetazone.readbinary(binaryIndex, binaryBuffer, binarySize);
//将 Binary 类型的 binaryIndex 位置存储的前 binarySize 大小的值读取到 binaryBuffer 中
AtcMetazone.writebinary(binaryIndex, binaryBuffer, binarySize);
//将 binaryBuffer 的前 binarySize 大小的值写到到 Binary 类型的 binaryIndex 位置
AtcMetazone.flush(flushFlag); //同上 flush
```



```
/*for metazone Reserved RW test*/  
AtcMetazone.specreadreserved(reservedAreaNum, reservedBuffer, reservedOffset,  
reservedSize);  
  
//将 Reserved 类型 reservedAreaNum 区域从 reservedOffset 开始的 reservedSize 大小值读取  
到 reservedBuffer 中  
  
AtcMetazone.specwritereserved(reservedAreaNum, reservedBuffer, reservedOffset,  
reservedSize, 1);  
  
//将 reservedBuffer 中的前 reservedSize 的值存储到 reservedAreaNum 区域的  
reservedOffset 开始位置  
  
AtcMetazone.flush(flushFlag); //同上 flush
```

33.4.4 Native 层使用 MetaZone

- 1、调用 MetaZone_Init () // 程序运行期间只需调用一次即可；
- 2、调用 Read/Write API 读、写 metazone 数据；
- 3、如果有写数据操作，则须调用 MetaZone_Flush(), 把内存中的修改写入 metazone 分区；
- 4、若有需要，重复 2、3，否则执行步骤 5；
- 5、调用 MetaZone_Deinit() // 所有读写完成之后，执行此操作。

说明：

API 的 header file 在 source/kernel/common/drivers/soc/autochips/include/metazone/metazone.h

Android.mk 中需增加 lib 库 link:

LOCAL_HEADER_LIBRARYS := device_kernel_headers\libatcmetazone_header

Native 进程访问 metazone 需要配置相应 SELinux 权限，具体方法是在本模块的 .te 文件中(一般位于 /device/mediatek/sepolicy/bsp/)，添加如下 3 条规则，可参考 /metalogo.te 文件：

- allow 模块名 metazone_device:chr_file {open read write ioctl};
- allow 模块名 block_device:dir {search};
- allow 模块名 metazone_block_device:blk_file {open read write};

Native 层去访问 metazone 的 Sample code 如下：

```
int Dword_index=0x10000+0x2; //选取 dword 类型的 index 值  
  
int Binary_index=0x10000+0x10; //选取 Binary 类型的 index 值  
  
int Dword_size=4; //定义 Dword 类型 data 的大小，为申请内存用
```

```

int Binary_size=100; //选取 Binary 类型读写的 size
int Reserved_size=512; //选取 Reserved 类型读写的 size
int Reserved_areanum = 1; //选取 Reserved 类型读写的 areanum
int data=10; // 定义 dword 写测试的 data 值
int *Dword_buff=NULL;
char *Binary_buff=NULL;
char *Reserved_buff=NULL;
int flush_flag=1;

Dword_buff=(int *)malloc(Dword_size); //创建 dword 读测试的 buffer
if(!Dword_buff){
    printf("malloc for Dwrod_buff fail (size %d)\n",Dword_size);
    return -1;
}
Binary_buff=(char *)malloc(Binary_size); //创建 Binary 读写测试的 buffer
if(!Binary_buff){
    printf("malloc for Binary_buff fail (size %d)\n",Binary_size);
    return -1;
}
Reserved_buff=(char *)malloc(Reserved_size); //创建 Reserved 读写测试的 buffer
if(!Reserved_buff){
    printf("malloc for Binary_buff fail (size %d)\n",Reserved_size);
    return -1;
}

MetaZone_Init(); //执行 init 操作
/*for metazone Dword RW test*/
MetaZone_Read(Dword_index,Dword_buff);//以下步骤含义均与 Java 层 sample code 相同
Metazone_Write(Dword_index,data);
Metazone_Flush(flush_flag);

```

```

/*for metazone Binary RW test*/
Metazone_ReadBinary(Binary_index,Binary_buff,Binary_size);
Metazone_WriteBinary(Binary_index,Binary_buff,Binary_size);
Metazone_Flush(flush_flag);

/*for metazone Reserved RW test*/
Metazone_SpecReadReserved(Reserved_areanum, Reserved_buff, Reserved_offset,
Reserved_size);
Metazone_SpecWriteReserved(Reserved_areanum , Reserved_buff, Reserved_offset,
Reserved_size);
Metazone_Flush(flush_flag);

MetaZone_Deinit();//执行 deinit 操作
free(Dword_buff); //释放内存
free(Binary_buff);
free(Reserved_buff);
Dword_buff = NULL;
Binary_buff =NULL;
Reserved_buff =NULL;

```

33.4.5 LK 中使用 Metazone

与 native 层使用方法相似，但不需要 Init / Deinit，直接使用 LK 中相关 APIs（APIs 的名称有区别），具体使用可参考 native 的 sample code 使用方式。

注意事项：

- 需要 include header file，其位置：

source/vendor/autochips/proprietary/bootable/lk/platform/autochips/\$(PROJECT)/include/platform/metazone_inter.h

- 在 lk 中使用时，需要在 makefile 文件中加入如下语句，添加 metazone_index.h 的依赖：

```
ifeq($(ATC_METAZONE_SUPPORT), yes)
```

```
LIBATCMETAZONE_HEADER_EXPORT_INCLUDES := $(call intermediates-dir-for,
```

```

HEADER_LIBRARIES, libatcmetazone_header)/exoint_includes

LK_MAKE_OPTION +=

EXPORT_INCLUDES="$(LIBATCMEATZONE_HEADER_EXPORT_INCLUDES)"

$(BUILD_LK_TARGET): libatcmetazone_header

endif

```

然后即可在头文件或者.c文件 include <metazone_index.h>。

- LK 整体编译成一个 image，注意：在 LK 中使用 metazone 读写接口，请在 source/vendor/autochips/proprietary/bootable/lk/platform/autochips/\$(PROJECT)/boot_flow.c 文件中 Metazone_Read_Partition()函数中_fgWriteableZoneInited = TRUE 语句之后。

33.4.6 Safety 中使用 Metazone

与 native 层使用相似，但不需要 Init / Deinit，直接使用 safety 中相关 APIs（APIs 的名称有区别），注意 safety 中只支持 read 接口，具体使用可参考 native 的 sample code 使用方式。

APIs 的 header file 在 source/vendor/autochips/proprietary/tinysys/hsm/os/include/common/metazone_driver.h

Safety 中模块使用 metazone_index.h 需要在 makefile 中添加如下语句：

```

ifneq ($(EXPORT_INCLUDES), )
LOCAL_INCULE += $(addprefix -I$(ROOTDIR)/, $(foreach i, $(EXPORT_INCLUDES),
$(subst -I, , $(strip $(shell cat &(ROOTDIR)/$(i))))))

```

Safety 整体编译成一个 image，注意：在 Safety 中使用 metazone read 接口，请在调用 ulMetazoneInit()中_fgWriteableZoneInited = TRUE 语句之后。

33.4.7 Kernel Driver 中使用 metazone

与 Native 层的使用相似，但不需要 Init / Deinit 操作，直接使用相关 APIs（APIs 名称和定义与 Native 层一致），可参考 native 的 sample code 使用方式。

如果有写数据操作，则调用 MetaZone_Flush，把内存中的修改写入 metazone 分区。Metazone driver 静态编译进内核，在 Kernel 层 API 通过 EXPORT_SYMBOL 方式提供给内核其它模块使用。

同时需要注意，在 Kernel 中使用 metazone API，需要确保在 metazone.ko insmod 后使用。

33.4.8 通过 adb 操作平台上的数据

通过 adb 命令打开 metazone 的 Debug 节点：proc/metazone 来进行读写 metazone 的数据信息进行 debug。

- Dword 数据：（index：是需要读取的 metazone index 需要用十六进制表示）
 - 读命令：`echo dword_read index 的名称 > proc/mtz_debug`
`cat proc/mtz_debug`
 - 写命令：`echo dword_write index 的名称 value > proc/mtz_debug`
- Binary 数据：
 - 读命令：`echo binary_read index 的名称 size > proc/mtz_debug`
`cat proc/mtz_debug`
 - 写命令：`echo binary_write index 的名称 size value > proc/mtz_debug`

33.4.9 Metazone Reserved 类型数据的使用注意事项

在目前 AC8025 的项目中，Reserved 类型的数据只提供一个给 display 模块使用，暂定为 1 个。目前单个 metazone 的使用空间为 512KB（metazone 的分区大小为 1M，其中包含 metazone 的备份）。

如需增加 reserved 数据类型的使用数量（最大支持 6 个）。

33.4.10 扩展 Metazone 导致 size 增加后注意事项

当增加 metazone 的 index 数据导致 metazone.bin 大小超过 512KB 时，注意需要修改 metazone 的分区大小和 reserved memory 大小分别为 2M，否则会导致系统加载 metazone 失败。

注意：

增加 metazone 的分区大小会影响系统启动时间 60ms 以上，具体时间根据 emmc 型号而不同。

建议：

非必要情况下建议使用 datazone 来存储 size 较大的数据，datazone 资源大小可自定义。datazone 的使用详见客户文档 datazone 的使用说明部分。

33.5 用户定制化

33.5.1 新增 index，修改 metazone.csv 文件

路径：build/projects/xxx(产品形态)/metazone.csv

修改 metazone.csv 文件

新增 index 的注意事项：

```

132 Dword,METAZONE_SENSOR_PATTERN_MODE,4,1,1,0,0,0
133 Dword,METAZONE_OTA_TEST,4,1,1,1,1,0
134 Dword,METAZONE_MIPI0_WORKING_OS,4,1,1,0,5,5
135 Dword,METAZONE_MIPI1_WORKING_OS,4,1,1,0,0,0
136 Dword,METAZONE_MIPI2_WORKING_OS,4,1,1,0,0,0
137 Binary,METAZONE_BT,6,1,0,0,0,0
138 Binary,METAZONE_WIFI,6,1,0,0,0,0
139 Binary,METAZONE_CARPLAY_CERTIFICATE_DATA0,100,1,1,0,0,0
140 Binary,METAZONE_CARPLAY_CERTIFICATE_DATA1,100,1,1,0,0,0
141 Binary,METAZONE_CARPLAY_CERTIFICATE_DATA2,100,1,1,0,0,0
142 Binary,METAZONE_CARPLAY_CERTIFICATE_DATA3,100,1,1,0,0,0
143 Binary,METAZONE_CARPLAY_CERTIFICATE_DATA4,100,1,1,0,0,0
144 Binary,METAZONE_CARPLAY_CERTIFICATE_DATA5,100,1,1,0,0,0
145 Binary,METAZONE_CARPLAY_CERTIFICATE_DATA6,100,1,1,0,0,0
146 Binary,METAZONE_CARPLAY_CERTIFICATE_DATA7,100,1,1,0,0,0
147 Binary,METAZONE_CARPLAY_CERTIFICATE_DATA8,100,1,1,0,0,0
148 Binary,METAZONE_CARPLAY_CERTIFICATE_DATA9,100,1,1,0,0,0
149 Binary,METAZONE_BT_RF_POWER,1,1,0,0,6,6
150 Binary,METAZONE_DSP_PO_120_0_1_0_0_0
151 Reserved,METAZONE_GAMMA,32768,1,1,0,reserved0.bin,reserved0.bin
152

```

Dword需要在dword后面追加新增

Binary需要在binary后面追加新增

图 33-9 metazone.csv 中新增 index 示意图

由于随意的插入新增 index，如果不同时更新 metazone_index.h 和 metazone.bin 文件，可能会造成和其他模块共用一个 index，会导致覆盖其他模块储存在改 index 的数据。

在编译中增加了读插入顺序的 check，如果不按照上图这种添加方式，编译会失败。

在 csv 表格中，我们会以"Dword_Customer"/"Binary_Customer"来区分客户定制化 index 和 ATC 原生 index。

绿色框内的第二列“200”为客户 index 的起始编号，后续客户 index 是以 200 开始进行编号。

47	Dword	METAZONE_VIDEOIN_BT601_ENABLE	4	1	1	0	0	0	0
48	Dword	METAZONE_RVC_SELECT_CHANNEL	4	1	1	0	0	0	0
49	Dword_Cus		200						
50	Binary	METAZONE_BT	6	1	0	0	0	0	0
51	Binary	METAZONE_WIFI	6	1	0	0	0	0	0
52	Binary	METAZONE_CARPLAY_CERTIFICATE_DATA0	100	1	1	0	0	0	0
53	Binary	METAZONE_CARPLAY_CERTIFICATE_DATA1	100	1	1	0	0	0	0
54	Binary	METAZONE_CARPLAY_CERTIFICATE_DATA2	100	1	1	0	0	0	0
55	Binary	METAZONE_CARPLAY_CERTIFICATE_DATA3	100	1	1	0	0	0	0
56	Binary	METAZONE_CARPLAY_CERTIFICATE_DATA4	100	1	1	0	0	0	0
57	Binary	METAZONE_CARPLAY_CERTIFICATE_DATA5	100	1	1	0	0	0	0
58	Binary	METAZONE_CARPLAY_CERTIFICATE_DATA6	100	1	1	0	0	0	0
59	Binary	METAZONE_CARPLAY_CERTIFICATE_DATA7	100	1	1	0	0	0	0
60	Binary	METAZONE_CARPLAY_CERTIFICATE_DATA8	100	1	1	0	0	0	0
61	Binary	METAZONE_CARPLAY_CERTIFICATE_DATA9	100	1	1	0	0	0	0
62	Binary	METAZONE_BT_RF_POWER	1	1	0	0	6	6	
63	Binary_Cu		100						
64									

图 33-10 客户定制化 index 和 ATC 原生的 index 分区域

如果需要添加 Dword Index/Binary index，只需在"Dword_Customer"/"Binary_Customer"后新增即可。

33.6 典型问题

1. DWORD 与 Binary 类型的[0, 0x10000)范围 index 值暂且不能使用，这是为后续开放预留的，在使用时需要加上一个 offset 0x10000。例如，文档中记录 BT 使用的 metazone 的 index 为 48，需注意在实际 coding 中，index 应该为 0x10000+48。
2. DWORD 与 Binary 的 index 值是独立的，针对于各自的 data area。
3. 在 kernel driver 层使用 metazone RW API，请注意其 driver 加载必须在 metazone driver 加载之后。如果出现调用 API fail，且 error 为"unknown symbol"，一般都属于此类错误。
4. 如果调用 metazone 的写操作，须调用 flush 操作刷写到 emmc，否则系统重启后，修改结果将丢失（回到上次 flush 的数据）。
5. Java 层调用 metazone API 时，请使用标准 SDK 的方法，不能在自己 Module 中 JNI 层封装调用 metazonenative api 供 Java 调用（会被 SELinux 阻止访问）。
6. Metazone 中每个 index item 只用来存储一个数据，不能出现多个数据公用一个 index，目前 AC8025 Android S 系统已经使用的 index。若您需要使用 metazone，请使用其他空闲位置。
7. 将 autochips.jar 导入 Android Studio 后可看到所要添加的 AtcMetazone.class 和 MetazoneIndex.class。也可将 autochips.jar 包解包，解压后在 autochips/com/autochips/metazone 下看到 AtcMetazone.class，在 autochips/com/autochips/metazone/index 下可看到 MetazoneIndex.class。

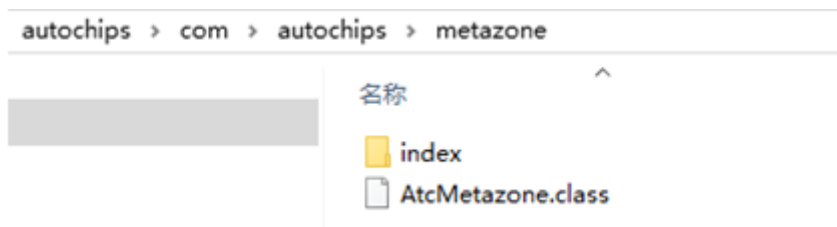


图 33-11 autochips.jar 解压后可看到 AtcMetazone.class



图 33-12 autochips.jar 解压后可看到 MetazoneIndex.class

34 Datazone

34.1 简介

34.1.1 功能

Datazone 用于存放显示屏、开机 Logo 图片、升级图片、RVC 和 BootAnimation 等部分的相关信息，Datazone 数据根据资源属性分别存放在 DatazoneRO 和 DatazoneRW 分区。在系统更新时被写入 eMMC 的 DatazoneRO/DatazoneRW 分区。Datazone 的具体功能如下：

- Safety OS 支持读写 Datazone 管理的各个资源
- LK 支持读写 Datazone 管理的各个资源
- Kernel 支持读写 Datazone 管理的各个资源
- Native 支持读写 Datazone 管理的各个资源
- Framework 支持读写 Datazone 管理的各个资源
- Cluster/Adas 支持读写 Datazone 管理的各个资源

34.1.2 代码结构及文件描述

表 34-1 datazone 代码结构和描述

文件名	文件路径	文件描述
AtcDatazone.java	source/android/vendor/autochips/proprietary/ frameworks/base/datazone/ java/com/autochips/datazone/	提供 framework 层读写 datazone 接 口
datazone_client.h datazone_client.cpp	source/android/vendor/autochips/proprietary/ frameworks/base/datazone/datazonejni	提供 HIDL 用户端 Datazone 接 口
com_autochips_datazone_ AtcDatazone.h AtcDatazone.cpp	source/android/vendor/autochips/proprietary/ frameworks/base/datazone/datazonejni	提供 datazone 在 framework 层的 JNI 接 口

文件名	文件路径	文件描述
Datazone.h Datazone.cpp service.cpp	source/android/vendor/autochips/proprietary/ hardware/datazone/	提供 framework 层 HIDL server 端 API
IDatalogo.hal types.hal	source/android/vendor/autochips/proprietary/ hardware/interfaces/datazone/1.0/	提供 datazone HIDL 接口
datazone.cpp datazone.h datazone_ota.cpp datazone_ota.h	source/android/vendor/autochips/proprietary /native/libs/datazone/	提供 native 层 datazone 的读写接口
datazone_opreation.c datazone.h	source/kernel/common/drivers/soc/ autochips/datazone	提供 datazone kernel driver
datazone.h datazone_ioctl.h	source/kernel/common/drivers/soc/ autochips/include/datazone	定义 datazone 数 据项 index 和 driver 功 能函数
datazone_inter.c	source/vendor/autochips/proprietary/bootable/ lk/platform/autochips/ac8x/	提供 lk 中 datazone 读 写接口
datazone_inter.c	source/vendor/autochips/proprietary/bootable /lk/platform/autochips/ac8x/include	提供 lk 中 datazone 函 数接口和宏
datazone_driver.c	source/vendor/autochips/proprietary/tinysys /hsm/os/drivers/misc	提供 safety metazone 读 写接口
datazone_driver.h	source/vendor/autochips/proprietary/tinysys /hsm/os/include/	提供 safety datazone 函 数接口和宏

34.2 软件设计

34.2.1 架构图

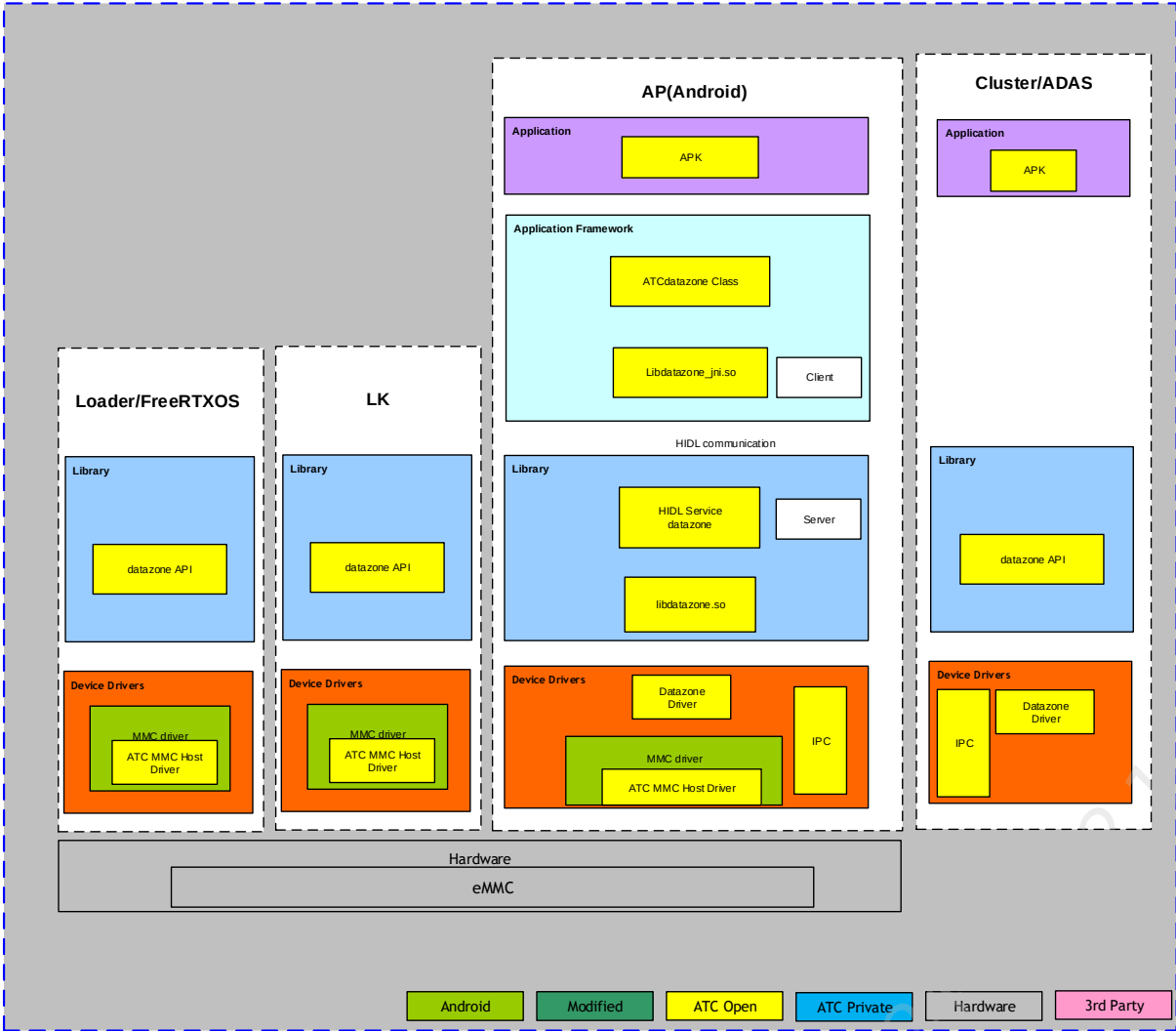


图 34-1 Datazone 系统结构图

34.2.2 Datazone 结构

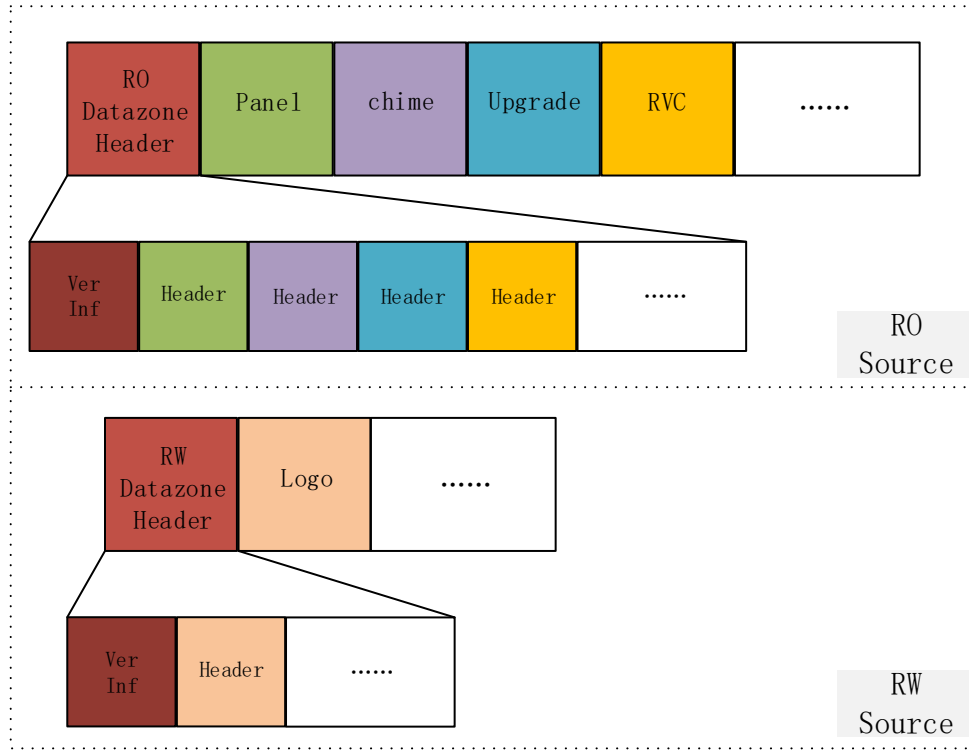


图 34-2 Datazone 结构图

如图所示，DatazoneRO/DatazoneRW 分区数据组织结构是一致的，只是存储的资源类型不同。以 DatazoneRO 分区资源为例介绍，由一个 Datazone_Header 和若干个资源数据区组成，其中 Datazone_Header 包括了一个 ver_info 和对应若干个资源头部（source_header）组成。

目前 DatazoneRW 支持一下几个属性：

- **Backup:** 表示资源可支持恢复出厂设置，注意： Datazone rw source 添加 backup 后，ro 区域会存放其备份。
- **Upgrade:** 表示该 RW 资源可支持升级。

34.2.3 流程图

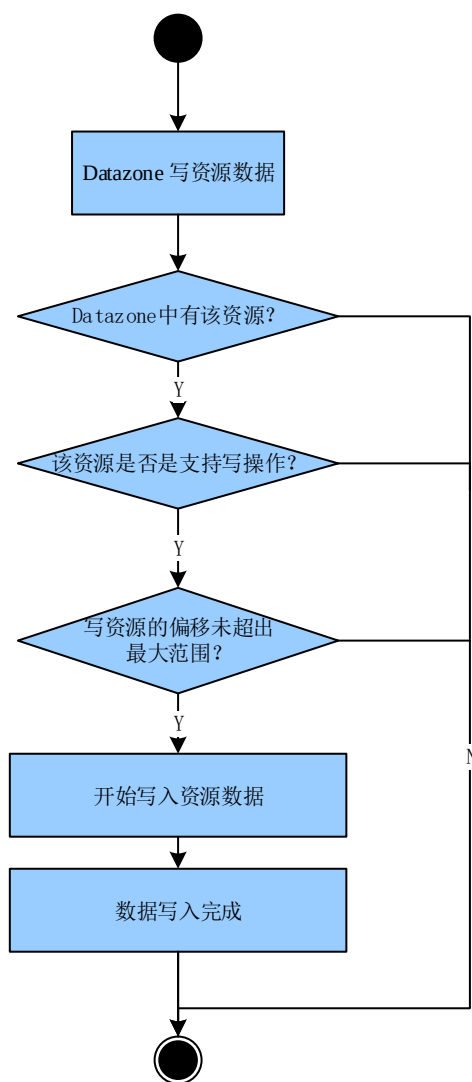


图 34-3 Datazone 写数据流程图

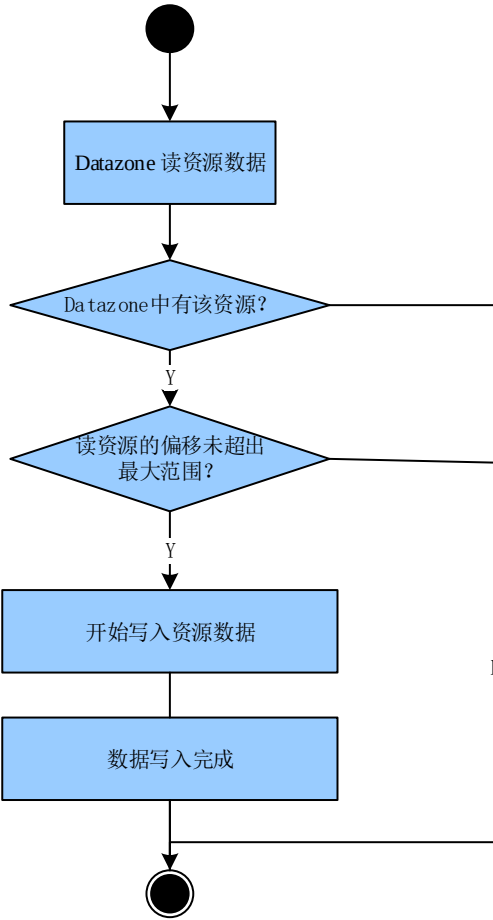


图 34-4 Datazone 读数据流程图

34.3 接口描述

表 34-2 Datazone API 错误码表

API	说明
DZ_ERR_OK	success
DZ_ERR_NO_SOURCE:	fail(no such source)
DZ_ERR_READ	read Datazone from emmc error
DZ_ERR_WRITE	write Datazone to emmc error
DZ_ERR_EXCEED	the size is not match
DZ_ERR_NOTSUPPORT	open Datazone mode is error
DZ_ERR_INVALID	size buffer is null
DZ_ERR_BROKEN	Datazone source is broken
DZ_ERR_INVALID	the size of read/write is error

DZ_ERR_EOF	iterate all the source error
DZ_ERR_BUFFER_SIZE	size is not aligned with 512
DZ_ERR_HL_MAP	translator addr error
DZ_ERR_HL_RSV	buffer size of read is small
DZ_ERR_HL_RSV	get reserved memory error
DZ_ERR_ALIGN	size or offset aligned with 512

34.3.1 Safety os API

表 34-3 Safety os API

API	说明
datazone_get_source_size	Get source size in Datazone
datazone_read_source	Get source in Datazone
datazone_write_source	Write source to Datazone
datazone_read_offset_source	Get source in Datazone according to offset
datazone_check_crc	Check the integrity of datazone source
datazone_update_crc	Update datazone source CRC value

34.3.1.1 datazone_get_source_size

【描述】

获取资源的大小

【语法】

```
int datazone_get_source_size(const char *name, unsigned int *size)
```

【参数】

- name: 资源名称
- size: 读取资源的大小的 buffer

【返回值】

参见表 34-2 datazone API 错误码表

34.3.1.2 datazone_read_source

【描述】

读取资源数据

【语法】

```
int datazone_read_source (const char *name, unsigned char *buffer, u32 buf_size)
```

【参数】

- name: 资源名称
- buf_size: 读取资源的大小
- buffer: 读取资源的 buffer

【返回值】

参见表 34-2 datazone API 错误码表

【注意】

- 该接口会将 source 资源中数据全部读取到 buffer 中，请 buffer 确保有足够的 size
- 该接口是读取整个资源，buf_size 需保证是资源的 size

34.3.1.3 datazone_write_source

【描述】

向 Datazone 中写入资源数据

【语法】

```
int datazone_write_source (const char *name, unsigned char *buffer, u32 buf_size)
```

【参数】

- name: 资源名称
- buf_size: 需要写入资源的大小
- buffer: 需要写入的数据

【返回值】

参见表 34-2 datazone API 错误码表

34.3.1.4 datazone_read_offset_source

【描述】

根据指定的 offset 来读取 Datazone 中的资源数据

【语法】

```
int datazone_read_offset_source (const char *name, u64 offset, u64 size, unsigned char *buffer)
```

【参数】

- name: 资源名称
- offset: 读取数据的偏移
- size: 需要读取资源的大小
- buffer: 读取数据的 buffer

【返回值】

参见表 34-2 datazone API 错误码表

34.3.1.5 datazone_check_crc

【描述】

根据指定的 offset 来检查 Datazone 中的资源数据的完整性

【语法】

```
int datazone_check_crc(const char *name, uint64_t offset, uint64_t size)
```

【参数】

- name: 资源名称
- offset: 需要校验数据的偏移
- size: 需要校验资源数据的大小

【返回值】

参见表 34-2 datazone API 错误码表

34.3.1.6 datazone_update_crc

【描述】

根据指定的 offset 来更新 Datazone 中的资源数据的 CRC 校验值

【语法】

```
int datazone_update_crc(const char *name, uint64_t offset, uint64_t size)
```

【参数】

- name: 资源名称
- offset: 需要更新校验值的数据偏移
- size: 需要计算校验值的数据的大小

【返回值】

参见表 34-2 datazone API 错误码表

【注意】

使能校验属性的资源，在更新资源数据之后是需要调用该接口去 update 资源的 CRC 值，否则会导致资源数据和 CRC 值不匹配，下次 check 资源数据完整性会失败。

34.3.2 LK API

表 34-4 LK API

API	说明
datazone_get_source_size	Get source size in Datazone
datazone_read_buf	Get source in Datazone
datazone_write_buf	Write source to Datazone
datazone_get_source_flags	Get source flag in Datazone
datazone_check_crc	Check the integrity of datazone source
datazone_update_crc	Update datazone source CRC value

34.3.2.1 datazone_get_source_size

【描述】

获取资源的大小

【语法】

```
int datazone_get_source_size(const char *name, unsigned int *size)
```

【参数】

- name: 资源名称
- size: 读取资源的大小的 buffer

【返回值】

参见表 34-2 datazone API 错误码表

34.3.2.2 datazone_read_buf

【描述】

读取资源数据

【语法】

```
int datazone_read_source (const char *name, u64 offset, u64 size, u8 *buffer)
```

【参数】

- name: 资源名称
- offset: 读取资源的偏移
- size: 读取资源的大小
- buffer: 读取资源的 buffer

【返回值】

参见表 34-2 datazone API 错误码表

34.3.2.3 datazone_write_buf

【描述】

向 Datazone 中写入资源数据

【语法】

```
int datazone_write_source (const char *name, u64 offset, u64 size, u8 *buffer)
```

【参数】

- name: 资源名称
- offset: 写资源的起始偏移
- size: 待写资源的大小
- buffer: 需要写入的资源数据

【返回值】

参见表 34-2 datazone API 错误码表

34.3.2.4 datazone_get_source_flags

【描述】

获取资源的 flag 信息

【语法】

```
int datazone_get_source_flags (const char *name, uint32_t *flags)
```

【参数】

- name: 资源名称
- flags: 读取 flags 的值

【返回值】

参见表 34-2 datazone API 错误码表

34.3.2.5 datazone_check_crc**【描述】**

根据指定的 offset 来检查 Datazone 中的资源数据的完整性

【语法】

```
int datazone_check_crc(const char *name, uint64_t offset, uint64_t size)
```

【参数】

- name: 资源名称
- offset: 需要校验数据的偏移
- size: 需要校验资源数据的大小

【返回值】

参见表 34-2 datazone API 错误码表

34.3.2.6 datazone_update_crc**【描述】**

根据指定的 offset 来更新 Datazone 中的资源数据的 CRC 校验值

【语法】

```
int datazone_update_crc(const char *name, uint64_t offset, uint64_t size)
```

【参数】

- name: 资源名称
- offset: 需要更新校验值的数据偏移
- size: 需要计算校验值的数据的大小

【返回值】

参见表 34-2 datazone API 错误码表

【注意】

添加校验属性的资源，在更新资源数据之后，需要调用该接口去 update 资源的 CRC 值，否则会导致资源数据和 CRC 值不匹配，下次 check 资源数据完整性会失败。

34.3.3 Kernel API



说明：

adas/custer/ivi 的 kernel 接口是复用的，以下接口的说明适用 adas/custer/ivi 。

表 34-5 Kernel API

API	说明
datazone_get_size	Get source size in Datazone
datazone_read_buf	Get source in Datazone
datazone_write_buf	Write source to Datazone
datazone_get_flags	Get source flag in Datazone
check_dtz_crc_kernel	Check the integrity of datazone source
update_dtz_crc_kernel	Update datazone source CRC value

34.3.3.1 datazone_get_size

【描述】

获取资源的大小

【语法】

```
int datazone_get_size(const char *name, uint64_t *size)
```

【参数】

- name: 资源名称
- size: 读取资源的大小

【返回值】

参见表 34-2 datazone API 错误码表

34.3.3.2 datazone_read_buf

【描述】

读取资源数据

【语法】

```
ssize_t datazone_read_buf (const char *name, uint64_t offset, uint64_t size, uint8_t *buffer)
```

【参数】

- name: 资源名称
- offset: 读取资源的偏移
- size: 读取资源的大小
- buffer: 读取资源的 buffer

【返回值】

参见表 34-2 datazone API 错误码表

34.3.3.3 datazone_write_buf**【描述】**

向 Datazone 中写入资源数据

【语法】

```
int datazone_write_source (const char *name, uint64_t offset, uint64_t size, uint8_t *buffer)
```

【参数】

- name: 资源名称
- offset: 写资源的起始偏移
- size: 待写资源的大小
- buffer: 需要写入的资源数据

【返回值】

参见表 34-2 datazone API 错误码表

34.3.3.4 datazone_get_flag**【描述】**

获取资源的 flag 信息

【语法】

```
int datazone_get_flags (const char *name, uint32_t *flags)
```

【参数】

- name: 资源名称
- flags: 读取 flags 的值

【返回值】

参见表 34-2 datazone API 错误码表

34.3.3.5 check_dtz_crc_kernel**【描述】**

根据指定的 offset 来检查 Datazone 中的资源数据的完整性

【语法】

```
int check_dtz_crc_kernel (const char *name, uint64_t offset, int size)
```

【参数】

- name: 资源名称
- offset: 需要校验数据的偏移
- size: 需要校验资源数据的大小

【返回值】

参见表 34-2 datazone API 错误码表

34.3.3.6 update_dtz_crc_kernel**【描述】**

根据指定的 offset 来更新 Datazone 中的资源数据的 CRC 校验值

【语法】

```
int update_dtz_crc_kernel (const char *name, uint64_t offset, int size)
```

【参数】

- name: 资源名称
- offset: 需要更新校验值的数据偏移
- size: 需要计算校验值的数据的大小

【返回值】

参见表 34-2 datazone API 错误码表

【注意】

添加校验属性的资源，在更新资源数据之后是需要调用该接口去 update 资源的 CRC 值，否则会导致资源数据和 CRC 值不匹配，下次 check 资源数据完整性会失败。

34.3.4 Native API



说明：

adas/cluster/ivi 的 native 接口是复用的，以下接口的说明适用 adas/custer/ivi 。

表 34-6 Native API

API	说明
datazone_get_size	Get source size in Datazone
datazone_read_buf	Get source in Datazone
datazone_write_buf	Write source to Datazone
datazone_get_flags	Get source flag in Datazone
datazone_truncate_source	Set source real size
datazone_source_realsize	get source real size
open_datazone	get source info
datazone_factory_reset	reset all Datazone source
datazone_factory_reset_source	reset spedified Datazone dource
datazone_check_crc	Check the integrity of datazone source
datazone_update_crc	Update datazone source CRC value

34.3.4.1 datazone_get_size

【描述】

获取资源的大小

【语法】

```
int datazone_get_size(cstruct datazone_handler *handler, uint64_t *size)
```

【参数】

- handler: 资源的基本信息，如 flag, name, size, Datazone 节点的 fd 等信息
- size: 读取资源的大小

【返回值】

参见表 34-2 datazone API 错误码表

34.3.4.2 datazone_read_buf

【描述】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

987 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

读取资源数据

【语法】

```
ssize_t datazone_read_buf (struct datazone_handler *handler, uint64_t offset, uint64_t size,  
uint8_t *buffer)
```

【参数】

- handler: 资源的基本信息，如 flag, name, size, Datazone 节点的 fd 等信息
- offset: 读取资源的偏移
- size: 读取资源的大小
- buffer: 读取资源的 buffer

【返回值】

参见表 34-2 datazone API 错误码表

34.3.4.3 datazone_write_buf

【描述】

向 Datazone 中写入资源数据

【语法】

```
int datazone_write_source (struct datazone_handler *handler, uint64_t offset, uint64_t size,  
uint8_t *buffer)
```

【参数】

- handler: 资源的基本信息，如 flag, name, size, Datazone 节点的 fd 等信息
- offset: 写资源的起始偏移
- size: 待写资源的大小
- buffer: 需要写入的资源数据

【返回值】

参见表 34-2 datazone API 错误码表

34.3.4.4 datazone_get_flag

【描述】

获取资源的 flag 信息

【语法】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

988 /1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露


```
int datazone_get_flags (struct datazone_handler *handler, uint32_t *flags)
```

【参数】

- handler: 资源的基本信息，如 flag, name, size, Datazone 节点的 fd 等信息
- flags: 读取 flags 的值

【返回值】

参见表 34-2 datazone API 错误码表

34.3.4.5 datazone_truncate_source

【描述】

设置资源的 size

【语法】

```
int datazone_truncate_source (struct datazone_handler *handler, uint64_t realsize)
```

【参数】

- handler: 资源的基本信息，如 flag, name, size, Datazone 节点的 fd 等信息
- realsize: 资源需要设置的大小

【返回值】

参见表 34-2 datazone API 错误码表

34.3.4.6 datazone_source_realsize

【描述】

获取资源的 size

【语法】

```
int datazone_source_realsize (struct datazone_handler *handler, uint64_t *realsize)
```

【参数】

- handler: 资源的基本信息，如 flag, name, size, Datazone 节点的 fd 等信息
- realsize: 获取资源的实际大小的 buffer

【返回值】

参见表 34-2 datazone API 错误码表

34.3.4.7 open_datazone

【描述】

获取资源的 size

【语法】

```
struct datazone_handler *open_datazone (const char *name, enum datazone_open_mode mode)
```

【参数】

- name: 资源的名字
- mode: 资源打开的模式，主要有以下几种：
 - DATAZONE_READ_ONLY: 以只读模式 open Datazone
 - DATAZONE_WRITABLE: 打开可读写模式 open Datazone
 - DATAZONE_TRUNC_SOURCE: 以设置资源大小的模式 open Datazone

【返回值】

参见表 34-2 datazone API 错误码表

34.3.4.8 datazone_factory_reset

【描述】

处理 Datazone 分区中所有支持 reset 的 source

【语法】

```
int datazone_factory_reset (void)
```

【参数】

none

【返回值】

参见表 34-2 datazone API 错误码表

34.3.4.9 datazone_factory_reset_source

【描述】

reset Datazone 分区中指定的 source

【语法】

```
int datazone_factory_reset_source (const char *source)
```

【参数】

source: 资源名字

【返回值】

参见表 34-2 datazone API 错误码表

34.3.4.10 datazone_check_crc**【描述】**

根据指定的 offset 来检查 Datazone 中的资源数据的完整性

【语法】

```
int datazone_check_crc(struct datazone_handler *handler, uint64_t offset, uint64_t size)
```

【参数】

- name: 资源名称
- offset: 需要校验数据的偏移
- size: 需要校验资源数据的大小

【返回值】

参见表 34-2 datazone API 错误码表

34.3.4.11 datazone_update_crc**【描述】**

根据指定的 offset 来更新 Datazone 中的资源数据的 CRC 校验值

【语法】

```
int datazone_update_crc(c struct datazone_handler *handler, uint64_t offset, uint64_t size)
```

【参数】

- name: 资源名称
- offset: 需要更新校验值的数据偏移
- size: 需要计算校验值的数据的大小

【返回值】

参见表 34-2 datazone API 错误码表

【注意】

添加校验属性的资源，在更新资源数据之后是需要调用该接口去 update 资源的 CRC 值，否则会导致资源数据和 CRC 值不匹配，下次 check 资源数据完整性会失败。

34.3.5 Framework API

表 34-7 Framework API

API	说明
getsize	Get source size in Datazone
readbuf	Get source in Datazone
writebuf	Write source to Datazone
getflags	Get source flag in Datazone
truncatesource	Set source real size
getrealsize	get source real size

34.3.5.1 getsize

【描述】

获取 Datazone source 的大小

【语法】

```
public native static int getsize(string name, int mode, int[] size)
```

【参数】

- name: 资源的名字
- mode: open 资源的模式（如以读写、只读的方式等）
- offset: 读取 Datazone source 的起始地址
- size: 获取到资源的最大 size

【返回值】

参见表 34-2 datazone API 错误码表

34.3.5.2 readbuf

【描述】

读取 Datazone source

【语法】

```
public native static int readbuf(string name, int mode, int offset, int size, byte[] buf)
```

【参数】

- name: 资源的名字
- mode: open 资源的模式（如以读写、只读的方式等）
- offset: 读取 Datazone source 的起始地址
- size: 读取资源的大小
- buf: 读取资源的 buffer

【返回值】

参见表 34-2 datazone API 错误码表

34.3.5.3 writebuf

【描述】

写 Datazone source 的数据

【语法】

```
public native static int write buf(string name, int mode, int offset, int size, byte[] buf)
```

【参数】

- name: 资源的名字
- mode: open 资源的模式（如以读写、只读的方式等）
- offset: 读取 Satazone source 的起始地址
- size: 读取资源的大小
- buf: 需要写入的资源数据

【返回值】

参见表 34-2 datazone API 错误码表

34.3.5.4 getflags

【描述】

获取 Datazone source 的 flag 信息

【语法】

```
public native static int getflags(string name, int mode, int [] flags)
```

【参数】

- **name:** 资源的名字
- **mode:** open 资源的模式（如以读写、只读的方式等）
- **flags:** 获取的资源的 flag 信息

【返回值】

参见表 34-2 datazone API 错误码表

34.3.5.5 truncatesource

【描述】

设置 Datazone source 的大小

【语法】

```
public native static int truncatesource(String name, int mode, int realsize)
```

【参数】

- **name:** 资源的名字
- **mode:** open 资源的模式（如以读写、只读的方式等）
- **realsize:** 需要设置资源的 size 的值

【返回值】

参见表 34-2 datazone API 错误码表

34.3.5.6 getrealsize

【描述】

获取 Datazone source 的实际大小

【语法】

```
public native static int getrealsize(String name, int mode, int[] realsize)
```

【参数】

- **name:** 资源的名字
- **mode:** open 资源的模式（如以读写、只读的方式等）
- **realsize:** 获取的资源实际大小的值

【返回值】

参见表 34-2 datazone API 错误码表



说明：

与原 JNI 封装方式相比，唯一变化的地方是，AtcDatazone 类由实例化类变为静态类，Java 层调用 Datazone API 时，不需要再实例化对象

34.4 使用说明和注意事项

34.4.1 各层 Datazone 接口使用注意

为了防止 Datazone reserved memory 中的数据被毁导致其他模块功能异常。建议各模块在 code 逻辑中读写 Datazone fail 的时候，根据各模块特性给定合适的默认值。这样即使 Datazone 使用异常，也能保证各模块功能正常。

34.4.2 java 层使用 Datazone

- 为了显示、逻辑、数据分离，建议显示相关在 Activity 中实现、逻辑与数据处理在 Service 中实现，可参考 Datazone 的实现。代码路径：

```
source/android/vendor/autochips/proprietary/framework/base/datazone/java/com/autochips/datazone
```

- 若使用 Datazone 功能，需要导入 Datazone 相关的接口与类：

```
import com.autochips.datazone.AtcDatazone;
```

- 使用案例

```
import com.autochips.datazone.AtcDatazone;

int mode = AtcDatazone.datazonewriteable;
int rt = 0;
byte[] buf = new byte[10];
int size = 100;
int flags = 0;
int realsize = 0;

rt = AtcDatazone.readbuf("test1", mode, 0, size, buf); //读 test1 的数据
rt = AtcDatazone.writebuf("test1", mode, 0, size, buf); //向 test1 中写数据
rt = AtcDatazone.getsize("test1", mode, &size); //获取 test1 的 size
rt = AtcDatazone.getflags("test1", mode, &flags); // 获取 test1 的 flags
```

```
rt = AtcDatazone.truncatesource("test1", mode, &realsize); //设置 test1 的大小
rt = AtcDatazone.getrealsize("test1", mode, &realsize); //获取 test1 的实际大小
```

34.4.3 Native 层使用 datazone

- 调用 open_datazone;
- 调用 Read/Write 接口读写 Datazone 的数据;

Native 进程访问 Datazone 需要配置相应 SELinux 权限，具体方法是在本模块的 .te 文件中(一般位于 source/android/device/autochips/ac8x/sepolicy/non_plat/)，添加如下 3 条规则，可参考 /hal_metalogo.te 文件：

- allow 模块名 datazone_device:chr_file {open read write ioctl};
- allow 模块名 block_device:dir {search};
- allow 模块名 datazone_block_device:blk_file {open read write};

```
#include<datazone.h>
.....
int mode = DATAZONE_WRITABLE;
int rt = 0;
char buf[100];
int size = 100;
int flags = 0;
int realsize = 0;
staruct datazone_handler *handler;
handler = open("test1", mode);
rt = datazone_read_buf(handler, mode, 0 , size, buf); //读 test1 的数据
rt = datazone_write_buf(handler, mode, 0 , size, buf); //向 test1 中写数据
rt = datazone_get_size(handler, mode, &size); //获取 test1 的 size
rt = datazone_get_flags(handler, mode, &flags); // 获取 test1 的 flags
rt = datazone_truncate_source(handler, mode, &realsize); //设置 test1 的大小
rt = datazone_get_realsize(handler, mode, &realsize); //获取 test1 的实际大小
close_datazone(handler);
```


34.4.4 LK 中使用 Datazone

与 native 层使用方法相似，但不需要调用 `open_datazone()/close_datazone()`，直接使用 LK 中相关 APIs（APIs 的名称有区别），具体使用可参考 native 的 sample code 使用方式。

注意事项：

- 需要 include header file，其位置：

`source/vendor/autochips/proprietary/bootable/lk/platform/autochips/$(PROJECT)/include/platform/datazone/datazone_inter.h`

- 在 LK 中使用时，需在头文件或者.c 文件 include `<datazone/datazone_inter.h>`。
- LK 整体编译成一个 image，注意：在 LK 中使用 Datazone 读写接口，需要保证在 lk 启动流程中执行 `datazone_init` 成功后。如果 LK 执行过程中未执行 `datazone_init`，可以自行添加。

34.4.5 Safety OS 中使用 Datazone

与 Native 层使用相似，但不需要 `open_datazone()/close_datazone()`，直接使用 Safety OS 中相关 APIs（APIs 的名称有区别），具体使用可参考 native 的 sample code 使用方式。

- 需要 include header file，APIs 的 header file 在 `source/android/vendor/autochips/proprietary/tinysys/hsm/include/datazone_driver.h`
- 在 Safety OS 中使用时，需在头文件或者.c 文件 include `<datazone_driver.h>`
- Safety OS 整体编译成一个 image，注意：在 Safety OS 中使用 Datazone 接口，需要保证在 Safety os 启动流程中执行 `datazone_init` 成功之后。

34.4.6 Kernel Driver 中使用 Datazone

与 Native 层的使用相似，但不需要 `open_datazone()/close_datazone()`，直接使用相关 APIs，可参考 native 的 sample code 使用方式。

- APIs 的 header file 在 `source/kernel/common/drivers/soc/autochips/include/datazone/datazone.h`
- Datazone driver 静态编译进内核，在 Kernel 层 API 通过 `EXPORT_SYMBOL` 方式提供给内核其它模块使用。
- 同时需要注意，在 Kernel 中使用 datazone API，请不要直接在 kernel 启动的 flow 中调用该接口需要，并且保证 init 进程启动后以及 `datazone_drv.ko insmod` 后开始调用 Datazone 相关接口。

34.4.7 通过 adb debug Datazone

通过执行命令查看 Datazone 的 source 情况

```
adb root
adb shell
mkdir mnt/datazone
mount -t datazone_fs none /mnt/datazone
ls /mnt/datazone
```

如查看 Datazone 中是否有资源: atc_volume。

```
ac8x_demo:/ # mkdir /mnt/datazone
ac8x_demo:/ # mount -t datazone_fs none /mnt/datazone
ac8x_demo:/ # cd mnt/datazone/
ac8x_demo:/mnt/datazone # ls -al
total 0
drwxr-xr-x  8 root root    0 2022-05-12 15:30 .
drwxr-xr-x 16 root system 340 2022-05-12 15:30 ..
dr-x----- 6 root root    0 2022-05-12 15:30 atc_volume
dr-x----- 6 root root    0 2022-05-12 15:30 test1
dr-x----- 6 root root    0 2022-05-12 15:30 test2
ac8x_demo:/mnt/datazone # cat atc_volume/
cat: atc_volume/: Is a directory
l|ac8x_demo:/mnt/datazone # cd atc_volume/
ac8x_demo:/mnt/datazone/atc_volume # ls
data  flags  offset  size
ac8x_demo:/mnt/datazone/atc_volume #
```

图 34-5 Datazone adb 查看所有资源

然后把对应资源 dump 出来，然后进行查看：

```
ac8x_demo:/mnt/datazone/atc_volume # dd if=data of=/mnt/atc_volume bs=1024 count=1
+0 records in
+0 records out
1024 bytes (1.0 K) copied, 0.002859 s, 350 K/s
ac8x_demo:/mnt/datazone/atc_volume # cat /mnt/atc_volume
adas      :0x10000
yba       :0x10000
rvc       :131072
safety    :0x10000
vehicle   :0x10000
```

图 34-6 Datazone adb 查看指定资源

34.4.8 Datazone 资源添加

1. 添加需要的资源文件：在/build/ projects/ <PROJECT_NAME>/ <DEVICE_NAME>/ datazone/路径下添加资源文件；
2. 进行资源配置：在 datazone_config.json 添加资源，按照如图 34-7 格式进行配置。

```
"test": {
  "attribute": "rw,verify",
  "file": "test.txt",
  "size": "1024"
},
"test2": {
  "attribute": "backup,rw",
  "file": "test.txt",
  "size": "1024"
}
```

图 34-7 datazone_config.json 中资源添加

其中：

- name: 资源名字；
- size: 资源大小（size 需要 512 对齐）；
- attribute: 资源属性。其中 ro / rw 为必须属性，对于 ro 的资源将会打包 datazone_ro.img 中，对于 rw 的资源将会打包在 datazone_rw.img 中。注意：datazone 的属性目前支持 ro、rw、backup（主要是设置资源的默认值，便于恢复出厂设置）、upgrade（支持升级）、verfiy（支持资源校验）；
- file: 资源文件（默认指向路径：根目录/build/projects/<PROJECT_NAME>/datazone/）。

34.4.9 Datazone CRC 校验

- 在 datazone_config.json 文件中添加 verify 字段（参考 34.4.8 章节说明）。
- 支持资源数据完整性校验的资源，通常是适用于对安全性要求较高的资源数据，在资源校验的过程中，如果资源较大，不建议使用校验，校验过程会耗时。
- MPTool 中 datazone tool 的 V1.2023_02_06 版本支持添加 verify 字段；该版本之前的版本均不支持，请注意 tool 的使用版本。Datazone tool 的版本检查：可通过执行 datazone tool 后的 log（路径：/MPtool/logs/）中查看。

34.5 典型问题

- Datazone 资源分为两种，Datazone ro 和 rw，ro 是不支持通过接口去修改数据的；
- 通过接口去访问资源名时，该资源名指的是在 datazone_config.json 中指定的文件名，而不是资源文件的名字。

35 USB

35.1 简介

35.1.1 软硬件环境

35.1.1.1 软件环境

AC8025 USB OTG 接口支持 host 和 device mode，支持最高 usb2.0 模式并且支持 host/device 的软件动态切换功能，Host30 接口仅支持 host 模式，最高支持 usb3.0 模式。

35.1.1.2 硬件环境

AC8025 支持两个 USB port，分别是 OTG 端口和 Host30 端口。

- USB OTG 使用的硬件电路连接示意图如下：

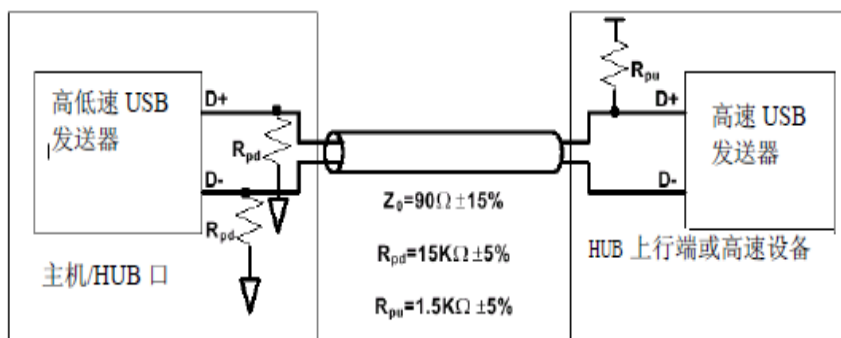


图 35-1 高速设备连接电路示意图

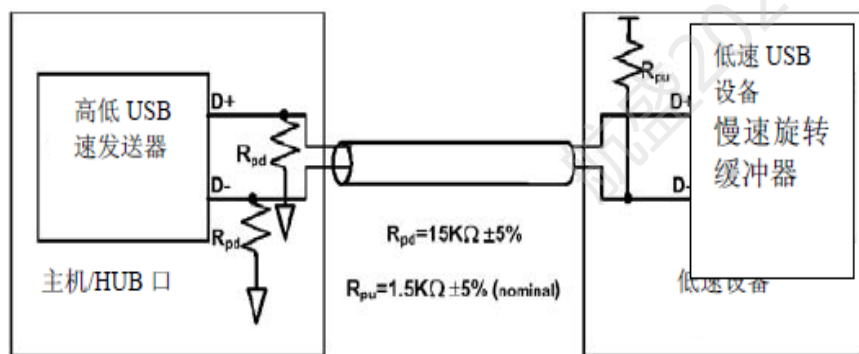


图 35-2 低速设备连接电路示意图

- Host30 使用的硬件电路连接示意图如下：

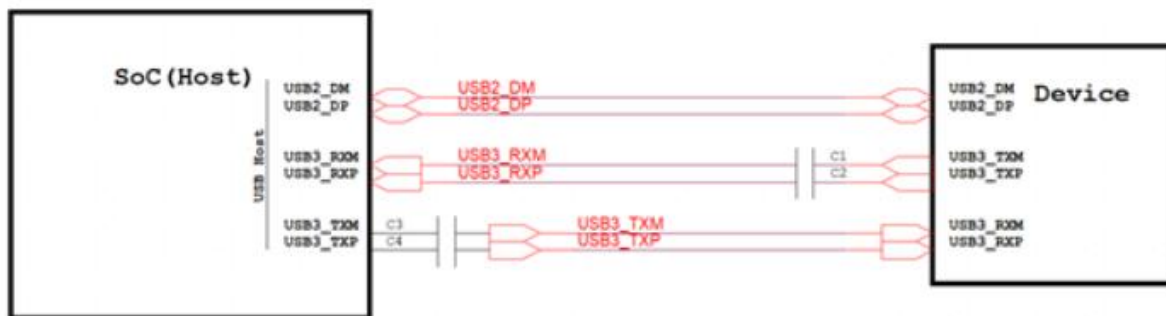


图 35-3 USB3.0 连接电路示意图

- CLK

用于配置 USB 的 clk source

- Uart

用于输出 debug log

- Power

用于输出 enable/disable USB Controller power

35.1.2 功能

USB OTG 口具有如下功能：

- usb 2.0 FS/HS
- 支持 host/device 切换
- 支持两级 HUB
- 支持 BC1.2
- 最多可同时连接 16 个设备

USB Host30 接口具有如下功能：

- 支持 usb3.0 SS 传输
- 支持 usb 2.0 FS/HS 传输
- Host only
- 支持 2 级 HUB
- 最多可同时连接 16 个设备

35.1.3 代码结构及文件描述

表 35-1 USB OTG/ USB Host30 Controller 代码结构及描述

文件名	文件路径	文件描述
cdns3-autochips.c	source/kernel/common/drivers/usb/cdns3	USB OTG/HOST reset/clock/phy 初始化
core.c	source/kernel/common/drivers/usb/cdns3	控制器初始化
gadget.c ep0.c	source/kernel/common/drivers/usb/cdns3	usb device 相关驱动代码
drd.c	source/kernel/common/drivers/usb/cdns3	Host/device 切换相关代码
host.c	source/kernel/common/drivers/usb/cdns3	usb host 相关驱动代码
xhci.c xhci-ring.c xhci-mem.c xhci-plat.c xhci-hub.c	source/kernel/common/drivers/usb/host	usb host xhci 相关驱动代码

表 35-2 USB 设备树配置

文件名	文件路径	文件描述
ac802x.dtsi	source/kernel/common/arch/arm64/boot/dts/autochips	包含 usb 设备描述，配置 usb 中断，clock 等信息，详细的可配置参数将在 usb 设备中描述

35.2 软件设计

35.2.1 OTG 架构图

35.2.1.1 OTG host 模式架构

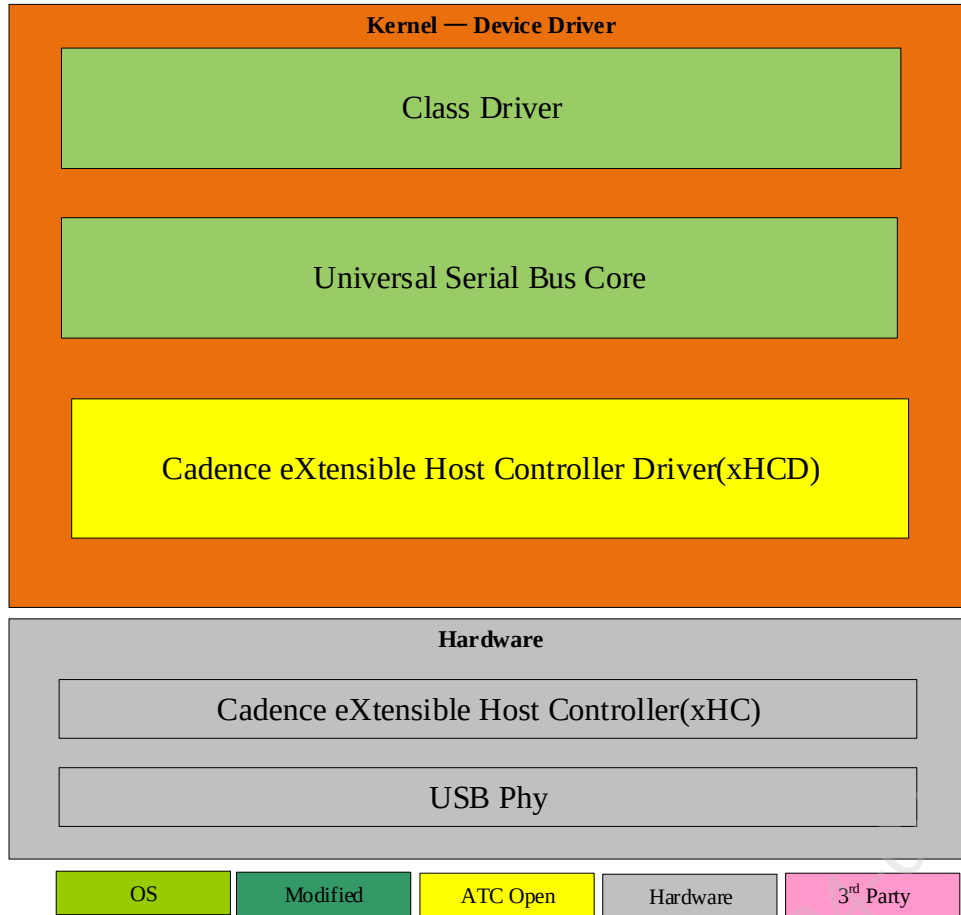


图 35-4 USB OTG host 模式架构

35.2.1.2 OTG host 模式数据流

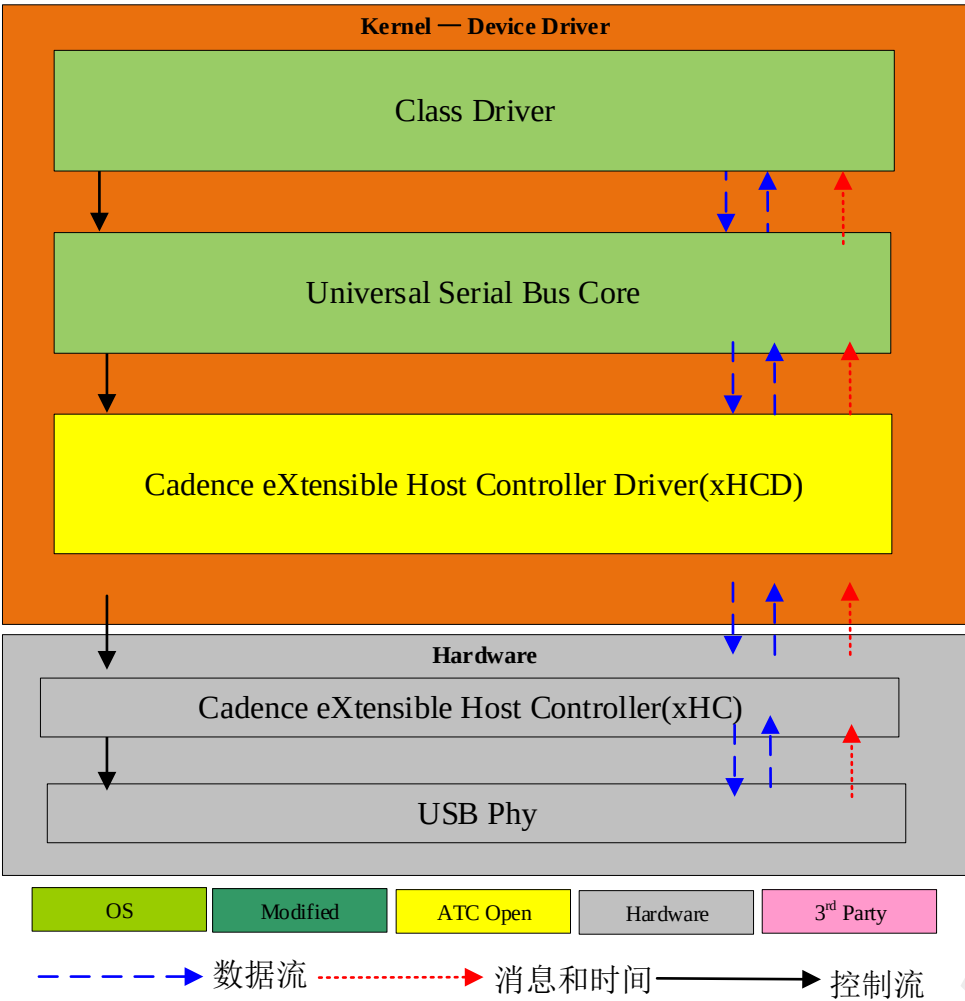


图 35-5 USB OTG host 数据流

35.2.1.3 OTG device 模式架构

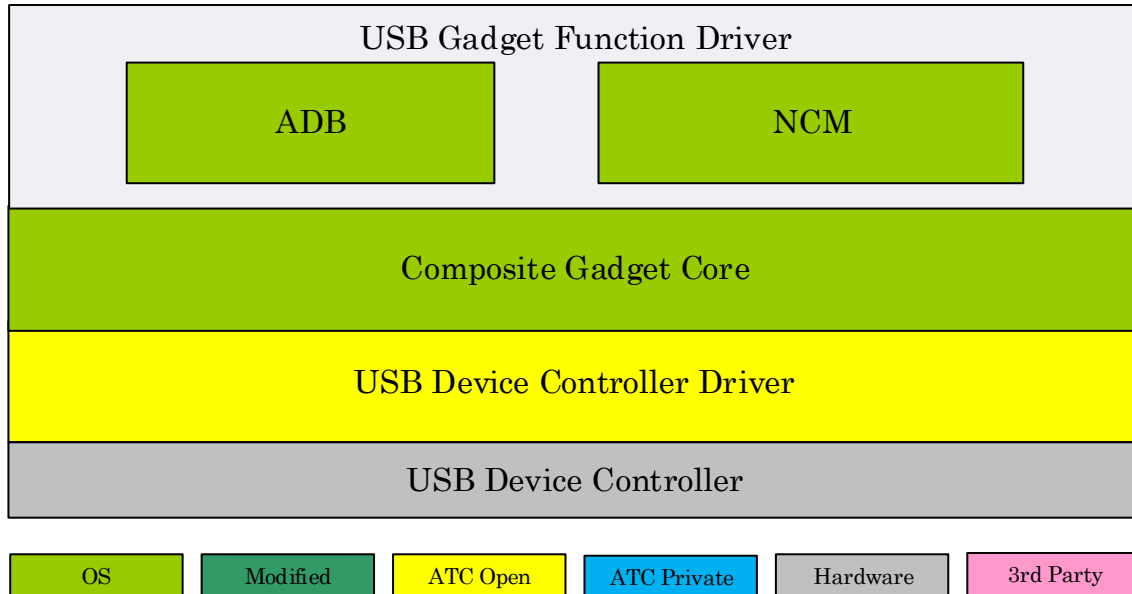


图 35-6 USB OTG device 架构

AC8025 OTG 支持 device mode，可以把 USB 根据需求配置为 ADB(Android Debug Bridge)。驱动架构和 Host 类似，也是分为三层：

- **USB Gadget Function Driver:** 包含具体的功能驱动，如 ADB & NCM 等，实现具体功能以及传输协议
- **Composite Gadget Core:** 完成被 Host 枚举的动作以及 USB 总线协议，其中 ADB 以及 Composite Gadget Core 层都是 kernel 原生的 code
- **USB Device Controller Driver:** 和硬件紧密相关，控制 USB Controller 工作，包括硬件初始化，注册设备，提供接口函数，响应 USB 中断等

35.2.1.4 OTG device 模式数据流

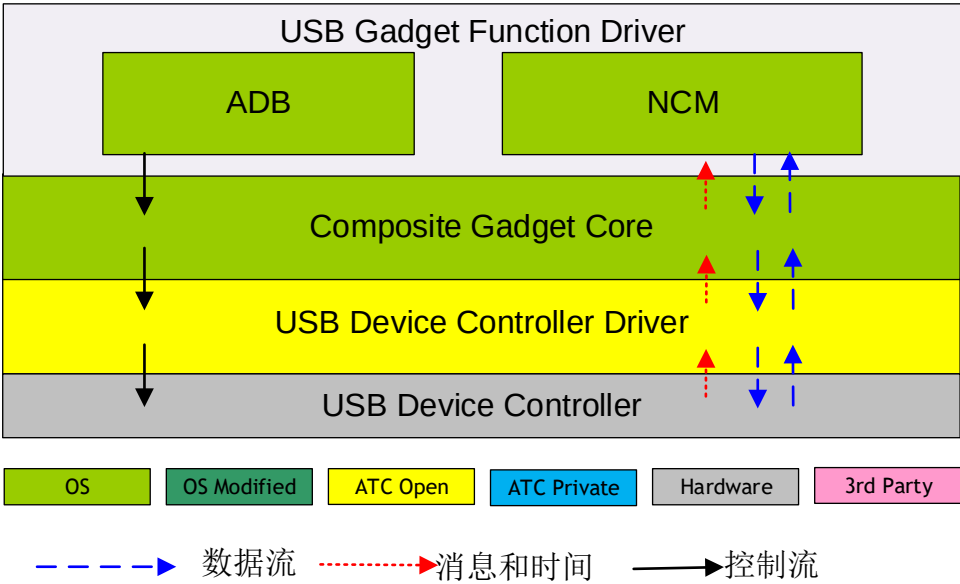


图 35-7 USB OTG device 数据流

35.2.2 Host30 架构图

35.2.2.1 USB Host 30 模式架构

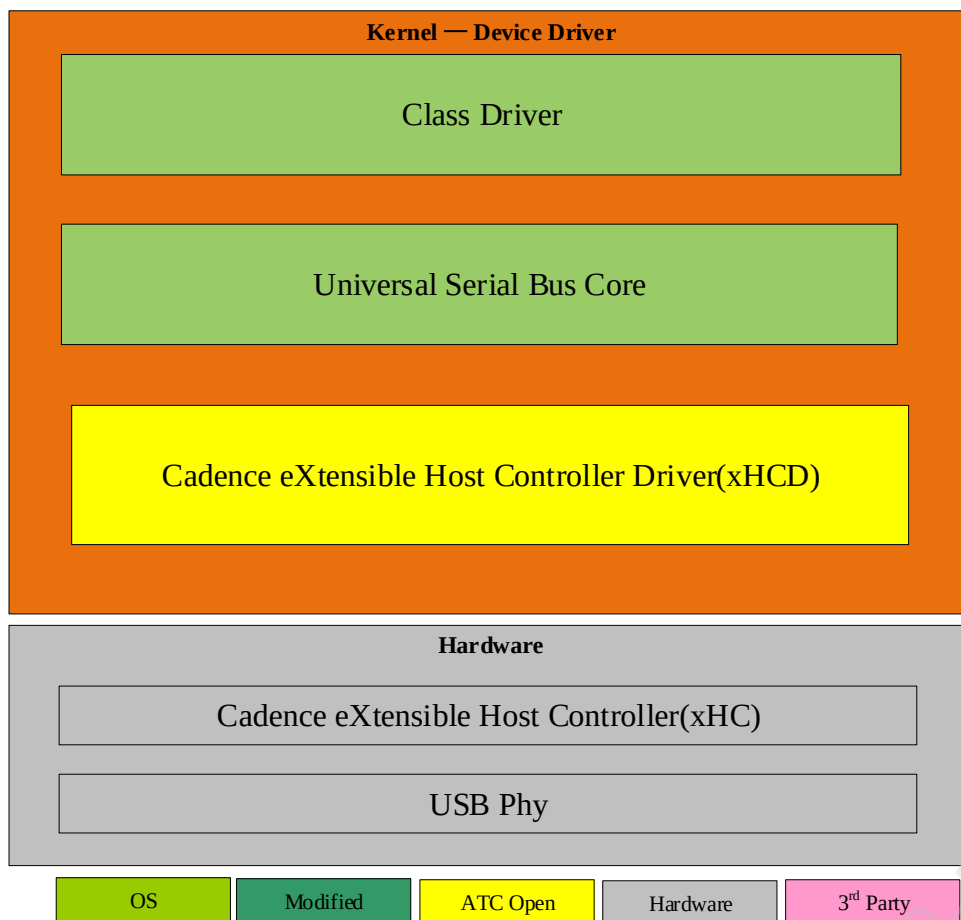


图 35-8 USB Host 30 模式架构

35.2.2.2 USB Host 30 模式数据流

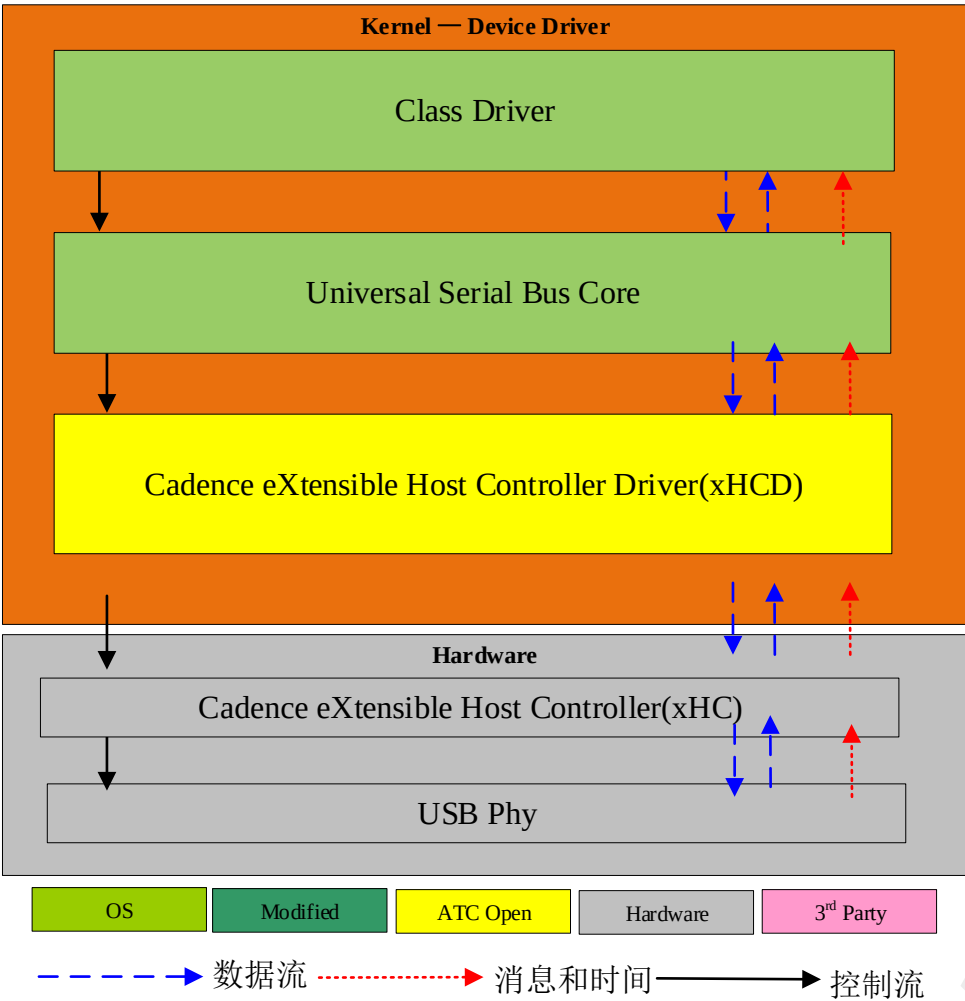


图 35-9 USB Host 30 模式数据流

35.3 使用说明和注意事项

35.3.1 设置 USB OTG host/device 模式切换

本章提供三种方式切换 USB mode，具体如下：

1. APP 切换

用户可以直接通过：Car Settings->ATC Settings->Other->usb mode，对 usb OTG 口的 USB mode 进行设置。如下图所示：

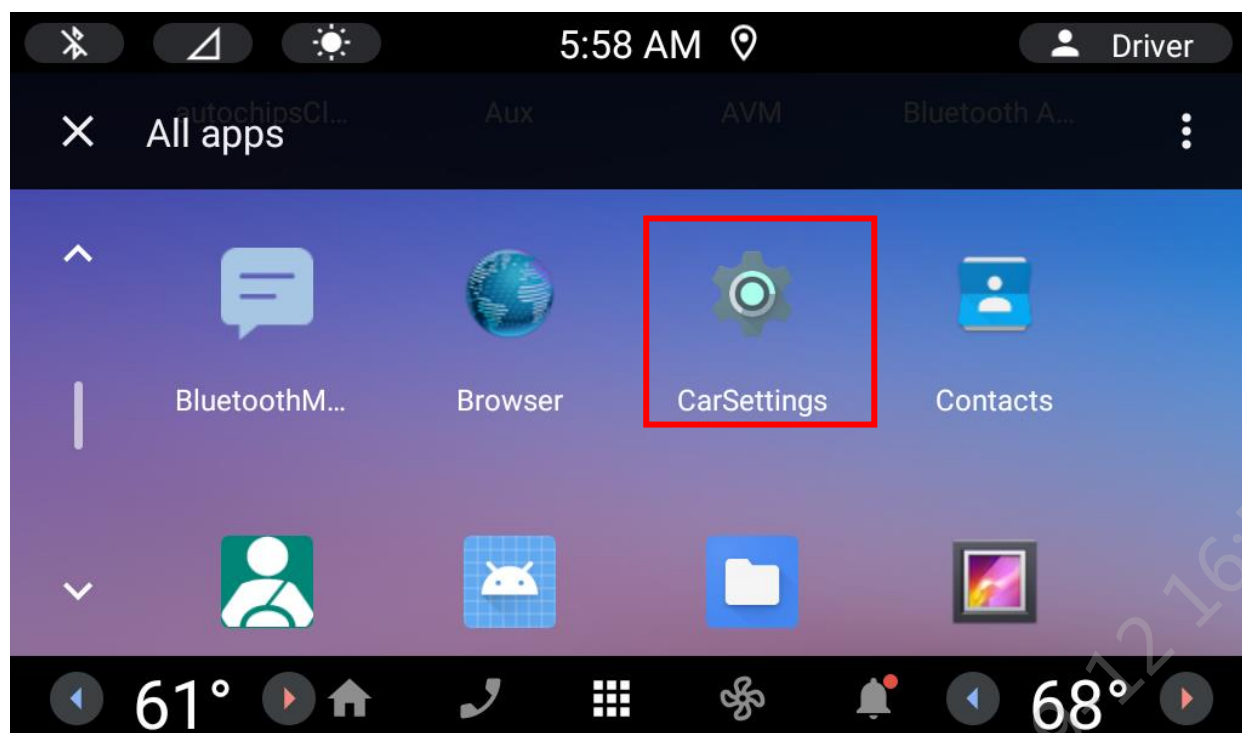


图 35-10 Car Settings

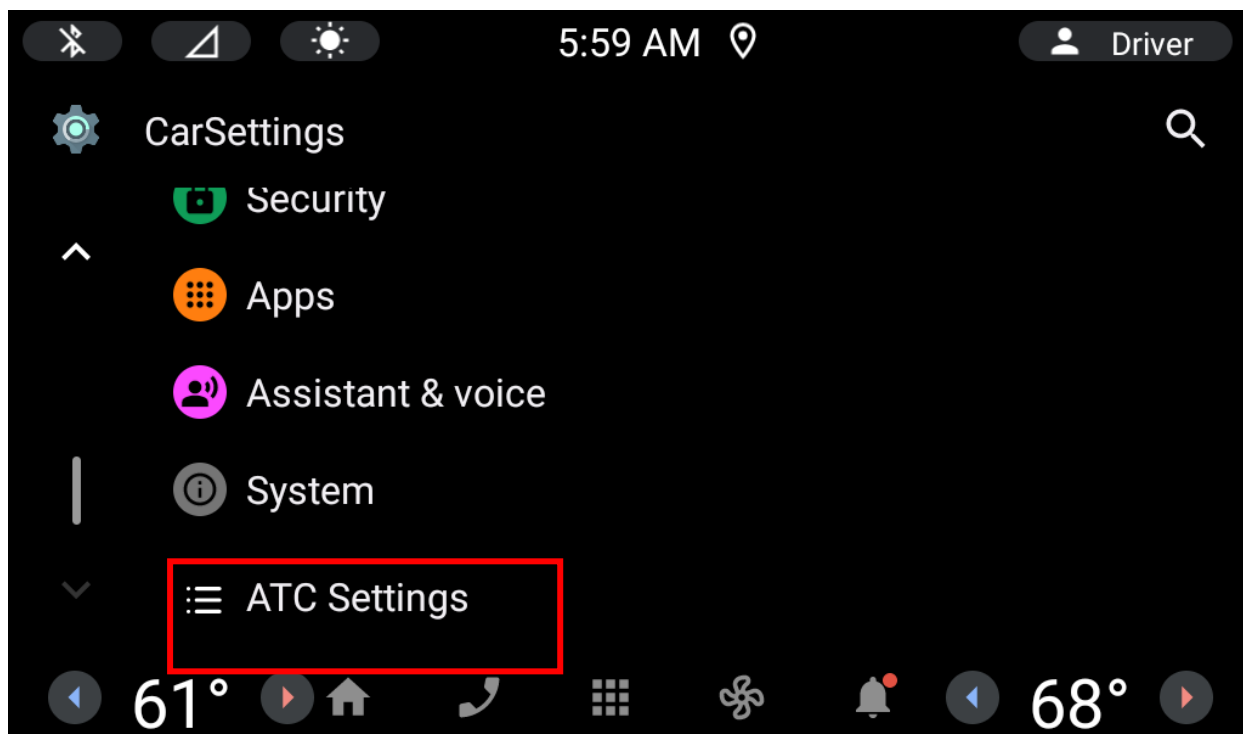


图 35-11 ATC Settings

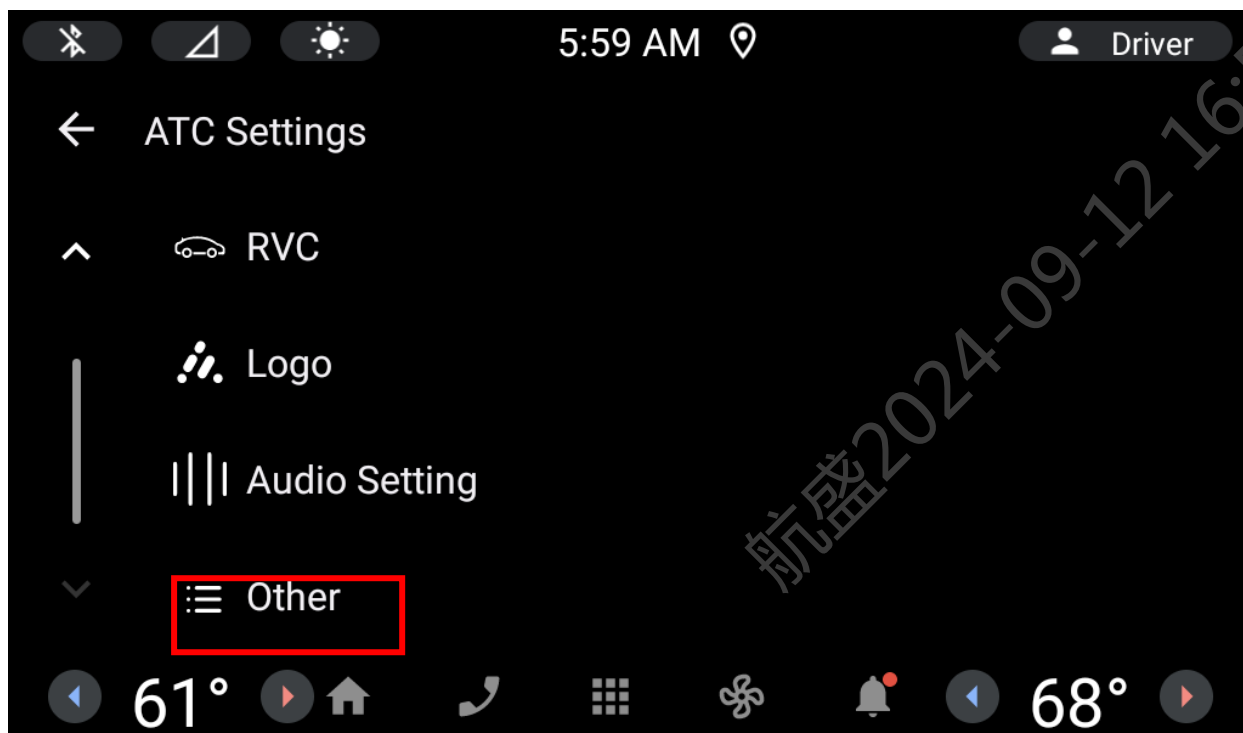


图 35-12 Other

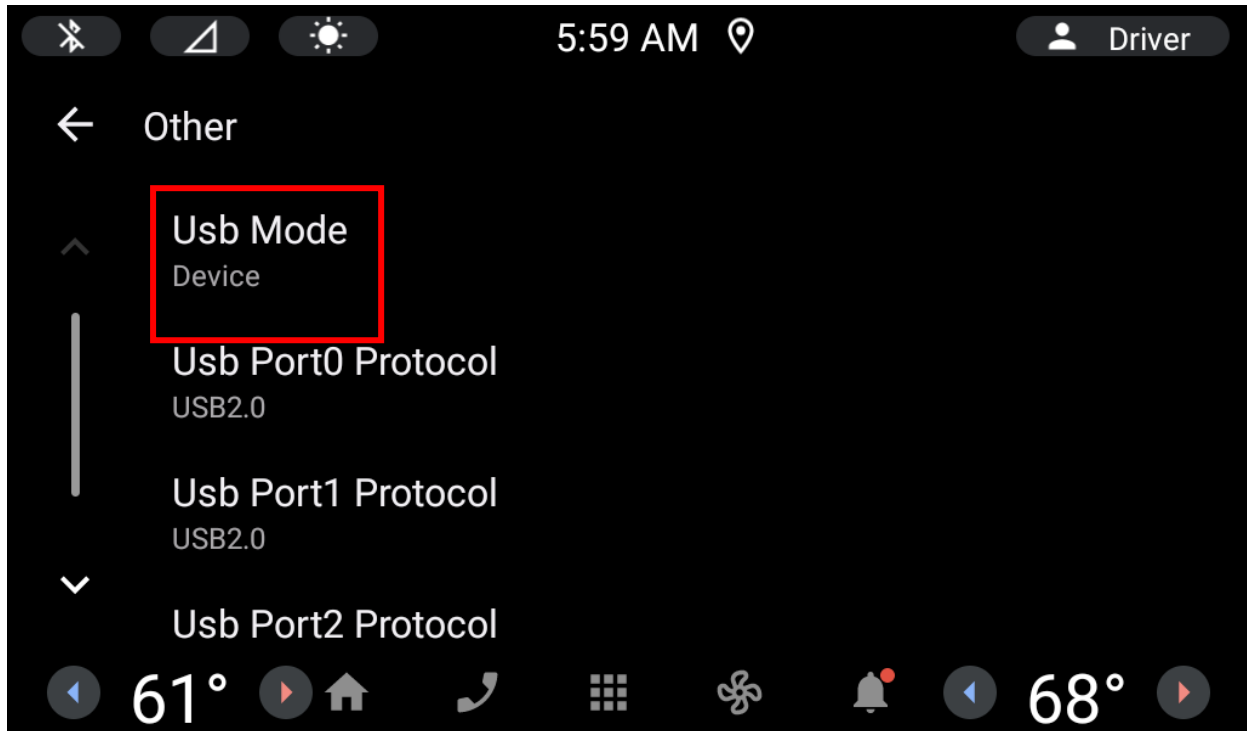


图 35-13 Usb Mode

2. 写 sys 设备节点

通过写如下设备节点来切换 usb host mode 和 device mode。

- 切换到 host 模式:

```
echo "host" > /sys/class/dual_role_usb/dual-role-usb20/data_role
```

```
230601_14:13:59.036)echo host >/sys/class/dual_role_usb/dual-role-usb20/data_role
230601_14:13:59.038) [ 1808.610651] [ C0] [N] [USB][DRD]:cdncusb: sduhc0tqAIDLEDetect
230601_14:13:59.038) [ 1808.610655] [ T3201] [N] [USB][DRD]:switch role to host succeed!
230601_14:13:59.038)console:/ # [ 1808.612138] [ C0] [N] [USB][DRD]:sdudcStop
230601_14:13:59.038) [ 1808.612188] [ T34] [N] [USB][DRD]:prop=1, usb_mode=1
230601_14:13:59.038) [ 1808.612798] [ C0] [N] [USB][DRD]:sdudcReset
230601_14:13:59.038) [ 1808.613418] [ T34] [N] [USB][DRD]:prop=2, usb_mode=1
230601_14:13:59.038) [ 1808.613956] [ C0] [N] [USB][DRD]:sdudcDisconnect
230601_14:13:59.038) [ 1808.614618] [ T34] [N] [USB][DRD]:prop=3, usb_mode=1
230601_14:13:59.038) [ 1808.615170] [ C0] [N] [USB][DRD]:cusb_g_reset
230601_14:13:59.038) [ 1808.616351] [ C0] [N] [USB][DRD]:cusb_g_disconnect
230601_14:13:59.038) [ 1808.617089] [ C0] [N] [USB][DRD]:cdn_cusb_get_usb_ode mode is 1 !
230601_14:13:59.038) [ 1808.617208] [ T662] [N] [USB][DRD]:prop=2, usb_mode=1
230601_14:13:59.038) [ 1808.619578] [ T662] [N] [USB][DRD]:prop=3, usb_mode=1
230601_14:13:59.038) [ 1808.620382] [ T662] [N] [USB][DRD]:prop=1, usb_mode=1
```

图 35-14 切换 host 模式成功

- 切换到 device 模式:

```
echo "device" > /sys/class/dual_role_usb/dual-role-usb20/data_role
```

```
30601_14:15:44.206)echo device >/sys/class/dual_role_usb/dual-role-usb20/data_role
30601_14:15:44.207)ce >/sys/class/dual_role_usb/dual-role-usb20/data_role
30601_14:15:44.207)e
30601_14:15:44.208)[ 1913.822754][ C0][N] [USB][DRD]:IRQ OTG: B IDLE Babble
30601_14:15:44.208)[ 1913.822758][ T3201][N] [USB][DRD]:switch role to device succeed!
30601_14:15:44.208)console:/ # [ 1913.824244][ T35][N] [USB][DRD]:prop=1, usb_mode=0
30601_14:15:44.208)[ 1913.824933][ T35][N] [USB][DRD]:prop=2, usb_mode=0
30601_14:15:44.208)[ 1913.825551][ T35][N] [USB][DRD]:prop=3, usb_mode=0
30601_14:15:44.208)[ 1913.826006][ C0][N] [USB][DRD]:USBIRQ: SUSPEND QUEUEWORK
30601_14:15:44.208)[ 1913.827606][ T662][N] [USB][DRD]:prop=2, usb_mode=0
30601_14:15:44.208)[ 1913.828356][ T662][N] [USB][DRD]:prop=3, usb_mode=0
30601_14:15:44.208)[ 1913.829032][ T662][N] [USB][DRD]:prop=1, usb_mode=0
30601_14:15:44.284)[ 1913.870217][ T3062][N] [USB][DRD]:sofcnt 0
30601_14:15:44.288)[ 1913.923325][ C0][N] [USB][DRD]:sdudcReset
30601_14:15:44.288)[ 1913.923873][ C0][N] [USB][DRD]:sdudcDisconnect
30601_14:15:44.288)[ 1913.924469][ C0][N] [USB][DRD]:cdn_cusb_get_usb_ode mode is 0 !
30601_14:15:44.425)[ 1914.011271][ T3062][I] android_work: sent uevent USB_STATE=CONNECTED
30601_14:15:44.425)[ 1914.038432][ T3062][I] android_work: sent uevent USB_STATE=CONFIGURED
```

图 35-15 切换 device 模式成功



注意:

也可以通过串口输入上面的命令, 注意使用串口切换需要 su 确保有足够的权限

3. 设置默认值

如需修改默认值, 请修改/build/projects/ivi_s_ac8025/metazone.csv 文件, 对应的 Index Name 是 METAZONE_USB_OTG_MODE。



注意:

通过 echo 命令进行模式切换没有记忆功能, usb mode 设置后立即生效。

35.3.2 设置 USB OTG protocol 为 full speed 或 high speed

1. APP 切换

用户可以直接通过: Car Settings->ATC Settings->Other ->usb OTG protocol, 对 USB OTG 口的 USB mode 进行设置。如下图所示:

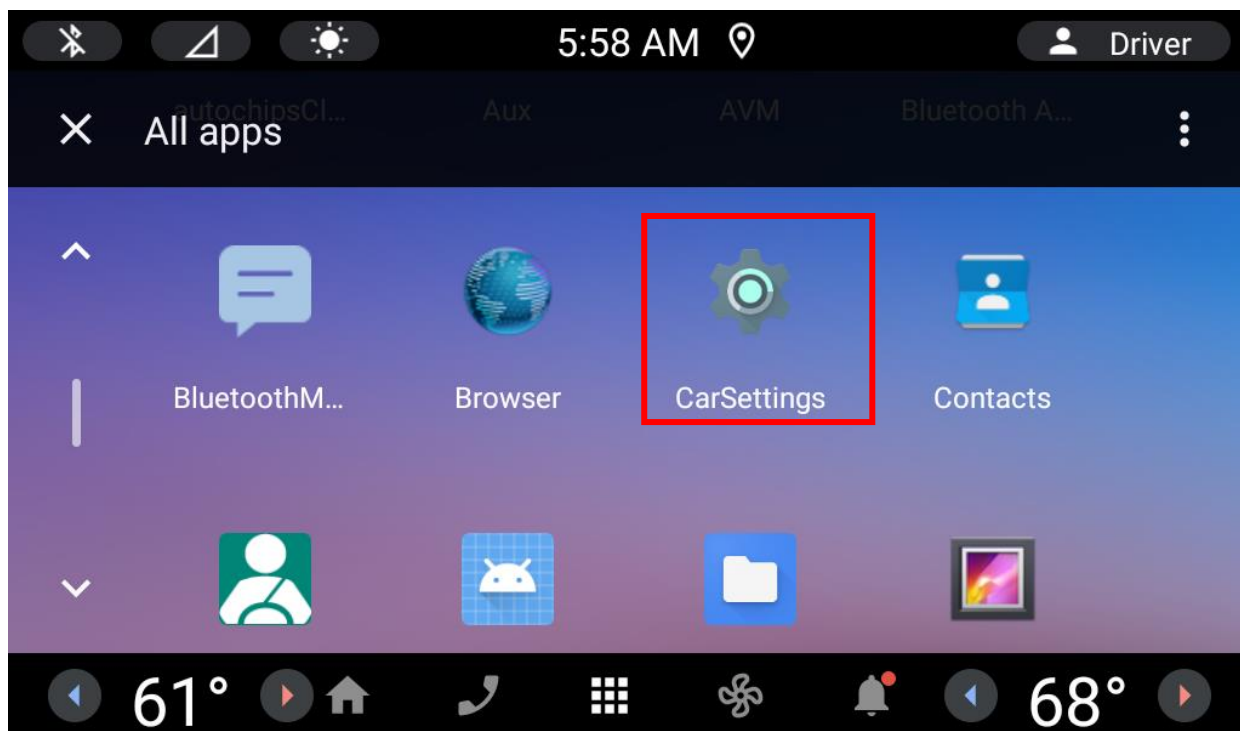


图 35-16 Car Settings

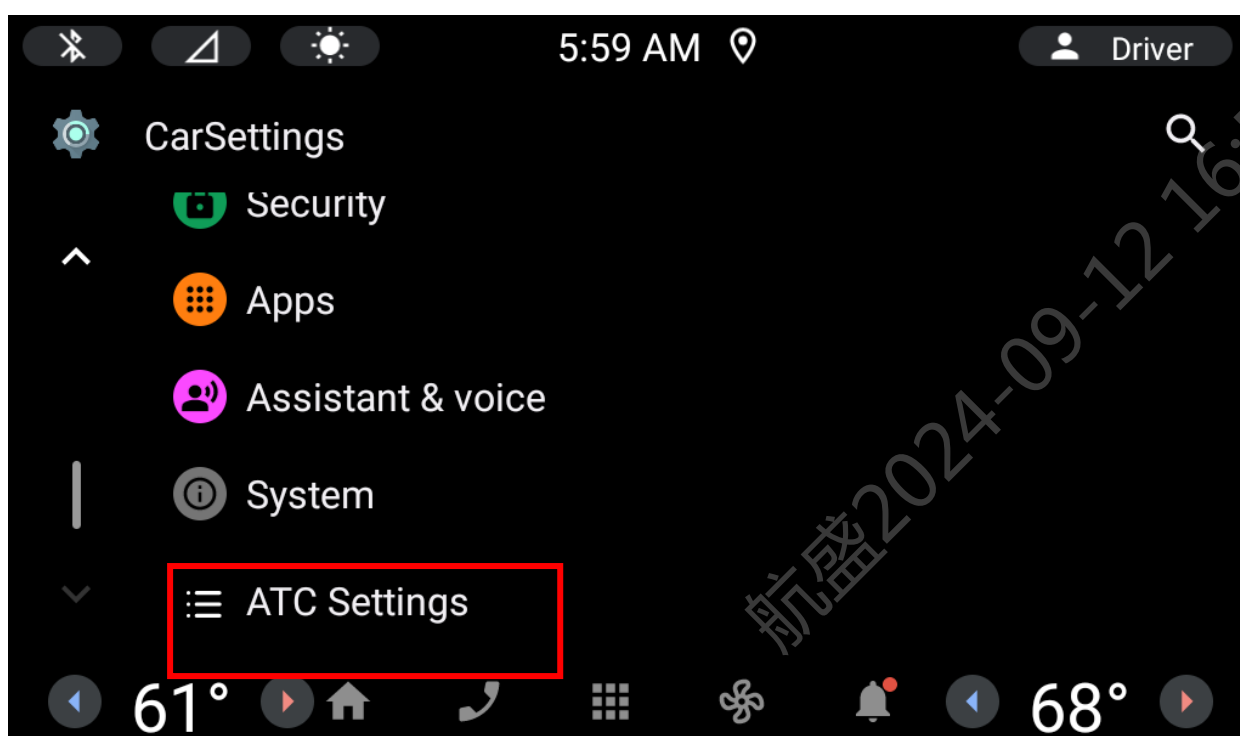


图 35-17 ATC Settings

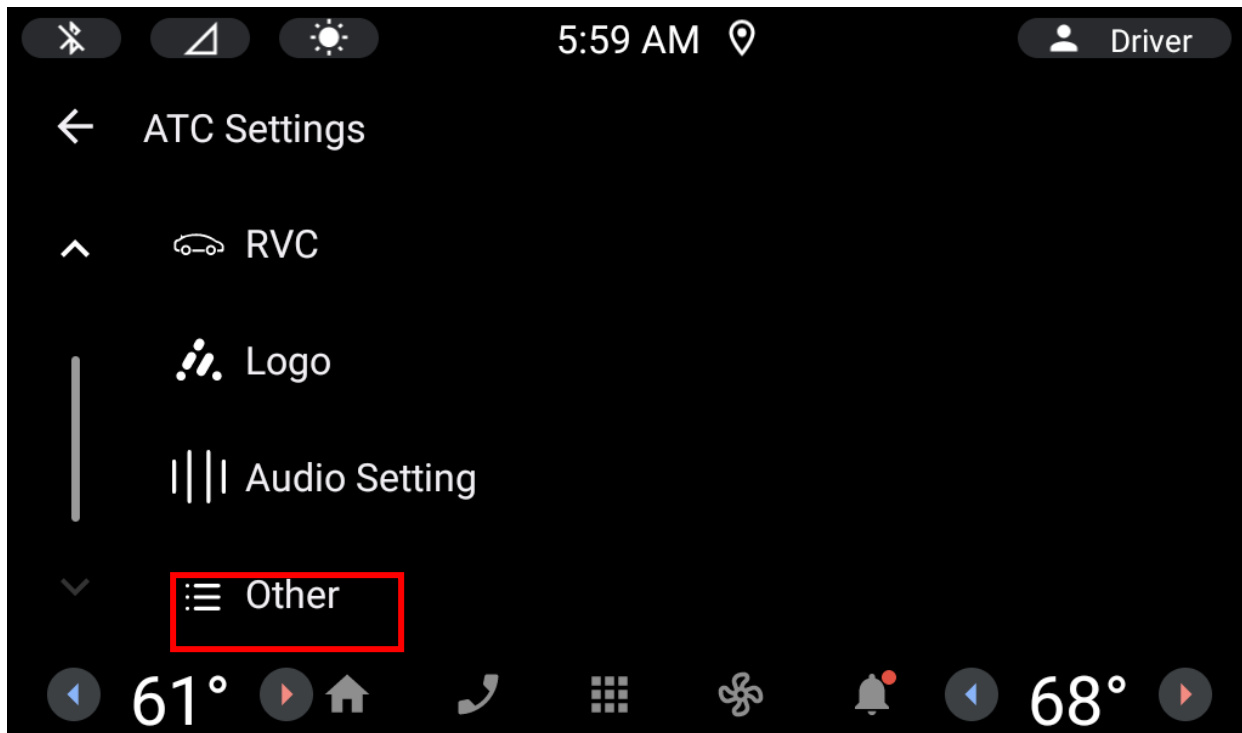


图 35-18 Other

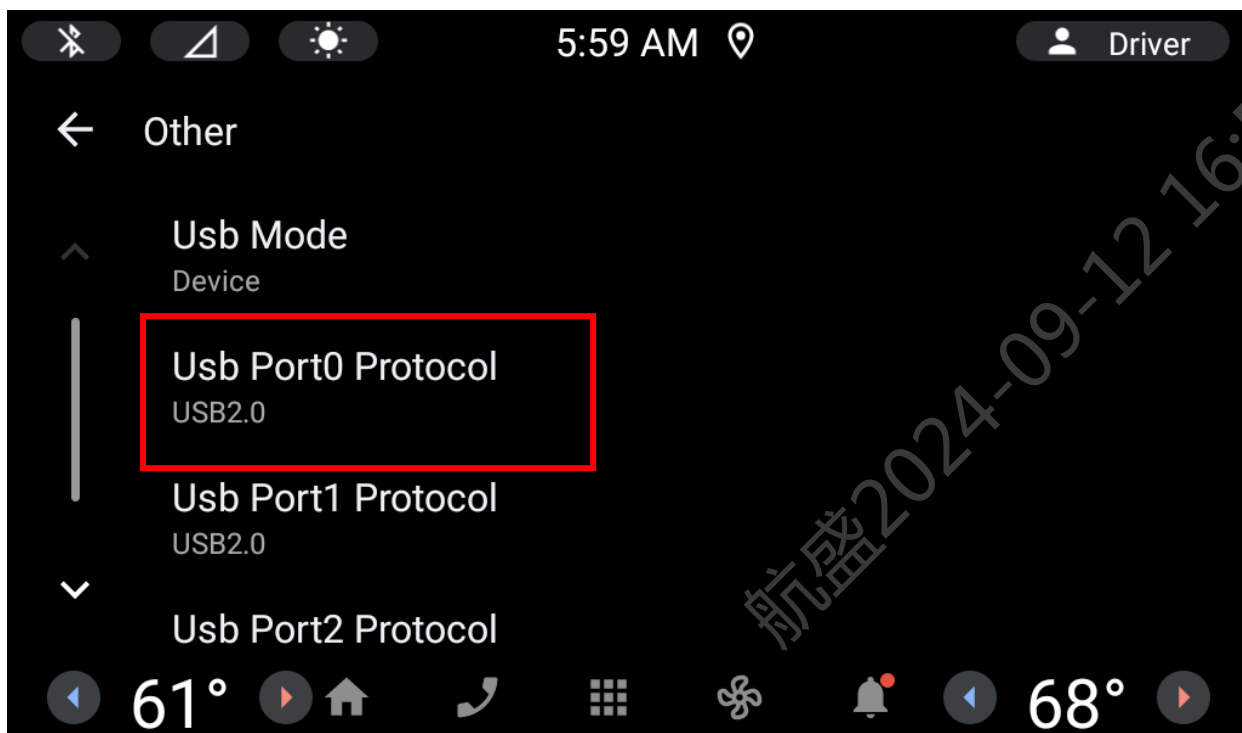


图 35-19 OTG Protocol

2. metazone Editor

usb OTG 口使用的 Metazone 值为 DWORD 类型，Index Name 为 METAZONE_USB_PROTOCOL，占据 0~3 位。

usb OTG 口的默认值如下：

```
metazone index Name = METAZONE_USB_PROTOCOL
```

```
usb2.0 writedata = 0x2 << 0
```

```
usb1.1 writedata = 0x1 << 0
```



注意：

配置后，需重启才能生效。

35.3.3 设置 USB Host 30 Protocol 为 Full Speed/High Speed/Super Speed

用户可以直接通过：Car Settings->ATC Settings->Other ->Usb Port1 Protocol，对 USB port1 口的 USB protocol 进行设置。如下图所示：

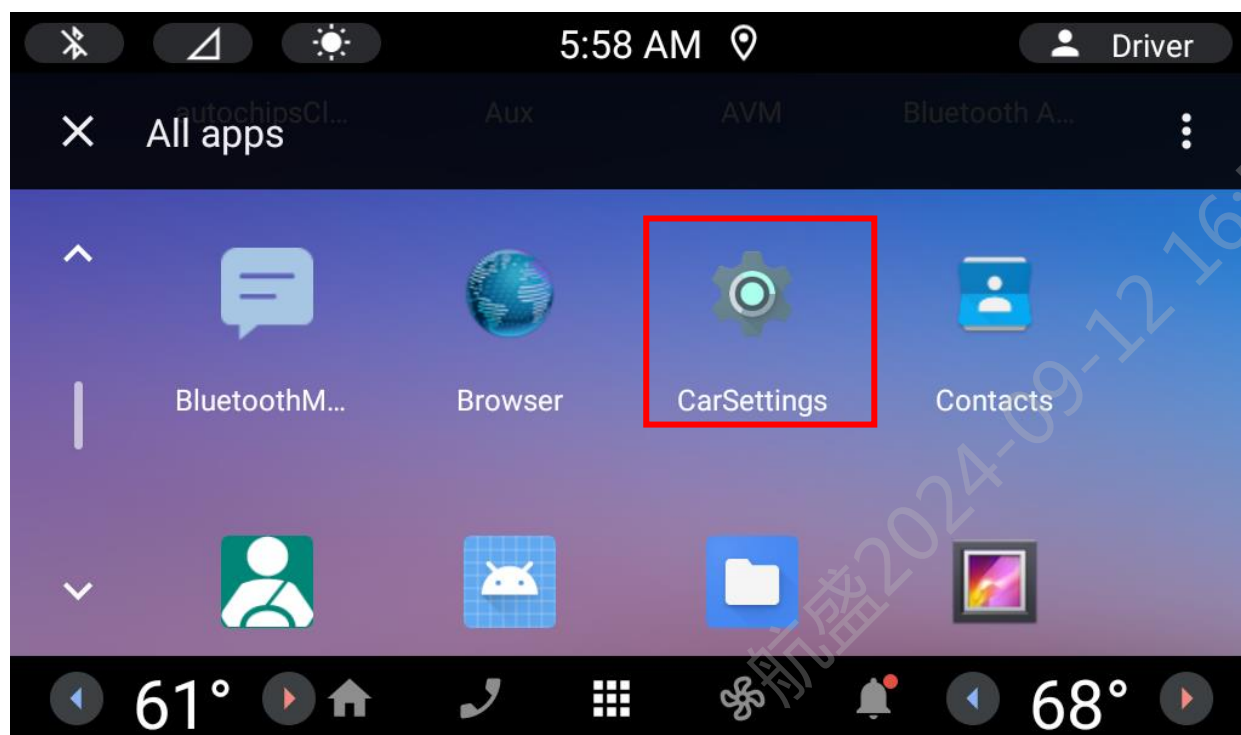


图 35-20 Car Settings

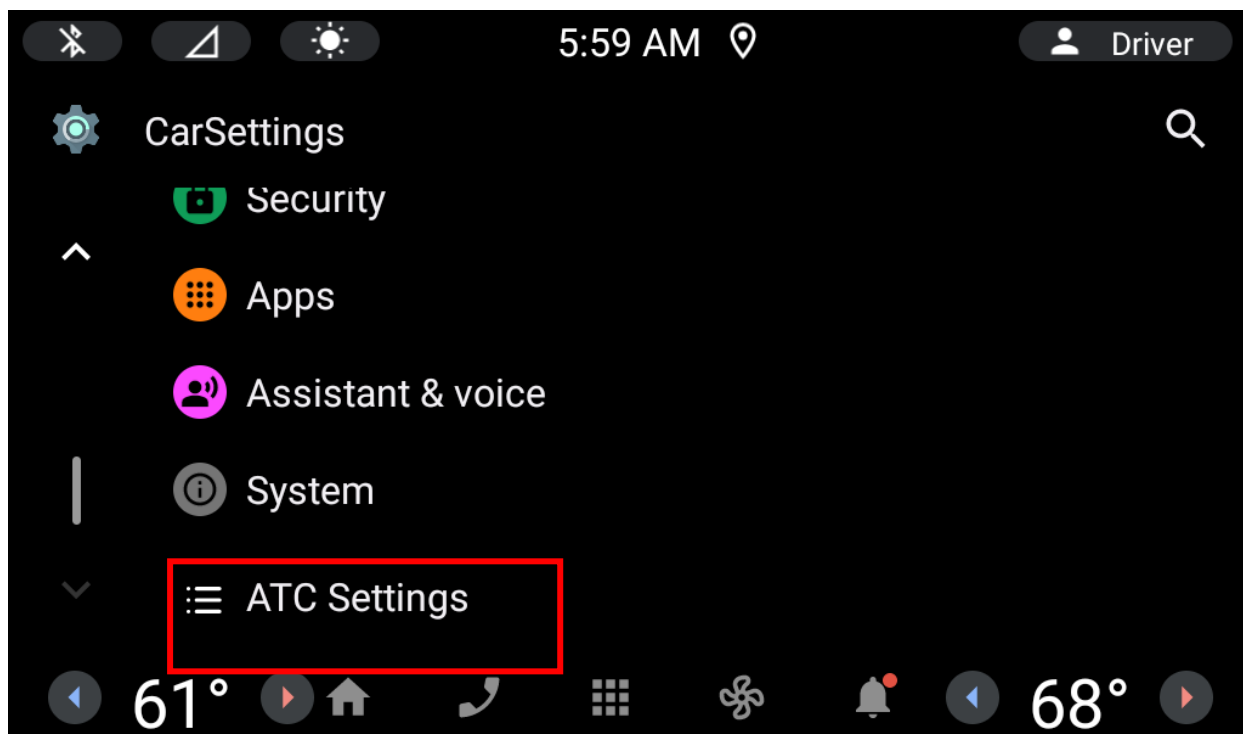


图 35-21 ATC Settings

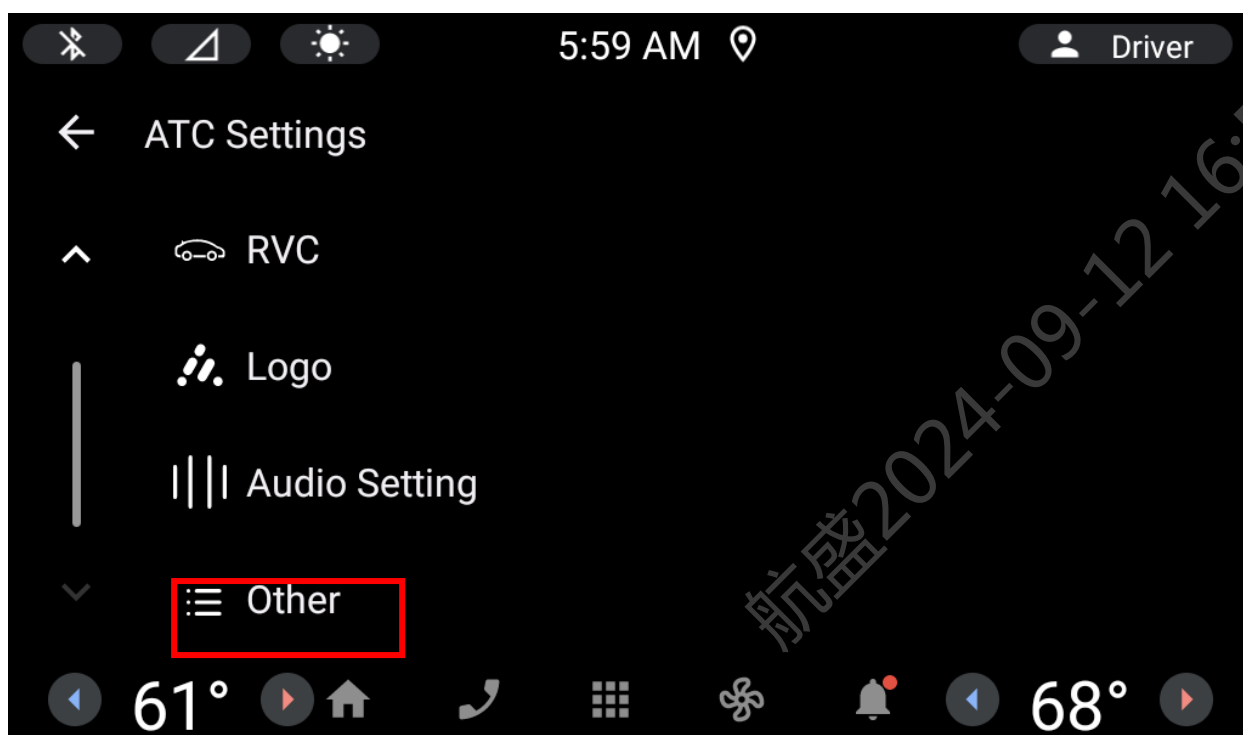


图 35-22 Other

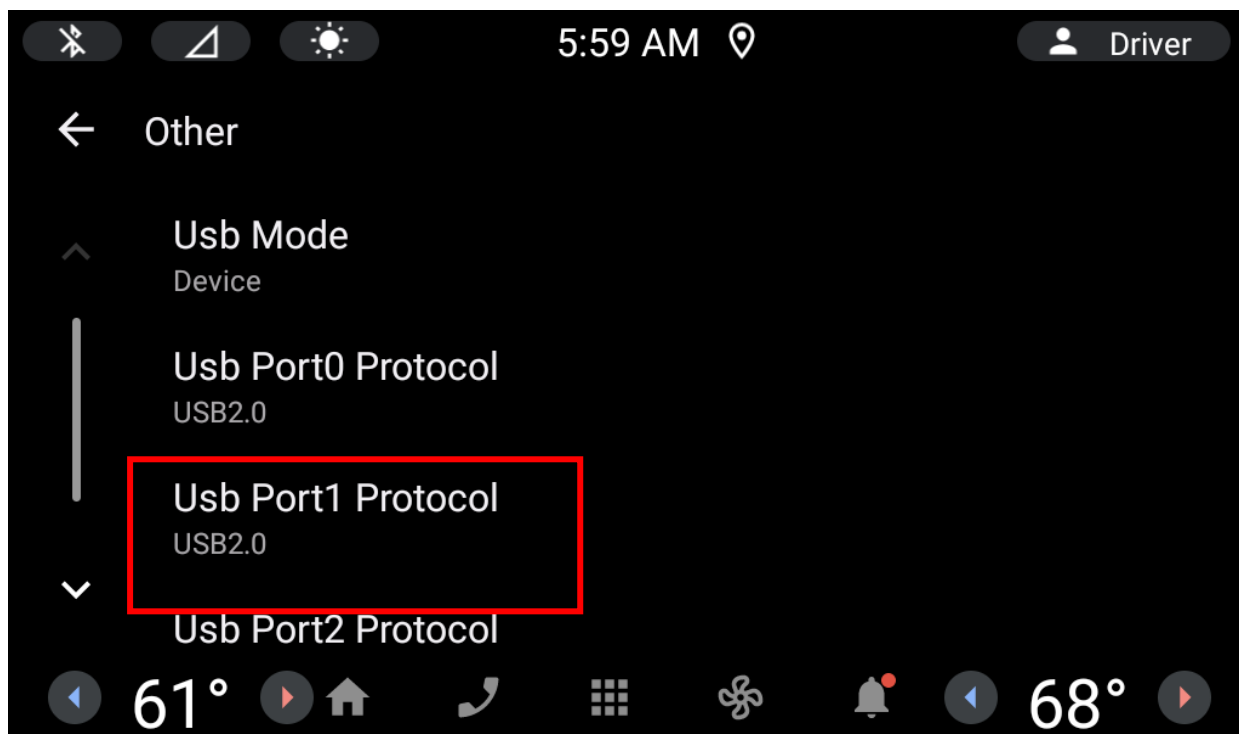


图 35-23 USB Port1 Protocol



注意:

配置后，需重启才能生效。

35.3.4 设置 USB Host 30 Protocol 默认值

USB Host 30 使用的 Metazone 值为 DWORD 类型，Index Name 为 METAZONE_USB_PROTOCOL (0x10014)

Host 30 Protocol 默认值如下:

```
metazone index Name = METAZONE_USB_PROTOCOL
usb3.0 writedata = 0x4 << 8
usb2.0 writedata = 0x2 << 8
usb1.1 writedata = 0x1 << 8
```

如需修改默认值，请修改/build/projects/ivi_s_ac8025/metazone.csv 文件，对应 Index Name 为 METAZONE_USB_PROTOCOL，修改 data 和 default_data 栏位对应的值即可。

35.3.5 切换 adb 连接域

对于 cockpit 版本，其存在多个域（ivi、adas、cluster），为了可以方便的调试各个域及 dump 不同域的信息，故提供了切换 adb 连接域的方法。

可通过 adb cmd 窗口或者串口执行 echo 命令进行系统切换，命令格式如下：

adb 命令（cli 命令需要 root 权限）：`adb shell "echo ${TARGET_SYSTEM} > /proc/usb_debug"`

串口命令：`echo ${TARGET_SYSTEM} > /proc/usb_debug`

参数\${TARGET_SYSTEM}说明：

表 35-3 切换参数说明

<code>\${TARGET_SYSTEM}</code>	意义	说明
ivi	IVI 系统	IVI 系统名称小写
adas	ADAS 系统	ADAS 系统名称小写，需要为全称
cluster	Cluster 系统	Cluster 系统名称小写，需要为全称

示例（切换到 ADAS 系统）：`adb shell "echo adas > /proc/usb_debug"`。



注意：

adb 切换后，重启也会在切换后的域。

例如：将 adb 切换到 adas 域，重启后依旧会在 adas 域

35.3.6 查看当前 adb 连接域

对于 cockpit 版本，可以通过 adb cmd 窗口或者串口执行 cat 命令查看当前 adb 连接域，命令格式如下：

adb 命令（cli 命令需要 root 权限）：`adb shell "cat /proc/usb_debug"`

串口命令：`cat /proc/usb_debug`

35.4 典型问题

35.4.1 如何查看 USB 工作规范和 USB OTG 当前工作模式

【问题描述】

如何查看 USB 工作的规范和 USBOTG 当前工作模式？

【解决方案】

可以通过系统主机面点击 Car Settings->ATC Settings->Other 查看。

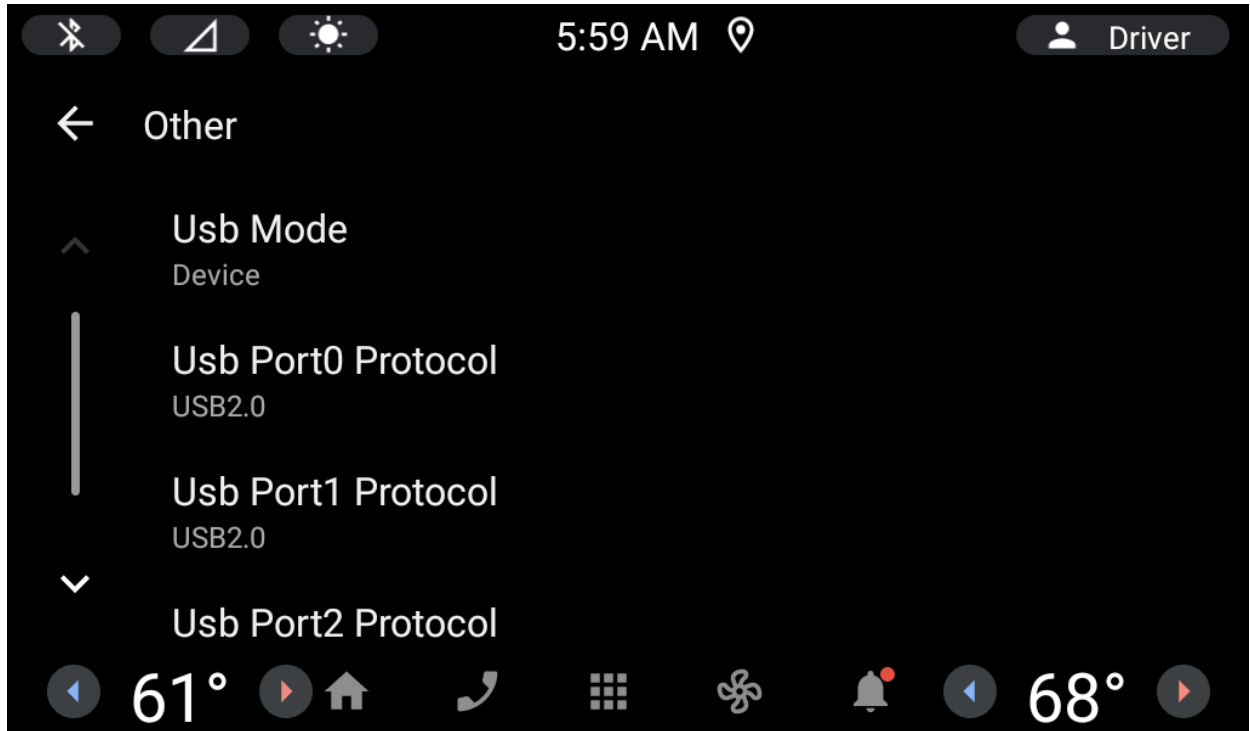


图 35-24 Mode & protocol

USB OTG 工作模式还可以通过串口输入命令来查询：

```
cat /sys/class/dual_role_usb/dual-role-usb20/data_role
```

当端口是 host 模式时输出：

```
host
```

当端口是 Device 模式时输出：

```
devices
```

35.4.2 如果 U 盘连接到车机的 USB port 上，没有出现挂载点，如何确定 driver 是否正确识别到 U 盘

【问题描述】

如果 U 盘连接到车机的 USB port 上，没有出现挂载点，如何确定 driver 是否正确识别到 U 盘？

【解决方案】

通过串口命令查询设备节点：ls /dev/block/

如果在 block 下出现 sd*(sda/sdb....)的节点，则说明 driver 已经正确识别到 U 盘设备，如下图所示。反之如果没有，则 driver 没有识别到 U 盘。当出现未识别 U 盘的情况可以进一步查看 /dev/bus/usb/00X/00X 节点是否有生成，来判断是 U 盘未识别还是说 USB 控制器未识别。如下图所示：

```
ls /dev/block

(230324_21:24:51.424)[11624.616255][ T909][E] audit: rate limit exceeded
(230324_21:24:51.627)[11624.815428][ T221][W] type=1400 audit(1678204296.611:59374): avc: deni
dev="tmpfs" ino=254 scontext=u:r:carservice_app:s0 tcontext=u:object_r:vendor_atc_aee_prop:s0 t
(230324_21:24:51.830)[11625.015461][ T221][W] type=1400 audit(1678204296.811:59375): avc: deni
dev="tmpfs" ino=254 scontext=u:r:carservice_app:s0 tcontext=u:object_r:vendor_atc_aee_prop:s0 t
(230324_21:24:51.970)ls dev/block
(230324_21:24:52.048)[11625.215652][ T221][W] type=1400 audit(1678204297.007:59376): avc: deni
dev="tmpfs" ino=254 scontext=u:r:carservice_app:s0 tcontext=u:object_r:vendor_atc_aee_prop:s0 t
(230324_21:24:52.049)aah-dtb_a dm-19 loop10 loop27 ram0 sda
(230324_21:24:52.049)aah-dtb_b dm-3 loop11 loop28 ram1 sdal
(230324_21:24:52.049)aah_a dm-4 loop12 loop29 ram10 sdb
(230324_21:24:52.049)aah_b dm-5 loop13 loop3 ram11 sdbl
(230324_21:24:52.049)aah_recovery-dtb_a dm-7 loop14 loop4 ram12 sdc
(230324_21:24:52.049)aah_recovery-dtb_b dm-8 loop15 loop5 ram13 system_a
(230324_21:24:52.049)avm dm-9 loop16 loop6 ram14 system_b
(230324_21:24:52.049)boot_a dtb_a loop17 loop7 ram15 teedata
(230324_21:24:52.049)boot_b dtb_b loop18 loop8 ram2 trustzone_a
(230324_21:24:52.049)cache dtbo loop19 loop9 ram3 trustzone_b
(230324_21:24:52.049)datazone-ro expdb loop2 mapper ram4 userdata
(230324_21:24:52.049)datazone-rw-a hsm_a loop20 metazone ram5 vbmeta_a
(230324_21:24:52.049)datazone-rw-b hsm_b loop21 misc ram6 vbmeta_b
(230324_21:24:52.049)dm-11 lk_a loop22 mmcblk0 ram7 vendor_a
(230324_21:24:52.049)dm-12 lk_b loop23 mmcblk0boot0 ram8 vendor_b
(230324_21:24:52.049)dm-14 logo loop24 mmcblk0boot1 ram9 vold
(230324_21:24:52.049)dm-17 loop0 loop25 platform recovery_a
(230324_21:24:52.049)dm-18 loop1 loop26 preloader recovery_b
```

图 35-25 USB disk sd*

35.4.3 BC1.2 功能支持多大的充电电流

【问题描述】

BC1.2 功能支持多大的充电电流？

【解决方案】

OTG BC1.2 功能默认支持的最大充电电流为 1.5A，被充电设备必须支持 BC1.2 协议，且该电流大小和具体的被充电设备、充电线的质量、被充电设备的电量有关，在恒流充电阶段内时充电电流范围应为 0.5~1.5A。在恒压充电阶段内，当被充电设备电量较高时，出现充电电流小于 0.5A 也是正常现象。

充电能力还跟板子限流芯片设置相关，AC8025 demo 板最大供电电流为 1.5A。

35.4.4 如何开关 OTG 口 BC 功能

【问题描述】

如何开关 OTG 口 BC 功能？

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

1020 /1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

【解决方案】

OTG BC1.2 功能默认关闭，如需开启参考如下表格修改 METAZONE_USB_OTG_MODE 即可。

表 35-4 参数说明

Metazone Index	Dec value	Description
METAZONE_USB_OTG_MODE	2	Hostmode,bc1.2 off
METAZONE_USB_OTG_MODE	34	Hostmode,bc1.2 on
METAZONE_USB_OTG_MODE	0	devicemode,bc1.2 off
METAZONE_USB_OTG_MODE	32	devicemode,bc1.2 on

备注：bc1.2 功能只在 hostmode 生效

调试指令(重启生效)

```
echo 0 > /sys/devices/platform/soc/13020320.cdns-usb/battery_charge //bc off
```

```
echo 2 > /sys/devices/platform/soc/13020320.cdns-usb/battery_charge //bc on
```

35.4.5 如何查看 USBHOST30 工作的规范和当前工作模式

【问题描述】

如何查看 USB 工作的规范和当前工作模式？

【解决方案】

可以通过系统主机面点击 Car Settings->ATC Settings->Other 查看。

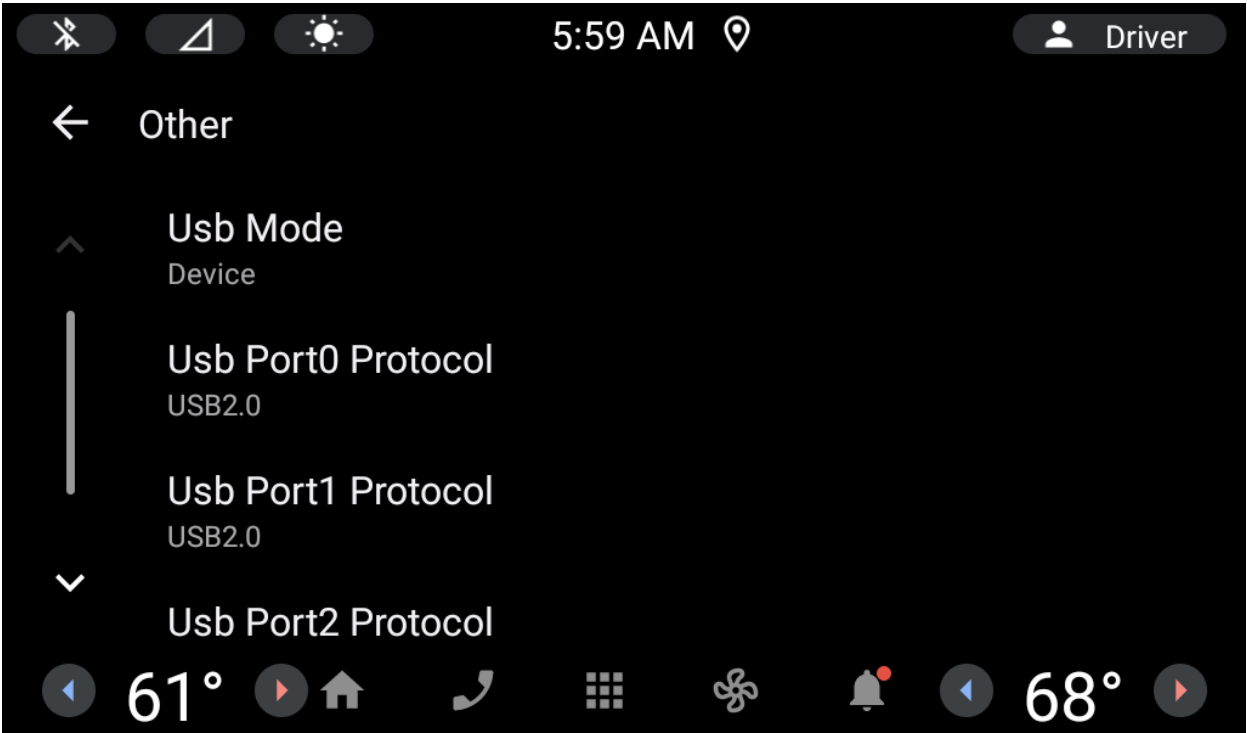


图 35-26 Protocol

36 SD4HC

36.1 简介

36.1.1 软硬件环境

36.1.1.1 软件环境

本章介绍的 SD4HC 驱动模块基于 kernel-5.10 内核，在 kernel-5.10 的 mmc sdhci 原生驱动框架的基础上开发。

36.1.1.2 硬件环境

本章介绍的 SD4HC 驱动模块运行于 AC8025 硬件平台，软件配置需要适配板级硬件电路的设计，例如每个 SD4HC IP 连接的 Device 类型，控制外部电压的 GPIO 引脚配置等。

AC8025 有 3 个 SD4HC IP，分别标记为 SD0、SD1、SD2。在 demo 设计中，SD0、SD1、SD2 的功能分配如下：

- SD0: 连接 eMMC Device，可运行 4bits/8bits 模式
- SD1: 连接 SD Card，可支持 SD2.0/SD3.0 模式
- SD2: 连接 SDIO Device，支持 MT6630 芯片，用于实现 WiFi/BT/GPS 功能

36.1.1.2.1 SD0 硬件环境

SD0 Host 和 eMMC Device 的连接如图 36-1 所示：

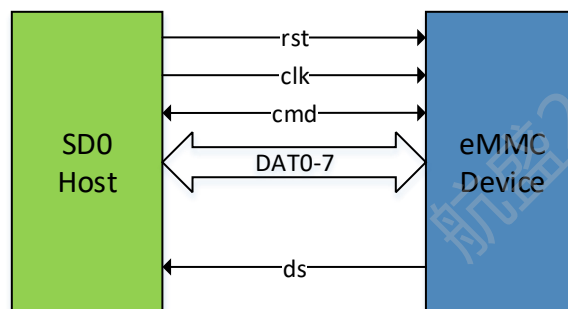


图 36-1 SD0 和 eMMC 连接示意图

- rst: eMMC hardware reset pin
- clk: eMMC clock pin

- cmd: eMMC command pin
- DAT0-7: eMMC data pins
- ds: eMMC data strobe pin , HS400 mode 下 Read 操作的 sync 信号

36.1.1.2.2 SD1 硬件环境

SD1 Host 和 SD 卡的连接如图 36-2 所示:

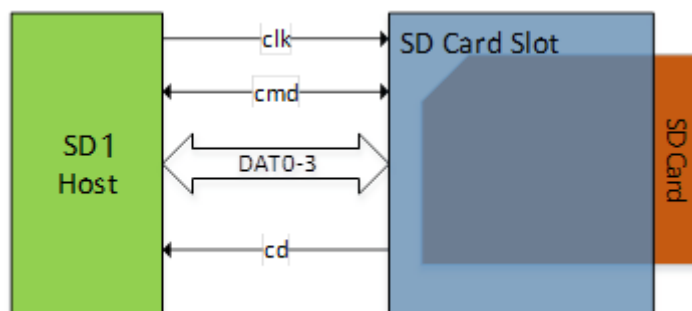


图 36-2 SD1 和 SD 卡连接示意图

- clk: SD Card clk pin
- cmd: SD Card cmd pin
- d0~d3: SD Card 4 个 data pin
- cd: 检测 SD 卡插入/拔出的 pin

36.1.1.2.3 SD2 硬件环境

SD2 Host 和 SDIO Device 的连接如图 36-3 所示:

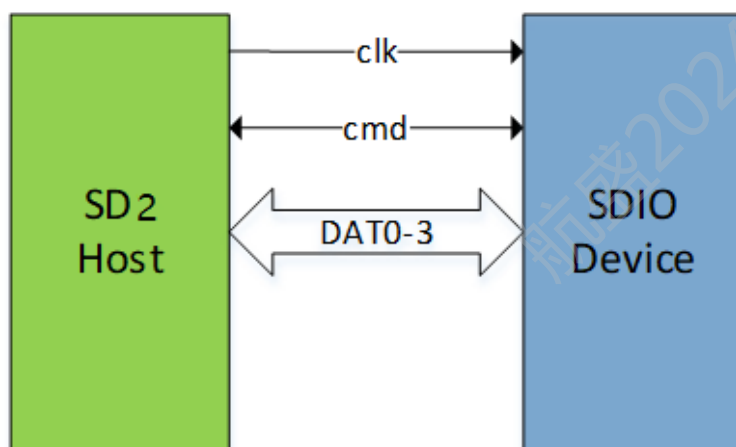


图 36-3 SD1 和 SDIO Device 连接示意图

- clk: SDIO clk pin
- cmd: SDIO cmd pin
- DAT0-3: SDIO 4 个 data pin

36.1.2 功能

36.1.2.1 硬件功能

SD4HC (The 4th Generation SD Host Controller) 支持如下版本:

- SD/SDIO devices up to 3.0
- eMMC devices up to 5.1

36.1.2.2 软件功能

该模块用于 SD4HC host driver, 主要功能如下:

- 初始化 eMMC/SD/SDIO device;
- 提供上电、掉电相关流程;
- 设置以及修改 eMMC/SD/SDIO device 的 mode/clock/bus width 等;
- 提供读写 eMMC/SD/SDIO device 的接口, 并提供 tuning 机制保证其高速模式稳定性;
- 当系统异常, 提供获取相关寄存器的接口, 便于分析;
- 对 SD4HC 模块提供 debug 的接口。

36.1.3 代码结构及文件描述

SD4HC 相关的 driver source code 存在于 hsm loader、hsm os、lk、kernel 中。

36.1.3.1 Preloader SD4HC driver

表 36-1 Preloader SD4HC driver 代码结构及描述

源文件	描述
source/vendor/autochips/proprietary/tinsys/hsm/loader/drivers/sd4hc	
mmc_common_inter.c	hsm loader 中 mmc core 层驱动源文件
mmc_core.c	
mmc_host.c	hsm loader 中 mmc host 层驱动源文件

源文件	描述
source/vendor/autochips/proprietary/tinysys/hsm/loader/include	
source/vendor/autochips/proprietary/tinysys/hsm/loader/include/ac8025	
mmc_common_inter.h	mmc_common_inter.h 为上层程序直接引用的接口头文件 sd4hc_cust.h 为 mmc driver 配置头文件 其他为 mmc driver 内部使用的头文件
mmc_core.h	
mmc_host.h	
mmc_rpmb.h	
sd4hc_cust.h	

36.1.3.2 Safety os SD4HC driver

表 36-2 Safety os SD4HC driver 代码结构及描述

源文件	描述
路径: source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/sd4hc/	
mmc_common_inter.c	mmc driver core 层驱动源文件
mmc_block.c	
mmc_rpmb.c	
mmc_core.c	
mmc_host.c	mmc driver host 层驱动源文件
路径: source/vendor/autochips/proprietary/tinysys/hsm/os/include/	
mmc_common_inter.h	mmc_common_inter.h 为上层程序直接引用的接口头文件 sd4hc_cust.h 为 mmc driver 配置头文件 其他为 mmc driver 内部使用的头文件

36.1.3.3 LK SD4HC driver

表 36-3 lk SD4HC driver 代码结构及描述

源文件	描述
source/vendor/autochips/proprietary/bootable/lk/platform/autochips/ac8x/sd4hc	
mmc_common_inter.c	mmc driver core 层驱动源文件
mmc_block.c	
mmc_rpmb.c	
mmc_core.c	
mmc_host.c	mmc driver host 层驱动源文件
source/vendor/autochips/proprietary/bootable/lk/platform/autochips/ac8x/include/platform	
mmc_common_inter.h	mmc_common_inter.h 为上层程序直接引用的接口头文件
mmc_core.h	
mmc_host.h	

源文件	描述
mmc_rpmb.h	sd4hc_cust.h 为 mmc driver 配置头文件
sd4hc_cust.h	其他为 mmc driver 内部使用的头文件

36.1.3.4 Kernel SD4HC driver

表 36-4 kernel SD4HC driver 代码结构及描述

源文件	描述
source/kernel/common/drivers/mmc/	
mmc/core 目录	该目录包含 kernel mmc driver 的 core 层代码，包括 card 部分的代码
mmc/host 目录	该目录包含 kernel mmc driver 的 host 层 sdhci 代码框架
source/kernel/common/drivers/mmc/host/autochips/ac8x	
sdhc-of-atc.c	SD4HC driver 的主要源文件，包含 SD4HC 特有的功能代码
sdhci-sd.c	针对 SD/SDIO 的 module 加载功能代码
sdhci-dbg.c	用于 debug 的功能代码
sdio_test.c	sdio 测试代码
source/kernel/common/arch/arm64/boot/dts/autochips	
ac8025.dtsi	dtsi 配置文件，包含 sd0/sd1/sd2 具体的寄存器信息以及 clk 配置等
ac8025_demo.dts	ivi 产品形态对应的 dts 配置文件，包含 sd0/sd1/sd2 的 driver strength、gpio 配置等
ac8025_demo-cluster.dts	cockpit 产品形态 cluster os 使用的 dts 配置文件，包含 sd0 的 driver strength、gpio 配置等

36.2 软件设计

36.2.1 架构图

eMMC（Embedded Multi Media Card）和 SD Card（Secure Digital Memory Card）都是从早期的 MMC 标准发展而来，二者在协议上有很多共通之处，SDIO 是在 SD Card 接口基础上衍生出来的外部设备数据传输标准。

SD4HC 符合 SD Host Controller Standard Specification version 4.00，可支持 eMMC/SD/SDIO，同一份驱动程序可同时支持这三种设备，因此本章中使用 mmc driver 表示 SD4HC 的驱动代码。

hsm loader 和 lk 中的 mmc driver 代码基本相同，其区别在于：hsm loader mmc driver 由于程序运行空间的限制，在 lk mmc driver 的基础上删减了不需要的功能。

36.2.1.1 Preloader/Safety os/LK mmc driver 软件架构图

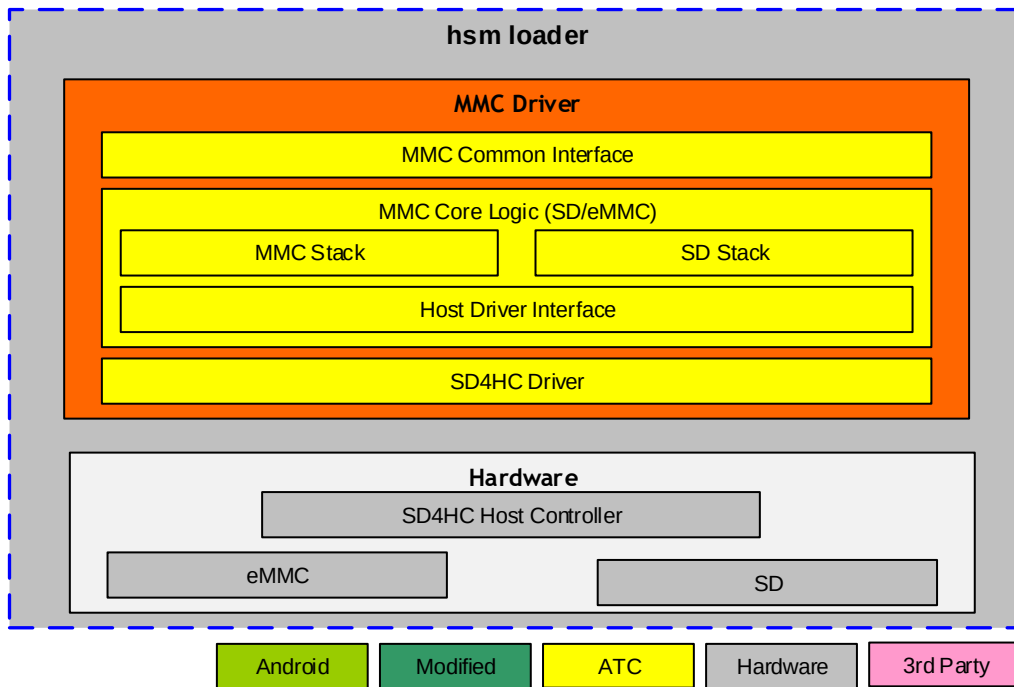


图 36-4 Preloader/Safety os/LK eMMC/SD driver 软件架构图

Safety os 的 mmc driver 架构和 Preloader 相同，也是通过 mmc 提供的 common 接口访问 SD4HC 驱动，下发读写请求等。

36.2.1.2 kernel mmc driver 软件架构图

位于 kernel 代码树的 drivers/mmc 目录，分为三层二个目录：

- mmc/core: 中间层为 core 层，是对 SD/eMMC Spec 标准的封装，对标准的流程进行封装；
- mmc/host: 最下层是 host 层，是 SD4HC 的 driver，封装对 HW 的操作，提供接口被 core 层调用。

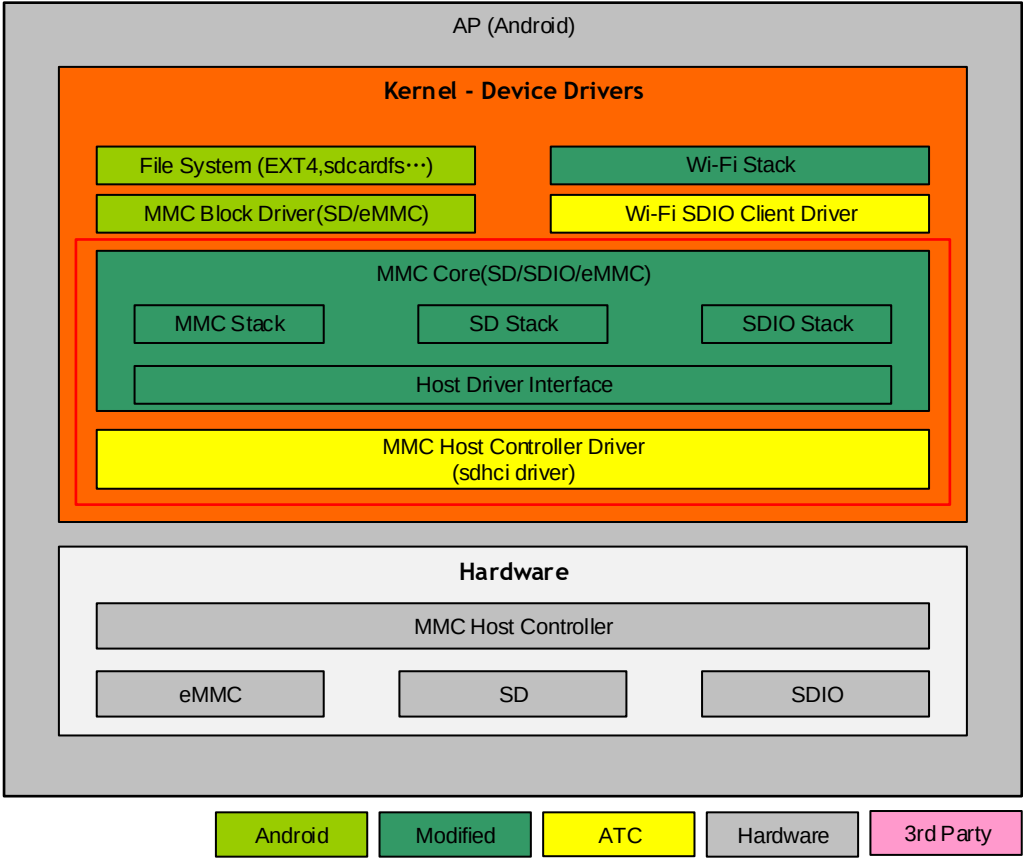


图 36-5 kernel mmc driver 软件架构图

36.2.2 控制流和数据流

36.2.2.1 Preloader/Safety os/LK mmc driver 控制流和数据流

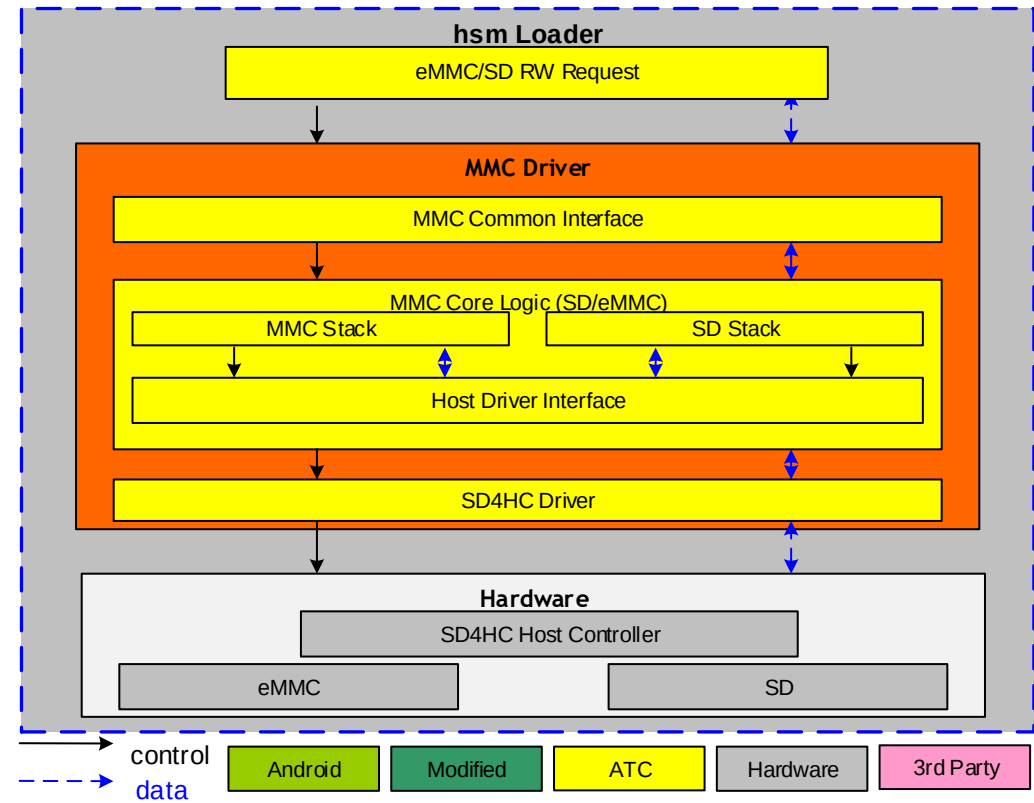


图 36-6 Preloader/Safety os/LK mmc driver 控制流和数据流

36.2.2.2 lk mmc driver 控制流和数据流

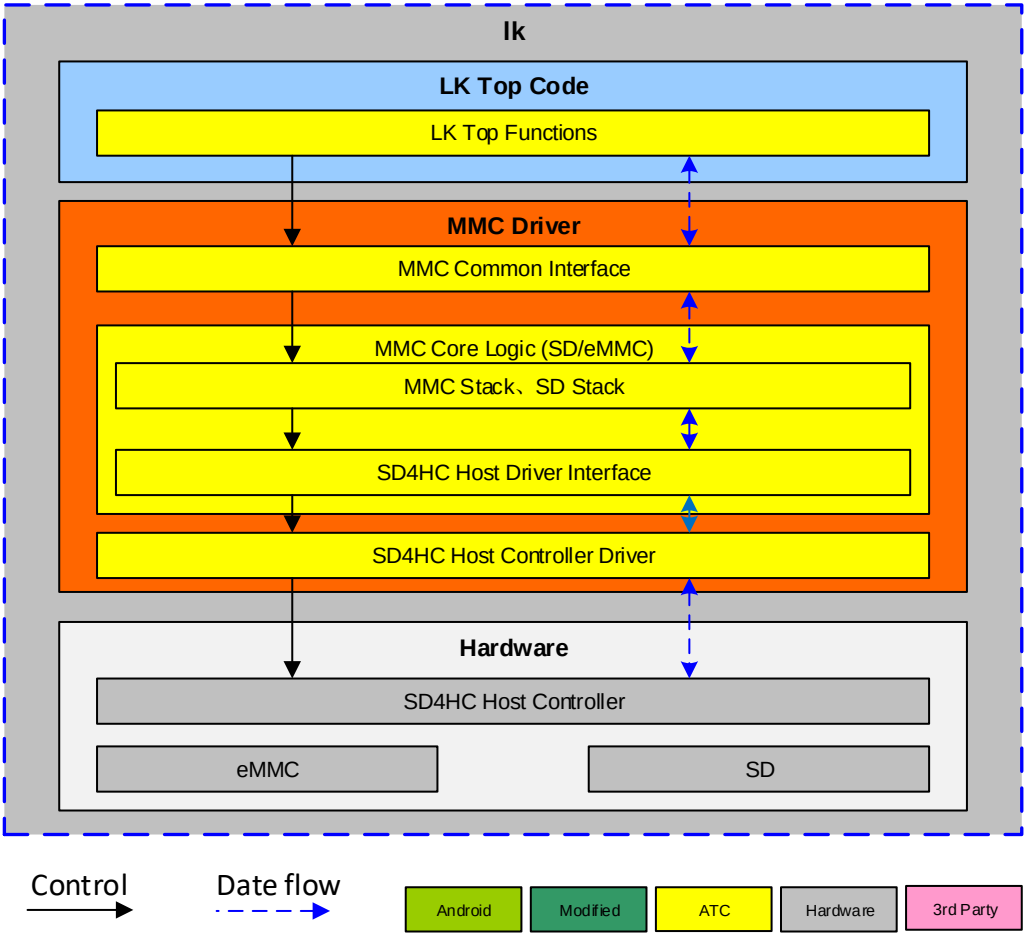


图 36-7 lk mmc driver 控制流和数据流

36.2.2.3 kernel mmc driver 控制流和数据流

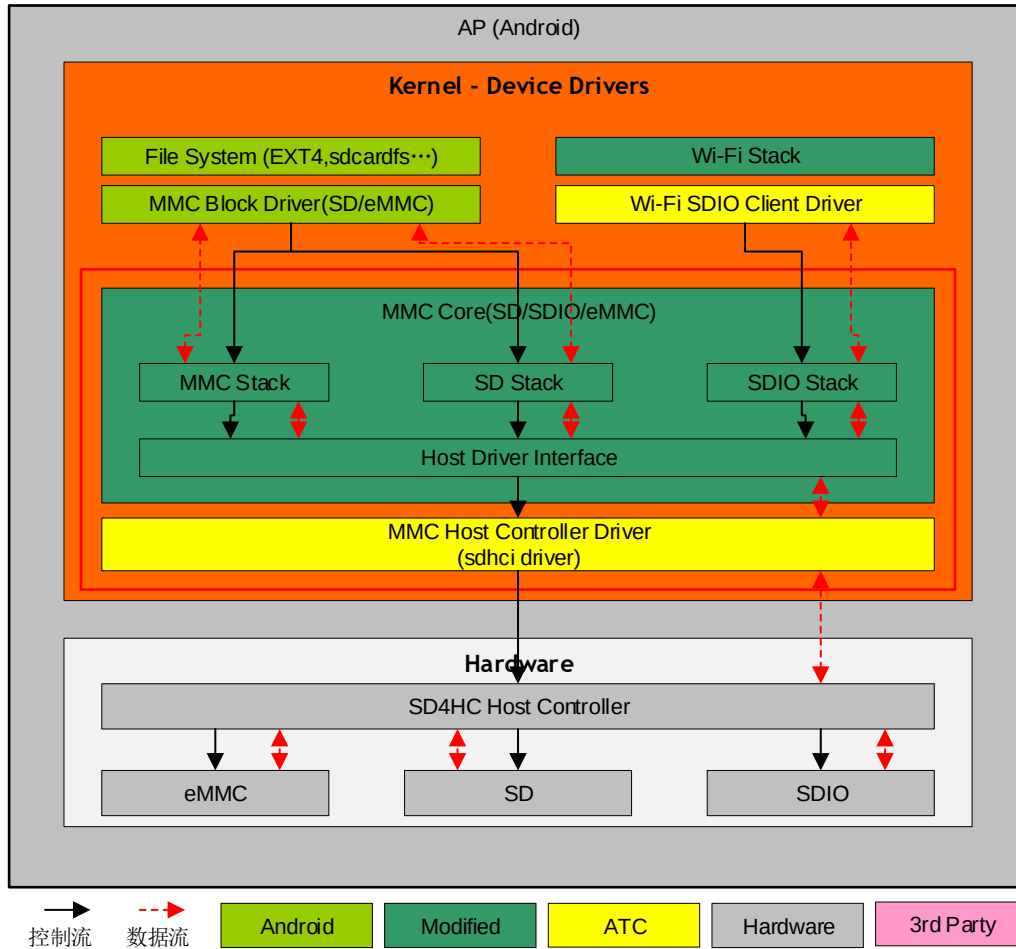


图 36-8 kernel mmc driver 控制流和数据流

36.2.3 dts 配置介绍

SD4HC 相关的 dts 配置文件位于 source/kernel/common/arch/arm64/boot/dts/autochips 路径下。

SD4HC 的配置信息在 ac8025.dtsi 文件中。

36.2.3.1 芯片级 dts 配置

ac8025.dtsi 文件包含芯片级的配置信息，其配置节点为 sd0@110D0000、sd1@13080000、sd2@13082000。

用户可修改的配置项如下表所示（其他配置项用户不可修改）：

表 36-5 kernel mmc driver dts 配置项描述

参数	说明
status	配置为 "okay" 或 "ok" 字符串, 可启用该设备, 否则关闭该设备
max-frequency	限制 Device 的最大工作频率, 单位为 Hz 通常可设置为 200000000, 即不对 Device 的工作频率做限制 (SD4HC 可支持的最大 bus clock 为 200MHz), 实际工作频率由具体的传输模式来确定
non-removable	标识设备是否可移除, SD0 连接 eMMC Device 需要配置该参数
bus-width	设置 Bus 宽度 SD/SDIO 可支持 4/1 bits, 推荐设置 bus-width = <4>
cap-mmc-highspeed	开启 eMMC High Speed 模式
mmc-ddr-1_8v	开启 eMMC HS DDR 模式
mmc-hs200-1_8v	开启 eMMC HS200 模式
mmc-hs400-1_8v	开启 eMMC HS400 模式
mmc-hs400-enhanced-strobe	开启 eMMC HS400ES 模式
cap-mmc-hw-reset	开启 eMMC HW reset 功能
no-sdio	该 Host 不需要支持 SDIO 设备
no-sd	该 Host 不需要支持 SD Card
no-mmc	该 Host 不需要支持 eMMC
cap-sd-highspeed	开启 SD/SDIO High Speed 模式
sd-uhs-sdr12	开启 SD/SDIO SDR12 模式
sd-uhs-sdr25	开启 SD/SDIO SDR25 模式
sd-uhs-sdr50	开启 SD/SDIO SDR50 模式
sd-uhs-ddr50	开启 SD/SDIO DDR50 模式
sd-uhs-sdr104	SD/SDIO 开启 SDR104 模式
broken-cd	对支持 SD Card 的 Host 有效, 配置该参数, driver 以 polling 方式检测 SD Card 的插入状态, 否则, driver 以中断的方式检测 SD Card 的插入状态



注意:

除表 36-5 所示的配置项外, ac8025.dtsi 中其他的配置项关联到 AC8025 内部的资源配置, 不可随意修改。

36.2.3.2 板级 dts 配置

ac8x_\$device.dts 文件（\$device 是 device 名称通配符，例如 \$device 可能为 demo 等值）包含板级配置信息，继承配置 &pio、&sd1、&sd2 三个节点。

其中 IVI 产品形态 sd0/sd1/sd2 的板级 dts 配置在 ac8025_demo.dts 文件；Cockpit 产品形态由于 eMMC 的虚拟化设计，当前 sd0 相关配置在 ac8025_demo-cluster.dts 文件。

如果 sd1、sd2 连接 SD Card，该 dts 可以分配 GPIO 来控制外部 SD Card 的电源开关，如果需要支持 SD3.0，还需要分配 GPIO 用于控制 SD Card IO 的电压选择 3.3 V/1.8 V。

对于连接 eMMC 和 SDIO 的情况，外部供电都是固定 1.8 V 的方案，不需要 GPIO 进行控制。

36.2.4 eMMC/SD 工作时钟说明

系统各阶段 eMMC/SD 的 clock 以及 mode 是可配置的，具体的配置方法见 36.4.1 和 36.4.2 章节。

系统默认的工作模式如下表所示：

表 36-6 eMMC/SD 各阶段 clock 及 mode 配置

阶段	eMMC	SD
Preloader	200MHz/HS200 mode	50MHz/HS mode
Safety os	200MHz/HS200 mode	50MHz/HS mode
lk	200MHz/HS200 mode	50MHz/HS mode
kernel	200MHz/HS400 mode	50MHz/HS mode

36.2.5 硬件写保护

在 AC8025 方案中对 eMMC 的保护有两种方式：硬件写保护、软件写保护。

其中硬件写保护是指通过硬件的方法，在 device 端进行配置，对于保护区域内所有地址的写请求均不会真正修改 device 内容；该方法依赖于 device 是否支持写保护功能；

软件写保护是指通过软件的方法，在 host driver 端对所有写请求进行过滤，对保护区域的写请求直接返回 I/O error；该方法比较灵活，不依赖与硬件，但是在 kernel 中才能生效，而启动到 kernel 之前不起作用；

本节介绍在 AC8025 上 eMMC 硬件写保护（power on）功能，主要介绍什么是临时硬件写保护以及如何对一个分区进行写保护。

此部分功能依赖于 eMMC device 是否支持临时硬件写保护。

36.2.5.1 背景介绍

写保护指对于 eMMC 上面某些 regions 进行写操作保护，即针对这些 regions 的写操作都会返回 fail，不能真正的修改 device 上面的数据。在 AC8025 中，对具有只读属性的关键系统分区进行写保护，防止意外破坏导致系统异常。

一般情况下，有三种硬件写保护机制：

1. 临时写保护

即跟 eMMC 通电状态没有关系，功能的 enable 以及 disable 都需要通过 cmd 进行设置。

2. 永久写保护

即对指定 regions 进行永久性的写保护，一旦使能不能解除，该方法使用场景较少。

3. 上电写保护

该方法在上电时对指定 regions 进行写保护，在不掉电的期间，写保护持续生效，一旦掉电，写保护失效，如果需要再次进行写保护，需要在每次上电时使能写保护功能。

硬件写保护具有一个限制条件：起始、结束地址必须按照 protect group 对齐。

Protect group 计算方法有两种，根据 EXT_CSD[175] 寄存器中 EXT_CSD_ERASE_GRP_DEF 的值。

- 如果该值为 0，wp_size 等于 CSD 寄存器中 WP_GRP_SIZE；
- 如果该值为 1，wp_size 等于 EXT_CSD 寄存器中 HC_WP_GRP_SIZE。

目前 AC8025 中该值为 0，即 wp_size 等于 WP_GRP_SIZE。

```
// source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/sd4hc/mmc_core.c
//compute wp_size
if (ext_csd[EXT_CSD_ERASE_GRP_DEF] & EXT_CSD_ERASE_GRP_DEF_EN) {
//should be the sector size
    card->wp_size = (card->ext_csd.hc_wp_grp_sz * card-
>ext_csd.hc_erase_grp_sz) >> 9;
    printf("[SD%d] EXT_CSD_ERASE_GRP_DEF is On, wp_size = %dKB\n",
        card->host->id, card->wp_size/2);
} else {
    card->wp_size = (card->csd.write_prot_grpsz + 1) * card->csd.erase_sctsz;
    printf("[SD%d] EXT_CSD_ERASE_GRP_DEF is Off, wp_size = %dKB\n"
        "[SD%d] csd.write_prot_grpsz = %d, csd.erase_sctsz = %d\n",
        card->host->id, card->wp_size/2,
```

```
card->host->id,card->csd.write_prot_grpsz,card-
>csd.erase_sctsz);
}
```

EXT_CSD_ERASE_GRP_DEF is Off, wp_size = 8MB, csd.write_prot_grpsz = 15,csd.erase_sctsz = 1024

即起始、结束地址需按照 8M 对齐，该 wp_size 根据 eMMC 厂家的不同，数值可能不一样。当前为了兼容不同 eMMC device 的写保护 group size 对齐要求，默认配置为 32M 对齐。

出于 32M 写保护区域大小对齐的要求，需要预留一部分 eMMC 空间，因此用户分区划分到 eMMC 上可能会出现写保护区域最后一个分区的结束地址和非写保护区域的第一个分区的起始地址不连续的情况。

如果起始、结束地址不满足该条件，则可能会导致 eMMC 硬件写保护功能配置异常。

36.2.5.2 Protect Groups

在 AC8025 中提供的 protect region 是 4 个，每个 region 的起始、结束地址需要按照 32M 进行对齐。当前默认使用 1 个 region 进行硬件写保护，当前该 region 包含分区如下：

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	partname	image_type	fs_type	region	size	imagename	dynamic	r_min	initial_size	operation	update	erase	fastboot	emmc_hw_wp	emmc_sw_wp	mountable	recovery
2	preloader	raw	raw	BOOT	512KB	preloader.img	Y		bootload	Y	Y	Y	Y	Y	Y	N	Y
3	pspt	raw	raw	USER	1MB		N		update	N	Y	N	N	Y	Y	N	N
4	hsm_a	raw	raw	USER	6MB	hsm.img	Y		update	Y	Y	Y	Y	Y	Y	N	Y
5	lk_a	raw	raw	USER	2MB	lk.img	Y		update	Y	Y	Y	Y	Y	Y	N	Y
6	trustzone_a	raw	raw	USER	4MB	trustzone.img	Y		update	Y	Y	Y	Y	Y	Y	N	Y
7	vendor_boot_a	raw	raw	USER	96MB	vendor_boot.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
8	boot_a	raw	raw	USER	64MB	boot.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
9	recovery_a	raw	raw	USER	70MB	recovery.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
10	vbmeta_a	raw	raw	USER	1MB	vbmeta.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
11	vbmeta_system_a	raw	raw	USER	1MB	vbmeta_system.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
12	vbmeta_vendor_a	raw	raw	USER	1MB	vbmeta_vendor.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
13	boot_misc_a	raw	raw	USER	1MB	boot_misc.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
14	vendor_a	sparse	ext4	USER	512MB	vendor.irmain	N		update	Y	Y	Y	Y	Y	Y	Y	Y
15	system_a	sparse	ext4	USER	3GB	system.irmain	N		update	Y	Y	Y	Y	Y	Y	Y	Y
16	vendor_dtlm_a	sparse	ext4	USER	512MB	vendor_dtlm	N		update	Y	Y	Y	Y	Y	Y	Y	Y
17	product_a	sparse	ext4	USER	1GB, 512MB	product.irmain	N		update	Y	Y	Y	Y	Y	Y	N	Y
18	datazone-ro_a	raw	raw	USER	100MB	datazone-ro.img	N		update	Y	Y	Y	Y	Y	Y	N	N
19	dtbo_a	raw	raw	USER	1MB	dtbo.img	N		update	Y	Y	Y	Y	Y	Y	N	N
20	hsm_b	raw	raw	USER	6MB	hsm.img	Y		update	Y	Y	Y	Y	Y	Y	N	Y
21	lk_b	raw	raw	USER	2MB	lk.img	Y		update	Y	Y	Y	Y	Y	Y	N	Y
22	trustzone_b	raw	raw	USER	4MB	trustzone.img	Y		update	Y	Y	Y	Y	Y	Y	N	Y
23	vendor_boot_b	raw	raw	USER	96MB	vendor_boot.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
24	boot_b	raw	raw	USER	64MB	boot.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
25	recovery_b	raw	raw	USER	70MB	recovery.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
26	vbmeta_b	raw	raw	USER	1MB	vbmeta.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
27	vbmeta_system_b	raw	raw	USER	1MB	vbmeta_system.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
28	vbmeta_vendor_b	raw	raw	USER	1MB	vbmeta_vendor.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
29	boot_misc_b	raw	raw	USER	1MB	boot_misc.img	N		update	Y	Y	Y	Y	Y	Y	N	Y
30	vendor_b	sparse	ext4	USER	512MB	vendor.irmain	N		update	Y	Y	Y	Y	Y	Y	Y	Y
31	system_b	sparse	ext4	USER	3GB	system.irmain	N		update	Y	Y	Y	Y	Y	Y	Y	Y
32	vendor_dtlm_b	sparse	ext4	USER	512MB	vendor_dtlm	N		update	Y	Y	Y	Y	Y	Y	Y	Y
33	product_b	sparse	ext4	USER	1GB, 512MB	product.irmain	N		update	Y	Y	Y	Y	Y	Y	N	Y
34	dtbo_b	raw	raw	USER	1MB	dtbo.img	N		update	Y	Y	Y	Y	Y	Y	N	N
35	super	sparse	raw	USER	14GB	super.img	N		update	Y	Y	Y	Y	Y	Y	N	N
36	datazone-ro_b	raw	raw	USER	100MB	datazone-ro.img	N		update	Y	Y	Y	Y	Y	Y	N	N
37	metazonebk	raw	raw	USER	1MB		N		update	Y	Y	Y	Y	Y	Y	N	N
38	datazone-rw-a	raw	raw	USER	100MB	datazone-rw.img	N		update	Y	Y	Y	Y	N	N	N	N
39	datazone-rw-b	raw	raw	USER	100MB	datazone-rw.img	N		update	Y	Y	Y	Y	N	N	N	N
40	expdb	raw	raw	USER	30MB		N		update	Y	Y	Y	Y	N	N	N	N
41	cache	sparse	ext4	USER	400MB	cache.img	N		update	Y	Y	Y	Y	N	N	Y	N
42	fastdata	sparse	ext4	USER	100MB	fastdata.img	N		update	Y	Y	Y	Y	N	N	Y	N
43	misc	raw	raw	USER	2MB		N		update	Y	Y	Y	Y	N	N	N	N

图 36-9 AC8025 Protect Region 示例

其中 emmc_hw_wp 为配置硬件写保护的 flag，emmc_sw_wp 为配置软件写保护的 flag。

36.2.5.3 enable/disable 硬件写保护

为了方便开关硬件写保护功能，提供如下接口。

首先执行如下命令登录 shell：

1. adb root
2. adb shell

然后通过如下命令开关及查询硬件写保护状态（以 boot_b 分区为例）：

1. 使能某个分区的硬件写保护：echo “emmc_hw_wp boot_b enable” > /proc/mmc_debug
2. 关闭某个分区的硬件写保护：echo “emmc_hw_wp boot_b disable” > /proc/mmc_debug
3. 查询写保护状态：echo emmc_check_wp > /proc/mmc_debug
4. 使能整体写保护区域的软硬件写保护：echo emmc_enable_all_wp > /proc/mmc_debug
5. 关闭整体写保护区域的软硬件写保护：echo emmc_disable_all_wp > /proc/mmc_debug

36.2.5.4 如何添加一个需要/不需要硬件写保护功能的分区

如果需要添加一个不需要写保护的分区，建议如下：

1. 如图 36-9 所示，在 metazonebk 分区之后添加该分区；
2. 设置 emmc_hw_wp 列为 0；

如果需要添加一个需要写保护的分区，根据前文介绍，建议如下：

1. 建议添加在 datazone-rw-a 分区之前；
2. 设置 emmc_hw_wp 列为 1；
3. 所有 protect group 总和不大大于 5。

36.2.6 软件写保护

36.2.6.1 enable/disable 软件写保护

为了方便开关软件写保护功能，提供如下接口。

首先执行如下命令登录 shell：

1. adb root
2. adb shell

然后通过如下命令开关及查询软件写保护状态（以 boot_b 分区为例）：

1. 使能某个分区的软件写保护：echo “emmc_sw_wp boot_b enable” > /proc/mmc_debug

2. 关闭某个分区的软件写保护: `echo "emmc_sw_wp boot_b disable" > /proc/mmc_debug`
3. 查询写保护状态: `echo emmc_check_wp > /proc/mmc_debug`
4. 使能整体写保护区域的软硬件写保护: `echo emmc_enable_all_wp > /proc/mmc_debug`
5. 关闭整体写保护区域的软硬件写保护: `echo emmc_disable_all_wp > /proc/mmc_debug`

36.2.6.2 如何添加一个需要/不需要软件写保护分区

在 partition_table Excel 中添加分区后, 在 emmc_sw_wp 列选择读写的属性, 如果需要进行软件写保护, 则选择 Y, 不需要软件写保护则选择 N。

Partition_table Excel 路径: build/projects/{\$project}/partition_table/emmc_ab/。

36.3 接口描述

为统一 emmc/sd/ufs 对外使用的 api, 在 emmc/sd/ufs 之上封装一层 storage common api, 如下所示, 建议使用 storage_read/storage_write api 去读写 emmc/sd/ufs, 通过设备 dev_num 区分。

1. UFS 设备 dev_num 是 BOOT_UFS_SLOT
2. eMMC 设备 dev_num 是 BOOT_SD_SLOT0; SD1、SD2 在 Preloader/Safety os/LK 阶段的 dev_num 对应 BOOT_SD_SLOT1、BOOT_SD_SLOT2

36.3.1 eMMC/SD Driver 在 preloader 中提供的接口

表 36-7 eMMC/SD 在 preloader 的接口描述

Methods in eMMC/SD System	Description
storage_read	eMMC/SD 读数据接口
storage_write	eMMC/SD 写数据接口
storage_get_blksize	获取eMMC/SD 的blk size
storage_switch_part	切换到eMMC的boot/rpmb/user

36.3.2 eMMC/SD Driver 在 Safety OS 中提供的接口

表 36-8 eMMC/SD 在 safety os 的接口描述

Methods in eMMC/SD System	Description
storage_read	eMMC/SD 读数据接口
storage_write	eMMC/SD 写数据接口
storage_get_blksize	获取eMMC/SD 的blk size
storage_switch_part	切换到eMMC 的boot/rpmb/user
storage_get_capacity_blk	获取eMMC的user region的size, 单位是blk
storage_clear_all_wp	解除eMMC 的写保护

36.3.3 eMMC/SD Driver 在 LK 中提供的接口

表 36-9 eMMC/SD 在 LK 的接口描述

Methods in eMMC/SD System	Description
storage_read	eMMC/SD 读数据接口
storage_write	eMMC/SD 写数据接口
storage_erase	eMMC/SD 擦除接口
storage_get_blksize	获取 eMMC/SD 的 blk size
storage_get_capacity_blk	获取 eMMC 的 user region 的 size，单位是 blk
storage_switch_part	切换到 eMMC 的 boot/rpmb/user
storage_clear_all_wp	解除 eMMC 的写保护

36.4 使用说明

36.4.1 如何在 Preloader 中配置 eMMC/SD 工作模式

在如下头文件的 sd4hc_cap[HOST_MAX_SLOTS] 结构体数组中修改配置项：

source/vendor/autochips/proprietary/tinysys/hsm/loader/include/ac8025/sd4hc_cust.h

sd4hc_cap[HOST_MAX_SLOTS] 包含三个元素（HOST_MAX_SLOTS = 3），分别对应 SD0、SD1、SD2。HSM Loader 阶段配置 SD0、SD1 对应的 sd4hc_cap[0]、sd4hc_cap[1]，分别对应 eMMC、SD 卡，SD2 对应的 SDIO 设备只在 kernel 阶段启用。

36.4.1.1 配置 eMMC 工作模式

```
struct sd4hc_cust sd4hc_cap[HOST_MAX_SLOTS] = {
{
    .clk_src = SRC_CLK_FREQ_200M,
    .clk_max = 200000,
    .dev_type = DEV_TYPE_EMMC,
    .data_pins = 8,
#ifdef SDHC_USE_DMA
    .flags = SD4HC_EMMC_MODE_HS200 | SD4HC_EMMC_MODE_HS |
SD4HC_EMMC_MODE_LEGACY | SD4HC_USE_SDMA,
#else
    .flags = SD4HC_EMMC_MODE_HS200 | SD4HC_EMMC_MODE_HS |
SD4HC_EMMC_MODE_LEGACY | SD4HC_USE_PIO,
#endif
},
},
};
```

图 36-10 eMMC 工作模式配置项

说明：

- clk_src：选择 SD4HC 的输入时钟源，此项不建议修改
- clk_max：限制 Device 工作的最高频率，单位为 KHz

- dev_type: Host 连接的设备类型，此项需要和 HW 设计一致，不可随意修改
- flags: 不需要开启的选项注释掉即可
 - SD4HC_USE_SDMA 和 SD4HC_64_BIT_ADDRESSING 需要同时设置，表示开启 SDMA 模式
 - SD4HC_USE_PIO 表示开启 PIO 传输模式，和 SD4HC_USE_SDMA 同时设置时优先使用 SDMA 模式
 - 其余选项为 SD4HC 支持的 eMMC 传输模式，默认支持 HS200/HighSpeed/Default Speed 模式。

36.4.1.2 配置 SD Card 工作模式

```

{
    .clk_src = SRC_CLK_FREQ_200M,
    .clk_max = 50000,
    .dev_type = DEV_TYPE_SD,
    .data_pins = 4,
    #if defined(SDHC_USE_DMA)
    .flags = SD4HC_SD_MODE_HS | SD4HC_SD_MODE_DEFAULT | SD4HC_USE_SDMA,
    #else
    .flags = SD4HC_SD_MODE_HS | SD4HC_SD_MODE_DEFAULT | SD4HC_USE_PIO,
    #endif
},
    
```

图 36-11 lk 中 SD Card 工作模式配置项

说明:

- clk_src: 选择 SD4HC 的输入时钟源，此项不建议修改
- clk_max: 限制 Device 工作的最高频率，单位为 KHz
- dev_type: Host 连接的设备类型，此项需要和 HW 设计一致，不可随意修改
- data_pins: Bus 宽度，SD Card 支持 4/1 bits 两种模式，建议设置为 4
- flags: 不需要开启的选项注释掉即可
 - SD4HC_USE_SDMA 表示开启 SDMA 模式
 - SD4HC_USE_PIO 表示开启 PIO 传输模式，和 SD4HC_USE_SDMA 同时设置时优先使用 SDMA 模式
 - 其余选项为 SD4HC 支持的 SD 传输模式，默认支持 HighSpeed/Default Speed 模式。注意：如需要使用 UHS 的 mode 需要同时将 SD4HC_SUPPORT_UHS1 选项打开。

36.4.2 如何在 Kernel 中配置 eMMC/SD/SDIO 工作模式

在 source/kernel/common/arch/arm64/boot/dts/autochips/ac8025.dtsi 文件以及 ac8025-demo.dts 中配置对应节点 sd0/sd1/sd2 的属性来控制 kernel 中 eMMC/SD/SDIO 的工作模式。相关的配置项参考 36.2.3.1 章节的内容。

以连接 eMMC 的 sd0 节点配置片段为例：

ac8025.dtsi:

```
max-frequency = <200000000>;
non-removable;
bus-width = <8>;
cap-mmc-highspeed;
mmc-hs200-1_8v;
no-sdio;
no-sd;
cap-mmc-hw-reset;
host-id = <0>;
status = "okay";
```

ac8025_demo.dts

```
&sd0 {
    mmc-hs400-1_8v;
    /*mmc-hs400-enhanced-strobe;*/
    phy-dll-delay-strobe = <8>;
    phy-dll-delay-sdclk-hsmmc = <0>;
    pinctrl-names = "sd-drive-strength";
    pinctrl-0 = <&sd0_drive_strength>;
};
```

图 36-12 kernel 中 eMMC dts 配置示例

该配置片段说明：

- max-frequency: 限制 eMMC Device 最高工作频率为 200MHz
- non-removable: eMMC 设备不可移除
- bus-width: Bus 宽度设置为 8
- cap-mmc-highspeed: 支持 eMMC HS 传输模式
- mmc-hs200-1_8v: 支持 eMMC HS200 传输模式
- mmc-hs400-1_8v: 支持 eMMC HS400 传输模式，driver 会根据 cap-mmc-highspeed、mmc-hs200-1_8v、mmc-hs400-1_8v 的配置以及 eMMC Device 的 abilities 选择性能最高的传输模式
- cap-mmc-hw-reset: 支持 eMMC HW reset 功能
- no-sdio: sd0 连接的 Device 没有 SDIO 功能，该项可加快 Device 初始化速度
- no-sd: sd0 连接的 Device 没有 SD Card 功能，该项可加快 Device 初始化速度

36.4.3 如何查看当前 SD/eMMC 工作状态

kernel 有提供接口显示当前 sd/emmc 的工作状态，它们在 /sys/kernel/debug/ 目录下：

(mmc0 表示 emmc, mmc1 表示 SD 卡)

```
/sys/kernel/debug # cat mmc0/ios
clock:                200000000 Hz
actual clock:         200000000 Hz
vdd:                  21(3.3~3.4v)
bus mode:             2(push-pull)
chip select:          0(don't care)
power mode:           2(on)
bus width:            3(8 bits)
timing spec:          10(mmc HS400)
signal voltage:       1 (1.80 V)
driver type:          0(driver type B)
```

上述状态说明如下：

1. clock: 表示当前该 device 的 clock 数值；
2. cdd: eMMC 的供电电压；
3. bus mode: 该处对应 MMC_BUSMODE_PUSHPULL，另外一种为 open_drain (MMC_BUSMODE_OPENDAIN)；
4. chip select: 只针对 SPI mode，当前没有意义；
5. power mode: 当前的电压状态，包括 MMC_POWER_OFF、MMC_POWER_UP、MMC_POWER_ON、MMC_POWER_UNDEFINED；
6. bus width: 总线的宽度，包含 1 bit (MMC_BUS_WIDTH_1)、4 bit (MMC_BUS_WIDTH_4)、8 bit (MMC_BUS_WIDTH_8)；
7. timing: 使用哪一种总线时序，分别有如下值：

```
MMC_TIMING_LEGACY
MMC_TIMING_MMC_HS
MMC_TIMING_SD_HS
MMC_TIMING_UHS_SDR12;
MMC_TIMING_UHS_SDR25
```

```
MMC_TIMING_UHS_SDR50
MMC_TIMING_UHS_SDR104
MMC_TIMING_UHS_DDR50
MMC_TIMING_MMC_DDR52
MMC_TIMING_MMC_HS200
MMC_TIMING_MMC_HS400
```

8. signal_voltage: 总线信号使用哪一种电压, 3.3v (MMC_SIGNAL_VOLTAGE_330)、1.8v (MMC_SIGNAL_VOLTAGE_180) 或者 1.2v (MMC_SIGNAL_VOLTAGE_120)。
9. driver type: eMMC device 的驱动类型, 默认为 driver type B。

36.4.4 硬件写保护功能验证

验证临时写保护可以用如下命令 (平台没有 remount 过) :

```
adb root
adb shell
# dd of=/dev/test.img if=/dev/zero count=1 bs=512
# echo 123456789 > /dev/test.img
# dd if=/dev/test.img of=/dev/block/boot_b count=1 bs=512
# dd of=/dev/readback.img if=/dev/block/boot_b count=1 bs=512
# cat /dev/readback.img
```

如果硬件写保护 enable 的情况下, readback.img 前面没有 test.img 中的 123456789。如果写保护没有 enable, 那边 readback.img 跟 test.img 前部分应该一致。

36.4.5 如何验证软件写保护

因为软件写保护与硬件写保护区域一致, 在验证软件写保护时需要先关闭硬件写保护, 在此前提下进行验证软件写保护功能:

1. 通过 adb 使能软件写保护;
2. 通过 dd 命令修改 boot_b 分区数据;
3. 分析 log, 有“write protect violation, abort it”, 此处表明 sd4hc driver 过滤了对受保护区间的写操作;
4. 重启机器看发现能正常进入系统。



注意：

在软件写保护没有使能的情况下进行以上操作，会发现无法正常进入系统，因为 kernel 分区数据被人为损坏了。

36.5 常见问题

36.5.1 系统 remount 之后的写保护状态

【问题描述】

针对 userdebug 系统可能存在用户执行 adb remount 的操作，如何判断系统是否被 remount 过？

【解决方案】

1. 通过 cat /proc/cmdline 查看属性"androidboot.device_remounted"的值，false 表示没有 remount，true 表示平台处于 remounted 状态；
2. 通过 getprop 命令获取属性"ro.boot.device_remounted"的值，false 表示没有 remount，true 表示平台处于 remounted 状态。

【问题描述】

针对 userdebug 版本，如果平台已经处于 remounted 状态，eMMC 的写保护是什么状态？

【解决方案】

1. 平台执行 adb remount 操作，则系统 eMMC 的软硬件写保护状态会被解除；
2. 重启系统也不会重新配置 eMMC 的硬件写保护；
3. 如果想要恢复 remount 之前的写保护状态需要重新烧写系统软件。

36.5.2 重要 error log 分析

【问题描述】

初步排查 SD4HC error log？

【解决方案】

有时候，会在 log 中看到 SD4HC driver 打印的如下错误类型：

- SDHCI_INT_TIMEOUT: host 在限定时间内没有收到 device 的 command response。
- SDHCI_INT_DATA_END_BIT: data line 传输的 end bit 错误。

- SDHCI_INT_DATA_TIMEOUT: 等待数据发生超时。
- SDHCI_INT_DATA_CRC: 数据 CRC 校验错误。
- SDHCI_INT_ADMA_ERROR: ADMA 传输错误。

部分情况下，出现上述 log 不会引起严重问题，因为 driver 可以通过 retry、tuning、reset 等机制来重试或调整传输参数来解决这些错误。这些错误恢复机制对上层是透明的，如果 driver 尝试后仍然不能解决相关的错误，才会向 block 层返回错误信息。

例如当 driver 在执行 tuning 操作对时序参数调优时，在某些 tuning 参数设置下可能会出现 SDHCI_INT_DATA_CRC 或 SDHCI_INT_DATA_TIMEOUT 等错误，tuning 算法会剔除掉发生错误的参数配置，这种打印出来的错误信息并不会影响系统正常运行。

如果系统运行出现异常，并且在异常点附近 log 中出现上述错误信息，那么就需要重点关注，并针对具体问题具体分析。

36.5.3 硬件写保护 group size

【问题描述】

如何计算硬件写保护 group size?

【解决方案】

如 36.2.5.1 中介绍，wp_size 的计算方法是根据 EXT_CSD、CSD 寄存器中的值进行计算的，常见的有 4M、8M、16M 等，需要根据相关寄存器的值进行具体计算。

37 UFS

37.1 简介

37.1.1 软硬件环境

37.1.1.1 软件环境

AC8025 SoC UFS Host 支持到 UFS 2.1，UFS 可作为系统的主存储，支持从 UFS 设备启动 Android 系统。

37.1.1.2 硬件环境

AC8025 上 UFS Host 和 UFS device 的接口原图如下。

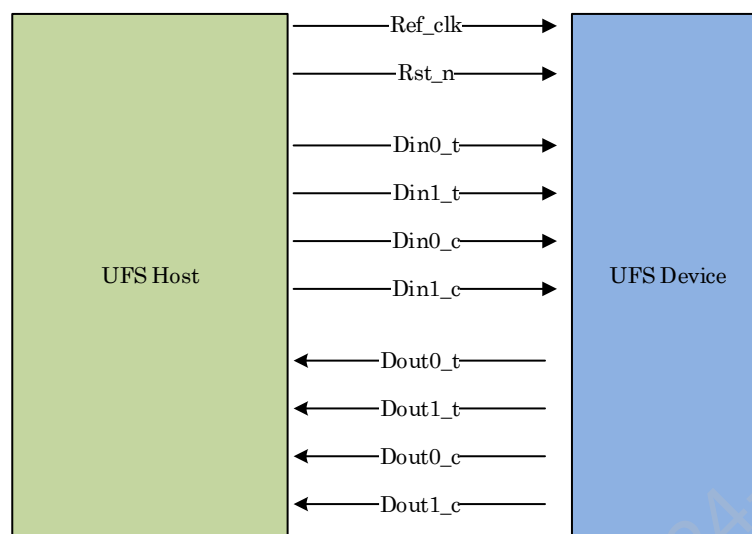


图 37-1 UFS Host 和 UFS Device 原图示意图

- R_st : ufs HW Reset pin
- Ref_Clk : reference clock
- Din0_t:/c downstream data lane0 input
- Din1_t:/c downstream data lane1 input
- Dout0_t:/c upstream data lane0 output
- Dout1_t:/c upstream data lane1 output

37.1.2 功能

UFS 具有功能：

- 支持 UFS 2.1
- 支持 HS Mode 的 g1/g2/g3
- 支持 1/2 line
- 支持 RPMB/BOOT/User 的 Lun 读写
- 支持 UFS Write Protection

37.1.3 代码结构及文件描述

Table 37-1 UFS 代码结构及描述

文件名	文件路径	文件描述
ufs-atc.c ufs-dbg.c ufs-rpmb.c	source/kernel/common/scsi/ufs/ac8x/	Kernel ufs host driver
ufs_interface.c ufs_core.c ufs_rpmb.c ufs_hcd.c	source/vendor/autochips/proprietary/tinysys/hsm/loader/driver/ufs	Hsm loader ufs driver
ufs_interface.c ufs_core.c ufs_rpmb.c ufs_hcd.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/ufs/	Hsm os ufs driver
ufs_interface.c ufs_core.c ufs_rpmb.c ufs_hcd.c	source/vendor/autochips/proprietary/bootable/lk/platform/autochips/ac8x/ufs/	Lk ufs driver

37.2 软件设计

37.2.1 UFS 在 HSM Loader/HSM OS/ LK 中的架构图

UFS Driver 在 loader 阶段，仅提供最基本的操作：

- UFS 的初始化
- 提供访问 UFS boot/rpmb/user 读写操作接口

- Hsm/LK 支持 enable/disable UFS 的 write protection

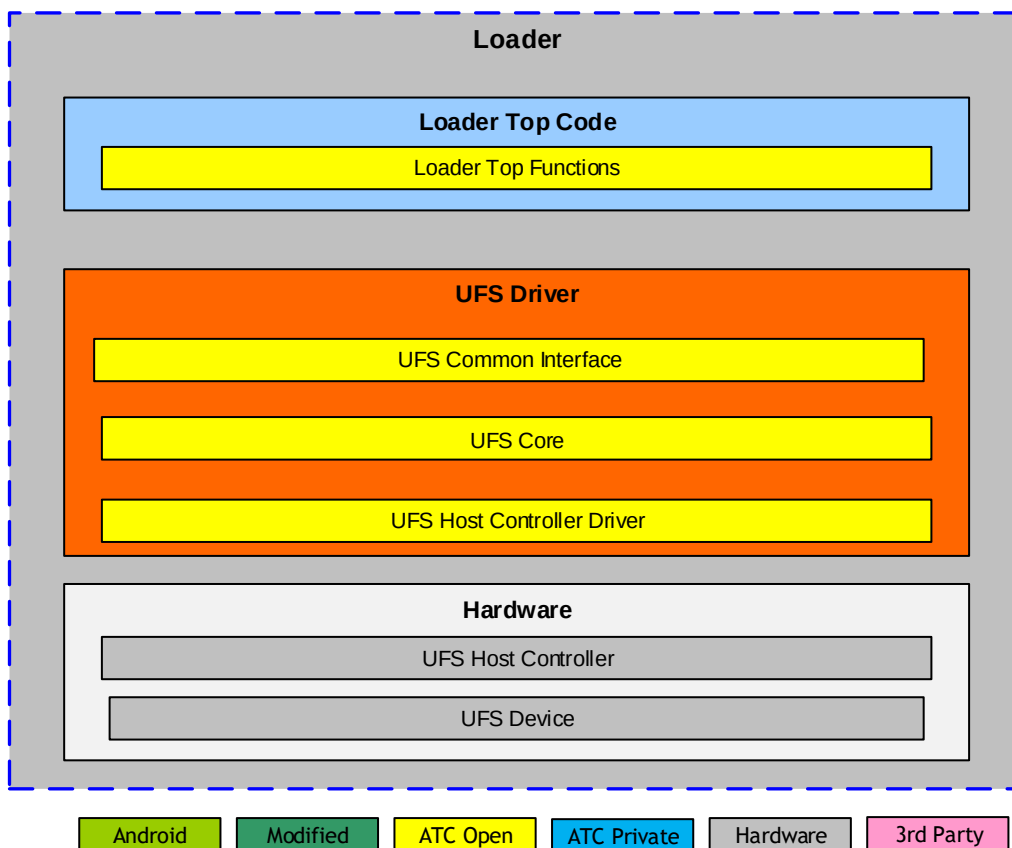


图 37-2 UFS HL/HSM/LK Driver 架构图

37.2.2 UFS 在 Kernel 中的架构图

Kernel 中的 UFS Driver 提供如下的功能:

- UFS 的初始化
- 支持 UFS HW/SW Write Protection
- 提供 UFS debug 节点操作

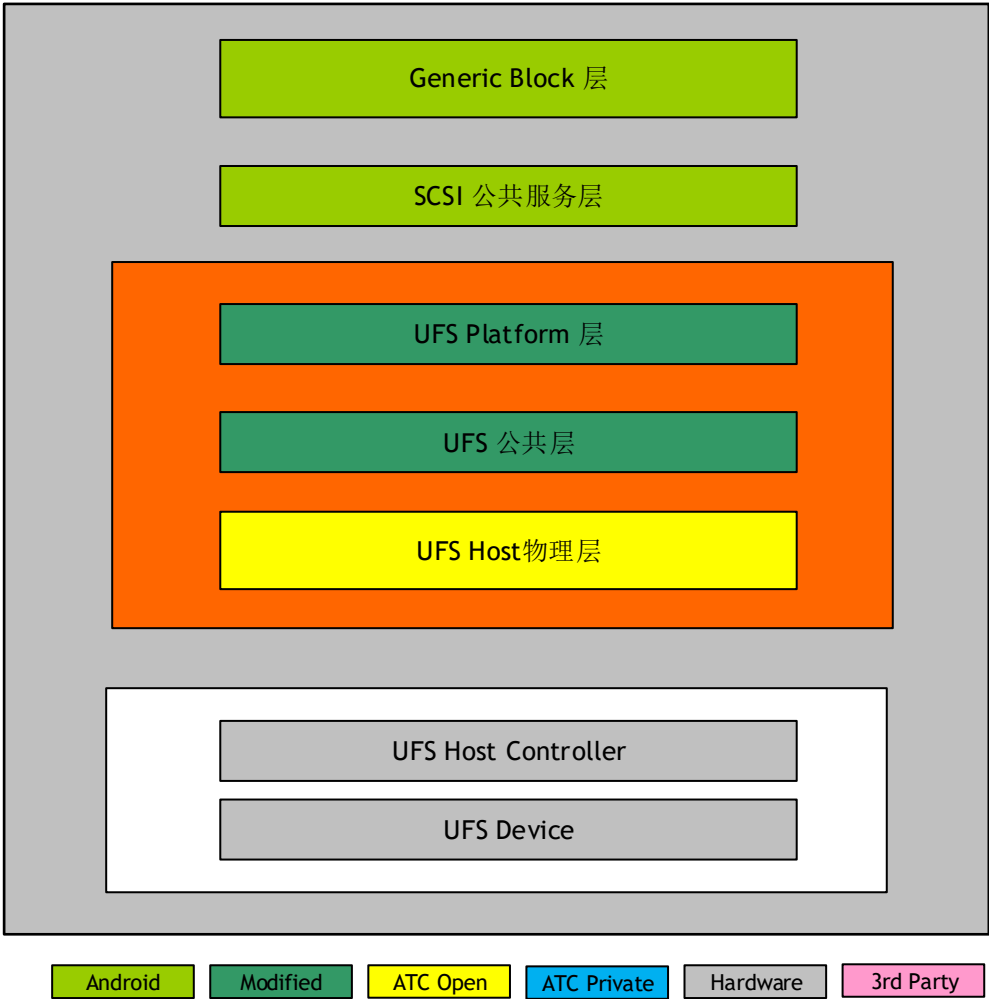


图 37-3 UFS Kernel driver 架构图

37.3 接口描述

37.3.1 UFS Driver 在 Loader 中提供的接口

表 37-1 UFS 在 hsm loader 的接口描述

Methods in UFS System	Description
storage_read	UFS 读数据接口
storage_write	UFS 写数据接口
Storage_get_blksz	获取UFS 的blk size
Storage_switch_part	切换到UFS 的boot/rpmb/user

37.3.2 UFS Driver 在 HSM OS 中提供的接口

表 37-2 UFS 在 hsm os 的接口描述

Methods in UFS System	Description
storage_read	UFS 读数据接口
storage_write	UFS 写数据接口
Storage_get_blksize	获取UFS 的blk size
Storage_switch_part	切换到UFS 的boot/rpmb/user
storage_get_capacity_blk	获取UFS 的LU 的size， 单位是blk
Storage_clear_all_wp	解除UFS 的写保护

37.3.3 UFS Driver 在 LK 中提供的接口

表 37-3 UFS 在 lk 的接口描述

Methods in UFS System	Description
storage_read	UFS 读数据接口
storage_write	UFS 写数据接口
Storage_erase	UFS 擦除接口
Storage_get_blksize	获取UFS 的LU 的size， 单位是blk
storage_get_capacity_blk	获取UFS的write protect group size， 以block为单位
Storage_switch_part	切换到UFS 的boot/rpmb/user
Storage_clear_all_wp	解除UFS 的写保护

37.3.3.1 storage_read

【描述】

device 读数据接口

【语法】

```
int storage_read(uint8 dev_num, uint64_t offset, uint64_t size, u64 dst);
```

【参数】

- dev_num: ufs 的 device num， 使用 ipc_lite.h 中 BOOT_UFS_SLOT
- offset: 访问 device 的偏移 地址， 单位是 byte
- size: 访问 device 的大小， 单位是 byte
- src: 读取到 memory 中的地址

【返回值】

- 0x0: succeed
- 非 0x0: 接口执行出错

37.3.3.2 storage_write

【描述】

device 写数据接口

【语法】

```
int storage_write(uint8 dev_num, uint64_t offset, uint64_t size, u64 src);
```

【参数】

- dev_num: ufs 的 device num，使用 ipc_lite.h 中 BOOT_UFS_SLOT
- offset: 访问 device 的偏移地址，单位是 byte
- size: 访问 device 的大小，单位是 byte
- src: 存储写数据的 memory 地址

【返回值】

- 0x0: succeed
- 非 0x0: 接口执行出错

37.3.3.3 storage_erase

【描述】

device 擦除数据接口

【语法】

```
int storage_erase(uint8 dev_num, uint64_t offset, uint64_t size);
```

【参数】

- dev_num: ufs 的 device num，使用 ipc_lite.h 中 BOOT_UFS_SLOT
- offset: erase device 的偏移地址，单位是 byte
- size: erase device 的大小，单位是 byte

【返回值】

- 0x0: succeed

- 非 0x0: 接口执行出错

37.3.3.4 storage_get_blksize

【描述】

获取 device 的 block size 大小

【语法】

```
int storage_get_blksize(uint8 dev_num, uint32_t *blksize);
```

【参数】

- dev_num: ufs 的 device num , 使用 ipc_lite.h 中 BOOT_UFS_SLOT
- blksize: 存储 blk size ,单位是 byte

【返回值】

- 0x0: succeed
- 非 0x0: 接口执行出错

37.3.3.5 storage_get_capacity_blk

【描述】

获取 device 的 容量大小, 单位 blk

【语法】

```
u32 storage_get_capacity_blk(uint8 dev_num, uint64_t *blkcnt);
```

【参数】

- dev_num: ufs 的 device num , 使用 ipc_lite.h 中 BOOT_UFS_SLOT
- blkcnt: 存储 blk count

【返回值】

- 0x0: succeed
- 非 0x0: 接口执行出错

37.3.3.6 storage_clear_all_wp

【描述】

解除 device 的硬件写保护区间

【语法】

```
u32 storage_clear_all_wp(uint8 dev_num);
```

【参数】

dev_num: ufs 的 device num，使用 ipc_lite.h 中 BOOT_UFS_SLOT

【返回值】

- 0x0: succeed
- 非 0x0: 接口执行出错

37.4 使用说明和注意事项

37.4.1 配置 UFS 的写保护区间

跟 eMMC 配置写保护区区间的原理一样，在 Partition Table 配置相应 Partition 的 hw_wp/sw_wp 的 flag 即可。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
	partname	image_type	fs_type	region	size	image name	dynamic	minimum	operation	upgrade	asab	fastboot	copy	hw_wp	sw_wp	mountable
1	preloader	raw	raw	BOOT	512KB	preloader.img	Y		bootload	Y	Y	Y	Y	Y	Y	N
2	pgpt	raw	raw	USER	1MB		N		invisible	N	Y	N	N	Y	Y	N
3	hsm_a	raw	raw	USER	2MB	hsm.img	Y		update	Y	Y	Y	Y	Y	Y	N
4	lk_a	raw	raw	USER	2MB	lk.img	Y		update	Y	Y	Y	Y	Y	Y	N
5	trustzone_raw	raw	raw	USER	4MB	trustzone.img	Y		update	Y	Y	Y	Y	Y	Y	N
6	vendor_boc	raw	raw	USER	96MB	vendor_boot.img	N		update	Y	Y	Y	Y	Y	Y	N
7	boot_a	raw	raw	USER	64MB	boot.img	N		update	Y	Y	Y	Y	Y	Y	N
8	recovery_araw	raw	raw	USER	70MB	recovery.img	N		update	Y	Y	Y	Y	Y	Y	N
9	vbmeta_a	raw	raw	USER	1MB	vbmeta.img	N		update	Y	Y	Y	Y	Y	Y	N
10	vbmeta_sys	raw	raw	USER	1MB	vbmeta_system.img	N		update	Y	Y	Y	Y	Y	Y	N
11	vbmeta_ver	raw	raw	USER	1MB	vbmeta_vendor.img	N		update	Y	Y	Y	Y	Y	Y	N
12	dtb_a	raw	raw	USER	1MB	dtb.img	N		update	Y	Y	Y	Y	Y	Y	N
13	vendor_a	sparse	ext4	USER	512MB	vendor.img main	N		update	Y	Y	Y	Y	Y	Y	Y
14	system_a	sparse	ext4	USER	2GB	system.img main	N		update	Y	Y	Y	Y	Y	Y	Y
15	vendor_dlk	sparse	ext4	USER	512MB	vendor_dlk main	N		update	Y	Y	Y	Y	Y	Y	Y
16	product_a	sparse	ext4	USER	1GB, 512MB	product.img main	N		update	Y	Y	Y	Y	Y	Y	N
17	hsm_b	raw	raw	USER	2MB	hsm.img	Y		update	Y	Y	Y	Y	Y	Y	N
18	lk_b	raw	raw	USER	2MB	lk.img	Y		update	Y	Y	Y	Y	Y	Y	N
19	trustzone_raw	raw	raw	USER	4MB	trustzone.img	Y		update	Y	Y	Y	Y	Y	Y	N
20	vendor_boc	raw	raw	USER	96MB	vendor_boot.img	N		update	Y	Y	Y	Y	Y	Y	N
21	boot_b	raw	raw	USER	64MB	boot.img	N		update	Y	Y	Y	Y	Y	Y	N
22	recovery_br	raw	raw	USER	70MB	recovery.img	N		update	Y	Y	Y	Y	Y	Y	N
23	vbmeta_b	raw	raw	USER	1MB	vbmeta.img	N		update	Y	Y	Y	Y	Y	Y	N
24	vbmeta_sys	raw	raw	USER	1MB	vbmeta_system.img	N		update	Y	Y	Y	Y	Y	Y	N
25	vbmeta_ver	raw	raw	USER	1MB	vbmeta_vendor.img	N		update	Y	Y	Y	Y	Y	Y	N
26	dtb_b	raw	raw	USER	1MB	dtb.img	N		update	Y	Y	Y	Y	Y	Y	N
27	vendor_b	sparse	ext4	USER	512MB	vendor.img main	N		update	Y	Y	Y	Y	Y	Y	Y



注意:

- UFS 支持最大 4 个写保护区间(4 个地址分别连续的地址空间)

38 FastTouch

38.1 简介

38.1.1 软硬件环境

38.1.1.1 软件环境

需要支持 HSM。

38.1.1.2 硬件环境

需要使用 I2C 接口的 touchscreen。

- AC8025 通过 I2C 接口配置、读取 touchscreen 中的坐标数据
- AC8025 使用中断引脚与 touchscreen 连接，接收 touchscreen 的中断信息通知

38.1.2 功能

- 此模块用于支持设备快速触摸功能，在设备上电后 850ms 内提供触摸点坐标。
- hsm 在 Android 启动后将 TP 控制权交接给 Android 系统。

38.1.3 代码结构及文件描述

表 38-1 FastTouch 代码结构及描述

文件名	文件路径	文件描述
fast_touch.c	source/kernel/common/drivers/misc/	FastTouch kernel logic
goodix.c	source/kernel/common/drivers/input/touchscreen/	Kernel tp driver
fast_touch.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/touchscreen/	FastTouch logic
goodix.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/touchscreen/	Hsm tp driver
touchscreen.h	source/vendor/autochips/proprietary/tinysys/hsm/os/include /	Hsm tp header

38.2 软件设计

方式一：

FastTouch 采用 R5、AP 分时复用 TP 实现，开机后 R5 完成 TP 初始化并独占 TP。

- 在 AP 启动阶段 AP 主动通过 IPC 发送交接请求给 R5，R5 接收到请求主动释放 TP 控制权并通知 AP；
- 交接完成后 AP 可直接使用支持 FastTouch 的 TP，不需再单独进行初始化，以实现快速交接。

方式二：FastTouch 只在 AP 端使用，采用 fastservice + touch driver 的方式实现开机 7 s 内 TP 可用。

38.2.1 架构图

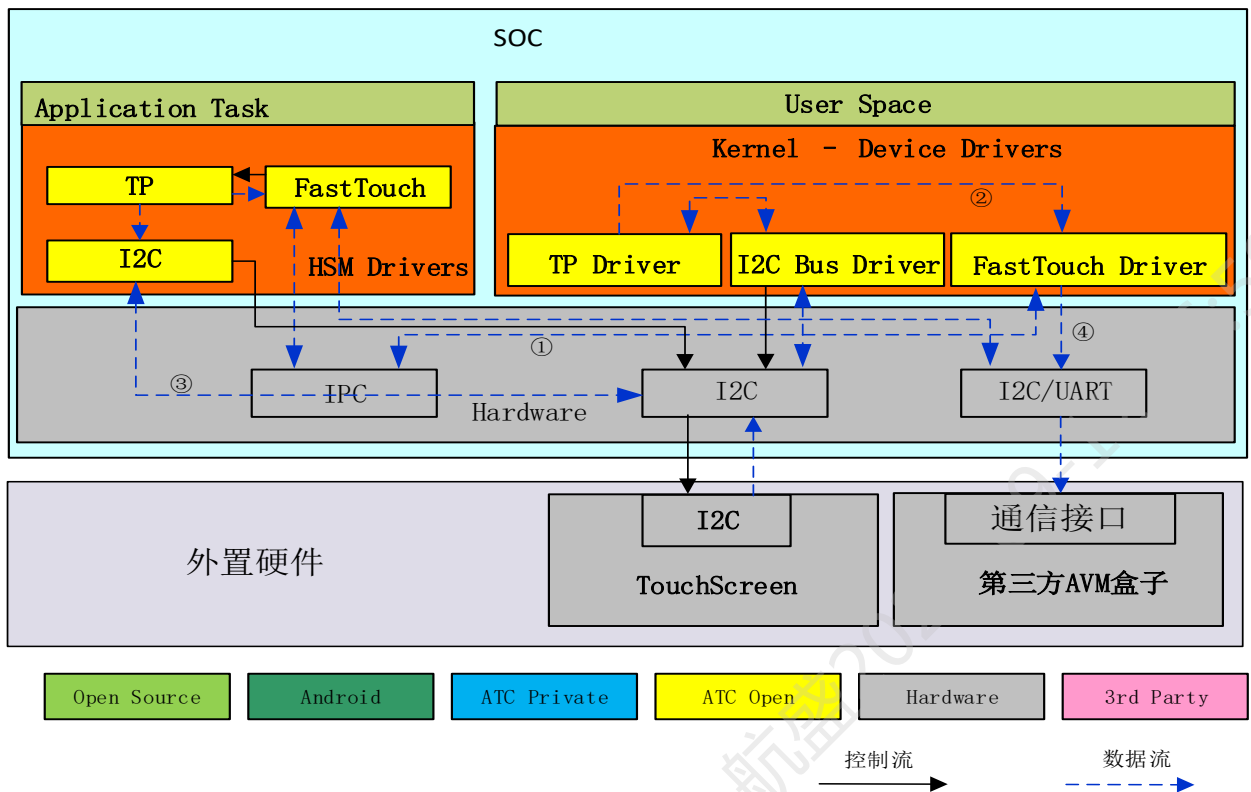
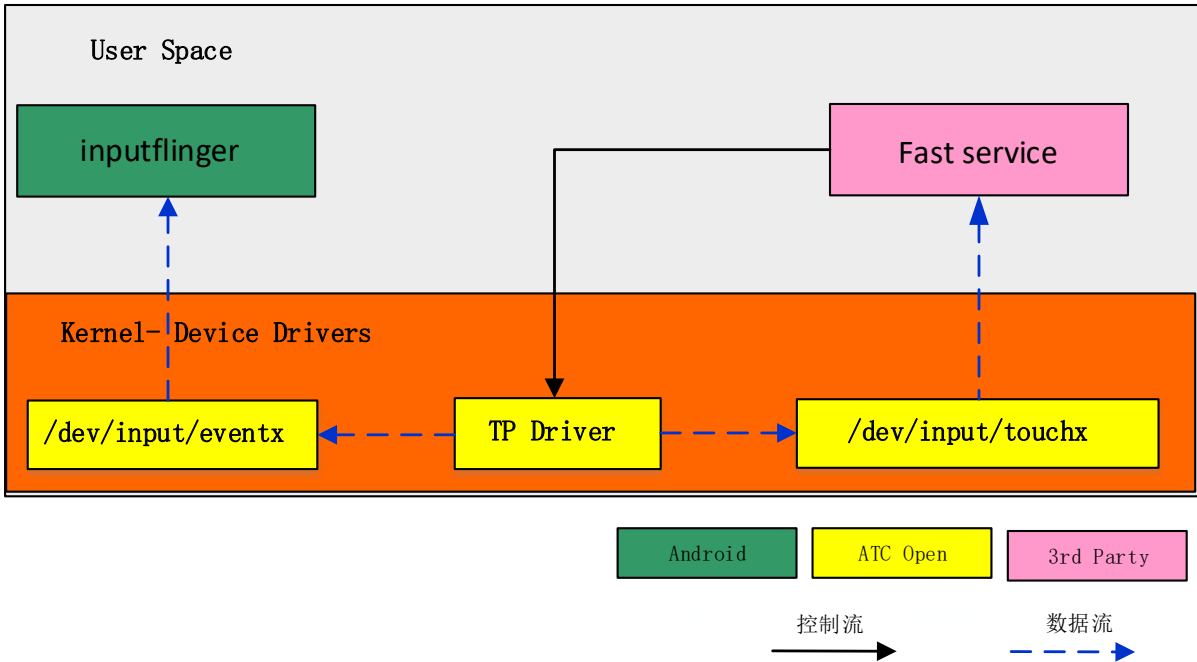


图 38-1 FastTouch 方式一架构图

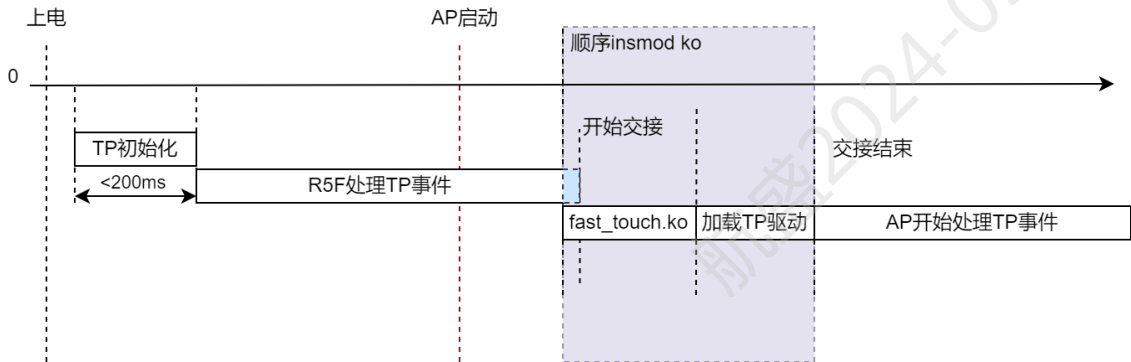
说明：

- ① AP 与 R5 之间的数据流通过 IPC 传输，用于传递诸如 AVM、倒车状态和交接数据；

- ② 在打开控制开关后（AVM start）TP driver 将 TP 数据发送至 FastTouch driver;
- ③ R5 与 TP 通过 I2C 通信，以获取触摸点或向 TP 寄存器写数据;
- ④ FastTouch driver 通过 I2C /UART 或其他通信接口将数据发送给第三方 AVM 盒子。



38.2.2 流程图



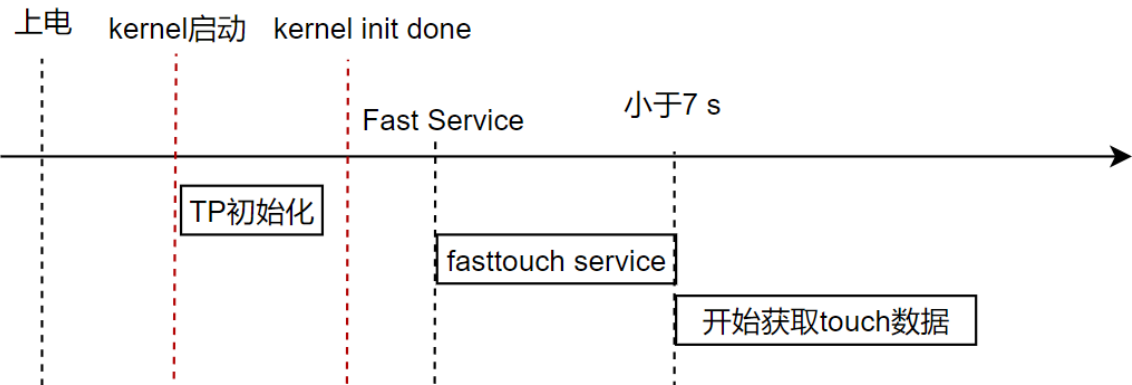


图 38-4 FastTouch 方案二工作流程图

38.3 接口描述

38.3.1 API in Kernel

表 38-2 API in kernel

API	说明
fast_touch_coordinate_report	Report coordinate to fasttouch driver

38.3.1.1 fast_touch_coordinate_report

【描述】

用于 kernel touchscreen driver 将坐标点发送至 fasttouch driver，导出由 touchscreen driver 调用

【语法】

int fast_touch_coordinate_report(int id, int x, int y, int w, int rls)

【参数】

- id: touch point id
- x: x coordinate
- y: y cordinate
- w: w coordinate
- rls: release status

【返回值】

- 0 : report success

- Other : report fail.

38.3.2 API in hsm

表 38-3 API in hsm

API	说明
fastTouchCoordinateReport	上报坐标点
goodixReportRegister	注册 TP driver report 回调函数

38.3.2.1 fastTouchCoordinateReport

【描述】

Fasttouch 上报坐标点到目标设备

【语法】

```
int fastTouchCoordinateReport(struct coordinate_data_t *data)
```

【参数】

data: 需要上报的坐标点数据

【返回值】

- 0: 上报成功.
- -1: 上报失败

38.3.2.2 goodixReportRegister

【描述】

TP driver 注册 report 回调函数，在 TP driver 获取到触摸点时调用注册的回调

【语法】

```
int goodixReportRegister(tp_report report)
```

【参数】

report : int (* tp_report)(struct coordinate_data_t *data) 类型 report 函数

【返回值】

0: 注册成功， -1: 注册失败。

38.4 使用说明与注意事项

1. 对于需要在 hsm 阶段就获取触摸信息的情况，可以使用 R5+AP 的 FastTouch 实现方案；
2. 若只需要在 AP 阶段实现早于 inputflinger 启动的快速触摸则可以使用标准的 kernel tp driver + 自定义的 fastservice 的方式二实现。

航盛2024-09-12 16:50:43

39 GPIO PINMUX

39.1 简介

39.1.1 软硬件环境

一般一个 PIN 脚可支持配置成不同功能。如下图所示，可支持配置成 function1,function2。

- AP 支持最多 77 个 GPIO（AGPIO），R5 与 AP 均可访问；
- Secure 支持最多 12 个 GPIO（SGPIO），只可在 Secure 下被 R5 访问。

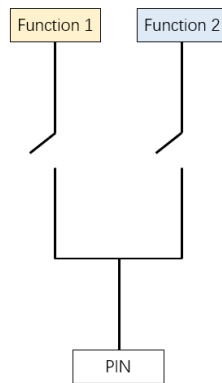


图 39-1 GPIO PINMUX 示意图



注意：

- 一个 PIN 脚同一时间只能被配成其中一种功能
- 每个 GPIO 功能同一时间只能被配置到一个 PIN 脚上
- Secure 下 UART/I2C/SPI 模块与 SGPIO 复用 PIN 脚。

39.1.2 功能

此模块用于支持芯片的 PIN 脚配置。AC8025 最多共支持 89 个 GPIO pin。

39.1.3 代码结构及文件描述

表 39-1 GPIO PINMUX 代码结构及描述

文件名	文件路径	文件描述
pinctrl-common.c	source/kernel/common/drivers/pinctrl/autochips	pinctrl operation function
gpio_v2.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/gpio/	gpio hsm operation function
gpio.h	source/vendor/autochips/proprietary/tinysys/hsm/os/include/	hsm gpio header file
sgpio.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/sgpio/	gpio hsm operation function
sgpio.h	source/vendor/autochips/proprietary/tinysys/hsm/os/include/ac8025/	hsm sgpio header file

39.2 软件设计

39.2.1 架构图

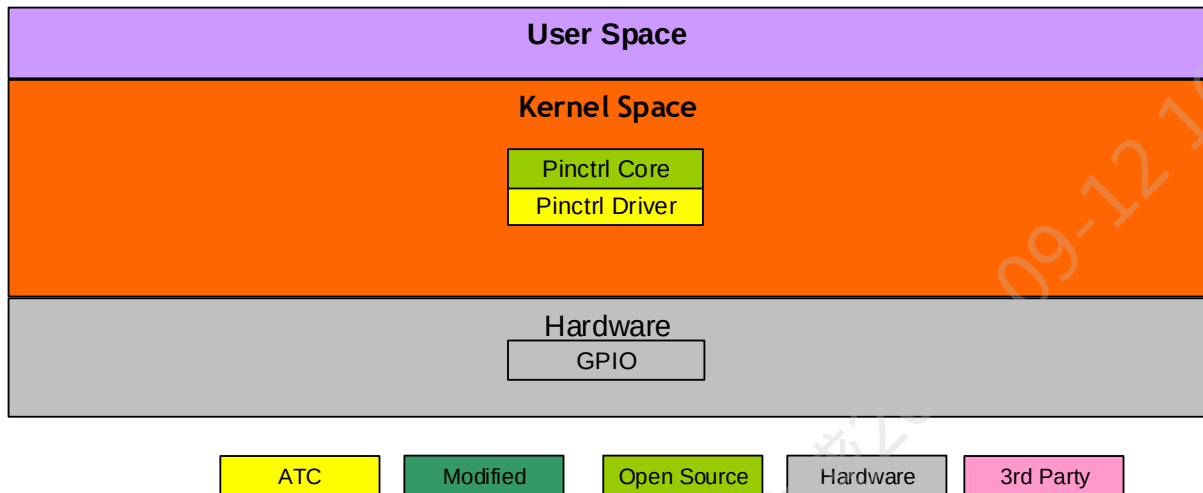


图 39-2 Pinctrl 架构图

39.3 接口描述

39.3.1 API in Kernel

表 39-2 API in kernel

API	说明
of_get_named_gpio	Get a GPIO number to use with GPIO API
gpio_request	Request a GPIO
gpio_direction_output	Set gpio direction to output
gpio_direction_input	Set gpio's direction to input
of_get_named_gpio	Get a GPIO number to use with GPIO API
gpio_request	Request a GPIO
gpio_direction_output	Set gpio direction to output
gpio_direction_input	Set gpio's direction to input
gpio_set_value	Set a gpio's value
gpio_get_value	Return a gpio's value
devm_pinctrl_get	Get pin control state holder

39.3.1.1 of_get_named_gpio

【描述】

Get a GPIO number to use with GPIO API.

【语法】

```
inline int of_get_named_gpio(struct device_node *np, const char *propname, int index)
```

【参数】

- np: device node to get GPIO from
- propname: Name of property containing gpio specifier(s)
- index: index of the GPIO

【返回值】

Returns GPIO number to use with generic GPIO API, or one of the errno value on the error condition.

39.3.1.2 gpio_request

【描述】

Request a GPIO.

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

1062 /1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

【语法】

```
int gpio_request(unsigned gpio, const char *label)
```

【参数】

- gpio: gpio number
- label: string

【注意】

These "optional" allocation calls help prevent drivers from stomping on each other, and help provide better diagnostics in debugfs. They're called even less than the "set direction" calls.

39.3.1.3 gpio_direction_output

【描述】

Set gpio direction to output.

【语法】

```
int gpio_direction_output(unsigned gpio, int value)
```

【参数】

- gpio: gpio number
- value: 1: output high 0:output low

39.3.1.4 gpio_direction_input

【描述】

Set gpio's direction to input.

【语法】

```
int gpio_direction_input(unsigned gpio)
```

【参数】

gpio: gpio number

39.3.1.5 gpio_set_value

【描述】

Set a gpio's value.

【语法】

```
static inline void gpio_set_value(unsigned int gpio, int value)
```

【参数】

- gpio: gpio number
- value: 1:output high 0:output low

39.3.1.6 gpio_get_value

【描述】

Return a gpio's value.

【语法】

```
static inline int gpio_get_value(unsigned int gpio)
```

【参数】

- gpio: gpio number
- return: gpio value

39.3.1.7 devm_pinctrl_get

【描述】

Get pin control state holder.

【语法】

```
struct pinctrl *devm_pinctrl_get(struct device *dev)
```

【参数】

- dev: the device to obtain the handle for
- return: pinctrl struct of dev.

39.3.1.8 pinctrl_lookup_state

【描述】

Retrieves a state handle from a pinctrl handle.

【语法】

```
struct pinctrl_state *pinctrl_lookup_state(struct pinctrl *p, const char *name)
```

【参数】

- p: the pinctrl handle to retrieve the state from

- name: the state name to retrieve

39.3.1.9 pinctrl_select_state

【描述】

Select/activate/program a pinctrl state to HW.

【语法】

```
int pinctrl_select_state(struct pinctrl *p, struct pinctrl_state *state)
```

【参数】

- p: the pinctrl handle for the device that requests configuration
- state: the state handle to select/activate/program

39.3.1.10 of_property_read_u32_array

【描述】

Find and read an array of 32 bit integers from a property.

【语法】

```
int of_property_read_u32_array(const struct device_node *np, const char *propname, u32
*out_values, size_t sz)
```

【参数】

- np: device node from which the property value is to be read.
- propname: name of the property to be searched.
- out_value: pointer to return value, modified only if return value is 0.

39.3.1.11 gpio_set_debounce

【语法】

```
static inline int gpio_set_debounce(unsigned gpio, unsigned debounce)
```

【参数】

- gpio: gpio num
- debounce

39.3.1.12 irq_of_parse_and_map

【描述】

Parse and map an interrupt into virq space.

【语法】

```
unsigned int irq_of_parse_and_map(struct device_node *dev, int index)
```

【参数】

- dev: Device node of the device whose interrupt is to be mapped
- index: Index of the interrupt to map

39.3.1.13 request_irq

【描述】

Request interrupt.

【语法】

```
static inline int __must_check request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags, const char *name, void *dev)
```

【参数】

- irq: interrupt number
- handler: interrupt handler function
- flags: flags (see IRQF_*)
- name: name of the device
- dev_id: cookie to identify the device

39.3.2 API in hsm (AGPIO)

表 39-3 AGPIO API in hsm

API	说明
xGpioSetMode	设置 AGPIO 管脚复用
xGpioSetDirection	设置 AGPIO 方向
xGpioSetPinValue	设置 AGPIO 电平值
xGpioGetPinValue	获取 AGPIO 电平值
xGpioEnableDebounce	使能 AGPIO 消抖
xGpioSetDebounceClk	设置 AGPIO 消抖时钟

xGpioSetDebounceCnt	设置 AGPIO 消抖计数
xGpioEnableSchmit	设置 AGPIO schmit
xGpioEnableE2	使能 AGPIO E2,用于配置 Drive strength
xGpioEnableE1	使能 AGPIO E1,用于配置 Drive strength
xGpioSetPullUpDown	配置 AGPIO 上下拉使能
xGpioIntStatusClr	清除 AGPIO 中断状态位
xGpioIntEnable	使能 AGPIO 中断
xGpioSetIntType	配置 AGPIO 中断触发类型
xGpioEnableSlewRate	使能 AGPIO Slew rate
xGpioGetDriving	获取 AGPIO 驱动能力
xGpioSetDriving	设置 AGPIO 驱动能力

39.3.2.1 xGpioSetMode

【描述】

设置管脚复用

【语法】

```
int xGpioSetMode(long lPin, unsigned char ucMode)
```

【参数】

- lPin : 需要设置的 GPIO
- ucMode: 需要设置的 mode

【返回值】

0: 配置成功.

39.3.2.2 xGpioSetDirection

【描述】

设置 GPIO 方向

【语法】

```
int xGpioSetDirection(long lPin, unsigned char ucValue)
```

【参数】

- lPin : 需要设置的 GPIO

- ucValue: 需要设置的方向, 推荐使用 GPIO_DIR 枚举
 - DIR_IN : 输入
 - DIR_OUT: 输出

【返回值】

0: 配置成功.

39.3.2.3 xGpioSetPinvalue

【描述】

设置电平值

【语法】

```
int xGpioSetPinvalue(long lPin,unsigned char ucValue)
```

【参数】

- lPin:需要设置的 GPIO
- ucValue:0: 低电平 1:高电平, 推荐使用 GPIO_VALUE 枚举
 - GPIO_IO_LOW: 低电平
 - GPIO_IO_HIGH: 高电平

【返回值】

0: 配置成功

39.3.2.4 xGpioGetPinValue

【描述】

获取电平值

【语法】

```
int xGpioGetPinValue(long lPin);
```

【参数】

lPin:需要操作的 GPIO

【返回值】

- -1: 获取失败
- 0 : 低电平

- 1: 高电平

39.3.2.5 xGpioEnableDebounce

【描述】

获取电平值

【语法】

```
int xGpioEnableDebounce(long lPin, unsigned char ucEnable);
```

【参数】

- lPin: 需要设置的 GPIO
- ucEnable: 1: 使能 0: 不使能

【返回值】

- -1: 设置失败
- 0: 设置成功

39.3.2.6 xGpioSetDebounceclk

【描述】

获取消抖时钟

【语法】

```
int xGpioSetDebounceclk(long lPin, char cClkSel);
```

【参数】

- lPin: 需要设置的 GPIO
- cClkSel: 设置的消抖时钟

【返回值】

- -1: 设置失败
- 0: 设置成功

39.3.2.7 xGpioSetDebounceCnt

【描述】

获取消抖计数

【语法】

```
int xGpioSetDebounceCnt(long lPin,char cDebounceCount);
```

【参数】

- lPin:需要设置的 GPIO
- cDebounceCount: 设置的消抖计数值

【返回值】

- -1: 设置失败
- 0 : 设置成功

39.3.2.8 xGpioEnableSchmit

【描述】

设置施密特触发器

【语法】

```
int xGpioEnableSchmit(long lPin,unsigned char ucEnable);
```

【参数】

- lPin:需要设置的 GPIO
- ucEnable: 1: 使能 0: 不使能

【返回值】

0: 设置成功

39.3.2.9 xGpioEnableE2

【描述】

使能 E2,用于配置 Drive strength

【语法】

```
int xGpioEnableE2(long lPin,unsigned char ucEnable);
```

【参数】

- lPin:需要设置的 GPIO
- ucEnable: 1: 使能 0: 不使能

【返回值】

0: 设置成功

39.3.2.10 xGpioEnableE1

【描述】

使能 E1，用于配置 Drive strength

【语法】

```
int xGpioEnableE1(long lPin, unsigned char ucEnable);
```

【参数】

- lPin: 需要设置的 GPIO
- ucEnable: 1: 使能 0: 不使能

【返回值】

0: 设置成功

39.3.2.11 xGpioSetPullUpDown

【描述】

配置上下拉使能

【语法】

```
int xGpioSetPullUpDown(long lPin, unsigned char ucUp);
```

【参数】

- lPin: 需要设置的 GPIO
- ucUp: 1: 上拉 0: 下拉，推荐使用 PAD_PULL_SEL 枚举
 - PULL_DOWN: 下拉
 - PULL_PU: 上拉
 - HIGH_Z: 关闭上下拉（高阻）

【返回值】

0: 设置成功

39.3.2.12 xGpioIntstatusclr

【描述】

清除 GPIO 中断状态

【语法】

```
int xGpioIntStatusClr(long lPin);
```

【参数】

lPin:需要设置的 GPIO

【返回值】

- 0: 清除成功
- 1: 清除失败

39.3.2.13 xGpioIntEnable

【描述】

使能 GPIO 中断

【语法】

```
int xGpioIntEnable(long lPin,unsigned char ucEnable);
```

【参数】

- lPin:需要设置的 GPIO
- ucEnable: 1: 使能 0: 不使能

【返回值】

- 0: 设置成功
- -1: 设置失败

39.3.2.14 xGpioSetIntType

【描述】

设置 GPIO 中断触发类型

【语法】

```
int xGpioSetIntType(long lPin, unsigned char ucIntType);
```

【参数】

- lPin: 需要设置的 GPIO
- ucIntType: 中断类型:
 - GPIO_IRQ_LEVEL_HIGH: 高电平触发
 - GPIO_IRQ_LEVEL_LOW: 低电平触发
 - GPIO_IRQ_EDGE_RISING 上升沿触发
 - GPIO_IRQ_EDGE_FALLING: 下降沿触发
 - GPIO_IRQ_EDGE_ANY: 双边沿触发

【返回值】

- 0: 设置成功
- -1: 设置失败

39.3.2.15 xGpioGetDriving

【描述】

获取 GPIO 驱动能力强度

【语法】

```
int xGpioGetDriving (long lPin);
```

【参数】

- lPin: 需要获取的 GPIO

【返回值】

- 0: 2 mA 1: 4 mA 2: 8 mA 3: 12mA
- -1: 获取失败

39.3.2.16 xGpioSetDriving

【描述】

设置 GPIO 驱动能力强度

【语法】

```
int xGpioSetDriving (long lPin, unsigned char driving);
```

【参数】

- lPin: 需要获取的 GPIO

- driving: 要设置的驱动能力强度 0: 2 mA 1: 4 mA 2: 8 mA 3: 12mA, 推荐使用 PAD_DRIVE 枚举
 - DRIVE_2MA
 - DRIVE_4MA
 - DRIVE_8MA
 - DRIVE_12MA

【返回值】

- 0: 设置成功
- -1: 设置失败

39.3.3 API in hsm (SGPIO)

表 39-4 SGPIO API in hsm

API	说明
xSGpioSetMode	设置 SGPIO 管脚模式
xSGpioGetMode	获取 SGPIO 管脚模式
xSGpioSetAltFunction	设置 SGPIO 复用功能
xSGpioGetAltFunction	获取 SGPIO 设置的复用功能
xSGpioSetPinValue	设置 SGPIO 电平值
xSGpioGetPinValue	获取 SGPIO 电平值
xSGpioEnableDebounce	使能 SGPIO 消抖
xSGpioSetDebounceCnt	设置 SGPIO 消抖计数
xSGpioGetDebounceCnt	获取 SGPIO 消抖计数
xSGpioEnableSlewRate	使能 SGPIO Slew rate
xSGpioEnableSchmit	设置 SGPIO schmit
xSGpioSetPullUpDown	配置 SGPIO 上下拉使能
xSGpioGetPullSel	获取 SGPIO 上下拉状态
xSGpioIntStatusClr	清除中断状态
xSGpioIntEnable	使能 SGPIO 中断
xSGpioSetIntType	设置 SGPIO 中断触发类型
xSGpioSetDriving	设置 SGPO 驱动能力
xSGpioGetDriving	获取 SGPIO 驱动能力
xSpinPadConfigSet	设置 SPIN Pad 属性
xSpinPadConfigGet	获取 SPIN Pad 属性

39.3.3.1 xSGpioSetMode

【描述】

设置 SGPIO 管脚模式

【语法】

```
int xSGpioSetMode(enum sgpio_pinIndex sgpioNum, enum sgpio_mode sgpioMode)
```

【参数】

- sgpioNum: 需要设置的 SGPIO num
- sgpioMode: 需要设置的 mode
 - 0: SGPIO_INPUT_MODE
 - 1: SGPIO_OUTPUT_MODE
 - 2: SGPIO_ALT_MODE

【返回值】

- 0: 设置成功.
- -1: 设置失败

39.3.3.2 xSGpioGetMode

【描述】

获取 SGPIO 管脚模式

【语法】

```
int xSGpioGetMode(enum sgpio_pinIndex sgpioNum)
```

【参数】

- sgpioNum: 需要获取的 SGPIO num

【返回值】

- other: 当前 SGPIO 的模式.
- -1: 设置失败

39.3.3.3 xSGpioSetAltFunction

【描述】

设置 SGPIO 的复用功能

【语法】

```
int xSGpioSetAltFunction(enum sgpio_pinIndex sgpioNum, enum sgpio_altIndex altIndex)
```

【参数】

- sgpioNum: 需要设置的 SGPIO num
- altIndex: 需要设置的 alt function index

【返回值】

- 0: 设置成功.
- -1: 设置失败

39.3.3.4 xSGpioGetAltFunction

【描述】

获取 SGPIO 的复用功能

【语法】

```
int xSGpioGetAltFunction(enum sgpio_pinIndex sgpioNum)
```

【参数】

- sgpioNum: 需要获取的 SGPIO num

【返回值】

- other: 指定 SGPIO 设置的复用功能.
- -1: 设置失败

39.3.3.5 xSGpioSetPinvalue

【描述】

设置 SGPIO 输出电平值

【语法】

```
int xSGpioSetPinValue(enum sgpio_pinIndex sgpioNum, uint8_t ucValue)
```

【参数】

- sgpioNum: 需要设置的 SGPIO
- ucValue: 0: 低电平 1: 高电平, 推荐使用 SGPIO_VALUE 枚举
 - SGPIO_IO_LOW: 低电平

➤ SGPIO_IO_HIGH: 高电平

【返回值】

- 0: 设置成功
- -1: 设置失败

39.3.3.6 xSGpioGetPinValue

【描述】

获取 SGPIO 电平值

【语法】

```
int xSGpioGetPinValue(enum sgpio_pinIndex sgpioNum)
```

【参数】

sgpioNum: 需要操作的 SGPIO

【返回值】

- -1: 获取失败
- 0: 低电平
- 1: 高电平

39.3.3.7 xSGpioEnableDebounce

【描述】

使能 SGPIO 消抖

【语法】

```
int xSGpioEnableDebounce(enum sgpio_pinIndex sgpioNum, uint8_t ucEnable)
```

【参数】

- sgpioNum: 需要设置的 SGPIO
- ucEnable: 1: 使能 0: 不使能

【返回值】

- -1: 设置失败
- 0: 设置成功

39.3.3.8 xSGpioSetDebounceCnt

【描述】

设置 SGPIO 消抖计数

【语法】

```
int xSGpioSetDebounceCnt(enum sgpio_pinIndex sgpioNum, char cDebounceCount)
```

【参数】

- sgpioNum: 需要设置的 GPIO
- cDebounceCount: 设置的消抖计数值

【返回值】

- -1: 设置失败
- 0: 设置成功

39.3.3.9 xGpioGetDebounceCnt

【描述】

获取 SGPIO 消抖计数

【语法】

```
int xSGpioGetDebounceCnt(enum sgpio_pinIndex sgpioNum)
```

【参数】

sgpioNum: 需要获取的 SGPIO

【返回值】

- -1: 获取失败
- other: 消抖计数值

39.3.3.10 xSGpioEnableSlewRate

【描述】

设置 SGPIO Slew Rate

【语法】

```
int xSGpioEnableSlewRate(enum sgpio_pinIndex sgpioNum, uint8_t ucEnable)
```

【参数】

- sgpioNum:需要设置的 SGPIO
- ucEnable: 1: 使能 0: 不使能

【返回值】

- 0: 设置成功
- -1: 设置失败

39.3.3.11 xGpioEnableSchmit

【描述】

设置 SGPIO 施密特触发器

【语法】

```
int xSGpioEnableSchmit(enum sgpio_pinIndex sgpioNum, uint8_t ucEnable);
```

【参数】

- sgpioNum:需要设置的 SGPIO
- ucEnable: 1: 使能 0: 不使能

【返回值】

- 0: 设置成功
- -1:设置失败

39.3.3.12 xSGpioSetPullUpDown

【描述】

配置 SGPIO 上下拉使能

【语法】

```
int xSGpioSetPullUpDown(enum sgpio_pinIndex sgpioNum, uint8_t ucUp)
```

【参数】

- sgpioNum:需要设置的 SGPIO
- ucUp: 1: 上拉 0: 下拉: 关闭上下拉, 推荐使用 SGPIO_PAD_PULL_SEL 枚举

【返回值】

- 0: 设置成功
- -1: 设置失败

39.3.3.13 xSGpioGetPullSel

【描述】

获取 SGPIO 上下拉状态

【语法】

```
int xSGpioGetPullSel(enum sgpio_pinIndex sgpioNum)
```

【参数】

- sgpioNum: 需要设置的 SGPIO

【返回值】

- other: 上下拉状态
- -1: 设置失败

39.3.3.14 xSGpioIntstatusClr

【描述】

清除 SGPIO 中断状态

【语法】

```
int xSGpioIntStatusClr(enum sgpio_pinIndex sgpioNum)
```

【参数】

sgpioNum: 需要设置的 SGPIO

【返回值】

- 0: 清除成功
- 1: 清除失败

39.3.3.15 xSGpioIntEnable

【描述】

使能 SGPIO 中断

【语法】

```
int xSGpioIntEnable(enum sgpio_pinIndex sgpioNum, uint8_t ucEnable)
```

【参数】

- sgpioNum: 需要设置的 SGPIO

- ucEnable: 1: 使能 0: 不使能

【返回值】

- 0: 配置成功
- -1: 配置失败

39.3.3.16 xSGpioSetIntType

【描述】

设置 SGPIO 中断触发类型

【语法】

```
int xSGpioSetIntType(enum sgpio_pinIndex sgpioNum, uint8_t ucIntType)
```

【参数】

- sgpioNum: 需要设置的 SGPIO
- ucIntType: 中断类型:
 - SGPIO_IRQ_LEVEL_HIGH: 高电平触发
 - SGPIO_IRQ_LEVEL_LOW: 低电平触发
 - SGPIO_IRQ_EDGE_RISING 上升沿触发
 - SGPIO_IRQ_EDGE_FALLING: 下降沿触发
 - SGPIO_IRQ_EDGE_ANY: 双边沿触发

【返回值】

- 0: 设置成功
- -1: 设置失败

39.3.3.17 xSGpioSetDriving

【描述】

设置 SGPIO 驱动能力

【语法】

```
int xSGpioSetDriving(enum sgpio_pinIndex sgpioNum, uint8_t driving)
```

【参数】

sgpioNum: 需要设置的 SGPIO

driving: 设置的驱动能力。0: 2 mA 1: 4 mA 2: 8 mA 3: 12mA，推荐使用 SGPIO_PAD_DRIVE 枚举

- SGPO_DRIVE_2MA: 2mA
- SGPO_DRIVE_4MA: 4mA
- SGPO_DRIVE_8MA: 8mA
- SGPO_DRIVE_12MA: 12mA

【返回值】

- 0: 设置成功
- -1: 设置失败

39.3.3.18 xSGpioGetDriving

【描述】

获取 SGPIO 驱动能力

【语法】

```
int xSGpioGetDriving(enum sgpio_pinIndex sgpioNum)
```

【参数】

sgpioNum: 需要获取的 SGPIO

【返回值】

- other: 当前 SGPIO 设置的驱动能力
- -1: 获取失败

39.3.3.19 xSpinPadSet

【描述】

设置 SPIN Pad 属性 (drive strength、SlewRate、schmitt-trigger、pull sel)

【语法】

```
int xSPinPadSet(uint8_t pin, enum SPIN_PAD_CFG cfg, uint8_t value)
```

【参数】

- pin: 需要设置的 SPIN
- cfg: 需要配置的 pad 属性, 0: slew rate 1: schmitt-triger 2: drive strength 3: pull sel, 推荐使用 SPIN_PAD_CFG 枚举。

- SPIN_CFG_SR: slew rate
- SPIN_CFG_SMT: Schmitt-trigger
- SPIN_CFG_DRIVE: 驱动能力
- SPIN_CFG_PULLSEL: 上下拉
- value: 需要设置的属性的值

【返回值】

- 0: 设置成功
- -1: 设置失败

39.3.3.20 xSpinPadGet

【描述】

获取 SPIN Pad 属性（驱动能力、SlewRate、schmitt-trigger、上下拉）

【语法】

```
int xSpinPadGet(uint8_t pin, enum SPIN_PAD_CFG cfg)
```

【参数】

- pin: 需要设置的 SPIN
- cfg: 需要配置的 pad 属性，0: slew rate 1: schmitt-triger 2: drive strength 3: pull se, 推荐使用 SPIN_PAD_CFG 枚举。
 - SPIN_CFG_SR: slew rate
 - SPIN_CFG_SMT: Schmitt-trigger
 - SPIN_CFG_DRIVE: 驱动能力
 - SPIN_CFG_PULLSEL: 上下拉

【返回值】

- other: 当前 pin 指定属性的值
- -1: 设置失败

39.4 使用说明与注意事项

39.4.1 Kernel 中使用 GPIO

AC8025 GPIO 在 Kernel 中的实现是基于 pinctrl 及 gpiolib 架构。因此，内核模块在 Kernel 中使用 GPIO 功能，需要通过 pinctrl 及 gpiolib 的 API 正确调用，以下介绍 pinctrl 及 gpiolib 的用法。

39.4.1.1 通过 pinctrl 设置管脚功能

【步骤 1】在 DTS 中声明该 GPIO

为了方便举例，假设某个模块名为 peripheral，其在 ac8025.dtsi 中有如下结点信息：

```
peripheral:peri@0x0 {  
    compatible = "Autochips,peripheral-test";  
};
```

该模块需要使用 pin86 引脚的 GPIO 功能，需要在 ac8025_demo.dts 中声明 GPIO 如下：

```
&pio0 {  
    test_pin86:testpin86 { D  
        pin86_t {  
            pinmux = <PINMUX_GPIO86_FUNC_GPIO0_31>; B  
            output-low; C  
        };  
    };  
};  
  
&peripheral {  
    pinctrl-names = "gpio86"; A  
    pinctrl-0 = <&test_pin86>;  
}
```

注意：

图片中 Peripheral 节点中 pinctrl-0 中依赖的名称与 pio0 节点指定的名称需要一致（如红箭头所标）

pinmux 中使用的宏，在内核目录下 include/dt-bindings/pinctrl/ac8025-pinfunc.h 中定义。

所有管脚允许配置的功能都用宏在 ac8025-pinfunc.h 中定义。如 pin86 引脚则有以下两个功能：

```
#define PINMUX_PIN86_FUNC_I2C5_SCL (ATC_PIN_NO(86) | 0)  
#define PINMUX_PIN86_FUNC_GPIO_75 (ATC_PIN_NO(86) | 1)
```

当前例子中需要配置成 GPIO 功能，则选用宏 PINMUX_PIN86_FUNC_GPIO0_75。

output-low 属性表示默认输出低电平。

- “testpin86”及“pin86_t”为自定义名称。

如果存在多个 Pin 需要设置功能，可以写成以下格式：

```
&pio0 {
    test_gpio86: testpin86 {
        pin86_t {
            pinmux = <PINMUX_GPIO86_FUNC_GPIO86>;
            output-low;
        };
    };
    test_gpio58: testpin58 {
        pin58_t {
            pinmux = <PINMUX_GPIO58_FUNC_GPIO0_28>;
            output-low;
        };
    };
};

&peripheral {
    pinctrl-names = "gpio86", "gpio58";
    pinctrl-0 = <&test_gpio86>;
    pinctrl-1 = <&test_gpio58>;
};
```

【步骤 2】在代码中配置 GPIO 管脚复用

代码中通过以下步骤配置 GPIO 管脚复用：

1. 调用 devm_pinctrl_get 函数获取 pinctrl 节点；
2. 调用 pinctrl_lookup_state 函数获取 pinctrl 节点中定义的 pin 状态 pinctrl_state；
3. 调用 pinctrl_select_state 函数设置 pin 的状态 pinctrl_state。

示例代码如下：

```
static int atc_peri_probe(struct platform_device *pdev)
{
    struct pinctrl *pinctrl;
    struct pinctrl_state *pins_default;
    int ret = 0;

    pinctrl = devm_pinctrl_get(&pdev->dev);
    if (IS_ERR(pinctrl)) {
        ret = PTR_ERR(pinctrl);
        dev_err(&pdev->dev, "cannot find pinctrl !\n");
        return ret;
    }
    pins_default = pinctrl_lookup_state(pinctrl, "gpio86");
    if (IS_ERR(pins_default)) {
        ret = PTR_ERR(pinctrl);
        dev_err(&pdev->dev, "cannot find pinctrl gpio86!\n");
        return ret;
    }

    pinctrl_select_state(pinctrl, pins_default);
}
```



注意：

pinctrl_lookup_state 函数第二个传参“gpio86”对应步骤 1 中 pinctrl-names。

39.4.2 通过 GPIOLIB 设置管脚方向及输入输出

【步骤 1】在 DTS 中声明该 GPIO

同样以 peripheral 模块来举例，在 ac8025_demo.dts 中声明 GPIO 如下：

```
&peripheral {
    gpio_pins = <&pio0 86 0>;
};
```



注意：

三元组中参数 1 表示依赖的模块，参数 2 表示声明的 GPIO 号，参数 3 表示 flag，默认配置为 0。

【步骤 2】在代码中配置 GPIO 状态

必需的头文件：

```
#include <linux/of_gpio.h>
```

```
#include <linux/gpio.h>
```

代码中通过以下步骤配置 GPIO 管脚复用：

1. 调用 of_get_named_gpio 从 DTS 节点中获取映射后的 gpio 号；
2. 调用 gpio_request 向 kernel 申请 gpio 使用；
3. 调用 gpio_direction_output / gpio_direction_input 设置输出/输入方向；
4. 调用 gpio_set_value / gpio_get_value 设置/获取 gpio 值。

示例代码如下：

```
gpio_num = of_get_named_gpio(pdev->dev.of_node, "gpio_pins", 0);
gpio_request(gpio_num, "GPIO86_TEST");

printk("Test gpio lib output set low!\n!\n");
gpio_direction_output(gpio_num, 0);

printk("Test gpio lib output set hight!\n!\n");
gpio_direction_output(gpio_num, 1);

printk("Gpio lib set as input and get value --!\n");
gpio_direction_input(gpio_num);
```



注意:

gpio_request 函数第二个传参 label 字符串为模块自定义字符串，不需要则设置为 NULL。

39.4.3 Kernel 注册 GPIO 外部中断

【步骤 1】在 DTS 中声明该 GPIO

同样以 peripheral 模块来举例，在 ac8025_demo.dts 中声明 GPIO 如下：

```
&peripheral {
    interrupt-parent = <&pio0>;
    interrupts = <276>;
};
```

【步骤 2】在代码中注册 GPIO 中断：

必需的头文件：

```
#include <linux/interrupt.h>
```

示例代码如下：

```
irq_num = irq_of_parse_and_map(pdev->dev.of_node, 0);
printk("Get irq_num = %d\n", irq_num);

ret = request_irq(irq_num, tpd_interrupt_handler, IRQF_TRIGGER_RISING,
    "pwmgpio", NULL);
if(ret) {
    printk("request irq fail ret = %d!\n", ret);
}
```

39.4.4 hsm 使用 AGPIO/SGPIO 外部中断

hsm 支持 AGPIO 与 SGPIO 的外部中断，相关接口如下：

AGPIO 中断配置接口：

```
int xGpioIntStatusClr(long lPin);
int xGpioIntEnable(long lPin, unsigned char ucEnable);
int xGpioSetIntType(long lPin, unsigned char ucIntType);
```

SGPIO 中断配置接口：

```
int xSGpioIntStatusClr(enum sgpio_pinIndex sgpioNum);
int xSGpioIntEnable(enum sgpio_pinIndex sgpioNum, uint8_t ucEnable);
int xSGpioSetIntType(enum sgpio_pinIndex sgpioNum, uint8_t ucIntType);
```

系统接口：

```
#define GicIRQConfig gic_irq_config
#define EnableIRQ    enable_irq
#define DisableIRQ   disable_irq
#define FreeIRQ      free_irq
```

中断号：

AGPIO 在 hsm 和 kernel 中断号不相同，当前仅 GPIO0_0-GPIO0_31 共 32 个 GPIO 可在 hsm 触发中断（中断号 176-207），AGPIO 中断号推荐使用 libsrc/include/ac8025/sec_irqno.h 中的 IRQ_ID_AGPIO0 宏(176)，GPIO0_1—GPIO0_31 通过宏偏移计算得到。

例如：GPIO0_2 中断号为 IRQ_ID_AGPIO0 + 2

SGPIO 中断号可使用 sec_irqno.h 中的 IRQ_ID_SGPIO0 (57) 作为起始通过偏移计算得到，中断号范围为 57-72，计算偏移时需要使用 sgpio_pinIndex 中的枚举值作为偏移量。

例如：SGPIO1 中断号为 IRQ_ID_SGPIO0 + SFT_SPI_0_CSN

Hsm 使用 AGPIO 外部中断示例（以 GPIO0_21 为例）：

```
#include "sec_irqno.h"
#include "interrupt_driver.h"
#include "gpio.h"

#define AGPIO_IO_A      21
#define AGPIO_TO_IRQ(X) (IRQ_ID_AGPIO0 + X)
#define IRQ_TO_AGPIO(X) (X - IRQ_ID_AGPIO0)

/* irq handler */
void __agpio_irq_handler(uint32_t irq, void *data)
{
    (void)irq;
    uint32_t gpio = (uint32_t)data;
    /* clear gpio interrupt status */
    xGpioIntStatusClr(gpio);
}

void enable_agpio_irq(uint32_t gpio, uint8_t IntType)
{
    uint32_t irq = AGPIO_TO_IRQ(gpio);

    /* set gpio mode */
    xGpioSetMode(gpio, 0);
    /* set dir: input */
    xGpioSetDirection(gpio, DIR_IN);
    /* set irq handler function */
    GicIRQConfig(irq, GICD_INT_CFG_GROUP1, GIC_CPU_MASK,
    | | | GICD_INT_CFG_LEVEL, (void *)__agpio_irq_handler, (void *)gpio);
    /* set EINT trig type */
    xGpioSetIntType(gpio, IntType);
    xGpioIntStatusClr(gpio);
    /* enable gpio interrupt */
    xGpioIntEnable(gpio, 1);
    /* enable Gic irq */
    EnableIRQ(irq);
}
```

Hsm 使用 SGPIO 外部中断示例（以 SGPIO1 为例）：

```
#include "sec_irqno.h"
#include "interrupt_driver.h"
#include "sgpio.h"

#define SGPIO_IO_A      SFT_SPI_0_CSN
#define SGPIO_TO_IRQ(X) (IRQ_ID_SGPIO0 + X)
#define IRQ_TO_SGPIO(X) (X - IRQ_ID_SGPIO0)

/* irq handler */
void __sgpio_irq_handler(uint32_t irq, void *data)
{
    (void)irq;
    uint32_t gpio = (uint32_t)data;
    /* clear sgpio interrupt status */
    xSGpioIntStatusClr(gpio);
}

void enable_sgpio_irq(uint32_t sgpio, uint8_t IntType)
{
    uint32_t irq = SGPIO_TO_IRQ(sgpio);
    /* set sgpio mode: input mode */
    xSGpioSetMode(sgpio, SGPIO_INPUT_MODE);
    /* set irq handler function */
    GicIRQConfig(irq, GICD_INT_CFG_GROUP1, GIC_CPU_MASK,
    | | | | GICD_INT_CFG_LEVEL, (void *)__sgpio_irq_handler, (void *)sgpio);
    /* set EINT trig type */
    xSGpioSetIntType(sgpio, IntType);
    xSGpioIntStatusClr(gpio);
    /* enable sgpio interrupt */
    xSGpioIntEnable(sgpio, 1);
    /* enable Gic irq */
    EnableIRQ(irq);
}
```

39.4.5 在 AP 端查看 SPIN 状态

【步骤 1】更新 SPIN 状态命令：echo sgpio > /proc/hsm_trace

【步骤 2】打印 SPIN 状态

命令：cat /proc/hsm_trace

输出结果：

```
ac8025_demo:/ # cat /proc/hsm_trace
*****show_sgpio*****
PinNum  Mode  AltFunc IO    Drive  SR    SMT    PullSel
00      -1    -1    -1    0      1     0     0
01      -1    -1    -1    0      1     0     1
02      -1    -1    -1    0      1     0     1
03      -1    -1    -1    0      0     0     2
04      -1    -1    -1    0      1     0     1
05      -1    -1    -1    0      1     1     0
06      -1    -1    -1    0      1     1     0
07      -1    -1    -1    0      1     1     0
08      -1    -1    -1    0      1     1     0
09      -1    -1    -1    0      1     1     0
10      -1    -1    -1    0      1     1     0
12      2     6    -1    0      0     0     2
13      2     7    -1    0      0     0     2
14      2     5    -1    0      0     0     1
15      2     4    -1    0      0     0     2
20      2     8    -1    0      0     0     2
21      2     9    -1    0      0     0     2
22      2    10    -1    0      0     0     2
23      2    11    -1    0      0     0     2
24      -1    -1    -1    3      0     0     0
25      -1    -1    -1    3      0     0     1
26      -1    -1    -1    3      0     0     0
27      -1    -1    -1    3      0     0     0
28      -1    -1    -1    3      0     0     0
29      -1    -1    -1    3      0     0     0
30      1     -1     0     0      0     0     2
31      0     -1     0     0      0     0     2
32      1     -1     0     0      0     0     2
33      0     -1     1     0      0     0     2
*****
```

- PinNum：对应《AC8025_PinAssignmetn.xml》中的 SW PIN No. 列
- Mode：SPIN 当前的模式（-1：专用 pin 0：GPIO 输入 1：GPIO 输出 2：复用模式）
- AltFunc：当前 pin 设置的复用功能，-1 表示非复用模式，其余值对应《AC8025_PinAssignmetn.xml》中的 SFT_AltFunc_List.
- IO：input/output 值（0：低电平 1：高电平）
- Drive：驱动能力（0~3 分别对应 2/4/8/12 mA）
- SR：Slew Rate 使能（0：disable 1：enable）
- SMT：施密特触发器使能（0：disable 1：enable）
- PullSel：上下拉状态（0：下拉 1：上拉 2：禁用上下拉）

【注意】cat /proc/hsm_trace 仅打印上一次更新的结果，不直接从寄存器中读取，若需要 debug 查看最新状态，需要先执行更新命令，再查看结果。

39.5 典型问题

39.5.1 如何通过节点方式进行 GPIO 相关设置/测试/查看

【问题描述】

如何通过节点方式进行 GPIO 相关设置/测试/查看？

【解决方案】

1. 查看或配置 GPIO 状态的调试节点位置：

```
/sys/devices/platform/soc/11090000.pinctrl/gpio_debug
```

2. 查看 GPIO 引脚状态

命令： `cat gpio_debug`

- MODE: 引脚模式
- DIR: 引脚方向（0: 输入 / 1: 输出）
- DIN: 引脚电平值（只读）
- DRV: 引脚驱动能力（0~3）
- SMT: 施密特触发器使能
- PULLEN: 上拉使能
- PULLSEL: 上拉选择（0: 下拉 / 1: 上拉）

说明：显示为-1 则表示该 pin 不支持该属性。

3. 配置 GPIO 引脚状态

格式： `echo 属性 引脚编号 值 > gpio_debug`

【Sample Command】

```
echo mode 86 1 > gpio_debug // 配置模式为 1
echo dir 86 0 > gpio_debug // 配置方向为输入
echo dir 86 1 > gpio_debug // 配置方向为输出
echo out 86 1 > gpio_debug // 配置输出为高电平
echo out 86 0 > gpio_debug // 配置输出为低电平
echo pullen 86 1 > gpio_debug //配置上拉使能
echo pullsel 86 1 > gpio_debug //配置上拉
echo pullsel 86 0 > gpio_debug //配置下拉
echo drv 86 3 > gpio_debug //配置驱动能力为 3
```


40 I2C

40.1 简介

40.1.1 软硬件环境

- AC8025 Secure Sub-system 提供了 2 组 I2C 接口，只能在 Secure 下通过 R5 访问
- AC8025 LPIO Sub-System 提供了 10 组 I2C 接口，R5 和 AP 均可访问



注意：

Secure 下 I2C 模块与 SGPIO 复用 PIN 脚，因此实际支持情况由 PIN 功能决定。

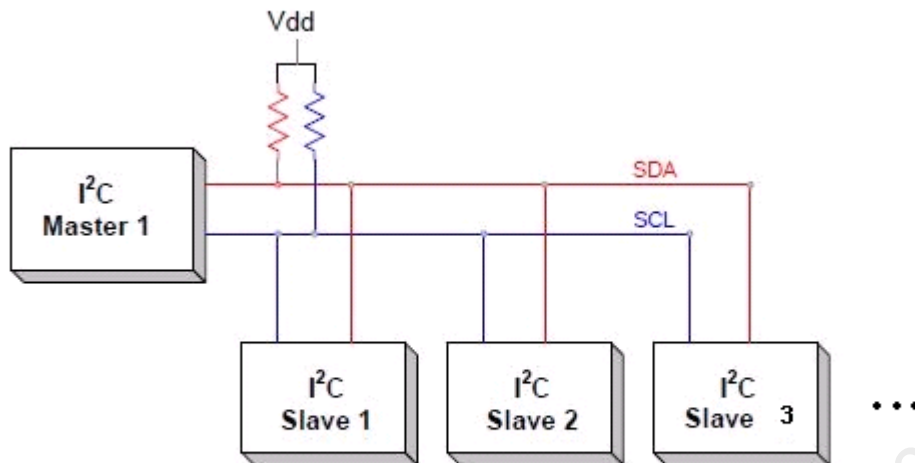


图 40-1 GPIO PINMUX 示意图

40.1.2 功能

此模块用于支持通过 I2C 总线传输数据，AC8025 共支持 12 组 I2C，支持 100KHz、400KHz 和最高 1MHz 的速率。

40.1.3 代码结构及文件描述

表 40-1 I2C 代码结构及描述

文件名	文件路径	文件描述
i2c-atc.c	source/kernel/common/drivers/soc/autochips/i2c/	I2C kernel logic code
i2c_v2.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/i2c/	I2C hsm logic code
i2c.h	source/vendor/autochips/proprietary/tinysys/hsm/os/include/	I2C hsm header file

40.2 软件设计

40.2.1 架构图

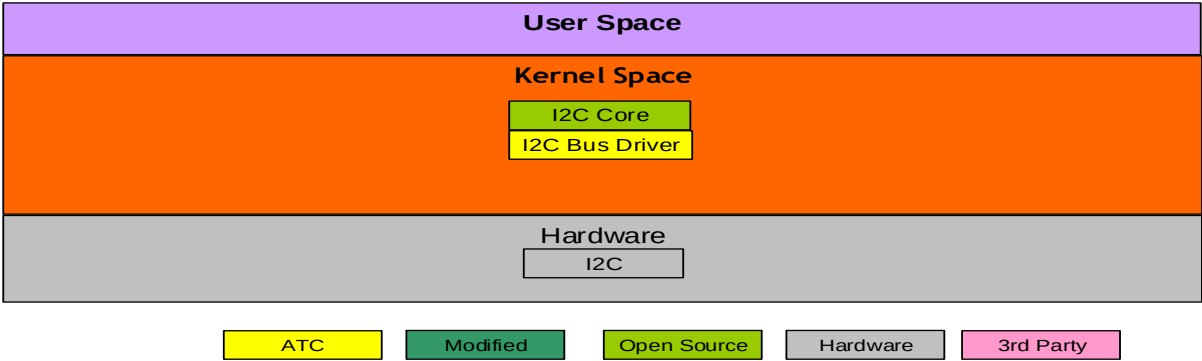


图 40-2 I2C 架构图

40.2.2 流程图

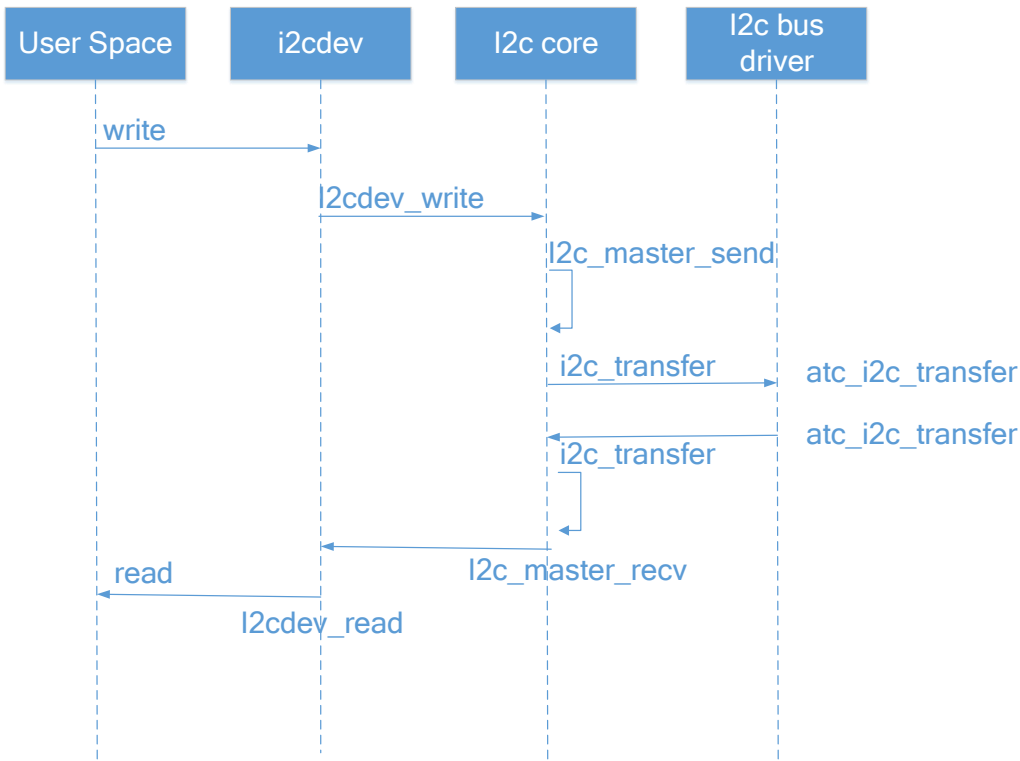


图 40-3 I2C 读写流程图

40.3 接口描述

40.3.1 API in Kernel

表 40-2 I2C kernel API

API	说明
atc_i2c_master_xfer	i2c master send/receive data

40.3.1.1 atc_i2c_master_xfer

【描述】

i2c master send data.

【语法】

int atc_i2c_master_xfer(struct i2c_adapter *adap, struct mt_i2c_msg msg[], s32 num)

【参数】

配置 I2C 需要填充 i2c_config 结构体，以下为该结构体成员

- struct i2c_adapter: i2c adapter
- struct mt_i2c_msg: i2c message
- s32 num: the number of i2c message

【返回值】

The value returned from sending data if it is successfully.

40.3.2 API in hsm

表 40-3 I2C hsm API

API	说明
xI2CInit	配置 i2c 为默认设置
xI2CSetSpeed	配置 i2c 访问速率
xI2CTransfer	访问 i2c
xI2CWrite	通过 i2c 写数据
xI2CRead	通过 i2c 读数据

40.3.2.1 xI2CInit

【描述】

配置 i2c 为默认设置

【语法】

BaseType_t xI2CInit(void)

【参数】

无

【返回值】

0: 配置成功.

40.3.2.2 xI2CSetSpeed

【描述】

配置 i2c 访问速率

【语法】

BaseType_t xI2CSetSpeed (char cBus, long lSpeed)

【参数】

- cBus : 要操作的 i2c 总线号
- lSpeed : 要设置的目标访问速率

【返回值】

- 0: 设置成功.
- Other : 设置失败

40.3.2.3 xI2CTransfer

【描述】

访问 i2c。用户需要配置好 xMessage 中的 lAddr(设备地址), lFlags(传输标志), lLength(传输长度), pcBuf (数据 buffer)。

【语法】

BaseType_t xI2CTransfer (char cBus, I2cMessageHandle_t xMessage, long lNum)

【参数】

- cBus : 要操作的 i2c 总线号
- xMessage:需要传输的 msg
- lNum: msg 个数

【返回值】

- lNum: 发送成功.
- Other : 发送失败

40.3.2.4 xI2CWrite

【描述】

通过 i2c 写数据

【语法】

BaseType_t xI2CWrite (char cBus, long lAddr, char pcBuf, long lLength)

【参数】

- cBus : i2c 总线号
- lAddr : 设备地址

- pcBuf : 待写入数据
- lLength: 写入数据长度

【返回值】

- lLength: 写成功
- Other : 写失败

40.3.2.5 xI2CRead

【描述】

通过 i2c 读数据

【语法】

BaseType_t xI2CRead (char cBus, long lAddr, char pcBuf, long lLength)

【参数】

- cBus : i2c 总线号
- lAddr : 设备地址
- pcBuf : 读取数据
- lLength: 读取数据长度

【返回值】

- lLength: 读成功
- Other : 读失败

40.3.3 数据结构

表 40-4 I2C 数据结构

数据结构	说明
struct I2cMessage	I2c msg

40.3.3.1 I2cMessage

```
struct I2cMessage{
    long lAddr;
    long lLength;
```

```
long lFlags;

char *pcBuf

}
```

40.4 使用说明和注意事项

下文介绍如何在内核中添加一个 i2c 设备，主要以 Atmel 触摸屏作为实例说明。

40.4.1 I2C 速率设置

修改 i2c 控制器的速率，需要修改 DTS 中 i2c 结点中 clock-frequency 属性：

Sample： 下图设置 i2c2 控制器默认频率为 400K。

```
&i2c0 {
    clock-frequency = <400000>;
    #address-cells = <1>;
    #size-cells = <0>;
    multiplexing = <0x1>;
    status = "okay";
}
```

40.4.2 在 DTS 中添加设备节点

假设 Atmel 触摸屏设备是挂接在 i2c0 总线上。则应该在 dts (ac8025_demo.dts) 里增加节点如下：

```
&i2c0 { //表示挂载在 i2c0 总线上

    touch: touch@4b {           //设备名，由用户自定义，@后面一般使用设备地址
        compatible = "atmel,maxtouch"; //匹配字段，需要与驱动保持一致
        reg = <0x4b>;           //从设备地址，必需
    };
};
```

40.4.3 设备驱动的编写

【步骤 1】使用 module_i2c_driver 注册 i2c 设备驱动程序

```
static const struct of_device_id mxt_of_match[] = {
    { .compatible = "atmel,maxtouch", }, //与 DTS 里的 compatible 对应
};
```

```

    {}},
};
MODULE_DEVICE_TABLE(of, mxt_of_match);
static struct i2c_driver mxt_driver = {
    .driver = {
        .name = "atmel_mxt_ts",          //驱动名称，用户自定义
        .of_match_table = mxt_of_match,
    },
    .probe      = mxt_probe,
    .remove     = mxt_remove,
};
module_i2c_driver(mxt_driver);

```

【步骤 2】在设备驱动中访问 i2c

当设备与驱动匹配后，会调用驱动注册的 probe 函数，probe 函数的函数定义为：

```
int (*probe)(struct i2c_client *, const struct i2c_device_id *);
```

probe 函数中的 i2c_client 是 i2c 设备驱动进行 i2c 读写操作的必要条件。获得 i2c_client 后，可通过调用如下 API 进行 i2c 访问：

```
int i2c_transfer(struct i2c_adapter *adap, struct i2c_msg *msgs, int num)
```

示例代码：

I2c 读操作：

```

static int mxt_bootloader_read(struct mxt_data *data,
                               u8 *val, unsigned int count)
{
    int ret;
    struct i2c_msg msg;

    /* 设置设备地址 */
    msg.addr = data->bootloader_addr;    //
    /* 如果设备地址是 10bit 地址，则设置 I2C_M_TEN，否则默认 7bit 地址*/
    msg.flags = data->client->flags & I2C_M_TEN;
    /* 设置读标志位 */

```



```

msg.flags |= I2C_M_RD;

/* 设置读取长度*/

msg.len = count;

/* 读取 buffer */

msg.buf = val;

/* 调用 i2c_transfer 函数，正常则返回 msg 个数 */

ret = i2c_transfer(data->client->adapter, &msg, 1);

if (ret == 1) {

    ret = 0;

} else {

    ret = ret < 0 ? ret : -EIO;

    dev_err(&data->client->dev, "%s: i2c recv failed (%d)\n",

        __func__, ret);

}

return ret;

}

```

I2c 写操作:

```

static int mxt_bootloader_write(struct mxt_data *data,

                                const u8 * const val, unsigned int count)

{

    int ret;

    struct i2c_msg msg;

    /* 设置设备地址 */

    msg.addr = data->bootloader_addr;

    /* 如果设备地址是 10bit 地址，则设置 I2C_M_TEN，否则默认 7bit 地址*/

    msg.flags = data->client->flags & I2C_M_TEN;

    /* 设置读取长度*/

    msg.len = count;

    /* 写入 buffer */

```

```
msg.buf = (u8 *)val;

/* 调用 i2c_transfer 函数，正常则返回 msg 个数 */
ret = i2c_transfer(data->client->adapter, &msg, 1);
if (ret == 1) {
    ret = 0;
} else {
    ret = ret < 0 ? ret : -EIO;
    dev_err(&data->client->dev, "%s: i2c send failed (%d)\n",
            __func__, ret);
}
return ret;
}
```

40.4.4 Kernel 打开 I2C 死锁检测

当前 I2C0-I2C5 支持死锁检测，打开 kernel 中的 i2c 死锁检测功能，需要在 dts 中对应的 i2c 节点下增加如下配置（以 i2c0 为例）：

```
&i2c0 {
    pinctrl-names = "i2c", "gpio";
    pinctrl-0 = <&i2c0_def>;
    pinctrl-1 = <&i2c0_gpio>;

    scl-gpios = <&pio0 76 0>;
    sda-gpios = <&pio0 77 0>;
}
```

I2cx_def 与 i2cx_gpio (x:0-5) 已定义在 ac8025_demo.dts 中，可直接引用。

```
/* default mode: i2c */
i2c0_def: i2c0-def {
    /* Note: scl_pin must define before sda_pin */
    scl_pin {
        pinmux = <PINMUX_PIN76_FUNC_I2C0_SCL>;
        bias-disable;
    };
    sda_pin {
        pinmux = <PINMUX_PIN77_FUNC_I2C0_SDA>;
        bias-disable;
    };
};

/* special mode: gpio */
i2c0_gpio: i2c0-gpio {
    /* Note: scl_pin must define before sda_pin */
    scl_pin {
        pinmux = <PINMUX_PIN76_FUNC_GPIO_65>;
        input-enable;
    };
    sda_pin {
        pinmux = <PINMUX_PIN77_FUNC_GPIO_66>;
        input-enable;
    };
};
```

scl-gpios 与 sda-gpios 属性配置的第二个参数 pin number 可参考对应的 i2cx_gpio 中的 pinmux 宏。
例如 PINMUX_PIN76_FUNC_GPIO_65，对应的 pin number 为 76。

40.5 典型问题

40.5.1 目前平台调通的触摸屏 IC

【问题描述】

目前平台调通的触摸屏 IC 有哪些？

【解决方案】

目前平台调通的触摸屏 IC：汇顶 GT9xx，奕力 2511。

40.5.2 I2C 支持的最大速率

【问题描述】

I2C 支持的最大速率有哪些？

【解决方案】

I2C 支持最大速度为 1M，但需要根据速率适当调整外部上拉电阻的阻值。

40.5.3 I2C 常见异常及确认方法

40.5.3.1 设备无应答

【异常状态】从机没有响应或没有发送 ACK 信号，通常表示指定地址的设备不存在或设备存在但状态异常（未正常工作）

【错误码】-6

【确认方法】

1. Kernel 异常 log

```
[ 4.375619][ T298][W] [I2C]atc_i2c_master_xfer,1132 WFail i2c6,A:0x3d,R:0xf5
[ 4.376510][ T298][W] [videoin][sensor_i2c] sensor_i2c_read(line:175): sensor_i2c_read failed, i2c_transfer ret=-6
```

2. Hsm 异常 log

```
[0][W][I2C] i2c_write,744 I2C5 addr = 0x28 write 2:2B, fifo size: 2, fail with NACK
[0][W][I2C] xl2CTransfer,1029 i2c5_write addr(0x28) fail!
[0][W][videoin] [sensor_i2c] sensor_i2c_read: sensor_i2c_read failed, i2c_transfer ret=-6
```

【产生原因】

1. 设备地址错误
2. 设备故障、设备为正常工作（未上电、未释放复位等）

40.5.3.2 传输超时

【异常状态】kernel 单笔传输时长超过 1s，hsm 单笔传输时长超过 100ms 未正常结束。

【错误码】-110

【确认方法】

1. Kernel 异常 log

```
[ 13.279711] [I2C]0x00000000:0x0000c91e 0x00000040 0x0000003d 0x00000000
[ 13.280696] [I2C]0x00000010:0x00000000 0x00000001 0x00000000 0x000000fc
[ 13.281659] [I2C]0x00000020:0x00000018 0x00000000 0x00000000 0x00000000
[ 13.282612] [I2C]i2c-6 do reset assert!
[ 13.283218] autochips_reset_program_hw: reg = 0x408, mask = 64
[ 13.284088] autochips_reset_program_hw: reg = 0x408, mask = 64
[ 13.285039] [I2C]i2c6, set break_after_reset_flag
[ 13.285759] [I2C]drivers/soc/autochips/i2c/i2c-atc-ac8025.c,atc_i2c_process_msg,1185 i2c-6: timeout waiting on completion
[ 13.287352] [I2C]atc_i2c_master_xfer,1329 atc_i2c_master_xfer,1329 ret = -110
[ 13.288456] [I2C]atc_i2c_master_xfer,1337 WFail i2c6,A:0x3d,R:0xf5
[ 13.289359] [videoin][sensor_i2c] sensor_i2c_read(line:166): sensor_i2c_read failed, i2c_transfer ret=-110\x0a
```

2. Hsm 异常 log

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

1104 /1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

```
[0][W][I2C] i2c_write,772 I2C0 write 2 timeout(100 ms) status 0x0
[0][W][I2C] xI2CTransfer,1030 i2c0_write addr(0x0c) fail!
[0][E][DISP] File:driver/panel/atc_panel.c FUNC:atc_push_init_table Line:222: [PANEL]:[atc_push_init_table:222]I2C write fail(-110), addr:0xc, cmd:0x1
```

【产生原因】

1. 未使能或不支持总线死锁检测的 I2C（I2C6-I2C9）SDA 或 SCL 被设备拉低；
2. AP 端 I2C 中断被禁用或其他模块中断处理耗时大于 1s。

40.5.3.3 总线电平异常

【异常状态】SCL 被异常拉低、SCL 正常但 SDA 被异常拉低且执行 bus clear 后 SDA 依然被拉低

【错误码】-5

【确认方法】

1. SCL 被拉低

- a) hsm 中默认打开了 I2C 总线死锁检测功能（仅 I2C0-I2C5 支持）传输前会检测总线电平状态是否为空闲状态，当检测到 SCL 时，会打印如下 log 并返回错误码 -5

```
[0][E][I2C] File:i2c_v2.c FUNC:i2c_dead_lock_detect Line:659: i2c-0 SCL in error status, please check hw.
[0][E][I2C] File:i2c_v2.c FUNC:xI2CTransfer Line:1008: i2c-0, i2c bus status error!
[0][E][DISP] File:driver/panel/atc_panel.c FUNC:atc_push_init_table Line:222: [PANEL]:[atc_push_init_table:222]I2C write fail(-5), addr:0xc, cmd:0x63
```

- b) kernel 中配置 I2C 总线死锁检测功能后（仅 I2C0-I2C5 支持，默认未打开）当检测到 SCL 被拉低时，打印如下 log 并返回错误码 -5

```
[ 4.098224][ T327][E] [I2C]drivers/soc/autochips/i2c/i2c-atc-ac8025.c,atc_i2c_deadlock_detect,1060 i2c-0 SCL in error(low) status, please check hw
[ 4.099888][ T327][I] [TP] 0-005d: i2c test failed attempt 2: -5
```

2. SCL 正常，SDA 被拉低

- a) hsm（I2C0-I2C5）传输前检测到 SDA 被拉低，会尝试执行 bus clear，当执行完成后 SDA 依旧被拉低时，打印如下 log 并返回错误码-5

```
[10.892][W][I2C] i2c_dead_lock_detect,670 i2c-0 SDA is low, the i2c bus may have deadlocked, try to restore.
[10.892][I][I2C] i2c-0 dead lock restore fail!
[10.893][E][I2C] File:i2c_v2.c FUNC:i2c_dead_lock_detect Line:673: the i2c-0 restore fail, try to reset device.
[10.894][E][I2C] File:i2c_v2.c FUNC:xI2CTransfer Line:1007: i2c-0, i2c bus status error!
[10.896][E][DISP] File:driver/panel/atc_panel.c FUNC:atc_push_init_table Line:222: [PANEL]:[atc_push_init_table:222]I2C write fail(-5), addr:0xc, cmd:0x5b
```

- b) kernel 中打开 I2C 总线死锁检测功能后，当检测到 SDA 被拉低时尝试执行 bus clear，当执行完成后 SDA 依旧被拉低时，打印如下 log 并返回错误码-5

```
[ 109.601512][ T1995][W] [I2C]atc_i2c_deadlock_detect,1079 i2c-0 SDA in error(low) status, the i2c bus may have deadlocked, try to restore.
[ 109.603168][ T1995][I] [I2C]i2c-0 dead lock restore fail!
[ 109.603840][ T1995][E] [I2C]drivers/soc/autochips/i2c/i2c-atc-ac8025.c,atc_i2c_deadlock_detect,1082 The i2c-0 restore fail, try to reset slave device.
```

【产生原因】

SCL 或 SDA 对地短路，需排查硬件。

41 PWM

41.1 简介

41.1.1 软硬件环境

AC8025 同时支持 4 路 PWM 输出。

41.1.2 功能

AC8025 PWM 支持可动态配置输出频率及占空比。

41.1.3 代码结构及文件描述

表 41-1 PWM 代码结构及描述

文件名	文件路径	文件描述
atc_pwm.c	source/kernel/common/drivers/soc/autochips/pwm/	PWM kernel logic code
atc_pwm.h	source/kernel/common/drivers/soc/autochips/include/	PWM kernel header file
pwm.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/	PWM hsm logic code
pwm.h	source/vendor/autochips/proprietary/tinysys/hsm/os/include/	PWM hsm header file

41.2 软件设计

41.2.1 架构图

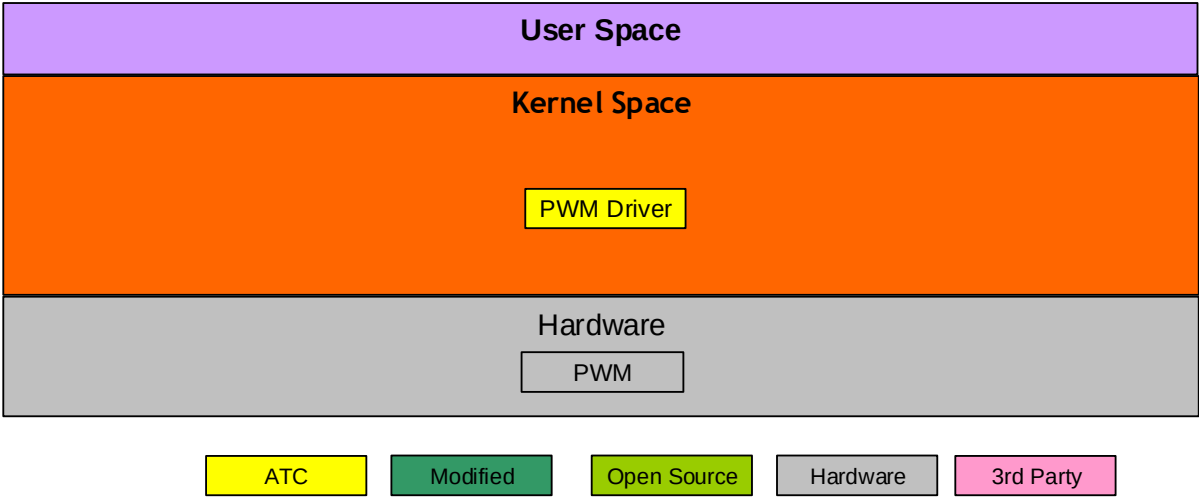


图 41-1 PWM 架构图

41.2.2 流程图

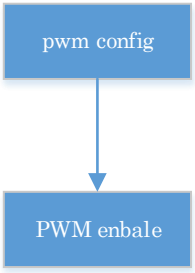


图 41-2 PWM 流程图

41.3 接口描述

41.3.1 API in Kernel

表 41-2 PWM API

API	说明
pwm_set_config	Config pwm

pwm_set_disable	Disable pwm
pwm_set_enable	Enable pwm
Pwm_set_duty	Set duty

41.3.1.1 pwm_set_config

【描述】

将 PWM 配置为 conf 参数指定的模式.

【语法】

s32 pwm_set_config (struct pwm_config *conf)

【参数】

配置 PWM 需要填充 pwm_config 结构体，以下为该结构体成员：

- pwm_no: pwm 通道号
- duty: 占空比，范围于 1~99
- freq: 频率
- mode:支持 NORMAL（默认）与 VSYNC
- vsynce_mode_edge: 极性设置。仅 VSYNC 有效，Normal 模式可设置为 0

【返回值】

- 0: config success.
- Other: config fail

41.3.1.2 pwm_set_disable

【描述】

禁用 PWM

【语法】

S32 pwm_set_disable (s32 pwm_no)

【参数】

pwm_no : pwm 通道号

【返回值】

- 0 : success
- Other : fail

41.3.1.3 pwm_set_enable

【描述】

使能 PWM

【语法】

S32 pwm_set_enable (s32 pwm_no)

【参数】

pwm_no : pwm 通道号

【返回值】

- 0 : success
- Other : fail.

41.3.1.4 pwm_set_duty

【描述】

设置 PWM 占空比

【语法】

S32 pwm_set_duty (s32 pwm_no, u32 duty)

【参数】

- pwm_no : pwm 通道号
- duty: 占空比 (1~99)

【返回值】

- 0 : success
- Other : fail.

41.3.2 API in hsm

表 41-3 PWM API

API	说明
xPWMDisable	关闭 PWM
xPWMConfig	配置 PWM
xPWMEnable	使能 pwm
pwm_set_duty	配置 PWM 占空比

41.3.2.1 xPWMDisable

【描述】

关闭 PWM

【语法】

```
in32_t xPWMDisable (int32_t pwm_no)
```

【参数】

pwm_no: PWM 控制器编号

【返回值】

- 0: 配置成功.
- Other:配置失败

41.3.2.2 xPWMConfig

【描述】

将 PWM 配置为 conf 参数指定的模式

【语法】

```
in32_t xPWMConfig (struct pwm_config *conf)
```

【参数】

配置 PWM,conf 结构体配置项为:

- pwm_no: pwm 通道号
- duty: 占空比, 范围于 1~99
- freq: 频率
- mode:支持 NORMAL (默认) 与 VSYNC
- vsynce_mode_edge: 极性设置, 仅 VSYNC 有效。Normal 模式可设置为 0

【返回值】

- 0: 配置成功
- Other:配置失败

41.3.2.3 xPWMEEnable

【描述】

使能 PWM

【语法】

in32_t xPWMEEnable (int32_t pwm_no)

【参数】

pwm_no: PWM 控制器编号

【返回值】

- 0: 配置成功.
- Other:配置失败

41.3.2.4 pwm_set_duty

【描述】

配置 PWM 占空比

【语法】

int pwm_set_duty (int32_t pwm_no, unsigned int duty)

【参数】

- pwm_no: PWM 控制器编号
- duty: 占空比（取值范围:1~99）

【返回值】

- 0: 配置成功.
- other:配置失败

41.3.3 数据结构

表 41-4 PWM 数据结构

数据结构	说明
struct pwm_config	Config pwm struct

41.3.3.1 pwm_config

```
struct pwm_config{
    U32 pwm_no;

    U32 duty;

    U32 freq;

    Enum pwm_mode_enum mode;

    S32 vsync_mode_edge;
}
```

具体说明如下：

- pwm_no: pwm 通道号
- duty: 占空比，范围于 1~99
- freq: 频率
- mode:支持 NORMAL（默认）与 VSYNC
- vsynce_mode_edge: 极性设置，配合 VSYNC mode 使用，Normal 模式配置为 0

41.4 使用说明和注意事项

41.4.1 时钟说明

1. PWM 时钟源最高为 200MHz，最大频率可置为 100MHz;
2. Sample Code

```
/******

*This sample code is for pwm programming reference.
******/

int pwm_set_spec_ex()
{
    int pwm_no = 0;

    pwm_set_disable(pwm_no);

    conf.pwm_no = pwm_no; //通道号

    conf.freq =100000;          //频率 100KHz

    conf.mode = PWM_MODE_NORMAL; //默认使用 NORMAL 模式
```

```
conf.duty = 50;    // 占空比设置为 50%

pwm_set_config(&conf); // 配置 PWM

pwm_set_enable(pwm_no); // 使能 pwm
}
```

41.4.2 Vsync Mode

Vsync mode 下，display 的 vsync 信号会连接至 pwm 控制器。pwm 模块在检测到 Vsync 到达配置的极性时，才会将 hightime 寄存器的变化更新到 pwm_output 波形上。

【作用】该模式作用在于只在 Vsync 消隐区间才会更新 pwm hightime 变化，防止 pwm 占空比变化导致的屏幕闪烁抖动。

【配置注意】一般配置 pwm sync mode 的 pol 为 Vsync 同步头的极性，目前 display 输出的 Vsync 信号同步头极性默认为负，所以 pwm VSYNC mode 时候，sync 极性也要配置为负。

42 SPI

42.1 简介

42.1.1 软硬件环境

- SPI 接口四个 PIN 如下：

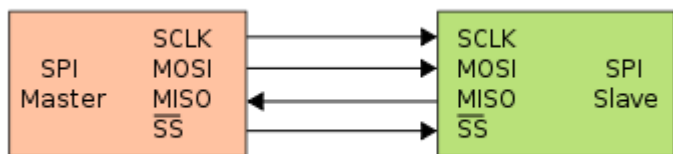


图 42-1 SPI PIN 示意图

- AC8025 共支持 5 组 SPI
 - AC8025 Secure Sub-System 提供了 2 组 SPI 接口(SPI0、SPI1)，只可被 R5 访问；
 - AC8025 LPIO Sub-System 提供了 3 组 SPI 接口(SPI2-SPI4)，R5 与 AP 均可访问。



注意：

Secure 下 SPI 模块与 SGPIO 复用 PIN 脚，因此实际支持情况由 PIN 功能决定。

42.1.2 功能

此模块用于支持 SPI 总线上主机和从机或其他外围设备之间的全双工、同步、串行通信。

42.1.3 代码结构及文件描述

表 42-1 SPI 代码结构及描述

文件名	文件路径	文件描述
spi-atc-ac8025.c	source/kernel/common/drivers/soc/autochips/spi/	SPI kernel logic code
spi.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/spi/	Hsm spi driver
spi.h	source/vendor/autochips/proprietary/tinysys/hsm/os/include/ac8025/	hsm spi header file

42.2 软件设计

42.2.1 架构图

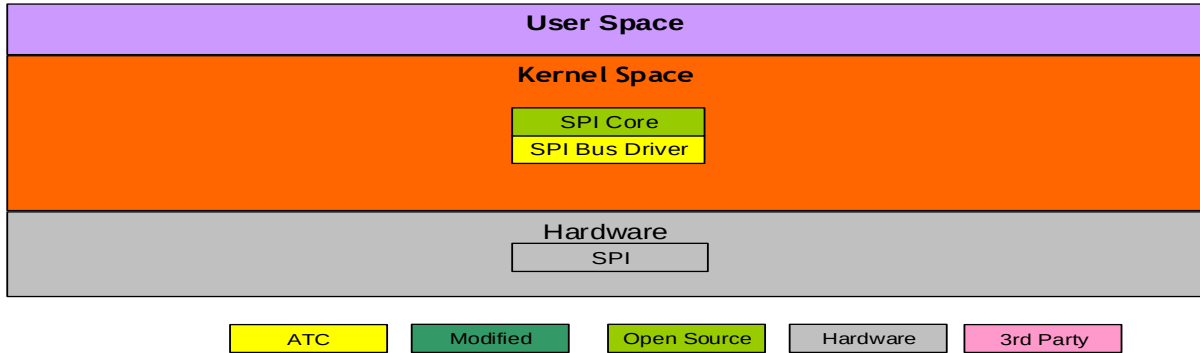


图 42-2 SPI 架构图

42.2.2 流程图

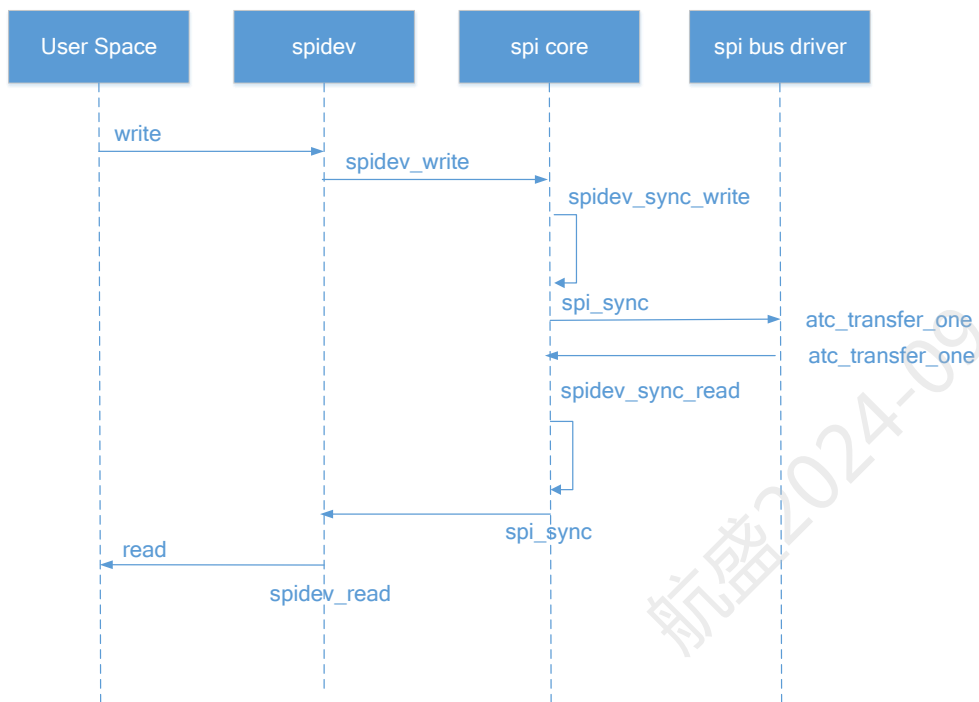


图 42-3 SPI 读写流程图

42.3 接口描述

42.3.1 API in Kernel

表 42-2 SPI kernel API

API	说明
atc_transfer_one	spi master send/receive data

42.3.1.1 atc_transfer_one

【描述】

spi master send/receive data.

【语法】

S32 atc_transfer_one (struct spi_master *master, struct spi_device *spi, struct spi_transfer *transfer)

【参数】

配置 SPI 需要填充 spi_config 结构体，以下为该结构体成员

- struct spi_master : spi master
- struct spi_device: spi device
- struct spi_transfer: spi message and transfer length

【返回值】

The value returned from register spi driver if transfer is successful.

42.3.2 API in Hsm

表 42-3 SPI API in hsm

API	说明
xSpiSetSpeed	设置 SPI 速率
xSpiSetMode	设置 SPI 模式
xSpiSetCs	设置 SPI 片选
xSpiXfer	SPI 传输接口

42.3.2.1 xSpiSetSpeed

【描述】

设置 SPI 速率

【语法】

```
int xSpiSetSpeed (uint8_t bus, uint32_t speed)
```

【参数】

- bus: 需要设置的 SPI
- speed: 需要设置的速率

【返回值】

- 0: 设置成功.
- -1: 设置失败

42.3.2.2 xSpiSetMode

【描述】

设置 SPI 传输模式

【语法】

```
int xSpiSetMode (uint8_t bus, uint32_t mode)
```

【参数】

- bus: 需要设置的 SPI
- mode: 需要设置的模式

【返回值】

- 0: 设置成功.
- -1: 设置失败

42.3.2.3 xSpiSetCs

【描述】

设置 SPI 片选

【语法】

```
void xSpiSetCs (uint8_t bus, uint8_t cs, uint8_t enable)
```

【参数】

- bus: 需要设置的 SPI

- cs: 需要设置的片选号
- enable: 0: 释放 1: 选中

42.3.2.4 xSpiXfer

【描述】

SPI 传输接口

【语法】

```
int xSpiXfer (uint8_t bus, uint32_t len, const char *dout, _char *din, uint64_t flags)
```

【参数】

- bus: 需要设置的 SPI
- len: 需要 exchange 的数据长度
- dout: 发送 buffer
- din: 接收 buffer
- flags: 传输标志位（可用于设置 cs 选择模式、指定 cs）

【返回值】

- len: 传输成功.
- -1: 传输失败

42.4 使用说明和注意事项

42.4.1 通过 spidev 使用 SPI

42.4.1.1 打开 SPI 设备

```
int fd;

static const char *device = "/dev/spidev1.0"; /*open spi1.1 device*/

fd = open(device, O_RDWR);
```

42.4.1.2 设置相关参数

```
ioctl(fd, SPI_IOC_WR_MODE32, &mode); /*set spi mode*/

ioctl(fd, SPI_IOC_RD_MODE32, &mode); /*get spi mode*/
```

```
ioctl(fd, SPI_IOC_WR_MAX_SPEED_HZ, &speed); /*set spi max speed hz*/
ioctl(fd, SPI_IOC_RD_MAX_SPEED_HZ, & speed); /*get spi max speed hz */
```

42.4.1.3 传输数据

```
struct spi_ioc_transfer xfer = { /*set data structure*/
    .tx_buf = (unsigned long)tx,
    .rx_buf = (unsigned long)rx,
    .len    = ARRAY_SIZE(tx),
    .delay_usecs = delay,      /* optional */
    .speed_hz = speed,        /* optional */
    .bits_per_word = bits,     /* optional */
    .cs_change = cs_change, /* optional */
};
ioctl(fd, SPI_IOC_MESSAGE(1), &xfer); /*transfer data*/
```

42.4.1.4 关闭 SPI 设备

```
close(fd); /*release spi device*/
```

42.4.2 Kernel 使用 SPI

42.4.2.1 使用 GPIO 作为 cs 配置方法

在对应的 SPI 设备节点中添加下图属性（以 SPI2 为例）

1. 增加如①所示的 cs gpio 信息，cs0 保持为<0>, 其后为 cs1（本例中 Pin70 脚配置为 spi2 cs1 脚）
2. 增加如②所示的 spi2cs1 设备节点，配置成功启动后会有/dev/spidev2.1 设备文件生成；
3. ③为配置 Pin 脚 pinmux 属性，将其设置为 gpio mode，输出高电平。

```
&spi2 {
    pinctrl-names = "default";
    pinctrl-0 = <&spi2_pins>;
    /* cs0 is dedicated pin */
    cs-gpios = <0>, <&pio0 70 0>;
    status = "okay";

    spi2cs0: spidev@0 {
        compatible = "atc,spidev";
        reg = <0>;
        spi-max-frequency = <21875000>;
    };

    spi2cs1: spidev@1 {
        compatible = "atc,spidev";
        reg = <1>;
        spi-max-frequency = <21875000>;
    };
};

&pio0 {
    spi2_pins: spi2-pins {
        spi2_csn {
            pinmux = <PINMUX_PIN105_FUNC_SPI3_CSN>;
            drive-strength = <DRIVE_8mA>;
        };
        spi2_sclk {
            pinmux = <PINMUX_PIN104_FUNC_SPI3_SCLK>;
            drive-strength = <DRIVE_8mA>;
        };
        spi2_miso {
            pinmux = <PINMUX_PIN107_FUNC_SPI3_MISO>;
            drive-strength = <DRIVE_8mA>;
        };
        spi2_mosi {
            pinmux = <PINMUX_PIN106_FUNC_SPI3_MOSI>;
            drive-strength = <DRIVE_8mA>;
        };
        spi2_cs1 {
            pinmux = <PINMUX_PIN70_FUNC_GPIO_61>;
            bias-pull-up;
            output-high;
        };
    };
};
```

42.4.2.2 SPI Pin 脚配置

说明：SPI 设备节点 `spidev0.x`、`spidev1.x` 和 `spidev2.x` 分别对应硬件上的 SPI1-SPI3。其中 SPI1 和 SPI2 只有一组 Pin 脚，且 Pin 脚默认为 GPIO 模式，分别为 Pin42-Pin45 和 Pin36-Pin39；SPI3 支持两组 Pin，其中 Pin104-Pin107 默认为 SPI 模式，Pin26-Pin29 默认为 GPIO 模式，复用模式为 SPI 模式。当使用 Pin104-Pin107 用作 SPI 功能时，必须将 Pin26-Pin29 配置为 GPIO 功能，同理，在使用 Pin26-Pin29 作为 SPI 功能时，需要将 Pin104-107 配置为非 SPI 功能。

由于 SPI1 和 SPI2 Pin 脚默认为 GPIO 模式，使用时需要通过 pinmux 将其配置为 SPI 模式，当前提供了 SPI1-SPI2 的 pinmux 配置，enable 对应 SPI 节点后启动过程中会自动设置（须确保其他模块不会使用对应的 Pin）

```
&spi0 {
    pinctrl-names = "default";
    pinctrl-0 = <&spi0_pins>;
    /* cs0 is dedicated pin */
    cs-gpios = <0>;
    status = "disabled";

    spi0cs0: spidev@0 {
        compatible = "atc,spidev";
        reg = <0>;
        spi-max-frequency = <43500000>;
    };
};

&spi1 {
    pinctrl-names = "default";
    pinctrl-0 = <&spi1_pins>;
    /* cs0 is dedicated pin */
    cs-gpios = <0>;
    status = "disabled";

    spi1cs0: spidev@0 {
        compatible = "atc,spidev";
        reg = <0>;
        spi-max-frequency = <43750000>;
    };
};
```

SPI3 默认使用 Pin104-Pin107，若需改用 Pin26-Pin29 需要根据说明修改对应 pinmux。

42.4.3 hsm 使用 SPI

hsm 中可访问所有的 Secure SPI 和所有的 LPIO SPI，当前使用同一套接口进行操作，因此访问时需要使用 spi.h 中的 SPI_ID 枚举作为接口中的 bus 参数。

```
/* SPI bus id definition */
enum SPI_ID {
    SPI_SS_0    = 0,
    SPI_SS_1    = 1,    /* Secure Subsystem SPI0/SPI1 */
    SPI_LS_0    = 2,
    SPI_LS_1    = 3,
    SPI_LS_2    = 4,    /* LPIO Subsystem SPI0/SPI1/SPI2 */
    SPI_MAX     = 5,
};
```

42.4.3.1 使能 SPI

Hsm 中 SPI 由 SPI driver 统一初始化，可通过配置 spi.h 中的 SSPI_X_ENABLE 和 SPI_X_ENABLE 宏使能或禁用。

```
/* Security Domain SPI config */
#define SSPI_0_ENABLE    0 /* 0: disable, 1: enable */
#define SSPI_1_ENABLE    1

#define SSPI_0_MOSI      SFT_SPI_0_MOSI /* Any pin in sgpio_pinIndex, select it in sgpio.h */
#define SSPI_0_MISO      SFT_SPI_0_MISO
#define SSPI_0_CSN       SFT_SPI_0_CSN
#define SSPI_0_SCLK      SFT_SPI_0_SCLK

#define SSPI_1_MOSI      SFT_GPIO_0
#define SSPI_1_MISO      SFT_GPIO_1
#define SSPI_1_CSN       SFT_GPIO_2
#define SSPI_1_SCLK      SFT_GPIO_3

/* Non-Security Domain SPI config */
#define SPI_0_ENABLE     0 /* 0: disable, 1: enable */
#define SPI_1_ENABLE     0
#define SPI_2_ENABLE     1
```

42.4.3.2 配置 SPI 引脚

SGPIO 支持复用功能全路由配置，因此 Secure SPI 可以将 Pin 脚配置到 SFT_IO 的任意一个 Pin 上（同一功能只能配置到一个 Pin 上）可通过下图①配置 secure SPI0 – SPI1 所使用的 Pin 脚（需使用枚举 sgpio_pinIndex 中的值）。

LPIO SPI 中 SPI2（硬件对应 SPI3）支持配置 Pin 脚配置，默认使用 IONUM_SPI3_MOSI、IONUM_SPI3_MISO、IONUM_SPI3_CSN 和 IONUM_SPI3_SCLK，另一组 Pin 脚可参考下图②注释。

```

/* Security Domain SPI config */
#define SSPI_0_ENABLE    0 /* 0: disable, 1: enable */
#define SSPI_1_ENABLE    1

#define SSPI_0_MOSI      SFT_SPI_0_MOSI /* Any pin in sgpio_pinIndex, select it in sgpio.h */
#define SSPI_0_MISO      SFT_SPI_0_MISO
#define SSPI_0_CSN       SFT_SPI_0_CSN
#define SSPI_0_SCLK      SFT_SPI_0_SCLK

#define SSPI_1_MOSI      SFT_GPIO_0
#define SSPI_1_MISO      SFT_GPIO_1
#define SSPI_1_CSN       SFT_GPIO_2
#define SSPI_1_SCLK      SFT_GPIO_3

/* Non-Security Domain SPI config */
#define SPI_0_ENABLE     0 /* 0: disable, 1: enable */
#define SPI_1_ENABLE     0
#define SPI_2_ENABLE     1

#define LSPI_2_MOSI      IONUM_SPI3_MOSI /* IONUM_SPI3_MOSI or IONUM_GPIO_28 */
#define LSPI_2_MISO      IONUM_SPI3_MISO /* IONUM_SPI3_MISO or IONUM_GPIO_29 */
#define LSPI_2_CSN       IONUM_SPI3_CSN  /* IONUM_SPI3_CSN or IONUM_GPIO_27 */
#define LSPI_2_SCLK      IONUM_SPI3_SCLK /* IONUM_SPI3_SCLK or IONUM_GPIO_26 */

```

42.4.3.3 配置 SPI 参数

设置 SPI 工作模式：

```

uint32_t mode = 0;
xSpiSetMode(bus, mode); // mode0
mode = SPI_CPHA;
xSpiSetMode(bus, mode); // mode1
mode = SPI_CPOL;
xSpiSetMode(bus, mode); // mode2
mode = SPI_CPOL | SPI_CPHA;
xSpiSetMode(bus, mode); // mode3

```

设置 SPI 速率：

```
xSpiSetSpeed(bus, 5*1000*1000); // 5M
```

设置 SPI CS：

SPI 片选支持 spi driver 自动设置或用户调用接口手动设置，设置方法如下：

1. 自动设置

```

uint64_t flags = 0;
flags = SPI_CS_0; // 使用 CS0

```

```
xSpiXfer(bus, len, dout, din, flags)
```

```
flags = SPI_CS_1; // 使用 CS1 (默认只支持 CS0, 其余 CS 需要配置 GPIO)
```

```
xSpiXfer(bus, len, dout, din, flags)
```

2. 手动设置

```
flags = SPI_CS_0 | SPI_CS_NO_CTRL; // 使用 CS0, 手动控制 CS
```

```
xSpiSetCs(bus, SPI_CS_0, 1); // 拉低 CS0
```

```
xSpiXfer(bus, len, dout, din, flags);
```

```
xSpiSetCs(bus, SPI_CS_0, 0); // 拉高 CS0
```

使用 GPIO 作为 CS 配置方法:

当前 Secure SPI 和 LPIO SPI 控制器均默认支持 1 个 CS。若需要连接多个设备, 需要使用 GPIO 作为片选, 使用 GPIO 用作片选后每个控制器最大支持 8 个 CS。

```
/* spi cs gpio define */
static struct cs_gpio_t cs_gpios_tbl[SPI_MAX_BUS][SPI_CS_NUM_MAX] = {
    {
        /* NOTE: SSPI-0 CS0 is dedicated pin */
        {.pin = 0, .mode = 0, .idle_level = 0},
    },
    {
        /* NOTE: SSPI-1 CS0 is dedicated pin */
        {.pin = 0, .mode = 0, .idle_level = 0},
        /* Add other cs gpio */
        // {.pin = SFT_SPI_0_MOSI, .mode = SGPIO_OUTPUT_MODE, .idle_level = 1},
    },
    {
        /* NOTE: SPI-0 CS0 is dedicated pin */
        {.pin = 0, .mode = 0, .idle_level = 0},
        /* Add other cs gpio */
        // {.pin = 36, .mode = 1, .idle_level = 1},
    },
    {
        /* NOTE: SPI-1 CS0 is dedicated pin */
        {.pin = 0, .mode = 0, .idle_level = 0},
    },
    {
        /* NOTE: SPI-2 CS0 is dedicated pin */
        {.pin = 0, .mode = 0, .idle_level = 0},
    },
};
```

Secure SPI 和 LPIO SPI 配置可参考上图①、②位置注释。SPI GPIO CS 需要配置 3 个参数, 分别为:

➤ pin : pin number

- mode: 配置为 GPIO 模式时的 mode id (SGPIO 固定配置为 SGPIO_OUTPUT_MODE)
- idle_level: CS 空闲电平

配置完成后 SPI 控制器初始化时会根据配置参数自动完成配置。

注意: CS0 为专用 Pin, 不可修改配置参数。

42.4.3.4 使用 SPI 进行数据传输

示例代码如下:

```
#define TEST_LEN (8)
int spi_test_demo(int id)
{
    char dout[TEST_LEN] = {0xaa, 0x55, 0x22, 0x11, 0xbb, 0x33, 0xdd, 0xaa};
    char din[TEST_LEN] = {0};
    uint64_t flags = 0;
    uint32_t mode = 0;

    mode = SPI_CPHA;    // mode1
    flags = SPI_CS_0;   // CS0
    xSpiSetMode(id, mode);
    xSpiSetSpeed(id, 10*1000*1000); // 10M

    if (xSpiXfer(id, TEST_LEN, dout, din, flags) != TEST_LEN) {
        pr_info("spi-%d test fail.\n", id);
        return -1;
    }

    return 0;
}
```

43 UART

43.1 简介

43.1.1 软硬件环境

- AC8025 芯片，同时支持 7 路 UART
 - AC8025 Secure Sub-system 提供 2 组 UART（UART0-1）只能在 Secure 下通过 R5 访问
 - AC8025 LPIO Sub-system 提供 5 组 UART（UART2-6），R5 与 AP 均可访问
 - UART1-UART4 为普通 UART 速率最高到 921600 baud
 - UART0/UART5/UART6 为高速 UART 速率可最高支持至 3692000 baud。



注意：

Secure 下 UART 模块与 SGPIO 复用 PIN 脚，因此实际支持情况由 PIN 功能决定。

43.1.2 功能

此模块用于支持通过 UART 的数据传输。

43.1.3 代码结构及文件描述

表 43-1 UART 代码结构及描述

文件名	文件路径	文件描述
atc_uart_ac8025.c	source/kernel/common/drivers/tty/serial/	Kernel Logic and relization of uart
uart.c	source/vendor/autochips/proprietary/tinysys/hsm/os/drivers/uart	Hsm logic and relization of uart

43.2 软件设计

43.2.1 架构图

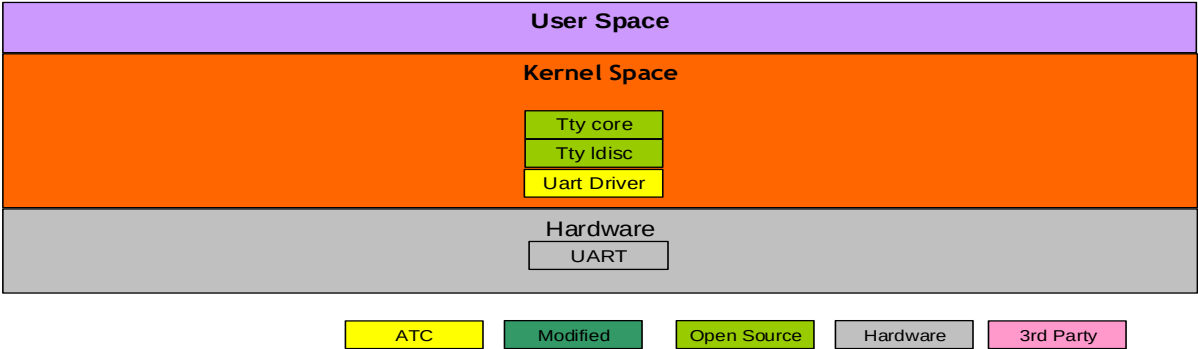


图 43-1 UART 架构图

43.2.2 流程图

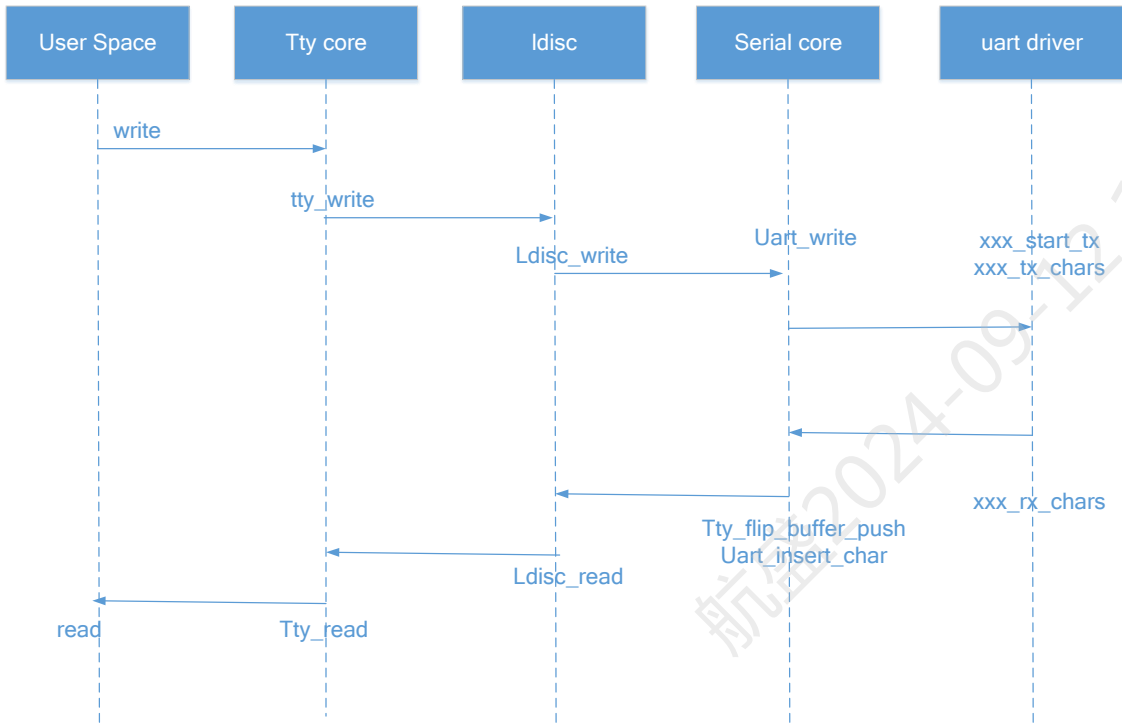


图 43-2 UART 读写流程图

43.3 接口描述

43.3.1 API in Kernel

表 43-2 UART kernel API

API	说明
ac8x_uart_init	uart Init
ac8x_startup	open uart and init
ac8x_shutdown	release uart
ac8x_start_tx	transfer data

43.3.1.1 ac8x_uart_init

【描述】

初始化 uart.

【语法】

```
int ac8x_uart_init(void)
```

【参数】

None

【返回值】

The value returned from register uart driver if registration is successful.

43.3.1.2 ac8x_startup

【描述】

Open and init uart port

【语法】

```
int ac8x_startup(struct uart_port *port)
```

【参数】

struct uart_port

【返回值】

The value returned from open and init uart if it is successful.

43.3.1.3 ac8x_shutdown

【描述】

Release uart port

【语法】

```
void ac8x_shutdown(struct uart_port *port)
```

【参数】

struct uart_port

【返回值】

None

43.3.1.4 ac8x_start_tx

【描述】

Transfer data from uart port

【语法】

```
void ac8x_start_tx(struct uart_port *port)
```

【参数】

struct uart_port

【返回值】

None

43.3.1.5 ac8x_receive

【描述】

Receive data from uart port.

【语法】

```
void ac8x_receive (struct ac8x_uart *uart)
```

【参数】

struct ac8x_uart

【返回值】

None

43.3.2 API in hsm

表 43-3 UART hsm API

API	说明
int uart_hw_init(uint8_t uart_num)	初始化 UART 为默认参数。
int uart_init(UART_ID uart_num, BAUD_TYPE baud, DATA_BITS bits, PARITY_TYPE parity, STOP_BITS stop)	设置 UART 参数并初始化 UART
int uart_read(uint8_t uart_num)	uart 读接口（阻塞）。
int uart_write(uint8_t uart_num, uint8_t data)	uart 写接口。
int setbaudrate(s32 uart_num, u32 baudrate)	uart 波特率设置（即将废弃）。
Int uart_set_baudrate(uint8_t uart_num, uint32_t baudrate)	uart 波特率设置(推荐使用)
int uart_read_noblock(uint8_t uart_num, uint8_t data)	uart 读接口（非阻塞）。
uart_monitor_register(uint8_t uart_num)	uart monitor 注册接口
uart_monitor_read(uint8_t uart_num)	uart monitor 读接口
uart_monitor_deregister(uint8_t uart_num)	uart monitor 注销接口

43.3.2.1 uart_hw_init

【描述】

初始化 UART 为默认参数(115200 波特率、8 位数据位、1 位停止位、无校验)

【语法】

```
int uart_hw_init(uint8_t uart_num)
```

【参数】

uart_num: 指定 UART 端口

【返回值】

- 0: 初始化成功
- -1: 初始化失败

43.3.2.2 uart_init

【描述】

以指定参数初始化 UART

【语法】

```
int uart_init(uint8_t uart_num)
```

【参数】

- **uart_num**: 指定 UART 端口，需要使用 UART_ID 枚举中的值
 - UART0: Secure UART0
 - UART1: Secure UART1
 - UART2: 对应硬件 Pin UART2
 - UART3: 对应硬件 Pin UART3
 - UART4: 对应硬件 Pin UART4
 - UART5: 对应硬件 Pin UART1
 - UART6: 对应硬件 Pin UART5
- **baud**: 指定要设置的波特率 ID，需要使用 BAUD_TYPE 枚举中的值
 - BAUD_600: 波特率 600
 -: 剩余枚举值见 uart.h 头文件
 - BAUD_3500000: 波特率 3500000
- **Bits**: 数据位长度，需要使用 DATA_BITS 枚举中的值
 - CS8: 8 位数据位
 - CS7: 7 位数据位
 - CS6: 6 位数据位
- **Parity**: 校验类型，需要使用 PARITY_TYPE 枚举中的值
 - PAR_EVEN: 奇校验
 - PAR_ODD: 偶校验
 - PAR_SPACE: 空校验
 - PAR_MARK: 标记校验
 - PAR_NONE: 无校验
- **Stop**: 停止位长度，需要使用 STOP_BITS 枚举中的值
 - NBSTOP1: 1 位停止位
 - NBSTOP1_5: 1.5 位停止位
 - NBSTOP2: 2 位停止位

【返回值】

- 0: 初始化成功
- -1: 初始化失败

43.3.2.3 uart_read

【描述】

uart 阻塞读接口，调用后直到读到 1byte 数据才返回

【语法】

```
int uart_read(uint8_t uart_num)
```

【参数】

uart_num: 指定 UART 端口

【返回值】

- Other : 读取的字节
- <0: 读取失败

43.3.2.4 uart_write

【描述】

Uart 写接口

【语法】

```
void uart_write(uint8_t uart_num, uint8_t data)
```

【参数】

- uart_num: 指定 UART 端口
- data: 写入的字符

【返回值】

- None.

43.3.2.5 setbaudrate

【描述】

设置 uart 波特率（未来将废弃，请使用 uart_set_baudrate 替代）

【语法】

```
int setbaudrate(s32 uart_num, u32 baudrate)
```

【参数】

- `uart_num`: 指定 UART 端口
- `baudrate`: 需要设置的波特率

【返回值】

- `Other`: 设置成功后的波特率
- `-1`: 设置失败.

43.3.2.6 `uart_set_baudrate`

【描述】

设置 `uart` 波特率

【语法】

```
int uart_set_baudrate(uint8_t uart_num, uint32_t baudrate)
```

【参数】

- `uart_num`: 指定 UART 端口
- `baudrate`: 需要设置的波特率值

【返回值】

- `Other`: 设置成功后的波特率
- `-1`: 设置失败.

43.3.2.7 `uart_read_noblock`

【描述】

`uart` 非阻塞读数据

【语法】

```
int uart_read_noblock(uint8_t uart_num, uint32_t *data)
```

【参数】

- `uart_num`: 指定 UART 端口
- `data`: 保存读取数据的指针

【返回值】

- 大于 0: 读取成功
- 0: 无数据
- -1: 读取失败.

43.3.2.8 uart_monitor_register

【描述】

注册 uart monitor, 注册后启动 monitor 功能, rx 数据被自动接收到循环缓冲区中

【语法】

```
int uart_monitor_register(uint8_t uart_num)
```

【参数】

- uart_num: 指定 UART 端口

【返回值】

- 0: 注册成功
- -1: 注册失败.

43.3.2.9 uart_monitor_deregister

【描述】

注销 uart monitor, 注销后 monitor 停止接收

【语法】

```
int uart_monitor_deregister(uint8_t uart_num)
```

【参数】

- uart_num: 指定 UART 端口

【返回值】

- 0: 注销成功
- -1: 注销失败.

43.3.2.10 uart_monitor_read

【描述】

从 uart monitor 的接收缓冲区中读取数据

【语法】

```
int uart_monitor_read(uint8_t uart_num, char *buf, uint32_t count)
```

【参数】

- uart_num: 指定 UART 端口
- buf: 存放 rx 数据的缓冲区
- count: 要读取的字节数，当缓冲区数据量小于 count 时返回实际读取的字节数

【返回值】

- other: 读取的数据长度
- -1: 读数据失败

43.4 使用说明和注意事项

43.4.1 用户空间使用 uart

43.4.1.1 打开 uart 设备

```
ttyAT0→uart0;    /* /dev/ttyAT0 is uart0 */
ttyAT1→uart1;    /* /dev/ttyAT1 is uart1 */
ttyAT2→uart2;    /* /dev/ttyAT2 is uart2 */
ttyAT3→uart3;    /* /dev/ttyAT3 is uart3*/
ttyAT4→uart3;    /* /dev/ttyAT4 is uart4*/

int fd;
char * device = "/dev/ttyAT1";
fd = open(device, O_RDWR);/*open uart1 device*/
if (fd < 0)
{
    printf("open uart1 error\n");
    return -1;
}
```

43.4.1.2 设置 uart 相关参数

基本的设置：baud rate, stop bits 等。

```
struct termios {
    tcflag_t c_iflag; /* input mode flags */
    tcflag_t c_oflag; /* output mode flags */
    tcflag_t c_cflag; /* control mode flags */
    tcflag_t c_lflag; /* local mode flags */
    cc_t c_line;      /* line discipline */
    cc_t c_cc[NCCS]; /* control characters */
};
```

Baud rate setting:

```
set_speed(int fd, int speed) {
    struct termios Opt;
    int ret;
    tcgetattr(fd, &Opt); /*get termios parameter*/
    switch(speed) {
        case 9600:
            cfsetispeed(&Opt, B9600); /*set baud rate 9600*/
            cfsetispeed(&Opt, B9600);
            break;

        case 38400:
            cfsetispeed(&Opt, B38400); /*set baud rate 38400*/
            cfsetispeed(&Opt, B38400);
            break;

        case 115200:
            cfsetispeed(&Opt, B115200); /*set baud rate 115200*/
            cfsetispeed(&Opt, B115200);
            break;

        case 230400:
```

```

        cfsetispeed(&Opt, B230400); /*set baud rate 230400*/
        cfsetispeed(&Opt, B230400);
        break;

    case 460800:

        cfsetispeed(&Opt, B460800); /*set baud rate 460800*/
        cfsetispeed(&Opt, B460800);
        break;

    case 921600:

        cfsetispeed(&Opt, B921600); /*set baud rate 921600*/
        cfsetispeed(&Opt, B921600);
        break;

    case 3200000:

        cfsetispeed(&Opt, B3200000); /*set baud rate 3200000*/
        cfsetispeed(&Opt, B3200000);
        break;

}

tcsetattr(fd, TCANOW, &Opt); /*set termios parameter*/
}

```

43.4.1.3 读写操作

```

#define BUFLLEN    2*1024

int ret;

char red_buf[BUFLLEN]; /*alloc read buffer*/
char wrt_buf[BUFLLEN]; /*alloc write buffer*/
char *prbuf = red_buf;
char *pwbuf = wrt_buf;

ret = read(fd, prbuf, BUFLLEN); /*receive data from uart device*/
ret = write(fd, pwbuf, BUFLLEN); /*transfer data to uart device*/

```

43.4.1.4 关闭 uart 设备

```

close(fd); /*release uart device*/

```

43.4.2 hsm 使用 uart

hsm 中可访问所有的 Secure UART 和所有的 LPIO UART，当前使用同一套接口进行操作，因此访问时需要使用 uart.h 中的 UART_ID 枚举作为接口中的 uart_num 参数。

```
/* UART id */
typedef enum {
    UART0 = 0,          /* Secure UART0 */
    UART1 = 1,          /* Secure UART1 */
    UART2 = 2,          /* LGC_UART0 -> HW Pin UART2 */
    UART3 = 3,          /* LGC_UART1 -> HW Pin UART3 */
    UART4 = 4,          /* LGC_UART2 -> HW Pin UART4 */
    UART5 = 5,          /* FLW_UART0 -> HW Pin UART1 */
    UART6 = 6,          /* FLW_UART1 -> HW Pin UART5 */
    UART_MAX,           /* Max UART id */
} UART_ID;
```

43.4.2.1 配置 uart pin 脚

SGPIO 支持复用功能全路由配置，因此 Secure UART 可将 Pin 脚配置到 SFT_IO 的任意一个 Pin 上（同一功能只能配置到一个 Pin 上）可通过下图①配置 Secure UART0 – UART1 所使用的 Pin 脚（需使用枚举 sgpio_pinIndex 中的值）

LPIO UART 中 UART2、UART3 支持两组 Pin 脚，默认使用 IONUM_UART2_TX、IONUM_UART2_RX、IONUM_UART3_TX 和 IONUM_UART3_RX，使用其他 Pin 脚可参考下图②中的注释。

```
/* Security Domain UART config */
#define UART0_TX_PIN    SFT_UART_0_TX /* Any pin in sgpio_pinIndex, select it in sgpio.h */
#define UART0_RX_PIN    SFT_UART_0_RX
#define UART0_CTS_PIN   SFT_UART_0_CTS
#define UART0_RTS_PIN   SFT_UART_0_RTS ①

#define UART1_TX_PIN    UART_PIN_RES /* Any pin in sgpio_pinIndex, select it in sgpio.h */
#define UART1_RX_PIN    UART_PIN_RES /* Any pin in sgpio_pinIndex, select it in sgpio.h */

/* Non-Security Domain UART config */
#define UART2_TX_PIN    IONUM_UART2_TX /* IONUM_GPIO_20 or IONUM_UART2_TX */
#define UART2_RX_PIN    IONUM_UART2_RX /* IONUM_GPIO_21 or IONUM_UART2_RX */

#define UART3_TX_PIN    IONUM_UART3_TX /* IONUM_GPIO_8 or IONUM_UART3_TX */
#define UART3_RX_PIN    IONUM_UART3_RX /* IONUM_GPIO_9 or IONUM_UART3_RX */

#define UART4_TX_PIN    IONUM_GPIO_30 ②
#define UART4_RX_PIN    IONUM_GPIO_31

#define UART5_TX_PIN    IONUM_UART1_TX
#define UART5_RX_PIN    IONUM_UART1_RX
#define UART5_CTS_PIN   IONUM_UART1_CTS
#define UART5_RTS_PIN   IONUM_UART1_RTS

#define UART6_TX_PIN    IONUM_UART5_TX
#define UART6_RX_PIN    IONUM_UART5_RX
#define UART6_CTS_PIN   UART_PIN_RES /* IONUM_GPIO_40 */
#define UART6_RTS_PIN   UART_PIN_RES /* IONUM_GPIO_41 */
```

Pin 脚将在 uart 初始化时被设置。

43.4.2.2 配置 uart 参数

使用 uart 前需要配置 uart 参数，当前提供以下接口用于 uart 参数配置：

```
/* uart configuration interface */
int uart_hw_init(uint8_t uart_num); /* Set default Mode as Normal, 115200 baudrate, 8 data bits, 1 stop bit */
int uart_init(UART_ID uart_num, BAUD_TYPE baud, DATA_BITS bits, PARITY_TYPE parity, STOP_BITS stop);
int uart_set_baudrate(uint8_t uart_num, uint32_t baudrate);
```

uart_hw_init() 接口使用默认参数（115200 波特率、8 位数据、无校验、1 位停止位）初始化 uart。

uart_init() 可根据提供的参数设置波特率、数据长度、校验类型和停止位长度，需使用各个参数的枚举中的成员进行设置。uart_set_baudrate() 接口用于单独设置 uart 波特率，与 uart_init() 接口的波特率参数不同，uart_set_baudrate() 的波特率参数为具体的波特率数值，支持设置 BAUD_TYPE 枚举未包含的波特率。

43.4.2.3 使用 uart 发送数据

```
char txbuf[10] = {0xaa, 0xbb, 0xff, 0x55, 0x23, 0x21, 0x22, 0x34, 0x3d, 0xbe};
int i;
for (i = 0; i < 10; i++) {
    uart_write(UART3, txbuf[i]);
}
```

43.4.2.4 使用 uart 接收数据

【方法 1】使用阻塞接口

```
char rxbuf[10] = {0};
int i;
for (i = 0; i < 10; i++) {
    rxbuf[i] = uart_read(UART3);
}
```

【方法 2】使用非阻塞接口

```
int ret;
char data;
while(1) {
```

```
ret = uart_read_noblock(UART3, &data);

if (ret > 0) { // 接收到数据
    pr_info("rx: 0x%02x\n", data);
}
}
```

方法 3: 使用 monitor 接口（推荐使用）

```
int ret;
char rxbuf[100];

uart_hw_init(UART3); // 初始化 uart3
uart_monitor_register(UART3) // 注册并启动 uart monitor
/* 循环从 uart monitor 缓冲区中读出数据 */
while(1) {
    ret = uart_monitor_read (UART3, rxbuf, 100); // 读出 100 个字节数据
    if (ret > 0) {
        for (int i = 0; i < ret; i++)
            pr_info("%d: 0x%02x\n", i, rxbuf[i]); // dump uart 接收到的数据
    }
    vTaskDelay(pdMS_TO_TICKS(5UL)); // 延时 5ms
}

uart_monitor_register(UART3); // 注销 uart monitor, uart monitor 停止接收数据
```

43.4.3 uart 软硬件对应关系

UART 软硬件对应关系			
控制器名称	节点名称	硬件 Pin 脚名称	备注
UART0	仅 hsm 访问	SFT_UART_0_TX	可修改至其他 SFT IO
		SFT_UART_0_RX	
		SFT_UART_0_CTS	
		SFT_UART_0_RTS	
UART1	仅 hsm 访问	-	可通过其他任意 SFT IO 设置得到，参考

			PinAssignment SFT_AltFunc_List, 无固定 Pin
UART2	/dev/ttyAT0	UART2_TX	可配置到 GPIO_20、GPIO_21
		UART2_RX	
UART3	/dev/ttyAT3	UART3_TX	可配置到 GPIO_8、GPIO_9
		UART3_RX	
UART4	/dev/ttyAT4	GPIO_30	Alternate Function 为 UART4_TX、UART4_RX
		GPIO_31	
UART5	/dev/ttyAT5	UART1_TX	
		UART1_RX	
		UART1_CTS	
		UART1_RTS	
UART6	/dev/ttyAT6	UART5_TX	
		UART5_RX	
		GPIO_40	Alternate Function 为 UART5_CTS
		GPIO_41	Alternate Function 为 UART5_RTS

44 PCIe

44.1 简介

44.1.1 软硬件环境

44.1.1.1 软件环境

driver 代码遵循 kernel PCIe 子系统的架构。

44.1.1.2 硬件环境

基于 AC8025 芯片内置 PCIe 模块进行开发。

44.1.2 功能

本章所述的 PCIe 功能模块主要包含：

- 支持 PCIe 1.1/2.1 协议，支持协议规范的电源管理功能；
- 支持 PCIe Gen2/1 1 Lane，Gen2 速率 5GT/s，Gen1 速率 2.5GT/s；
- 支持配置为 RP（Root Port）或者 EP（Endpoint），当前软件主要用于 RP，如果要配置为 EP，则需要定制相关功能；
- 作为 RP 时，支持参考时钟差分输出；
- 作为 RP 时，最大支持访问及配置 256 PCI Functions；
- 作为 RP 时，最大支持 32 个区域 Outbound 地址转换，最大支持 3 个区域 Inbound 地址转换；
- 作为 EP 时，支持 1 PF, 0 VFs PCI Functions；
- 作为 EP 时，支持 4 通道 DMA 传输，包括 Bulk / Scatter / Gather 传输方式；
- 作为 EP 时，最大支持 32 个区域 Outbound 地址转换，最大支持 7 个区域 Inbound 地址转换；
- 作为 EP 时，支持外部时钟输入，也能依靠内部时钟工作；
- 支持 INTx/MSI 中断功能，其中 INTx 包含 INTA/INTB/INTC/INTD；
- 支持协议规定的各种 Error 检测功能。

44.1.3 代码结构及文件描述

表 44-1 PCIe 代码结构及描述

文件名	文件路径	文件描述
all	source/kernel/atc_modules/pcie/rootport	ATC PCIe RP 核心代码
all	source/kernel/atc_modules/pcie/msi_controller	ATC MSI 控制器核心代码
all	source/kernel/atc_modules/pcie/common	硬件相关接口实现代码
all	source/kernel/atc_modules/pcie/include	硬件相关头文件
all	source/kernel/atc_modules/pcie/endpoint	ATC PCIe EP 核心代码，包含 EP 控制器代码及 EP Driver，当作为 EP 时，EP Driver 需要移植到 RP 端的系统环境中编译运行。

44.2 使用说明和注意事项

在适配 PCIe EP Device 的时候，可能会遇到如下问题，可以参考相关解决方法。

44.2.1 EP DMA 支持情况

AC8025 PCIe RP 支持 32/64Bit 寻址，可以修改 rootport/Makefile 选择。

相关宏 CONFIG_PCIE_DEVICE_DMA64，表示支持 64 位地址，否则是 32 位地址。

44.2.1.1 支持 64 位 DMA

增加 64 位支持，可以在 Makefile 中增加如下代码：

```
subdir-ccflags-y += -DCONFIG_PCIE_DEVICE_DMA64
```

44.2.1.2 支持 32 位 DMA

由于 AC8025 的物理地址都是 64 位，所以只支持 32 位地址的 Device 的 Driver 需要修改。

1. 修改 dma mask，改为 64 位地址，如下代码：

```
if (pci_set_dma_mask(pdev, DMA_BIT_MASK(64)) ||
    (pci_set_consistent_dma_mask(pdev, DMA_BIT_MASK(64)) {
    return -EINVAL;
}
```

2. 由于 Device 只支持 32 位地址，所以申请到的 64 位物理地址是不能够直接传递给 Device 的，需要在 Device Driver 端进行地址截短操作：低 32 位地址保留并传递给 Device，高位地址丢弃（高位地址一般是 0x8 等固定值），请注意，此时，该物理地址已经被截短了，要保证 Device Driver 中不再使用该地址进行地址访问。

44.2.2 参考时钟

AC8025 PCIe 作为 RP 时，支持输出内部时钟给到 Device，此种方案可以节省额外的外部时钟，但是也支持使用外部时钟，相关宏 AC8025_PCIE_INTERNAL_CLOCK，表示支持内部时钟输出，否则需要连接外部时钟。

```
subdir-ccflags += -DAC8025_PCIE_INTERNAL_CLOCK
```

45 AtcCamera App

45.1 简介

45.1.1 软硬件环境

- 摄像头
- 高速 SD 卡

45.1.2 功能

- 倒车
- 预览
- 拍照

支持预览、拍照、录像并发操作。

- 普通录像
- 紧急录像

普通录像和紧急录像的功能和区别如下：

- 两者都是从摄像头取数据，存储到本地文件夹；
- 普通录制的情况下，文件存储到 **Normal** 文件夹下，当录制容量不够时，会将最早录制的文件删掉（所谓循环录制），以存储新的文件；
- 紧急录像情况下，文件存储到 **Urgent** 文件夹下，当录制容量不够时，不会将 **Urgent** 文件夹下最早录制的文件删掉。当容量不够时，提示卡满并停止录制。
- 触发紧急录像事件后，APP 将做 3 件事情（一为录制条件下才做，二、三为必做）。
 - 一、如果当前正在录制，则将当前文件存储到 **Urgent** 目录；
 - 二、将此后 30s 内的视频存储到 **Urgent** 文件夹；
 - 三、按照当前时间，去普通录制目录查找紧急录像事件触发前 30s 内的视频文件，当存在此类文件时，将文件存入到 **Urgent** 文件夹。

- 水印

支持分别设置预览、拍照、录像水印。

- 录制时长切换

UI 上提供 3min、5min、和 10min 这三个选项。

- 分辨率切换

UI 上提供摄像头所支持的分辨率列表。

- 麦克风语音录入
 - 开启时，麦克风语音会被录入进视频；
 - 关闭时，麦克风语音不会被录入进视频。

- TF 卡热拔插恢复录制

TF 卡拔出时，记录当前的录制状态，若正在录制则调用相关函数停止录制；TF 卡插入时，根据记录的录制状态，选择性的恢复录制。

- 后台录制

Home 按键切换进入后台继续录制，Back 按键退出录制。

- 多路录制

AtcCamera App 提供的功能 UI 如下所示。

预览录制 UI:

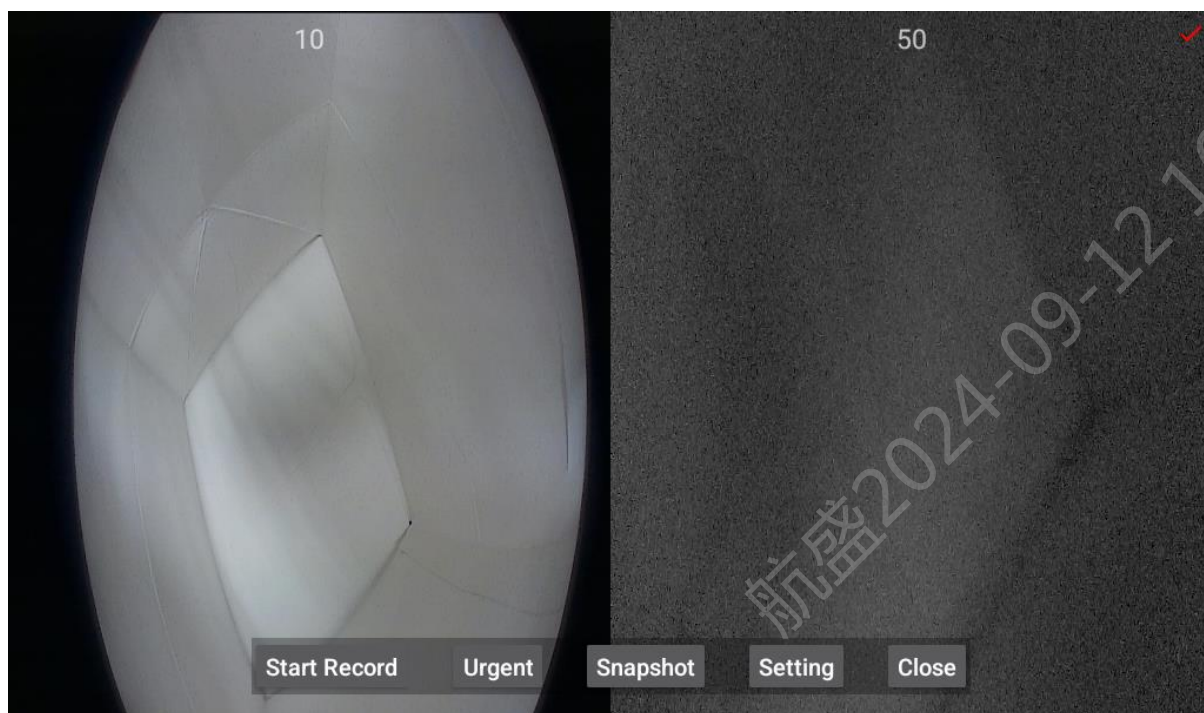


图 45-1 预览和录制 UI

倒车 UI:

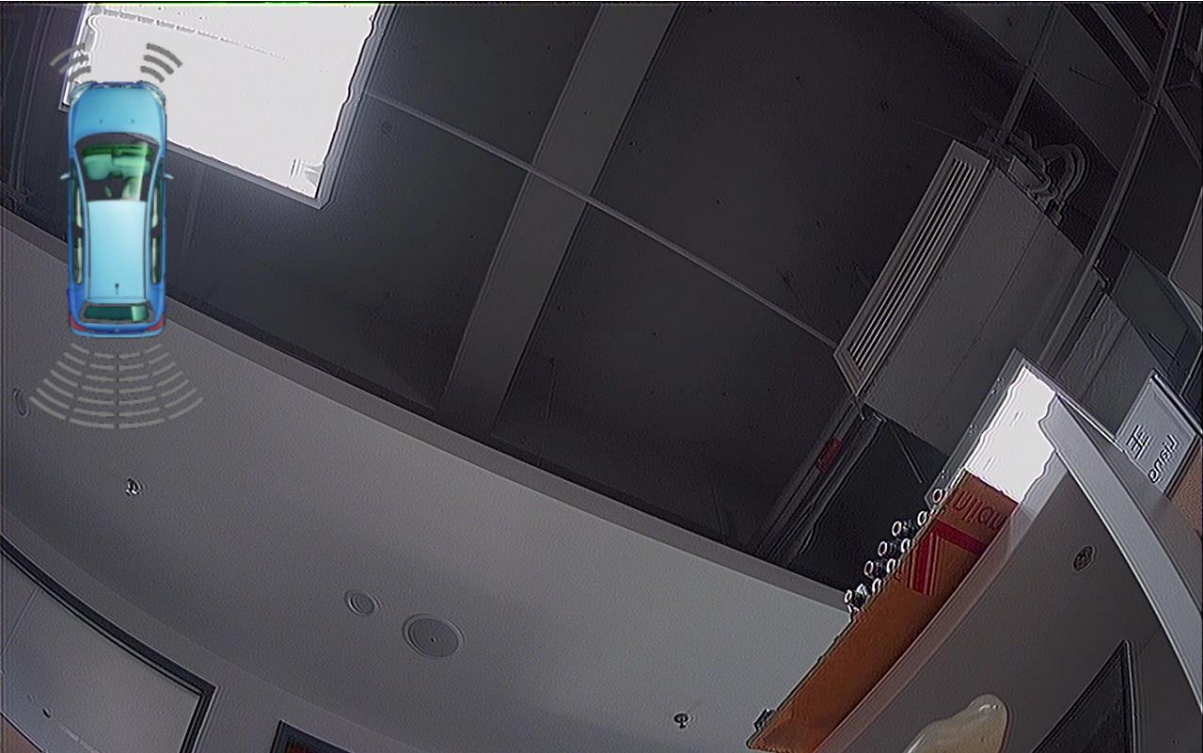


图 45-2 倒车 UI

45.1.3 代码结构及文件描述

表 45-1 AtcCamera APP 代码结构及描述

文件名	文件路径	文件描述
MainActivity.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/activity	用于处理 AtcCamera 的 UI 显示。包括拍照、预览、录制等开关，及录制参数的设置。
CameraItemView.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/view	单个摄像头预览和设置 UI 组件
CameraOperation.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/operator	用于实际控制与 Camera Framework 层的交互，控制、监听 Camera 的行为和反应。
CameraService.java	source/android/vendor/autochips/proprietary/package/apps/AtcCa	后台服务类

文件名	文件路径	文件描述
	mera/src/com/autochips/camera/service	
BootCompletedReceiver.java & HomeReadyReceiver.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/receiver	用于监听开机消息，以启动 CameraService 服务。
CameraServiceImpl.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/service	AIDL 接口 ICameraService 的实现类。
CameraStateMachine.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/operator	Camera 状态管理类，使用子线程操作各个 Camera。
BackcarSurfaceCallback.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera	SurfaceHolder.Callback 接口实现类，用于监听倒车 SurfaceView 状态改变。
EmptyActivity.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/activity	空 Activity，主要用于在倒车起来之后覆盖在其它 Activity 之前，从而触发其它 Activity 的生命周期变化。
IState.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/common	camera 状态接口
State.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/common	camera 状态接口实现类
StateMachine.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/common	状态机实现类
Resolution.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/bean	分辨率实体类
WatermarkInfo.java	source/android/vendor/autochips/proprietary/package/apps/AtcCa	预览水印实体类

文件名	文件路径	文件描述
	mera/src/com/autochips/camera/bean	
LogUtil.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/util	日志工具类
DvrUtil.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/util	获取系统休眠、ARM2 状态、版本等信息
MetazoneUtil.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/util	Metazone 工具类
StorageUtil.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/util	Storage 存储相关工具类
ToastUtil.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/util	提示工具类
SharePreferencesUtil.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/util	SharePreference 工具类
Constants.java	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/src/com/autochips/camera/util	DVR 常量定义
ICameraService.aidl	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/aidl/src/com/autochips/camera	Service 端接口 AIDL 定义
ICameraServiceCallback.aidl	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/aidl/src/com/autochips/camera	Service 回调接口 AIDL 定义
Resolution.aidl	source/android/vendor/autochips/proprietary/package/apps/AtcCa	分辨率实体类 AILD 定义

文件名	文件路径	文件描述
	mera/aidl/src/com/autochips/camera/bean	
WatermarkInfo.aidl	source/android/vendor/autochips/proprietary/package/apps/AtcCamera/aidl/src/com/autochips/camera/bean	水印实体类 AILD 定义

45.2 软件设计

45.2.1 架构图

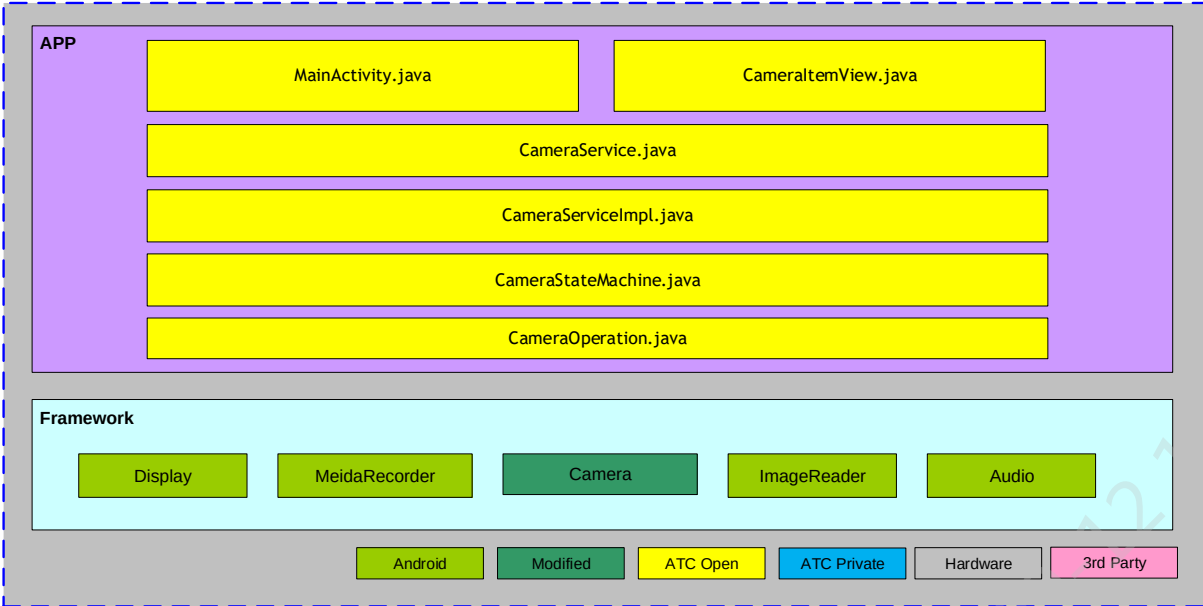


图 45-3 AtcCamera 架构图

上图中各个类实现的功能如下：

1. MainActivity 和 CameraItemView 主要用于 UI 界面显示。
2. CameraService 主要为作为 Client 端的 MIPIDVRActivity 提供具体功能实现，例如预览、录制等。
3. CameraServiceImpl 是 AIDL 的接口实现。
4. CameraStateMachine 用于记录 Camera 的状态，每个 Camera 对应一个 CameraStateMachine，而每个 CameraStateMachine 又是运行在单独的线程中。
5. CameraOperation 为 Camera 的具体操作类，通过调用 Camera 相关接口实现预览和录制。

45.2.2 流程图

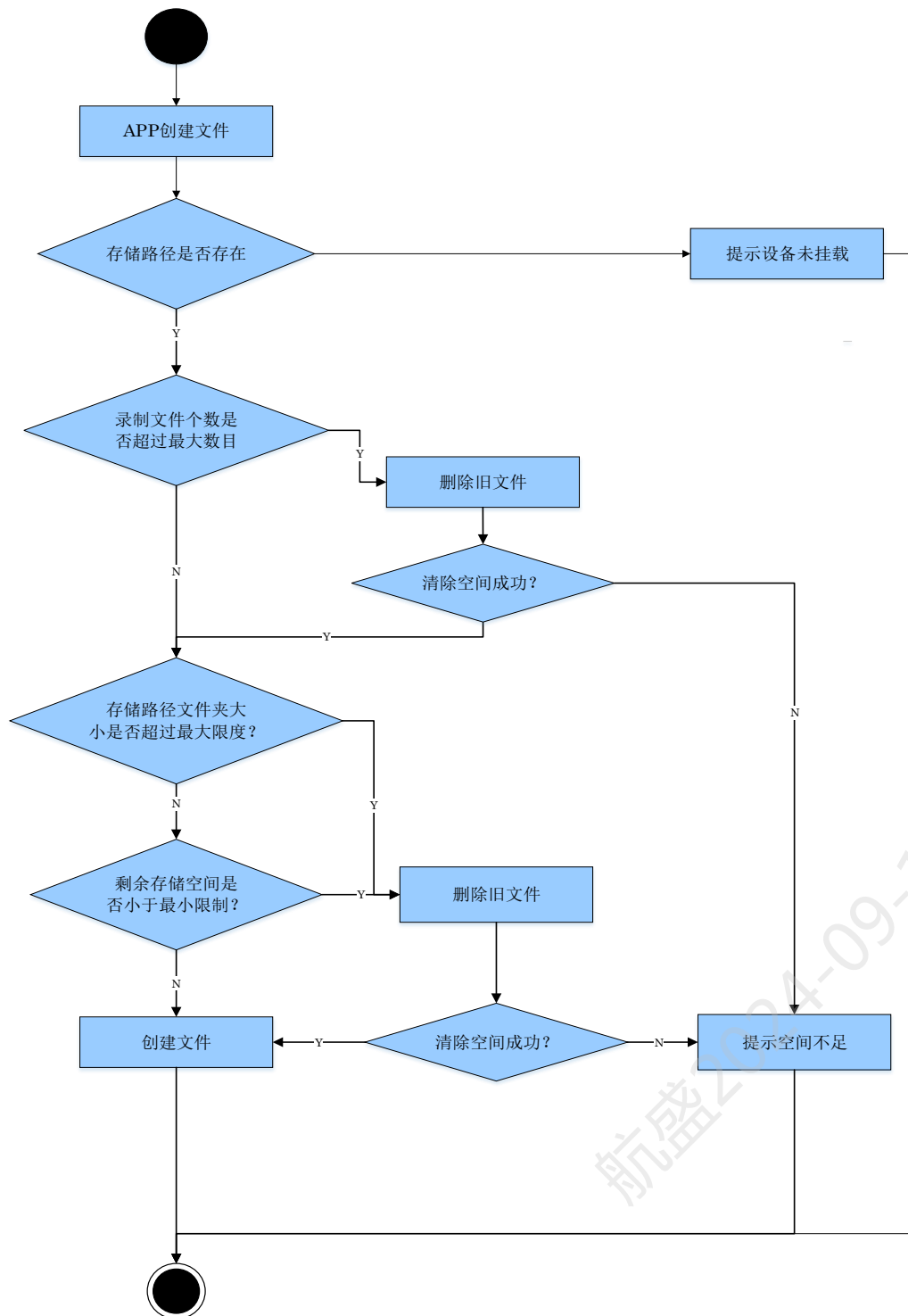


图 45-4 创建录制文件

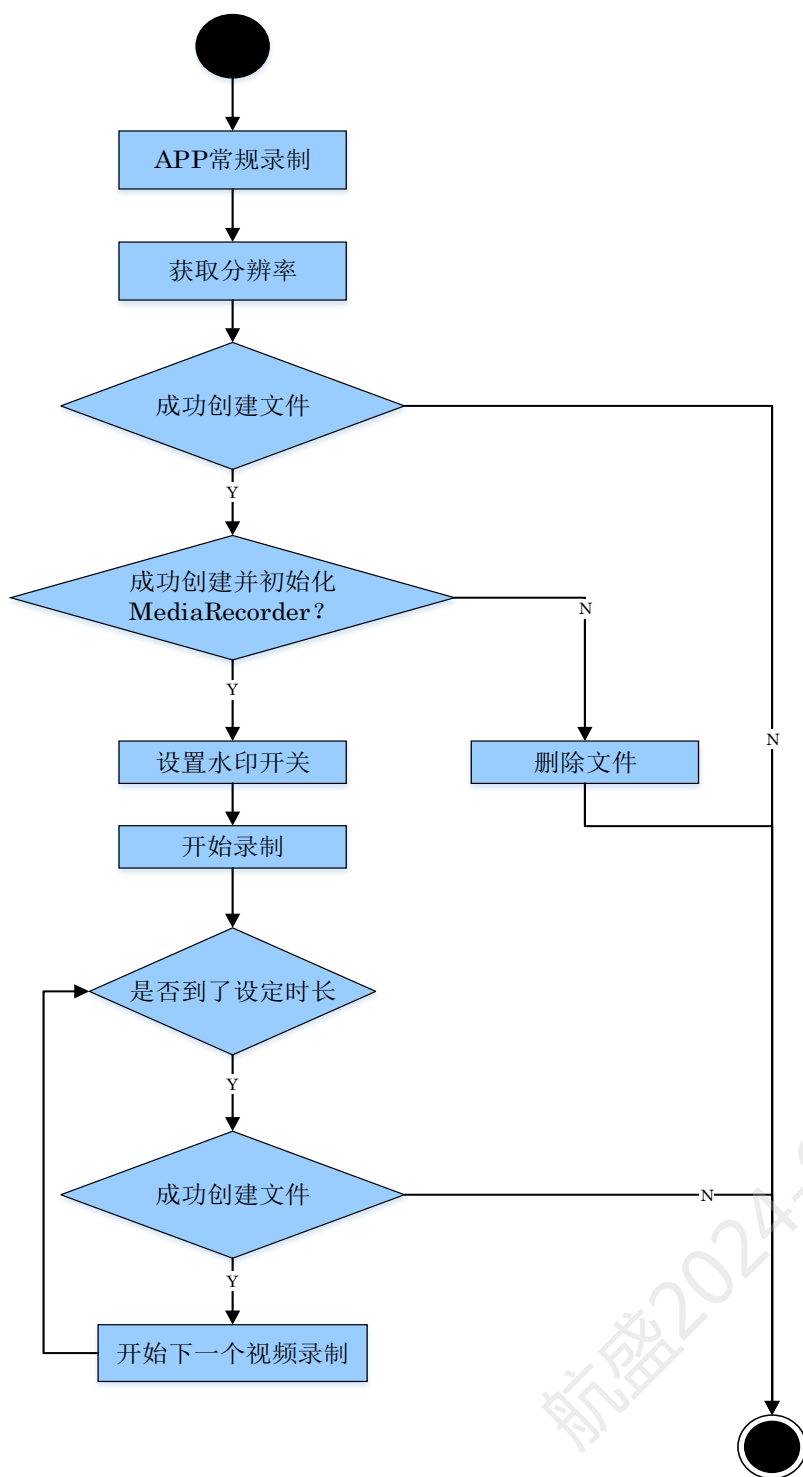


图 45-5 常规录制

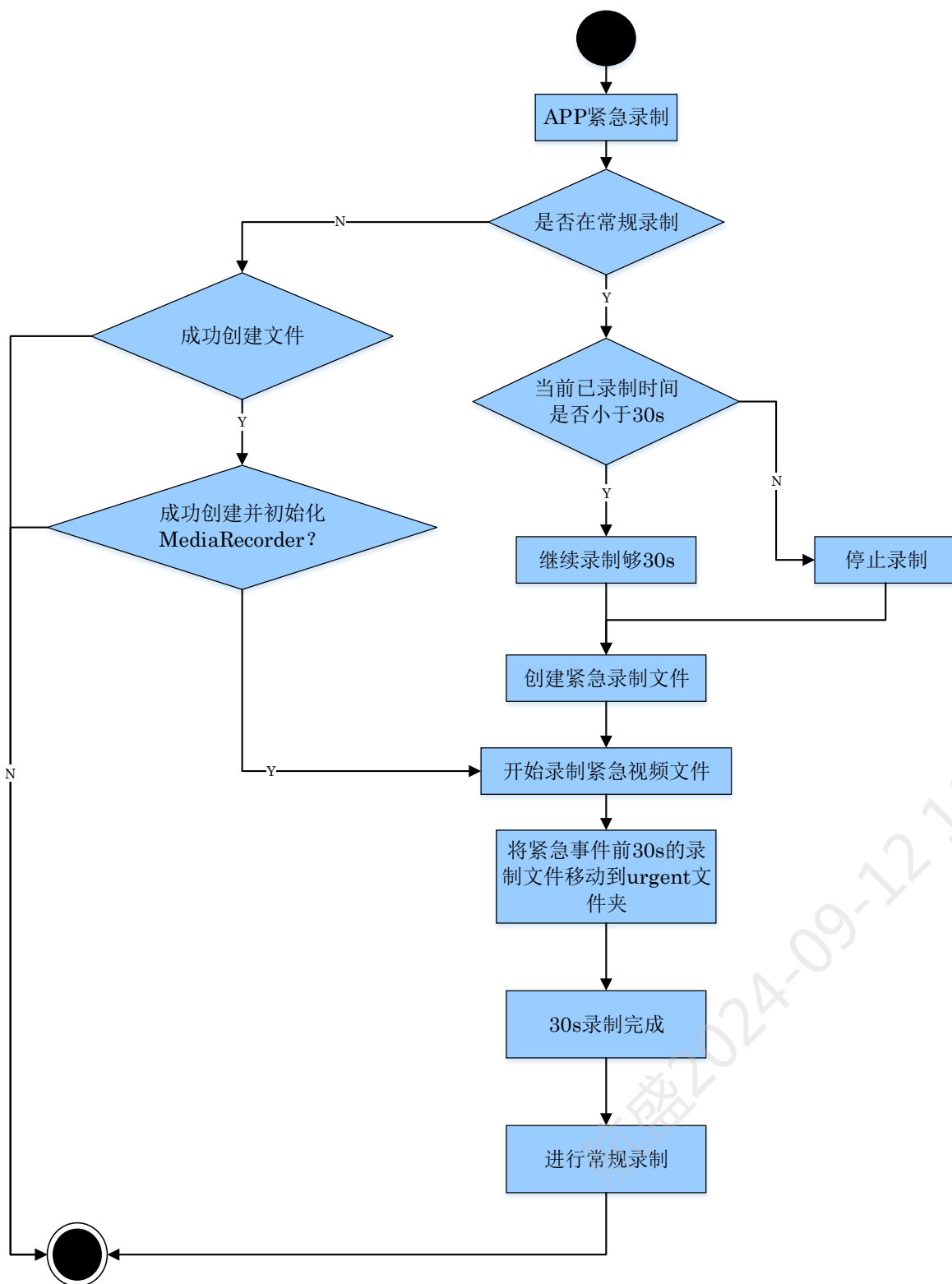


图 45-6 紧急录制

45.3 接口描述

45.3.1 类 MIPIDVRActivity

该类只负责 UI 显示相关内容，Camera 相关功能由 CameraService 提供。在应用刚起来时通过 bindService 获取服务提供的接口，之后便可以通过服务提供的接口获取相关的功能，如预览、录制等。

表 45-2 MIPIDVRActivity 类方法

方法名	说明
initView	初始化所有的 View
loadCameraItemsView	加载 Camera 预览组件画面
initCameraItemData	初始化 Camera 数据
ShowStopRecordConfirmDialog	弹出是否停止录制的对话框
BindCameraService/unbindCameraService	绑定/解绑 CameraService
isCameraResourceAvailable	判断 camera 资源是否可用
updateButtons	更新 UI 上的 Button 按键显示状态
initSettingView	初始化设置界面
updateSettingView	更新设置界面
updateResolutions	更新分辨率
addResolutionRadioButtons	添加分辨率选择按钮
updateWatermark	更新水印信息
handleCameraError	处理错误消息
showSettingView/ hideSettingView	显示/隐藏设置界面
showPreAllocDialog	显示预分配对话框
updateDuration	更新录制时长

45.3.2 类 CameraServiceImpl

跨进程 AIDL 接口 ICameraService 实现类，继承自 ICameraService.Stub。在 CameraService 的 onBind 函数中将该类对象进行返回。

表 45-3 类 CameraServiceImpl 方法说明

方法名	说明
startPreview	开启预览
stopPreview	停止预览
isPreviewStarted	判断当前预览是否开启
startRecord	开启录制

方法名	说明
stopRecord	停止录制
isRecording	判断普通录制是否开启
isUrgentRecording	判断紧急录制是否开启
snapshot	拍照
setMicState	开关 Mic
getMicState	获取 Mic 开关状态
setResolution	设置预览和录制的分辨率
getResolutions	获取摄像头分辨率
getCurrentResolutions	获取当前支持的分辨率
setDuration	设置普通录制时长
getDuration	获取当前普通录制时长
resetSettings	恢复设置参数
setWatermarkState	设置开关水印
getWatermarkState	获取水印开关状态
setWatermarkStartX	设置水印 x 坐标
setWatermarkStartY	设置水印 y 坐标
setWatermarkWidth	设置水印宽度
setWatermarkHeight	设置水印高度
getWatermarkInfo	获取水印信息
setWatermarkInfo	设置水印信息
cameraResourceAvailable	Camera 资源是否可用
setStoragePath	设置存储路径
getCameraState	获取 Camera 状态
openCamera	打开 Camera
closeCamera	关闭 Camera
isCameraClosedManual	Camera 是否手动关闭
registerCameraServiceCallback	注册回调
unregisterCameraServiceCallback	注销回调

45.3.3 类 CameraStateMachine

Camera 状态管理类，共定义 7 个状态，分别为 ClosedState、OpeningState、OpenedState、ClosingState、PreviewingState、RecordingState、PreviewingAndRecordingState。

表 45-4 类 CameraStateMachine 方法说明

方法名	说明
startPreview	向状态机发送 MSG_START_PREVIEW 消息

方法名	说明
stopPreview	向状态机发送 MSG_STOP_PREVIEW 消息
getPreviewState	获取当前预览状态
startRecord	向状态机发送 MSG_START_RECORD/MSG_START_URGENT 消息
stopRecord	向状态机发送 MSG_STOP_RECORD 消息
snapshot	向状态机发送 MSG_SNAPSHOT 消息
setMic	开关 Mic
getMicState	获取 Mic 开关状态
setResolution	设置预览和录制的分辨率
getResolutions	获取摄像头分辨率
setDuration	设置普通录制时长
getDuration	获取当前普通录制时长
setWatermark	设置开关水印
getWatermarkState	获取水印开关状态
setWatermarkStartX	设置水印 x 坐标
setWatermarkStartY	设置水印 y 坐标
setWatermarkWidth	设置水印宽度
setWatermarkHeight	设置水印高度
registerCallback	注册状态机回调监听
unregisterCallback	反注册状态机回调监听

45.3.4 类 CameraOperation

Camera 操作类，所有 camera 相关的操作都由该类提供，如打开 camera，开启预览和录制等。

表 45-5 类 CameraOperation 方法说明

方法名	方法说明
initStates	初始化状态机
handleSetParamMessage	处理参数设置的消息
openCamera	打开 Camera，最多尝试 3 次，每次间隔 200 毫秒
closeCamera	关闭 camera
startPreview	开启预览
stopPreview	关闭预览
startRecord	开启录制
stopRecord	关闭录制
snapshot	拍照
setResolution	设置分辨率

方法名	方法说明
setDuration	设置录制时长
setStoragePath	设置存储路径
setMic	设置麦克风开关
hasPendingPreivewSurface	是否存在 pending 的预览 surface
updatePreviewAndRecord	session 创建完成之后更新预览和录制的状态
setUpCaptureRequestBuilder	设置预览和录制
getMetazoneInfo	从 metazone 获取倒车裁剪等信息
closePreviewSession	关闭 session
restartRecord	重启录制
setWaterMark	设置水印开关
setWatermarkStartX/setWatermarkStartY	设置水印坐标
setWatermarkWidth	设置水印宽度
setWatermarkHeight	设置水印高度
resetSettings	恢复设置参数
setHue	设置 camera 色调
setBrightness	设置 camera 亮度
setSaturation	设置 camera 饱和度
setContrast	设置 camera 对比度
handleQBoff	处理 QB OFF
handleQBon	处理 QB ON
acquireWakeLock	申请 wake lock
releaseWakeLock	释放 wake lock
getPreviewState	获取预览状态
isRecording	是否正在录制
isUrgentRecording	是否正在紧急录制中
getResolution	获取分辨率
getResolutions	获取分辨率列表
getMicState	获取麦克风状态
setWatermarkSize	设置水印高度和宽度
getWatermarkState	获取水印状态
getDuration	获取时长
getCameraState	获取 Camera 状态
isCameraClosedManual	是否手动关闭

45.3.5 类 StorageUtil

外置存储工具类，主要提供 SD 卡的文件创建、删除等操作。

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

1157 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

表 45-6 类 StorageUtil 方法说明

方法名	方法说明
setStoragePath	设置存储路径
setNormalPolicy	设置普通录制文件夹下的限制规则
setUrgentPolicy	设置紧急录制文件夹下的限制规则
setImagePolicy	设置拍照文件夹下的限制规则
getSpaceReusePolicy	获取相应文件夹下的限制规则
reuseStorageSpace	复用预分配的空间
deleteInvalidFile	删除无效文件
isExist	判断存储卡是否挂载
estimateVideoFileSize	预估视频文件大小
formatPreallocVolume	格式化预分配空间
getPreAllocFile	获取预分配文件
getOutputFile	获取录制或拍照存储文件
createFile	创建文件
updateFileWithMediaScanner	通知 MediaScanner 扫描更新某个文件
deleteOldFiles	删除最早创建的文件
moveUrgentFileFromNormal	将文件从 normal 文件夹下移动到 urgent 文件夹下

45.3.6 类 DVRConst

该类定义所有 camera id 及操作相关的信息常量和错误常量。

45.4 使用说明和注意事项

45.4.1 水印开关

开启水印的方法如下。

1. 预览、录制、拍照水印所对应的标签名称

```
private static final String ATC_WATERMARK_PREVIEW_KEY_NAME =
"com.atc.watermark.info.preview";

private static final String ATC_WATERMARK_RECORD_KEY_NAME =
"com.atc.watermark.info.record";

private static final String ATC_WATERMARK_SNAPSHOT_KEY_NAME =
"com.atc.watermark.info.snapshot";
```

2. 查询 camera 是否支持水印属性的方法，遍历 camera 支持的 request key， 如果包含某个 request key 的 name 与标签名称一致，则说明对应 camera 支持水印功能。

```
private CaptureRequest.Key<int[]> mWatermarkPreviewKey ;
private CaptureRequest.Key<int[]> mWatermarkRecordKey;
private CaptureRequest.Key<int[]> mWatermarkSnapshotKey;
List<CaptureRequest.Key<?>> requestKeyList =
cameraCharacteristics.getAvailableCaptureRequestKeys();
for (CaptureRequest.Key<?> key : requestKeyList ) {
    if (ATC_WATERMARK_PREVIEW_KEY_NAME.equals(key.getName())) {
        mWatermarkPreviewKey = (CaptureRequest.Key<int[]>)key;
    } else if (ATC_WATERMARK_RECORD_KEY_NAME.equals(key.getName())) {
        mWatermarkRecordKey = (CaptureRequest.Key<int[]>)key;
    } else if (ATC_WATERMARK_SNAPSHOT_KEY_NAME.equals(key.getName())) {
        mWatermarkSnapshotKey = (CaptureRequest.Key<int[]>)key;
    }
}
```

3. 开启水印

```
if (mWatermarkPreviewKey != null) {
mCaptureRequestBuilder.set(mWatermarkPreviewKey,new int[]{OnOff,x,y,w,h});
}
if (mWatermarkRecordKey != null) {
    mCaptureRequestBuilder.set(mWatermarkRecordKey,new int[]{OnOff,x,y,w,h});
}
if (mWatermarkSnapshotKey != null) {
    mCaptureRequestBuilder.set(mWatermarkSnapshotKey,new int[]{OnOff,x,y,w,h});
}
```

【参数】

OnOff ： 是否开启水印，1 为开启，0 为关闭

X ： 水印的 X 坐标值

Y ： 水印的 Y 坐标值

w ： 水印的宽度

h : 水印的高度

45.4.2 AtcCamera 所使用的摄像头

目前公版上支持 MIPI1、MIPI2 和 BT656 三种类型的摄像头。

45.4.3 倒车事件的监听

注册倒车事件监听

```
private void registerRvcEvent() {  
    if (mIsRvcMode) {  
        mCarProperty = new CarProperty();  
        ArrayList<Integer> evts = new ArrayList<>(1);  
        evts.add(VehiclePropertyIds.CURRENT_GEAR);  
        mCarProperty.registerCallback(mEventCallback, evts);  
    }  
}
```

回调接口监听

```
private CarProperty.CarPropertyEventCallback mEventCallback = new  
CarProperty.CarPropertyEventCallback() {  
    @Override  
    public void onChangeEvent(CarPropertyValue value) {  
        if (value == null) {  
            LogUtil.d(TAG, "invalid car event");  
            return;  
        }  
        if (VehiclePropertyIds.CURRENT_GEAR == value.getPropertyId()) {  
            Integer[] gearValue = new Integer[1];  
            gearValue = (Integer[]) value.getValue();  
            LogUtil.d(TAG, "onChangeEvent - event:" +  
VehiclePropertyIds.toString(value.getPropertyId()) + ", value = " +  
VehicleGear.toString(gearValue[0]));  
            if (VehicleGear.GEAR_REVERSE == gearValue[0]) {
```

```
        mHandler.post(() -> {
            LogUtil.i(TAG, "Car in reverse gear!");
            handleNativeRvcStarted();

        });
    } else {
        mHandler.post(() -> {
            LogUtil.i(TAG, "Car without reverse gear!");
            handleNativeRvcStopped();

        });
    }
}

};}
```

45.4.4 Metazone 在倒车中的使用

倒车的类型、镜像、视频裁剪等参数都是通过 metazone 来保存，CarSettings 设置好后，会保存到 metazone 对应区域中。在每次收到倒车信号打开倒车 camera 之前都会读取 metazone 相应地址中的值来进行相应的参数设置，例如，读取倒车类型的 metazone 中对应地址的值可以得知开启的是 AVM 类型的倒车还是普通类型的倒车，如果是 AVM 类型倒车则忽略该信号，由 AVM App 进行响应，否则响应普通倒车。

其中 metazone 定义的 index 有：

表 45-7 倒车中用到的 metazone 地址

DWORD Item Index	Comments	Value
METAZONE_BACKCAR_VDO_SOURCE	倒车类型	(由高到低顺序) 第一个字节：倒车类型 0: 普通倒车 1: R5F 倒车 2: AVM倒车
METAZONE_RVC_AVM_CAMERA_ID	倒车摄像头ID	取值：-1 ~ 14
METAZONE_BACKCAR_VDO_MIRROR	倒车镜像	0: 无镜像 1: 左右镜像 2: 上下镜像 3: 左右+上下镜像

METAZONE_BACKCAR_DISP_RECT_X	倒车区域x值	屏幕左上角坐标
METAZONE_BACKCAR_DISP_RECT_Y	倒车区域y值	屏幕左上角坐标
METAZONE_BACKCAR_DISP_RECT_W	倒车区域w值	最大为屏幕宽度
METAZONE_BACKCAR_DISP_RECT_H	倒车区域h值	最大为屏幕高度
METAZONE_BACKCAR_CROP_RECT_X	视频裁剪x值	左边裁剪掉的像素值
METAZONE_BACKCAR_CROP_RECT_Y	视频裁剪y值	上方裁剪掉的像素值
METAZONE_BACKCAR_CROP_RECT_W	视频裁剪w值	裁剪后视频区域的宽度值
METAZONE_BACKCAR_CROP_RECT_H	视频裁剪h值	裁剪后视频区域的高度值
METAZONE_BACKCAR_COLOR_PROCESS_HUE	倒车色度	取值范围: -128 ~ 127
METAZONE_BACKCAR_COLOR_PROCESS_BRIGHTNESS	倒车亮度	取值范围: -128 ~ 127
METAZONE_BACKCAR_COLOR_PROCESS_SATURATION	倒车饱和度	取值范围: 0 ~ 255
METAZONE_BACKCAR_COLOR_PROCESS_CONTRAST	倒车对比度	取值范围: 0 ~ 255

45.4.5 倒车镜像

倒车镜像的设置方法如下。

```
private static final int MIR_ADDR =
MetazoneIndex.DwordIndex.METAZONE_BACKCAR_DVO_MIRROR;

int[] result = new int[4];

AtcMetazone.readval(MIR_ADDR, result);//获取倒车镜像值
```

将 result[0] 设置给 CameraDevice（mirror 功能对应的 request key name 为“com.atc.mirror”）。

```
mPreviewBuilder.set(mMirrorKey, new int[]{result[0], 0, 0, 0});
```

45.4.6 倒车视频源的裁剪

倒车裁剪范围设置方法如下：

```
private static final int X_CUT_ADDR =
MetazoneIndex.DwordIndex.METZZONE_BACKCAR_CROP_RECT_X;

private static final int Y_CUT_ADDR =
MetazoneIndex.DwordIndex.METZZONE_BACKCAR_CROP_RECT_Y;
```

```
private static final int W_CUT_ADDR =  
MetazoneIndex.DwordIndex.METZZONE_BACKCAR_CROP_RECT_W;  
  
private static final int H_CUT_ADDR =  
MetazoneIndex.DwordIndex.METZZONE_BACKCAR_CROP_RECT_H;  
  
int[] tmp = new int[4]; //临时变量  
AtcMetazone.readval(X_CUT_ADDR, tmp);  
int x = tmp[0];  
AtcMetazone.readval(Y_CUT_ADDR, tmp);  
int y = tmp[0];  
AtcMetazone.readval(W_CUT_ADDR, tmp);  
int w = tmp[0];  
AtcMetazone.readval(H_CUT_ADDR, tmp);  
int h = tmp[0];  
mCamRect = new Rect(x, y, (x + w), (y + h));
```

将 mCamRect 设置给 CameraDevice（裁剪范围 request key 为 CaptureRequest.SCALER_CROP_REGION）。

```
mPreviewBuilder.set(CaptureRequest.SCALER_CROP_REGION, mCamRect);
```

45.4.7 同步刷新紧急视频到外置存储中

通常使用 MediaRecorder 录制视频将视频数据写到外置存储中时会先将数据写到缓存中，如果出现意外拔卡或掉电的情况则缓存中的数据不能刷新到外置存储中。为防止紧急视频数据在这种情况下出现丢失，特在写卡逻辑中加入同步刷新功能（即将数据直接写到外置存储而不经缓存）。通过设置属性“persist.sys.media.recorder.setsync.storage”为 1 将开启同步刷新功能，否则不设置或设置为值为 0 将关闭该功能。默认情况下该功能是关闭的，只建议在录制紧急视频时开启该功能。需要注意的是在设置该属性时需要将应用 UID 设置为 system，即在 AndroidManifest.xml 中将 android:sharedUserId 设置为“android.uid.system”，否则将无权限设置该属性。

```
SytemProperties.set("persist.sys.media.recorder.setsync.storage",  
String.valueOf(1));
```

45.5 用户定制化

45.5.1 录制时长设置

普通录制分为三种录制时长，分别为 3、5 和 10 分钟，默认时长为 5 分钟。紧急录制默认时长为 30 秒。默认时长设定在 `CameraOperation.java` 中，客户可进行自定义。而对于录制时长的控制是通过在开启录制之后设置一个相应长度的延时消息，当延时消息到来时即可结束当前文件的录制并开启下一个文件的录制。

```
mRecorder.start();
mRecordThreadHandler.postDelayed(new Runnable(){
    mRecorder.setNextOutputFile(newFile);
    mRecordThreadHandler.postDelayed(this, mVideoDuration);
},mVideoDuration);
```

45.5.2 循环录制策略设置

考虑到外置存储空间有限，且有些早期录制的视频文件无保存意义，特删除早期录制文件以进行循环录制，对普通录制、紧急录制及拍照各采取了不同的删除策略。

- 普通录制：设定最大普通录制文件总大小不超过 8GB，最小可用空间不低于 50MB，且最大文件数量不超过 150 个，当以上 3 个条件有一个不符合时将停止录制。在录制的过程中如果检查到可用空间低于 100MB，则会循环删除最早录制的文件直到可用空间大于 100MB 且最大文件数量不超过 150 个。
- 紧急录制：设定最大紧急录制文件总大小不超过 2GB，且最小可用空间不低于 50MB，当以上 2 个条件有一个不符合时将停止录制。紧急录制在录制过程中不进行可用空间检查，只会在开启紧急录制前进行检查，如果可用空间低于 50MB，则删除最早录制的普通录制文件直到可用空间大于 50MB，如果最终可用空间低于 50MB 则提示空间不足而不开启紧急录制。
- 拍照：只要可用空间不低于 2MB 就可以进行拍照。

对于以上策略可以在 `StorageUtil` 中的 `setNormalPolicy`、`setUrgentPolicy` 和 `setImagePolicy` 进行自定义设置。定义一个继承自 `StorageUtil.SpaceReusePolicy` 接口的类，重写 `enoughSpaceAfterReuse` 和 `ensureFileNumWithinLimits` 这两个方法。

45.5.3 打开 Camera id 设置

`AtcCamera` 支持通过配置文件配置所要打开的 camera id，配置文件的路径是 `system/etc/atccamera_ids.xml`，`AtcCamera` 会打开配置文件中所定义 id 的 camera，若需要打开其他 id 的 camera，可通过修改配置，添加 id 值的方式来实现。配置文件格式如下：


```
<?xml version="1.0" encoding="utf-8"?>
<CameraIds>
    <id>10</id>
    <id>50</id>
</CameraIds>
```

45.6 典型问题

45.6.1 写外置存储卡权限

Android 上即使申请了“android.permission.WRITE_EXTERNAL_STORAGE”和“android.permission.WRITE_MEDIA_STORAGE”权限也无法对/storage/下的 SD 卡或 U 盘挂载点进行写操作，但/mnt/media_rw 下的相应挂载点还是可以进行写操作。所以在关于读写外置存储卡时，一般需要通过/storage/下的挂载点进行读操作，而通过/mnt/media_rw 下的挂载点进行写操作。Android S 上可以在/storage/下的挂载点进行读写操作。

46 Cluster App

46.1 简介

46.1.1 功能

AC8025 Cluster App 作为 Cluster 端仪表 App，提供如下功能：

- Gauge 转速、车速、油量、水温数据显示
- Telltale 仪表指示灯显示
- 行车档位信息显示
- 音乐播放、通话信息显示
- 车模和车道显示
- 支持行车电脑菜单设置，包括正确显示里程信息，空调设置，TPMS 等信息。
- 支持仪表转场动画
- 支持全屏导航模式

Cluster App 和 IVI 通信支持：

- 支持 IVI 向仪表传输文本信息(音乐和通话信息)
- 支持 IVI 向仪表传输图片信息(传送联系人头像、音乐专辑图片信息)
- 支持 IVI 向仪表传输视频信息(IVI 指定 Activity 界面投射在 Cluster 端显示)

46.1.2 代码结构及文件描述

表 46-1 Cluster 代码结构及描述

文件名	文件路径	文件描述
ClusterCtrlManager.java	source/vendor/autochips/proprietary/frameworks/base/ clusterctrl/java/com/autochips/ clusterctrl/	发送 IVI 端信息给 Cluster 功能接口
IAtcCluster.hal	source/vendor/autochips/proprietary/hardware/interfaces/ atcccluster/1.0/	发送音 乐，蓝牙 电话投屏 信息的

文件名	文件路径	文件描述
		hidl 接口
atcccluster.cpp、 atcccluster.h	source/vendor/autochips/proprietary/hardware/atcccluster/	发送音 乐，蓝牙 电话投屏 信息的 hidl 服务 实现
idc_server.h	source/vendor/autochips/proprietary/hardware/libidc/include/	server 端 监听 idc 事件回调 功能接口

46.2 软件设计

46.2.1 架构图

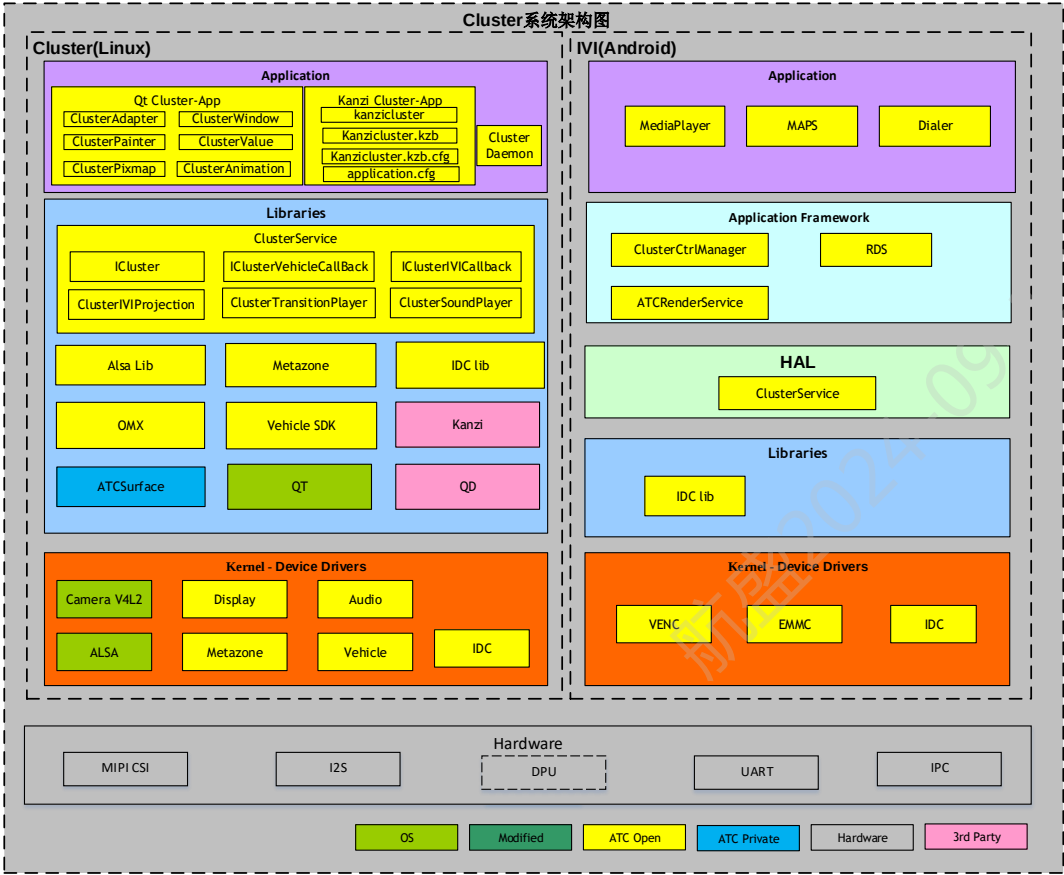


图 46-1 Cluster App 架构图

Cluster 可以接受以下 Android 传递过来的数据。

1. Message 数据: Android 多媒体, 蓝牙电话可以通过调用 `AtcCluster hidl` 接口传输图片, 文本信息, 底层通过 IDC 机制跨系统将信息打包传输到 Cluster 端处理, 显示;
2. 投屏数据: Android 的投屏应用 `maps` 可以将显示 Buffer 通过 RDS 模块调用 IDC 模块跨系统传输 Android 端 buffers 到 Cluster 端显示。

Cluster App 由 Qt 或者 kanzi 开发的 App 界面, 在 Cluster 系统上 C++ 开发的 `ClusterService` 动态链接库, 在 Android 端 HAL 层的 `ClusterService` 和 Framework 层的 `ClusterServiceManager` 组成。

Cluster Daemon 为守护进程管理 Cluster App 生命周期, Cluster App 异常挂掉后可重启。

1. 基于 Qt QPainter 开发的仪表 App UI 界面, 使用 Qt OpenGL 进行仪表纹理绘制及 3D 车模渲染。
 - `ClusterAdapter`: `ICluster` 接口适配类, 实现 `IClusterVehicleCallback` 和 `IClusterIVICallback`, 并注册回调。
 - `ClusterWindow`: 继承 `QOpenGLWindow` 和 `QOpenGLFunctions` 类, 基于 Qt OpenGL 渲染仪表。
 - `ClusterPainter`: 使用 `QPainter` 绘制仪表 UI
 - `ClusterPixmap`: 使用 `QPixmap` 加载仪表图片资源文件, 管理仪表资源。
 - `ClusterValue`: 仪表数据管理单例类, 包括车速、转速、油耗、文件、指示灯状态等。
 - `ClusterAnimation`: 仪表动画管理类, 为演示仪表动画提供模拟数据。
2. 基于 Kanzi Studio 开发仪表 App UI 界面, 使用 Kanzi 引擎进行仪表和 3D 车模的渲染。
 - `application.cfg`: kanzi 配置文件, 例如: 指定 `DisplayId`、界面显示 GPU FPS 等 debug 信息等。
 - `KanziCluster.kzb.cfg`: 指定资源文件名称。
 - `KanziCluster.kzb`: 使用 Kanzi Studio 导出的 Cluster App 运行资源文件, 由 Kanzi 引擎加载并解析动态运行。
 - `KanziCluster`: 基于 AC8025 交叉编译的 Kanzi 引擎执行文件, `KanziCluster` 会加载解析 `KanziCluster.kzb` 即显示 Cluster App 仪表界面。

46.2.2 流程图

46.2.2.1 IVI 信息显示流程

KanziCluster 显示 IVI 端音乐和通话信息流程:

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

1168 / 1254

本文档包含信息归杰发科技所有, 未经许可, 严禁转载或披露

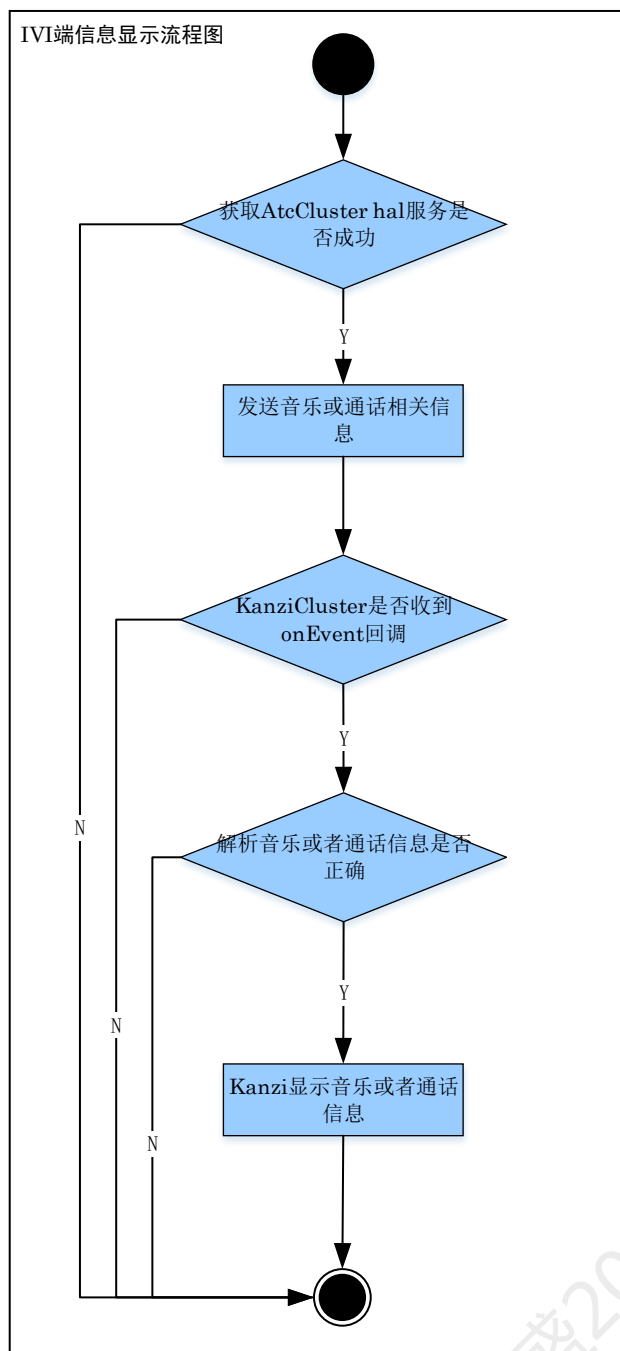


图 46-2 IVI 信息显示流程图

46.2.2.2 Cluster 转场动画流程

Cluster 在不同主题切换时，增加了转场动画功能，在主题切换期间，全屏播放一段视频画面，作为主题切换的过渡。Cluster 转场动画流程如下：

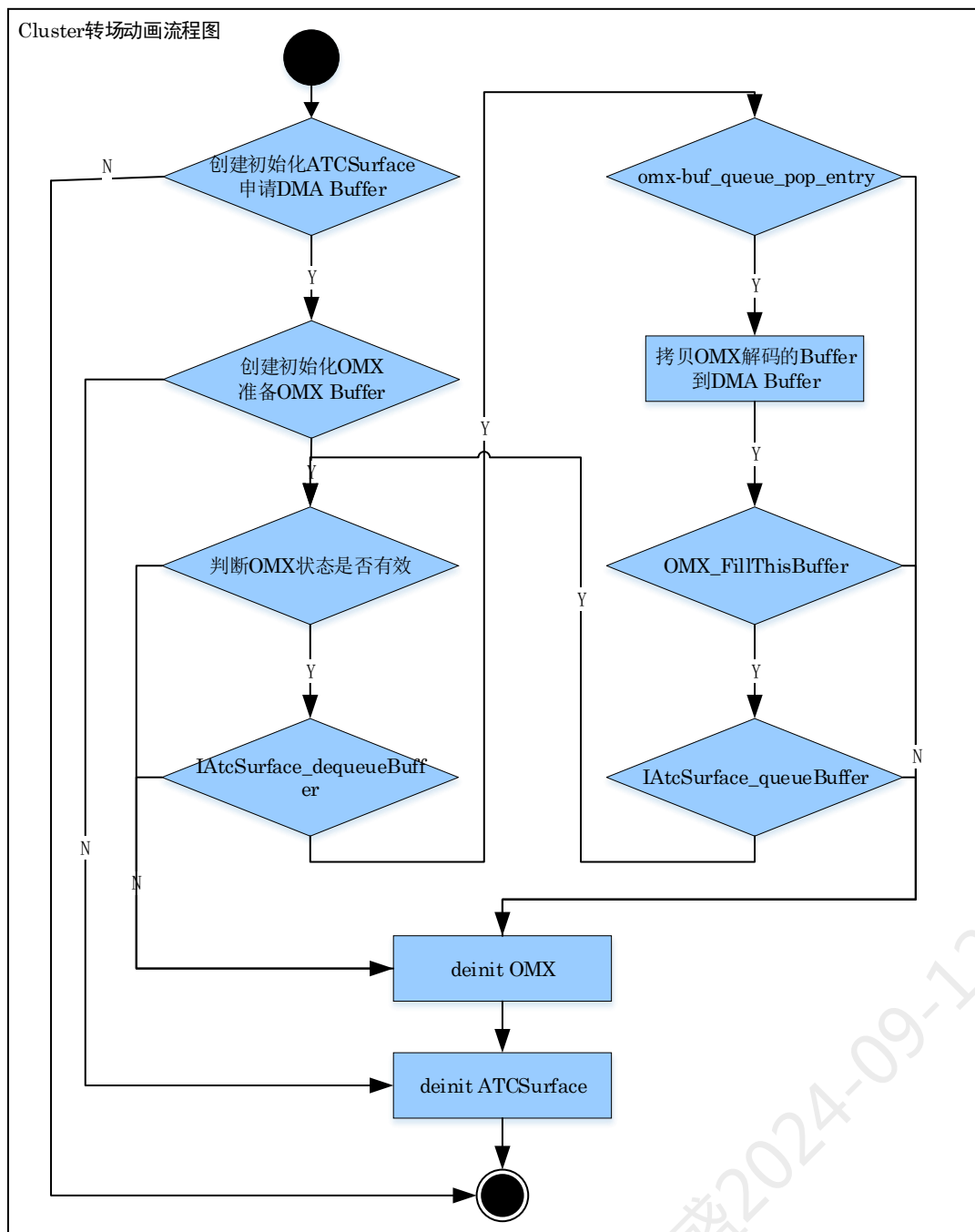


图 46-3 Cluster 转场动画流程图

46.2.2.3 Cluster 接受 IVI 信息流程

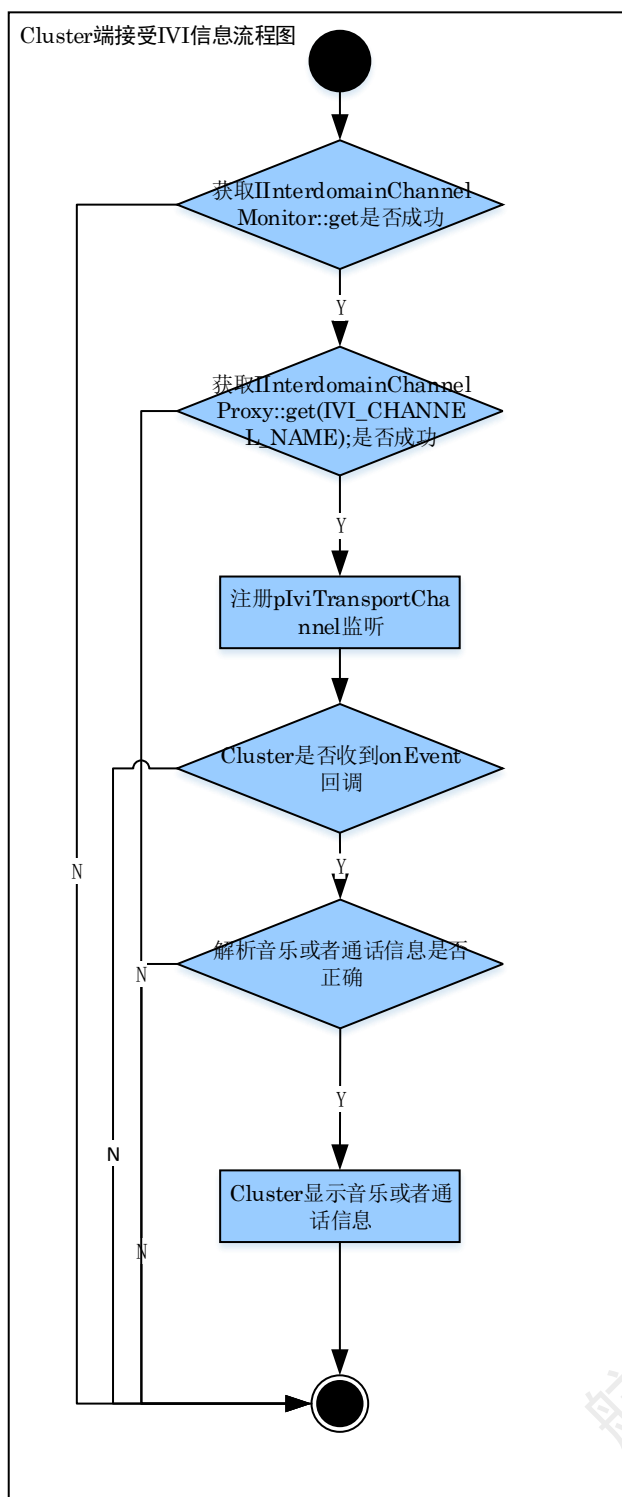


图 46-4 Cluster 端接受 IVI 信息流程图

46.2.2.4 Cluster 接受 IVI 投屏信息流程

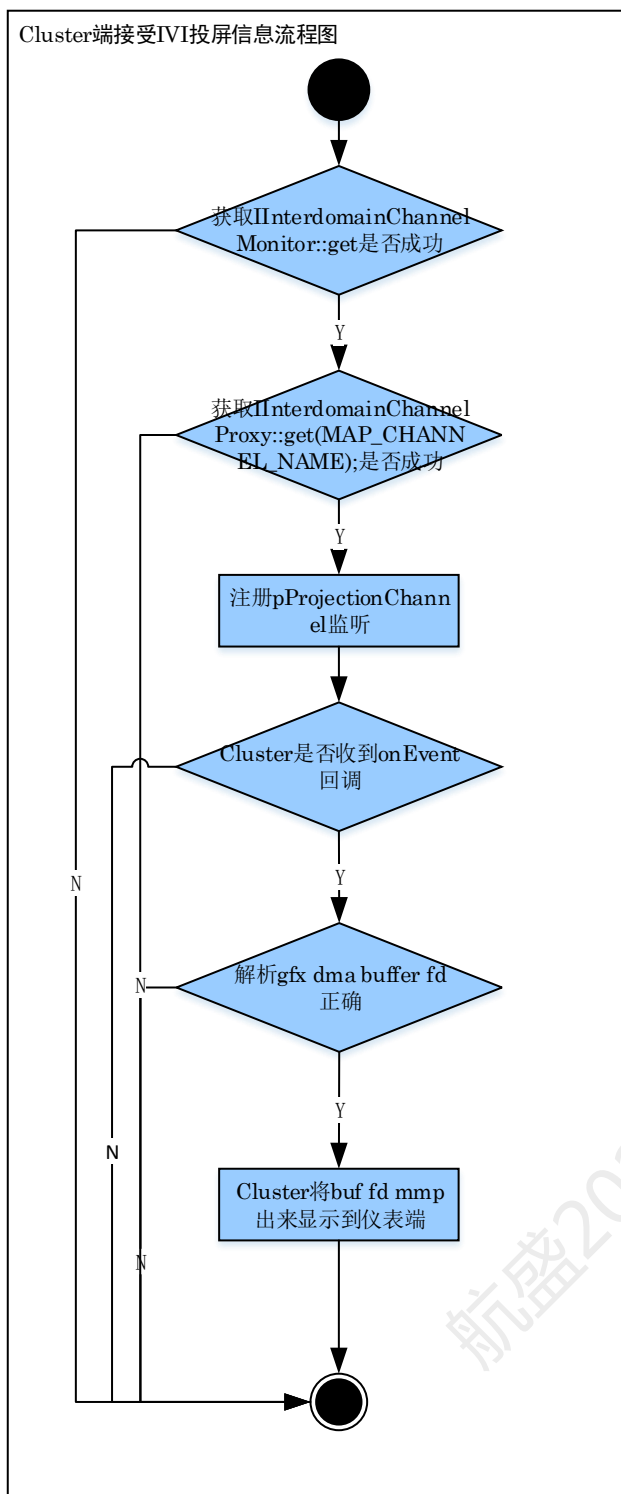


图 46-5 Cluster 端接受 IVI 投屏信息流程图

46.3 接口描述

46.3.1 ClusterCtrlManager

ClusterCtrlManager 在 Android 系统提供给 Android App 的接口，该接口文件打包在 Autochips.jar 包下。

表 46-2 ClusterCtrlManager API

API	说明
sendPlaybackState	发送歌曲播放状态
sendPhoneNum	发送电话号码
sendContactName	发送联系人名称
sendContactImage	发送联系人头像
sendPhoneTime	发送通话时间
sendPhoneState	发送通话状态
sendMediaMetaData	发送歌曲信息，包括歌曲 id，歌曲名、歌手名
sendRouteGuidanceInformation	发送导航信息

46.3.1.1 sendMediaState

【描述】

Send media information to cluster.

【语法】

```
void sendMediaState (int clientId , int state);
```

【参数】

- clientId:客户端信息
- state: Playing state

【返回值】

None

46.3.1.2 sendMediaMetaData

【描述】

发送歌曲信息，包括歌曲 id，歌曲名、歌手名。

【语法】

```
void sendMediaMetaData (int clientId, Const AtcClusterMediaMetadata &data);
```

【参数】

- clientId: Id of client
- data: Bitmap of the Album and song of name singer info

【返回值】

None

46.3.1.3 sendPhoneNum

【描述】

Send phone number to cluster.

【语法】

```
void sendPhoneNum (String phoneNum);
```

【参数】

phoneNum: phone number

【返回值】

None

46.3.1.4 sendContactName

【描述】

Send contact name to cluster.

【语法】

```
void sendContactName (String name);
```

【参数】

name: contact name

【返回值】

None

46.3.1.5 sendContactImage

【描述】

Send contact Image to cluster.

【语法】

```
void sendContactImage (Bitmap bitmap);
```

【参数】

bitmap: contact image

【返回值】

None

46.3.1.6 sendPhoneTime

【描述】

Send duration of call to cluster.

【语法】

```
void sendPhoneTime (String time);
```

【参数】

time: duration of call

【返回值】

None

46.3.1.7 sendPhoneState

【描述】

Send state of call to cluster.

【语法】

```
void sendPhoneState (int state);
```

【参数】

state: state of call

【返回值】

None

46.3.1.8 sendRouteGuidanceInformation

【描述】

发送道路导航信息。

【语法】

```
void sendRouteGuidanceInformation (int clientId, const hidl_vec<RouteGuidanceInformation>
&routeGuidanceInformationList);
```

【参数】

- clientId: 客户端 id
- routeGuidanceInformationList: 道路导航信息集合

【返回值】

- Int 0: success
- Int 非 0 : fail

46.3.2 IInterdomainChannelProxy

表 46-3 IInterdomainChannelProxy API

API	说明
addListener	注册事件通知回调函数
removeListener	移除事件通知回调函数
getName	获取 Inter-Domain Channel 对象的名字

46.3.2.1 addListener

【描述】

注册事件通知回调函数。

【语法】

```
void addListener(IIDCProxyListener *listener)
```

【参数】

listener [IN] : 用于接受的 event 事件，其中包含 idc_event, reply 等数据。

【返回值】

无。

46.3.2.2 removeListener

【描述】

移除事件通知回调函数。

【语法】

```
void removeListener(IIDCProxyListener *listener)
```

【参数】

listener [IN] : 用于接受的 event 事件，其中包含 idc_event, reply 等数据。

【返回值】

无。

46.3.2.3 getName**【描述】**

获取 Inter-Domain Channel 对象的名字。

【语法】

```
void getName(char*name)
```

【参数】

name[OUT] : 用于接受 Inter-Domain Channel 对象的名字。

【返回值】

无。

46.3.2.4 onEvent**【描述】**

Idc 发送给 server 端 event 事件，服务端需要实现此方法。

【语法】

```
onEvent(IInterdomainChannelProxy *proxy,idc_event_t *event)
```

【参数】

- proxy [int] : IInterdomainChannelProxy 指针对象
- event[int]: idc_event_t event 事件

【返回值】

无。

46.3.3 数据结构

表 46-4 idc 数据结构

数据结构	说明
idc_gfx_buffer	跨系统传输的graphicbuffer结构体
idc_buffer	跨系统传输的buffer结构体
idc_event	跨系统传输的msg 原始的字节流 结构体

```
typedef struct idc_gfx_buffer {
    uint32_t width;
    uint32_t height;
    uint32_t stride;
    uint32_t format;
    int fd;
    int fence;

    uint32_t priv[8];
} idc_gfx_buffer_t;

struct idc_buffer {
    int fd;
    int size;
    int fence;
};

struct idc_event {
    uint32_t id;
    uint64_t param1;
    uint64_t param2;
};

#define IDC_CLUSTER_DOMAIN_NAME "cluster" //cluster 域
#define IDC_ADAS_DOMAIN_NAME "adas" //adas 域
#define IDC_IVI_DOMAIN_NAME "ivi" //ivi 域

Event 事件类型
```

```
#define IDC_EVENT_CONNECTED 0
#define IDC_EVENT_DISCONNECTED 1
#define IDC_EVENT_RAW_DATA 2
#define IDC_EVENT_MESSAGE 3
#define IDC_EVENT_DMA_BUFFER 4
#define IDC_EVENT_GFX_DMA_BUFFER 5
#define IDC_EVENT_EVENT_PARCEL 6
```

46.4 使用说明和注意事项

46.4.1 Kanzi app 使用问题

- Cluster App 运行在 Cluster 系统，基于 Kanzi 实现，通过 application.cfg 中的 DefaultDisplayIndex 属性来配置显示到哪个屏。
- 8025 仪表分辨率为 1920x720，因此副屏需要连接分辨率为 1920x720 的屏才能正常显示。
- Kanzi UI 界面更新只能在 onupdate 的主线程去做，不然更新 ui 不起作用

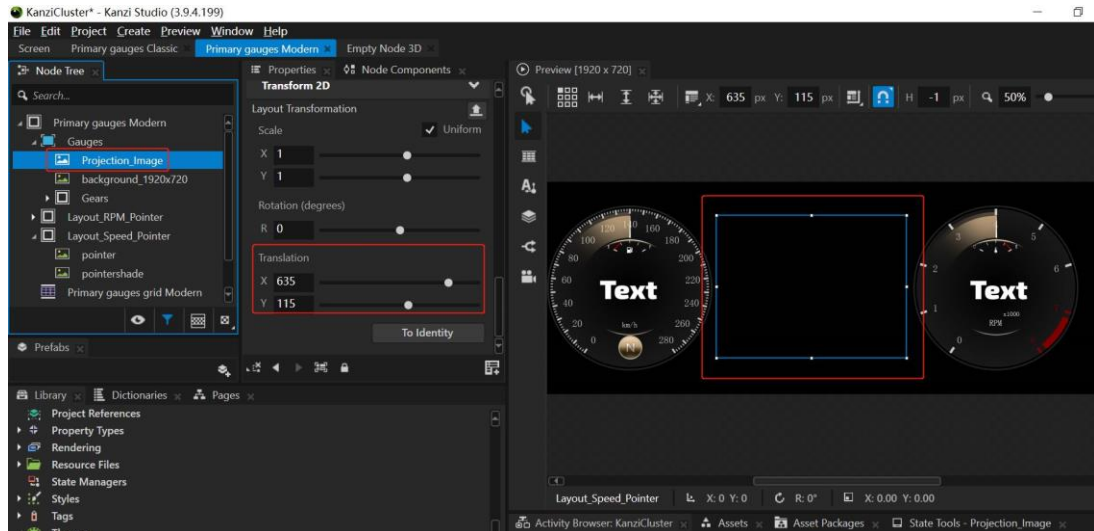
Cluster app 接收到 android 发过来的 buffer 数据的坐标系是和 opengl 的是反着的，需要做一次逆向 copy 或者使用 BitmapImage::createFromMemoryFlipped，创建 bitmapimage 的时候垂直翻转一下，不然画面显示会出现反着的效果。

46.4.2 投屏功能

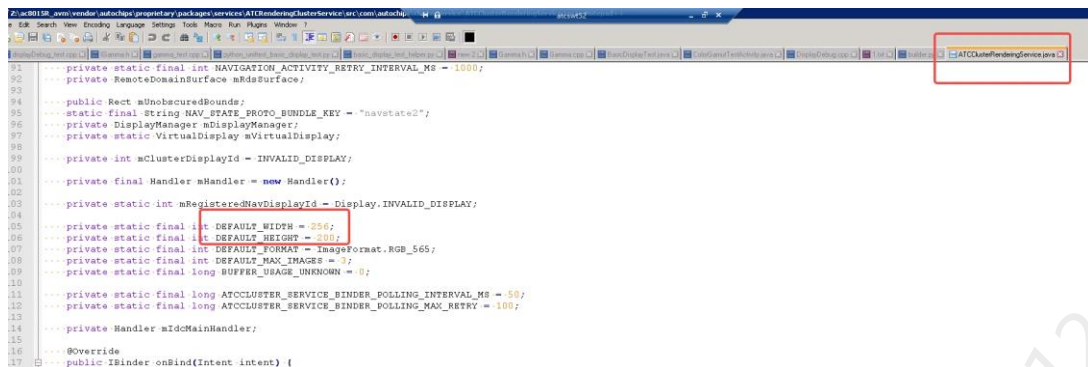
1. 设置投屏显示区域：

- KanziCluster 投屏是显示在 projection_image 组件中，因此可以改变此 Image 组件的大小和位置来设置投屏显示的位置。如下图所示，Kanzi 相关介绍参考：

<https://docs.kanzi.com/3.9.4/en/working-with/deploying-applications/deploying-kanzi-applications-for-qnx.html>



- 同时还要改变 android 中 Activity 投屏数据源的大小与投屏 Image 组件相对应。数投屏数据大小设置在 ATCClusterRenderingService.java 中，如下图所示：



2. Android 中 Activity 投屏如何配置：

需要投屏的 activity 要在对应的 AndroidManifest.xml 中

定义以下 category `<category android:name="android.car.cluster.NAVIGATION" />`

添加 `android:resizeableActivity="true"`和

`android:permission="android.car.permission.CAR_INSTRUMENT_CLUSTER_CONTROL"`标签。

例如：


```
<activity
    android:name=".FakeClusterNavigationActivity"
    android:allowEmbedded="true"
    android:launchMode="singleInstance"
    android:permission="android.car.permission.CAR_INSTRUMENT_CLUSTER_CONTROL"
    android:resizeableActivity="true"
    android:theme="@android:style/Theme.NoTitleBar.Fullscreen">
    <intent-filter android:priority="-1">
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.car.cluster.NAVIGATION" />
    </intent-filter>
</activity>
```

46.4.3 QT/QD /kancluster app 集成 idc 功能步骤

- 引入 libidc 的库，导入头文件<idc_server.h>

```
1 #include <stdio.h>
2
3 #include <stdio.h>
4 #include <idc_server.h>
5
6 #include <unistd.h>
7 #include <sys/mman.h>
```

```
CPPFLAGS += -std=c++11
LOCAL_SHARE_LIBRARIES += -lidc -lstdc++ -lpthread -lsurface_atc -lCommonAPI -lCommonAPI-DBus -ldbus-1
```

- 根据不同的 channel，获取 IInterdomainChannelProxy 对象注册 evnet 监听回调，目前有如下 3 种 channel：

MAP_CHANNEL_NAME "map.projection": 监听投屏数据

IVI_CHANNEL_NAME "cluster.transport": 监听 ivi 发送过来的 message 数据包

ADAS_CHANNEL_NAME "idc.dms_server_cluster": adas 域发过来的 dms 信息

```

{
    KANZILOGD(TAG, "start()");
    pProjectionCallback = callback;
    IInterdomainChannelMonitor *idc_monitor = NULL;

    idc_monitor = IInterdomainChannelMonitor::get();
    if (!idc_monitor) {
        KANZILOGE(TAG, "ChannelMonitor::get() fail");
        return;
    }

    pProjectionChannel = IInterdomainChannelProxy::get(MAP_CHANNEL_NAME);
    if (!pProjectionChannel) {
        KANZILOGE(TAG, "Failed to get map.projection channel proxy!\n");
        return;
    }
    pIviTransportChannel = IInterdomainChannelProxy::get(IVI_CHANNEL_NAME);
    if (!pIviTransportChannel) {
        KANZILOGE(TAG, "Failed to get cluster.transport channel proxy!\n");
        return;
    }
    pAdasTransportChannel = IInterdomainChannelProxy::get(ADAS_CHANNEL_NAME);
    if (!pAdasTransportChannel) {
        KANZILOGE(TAG, "Failed to get idc.dms_server_cluster channel proxy!\n");
        return;
    }
    pProjectionChannel->addListener(this);
    pIviTransportChannel->addListener(this);
    pAdasTransportChannel->addListener(this);
}

```

6. 实现回调方法 onEvent，并根据不同的 channelName，和 message 类型来解析不同的数据 data，比如：

IDC_EVENT_MESSAGE 传输过来的一般是当前播放的音乐，专辑图片，蓝牙电话的状态，联系人头像

IDC_EVENT_GFX_DMA_BUFFER，android 投屏的数据流。

IDC_EVENT_MESSAGE:解析出来文本信息, 图像信息

```
} else if (IDC_EVENT_MESSAGE == event->id) {
    char channelName[64] = {0};
    proxy->getName(channelName);
    KANZILOGI(TAG, " received idc message event, channelName:%s\n", channelName);
    if (0 == strcmp(channelName, ADAS_CHANNEL_NAME)) {
        KANZILOGI(TAG, " ADAS_CHANNEL_NAME\n");
        if (pProjectionCallback)
            pProjectionCallback->onDmsRespond();
    } else if (0 == strcmp(channelName, IVI_CHANNEL_NAME)) {
        KANZILOGI(TAG, " IVI_CHANNEL_NAME\n");
        IDCMessage *message = (IDCMessage*)event->param2;
        KANZILOGI(TAG, " message->m_msg.what:%d\n", message->m_msg.what);
        switch (message->m_msg.what) {
            case RECEIVE_PLAYBACK_STATE: {
                int state = message->getIntExtra(ARGV1);

                if (state == MEDIA_STATE_PAUSED || state == MEDIA_STATE_STOPPED) {
                    pProjectionCallback->onPlayStateChanged(state);
                }
            }
            break;
            case RECEIVE_MEDIA_METADATA: {
                std::string mediaName = message->getStringExtra(ARGV2);
                int action = message->getIntExtra(ARGV4);
                unsigned int dataLength = 0;
                const unsigned char* data = message->getArrayExtra(ARGV5, &dataLength);
                KANZILOGD(TAG, "handleAsyncMessage, RECEIVE_MEDIA_METADATA dataLength:%d", dataLength);

                pProjectionCallback->onCurrentSongNameChanged(mediaName);
                pProjectionCallback->onCurrentAlbumPixmapChanged(data, dataLength);
                pProjectionCallback->onPlayStateChanged(MEDIA_STATE_PLAYING);

                if (action == MEDIA_ACTION_NEXT) {
                    pProjectionCallback->onPlayActionChanged(0);
                } else if (action == MEDIA_ACTION_PREVIOUS) {
                    pProjectionCallback->onPlayActionChanged(1);
                }
            }
            break;
            case RECEIVE_ALBUM_IMAGE: {
                std::string mediaId = message->getStringExtra(ARGV1);
                unsigned int dataLength = 0;
                const unsigned char* data = message->getArrayExtra(ARGV2, &dataLength);
            }
        }
    }
}
```

IDC_EVENT_GFX_DMA_BUFFER:解析出来 fd, 通过 fd, mmap 出来 buffer 指针地址, 来显示。

```
} else if (IDC_EVENT_GFX_DMA_BUFFER == event->id) {
    idc_gfx_buffer_t *gfx_buf = (idc_gfx_buffer_t *)event->param1;
    KANZILOGI(TAG, " %s -> gfx buffer event, fd: %d, width: %u, height: %u\n",
        __func__, (int)gfx_buf->fd, gfx_buf->width, gfx_buf->height);
    KANZILOGI(TAG, "DRM_FORMAT_NV12:%d, DRM_FORMAT_RGBA8888:%d, DRM_FORMAT_RGB888:%d", DRM_FORMAT_NV12, DRM_FORMAT_RGBA8888, DRM_FORMAT_RGB888);
    int size = gfx_buf->width * gfx_buf->height * ARG8888CHANNELS;
    if (DRM_FORMAT_NV12 == gfx_buf->format) {
        size = gfx_buf->width * gfx_buf->height * 1.5;
        KANZILOGI(TAG, "DRM_FORMAT_NV12");
    }
    KANZILOGI(TAG, "gfx_buf->format:%d, size:%d", gfx_buf->format, size);
    if (pProjectionCallback && size == PROJECTION_WIDTH * PROJECTION_HEIGHT * ARG8888CHANNELS) {
        unsigned char *data_buf = (unsigned char *)mmap(NULL, size, PROT_READ,
            MAP_SHARED, gfx_buf->fd, 0);
        KANZILOGI(TAG, " %s -> data buf: %p\n", __func__, data_buf);
        pProjectionCallback->onProjectionPixmapChanged(data_buf, gfx_buf->width, gfx_buf->height, gfx_buf->format);
        munmap(data_buf, size);
    }
}
```

7. 停止 idc 监听的时候需要 remove 掉对应的监听。

```
void IviProjectionPlayer::stop()
{
    KANZILOGI(TAG, "stop()");
    pProjectionChannel->removeListener(this);
    pIviTransportChannel->removeListener(this);
    pAdasTransportChannel->removeListener(this);
}
```

46.5 典型问题

kanzicluster：如果 ui 画面有 3d 小车的话，如果放在整个 cfg 资源文件加载的话，会导致仪表第一帧的出图时间变慢，一般考虑到仪表启动性能的话，建议将 3d 模型进行异步加载，然后再在主线程里面加到整个 ui 里面去，这样出图更快。

47 Control Panel UI

47.1 简介

47.1.1 软硬件环境

47.1.1.1 软件环境

运行于 AC8025 平台 cockpit 产品的 Adas 域，即搭载了 QT5.11（对应 QtQuick 2.11）的 Linux 系统。音量及亮度调节功能依赖 Audio、Display 提供 audiovolumeclient、light 库。

47.1.1.2 硬件环境

插入控制屏并正确配置 dpim 的 AC8025 平台（配置方法见 [47.4 使用说明和注意事项](#)）。

47.1.2 功能

控制屏界面左侧菜单栏提供了 4 个页面入口，分别是暖通空调、音量、座椅角度、亮度。

- 暖通空调：即 HVAC，空调、通风、暖气相关设置，仅实现了 UI 效果，未实现功能；
- 音量：调用 audiovolumeclient 库接口调节当前 IVI 系统中正在播放音频的音量；
- 座椅角度：控制左右座椅靠背的倾斜度，仅实现了 UI 效果，未实现功能；
- 亮度：调用 light 库接口设置控制屏自身的亮度，设置多久进入空闲状态，空闲状态会将亮度设置为最低值，用户触摸屏幕任意位置可退出空闲状态，即屏幕亮度恢复到正常值。

47.1.3 代码结构及文件描述

表 47-1 ControlPanelUI 代码结构及描述

文件名	文件路径	文件描述
ControlPanelUI.pro	source/vendor/autochips/proprietary/packages/ControlPanelUI/	编译配置文件
control_panel_ui.sh	source/vendor/autochips/proprietary/packages/ControlPanelUI/	进程自启动控制脚本
qml.qrc	source/vendor/autochips/proprietary/packages/ControlPanelUI/	资源文件列表
main.cpp	source/vendor/autochips/proprietary/packages/ControlPanelUI/src	程序入口

文件名	文件路径	文件描述
brightness_controller.cpp volume_controller.cpp	source/vendor/autochips/proprietary/packages/ ControlPanelUI/src	包装了对 light 和 audiovolumeclient 库 接口，供 qml 调用
main.qml	source/vendor/autochips/proprietary/packages/ ControlPanelUI/qml	定义主界面页面布局； 实现触摸时退出空闲状态
page_xxx.qml	source/vendor/autochips/proprietary/packages/ ControlPanelUI/qml	定义了各功能页面，参 考 1.2 章节功能描述
SeatSettings	source/vendor/autochips/proprietary/packages/ ControlPanelUI/qml	点击 page_hvac.qml 中 的座位按钮可进入此座 位冷/暖气设置页面
AtcXxx.qml MyXxx.qml	source/vendor/autochips/proprietary/packages/ ControlPanelUI/qml	自定义控件
xxx.ttf/.pfa/.pfb/.qpf	source/vendor/autochips/proprietary/packages/ ControlPanelUI/fonts	系统字库，在编译时被 拷贝到 /usr/lib/fonts 目 录
control_panel_ui.mk	source/vendor/autochips/proprietary/buildroot- configs/configs/package/control_panel_ui	buildroot 编译配置文件
ControlPanelUI	adas/out/build/ac8025-adas/control_panel_ui- 1.0/ & adas/out/target/ac8025-adas/usr/bin/	编译生成的可执行程序 (拉取代码后编译完成 才可见)

47.2 软件设计

47.2.1 架构图

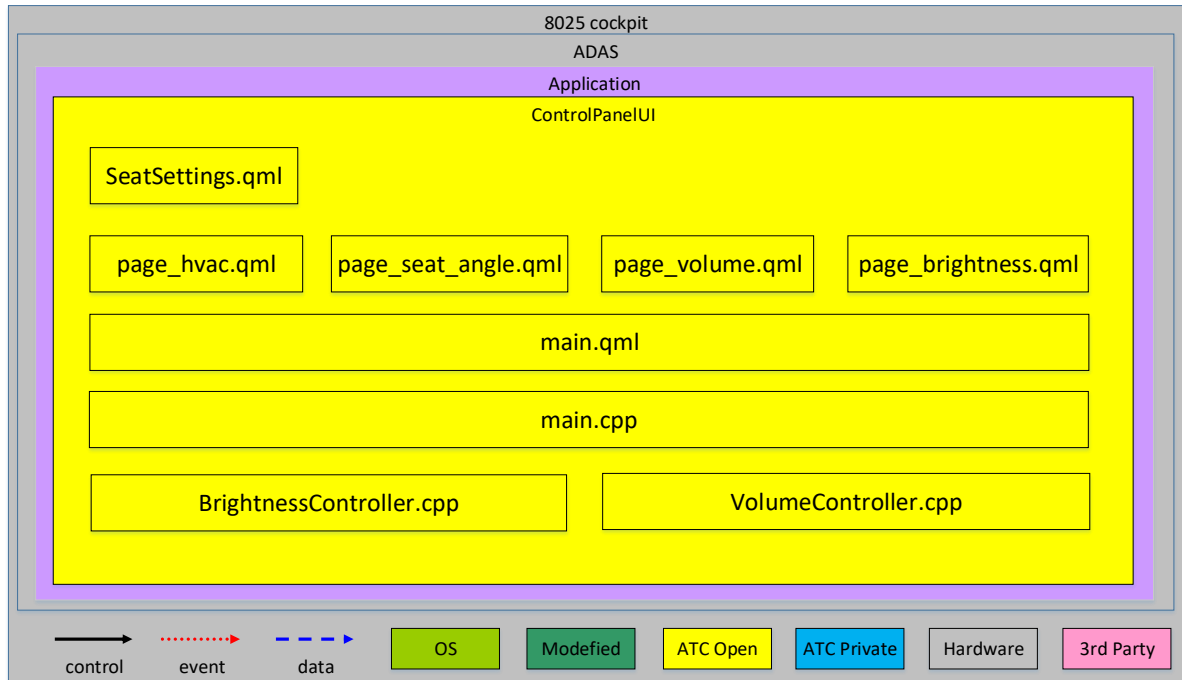


图 47-1 ControlPanelUI 架构图

47.2.2 流程图

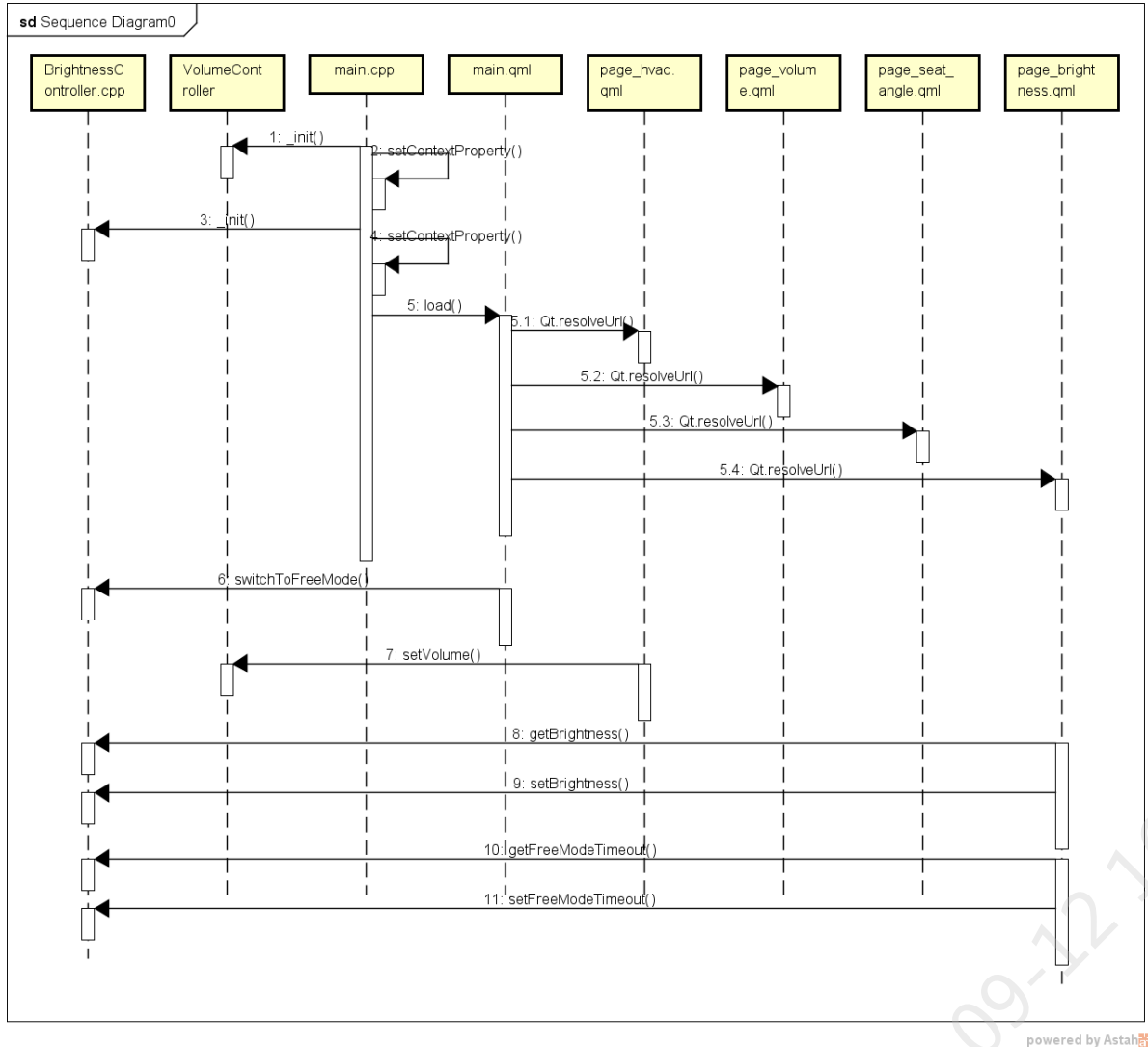


图 47-2 流程图

47.3 接口描述

本模块属于 demo App 程序，不对外提供接口，仅作为车内控制屏 App 的一种参考实现。

这里简单介绍此 demo 程序中 qml 调用 cpp 接口的方法。

1. 创建 cpp 类实例，将实例地址设置为 QT property（main.cpp）


```
QQmlApplicationEngine engine;  
  
VolumeController volumeCtrl;  
engine.rootContext()->setContextProperty("volumeCtrl", &volumeCtrl);
```

2. 在 qml 文件中直接使用此实例 (page_volume.qml)

```
MouseArea {  
    anchors.fill: parent  
    onPressed: {  
        parent.color = "gray"  
        volumeCtrl.setVolume(false)  
        console.log(tag, "volume- icon pressed")  
    }  
}
```

47.4 使用说明和注意事项

在 cockpit 产品形态下，如需使用控制屏，请配置带控制屏信息的 DPIM，且连接控制屏。

48 HUD App

48.1 简介

48.1.1 功能

AC8025 HUD App，提供如下功能：

- Gauge 车速数据显示
- Telltale 指示灯（如转向灯、安全带、燃油不足、车门未关）显示
- 导航信息显示

48.1.2 代码结构及文件描述

表 48-1 HUD 代码结构及描述

文件名	文件路径	文件描述
HudServiceManager.java	source/vendor/autochips/proprietary/frameworks/base/atchud/java/com/autochips/hud/	IVI 端发送导航信息给 HUD 功能接口
IAtcHud.hal	source/vendor/autochips/proprietary/hardware/interfaces/atchud/1.0/	HUD hidl 接口
atchud.cpp、atchud.h	source/vendor/autochips/proprietary/hardware/atchud/	HUD hidl 服务实现

48.2 软件设计

48.2.1 架构图

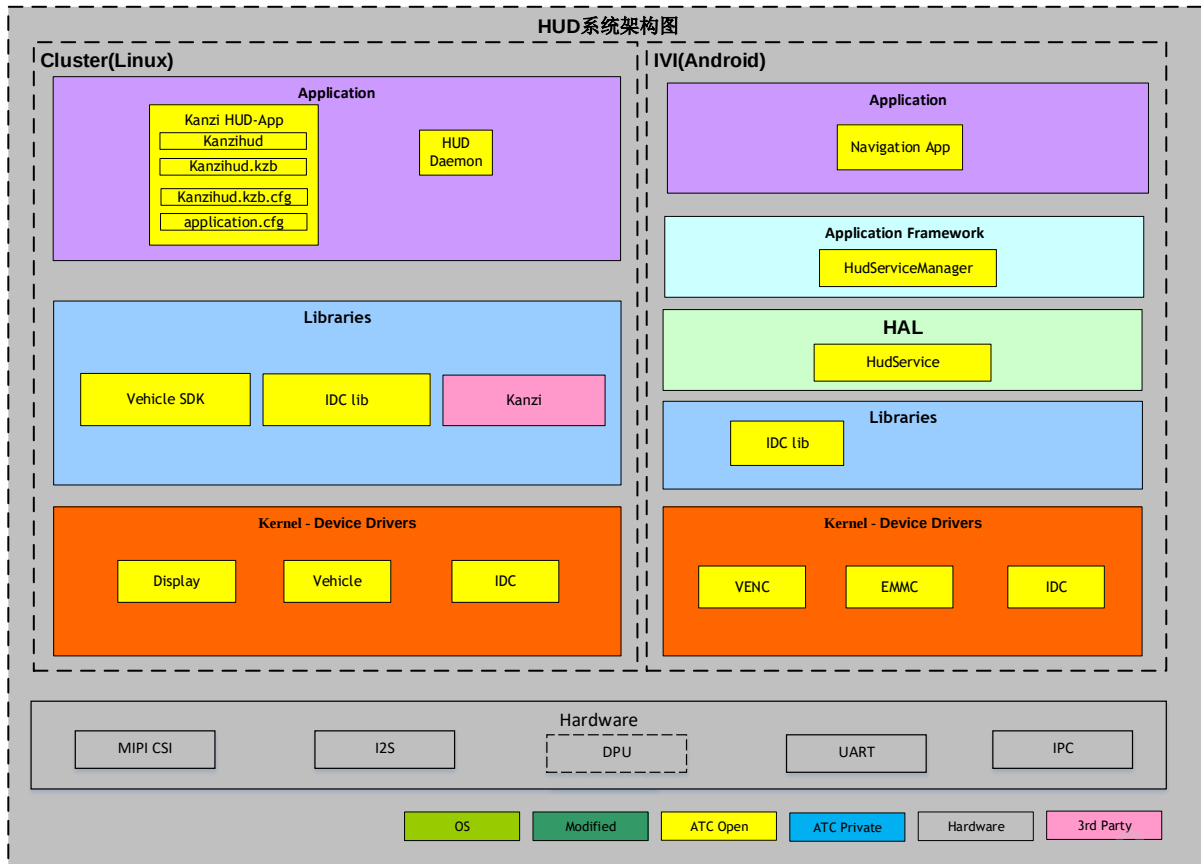


图 48-1 HUD App 架构图

HUD App 由 kanzi 开发的 App 界面，在 Cluster 系统上通过 IDC 和 Vehicle 获取导航和车辆信息，Cluster 端和 Android 端使用 IDC 模块进行通信。在 Android 端由 HAL 层的 HudService 和 Framework 层的 HudServiceManager 组成，通过 IDC 模块发送数据给 Cluster。

HUD Daemon 为守护进程管理 HUD App 生命周期，HUD App 异常挂掉后可重启。

基于 KanziStudio 开发 HUD UI 界面，其相关文件如下：

- application.cfg: kanzi 配置文件，例如：指定 DisplayId、界面显示 GPU FPS 等 debug 信息等。
- KanziHud.kzb.cfg: 指定资源文件名称。
- KanziHud.kzb: 使用 Kanzi Studio 导出的 HUD App 运行资源文件，由 Kanzi 引擎加载并解析动态运行。

48.3 接口描述

48.3.1 HudServiceManager

HudServiceManager 在 Android 系统提供给 Android App 的接口，该接口文件打包在 Autochips.jar 包下。

表 48-2 HudServiceManager API

API	说明
sendRouteGuidanceInfo	发送导航信息

48.3.1.1 sendRouteGuidanceInfo

【描述】

Send navigation information to HUD.

【语法】

```
int sendRouteGuidanceInfo(vec<RouteGuidanceInformation> infoList);
```

【参数】

infoList: route guidance information

【返回值】

- -1: fail
- 0: success.

48.4 使用说明和注意事项

HUD App 运行在 Cluster 系统，基于 Kanzi 实现，通过 application.cfg 中的 DefaultDisplayIndex 属性来配置显示到哪个屏。

49 NPU

49.1 简介

NPU 是神经处理单元 (Neural Processing Unit) 的缩写，它是一种专门用于进行神经网络计算加速的模块。NPU 芯片通常具有高度集成和低功耗的特点，能够帮助设备更快地执行人工智能 (AI) 算法，提高设备的 AI 计算能力。

ATC NPU 模块包含两部分内容：

- (1) 运行于 PC 端的 ATCNN ToolKit：用于模型转换和优化，并导出优化后的模型；
- (2) 运行于 SOC 端的 ATCNN SDK：用于调用 NPU 资源的板端 API。

49.1.1 软硬件环境

49.1.1.1 软件环境

软件运行环境，包括：

- ATCNN SDK：根据产品配置不同，可基于 Linux 或者 Android S 系统调用本 API。
- ATCNN ToolKit：
 - Ubuntu 20.04 LTS 64-bit with python 3.8
 - Ubuntu 18.04 LTS 64-bit with python 3.8

第三方依赖软件：tensorflow、pillow、networkx、lmdb、onnx、dill、flatbuffers、matplotlib、ruamel.yaml、torch、ply、opencv 等，具体版本信息参考表 49-1。

表 49-1 ToolKit 第三方依赖库

依赖库	protobuf == 3.20.3 lmdb == 0.93 networkx == 3.1 onnx >=1.12.0 dill == 0.2.8.2 ruamel.yaml == 0.15.81 ply == 3.11 torch == 1.5.1 tensorflow >= 2.12.0 flatbuffers >= 22.10.26 onnxoptimizer==0.3.13
-----	--

49.1.1.2 硬件环境

- ATCNN SDK: AC8025
- ATCNN ToolKit: 参考表 49-2。

表 49-2 ToolKit 依赖的硬件环境

Resource	Note
CPU	Intel CoreTm5-6500 CPU@3.2GHz x4 with support of the Intel Advanced Vector Extensions
GPU	(Optional)NVIDIA GPU cards with CUDA architectures. Currently, support CUDA 10.0 with compatibility of cuDNN 7.4
RAM	8GB at least
Disk	160GB
OS	Ubuntu 20.04 LTS 64-bit with python 3.8(建议) , Ubuntu 18.04 LTS 64-bit with python 3.8 , or Ubuntu 16.04 LTS 64-bit with python 3.8 Note: Other Ubuntu version are not recommended Important: Ubuntu 16.04 will be deprecated. Use the 20.04 version instead

49.1.2 功能

49.1.2.1 ATCNN ToolKit

ATCNN-Toolkit 为只在 pc 端运行的开发套件，主要包含以下功能：

- 模型转换：支持将多种不同 AI 框架训练的模型转换为板端（NPU 芯片）能正确加载推理的 atcnn 模型，目前支持的 AI 框架包括 ONNX、Caffe、Tensorflow、Darknet、Keras、Tensorflow Lite 及 Pytorch。
- 量化功能：支持将浮点模型转换成整数模型，可配置不同的量化策略，如非对称 8bit per-tensor 均匀量化 (asymmetric_affine_u8) 、对 称 8bit per-channel 均 匀 量 化 (perchannel_symmetric_affine_i8)等。
- 剪枝功能，支持将浮点模型按照权重“重要性”进行剪枝，减少模型计算量，结合板端(NPU 芯片)硬件特性进行加速。
- 编译功能：支持将量化后的模型转换成板端 NPU 可直接运行的二进制模型。
- 模型推理功能，支持推理量化后的 atcnn 模型并与原始的 float 模型进行比较，打印中间特征层及最终输出层的余弦相似度及欧式距离信息。
- 模型性能评估：支持估算转换后的 atcnn 模型的 MAC 数信息。

ATCNN ToolKit 输入为预训练模型（比如使用 caffe/tensorflow/pytorch 等训练后得到模型），输出为 ATC 自定义格式且经过优化的.atcnn 模型。

49.1.2.2 ATCNN SDK

待.atcnn 模型生成完毕，需将其移植至平台端，调用 ATCNN SDK 接口，开始前向推理。ATCNN SDK 可提供如下功能：

- 可调用 NPU 资源，加速前向推理过程；
- 结合 ToolKit，可提供预处理功能，包括：减均值除方差（norm）、缩放（resize）、裁剪（crop）和置换（permute）；
- 可接受 ToolKit 处理完毕的 xxx_board.atcnn 或者 xxx_board_binary.atcnn 格式模型；
- 提供如下两种驱动，两者使用相同接口进行调用：
 - a. Unified driver：包含全功能，客户可基于此驱动进行 debug（如 dump），分析模型问题；可解析 xxx_board.atcnn 或者 xxx_board_binary.atcnn 格式模型；
 - b. Lite driver：裁剪版驱动，使用时需要的资源及依赖的底层库更少，主要用于前向推理，对于资源受限环境或者生产环境，客户可基于 Lite 版本驱动进行开发；Lite driver 只可解析 xxx_board_binary.atcnn 格式模型。

客户可通过使用不同命令可动态切换使能何种版本 driver(不可同时使能)：

- Android 系统：setprop vendor. ATC_NN_DRIVER_TYPE L
- Linux 系统：export ATC_NN_DRIVER_TYPE =L

1) Android 系统驱动切换示例(默认使用 Lite driver, ATC_NN_DRIVER_TYPE=L)：

a) Unified driver -> Lite driver:

```
F:\>adb root
F:\>adb remount
F:\>adb shell
ac8025_demo:/ # cd vendor
ac8025_demo:/vendor # rmmod galcore
ac8025_demo:/vendor # insmod /vendor/lib/modules/vipcore.ko
ac8025_demo:/vendor # setprop vendor.ATC_NN_DRIVER_TYPE L
```

b) Lite driver -> Unified driver:

```
F:\>adb root
F:\>adb remount
F:\>adb shell
ac8025_demo:/ # cd vendor
```



```
ac8025_demo: /vendor # rmmod vipcore
ac8025_demo: /vendor # insmod /vendor /lib/modules/galcore.ko
ac8025_demo: /vendor # setprop vendor.ATC_NN_DRIVER_TYPE U
```

2) ADAS (linux) 系统驱动切换示例(默认使用 Lite driver

ATC_NN_DRIVER_TYPE=L):

以下示例前提条件是切换到 ADAS 域:

a) Unified driver -> Lite driver:

```
F:\>adb root
F:\>adb remount
F:\>adb shell
sh-4.4: # rmmod galcore
sh-4.4: # insmod /lib/modules/ vipcore.ko
sh-4.4: # export ATC_NN_DRIVER_TYPE =L
```

b) Lite driver -> Unified driver:

```
F:\>adb root
F:\>adb remount
F:\>adb shell
sh-4.4: # rmmod vipcore
sh-4.4: # insmod /lib/modules/galcore.ko
sh-4.4: # export ATC_NN_DRIVER_TYPE =U
```

- 提供 C++/Java 两种接口: 其中 [49.3.2.1](#) 中接口可用于 adas(Linux)系统或者 Android 系统 vendor 域; 若需采用 Android service 形式调用, 请使用 [49.3.2.2](#) 中 Java 接口或者 C++接口。

49.2 软件设计

49.2.1 架构图

49.2.1.1 ATCNN ToolKit 架构图

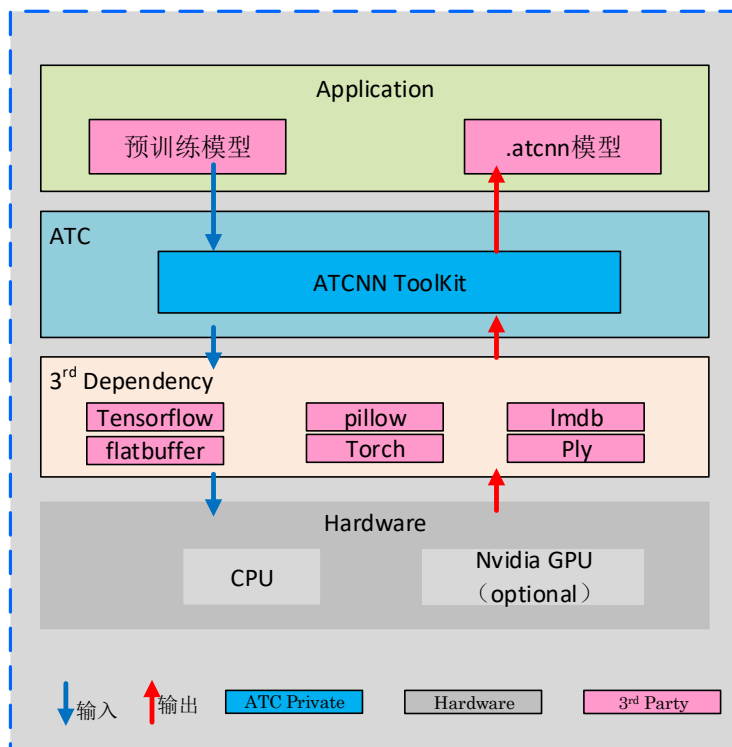


图 49-1 ToolKit 架构图

49.2.1.2 ATCNN SDK 架构图

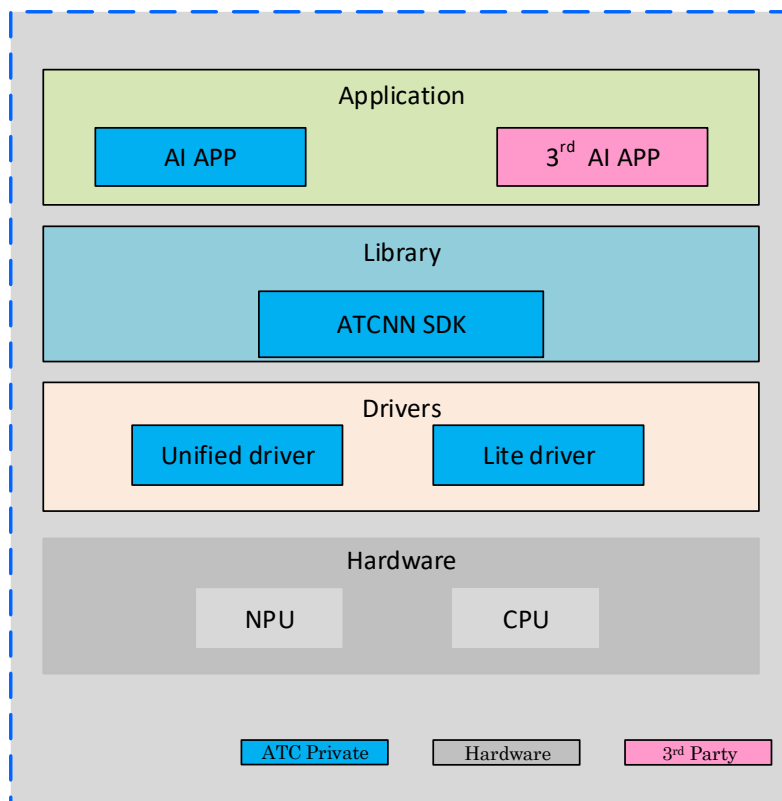


图 49-2 ATCNN SDK 架构图

49.2.2 流程图

49.2.2.1 ATCNN Toolkit 流程图

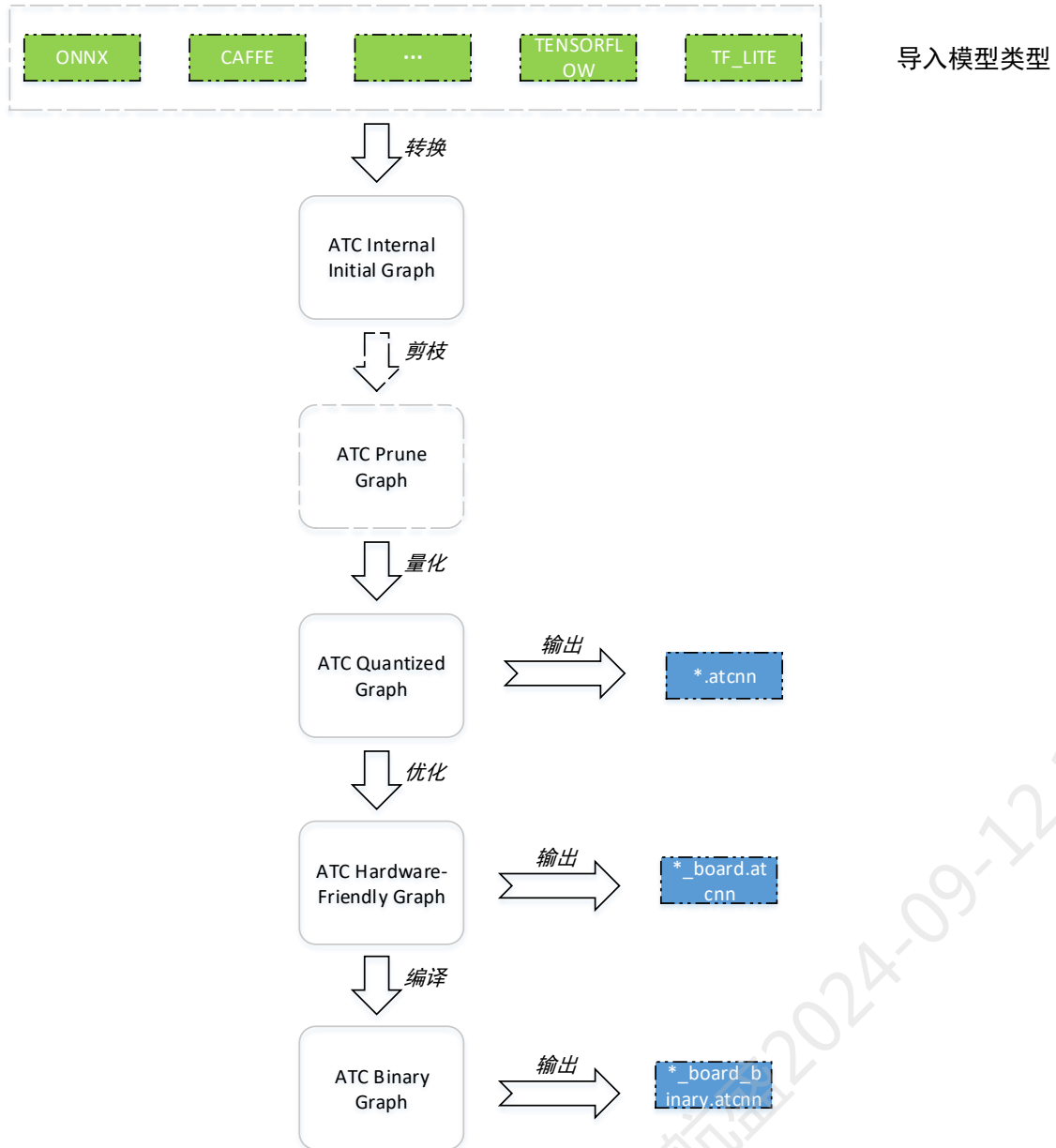


图 49-3 ATCNN ToolKit 流程图

ATCNN Toolkit 支持多种不同深度学习框架模型的导入，如图 49-3 所示，同时输出多个 atcnn 模型文件，可用于不同硬件环境推理。完整流程包括转换、剪枝、量化、图优化、编译，其中剪枝操作非必选项，可利用外部算法剪枝后再将模型导入 ATCNN Toolkit。量化过程也非必选项，ATCNN Toolkit 支持外部量化的 TF_LITE 模型导入。由于 AC8025 硬件平台 NPU 只支持整型计算，因此最后获取的板端模型必须为量化的整型模型表达。

49.2.2.2 ATCNN C++接口调用时序图

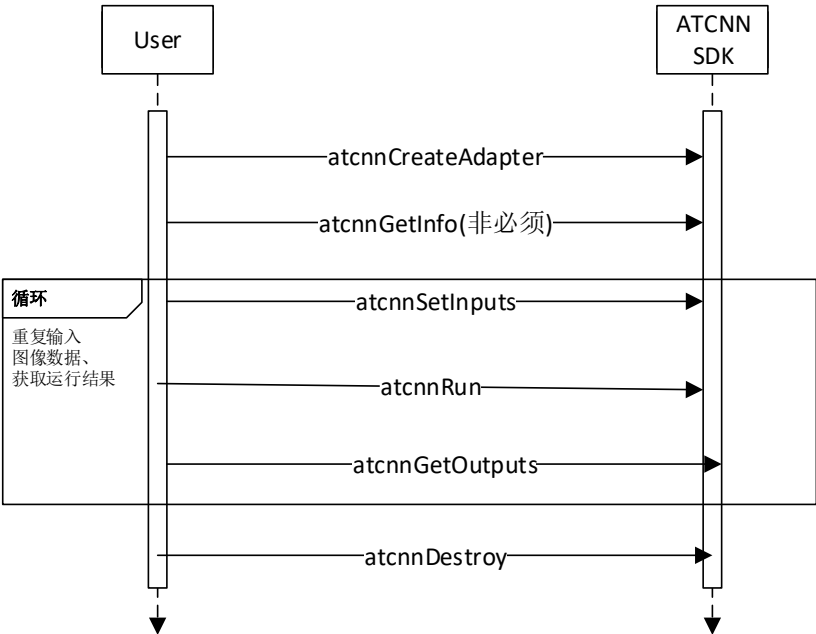


图 49-4 ATCNN C++接口调用时序图

具体调用流程可参考 [49.4.2 ATCNN C++接口基本调用流程](#)。

49.2.2.3 ATCNN Service 接口调用时序图

49.2.2.3.1 C++接口调用时序图

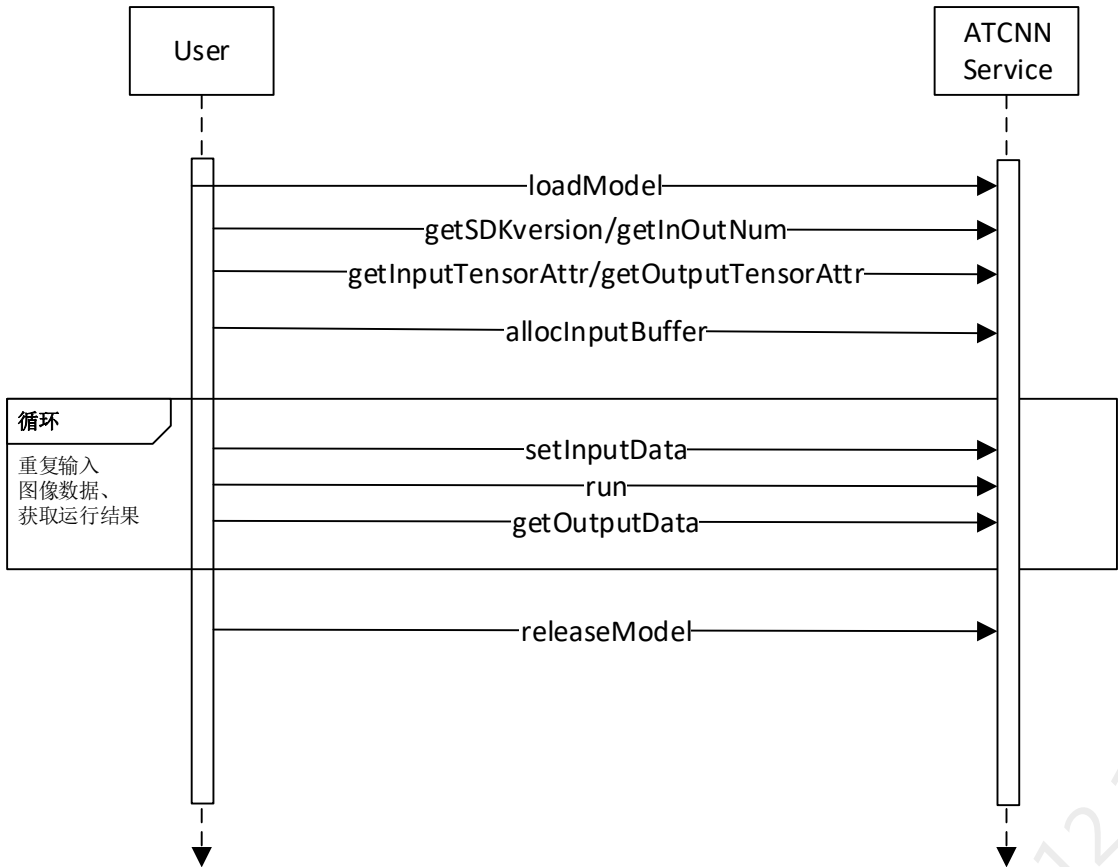


图 49-5 ATCNN Service C++接口调用时序图

具体调用流程可参考：[49.4.3.1 ATCNN Service C++接口基本调用流程](#)。

49.2.2.3.2 Java 接口调用时序图

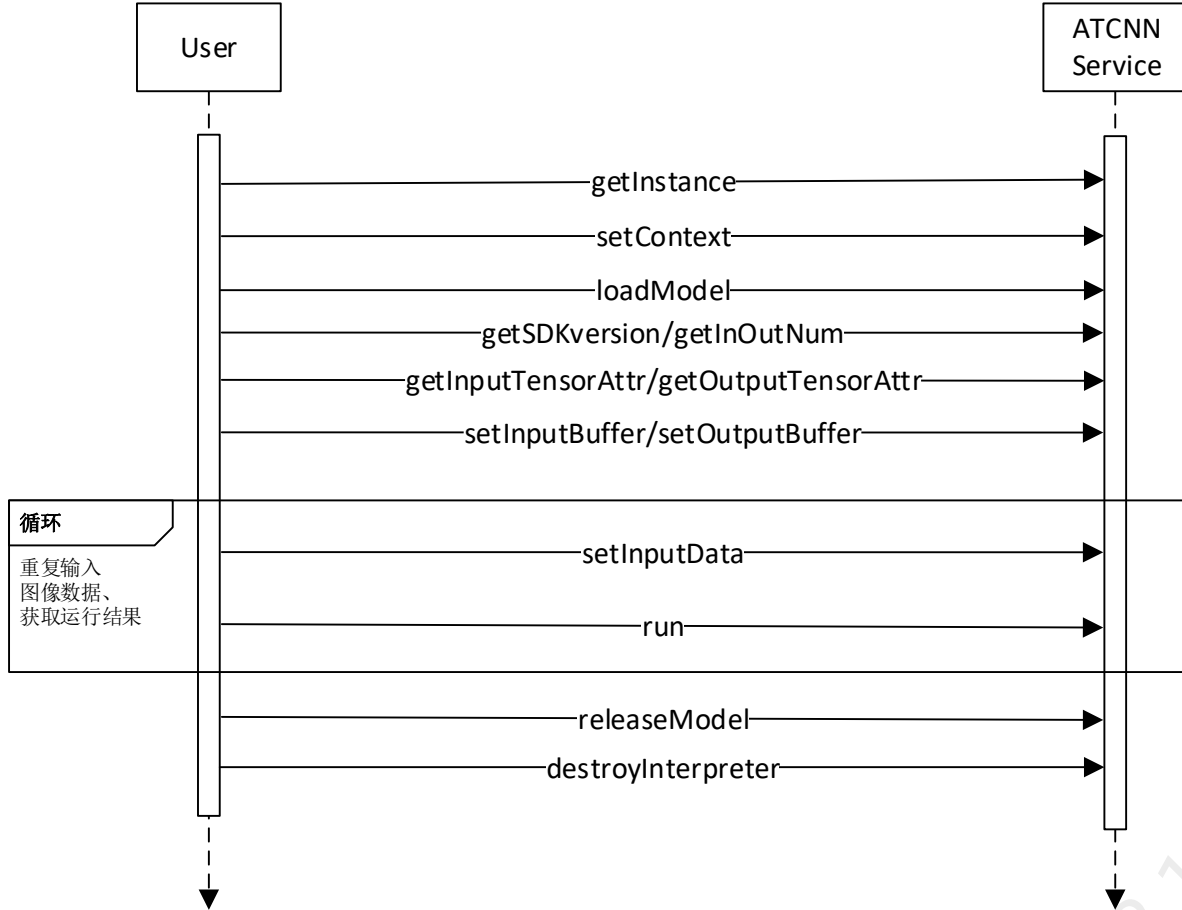


图 49-6 ATCNN Java 接口调用时序图

具体调用流程可参考：[49.4.3.2 ATCNN Service Java 接口基本调用流程](#)。

49.3 接口及数据结构描述

49.3.1 ATCNN ToolKit

ATCNN Toolkit 可通过解析配置文件的方式对模型进行转换和推理等操作，提供的接口 API 如下表所示，均为 python 接口。

表 49-3 ToolKit 类名/接口

类名	成员函数	说明
ATCnn	ATCnn	ATCnn 类的构造函数，创建并初始化 Toolkit 运行环境

	load_model_file_config	读取配置文件信息
	build	转换模型
	inference	推理模型
	eval_performance	性能评估

49.3.1.1 ATCnn

【描述】

创建并返回一个 atcnn 对象实例，并设置相应的参数用于初始化 ATCNN Toolkit 的初始运行环境。

【语法】

```
CLASS atcnlib.atcnn. ATCnn (log_level='ERROR', cmd_info=False)
```

【参数】

- log_level: 定义日志输入的级别，只支持三种类型“ERROR”、“INFO”、“DEBUG”，默认值为“ERROR”
- cmd_info: 定义是否生成二进制模型或者是否推理二进制模型，默认值为 False

【返回值】

- success: 返回一个 atcnn 对象实例 atcnobj
- otherwise: 返回 None

49.3.1.2 load_model_file_config

【描述】

解析配置文件信息

【语法】

```
atcnobj.load_model_file_config(config)
```

【参数】

config: 配置文件名

【返回值】

success: True, otherwise: False

49.3.1.3 build

【描述】

转换模型，并保存转换结果

【语法】

```
atcnnobj.build()
```

【参数】

无

【返回值】

success: True, otherwise: False

49.3.1.4 inference

【描述】

推理模型，并打印推理信息，保存最后输出的 tensor 值

【语法】

```
atcnnobj.inference(image= ' ', output_dir=' debug_atcnn')
```

【参数】

- image: 定义推理的图片名称，默认值为空（此时将从量化数据集里随机选择一张图片进行推理）
- output_dir: 定义文件夹名用于保存推理后的输出 tensor 值

【返回值】

success: True, otherwise: False

49.3.1.5 eval_performance

【描述】

性能评估，并打印性能数据

【语法】

```
atcnnobj.eval_performance()
```

【参数】

无

【返回值】

success: True, otherwise: False

49.3.2 ATCNN SDK

49.3.2.1 C++ Interface

注意：C++接口不支持跨域访问。

ATCNN SDK 相关的各版本 C++通用接口如下表所示：

表 49-4 ATCNN SDK

API	说明
atcnnCreateAdapter	初始化设备环境，并加载 atcnn 模型
atcnnGetInfo	查询版本信息、输入输出层信息
atcnnSetInputs	设置输入图像信息
atcnnRun	执行前向推理过程
atcnnGetOutputs	获取前向推理结果数据
atcnnDestroy	释放相关资源

49.3.2.1.1 atcnnCreateAdapter

【描述】

创建 adapter 并加载 atcnn 模型，是否自动保存 xxx_board_binary.atcnn 模型。

【语法】

```
atcnn_return_value atcnnCreateAdapter(atcnn_adapter* adapter, const char* buffer, bool
saveNBG=false);
```

【返回值】

atcnn_return_value 错误码（见[返回值错误码](#)）

【参数】

- atcnn_adapter * adapter：创建设备环境
- const char* buffer：atcnn 模型路径或者 atcnn 模型 buffer
- bool saveNBG：是否自动保存 xxx_board_binary.atcnn 模型，默认为 false

【注意】

- xxx_board.atcnn 和 xxx_board_binary.atcnn 使用同一接口；
- 使用 xxx_board.atcnn 初始化耗时会比使用 xxx_board_binary.atcnn 耗时长；若设置 saveNBG 为 true，则在程序第一次初始化时会自动保存一个以哈希值命名的中间文件，当程序再此启动

时，驱动会自动加载该中间文件，从而减少初始化时间；android 系统默认保存地址为“/early/nn/”，linux 系统会默认保存到当前目录下；

- c) Unified driver 支持 xxx_board.atcnn 和 xxx_board_binary.atcnn 格式输入，且 saveNBG 只有在输入为 xxx_board.atcnn 格式模型时生效；
- d) Lite driver 只支持 xxx_board_binary.atcnn 格式输入，saveNBG 不生效。

49.3.2.1.2 atcnnGetInfo

【描述】

查询模型和 SDK API 相关信息。

【语法】

```
atcnn_return_value atcnnGetInfo(atcnn_adapter* adapter, atcnn_get_info infoFlag, void* info,
int32_t infoSize)
```

【返回值】

atcnn_return_value 错误码（见[返回值错误码](#)）

【参数】

- atcnn_adapter* adapter: 设备环境
- atcnn_get_info infoFlag: 查询的信息类别（见[查询信息类别](#)）
- void* info: 存放返回结果的结构体变量
- int32_t infoSize: info 对应的结构体变量的大小

49.3.2.1.3 atcnnSetInputs

【描述】

设置输入 buffer 及相关属性。

【语法】

```
atcnn_return_value atcnnSetInputs(atcnn_adapter* adapter, atcnn_data input[], int32_t n_input)
```

【返回值】

atcnn_return_value 错误码（见[返回值错误码](#)）

【参数】

- atcnn_adapter* adapter: 设备环境
- atcnn_data *input: 输入数据 buffer（见[atcnn_data](#) 数据结构）

- `int32_t n_input`: 共有几个输入

【注意】

- `atcnn_data *input` 由用户创建和释放;
- 模型输入图像格式需在 ToolKit 模型转换时设置, 默认 `ATCNN_FORMAT_IMAGE_RGB`;
- 若在模型转换时设置了预处理, SDK 会解析存于 `.atcnn` 结构文件中的预处理信息, 驱动层预处理顺序为: `crop->resize->color convert->reverse->scale/mean -> permute`;
- 当输入为 `ATCNN_FORMAT_IMAGE_YUV420/ ATCNN_FORMAT_IMAGE_YUV444` 时, 输入节点会变成 3 个; 当输入为 `NV12` 或者 `NV21` 输入时, 输入节点会变成 2 个, 详细维度信息可通过 `atcnnGetInfo` 查询。

49.3.2.1.4 atcnnRun

【描述】

执行推理操作。

【语法】

```
atcnn_return_value atcnnRun(atcnn_adapter* adapter, bool dump=false, bool frameTime=false,
bool layerTime=false)
```

【返回值】

`atcnn_return_value` 错误码 (见[返回值错误码](#))

【参数】

- `atcnn_adapter* adapter`: 设备环境
- `bool dump`: 是否 dump 中间特征层数据 (默认为 `false`)
- `bool frameTime`: 是否打印模型推理时间 (默认为 `false`)
- `bool layerTime`: 是否打印单层执行时间 (默认为 `false`)

【注意】

`dump/ frameTime/ layerTime` 为 debug 方法, 具体执行建议关闭, 会影响执行效率。`dump` 和 `layerTime` 在 Lite driver 中无效。

49.3.2.1.5 atcnnGetOutputs

【描述】

获取推理结果

【语法】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

1208 /1254

本文档包含信息归杰发科技所有, 未经许可, 严禁转载或披露

atcnn_return_value atcnnGetOutputs(atcnn_adapter* adapter, atcnn_data output[], int32_t n_output)

【返回值】

atcnn_return_value 错误码（见[返回值错误码](#)）

【参数】

- atcnn_adapter* adapter: 设备环境
- atcnn_data *output: 输出数据 buffer（见[atcnn_data](#) 数据结构）
- int32_t n_output: 获取第几个输出，输出 id 从 0 开始

【注意】

- a) atcnn_data *output 根据 atcnn_data 中的 is_prealloc 参数确定承接输出结果的 buffer 由内部创建还是外部创建；若为内部创建（默认），用户无需释放，由 SDK 内部管理；若为外部创建（is_prealloc=true），用户需自行管理申请的内存；
- b) 默认所有输出格式为 [ATCNN_TYPE_FLOAT32](#)。

49.3.2.1.6 atcnnDestroy

【描述】

释放 NPU 资源。

【语法】

atcnn_return_value atcnnDestroy(atcnn_adapter* adapter)

【参数】

- atcnn_adapter* adapter: 设备环境

【返回值】

atcnn_return_value 错误码（见[返回值错误码](#)）

【参数】

无

49.3.2.2 Service Interface

系统中已集成 ATCNN 的 vendor HAL 服务：[vendor.autochips.hardware.atcnn@1.0-service](#)，用户在做 System 域或 App 应用开发时，可以使用 ATCNN Service Interface 进行模型部署，当前有提供 C++及 Java 版本接口。

49.3.2.2.1 C++ Interface

ATCNN Service Interface 的 C++通用接口如下表所示：

表 49-5 ATCNN Service C++ SDK

API	说明
loadModel	加载.atcnn 模型，获取模型句柄
getSDKVersion	获取 SDK 版本号
getInOutNum	获取模型的输入/输出个数
getInputTensorAttr	获取输入 tensor 的属性
getOutputTensorAttr	获取输出 tensor 的属性
allocInputBuffers	分配输入图像的缓冲区
setInputData	设置输入图像信息
run	运行模型推理
getOutputData	获取模型推理结果
releaseModel	释放模型资源

49.3.2.2.1.1 loadModel

【描述】

加载模型文件

【语法】

```
bool loadModel(int& modelId, const char* filename, bool saveNBG, int size);
```

【返回值】

bool 指示接口是否运行成功

【参数】

- int& modelId: 引用传递，接收对应的模型句柄
- const char* filename: 可以是模型文件的完整路径，或模型字节流的地址
- boolean saveNBG: 是否存储 NBG 文件，缺省值为 false
- int size: 模型字节流的长度，若传入的 filename 为文件路径，此长度可不填，缺省为-1

【注意】

若模型文件解析正常，modelId >= 0

49.3.2.2.1.2 getSDKVersion

【描述】

获取 SDK 版本信息

【语法】

```
bool getSDKVersion(int modelId, atcnn_sdk_version& sdkInfo);
```

【返回值】

bool 指示接口是否运行成功

【参数】

- int modelId：模型文件句柄
- atcnn_sdk_version& sdkInfo：引用传递，用于接收 SDK 版本信息

【注意】

无

49.3.2.2.1.3 getInOutNum

【描述】

获取输入输出个数

【语法】

```
bool getInOutNum(int modelId, atcnn_tensor_in_out_num& inoutNum);
```

【返回值】

bool 指示接口是否运行成功

【参数】

- int modelId：模型文件句柄
- atcnn_tensor_in_out_num& inoutNum：引用传递，用于接收输入、输出个数信息

【注意】

无

49.3.2.2.1.4 getInputTensorAttr

【描述】

获取输入 tensor 的属性信息

【语法】

```
bool getInputTensorAttr(int modelId, int index, atcnn_tensor_attr& attr);
```

【返回值】

bool 指示接口是否运行成功

【参数】

- int modelId: 模型文件句柄
- int index: 第几个输入 tensor
- atcnn_tensor_attr& attr: 引用传递，用于接收指定输入 tensor 的属性信息

【注意】

无

49.3.2.2.1.5 **getOutputTensorAttr**

【描述】

获取输出 tensor 的属性信息

【语法】

```
bool getOutputTensorAttr(int modelId, int index, atcnn_tensor_attr& attr);
```

【返回值】

bool 指示接口是否运行成功

【参数】

- int modelId: 模型文件句柄
- int index: 第几个输出 tensor
- atcnn_tensor_attr& attr: 引用传递，用于接收指定输出 tensor 的属性信息

【注意】

无

49.3.2.2.1.6 **setInputData**

【描述】

设置输入图像信息

【语法】


```
bool setInputData(int modelId, std::vector<atcnn_data> inputDatas);
```

【返回值】

bool 指示接口是否运行成功

【参数】

- int modelId: 模型文件句柄
- std::vector<atcnn_data> inputDatas: 模型的所有输入信息

【注意】

atcnn_data 中的 data 成员变量对应输入 buffer 的地址，若不为 nullptr，则会使用此 buffer 做为输入，若为 nullptr，则使用 allocInputBuffers 分配的 buffer 做为输入。建议使用 allocInputBuffers 分配输入 buffer，效率会更高

49.3.2.2.1.7 allocInputBuffers

【描述】

分配输入图像的缓冲区

【语法】

```
bool allocInputBuffers(int modelId, std::vector<int> sizes, std::vector<unsigned char*>& buffers);
```

【返回值】

bool 指示接口是否运行成功

【参数】

- int modelId: 模型文件句柄
- std::vector<int> sizes: 每个输入的 buffer 长度信息
- std::vector<unsigned char*>& buffers: 引用传递，用于接收每个输入 buffer 对应的地址

【注意】

推荐使用这种方式获取输入 buffer，来填入输入图像数据，效率会更高

49.3.2.2.1.8 run

【描述】

运行模型推理

【语法】

```
bool run(int modelId, bool dump, bool frameTime, bool layerTime);
```

【返回值】

bool 指示接口是否运行成功

【参数】

- int modelId: 模型文件句柄
- bool dump: 是否 dump 中间特征层数据
- bool frameTime: 是否打印模型推理时间
- bool layerTime: 是否打印单层执行时间

【注意】

dump/ frameTime/ layerTime 为 debug 方法，具体执行建议关闭，会影响执行效率

49.3.2.2.1.9 getOutputData**【描述】**

获取输出的推理结果数据

【语法】

```
bool getOutputData(int modelId, int outputSize, std::vector<atcnn_data>& datas);
```

【返回值】

bool 指示接口是否运行成功

【参数】

- int modelId: 模型文件句柄
- int outputSize: 指定获取的输出个数，需要小于或等于模型的最大输出个数
- std::vector< atcnn_data>& datas: 引用传递，用于接收指定输出个数的输出数据

【注意】

atcnn_data 中的 data 成员变量对应输出 buffer 的地址，应用可通过此地址获取推理结果

49.3.2.2.1.10 releaseModel**【描述】**

释放模型解析占用的资源

【语法】

```
bool releaseModel(int modelId);
```

【返回值】

bool 指示接口是否运行成功

【参数】

int modelId: 模型文件句柄

【注意】

无

49.3.2.2.2 Java Interface

ATCNN Service Interface 的 Java 通用接口如下表所示:

表 49-6 ATCNN JAVA SDK

API	说明
getInstance	获取 AtcnnInterpreter, 解释器为单例模式
setContext	设置环境上下文
loadModel	加载.atcnn 模型, 返回模型句柄
getSDKVersion	获取 SDK 版本号
getInOutNum	获取模型的输入/输出个数
getInputTensorAttr	获取输入 tensor 的属性
getOutputTensorAttr	获取输出 tensor 的属性
setInputData	设置输入图像信息
setInputBuffers	设置输入图像的缓冲区
setOutputBuffers	设置输出结果的缓冲区
run	运行模型推理
releaseModel	释放模型资源
destroyInterpreter	销毁解释器

49.3.2.2.2.1 getInstance**【描述】**

获取 Atcnn 模型解释器。

【语法】

```
AtcnnInterpreter getInstance();
```

【返回值】

AtcnnInterpreter

【参数】

无

【注意】

AtcnnInterpreter 为单例模式，只能通过此接口获取解释器对象。

49.3.2.2.2.2 setContext**【描述】**

设置上下文信息。

【语法】

```
void setContext(Context context);
```

【返回值】

无

【参数】

Context context: 上下文信息

【注意】

无

49.3.2.2.2.3 loadModel**【描述】**

加载模型文件

【语法】

```
int loadModel(String modelName, boolean saveNBG, boolean localParse);
```

【返回值】

Int 对应模型句柄，若模型解析成功将返回大于 0，若失败则返回小于 0

【参数】

- String modelName: 若 localParse 为 true，则对应 Assets 资源包中的模型文件名，若 localParse 为 false，则对应模型文件在平台中的完整路径
- boolean saveNBG: 是否存储 NBG 文件
- boolean localParse: 模型文件是否在本地 Assets 资源包中

【注意】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

1216 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

无

49.3.2.2.2.4 getSDKVersion

【描述】

获取 SDK 版本信息

【语法】

```
Atcnn_sdk_version getSDKVersion(int modelId);
```

【返回值】

Atcnn_sdk_version SDK 版本信息结构体

【参数】

int modelId : 模型文件句柄

【注意】

无

49.3.2.2.2.5 getInOutNum

【描述】

获取输入输出个数

【语法】

```
Atcnn_tensor_in_out_num getInOutNum(int modelId);
```

【返回值】

Atcnn_tensor_in_out_num 模型输入输出个数结构体

【参数】

int modelId : 模型文件句柄

【注意】

无

49.3.2.2.2.6 getInputTensorAttr

【描述】

获取输入 tensor 的属性信息

【语法】

杰发科技机密文件

© 2023 - 2024 杰发科技有限公司

1217 / 1254

本文档包含信息归杰发科技所有，未经许可，严禁转载或披露

Atcnn_tensor_attr getInputTensorAttr(int modelId, int index);

【返回值】

Atcnn_tensor_attr tensor 属性信息

【参数】

- int modelId: 模型文件句柄
- int index: 第几个输入 tensor

【注意】

无

49.3.2.2.7 **getOutputTensorAttr**

【描述】

获取输入 tensor 的属性信息

【语法】

Atcnn_tensor_attr getOutputTensorAttr(int modelId, int index);

【返回值】

Atcnn_tensor_attr tensor 属性信息

【参数】

- int modelId: 模型文件句柄
- int index: 第几个输出 tensor

【注意】

无

49.3.2.2.8 **setInputData**

【描述】

设置输入图像信息

【语法】

boolean setInputData(int modelId, ArrayList<Atcnn_data> inputData);

【返回值】

Boolean 返回是否设置成功

【参数】

- int modelId: 模型文件句柄
- ArrayList<Atcnn_data> inputData: 模型的所有输入信息

【注意】

无

49.3.2.2.2.9 setInputBuffers**【描述】**

设置输入图像的缓冲区

【语法】

```
void setInputBuffers(int modelId, Map<Integer, byte[]> bufferMap);
```

【返回值】

无

【参数】

- int modelId: 模型文件句柄
- Map<Integer, byte[]> bufferMap: 每个输入与其对应 buffer 的映射关系

【注意】

无

49.3.2.2.2.10 setOutputBuffers**【描述】**

设置输出结果的缓冲区

【语法】

```
setOutputBuffers(int modelId, Map<Integer, byte[]> bufferMap);
```

【返回值】

无

【参数】

- int modelId: 模型文件句柄
- Map<Integer, byte[]> bufferMap: 每个输出与其对应 buffer 的映射关系

【注意】

无

49.3.2.2.11 run**【描述】**

运行模型推理

【语法】

```
boolean run(int modelId, boolean dump, boolean frameTime, boolean layerTime);
```

【返回值】

boolean 返回是否运行成功

【参数】

- int modelId: 模型文件句柄
- boolean dump: 是否 dump 中间特征层数据
- boolean frameTime: 是否打印模型推理时间
- boolean layerTime: 是否打印单层执行时间

【注意】

dump/ frameTime/ layerTime 为 debug 方法，具体执行建议关闭，会影响执行效率

49.3.2.2.12 releaseModel**【描述】**

释放模型解析占用的资源

【语法】

```
boolean releaseModel(int modelId);
```

【返回值】

boolean 返回是否释放成功

【参数】

int modelId: 模型文件句柄

【注意】

无

49.3.2.2.2.13 destroyInterpreter

【描述】

销毁解释器，释放对应资源

【语法】

```
boolean destroyInterpreter();
```

【返回值】

boolean 返回是否释放成功

【参数】

无

【注意】

无

49.3.3 ATCNN SDK 数据结构

49.3.3.1 C++ 数据结构

表 49-7 ATCNN SDK 数据结构

数据结构	说明
atcnn_adapter	设备环境句柄
atcnn_dim_layout	Tensor layout排列
atcnn_source_format	ATCNN API支持输入的数据格式
atcnn_tensor_type	支持的数据类型
atcnn_return_value	ATCNN API返回值错误码
atcnn_get_info	查询信息类别
atcnn_sdk_version	ATCNN API版本信息，包括API版本及所基于的驱动版本
atcnn_data	输入/输出数据结构体
atcnn_tensor_in_out_num	表示input和output的Tensor个数
atcnn_tensor_attr	表示tensor属性，目前主要用于查询网络输入/输出tensor属性

49.3.3.1.1 atcnn_adapter

```
typedef void* atcnn_adapter;
```

49.3.3.1.2 atcnn_dim_layout

```
typedef enum _atcnn_dim_layout{
    ATCNN_DIM_NCHW = 0,
    ATCNN_DIM_NHWC,
    ATCNN_DIM_NA
}atcnn_dim_layout;
```

49.3.3.1.3 atcnn_source_format

```
typedef enum _atcnn_source_format{
    ATCNN_FORMAT_TENSOR = 0,
    ATCNN_FORMAT_IMAGE_GRAY,
    ATCNN_FORMAT_IMAGE_RGB,
    ATCNN_FORMAT_IMAGE_YUV420,
    ATCNN_FORMAT_IMAGE_BGRA,
    ATCNN_FORMAT_IMAGE_RGB888_PLANAR,
    ATCNN_FORMAT_IMAGE_YUV444,
    ATCNN_FORMAT_IMAGE_NV12,
    ATCNN_FORMAT_IMAGE_NV21
}atcnn_source_format;
```

49.3.3.1.4 atcnn_tensor_type

```
typedef enum _atcnn_tensor_type{
    ATCNN_TYPE_NONE = 0,
    ATCNN_TYPE_INT8,
    ATCNN_TYPE_UINT8,
    ATCNN_TYPE_INT16,
    ATCNN_TYPE_FLOAT16,
    ATCNN_TYPE_FLOAT32,
    ATCNN_TYPE_INT32
}atcnn_tensor_type;
```

49.3.3.1.5 atcnn_return_value

```
typedef enum _atcnn_return_value{
```

```
    ATCNN_SUCCESS = 0,  
    ATCNN_FAILURE,  
    ATCNN_UNINIT,  
    ATCNN_TIMEOUT,  
    ATCNN_DEVICE_UNAVAILABLE,  
    ATCNN_FILE_INVALID,  
    ATCNN_PARAM_WRONG,  
    ATCNN_MEMORY_WRONG,  
    ATCNN_VERSION_UNMATCH,  
    ATCNN_EXPOER_BOARD_BINARY_FAILURE,  
}atcnn_return_value;
```

49.3.3.1.6 atcnn_get_info

```
typedef enum _atcnn_get_info{  
    ATCNN_SDK_VERSION = 0,  
    ATCNN_IN_OUT_NUM,  
    ATCNN_INPUT_ATTR,  
    ATCNN_OUTPUT_ATTR,  
}atcnn_get_info;
```

49.3.3.1.7 atcnn_quant_type

```
typedef enum _atcnn_quant_type{  
    ATCNN_QNT_NONE = 0,  
    ATCNN_QNT_DFT,  
    ATCNN_QNT_ASYMM,  
    ATCNN_QNT_SYMMETRIC,  
}atcnn_quant_type;
```

49.3.3.1.8 atcnn_sdk_version

```
typedef struct _atcnn_sdk_version{  
    string apiVersion;    //sdk 版本  
    string drvVersion;    //底层 driver 版本  
    string toolKitVersion; //当前 atcnn 模型来源于 toolKit 的版本
```

```
}atcnn_sdk_version;
```

49.3.3.1.9 atcnn_data

```
typedef struct _atcnn_data{
    void* data;
    int32_t n_dims;
    int32_t dims[ATCNN_MAX_DIMS];
    atcnn_dim_layout layout;
    atcnn_tensor_type dataType;
    bool is_prealloc,
    _atcnn_data() {
        data = NULL;
        is_prealloc = false;
        datatype = ATCNN_TYPE_UINT8;
    }
}atcnn_data;
```

49.3.3.1.10 atcnn_tensor_in_out_num

```
typedef struct _atcnn_tensor_in_out_num{
    int32_t n_input;
    int32_t n_output;
}atcnn_tensor_in_out_num;
```

49.3.3.1.11 atcnn_tensor_attr

```
typedef struct _atcnn_tensor_attr{
    int32_t index;
    std::string name;
    int32_t n_dims;
    int32_t dims[ATCNN_MAX_DIMS];
    atcnn_dim_layout layout;
    atcnn_quant_type quantType;
}atcnn_tensor_attr;
```

49.3.3.2 Java 数据结构

表 49-8 ATCNN SDK 数据结构

数据结构	说明
ATCNN_DIM_LAYOUT	Tensor layout排列
ATCNN_SOURCE_FORMAT	ATCNN API支持输入的数据格式
ATCNN_TENSOR_TYPE	支持的数据类型
ATCNN_RETURN_VALUE	ATCNN API返回值错误码
ATCNN_GET_INFO	查询信息类别
ATCNN_QUANT_TYPE	支持的量化类型
Atcnn_sdk_version	ATCNN API版本信息，包括API版本及所基于的驱动版本
Atcnn_data	输入/输出数据结构体
Atcnn_tensor_in_out_num	表示input和output的Tensor个数
Atcnn_tensor_attr	表示tensor属性，目前主要用于查询网络输入/输出tensor属性

49.3.3.2.1 ATCNN_DIM_LAYOUT

```
enum ATCNN_DIM_LAYOUT : int32_t {
    ATCNN_DIM_NCHW = 0,
    ATCNN_DIM_NHWC,
    ATCNN_DIM_NA
};
```

49.3.3.2.2 ATCNN_SOURCE_FORMAT

```
enum ATCNN_SOURCE_FORMAT : int32_t {
    ATCNN_FORMAT_TENSOR = 0,
    ATCNN_FORMAT_IMAGE_GRAY,
    ATCNN_FORMAT_IMAGE_RGB,
    ATCNN_FORMAT_IMAGE_YUV420,
    ATCNN_FORMAT_IMAGE_BGRA,
    ATCNN_FORMAT_IMAGE_RGB888_PLANAR,
    ATCNN_FORMAT_IMAGE_YUV444,
    ATCNN_FORMAT_IMAGE_NV12
};
```

49.3.3.2.3 ATCNN_TENSOR_TYPE

```
enum ATCNN_TENSOR_TYPE : int32_t {  
    ATCNN_TYPE_NONE = 0,  
    ATCNN_TYPE_INT8,  
    ATCNN_TYPE_UINT8,  
    ATCNN_TYPE_INT16,  
    ATCNN_TYPE_FLOAT16,  
    ATCNN_TYPE_FLOAT32  
};
```

49.3.3.2.4 ATCNN_RETURN_VALUE

```
enum ATCNN_RETURN_VALUE : int32_t {  
    ATCNN_SUCCESS = 0,  
    ATCNN_FAILURE,  
    ATCNN_UNINIT,  
    ATCNN_TIMEOUT,  
    ATCNN_DEVICE_UNAVAILABLE,  
    ATCNN_FILE_INVALID,  
    ATCNN_PARAM_WRONG,  
    ATCNN_MEMORY_WRONG,  
    ATCNN_VERSION_UNMATCH,  
    ATCNN_EXPOER_BOARD_BINARY_FAILURE  
};
```

49.3.3.2.5 ATCNN_GET_INFO

```
enum ATCNN_GET_INFO : int32_t {  
    ATCNN_SDK_VERSION = 0,  
    ATCNN_IN_OUT_NUM,  
    ATCNN_INPUT_ATTR,  
    ATCNN_OUTPUT_ATTR  
};
```

49.3.3.2.6 ATCNN_QUANT_TYPE

```
enum ATCNN_QUANT_TYPE : int32_t {
    ATCNN_QNT_NONE = 0,
    ATCNN_QNT_DFP,
    ATCNN_QNT_ASYMM,
    ATCNN_QNT_SYMMETRIC
};
```

49.3.3.2.7 Atcnn_sdk_version

```
struct Atcnn_sdk_version {
    string apiVersion;
    string drvVersion;
    string toolKitVersion;
};
```

49.3.3.2.8 Atcnn_data

```
struct Atcnn_data {
    handle data;
    int32_t n_dims;
    vec<uint32_t> dims;
    ATCNN_DIM_LAYOUT layout;
    ATCNN_TENSOR_TYPE dataType;
};
```

49.3.3.2.9 Atcnn_tensor_in_out_num

```
struct Atcnn_tensor_in_out_num {
    int32_t n_input;
    int32_t n_output;
};
```

49.3.3.2.10 Atcnn_tensor_attr

```
struct Atcnn_tensor_attr {
    int32_t index;
```

```
string name;

int32_t n_dims;

vec<uint32_t> dims;

ATCNN_DIM_LAYOUT layout;

ATCNN_QUANT_TYPE quantType;

}
```

49.4 使用说明和注意事项

49.4.1 ATCNN TooKit 安装使用说明

49.4.1.1 安装说明

ATCNN-Toolkit 提供 wheel 包安装方式，通过 `pip install` 命令进行安装。Wheel 文件被包含于提供的 `atcnn-toolkit` tar 包里，如图 49-7 所示，该 tar 包还包含一个 demo 示例（示例模型为 `lenet.onnx`，量化数据集位于 `dataset` 文件夹下，调用配置文件为 `example.prototxt`，`atcnn_cmd.py` 封装了转换和推理等子过程）。

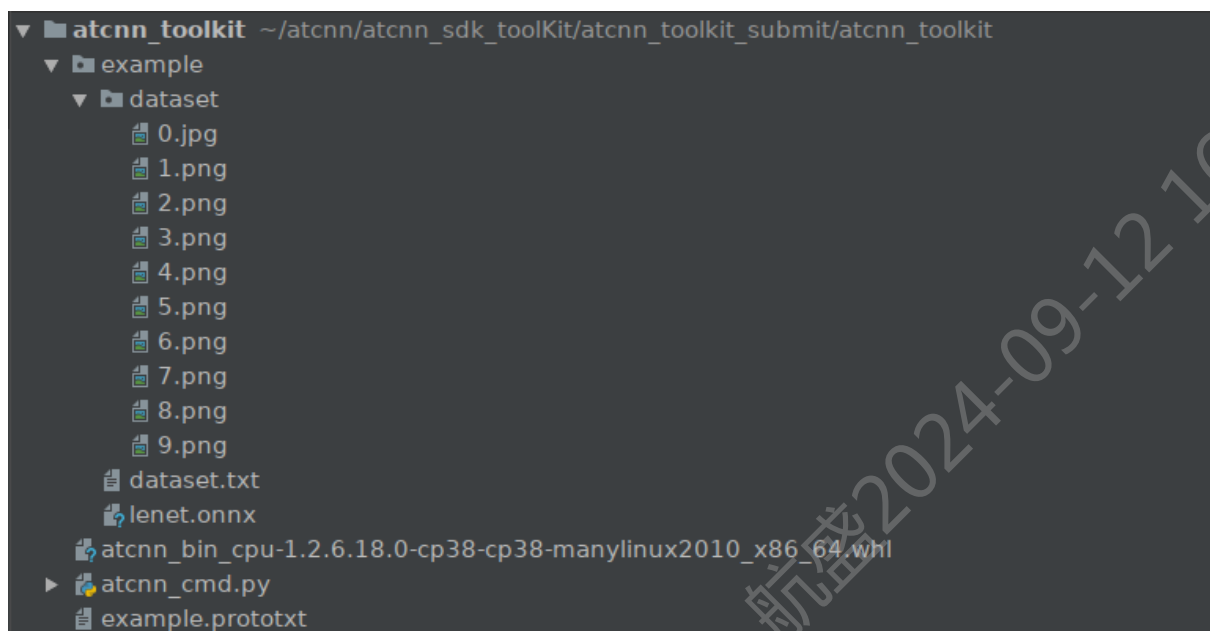


图 49-7 ATCNN-Toolkit tar 包内容示意图

(1) 创建 `virtualenv` 环境（建议使用，可以隔离原有 `python` 环境），执行命令如下：

```
sudo apt update
sudo apt install virtualenv
```



```
sudo apt install python3.8
sudo apt install libpython3.8-dev
sudo apt install python3.8-tk
sudo apt install build-essential

virtualenv -p /usr/bin/python3.8 atcnn
source atcnn/bin/activate
```

(2) 安装依赖库

```
pip install tensorflow==2.12.0
pip install protobuf==3.20.3
pip install lmdb==0.93
pip install torch==1.5.1
pip install torchvision==0.6.1
pip install networkx==3.1
pip install onnxoptimizer==0.3.13
pip install onnx==1.14.0
pip install dill == 0.2.8.2
pip install ruamel.yaml == 0.15.81
pip install ply == 3.11
pip install flatbuffers == 23.5.26
```

(3) 安装 ATCNN-Toolkit

```
pip install atcnn_bin_cpu-1.2.6.18.0-cp38-cp38-manylinux2010_x86_64.whl
```

49.4.1.2 使用说明

ATCNN-Toolkit 可通过配置文件的方式对指定模型进行转换、推理和性能评估，配置文件使用 google protobuf 进行表达。

49.4.1.2.1 配置文件说明

在模型转换、推理和性能评估前先要对模型配置文件进行设置，包括模型信息、量化信息及预处理信息等。详细的配置文件属性介绍如下所示。

49.4.1.2.1.1 配置文件属性域信息

表 49-9 配置文件属性域信息

属性名称	属性说明
target_hardware	设置硬件平台信息，目前仅支持 AC8025，默认值为 AC8025
atc_net_config	设置模型描述信息，主要用于描述模型的类型、输入输出节点信息等，请参见 49.4.1.2.1.2
atc_preproc_config	设置模型预处理信息，主要用于描述输入节点预处理信息，请参见 49.4.1.2.1.3
atc_opt_quantization_config	设置模型优化及量化信息，主要用于定义模型裁剪、模型量化信息等，请参见 49.4.1.2.1.4

49.4.1.2.1.2 模型描述信息(atc_net_config)

表 49-10 模型描述信息

属性名称	属性说明
model_type	设置模型类型，可配置值为 CAFFE、TENSORFLOW、PYTORCH、ONNX、TF_LITE、DARKNET、KERAS、ATCNN、UNKNOWN_TYPE，默认值为 UNKNOWN_TYPE
model_file	设置模型定义文件的位置，数据类型为字符串
model_weights	设置模型权重文件的位置，数据类型为字符串
caffe_proto_type	设置解析 caffe 模型的 python 文件位置，数据类型为字符串，默认值为“caffe”（代表调用 ATCNN Toolkit 默认的 caffe 模型解析文件）
input_node	设置模型输入节点描述信息，可多次定义，对应模型多个输入节点，请参见表 49-11
output_model_stage1_name	设置保存量化的 atcnn 模型文件的位置，数据类型为字符串，若不指定，程序将自动填补
output_model_stage2_name	设置保存 Hardware-Friendly 的 atcnn 模型文件的位置，数据类型为字符串，若不指定，程序将自动填补
output_model_stage3_name	设置保存量化 Binary 的 atcnn 模型文件的位置，数据类型为字符串，若不指定，程序将自动填补

表 49-11 输入节点描述信息(input_node)

属性名称	属性说明
layer_name	设置节点名称，用于指定模型输入层的名字，输入层若只有一个输入可不配置节点名称，默认值为“unknown”
shape	设置输入节点的 shape 信息，可多次定义，对应不同的输入 tensor 不同维度的输入大小，数据类型为整型
size_with_batch	设置输入节点的 shape 定义是否包含 batch 维度信息，默认值为 false，数据类型为布尔型

49.4.1.2.1.3 模型预处理信息(atc_preproc_config)

表 49-12 预处理节点描述信息

属性名称	属性说明
preproc_node	设置模型输入节点预处理信息，可多次定义，用于对应模型多个输入节点，请参见表 49-13
add_pre_to_net	设置是否将预处理信息保存到网络
add_post_to_net	设置是否将后处理转换 float32 输出保存到网络，默认值为 true

表 49-13 预处理节点具体操作描述信息

属性名称	属性说明
layer_name	设置模型输入节点名称，即该预处理节点关联的输入 tensor，若模型只有一个输入，可不配置，数据类型为字符串，默认值为“unkown”
mean	设置预处理均值，可多次定义，用于对应输入节点不同通道均值。
scale	设置预处理方差，可多次定义，用于对应输入节点不同通道方差。其和均值 mean 一起搭配的公式如下(这里 x 代表输入，y 代表预处理后的输出： $y = (x - mean) * scale$
reverse_channel	设置模型输入节点的通道是否反转，ATCNN-Toolkit 读入图片的方式默认格式为 RGB，若模型输入为 BGR 格式，则该选项需设置为 true，默认

属性名称	属性说明
	值为 false,
scale_type	设置数据缩放方式，目前只支持非等比例缩放，默认值为非等比例缩放(Non_Equal)
sdk_input_format	设置板端推理的数据类型，目前支持 ATCNN_FORMAT_TENSOR、 ATCNN_FORMAT_IMAGE_GRAY、 ATCNN_FORMAT_IMAGE_RGB、 ATCNN_FORMAT_IMAGE_YUV420、 ATCNN_FORMAT_IMAGE_NV12、 ATCNN_FORMAT_IMAGE_NV21、 ATCNN_FORMAT_DEFAULT，默认值为 ATCNN_FORMAT_DEFAULT
resizeProc	设置板端预处理 resize 参数，请参见 缩放节点信息
cropProc	设置板端预处理 crop 参数，请参见 裁剪节点信息

(1) 缩放节点信息(resizeProc)

表 49-14 预处理节点之缩放操作描述信息

属性名称	属性说明
resize_type	设置缩放类型，目前仅支持 ATCNN_IMAGE_RESIZE_BILINEAR
src_shape	设置输入 tensor 宽高信息，如 src_shape{ dim: 256 dim: 256 }

(2) 裁剪节点信息(cropProc)

表 49-15 预处理节点之裁剪操作描述信息

属性名称	属性说明
enable_preproc_crop	设置缩放功能是否开启，默认值为 False
crop_rect	设置 crop 信息，描述方式为起始点坐标(x,y)及宽高(w, h)，op_rect{ dim: 0 dim: 0 dim: 256 dim: 256 }

49.4.1.2.1.4 网络优化描述信息(atc_opt_quantization_config)

表 49-16 网络优化项描述信息

属性名称	属性说明
dataset_node	设置量化数据集信息，请参见表 49-17
opt_prune_node	设置剪枝方法信息，请参见表 49-18
quantization_node	设置量化方法信息，请参见表 49-19

表 49-17 量化数据集描述信息(dataset_node)

属性名称	属性说明
path	设置量化数据库描述文件位置，数据类型为字符串
type	设置数据描述文件解析类型，目前仅支持文本格式(TEXT)，默认值为 TEXT。toolKit 支持 npy 格式以及 jpg、png 等图像格式的数据输入

表 49-18 输入数据描述信息(node_info)

属性名称	属性说明
layer_name	设置关联模型输入的 tensor，当模型只有一个输入时可不配置，数据类型为字符串
dtype	设置数据解析类型，可配置 INT8、INT16、UINT8、UINT16、INT32、FLOAT16、FLOAT32，默认值为 FLOAT32
node_decode_type	设置解码方式，可配置 IMAGE_RGB、IMAGE_RGB888_PLANAR、IMAGE_YUV420、IMAGE_GRAY、IMAGE_BGRA、IMAGE_TENSOR、NPY、UN_TYPE，默认值为 IMAGE_RGB

表 49-19 剪枝方法描述(opt_prune_node)

属性名称	属性说明
do_prune_flag	设置模型是否进行剪枝操作，默认值为 false
prune_percent	设置裁剪比例，值域为 0~100 之间，该比例设置对模型的每一层都有效，默认值为 0
prune_type	设置裁剪方法，可配置 ELEMENT、VECTOR、KERNEL、FILTER，默认值为 ELEMENT

属性名称	属性说明
prune_config	设置剪枝描述文件位置，剪枝描述文件可定义不同层不同的裁剪比例，不可与 prune_percent 同时使用，数据类型为字符串
prune_priority	设置剪枝方式优先级，可配置 FILE_FIRST（使用 prune_config 配置进行剪枝）、PARAMETER_FIRST（使用 prune_percent 配置进行剪枝），默认值为 PARAMETER_FIRST

表 49-20 量化方法描述(quantization_node)



注意：

对于预训练模型为已量化的 tensorflow 或者 tensorflow-lite 模型，请勿使用 ATCNN ToolKit 再次量化。

属性名称	属性说明
do_quantize_flag	设置是否进行量化，默认值为 true，数据类型为布尔型
batch_size	设置量化过程中推理的数据条目数，默认值为 1
Iterations	设置量化过程中推理的次数，默认值为 1
device_type	设置运行设备类型，目前仅支持 CPU，默认值为 CPU
quan_method	设置量化方法，可配置 asymmetric_affine_u8、dynamic_fixed_point_i8、dynamic_fixed_point_i16、tf_quantized、perchannel_symmetric_affine_i8，默认值为 asymmetric_affine_u8（注:其中当输入的为量化了的模型，则设置量化方法为 tf_quantized）
quantized_file	设置量化描述文件位置，量化描述文件可定义不同层不同的量化方法，不可与 quan_method 一起使用，数据类型为字符串
quan_observer	设置量化统计方式，可配置 NORMAL、KL_DIVERGENCE 和

属性名称	属性说明
	MOVING_AVERAGE, 默认值为 NORMAL
quan_hybrid	设置是否进行混合量化, 默认值为 false
hybrid_layers	定义混合量化层, 请参见 混合量化层描述信息
save_quantized_file	设置是否保存量化信息, 默认值为 false

表 49-21 混合量化层描述信息

属性名称	属性说明
item	可重复定义, 单个层的量化描述信息见 单层量化描述信息

表 49-22 单层量化描述信息

属性名称	属性说明
layer_name	字符串类型, 定义待量化的层名
qtype	量化类型, 可配置 asymmetric_affine_u8、dynamic_fixed_point_i8、dynamic_fixed_point_i16、tf_quantized、perchannel_symmetric_affine_i8

49.4.1.2.2 模型转换

ATCNN-Toolkit 的模型转换功能将不同 AI 框架的模型转换为端测（SOC 芯片）能正确加载推理的 atcnn 模型, 如图 49-3 所示。

49.4.1.2.3 模型转换配置文件示例说明

(1) 模型导入设置

- Caffe 模型导入配置

表 49-23 Caffe 模型导入配置

参数名称	参数说明
model_type	必填项, 设置为 CAFFE
model_file	必填项, 设置为模型定义文件, 如图 49-8 所示 example/lenet.prototxt
model_weights	可选项, 设置为模型权重文件, 如图 49-8 所示

参数名称	参数说明
	example/lenet.caffemodel。若不配置，程序会随机生成 weights 值供调试
caffe_proto_type	可选项，设置为解析 caffe 模型的文件路径，若不配置，则默认调用 ATCNN Toolkit 自带的解析文件解析 caffe 模型

```
atcnn_config {
  target_hardware: AC8025
  atc_net_config {
    model_type: CAFFE
    model_file: "example/lenet.prototxt"
    model_weights: "example/lenet.caffemodel"
  }
}
```

图 49-8 Caffe 模型配置示例

■ ONNX 模型导入配置

表 49-24 ONNX 模型导入配置

参数名称	参数说明
model_type	必填项，设置为 ONNX
model_file	必填项，设置为模型文件，如 图 49-9 所示 example/lenet.onnx。

```
atcnn_config {
  target_hardware: AC8025
  atc_net_config {
    model_type: ONNX
    model_file: "example/lenet.onnx"
  }
}
```

图 49-9 ONNX 模型配置示例

■ Tensorflow-Lite 模型导入配置

表 49-25 Tensorflow-Lite 模型导入配置

参数名称	参数说明
model_type	必填项，设置为 TF_LITE
model_file	必填项，设置为模型文件，如 图 49-10 所示 example/lenet.tflite。（注：Tensorflow-Lite 建议版本为 Tensorflow 2.12.0 原生版本）

```
atcnn_config {
  target_hardware: AC8025
  atc_net_config {
    model_type: TF_LITE
    model_file: "example/lenet.tflite"
  }
}
```

图 49-10 Tensorflow-Lite 模型配置示例

- Darknet 模型导入配置

表 49-26 Darknet 模型导入配置

参数名称	参数说明
model_type	必填项，设置为 DARKNET
model_file	必填项，设置为模型文件，如 图 49-11 所示 example/lenet.cfg
model_weights	必填项，设置为模型权重文件，如 图 49-11 所示 example/lenet.weights

```
atcnn_config {
  target_hardware: AC8025
  atc_net_config {
    model_type: DARKNET
    model_file: "example/lenet.cfg"
    model_weights: "example/lenet.weights"
  }
}
```

图 49-11 Darknet 模型配置示例

- Keras 模型导入配置

表 49-27 Keras 模型导入配置

参数名称	参数说明
model_type	必填项，设置为 KERAS
model_file	必填项，设置为模型文件，如 图 49-12 所示 example/lenet.h5。

```
atcnn_config {
  target_hardware: AC8025
  atc_net_config {
    model_type: KERAS
    model_file: "example/lenet.h5"
  }
}
```

图 49-12 Keras 模型配置示例

Tensorflow 模型导入配置

表 49-28 Tensorflow 模型导入配置

参数名称	参数说明
model_type	必填项，设置为 TENSORFLOW
model_file	必填项，设置为模型文件，如 图 49-13 所示 example/lenet.pb。（注：建议 tensorflow 模型的输入层类型需为 Placeholder）

```
atcnn_config {
  target_hardware: AC8025
  atc_net_config {
    model_type: TENSORFLOW
    model_file: "example/lenet.pb"
  }
}
```

图 49-13 Tensorflow 模型配置示例

Pytorch 模型导入配置

表 49-29 Pytorch 模型导入配置

参数名称	参数说明
------	------

参数名称	参数说明
model_type	必填项，设置为 PYTORCH
model_file	必填项，设置为模型文件，如图 49-14 所示 example/lenet.pt。（Pytorch 模型必须来源于 torch.jit.trace()保存的 pt 模型）
input_node	必填项，设置模型输入层的信息，如 shape 信息、layer_name 信息、layout 信息等。



注意：

Pytorch 模型必须来源于 torch.jit.trace()

```
atcnn_config {
  target_hardware: AC8025
  atc_net_config {
    model_type: PYTORCH
    model_file: "example/lenet.pt"
    input_node {
      layer_name: "input"
      shape: 1
      shape: 1
      shape: 28
      shape: 28
      size_with_batch: true
    }
  }
}
```

图 49-14 Pytorch 模型配置示例

(2) 模型预处理设置

表 49-30 模型预处理设置示例

参数名称	参数说明
preproc_node	可选项，设置预处理节点，请参见图 49-12
add_preproc_to_net	可选项，当设置为 True，会将设置的预处理加入网络生成最终二进制模型，默认值为 False

```
atc_preproc_config {
  preproc_node {
    reverse_channel: true
    mean: 0
    mean: 0
    mean: 0
    scale: 0.0039
    scale: 0.0039
    scale: 0.0039
    sdk_input_format: ATCNN_FORMAT_IMAGE_NV12
    resizeProc {
      resize_type: ATCNN_IMAGE_RESIZE_BILINEAR
      src_shape {
        dim: 289
        dim: 289
      }
    }
    cropProc {
      enable_preproc_crop: true
      crop_rect {
        dim: 0
        dim: 0
        dim: 256
        dim: 256
      }
    }
  }
  add_pre_to_net: true
}
```

图 49-15 yolov3 检测网络预处理参数设置

(3) 模型剪枝设置

表 49-31 模型剪枝设置示例

参数名称	参数说明
do_prune_flag	可选项，设置是否开启剪枝功能，默认值为 false
prune_percent	可选项，设置剪枝比例，值域为 0~100，对模型所有层有效，默认值为 0，如图 49-16 所示
prune_type	可选项，设置剪枝方法，可配置 ELEMENT、VECTOR、KERNEL、FILTER，默认值为 ELEMENT
prune_config	可选项，设置剪枝描述文件位置
prune_priority	可选项，设置剪枝方式优先级，可配置 FILE_FIRST、PARAMETER_FIRST，默认值为 PARAMETER_FIRST

```
opt_prune_node {
  do_prune_flag: true
  prune_percent: 5.0
  prune_type: ELEMENT
  prune_config: "default_prune.txt"
  prune_priority: PARAMETER_FIRST
}
```

图 49-16 模型剪枝示例

(4) 模型量化配置

模型量化配置一般需要配置两个选项：量化数据库信息和量化方法信息。

■ 量化数据库配置

表 49-32 量化数据库配置示例

参数名称	参数说明
path	必选项，设置量化数据库描述文件位置，模型描述文件格式如图 49-17 所示，当模型有多个输入时，用空格隔开。
type	可选项，默认值为 TEXT
node_info	可选项，可多次定义，当模型有多个输入时候需配置 layer_name，当模型为单输入时且输入类型为 IMAGE_RGB 时可不配置。

```
dataset_node {
  path: "/workspace/dataset.txt"
  type: TEXT
  node_info {
    layer_name: "input"
    dtype: FLOAT32
  }
}
```

图 49-17 模型量化数据集配置示例

```
datasets/ILSVRC2012_val_00000001.JPEG datasets/ILSVRC2012_val_00000001.JPEG
datasets/ILSVRC2012_val_00000002.JPEG datasets/ILSVRC2012_val_00000002.JPEG
datasets/ILSVRC2012_val_00000003.JPEG datasets/ILSVRC2012_val_00000003.JPEG
datasets/ILSVRC2012_val_00000004.JPEG datasets/ILSVRC2012_val_00000004.JPEG
datasets/ILSVRC2012_val_00000005.JPEG datasets/ILSVRC2012_val_00000005.JPEG
datasets/ILSVRC2012_val_00000006.JPEG datasets/ILSVRC2012_val_00000006.JPEG
datasets/ILSVRC2012_val_00000007.JPEG datasets/ILSVRC2012_val_00000007.JPEG
datasets/ILSVRC2012_val_00000008.JPEG datasets/ILSVRC2012_val_00000008.JPEG
datasets/ILSVRC2012_val_00000009.JPEG datasets/ILSVRC2012_val_00000009.JPEG
datasets/ILSVRC2012_val_00000010.JPEG datasets/ILSVRC2012_val_00000010.JPEG
```

图 49-18 多输入模型数据集描述文件示例

■ 量化方法描述

表 49-33 量化方式设置示例

参数名称	参数说明
quan_method	可选项，可配置 asymmetric_affine_u8、dynamic_fixed_point_i8、dynamic_fixed_point_i16、tf_quantized、perchannel_symmetric_affine_i8，默

参数名称	参数说明
	默认值为 asymmetric_affine_u8
quan_observer	可选项，设置量化统计方式，可配置 NORMAL、KL_DIVERGENCE 和 MOVING_AVERAGE，默认值为 NORMAL
quantized_file	可选项，设置量化描述文件位置
quan_hybrid	可选项，默认值为 false
hybrid_layers	可选项，设置混合量化层信息
batch_size	可选项，默认值为 1
iterations	可选项，默认值为 1（注：该参数与 batch_size 相乘代表总的量化数据集大小）
moving_average_weight	可选项，一般与量化方法 MOVING_AVERAGE 一起使用，默认值为 0.01

```
quantization_node {
  divergence_bins: 0
  quantized_file: "atcnn_quantized_info.quantize"
  batch_size: 1
  iterations: 10
  device_type: CPU
  quan_method: asymmetric_affine_u8
  quan_hybrid: true
  quan_observer: NORMAL
  moving_average_weight: 0.009999999776482582
  hybrid_layers {
    item {
      layer_name: "Conv_conv2"
      qtype: dynamic_fixed_point_i16
    }
    item {
      layer_name: "Conv_conv1"
      qtype: dynamic_fixed_point_i16
    }
  }
  save_quantized_file: true
}
```

图 49-19 模型量化信息配置示例

49.4.1.2.4 模型转换执行命令

模型转换调用的是 `atcnn_cmd.py` 来执行，具体执行命令如下：

```
python3 atcnn_cmd.py build --config xxx.prototxt --gen_cmd
```

其中 `xxx.prototxt` 为模型转换的配置文件信息。执行完成后若配置文件未设置 `atcnn` 保存路径，则会在模型目录下保存转换好的 `atcnn` 模型。包含三个 `atcnn` 模型，分别为 `xxx.atcnn`、`xxx_board.atcnn` 和 `xxx_board_binary.atcnn`，其中 `xxx.atcnn` 主要用于 PC 端的推理，`xxx_board.atcnn` 和 `xxx_board_binary.atcnn` 则用于板端(SOC 芯片)的 SDK 调用，两者推理的结果基本相同（注：PC 端采用伪量化(fake quantization)方式推理，实际为 float 表达和板端的推理会有些许差异）。同时在前执行目录下保存模型量化描述文件(`atcnn_quantized_info.quantize`)，该文件可供混合量化使用；以及保存了 `normalization` 的配置文件，`xxx_normalization.prototxt`，相较于原始的配置文件(`xxx.prototxt`)，填补一些缺省值，比如 `atcnn` 模型保存的位置。`gen_cmd` 参数作为可选项，主要用于生成二进制指令模型，若在转换中无此选项则上板推理时会在板端进行编译生成对应的二进制指令。

转换过程中会打印每一层的精度信息，如图 49-20 所示，比较原始 float 模型的每一层激活值输出和转换后的模型对应层的激活值输出的欧式距离和余弦距离（注：若模型结构中存在 `conv+relu` 或者 `fully connected+relu` 结构时，打印的 `conv` 或者 `fully connected` 的精度会存在误差，请以 `relu` 层输出信息为准。该精度信息是抽取量化数据库中单个输入样本进行逐层比较分析的结果，任务实际的转换精度还需使用对应的测试集来反映）。

```
Quantization Error -----
data_input          eculidean=0.000000 cosine=1.000000
Conv_conv1          eculidean=0.011757 cosine=0.999931
MaxPool_pool1       eculidean=0.012162 cosine=0.999926
Conv_conv2          eculidean=0.012182 cosine=0.999926
MaxPool_pool2       eculidean=0.014309 cosine=0.999898
Reshape_ip1_Reshape eculidean=0.014309 cosine=0.999898
Gemm_ip1            eculidean=0.802898 cosine=0.677678
Relu_relu1          eculidean=0.012125 cosine=0.999926
Gemm_ip2            eculidean=0.007179 cosine=0.999974
Softmax_prob        eculidean=0.000004 cosine=1.000000
attach_Softmax_prob eculidean=0.000004 cosine=1.000000
End -----
```

图 49-20 lenet 模型转换过程打印的逐层精度信息

49.4.1.3 模型推理

模型推理主要用于比较转换后的模型推理结果与转换前的模型推理差异（余弦相似度差异及欧式距离差异）。

49.4.1.3.1 模型推理配置文件说明

模型推理一般使用的配置文件是模型转换后生成 `xxx_normalization.prototxt`，该配置文件包含转换后的模型文件位置等信息。模型推理功能调用的模型为 `xxx.atcnn`，同时也会读取配置文件中原始模型的信息及其相应的预处理。

49.4.1.3.2 模型推理执行命令

模型推理调用的是 `atcnn_cmd.py` 来执行，具体执行命令如下：

```
python3 atcnn_cmd.py inference --config xxx_normalization.prototxt --image xxx.jpg
```

其中 `--config` 指向配置文件，`--image` 则指向要测试的图片，执行完成后打印模型所有层的余弦相似度信息及欧式距离信息（如图 49-21 示例所示），并在当前目录下构建新的目录 `debug_atcnn`，`debug_atcnn` 目录下又包含两个子目录（`float` 目录、`quantize_net` 目录）分别用于保存原始模型及转换后的模型输入输出层的 `tensor` 值。

```
/workspace/atcnn_toolkit_v0.14 python atcnn_converter.py inference --config example_normalization.prototxt --image example/dataset/9.png
2022-12-08 06:56:41.370549: W tensorflow/stream_executor/platform/default/dso_loader.cc:64] Could not load dynamic library 'libcudart.so.11.0'; dlopen: libcudart
ocal/nvidia/lib:/usr/local/nvidia/lib64
2022-12-08 06:56:41.370575: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart dlerror if you do not have a GPU set up on your machine.
1.0
Beginning to load the configuration in the filepath example_normalization.prototxt
Load model file example/lenet.onnx
WARNING: ONNX Optimizer has been moved to https://github.com/onnx/optimizer.
All further enhancements and fixes to optimizers will be done in this new repo.
The optimizer code in onnx/onnx repo will be removed in 1.9 release.
Load model file example/lenet_stage1.atcnn
2022-12-08 06:56:43.654926: W tensorflow/stream_executor/platform/default/dso_loader.cc:64] Could not load dynamic library 'libcudart.so.11'; dlopen: libcudart.so.1
y/lib:/usr/local/nvidia/lib64
2022-12-08 06:56:43.654953: W tensorflow/stream_executor/cuda/cuda_driver.cc:269] failed call to cuInit: UNKNOWN ERROR (303)
2022-12-08 06:56:43.654977: I tensorflow/stream_executor/cuda/cuda_diagnostics.cc:163] no NVIDIA GPU device is present: /dev/nvidia0 does not exist
2022-12-08 06:56:43.655327: I tensorflow/core/platform/cpu_feature_guard.cc:142] This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (on
FMA
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
W tensorflow/AutoGraph could not transform <acuitylib.core.execution.BackendFunction object at 0x7f06c36eca20> and will run it as-is.
Please report this to the TensorFlow team. When filing the bug, set the verbosity to 10 (on Linux, 'export AUTOGRAPH_VERBOSITY=10') and attach the full output.
Cause: Non-function: <class 'cython_function_or_method'>
To silence this warning, decorate the function with @tf.autograph.experimental.do_not_convert
2022-12-08 06:56:43.732592: I tensorflow/compiler/mlir/mlir_graph_optimization_pass.cc:185] None of the MLIR Optimization Passes are enabled (registered 2)
2022-12-08 06:56:43.740759: W tensorflow/core/grappler/costs/op_level_cost_estimator.cc:689] Error in PredictCost() for the op: "Softmax" attr { key: "T" va
: "CPU" vendor: "GenuineIntel" model: "101" frequency: 3199 num_cores: 32 environment { key: "cpu_instruction_set" value: "AVX SSE, SSE2, SSE3, SSSE3, SSE4.1, S
576 kb cache size: 25952256 memory size: 268435456 } outputs { dtype: DT_FLOAT shape { unknown rank: true } }
W tensorflow/AutoGraph could not transform <acuitylib.core.execution.BackendFunction object at 0x7f06c2db5be0> and will run it as-is.
Please report this to the TensorFlow team. When filing the bug, set the verbosity to 10 (on Linux, 'export AUTOGRAPH_VERBOSITY=10') and attach the full output.
Cause: Non-function: <class 'cython_function_or_method'>
To silence this warning, decorate the function with @tf.autograph.experimental.do_not_convert
2022-12-08 06:56:43.948063: W tensorflow/core/grappler/costs/op_level_cost_estimator.cc:689] Error in PredictCost() for the op: "Softmax" attr { key: "T" va
: "CPU" vendor: "GenuineIntel" model: "101" frequency: 3199 num_cores: 32 environment { key: "cpu_instruction_set" value: "AVX SSE, SSE2, SSE3, SSSE3, SSE4.1, S
576 kb cache size: 25952256 memory size: 268435456 } outputs { dtype: DT_FLOAT shape { unknown rank: true } }
Quantization Error
data_input                                ecclidean=0.000000 cosine=1.000000
conv_conv1                                ecclidean=0.014338 cosine=0.999897
MaxPool_pool1                             ecclidean=0.015965 cosine=0.999887
conv_conv2                                ecclidean=0.015904 cosine=0.999874
MaxPool_pool2                             ecclidean=0.018624 cosine=0.999826
Reshape_ip1 Reshape                       ecclidean=0.018624 cosine=0.999826
gemm_ip1                                  ecclidean=0.700017 cosine=0.754988
Relu_relu1                                ecclidean=0.015926 cosine=0.999873
gemm_ip2                                  ecclidean=0.009661 cosine=0.999953
softmax_prob                              ecclidean=0.000623 cosine=1.000000
attach_softmax_prob/out0                  ecclidean=0.000023 cosine=1.000000
End
Inference done!
```

图 49-21 lenet 推理示例

49.4.1.4 模型性能分析

模型性能分析可评估模型每一层的运算量(MAC 数为计量单位)，整个模型的总体运算量，以及模型参数规模和输出层大小。

49.4.1.4.1 模型性能分析执行命令

模型性能分析调用 `atcnn_cmd.py` 来执行，具体的执行命令如下：

```
python3 atcnn_cmd.py eval --config xxx_normalization.prototxt
```

其中 `--config` 指向配置文件，运行完后打印模型的 MAC 数信息（如图 49-22 所示）。


```
workspace/atcnn_toolkit_v0.16 python atcnn_converter.py eval --config example_normalization.prototxt
2022-12-08 07:18:20.994443: I tensorflow/stream_executor/platform/default/dso_loader.cc:44] Could not load dynamic library 'libcudart.so.11.0': dlopen: libcudart.so.11.0: cannot open shared object file: No such file or
local/nvidia/lib:/usr/local/nvidia/lib64
2022-12-08 07:18:20.994473: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart dlerror if you do not have a GPU set up on your machine.
1.9
Beginning to load the configuration in the filepath example_normalization.prototxt
Load model file example/lenet_stage1.atcnn
=====performance=====
layer      input shape      output shape      macc      param      activation
Conv conv1_9 acuity_mark_perm_12 preprocess_13
[1, 1, 28, 28] [1, 28, 28, 1] 784 0 0
[1, 1, 28, 28] [1, 28, 28, 1] 784 0 784
Reshape ip1 Reshape_5 acuity_mark_perm_11
[1, 4, 4, 50] [1, 50, 4, 4] 800 0 800
data input 10
[1, 1, 28, 28] [1, 1, 28, 28] 0 0 784
Conv conv1_9
[1, 28, 28, 1] [1, 24, 24, 20] 288000.0 500.0 11520
MaxPool pool1 8
[1, 24, 24, 20] [1, 12, 12, 20] 0 0 2880
Conv conv2_7
[1, 12, 12, 20] [1, 8, 8, 50] 160000.0 2500.0 3200
MaxPool pool2 6
[1, 8, 8, 50] [1, 4, 4, 50] 0 0 800
Reshape ip1 Reshape_5
[1, 50, 4, 4] [1, 800] 800 0 800
Gemm ip1 4
[1, 800] [1, 500] 40000 500 500
Relu relu1 3
[1, 500] [1, 500] 500 0 500
Gemm ip2 2
[1, 500] [1, 10] 5000 5000 10
Softmax prob 1
[1, 10] [1, 10] 0 0 10
attach_Softmax_prob/out0_0
sum_macc:2.3M sum_param:420.41K sum_activation:22.07K
=====performance end=====
eval performance done!
```

图 49-22 lenet 性能分析示例

49.4.2 ATCNN C++接口基本调用流程

- (1) 准备 xxx_board.atcnn 或者 xxx_board_binary.atcnn 模型文件;
- (2) 调用 atcnnCreateAdapter 接口创建设备环境并加载 atcnn 模型，代码如下:

```
atcnn_return_value status;
atcnn_adapter adapter;

status =
atcnnCreateAdapter(&adapter, "./mobilenet_v1_1_0_224_frozen_board_binary.atcnn", false);

if( ATCNN_SUCCESS != status){
printf("atcnnCreateAdapter fail! Status = %d\n", status);
goto Error;
}
```

- (3) atcnn 模型的 input/output 属性可能已经和原始预训练模型不同，因此需要调用 atcnnGetInfo 接口获取 input 和 output 属性（若已知网络结构，此为非必选项）。代码如下:

```
atcnn_return_value status;
atcnn_sdk_version version;

status = atcnnGetInfo(&adapter, ATCNN_SDK_VERSION, &version,
sizeof(atcnn_sdk_version));

if( ATCNN_SUCCESS != status) {
printf("atcnn get version info fail! Status = %d\n", status);
goto Error;
}

printf("apiVersion: %s, drvVersion: %s, toolkitVersion: %s\n",
version.apiVersion.c_str(),
version.drvVersion.c_str(), version.toolKitVersion.c_str());

atcnn_tensor_in_out_num inoutNum;

status = atcnnGetInfo(&adapter, ATCNN_IN_OUT_NUM, &inoutNum,
sizeof(atcnn_tensor_in_out_num));
```

```
if( ATCNN_SUCCESS != status){
    printf("atcnn get in_out_num info fail!  Status = %d\n", status);
    goto Error;
}
printf("n_input: %d, n_output: %d\n",inoutNum.n_input,inoutNum.n_output);

atcnn_tensor_attr inputAttr[1]; //此处值需>= inoutNum.n_input
for(int i=0;i<inoutNum.n_input;i++){
    inputAttr[i].index = i;
    status = atcnnGetInfo(&adapter, ATCNN_INPUT_ATTR,  &inputAttr[i],
        sizeof(atcnn_tensor_attr));
        TEST_CHECK_STATUS( status, "atcnnGetInfo-ATCNN_INPUT_ATTR");
    if( ATCNN_SUCCESS != status){
        printf("atcnn get input tensor info fail!  Status = %d\n", status);
        goto Error;
    }
    printf("inputAttr[%d].index: %d, inputAttr[%d].n_dims: %d\n",i,
        inputAttr[i].index,i,inputAttr[i].n_dims);
}

atcnn_tensor_attr outputAttr[1]; //此处值需>= inoutNum.n_input
for(int i=0;i<inoutNum.n_output;i++){
    outputAttr[i].index = 0;
    status = atcnnGetInfo(&adapter, ATCNN_OUTPUT_ATTR,  &outputAttr[i],
        sizeof(atcnn_tensor_attr));
    if( ATCNN_SUCCESS != status){
        printf("atcnn get output tensor info fail!  Status = %d\n", status);
        goto Error;
    }
    printf("outputAttr[%d].index: %d, outputAttr[%d].n_dims: %d\n",i,
        outputAttr[i].index, i, outputAttr[i].n_dims);
}
```

(4) 根据模型 input 参数的具体格式，调用 atcnnSetInputs 进行设置，代码如下：

```
atcnn_data data[inoutNum.n_input];// inoutNum.n_input 此处仅为示例
data[0].data = graydata;
data[0].dims[0] = width;
data[0].dims[1] = height;
```

```
data[0].dims[2] = channel;
data[0].dims[3] = 1;
data[0].n_dims = 4;//nchw
data[0].dataType = ATCNN_TYPE_INT8;
data[0].layout = ATCNN_DIM_NHWC;
status = atcnnSetInputs(&adapter, data, inoutNum.n_input);
if( ATCNN_SUCCESS != status){
    printf("atcnn set input fail! Status = %d\n", status);
    goto Error;
}
```

此处输入的图像格式与 toolkit 生成 atcnn 模型时设置的图像格式需保持一致。

(5) 执行推理

```
status = atcnnRun (&adapter, false, false, false);
if( ATCNN_SUCCESS != status){
    printf("atcnn run fail! Status = %d\n", status);
    goto Error;
}
```

(6) 待推理完毕，调用 atcnnGetOutputs 获取推理结果，对获取的结果可根据实际需求进行其它后处理操作：

```
atcnn_data outputData[inoutNum.n_output];//此处仅为示例
status = atcnnGetOutputs(outputData,inoutNum.n_output);
if( ATCNN_SUCCESS != status){
    printf("atcnn get result fail! Status = %d\n", status);
    goto Error;
}
```

(7) 循环执行(4)、(5)、(6)三步；

(8) 待所有图像处理完毕，调用 atcnnDestroy 接口释放 NPU 资源。

```
status = atcnnDestroy();
if( ATCNN_SUCCESS != status){
    printf("release npu resource fail! Status = %d\n", status);
    goto Error;
}
```

(9) 调整日志输出级别：SDK 支持四种级别的日志输出：D(debug)、I(information)、W(warning)、E(error)。其中，D 级别包含 I、W、E 级别的日志，I 级别包含 W、E 级别的日志，W 级别包含 E 级别的日志。日志级别设置需在应用启动前设置，设置方式示例如下：

Android 系统: setprop vendor.ATC_NN_LOG_LEVEL D

Linux 系统: export ATC_NN_LOG_LEVEL=D

49.4.3 ATCNN Service 接口基本调用流程

49.4.3.1 ATCNN Service C++接口基本调用流程

- (1) 准备 xxx_board.atcnn 或者 xxx_board_binary.atcnn 模型文件;
- (2) 调用 loadModel 接口创建设备环境并加载 atcnn 模型, 代码如下:

```
int mModelId = -1;
FILE* fp = fopen("./xxx_board_binary.atcnn", "rb");
if (fp == NULL) {
    LOGE("open ./xxx_board_binary.atcnn fail.");
    return;
}
fseek(fp, 0, SEEK_END);
long fileSize = ftell(fp);
LOGI("model size: %ld", fileSize);
char* modelBuffer = (char*)malloc(fileSize);
fseek(fp, 0, SEEK_SET);
fread(modelBuffer, 1, fileSize, fp);
fclose(fp);

AtcnnInterpreter interpreter;
int modelId = -1;
//interpreter.loadModel(modelId, "./xxx_board_binary.atcnn", true);
interpreter.loadModel(modelId, modelBuffer, true, fileSize);
LOGD("modelId(%d)", modelId);
if (modelId < 0) {
    return false;
}
```

- (3) atcnn 模型的 input/output 属性可能已经和原始预训练模型不同, 因此需要调用相关 get 接口来获取 input 和 output 属性 (若已知网络结构, 此为非必选项)。代码如下:

```
atcnn_tensor_in_out_num number;
interpreter.getInOutNum(modelId, number);
LOGI("n_input(%d) n_output(%d)", number.n_input, number.n_output);
```

```
for (int i = 0; i < number.n_input; i++) {
    atcnn_tensor_attr input;
    interpreter.getInputTensorAttr(modelId, i, input);
}
...
```

(4) 根据模型 input 参数的具体格式，调用 setInputData 进行设置，代码如下：

```
std::vector<atcnn_data> inputs;
for (int i = 0; i < number.n_input; i++) {
    atcnn_tensor_attr input;
    interpreter.getInputTensorAttr(modelId, i, input);

    atcnn_data data;
    for (int j = 0; j < ATCNN_MAX_DIMS; j++) {
        data.dims[j] = input.dims[j];
    }
    data.n_dims = input.n_dims;
    data.layout = input.layout;
    data.dataType = atcnn_tensor_type::ATCNN_TYPE_INT8;
    inputs.push_back(data);
}
interpreter.setInputData(modelId, inputs);
```

此处输入的图像格式与 toolkit 生成 atcnn 模型时设置的图像格式需保持一致。

(5) 执行推理

```
interpreter.run(modelId);
```

(6) 待推理完毕，再通过 getOutputData 获取推理结果

```
std::vector<atcnn_data> outputs;
interpreter.getOutputData(modelId, number.n_output, &outputs);
for (int i = 0; i < (int)outputs.size(); i++) {
    std::string name = "./result_.txt";
    std::ofstream file(name.c_str());
    if (file.is_open()) {
        int shared_size = 0;
        switch ((int)outputs[i].dataType) {
            case (int)atcnn_tensor_type::ATCNN_TYPE_INT8:
            case (int)atcnn_tensor_type::ATCNN_TYPE_UINT8:
```

```

        shared_size = 1;
        break;
    case (int)atcnn_tensor_type::ATCNN_TYPE_INT16:
    case (int)atcnn_tensor_type::ATCNN_TYPE_FLOAT16:
        shared_size = 2;
        break;
    case (int)atcnn_tensor_type::ATCNN_TYPE_FLOAT32:
        shared_size = 4;
        break;
    }
    for (int j = 0; j < outputs[i].n_dims; j++) {
        shared_size *= outputs[i].dims[j];
    }
    shared_size /= 4;
    LOGI("outputs(%d) size(%d)", i, shared_size);
    float* floatArray = reinterpret_cast<float*>(outputs[i].data);
    for (int j = 0; j < shared_size; j++) {
        file << floatArray[j] << " ";
    }
    file.close();
}
}

```

(7) 循环执行(4)、(5)、(6)三步;

(8) 待所有图像处理完毕, 调用 `releaseModel` 接口释放资源。

49.4.3.2 ATCNN Service Java 接口基本调用流程

- (1) 准备 `xxx_board.atcnn` 或者 `xxx_board_binary.atcnn` 模型文件;
- (2) 调用 `getInstance` 接口获取 `atcnn` 模型解释器;
- (3) 调用 `setContext` 接口, 设置上下文环境 (若模型文件不在本地 `Assets` 资源包中, 则无需设置);
- (4) 调用 `loadModel` 接口创建设备环境并加载 `atcnn` 模型, 代码如下;

```

int mModelId =
mAtcnnInterpreter.loadModel("mobilenet_v1_1_0_224_frozen_board_binary.atcnn",
false, true);
if (mModelId < 0) {
    Log.d(TAG, "load model fail");
}

```

```
return;  
};
```

- (5) atcnn 模型的 input/output 属性可能已经和原始预训练模型不同，因此需要调用相关 get 接口来获取 input 和 output 属性（若已知网络结构，此为非必选项）。代码如下；

```
Atcnn_sdk_version version = mAtcnnInterpreter.getSDKVersion(mModelId);  
Log.d(TAG, "apiVersion:" + version.apiVersion + ", drvVersion:" +  
version.drvVersion + ", toolKitVersion:" + version.toolKitVersion);  
  
Atcnn_tensor_in_out_num inoutNum = mAtcnnInterpreter.getInOutNum(mModelId);  
Log.d(TAG, "n_input:" + inoutNum.n_input + ", n_output:" + inoutNum.n_output);  
...
```

- (6) 根据模型 input、output 参数的具体格式，调用 setInputBuffers 和 setOutputBuffers 设置存放输入输出数据的 buffer，代码如下：

```
int mInputWidth = 640;  
int mInputHeight = 480;  
byte[] mBuffer_input = new byte[mInputWidth * mInputHeight * 3];  
byte[] mBuffer_out_1 = new byte[4420 * 4 * 4];  
byte[] mBuffer_out_2 = new byte[4420 * 2 * 4];  
byte[] mBuffer_out_3 = new byte [4420 * 10 *4];  
  
Map<Integer, byte[]> inputBuffers = new HashMap<>();  
inputBuffers.put(0, mBuffer_input);  
mAtcnnInterpreter.setInputBuffers(mModelId, inputBuffers);  
  
Map<Integer, byte[]> outputBuffers = new HashMap<>();  
outputBuffers.put(0, mBuffer_out_1);  
outputBuffers.put(1, mBuffer_out_2);  
outputBuffers.put(2, mBuffer_out_3);  
mAtcnnInterpreter.setOutputBuffers(mModelId, outputBuffers);
```

- (7) 根据模型 input 参数的具体格式，调用 setInputData 进行设置，代码如下：

```
ArrayList<Atcnn_data> inputs = new ArrayList<>();  
Atcnn_data data = new Atcnn_data();  
data.dims.add(mInputWidth);  
data.dims.add(mInputHeight);  
data.dims.add(3);  
data.dims.add(1);
```

```
data.n_dims = 4;
data.dataType = ATCNN_TENSOR_TYPE.ATCNN_TYPE_INT8;
data.layout = ATCNN_DIM_LAYOUT.ATCNN_DIM_NCHW;
inputs.add(data);
boolean result = mAtcnnInterpreter.setInputData(mModelId, inputs);
```

此处输入的图像格式与 toolkit 生成 atcnn 模型时设置的图像格式需保持一致，默认采用 RGB888。

(8) 执行推理

```
result = mAtcnnInterpreter.run(mModelId, false, false, false);
```

(9) 待推理完毕，推理结果会自动填充到 outputBuffers 中；

(10) 循环执行(7)、(8)、(9) 三步；

(11) 待所有图像处理完毕，调用 releaseModel 和 destroyInterpreter 接口释放资源。

49.4.4 ATCNN SDK 接口调用示例

已提供如下示例：

- 1、基于 lenet 网络的模型转换 demo
- 2、基于 C++接口的 lenet 模型部署 demo
- 3、基于 java 接口的人脸检测部署 demo

50 参考文档

1. Android 官方网站

<https://source.android.com/source/index.html>

<https://developer.android.com/>

2. Android git repositories

<https://android.googlesource.com/>

3. [GUIDE] Understanding the Android Source Code

<https://forum.xda-developers.com/showthread.php?t=2620389>

4. Android-- bionic 介绍

<http://blog.csdn.net/andyhuabing/article/details/7183369>

5. Android 目录下的内容学习---abi

<http://blog.csdn.net/fristkey2/article/details/7870593>

6. Android Developers Bluetooth 介绍

<https://developer.android.com/guide/topics/connectivity/bluetooth>

7. Android Developers Telecom 介绍

<https://developer.android.com/guide/topics/connectivity/telecom/>

8. Android Developers Media 框架介绍

<https://developer.android.com/guide/topics/media-apps/media-apps-overview>

9. Android Developers Contacts 介绍

<https://developer.android.com/guide/topics/providers/contacts-provider>

10. Android OTA 升级官方网站

<https://source.android.google.cn/docs/core/ota?hl=zh-cn>

11. Android 官方网站

<https://source.android.com/source/index.html>

12. Android NuPlayer 介绍

<https://blog.csdn.net/miaomiao12345678/article/details/57415505>

13. Android MediaPlayer 使用简介

https://blog.csdn.net/android_cmos/article/details/51399471

<https://blog.csdn.net/u014365133/article/details/53330776>

14. Android NuPlayer 介绍

<https://blog.csdn.net/miaomiao12345678/article/details/57415505>

15. Android MediaPlayer 使用简介

https://blog.csdn.net/android_cmos/article/details/51399471

<https://blog.csdn.net/u014365133/article/details/53330776>

16. FreeRTOS

www.freertos.org

17. Universal Serial Bus Specification Revision 2.0

18. Universal Serial Bus 3.0 Specification

19. extensible-host-controller-interface-usb-xhci.pdf

20. Cadence USB 30 DRD Controller - Programmers Guide

21. Accessory Interface Specification CarPlay Addendum.pdf

22. <https://android.googlesource.com/kernel/msm.git/+android-msm-bullhead-3.10-n-preview-1/drivers/usb/phy/class-dual-role.c?autodive=0%2F%2F%2F%2F>

23. Universal Flash Storage (UFS) Version 2.1

24. eMMC 5.1 Spec

https://www.jedec.org/document_search?search_api_views_fulltext=jesd84-b51

25. WD090GHL60AC-A7.pdf (1024*600 LCM specification)

26. Virtqueues and virtio ring: How the data travels

<https://www.redhat.com/en/blog/virtqueues-and-virtio-ring-how-data-travels>

27. [BeiDou]官方网站

<http://www.beidou.gov.cn/>

28. [GPS]官方网站

<https://www.gps.gov/>

29. PCIE

<https://pcisig.com/>

30. AC8025 SWIP Tool MPTool 用户使用指南